Probabilistic Machine Learning: An Introduction

Adaptive Computation and Machine Learning
Thomas Dietterich, Editor
Christopher Bishop, David Heckerman, Michael Jordan, and Michael Kearns, Associate Editors

Bioinformatics: The Machine Learning Approach, Pierre Baldi and Søren Brunak
Reinforcement Learning: An Introduction, Richard S. Sutton and Andrew G. Barto
Graphical Models for Machine Learning and Digital Communication, Brendan J. Frey
Learning in Graphical Models, Michael I. Jordan
Causation, Prediction, and Search, second edition, Peter Spirtes, Clark Glymour, and Richard Scheines
Principles of Data Mining, David Hand, Heikki Mannila, and Padhraic Smyth
Bioinformatics: The Machine Learning Approach, second edition, Pierre Baldi and Søren Brunak
Learning Kernel Classifiers: Theory and Algorithms, Ralf Herbrich
Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond, Bernhard Schölkopf and Alexander J. Smola
Introduction to Machine Learning, Ethem Alpaydin
Gaussian Processes for Machine Learning, Carl Edward Rasmussen and Christopher K.I. Williams
Semi-Supervised Learning, Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien, Eds.
The Minimum Description Length Principle, Peter D. Grünwald
Introduction to Statistical Relational Learning, Lise Getoor and Ben Taskar, Eds.
Probabilistic Graphical Models: Principles and Techniques, Daphne Koller and Nir Friedman
Introduction to Machine Learning, second edition, Ethem Alpaydin
Boosting: Foundations and Algorithms, Robert E. Schapire and Yoav Freund
Machine Learning: A Probabilistic Perspective, Kevin P. Murphy
Foundations of Machine Learning, Mehryar Mohri, Afshin Rostami, and Ameet Talwalker

Probabilistic Machine Learning
An Introduction

Kevin P. Murphy

The MIT Press
Cambridge, Massachusetts
London, England

© 2021 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

For information about special quantity discounts, please email special_sales@mitpress.mit.edu

This book was set in the LaTeX programming language by the author. Printed and bound in the United States of America.

This book is dedicated to my mother, Brigid Murphy,
who introduced me to the joy of learning and teaching.

Brief Contents

1 Introduction 1

I Foundations 29
2 Probability: univariate models 31
3 Probability: multivariate models 73
4 Statistics 99
5 Decision theory 159
6 Information theory 209
7 Linear algebra 231
8 Optimization 279

II Linear models 331
9 Linear discriminant analysis 333
10 Logistic regression 349
11 Linear regression 379
12 Generalized linear models * 419

III Deep neural networks 437
13 Neural networks for unstructured data 439
14 Neural networks for images 485
15 Neural networks for sequences 523

IV Nonparametric models 563
16 Exemplar-based methods 565
17 Kernel methods 585
18 Trees, forests, bagging and boosting 627

V Beyond supervised learning 647
19 Learning with fewer labeled examples 649
20 Dimensionality reduction 675
21 Clustering 733
22 Recommender systems 759
23 Graph embeddings * 771

VI Appendix 793
A Notation 795

Contents

Preface **xi**

1 Introduction **1**
1.1 What is machine learning? 1
1.2 Supervised learning 1
1.2.1 Classification 2
1.2.2 Regression 8
1.2.3 Overfitting and generalization 12
1.2.4 No free lunch theorem 13
1.3 Unsupervised learning 13
1.3.1 Clustering 14
1.3.2 Discovering latent "factors of variation" 14
1.3.3 Self-supervised learning 15
1.3.4 Evaluating unsupervised learning 15
1.4 Reinforcement learning 16
1.5 Data 18
1.5.1 Some common image datasets 18
1.5.2 Some common text datasets 21
1.5.3 Preprocessing discrete input data 21
1.5.4 Preprocessing text data 22
1.5.5 Handling missing data 25
1.6 Discussion 25
1.6.1 The relationship between ML and other fields 26
1.6.2 Structure of the book 26
1.6.3 Caveats 26

I Foundations **29**

2 Probability: univariate models **31**
2.1 Introduction 31
2.1.1 What is probability? 31

2.1.2 Types of uncertainty 31
2.1.3 Probability as an extension of logic 32
2.2 Random variables 33
2.2.1 Discrete random variables 33
2.2.2 Continuous random variables 34
2.2.3 Sets of related random variables 36
2.2.4 Independence and conditional independence 37
2.2.5 Moments of a distribution 38
2.3 Bayes' rule 41
2.3.1 Example: Testing for COVID-19 42
2.3.2 Example: The Monty Hall problem 43
2.3.3 Inverse problems * 45
2.4 Bernoulli and binomial distributions 45
2.4.1 Definition 46
2.4.2 Sigmoid (logistic) function 46
2.4.3 Binary logistic regression 48
2.5 Categorical and multinomial distributions 49
2.5.1 Definition 49
2.5.2 Softmax function 50
2.5.3 Multiclass logistic regression 51
2.5.4 Log-sum-exp trick 52
2.6 Univariate Gaussian (normal) distribution 53
2.6.1 Cumulative distribution function 53
2.6.2 Probability density function 54
2.6.3 Regression 55
2.6.4 Why is the Gaussian distribution so widely used? 56
2.6.5 Dirac delta function as a limiting case 57
2.7 Some other common univariate distributions * 57
2.7.1 Student t distribution 57
2.7.2 Cauchy distribution 59
2.7.3 Laplace distribution 59
2.7.4 Beta distribution 60
2.7.5 Gamma distribution 60
2.8 Transformations of random variables * 62
2.8.1 Discrete case 62
2.8.2 Continuous case 62
2.8.3 Invertible transformations (bijectors) 62
2.8.4 Moments of a linear transformation 64
2.8.5 The convolution theorem 65
2.8.6 Central limit theorem 67
2.8.7 Monte Carlo approximation 68
2.9 Exercises 69

3 Probability: multivariate models 73
3.1 Joint distributions for multiple random variables 73

3.1.1 Covariance 73
3.1.2 Correlation 74
3.1.3 Uncorrelated does not imply independent 75
3.1.4 Correlation does not imply causation 75
3.1.5 Simpsons' paradox 76
3.2 The multivariate Gaussian (normal) distribution 77
3.2.1 Definition 77
3.2.2 Mahalanobis distance 79
3.2.3 Marginals and conditionals of an MVN * 80
3.2.4 Example: Imputing missing values * 81
3.3 Linear Gaussian systems * 81
3.3.1 Example: inferring a latent vector from a noisy sensor 83
3.3.2 Example: inferring a latent vector from multiple noisy sensors 84
3.4 Mixture models 84
3.4.1 Gaussian mixture models 85
3.4.2 Mixtures of Bernoullis 89
3.4.3 Gaussian scale mixtures * 89
3.5 Probabilistic graphical models * 90
3.5.1 Representation 90
3.5.2 Inference 93
3.5.3 Learning 95
3.6 Exercises 97

4 Statistics 99
4.1 Introduction 99
4.2 Maximum likelihood estimation (MLE) 99
4.2.1 Definition 99
4.2.2 Justification for MLE 100
4.2.3 Example: MLE for the Bernoulli distribution 102
4.2.4 Example: MLE for the categorical distribution 103
4.2.5 Example: MLE for the univariate Gaussian 103
4.2.6 Example: MLE for the multivariate Gaussian 104
4.2.7 Example: MLE for linear regression 106
4.3 Empirical risk minimization (ERM) 107
4.3.1 Example: minimizing the misclassification rate 107
4.3.2 Surrogate loss 108
4.4 Other estimation methods * 108
4.4.1 The method of moments 108
4.4.2 Online (recursive) estimation 110
4.5 Regularization 112
4.5.1 Example: MAP estimation for the Bernoulli distribution 113
4.5.2 Example: MAP estimation for the multivariate Gaussian * 114
4.5.3 Example: weight decay 115
4.5.4 Picking the regularizer using a validation set 117
4.5.5 Cross-validation 117

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

4.5.6 Early stopping 119
4.5.7 Using more data 120
4.6 Bayesian statistics * 121
4.6.1 Conjugate priors 121
4.6.2 The beta-binomial model 122
4.6.3 The Dirichlet-multinomial model 129
4.6.4 The Gaussian-Gaussian model 132
4.6.5 Beyond conjugate priors 135
4.6.6 Credible intervals 137
4.6.7 Bayesian machine learning 138
4.6.8 Computational issues 142
4.7 Frequentist statistics * 145
4.7.1 Sampling distributions 145
4.7.2 Gaussian approximation of the sampling distribution of the MLE 146
4.7.3 Bootstrap approximation of the sampling distribution of any estimator 147
4.7.4 Confidence intervals 148
4.7.5 Caution: Confidence intervals are not credible 149
4.7.6 The bias-variance tradeoff 150
4.8 Exercises 155

5 Decision theory 159
5.1 Bayesian decision theory 159
5.1.1 Basics 159
5.1.2 Classification problems 161
5.1.3 ROC curves 162
5.1.4 Precision-recall curves 165
5.1.5 Regression problems 168
5.1.6 Probabilistic prediction problems 169
5.2 A/B testing * 171
5.2.1 A Bayesian approach 172
5.2.2 Example 174
5.3 Bandit problems * 176
5.3.1 Contextual bandits 176
5.3.2 Markov decision processes 177
5.3.3 Exploration-exploitation tradeoff 178
5.3.4 Optimal solution 178
5.3.5 Regret 180
5.3.6 Upper confidence bounds (UCB) 181
5.3.7 Thompson sampling 182
5.3.8 Simple heuristics 183
5.4 Bayesian hypothesis testing 184
5.4.1 Example: Testing if a coin is fair 185
5.4.2 Bayesian model selection 185
5.4.3 Occam's razor 186
5.4.4 Connection between cross validation and marginal likelihood 189

5.4.5 Information criteria * 190
5.5 Frequentist decision theory 192
5.5.1 Computing the risk of an estimator 192
5.5.2 Consistent estimators 195
5.5.3 Admissible estimators 195
5.6 Empirical risk minimization 196
5.6.1 Empirical risk 196
5.6.2 Structural risk 198
5.6.3 Cross-validation 199
5.6.4 Statistical learning theory * 200
5.7 Frequentist hypothesis testing * 201
5.7.1 Likelihood ratio test 201
5.7.2 Null hypothesis significance testing (NHST) 202
5.7.3 p-values 203
5.7.4 p-values considered harmful 203
5.7.5 Why isn't everyone a Bayesian? 206
5.8 Exercises 207

6 Information theory 209
6.1 Entropy 209
6.1.1 Entropy for discrete random variables 209
6.1.2 Cross entropy 211
6.1.3 Joint entropy 211
6.1.4 Conditional entropy 212
6.1.5 Perplexity 213
6.1.6 Differential entropy for continuous random variables * 214
6.2 Relative entropy (KL divergence) * 215
6.2.1 Definition 215
6.2.2 Interpretation 216
6.2.3 Example: KL divergence between two Gaussians 216
6.2.4 Non-negativity of KL 216
6.2.5 KL divergence and MLE 217
6.2.6 Forward vs reverse KL 218
6.3 Mutual information * 219
6.3.1 Definition 219
6.3.2 Interpretation 219
6.3.3 Example 220
6.3.4 Conditional mutual information 221
6.3.5 MI as a "generalized correlation coefficient" 221
6.3.6 Normalized mutual information 222
6.3.7 Maximal information coefficient 223
6.3.8 Data processing inequality 225
6.3.9 Sufficient Statistics 226
6.3.10 Fano's inequality * 226
6.4 Exercises 227

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7 Linear algebra 231
7.1 Introduction 231
7.1.1 Notation 231
7.1.2 Vector spaces 234
7.1.3 Norms of a vector and matrix 236
7.1.4 Properties of a matrix 238
7.1.5 Special types of matrices 240
7.2 Matrix multiplication 244
7.2.1 Vector-Vector Products 244
7.2.2 Matrix-Vector Products 244
7.2.3 Matrix-Matrix Products 245
7.2.4 Application: manipulating data matrices 247
7.2.5 Kronecker products * 249
7.2.6 Einstein summation * 250
7.3 Matrix inversion 251
7.3.1 The inverse of a square matrix 251
7.3.2 Schur complements * 251
7.3.3 The matrix inversion lemma * 253
7.3.4 Matrix determinant lemma * 253
7.4 Eigenvalue decomposition (EVD) 254
7.4.1 Basics 254
7.4.2 Diagonalization 255
7.4.3 Eigenvalues and eigenvectors of symmetric matrices 255
7.4.4 Geometry of quadratic forms 256
7.4.5 Standardizing and whitening data 256
7.4.6 Power method 258
7.4.7 Deflation 259
7.4.8 Eigenvectors optimize quadratic forms 259
7.5 Singular value decomposition (SVD) 259
7.5.1 Basics 259
7.5.2 Connection between SVD and EVD 260
7.5.3 Pseudo inverse 261
7.5.4 SVD and the range and null space of a matrix * 262
7.5.5 Truncated SVD 263
7.6 Other matrix decompositions * 264
7.6.1 LU factorization 264
7.6.2 QR decomposition 264
7.6.3 Cholesky decomposition 265
7.7 Solving systems of linear equations * 266
7.7.1 Solving square systems 267
7.7.2 Solving underconstrained systems (least norm estimation) 267
7.7.3 Solving overconstrained systems (least squares estimation) 268
7.8 Matrix calculus 269
7.8.1 Derivatives 269
7.8.2 Gradients 270

7.8.3 Directional derivative 271
7.8.4 Total derivative * 271
7.8.5 Jacobian 271
7.8.6 Hessian 272
7.8.7 Gradients of commonly used functions 272
7.8.8 Functional derivative notation * 274
7.9 Exercises 277

8 Optimization 279

8.1 Introduction 279
8.1.1 Local vs global optimization 279
8.1.2 Constrained vs unconstrained optimization 281
8.1.3 Convex vs nonconvex optimization 281
8.1.4 Smooth vs nonsmooth optimization 285
8.2 First-order methods 286
8.2.1 Descent direction 287
8.2.2 Step size (learning rate) 288
8.2.3 Convergence rates 290
8.2.4 Momentum methods 291
8.3 Second-order methods 292
8.3.1 Newton's method 293
8.3.2 BFGS and other quasi-Newton methods 294
8.3.3 Trust region methods 295
8.3.4 Natural gradient descent * 296
8.4 Stochastic gradient descent 299
8.4.1 Application to finite sum problems 299
8.4.2 Example: SGD for fitting linear regression 300
8.4.3 Choosing the step size (learning rate) 301
8.4.4 Iterate averaging 303
8.4.5 Variance reduction * 303
8.4.6 Preconditioned SGD 305
8.5 Constrained optimization 307
8.5.1 Lagrange multipliers 308
8.5.2 The KKT conditions 309
8.5.3 Linear programming 311
8.5.4 Quadratic programming 312
8.5.5 Mixed integer linear programming * 313
8.6 Proximal gradient method * 313
8.6.1 Projected gradient descent 314
8.6.2 Proximal operator for ℓ_1-norm regularizer 315
8.6.3 Proximal operator for quantization 316
8.7 Bound optimization * 317
8.7.1 The general algorithm 317
8.7.2 The EM algorithm 318
8.7.3 Example: EM for a GMM 321

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.7.4 Example: EM for an MVN with missing data 325
8.8 Blackbox and derivative free optimization 328
8.8.1 Grid search and random search 328
8.8.2 Simulated annealing * 328
8.8.3 Model-based blackbox optimization * 329
8.9 Exercises 330

II Linear models 331

9 Linear discriminant analysis 333
9.1 Introduction 333
9.2 Gaussian discriminant analysis 333
9.2.1 Quadratic decision boundaries 334
9.2.2 Linear decision boundaries 335
9.2.3 The connection between LDA and logistic regression 335
9.2.4 Model fitting 336
9.2.5 Nearest centroid classifier 338
9.2.6 Fisher's linear discriminant analysis * 338
9.3 Naive Bayes classifiers 342
9.3.1 Example models 342
9.3.2 Model fitting 343
9.3.3 Bayesian naive Bayes 344
9.3.4 The connection between naive Bayes and logistic regression 345
9.4 Generative vs discriminative classifiers 346
9.4.1 Advantages of discriminative classifiers 346
9.4.2 Advantages of generative classifiers 347
9.4.3 Handling missing features 347
9.5 Exercises 348

10 Logistic regression 349
10.1 Introduction 349
10.2 Binary logistic regression 349
10.2.1 Linear classifiers 349
10.2.2 Nonlinear classifiers 350
10.2.3 Maximum likelihood estimation 352
10.2.4 Stochastic gradient descent 355
10.2.5 Perceptron algorithm 355
10.2.6 Iteratively reweighted least squares 356
10.2.7 MAP estimation 357
10.2.8 Standardization 359
10.3 Multinomial logistic regression 360
10.3.1 Linear and nonlinear classifiers 360
10.3.2 Maximum likelihood estimation 360
10.3.3 Gradient-based optimization 363

10.3.4 Bound optimization 363
10.3.5 MAP estimation 364
10.3.6 Maximum entropy classifiers 365
10.3.7 Hierarchical classification 366
10.3.8 Handling large numbers of classes 367
10.4 Robust logistic regression * 368
10.4.1 Mixture model for the likelihood 368
10.4.2 Bi-tempered loss 369
10.5 Bayesian logistic regression * 371
10.5.1 Laplace approximation 372
10.5.2 Approximating the posterior predictive 374
10.6 Exercises 376

11 Linear regression 379

11.1 Introduction 379
11.2 Standard linear regression 379
11.2.1 Terminology 379
11.2.2 Least squares estimation 380
11.2.3 Other approaches to computing the MLE 384
11.2.4 Measuring goodness of fit 387
11.3 Ridge regression 389
11.3.1 Computing the MAP estimate 389
11.3.2 Connection between ridge regression and PCA 391
11.3.3 Choosing the strength of the regularizer 393
11.4 Robust linear regression * 393
11.4.1 Robust regression using the Student t distribution 394
11.4.2 Robust regression using the Laplace distribution 395
11.4.3 Robust regression using Huber loss 396
11.4.4 Robust regression by randomly or iteratively removing outliers 397
11.5 Lasso regression 397
11.5.1 MAP estimation with a Laplace prior (ℓ_1 regularization) 397
11.5.2 Why does ℓ_1 regularization yield sparse solutions? 398
11.5.3 Hard vs soft thresholding 399
11.5.4 Regularization path 401
11.5.5 Comparison of least squares, lasso, ridge and subset selection 402
11.5.6 Variable selection consistency 404
11.5.7 Group lasso 405
11.5.8 Elastic net (ridge and lasso combined) 407
11.5.9 Optimization algorithms 407
11.6 Bayesian linear regression * 409
11.6.1 Computing the posterior 409
11.6.2 Computing the posterior predictive 412
11.6.3 Empirical Bayes (Automatic relevancy determination) 414
11.7 Exercises 416

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

12 Generalized linear models * 419
12.1 Introduction 419
12.2 The exponential family 419
12.2.1 Definition 419
12.2.2 Examples 420
12.2.3 Log partition function is cumulant generating function 425
12.2.4 MLE for the exponential family 426
12.2.5 Exponential dispersion family 427
12.2.6 Maximum entropy derivation of the exponential family 427
12.3 Generalized linear models (GLMs) 428
12.3.1 Examples 429
12.3.2 Maximum likelihood estimation 431
12.3.3 GLMs with non-canonical link functions 431
12.4 Probit regression 432
12.4.1 Latent variable interpretation 432
12.4.2 Maximum likelihood estimation 433
12.4.3 Ordinal probit regression * 435
12.4.4 Multinomial probit models * 435

III Deep neural networks 437

13 Neural networks for unstructured data 439
13.1 Introduction 439
13.2 Multilayer perceptrons (MLPs) 440
13.2.1 The XOR problem 440
13.2.2 Differentiable MLPs 442
13.2.3 Activation functions 442
13.2.4 Example models 443
13.2.5 The importance of depth 448
13.2.6 Connections with biology 449
13.3 Backpropagation 451
13.3.1 Forward vs reverse mode differentiation 452
13.3.2 Reverse mode differentiation for multilayer perceptrons 453
13.3.3 Vector-Jacobian product for common layers 455
13.3.4 Computation graphs 457
13.3.5 Automatic differentiation in functional form * 460
13.4 Training neural networks 464
13.4.1 Tuning the learning rate 464
13.4.2 Vanishing and exploding gradients 464
13.4.3 Non-saturating activation functions 465
13.4.4 Residual connections 468
13.4.5 Parameter initialization 469
13.4.6 Multi-GPU training 472
13.5 Regularization 473

13.5.1 Early stopping 473
13.5.2 Weight decay 473
13.5.3 Sparse DNNs 473
13.5.4 Dropout 474
13.5.5 Bayesian neural networks 475
13.5.6 Regularization effects of (stochastic) gradient descent * 475
13.6 Other kinds of feedforward networks 477
13.6.1 Radial basis function networks 477
13.6.2 Mixtures of experts 479
13.7 Exercises 482

14 Neural networks for images 485
14.1 Introduction 485
14.2 Common layers 486
14.2.1 Convolutional layers 486
14.2.2 Pooling layers 493
14.2.3 Putting it altogether 493
14.2.4 Normalization layers 494
14.3 Common architectures for image classification 497
14.3.1 LeNet 497
14.3.2 AlexNet 498
14.3.3 GoogLeNet (Inception) 499
14.3.4 ResNet 500
14.3.5 DenseNet 501
14.3.6 Neural architecture search 502
14.4 Other forms of convolution * 502
14.4.1 Dilated convolution 502
14.4.2 Transposed convolution 503
14.4.3 Depthwise separable convolution 504
14.5 Solving other discriminative vision tasks with CNNs 505
14.5.1 Image tagging 505
14.5.2 Object detection 506
14.5.3 Instance segmentation 507
14.5.4 Semantic segmentation 507
14.5.5 Human pose estimation 508
14.6 Generating images by inverting CNNs * 509
14.6.1 Converting a trained classifier into a generative model 509
14.6.2 Image priors 510
14.6.3 Visualizing the features learned by a CNN 511
14.6.4 Deep Dream 512
14.6.5 Neural style transfer 513
14.7 Adversarial Examples * 516
14.7.1 Whitebox (gradient-based) attacks 517
14.7.2 Blackbox (gradient-free) attacks 518
14.7.3 Real world adversarial attacks 519

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

14.7.4 Defenses based on robust optimization 520
14.7.5 Why models have adversarial examples 521

15 Neural networks for sequences 523

15.1 Introduction 523
15.2 Recurrent neural networks (RNNs) 523
15.2.1 Vec2Seq (sequence generation) 523
15.2.2 Seq2Vec (sequence classification) 526
15.2.3 Seq2Seq (sequence translation) 527
15.2.4 Teacher forcing 529
15.2.5 Backpropagation through time 530
15.2.6 Vanishing and exploding gradients 531
15.2.7 Gating and long term memory 532
15.2.8 Beam search 535
15.3 1d CNNs 536
15.3.1 1d CNNs for sequence classification 536
15.3.2 Causal 1d CNNs for sequence generation 537
15.4 Attention 538
15.4.1 Attention as soft dictionary lookup 538
15.4.2 Kernel regression as non-parametric attention 539
15.4.3 Parametric attention 540
15.4.4 Seq2Seq with attention 541
15.4.5 Seq2vec with attention (text classification) 543
15.4.6 Seq+Seq2Vec with attention (text pair classification) 543
15.4.7 Soft vs hard attention 545
15.5 Transformers 546
15.5.1 Self-attention 546
15.5.2 Multi-headed attention 546
15.5.3 Positional encoding 547
15.5.4 Putting it altogether 549
15.5.5 Comparing transformers, CNNs and RNNs 550
15.5.6 Transformers for images * 551
15.6 Efficient transformers * 552
15.6.1 Fixed non-learnable localized attention patterns 552
15.6.2 Learnable sparse attention patterns 553
15.6.3 Memory and recurrence methods 554
15.6.4 Low-rank and kernel methods 554
15.7 Language models and unsupervised representation learning 556
15.7.1 ELMo 557
15.7.2 BERT 557
15.7.3 GPT 561
15.7.4 T5 562
15.7.5 Discussion 562

IV Nonparametric models 563

16 Exemplar-based methods 565

16.1 K nearest neighbor (KNN) classification 565
16.1.1 Example 566
16.1.2 The curse of dimensionality 566
16.1.3 Reducing the speed and memory requirements 568
16.1.4 Open set recognition 568
16.2 Learning distance metrics 569
16.2.1 Linear and convex methods 570
16.2.2 Deep metric learning 571
16.2.3 Classification losses 572
16.2.4 Ranking losses 572
16.2.5 Speeding up ranking loss optimization 574
16.2.6 Other training tricks for DML 577
16.3 Kernel density estimation (KDE) 577
16.3.1 Density kernels 578
16.3.2 Parzen window density estimator 578
16.3.3 How to choose the bandwidth parameter 580
16.3.4 From KDE to KNN classification 580
16.3.5 Kernel regression 581

17 Kernel methods 585

17.1 Inferring functions from data 585
17.1.1 Smoothness prior 586
17.1.2 Inference from noise-free observations 586
17.1.3 Inference from noisy observations 588
17.2 Mercer kernels 588
17.2.1 Mercer's theorem 589
17.2.2 Some popular Mercer kernels 589
17.3 Gaussian processes 594
17.3.1 Noise-free observations 594
17.3.2 Noisy observations 595
17.3.3 Comparison to kernel regression 596
17.3.4 Weight space vs function space 597
17.3.5 Numerical issues 598
17.3.6 Estimating the kernel 598
17.3.7 GPs for classification 601
17.3.8 Connections with deep learning 602
17.4 Scaling GPs to large datasets 603
17.4.1 Sparse (inducing-point) approximations 603
17.4.2 Exploiting parallelization and kernel matrix structure 603
17.4.3 Random feature approximation 603
17.5 Support vector machines (SVMs) 605
17.5.1 Large margin classifiers 605

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17.5.2 The dual problem 607
17.5.3 Soft margin classifiers 609
17.5.4 The kernel trick 610
17.5.5 Converting SVM outputs into probabilitues 611
17.5.6 Connection with logistic regression 611
17.5.7 Multi-class classification with SVMs 612
17.5.8 How to choose the regularizer C 613
17.5.9 Kernel ridge regression 614
17.5.10 SVMs for regression 615
17.6 Sparse vector machines 617
17.6.1 Relevance vector machines (RVMs) 618
17.6.2 Comparison of sparse and dense kernel methods 618
17.7 Optimizing in function space * 620
17.7.1 Functional analysis 621
17.7.2 Hilbert space 622
17.7.3 Reproducing Kernel Hilbert Space 622
17.7.4 Representer theorem 623
17.7.5 Kernel ridge regression revisited 625
17.8 Exercises 625

18 Trees, forests, bagging and boosting 627

18.1 Classification and regression trees (CART) 627
18.1.1 Model definition 627
18.1.2 Model fitting 629
18.1.3 Regularization 630
18.1.4 Handling missing input features 630
18.1.5 Pros and cons 630
18.2 Ensemble learning 632
18.2.1 Stacking 632
18.2.2 Ensembling is not Bayes model averaging 633
18.3 Bagging 633
18.4 Random forests 634
18.5 Boosting 635
18.5.1 Forward stagewise additive modeling 636
18.5.2 Quadratic loss and least squares boosting 636
18.5.3 Exponential loss and AdaBoost 637
18.5.4 LogitBoost 640
18.5.5 Gradient boosting 640
18.6 Interpreting tree ensembles 644
18.6.1 Feature importance 644
18.6.2 Partial dependency plots 645

V Beyond supervised learning 647

19 Learning with fewer labeled examples 649

19.1 Data augmentation 649
19.1.1 Examples 649
19.1.2 Theoretical justification 650
19.2 Transfer learning 650
19.2.1 Fine-tuning 651
19.2.2 Supervised pre-training 652
19.2.3 Unsupervised pre-training (self-supervised learning) 653
19.2.4 Domain adaptation 656
19.3 Semi-supervised learning 656
19.3.1 Self-training and pseudo-labeling 657
19.3.2 Entropy minimization 658
19.3.3 Co-training 661
19.3.4 Label propagation on graphs 661
19.3.5 Consistency regularization 662
19.3.6 Deep generative models * 664
19.3.7 Combining self-supervised and semi-supervised learning 668
19.4 Active learning 668
19.4.1 Decision-theoretic approach 669
19.4.2 Information-theoretic approach 669
19.4.3 Batch active learning 670
19.5 Meta-learning * 670
19.5.1 Model-agnostic meta-learning (MAML) 670
19.6 Few-shot learning * 671
19.6.1 Matching networks 672
19.7 Exercises 673

20 Dimensionality reduction 675

20.1 Principal components analysis (PCA) 675
20.1.1 Examples 675
20.1.2 Derivation of the algorithm 677
20.1.3 Computational issues 680
20.1.4 Choosing the number of latent dimensions 682
20.2 Factor analysis * 684
20.2.1 Generative model 685
20.2.2 Probabilistic PCA 686
20.2.3 EM algorithm for FA/PPCA 687
20.2.4 Unidentifiability of the parameters 689
20.2.5 Nonlinear factor analysis 691
20.2.6 Mixtures of factor analysers 692
20.2.7 Exponential family factor analysis 693
20.2.8 Factor analysis models for paired data 695
20.3 Autoencoders 698

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.3.1 Bottleneck autoencoders 698
20.3.2 Denoising autoencoders 699
20.3.3 Contractive autoencoders 700
20.3.4 Sparse autoencoders 701
20.3.5 Variational autoencoders 703
20.4 Manifold learning * 706
20.4.1 What are manifolds? 707
20.4.2 The manifold hypothesis 708
20.4.3 Approaches to manifold learning 708
20.4.4 Multi-dimensional scaling (MDS) 709
20.4.5 Isomap 712
20.4.6 Kernel PCA 713
20.4.7 Maximum variance unfolding (MVU) 715
20.4.8 Local linear embedding (LLE) 715
20.4.9 Laplacian eigenmaps 717
20.4.10 t-SNE 719
20.5 Word embeddings 723
20.5.1 Latent semantic analysis / indexing 723
20.5.2 Word2vec 725
20.5.3 GloVE 728
20.5.4 Word analogies 728
20.5.5 RAND-WALK model of word embeddings 729
20.5.6 Contextual word embeddings 730
20.6 Exercises 730

21 Clustering 733
21.1 Introduction 733
21.1.1 Evaluating the output of clustering methods 733
21.2 Hierarchical agglomerative clustering 735
21.2.1 The algorithm 736
21.2.2 Example 738
21.3 K means clustering 739
21.3.1 The algorithm 740
21.3.2 Examples 740
21.3.3 Vector quantization 741
21.3.4 The K-means++ algorithm 743
21.3.5 The K-medoids algorithm 743
21.3.6 Speedup tricks 744
21.3.7 Choosing the number of clusters K 744
21.4 Clustering using mixture models 748
21.4.1 Mixtures of Gaussians 748
21.4.2 Mixtures of Bernoullis 752
21.5 Spectral clustering * 752
21.5.1 Normalized cuts 752
21.5.2 Eigenvectors of the graph Laplacian encode the clustering 753

21.5.3 Example 754
21.5.4 Connection with other methods 754
21.6 Biclustering * 755
21.6.1 Basic biclustering 755
21.6.2 Nested partition models (Crosscat) 756

22 Recommender systems 759

22.1 Explicit feedback 759
22.1.1 Datasets 759
22.1.2 Collaborative filtering 760
22.1.3 Matrix factorization 761
22.1.4 Autoencoders 763
22.2 Implicit feedback 764
22.2.1 Bayesian personalized ranking 765
22.2.2 Factorization machines 765
22.2.3 Neural matrix factorization 766
22.3 Leveraging side information 767
22.4 Exploration-exploitation tradeoff 768

23 Graph embeddings * 771

23.1 Introduction 771
23.2 Graph Embedding as an Encoder/Decoder Problem 772
23.3 Shallow graph embeddings 774
23.3.1 Unsupervised embeddings 775
23.3.2 Distance-based: Euclidean methods 775
23.3.3 Distance-based: non-Euclidean methods 776
23.3.4 Outer product-based: Matrix factorization methods 776
23.3.5 Outer product-based: Skip-gram methods 777
23.3.6 Supervised embeddings 778
23.4 Graph Neural Networks 779
23.4.1 Message passing GNNs 779
23.4.2 Spectral Graph Convolutions 781
23.4.3 Spatial Graph Convolutions 781
23.4.4 Non-Euclidean Graph Convolutions 783
23.5 Deep graph embeddings 783
23.5.1 Unsupervised embeddings 783
23.5.2 Semi-supervised embeddings 786
23.6 Applications 787
23.6.1 Unsupervised applications 787
23.6.2 Supervised applications 789

Appendices 791

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

VI Appendix 793

A Notation 795

A.1 Introduction 795
A.2 Common mathematical symbols 795
A.3 Functions 796
A.3.1 Common functions of one argument 796
A.3.2 Common functions of two arguments 796
A.3.3 Common functions of > 2 arguments 796
A.4 Linear algebra 797
A.4.1 General notation 797
A.4.2 Vectors 797
A.4.3 Matrices 797
A.4.4 Matrix calculus 798
A.5 Optimization 798
A.6 Probability 799
A.7 Information theory 799
A.8 Statistics and machine learning 799
A.8.1 Supervised learning 800
A.8.2 Unsupervised learning and generative models 800
A.8.3 Bayesian inference 800
A.9 Abbreviations 801

Bibliography 815

Preface

In 2012, I published a 1200-page book called "Machine learning: a probabilistic perspective", which provided a fairly comprehensive coverage of the field of machine learning (ML) at that time, under the unifying lens of probabilistic modeling. The book was well received, and won the De Groot prize in 2013.

The year 2012 is also generally considered the start of the "deep learning revolution". The term "deep learning" refers to a branch of ML that is based on neural networks with many layers (hence the term "deep"). Although this basic technology had been around for many years, it was in 2012 when [KSH12] used deep neural networks (DNNs) to win the ImageNet image classification challenge by such a large margin that it caught the attention of the community. Related work appeared around the same time in several other papers, including [Cir+10; Cir+11; Hin+12]. These breakthroughs were enabled by advances in hardware technology (in particular, the repurposing of fast graphics processing units from video games to ML), data collection technology (in particular, the use of crowd sourcing to collect large labeled datasets such as ImageNet), as well as various new algorithmic ideas.

Since 2012, the field of deep learning has exploded, with new advances coming at an increasing pace. Interest in the field has also exploded, fueled by the commercial success of the technology, and the breadth of applications to which it can be applied. Therefore, in 2018, I decided to write a second edition of my book, to attempt to summarize some of this progress.

By March 2020, my draft of the second edition had swollen to about 1600 pages, and I was still not done. Then the COVID-19 pandemic struck. I decided to put the book writing on hold, and to "pivot" towards helping with various COVID-19 projects (see e.g., [MKS21; Wah+21]). However, in the Fall, when these projects were taking less of my cycles, I decided to try to finish the book. To make up for lost time, I asked several colleagues to help me finish the last $\sim 10\%$ of "missing content". (See acknowledgements below.)

In the meantime, MIT Press told me they could not publish a 1600 page book, and that I would need to split it into two volumes. The result of all this is two new books, "Probabilistic Machine Learning: An Introduction", which you are currently reading, and "Probabilistic Machine Learning: Advanced Topics", which is the sequel to this book [Mur22]. Together these two books attempt to present a fairly broad coverage of the field of ML c. 2021, using the same unifying lens of probabilistic modeling and Bayesian decision theory that I used in the first book.

Most of the content from the first book has been reused, but it is now split fairly evenly between the two new books. In addition, each book has lots of new material, covering some topics from deep learning, but also advances in other parts of the field, such as generative models, variational inference

and reinforcement learning. To make the book more self-contained and useful for students, I have also added some more background content, on topics such as optimization and linear algebra, that was omitted from the first book due to lack of space. Advanced material, that can be skipped during an introductory level course, is denoted by an asterisk * in the section or chapter title. In the future, we hope to post sample syllabuses and slides online.

Another major change is that nearly all of the software now uses Python instead of Matlab. (In the future, we hope to have a Julia version of the code.) The new code leverages standard Python libraries, such as numpy, scipy, scikit-learn, etc. Some examples also rely on various deep learning libraries, such as TensorFlow, PyTorch, and JAX. In addition to the code to create all the figures, there are supplementary Jupyter notebooks to accompany each chapter, which discuss practical aspects that we don't have space to cover in the main text. Details can be found at probml.ai.

Acknowledgements

I would like to thank the following people for helping me to write various parts of this book:

- Frederik Kunstner, Si Yi Meng, Aaron Mishkin, Sharan Vaswani, and Mark Schmidt who helped write parts of Chapter 8 (Optimization).
- Lihong Li, who helped write Sec. 5.3 (Bandit problems *).
- Mathieu Blondel, who helped write Sec. 13.3 (Backpropagation).
- Roy Frostig, who wrote Sec. 7.8.8 (Functional derivative notation *) and Sec. 13.3.5 (Automatic differentiation in functional form *).
- Justin Gilmer, who helped write Sec. 14.7 (Adversarial Examples *).
- Krzysztof Choromanski, who helped write Sec. 15.6 (Efficient transformers *).
- Colin Raffel, who helped write Sec. 19.2 (Transfer learning) and Sec. 19.3 (Semi-supervised learning).
- Bryan Perozzi, who helped write Chapter 23 (Graph embeddings *).
- Zico Kolter, who helped write parts of Chapter 7 (Linear algebra).

I would like to thank John Fearns and Peter Cerno for carefully proofreading the book, as well as feedback from many other people, including 4 anonymous reviewers solicited by MIT Press.

I would like to thank Mahmoud Soliman for writing all the "back-office" code, that connects latex, colab, github and GCP. I would like to thank the authors of [Zha+20], [Gér17] and [Mar18] for letting me reuse or modify some of their open source code from their own excellent books. I would also like to thank many other members of the github community for their code contrbutions (see the full list of names).

Finally I would like to thank my manager at Google, Doug Eck, for letting me spend company time on this book, and my wife Margaret for letting me spend family time on it, too. I hope my efforts to synthesize all this material together in one place will help to save you time in your journey of discovery into the "land of ML".

Kevin Patrick Murphy
Palo Alto, California
April 2021.

1 Introduction

1.1 What is machine learning?

A popular definition of **machine learning** or **ML**, due to Tom Mitchell [Mit97], is as follows:

> A computer program is said to learn from experience E with respect to some class of tasks T, and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Thus there are many different kinds of machine learning, depending on the nature of the task T we wish the system to learn, the nature of the performance measure P we use to evaluate the system, and the nature of the training signal or experience E we give it.

In this book, we will cover the most common types of ML, but from a **probabilistic perspective**. Roughly speaking, this means that we treat all unknown quantities (e.g., predictions about the future value of some quantity of interest, such as tomorrow's temperature, or the parameters of some model) as **random variables**, that are endowed with **probability distributions** which describe a weighted set of possible values the variable may have. (See Chapter 2 for a quick refresher on the basics of probablity, if necessary.)

There are two main reasons we adopt a probabilistic approach. First, it is the optimal approach to **decision making under uncertainty**, as we explain in Sec. 5.1. Second, probabilistic modeling is the language used by most other areas of science and engineering, and thus provides a unifying framework between these fields. As Shakir Mohamed, a researcher at DeepMind, put it:[1]

> Almost all of machine learning can be viewed in probabilistic terms, making probabilistic thinking fundamental. It is, of course, not the only view. But it is through this view that we can connect what we do in machine learning to every other computational science, whether that be in stochastic optimisation, control theory, operations research, econometrics, information theory, statistical physics or bio-statistics. For this reason alone, mastery of probabilistic thinking is essential.

1.2 Supervised learning

The most common form of ML is **supervised learning**. In this problem, the task T is to learn a mapping f from inputs $\mathbf{x} \in \mathcal{X}$ to outputs $\mathbf{y} \in \mathcal{Y}$. The inputs $\mathbf{x}$ are also called the **features**,

1. Source: Slide 2 of https://bit.ly/3pyHyPn

(a) (b) (c)

Figure 1.1: Three types of iris flowers: setosa, versicolor and virginica. Used with kind permission of Dennis Kramb and SIGNA.

index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
	...				
50	7.0	3.2	4.7	1.4	versicolor
	...				
149	5.9	3.0	5.1	1.8	virginica

Table 1.1: A subset of the Iris design matrix. The features are: sepal length, sepal width, petal length, petal width. There are 50 examples of each class.

covariates, or **predictors**; this is often a fixed dimensional vector of numbers, such as the height and weight of a person, or the pixels in an image. In this case, $\mathcal{X} = \mathbb{R}^D$, where D is the dimensionality of the vector (i.e., the number of input features). The output $\mathbf{y}$ is also known as the **label**, **target**, or **response**.[2] The experience E is given in the form of a set of N input-output pairs $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$, known as the **training set**. (N is called the **sample size**.) The performance measure P depends on the type of output we are predicting, as we discuss below.

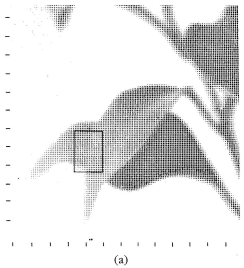
1.2.1 Classification

In **classification** problems, the output space is a set of C unordered and mutually exclusive labels known as **classes**, $\mathcal{Y} = \{1, 2, \ldots, C\}$. The problem of predicting the class label given an input is also called **pattern recognition**. (If there are just two classes, often denoted by $y \in \{0, 1\}$ or $y \in \{-1, +1\}$, it is called **binary classification**.)

1.2.1.1 Example: classifying Iris flowers

As an example, consider the problem of classifying iris flowers into their 3 subspecies, setosa, versicolor and virginica. Fig. 1.1 shows one example of each of these classes.

2. Sometimes (e.g., in the statsmodels Python package) $\mathbf{x}$ are called the **exogeneous variables** and $\mathbf{y}$ are called the **endogeneous variables**.



(a)

X =	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49
Y																
58	171	169	167	167	166	165	166	164	167	171	171	174	174	175	173	171
57	168	168	168	167	166	167	167	165	169	168	174	176	175	175	175	172
56	168	167	167	165	166	166	166	167	167	168	170	178	177	176	174	174	173
55	168	168	165	169	167	168	167	165	168	175	177	177	175	175	172	171
54	169	170	167	169	169	168	163	166	172	169	174	173	175	178	173	173
53	171	169	170	168	169	168	169	168	168	170	175	173	175	177	178	176
52	172	171	170	168	169	169	167	168	173	172	173	177	174	175	178	176
51	172	174	171	170	166	168	167	168	172	172	172	177	179	172	175	175
50	171	167	176	169	170	169	168	169	171	172	174	174	173	173	174	178
49	174	172	173	173	173	174	171	171	172	174	172	172	172	169	173	173
48	173	173	175	176	178	172	171	174	174	173	175	175	175	173	173	171
47	173	175	178	173	173	171	171	175	175	177	178	175	174	173	175	178
46	178	175	174	169	173	175	177	175	177	177	174	175	176	177	177	174
45	173	175	173	174	172	173	174	175	174	171	173	174	175	174	172	171
44	177	174	175	175	172	171	172	176	172	173	172	172	173	170	170	175
43	173	171	174	168	176	172	173	173	173	174	171	174	175	173	174	174
42	175	173	171	172	170	171	176	175	178	172	174	175	175	175	175	172
41	181	179	177	172	170	170	169	179	175	174	175	174	172	175	174	175
40	188	184	179	178	176	176	176	174	172	178	172	174	173	172	174	173
39	195	191	188	186	185	185	180	177	178	175	174	176	175	174	176	176
38	200	199	197	193	190	187	185	180	176	175	180	177	175	175	176	177
37	202	202	199	202	199	194	187	180	175	179	177	176	174	175	176	173

(b)

(a) *(b)*

Figure 1.2: (a) An image of a flower as seen by a human. (b) What the computer sees. The array of numbers represents pixel intensities in the highlighted box. This illustrates the difficulty of learning models that work with image data. From [Mar82, p273]. Used with kind permission of MIT Press.

In **image classification**, the input space $\mathcal{X}$ is the set of images, which is a very high dimensional space: for a color image with $C = 3$ channels (e.g., RGB) and $D_1 \times D_2$ pixels, we have $\mathcal{X} = \mathbb{R}^D$, where $D = C \times D_1 \times D_2$. (In practice we represent each pixel intensity with an integer, typically from the range $\{0, 1, \ldots, 255\}$, but we assume real valued inputs for notational simplicity.) Learning a mapping $f : \mathcal{X} \to \mathcal{Y}$ from images to labels is quite challenging, as illustrated in Fig. 1.2. However, it can be tackled using certain kinds of functions, such as a **convolutional neural network** or **CNN**, which we discuss in Sec. 14.1.

Fortunately for us, some botanists have already identified 4 simple, but highly informative, numeric features — sepal length, sepal width, petal length, petal width — which can be used to distinguish the three kinds of Iris flowers. In this section, we will use this much lower dimensional input space, $\mathcal{X} = \mathbb{R}^4$, for simplicity. The **iris dataset** is a collection of 150 labeled examples of iris flowers, 50 of each type, described by these 4 features. It is widely used as an example, because it is small and simple to understand. (We will discuss larger and more complex datasets later in the book.)

When we have small datasets of features, it is common to store them in an $N \times D$ matrix, in which each row represents an example, and each column represents a feature. This is known as a **design matrix**; see Table 1.1 for an example.[3]

The iris dataset is an example of **tabular data**. When the inputs are of variable size (e.g., sequences of words, or social networks), rather than fixed-length vectors, the data is usually stored in some other format rather than in a design matrix. However, such data is often converted to a fixed-sized feature representation (a process known as **featurization**), thus implicitly creating a design matrix for further processing. We give an example of this in Sec. 1.5.4.1, where we discuss the bag of words representation for sequences.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

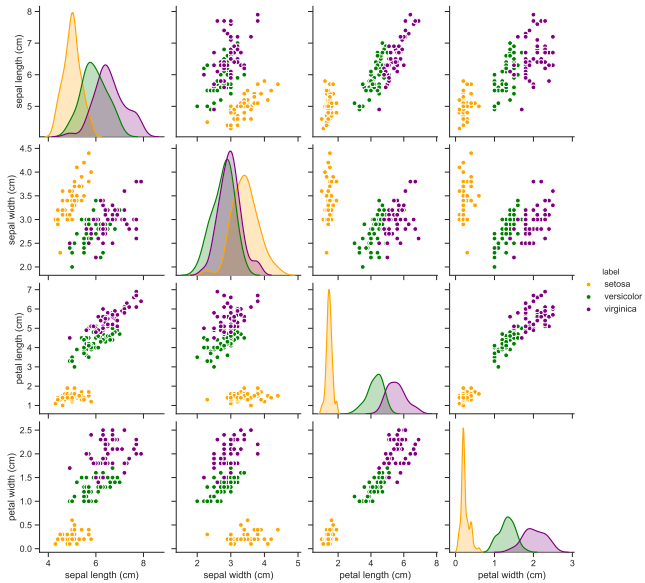


Figure 1.3: Visualization of the Iris data as a pairwise scatter plot. On the diagonal we plot the marginal distribution of each feature for each class. The off-diagonals contain scatterplots of all possible pairs of features. Generated by code at figures.probml.ai/book1/1.3

1.2.1.2 Exploratory data analysis

Before tackling a problem with ML, it is usually a good idea to perform **exploratory data analysis**, to see if there are any obvious patterns (which might give hints on what method to choose), or any obvious problems with the data (e.g., label noise or outliers).

For tabular data with a small number of features, it is common to make a **pair plot**, in which panel (i, j) shows a scatter plots of variables i and j, and the diagonal entries (i, i) show the marginal density of variable i; all plots are optionally color coded by class label, See Fig. 1.3 for an example.

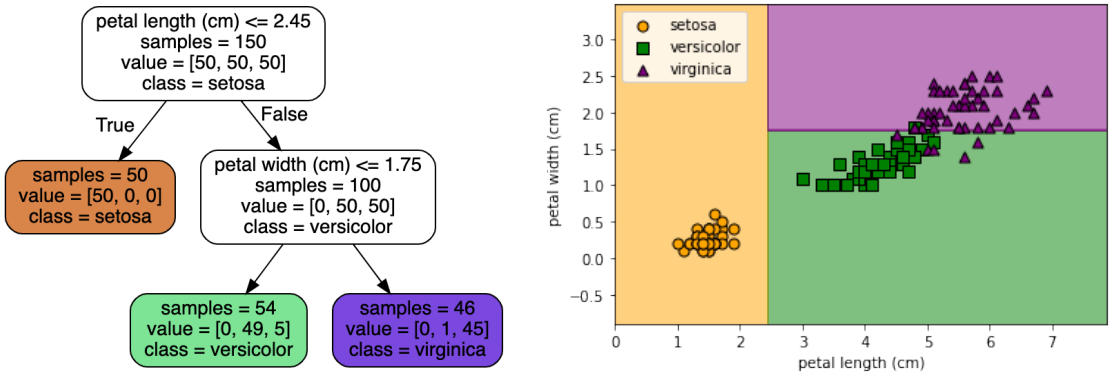
For higher dimensional data, it is common to first perform **dimensionality reduction**, and then to visualize the data in 2d or 3d. We discuss methods for dimensionality reduction in Chapter 20.

1.2.1.3 Learning a classifier

From Fig. 1.3, we can see that the setosa class is easy to distinguish from the other two classes. For example, suppose we create the following **decision rule**:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \begin{cases} \text{Setosa if petal length} < 2.45 \\ \text{Versicolor or Virginica otherwise} \end{cases} \tag{1.1}$$

3. This particular design matrix has $N = 150$ rows and $D = 4$ columns, and hence has a **tall and skinny** shape, since $N \gg D$. By contrast, some datasets (e.g., genomics) have more features than examples, $D \gg N$; their design matrices are **short and fat**. The term "**big data**" usually means that N is large, whereas the term "**wide data**" means that D is large (relative to N).



(a) *(b)*

Figure 1.4: Example of a decision tree of depth 2 applied to the iris data, using just the petal length and petal width features. Leaf nodes are color coded according to the predicted class. The number of training samples that pass from the root to a node is shown inside each box; we show how many values of each class fall into this node. This vector of counts can be normalized to get a distribution over class labels for each node. We can then pick the majority class. Adapted from Figures 6.1 and 6.2 of [Gér19]. Generated by code at figures.probml.ai/book1/1.4.

This is a very simple example of a classifier, in which we have partitioned the input space into two regions, defined by the one-dimensional (1d) **decision boundary** at $x_{\text{petal length}} = 2.45$. Points lying to the left of this boundary are classified as setosa; points to the right are either versicolor or virginica.

We see that this rule perfectly classifies the setosa examples, but not the virginica and versicolor ones. To improve performance, we can recursively partition the space, by splitting regions in which the classifier makes errors. For example, we can add another decision rule, to be applied to inputs that fail the first test, to check if the petal width is below 1.75cm (in which case we predict versicolor) or above (in which case we predict virginica). We can arrange these nested rules in to a tree structure, called a **decision tree**, as shown in Fig. 1.4a This induces the 2d **decision surface** shown in Fig. 1.4b.

We can represent the tree by storing, for each internal node, the feature index that is used, as well as the corresponding threshold value. We denote all these **parameters** by $\boldsymbol{\theta}$. We discuss how to learn these parameters in Sec. 18.1.

1.2.1.4 Model fitting

The goal of supervised learning is to automatically come up with classification models such as the one shown in Fig. 1.4a, so as to reliably predict the labels for any given input. A common way to measure performance on this task is in terms of the **misclassification rate** on the training set:

$$\mathcal{L}(\boldsymbol{\theta}) \triangleq \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}\left(y_n \neq f(\mathbf{x}_n; \boldsymbol{\theta})\right) \tag{1.2}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

		Estimate		
		Setosa	Versicolor	Virginica
Truth	Setosa	0	1	1
	Versicolor	1	0	1
	Virginica	10	10	0

Table 1.2: Hypothetical asymmetric loss matrix for iris classification.

where $\mathbb{I}(e)$ is the binary **indicator function**, which returns 1 iff (if and only if) the condition e is true, and returns 0 otherwise, i.e.,

$$\mathbb{I}(e) = \begin{cases} 1 & \text{if } e \text{ is true} \\ 0 & \text{if } e \text{ is false} \end{cases} \tag{1.3}$$

This assumes all errors are equal. However it may be the case that some errors are more costly than others. For example, suppose we are foraging in the wilderness and we find some iris flowers. Furthermore, suppose that setosa and versicolor are tasty, but virginica is poisonous. In this case, we might use the asymmetric **loss function** $\ell(y, \hat{y})$ shown in Table 1.2.

We can then define **empirical risk** to be the average loss of the predictor on the training set:

$$\mathcal{L}(\boldsymbol{\theta}) \triangleq \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(\mathbf{x}_n; \boldsymbol{\theta})) \tag{1.4}$$

We see that the misclassification rate Eq. (1.2) is equal to the empirical risk when we use **zero-one loss** for comparing the true label with the prediction:

$$\ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y}) \tag{1.5}$$

See Sec. 5.1 for more details.

One way to define the problem of **model fitting** or **training** is to find a setting of the parameters that minimizes the empirical risk on the training set:

$$\hat{\boldsymbol{\theta}} = \operatorname*{argmin}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \operatorname*{argmin}_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(\mathbf{x}_n; \boldsymbol{\theta})) \tag{1.6}$$

This is called **empirical risk minimization**.

However, our true goal is to minimize the expected loss on *future data* that we have not yet seen. That is, we want to **generalize**, rather than just do well on the training set. We discuss this important point in Sec. 1.2.3.

1.2.1.5 Uncertainty

> [We must avoid] false confidence bred from an ignorance of the probabilistic nature of the world, from a desire to see black and white where we should rightly see gray. — Immanuel Kant, as paraphrased by Maria Konnikova [Kon20].

In many cases, we will not be able to perfectly predict the exact output given the input, due to lack of knowledge of the input-output mapping (this is called **epistemic uncertainty** or **model uncertainty**), and/or due to intrinsic (irreducible) stochasticity in the mapping (this is called **aleatoric uncertainty** or **data uncertainty**).

For example, in Fig. 1.4b, we see that an input with petal-length=5 and petal-width=1.75 is right on the decision boundary beween virginica (purple) and versicolor (green), so there may be some uncertainty about the correct label. We can capture our uncertainty using the following **conditional probability distribution**:

$$p(y = c|\mathbf{x}; \boldsymbol{\theta}) = f_c(\mathbf{x}; \boldsymbol{\theta}) \tag{1.7}$$

where $f : \mathcal{X} \to [0, 1]^C$ maps inputs to a probability distribution over the C possible output labels. For example, given the decision tree in Fig. 1.4a, we can estimate the empirical distribution over class labels for each leaf node. If we consider the second (green) leaf node , we get $p(y|\mathbf{x} \in L_3(\boldsymbol{\theta})) = [0/54, 49/54, 5/54]$, which reflects the fact that most examples in that region of input space are versicolor, but some are virginica.

Since $f_c(\mathbf{x}; \boldsymbol{\theta})$ returns the probability of class label c, we require $0 \leq f_c \leq 1$ for each c, and $\sum_{c=1}^{C} f_c = 1$. To avoid this restriction, it is common to instead require the model to return unnormalized log-probabilities. We can then convert these to probabilities using the **softmax function**, which is defined as follows

$$\mathcal{S}(\mathbf{a}) \triangleq \left[\frac{e^{a_1}}{\sum_{c'=1}^{C} e^{a_{c'}}}, \ldots, \frac{e^{a_C}}{\sum_{c'=1}^{C} e^{a_{c'}}} \right] \tag{1.8}$$

This maps $\mathbb{R}^C$ to $[0, 1]^C$, and satisfies the constraints that $0 \leq \mathcal{S}(\mathbf{a})_c \leq 1$ and $\sum_{c=1}^{C} \mathcal{S}(\mathbf{a})_c = 1$. The inputs to the softmax, $\mathbf{a} = f(\mathbf{x}; \boldsymbol{\theta})$, are called **logits**. See Sec. 2.5.2 for details. We thus define the overall model as follows:

$$p(y = c|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{S}_c(\mathbf{f}(\mathbf{x}; \boldsymbol{\theta})) \tag{1.9}$$

Representing uncertainty in our prediction can be important for certain applications. For example, let us return to our poisonous flower example. If we are confident that the flower is setosa, we can enjoy a healthy snack, but if we are uncertain about whether the flower is versicolor or virginica, we would be better off going hungry. Alternatively, if we are uncertain, we may choose to perform an **information gathering action**, such as performing a diagnostic test. For more information about how to make optimal decisions in the presence of uncertainty, see Sec. 5.1.

When fitting probabilistic models, it is common to use the negative log probability as our loss function:

$$\ell(y, f(\mathbf{x}; \boldsymbol{\theta})) = -\log p(y|f(\mathbf{x}; \boldsymbol{\theta})) \tag{1.10}$$

The reasons for this are explained in Sec. 5.1.6.1, but the intuition is that a good model (with low loss) is one that assigns a high probability to the true output y for each corresponding input $\mathbf{x}$. The average negative log probability of the training set is given by

$$\text{NLL}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{n=1}^{N} \log p(y_n|f(\mathbf{x}_n; \boldsymbol{\theta})) \tag{1.11}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is called the **negative log likelihood**. If we minimize this, we can compute the **maximum likelihood estimate** or **MLE**:

$$\hat{\boldsymbol{\theta}}_{\text{mle}} = \operatorname*{argmin}_{\boldsymbol{\theta}} \text{NLL}(\boldsymbol{\theta}) \tag{1.12}$$

This is a very common way to fit models to data, as we will see.

1.2.2 Regression

Now suppose that we want to predict a real-valued quantity $y \in \mathbb{R}$ instead of a class label $y \in \{1, \ldots, C\}$; this is known as **regression**. For example, in the case of Iris flowers, y might be the degree of toxicity if the flower is eaten, or the average height of the plant.

Regression is very similar to classification. However, since the output is real-valued, we need to use a different loss function. For regression, the most common choice is to use **quadratic loss**, or ℓ_2 **loss**:

$$\ell_2(y, \hat{y}) = (y - \hat{y})^2 \tag{1.13}$$

This penalizes large **residuals** $y - \hat{y}$ more than small ones.[4] The empirical risk when using quadratic loss is equal to the **mean squared error** or **MSE**:

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2 \tag{1.14}$$

Based on the discussion in Sec. 1.2.1.5, we should also model the uncertainty in our prediction. In regression problems, it is common to assume the output distribution is a **Gaussian** or **normal**. As we explain in Sec. 2.6, this distribution is defined by

$$\mathcal{N}(y|\mu, \sigma^2) \triangleq \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y-\mu)^2} \tag{1.15}$$

where μ is the mean, σ^2 is the variance, and $\sqrt{2\pi\sigma^2}$ is the normalization constant needed to ensure the density integrates to 1. In the context of regression, we can make the mean depend on the inputs by defining $\mu = f(\mathbf{x}_n; \boldsymbol{\theta})$. We therefore get the following conditional probability distribution:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y|f(\mathbf{x}; \boldsymbol{\theta}), \sigma^2) \tag{1.16}$$

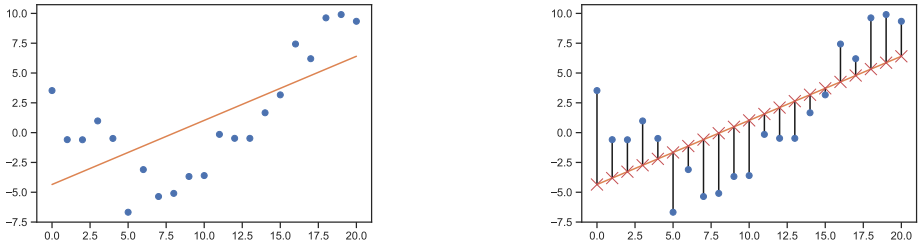
If we assume that the variance σ^2 is fixed (for simplicity), the corresponding negative log likelihood becomes

$$\text{NLL}(\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2} (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2 \right) \right] \tag{1.17}$$

$$= \frac{N}{2\sigma^2} \text{MSE}(\boldsymbol{\theta}) + \text{const} \tag{1.18}$$

We see that the NLL is proportional to the MSE. Hence computing the maximum likelihood estimate of the parameters will result in minimizing the squared error, which seems like a sensible approach to model fitting.

4. If the data has outliers, the quadratic penalty can be too severe. In such cases, it can be better to use ℓ_1 loss instead, which is more **robust**. See Sec. 11.4 for details.



(a) *(b)*

Figure 1.5: (a) Linear regression on some 1d data. (b) The vertical lines denote the residuals between the observed output value for each input (blue circle) and its predicted value (red cross). The goal of least squares regression is to pick a line that minimizes the sum of squared residuals. Generated by code at figures.probml.ai/book1/1.5.

1.2.2.1 Linear regression

As an example of a regression model, consider the 1d data in Fig. 1.5a. We can fit this data using a **simple linear regression** model of the form

$$f(x;\boldsymbol{\theta}) = b + wx \tag{1.19}$$

where w is the **slope**, b is the **offset**, and $\boldsymbol{\theta} = (w, b)$ are all the **parameters** of the model. By adjusting $\boldsymbol{\theta}$, we can minimize the sum of squared errors, shown by the vertical lines in Fig. 1.5b. until we find the **least squares solution**

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, \mathrm{MSE}(\boldsymbol{\theta}) \tag{1.20}$$

See Sec. 11.2.2.1 for details.

If we have multiple input features, we can write

$$f(\mathbf{x};\boldsymbol{\theta}) = b + w_1 x_1 + \cdots + w_D x_D = b + \mathbf{w}^\top \mathbf{x} \tag{1.21}$$

where $\boldsymbol{\theta} = (\mathbf{w}, b)$. This is called **multiple linear regression**.

For example, consider the task of predicting temperature as a function of 2d location in a room. Fig. 1.6(a) plots the results of a linear model of the following form:

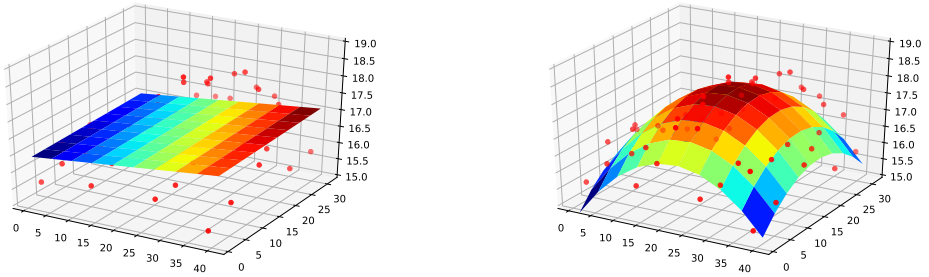
$$f(\mathbf{x};\boldsymbol{\theta}) = b + w_1 x_1 + w_2 x_2 \tag{1.22}$$

We can extend this model to use $D > 2$ input features (such as time of day), but then it becomes harder to visualize.

To reduce notational clutter, it is standard to absorb the bias term b into the weights $\mathbf{w}$ by defining $\tilde{\mathbf{w}} = [b, w_1, \ldots, w_D]$ and defining $\tilde{\mathbf{x}} = [1, x_1, \ldots, x_D]$, so that

$$\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = \mathbf{w}^\top \mathbf{x} + b \tag{1.23}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 1.6: Linear and polynomial regression applied to 2d data. Vertical axis is temperature, horizontal axes are location within a room. Data was collected by some remote sensing motes at Intel's lab in Berkeley, CA (data courtesy of Romain Thibaux). (a) The fitted plane has the form $\hat{f}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2$. (b) Temperature data is fitted with a quadratic of the form $\hat{f}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2$. Generated by code at figures.probml.ai/book1/1.6.

This converts the **affine function** into a linear function. We will usually assume that this has been done, so we can just write $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \mathbf{x}$.

In statistics, the $\mathbf{w}$ parameters are usually called **regression coefficients** (and are typically denoted by $\boldsymbol{\beta}$) and b is called the **intercept**. In ML, the parameters $\mathbf{w}$ are called the **weights** and b is called the **bias**. This terminology arises from electrical engineering, where we view the function f as a circuit which takes in $\mathbf{x}$ and returns $f(\mathbf{x})$. Each input is fed to the circuit on "wires", which have weights $\mathbf{w}$. The circuit computes the weighted sum of its inputs, and adds a constant bias or offset term b. (This use of the term "bias" should not be confused with the statistical concept of bias discussed in Sec. 4.7.6.1.)

1.2.2.2 Polynomial regression

The linear model in Fig. 1.5a is obviously not a very good fit to the data. We can improve the fit by using a **polynomial regression** model of degree D. This has the form $f(x; \mathbf{w}) = \mathbf{w}^\top \boldsymbol{\phi}(x)$, where $\boldsymbol{\phi}(x)$ is a feature vector derived from the input, which has the following form:

$$\boldsymbol{\phi}(x) = [1, x, x^2, \ldots, x^D] \tag{1.24}$$

This is a simple example of **feature preprocessing**, also called **feature engineering**.

In Fig. 1.7a, we see that using $D = 2$ results in a much better fit. We can keep increasing D, and hence the number of parameters in the model, until $D = N - 1$; in this case, we have one parameter per data point, so we can perfectly **interpolate** the data. The resulting model will have 0 MSE, as shown in Fig. 1.7c. However, intuitively the resulting function will not be a good predictor for future inputs, since it is too "wiggly". We discuss this in more detail in Sec. 1.2.3.

We can also apply polynomial regression to multi-dimensional inputs. For example, Fig. 1.6(b) plots the predictions for the temperature model after performing a quadratic expansion of the inputs

$$f(\mathbf{x}; \mathbf{w}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 \tag{1.25}$$

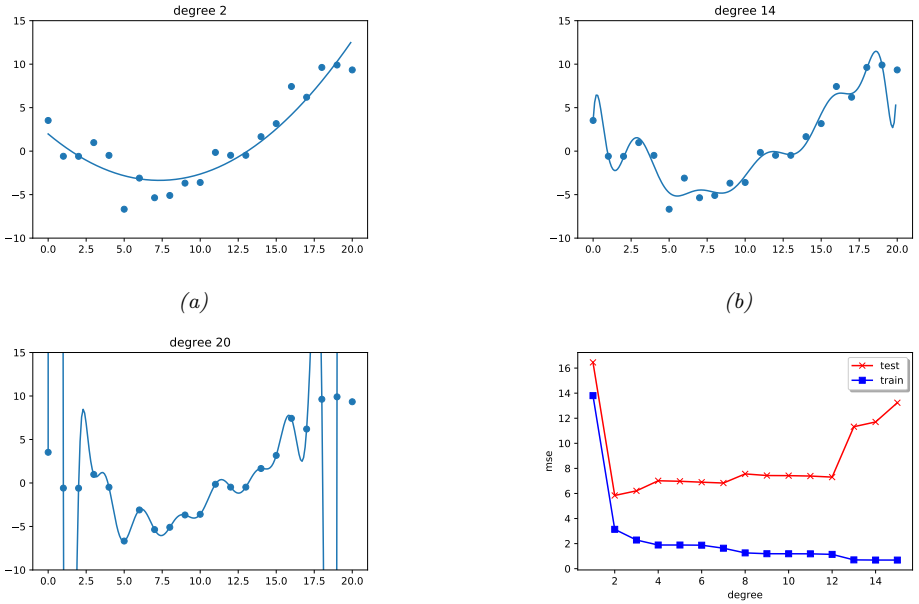


Figure 1.7: (a-c) Polynomials of degrees 2, 14 and 20 fit to 21 datapoints (the same data as in Fig. 1.5). (d) MSE vs degree. Generated by code at figures.probml.ai/book1/1.7.

The quadratic shape is a better fit to the data than the linear model in Fig. 1.6(a), since it captures the fact that the middle of the room is hotter. We can also add cross terms, such as $x_1 x_2$, to capture interaction effects. See Sec. 1.5.3.2 for details.

Note that the above models still use a prediction function that is a linear function of the parameters $\mathbf{w}$, even though it is a nonlinear function of the original input $\mathbf{x}$. The reason this is important is that a linear model induces an MSE loss function $\text{MSE}(\boldsymbol{\theta})$ that has a unique global optimum, as we explain in Sec. 11.2.2.1.

1.2.2.3 Deep neural networks

In Sec. 1.2.2.2, we manually specified the transformation of the input features, namely polynomial expansion, $\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_2^2, \cdots]$. We can create much more powerful models by learning to do such nonlinear **feature extraction** automatically. If we let $\boldsymbol{\phi}(\mathbf{x})$ have its own set of parameters, say $\mathbf{V}$, then the overall model has the form

$$f(\mathbf{x}; \mathbf{w}, \mathbf{V}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}; \mathbf{V}) \tag{1.26}$$

We can recursively decompose the feature extractor $\boldsymbol{\phi}(\mathbf{x}; \mathbf{V})$ into a composition of simpler functions. The resulting model then becomes a stack of L nested functions:

$$f(\mathbf{x}; \boldsymbol{\theta}) = f_L(f_{L-1}(\cdots(f_1(\mathbf{x}))\cdots)) \tag{1.27}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $f_\ell(\mathbf{x}) = f(\mathbf{x}; \boldsymbol{\theta}_\ell)$ is the function at layer ℓ. The final layer is linear and has the form $f_L(\mathbf{x}) = \mathbf{w}^\top f_{1:L-1}(\mathbf{x})$, where $f_{1:L-1}(\mathbf{x})$ is the learned feature extractor. This is the key idea behind **deep neural networks** or **DNN**s, which includes common variants such as **convolutional neural networks** (CNNs) for images, and **recurrent neural networks** (RNNs) for sequences. See Part III for details.

1.2.3 Overfitting and generalization

We can rewrite the empirical risk in Eq. (1.4) in the following equivalent way:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_{\text{train}}) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}_n, y_n) \in \mathcal{D}_{\text{train}}} \ell(y_n, f(\mathbf{x}_n; \boldsymbol{\theta})) \tag{1.28}$$

where $N = |\mathcal{D}_{\text{train}}|$ is the size of the training set $\mathcal{D}_{\text{train}}$. This formulation is useful because it makes explicit which dataset the loss is being evaluated on.

With a suitably flexible model, we can drive the training loss to zero (assuming no label noise), by simply memorizing the correct output for each input. For example, Fig. 1.7(c) perfectly interpolates the training data. But what we care about is prediction accuracy on new data, which may not be part of the training set. A model that perfectly fits the training data, but which is too complex, is said to suffer from **overfitting**.

To detect if a model is overfitting, let us assume (for now) that we have access to the true (but unknown) distribution $p^*(\mathbf{x}, \mathbf{y})$ used to generate the training set. Then, instead of computing the empirical risk we compute the theoretical expected loss or **population risk**

$$\mathcal{L}(\boldsymbol{\theta}; p^*) \triangleq \mathbb{E}_{p^*(\mathbf{x},\mathbf{y})} \left[\ell(\mathbf{y}, f(\mathbf{x}; \boldsymbol{\theta}))\right] \tag{1.29}$$

The difference $\mathcal{L}(\boldsymbol{\theta}; p^*) - \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_{\text{train}})$ is called the **generalization gap**. If a model has a large generalization gap (i.e., low training error but high test error), it is a sign that it is overfitting.

In practice we don't know p^*. However, we can partition the data we do have into two subsets, known as the training set and the **test set**. Then we can approximate the population risk using the **test risk**:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_{\text{test}}) \triangleq \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{\mathbf{x},\mathbf{y} \in \mathcal{D}_{\text{test}}} \ell(y, f(\mathbf{x}; \boldsymbol{\theta})) \tag{1.30}$$

As an example, in Fig. 1.7d, we plot the training error and test error for polynomial regression as a function of degree D. We see that the training error goes to 0 as the model becomes more complex. However, the test error has a characteristic **U-shaped curve**: on the left, where $D = 1$, the model is **underfitting**; on the right, where $D \gg 1$, the model is **overfitting**; and when $D = 2$, the model complexity is "just right".

How can we pick a model of the right complexity? If we use the training set to evaluate different models, we will always pick the most complex model, since that will have the most **degrees of freedom**, and hence will have minimum loss. So instead we should pick the model with minimum test loss.

In practice, we need to partition the data into three sets, namely the training set, the test set and a **validation set**; the latter is used for model selection, and we just use the test set to estimate future performance (the population risk), i.e., the test set is not used for model fitting or model selection. See Sec. 4.5.4 for further details.

1.2.4 No free lunch theorem

All models are wrong, but some models are useful. — George Box [BD87, p424].[5]

Given the large variety of models in the literature, it is natural to wonder which one is best. Unfortunately, there is no single best model that works optimally for all kinds of problems — this is sometimes called the **no free lunch theorem** [Wol96]. The reason is that a set of assumptions (also called **inductive bias**) that works well in one domain may work poorly in another. The best way to pick a suitable model is based on domain knowledge, and/or trial and error (i.e., using model selection techniques such as cross validation (Sec. 4.5.4) or Bayesian methods (Sec. 5.4.2)). For this reason, it is important to have many models and algorithmic techniques in one's toolbox to choose from.

1.3 Unsupervised learning

In supervised learning, we assume that each input example $\mathbf{x}$ in the training set has an associated set of output targets $\mathbf{y}$, and our goal is to learn the input-output mapping. Although this is useful, and can be difficult, supervised learning is essentially just "glorified curve fitting" [Pea18].

An arguably much more interesting task is to try to "make sense of" data, as opposed to just learning a mapping. That is, we just get observed "inputs" $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$ without any corresponding "outputs" $\mathbf{y}_n$. This is called **unsupervised learning**.

From a probabilistic perspective, we can view the task of unsupervised learning as fitting an unconditional model of the form $p(\mathbf{x})$, which can generate new data $\mathbf{x}$, whereas supervised learning involves fitting a conditional model, $p(\mathbf{y}|\mathbf{x})$, which specifies (a distribution over) outputs given inputs.[6]

Unsupervised learning avoids the need to collect large labeled datasets for training, which can often be time consuming and expensive (think of asking doctors to label medical images).

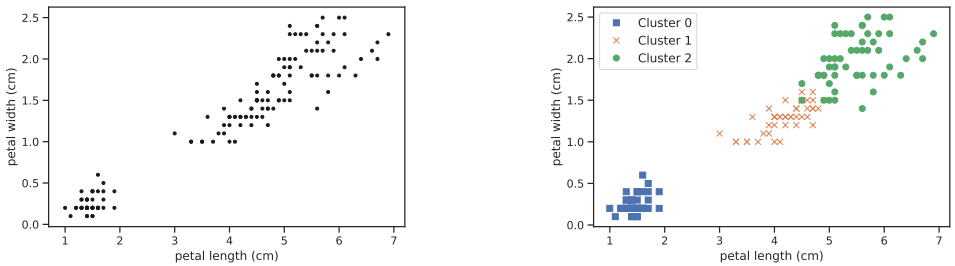
Unsupervised learning also avoids the need to learn how to partition the world into often arbitrary categories. For example, consider the task of labeling when an action, such as "drinking" or "sipping", occurs in a video. Is it when the person picks up the glass, or when the glass first touches the mouth, or when the liquid pours out? What if they pour out some liquid, then pause, then pour again — is that two actions or one? Humans will often disagree on such issues [Idr+17], which means the task is not well defined. It is therefore not reasonable to expect machines to learn such mappings.[7]

Finally, unsupervised learning forces the model to "explain" the high dimensional inputs, rather than just the low dimensional outputs. This allows us to learn richer models of "how the world works". As Geoff Hinton, who is a famous professor of ML at the University of Toronto, has said:

> When we're learning to see, nobody's telling us what the right answers are — we just look. Every so often, your mother says "that's a dog", but that's very little information. You'd be lucky if you got a few bits of information — even one bit per second — that way. The brain's

5. George Box is a retired statistics professor at the University of Wisconsin.
6. In the statistics community, it is common to use $\mathbf{x}$ to denote exogeneous variables that are not modeled, but are simply given as inputs. Therefore an unconditional model would be denoted $p(\mathbf{y})$ rather than $p(\mathbf{x})$.
7. A more reasonable approach is to try to capture the probability distribution over labels produced by a "crowd" of annotators (see e.g., [Dum+18; Aro+19]). This embraces the fact that there can be multiple "correct" labels for a given input due to the ambiguity of the task itself.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 1.8: (a) A scatterplot of the petal features from the iris dataset. (b) The result of unsupervised clustering using $K = 3$. Generated by code at figures.probml.ai/book1/1.8.

> visual system has 10^{14} neural connections. And you only live for 10^9 seconds. So it's no use learning one bit per second. You need more like 10^5 bits per second. And there's only one place you can get that much information: from the input itself. — Geoffrey Hinton, 1996 (quoted in [Gor06]).

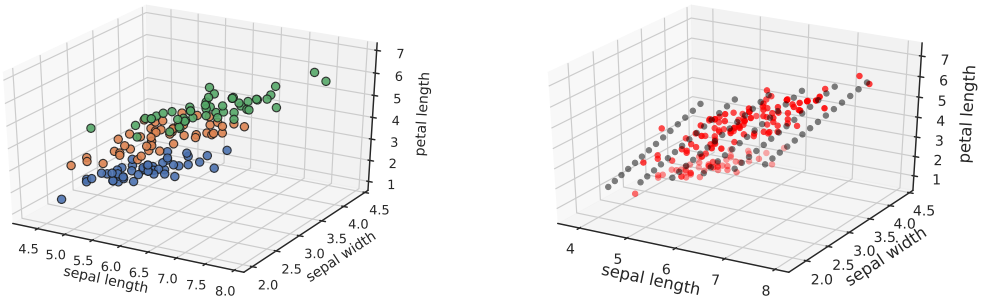
1.3.1 Clustering

A simple example of unsupervised learning is the problem of finding **clusters** in data. The goal is to partition the input into regions that contain "similar" points. As an example, consider a 2d version of the Iris dataset. In Fig. 1.8a, we show the points without any class labels. Intuitively there are at least two clusters in the data, one in the bottom left and one in the top right. Furthermore, if we assume that a "good" set of clusters should be fairly compact, then we might want to split the top right into (at least) two subclusters. The resulting partition into three clusters is shown in Fig. 1.8b. (Note that there is no correct number of clusters; instead, we need to consider the tradeoff between model complexity and fit to the data. We discuss ways to make this tradeoff in Sec. 21.3.7.)

1.3.2 Discovering latent "factors of variation"

When dealing with high dimensional data, it is often useful to reduce the dimensionality by projecting it to a lower dimensional subspace which captures the "essence" of the data. One approach to this problem is to assume that each observed high-dimensional output $\mathbf{x}_n \in \mathbb{R}^D$ was generated by a set of hidden or unobserved low-dimensional **latent factors** $\mathbf{z}_n \in \mathbb{R}^K$. We can represent the model diagrammatically as follows: $\mathbf{z}_n \rightarrow \mathbf{x}_n$, where the arrow represents causation. Since we don't know the latent factors $\mathbf{z}_n$, we often assume a simple prior probability model for $p(\mathbf{z}_n)$ such as a Gaussian, which says that each factor is a random vector K-dimensional vector. If the data is real-valued, we can use a Gaussian likelihood as well.

The simplest example is when we use a linear model, $p(\mathbf{x}_n|\mathbf{z}_n;\boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \boldsymbol{\Sigma})$. The resulting model is called **factor analysis** (FA). It is similar to linear regression, except we only observe the outputs $\mathbf{x}_n$, and not the inputs $\mathbf{z}_n$. In the special case that $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}$, this reduces to a model called probabilistic **principal components analysis** (**PCA**), which we will explain in



(a) (b)

Figure 1.9: (a) Scatterplot of iris data (first 3 features). Points are color coded by class. (b) We fit a 2d linear subspace to the 3d data using PCA. The class labels are ignored. Red dots are the original data, black dots are points generated from the model using $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu}$, where $\mathbf{z}$ are latent points on the underlying inferred 2d linear manifold. Generated by code at figures.probml.ai/book1/1.9.

Sec. 20.1. In Fig. 1.9, we give an illustration of how this method can find a 2d linear subspace when applied to some simple 3d data.

Of course, assuming a linear mapping from $\mathbf{z}_n$ to $\mathbf{x}_n$ is very restrictive. However, we can create nonlinear extensions by defining $p(\mathbf{x}_n|\mathbf{z}_n;\boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|f(\mathbf{z}_n;\boldsymbol{\theta}), \sigma^2\mathbf{I})$, where $f(\mathbf{z};\boldsymbol{\theta})$ is a nonlinear model, such as a deep neural network. It becomes much harder to fit such a model (i.e., to estimate the parameters $\boldsymbol{\theta}$), because the inputs to the neural net have to be inferred, as well as the parameters of the model. However, there are various approximate methods, such as the **variational autoencoder** which can be applied (see Sec. 20.3.5).

1.3.3 Self-supervised learning

A recently popular approach to unsupervised learning is known as **self-supervised learning**. In this approach, we create proxy supervised tasks from unlabeled data. For example, we might try to learn to predict a color image from a gray scale image, or to mask out words in a sentence and then try to predict them given the surrounding context. The hope is that the resulting predictor $\hat{\mathbf{x}}_1 = f(\mathbf{x}_2;\boldsymbol{\theta})$, where $\mathbf{x}_2$ is the observed input and $\hat{\mathbf{x}}_1$ is the predicted output, will learn useful features from the data, that can then be used in standard, downstream supervised tasks. This avoids the hard problem of trying to infer the "true latent factors" $\mathbf{z}$ behind the observed data, and instead relies on standard supervised learning methods. We discuss this approach in more detail in Sec. 19.2.

1.3.4 Evaluating unsupervised learning

Although unsupervised learning is appealing, it is very hard to evaluate the quality of the output of an unsupervised learning method, because there is no ground truth to compare to [TOB16].

A common method for evaluating unsupervised models is to measure the probability assigned by the model to unseen test examples. We can do this by computing the (unconditional) negative log

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

likelihood of the data:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = -\frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \log p(\mathbf{x}|\boldsymbol{\theta}) \tag{1.31}$$

This treats the problem of unsupervised learning as one of **density estimation**. The idea is that a good model will not be "surprised" by actual data samples (i.e., will assign them high probability). Furthermore, since probabilities must sum to 1.0, if the model assigns high probability to regions of data space where the data samples come from, it implicitly assigns low probability to the regions where the data does not come from. Thus the model has learned to capture the **typical patterns** in the data. This can be used inside of a **data compression** algorithm.

Unfortunately, density estimation is difficult, especially in high dimensions. Furthermore, a model that assigns high probability to the data may not have learned useful high level patterns (after all, the model could just memorize all the training examples).

An alternative evaluation metric is to use the learned unsupervised representation as features or input to a downstream supervised learning method. If the unsupervised method has discovered useful patterns, then it should be possible to use these patterns to perform supervised learning using much less labeled data than when working with the original features. For example, in Sec. 1.2.1.1, we saw how the 4 manually defined features of iris flowers contained most of the information needed to perform classification. We were thus able to train a classifier with perfect perfomance using just 150 examples. If the input was raw pixels, we would need many more examples to achieve comparable performance (see Sec. 14.1). That is, we can increase the **sample efficiency** of learning (i.e., reduce the number of labeled examples needed to get good performance) by first learning a good representation.

Increased sample efficiency is a useful evaluation metric, but in many applications, especially in science, the goal of unsupervised learning is to *gain understanding*, not to improve performance on some prediction task. This requires the use of models that are **interpretable**, but which can also generate or "explain" most of the observed patterns in the data. To paraphrase Plato, the goal is to discover how to "carve nature at its joints". Of course, evaluating whether we have succesfully discovered the true underlying structure behind some dataset often requires performing experiments and thus interacting with the world. We discuss this topic further in Sec. 1.4.

1.4 Reinforcement learning

In addition to supervised and unsupervised learning, there is a third kind of ML known as **reinforcement learning** (**RL**). In this class of problems, the system or **agent** has to learn how to interact with its environment. This can be encoded by means of a **policy** $\mathbf{a} = \pi(\mathbf{x})$, which specifies which action to take in response to each possible input $\mathbf{x}$ (derived from the environment state).

For example, consider an agent that learns to play a video game, such as Atari Space Invaders (see Fig. 1.10a). In this case, the input $\mathbf{x}$ is the image (or sequence of past images), and the output $\mathbf{a}$ is the direction to move in (left or right) and whether to fire a missile or not. As a more complex example, consider the problem of a robot learning to walk (see Fig. 1.10b). In this case, the input $\mathbf{x}$ is the set of joint positions and angles for all the limbs, and the output $\mathbf{a}$ is a set of actuation or motor control signals.

The difference from supervised learning (SL) is that the system is not told which action is the best one to take (i.e., which output to produce for a given input). Instead, the system just receives

(a) (b)

Figure 1.10: Examples of some control problems. (a) Space Invaders Atari game. From https://gym.openai.com/envs/SpaceInvaders-v0/. (b) Controlling a humanoid robot in the MuJuCo simulator so it walks as fast as possible without falling over. From https://gym.openai.com/envs/Humanoid-v2/

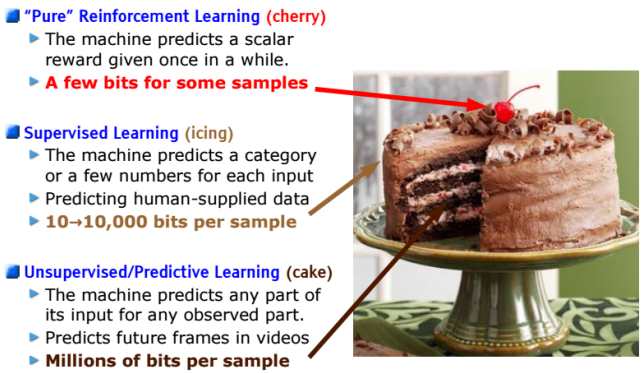


Figure 1.11: The three types of machine learning visualized as layers of a chocolate cake. This figure (originally from https://bit.ly/2m65Vs1) was used in a talk by Yann Le Cun at NIPS'16, and is used with his permission.

an occasional **reward** (or punishment) signal in response to the actions that it takes. This is like **learning with a critic**, who gives an occasional thumbs up or thumbs down, as opposed to **learning with a teacher**, who tells you what to do at each step.

RL has grown in popularity recently, due to its broad applicability (since the reward signal that the agent is trying to optimize can be any metric of interest). However, it can be harder to make RL work than it is for supervised or unsupervised learning, for a variety of reasons. A key difficulty is that the reward signal may only be given occasionally (e.g., if the agent eventually reaches a desired state), and even then it may be unclear to the agent which of its many actions were responsible for getting the reward. (Think of playing a game like chess, where there is a single win or lose signal at the end of the game.)

To compensate for the minimal amount of information coming from the reward signal, it is common to use other information sources, such as expert demonstrations, which can be used in a supervised

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

way, or unlabeled data, which can be used by an unsupervised learning system to discover the underlying structure of the environment. This can make it feasible to learn from a limited number of trials (interactions with the environment). As Yann Le Cun put it, in an invited talk at the NIPS[8] conference in 2016: "If intelligence was a cake, unsupervised learning would be the chocolate sponge, supervised learning would be the icing, and reinforcement learning would be the cherry." This is illustrated in Fig. 1.11.

More information on RL can be found in the sequel to this book, [Mur22].

1.5 Data

Machine learning is concerned with fitting models to data using various algorithms. Although we focus on the modeling and algorithm aspects, it is important to mention that the nature and quality of the training data also plays a vital role in the success of any learned model.

In this section, we briefly describe some common image and text datasets that are widely used in the literature. We will use some of these datasets in later chapters. We also briefly discuss the topic of data preprocessing.

1.5.1 Some common image datasets

The ML literature frequently uses various standard labeled imaged datasets in order to benchmark models and algorithms. We discuss some of these below, since we will use them later in the book.

1.5.1.1 Small datasets

One of the simplest and most widely used is known as **MNIST** [LeC+98; YB19].[9] This is a dataset of 60k training images and 10k test images, each of size 28×28 (gray scale), illustrating handwritten digits from 10 categories; Each pixel is an integer in the range $\{0, 1, \ldots, 255\}$; these are usually rescaled to $[0, 1]$, to represent pixel intensity. We can optionally convert this to a binary image by thresholding. See Fig. 1.12a for an illustration.

MNIST is so widely used in the ML community that Geoff Hinton, a famous ML researcher, has called it the "drosophila of machine learning", since if we cannot make a method work well on MNIST, However, nowadays MNIST classification is considered "too easy", since it is possible to distinguish most pairs of digits by looking at just a single pixel. [XRV17] therefore proposed a dataset with the same size and shape as MNIST, but where each image is the picture of a piece of clothing instead of a handwritten digit. This is called **Fashion-MNIST dataset** (see Fig. 1.12b) and is also widely used in the more recent literature.

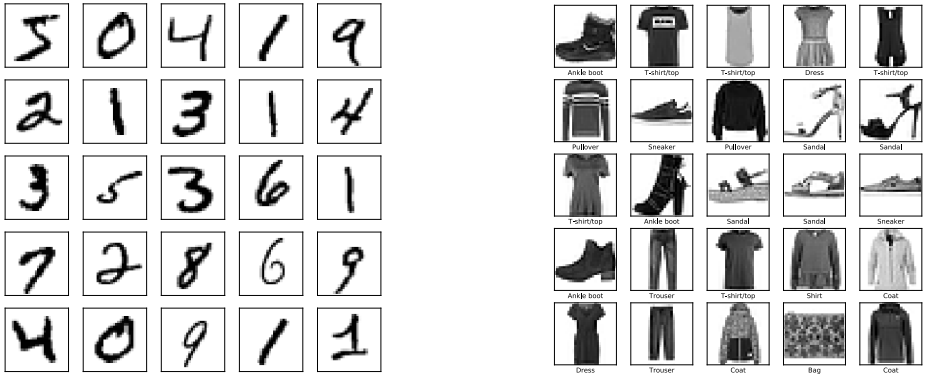
Another grayscale dataset is **EMNIST** [Coh+17] which is a dataset of $\sim 730k$ images of size 28×28 illustrating handwritten digits and letters from 47 classes.

For small color images, the most common dataset is **CIFAR** [KH09].[10] This is a dataset of $\sim 50k$

8. NIPS stands for "Neural Information Processing Systems". It is one of the premier ML conferences. It has recently been renamed to NeurIPS.

9. The term "MNIST" stands for "Modified National Institute of Standards"; The term "modified" is used because the images have been preprocessed to ensure the digits are mostly in the center of the image.

10. CIFAR stands for "Canadian Institute For Advanced Research". This is the agency that funded labeling of the dataset, which was derived from the TinyImages dataset at http://groups.csail.mit.edu/vision/TinyImages/ created by Antonio Torralba. See [KH09] for details.



(a) (b)

Figure 1.12: (a) Visualization of the first 25 digits from the **MNIST** *dataset [LeC+98; YB19]. Each image is* 28×28*. There are 60k training examples and 10k test examples. Generated by code at figures.probml.ai/book1/1.12. (b) Visualization of the* **FashionMNIST** *dataset [XRV17]. The dataset has the same size as MNIST, but is harder to classify. There are 10 classes: 'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'. We show the first 25 images from the training set. Generated by code at figures.probml.ai/book1/1.12.*

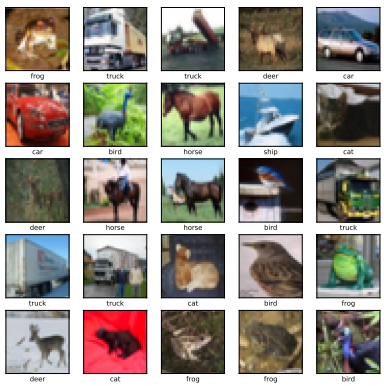
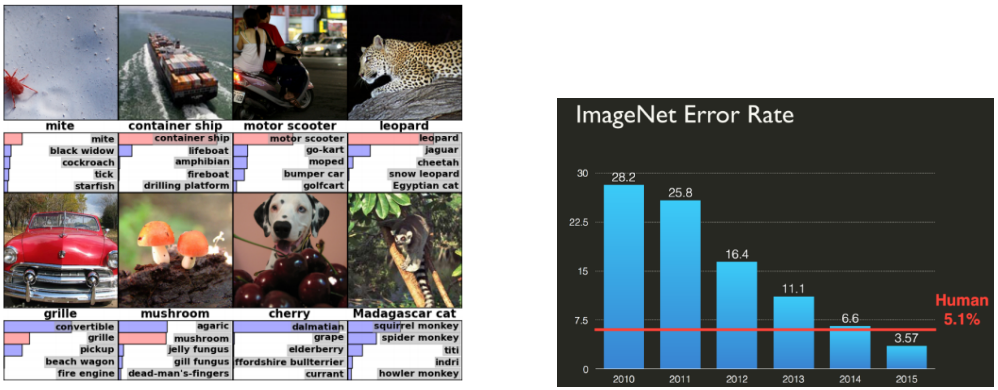


Figure 1.13: Some images from the **CIFAR-10** *dataset [KH09]. Each image is* $32 \times 32 \times 3$*, where the final dimension of size 3 refers to RGB. There are 50k training examples and 10k test examples. There are 10 classes: plane, car, bird, cat, deer, dog, frog, horse, ship, and truck. We show the first 25 images from the training set. Generated by code at figures.probml.ai/book1/1.13.*

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 1.14: (a) Sample images from the **ImageNet** *dataset [Rus+15]. This subset consists of 1.3M color training images, each of which is 256x256 pixels in size. There are 1000 possible labels, one per image, and the task is to minimize the top 5 error rate, i.e., to ensure the correct label is within the 5 most probable predictions. Below each image we show the true label, and a distribution over the top 5 predicted labels. If the true label is in the top 5, its probability bar is colored red. Predictions are generated by a convolutional neural network (CNN) called "AlexNet" (Sec. 14.3.2). From Figure 4 of [KSH12]. Used with kind permission of Alex Krizhevsky. (b) Misclassification rate (top 5) on the ImageNet competition over time. Used with kind permission of Andrej Karpathy.*

images of size $3 \times 32 \times 32$ representing everyday objects from 10 or 100 classes; see Fig. 1.13 for an illustration.

1.5.1.2 The ImageNet competition

Small datasets are useful for prototyping ideas, but it is also important to test methods on larger datasets, both in terms of image size and number of labeled examples. The most widely used dataset of this type is called **ImageNet** [Rus+15]. This is a dataset of $\sim 14M$ images of size $256 \times 256 \times 3$ illustrating various objects from 20,000 classes; see Fig. 1.14a for some examples

The ImageNet dataset was used as the basis of the ImageNet Large Scale Visual Recognition Challenge (ILSRVC), which ran from 2010 to 2018. This used a subset of 1.3M images from 1000 classes. During the course of the competition, significant progress was made by the commmunity, as shown in Fig. 1.14b. In particular, 2015 marked the first year in which CNNs could outperform humans (or at least one human, namely Andrej Karpathy) at the task of classifying images from ImageNet. Note that this does not mean that CNNs are better at vision than humans (see e.g., [YL18] for some common failure modes). Instead, it mostly likely reflects the fact the dataset makes many **fine-grained classification** distinctions — such as between a "tiger" and a "tiger cat" — which humans find difficult to understand; by contrast, sufficiently flexible CNNs can learn arbitrary patterns, including random labels [Zha+17a].

T: In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

Q1: What causes precipitation to fall? A1: **gravity**
Q2: What is another main form of precipitation besides drizzle, rain, snow, sleet and hail? A2: **graupel**
Q3: Where do water droplets collide with ice crystals to form precipitation? A3: **within a cloud**

Figure 1.15: Question-answer pairs for a sample passage in the SQuAD dataset. Each of the answers is a segment of text from the passage. This can be solved using sentence pair tagging. The input is the paragraph text T and the question Q. The output is a tagging of the relevant words in T that answer the question in Q. From Figure 1 of [Raj+16]. Used with kind permission of Percy Liang.

1.5.2 Some common text datasets

Supervised machine learning is often applied to text documents, for tasks such as **email spam classification**, **sentiment analysis** (e.g., is a movie or product review positive or negative), etc.

The "MNIST of text classification" is arguably the **IMDB movie review dataset** from [Maa+11]. (IMDB stands for "internet movie database".) This contains 25k labeled examples for training, and 25k for testing. Each example has a binary label, representing a positive or negative rating.

There are also many other tasks that arise in the field of **natural language processing** or **NLP**. For example, instead of mapping an input sentence $\mathbf{x}$ to a class label $y \in \{1, \ldots, C\}$, we may want to another sentence $\mathbf{y}$, perhaps in another language; this is called **machine translation**. This requires aligned $(\mathbf{x}, \mathbf{y})$ pairs. Fortunately, several such datasets exist, e.g., from the Canadian parliament (English-French pairs), and the European Union (Europarl). A subset of the latter, known as the **WMT dataset** (Workshop on Machine Translation), consists of English-German pairs, and is widely used as a benchmark dataset.

A generalization of machine translation is to learn a mapping from one sequence $\mathbf{x}$ to any other sequence $\mathbf{y}$. This is called a **seq-to-seq model**, and can be viewed as a form of high-dimensional classification. This framing of the problem is very general. For example, it includes **document summarization**, **question answering**, etc. (For example, Fig. 1.15 shows how to formulate question answering as a seq-to-seq problem: the input is the text T and question Q, and the output is the answer A, which is a set of words, possibly extracted from the input.)

We can also consider the case where the input $\mathbf{x}$ is a prefix of a sentence, and the output y is the next word or token. We can then sample from $p(y|\mathbf{x})$, append the output to the input, and recurse. This will let us generate sentences unconditionally, a task known as **language modeling**. This only requires input sentences $\mathbf{x}$, without any corresponding "labels" $\mathbf{y}$. We can therefore think of this as a form of unsupervised learning, which we discuss in Sec. 1.3.

More details on NLP can be found in [JM20].

1.5.3 Preprocessing discrete input data

Many ML modes assume that the data consists of real-valued feature vectors, $\mathbf{x} \in \mathbb{R}^D$. However, sometimes the input may have discrete input features, such as categorical variables like race and gender, or words from some vocabularly. In the sections below, we discuss some ways to preprocess

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

such data to convert it to vector form. This is a common operation that is used for many different kinds of models.

1.5.3.1 One-hot encoding

When we have categorical features, we need to convert them to a numerical scale, so that computing weighted combinations of the inputs makes sense. The standard way to preprocess such categorical variables is to use a **one-hot encoding**, also called a **dummy encoding**. If a variable x has K values, we will denote its dummy encoding as follows: one-hot$(x) = [\mathbb{I}(x = 1), \ldots, \mathbb{I}(x = K)]$. For example, if there are 3 colors (say red, green and blue), the corresponding one-hot vectors will be one-hot(red) $= [1, 0, 0]$, one-hot(green) $= [0, 1, 0]$, and one-hot(blue) $= [0, 0, 1]$.

1.5.3.2 Feature crosses

A linear model using a dummy encoding for each categorical variable can capture the **main effects** of each variable, but cannot capture **interaction effects** between them. For example, suppose we want to predict the fuel efficiency of a vehicle given two categorical input variables: the type (say SUV, Truck, or Family car), and the country of origin (say USA or Japan). If we concatenate the one-hot encodings for the ternary and binary features, we get the following input encoding:

$$\boldsymbol{\phi}(\mathbf{x}) = [1, \mathbb{I}(x_1 = S), \mathbb{I}(x_1 = T), \mathbb{I}(x_1 = F), \mathbb{I}(x_2 = U), \mathbb{I}(x_2 = J)] \tag{1.32}$$

where x_1 is the color and x_2 is the country of origin.

This model cannot capture dependencies between the features. For example, we expect trucks to be less fuel efficient, but perhaps trucks from the USA are even less efficient than trucks from Japan. This cannot be captured using the linear model in Eq. (1.32) since the contribution from the country of origin is independent of the car type.

We can fix this by computing explicit **feature crosses**. For example, we can define a new composite feature with 3×2 possible values, to capture the interaction of type and country of origin. The new model becomes

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) \tag{1.33}$$

$$\begin{aligned} &= w_0 + w_1 \mathbb{I}(x_1 = R) + w_2 \mathbb{I}(x_1 = G) + w_3 \mathbb{I}(x_1 = B) \\ &+ w_4 \mathbb{I}(x_2 = U) + w_5 \mathbb{I}(x_2 = J) \\ &+ w_6 \mathbb{I}(x_1 = R, x_2 = U) + w_7 \mathbb{I}(x_1 = G, x_2 = U) + w_8 \mathbb{I}(x_1 = B, x_2 = U) \\ &+ w_9 \mathbb{I}(x_1 = R, x_2 = J) + w_{10} \mathbb{I}(x_1 = G, x_2 = J) + w_{11} \mathbb{I}(x_1 = B, x_2 = J) \end{aligned} \tag{1.34}$$

We can see that the use of feature crosses converts the original dataset into a **wide format**, with many more columns.

1.5.4 Preprocessing text data

In Sec. 1.5.2, we briefly discussed text classification and other NLP tasks. To feed text data into a classifier, we need to tackle various issues. First, documents have a variable length, and are thus not fixed-length feature vectors, as assumed by many kinds of models. Second, words are categorical variables with many possible values (equal to the size of the vocabulary), so the corresponding

one-hot encodings will be very high dimensional, with no natural notion of similarity. Third, we may encounter words at test time that have not been seen during training (so-called **out-of-vocabulary** or **OOV** words). We discuss some solutions to these problems below. More details can be found in e.g., [BKL10; MRS08; JM20].

1.5.4.1 Bag of words model

A simple approach to dealing with variable-length text documents is to interpret them as a **bag of words**, in which we ignore word order. To convert this to a vector from a fixed input space, we first map each word to a **token** from some vocabulary.

To reduce the number of tokens, we often use various pre-processing techniques such as the following: dropping punctuation, converting all words to lower case; dropping common but uninformative words, such as "and" and "the" (this is called **stop word removal**); replacing words with their base form, such as replacing "running" and "runs" with "run" (this is called **word stemming**); etc. For details, see e.g., [BL12], and for some sample code, see code.probml.ai/book1/text_preproc_torch.

Let x_{nt} be the token at location t in the n'th document. If there are D unique tokens in the vocabulary, then we can represent the n'th document as a D-dimensional vector $\tilde{\mathbf{x}}_n$, where x'_{nv} is the number of times that word v occurs in document n:

$$\tilde{x}_{nv} = \sum_{t=1}^{T} \mathbb{I}\left(x_{nt} = v\right) \tag{1.35}$$

where T is the length of document n. We can now interpret documents as vectors in $\mathbb{R}^D$. This is called the **vector space model** of text [SWY75; TP10].

We traditionally store input data in an $N \times D$ design matrix denoted by $\mathbf{X}$, where D is the number of features. In the context of vector space models, it is more common to represent the input data as a $D \times N$ **term frequency matrix**, where TF_{ij} is the frequency of term i in document j. See Fig. 1.16 for an illustration.

1.5.4.2 TF-IDF

One problem with representing documents as word count vectors is that frequent words may have undue influence, just because the magnitude of their word count is higher, even if they do not carry much semantic content. A common solution to this is to transform the counts by taking logs, which reduces the impact of words that occur many times within a single document.

To reduce the impact of words that occur many times in general (across all documents), we compute a quantity called the **inverse document frequency**, defined as follows: $\text{IDF}_i \triangleq \log \frac{N}{1+\text{DF}_i}$, where DF_i is the number of documents with term i. We can combine these transformations to compute the **TF-IDF** matrix as follows:

$$\text{TFIDF}_{ij} = \log(\text{TF}_{ij} + 1) \times \text{IDF}_i \tag{1.36}$$

(We often normalize each row as well.) This provides a more meaningful representation of documents, and can be used as input to many ML algorithms. See code.probml.ai/book1/tfidf_demo for an example.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

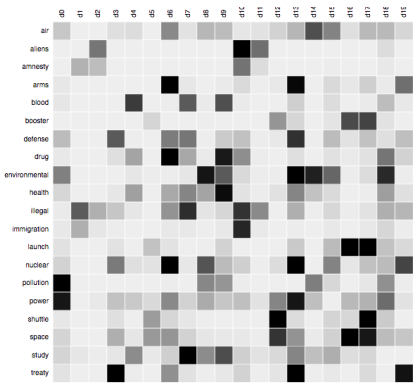


Figure 1.16: Example of a term-document matrix, where raw counts have been replaced by their TF-IDF values (see Sec. 1.5.4.2). Darker cells are larger values. From `https://bit.ly/2kByLQI`. *Used with kind permission of Christoph Carl Kling.*

1.5.4.3 Word embeddings

Although the TF-IDF transformation improves the vector representation of words by placing more weight on "informative" words and less on "uninformative" words, it does not overcome the fundamental issue that semantically similar words, such as "man" and "woman", may be further apart (in vector space) than semantically dissimilar words, such as "man" and "banana". Thus the assumption that points that are close in input space should have similar outputs, which is implicitly made by logistic regression models, is invalid.

The standard way to solve this problem is to use **word embeddings**, in which we map each sparse one-hot vector, $\mathbf{x}_{n,t} \in \{0,1\}^V$, to a lower-dimensional dense vector, $\mathbf{e}_{n,t} \in \mathbb{R}^K$ using $\mathbf{e}_{nt} = \mathbf{E}\mathbf{x}_{nt}$, where $\mathbf{E}$ is learned such that semantically similar words are placed close by. There are many ways to learn such embeddings, as we discuss in Sec. 20.5.

Once we have an embedding matrix, we can represent a variable-length text document as a bag **bag of word embeddings**. We can then convert this to a fixed length vector by summing (or averaging) the embeddings:

$$\overline{\mathbf{e}}_n = \sum_{t=1}^{T} \mathbf{e}_{nt} = \mathbf{E}\tilde{\mathbf{x}}_n \tag{1.37}$$

where $\tilde{\mathbf{x}}_n$ is the bag of words representation from Eq. (1.35). Thus the overall model has the form

$$p(y = c|\mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{S}_c(\mathbf{W}\mathbf{E}\tilde{\mathbf{x}}_n) \tag{1.38}$$

We often use a **pre-trained word embedding** matrix $\mathbf{E}$, in which case the model is linear in $\mathbf{W}$. See also Sec. 15.7 for a discussion of contextual word embeddings.

1.5.4.4 Dealing with novel words

At test time, the model may encounter a completely novel word that it has not seen before. This is known as the **out of vocabulary** or **OOV** problem. Such novel words are bound to occur, because

the set of words is an **open class**. For example, the set of proper nouns (names of people and places) is unbounded.

A standard heuristic to solve this problem is to replace all novel words with the special symbol **UNK**, which stands for "unknown". However, this loses information. For example, if we encounter the word "athazagoraphobia", we may guess it means "fear of something", since phobia is a common suffix in English (derived from Greek) to mean "fear of". (It turns out that athazagoraphobia means "fear of being forgotten about or ignored".)

We could work at the character level, but this would require the model to learn how to group common letter combinations together into words. It is better to leverage the fact that words have substructure, and then to take as input **subword units** or **wordpieces** [SHB16; Wu+16]; these are often created using a method called **byte-pair encoding** [Gag94], which is a form of data compression that creates new symbols to represent common substrings.

1.5.5 Handling missing data

Sometimes we may have **missing data**, in which parts of the input $\mathbf{x}$ or output y may be unknown. If the output is unknown during training, the example is unlabeled; we consider such semi-supervised learning scenarios in Sec. 19.3. We therefore focus on the case where some of the input features may be missing, either at training or testing time, or both.

To model this, let $\mathbf{M}$ be an $N \times D$ matrix of binary variables, where $M_{nd} = 1$ if feature d in example n is missing, and $M_{nd} = 0$ otherwise. Let $\mathbf{X}_v$ be the visible parts of the input feature matrix, corresponding to $M_{nd} = 0$, and $\mathbf{X}_h$ be the missing parts, corresponding to $M_{nd} = 1$. Let $\mathbf{Y}$ be the output label matrix, which we assume is fully observed. If we assume $p(\mathbf{M}|\mathbf{X}_v, \mathbf{X}_h, \mathbf{Y}) = p(\mathbf{M})$, we say the data is **missing completely at random** or **MCAR**, since the missingness does not depend on the hidden or observed features. If we assume $p(\mathbf{M}|\mathbf{X}_v, \mathbf{X}_h, \mathbf{Y}) = p(\mathbf{M}|\mathbf{X}_v, \mathbf{Y})$, we say the data is **missing at random** or **MAR**, since the missingness does not depend on the hidden features, but may depend on the visible features. If neither of these assumptions hold, we say the data is **not missing at random** or **NMAR**.

In the MCAR and MAR cases, we can ignore the missingess mechanism, since it tells us nothing about the hidden features. However, in the NMAR case, we need to model the **missing data mechanism**, since the lack of information may be informative. For example, the fact that someone did not fill out an answer to a sensitive question on a survey (e.g., "Do you have COVID?") could be informative about the underlying value. See e.g., [LR87; Mar08] for more information on missing data models.

In this book, we will always make the MAR assumption. However, even with this assumption, we cannot directly use a discriminative model, such as a DNN, when we have missing input features, since the input $\mathbf{x}$ will have some unknown values. A common heuristic is called **mean value imputation**, in which missing values are replaced by their empirical mean. More generally, we can fit a generative model to the input, and use that to **fill in** the missing values. We briefly discuss some suitable generative models for this task in Chapter 20, and in more detail in the sequel to this book, [Mur22].

1.6 Discussion

In this section, we situate ML and this book into a larger context.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

1.6.1 The relationship between ML and other fields

There are several subcommunities that work on ML-related topics, each of which have different names. The field of **predictive analytics** is similar to supervised learning (in particular, classification and regression), but focuses more on business applications. **Data mining** covers both supervised and unsupervised machine learning, but focuses more on structured data, usually stored in large commercial databases. **Data science** uses techniques from machine learning and statistics, but also emphasizes other topics, such as data integration, data visualization, and working with domain experts, often in an iterative feedback loop (see e.g., [BS17]). The difference between these areas is often just one of terminology.[11]

ML is also very closely related to the field of **statistics**. Indeed, Jerry Friedman, a famous statistics professor at Stanford, said[12]

> [If the statistics field had] incorporated computing methodology from its inception as a fundamental tool, as opposed to simply a convenient way to apply our existing tools, many of the other data related fields [such as ML] would not have needed to exist — they would have been part of statistics. — Jerry Friedman [Fri97b]

Machine learning is also related to **artificial intelligence** (**AI**). Historically, the field of AI assumed that we could program "intelligence" by hand (see e.g., [RN10; PM17]), but this approach has largely failed to live up to expectations, mostly because it proved to be too hard to explicitly encode all the knowledge such systems need. Consequently, there is renewed interest in using ML to help an AI system acquire its own knowledge. (Indeed the connections are so close that sometimes the terms "ML" and "AI" are used interchangeably.)

1.6.2 Structure of the book

We have seen that ML is closely related to many other subjects in mathematics, statistics, computer science, etc. It can be hard to know where to start.

In this book, we take one particular path through this interconnected landscape, using probability theory as our unifying lens. We cover statistical foundations in Part I, supervised learning in Part II–Part IV, and unsupervised learning in Part V. For more information on these (and other) topics, please see the sequel to this book, [Mur22],

In addition to the book, you may find the online Python notebooks that accompany this book helpful. See `http://probml.ai` for details.

1.6.3 Caveats

In this book, we will see how machine learning can be used to create systems that can (attempt to) predict outputs given inputs, and these predictions can then be used to choose actions so as to minimize expected loss. When designing such systems, it can be hard to design a loss function that correctly specifies all of our preferences; this can result in "**reward hacking**" in which the machine optimizes the reward function we give it, but then we realize that the function did not capture various

11. See `https://developers.google.com/machine-learning/glossary/` for a useful "ML glossary".
12. Quoted in `https://brenocon.com/blog/2008/12/statistics-vs-machine-learning-fight/`

constraints or preferences that we forgot to specify [Wei76; Amo+16; D'A+20]. (This is particularly important when tradeoffs need to be made between multiple objectives.)

Reward hacking has raised various concerns in the context of **AI ethics** and **AI safety** (see e.g., [KR19; Lia20]). Russell [Rus19] proposes to solve this problem by not specifying the reward function, but instead forcing the machine to infer the reward by observing human behavior, an approach known as **inverse reinforcement learning**. However, emulating current or past human behavior too closely may be undesirable, and can be biased by the data that is available for training (see e.g., [Pau+20]).

Another approach is to view ML as a tool for building adaptive components which form part of a larger system. Such a system should be designed and regulated in the same way that we design and regulate other complex, semi-autonomous human artefacts, such as aeroplanes, online trading platforms or medical diagnostic systems (c.f. [Jor19]). ML plays a key role in such systems, but additional checks and balances should be put in place to encode our prior knowledge and preferences, and to ensure the system acts in the way that we intended.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Part I

Foundations

2 Probability: univariate models

2.1 Introduction

In this chapter, we give a brief introduction to the basics of probability theory. There are many good books that go into more detail, see e.g., [GS97; BT08].

2.1.1 What is probability?

> Probability theory is nothing but common sense reduced to calculation. — Pierre Laplace, 1812

We are all comfortable saying that the probability that a (fair) coin will land heads is 50%. But what does this mean? There are actually two different interpretations of probability. One is called the **frequentist** interpretation. In this view, probabilities represent long run frequencies of **events** that can happen multiple times. For example, the above statement means that, if we flip the coin many times, we expect it to land heads about half the time.[1]

The other interpretation is called the **Bayesian** interpretation of probability. In this view, probability is used to quantify our **uncertainty** or ignorance about something; hence it is fundamentally related to information rather than repeated trials [Jay03; Lin06]. In the Bayesian view, the above statement means we believe the coin is equally likely to land heads or tails on the next toss.

One big advantage of the Bayesian interpretation is that it can be used to model our uncertainty about one-off events that do not have long term frequencies. For example, we might want to compute the probability that the polar ice cap will melt by 2030 CE. This event will happen zero or one times, but cannot happen repeatedly. Nevertheless, we ought to be able to quantify our uncertainty about this event; based on how probable we think this event is, we can decide how to take the optimal action, as discussed in Chapter 5. We shall therefore adopt the Bayesian interpretation in this book. Fortunately, the basic rules of probability theory are the same, no matter which interpretation is adopted.

2.1.2 Types of uncertainty

The uncertainty in our predictions can arise for two fundamentally different reasons. The first is due to our ignorance of the underlying hidden causes or mechanism generating our data. This is

1. Actually, the Stanford statistician (and former professional magician) Persi Diaconis has shown that a coin is about 51% likely to land facing the same way up as it started, due to the physics of the problem [DHM07].

called **epistemic uncertainty**, since epistemology is the philosophical term used to describe the study of knowledge. However, a simpler term for this is **model uncertainty**. The second kind of uncertainty arises from intrinsic variability, which cannot be reduced even if we collect more data. This is sometimes called **aleatoric uncertainty** [Hac75; KD09], derived from the Latin word for "dice", although a simpler term would be **data uncertainty**. As a concrete example, consider tossing a fair coin. We might know for sure that the probability of heads is $p = 0.5$, so there is no epistemic uncertainty, but we still cannot perfectly predict the outcome.

This distinction can be important for applications such as active learning. A typical strategy is to query examples for which $\mathbb{H}(p(y|\mathbf{x}, \mathcal{D}))$ is large (where $\mathbb{H}(p)$ is the entropy, discussed in Sec. 6.1). However, this could be due to uncertainty about the parameters, i.e., large $\mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D}))$, or just due to inherent label noise or variability, corresponding to large entropy of $p(y|\mathbf{x}, \boldsymbol{\theta})$. See [Osb16] for further discussion.

2.1.3 Probability as an extension of logic

In this section, we review the basic rules of probability, following the presentation of [Jay03], in which we view probability as an extension of **Boolean logic**.

2.1.3.1 Probability of an event

We define an **event**, denoted by the binary variable A, as some state of the world that either holds or does not hold. For example, A might be event "it will rain tomorrow", or "it rained yesterday", or "the label is $y = 1$", or "the parameter θ is between 1.5 and 2.0", etc. The expression $\Pr(A)$ denotes the probability with which you believe event A is true (or the long run fraction of times that A will occur). We require that $0 \leq \Pr(A) \leq 1$, where $\Pr(A) = 0$ means the event definitely will not happen, and $\Pr(A) = 1$ means the event definitely will happen. We write $\Pr(\overline{A})$ to denote the probability of event A not happening; this is defined to $\Pr(\overline{A}) = 1 - \Pr(A)$.

2.1.3.2 Probability of a conjunction of two events

We denote the **joint probability** of events A and B both happening as follows:

$$\Pr(A \wedge B) = \Pr(A, B) \tag{2.1}$$

If A and B are independent events, we have

$$\Pr(A, B) = \Pr(A) \Pr(B) \tag{2.2}$$

For example, suppose X and Y are chosen uniformly at random from the set $\mathcal{X} = \{1, 2, 3, 4\}$. Let A be the event that $X \in \{1, 2\}$, and B be the event that $Y \in \{3\}$. Then we have $\Pr(A, B) = \Pr(A) \Pr(B) = \frac{1}{2} \cdot \frac{1}{4}$.

2.1.3.3 Probability of a union of two events

The probability of event A or B happening is given by

$$\Pr(A \vee B) = \Pr(A) + \Pr(B) - \Pr(A \wedge B) \tag{2.3}$$

If the events are mutually exclusive (so they cannot happen at the same time), we get

$$\Pr(A \vee B) = \Pr(A) + \Pr(B) \tag{2.4}$$

For example, suppose X is chosen uniformly at random from the set $\mathcal{X} = \{1, 2, 3, 4\}$. Let A be the event that $X \in \{1, 2\}$ and B be the event that $X \in \{3\}$. Then we have $\Pr(A \vee B) = \frac{2}{4} + \frac{1}{4}$.

2.1.3.4 Conditional probability of one event given another

We define the **conditional probability** of event B happening given that A has occurred as follows:

$$\Pr(B|A) \triangleq \frac{\Pr(A, B)}{\Pr(A)} \tag{2.5}$$

This is not defined if $\Pr(A) = 0$, since we cannot condition on an impossible event.

2.1.3.5 Conditional independence of events

We say that event A is **conditionally independent** of event B if

$$\Pr(A|B) = \Pr(A) \tag{2.6}$$

Of course, this is a symmetric relationship, so we also have $\Pr(B|A) = \Pr(B)$. Hence $\Pr(A, B) = \Pr(A)\Pr(B)$. We use the notation $A \perp B$ to denote this property.

Two events, A and B, are conditionally independent given a third event C if $\Pr(A|B, C) = \Pr(A|C)$. Equivalently, we can write this as $\Pr(A, B|C) = \Pr(A|C)\Pr(B|C)$. This is written as $A \perp B|C$.

For example, let A be the event "the smoke detector is on", B be the event "there is a fire nearby", and C be the event "there is smoke nearby". Clearly A and B depend on each other, but $A \perp B|C$, since if there is smoke nearby (i.e., event C is true), the detector will turn on, regardless of whether there is a fire. Events are often dependent on each other, but may be rendered independent if we condition on the relevant intermediate variables, as we discuss in more detail later in this chapter.

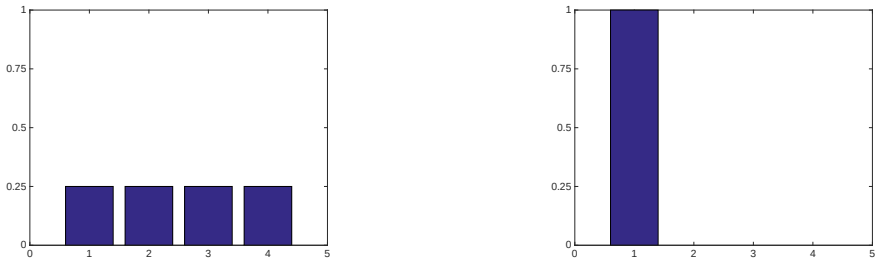
2.2 Random variables

Suppose X represents some unknown quantity of interest, such as which way a dice will land when we roll it, or the temperature outside your house at the current time. If the value of X is unknown and/or could change, we call it a **random variable** or **rv**. The set of possible values, denoted $\mathcal{X}$, is known as the **sample space** or **state space**. An **event** is a set of outcomes from a given sample space. For example, if X represents the face of a dice that is rolled, so $\mathcal{X} = \{1, 2, \ldots, 6\}$, the event of "seeing a 1" is denoted $X = 1$, the event of "seeing an odd number" is denoted $X \in \{1, 3, 5\}$, the event of "seeing a number between 1 and 3" is denoted $1 \leq X \leq 3$, etc.

2.2.1 Discrete random variables

If the sample space $\mathcal{X}$ is finite or countably infinite, then X is called a **discrete random variable**. In this case, we denote the probability of the event that X has value x by $\Pr(X = x)$. We define the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 2.1: Some discrete distributions on the state space $\mathcal{X} = \{1, 2, 3, 4\}$. (a) A uniform distribution with $p(x = k) = 1/4$. (b) A degenerate distribution (delta function) that puts all its mass on $x = 1$. Generated by code at figures.probml.ai/book1/2.1.

probability mass function or **pmf** as a function which computes the probability of events which correspond to setting the rv to each possible value:

$$p(x) \triangleq \Pr(X = x) \tag{2.7}$$

The pmf satisfies the properties $0 \leq p(x) \leq 1$ and $\sum_{x \in \mathcal{X}} p(x) = 1$.

If X has a finite number of values, say K, the pmf can be represented as a list of K numbers, which we can plot as a histogram. For example, Fig. 2.1 shows two pmf's defined on $\mathcal{X} = \{1, 2, 3, 4\}$. On the left we have a uniform distribution, $p(x) = 1/4$, and on the right, we have a degenerate distribution, $p(x) = \mathbb{I}(x = 1)$, where $\mathbb{I}()$ is the binary indicator function. Thus the distribution in Fig. 2.1(b) represents the fact that X is always equal to the value 1. (Thus we see that random variables can also be constant.)

2.2.2 Continuous random variables

If $X \in \mathbb{R}$ is a real-valued quantity, it is called a **continuous random variable**. In this case, we can no longer create a finite (or countable) set of distinct possible values it can take on. However, there are a countable number of *intervals* which we can partition the real line into. If we associate events with X being in each one of these intervals, we can use the methods discussed above for discrete random variables. By allowing the size of the intervals to shrink to zero, we can represent the probability of X taking on a specific real value, as we show below.

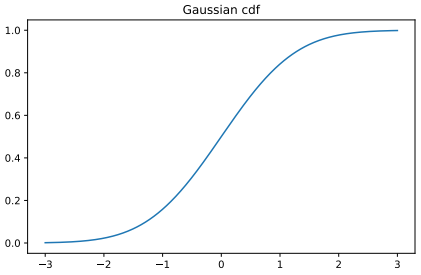
2.2.2.1 Cumulative distribution function (cdf)

Define the events $A = (X \leq a)$, $B = (X \leq b)$ and $C = (a < X \leq b)$, where $a < b$. We have that $B = A \vee C$, and since A and C are mutually exclusive, the sum rules gives

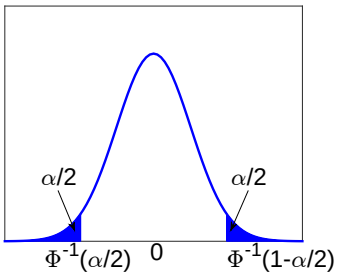
$$\Pr(B) = \Pr(A) + \Pr(C) \tag{2.8}$$

and hence the probability of being in interval C is given by

$$\Pr(C) = \Pr(B) - \Pr(A) \tag{2.9}$$



(a)



(b)

Figure 2.2: (a) Plot of the cdf for the standard normal, $\mathcal{N}(0,1)$. Generated by code at figures.probml.ai/book1/2.2. (b) Corresponding pdf. The shaded regions each contain $\alpha/2$ of the probability mass. Therefore the nonshaded region contains $1-\alpha$ of the probability mass. The leftmost cutoff point is $\Phi^{-1}(\alpha/2)$, where Φ is the cdf of the Gaussian. By symmetry, the rightmost cutoff point is $\Phi^{-1}(1-\alpha/2) = -\Phi^{-1}(\alpha/2)$. Generated by code at figures.probml.ai/book1/2.2.

In general, we define the **cumulative distribution function** or **cdf** of the rv X as follows:

$$P(x) \triangleq \Pr(X \leq x) \tag{2.10}$$

(Note that we use a capital P to represent the cdf.) Using this, we can compute the probability of being in any interval as follows:

$$\Pr(a < X \leq b) = P(b) - P(a) \tag{2.11}$$

Cdf's are monotonically non-decreasing functions. See Fig. 2.2a for an example, where we illustrate the cdf of a standard normal distribution, $\mathcal{N}(x|0,1)$; this cdf is usually denoted by $\Phi(x)$.

2.2.2.2 Probability density function (pdf)

We define the **probability density function** or **pdf** as the derivative of the cdf:

$$p(x) \triangleq \frac{d}{dx} P(x) \tag{2.12}$$

See Fig. 2.2b for an example, where we illustrate the pdf of a univariate Gaussian. (Note that this derivative does not always exist, in which case the pdf is not defined.)

Given a pdf, we can compute the probability of a continuous variable being in a finite interval as follows:

$$\Pr(a < X \leq b) = \int_a^b p(x)dx = P(b) - P(a) \tag{2.13}$$

As the size of the interval gets smaller, we can write

$$\Pr(x \leq X \leq x + dx) \approx p(x)dx \tag{2.14}$$

Intuitively, this says the probability of X being in a small interval around x is the density at x times the width of the interval.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

2.2.2.3 Quantiles

If the cdf P is strictly monotonically increasing, it has an inverse, called the **inverse cdf**, or **percent point function** (**ppf**), or **quantile function**.

If P is the cdf of X, then $P^{-1}(q)$ is the value x_q such that $\Pr(X \leq x_q) = q$; this is called the q'th **quantile** of P. The value $P^{-1}(0.5)$ is the **median** of the distribution, with half of the probability mass on the left, and half on the right. The values $P^{-1}(0.25)$ and $P^{-1}(0.75)$ are the lower and upper **quartiles**.

For example, let Φ be the cdf of the Gaussian distribution $\mathcal{N}(0, 1)$, and Φ^{-1} be the inverse cdf. Then points to the left of $\Phi^{-1}(\alpha/2)$ contain $\alpha/2$ of the probability mass, as illustrated in Fig. 2.2b. By symmetry, points to the right of $\Phi^{-1}(1 - \alpha/2)$ also contain $\alpha/2$ of the mass. Hence the central interval $(\Phi^{-1}(\alpha/2), \Phi^{-1}(1 - \alpha/2))$ contains $1 - \alpha$ of the mass. If we set $\alpha = 0.05$, the central 95% interval is covered by the range

$$(\Phi^{-1}(0.025), \Phi^{-1}(0.975)) = (-1.96, 1.96) \tag{2.15}$$

If the distribution is $\mathcal{N}(\mu, \sigma^2)$, then the 95% interval becomes $(\mu - 1.96\sigma, \mu + 1.96\sigma)$. This is often approximated by writing $\mu \pm 2\sigma$.

2.2.3 Sets of related random variables

In this section, we discuss distributions over sets of related random variables.

Suppose, to start, that we have two random variables, X and Y. We can define the **joint distribution** of two random variables using $p(x, y) = p(X = x, Y = y)$ for all possible values of X and Y. If both variables have finite cardinality, we can represent the joint distribution as a 2d table, all of whose entries sum to one. For example, consider the following example with two binary variables:

$p(X, Y)$	$Y = 0$	$Y = 1$
$X = 0$	0.2	0.3
$X = 1$	0.3	0.2

If two variables are conditionally independent, we can represent the joint as the product of the two marginals. If both variables have finite cardinality, we can factorize the 2d joint table into a product of two 1d vectors, as shown in Fig. 2.3.

Given a joint distribution, we define the **marginal distribution** of an rv as follows:

$$p(X = x) = \sum_y p(X = x, Y = y) \tag{2.16}$$

where we are summing over all possible states of Y. This is sometimes called the **sum rule** or the **rule of total probability**. We define $p(Y = y)$ similarly. For example, from the above 2d table, we see $p(X = 0) = 0.2 + 0.3 = 0.5$ and $p(Y = 0) = 0.2 + 0.3 = 0.5$. (The term "marginal" comes from the accounting practice of writing the sums of rows and columns on the side, or margin, of a table.)

We define the **conditional distribution** of an rv using

$$p(Y = y|X = x) = \frac{p(X = x, Y = y)}{p(X = x)} \tag{2.17}$$

We can rearrange this equation to get

$$p(x, y) = p(x)p(y|x) \tag{2.18}$$

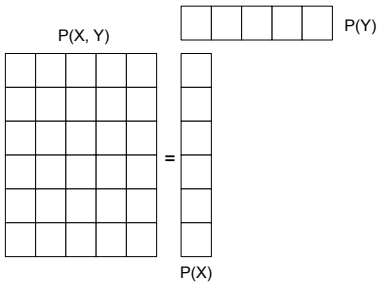


Figure 2.3: Computing $p(x, y) = p(x)p(y)$, where $X \perp Y$. Here X and Y are discrete random variables; X has 6 possible states (values) and Y has 5 possible states. A general joint distribution on two such variables would require $(6 \times 5) - 1 = 29$ parameters to define it (we subtract 1 because of the sum-to-one constraint). By assuming (unconditional) independence, we only need $(6 - 1) + (5 - 1) = 9$ parameters to define $p(x, y)$.

This is called the **product rule**.

By extending the product rule to D variables, we get the **chain rule of probability**:

$$p(\mathbf{x}_{1:D}) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)p(x_4|x_1, x_2, x_3) \dots p(x_D|\mathbf{x}_{1:D-1}) \tag{2.19}$$

This provides a way to create a high dimensional joint distribution from a set of conditional distributions. We discuss this in more detail in Sec. 3.5.

2.2.4 Independence and conditional independence

We say X and Y are **unconditionally independent** or **marginally independent**, denoted $X \perp Y$, if we can represent the joint as the product of the two marginals (see Figure 2.3), i.e.,

$$X \perp Y \iff p(X, Y) = p(X)p(Y) \tag{2.20}$$

In general, we say a set of variables $X_1, \dots, X_n$ is **independent** if the joint can be written as a product of marginals, i.e.,

$$p(X_1, \dots, X_n) = \prod_{i=1}^{n} p(X_i) \tag{2.21}$$

Unfortunately, unconditional independence is rare, because most variables can influence most other variables. However, usually this influence is mediated via other variables rather than being direct. We therefore say X and Y are **conditionally independent** (CI) given Z iff the conditional joint can be written as a product of conditional marginals:

$$X \perp Y \mid Z \iff p(X, Y|Z) = p(X|Z)p(Y|Z) \tag{2.22}$$

We can write this assumption as a graph $X - Z - Y$, which captures the intuition that all the dependencies between X and Y are mediated via Z. By using larger graphs, we can define complex joint distributions; these are known as **graphical models**, and are discussed in Sec. 3.5.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

2.2.5 Moments of a distribution

In this section, we describe various summary statistics that can be derived from a probability distribution (either a pdf or pmf).

2.2.5.1 Mean of a distribution

The most familiar property of a distribution is its **mean**, or **expected value**, often denoted by μ. For continuous rv's, the mean is defined as follows:

$$\mathbb{E}[X] \triangleq \int_{\mathcal{X}} x\, p(x) dx \tag{2.23}$$

If the integral is not finite, the mean is not defined; we will see some examples of this later.

For discrete rv's, the mean is defined as follows:

$$\mathbb{E}[X] \triangleq \sum_{x \in \mathcal{X}} x\, p(x) \tag{2.24}$$

However, this is only meaningful if the values of x are ordered in some way (e.g., if they represent integer counts).

Since the mean is a linear operator, we have

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b \tag{2.25}$$

This is called the **linearity of expectation**.

For a set of n independent random variables, one can show that the expectation of their sum and product is as follows:

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i] \tag{2.26}$$

$$\mathbb{E}\left[\prod_{i=1}^{n} X_i\right] = \prod_{i=1}^{n} \mathbb{E}[X_i] \tag{2.27}$$

2.2.5.2 Variance of a distribution

The **variance** is a measure of the "spread" of a distribution, often denoted by σ^2. This is defined as follows:

$$\mathbb{V}[X] \triangleq \mathbb{E}\left[(X-\mu)^2\right] = \int (x-\mu)^2 p(x) dx \tag{2.28}$$

$$= \int x^2 p(x) dx + \mu^2 \int p(x) dx - 2\mu \int x p(x) dx = \mathbb{E}\left[X^2\right] - \mu^2 \tag{2.29}$$

from which we derive the useful result

$$\mathbb{E}\left[X^2\right] = \sigma^2 + \mu^2 \tag{2.30}$$

The **standard deviation** is defined as

$$\text{std}\left[X\right] \triangleq \sqrt{\mathbb{V}\left[X\right]} = \sigma \tag{2.31}$$

This is useful since it has the same units as X itself.

The variance of a shifted and scaled version of a random variable is given by

$$\mathbb{V}\left[aX + b\right] = a^2\mathbb{V}\left[X\right] \tag{2.32}$$

If we have a set of n independent random variables, the variance of their sum is given by the sum of their variances:

$$\mathbb{V}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{V}\left[X_i\right] \tag{2.33}$$

The variance of their product can also be derived, as follows:

$$\mathbb{V}\left[\prod_{i=1}^{n} X_i\right] = \mathbb{E}\left[(\prod_i X_i)^2\right] - (\mathbb{E}\left[\prod_i X_i\right])^2 \tag{2.34}$$

$$= \mathbb{E}\left[\prod_i X_i^2\right] - (\prod_i \mathbb{E}\left[X_i\right])^2 \tag{2.35}$$

$$= \prod_i \mathbb{E}\left[X_i^2\right] - \prod_i (\mathbb{E}\left[X_i\right])^2 \tag{2.36}$$

$$= \prod_i (\mathbb{V}\left[X_i\right] + (\mathbb{E}\left[X_i\right])^2) - \prod_i (\mathbb{E}\left[X_i\right])^2 \tag{2.37}$$

$$= \prod_i (\sigma_i^2 + \mu_i^2) - \prod_i \mu_i^2 \tag{2.38}$$

2.2.5.3 Mode of a distribution

The **mode** of a distribution is the value with the highest probability mass or probability density:

$$\mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, p(\mathbf{x}) \tag{2.39}$$

If the distribution is **multimodal**, this may not be unique, as illustrated in Fig. 2.4. Furthermore, even if there is a unique mode, this point may not be a good summary of the distribution.

2.2.5.4 Conditional moments

When we have two or more dependent random variables, we can compute the moments of one given knowledge of the other. For example, the **law of iterated expectations**, also called the **law of total expectation**, tells us that

$$\mathbb{E}\left[X\right] = \mathbb{E}\left[\mathbb{E}\left[X|Y\right]\right] \tag{2.40}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

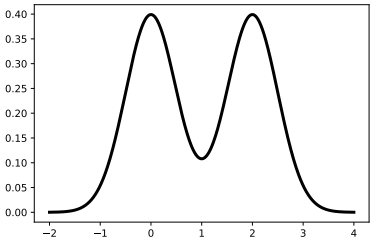


Figure 2.4: Illustration of a mixture of two 1d Gaussians, $p(x) = 0.5\mathcal{N}(x|0, 0.5) + 0.5\mathcal{N}(x|2, 0.5)$. Generated by code at figures.probml.ai/book1/2.4.

To prove this, let us suppose, for simplicity, that X and Y are both discrete rv's. Then we have

$$\mathbb{E}\left[\mathbb{E}\left[X|Y\right]\right] = \mathbb{E}\left[\sum_x xp(X = x|Y)\right] \tag{2.41}$$

$$= \sum_y \left[\sum_x xp(X = x|Y)\right] p(Y = y) = \sum_{x,y} xp(X = x, Y = y) = \mathbb{E}\left[X\right] \tag{2.42}$$

To give a more intuitive explanation, consider the following simple example.[2] Let X be the lifetime duration of a lightbulb, and let Y be the factory the lightbulb was produced in. Suppose $\mathbb{E}\left[X|Y = 1\right] = 5000$ and $\mathbb{E}\left[X|Y = 2\right] = 4000$, indicating that factory 1 produces longer lasting bulbs. Suppose factory 1 supplies 60% of the lightbulbs, so $p(Y = 1) = 0.6$ and $p(Y = 2) = 0.4$. Then the overal expected duration of a random lightbulb is given by

$$\mathbb{E}\left[X\right] = \mathbb{E}\left[X|Y = 1\right] p(Y = 1) + \mathbb{E}\left[X|Y = 2\right] p(Y = 2) = 5000 \times 0.6 + 4000 \times 0.4 = 4600 \tag{2.43}$$

There is a similar formula for the variance. In particular, the **law of total variance**, also called the **conditional variance formula**, tells us that

$$\mathbb{V}\left[X\right] = \mathbb{E}\left[\mathbb{V}\left[X|Y\right]\right] + \mathbb{V}\left[\mathbb{E}\left[X|Y\right]\right] \tag{2.44}$$

To see this, let us define the conditional moments, $\mu_{X|Y} = \mathbb{E}\left[X|Y\right]$, $s_{X|Y} = \mathbb{E}\left[X^2|Y\right]$, and $\sigma^2_{X|Y} = \mathbb{V}\left[X|Y\right] = s_{X|Y} - \mu^2_{X|Y}$, which are functions of Y (and therefore are random quantities). Then we have

$$\mathbb{V}\left[X\right] = \mathbb{E}\left[X^2\right] - \left(\mathbb{E}\left[X\right]\right)^2 = \mathbb{E}\left[s_{X|Y}|Y\right] - \left(\mathbb{E}\left[\mu_{X|Y}|Y\right]\right)^2 \tag{2.45}$$

$$= \mathbb{E}\left[\sigma^2_{X|Y}|Y\right] + \mathbb{E}\left[\mu^2_{X|Y}|Y\right] - \left(\mathbb{E}\left[\mu_{X|Y}|Y\right]\right)^2 \tag{2.46}$$

$$= E_Y[\mathbb{V}\left[X|Y\right]] + \mathbb{V}_Y[\mu_{X|Y}] \tag{2.47}$$

2. This example is from https://en.wikipedia.org/wiki/Law_of_total_expectation, but with modified notation.

To get some intuition for these formulas, consider a mixture of K univariate Gaussians. Let Y be the hidden indicator variable that specifies which mixture component we are using, and let $X = \sum_{y=1}^{K} \pi_y \mathcal{N}(X|\mu_y, \sigma_y)$. In Fig. 2.4, we have $\pi_1 = \pi_2 = 0.5$, $\mu_1 = 0$, $\mu_2 = 2$, $\sigma_1 = \sigma_2 = 0.5$. Thus

$$\mathbb{E}\left[\mathbb{V}\left[X|Y\right]\right] = \pi_1\sigma_1 + \pi_2\sigma_2 = 0.5 \tag{2.48}$$

$$\mathbb{V}\left[\mathbb{E}\left[X|Y\right]\right] = \pi_1(\mu_1 - \overline{\mu})^2 + \pi_2(\mu_2 - \overline{\mu})^2 = 0.5(0-1)^2 + 0.5(2-1)^2 = 0.5 + 0.5 = 1 \tag{2.49}$$

So we get the intuitive result that the variance of Y is dominated by which centroid it is drawn from (i.e., difference in the means), rather than the local variance around each centroid.

2.3 Bayes' rule

> Bayes's theorem is to the theory of probability what Pythagoras's theorem is to geometry. — Sir Harold Jeffreys, 1973 [Jef73].

In this section, we discuss the basics of **Bayesian inference**. According to the Merriam-Webster dictionary, the term "inference" means "the act of passing from sample data to generalizations, usually with calculated degrees of certainty". The term "Bayesian" is used to refer to inference methods that represent "degrees of certainty" using probability theory, and which leverage **Bayes' rule**[3], to update the degree of certainty given data.

Bayes' rule itself is very simple: it is just a formula for computing the probability distribution over possible values of an unknown (or **hidden**) quantity H given some observed data $Y = y$:

$$p(H = h|Y = y) = \frac{p(H = h)p(Y = y|H = h)}{p(Y = y)} \tag{2.50}$$

This follows automatically from the identity

$$p(h|y)p(y) = p(h)p(y|h) = p(h, y) \tag{2.51}$$

which itself follows from the **product rule of probability**.

In Eq. (2.50), the term $p(H)$ represents what we know about possible values of H before we see any data; this is called the **prior distribution**. (If H has K possible values, then $p(H)$ is a vector of K probabilities, that sum to 1.) The term $p(Y|H = h)$ represents the distribution over the possible outcomes Y we expect to see if $H = h$; this is called the **observation distribution**. When we evaluate this at a point corresponding to the actual observations, y, we get the function $p(Y = y|H = h)$, which is called the **likelihood**. (Note that this is a function of h, since y is fixed, but it is not a probability distribution, since it does not sum to one.) Multiplying the prior distribution $p(H = h)$ by the likelihood function $p(Y = y|H = h)$ for each h gives the unnormalized joint distribution $p(H = h, Y = y)$. We can convert this into a normalized distribution by dividing by $p(Y = y)$, which is known as the **marginal likelihood**, since it is computed by marginalizing over the unknown H:

$$p(Y = y) = \sum_{h' \in \mathcal{H}} p(H = h')p(Y = y|H = h') = \sum_{h' \in \mathcal{H}} p(H = h', Y = y) \tag{2.52}$$

3. Thomas Bayes (1702–1761) was an English mathematician and Presbyterian minister. For a discussion of whether to spell this as Bayes' rule or Bayes's rule, see https://bit.ly/2kDtLuK.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

		Observation	
		0	1
Truth	0	TNR=Specificity=0.975	FPR=1-TNR=0.025
	1	FNR=1-TPR=0.125	TPR=Sensitivity=0.875

Table 2.1: Likelihood function $p(Y|H)$ for a binary observation Y given two possible hidden states H. Each row sums to one. Abbreviations: TNR is true negative rate, TPR is true positive rate, FNR is false negative rate, FPR is false positive rate.

Normalizing the joint distribution by computing $p(H = h, Y = y)/p(Y = y)$ for each h gives the **posterior distribution** $p(H = h|Y = y)$; this represents our new **belief state** about the possible values of H.

We can summarize Bayes rule in words as follows:

$$\text{posterior} \propto \text{prior} \times \text{likelihood} \tag{2.53}$$

Here we use the symbol $\propto$ to denote "proportional to", since we are ignoring the denominator, which is just a constant, independent of H. Using Bayes rule to update a distribution over unknown values of some quantity of interest, given relevant observed data, is called **Bayesian inference**, or **posterior inference**. It can also just be called **probabilistic inference**.

Below we give some simple examples of Bayesian inference in action. We will see many more interesting examples later in this book.

2.3.1 Example: Testing for COVID-19

Suppose you think you may have contracted **COVID-19**, which is an infectious disease caused by the **SARS-CoV-2** virus. You decide to take a diagnostic test, and you want to use its result to determine if you are infected or not.

Let $H = 1$ be the event that you are infected, and $H = 0$ be the event you are not infected. Let $Y = 1$ if the test is positive, and $Y = 0$ if the test is negative. We want to compute $p(H = h|Y = y)$, for $h \in \{0, 1\}$, where y is the observed test outcome. (We will write the distribution of values, $[p(H = 0|Y = y), p(H = 1|Y = y)]$ as $p(H|y)$, for brevity.) We can think of this as a form of **binary classification**, where H is the unknown class label, and y is the feature vector.

First we must specify the likelihood. This quantity obviously depends on how reliable the test is. There are two key parameters. The **sensitivity** (aka **true positive rate**) is defined as $p(Y = 1|H = 1)$, i.e., the probability of a positive test given that the truth is positive. The **false negative rate** is defined as one minus the sensitivity. The **specificity** (aka **true negative rate**) is defined as $p(Y = 0|H = 0)$, i.e., the probability oof a negative test given that the truth is negative. The **false positive rate** is defined as one minus the specificity. We summarize all these quantities in Table 2.1. (See Sec. 5.1.3.1 for more details.) Following https://nyti.ms/31MTZgV, we set the sensitivity to 87.5% and the specificity to 97.5%.

Next we must specify the prior. The quantity $p(H = 1)$ represents the **prevalence** of the disease in the area in which you live. We set this to $p(H = 1) = 0.1$ (i.e., 10%), which was the prevalence in New York City in Spring 2020. (This example was chosen to match the numbers in https://nyti.ms/31MTZgV.)

Now suppose you test positive. We have

$$p(H=1|Y=1) = \frac{p(Y=1|H=1)p(H=1)}{p(Y=1|H=1)p(H=1)+p(Y=1|H=0)p(H=0)} \tag{2.54}$$

$$= \frac{\text{TPR} \times \text{prior}}{\text{TPR} \times \text{prior} + \text{FPR} \times (1-\text{prior})} \tag{2.55}$$

$$= \frac{0.875 \times 0.1}{0.875 \times 0.1 + 0.025 \times 0.9} = 0.795 \tag{2.56}$$

So there is a 79.5% chance you are infected.

Now suppose you test negative. The probability you are infected is given by

$$p(H=1|Y=0) = \frac{p(Y=0|H=1)p(H=1)}{p(Y=0|H=1)p(H=1)+p(Y=0|H=0)p(H=0)} \tag{2.57}$$

$$= \frac{\text{FNR} \times \text{prior}}{\text{FNR} \times \text{prior} + \text{TNR} \times (1-\text{prior})} \tag{2.58}$$

$$= \frac{0.125 \times 0.1}{0.125 \times 0.1 + 0.975 \times 0.9} = 0.014 \tag{2.59}$$

So there is just a 1.4% chance you are infected.

Nowadays COVID-19 prevalence is much lower. Suppose we repeat these calculations using a base rate of 1%; now the posteriors reduce to 26% and 0.13% respectively.

The fact that you only have a 26% chance of being infected with COVID-19, even after a positive test, is very counter-intuitive. The reason is that a single positive test is more likely to be a false positive than due to the disease, since the disease is rare. To see this, suppose we have a population of 100,000 people, of whom 1000 are infected. Of those who are infected, $875 = 0.875 \times 100,000$ test positive, and of those who are uninfected, $2475 = 0.025 \times 99,000$ test positive. Thus the total number of positives is $3350 = 865 + 2475$, so the posterior probability of being infected given a positive test is $875/3350 = 0.26$.

Of course, the above calculations assume we know the sensitivity and specificity of the test. See [GC20] for how to apply Bayes rule for diagnostic testing when there is uncertainty about these parameters.

2.3.2 Example: The Monty Hall problem

In this section, we consider a more "frivolous" application of Bayes rule. In particular, we apply it to the famous **Monty Hall problem**.

Imagine a game show with the following rules: There are three doors, labelled 1, 2, 3. A single prize (e.g., a car) has been hidden behind one of them. You get to select one door. Then the gameshow host opens one of the other two doors (not the one you picked), in such a way as to not reveal the prize location. At this point, you will be given a fresh choice of door: you can either stick with your first choice, or you can switch to the other closed door. All the doors will then be opened and you will receive whatever is behind your final choice of door.

For example, suppose you choose door 1, and the gameshow host opens door 3, revealing nothing behind the door, as promised. Should you (a) stick with door 1, or (b) switch to door 2, or (c) does it make no difference?

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Door 1	Door 2	Door 3	Switch	Stay
Car	-	-	Lose	Win
-	Car	-	Win	Lose
-	-	Car	Win	Lose

Table 2.2: 3 possible states for the Monty Hall game, showing that switching doors is two times better (on average) than staying with your original choice. Adapted from Table 6.1 of [PM18].

Intuitively, it seems it should make no difference, since your initial choice of door cannot influence the location of the prize. However, the fact that the host opened door 3 tells us something about the location of the prize, since he made his choice conditioned on the knowledge of the true location and on your choice. As we show below, you are in fact twice as likely to win the prize if you switch to door 2.

To show this, we will use Bayes' rule. Let H_i denote the hypothesis that the prize is behind door i. We make the following assumptions: the three hypotheses H_1, H_2 and H_3 are equiprobable *a priori*, i.e.,

$$P(H_1) = P(H_2) = P(H_3) = \frac{1}{3}. \tag{2.60}$$

The datum we receive, after choosing door 1, is either $Y = 3$ and $Y = 2$ (meaning door 3 or 2 is opened, respectively). We assume that these two possible outcomes have the following probabilities. If the prize is behind door 1, then the host selects at random between $Y = 2$ and $Y = 3$. Otherwise the choice of the host is forced and the probabilities are 0 and 1.

$$\left| \begin{array}{l} P(Y=2|H_1)=\frac{1}{2} \\ P(Y=3|H_1)=\frac{1}{2} \end{array} \right| \left| \begin{array}{l} P(Y=2|H_2)=0 \\ P(Y=3|H_2)=1 \end{array} \right| \left| \begin{array}{l} P(Y=2|H_3)=1 \\ P(Y=3|H_3)=0 \end{array} \right| \tag{2.61}$$

Now, using Bayes' theorem, we evaluate the posterior probabilities of the hypotheses:

$$P(H_i|Y=3) = \frac{P(Y=3|H_i)P(H_i)}{P(Y=3)} \tag{2.62}$$

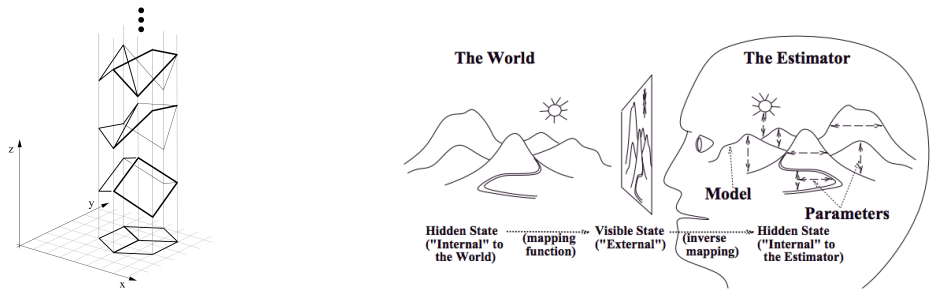
$$\left| P(H_1|Y=3) = \frac{(1/2)(1/3)}{P(Y=3)} \right| P(H_2|Y=3) = \frac{(1)(1/3)}{P(Y=3)} \left| P(H_3|Y=3) = \frac{(0)(1/3)}{P(Y=3)} \right| \tag{2.63}$$

The denominator $P(Y = 3)$ is $P(Y = 3) = \frac{1}{6} + \frac{1}{3} = \frac{1}{2}$. So

$$\left| P(H_1|Y=3) = \frac{1}{3} \right| P(H_2|Y=3) = \frac{2}{3} \left| P(H_3|Y=3) = 0. \right| \tag{2.64}$$

So the contestant should switch to door 2 in order to have the biggest chance of getting the prize. See Table 2.2 for a worked example.

Many people find this outcome surprising. One way to make it more intuitive is to perform a thought experiment in which the game is played with a million doors. The rules are now that the contestant chooses one door, then the game show host opens 999,998 doors in such a way as not to reveal the prize, leaving the *contestant's* selected door and *one other door* closed. The contestant may now stick or switch. Imagine the contestant confronted by a million doors, of which doors 1 and 234,598 have not been opened, door 1 having been the contestant's initial guess. Where do you think the prize is?



(a) (b)

Figure 2.5: (a) Any planar line-drawing is geometrically consistent with infinitely many 3-D structures. From Figure 11 of [SA93]. Used with kind permission of Pawan Sinha. (b) Vision as inverse graphics. The agent (here represented by a human head) has to infer the scene h given the image y using an estimator, which computes $\hat{h}(y) = \text{argmax}_h \, p(h|y)$. From Figure 1 of [Rao99]. Used with kind permission of Rajesh Rao.

2.3.3 Inverse problems *

Probability theory is concerned with predicting a distribution over outcomes y given knowledge (or assumptions) about the state of the world, h. By contrast, **inverse probability** is concerned with inferring the state of the world from observations of outcomes. We can think of this as inverting the $h \to y$ mapping.

For example, consider trying to infer a 3d shape h from a 2d image y. This is a fundamentally **ill-posed** problem, as illustrated in Fig. 2.5a, since there are multiple possible hidden h's consistent with the same observed y. To tackle such **inverse problems**, we can use Bayes' rule to compute the posterior, $p(h|y)$, as we saw above. This requires specifying the **forwards model**, $p(y|h)$, as well as a prior $p(h)$, which can be used to rule out (or downweight) implausible inverse mappings.

Fig. 2.5b illustrates how this idea can be used in the context of computer vision. This approach – known as **vision as inverse graphics** or **analysis by synthesis** — assumes that humans infer the underlying 3d structure of a scene by considering the set of possible 3d models h which best "explain" the observed data y. There is considerable evidence for this [KMY04; YK06; Doy+07; MC19], although the brain has evolved to use various shortcuts to make the computation efficient (see e.g., [Lak+17; Gri20]). We give some simple (2d) examples of this approach in practice in Sec. 3.4.1.2, and discuss the topic in more detail in the sequel to this book, [Mur22].

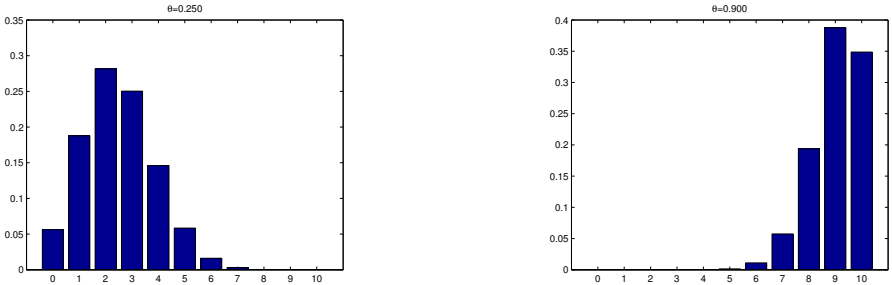
2.4 Bernoulli and binomial distributions

Perhaps the simplest probability distribution is the **Bernoulli distribution**, which can be used to model binary events, as we discuss below.

2.4.1 Definition

Consider tossing a coin, where the probability of event that it lands heads is given by $0 \leq \theta \leq 1$. Let $Y = 1$ denote this event, and let $Y = 0$ denote the event that the coin lands tails. Thus we are

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 2.6: Illustration of the binomial distribution with $N = 10$ and (a) $\theta = 0.25$ and (b) $\theta = 0.9$. Generated by code at figures.probml.ai/book1/2.6.

assuming that $p(Y = 1) = \theta$ and $p(Y = 0) = 1 - \theta$. This is called the **Bernoulli distribution**, and can be written as follows

$$Y \sim \text{Ber}(\theta) \tag{2.65}$$

where the symbol $\sim$ means "is sampled from" or "is distributed as", and Ber refers to Bernoulli. The probability mass function (pmf) of this distribution is defined as follows:

$$\text{Ber}(y|\theta) = \begin{cases} 1-\theta & \text{if } y = 0 \\ \theta & \text{if } y = 1 \end{cases} \tag{2.66}$$

(See Sec. 2.2.1 for details on pmf's.) We can write this in a more concise manner as follows:

$$\text{Ber}(y|\theta) \triangleq \theta^y (1-\theta)^{1-y} \tag{2.67}$$

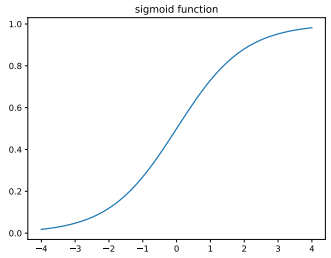
The Bernoulli distribution is a special case of the **binomial distribution**. To explain this, suppose we observe a set of N Bernoulli trials, denoted $y_n \sim \text{Ber}(\cdot|\theta)$, for $n = 1 : N$. Concretely, think of tossing a coin N times. Let us define s to be the total the number of heads, $s \triangleq \sum_{n=1}^{N} \mathbb{I}(y_n = 1)$. The distribution of s is given by the binomial distribution:

$$\text{Bin}(s|N,\theta) \triangleq \binom{N}{s} \theta^s (1-\theta)^{N-s} \tag{2.68}$$

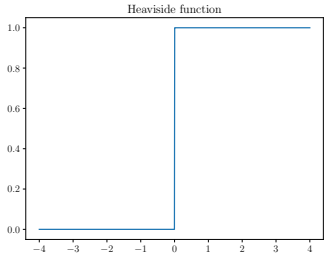
where

$$\binom{N}{k} \triangleq \frac{N!}{(N-k)!k!} \tag{2.69}$$

is the number of ways to choose k items from N (this is known as the **binomial coefficient**, and is pronounced "N choose k"). See Fig. 2.6 for some examples of the binomial distribution. If $N = 1$, the binomial distribution reduces to the Bernoulli distribution.



(a)



(b)

Figure 2.7: (a) The sigmoid (logistic) function $\sigma(a) = (1 + e^{-a})^{-1}$. (b) The Heaviside function $\mathbb{I}(a > 0)$. Generated by code at figures.probml.ai/book1/2.7.

$$\sigma(x) \triangleq \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x} \tag{2.71}$$

$$\frac{d}{dx}\sigma(x) = \sigma(x)(1-\sigma(x)) \tag{2.72}$$

$$1 - \sigma(x) = \sigma(-x) \tag{2.73}$$

$$\sigma^{-1}(p) = \log\left(\frac{p}{1-p}\right) \triangleq \text{logit}(p) \tag{2.74}$$

$$\sigma_+(x) \triangleq \log(1+e^x) \triangleq \text{softplus}(x) \tag{2.75}$$

$$\frac{d}{dx}\sigma_+(x) = \sigma(x) \tag{2.76}$$

Table 2.3: Some useful properties of the sigmoid (logistic) and related functions. Note that the **logit** *function is the inverse of the sigmoid function, and has a domain of* $[0, 1]$.

2.4.2 Sigmoid (logistic) function

When we want to predict a binary variable $y \in \{0, 1\}$ given some inputs $\mathbf{x} \in \mathcal{X}$, we need to use a **conditional probability distribution** of the form

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y|f(\mathbf{x}; \boldsymbol{\theta})) \tag{2.70}$$

where $f(\mathbf{x}; \boldsymbol{\theta})$ is some function that predicts the mean parameter of the output distribution. We will consider many different kinds of function f in Part II–Part IV.

To avoid the requirement that $0 \leq f(\mathbf{x}; \boldsymbol{\theta}) \leq 1$, we can let f be an unconstrained function, and use the following model:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y|\sigma(f(\mathbf{x}; \boldsymbol{\theta}))) \tag{2.77}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Here $\sigma()$ is the **sigmoid** or **logistic** function, defined as follows:

$$\sigma(a) \triangleq \frac{1}{1+e^{-a}} \tag{2.78}$$

where $a = f(\mathbf{x};\boldsymbol{\theta})$. The term "sigmoid" means S-shaped: see Fig. 2.7a for a plot. We see that it maps the whole real line to $[0,1]$, which is necessary for the output to be interpreted as a probability (and hence a valid value for the Bernoulli parameter θ). The sigmoid function can be thought of as a "soft" version of the **heaviside step function**, defined by

$$H(a) \triangleq \mathbb{I}(a > 0) \tag{2.79}$$

as shown in Fig. 2.7b.

Plugging the definition of the sigmoid function into Eq. (2.77) we get

$$p(y=1|\mathbf{x},\boldsymbol{\theta}) = \frac{1}{1+e^{-a}} = \frac{e^a}{1+e^a} = \sigma(a) \tag{2.80}$$

$$p(y=0|\mathbf{x},\boldsymbol{\theta}) = 1 - \frac{1}{1+e^{-a}} = \frac{e^{-a}}{1+e^{-a}} = \frac{1}{1+e^a} = \sigma(-a) \tag{2.81}$$

The quantity a is equal to the **log odds**, $\log(\frac{p}{1-p})$, where $p = p(y=1|\mathbf{x};\boldsymbol{\theta})$. To see this, note that

$$\log\left(\frac{p}{1-p}\right) = \log\left(\frac{e^a}{1+e^a}\frac{1+e^a}{1}\right) = \log(e^a) = a \tag{2.82}$$

The **logistic function** or **sigmoid** function maps the log-odds a to p:

$$p = \text{logistic}(a) = \sigma(a) \triangleq \frac{1}{1+e^{-a}} = \frac{e^a}{1+e^a} \tag{2.83}$$

The inverse of this is called the **logit function**, and maps p to the log-odds a:

$$a = \text{logit}(p) = \sigma^{-1}(p) \triangleq \log\left(\frac{p}{1-p}\right) \tag{2.84}$$

See Table 2.3 for some useful properties of these functions.

2.4.3 Binary logistic regression

In this section, we use a conditional Bernoulli model, where we use a linear predictor of the form $f(\mathbf{x};\boldsymbol{\theta}) = \mathbf{w}^\top\mathbf{x}$. Thus the model has the form

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^\top\mathbf{x}+b)) \tag{2.85}$$

In other words,

$$p(y=1|\mathbf{x};\boldsymbol{\theta}) = \sigma(\mathbf{w}^\top\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}^\top\mathbf{x}+b)}} \tag{2.86}$$

This is called **logistic regression**.

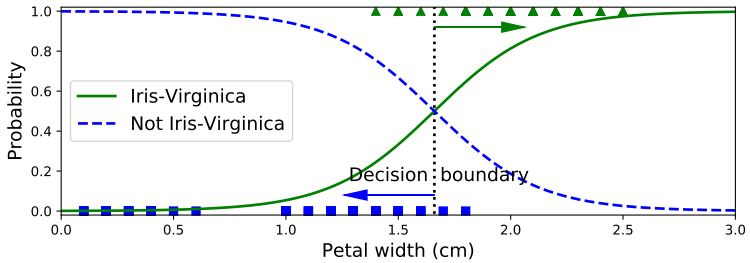


Figure 2.8: Logistic regression applied to a 1-dimensional, 2-class version of the Iris dataset. Generated by code at figures.probml.ai/book1/2.8. Adapted from Figure 4.23 of [Gér19].

For example consider a 1-dimensional, 2-class version of the iris dataset, where the positive class is "virginica" and the negative class is "not virginica", and the feature x we use is the petal width. We fit a logistic regression model to this and show the results in Fig. 2.8. The **decision boundary** corresponds to the value x^* where $p(y = 1|x = x^*, \boldsymbol{\theta}) = 0.5$. We see that, in this example, $x^* \approx 1.7$. As x moves away from this boundary, the classifier becomes more confident in its prediction about the class label.

It should be clear from this example why it would be inappropriate to use linear regression for a (binary) classification problem. In such a model, the probabilities would increase above 1 as we move far enough to the right, and below 0 as we move far enough to the left.

For more detail on logistic regression, see Chapter 10.

2.5 Categorical and multinomial distributions

To represent a distribution over a finite set of labels, $y \in \{1, \ldots, C\}$, we can use the **categorical** distribution, which generalizes the Bernoulli to $C > 2$ values.

2.5.1 Definition

The categorical distribution is a discrete probability distribution with one parameter per class:

$$\text{Cat}(y|\boldsymbol{\theta}) \triangleq \prod_{c=1}^{C} \theta_c^{\mathbb{I}(y=c)} \tag{2.87}$$

In other words, $p(y = c|\boldsymbol{\theta}) = \theta_c$. Note that the parameters are constrained so that $0 \leq \theta_c \leq 1$ and $\sum_{c=1}^{C} \theta_c = 1$; thus there are only $C - 1$ independent parameters.

We can write the categorical distribution in another way by converting the discrete variable y into a **one-hot vector** with C elements, all of which are 0 except for the entry corresponding to the class label. (The term "one-hot" arises from electrical engineering, where binary vectors are encoded as electrical current on a set of wires, which can be active ("hot") or not ("cold").) For example, if $C = 3$, we encode the classes 1, 2 and 3 as $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$. More generally, we can encode the classes using **unit vectors**, where $\mathbf{e}_c$ is all 0s except for dimension c. (This is also called a **dummy**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

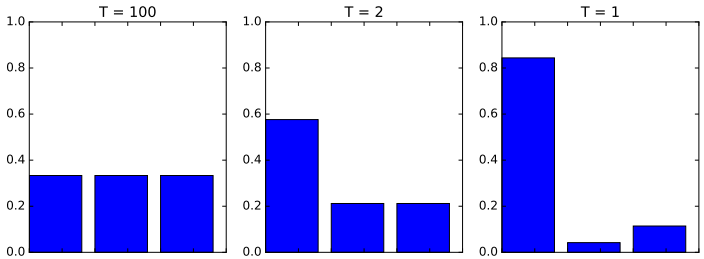


Figure 2.9: Softmax distribution $\mathcal{S}(\mathbf{a}/T)$, where $\mathbf{a} = (3, 0, 1)$, at temperatures of $T = 100$, $T = 2$ and $T = 1$. When the temperature is high (left), the distribution is uniform, whereas when the temperature is low (right), the distribution is "spiky", with most of its mass on the largest element. Generated by code at figures.probml.ai/book1/2.9.

encoding.) Using one-hot encodings, we can write the categorical distribution as follows:

$$\text{Cat}(\mathbf{y}|\boldsymbol{\theta}) \triangleq \prod_{c=1}^{C} \theta_c^{y_c} \tag{2.88}$$

The categorical distribution is a special case of the **multinomial distribution**. To explain this, suppose we observe N categorical trials, $y_n \sim \text{Cat}(\cdot|\boldsymbol{\theta})$, for $n = 1 : N$. Concretely, think of rolling a C-sided dice N times. Let us define $\mathbf{s}$ to be a vector that counts the number of times each face shows up, i.e., $s_c \triangleq \sum_{n=1}^{N} \mathbb{I}(y_n = c)$. (Equivalently, if we use one-hot encodings, we have $\mathbf{s} = \sum_n \mathbf{y}_n$.) The distribution of $\mathbf{s}$ is given by the **multinomial distribution**:

$$\text{Mu}(\mathbf{s}|N, \boldsymbol{\theta}) \triangleq \binom{N}{s_1 \dots s_C} \prod_{c=1}^{C} \theta_c^{s_c} \tag{2.89}$$

where θ_c is the probability that side c shows up, and

$$\binom{N}{s_1 \dots s_C} \triangleq \frac{N!}{s_1! s_2! \cdots s_C!} \tag{2.90}$$

is the **multinomial coefficient**, which is the number of ways to divide a set of size $N = \sum_{c=1}^{C} s_c$ into subsets with sizes s_1 up to s_C. If $N = 1$, the multinomial distribution becomes the categorical distribution.

2.5.2 Softmax function

In the conditional case, we can define

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y|f(\mathbf{x}; \boldsymbol{\theta})) \tag{2.91}$$

which we can also write as

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Mu}(\mathbf{y}|1, f(\mathbf{x}; \boldsymbol{\theta})) \tag{2.92}$$

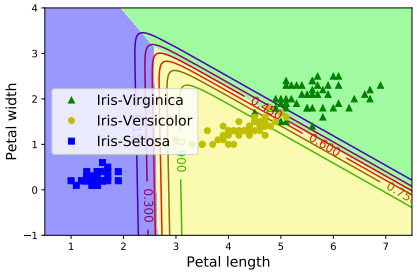


Figure 2.10: Logistic regression on the 3-class, 2-feature version of the Iris dataset. Adapted from Figure of 4.25 [Gér19]. Generated by code at figures.probml.ai/book1/2.10.

We require that $0 \leq f_c(\mathbf{x};\boldsymbol{\theta}) \leq 1$ and $\sum_{c=1}^{C} f_c(\mathbf{x};\boldsymbol{\theta}) = 1$.

To avoid the requirement that f directly predict a probability vector, it is common to pass the output from f into the **softmax** function [Bri90], also called the **multinomial logit**. This is defined as follows:

$$\mathcal{S}(\mathbf{a}) \triangleq \left[\frac{e^{a_1}}{\sum_{c'=1}^{C} e^{a_{c'}}}, \ldots, \frac{e^{a_C}}{\sum_{c'=1}^{C} e^{a_{c'}}}\right] \tag{2.93}$$

This maps $\mathbb{R}^C$ to $[0,1]^C$, and satisfies the constraints that $0 \leq \mathcal{S}(\mathbf{a})_c \leq 1$ and $\sum_{c=1}^{C} \mathcal{S}(\mathbf{a})_c = 1$. The inputs to the softmax, $\mathbf{a} = f(\mathbf{x};\boldsymbol{\theta})$, are called **logits**, and are a generalization of the log odds.

The softmax function is so-called since it acts a bit like the argmax function. To see this, let us divide each a_c by a constant T called the **temperature**.[4] Then as $T \to 0$, we find

$$\mathcal{S}(\mathbf{a}/T)_c = \begin{cases} 1.0 & \text{if } c = \text{argmax}_{c'}\, a_{c'} \\ 0.0 & \text{otherwise} \end{cases} \tag{2.94}$$

In other words, at low temperatures, the distribution puts most of its probability mass in the most probable state (this is called **winner takes all**), whereas at high temperatures, it spreads the mass uniformly. See Fig. 2.9 for an illustration.

2.5.3 Multiclass logistic regression

If we use a linear predictor of the form $f(\mathbf{x};\boldsymbol{\theta}) = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{W}$ is a $C \times D$ matrix, and $\mathbf{b}$ is C-dimensional bias vector, the final model becomes

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{x} + \mathbf{b})) \tag{2.95}$$

Let $\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$ be the C-dimensional vector of **logits**. Then we can rewrite the above as follows:

$$p(y = c|\mathbf{x};\boldsymbol{\theta}) = \frac{e^{a_c}}{\sum_{c'=1}^{C} e^{a_{c'}}} \tag{2.96}$$

4. This terminology comes from the area of statistical physics. The **Boltzmann distribution** is a distribution over states which has the same form as the softmax function.

This is known as **multinomial logistic regression**.

If we have just two classes, this reduces to binary logistic regression. To see this, note that

$$\mathcal{S}(\mathbf{a})_0 = \frac{e^{a_0}}{e^{a_0} + e^{a_1}} = \frac{1}{1 + e^{a_1 - a_0}} = \sigma(a_0 - a_1) \tag{2.97}$$

so we can just train the model to predict $a = a_1 - a_0$. This can be done with a single weight vector $\mathbf{w}$; if we use the multi-class formulation, we will have two weight vectors, $\mathbf{w}_0$ and $\mathbf{w}_1$. Such a model is **over-parameterized**, which can hurt interpretability, but the predictions will be the same.

We discuss this in more detail in Sec. 10.3. For now, we just give an example. Fig. 2.10 shows what happens when we fit this model to the 3-class iris dataset, using just 2 features. We see that the decision boundaries between each class are linear. We can create nonlinear boundaries by using transforming the features (e.g., using polynomials), as we discuss in Sec. 10.3.1.

2.5.4 Log-sum-exp trick

In this section, we discuss one important practical detail to pay attention to when working with the softmax distribution. Suppose we want to compute the normalized probability $p_c = p(y = c|\mathbf{x})$, which is given by

$$p_c = \frac{e^{a_c}}{Z(\mathbf{a})} = \frac{e^{a_c}}{\sum_{c'=1}^{C} e^{a_{c'}}} \tag{2.98}$$

where $\mathbf{a} = f(\mathbf{x}; \boldsymbol{\theta})$ are the logits. We might encounter numerical problems when computing the **partition function** Z. For example, suppose we have 3 classes, with logits $\mathbf{a} = (0, 1, 0)$. Then we find $Z = e^0 + e^1 + e^0 = 4.71$. But now suppose $\mathbf{a} = (1000, 1001, 1000)$; we find $Z = \infty$, since on a computer, even using 64 bit precision, `np.exp(1000)=inf`. Similarly, suppose $\mathbf{a} = (-1000, -999, -1000)$; now we find $Z = 0$, since `np.exp(-1000)=0`. To avoid numerical problems, we can use the following identity:

$$\log \sum_{c=1}^{C} \exp(a_c) = m + \log \sum_{c=1}^{C} \exp(a_c - m) \tag{2.99}$$

This holds for any m. It is common to use $m = \max_c a_c$ which ensures that the largest value you exponentiate will be zero, so you will definitely not overflow, and even if you underflow, the answer will be sensible. This is known as the **log-sum-exp trick**. We use this trick when implementing the **lse** function:

$$\text{lse}(\mathbf{a}) \triangleq \log \sum_{c=1}^{C} \exp(a_c) \tag{2.100}$$

We can use this to compute the probabilities from the logits:

$$p(y = c|\mathbf{x}) = \exp(a_c - \text{lse}(\mathbf{a})) \tag{2.101}$$

We can then pass this to the cross-entropy loss, defined in Eq. (5.40).

However, to save computational effort, and for numerical stability, it is quite common to modify the cross-entropy loss so that it takes the logits **a** as inputs, instead of the probability vector **p**. For example, consider the binary case. The CE loss for one example is

$$\mathcal{L} = -\left[\mathbb{I}\left(y=0\right)\log p_0 + \mathbb{I}\left(y=1\right)\log p_1\right] \tag{2.102}$$

where

$$\log p_1 = \log\left(\frac{1}{1+\exp(-a)}\right) = \log(1) - \log(1+\exp(-a)) = 0 - \text{lse}([0,-a]) \tag{2.103}$$

$$\log p_0 = 0 - \text{lse}([0,+a]) \tag{2.104}$$

2.6 Univariate Gaussian (normal) distribution

The most widely used distribution of real-valued random variables $y \in \mathbb{R}$ is the **Gaussian distribution**, also called the **normal distribution** (see Sec. 2.6.4 for a discussion of these names).

2.6.1 Cumulative distribution function

We define the **cumulative distribution function** or **cdf** of a continuous random variable Y as follows:

$$P(y) \triangleq \Pr(Y \leq y) \tag{2.105}$$

(Note that we use a capital P to represent the cdf.) Using this, we can compute the probability of being in any interval as follows:

$$\Pr(a < Y \leq b) = P(b) - P(a) \tag{2.106}$$

Cdf's are monotonically non-decreasing functions.

The cdf of the Gaussian is defined by

$$\Phi(y;\mu,\sigma^2) \triangleq \int_{-\infty}^{y} \mathcal{N}(z|\mu,\sigma^2)dz = \frac{1}{2}[1+\text{erf}(z/\sqrt{2})] \tag{2.107}$$

where $z = (y-\mu)/\sigma$ and $\text{erf}(u)$ is the **error function**, defined as

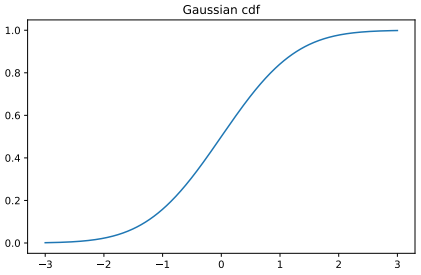
$$\text{erf}(u) \triangleq \frac{2}{\sqrt{\pi}}\int_0^u e^{-t^2}dt \tag{2.108}$$

This latter function is built in to most software packages. See Fig. 2.11a for a plot.

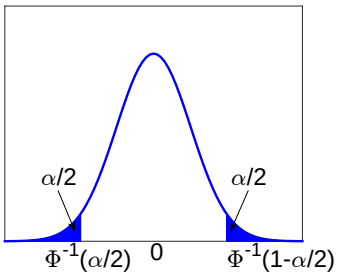
The parameter μ encodes the mean of the distribution, which is the same as the mode, since the distribution is unimodal. The parameter σ^2 encodes the variance. (Sometimes we talk about the **precision** of a Gaussian, which is the inverse variance, denoted $\lambda = 1/\sigma^2$.) When $\mu = 0$ and $\sigma = 1$, the Gaussian is called the **standard normal** distribution.

If P is the cdf of Y, then $P^{-1}(q)$ is the value y_q such that $p(Y \leq y_q) = q$; this is called the q'th **quantile** of P. The value $P^{-1}(0.5)$ is the **median** of the distribution, with half of the probability

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a)



(b)

Figure 2.11: (a) Cumulative distribution function (cdf) for the standard normal. Generated by code at figures.probml.ai/book1/2.11. (b) Probability density function (pdf) for the standard normal. The shaded regions each contain $\alpha/2$ of the probability mass. Therefore the nonshaded region contains $1-\alpha$ of the probability mass. The leftmost cutoff point is $\Phi^{-1}(\alpha/2)$, where Φ is the cdf of the Gaussian. By symmetry, the rightmost cutoff point is $\Phi^{-1}(1-\alpha/2) = -\Phi^{-1}(\alpha/2)$. Generated by code at figures.probml.ai/book1/2.11.

mass on the left, and half on the right. The values $P^{-1}(0.25)$ and $P^{-1}(0.75)$ are the lower and upper **quartiles**.

For example, let Φ be the cdf of the Gaussian distribution $\mathcal{N}(0,1)$, and Φ^{-1} be the inverse cdf (also known as the **probit function**). Then points to the left of $\Phi^{-1}(\alpha/2)$ contain $\alpha/2$ of the probability mass, as illustrated in Fig. 2.11b. By symmetry, points to the right of $\Phi^{-1}(1-\alpha/2)$ also contain $\alpha/2$ of the mass. Hence the central interval $(\Phi^{-1}(\alpha/2), \Phi^{-1}(1-\alpha/2))$ contains $1-\alpha$ of the mass. If we set $\alpha = 0.05$, the central 95% interval is covered by the range

$$(\Phi^{-1}(0.025), \Phi^{-1}(0.975)) = (-1.96, 1.96) \tag{2.109}$$

If the distribution is $\mathcal{N}(\mu, \sigma^2)$, then the 95% interval becomes $(\mu - 1.96\sigma, \mu + 1.96\sigma)$. This is often approximated by writing $\mu \pm 2\sigma$.

2.6.2 Probability density function

We define the **probability density function** or **pdf** as the derivative of the cdf:

$$p(y) \triangleq \frac{d}{dy} P(y) \tag{2.110}$$

The pdf of the Gaussian is given by

$$\mathcal{N}(y|\mu, \sigma^2) \triangleq \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y-\mu)^2} \tag{2.111}$$

where $\sqrt{2\pi\sigma^2}$ is the normalization constant needed to ensure the density integrates to 1 (see Exercise 2.12). See Fig. 2.11b for a plot.

Given a pdf, we can compute the probability of a continuous variable being in a finite interval as follows:

$$\Pr(a < Y \leq b) = \int_a^b p(y)dy = P(b) - P(a) \tag{2.112}$$

As the size of the interval gets smaller, we can write

$$\Pr(y \leq Y \leq y + dy) \approx p(y)dy \tag{2.113}$$

Intuitively, this says the probability of Y being in a small interval around y is the density at y times the width of the interval. One important consequence of the above result is that the pdf at a point can be larger than 1. For example, $\mathcal{N}(0|0, 0.1) = 3.99$.

We can use the pdf to compute the **mean**, or **expected value**, of the distribution:

$$\mathbb{E}[Y] \triangleq \int_{\mathcal{Y}} y\, p(y)dy \tag{2.114}$$

For a Gaussian, we have the familiar result that $\mathbb{E}\left[\mathcal{N}(\cdot|\mu, \sigma^2)\right] = \mu$. (Note, however, that for some distributions, this integral is not finite, so the mean is not defined.)

We can also use the pdf to compute the **variance** of a distribution. This is a measure of the "spread", and is often denoted by σ^2. The variance is defined as follows:

$$\mathbb{V}[Y] \triangleq \mathbb{E}\left[(Y - \mu)^2\right] = \int (y - \mu)^2 p(y)dy \tag{2.115}$$

$$= \int y^2 p(y)dy + \mu^2 \int p(y)dy - 2\mu \int yp(y)dy = \mathbb{E}\left[Y^2\right] - \mu^2 \tag{2.116}$$

from which we derive the useful result

$$\mathbb{E}\left[Y^2\right] = \sigma^2 + \mu^2 \tag{2.117}$$

The **standard deviation** is defined as

$$\text{std}[Y] \triangleq \sqrt{\mathbb{V}[Y]} = \sigma \tag{2.118}$$

(The standard deviation can be more intepretable than the variance since it has the same units as Y itself.) For a Gaussian, we have the familiar result that $\text{std}\left[\mathcal{N}(\cdot|\mu, \sigma^2)\right] = \sigma$.

2.6.3 Regression

So far we have been considering the unconditional Gaussian distribution. In some cases, it is helpful to make the parameters of the Gaussian be functions of some input variables, i.e., we want to create a conditional density model of the form

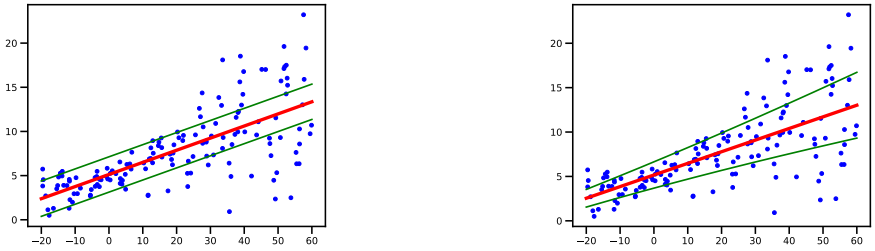
$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y|f_\mu(\mathbf{x}; \boldsymbol{\theta}), f_\sigma(\mathbf{x}; \boldsymbol{\theta})^2) \tag{2.119}$$

where $f_\mu(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}$ predicts the mean, and $f_\sigma(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}_+$ predicts the variance.

It is common to assume that the variance is fixed, and is independent of the input. This is called **homoscedastic regression**. Furthermore it is common to assume the mean is a linear function of the input. The resulting model is called **linear regression**:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x} + b, \sigma^2) \tag{2.120}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 2.12: Linear regression using Gaussian output with mean $\mu(x) = b + wx$ and (a) fixed variance σ^2 (homoskedastic) or (b) input-dependent variance $\sigma(x)^2$ (heteroscedastic). Generated by code at figures.probml.ai/book1/2.12.

where $\boldsymbol{\theta} = (\mathbf{w}, b, \sigma^2)$. See Fig. 2.12(a) for an illustration of this model in 1d. and Sec. 11.2 for more details on this model.

However, we can also make the variance depend on the input; this is called **heteroskedastic regression**. In the linear regression setting, we have

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}_\mu^\top \mathbf{x} + b, \sigma_+(\mathbf{w}_\sigma^\top \mathbf{x})) \tag{2.121}$$

where $\boldsymbol{\theta} = (\mathbf{w}_\mu, \mathbf{w}_\sigma)$ are the two forms of regression weights, and

$$\sigma_+(a) = \log(1 + e^a) \tag{2.122}$$

is the **softplus** function, that maps from $\mathbb{R}$ to $\mathbb{R}_+$, to ensure the predicted standard deviation is non-negative. See Fig. 2.12(b) for an illustration of this model in 1d.

Note that Fig. 2.12 plots the 95% predictive interval, $[\mu(x) - 2\sigma(x), \mu(x) + 2\sigma(x)]$. This is the uncertainty in the predicted *observation* y given $\mathbf{x}$, and captures the variability in the blue dots. By contrast, the uncertainty in the underlying (noise-free) function is represented by $\sqrt{\mathbb{V}\left[f_\mu(\mathbf{x};\boldsymbol{\theta})\right]}$, which does not involve the σ term; now the uncertainty is over the parameters $\boldsymbol{\theta}$, rather than the output y. See Sec. 11.6 for details on how to model parameter uncertainty.

2.6.4 Why is the Gaussian distribution so widely used?

The Gaussian distribution is the most widely used distribution in statistics and machine learning. There are several reasons for this. First, it has two parameters which are easy to interpret, and which capture some of the most basic properties of a distribution, namely its mean and variance. Second, the central limit theorem (Sec. 2.8.6) tells us that sums of independent random variables have an approximately Gaussian distribution, making it a good choice for modeling residual errors or "noise". Third, the Gaussian distribution makes the least number of assumptions (has maximum entropy), subject to the constraint of having a specified mean and variance, as we show in Sec. 12.2.6; this makes it a good default choice in many cases. Finally, it has a simple mathematical form, which results in easy to implement, but often highly effective, methods, as we will see in Sec. 3.2.

From a historical perspective, it's worth remarking that the term "Gaussian distribution" is a bit misleading, since, as Jaynes [Jay03, p241] notes: "The fundamental nature of this distribution and its main properties were noted by Laplace when Gauss was six years old; and the distribution itself had been found by de Moivre before Laplace was born". However, Gauss popularized the use of the distribution in the 1800s, and the term "Gaussian" is now widely used in science and engineering.

The name "normal distribution" seems to have arisen in connection with the normal equations in linear regression (see Sec. 11.2.2.2). However, we prefer to avoid the term "normal", since it suggests other distributions are "abnormal", whereas, as Jaynes [Jay03] points out, it is the Gaussian that is abnormal in the sense that it has many special properties that are untypical of general distributions.

2.6.5 Dirac delta function as a limiting case

As the variance of a Gaussian goes to 0, the distribution approaches an infinitely narrow, but infinitely tall, "spike" at the mean. We can write this as follows:

$$\lim_{\sigma \to 0} \mathcal{N}(y|\mu, \sigma^2) \to \delta(y - \mu) \tag{2.123}$$

where δ is the **Dirac delta function**, defined by

$$\delta(\theta) = \begin{cases} +\infty & \text{if } \theta = 0 \\ 0 & \text{if } \theta \neq 0 \end{cases} \tag{2.124}$$

where

$$\int_{-\infty}^{\infty} \delta(\theta) d\theta = 1 \tag{2.125}$$

The delta function distribution satisfies the following **sifting property**, which we will use later on:

$$\int_{-\infty}^{\infty} f(y)\delta(x - y)dy = f(x) \tag{2.126}$$

2.7 Some other common univariate distributions *

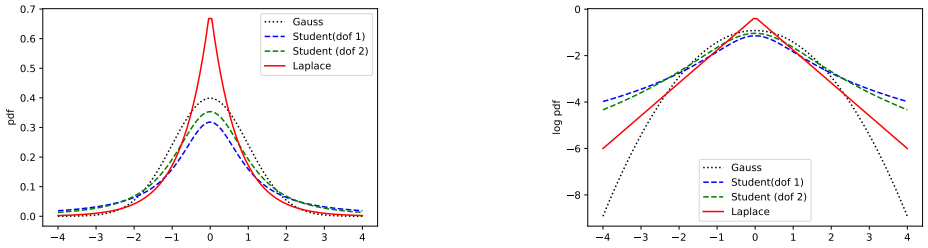
In this section, we briefly introduce some other univariate distributions that we will use in this book.

2.7.1 Student t distribution

The Gaussian distribution is quite sensitive to **outliers**. A **robust** alternative to the Gaussian is the **Student t-distribution**, which we shall call the **Student distribution** for short.[5] Its pdf is

5. This distribution has a colorful etymology. It was first published in 1908 by William Sealy Gosset, who worked at the Guinness brewery in Dublin, Ireland. Since his employer would not allow him to use his own name, he called it the "Student" distribution. The origin of the term t seems to have arisen in the context of tables of the Student distribution, used by Fisher when developing the basis of classical statistical inference. See http://jeff560.tripod.com/s.html for more historical details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 2.13: (a) The pdf's for a $\mathcal{N}(0,1)$, $\mathcal{T}(\mu=0,\sigma=1,\nu=1)$, $\mathcal{T}(\mu=0,\sigma=1,\nu=2)$, and $\text{Lap}(0,1/\sqrt{2})$. The mean is 0 and the variance is 1 for both the Gaussian and Laplace. When $\nu=1$, the Student is the same as the Cauchy, which does not have a well-defined mean and variance. (b) Log of these pdf's. Note that the Student distribution is not log-concave for any parameter value, unlike the Laplace distribution. Nevertheless, both are unimodal. Generated by code at figures.probml.ai/book1/2.13.

as follows:

$$\mathcal{T}(y|\mu,\sigma^2,\nu) \propto \left[1+\frac{1}{\nu}\left(\frac{y-\mu}{\sigma}\right)^2\right]^{-(\frac{\nu+1}{2})} \tag{2.127}$$

where μ is the mean, $\sigma>0$ is the scale parameter (not the standard deviation), and $\nu>0$ is called the **degrees of freedom** (although a better term would be the **degree of normality** [Kru13], since large values of ν make the distribution act like a Gaussian).

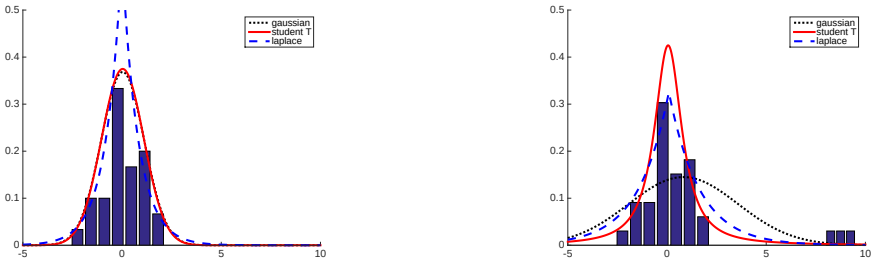
We see that the probability density decays as a polynomial function of the squared distance from the center, as opposed to an exponential function, so there is more probability mass in the tail than with a Gaussian distribution, as shown in Fig. 2.13. We say that the Student distribution has **heavy tails**, which makes it robust to outliers.

To illustrate the robustness of the Student distribution, consider Fig. 2.14. On the left, we show a Gaussian and a Student distribution fit to some data with no outliers. On the right, we add some outliers. We see that the Gaussian is affected a lot, whereas the Student hardly changes. We discuss how to use the Student distribution for robust linear regression in Sec. 11.4.1.

For later reference, we note that the Student distribution has the following properties:

$$\text{mean}=\mu,\ \text{mode}=\mu,\ \text{var}=\frac{\nu\sigma^2}{(\nu-2)} \tag{2.128}$$

The mean is only defined if $\nu>1$. The variance is only defined if $\nu>2$. For $\nu \gg 5$, the Student distribution rapidly approaches a Gaussian distribution and loses its robustness properties. It is common to use $\nu=4$, which gives good performance in a range of problems [LLT89].



(a) (b)

Figure 2.14: Illustration of the effect of outliers on fitting Gaussian, Student and Laplace distributions. (a) No outliers (the Gaussian and Student curves are on top of each other). (b) With outliers. We see that the Gaussian is more affected by outliers than the Student and Laplace distributions. Adapted from Figure 2.16 of [Bis06]. Generated by code at figures.probml.ai/book1/2.14

2.7.2 Cauchy distribution

If $\nu = 1$, the Student distribution is known as the **Cauchy** or **Lorentz** distribution. Its pdf is defined by

$$\mathcal{C}(x|\mu,\gamma) = \frac{1}{\gamma\pi}\left[1 + \left(\frac{x-\mu}{\gamma}\right)^2\right]^{-1} \tag{2.129}$$

This distribution has very heavy tails compared to a Gaussian. For example, 95% of the values from a standard normal are between -1.96 and 1.96, but for a standard Cauchy they are between -12.7 and 12.7. In fact the tails are so heavy that the integral that defines the mean does not converge. The **half Cauchy** distribution is a version of the Cauchy (with mean 0) that is "folded over" on itself, so all its probability density is on the positive reals. Thus it has the form

$$\mathcal{C}_+(x|\gamma) \triangleq \frac{2}{\pi\gamma}\left[1 + \left(\frac{x}{\gamma}\right)^2\right]^{-1} \tag{2.130}$$

This is useful in Bayesian modeling, where we want to use a distribution over positive reals with heavy tails, but finite density at the origin.

2.7.3 Laplace distribution

Another distribution with heavy tails is the **Laplace distribution**[6], also known as the **double sided exponential** distribution. This has the following pdf:

$$\text{Lap}(y|\mu,b) \triangleq \frac{1}{2b}\exp\left(-\frac{|y-\mu|}{b}\right) \tag{2.131}$$

6. Pierre-Simon Laplace (1749–1827) was a French mathematician, who played a key role in creating the field of Bayesian statistics.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

See Fig. 2.13 for a plot. Here μ is a location parameter and $b > 0$ is a scale parameter. This distribution has the following properties:

$$\text{mean} = \mu, \ \text{mode} = \mu, \ \text{var} = 2b^2 \tag{2.132}$$

In Sec. 11.4.2, we discuss how to use the Laplace distribution for robust linear regression, and in Sec. 11.5, we discuss how to use the Laplace distribution for sparse linear regression.

2.7.4 Beta distribution

The **beta distribution** has support over the interval $[0, 1]$ and is defined as follows:

$$\text{Beta}(x|a, b) = \frac{1}{B(a, b)} x^{a-1}(1-x)^{b-1} \tag{2.133}$$

where $B(a, b)$ is the **beta function**, defined by

$$B(a, b) \triangleq \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)} \tag{2.134}$$

where $\Gamma(a)$ is the Gamma function defined by

$$\Gamma(a) \triangleq \int_0^\infty x^{a-1} e^{-x} dx \tag{2.135}$$

See Fig. 2.15a for plots of some beta distributions.

We require $a, b > 0$ to ensure the distribution is integrable (i.e., to ensure $B(a, b)$ exists). If $a = b = 1$, we get the uniform distribution. If a and b are both less than 1, we get a bimodal distribution with "spikes" at 0 and 1; if a and b are both greater than 1, the distribution is unimodal.

For later reference, we note that the distribution has the following properties (Exercise 2.8):

$$\text{mean} = \frac{a}{a+b}, \ \text{mode} \ = \frac{a-1}{a+b-2}, \ \text{var} = \frac{ab}{(a+b)^2(a+b+1)} \tag{2.136}$$

2.7.5 Gamma distribution

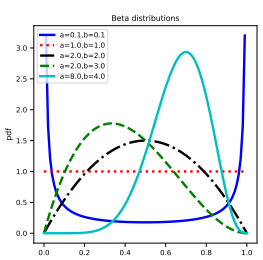
The **gamma distribution** is a flexible distribution for positive real valued rv's, $x > 0$. It is defined in terms of two parameters, called the shape $a > 0$ and the rate $b > 0$:

$$\text{Ga}(x|\text{shape} = a, \text{rate} = b) \triangleq \frac{b^a}{\Gamma(a)} x^{a-1} e^{-xb} \tag{2.137}$$

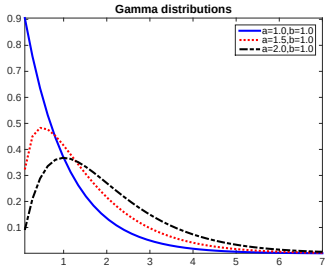
Sometimes the distribution is parameterized in terms of the shape a and the **scale** $s = 1/b$:

$$\text{Ga}(x|\text{shape} = a, \text{scale} = s) \triangleq \frac{1}{s^a \Gamma(a)} x^{a-1} e^{-x/s} \tag{2.138}$$

See Fig. 2.15b for some plots of the gamma pdf.



(a)



(b)

Figure 2.15: (a) Some beta distributions. Generated by code at figures.probml.ai/book1/2.15. (b) Some $\text{Ga}(a, b = 1)$ *distributions. If* $a \leq 1$*, the mode is at 0, otherwise it is* > 0*. As we increase the rate b, we reduce the horizontal scale, thus squeezing everything leftwards and upwards. Generated by code at figures.probml.ai/book1/2.15.*

For reference, we note that the distribution has the following properties:

$$\text{mean} = \frac{a}{b}, \ \text{mode} = \frac{a-1}{b}, \ \text{var} = \frac{a}{b^2} \tag{2.139}$$

There are several distributions which are just special cases of the Gamma, which we discuss below.

- **Exponential distribution**. This is defined by

$$\text{Expon}(x|\lambda) \triangleq \text{Ga}(x|\text{shape} = 1, \text{rate} = \lambda) \tag{2.140}$$

  This distribution describes the times between events in a Poisson process, i.e. a process in which events occur continuously and independently at a constant average rate λ.

- **Chi-squared distribution**. This is defined by

$$\chi^2_\nu(x) \triangleq \text{Ga}(x|\text{shape} = \frac{\nu}{2}, \text{rate} = \frac{1}{2}) \tag{2.141}$$

  where ν is called the degrees of freedom. This is the distribution of the sum of squared Gaussian random variables. More precisely, if $Z_i \sim \mathcal{N}(0, 1)$, and $S = \sum_{i=1}^{\nu} Z_i^2$, then $S \sim \chi^2_\nu$.

- The **inverse Gamma distribution** is the distribution of $Y = 1/X$ assuming $X \sim \text{Ga}(a, b)$. One can show (Exercise 2.7) that the pdf is given by

$$\text{IG}(x|\text{shape} = a, \text{scale} = s) \triangleq \frac{b^a}{\Gamma(a)} x^{-(a+1)} e^{-s/x} \tag{2.142}$$

  The distribution has these properties

$$\text{mean} = \frac{s}{a-1}, \text{mode} = \frac{s}{a+1}, \text{var} = \frac{s^2}{(a-1)^2(a-2)} \tag{2.143}$$

  The mean only exists if $a > 1$. The variance only exists if $a > 2$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

2.8 Transformations of random variables *

Suppose $\mathbf{x} \sim p()$ is some random variable, and $\mathbf{y} = f(\mathbf{x})$ is some deterministic transformation of it. In this section, we discuss how to compute $p(\mathbf{y})$.

2.8.1 Discrete case

If X is a discrete rv, we can derive the pmf for Y by simply summing up the probability mass for all the x's such that $f(x) = y$:

$$p_y(y) = \sum_{x:f(x)=y} p_x(x) \tag{2.144}$$

For example, if $f(X) = 1$ if X is even and $f(X) = 0$ otherwise, and $p_x(X)$ is uniform on the set $\{1, \ldots, 10\}$, then $p_y(1) = \sum_{x\in\{2,4,6,8,10\}} p_x(x) = 0.5$, and hence $p_y(0) = 0.5$ also. Note that in this example, f is a many-to-one function.

2.8.2 Continuous case

If X is continuous, we cannot use Eq. (2.144) since $p_x(x)$ is a density, not a pmf, and we cannot sum up densities. Instead, we work with cdf's, as follows:

$$P_y(y) \triangleq \Pr(Y \leq y) = \Pr(f(X) \leq y) = \Pr(X \in \{x|f(x) \leq y\}) \tag{2.145}$$

If f is invertible, we can derive the pdf of y by differentiating the cdf, as we show below. If f is not invertible, we can use numerical integration, or a Monte Carlo approximation.

2.8.3 Invertible transformations (bijectors)

In this section, we consider the case of monotonic and hence invertible functions. (Note a function is invertible iff it is a **bijector**). With this assumption, there is a simple formula for the pdf of y, as we will see. (This can be generalized to invertible, but non-monotonic, functions, but we ignore this case.)

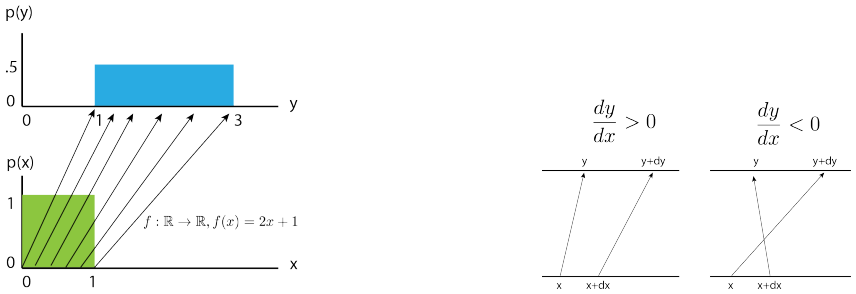
2.8.3.1 Change of variables: scalar case

We start with an example. Suppose $x \sim \text{Unif}(0, 1)$, and $y = f(x) = 2x + 1$. This function stretches and shifts the probability distribution, as shown in Fig. 2.16(a). Now let us zoom in on a point x and another point that is infinitesimally close, namely $x + dx$. We see this interval gets mapped to $(y, y + dy)$. The probability mass in these intervals must be the same, hence $p(x)dx = p(y)dy$, and so $p(y) = p(x)dx/dy$. However, since it does not matter (in terms of probability preservation) whether $dx/dy > 0$ or $dx/dy < 0$, we get

$$p_y(y) = p_x(x)|\frac{dx}{dy}| \tag{2.146}$$

Now consider the general case for any $p_x(x)$ and any monotonic function $f : \mathbb{R} \to \mathbb{R}$. Let $g = f^{-1}$, so $y = f(x)$ and $x = g(y)$. If we assume that $f : \mathbb{R} \to \mathbb{R}$ is monotonically increasing we get

$$P_y(y) = \Pr(f(X) \leq y) = \Pr(X \leq f^{-1}(y)) = P_x(f^{-1}(y)) = P_x(g(y)) \tag{2.147}$$



(a) (b)

Figure 2.16: (a) Mapping a uniform pdf through the function $f(x) = 2x + 1$. (b) Illustration of how two nearby points, x and $x + dx$, get mapped under f. If $\frac{dy}{dx} > 0$, the function is locally increasing, but if $\frac{dy}{dx} < 0$, the function is locally decreasing. From [Jan18]. Used with kind permission of Eric Jang

Taking derivatives we get

$$p_y(y) \triangleq \frac{d}{dy} P_y(y) = \frac{d}{dy} P_x(x) = \frac{dx}{dy}\frac{d}{dx} P_x(x)) = \frac{dx}{dy} p_x(x) \tag{2.148}$$

We can derive a similar expression (but with opposite signs) for the case where f is monotonically decreasing. To handle the general case we take the absolute value to get

$$p_y(y) = p_x\left(g(y)\right) \left| \frac{d}{dy} g(y) \right| \tag{2.149}$$

This is called **change of variables** formula.

2.8.3.2 Change of variables: multivariate case

We can extend the previous results to multivariate distributions as follows. Let $\mathbf{f}$ be an invertible function that maps $\mathbb{R}^n$ to $\mathbb{R}^n$, with inverse $\mathbf{g}$. Suppose we want to compute the pdf of $\mathbf{y} = \mathbf{f}(\mathbf{x})$. By analogy with the scalar case, we have

$$p_y(\mathbf{y}) = p_x\left(\mathbf{g}(\mathbf{y})\right) \left| \det\left[\mathbf{J}_g(\mathbf{y})\right] \right| \tag{2.150}$$

where $\mathbf{J}_g = \frac{d\mathbf{g}(\mathbf{y})}{d\mathbf{y}^\top}$ is the Jacobian of $\mathbf{g}$, and $|\det \mathbf{J}(\mathbf{y})|$ is the absolute value of the determinant of $\mathbf{J}$ evaluated at $\mathbf{y}$. (See Sec. 7.8.5 for a discussion of Jacobians.) In Exercise 3.6 you will use this formula to derive the normalization constant for a multivariate Gaussian.

Fig. 2.17 illustrates this result in 2d, for the case where $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$, where $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. We see that the area of the unit square changes by a factor of $\det(\mathbf{A}) = ad - bc$, which is the area of the parallelogram.

As another example, consider transforming a density from Cartesian coordinates $\mathbf{x} = (x_1, x_2)$ to

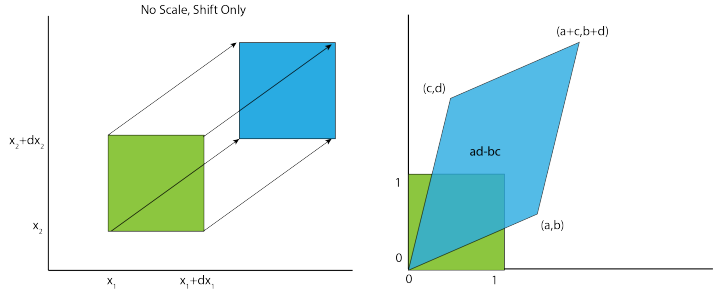


Figure 2.17: Illustration of an affine transformation applied to a unit square, $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$. (a) Here $\mathbf{A} = \mathbf{I}$. (b) Here $\mathbf{b} = \mathbf{0}$. From [Jan18]. Used with kind permission of Eric Jang

polar coordinates $\mathbf{y} = \mathbf{f}(x_1, x_2)$, so $\mathbf{g}(r, \theta) = (r\cos\theta, r\sin\theta)$. Then

$$\mathbf{J}_g = \begin{pmatrix} \frac{\partial x_1}{\partial r} & \frac{\partial x_1}{\partial \theta} \\ \frac{\partial x_2}{\partial r} & \frac{\partial x_2}{\partial \theta} \end{pmatrix} = \begin{pmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{pmatrix} \tag{2.151}$$

$$|\det(\mathbf{J}_g)| = |r\cos^2\theta + r\sin^2\theta| = |r| \tag{2.152}$$

Hence

$$p_{r,\theta}(r, \theta) = p_{x_1,x_2}(r\cos\theta, r\sin\theta)\, r \tag{2.153}$$

To see this geometrically, notice that the area of the shaded patch in Fig. 2.18 is given by

$$\Pr(r \le R \le r + dr, \theta \le \Theta \le \theta + d\theta) = p_{r,\theta}(r, \theta) dr d\theta \tag{2.154}$$

In the limit, this is equal to the density at the center of the patch times the size of the patch, which is given by $r\ dr\ d\theta$. Hence

$$p_{r,\theta}(r, \theta)\ dr\ d\theta = p_{x_1,x_2}(r\cos\theta, r\sin\theta)\ r\ dr\ d\theta \tag{2.155}$$

2.8.4 Moments of a linear transformation

Suppose f is an affine function, so $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$. In this case, we can easily derive the mean and covariance of $\mathbf{y}$ as follows. First, for the mean, we have

$$\mathbb{E}[\mathbf{y}] = \mathbb{E}[\mathbf{A}\mathbf{x} + \mathbf{b}] = \mathbf{A}\boldsymbol{\mu} + \mathbf{b} \tag{2.156}$$

where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$. If f is a scalar-valued function, $f(\mathbf{x}) = \mathbf{a}^\top\mathbf{x} + b$, the corresponding result is

$$\mathbb{E}\left[\mathbf{a}^\top\mathbf{x} + b\right] = \mathbf{a}^\top\boldsymbol{\mu} + b \tag{2.157}$$

For the covariance, we have

$$\mathrm{Cov}[\mathbf{y}] = \mathrm{Cov}[\mathbf{A}\mathbf{x} + \mathbf{b}] = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top \tag{2.158}$$

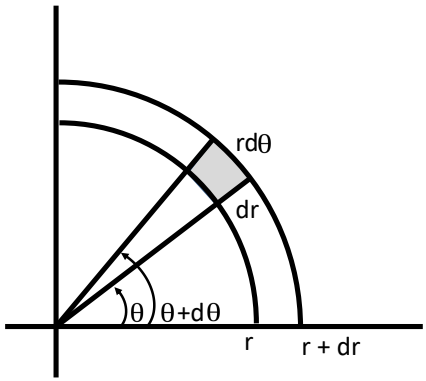


Figure 2.18: Change of variables from polar to Cartesian. The area of the shaded patch is r dr dθ. Adapted from Figure 3.16 of [Ric95]

where $\boldsymbol{\Sigma} = \text{Cov}[\mathbf{x}]$. We leave the proof of this as an exercise.

As a special case, if $y = \mathbf{a}^\top \mathbf{x} + b$, we get

$$\mathbb{V}[y] = \mathbb{V}\left[\mathbf{a}^\top \mathbf{x} + b\right] = \mathbf{a}^\top \boldsymbol{\Sigma} \mathbf{a} \tag{2.159}$$

For example, to compute the variance of the sum of two scalar random variables, we can set $\mathbf{a} = [1, 1]$ to get

$$\mathbb{V}[x_1 + x_2] = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \tag{2.160}$$

$$= \Sigma_{11} + \Sigma_{22} + 2\Sigma_{12} = \mathbb{V}[x_1] + \mathbb{V}[x_2] + 2\text{Cov}[x_1, x_2] \tag{2.161}$$

Note, however, that although some distributions (such as the Gaussian) are completely characterized by their mean and covariance, in general we must use the techniques described above to derive the full distribution of $\mathbf{y}$.

2.8.5 The convolution theorem

Let $y = x_1 + x_2$, where x_1 and x_2 are independent rv's. If these are discrete random variables, we can compute the pmf for the sum as follows:

$$p(y = j) = \sum_k p(x_1 = j) p(x_2 = j - k) \tag{2.162}$$

for $j = \ldots, -2, -1, 0, 1, 2, \ldots$.

If x_1 and x_2 have pdf's $p_1(x_1)$ and $p_2(x_2)$, what is the distribution of y? The cdf for y is given by

$$P_y(y^*) = \Pr(y \leq y^*) = \int_{-\infty}^{\infty} p_1(x_1) dx_1 \int_{-\infty}^{y^* - x_1} p_2(x_2) dx_2 \tag{2.163}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

-	-	1	2	3	4	-	-	
7	6	5	-	-	-	-	-	$z_0 = x_0y_0 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_0y_1 + x_1y_0 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_0y_2 + x_1y_1 + x_2y_0 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_1y_2 + x_2y_1 + x_3y_0 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_2y_2 + x_3y_1 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_3y_2 = 28$

Table 2.4: Discrete convolution of $\mathbf{x} = [1, 2, 3, 4]$ *with* $\mathbf{y} = [5, 6, 7]$ *to yield* $\mathbf{z} = [5, 16, 34, 52, 45, 28]$. *In general,* $z_n = \sum_{k=-\infty}^{\infty} x_k y_{n-k}$. *We see that this operation consists of "flipping"* $\mathbf{y}$ *and then "dragging" it over* $\mathbf{x}$, *multiplying elementwise, and adding up the results.*

where we integrate over the region R defined by $x_1 + x_2 < y^*$. Thus the pdf for y is

$$p(y) = \left[\frac{d}{dy^*} P_y(y^*)\right]_{y^*=y} = \int p_1(x_1)p_2(y - x_1)dx_1 \tag{2.164}$$

where we used the rule of **differentiating under the integral sign**:

$$\frac{d}{dx}\int_{a(x)}^{b(x)} f(t)dt = f(b(x))\frac{db(x)}{dx} - f(a(x))\frac{da(x)}{dx} \tag{2.165}$$

We can write Eq. (2.164) as follows:

$$p = p_1 \circledast p_2 \tag{2.166}$$

where $\circledast$ represents the **convolution** operator. For finite length vectors, the integrals become sums, and convolution can be thought of as a "flip and drag" operation, as illustrated in Table 2.4. Consequently, Eq. (2.164) is called the **convolution theorem**.

For example, suppose we roll two dice, so p_1 and p_2 are both the discrete uniform distributions over $\{1, 2, \ldots, 6\}$. Let $y = x_1 + x_2$ be the sum of the dice. We have

$$p(y = 2) = p(x_1 = 1)p(x_2 = 1) = \frac{1}{6}\frac{1}{6} = \frac{1}{36} \tag{2.167}$$

$$p(y = 3) = p(x_1 = 1)p(x_2 = 2) + p(x_1 = 2)p(x_2 = 1) = \frac{1}{6}\frac{1}{6} + \frac{1}{6}\frac{1}{6} = \frac{2}{36} \tag{2.168}$$

$$\ldots \tag{2.169}$$

Continuing in this way, we find $p(y = 5) = 4/36$, $p(y = 6) = 5/36$, $p(y = 7) = 6/36$, $p(y = 8) = 5/36$, $p(y = 9) = 4/36$, $p(y = 10) = 3/36$, $p(y = 11) = 2/36$ and $p(y = 12) = 1/36$. The result, of course, is equal to the binomial distribution, $\text{Bin}(y|2, (1/6, \ldots, 1/6))$. See Fig. 2.19 for a plot. We see that the distribution looks like a Gaussian; we explain the reasons for this in Sec. 2.8.6.

We can also compute the pdf of the sum of two continuous rv's. For example, in the case of Gaussians, where $x_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \sigma_1^2)$ and $x_2 \sim \mathcal{N}(\boldsymbol{\mu}_2, \sigma_2^2)$, one can show (Exercise 2.5) that if $y = x_1 + x_2$ then

$$p(y) = \mathcal{N}(x_1|\boldsymbol{\mu}_1, \sigma_1^2) \otimes \mathcal{N}(x_2|\boldsymbol{\mu}_2, \sigma_2^2) = \mathcal{N}(y|\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2, \sigma_1^2 + \sigma_2^2) \tag{2.170}$$

Hence the convolution of two Gaussians is a Gaussian.

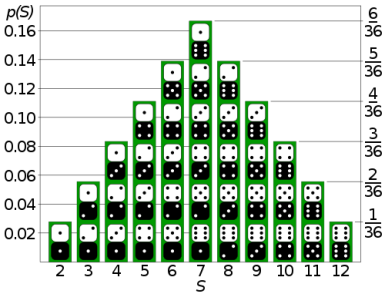
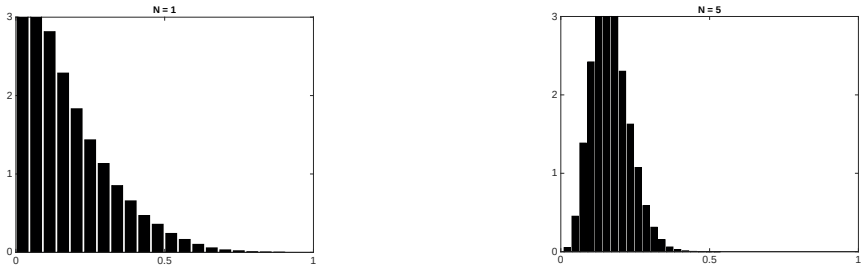


Figure 2.19: Distribution of the sum of two dice rolls, i.e., $p(y)$ where $y = x_1 + x_2$ and $x_i \sim \text{Unif}(\{1, 2, \ldots, 6\})$. From https://en.wikipedia.org/wiki/Probability_distribution. Used with kind permission of Wikipedia author Tim Stellmach.



(a) *(b)*

Figure 2.20: The central limit theorem in pictures. We plot a histogram of $\hat{\mu}_N^s = \frac{1}{N}\sum_{n=1}^N x_{ns}$, where $x_{ns} \sim \text{Beta}(1, 5)$, for $s = 1 : 10000$. As $N \to \infty$, the distribution tends towards a Gaussian. (a) $N = 1$. (b) $N = 5$. Adapted from Figure 2.6 of [Bis06]. Generated by code at figures.probml.ai/book1/2.20.

2.8.6 Central limit theorem

Now consider N random variables with pdf's (not necessarily Gaussian) $p_n(x)$, each with mean μ and variance σ^2. We assume each variable is **independent and identically distributed** or **iid** for short, which means $X_n \sim p(X)$ are independent samples from the same distribution. Let $S_N = \sum_{n=1}^N X_n$ be the sum of the rv's. One can show that, as N increases, the distribution of this sum approaches

$$p(S_N = u) = \frac{1}{\sqrt{2\pi N\sigma^2}} \exp\left(-\frac{(u - N\mu)^2}{2N\sigma^2}\right) \tag{2.171}$$

Hence the distribution of the quantity

$$Z_N \triangleq \frac{S_N - N\mu}{\sigma\sqrt{N}} = \frac{\overline{X} - \mu}{\sigma/\sqrt{N}} \tag{2.172}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

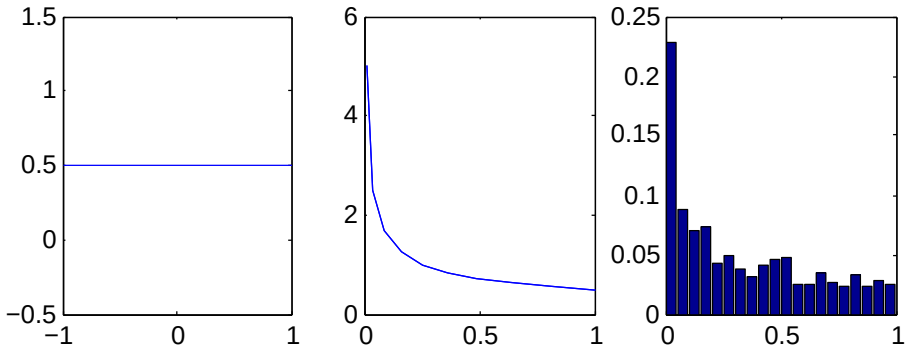


Figure 2.21: Computing the distribution of $y = x^2$, where $p(x)$ is uniform (left). The analytic result is shown in the middle, and the Monte Carlo approximation is shown on the right. Generated by code at figures.probml.ai/book1/2.21.

converges to the standard normal, where $\overline{X} = S_N/N$ is the sample mean. This is called the **central limit theorem**. See e.g., [Jay03, p222] or [Ric95, p169] for a proof.

In Fig. 2.20 we give an example in which we compute the sample mean of rv's drawn from a beta distribution. We see that the sampling distribution of this mean rapidly converges to a Gaussian distribution.

2.8.7 Monte Carlo approximation

Suppose $\mathbf{x}$ is a random variable, and $\mathbf{y} = f(\mathbf{x})$ is some function of $\mathbf{x}$. It is often difficult to compute the induced distribution $p(\mathbf{y})$ analytically. One simple but powerful alternative is to draw a large number of samples from the $\mathbf{x}$'s distribution, and then to use these samples (instead of the distribution) to approximate $p(\mathbf{y})$.

For example, suppose $x \sim \text{Unif}(-1, 1)$ and $y = f(x) = x^2$. We can approximate $p(y)$ by drawing many samples from $p(x)$ (using a uniform **random number generator**), squaring them, and computing the resulting **empirical distribution**, which is given by

$$p_S(y) \triangleq \frac{1}{S} \sum_{s=1}^{S} \delta(y - y_s) \tag{2.173}$$

This is just an equally weighted "sum of spikes", each centered on one of the samples. By using enough samples, we can approximate $p(y)$ rather well. See Fig. 2.21 for an illustration.

This approach is called a **Monte Carlo approximation** to the distribution. (The term "Monte Carlo" comes from the name of a famous gambling casino in Monaco.) Monte Carlo techniques were first developed in the area of statistical physics — in particular, during development of the atomic bomb — but are now widely used in statistics and machine learning as well. More details can be found in the sequel to this book, [Mur22], as well as specialized books on the topic, such as [Liu01; RC04; KTB11; BZ20].

2.9 Exercises

Exercise 2.1 [Conditional independence]

(Source: Koller.)

a. Let $H \in \{1, \ldots, K\}$ be a discrete random variable, and let e_1 and e_2 be the observed values of two other random variables E_1 and E_2. Suppose we wish to calculate the vector

$$\vec{P}(H|e_1, e_2) = (P(H = 1|e_1, e_2), \ldots, P(H = K|e_1, e_2))$$

Which of the following sets of numbers are sufficient for the calculation?

i. $P(e_1, e_2)$, $P(H)$, $P(e_1|H)$, $P(e_2|H)$
ii. $P(e_1, e_2)$, $P(H)$, $P(e_1, e_2|H)$
iii. $P(e_1|H)$, $P(e_2|H)$, $P(H)$

b. Now suppose we now assume $E_1 \perp E_2|H$ (i.e., E_1 and E_2 are conditionally independent given H). Which of the above 3 sets are sufficient now?

Show your calculations as well as giving the final result. Hint: use Bayes rule.

Exercise 2.2 [Pairwise independence does not imply mutual independence]

We say that two random variables are pairwise independent if

$$p(X_2|X_1) = p(X_2) \tag{2.174}$$

and hence

$$p(X_2, X_1) = p(X_1)p(X_2|X_1) = p(X_1)p(X_2) \tag{2.175}$$

We say that n random variables are mutually independent if

$$p(X_i|X_S) = p(X_i) \quad \forall S \subseteq \{1, \ldots, n\} \setminus \{i\} \tag{2.176}$$

and hence

$$p(X_{1:n}) = \prod_{i=1}^{n} p(X_i) \tag{2.177}$$

Show that pairwise independence between all pairs of variables does not necessarily imply mutual independence. It suffices to give a counter example.

Exercise 2.3 [Conditional independence iff joint factorizes]

In the text we said $X \perp Y|Z$ iff

$$p(x, y|z) = p(x|z)p(y|z) \tag{2.178}$$

for all x, y, z such that $p(z) > 0$. Now prove the following alternative definition: $X \perp Y|Z$ iff there exist functions g and h such that

$$p(x, y|z) = g(x, z)h(y, z) \tag{2.179}$$

for all x, y, z such that $p(z) > 0$.

Exercise 2.4 [Expected value of the minimum of two rv's]

Suppose X, Y are two points sampled independently and uniformly at random from the interval $[0, 1]$. What is the expected location of the leftmost point?

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 2.5 [Convolution of two Gaussians is a Gaussian]
Show that the convolution of two Gaussians is a Gaussian, i.e.,

$$p(y) = \mathcal{N}(x_1|\mu_1, \sigma_1^2) \otimes \mathcal{N}(x_2|\mu_2, \sigma_2^2) = \mathcal{N}(y|\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2) \tag{2.180}$$

where $y = x_1 + x_2$, $x_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $x_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$.

Exercise 2.6 [Variance of a sum]
Show that the variance of a sum is

$$\mathbb{V}[X + Y] = \mathbb{V}[X] + \mathbb{V}[Y] + 2\text{Cov}[X, Y], \tag{2.181}$$

where $\text{Cov}[X, Y]$ is the covariance between X and Y.

Exercise 2.7 [Deriving the inverse gamma density]
Let $X \sim \text{Ga}(a, b)$, and $Y = 1/X$. Derive the distribution of Y.

Exercise 2.8 [Mean, mode, variance for the beta distribution]
Suppose $\theta \sim \text{Beta}(a, b)$. Show that the mean, mode and variance are given by

$$\mathbb{E}[\theta] = \frac{a}{a+b} \tag{2.182}$$

$$\mathbb{V}[\boldsymbol{\theta}] = \frac{ab}{(a+b)^2(a+b+1)} \tag{2.183}$$

$$\text{mode}[\boldsymbol{\theta}] = \frac{a-1}{a+b-2} \tag{2.184}$$

Exercise 2.9 [Bayes rule for medical diagnosis]

After your yearly checkup, the doctor has bad news and good news. The bad news is that you tested positive for a serious disease, and that the test is 99% accurate (i.e., the probability of testing positive given that you have the disease is 0.99, as is the probability of testing negative given that you don't have the disease). The good news is that this is a rare disease, striking only one in 10,000 people. What are the chances that you actually have the disease? (Show your calculations as well as giving the final result.)

Exercise 2.10 [Legal reasoning]

(Source: Peter Lee.) Suppose a crime has been committed. Blood is found at the scene for which there is no innocent explanation. It is of a type which is present in 1% of the population.

a. The prosecutor claims: "There is a 1% chance that the defendant would have the crime blood type if he were innocent. Thus there is a 99% chance that he is guilty". This is known as the **prosecutor's fallacy**. What is wrong with this argument?

b. The defender claims: "The crime occurred in a city of 800,000 people. The blood type would be found in approximately 8000 people. The evidence has provided a probability of just 1 in 8000 that the defendant is guilty, and thus has no relevance." This is known as the **defender's fallacy**. What is wrong with this argument?

Exercise 2.11 [Probabilities are sensitive to the form of the question that was used to generate the answer]

(Source: Minka.) My neighbor has two children. Assuming that the gender of a child is like a coin flip, it is most likely, a priori, that my neighbor has one boy and one girl, with probability 1/2. The other possibilities—two boys or two girls—have probabilities 1/4 and 1/4.

a. Suppose I ask him whether he has any boys, and he says yes. What is the probability that one child is a girl?

b. Suppose instead that I happen to see one of his children run by, and it is a boy. What is the probability that the other child is a girl?

Exercise 2.12 [Normalization constant for a 1D Gaussian]

The normalization constant for a zero-mean Gaussian is given by

$$Z = \int_a^b \exp\left(-\frac{x^2}{2\sigma^2}\right) dx \tag{2.185}$$

where $a = -\infty$ and $b = \infty$. To compute this, consider its square

$$Z^2 = \int_a^b \int_a^b \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) dxdy \tag{2.186}$$

Let us change variables from cartesian (x, y) to polar (r, θ) using $x = r\cos\theta$ and $y = r\sin\theta$. Since $dxdy = rdrd\theta$, and $cos^2\theta + \sin^2\theta = 1$, we have

$$Z^2 = \int_0^{2\pi} \int_0^{\infty} r \exp\left(-\frac{r^2}{2\sigma^2}\right) drd\theta \tag{2.187}$$

Evaluate this integral and hence show $Z = \sqrt{\sigma^2 2\pi}$. Hint 1: separate the integral into a product of two terms, the first of which (involving $d\theta$) is constant, so is easy. Hint 2: if $u = e^{-r^2/2\sigma^2}$ then $du/dr = -\frac{1}{\sigma^2} re^{-r^2/2\sigma^2}$, so the second integral is also easy (since $\int u'(r)dr = u(r)$).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

3 Probability: multivariate models

3.1 Joint distributions for multiple random variables

In this section, we discuss various ways to measure the dependence of one or more variables on each other.

3.1.1 Covariance

The **covariance** between two rv's X and Y measures the degree to which X and Y are (linearly) related. Covariance is defined as

$$\mathrm{Cov}\left[X, Y\right] \triangleq \mathbb{E}\left[(X - \mathbb{E}\left[X\right])(Y - \mathbb{E}\left[Y\right])\right] = \mathbb{E}\left[XY\right] - \mathbb{E}\left[X\right]\mathbb{E}\left[Y\right] \tag{3.1}$$

If $\mathbf{x}$ is a D-dimensional random vector, its **covariance matrix** is defined to be the following symmetric, positive semi definite matrix:

$$\mathrm{Cov}\left[\mathbf{x}\right] \triangleq \mathbb{E}\left[(\mathbf{x} - \mathbb{E}\left[\mathbf{x}\right])(\mathbf{x} - \mathbb{E}\left[\mathbf{x}\right])^\top\right] \triangleq \boldsymbol{\Sigma} \tag{3.2}$$

$$= \begin{pmatrix} \mathbb{V}\left[X_1\right] & \mathrm{Cov}\left[X_1, X_2\right] & \cdots & \mathrm{Cov}\left[X_1, X_D\right] \\ \mathrm{Cov}\left[X_2, X_1\right] & \mathbb{V}\left[X_2\right] & \cdots & \mathrm{Cov}\left[X_2, X_D\right] \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}\left[X_D, X_1\right] & \mathrm{Cov}\left[X_D, X_2\right] & \cdots & \mathbb{V}\left[X_D\right] \end{pmatrix} \tag{3.3}$$

from which we get the important result

$$\mathbb{E}\left[\mathbf{x}\mathbf{x}^\top\right] = \boldsymbol{\Sigma} + \boldsymbol{\mu}\boldsymbol{\mu}^\top \tag{3.4}$$

Another useful result is that the covariance of a linear transformation is given by

$$\mathrm{Cov}\left[\mathbf{A}\mathbf{x} + \mathbf{b}\right] = \mathbf{A}\mathrm{Cov}\left[\mathbf{x}\right]\mathbf{A}^\top \tag{3.5}$$

as shown in Exercise 3.4.

The **cross-covariance** between two random vectors is defined as

$$\mathrm{Cov}\left[\mathbf{x}, \mathbf{y}\right] = \mathbb{E}\left[(\mathbf{x} - \mathbb{E}\left[\mathbf{x}\right])(\mathbf{y} - \mathbb{E}\left[\mathbf{y}\right])^\top\right] \tag{3.6}$$

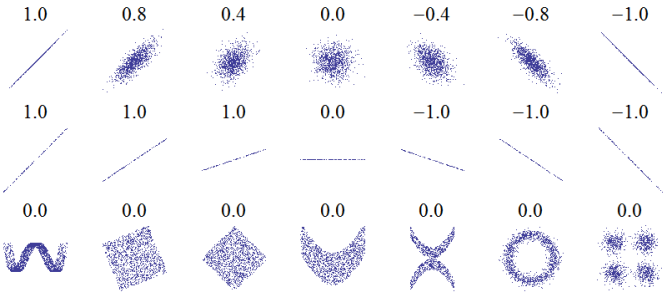


Figure 3.1: Several sets of (x, y) points, with the correlation coefficient of x and y for each set. Note that the correlation reflects the noisiness and direction of a linear relationship (top row), but not the slope of that relationship (middle), nor many aspects of nonlinear relationships (bottom). (Note: the figure in the center has a slope of 0 but in that case the correlation coefficient is undefined because the variance of Y is zero.) From https://en.wikipedia.org/wiki/Pearson_correlation_coefficient. Used with kind permission of Wikipedia author Imagecreator.

3.1.2 Correlation

Covariances can be between negative and positive infinity. Sometimes it is more convenient to work with a normalized measure, with a finite lower and upper bound. The (Pearson) **correlation coefficient** between X and Y is defined as

$$\rho \triangleq \text{corr}\left[X, Y\right] \triangleq \frac{\text{Cov}\left[X, Y\right]}{\sqrt{\mathbb{V}\left[X\right]\mathbb{V}\left[Y\right]}} \tag{3.7}$$

One can show (Exercise 3.2) that $-1 \leq \rho \leq 1$.

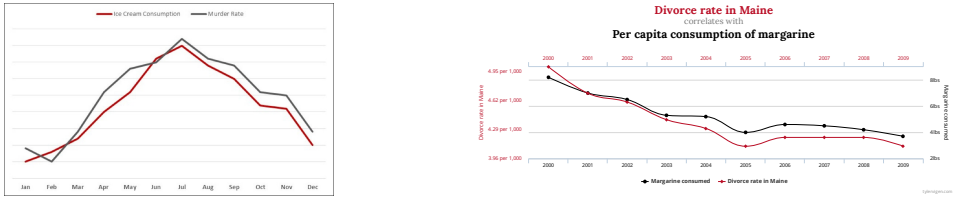
One can also show that $\text{corr}\left[X, Y\right] = 1$ if and only if $Y = aX + b$ for some parameters a and b, i.e., if there is a *linear* relationship between X and Y (see Exercise 3.3). Intuitively one might expect the correlation coefficient to be related to the slope of the regression line, i.e., the coefficient a in the expression $Y = aX + b$. However, as we show in Eq. (11.27), the regression coefficient is in fact given by $a = \text{Cov}\left[X, Y\right] / \mathbb{V}\left[X\right]$. In Fig. 3.1, we show that the correlation coefficient can be 0 for strong, but nonlinear, relationships. (Compare to Fig. 6.6.) Thus a better way to think of the correlation coefficient is as *a degree of linearity*. (See code.probml.ai/book1/correlation2d for a demo to illustrate this idea.)

In the case of a vector $\mathbf{x}$ of related random variables, and the **correlation matrix** is given by

$$\text{corr}(\mathbf{x}) = \begin{pmatrix} 1 & \frac{\mathbb{E}[(X_1-\mu_1)(X_2-\mu_2)]}{\sigma_1\sigma_2} & \cdots & \frac{\mathbb{E}[(X_1-\mu_1)(X_D-\mu_D)]}{\sigma_1\sigma_D} \\ \frac{\mathbb{E}[(X_2-\mu_2)(X_1-\mu_1)]}{\sigma_2\sigma_1} & 1 & \cdots & \frac{\mathbb{E}[(X_2-\mu_2)(X_D-\mu_D)]}{\sigma_2\sigma_D} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\mathbb{E}[(X_D-\mu_D)(X_1-\mu_1)]}{\sigma_D\sigma_1} & \frac{\mathbb{E}[(X_D-\mu_D)(X_2-\mu_2)]}{\sigma_D\sigma_2} & \cdots & 1 \end{pmatrix} \tag{3.8}$$

This can be written more compactly as

$$\text{corr}(\mathbf{x}) = (\text{diag}(\mathbf{K}_{xx}))^{-\frac{1}{2}} \mathbf{K}_{xx} (\text{diag}(\mathbf{K}_{xx}))^{-\frac{1}{2}} \tag{3.9}$$



(a) (b)

Figure 3.2: Examples of spurious correlation between causally unrelated time series. (a) Consumption is ice cream (red) and US murder rate (black) over time. From `https://bit.ly/2zfbuuf`. *Used with kind permission of Karen Cyphers. (b) Divorce rate in Maine strongly correlates with per-capita US consumption of margarine (black). From* `http://www.tylervigen.com/spurious-correlations`. *Used with kind permission of Tyler Vigen.*

where $\mathbf{K}_{xx}$ is the **auto-covariance matrix**

$$\mathbf{K}_{xx} = \boldsymbol{\Sigma} = \mathbb{E}\left[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^\top\right] = \mathbf{R}_{xx} - \boldsymbol{\mu}\boldsymbol{\mu}^\top \tag{3.10}$$

and $\mathbf{R}_{xx} = \mathbb{E}\left[\mathbf{x}\mathbf{x}^\top\right]$ is the **autocorrelation matrix**.

3.1.3 Uncorrelated does not imply independent

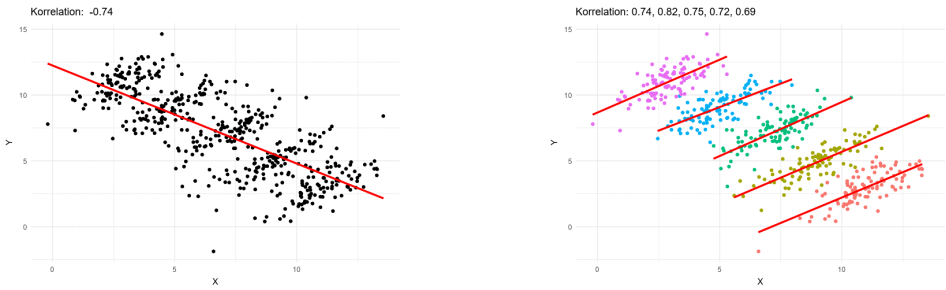
If X and Y are independent, meaning $p(X, Y) = p(X)p(Y)$, then $\text{Cov}[X, Y] = 0$, and hence $\text{corr}[X, Y] = 0$. So independent implies uncorrelated. However, the converse is not true: *uncorrelated does not imply independent*. For example, let $X \sim U(-1, 1)$ and $Y = X^2$. Clearly Y is dependent on X (in fact, Y is uniquely determined by X), yet one can show (Exercise 3.1) that $\text{corr}[X, Y] = 0$. Some striking examples of this fact are shown in Fig. 3.1. This shows several data sets where there is clear dependence between X and Y, and yet the correlation coefficient is 0. A more general measure of dependence between random variables is mutual information, discussed in Sec. 6.3. This is zero only if the variables truly are independent.

3.1.4 Correlation does not imply causation

It is well known that "**correlation does not imply causation**". For example, consider Fig. 3.2a. In red, we plot $x_{1:T}$, where x_t is the amount of ice cream sold in month t. In black, we plot $y_{1:T}$, where y_t is the murder rate in month t. (Quantities have been rescaled to make the plots overlap.) We see a strong correlation between these signals. Indeed, it is sometimes claimed that "eating ice cream causes murder" [Pet13]. Of course, this is just a **spurious correlation**, due to a **hidden common cause**, namely the weather. Hot weather increases ice cream sales, for obvious reasons. Hot weather also increases violent crime; the reason for this is hotly (ahem) debated; some claim it is due to an increase in anger [And01], but other claim it is merely due to more people being outside [Ash18], where most murders occur.

Another example is shown in Fig. 3.2b. This time we plot the divorce rate in Maine (red) and margarine consumption (black). It is less obvious why these should be correlated. Indeed, this is

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 3.3: Illustration of Simpson's paradox. (a) Overall, y decreases y with x. (b) Within each group, y increases with x. From https://en.wikipedia.org/wiki/Simpson's_paradox. Used with kind permission of Wikipedia author Pace svwiki

very likely to not be a robust result; for example, if we looked over a longer period of time, we may find that these signals are uncorrelated.

Another famous example concerns the positive correlation between birth rates and the presence of storks (a kind of bird). This has given rise to the urban legend that storks deliver babies [Mat00]. Of course, the true reason for the correlation is more likely due to hidden factors, such as increased living standards and hence more food. Many more amusing examples of such spurious correlations can be found in [Vig15].

These examples serve as a "warning sign", that we should not treat the ability for x to predict y as an indicator that x causes y.

3.1.5 Simpsons' paradox

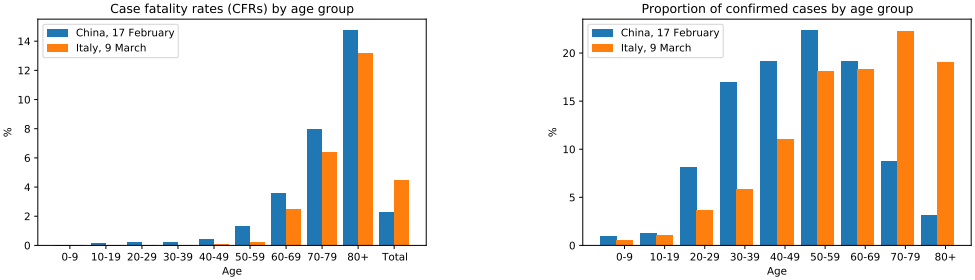
Simpson's paradox says that a statistical trend or relationship that appears in several different groups of data can disappear or reverse sign when these groups are combined. This results in counterintuitive behavior if we misinterpret claims of statistical dependence in a causal way.

A visualization of the paradox is given in Fig. 3.3. Overall, we see that y decreases with x, but within each subpopulation, y increases with x.

For a recent real-world example of Simpson's paradox in the context of COVID-19, consider Fig. 3.4(a). This shows that the case fatality rate (CFR) of COVID-19 in Italy is less than in China in each age group, but is higher overall. The reason for this is that there are more older people in Italy, as shown in Fig. 3.4(b). In other words, Fig. 3.4(a) shows $p(F = 1|A, C)$, where A is age, C is country, and $F = 1$ is the event that someone dies from COVID-19, and Fig. 3.4(b) shows $p(A|C)$, which is the probability someone is in age bucket A for country C. Combining these, we find $p(F = 1|C = \text{Italy}) > p(F = 1|C = \text{China})$. See [KGS20] for more details.

3.2 The multivariate Gaussian (normal) distribution

The most widely used joint probability distribution for continuous random variables is the **multivariate Gaussian** or **multivariate normal** (**MVN**). This is mostly because it is mathematically



(a) *(b)*

Figure 3.4: Illustration of Simpson's paradox using COVID-19, (a) Case fatality rates (CFRs) in Italy and China by age group, and in aggregated form ("Total", last pair of bars), up to the time of reporting (see legend). (b) Proportion of all confirmed cases included in (a) within each age group by country. From Figure 1 of [KGS20]. Used with kind permission of Julius von Kügelgen.

convenient, but also because the Gaussian assumption is fairly reasonable in many cases (see the discussion in Sec. 2.6.4).

3.2.1 Definition

The MVN density is defined by the following:

$$\mathcal{N}(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \triangleq \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{y}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu})\right] \tag{3.11}$$

where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{y}] \in \mathbb{R}^D$ is the mean vector, and $\boldsymbol{\Sigma} = \mathrm{Cov}[\mathbf{y}]$ is the $D \times D$ **covariance matrix**, defined as follows:

$$\mathrm{Cov}[\mathbf{y}] \triangleq \mathbb{E}\left[(\mathbf{y}-\mathbb{E}[\mathbf{y}])(\mathbf{y}-\mathbb{E}[\mathbf{y}])^\top\right] \tag{3.12}$$

$$= \begin{pmatrix} \mathbb{V}[Y_1] & \mathrm{Cov}[Y_1, Y_2] & \cdots & \mathrm{Cov}[Y_1, Y_D] \\ \mathrm{Cov}[Y_2, Y_1] & \mathbb{V}[Y_2] & \cdots & \mathrm{Cov}[Y_2, Y_D] \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}[Y_D, Y_1] & \mathrm{Cov}[Y_D, Y_2] & \cdots & \mathbb{V}[Y_D] \end{pmatrix} \tag{3.13}$$

where

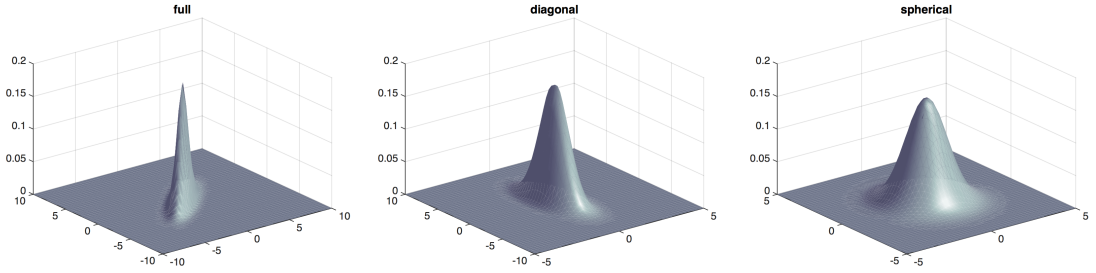
$$\mathrm{Cov}[Y_i, Y_j] \triangleq \mathbb{E}[(Y_i - \mathbb{E}[Y_i])(Y_j - \mathbb{E}[Y_j])] = \mathbb{E}[Y_i Y_j] - \mathbb{E}[Y_i]\,\mathbb{E}[Y_j] \tag{3.14}$$

and $\mathbb{V}[Y_i] = \mathrm{Cov}[Y_i, Y_i]$. From Eq. (3.12), we get the important result

$$\mathbb{E}[\mathbf{y}\mathbf{y}^\top] = \boldsymbol{\Sigma} + \boldsymbol{\mu}\boldsymbol{\mu}^\top \tag{3.15}$$

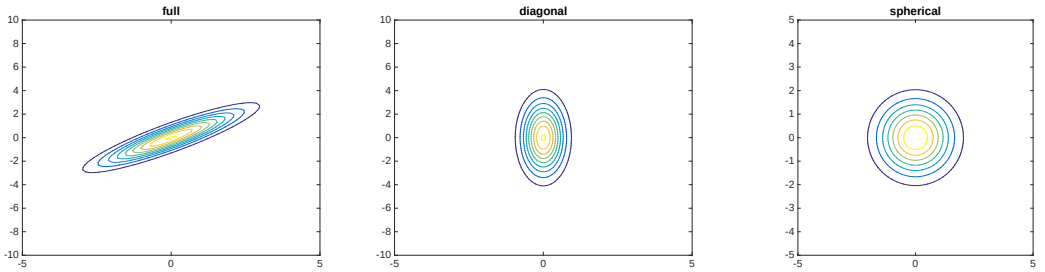
The normalization constant in Eq. (3.11) $Z = (2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}$ just ensures that the pdf integrates to 1 (see Exercise 3.6).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 3.5: Visualization of a 2d Gaussian density as a surface plot. (a) Distribution using a full covariance matrix can be oriented at any angle. (b) Distribution using a diagonal covariance matrix must be parallel to the axis. (c) Distribution using a spherical covariance matrix must have a symmetric shape. Generated by code at figures.probml.ai/book1/3.5.



(a) (b) (c)

Figure 3.6: Visualization of a 2d Gaussian density in terms of level sets of constant probability density. (a) A full covariance matrix has elliptical contours. (b) A diagonal covariance matrix is an **axis aligned** *ellipse. (c) A spherical covariance matrix has a circular shape. Generated by code at figures.probml.ai/book1/3.6.*

In 2d, the MVN is known as the **bivariate Gaussian** distribution. Its pdf can be represented as $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\mathbf{y} \in \mathbb{R}^2$, $\boldsymbol{\mu} \in \mathbb{R}^2$ and

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \sigma_{12}^2 \\ \sigma_{21}^2 & \sigma_2^2 \end{pmatrix} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \tag{3.16}$$

where ρ is the **correlation coefficient**, defined by

$$\text{corr}\left[X, Y\right] \triangleq \frac{\text{Cov}\left[X, Y\right]}{\sqrt{\mathbb{V}\left[X\right]\mathbb{V}\left[Y\right]}} = \frac{\sigma_{12}^2}{\sigma_1\sigma_2} \tag{3.17}$$

One can show (Exercise 3.2) that $-1 \leq \text{corr}\left[X, Y\right] \leq 1$. Expanding out the pdf in the 2d case gives

the following rather intimidating-looking result:

$$p(y_1, y_2) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \exp\left(-\frac{1}{2(1-\rho^2)} \times \left[\frac{(y_1-\mu_1)^2}{\sigma_1^2} + \frac{(y_2-\mu_2)^2}{\sigma_2^2} - 2\rho\frac{(y_1-\mu_1)}{\sigma_1}\frac{(y_2-\mu_2)}{\sigma_2}\right]\right) \tag{3.18}$$

Fig. 3.5 and Fig. 3.6 plot some MVN densities in 2d for three different kinds of covariance matrices. A **full covariance matrix** has $D(D+1)/2$ parameters, where we divide by 2 since $\boldsymbol{\Sigma}$ is symmetric. (The reason for the elliptical shape is explained in Sec. 7.4.4, where we discuss the geometry of quadratic forms.) A **diagonal covariance matrix** has D parameters, and has 0s in the off-diagonal terms. A **spherical covariance matrix**, also called **isotropic covariance matrix**, has the form $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}_D$, so it only has one free parameter, namely σ^2.

3.2.2 Mahalanobis distance

In this section, we attempt to gain some insights into the geometric shape of the Gaussian pdf in multiple dimensions. To do this, we will consider the shape of the **level sets** of constant (log) probability.

The log probability at a specific point $\mathbf{y}$ is given by

$$\log p(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{1}{2}(\mathbf{y}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu}) + \text{const} \tag{3.19}$$

The dependence on $\mathbf{y}$ can be expressed in terms of the **Mahalanobis distance** Δ between $\mathbf{y}$ and $\boldsymbol{\mu}$, whose square is defined as follows:

$$\Delta^2 \triangleq (\mathbf{y}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu}) \tag{3.20}$$

Thus contours of constant (log) probability are equivalent to contours of constant Mahalanobis distance.

To gain insight into the contours of constant Mahalanobis distance, we exploit the fact that $\boldsymbol{\Sigma}$, and hence $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$, are both positive definite matrices (by assumption). Consider the following eigendecomposition (Sec. 7.4) of $\boldsymbol{\Sigma}$:

$$\boldsymbol{\Sigma} = \sum_{d=1}^{D} \lambda_d \mathbf{u}_d \mathbf{u}_d^\top \tag{3.21}$$

We can similarly write

$$\boldsymbol{\Sigma}^{-1} = \sum_{d=1}^{D} \frac{1}{\lambda_d} \mathbf{u}_d \mathbf{u}_d^\top \tag{3.22}$$

Let us define $z_d \triangleq \mathbf{u}_d^\top(\mathbf{y}-\boldsymbol{\mu})$, so $\mathbf{z} = \mathbf{U}(\mathbf{y}-\boldsymbol{\mu})$. Then we can rewrite the Mahalanobis distance as

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

follows:

$$(\mathbf{y}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu}) = (\mathbf{y}-\boldsymbol{\mu})^\top \left(\sum_{d=1}^{D} \frac{1}{\lambda_d}\mathbf{u}_d\mathbf{u}_d^\top\right)(\mathbf{y}-\boldsymbol{\mu}) \tag{3.23}$$

$$= \sum_{d=1}^{D} \frac{1}{\lambda_d}(\mathbf{y}-\boldsymbol{\mu})^\top \mathbf{u}_d\mathbf{u}_d^\top(\mathbf{y}-\boldsymbol{\mu}) = \sum_{d=1}^{D}\frac{z_d^2}{\lambda_d} \tag{3.24}$$

As we discuss in Sec. 7.4.4, this means we can interpret the Mahalanobis distance as Euclidean distance in a new coordinate frame $\mathbf{z}$ in which we rotate $\mathbf{y}$ by $\mathbf{U}$ and scale by $\boldsymbol{\Lambda}$.

For example, in 2d, let us consider the set of points (z_1, z_2) that satisfy this equation:

$$\frac{z_1^2}{\lambda_1} + \frac{z_2^2}{\lambda_2} = r \tag{3.25}$$

Since these points have the same Mahalanobis distance, they correspond to points of equal probability. Hence we see that the contours of equal probability density of a 2d Gaussian lie along ellipses. This is illustrated in Fig. 7.6. The eigenvectors determine the orientation of the ellipse, and the eigenvalues determine how elongated it is.

We can also define the Mahalanobis distance in terms of a linear mapping $\mathbf{L}$ such that $\boldsymbol{\Sigma}^{-1} = \mathbf{L}^\top\mathbf{L}$, where $\mathbf{L}$ may map from $\mathbb{R}^D$ to $\mathbb{R}^d$ where $d \leq D$. Then we have

$$(\mathbf{y}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{y}-\boldsymbol{\mu}) = (\mathbf{y}-\boldsymbol{\mu})^\top \mathbf{L}^\top\mathbf{L}(\mathbf{y}-\boldsymbol{\mu}) = (\mathbf{L}\boldsymbol{\delta})^\top(\mathbf{L}\boldsymbol{\delta}) = ||\mathbf{L}\boldsymbol{\delta}||_2^2 \tag{3.26}$$

where $\boldsymbol{\delta} = \mathbf{y} - \boldsymbol{\mu}$. This can be thought of as measuring Euclidean distance in a lower dimensional space.

3.2.3 Marginals and conditionals of an MVN *

Suppose we have two *discrete* random variables, Y_1 and Y_2. We can represent their joint distribution as a 2d table, all of whose entries sum to one. For example, consider the following example with two binary variables:

$p(Y_1, Y_2)$	$Y_1 = 0$	$Y_2 = 1$
$Y_1 = 0$	0.2	0.3
$Y_2 = 1$	0.3	0.2

Given a joint distribution, we define the **marginal distribution** as follows:

$$p(Y_2 = y_2) = \sum_{y_1} p(Y_1 = y_1, Y_2 = y_2) \tag{3.27}$$

We define the **conditional distribution** as follows:

$$p(Y_1 = y_1 | Y_2 = y_2) = \frac{p(Y_1 = y_1, Y_2 = y_2)}{p(Y_2 = y_2)} \tag{3.28}$$

We now discuss how to extend these results to jointly Gaussian random variables, where we replace sums with integrals. We will partition our vector of random variables $\mathbf{y}$ into two parts, $\mathbf{y}_1$ and

$\mathbf{y}_2$, where $\mathbf{y}_2$ are the variables of interest, and $\mathbf{y}_1$ are called **nuisance variables**. The marginal distribution for $\mathbf{y}_2$ can be computed by integrating out $\mathbf{y}_1$ as follows:

$$p(\mathbf{y}_2) = \int \mathcal{N}(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) d\mathbf{y}_1 = \mathcal{N}(\mathbf{y}_2|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \tag{3.29}$$

where we have defined the following partition of the (moment) parameters:

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \tag{3.30}$$

Now suppose we observe the value of $\mathbf{y}_2$, and we want to predict the value of $\mathbf{y}_1$ conditional on this observation. That is, we want to transform the prior distribution $p(\mathbf{y}_1)$, which represents our beliefs about the values of $\mathbf{y}_1$ before we have seen any observations, into the posterior distribution $p(\mathbf{y}_1|\mathbf{y}_2)$, which represents our beliefs about the values of $\mathbf{y}_1$ after (posterior to) seeing $\mathbf{y}_2$. Exercise 3.11 asks you to derive the following important result:

$$p(\mathbf{y}_1|\mathbf{y}_2) = \mathcal{N}(\mathbf{y}_1|\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{y}_2 - \boldsymbol{\mu}_2),\ \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}) \tag{3.31}$$

Note that the posterior mean is a linear function of $\mathbf{y}_2$, but the posterior covariance is independent of $\mathbf{y}_2$. See Exercise 3.8 for a simple example.

3.2.4 Example: Imputing missing values *

As an example application of the above results, suppose we observe some parts (dimensions) of $\mathbf{y}$, with the remaining parts being missing or unobserved. We can exploit the correlation amongst the dimensions (encoded by the covariance matrix) to infer the missing entries; this is called **missing value imputation**.

Fig. 3.7 shows a simple example. We sampled N vectors from a $D = 10$-dimensional Gaussian, and then deliberately "hid" 50% of the data in each sample (row). We then inferred the missing entries given the observed entries and the true model parameters.[1] More precisely, for each row n of the data matrix, we compute $p(\mathbf{y}_{n,\mathbf{h}}|\mathbf{y}_{n,\mathbf{v}}, \boldsymbol{\theta})$, where $\mathbf{v}$ are the indices of the visible entries in that row, $\mathbf{h}$ are the remaining indices of the hidden entries, and $\boldsymbol{\theta} = (\boldsymbol{\mu}, \boldsymbol{\Sigma})$. From this, we compute the marginal distribution of each missing variable $i \in \mathbf{h}$, $p(y_{n,i}|\mathbf{y}_{n,\mathbf{v}}, \boldsymbol{\theta})$. From the marginal, we compute the posterior mean, $\bar{y}_{n,i} = \mathbb{E}\left[y_{n,i}|\mathbf{y}_{n,\mathbf{v}}, \boldsymbol{\theta}\right]$.

The posterior mean represents our "best guess" about the true value of that entry, in the sense that it minimizes our expected squared error, as explained in Chapter 5. We can use $\mathbb{V}\left[y_{n,i}|\mathbf{y}_{n,\mathbf{v}}, \boldsymbol{\theta}\right]$ as a measure of confidence in this guess, although this is not shown. Alternatively, we could draw multiple posterior samples from $p(\mathbf{y}_{n,\mathbf{h}}|\mathbf{y}_{n,\mathbf{v}}, \boldsymbol{\theta})$; this is called **multiple imputation**, and provides a more robust estimate to downstream algorithms that consume the "filled in" data.

3.3 Linear Gaussian systems *

In Sec. 3.2.3, we conditioned on noise-free observations to infer the posterior over the hidden parts of a Gaussian random vector. In this section, we extend this approach to handle noisy observations.

1. In practice, we would need to estimate the parameters from the partially observed data. Unfortunately the MLE results in Sec. 4.2.6 no longer apply, but we can use the EM algorithm to derive an approximate MLE in the presence of missing data, as explained in Sec. 8.7.4.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

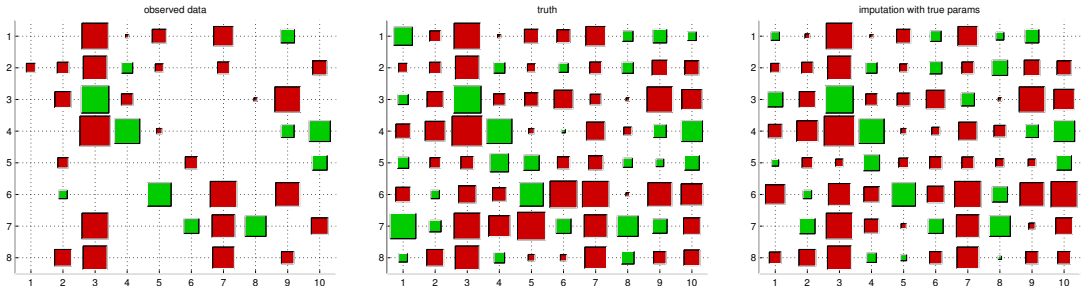


Figure 3.7: Illustration of data imputation using an MVN. (a) Visualization of the data matrix. Blank entries are missing (not observed). Red are positive, green are negative. Area of the square is proportional to the value. (This is known as a **Hinton diagram**, *named after Geoff Hinton, a famous ML researcher.) (b) True data matrix (hidden). (c) Mean of the posterior predictive distribution, based on partially observed data in that row, using the true model parameters. Generated by code at figures.probml.ai/book1/3.7.*

Let $\mathbf{z} \in \mathbb{R}^D$ be an unknown vector of values, and $\mathbf{y} \in \mathbb{R}^K$ be some noisy measurement of $\mathbf{z}$. We assume these variables are related by the following joint distribution:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z) \tag{3.32}$$

$$p(\mathbf{y}|\mathbf{z}) = \mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z} + \mathbf{b}, \boldsymbol{\Sigma}_y) \tag{3.33}$$

where $\mathbf{W}$ is a matrix of size $K \times D$. This is an example of a **linear Gaussian system**.

The corresponding joint distribution, $p(\mathbf{z}, \mathbf{y}) = p(\mathbf{z})p(\mathbf{y}|\mathbf{z})$, is a $D + K$ dimensional Gaussian, with mean and covariance given by

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_z \\ \mathbf{W}\boldsymbol{\mu}_z + \mathbf{b} \end{pmatrix} \tag{3.34}$$

$$\boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_z & \boldsymbol{\Sigma}_z\mathbf{W}^\top \\ \mathbf{W}\boldsymbol{\Sigma}_z & \boldsymbol{\Sigma}_y + \mathbf{W}\boldsymbol{\Sigma}_z\mathbf{W}^\top \end{pmatrix} \tag{3.35}$$

By conditioning on $\mathbf{y}$, we can compute the posterior $p(\mathbf{z}|\mathbf{y})$ using **Bayes' rule for Gaussians**, which is as follows:

$$p(\mathbf{z}|\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{y})} = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_{z|y}, \boldsymbol{\Sigma}_{z|y}) \tag{3.36}$$

$$\boldsymbol{\Sigma}_{z|y}^{-1} = \boldsymbol{\Sigma}_z^{-1} + \mathbf{W}^\top\boldsymbol{\Sigma}_y^{-1}\mathbf{W} \tag{3.37}$$

$$\boldsymbol{\mu}_{z|y} = \boldsymbol{\Sigma}_{z|y}[\mathbf{W}^\top\boldsymbol{\Sigma}_y^{-1}\,(\mathbf{y} - \mathbf{b}) + \boldsymbol{\Sigma}_z^{-1}\boldsymbol{\mu}_z] \tag{3.38}$$

The normalization constant is given by

$$p(\mathbf{y}) = \int \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)\mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z} + \mathbf{b}, \boldsymbol{\Sigma}_y)d\mathbf{z} = \mathcal{N}(\mathbf{y}|\mathbf{W}\boldsymbol{\mu}_z + \mathbf{b}, \boldsymbol{\Sigma}_y + \mathbf{W}\boldsymbol{\Sigma}_z\mathbf{W}^\top) \tag{3.39}$$

This result follows by applying the Gaussian conditioning formula, Eq. (3.31), to the joint $p(\mathbf{y}, \mathbf{z})$.

We see that the Gaussian prior $p(\mathbf{z})$, combined with the Gaussian likelihood $p(\mathbf{y}|\mathbf{z})$, results in a Gaussian posterior $p(\mathbf{z}|\mathbf{y})$. Thus Gaussians are closed under Bayesian conditioning. To describe this more generally, we say that the Gaussian prior is a **conjugate prior** for the Gaussian likelihood, since the posterior distribution has the same type as the prior. We discuss the notion of conjugate priors in more detail in Sec. 4.6.1.

In the sections below, we give various applications of this result.

3.3.1 Example: inferring a latent vector from a noisy sensor

Suppose we have an unknown quantity of interest, $\mathbf{z} \in \mathbb{R}^D$, which we endow with a Gaussian prior, $p(\mathbf{z}) = \mathcal{N}(\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$. If we "know nothing" about $\mathbf{z}$ a priori, we can set $\boldsymbol{\Sigma}_z = \infty\mathbf{I}$, which means we are completely uncertain about what the value of $\mathbf{z}$ should be. (In practice, we can use a large but finite value for the covariance.) By symmetry, it seems reasonable to set $\boldsymbol{\mu}_z = \mathbf{0}$.

Now suppose we make N noisy but independent measurements of $\mathbf{z}$, $\mathbf{y}_n \sim \mathcal{N}(\mathbf{z}, \boldsymbol{\Sigma}_y)$, each of size K, We can represent the likelihood as follows:

$$p(\mathcal{D}|\mathbf{z}) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{y}_n|\mathbf{z}, \boldsymbol{\Sigma}_y) = \mathcal{N}(\overline{\mathbf{y}}|\mathbf{z}, \frac{1}{N}\boldsymbol{\Sigma}_y) \tag{3.40}$$

Note that we can replace the N observations with their average, $\overline{\mathbf{y}}$, provided we scale down the covariance by $1/N$ to compensate. Setting $\mathbf{W} = \mathbf{I}$, $\mathbf{b} = \mathbf{0}$, we can then use Bayes rule for Gaussian to compute the posterior over $\mathbf{z}$:

$$p(\mathbf{z}|\mathbf{y}_1, \ldots, \mathbf{y}_N) = \mathcal{N}(\mathbf{z}|\, \widehat{\boldsymbol{\mu}}, \widehat{\boldsymbol{\Sigma}}) \tag{3.41}$$

$$\widehat{\boldsymbol{\Sigma}}^{-1} = \boldsymbol{\Sigma}_z^{-1} + N\boldsymbol{\Sigma}_y^{-1} \tag{3.42}$$

$$\widehat{\boldsymbol{\mu}} = \widehat{\boldsymbol{\Sigma}}\, (\boldsymbol{\Sigma}_y^{-1}(N\overline{\mathbf{y}}) + \boldsymbol{\Sigma}_z^{-1}\boldsymbol{\mu}_z) \tag{3.43}$$

where $\widehat{\boldsymbol{\mu}}$ and $\widehat{\boldsymbol{\Sigma}}$ are the parameters of the posterior.

Fig. 3.8 gives a 2d example. We can think of $\mathbf{z}$ as representing the true, but unknown, location of an object in 2d space, such as a missile or airplane, and the $\mathbf{y}_n$ as being noisy observations, such as radar "blips". As we receive more blips, we are better able to localize the source. (In the sequel to this book, [Mur22], we discuss the **Kalman filter** algorithm, which extends this idea to a temporal sequence of observations.)

The posterior uncertainty about each component of $\mathbf{z}$ location vector depends on how reliable the sensor is in each of these dimensions. In the above example, the measurement noise in dimension 1 is higher than in dimension 2, so we have more posterior uncertainty about z_1 (horizontal axis) than about z_2 (vertical axis).

3.3.2 Example: inferring a latent vector from multiple noisy sensors

In this section, we extend Sec. 3.3.1, to the case where we have multiple measurements, coming from different sensors, each with different reliabilities. That is, the model has the form

$$p(\mathbf{z}, \mathbf{y}) = p(\mathbf{z}) \prod_{m=1}^{M} \prod_{n=1}^{N_m} \mathcal{N}(\mathbf{y}_{n,m}|\mathbf{z}, \boldsymbol{\Sigma}_m) \tag{3.44}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

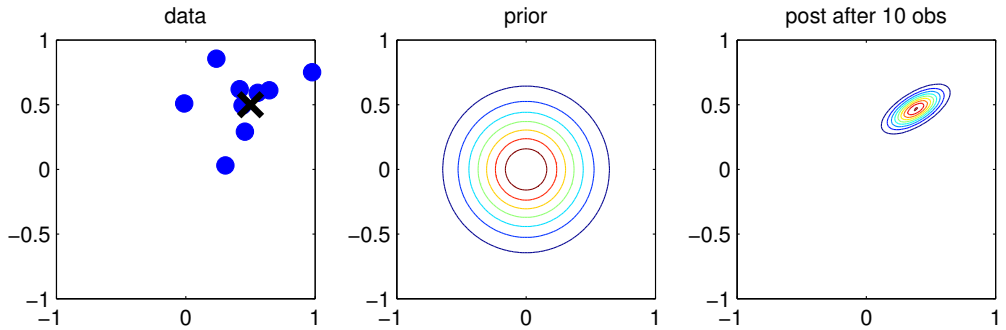


Figure 3.8: Illustration of Bayesian inference for a 2d Gaussian random vector $\mathbf{z}$*. (a) The data is generated from* $\mathbf{y}_n \sim \mathcal{N}(\mathbf{z}, \boldsymbol{\Sigma}_y)$*, where* $\mathbf{z} = [0.5, 0.5]^\top$ *and* $\boldsymbol{\Sigma}_y = 0.1[2, 1; 1, 1]$*). We assume the sensor noise covariance* $\boldsymbol{\Sigma}_y$ *is known but* $\mathbf{z}$ *is unknown. The black cross represents* $\mathbf{z}$*. (b) The prior is* $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, 0.1\mathbf{I}_2)$*. (c) We show the posterior after 10 data points have been observed. Generated by code at figures.probml.ai/book1/3.8.*

where M is the number of sensors (measurement devices), and N_m is the number of observations from sensor m, and $\mathbf{y} = \mathbf{y}_{1:N,1:M} \in \mathbb{R}^K$. Our goal is to combine the evidence together, to compute $p(\mathbf{z}|\mathbf{y})$. This is known as **sensor fusion**.

We now give a simple example, where there are just two sensors, so $\mathbf{y}_1 \sim \mathcal{N}(\mathbf{z}, \boldsymbol{\Sigma}_1)$ and $\mathbf{y}_2 \sim \mathcal{N}(\mathbf{z}, \boldsymbol{\Sigma}_2)$. Pictorially, we can represent this example as $\mathbf{y}_1 \leftarrow \mathbf{z} \rightarrow \mathbf{y}_2$. We can combine $\mathbf{y}_1$ and $\mathbf{y}_2$ into a single vector $\mathbf{y}$, so the model can be represented as $\mathbf{z} \rightarrow [\mathbf{y}_1, \mathbf{y}_2]$, where $p(\mathbf{y}|\mathbf{z}) = \mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z}, \boldsymbol{\Sigma}_y)$, where $\mathbf{W} = [\mathbf{I}; \mathbf{I}]$ and $\boldsymbol{\Sigma}_y = [\boldsymbol{\Sigma}_1, \mathbf{0}; \mathbf{0}, \boldsymbol{\Sigma}_2]$ are block-structured matrices. We can then apply Bayes' rule for Gaussians to compute $p(\mathbf{z}|\mathbf{y})$.

Fig. 3.9(a) gives a 2d example, where we set $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = 0.01\mathbf{I}_2$, so both sensors are equally reliable. In this case, the posterior mean is halfway between the two observations, $\mathbf{y}_1$ and $\mathbf{y}_2$. In Fig. 3.9(b), we set $\boldsymbol{\Sigma}_1 = 0.05\mathbf{I}_2$ and $\boldsymbol{\Sigma}_2 = 0.01\mathbf{I}_2$, so sensor 2 is more reliable than sensor 1. In this case, the posterior mean is closer to $\mathbf{y}_2$. In Fig. 3.9(c), we set

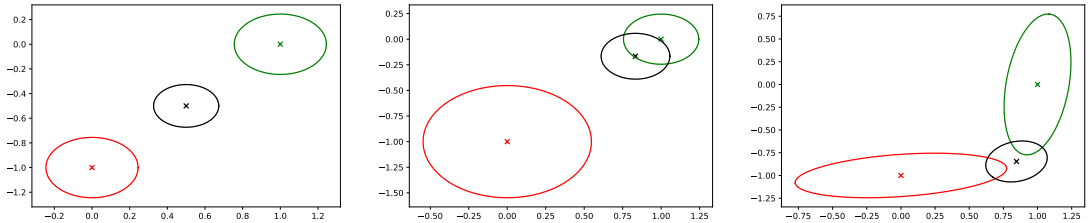
$$\boldsymbol{\Sigma}_1 = 0.01 \begin{pmatrix} 10 & 1 \\ 1 & 1 \end{pmatrix}, \quad \boldsymbol{\Sigma}_2 = 0.01 \begin{pmatrix} 1 & 1 \\ 1 & 10 \end{pmatrix} \tag{3.45}$$

so sensor 1 is more reliable in the second component (vertical direction), and sensor 2 is more reliable in the first component (horizontal direction). In this case, the posterior mean uses $\mathbf{y}_1$'s vertical component and $\mathbf{y}_2$'s horizontal component.

3.4 Mixture models

One way to create more complex probability models is to take a convex combination of simple distributions. This is called a **mixture model**. This has the form

$$p(\mathbf{y}|\boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k p_k(\mathbf{y}) \tag{3.46}$$



(a) (b) (c)

Figure 3.9: We observe $\mathbf{y}_1 = (0, -1)$ (red cross) and $\mathbf{y}_2 = (1, 0)$ (green cross) and estimate $\mathbb{E}[\mathbf{z}|\mathbf{y}_1, \mathbf{y}_2]$ (black cross). (a) Equally reliable sensors, so the posterior mean estimate is in between the two circles. (b) Sensor 2 is more reliable, so the estimate shifts more towards the green circle. (c) Sensor 1 is more reliable in the vertical direction, Sensor 2 is more reliable in the horizontal direction. The estimate is an appropriate combination of the two measurements. Generated by code at figures.probml.ai/book1/3.9.

where p_k is the k'th mixture component, and π_k are the mixture weights which satisfy $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^{K} \pi_k = 1$.

We can re-express this model as a hierarchical model, in which we introduce the discrete **latent variable** $z \in \{1, \ldots, K\}$, which specifies which distribution to use for generating the output $\mathbf{y}$. The prior on this latent variable is $p(z = k) = \pi_k$, and the conditional is $p(\mathbf{y}|z = k) = p_k(\mathbf{y}) = p(\mathbf{y}|\boldsymbol{\theta}_k)$. That is, we define the following joint model:

$$p(z|\boldsymbol{\theta}) = \text{Cat}(z|\boldsymbol{\pi}) \tag{3.47}$$

$$p(\mathbf{y}|z = k, \boldsymbol{\theta}) = p(\mathbf{y}|\boldsymbol{\theta}_k) \tag{3.48}$$

The "generative story" for the data is that we first sample a specific component z, and then we generate the observations $\mathbf{y}$ using the parameters chosen according to the value of z. By marginalizing out z, we recover Eq. (3.46):

$$p(\mathbf{y}|\boldsymbol{\theta}) = \sum_{k=1}^{K} p(z = k|\boldsymbol{\theta}) p(\mathbf{y}|z = k, \boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k p(\mathbf{y}|\boldsymbol{\theta}_k) \tag{3.49}$$

We can create different kinds of mixture model by varying the base distribution p_k, as we illustrate below.

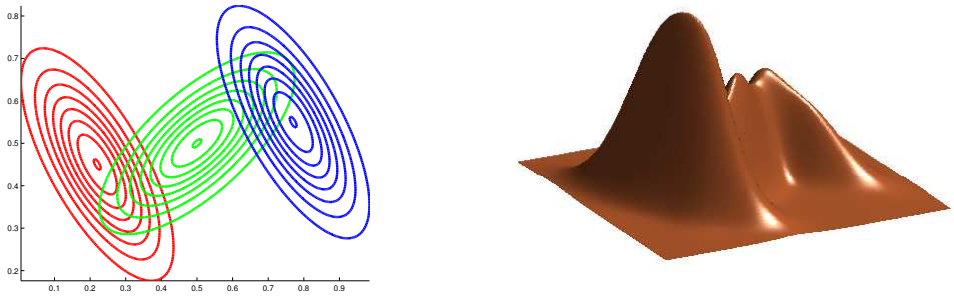
3.4.1 Gaussian mixture models

A **Gaussian mixture model** or **GMM**, also called a **mixture of Gaussians** (**MoG**), is defined as follows:

$$p(\mathbf{y}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{y}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{3.50}$$

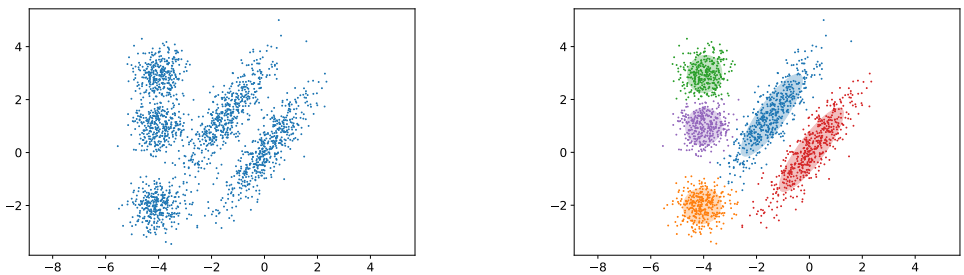
In Fig. 3.10 we show the density defined by a mixture of 3 Gaussians in 2d. Each mixture component is represented by a different set of elliptical contours. If we let the number of mixture components grow sufficiently large, a GMM can approximate any smooth distribution over $\mathbb{R}^D$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 3.10: A mixture of 3 Gaussians in 2d. (a) We show the contours of constant probability for each component in the mixture. (b) A surface plot of the overall density. Adapted from Figure 2.23 of [Bis06]. Generated by code at figures.probml.ai/book1/3.10.



(a) (b)

Figure 3.11: (a) Some data in 2d. (b) A possible clustering using $K = 3$ clusters computed using a GMM. Generated by code at figures.probml.ai/book1/3.11.

3.4.1.1 Using GMMs for clustering

GMMs are often used for unsupervised **clustering** of real-valued data samples $\mathbf{y}_n \in \mathbb{R}^D$. This works in two stages. First we fit the model e.g., by computing the MLE $\hat{\boldsymbol{\theta}} = \text{argmax} \log p(\mathcal{D}|\boldsymbol{\theta})$, where $\mathcal{D} = \{\mathbf{y}_n : n = 1 : N\}$. (We discuss how to compute this MLE in Sec. 8.7.3.) Then we associate each data point $\mathbf{y}_n$ with a discrete latent or hidden variable $z_n \in \{1, \ldots, K\}$ which specifies the identity of the mixture component or cluster which was used to generate $\mathbf{y}_n$. These latent identities are unknown, but we can compute a posterior over them using Bayes rule:

$$r_{nk} \triangleq p(z_n = k|\mathbf{x}_n, \boldsymbol{\theta}) = \frac{p(z_n = k|\boldsymbol{\theta})p(\mathbf{x}_n|z_n = k, \boldsymbol{\theta})}{\sum_{k'=1}^{K} p(z_n = k'|\boldsymbol{\theta})p(\mathbf{x}_n|z_n = k', \boldsymbol{\theta})} \tag{3.51}$$

The quantity r_{nk} is called the **responsibility** of cluster k for data point n. Given the responsibilities, we can compute the most probable cluster assignment as follows:

$$\hat{z}_n = \arg\max_k r_{nk} = \arg\max_k \left[\log p(\mathbf{x}_n|z_n = k, \boldsymbol{\theta}) + \log p(z_n = k|\boldsymbol{\theta})\right] \tag{3.52}$$

This is known as **hard clustering**. (If we use the responsibilities to fractionally assign each data point to different clusters, it is called **soft clustering**.) See Fig. 3.11 for an example.

If we have a uniform prior over z_n, and we use spherical Gaussians with $\boldsymbol{\Sigma}_k = \mathbf{I}$, the hard clustering problem reduces to

$$z_n = \underset{k}{\operatorname{argmin}} ||\mathbf{y}_n - \hat{\boldsymbol{\mu}}_k||_2^2 \tag{3.53}$$

In other words, we assign each data point to its closest centroid, as measured by Euclidean distance. This is the basis of the **K-means clustering** algorithm, which we discuss in Sec. 21.3.

3.4.1.2 Using GMMs as a prior to regularize an inverse problem

In this section, we consider using GMMs as a blackbox density model to regularize the inversion of a many-to-one mapping. Specifically, we consider the problem of inferring a "clean" image $\mathbf{z}$ from a corrupted version $\mathbf{y}$. We use a linear-Gaussian forwards model of the form

$$p(\mathbf{y}|\mathbf{z}) = \mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I}) \tag{3.54}$$

where σ^2 is the variance of the measurement noise. The form of the matrix $\mathbf{W}$ depends on the nature of the corruption, which we assume is known, for simplicity. Here are some common examples of different kinds of corruption we can model in our approach:

- If the corruption is due to additive noise (as in Fig. 3.12a), we can set $\mathbf{W} = \mathbf{I}$. The resulting MAP estimate can be used for **image denoising**, as in Fig. 3.12b.
- If the corruption is due to blurring (as in Fig. 3.12c), we can set $\mathbf{W}$ to be a fixed convolutional kernel [KF09]. The resulting MAP estimate can be used for **image deblurring**, as in Fig. 3.12d.
- If the corruption is due to occlusion (as in Fig. 3.12e), we can set $\mathbf{W}$ to be a diagonal matrix, with 0s in the locations corresponding to the occluders. The resulting MAP estimate can be used for **image inpainting**, as in Fig. 3.12f.
- If the corruption is due to downsampling, we can set $\mathbf{W}$ to a convolutional kernel. The resulting MAP estimate can be used for **image super-resolution**.

Thus we see that the linear-Gaussian likelihood model is surprisingly flexible. Given the model, our goal is to invert it, by computing the MAP estimate $\hat{\mathbf{z}} = \operatorname{argmax} p(\mathbf{z}|\mathbf{y})$. However, the problem of inverting this model is ill-posed, since there are many possible latent images $\mathbf{z}$ that map to the same observed image $\mathbf{y}$. Therefore we need to use a prior to regularize the inversion process.

In [ZW11], they propose to use a GMM prior of the form $p(\mathbf{z}) = \sum_k p(c=k)\mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, using $K = 200$ mixture components, which they fit a dataset of 2M clean image patches. The full joint model therefore has the form

$$p(c=k, \mathbf{z}, \mathbf{y}|\boldsymbol{\theta}) = p(c=k)\mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I}) \tag{3.55}$$

To compute the marginal MAP estimate $\operatorname{argmax}_{\mathbf{z}} p(\mathbf{z}|\mathbf{y})$, we can use the approximation

$$p(\mathbf{z}|\mathbf{y}) \approx p(\mathbf{z}|\mathbf{y}, c^*) \propto \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_{c*}, \boldsymbol{\Sigma}_{c^*})\mathcal{N}(\mathbf{y}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I}) \tag{3.56}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

(a) *(b)* *(c)* *(d)* *(e)* *(f)*

Figure 3.12: Example of recovering a clean image (right) from a corrupted version (left) using MAP estimation with a GMM patch prior and Gaussian likelihood. First row: image denoising. Second row: image deblurring. Third row: image inpainting. From [RW15] and [ZW11]. Used with kind permission of Dan Rosenbaum and Daniel Zoran.

If we know c^*, we can compute the above using Bayes rule for Gaussians in Eq. (3.36). To compute the MAP mixture component, c^*, we can use Eq. (3.39) to get

$$c^* = \operatorname*{argmax}_c p(c)p(\mathbf{y}|c) = \operatorname*{argmax}_c p(c)\mathcal{N}(\mathbf{y}|\mathbf{W}\boldsymbol{\mu}_c, \sigma^2\mathbf{I} + \mathbf{W}\boldsymbol{\Sigma}_c\mathbf{W}^\top) \tag{3.57}$$

See Fig. 3.12 for some examples of this process in action.

When the amount of corruption is more severe, more sophisticated prior models, such as normalizing flows proposed in [WLD20], may be necessary. If the corruption is not known, it is called a "**blind inverse problem**", which is significantly harder. See e.g., [Ani+18; XC19; Wan+20c] for some relevant works.

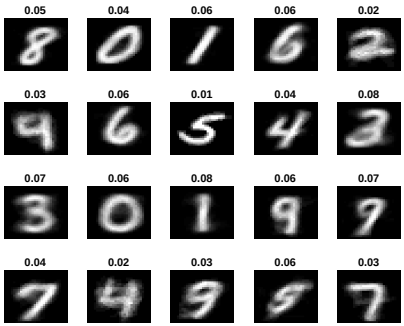


Figure 3.13: We fit a mixture of 20 Bernoullis to the binarized MNIST digit data. We visualize the estimated cluster means $\hat{\boldsymbol{\mu}}_k$. The numbers on top of each image represent the estimated mixing weights $\hat{\pi}_k$. No labels were used when training the model. Generated by code at figures.probml.ai/book1/3.13.

3.4.2 Mixtures of Bernoullis

If the data is binary valued, we can use a **mixture of Bernoullis**, where each mixture component has the following form:

$$p(\mathbf{y}|z=k,\boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Ber}(y_d|\mu_{dk}) = \prod_{d=1}^{D} \mu_{dk}^{y_d}(1-\mu_{dk})^{1-y_d} \tag{3.58}$$

Here μ_{dk} is the probability that bit d turns on in cluster k.

For example, consider building an unsupervised density model for the famous **MNIST** datataset [LeC+98; YB19]. This is a dataset of 60k training images of size $28 \times 28 \times 1$ (the final dimension of size 1 means the images are gray scale) illustrating handwritten digits from 10 categories; see Fig. 1.12a for an illustration of the data. There are 60k training examples and 10k test examples.

We fit a mixture of Bernoullis using $K = 20$ components to the traninig set data. (We use the EM algorithm to do this fitting — see Sec. 8.7.3 for details.) The resulting parameters for each mixture component (i.e., $\boldsymbol{\mu}_k$ and π_k) are shown in Fig. 3.13. We see that the model has "discovered" a representation of each type of digit. (Some digits are represented multiple times, since the model does not know the "true" number of classes. See Sec. 21.3.7 for more information on how to choose the number K of mixture components.)

3.4.3 Gaussian scale mixtures *

A **Gaussian scale mixture** of **GSM** [AM74; Wes87] is like an "infinite" mixture of Gaussians, each with a different scale (variance). More precisely, let $x = \epsilon z$, where $z \sim \mathcal{N}(0, \sigma_0^2)$ and $\epsilon \sim p(\epsilon)$. We can think of this as **multiplicative noise** being applied to the Gaussian rv z. We have $x|\epsilon \sim \mathcal{N}(0, \epsilon^2\sigma_0^2)$. Marginalizing out the scale ϵ gives

$$p(x) = \int \mathcal{N}(x|0, \sigma_0^2\epsilon^2)p(\epsilon^2)d\epsilon \tag{3.59}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

By changing the prior $p(\epsilon)$, we can create various interesting distributions. We give some examples below.

The main advantage of this approach is that it is often computationally more convenient to work with the **expanded parameterization**, in which we explicitly include the scale term ϵ, since, conditional on that, the distribution is Gaussian. We use this formulation in Sec. 11.4.1.1, where we discuss robust regression.

3.4.3.1 Example: Student t distribution as GSM

We can represent the Student distribution as a GSM as follows:

$$\mathcal{T}(y|0,\sigma^2,\nu) = \int_0^\infty dz \mathcal{N}(y|0,z\sigma^2)\text{IG}(z|\frac{\nu}{2},\frac{\nu}{2}) = \int_0^\infty dz \mathcal{N}(y|0,z\sigma^2)\chi^{-2}(z|\nu,1)dz \tag{3.60}$$

where IG is the inverse Gamma distribution (Sec. 2.7.5). (See Exercise 3.10 for the derivation.) Thus we can think of the Student as an infinite superposition of Gaussians of different widths; marginalizing this out induces a distribution with wider tails than a Gaussian with the same variance. This result also explains why the Student distribution approaches a Gaussian as the dof gets large, since when $\nu = \infty$, the inverse Gamma distribution becomes a delta function.

3.4.3.2 Example: Laplace distribution as GSM

Similarly one can show that the Laplace distribution is an infinite weighted sum of Gaussians, where the precision comes from a Gamma distribution:

$$\text{Lap}(w|0,\lambda) = \int \mathcal{N}(w|0,\tau^2)\text{Ga}(\tau^2|1,\frac{\lambda^2}{2})d\tau^2 \tag{3.61}$$

3.5 Probabilistic graphical models *

> I basically know of two principles for treating complicated systems in simple ways: the first is the principle of modularity and the second is the principle of abstraction. I am an apologist for computational probability in machine learning because I believe that probability theory implements these two principles in deep and intriguing ways — namely through factorization and through averaging. Exploiting these two mechanisms as fully as possible seems to me to be the way forward in machine learning. — Michael Jordan, 1997 (quoted in [Fre98]).

We have now introduced a few simple probabilistic building blocks. In Sec. 3.3, we showed one way to combine some Gaussian building blocks to build a high dimensional distribution $p(\mathbf{y})$ from simpler parts, namely the marginal $p(\mathbf{y}_1)$ and the conditional $p(\mathbf{y}_2|\mathbf{y}_1)$. This idea can be extended to define joint distributions over sets of many random variables. The key assumption we will make is that some variables are **conditionally independent** of others. We will represent our CI assumptions using graphs, as we briefly explain below. (See the sequel to this book, [Mur22], for more information.)

3.5.1 Representation

A **probabilistic graphical model** or **PGM** is a joint probability distribution that uses a graph structure to encode conditional independence assumptions. When the graph is a **directed acyclic**

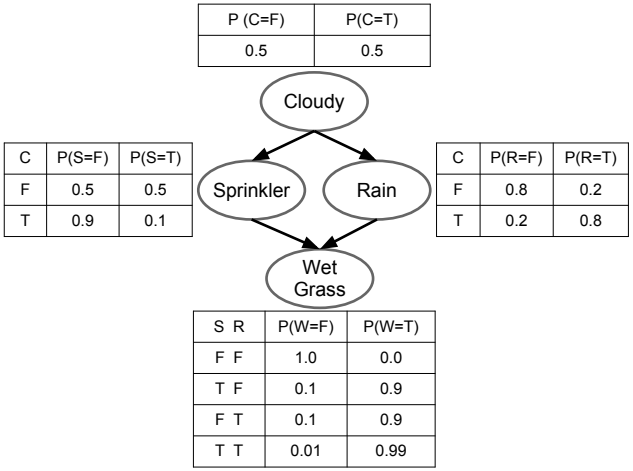


Figure 3.14: Water sprinkler PGM with corresponding binary CPTs. T and F stand for true and false.

graph or **DAG**, the model is sometimes called a **Bayesian network**, although there is nothing inherently Bayesian about such models.

The basic idea in PGMs is that each node in the graph represents a random variable, and each edge represents a direct dependency. More precisely, each lack of edge represents a conditional independency. In the DAG case, we can number the nodes in **topological order** (parents before children), and then we connect them such that each node is conditionally independent of all its predecesors given its parents:

$$Y_i \perp \mathbf{Y}_{\text{pred}(i) \setminus \text{pa}(i)} | \mathbf{Y}_{\text{pa}(i)} \tag{3.62}$$

where $\text{pa}(i)$ are the parents of node i, and $\text{pred}(i)$ are the predecessors of node i in the ordering. (This is called the **ordered Markov property**.) Consequently, we can represent the joint distribution as follows:

$$p(\mathbf{Y}_{1:V}) = \prod_{i=1}^{V} p(Y_i | \mathbf{Y}_{\text{pa}(i)}) \tag{3.63}$$

where V is the number of nodes in the graph.

3.5.1.1 Example: water sprinkler network

For example, suppose we want to model the dependencies between 4 discrete (binary) random variables related to the weather: C (whether it is cloudy or not), R (whether it is raining or not), S (whether the water sprinkler is on or not), and W (whether the grass is wet or not). Intuitively, clouds cause rain to fall, but prevent sprinklers from turning on (either because of a sensor or human intervention). The rain or the sprinklers can both cause the grass to get wet. We can represent these assumptions using the DAG in Fig. 3.14, where we number the nodes as follows: $C = Y_1$, $S = Y_2$,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

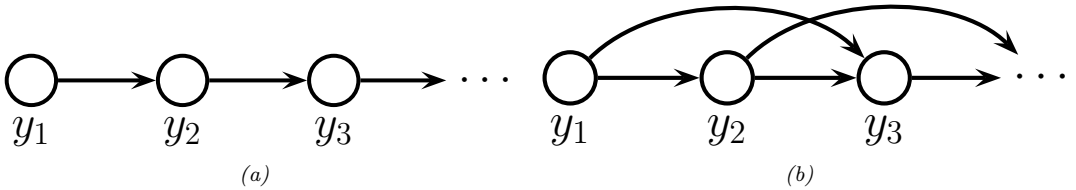


Figure 3.15: Illustration of first and second order autoregressive (Markov) models.

$R = Y_3$ and $W = Y_4$. This model defines the following joint distribution:

$$p(C, S, R, W) = p(C)p(S|C)p(R|C, \not{S})p(W|\not{C}, S, R) \tag{3.64}$$

where we strike through terms that are not needed due to the conditional independence properties of the model.

Each term $p(Y_i|\mathbf{Y}_{\text{pa}(i)})$ is a called the **conditional probability distribution** or **CPD** for node i. This can be any kind of distribution we like. In Fig. 3.14, we assume each CPD is a conditional categorical distribution, which can be represented as a **conditional probability table** or **CPT**. We can represent the i'th CPT as follows:

$$\theta_{ijk} \triangleq p(Y_i = k|\mathbf{Y}_{\text{pa}(i)} = j) \tag{3.65}$$

This satisfies the properties $0 \leq \theta_{ijk} \leq 1$ and $\sum_{k=1}^{K_i} \theta_{ijk} = 1$ for each row j. Here i indexes nodes, $i \in [V]$; k indexes node states, $k \in [K_i]$, where K_i is the number of states for node i; and j indexes joint parent states, $j \in [J_i]$, where $J_i = \prod_{p \in \text{pa}(i)} K_p$. For example, the wet grass node has 2 binary parents, so there are 4 parent states.

3.5.1.2 Example: Markov chain

Now suppose we want to create a joint probability distribution over variable-length sequences, $p(y_{1:T})$. If each variable y_t represents a word from a vocabulary with K possible values, so $y_t \in \{1, \ldots, K\}$, the resulting model represents a distribution over possible sentences of length T; this is often called a **language model**.

By the chain rule of probability, we can represent any joint distribution over T variables as follows:

$$p(\mathbf{y}_{1:T}) = p(y_1)p(y_2|y_1)p(y_3|y_2, y_1)p(y_4|y_3, y_2, y_1)\ldots = \prod_{t=1}^{T} p(y_t|\mathbf{y}_{1:t-1}) \tag{3.66}$$

Unfortunately, the number of parameters needed to represent each conditional distribution $p(y_t|\mathbf{y}_{1:t-1})$ grows exponentially with t. However, suppose we make the conditional independence assumption that the future, $\mathbf{y}_{t+1:T}$, is independent of the past, $\mathbf{y}_{1:t-1}$, given the present, y_t. This is called the **first order Markov condition**, and is repesented by the PGM in Fig. 3.15(a). With this assumption, we can write the joint distribution as follows:

$$p(\mathbf{y}_{1:T}) = p(y_1)p(y_2|y_1)p(y_3|y_2)p(y_4|y_3)\ldots = p(y_1)\prod_{t=2}^{T} p(y_t|y_{t-1}) \tag{3.67}$$

This is called a **Markov chain**, **Markov model** or **autoregressive model** of order 1.

The function $p(y_t|y_{t-1})$ is called the **transition function**, **transition kernel** or **Markov kernel**. This is just a conditional distribution over the states at time t given the state at time $t-1$, and hence it satisfies the conditions $p(y_t|y_{t-1}) \geq 0$ and $\sum_{k=1}^{K} p(y_t = k|y_{t-1} = j) = 1$. We can represent this CPT as a **stochastic matrix**, $A_{jk} = p(y_t = k|y_{t-1} = j)$, where each row sums to 1. This is known as the **state transition matrix**. We assume this matrix is the same for all time steps, so the model is said to be **homogeneous**, **stationary**, or **time-invariant**. This is an example of **parameter tying**, since the same parameter is shared by multiple variables. This assumption allows us to model an arbitrary number of variables using a fixed number of parameters.

The first-order Markov assumption is rather strong. Fortunately, we can easily generalize first-order models to depend on the last M observations, thus creating a model of order (memory length) M:

$$p(\mathbf{y}_{1:T}) = p(\mathbf{y}_{1:M}) \prod_{t=M+1}^{T} p(y_t|\mathbf{y}_{t-M:t-1}) \tag{3.68}$$

This is called an M**'th order Markov model**. For example, if $M = 2$, y_t depends on y_{t-1} and y_{t-2}, as shown in Fig. 3.15(b). This is called a **trigram model**, since it models the distribution over word triples. If we use $M = 1$, we get a **bigram model**, which models the distribution over word pairs.

For large vocabulary sizes, the number of parameters needed to estimate the conditional distributions for M-gram models for large M can become prohibitive. In this case, we need to make additional assumptions beyond conditional independence. For example, we can assume that $p(y_t|\mathbf{y}_{t-M:t-1})$ can be represented as a low-rank matrix, or in terms of some kind of neural network. This is called a **neural language model**. See Chapter 15 for details.

3.5.2 Inference

A PGM defines a joint probability distribution. We can therefore use the rules of marginalization and conditioning to compute $p(\mathbf{Y}_i|\mathbf{Y}_j = \mathbf{y}_j)$ for any sets of variables i and j.

3.5.2.1 Example: water sprinkler

For example, consider the water sprinkler example in Fig. 3.14. We can compute our prior belief that it has rained as follows:

$$p(R = 1) = \sum_{c=0}^{1} p(C = c)p(R = 1|C = c) = 0.5 \tag{3.69}$$

If we see that the grass is wet, then our (posterior) belief that it has rained changes to $p(R = 1|W = 1) = 0.7079$. Now suppose we also notice the water sprinkler was turned on: our belief that it rained goes down to $p(R = 1|W = 1, S = 1) = 0.3204$. This negative mutual interaction between multiple causes of some observations is called the **explaining away** effect, also known as **Berkson's paradox**. (See code.probml.ai/book1/sprinkler_pgm for some code that reproduces these calculations.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

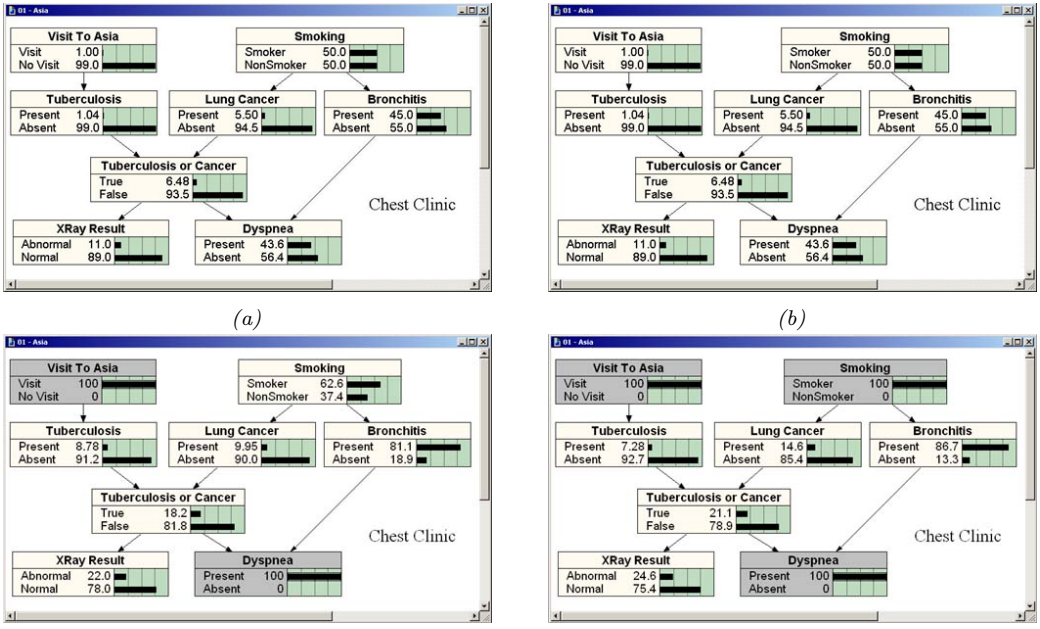


Figure 3.16: Illustration of belief updating in the "Asia" PGM. The histograms show the marginal distribution of each node. (a) Prior. (b) Posterior after conditioning on Dyspnea=1. (c) Posterior after also conditioning on VisitToAsia=1. (d) Posterior after also conditioning on Smoking=1. From `https://bit.ly/3h8DWQP`. *Used with kind permission of Brent Boerlage.*

3.5.2.2 Example: Asia network

Let us now give a more interesting example. We will consider a hypothetical model proposed in [LS88]; this is known as the "**Asia network**", since it was designed to model various lung diseases in Western patients who may have recently visited Asia. (Note: this is purely for pedagogical purposes, and it is not a realistic model. Also, it should be apparent that we could, in principle, create a similar model for COVID-19 diagnosis.)

Fig. 3.16a shows the model, as well as the prior marginal distributions over each node (assumed to be binary). Now suppose the patient reports that they have **Dyspnea**, aka shortness of breath. We can represent this fact by "shading" that node a dark color, and then clamping the distribution to be 100% probability that Dyspnea=1, and 0% probability that Dyspnea=0. We then propagate this new information through the network to get the updated marginal distributions shown in Fig. 3.16b. We see that the probability of bronchitis has gone up, from 45% to 83.4%, since this is the most likely cause or explanation of the evidence. However, there is some probability that the symptom (Dyspnea) was caused by tuberculosis (TB) or Lung Cancer.

Now suppose the doctor asks the patient if they have recently been to Asia, and they say yes. Fig. 3.16c shows the new belief state of each node. We see that the probability of TB has increased from 2% to 9%. However, Bronchitis remains the most likely explanation, so the doctor asks if the

patient smokes, and they say yes. Fig. 3.16c shows the new belief state of each node. We see that we are now even more confident that the patient has Bronchitis, and not TB or Lung Cancer. The predicted probability that an X-ray will show an abnormal result has gone up, to 24%, so the doctor may decide it is worth ordering a test to verify this hypothesis.

3.5.2.3 Computational issues

We have seen that Bayesian inference allows us to update our beliefs about the world by combining noisy evidence with prior knowledge. Unfortunately, the process of computing marginals and conditionals from joint distributions can be computationally expensive. Fortunately, we can often exploit the CI properties of the distribution to make the computation much more efficient. We discuss this topic briefly in Sec. 4.6.8, but go into more detail in the sequel to this book, [Mur22].

3.5.3 Learning

If the parameters of the CPDs are unknown, we can view them as additional random variables, add them as nodes to the graph, and then treat them as **hidden variables** to be inferred. Fig. 3.17(a) shows a simple example, in which we have N iid random variables, $\mathbf{y}_n$, all drawn from the same distribution with common parameter $\boldsymbol{\theta}$. (The **shaded nodes** represent observed values, whereas the unshaded (hollow) nodes represent latent variables or parameters.)

More precisely, the model encodes the following "generative story" about the data:

$$\boldsymbol{\theta} \sim p(\boldsymbol{\theta}) \tag{3.70}$$

$$\mathbf{y}_n \sim p(\mathbf{y}|\boldsymbol{\theta}) \tag{3.71}$$

where $p(\boldsymbol{\theta})$ is some (unspecified) prior over the parameters, and $p(\mathbf{y}|\boldsymbol{\theta})$ is some specified likelihood function. The corresponding joint distribution has the form

$$p(\mathcal{D}, \boldsymbol{\theta}) = p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta}) \tag{3.72}$$

where $\mathcal{D} = (\mathbf{y}_1, \ldots, \mathbf{y}_N)$. By virtue of the iid assumption, the likelihood can be rewritten as follows:

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} p(\mathbf{y}_n|\boldsymbol{\theta}) \tag{3.73}$$

Notice that the order of the data vectors is not important for defining this model, i.e., we can permute the numbering of the leaf nodes in the PGM. When this property holds, we say that the data is **exchangeable**.

3.5.3.1 Plate notation

In Fig. 3.17(a), we see that the $\mathbf{y}$ nodes are repeated N times. To avoid visual clutter, it is common to use a form of **syntactic sugar** called **plates**. This is a notational convention in which we draw a little box around the repeated variables, with the understanding that nodes within the box will get repeated when the model is **unrolled**. We often write the number of copies or repetitions in the bottom right corner of the box. This is illustrated in Fig. 3.17(b). This notation is widely used to represent certain kinds of Bayesian model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

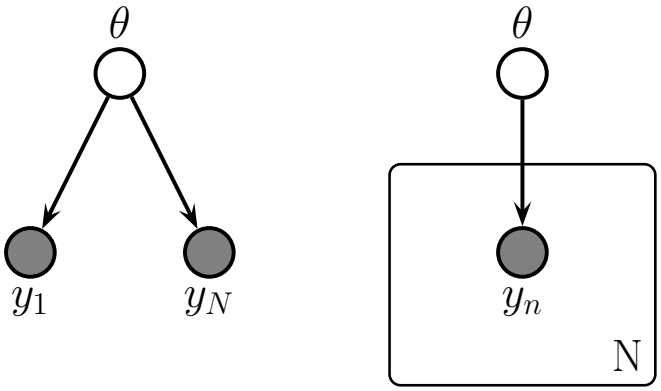


Figure 3.17: Left: data points $\mathbf{y}_n$ are conditionally independent given $\boldsymbol{\theta}$. Right: Same model, using plate notation. This represents the same model as the one on the left, except the repeated $\mathbf{y}_n$ nodes are inside a box, known as a plate; the number in the lower right hand corner, N, specifies the number of repetitions of the $\mathbf{y}_n$ node.

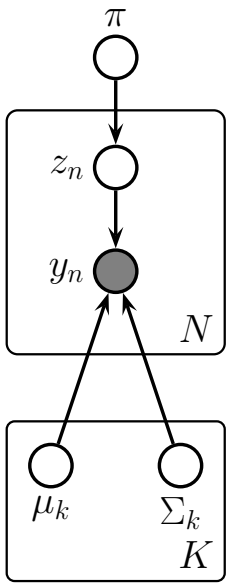


Figure 3.18: A Gaussian mixture model represented as a graphical model.

Fig. 3.18 shows a more interesting example, in which we represent a GMM (Sec. 3.4.1) as a graphical model. We see that this encodes the joint distribution

$$p(\mathbf{y}_{1:N}, \mathbf{z}_{1:N}, \boldsymbol{\theta}) = p(\boldsymbol{\pi}) \left[\prod_{k=1}^{K} p(\boldsymbol{\mu}_k) p(\boldsymbol{\Sigma}_k)\right] \left[\prod_{n=1}^{N} p(z_n|\boldsymbol{\pi}) p(\mathbf{y}_n|z_n, \boldsymbol{\mu}_{1:K}, \boldsymbol{\Sigma}_{1:K})\right] \tag{3.74}$$

We see that the latent variables z_n as well as the unknown paramters, $\boldsymbol{\theta} = (\boldsymbol{\pi}, \boldsymbol{\mu}_{1:K}, \boldsymbol{\Sigma}_{1:K})$, are all shown as unshaded nodes.

3.6 Exercises

Exercise 3.1 [Uncorrelated does not imply independent]

Let $X \sim U(-1, 1)$ and $Y = X^2$. Clearly Y is dependent on X (in fact, Y is uniquely determined by X). However, show that $\rho(X, Y) = 0$. Hint: if $X \sim U(a, b)$ then $E[X] = (a + b)/2$ and $\mathbb{V}[X] = (b - a)^2/12$.

Exercise 3.2 [Correlation coefficient is between -1 and +1]

Prove that $-1 \leq \rho(X, Y) \leq 1$

Exercise 3.3 [Correlation coefficient for linearly related variables is ± 1]

Show that, if $Y = aX + b$ for some parameters $a > 0$ and b, then $\rho(X, Y) = 1$. Similarly show that if $a < 0$, then $\rho(X, Y) = -1$.

Exercise 3.4 [Linear combinations of random variables]

Let $\mathbf{x}$ be a random vector with mean $\mathbf{m}$ and covariance matrix Σ. Let $\mathbf{A}$ and $\mathbf{B}$ be matrices.

a. Derive the covariance matrix of $\mathbf{Ax}$.
b. Show that $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$.
c. Derive an expression for $\mathbb{E}\left[\mathbf{x}^T \mathbf{Ax}\right]$.

Exercise 3.5 [Gaussian vs *jointly* Gaussian]

Let $X \sim \mathcal{N}(0, 1)$ and $Y = WX$, where $p(W = -1) = p(W = 1) = 0.5$. It is clear that X and Y are not independent, since Y is a function of X.

a. Show $Y \sim \mathcal{N}(0, 1)$.
b. Show $\text{Cov}[X, Y] = 0$. Thus X and Y are uncorrelated but dependent, even though they are Gaussian. Hint: use the definition of covariance

$$\text{Cov}[X, Y] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] \tag{3.75}$$

and the **rule of iterated expectation**

$$\mathbb{E}[XY] = \mathbb{E}[\mathbb{E}[XY|W]] \tag{3.76}$$

Exercise 3.6 [Normalization constant for a multidimensional Gaussian]

Prove that the normalization constant for a d-dimensional Gaussian is given by

$$(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{\frac{1}{2}} = \int \exp(-\frac{1}{2}(\mathbf{x} - \mu)^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}))d\mathbf{x} \tag{3.77}$$

Hint: diagonalize $\boldsymbol{\Sigma}$ and use the fact that $|\boldsymbol{\Sigma}| = \prod_i \lambda_i$ to write the joint pdf as a product of d one-dimensional Gaussians in a transformed coordinate system. (You will need the change of variables formula.) Finally, use the normalization constant for univariate Gaussians.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 3.7 [Bivariate Gaussian]

Derive an explicit expression for the pdf of a bivariate Gaussian.

Exercise 3.8 [Conditioning a bivariate Gaussian]

Consider a bivariate Gaussian distribution $p(x_1, x_2)$.

a. What is $p(x_2|x_1)$? Simplify your answer by expressing it in terms of ρ, σ_2, σ_1, μ_1,μ_2 and x_1.

b. Assume $\sigma_1 = \sigma_2 = 1$. What is $p(x_2|x_1)$ now?

Exercise 3.9 [Sensor fusion with known variances in 1d]

Suppose we have two sensors with known (and different) variances v_1 and v_2, but unknown (and the same) mean μ. Suppose we observe n_1 observations $y_i^{(1)} \sim \mathcal{N}(\mu, v_1)$ from the first sensor and n_2 observations $y_i^{(2)} \sim \mathcal{N}(\mu, v_2)$ from the second sensor. (For example, suppose μ is the true temperature outside, and sensor 1 is a precise (low variance) digital thermosensing device, and sensor 2 is an imprecise (high variance) mercury thermometer.) Let $\mathcal{D}$ represent all the data from both sensors. What is the posterior $p(\mu|\mathcal{D})$, assuming a non-informative prior for μ (which we can simulate using a Gaussian with a precision of 0)? Give an explicit expression for the posterior mean and variance.

Exercise 3.10 [Show that the Student distribution can be written as a Gaussian scale mixture]

Show that a Student distribution is a GSM where we use a Gamma mixing distribution on the precision α (or equivalently, an inverse Gamma on the variance).

Exercise 3.11 [Derivation of the conditionals of an MVN]

Consider a joint Gaussian of the form $p(\mathbf{x}_1, \mathbf{x}_2) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \tag{3.78}$$

By using Schur complenents, show that

$$p(\mathbf{x}_1|\mathbf{x}_2) = \mathcal{N}(\mathbf{x}_1|\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2),\ \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}) \tag{3.79}$$

Exercise 3.12 [Information form formulae for marginalizing and conditioning an MVN]

Derive the equations for the marginal and conditional of an MVN in information form, i.e., show that

$$p(\mathbf{x}_2) = \mathcal{N}_c(\mathbf{x}_2|\boldsymbol{\xi}_2 - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\xi}_1, \boldsymbol{\Lambda}_{22} - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\Lambda}_{12}) \tag{3.80}$$

$$p(\mathbf{x}_1|\mathbf{x}_2) = \mathcal{N}_c(\mathbf{x}_1|\boldsymbol{\xi}_1 - \boldsymbol{\Lambda}_{12}\mathbf{x}_2, \boldsymbol{\Lambda}_{11}) \tag{3.81}$$

4 Statistics

4.1 Introduction

In Chapter 2–Chapter 3, we assumed all the parameters $\boldsymbol{\theta}$ of our probability models were known. In this chapter, we discuss how to learn these parameters from data.

The process of estimating $\boldsymbol{\theta}$ from $\mathcal{D}$ is called **model fitting**, or **training**, and is at the heart of machine learning. There are many methods for producing such estimates, but most boil down to an optimization problem of the form

$$\hat{\boldsymbol{\theta}} = \operatorname*{argmin}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) \tag{4.1}$$

where $\mathcal{L}(\boldsymbol{\theta})$ is some kind of loss function or objective function. We discuss several different loss functions in this chapter. In some cases, we also discuss how to solve the optimization problem in closed form. In general, however, we will need to use some kind of generic optimization algorithm, which we discuss in Chapter 8.

In addition to computing a **point estimate**, $\hat{\boldsymbol{\theta}}$, we discuss how to model our uncertainty or confidence in this estimate. In statistics, the process of quantifying uncertainty about an unknown quantity estimated from a finite sample of data is called **inference**. We will discuss both Bayesian and frequentist approaches to inference.[1]

4.2 Maximum likelihood estimation (MLE)

The most common approach to parameter estimation is to pick the parameters that assign the highest probability to the training data; this is called **maximum likelihood estimation** or **MLE**. We give more details below, and then give a series of worked examples.

4.2.1 Definition

We define the MLE as follows:

$$\hat{\boldsymbol{\theta}}_{\text{mle}} \triangleq \operatorname*{argmax}_{\boldsymbol{\theta}} p(\mathcal{D}|\boldsymbol{\theta}) \tag{4.2}$$

1. In the deep learning community, the term "inference" refers to what we will call "prediction", namely computing $p(y|\mathbf{x}, \hat{\boldsymbol{\theta}})$.

We usually assume the training examples are independently sampled from the same distribution, so the (conditional) likelihood becomes

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta}) \tag{4.3}$$

This is known as the **iid** assumption, which stands for "independent and identically distributed". We usually work with the **log likelihood**, which is given by

$$\text{LL}(\boldsymbol{\theta}) \triangleq \log p(\mathcal{D}|\boldsymbol{\theta}) = \sum_{n=1}^{N} \log p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta}) \tag{4.4}$$

This decomposes into a sum of terms, one per example. Thus the MLE is given by

$$\hat{\boldsymbol{\theta}}_{\text{mle}} = \underset{\boldsymbol{\theta}}{\text{argmax}} \sum_{n=1}^{N} \log p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta}) \tag{4.5}$$

Since most optimization algorithms (such as those discussed in Chapter 8) are designed to *minimize* cost functions, we can redefine the **objective function** to be the (conditional) **negative log likelihood** or **NLL**:

$$\text{NLL}(\boldsymbol{\theta}) \triangleq -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta}) \tag{4.6}$$

Minimizing this will give the MLE. If the model is unconditional (unsupervised), the MLE becomes

$$\hat{\boldsymbol{\theta}}_{\text{mle}} = \underset{\boldsymbol{\theta}}{\text{argmin}} -\sum_{n=1}^{N} \log p(\mathbf{y}_n|\boldsymbol{\theta}) \tag{4.7}$$

since we have outputs $\mathbf{y}_n$ but no inputs $\mathbf{x}_n$.[2]

Alternatively we may want to maximize the *joint* likelihood of inputs and outputs. The MLE in this case becomes

$$\hat{\boldsymbol{\theta}}_{\text{mle}} = \underset{\boldsymbol{\theta}}{\text{argmin}} -\sum_{n=1}^{N} \log p(\mathbf{y}_n, \mathbf{x}_n|\boldsymbol{\theta}) \tag{4.8}$$

4.2.2 Justification for MLE

There are several ways to justify the method of MLE. One way is to view it as simple point approximation to the Bayesian posterior $p(\boldsymbol{\theta}|\mathcal{D})$ using a uniform prior, as explained in Sec. 4.6.7.1. In

2. In statistics, it is standard to use $\mathbf{y}$ to represent variables whose generative distribution we choose to model, and to use $\mathbf{x}$ to represent exogoneous inputs which are given but not generated. Thus supervised learning concerns fitting conditional models of the form $p(\mathbf{y}|\mathbf{x})$, and unsupervised learning is the special case where $\mathbf{x} = \emptyset$, so we are just fitting the unconditional distribution $p(\mathbf{y})$. In the ML literature, supervised learning treats $\mathbf{y}$ as generated and $\mathbf{x}$ as given, but in the unsupervised case, it often switches to using $\mathbf{x}$ to represent generated variables.

particular, suppose we approximate the posterior by a delta function, $p(\boldsymbol{\theta}|\mathcal{D}) = \delta(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_{\text{map}})$, where $\hat{\boldsymbol{\theta}}_{\text{map}}$ is the posterior mode, given by

$$\hat{\boldsymbol{\theta}}_{\text{map}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log p(\boldsymbol{\theta}|\mathcal{D}) = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) \tag{4.9}$$

If we use a uniform prior, $p(\boldsymbol{\theta}) \propto 1$, the MAP estimate becomes equal to the MLE, $\hat{\boldsymbol{\theta}}_{\text{map}} = \hat{\boldsymbol{\theta}}_{\text{mle}}$.

Another way to justify the use of the MLE is that the resulting predictive distribution $p(\mathbf{y}|\hat{\boldsymbol{\theta}}_{\text{mle}})$ is as close as possible (in a sense to be defined below) to the **empirical distribution** of the data. In the unconditional case, the empirical distribution is defined by

$$p_D(\mathbf{y}) \triangleq \frac{1}{N} \sum_{n=1}^{N} \delta(\mathbf{y} - \mathbf{y}_n) \tag{4.10}$$

We see that the empirical distribution is a series of delta functions or "spikes" at the observed training points. We want to create a model whose distribution $q(\mathbf{y}) = p(\mathbf{y}|\boldsymbol{\theta})$ is similar to $p_D(\mathbf{y})$.

A standard way to measure the (dis)similarity between probability distributions p and q is the **Kullback Leibler divergence**, or **KL divergence**. We give the details in Sec. 6.2, but in brief this is defined as

$$\mathbb{KL}\left(p \| q\right) = \sum_{\mathbf{y}} p(\mathbf{y}) \log \frac{p(\mathbf{y})}{q(\mathbf{y})} \tag{4.11}$$

$$= \underbrace{\sum_{\mathbf{y}} p(\mathbf{y}) \log p(\mathbf{y})}_{-\mathbb{H}(p)} \underbrace{- \sum_{\mathbf{y}} p(\mathbf{y}) \log q(\mathbf{y})}_{\mathbb{H}(p,q)} \tag{4.12}$$

where $\mathbb{H}\left(p\right)$ is the entropy of p (see Sec. 6.1), and $\mathbb{H}\left(p, q\right)$ is the cross-entropy of p and q (see Sec. 6.1.2). One can show that $\mathbb{KL}\left(p \| q\right) \geq 0$, with equality iff $p = q$.

If we define $q(\mathbf{y}) = p(\mathbf{y}|\boldsymbol{\theta})$, and set $p(\mathbf{y}) = p_D(\mathbf{y})$, then the KL divergence becomes

$$\mathbb{KL}\left(p \| q\right) = \sum_{\mathbf{y}} p_D(\mathbf{y}) \log p_D(\mathbf{y}) - p_D(\mathbf{y}) \log q(\mathbf{y}) \tag{4.13}$$

$$= -\mathbb{H}(p_D) - \frac{1}{N} \sum_{n=1}^{N} \log p(\mathbf{y}_n|\boldsymbol{\theta}) \tag{4.14}$$

$$= \text{const} + \text{NLL}(\boldsymbol{\theta}) \tag{4.15}$$

The first term is a constant which we can ignore, leaving just the NLL. Thus minimizing the KL is equivalent to minimizing the NLL which is equivalent to computing the MLE, as in Eq. (4.7).

We can generalize the above results to the supervised (conditional) setting by using the following empirical distribution:

$$p_D(\mathbf{x}, \mathbf{y}) = p_D(\mathbf{y}|\mathbf{x}) p_D(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^{N} \delta(\mathbf{x} - \mathbf{x}_n) \delta(\mathbf{y} - \mathbf{y}_n) \tag{4.16}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The expected KL then becomes

$$\mathbb{E}_{p_D(\mathbf{x})}\left[\mathbb{KL}\left(p_D(Y|\mathbf{x})\|q(Y|\mathbf{x})\right)\right] = \mathbb{E}_{p_D(\mathbf{x})}\left[-\mathbb{H}(p_D(Y|\mathbf{x}))\right] - \mathbb{E}_{p_D(\mathbf{x})}\left[\frac{1}{N}\sum_{n=1}^{N}\log p(\mathbf{y}_n|\mathbf{x},\boldsymbol{\theta})\right] \tag{4.17}$$

$$= \text{const} - \frac{1}{N}\sum_{n=1}^{N}\log p(\mathbf{y}_n|\mathbf{x}_n,\boldsymbol{\theta}) \tag{4.18}$$

Minimizing this is equivalent to minimizing the conditional NLL in Eq. (4.6).

4.2.3 Example: MLE for the Bernoulli distribution

Suppose Y is a random variable representing a coin toss, where the event $Y = 1$ corresponds to heads and $Y = 0$ corresponds to tails. Let $\theta = p(Y = 1)$ be the probability of heads. The probability distribution for this rv is the Bernoulli, which we introduced in Sec. 2.4.

The NLL for the Bernoulli distribution is given by

$$\text{NLL}(\theta) = -\log\prod_{n=1}^{N} p(y_n|\theta) \tag{4.19}$$

$$= -\log\prod_{n=1}^{N}\theta^{\mathbb{I}(y_n=1)}(1-\theta)^{\mathbb{I}(y_n=0)} \tag{4.20}$$

$$= -\sum_{n=1}^{N}\mathbb{I}\left(y_n=1\right)\log\theta + \mathbb{I}\left(y_n=0\right)\log(1-\theta) \tag{4.21}$$

$$= -[N_1\log\theta + N_0\log(1-\theta)] \tag{4.22}$$

where we have defined $N_1 = \sum_{n=1}^{N}\mathbb{I}\left(y_n=1\right)$ and $N_0 = \sum_{n=1}^{N}\mathbb{I}\left(y_n=0\right)$, representing the number of heads and tails. (The NLL for the binomial is the same as for the Bernoulli, modulo an irrevant $\binom{N}{c}$ term, which is a constant independent of θ.) These two numbers are called the **sufficient statistics** of the data, since they summarize everything we need to know about $\mathcal{D}$. The total count, $N = N_0 + N_1$, is called the **sample size**.

The MLE can be found by solving $\frac{d}{d\theta}\text{NLL}(\theta) = 0$. The derivative of the NLL is

$$\frac{d}{d\theta}\text{NLL}(\theta) = \frac{-N_1}{\theta} + \frac{N_0}{1-\theta} \tag{4.23}$$

and hence the MLE is given by

$$\hat{\theta}_{\text{mle}} = \frac{N_1}{N_0+N_1} \tag{4.24}$$

We see that this is just the empirical fraction of heads, which is an intuitive result.

4.2.4 Example: MLE for the categorical distribution

Suppose we roll a K-sided dice N times. Let $Y_n \in \{1, \ldots, K\}$ be the n'th outcome, where $Y_n \sim \text{Cat}(\boldsymbol{\theta})$. We want to estimate the probabilities $\boldsymbol{\theta}$ from the dataset $\mathcal{D} = \{y_n : n = 1 : N\}$. The NLL is given by

$$\text{NLL}(\boldsymbol{\theta}) = -\sum_k N_k \log \theta_k \tag{4.25}$$

where N_k is the number of times the event $Y = k$ is observed. (The NLL for the multinomial is the same, up to irrelevant scale factors.)

To compute the MLE, we have to minimize the NLL subject to the constraint that $\sum_{k=1}^K \theta_k = 1$. To do this, we will use the method of Lagrange multipliers (see Sec. 8.5.1).[3]

The Lagrangian is as follows:

$$\mathcal{L}(\boldsymbol{\theta}, \lambda) \triangleq -\sum_k N_k \log \theta_k - \lambda \left(1 - \sum_k \theta_k\right) \tag{4.26}$$

Taking derivatives with respect to λ yields the original constraint:

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 1 - \sum_k \theta_k = 0 \tag{4.27}$$

Taking derivatives with respect to θ_k yields

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = -\frac{N_k}{\theta_k} + \lambda = 0 \implies N_k = \lambda \theta_k \tag{4.28}$$

We can solve for λ using the sum-to-one constraint:

$$\sum_k N_k = N = \lambda \sum_k \theta_k = \lambda \tag{4.29}$$

Thus the MLE is given by

$$\hat{\theta}_k = \frac{N_k}{\lambda} = \frac{N_k}{N} \tag{4.30}$$

which is just the empirical fraction of times event k occurs.

4.2.5 Example: MLE for the univariate Gaussian

Suppose $Y \sim \mathcal{N}(\mu, \sigma^2)$ and let $\mathcal{D} = \{y_n : n = 1 : N\}$ be an iid sample of size N. We can estimate the parameters $\boldsymbol{\theta} = (\mu, \sigma^2)$ using MLE as follows. First, we derive the NLL, which is given by

$$\text{NLL}(\mu, \sigma^2) = -\sum_{n=1}^N \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \mu)^2 \right) \right] \tag{4.31}$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \mu)^2 + \frac{N}{2} \log(2\pi\sigma^2) \tag{4.32}$$

3. We do not need to explicitly enforce the constraint that $\theta_k \geq 0$ since the gradient of the Lagrangian has the form $-N_k/\theta_k - \lambda$; so negative values of θ_k would increase the objective, rather than minimize it. (Of course, this does not preclude setting $\theta_k = 0$, and indeed this is the optimal solution if $N_k = 0$.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The minimum of this function must satisfy the following conditions, which we explain in Sec. 8.1.1.1:

$$\frac{\partial}{\partial \mu}\text{NLL}(\mu, \sigma^2) = 0, \;\; \frac{\partial}{\partial \sigma^2}\text{NLL}(\mu, \sigma^2) = 0 \tag{4.33}$$

So all we have to do is to find this stationary point. Some simple calculus (Exercise 4.1) shows that the solution is given by the following:

$$\hat{\mu}_{\text{mle}} = \frac{1}{N}\sum_{n=1}^{N} y_n = \overline{y} \tag{4.34}$$

$$\hat{\sigma}^2_{\text{mle}} = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{\mu}_{\text{mle}})^2 = \frac{1}{N}\left[\sum_{n=1}^{N} y_n^2 + \hat{\mu}^2_{\text{mle}} - 2y_n\hat{\mu}_{\text{mle}}\right] = s^2 - \overline{y}^2 \tag{4.35}$$

$$s^2 \triangleq \frac{1}{N}\sum_{n=1}^{N} y_n^2 \tag{4.36}$$

The quantities $\overline{y}$ and s^2 are called the **sufficient statistics** of the data, since they are sufficient to compute the MLE, without loss of information relative to using the raw data itself.

Note that you might be used to seeing the estimate for the variance written as

$$\hat{\sigma}^2_{\text{unb}} = \frac{1}{N-1}\sum_{n=1}^{N}(y_n - \hat{\mu}_{\text{mle}})^2 \tag{4.37}$$

where we divide by $N-1$. This is not the MLE, but is a different kind of estimate, which happens to be unbiased (unlike the MLE); see Sec. 4.7.6.1 for details.

4.2.6 Example: MLE for the multivariate Gaussian

In this section, we derive the maximum likelihood estimate for the parameters of a multivariate Gaussian.

First, let us write the log-likelihood, dropping irrelevant constants:

$$\text{LL}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \log p(\mathcal{D}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{N}{2}\log|\boldsymbol{\Lambda}| - \frac{1}{2}\sum_{n=1}^{N}(\mathbf{y}_n - \boldsymbol{\mu})^\top \boldsymbol{\Lambda}(\mathbf{y}_n - \boldsymbol{\mu}) \tag{4.38}$$

where $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ is the **precision matrix** (inverse covariance matrix).

4.2.6.1 MLE for the mean

Using the substitution $\mathbf{z}_n = \mathbf{y}_n - \boldsymbol{\mu}$, the derivative of a quadratic form (Eq. (7.249)) and the chain rule of calculus, we have

$$\frac{\partial}{\partial \boldsymbol{\mu}}(\mathbf{y}_n - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{y}_n - \boldsymbol{\mu}) = \frac{\partial}{\partial \mathbf{z}_n}\mathbf{z}_n^\top \boldsymbol{\Sigma}^{-1}\mathbf{z}_n \frac{\partial \mathbf{z}_n}{\partial \boldsymbol{\mu}^\top} \tag{4.39}$$

$$= -1(\boldsymbol{\Sigma}^{-1} + \boldsymbol{\Sigma}^{-T})\mathbf{z}_n \tag{4.40}$$

since $\frac{\partial \mathbf{z}_n}{\partial \boldsymbol{\mu}^\top} = -\mathbf{I}$. Hence

$$\frac{\partial}{\partial \boldsymbol{\mu}} \text{LL}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{1}{2} \sum_{n=1}^{N} -2\boldsymbol{\Sigma}^{-1}(\mathbf{y}_n - \boldsymbol{\mu}) = \boldsymbol{\Sigma}^{-1} \sum_{n=1}^{N} (\mathbf{y}_n - \boldsymbol{\mu}) = 0 \tag{4.41}$$

$$\hat{\boldsymbol{\mu}} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{y}_n = \overline{\mathbf{y}} \tag{4.42}$$

So the MLE of $\boldsymbol{\mu}$ is just the empirical mean.

4.2.6.2 MLE for the covariance matrix

We can use the trace trick (Eq. (7.36)) to rewrite the log-likelihood in terms of the precision matrix $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ as follows:

$$\text{LL}(\hat{\boldsymbol{\mu}}, \boldsymbol{\Lambda}) = \frac{N}{2} \log |\boldsymbol{\Lambda}| - \frac{1}{2} \sum_n \text{tr}[(\mathbf{y}_n - \hat{\boldsymbol{\mu}})(\mathbf{y}_n - \hat{\boldsymbol{\mu}})^\top \boldsymbol{\Lambda}] \tag{4.43}$$

$$= \frac{N}{2} \log |\boldsymbol{\Lambda}| - \frac{1}{2} \text{tr}\left[\mathbf{S}_{\overline{\mathbf{y}}} \boldsymbol{\Lambda}\right] \tag{4.44}$$

$$\mathbf{S}_{\overline{\mathbf{y}}} \triangleq \sum_{n=1}^{N} (\mathbf{y}_n - \overline{\mathbf{y}})(\mathbf{y}_n - \overline{\mathbf{y}})^\top = \left(\sum_n \mathbf{y}_n \mathbf{y}_n^\top \right) - N \overline{\mathbf{y}} \overline{\mathbf{y}}^\top \tag{4.45}$$

where $\mathbf{S}_{\overline{\mathbf{y}}}$ is the **scatter matrix** centered on $\overline{\mathbf{y}}$.

We can rewrite the scatter matrix in a more compact form as follows:

$$\mathbf{S}_{\overline{\mathbf{y}}} = \tilde{\mathbf{Y}}^\top \tilde{\mathbf{Y}} = \mathbf{Y}^\top \mathbf{C}_N^\top \mathbf{C}_N \mathbf{Y} = \mathbf{Y}^\top \mathbf{C}_N \mathbf{Y} \tag{4.46}$$

where

$$\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^\top \tag{4.47}$$

is the **centering matrix**, which converts $\mathbf{Y}$ to $\tilde{\mathbf{Y}}$ by subtracting the mean $\overline{\mathbf{y}} = \frac{1}{N} \mathbf{Y}^\top \mathbf{1}_N$ off every row.

Using results from Sec. 7.8, we can compute derivatives of the loss with respect to $\boldsymbol{\Lambda}$ to get

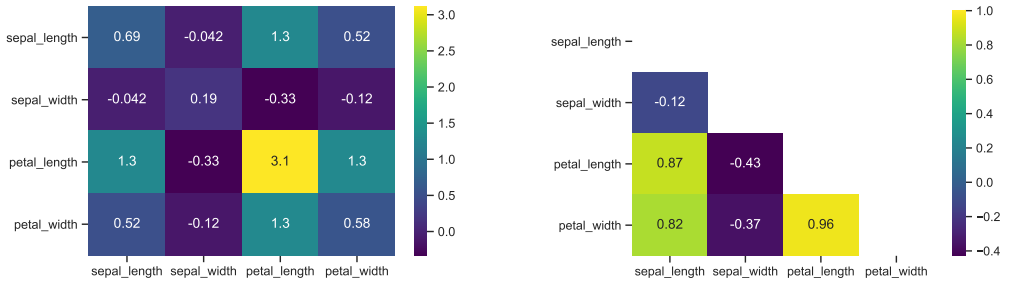
$$\frac{\partial \text{LL}(\hat{\boldsymbol{\mu}}, \boldsymbol{\Lambda})}{\partial \boldsymbol{\Lambda}} = \frac{N}{2} \boldsymbol{\Lambda}^{-T} - \frac{1}{2} \mathbf{S}_{\overline{\mathbf{y}}}^\top = \mathbf{0} \tag{4.48}$$

$$\boldsymbol{\Lambda}^{-\top} = \boldsymbol{\Lambda}^{-1} = \boldsymbol{\Sigma} = \frac{1}{N} \mathbf{S}_{\overline{\mathbf{y}}} \tag{4.49}$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{y}_n - \overline{\mathbf{y}})(\mathbf{y}_n - \overline{\mathbf{y}})^\top = \frac{1}{N} \mathbf{Y}^\top \mathbf{C}_N \mathbf{Y} \tag{4.50}$$

Thus the MLE for the covariance matrix is the empirical covariance matrix. See Fig. 4.1a for an example.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 4.1: (a) Covariance matrix for the features in the iris dataset from Sec. 1.2.1.1. (b) Correlation matrix. We only show the lower triangle, since the matrix is symmetric and has a unit diagonal. Compare this to Fig. 1.3. Generated by code at figures.probml.ai/book1/4.1.

Sometimes it is more convenient to work with the correlation matrix defined in Eq. (3.8). This can be computed using

$$\text{corr}(\mathbf{Y}) = (\text{diag}(\boldsymbol{\Sigma}))^{-\frac{1}{2}}\ \boldsymbol{\Sigma}\ (\text{diag}(\boldsymbol{\Sigma}))^{-\frac{1}{2}} \tag{4.51}$$

where $\text{diag}(\boldsymbol{\Sigma})^{-\frac{1}{2}}$ is a diagonal matrix containing the entries $1/\sigma_i$. See Fig. 4.1b for an example.

Note, however, that the MLE may overfit or be numerically unstable, especially when the number of samples N is small compared to the number of dimensions D. The main problem is that $\boldsymbol{\Sigma}$ has $O(D^2)$ parameters, so we may need a lot of data to reliably estimate it. In particular, as we see from Eq. (4.50), the MLE for a full covariance matrix is singular if $N < D$. And even when $N > D$, the MLE can be ill-conditioned, meaning it is close to singular. We discuss solutions to this problem in Sec. 4.5.2.

4.2.7 Example: MLE for linear regression

We briefly mentioned linear regression in Sec. 2.6.3. Recall that it corresponds to the following model:

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top\mathbf{x}, \sigma^2) \tag{4.52}$$

where $\boldsymbol{\theta} = (\mathbf{w}, \sigma^2)$. Let us assume for now that σ^2 is fixed, and focus on estimating the weights $\mathbf{w}$.

The negative log likelihood or NLL is given by

$$\text{NLL}(\mathbf{w}) = -\sum_{n=1}^{N} \log\left[\left(\frac{1}{2\pi\sigma^2}\right)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \mathbf{w}^\top\mathbf{x}_n)^2\right)\right] \tag{4.53}$$

Dropping the irrelevant additive constants gives the following simplified objective, known as the **residual sum of squares** or **RSS**:

$$\text{RSS}(\mathbf{w}) \triangleq \sum_{n=1}^{N}(y_n - \mathbf{w}^\top\mathbf{x}_n)^2 = \sum_{n=1}^{N} r_n^2 \tag{4.54}$$

where r_n the n'th **residual error**. Scaling by the number of examples N gives the **mean squared error** or **MSE**:

$$\text{MSE}(\mathbf{w}) = \frac{1}{N}\text{RSS}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2 \tag{4.55}$$

Finally, taking the square root gives the **root mean squared error** or **RMSE**:

$$\text{RMSE}(\mathbf{w}) = \sqrt{\text{MSE}(\mathbf{w})} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2} \tag{4.56}$$

We can compute the MLE by minimizing the NLL, RSS, MSE or RMSE. All will give the same results, since these objective functions are all the same, up to irrelevant constants

Let us focus on the RSS objective. It can be written in matrix notation as follows:

$$\text{RSS}(\mathbf{w}) = \sum_{n=1}^{N}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) \tag{4.57}$$

In Sec. 11.2.2.1, we prove that the optimum, which occurs where $\nabla_{\mathbf{w}}\text{RSS}(\mathbf{w}) = \mathbf{0}$, satisfies the following equation:

$$\hat{\mathbf{w}}_{\text{mle}} \triangleq \underset{\mathbf{w}}{\text{argmin}}\, \text{RSS}(\mathbf{w}) = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{4.58}$$

This is called the **ordinary least squares** or **OLS** estimate, and is equivalent to the MLE.

4.3 Empirical risk minimization (ERM)

We can generalize MLE by replacing the (conditional) log loss term in Eq. (4.6), $\ell(\mathbf{y}_n, \boldsymbol{\theta}; \mathbf{x}_n) = -\log p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta})$, with any other loss function, to get

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N}\ell(\mathbf{y}_n, \boldsymbol{\theta}; \mathbf{x}_n) \tag{4.59}$$

This is known as **empirical risk minimization** or **ERM**, since it is the expected loss where the expectation is taken wrt the empirical distribution. See Sec. 5.6 for more details.

4.3.1 Example: minimizing the misclassification rate

If we are solving a classification problem, we might want to use 0-1 loss:

$$\ell_{01}(\mathbf{y}_n, \boldsymbol{\theta}; \mathbf{x}_n) = \begin{cases} 0 & \text{if } \mathbf{y}_n = f(\mathbf{x}_n; \boldsymbol{\theta}) \\ 1 & \text{if } \mathbf{y}_n \neq f(\mathbf{x}_n; \boldsymbol{\theta}) \end{cases} \tag{4.60}$$

where $f(\mathbf{x}; \boldsymbol{\theta})$ is some kind of predictor. The empirical risk becomes

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N}\ell_{01}(\mathbf{y}_n, \boldsymbol{\theta}; \mathbf{x}_n) \tag{4.61}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is just the empirical **misclassification rate** on the training set.

Note that for binary problems, we can rewrite the misclassifcation rate in the following notation. Let $\tilde{y} \in \{-1, +1\}$ be the true label, and $\hat{y} \in \{-1, +1\} = f(\mathbf{x}; \boldsymbol{\theta})$ be our prediction. We define the 0-1 loss as follows:

$$\ell_{01}(\tilde{y}, \hat{y}) = \mathbb{I}(\tilde{y} \neq \hat{y}) = \mathbb{I}(\tilde{y}\, \hat{y} < 0) \tag{4.62}$$

The corresponding empirical risk becomes

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} \ell_{01}(y_n, \hat{y}_n) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(\tilde{y}_n\, \hat{y}_n < 0) \tag{4.63}$$

where the dependence on $\mathbf{x}_n$ and $\boldsymbol{\theta}$ is implicit.

4.3.2 Surrogate loss

Unfortunately, the 0-1 loss used in Sec. 4.3.1 is a non-smooth step function, as shown in Fig. 4.2, making it difficult to optimize. (In fact, it is NP-hard [BDEL03].) In this section we consider the use of a **surrogate loss function** [BJM06]. The surrogate is usually chosen to be a maximally tight convex upper bound, which is then easy to minimize.

For example, consider a probabilistic binary classifier, which produces the following distribution over labels:

$$p(\tilde{y}|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\tilde{y}\eta) = \frac{1}{1 + e^{-\tilde{y}\eta}} \tag{4.64}$$

where $\eta = f(\mathbf{x}; \boldsymbol{\theta})$ is the log odds. Hence the log loss is given by

$$\ell_{ll}(\tilde{y}, \eta) = -\log p(\tilde{y}|\eta) = \log(1 + e^{-\tilde{y}\eta}) \tag{4.65}$$

Fig. 4.2 shows that this is a smooth upper bound to the 0-1 loss, where we plot the loss vs the quantity $\tilde{y}\eta$, known as the **margin**, since it defines a "margin of safety" away from the threshold value of 0. Thus we see that minimizing the negative log likelihood is equivalent to minimizing a (fairly tight) upper bound on the empirical 0-1 loss.

Another convex upper bound to 0-1 loss is the **hinge loss**, which is defined as follows:

$$\ell_{\text{hinge}}(\tilde{y}, \eta) = \max(0, 1 - \tilde{y}\eta) \triangleq (1 - \tilde{y}\eta)_+ \tag{4.66}$$

This is plotted in Fig. 4.2; we see that it has the shape of a partially open door hinge. This is convex upper bound to the 0-1 loss, although it is only piecewise differentiable, not everywhere differentiable.

4.4 Other estimation methods *

4.4.1 The method of moments

Computing the MLE requires solving the equation $\nabla_{\boldsymbol{\theta}} \text{NLL}(\boldsymbol{\theta}) = \mathbf{0}$. Sometimes this is computationally difficult. In such cases, we may be able to use a simpler approach known as the **method of moments**

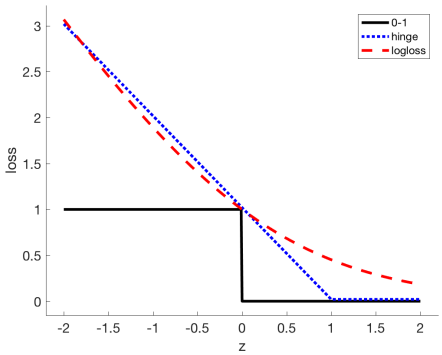


Figure 4.2: Illustration of various loss functions for binary classification. The horizontal axis is the margin $z = \tilde{y}\eta$, the vertical axis is the loss. The log loss uses log base 2. Generated by code at figures.probml.ai/book1/4.2.

(MOM). In this approach, we equate the theoretical moments of the distribution to the empirical moments, and solve the resulting set of K simultaneous equations, where K is the number of parameters. The theoretical moments are given by $\mu_k = \mathbb{E}\left[Y^k\right]$, for $k = 1 : K$, and the empirical moments are given by

$$\hat{\mu}_k = \frac{1}{N}\sum_{n=1}^{n} y_n^k \tag{4.67}$$

so we just need to solve $\mu_k = \hat{\mu}_k$ for each k. We give some examples below.

The method of moments is simple, but it is theoretically inferior to the MLE approach, since it may not use all the data as efficiently. (For details on these theoretical results, see e.g., [CB02].) Furthermore, it can sometimes produce inconsistent results (see Sec. 4.4.1.2). However, when it produces valid estimates, it can be used to initialize iterative algorithms that are used to optimize the NLL (see e.g., [AHK12]), thus combining the computational efficiency of MOM with the statistical accuracy of MLE.

4.4.1.1 Example: MOM for the univariate Gaussian

For example, consider the case of a univariate Gaussian distribution. From Sec. 4.2.5, we have

$$\mu_1 = \mu = \overline{y} \tag{4.68}$$

$$\mu_2 = \sigma^2 + \mu = s^2 \tag{4.69}$$

where $\overline{y}$ is the empirical mean and s^2 is the empirical average sum of squares. so $\hat{\mu} = \overline{y}$ and $\hat{\sigma}^2 = s^2 - \overline{y}^2$. In this case, the MOM estimate is the same as the MLE, but this is not always the case.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

4.4.1.2 Example: MOM for the uniform distribution

In this section, we give an example of the MOM applied to the uniform distribution. Our presentation follows the wikipedia page.[4] Let $Y \sim \text{Unif}(\theta_1, \theta_2)$ be a uniform random variable, so

$$p(y|\theta) = \frac{1}{\theta_2 - \theta_1} \mathbb{I}(\theta_1 \leq y \leq \theta_2) \tag{4.70}$$

The first two moments are

$$\mu_1 = \mathbb{E}[Y] = \frac{1}{2}(\theta_1 + \theta_2) \tag{4.71}$$

$$\mu_2 = \mathbb{E}\left[Y^2\right] = \frac{1}{3}(\theta_1^2 + \theta_1\theta_2 + \theta_2^2) \tag{4.72}$$

Inverting these equations gives

$$(\theta_1, \theta_2) = \left(\mu_1 - \sqrt{3(\mu^2 - \mu_1^2)}, 2\mu_1 - \theta_1\right) \tag{4.73}$$

Unfortunately this estimator can give sometimes give invalid results. For example, suppose $\mathcal{D} = \{0, 0, 0, 0, 1\}$. The empirical moments are $\hat{\mu}_1 = \frac{1}{5}$ and $\hat{\mu}_2 = \frac{1}{5}$, so the estimated parameters are $\hat{\theta}_1 = \frac{1}{5} - \frac{2\sqrt{3}}{5} = -0.493$ and $\hat{\theta}_2 = \frac{1}{5} + \frac{2\sqrt{3}}{5} = 0.893$. However, these cannot possibly be the correct parameters, since if $\theta_2 = 0.893$, we cannot generate a sample as large as 1.

By contrast, consider the MLE. Let $y_{(1)} \leq y_{(2)} \leq \cdots \leq y_{(N)}$ be the **order statistics** of the data (i.e., the values sorted in increasing order). Let $\theta = \theta_2 - \theta_1$. Then the likelihood is given by

$$p(\mathcal{D}|\boldsymbol{\theta}) = (\theta)^{-N} \mathbb{I}\left(y_{(1)} \geq \theta_1\right) \mathbb{I}\left(y_{(N)} \leq \theta_2\right) \tag{4.74}$$

Within the permitted bounds for θ, the derivative of the log likelihood is given by

$$\frac{d}{d\theta} \log p(\mathcal{D}|\theta) = -\frac{N}{\theta} < 0 \tag{4.75}$$

Hence the likelihood is a decreasing function of θ, so we should pick

$$\hat{\theta}_1 = y_{(1)}, \hat{\theta}_2 = y_{(N)} \tag{4.76}$$

In the above example, we get $\hat{\theta}_1 = 0$ and $\hat{\theta}_2 = 1$, as one would expect.

4.4.2 Online (recursive) estimation

If the entire dataset $\mathcal{D}$ is available before training starts, we say that we are doing **batch learning**. However, in some cases, the data set arrives sequentially, so $\mathcal{D} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots\}$ in an unbounded stream. In this case, we want to perform **online learning**.

Let $\hat{\boldsymbol{\theta}}_{t-1}$ be our estimate (e.g., MLE) given $\mathcal{D}_{1:t-1}$. To ensure our learning algorithm takes constant time per update, we need to find a learning rule of the form

$$\boldsymbol{\theta}_t = f(\hat{\boldsymbol{\theta}}_{t-1}, \mathbf{y}_t) \tag{4.77}$$

This is called a **recursive update**. Below we give some examples of such online learning methods.

4. https://en.wikipedia.org/wiki/Method_of_moments_(statistics).

4.4.2.1 Example: recursive MLE for the mean of a Gaussian

Let us reconsider the example from Sec. 4.2.5 where we computed the MLE for a univariate Gaussian. We know that the batch estimate for the mean is given by

$$\hat{\boldsymbol{\mu}}_t = \frac{1}{t}\sum_{n=1}^{t} \mathbf{y}_n \tag{4.78}$$

This is just a **running sum** of the data, so we can easily convert this into a recursive estimate as follows:

$$\hat{\boldsymbol{\mu}}_t = \frac{1}{t}\sum_{n=1}^{t} \mathbf{y}_n = \frac{1}{t}\left((t-1)\hat{\boldsymbol{\mu}}_{t-1} + \mathbf{y}_t\right) \tag{4.79}$$

$$= \hat{\boldsymbol{\mu}}_{t-1} + \frac{1}{t}(\mathbf{y}_t - \hat{\boldsymbol{\mu}}_{t-1}) \tag{4.80}$$

This is known as a **moving average**.

We see from Eq. (4.80) that the new estimate is the old estimate plus a correction term. The size of the correction diminishes over time (i.e., as we get more samples). However, if the distribution is changing, we want to give more weight to more recent data examples. We discuss how to do this in Sec. 4.4.2.2.

4.4.2.2 Exponentially-weighted moving average (EMA)

Eq. (4.80) shows how to compute the moving average of a signal. In this section, we show how to adjust this to give more weight to more recent examples. In particular, we will compute the following **exponentially weighted moving average** (**EWMA**):

$$\hat{\boldsymbol{\mu}}_t = \beta\boldsymbol{\mu}_{t-1} + (1-\beta)\mathbf{y}_t \tag{4.81}$$

where $0 \leq \beta \leq 1$. The contribution of a data point k steps in the past is weighted by $\beta^k(1-\beta)$. Thus the contribution from old data is exponentially decreasing. In particular, we have

$$\hat{\boldsymbol{\mu}}_t = \beta\boldsymbol{\mu}_{t-1} + (1-\beta)\mathbf{y}_t \tag{4.82}$$

$$= \beta^2\boldsymbol{\mu}_{t-2} + \beta(1-\beta)\mathbf{y}_{t-1} + (1-\beta)\mathbf{y}_t \quad \vdots \tag{4.83}$$

$$= (1-\beta)\beta^t\mathbf{y}_0 + (1-\beta)\beta^{t-1}\mathbf{y}_1 + \cdots + (1-\beta)\beta\mathbf{y}_{t-1} + (1-\beta)\beta^0\mathbf{y}_t \tag{4.84}$$

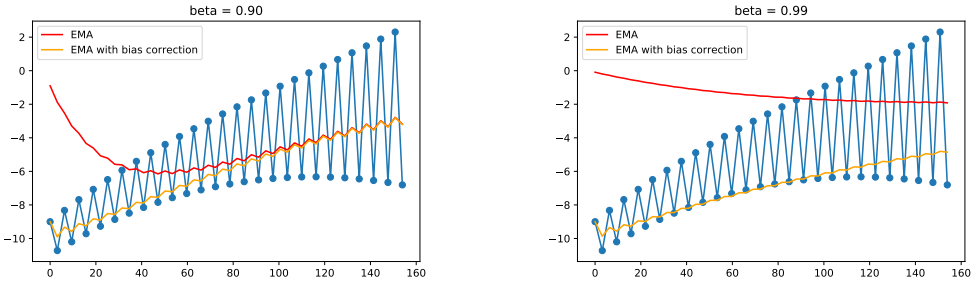
The sum of a **geometric series** is given by

$$\beta^t + \beta^{t-1} + \cdots + \beta^1 + \beta^0 = \frac{1-\beta^{t+1}}{1-\beta} \tag{4.85}$$

Hence

$$(1-\beta)\sum_{k=0}^{t}\beta^k = (1-\beta)\frac{1-\beta^{t+1}}{1-\beta} = 1-\beta^{t+1} \tag{4.86}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 4.3: Illustration of exponentially-weighted moving average with and without bias correction. (a) Short memory: $\beta = 0.9$. (a) Long memory: $\beta = 0.99$. Generated by code at figures.probml.ai/book1/4.3.

Since $\beta < 0$, we have $\beta^{t+1} \to 0$ as $t \to \infty$, so smaller β forgets the past more quickly, and adapts to the more recent data more rapidly. This is illustrated in Fig. 4.3.

Since the initial estimate starts from $\hat{\boldsymbol{\mu}}_0 = \mathbf{0}$, there is an initial bias. This can be corrected by scaling as follows [KB15]:

$$\tilde{\boldsymbol{\mu}}_t = \frac{\hat{\boldsymbol{\mu}}_t}{1 - \beta^t} \tag{4.87}$$

(Note that the update in Eq. (4.81) is still applied to the uncorrected EWMA, $\hat{\boldsymbol{\mu}}_{t-1}$, before being corrected for the current time step.) The benefit of this is illustrated in Fig. 4.3.

4.5 Regularization

A fundamental problem with MLE, and ERM, is that it will try to pick parameters that minimize loss on the training set, but this may not result in a model that has low loss on future data. This is called **overfitting**.

As a simple example, suppose we want to predict the probability of heads when tossing a coin. We toss it $N = 3$ times and observe 3 heads. The MLE is $\theta_{\text{mle}} = N_1/(N_0 + N + 1) = 3/(3 + 0) = 1$ (see Sec. 4.2.3). However, if we use $\text{Ber}(y|\theta_{mle})$ as our model, we will predict that all future coin tosses will also be heads, which seems rather unlikely.

The core of the problem is that the model has enough parameters to perfectly fit the observed training data, so it can perfectly match the empirical distribution. However, in most cases the empirical distribution is not the same as the true distribution, so putting all the probability mass on the observed set of N examples will not leave over any probability for novel data in the future. That is, the model may not **generalize**.

The main solution to overfitting is to use **regularization**, which means to add a penalty term to the NLL (or empirical risk). Thus we optimize an objective of the form

$$\mathcal{L}(\boldsymbol{\theta}; \lambda) = \left[\frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{y}_n, \boldsymbol{\theta}; \mathbf{x}_n) \right] + \lambda C(\boldsymbol{\theta}) \tag{4.88}$$

where $\lambda \geq 0$ is the **regularization parameter**, and $C(\boldsymbol{\theta})$ is some form of **complexity penalty**. A common way to define the complexity penalty is to use $C(\boldsymbol{\theta}) = \log p(\boldsymbol{\theta})$, where $p(\boldsymbol{\theta})$ is the **prior** for $\boldsymbol{\theta}$. If ℓ is the log loss, the regularized objective becomes

$$\mathcal{L}(\boldsymbol{\theta}; \lambda) = \left[\frac{1}{N} - \sum_{n=1}^{N} \log p(\mathbf{y}_n | \mathbf{x}_n, \boldsymbol{\theta})\right] + \lambda \log p(\boldsymbol{\theta}) \tag{4.89}$$

If we set λ appropriately, this becomes proportional to the negative log posterior (up to additive factors):

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta}|\mathcal{D}) + \text{const} \tag{4.90}$$

Minimizing this is equivalent to computing the **posterior mode**. This is known as **MAP estimation**, which stands for **maximum a posterior estimation**.

4.5.1 Example: MAP estimation for the Bernoulli distribution

Consider again the coin tossing example. If we observe just one head, the MLE is $\theta_{mle} = 1$. To avoid this, we can add a penalty to θ to discourage "extreme" values, such as $\theta = 0$ or $\theta = 1$. We can do this by using a beta distribution as our prior, $p(\theta) = \text{Beta}(\theta|a, b)$, where $a, b > 1$ encourages values of θ near to $a/(a + b)$ (see Sec. 2.7.4 for details). The log likelihood plus log prior becomes

$$\text{LL}(\theta) = \log p(\mathcal{D}|\theta) + \log p(\theta) \tag{4.91}$$

$$= [N_1 \log \theta + N_0 \log(1 - \theta)] + [(a - 1) \log(\theta) + (b - 1) \log(1 - \theta)] \tag{4.92}$$

Using the method from Sec. 4.2.3 we find that the MAP estimate is

$$\theta_{\text{map}} = \frac{N_1 + a - 1}{N_1 + N_0 + a + b - 2} \tag{4.93}$$

If we set $a = b = 2$ (which weakly favors a value of θ near 0.5), the estimate becomes

$$\theta_{\text{map}} = \frac{N_1 + 1}{N_1 + N_0 + 2} \tag{4.94}$$

This is called **add-one smoothing**, and is a simple but widely used technique to avoid the **zero count** problem. (See also Sec. 4.6.2.9.)

The zero-count problem, and overfitting more generally, is analogous to a problem in philosophy called the **black swan paradox**. This is based on the ancient Western conception that all swans were white. In that context, a black swan was a metaphor for something that could not exist. (Black swans were discovered in Australia by European explorers in the 17th Century.) The term "black swan paradox" was first coined by the famous philosopher of science Karl Popper; the term has also been used as the title of a recent popular book [Tal07]. This paradox was used to illustrate the problem of **induction**, which is the problem of how to draw general conclusions about the future from specific observations from the past. The solution to the paradox is to admit that induction is in general impossible, and that the best we can do is to make plausible guesses about what the future might hold, by combining the empirical data with prior knowledge.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

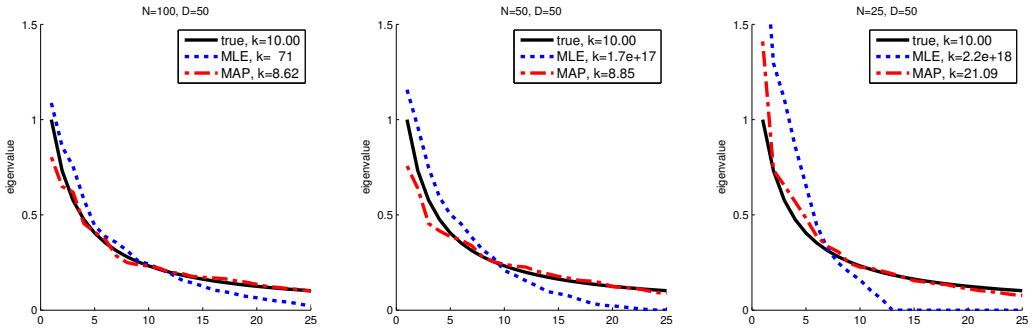


Figure 4.4: Estimating a covariance matrix in $D = 50$ dimensions using $N \in \{100, 50, 25\}$ samples. We plot the eigenvalues in descending order for the true covariance matrix (solid black), the MLE (dotted blue) and the MAP estimate (dashed red), using Eq. (4.96) with $\lambda = 0.9$. We also list the condition number of each matrix in the legend. We see that the MLE is often poorly conditioned, but the MAP estimate is numerically well behaved. Adapted from Figure 1 of [SS05]. Generated by code at figures.probml.ai/book1/4.4.

4.5.2 Example: MAP estimation for the multivariate Gaussian *

In Sec. 4.2.6, we showed that the MLE for the mean of an MVN is the empirical mean, $\hat{\boldsymbol{\mu}}_{\text{mle}} = \overline{\mathbf{y}}$. We also showed that the MLE for the covariance is the empirical covariance, $\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\mathbf{S}_{\overline{\mathbf{y}}}$.

In high dimensions the estimate for $\boldsymbol{\Sigma}$ can easily become singular. One solution to this is to perform MAP estimation, as we explain below.

4.5.2.1 Shrinkage estimate

A convenient prior to use for $\boldsymbol{\Sigma}$ is the inverse Wishart prior. This is a distribution over positive definite matrices, where the parameters are defined in terms of a prior scatter matrix, $\breve{\mathbf{S}}$, and a prior sample size or strength $\breve{N}$. One can show that the resulting MAP estimate is given by

$$\hat{\boldsymbol{\Sigma}}_{\text{map}} = \frac{\breve{\mathbf{S}} + \mathbf{S}_{\overline{\mathbf{y}}}}{\breve{N} + N} = \frac{\breve{N}}{\breve{N} + N}\frac{\breve{\mathbf{S}}}{\breve{N}} + \frac{N}{\breve{N} + N}\frac{\mathbf{S}_{\overline{\mathbf{y}}}}{N} = \lambda\boldsymbol{\Sigma}_0 + (1-\lambda)\hat{\boldsymbol{\Sigma}}_{\text{mle}} \tag{4.95}$$

where $\lambda = \frac{\breve{N}}{\breve{N}+N}$ controls the amount of regularization.

A common choice (see e.g., [FR07, p6]) for the prior scatter matrix is to use $\breve{\mathbf{S}} = \breve{N}\,\text{diag}(\hat{\boldsymbol{\Sigma}}_{\text{mle}})$. With this choice, we find that the MAP estimate for $\boldsymbol{\Sigma}$ is given by

$$\hat{\boldsymbol{\Sigma}}_{\text{map}}(i,j) = \begin{cases} \hat{\boldsymbol{\Sigma}}_{\text{mle}}(i,j) & \text{if } i = j \\ (1-\lambda)\hat{\boldsymbol{\Sigma}}_{\text{mle}}(i,j) & \text{otherwise} \end{cases} \tag{4.96}$$

Thus we see that the diagonal entries are equal to their ML estimates, and the off-diagonal elements are "shrunk" somewhat towards 0. This technique is therefore called **shrinkage estimation**.

The other parameter we need to set is λ, which controls the amount of regularization (shrinkage towards the MLE). It is common to set λ by cross validation (Sec. 4.5.5). Alternatively, we can use the closed-form formula provided in [LW04a; LW04b; SS05], which is the optimal

frequentist estimate if we use squared loss. This is implemented in the sklearn function https://scikit-learn.org/stable/modules/generated/sklearn.covariance.LedoitWolf.html.

The benefits of this approach are illustrated in Fig. 4.4. We consider fitting a 50-dimensional Gaussian to $N = 100$, $N = 50$ and $N = 25$ data points. We see that the MAP estimate is always well-conditioned, unlike the MLE (see Sec. 7.1.4.4 for a discussion of condition numbers). In particular, we see that the eigenvalue spectrum of the MAP estimate is much closer to that of the true matrix than the MLE's spectrum. The eigenvectors, however, are unaffected.

4.5.2.2 Inverting the MAP estimate

We have seen how to compute a numerically stable estimate of the covariance matrix using $\hat{\mathbf{\Sigma}}_{\text{map}}$. However, we often need to compute the precision matrix $\hat{\mathbf{\Sigma}}_{\text{map}}^{-1}$. Since $\hat{\mathbf{\Sigma}}_{\text{map}}$ is just a scaled version of $\hat{\mathbf{\Sigma}}_{\text{mle}}$, when we try to invert $\hat{\mathbf{\Sigma}}_{\text{map}}$, we will encounter numerical problems whenever $D > N$. Fortunately we can use the SVD of $\mathbf{Y}$ to get around this, as we show below. (See Sec. 7.5 for details on SVD.)

Let $\mathbf{Y} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ be the SVD of the design matrix, where $\mathbf{V}$ is $D \times N$, $\mathbf{U}$ is an $N \times N$ orthogonal matrix, and $\mathbf{D}$ is a diagonal matrix of size $N \times N$. Furthermore, define the $N \times N$ matrix $\mathbf{Z} = \mathbf{U}\mathbf{D}$; this is like a design matrix in a lower dimensional space (since we assume $N < D$). Also, define $\boldsymbol{\mu}_z = \mathbf{V}^\top \hat{\boldsymbol{\mu}}$ as the mean of the data in this reduced space, where $\hat{\boldsymbol{\mu}} = \overline{\mathbf{y}}$. With these definitions, we can rewrite the MLE as follows:

$$\hat{\mathbf{\Sigma}}_{\text{mle}} = \frac{1}{N}\mathbf{Y}^\top\mathbf{Y} - \overline{\mathbf{y}}\,\overline{\mathbf{y}}^\top \tag{4.97}$$

$$= \frac{1}{N}(\mathbf{Z}\mathbf{V}^\top)^\top(\mathbf{Z}\mathbf{V}^\top) - (\mathbf{V}\boldsymbol{\mu}_z)(\mathbf{V}\boldsymbol{\mu}_z)^\top \tag{4.98}$$

$$= \frac{1}{N}\mathbf{V}\mathbf{Z}^\top\mathbf{Z}\mathbf{V}^\top - \mathbf{V}\boldsymbol{\mu}_z\boldsymbol{\mu}_z^\top\mathbf{V}^\top \tag{4.99}$$

$$= \mathbf{V}(\frac{1}{N}\mathbf{Z}^\top\mathbf{Z} - \boldsymbol{\mu}_z\boldsymbol{\mu}_z^\top)\mathbf{V}^\top \tag{4.100}$$

$$= \mathbf{V}\hat{\mathbf{\Sigma}}_z\mathbf{V}^\top \tag{4.101}$$

where $\hat{\mathbf{\Sigma}}_z$ is the empirical covariance of $\mathbf{Z}$. Hence we can rewrite the MAP estimate as

$$\hat{\mathbf{\Sigma}}_{\text{map}} = \mathbf{V}\mathbf{\Omega}_z\mathbf{V}^\top \tag{4.102}$$

where

$$\mathbf{\Omega}_z \triangleq \lambda\text{diag}(\hat{\mathbf{\Sigma}}_z) + (1-\lambda)\hat{\mathbf{\Sigma}}_z \tag{4.103}$$

This is an invertible matrix, unlike $\hat{\mathbf{\Sigma}}_{\text{mle}}$.

4.5.3 Example: weight decay

In Fig. 1.7, we saw how using polynomial regression with too high of a degree can result in overfitting. One solution is to reduce the degree of the polynomial. However, a more general solution is to

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

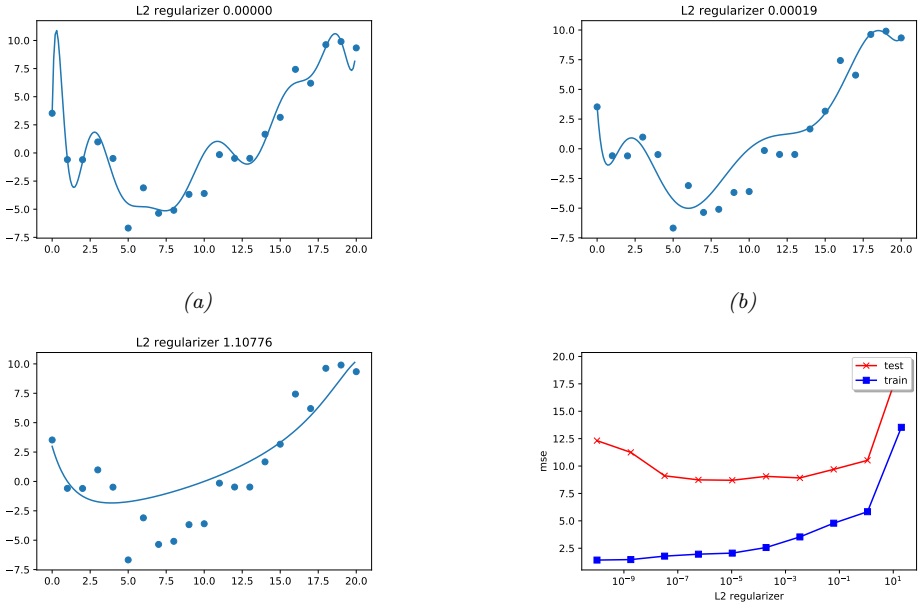


Figure 4.5: (a-c) Ridge regression applied to a degree 14 polynomial fit to 21 datapoints. (d) MSE vs strength of regularizer. The degree of regularization increases from left to right, so model complexity decreases from left to right. Generated by code at figures.probml.ai/book1/4.5.

penalize the magnitude of the weights (regression coefficients). We can do this by using a zero-mean Gaussian prior, $p(\mathbf{w})$. The resulting MAP estimate is given by

$$\hat{\mathbf{w}}_{\text{map}} = \underset{\mathbf{w}}{\text{argmin}}\, \text{NLL}(\mathbf{w}) + \lambda ||\mathbf{w}||_2^2 \tag{4.104}$$

where $||\mathbf{w}||_2^2 = \sum_{d=1}^{D} w_d^2$. (We write $\mathbf{w}$ rather than $\boldsymbol{\theta}$, since it only really make sense to penalize the magnitude of weight vectors, rather than other parameters, such as bias terms or noise variances.)

Eq. (4.104) is called ℓ_2 **regularization** or **weight decay**. The larger the value of λ, the more the parameters are penalized for being "large" (deviating from the zero-mean prior), and thus the less flexible the model.

In the case of linear regression, this kind of penalization scheme is called **ridge regression**. For example, consider the polynomial regression example from Sec. 1.2.2.2, where the predictor has the form

$$f(x; \mathbf{w}) = \sum_{d=0}^{D} w_d x^d = \mathbf{w}^\top [1, x, x^2, \ldots, x^D] \tag{4.105}$$

Suppose we use a high degree polynomial, say $D = 14$, even though we have a small dataset with just $N = 21$ examples. MLE for the parameters will enable the model to fit the data very well, by

carefully adjusting the weights, but the resulting function is very "wiggly", thus resulting in overfitting. Fig. 4.5 illustrates how increasing λ can reduce overfitting. For more details on ridge regression, see Sec. 11.3.

4.5.4 Picking the regularizer using a validation set

A key question when using regularization is how to choose the strength of the regularizer λ: a small value means we will focus on minimizing empirical risk, which may result in overfitting, whereas a large value means we will focus on staying close to the prior, which may result in **underfitting**.

In this section, we describe a simple but very widely used method for choosing λ. The basic idea is to partition the data into two disjoint sets, the training set $\mathcal{D}_{\text{train}}$ and a **validation set** $\mathcal{D}_{\text{valid}}$ (also called a **development set**). (Often we use about 80% of the data for the training set, and 20% for the validation set.) We fit the model on $\mathcal{D}_{\text{train}}$ (for each setting of λ) and then evaluate its performance on $\mathcal{D}_{\text{valid}}$. We then pick the value of λ that results in the best validation performance. (This optimization method is a 1d example of grid search, discussed in Sec. 8.8.1.)

To explain the method in more detail, we need some notation. Let us define the regularized empirical risk on a dataset as follows:

$$R_\lambda(\mathcal{D}, \boldsymbol{\theta}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x},\mathbf{y}) \in \mathcal{D}} \ell(\mathbf{y}, f(\mathbf{x}; \boldsymbol{\theta})) + \lambda C(\boldsymbol{\theta}) \tag{4.106}$$

For each λ, we compute the parameter estimate

$$\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}}) = \operatorname*{argmin}_{\boldsymbol{\theta}} R_\lambda(\mathcal{D}_{\text{train}}, \boldsymbol{\theta}) \tag{4.107}$$

We then compute the **validation risk**:

$$R_\lambda^{\text{val}} \triangleq R_0(\mathcal{D}_{\text{valid}}, \hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}})) \tag{4.108}$$

This is an estimate of the **population risk**, which is the expected loss under the true distribution $p^*(\mathbf{x}, \mathbf{y})$. Finally we pick

$$\lambda^* = \operatorname*{argmin}_{\lambda \in \mathcal{S}} R_\lambda^{\text{val}} \tag{4.109}$$

(This requires fitting the model once for each value of λ in $\mathcal{S}$, although in some cases, this can be done more efficiently.)

After picking λ^*, we can refit the model to the entire dataset, $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{valid}}$, to get

$$\hat{\boldsymbol{\theta}}^* = \operatorname*{argmin}_{\boldsymbol{\theta}} R_{\lambda^*}(\mathcal{D}, \boldsymbol{\theta}) \tag{4.110}$$

4.5.5 Cross-validation

The above technique in Sec. 4.5.4 can work very well. However, if the size of the training set is small, leaving aside 20% for a validation set can result in an unreliable estimate of the model parameters.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

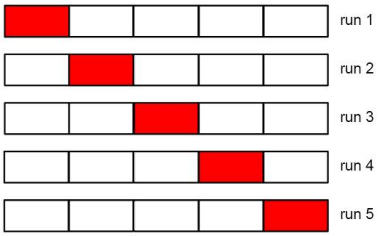


Figure 4.6: Schematic of 5-fold cross validation.

A simple but popular solution to this is to use **cross validation** (**CV**). The idea is as follows: we split the training data into K **folds**; then, for each fold $k \in \{1, \ldots, K\}$, we train on all the folds but the k'th, and test on the k'th, in a round-robin fashion, as sketched in Fig. 4.6. Formally, we have

$$R_\lambda^{\text{cv}} \triangleq \frac{1}{K} \sum_{k=1}^{K} R_0(\mathcal{D}_k, \hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{-k})) \tag{4.111}$$

where $\mathcal{D}_k$ is the data in the k'th fold, and $\mathcal{D}_{-k}$ is all the other data. This is called the **cross-validated risk**. Fig. 4.6 illustrates this procedure for $K = 5$. If we set $K = N$, we get a method known as **leave-one-out cross-validation**, since we always train on $N - 1$ items and test on the remaining one.

We can use the CV estimate as an objective inside of an optimization routine to pick the optimal hyperparameter, $\hat{\lambda} = \text{argmin}_\lambda R_\lambda^{\text{cv}}$. Finally we combine all the available data (training and validation), and re-estimate the model parameters using $\hat{\boldsymbol{\theta}} = \text{argmin}_{\boldsymbol{\theta}} R_{\hat{\lambda}}(\mathcal{D}, \boldsymbol{\theta})$. See Sec. 5.6.3 for more details.

4.5.5.1 The one standard error rule

CV gives an estimate of $\hat{R}(\mathcal{D}, \lambda)$, but does not give any measure of uncertainty. A standard frequentist measure of uncertainty of an estimate is the **standard error of the mean**, which is the mean of the sampling distribution of the estimate (see Sec. 4.7.1). We can compute this as follows. First let $L_n = \ell(\mathbf{y}_n, f(\mathbf{x}_n; \hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{-n}))$ be the loss on the n'th example, where we use the parameters that were estimated using whichever training fold excludes n. (Note that L_n depends on λ, but we drop this from the notation.) Next let $\hat{\mu} = \frac{1}{N} \sum_{n=1}^{N} L_n$ be the empirical mean and $\hat{\sigma}^2 = \frac{1}{N} \sum_{n=1}^{N} (L_n - \hat{\mu})^2$ be the empirical variance. Given this, we define our estimate to be $\hat{\mu}$, and the standard error of this estimate to be $\text{se}(\hat{\mu}) = \frac{\hat{\sigma}}{\sqrt{N}}$. Note that σ measures the intrinsic variability of L_n across samples, whereas $\text{se}(\hat{\mu})$ measures our uncertainty about the mean $\hat{\mu}$.

Suppose we apply CV to a set of models and compute the mean and se of their estimated risks. A common heuristic for picking a model from these noisy estimates is to pick the value which corresponds to the simplest model whose risk is no more than one standard error above the risk of the best model; this is called the **one-standard error rule** [HTF01, p216].

(a) (b)

Figure 4.7: Ridge regression is applied to a degree 14 polynomial fit to 21 datapoints shown in Fig. 4.5 for different values of the regularizer λ. The degree of regularization increases from left to right, so model complexity decreases from left to right. (a) MSE on train (blue) and test (red) vs $\log(\lambda)$. (b) 5-fold cross-validation estimate of test MSE; error bars are standard error of the mean. Generated by code at figures.probml.ai/book1/4.7.

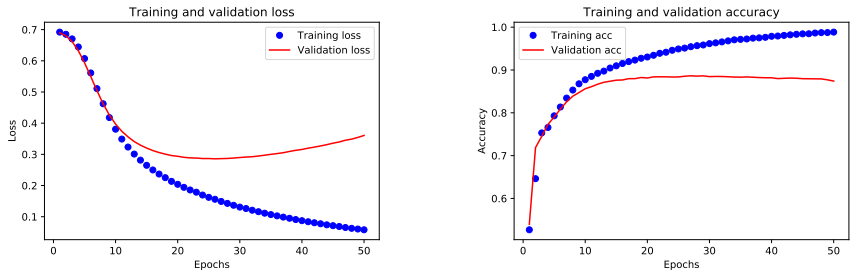


Figure 4.8: Performance of a text classifier (a neural network applied to a bag of word embeddings using average pooling) vs number of training epochs on the IMDB movie sentiment dataset. Blue = train, red = validation. (a) Cross entropy loss. Early stopping is triggered at about epoch 25. (b) Classification accuracy. Generated by code at figures.probml.ai/book1/4.8.

4.5.5.2 Example: ridge regression

As an example, consider picking the strength of the ℓ_2 regularizer for the ridge regression problem in Sec. 4.5.3. In Fig. 4.7a, we plot the error vs $\log(\lambda)$ on the train set (blue) and test set (red curve). We see that the test error has a U-shaped curve, where it decreases as we increase the regularizer, and then increases as we start to underfit. In Fig. 4.7b, we plot the 5-fold CV estimate of the test MSE vs $\log(\lambda)$. We see that the minimum CV error is close the optimal value for the test set (although it does underestimate the spike in the test error for large lambda, due to the small sample size.)

4.5.6 Early stopping

A very simple form of regularization, which is often very effective in practice (especially for complex models), is known as **early stopping**. This leverages the fact that optimization algorithms are

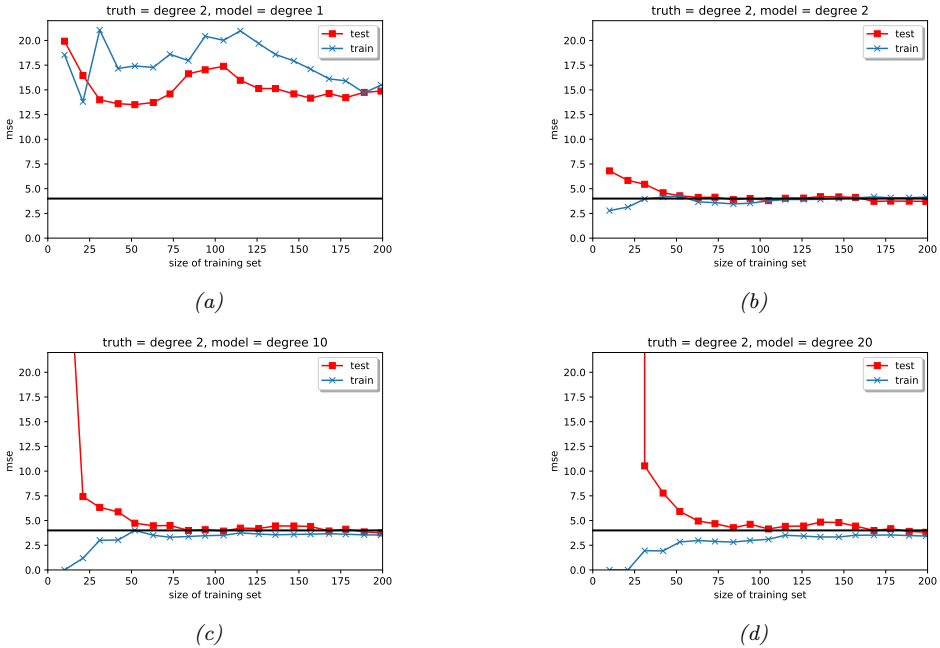


Figure 4.9: MSE on training and test sets vs size of training set, for data generated from a degree 2 polynomial with Gaussian noise of variance $\sigma^2 = 4$. We fit polynomial models of varying degree to this data. Generated by code at figures.probml.ai/book1/4.9.

iterative, and so they take many steps to move away from the initial parameter estimates. If we detect signs of overfitting (by monitoring performance on the validation set), we can stop the optimization process, to prevent the model memorizing too much information about the training set. See Fig. 4.8 for an illustration.

4.5.7 Using more data

As the amount of data increases, the chance of overfitting (for a model of fixed complexity) decreases (assuming the data contains suitably informative examples, and is not too redundant). This is illustrated in Fig. 4.9. We show the MSE on the training and test sets for four different models (polynomials of increasing degree) as a function of the training set size N. (A plot of error vs training set size is known as a **learning curve**.) The horizontal black line represents the **Bayes error**, which is the error of the optimal predictor (the true model) due to inherent noise. (In this example, the true model is a degree 2 polynomial, and the noise has a variance of $\sigma^2 = 4$; this is called the **noise floor**, since we cannot go below it.)

We notice several interesting things. First, the test error for degree 1 remains high, even as N increases, since the model is too simple to capture the truth; this is called underfitting. The test error for the other models decreases to the optimal level (the noise floor), but it decreases more rapidly for the simpler models, since they have fewer parameters to estimate. The gap between the

test error and training error is larger for more complex models, but decreases as N grows.

Another interesting thing we can note is that the training error (blue line) initially *increases* with N, at least for the models that are sufficiently flexible, The reason for this is as follows: as the data set gets larger, we observe more distinct input-output pattern combinations, so the task of fitting the data becomes harder. However, eventually the training set will come to resemble the test set, and the error rates will converge, and will reflect the optimal performance of that model.

4.6 Bayesian statistics *

So far, we have discussed several ways to estimate parameters from data. However, these approaches ignore any uncertainty in the estimates, which can be important for some applications, such as active learning, or avoiding overfitting, or just knowing how much to trust the estimate of some scientifically meaningful quantity. In statistics, modeling uncertainty about parameters using a probability distribution (as opposed to just computing a point estimate) is known as **inference**.

In this section, we use the **posterior distribution** to represent out uncertainty. This is the approach adopted in the field of **Bayesian statistics**. To compute the posterior, we start with a **prior** distribution $p(\boldsymbol{\theta})$, which reflects what we know before seeing the data. We then define a **likelihood function** $p(\mathcal{D}|\boldsymbol{\theta})$, which reflects the data we expect to see for each setting of the parameters. We then use Bayes rule to condition the prior on the observed data to compute the posterior $p(\boldsymbol{\theta}|\mathcal{D})$ as follows:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta})}{p(\mathcal{D})} = \frac{p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta}')p(\mathcal{D}|\boldsymbol{\theta}')d\boldsymbol{\theta}'} \tag{4.112}$$

The denominator $p(\mathcal{D})$ is called the **marginal likelihood**, since it is computed by **marginalizing over** (or **integrating out**) the unknown $\boldsymbol{\theta}$. This can be interpreted as the average probability of the data, where the average is wrt the prior. Note, however, that $p(\mathcal{D})$ is a constant, independent of $\boldsymbol{\theta}$, so we will often ignore it when we just want to infer the relative probabilities of $\boldsymbol{\theta}$ values.

Equation (4.112) is analogous to the use of Bayes rule for COVID-19 testing in Sec. 2.3.1. The difference is that the unknowns correspond to parameters of a statistical model, rather than the unknown disease state of a patient. In addition, we usually condition on a set of observations $\mathcal{D}$, as opposed to a single observation (such as a single test outcome). In particular, for a supervised or conditional model, the observed data has the form $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n) : n = 1 : N\}$. For an unsupervised or unconditional model, the observed data has the form $\mathcal{D} = \{(\mathbf{y}_n) : n = 1 : N\}$.

Once we have computed the posterior over the parameters, we can compute the **posterior predictive distribution** over outputs given inputs by **marginalizing out** the unknown parameters. In the supervised/ conditional case, this becomes

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta} \tag{4.113}$$

This can be viewed as a form of **Bayes model averaging** (**BMA**), since we are making predictions using an infinite set of models (parameter values), each one weighted by how likely it is. The use of BMA reduces the chance of overfitting (Sec. 1.2.3), since we are not just using the single best model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

4.6.1 Conjugate priors

In this section, we consider a set of (prior, likelihood) pairs for which we can compute the posterior in closed form. In particular, we will use priors that are "conjugate" to the likelihood. We say that a prior $p(\boldsymbol{\theta}) \in \mathcal{F}$ is a **conjugate prior** for a likelihood function $p(\mathcal{D}|\boldsymbol{\theta})$ if the posterior is in the same parameterized family as the prior, i.e., $p(\boldsymbol{\theta}|\mathcal{D}) \in \mathcal{F}$. In other words, $\mathcal{F}$ is closed under Bayesian updating. If the family $\mathcal{F}$ corresponds to the exponential family (defined in Sec. 12.2), then the computations can be performed in closed form.

In the sections below, we give some common examples of this framework, which we will use later in the book. For simplicity, we focus on unconditional models (i.e., there are only outcomes or targets y, and no inputs or features $\mathbf{x}$); we relax this assumption in Sec. 4.6.7.

4.6.2 The beta-binomial model

Suppose we toss a coin N times, and want to infer the probability of heads. Let $y_n = 1$ denote the event that the n'th trial was heads, $y_n = 0$ represent the event that the n'th trial was tails, and let $\mathcal{D} = \{y_n : n = 1 : N\}$ be all the data. We assume $y_n \sim \text{Ber}(\theta)$, where $\theta \in [0, 1]$ is the rate parameter (probability of heads). In this section, we discuss how to compute $p(\theta|\mathcal{D})$.

4.6.2.1 Bernoulli likelihood

We assume the data are **iid** or **independent and identically distributed**. Thus the likelihood has the form

$$p(\mathcal{D}|\theta) = \prod_{n=1}^{N} \theta^{y_n}(1-\theta)^{1-y_n} = \theta^{N_1}(1-\theta)^{N_0} \tag{4.114}$$

where we have defined $N_1 = \sum_{n=1}^{N} \mathbb{I}(y_n = 1)$ and $N_0 = \sum_{n=1}^{N} \mathbb{I}(y_n = 0)$, representing the number of heads and tails. These counts are called the **sufficient statistics** of the data, since this is all we need to know about $\mathcal{D}$ to infer θ. The total count, $N = N_0 + N_1$, is called the sample size.

4.6.2.2 Binomial likelihood

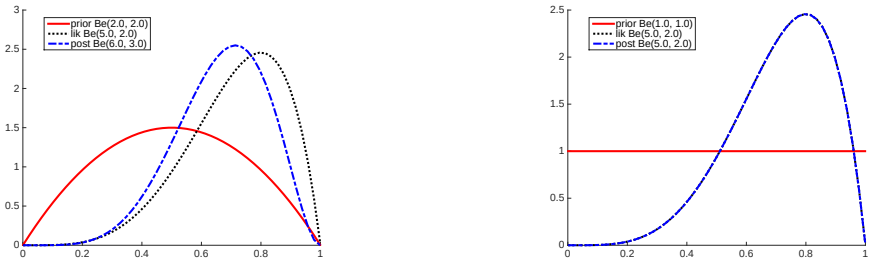
Note that we can also consider a Binomial likelihood model, in which we perform N trials and observe the number of heads, y, rather than observing a sequence of coin tosses. Now the likelihood has the following form:

$$p(\mathcal{D}|\theta) = \text{Bin}(y|N, \theta) = \binom{N}{y}\theta^y(1-\theta)^{N-y} \tag{4.115}$$

The scaling factor $\binom{N}{y}$ is independent of θ, so we can ignore it. Thus this likelihood is proportional to the Bernoulli likelihood in Eq. (4.114), so our inferences about θ will be the same for both models.

4.6.2.3 Prior

To simplify the computations, we will assume that the prior $p(\boldsymbol{\theta}) \in \mathcal{F}$ is a conjugate prior for the likelihood function $p(\mathbf{y}|\boldsymbol{\theta})$. This means that the posterior is in the same parameterized family as the prior, i.e., $p(\boldsymbol{\theta}|\mathcal{D}) \in \mathcal{F}$.



(a) *(b)*

Figure 4.10: Updating a Beta prior with a Bernoulli likelihood with sufficient statistics $N_1 = 4, N_0 = 1$. (a) Beta(2,2) prior. (b) Uniform Beta(1,1) prior. Generated by code at figures.probml.ai/book1/4.10.

To ensure this property when using the Bernoulli (or Binomial) likelihood, we should use a prior of the following form:

$$p(\theta) \propto \theta^{\breve{\alpha}-1}(1-\theta)^{\breve{\beta}-1} = \text{Beta}(\theta|\, \breve{\alpha}, \breve{\beta}) \tag{4.116}$$

We recognize this as the pdf of a beta distribution (see Sec. 2.7.4).

4.6.2.4 Posterior

If we multiply the Bernoulli likelihood in Eq. (4.114) with the beta prior in Eq. (2.133) we get a beta posterior:

$$p(\theta|\mathcal{D}) \propto \theta^{N_1}(1-\theta)^{N_0}\, \theta^{\breve{\alpha}-1}(1-\theta)^{\breve{\beta}-1} \tag{4.117}$$

$$\propto \text{Beta}(\theta|\, \breve{\alpha}+N_1, \breve{\beta}+N_0) \tag{4.118}$$

$$= \text{Beta}(\theta|\, \widehat{\alpha}, \widehat{\beta}) \tag{4.119}$$

where $\widehat{\alpha} \triangleq \breve{\alpha}+N_1$ and $\widehat{\beta} \triangleq \breve{\beta}+N_0$ are the parameters of the posterior. Since the posterior has the same functional form as the prior, we say that the beta distribution is a conjugate prior for the Bernoulli likelihood.

The parameters of the prior are called **hyper-parameters**. It is clear that (in this example) the hyper-parameters play a role analogous to the sufficient statistics; they are therefore often called **pseudo counts**. We see that we can compute the posterior by simply adding the observed counts (from the likelihood) to the pseudo counts (from the prior).

The strength of the prior is controlled by $\breve{N}=\breve{\alpha}+\breve{\beta}$; this is called the **equivalent sample size**, since it plays a role analogous to the observed sample size, $N = N_0 + N_1$.

4.6.2.5 Example

For example, suppose we set $\breve{\alpha}=\breve{\beta}=2$. This is like saying we believe we have already seen two heads and two tails before we see the actual data; this is a very weak preference for the value of

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\theta = 0.5$. The effect of using this prior is illustrated in Fig. 4.10a. We see the posterior (blue line) is a "compromise" between the prior (red line) and the likelihood (black line).

If we set $\breve{\alpha}$=$\breve{\beta}$= 1, the corresponding prior becomes the uniform distribution:

$$p(\theta) = \text{Beta}(\theta|1,1) \propto \theta^0(1-\theta)^0 = \text{Unif}(\theta|0,1) \tag{4.120}$$

The effect of using this prior is illustrated in Fig. 4.10b. We see that the posterior has exactly the same shape as the likelihood, since the prior was "**uninformative**".

4.6.2.6 Posterior mode (MAP estimate)

The most probable value of the parameter is given by the MAP estimate

$$\hat{\theta}_{\text{map}} = \arg\max_\theta p(\theta|\mathcal{D}) \tag{4.121}$$

$$= \arg\max_\theta \log p(\theta|\mathcal{D}) \tag{4.122}$$

$$= \arg\max_\theta \log p(\theta) + \log p(\mathcal{D}|\theta) \tag{4.123}$$

One can show that this is given by

$$\hat{\theta}_{\text{map}} = \frac{\breve{\alpha} + N_1 - 1}{\breve{\alpha} + N_1 - 1 + \breve{\beta} + N_0 - 1} \tag{4.124}$$

If we use a $\text{Beta}(\theta|2,2)$ prior, this amounts to **add-one smoothing**:

$$\hat{\theta}_{\text{map}} = \frac{N_1 + 1}{N_1 + 1 + N_0 + 1} = \frac{N_1 + 1}{N + 2} \tag{4.125}$$

If we use a uniform prior, $p(\theta) \propto 1$, the MAP estimate becomes the MLE, since $\log p(\theta) = 0$:

$$\hat{\theta}_{\text{mle}} = \arg\max_\theta \log p(\mathcal{D}|\theta) \tag{4.126}$$

When we use a Beta prior, the uniform distribution is $\breve{\alpha}$=$\breve{\beta}$= 1. In this case, the MAP estimate reduces to the MLE:

$$\hat{\theta}_{\text{mle}} = \frac{N_1}{N_1 + N_0} = \frac{N_1}{N} \tag{4.127}$$

If $N_1 = 0$, we will estimate that $p(Y = 1) = 0.0$, which says that we do not predict any future observations to be 1. This is a very extreme estimate, that is likely due to insufficient data. We can solve this problem using a MAP estimate with a stronger prior, or using a fully Bayesian approach, in which we marginalize out θ instead of estimating it, as explained in Sec. 4.6.2.9.

4.6.2.7 Posterior mean

The posterior mode can be a poor summary of the posterior, since it corresponds to a single point. The posterior mean is a more robust estimate, since it integrates over the whole space.

If $p(\theta|\mathcal{D}) = \text{Beta}(\theta|\, \widehat{\alpha}, \widehat{\beta})$, then the posterior mean is given by

$$\overline{\theta} \triangleq \mathbb{E}\left[\theta|\mathcal{D}\right] = \frac{\widehat{\alpha}}{\widehat{\beta} + \widehat{\alpha}} = \frac{\widehat{\alpha}}{\widehat{N}} \tag{4.128}$$

where $\widehat{N}{=}\widehat{\beta} + \widehat{\alpha}$ is the strength (equivalent sample size) of the posterior.

We will now show that the posterior mean is a convex combination of the prior mean, $m = \breve{\alpha} \,/\, \breve{N}$ (where $\breve{N}{\triangleq}\breve{\alpha} + \breve{\beta}$ is the prior strength), and the MLE: $\hat{\theta}_{\text{mle}} = \frac{N_1}{N}$:

$$\mathbb{E}\left[\theta|\mathcal{D}\right] = \frac{\breve{\alpha} + N_1}{\breve{\alpha} + N_1 + \breve{\beta} + N_0} = \frac{\breve{N}\, m + N_1}{N + \breve{N}} = \frac{\breve{N}}{N + \breve{N}} m + \frac{N}{N + \breve{N}} \frac{N_1}{N} = \lambda m + (1 - \lambda)\hat{\theta}_{\text{mle}} \tag{4.129}$$

where $\lambda = \frac{\breve{N}}{\widehat{N}}$ is the ratio of the prior to posterior equivalent sample size. So the weaker the prior, the smaller is λ, and hence the closer the posterior mean is to the MLE.

4.6.2.8 Posterior variance

To capture some notion of uncertainty in our estimate, a common approach is to compute the **standard error** of our estimate, which is just the posterior standard deviation:

$$\text{se}(\theta) = \sqrt{\mathbb{V}\left[\theta|\mathcal{D}\right]} \tag{4.130}$$

In the case of the Bernoulli model, we showed that the posterior is a beta distribution. The variance of the beta posterior is given by

$$\mathbb{V}\left[\theta|\mathcal{D}\right] = \frac{\widehat{\alpha}\widehat{\beta}}{(\widehat{\alpha} + \widehat{\beta})^2(\widehat{\alpha} + \widehat{\beta} + 1)} \tag{4.131}$$

where $\widehat{\alpha}{=}\breve{\alpha} + N_1$ and $\widehat{\beta}{=}\breve{\beta} + N_0$. If $N \gg \breve{\alpha} + \breve{\beta}$, this simplifies to

$$\mathbb{V}\left[\theta|\mathcal{D}\right] \approx \frac{N_1 N_0}{N^3} = \frac{\hat{\theta}(1 - \hat{\theta})}{N} \tag{4.132}$$

where $\hat{\theta}$ is the MLE. Hence the standard error is given by

$$\sigma = \sqrt{\mathbb{V}\left[\theta|\mathcal{D}\right]} \approx \sqrt{\frac{\hat{\theta}(1 - \hat{\theta})}{N}} \tag{4.133}$$

We see that the uncertainty goes down at a rate of $1/\sqrt{N}$. We also see that the uncertainty (variance) is maximized when $\hat{\theta} = 0.5$, and is minimized when $\hat{\theta}$ is close to 0 or 1. This makes sense, since it is easier to be sure that a coin is biased than to be sure that it is fair.

4.6.2.9 Posterior predictive

Suppose we want to predict future observations. A very common approach is to first compute an estimate of the parameters based on training data, $\hat{\boldsymbol{\theta}}(\mathcal{D})$, and then to plug that parameter back into the model and use $p(y|\hat{\boldsymbol{\theta}})$ to predict the future; this is called a **plug-in approximation**. However, this can result in overfitting. As an extreme example, suppose we have seen $N = 3$ heads in a row. The MLE is $\hat{\theta} = 3/3 = 1$. However, if we use this estimate, we would predict that tails are impossible.

One solution to this is to compute a MAP estimate, and plug that in, as we discussed in Sec. 4.5.1. Here we discuss a fully Bayesian solution, in which we marginalize out θ.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

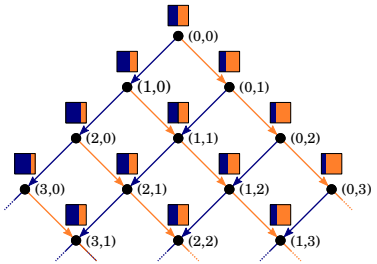


Figure 4.11: Illustration of sequential Bayesian updating for the beta-Bernoulli model. Each colored box represents the predicted distribution $p(x_t|\mathbf{h}_t)$, where $\mathbf{h}_t = (N_{1,t}, N_{0,t})$ is the sufficient statistic derived from history of observations up until time t, namely the total number of heads and tails. The probability of heads (blue bar) is given by $p(x_t = 1|\mathbf{h}_t) = (N_{t,1} + 1)/(t + 2)$, assuming we start with a uniform $\text{Beta}(\theta|1, 1)$ *prior. From Figure 3 of [Ort+19]. Used with kind permission of Pedro Ortega.*

Bernoulli model

For the Bernoulli model, the resulting **posterior predictive distribution** has the form

$$p(y = 1|\mathcal{D}) = \int_0^1 p(y = 1|\theta)p(\theta|\mathcal{D})d\theta \tag{4.134}$$

$$= \int_0^1 \theta \, \text{Beta}(\theta|\, \widehat{\alpha}, \widehat{\beta})d\theta = \mathbb{E}\left[\theta|\mathcal{D}\right] = \frac{\widehat{\alpha}}{\widehat{\alpha} + \widehat{\beta}} \tag{4.135}$$

In Sec. 4.5.1, we had to use the Beta(2,2) prior to recover add-one smoothing, which is a rather unnatural prior. In the Bayesian approach, we can get the same effect using a uniform prior, $p(\theta) = \text{Beta}(\theta|1, 1)$, since the predictive distribution becomes

$$p(y = 1|\mathcal{D}) = \frac{N_1 + 1}{N_1 + N_0 + 2} \tag{4.136}$$

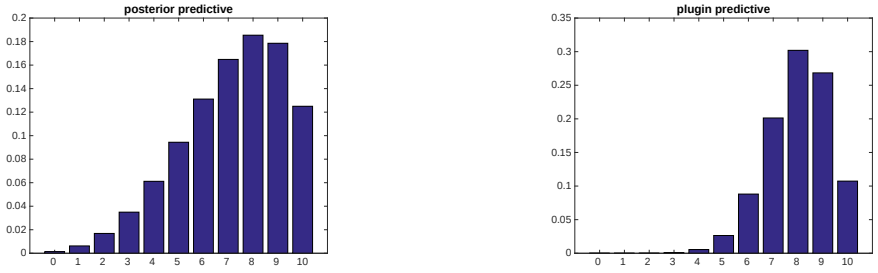
This is known as **Laplace's rule of succession**. See Fig. 4.11 for an illustration of this in the sequential setting.

Binomial model

Now suppose we were interested in predicting the number of heads in $M > 1$ future coin tossing trials, i.e., we are using the binomial model instead of the Bernoulli model. The posterior over θ is the same as before, but the posterior predictive distribution is different:

$$p(y|\mathcal{D}, M) = \int_0^1 \text{Bin}(y|M, \theta)\text{Beta}(\theta|\, \widehat{\alpha}, \widehat{\beta})d\theta \tag{4.137}$$

$$= \binom{M}{y} \frac{1}{B(\widehat{\alpha}, \widehat{\beta})} \int_0^1 \theta^y (1 - \theta)^{M-y} \theta^{\widehat{\alpha}-1} (1 - \theta)^{\widehat{\beta}-1} d\theta \tag{4.138}$$



(a) (b)

Figure 4.12: (a) Posterior predictive distributions for 10 future trials after seeing $N_1 = 4$ heads and $N_0 = 1$ tails. (b) Plug-in approximation based on the same data. In both cases, we use a uniform prior. Generated by code at figures.probml.ai/book1/4.12.

We recognize the integral as the normalization constant for a Beta$(\widehat{\alpha} + y, M - y + \widehat{\beta})$ distribution. Hence

$$\int_0^1 \theta^{y+\widehat{\alpha}-1}(1-\theta)^{M-y+\widehat{\beta}-1}d\theta = B(y+\widehat{\alpha}, M-y+\widehat{\beta}) \tag{4.139}$$

Thus we find that the posterior predictive is given by the following, known as the (compound) **beta-binomial** distribution:

$$Bb(x|M,\widehat{\alpha},\widehat{\beta}) \triangleq \binom{M}{x} \frac{B(x+\widehat{\alpha}, M-x+\widehat{\beta})}{B(\widehat{\alpha},\widehat{\beta})} \tag{4.140}$$

In Fig. 4.12(a), we plot the posterior predictive density for $M = 10$ after seeing $N_1 = 4$ heads and $N_0 = 1$ tails, when using a uniform Beta(1,1) prior. In Fig. 4.12(b), we plot the plug-in approximation, given by

$$p(\theta|\mathcal{D}) \approx \delta(\theta - \hat{\theta}) \tag{4.141}$$

$$p(y|\mathcal{D}, M) = \int_0^1 \text{Bin}(y|M,\theta)p(\theta|\mathcal{D})d\theta = \text{Bin}(y|M,\hat{\theta}) \tag{4.142}$$

where $\hat{\theta}$ is the MAP estimate. Looking at Fig. 4.12, we see that the Bayesian prediction has longer tails, spreading its probablity mass more widely, and is therefore less prone to overfitting and black-swan type paradoxes. (Note that we use a uniform prior in both cases, so the difference is not arising due to the use of a prior; rather, it is due to the fact that the Bayesian approach integrates out the unknown parameters when making its predictions.)

4.6.2.10 Marginal likelihood

The **marginal likelihood** or **evidence** for a model $\mathcal{M}$ is defined as

$$p(\mathcal{D}|\mathcal{M}) = \int p(\boldsymbol{\theta}|\mathcal{M})p(\mathcal{D}|\boldsymbol{\theta},\mathcal{M})d\boldsymbol{\theta} \tag{4.143}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

When performing inference for the parameters of a specific model, we can ignore this term, since it is constant wrt $\boldsymbol{\theta}$. However, this quantity plays a vital role when choosing between different models, as we discuss in Sec. 5.4.2. It is also useful for estimating the hyperparameters from data (an approach known as empirical Bayes), as we discuss in Sec. 4.6.5.3.

In general, computing the marginal likelihood can be hard. However, in the case of the beta-Bernoulli model, the marginal likelihood is proportional to the ratio of the posterior normalizer to the prior normalizer. To see this, recall that the posterior for the beta-binomial models is given by $p(\theta|\mathcal{D}) = \text{Beta}(\theta|a', b')$, where $a' = a + N_1$ and $b' = b + N_0$. We know the normalization constant of the posterior is $B(a', b')$. Hence

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \tag{4.144}$$

$$= \frac{1}{p(\mathcal{D})}\left[\frac{1}{B(a,b)}\theta^{a-1}(1-\theta)^{b-1}\right]\left[\binom{N}{N_1}\theta^{N_1}(1-\theta)^{N_0}\right] \tag{4.145}$$

$$= \binom{N}{N_1}\frac{1}{p(\mathcal{D})}\frac{1}{B(a,b)}\left[\theta^{a+N_1-1}(1-\theta)^{b+N_0-1}\right] \tag{4.146}$$

So

$$\frac{1}{B(a+N_1, b+N_0)} = \binom{N}{N_1}\frac{1}{p(\mathcal{D})}\frac{1}{B(a,b)} \tag{4.147}$$

$$p(\mathcal{D}) = \binom{N}{N_1}\frac{B(a+N_1, b+N_0)}{B(a,b)} \tag{4.148}$$

The marginal likelihood for the beta-Bernoulli model is the same as above, except it is missing the $\binom{N}{N_1}$ term.

4.6.2.11 Mixtures of conjugate priors

The beta distribution is a conjugate prior for the binomial likelihood, which enables us to easily compute the posterior in closed form, as we have seen. However, this prior is rather restrictive. For example, suppose we want to predict the outcome of a coin toss at a casino, and we believe that the coin may be fair, but may equally likely be biased towards heads. This prior cannot be represented by a beta distribution. Fortunately, it can be represented as a **mixture of beta distributions**. For example, we might use

$$p(\theta) = 0.5\ \text{Beta}(\theta|20, 20) + 0.5\ \text{Beta}(\theta|30, 10) \tag{4.149}$$

If θ comes from the first distribution, the coin is fair, but if it comes from the second, it is biased towards heads.

We can represent a mixture by introducing a latent indicator variable h, where $h = k$ means that θ comes from mixture component k. The prior has the form

$$p(\theta) = \sum_k p(h = k)p(\theta|h = k) \tag{4.150}$$

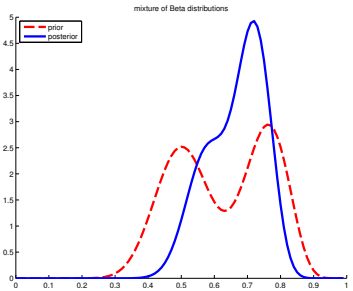


Figure 4.13: A mixture of two Beta distributions. Generated by code at figures.probml.ai/book1/4.13.

where each $p(\theta|h=k)$ is conjugate, and $p(h=k)$ are called the (prior) mixing weights. One can show (Exercise 4.6) that the posterior can also be written as a mixture of conjugate distributions as follows:

$$p(\theta|\mathcal{D}) = \sum_k p(h=k|\mathcal{D}) p(\theta|\mathcal{D}, h=k) \tag{4.151}$$

where $p(h=k|\mathcal{D})$ are the posterior mixing weights given by

$$p(h=k|\mathcal{D}) = \frac{p(h=k)p(\mathcal{D}|h=k)}{\sum_{k'} p(h=k')p(\mathcal{D}|h=k')} \tag{4.152}$$

Here the quantity $p(\mathcal{D}|h=k)$ is the marginal likelihood for mixture component k (see Sec. 4.6.2.10).

Returning to our example above, if we have the prior in Eq. (4.149), and we observe $N_1 = 20$ heads and $N_0 = 10$ tails, then, using Eq. (4.148), the posterior becomes

$$p(\theta|\mathcal{D}) = 0.346 \text{ Beta}(\theta|40, 30) + 0.654 \text{ Beta}(\theta|30, 20) \tag{4.153}$$

See Fig. 4.13 for an illustration.

We can compute the posterior probability that the coin is biased towards heads as follows:

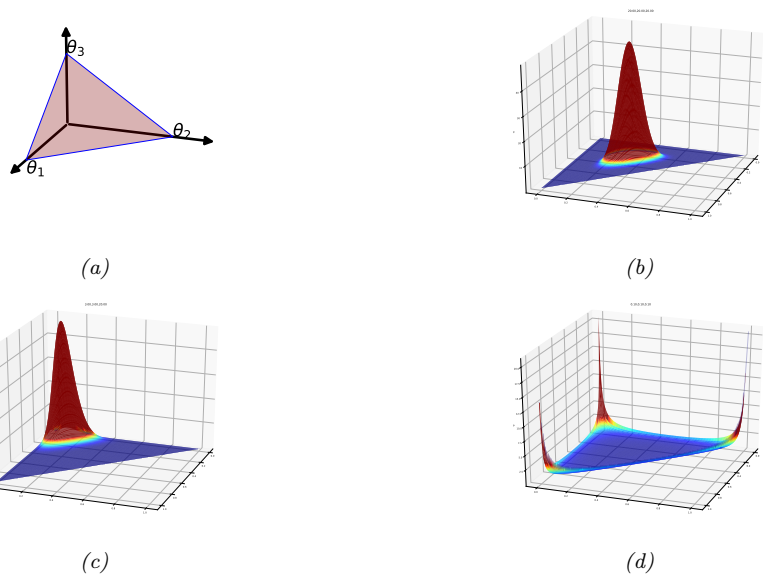
$$\Pr(\theta > 0.5|\mathcal{D}) = \sum_k \Pr(\theta > 0.5|\mathcal{D}, h=k) p(h=k|\mathcal{D}) = 0.9604 \tag{4.154}$$

If we just used a single Beta(20,20) prior, we would get a slightly smaller value of $\Pr(\theta > 0.5|\mathcal{D}) = 0.8858$. So if we were "suspicious" initially that the casino might be using a biased coin, our fears would be confirmed more quickly than if we had to be convinced starting with an open mind.

4.6.3 The Dirichlet-multinomial model

In this section, we generalize the results from Sec. 4.6.2 from binary variables (e.g., coins) to K-ary variables (e.g., dice).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c) (d)

Figure 4.14: (a) The Dirichlet distribution when $K = 3$ defines a distribution over the simplex, which can be represented by the triangular surface. Points on this surface satisfy $0 \leq \theta_k \leq 1$ and $\sum_{k=1}^{3} \theta_k = 1$. Generated by code at figures.probml.ai/book1/4.14. (b) Plot of the Dirichlet density for $\breve{\boldsymbol{\alpha}} = (20, 20, 20)$. (c) Plot of the Dirichlet density for $\breve{\boldsymbol{\alpha}} = (3, 3, 20)$. (d) Plot of the Dirichlet density for $\breve{\boldsymbol{\alpha}} = (0.1, 0.1, 0.1)$. Generated by code at figures.probml.ai/book1/4.14.

4.6.3.1 Likelihood

Let $Y \sim \text{Cat}(\boldsymbol{\theta})$ be a discrete random variable drawn from a categorical distribution. The likelihood has the form

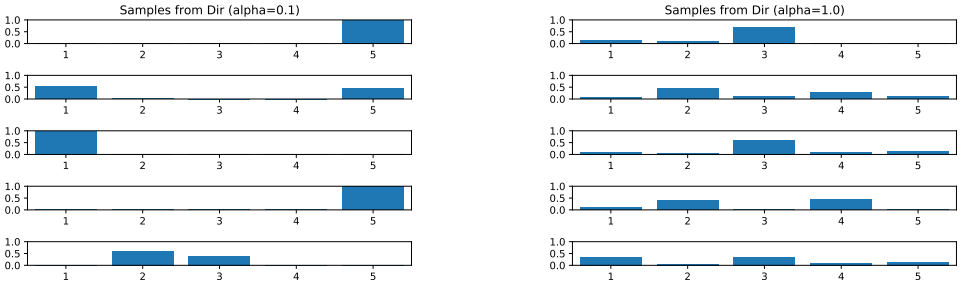
$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} \text{Cat}(y_n|\boldsymbol{\theta}) = \prod_{n=1}^{N} \prod_{c=1}^{C} \theta_c^{\mathbb{I}(y_n=c)} = \prod_{c=1}^{C} \theta_c^{N_c} \tag{4.155}$$

where $N_c = \sum_n \mathbb{I}(y_n = c)$.

4.6.3.2 Prior

The conjugate prior for a categorical distribution is the **Dirichlet distribution**, which is a multivariate generalization of the beta distribution. This has support over the **probability simplex**, defined by

$$S_K = \{\boldsymbol{\theta} : 0 \leq \theta_k \leq 1, \sum_{k=1}^{K} \theta_k = 1\} \tag{4.156}$$



(a) (b)

Figure 4.15: Samples from a 5-dimensional symmetric Dirichlet distribution for different parameter values. (a) $\breve{\boldsymbol{\alpha}}= (0.1, \ldots, 0.1)$. This results in very sparse distributions, with many 0s. (b) $\breve{\boldsymbol{\alpha}}= (1, \ldots, 1)$. This results in more uniform (and dense) distributions. Generated by code at figures.probml.ai/book1/4.15.

The pdf of the Dirichlet is defined as follows:

$$\text{Dir}(\boldsymbol{\theta}|\, \breve{\boldsymbol{\alpha}}) \triangleq \frac{1}{B(\breve{\boldsymbol{\alpha}})} \prod_{k=1}^{K} \theta_k^{\breve{\alpha}_k - 1} \mathbb{I}\left(\boldsymbol{\theta} \in S_K\right) \tag{4.157}$$

where $B(\breve{\boldsymbol{\alpha}})$ is the multivariate beta function,

$$B(\breve{\boldsymbol{\alpha}}) \triangleq \frac{\prod_{k=1}^{K} \Gamma(\breve{\alpha}_k)}{\Gamma(\sum_{k=1}^{K} \breve{\alpha}_k)} \tag{4.158}$$

Fig. 4.14 shows some plots of the Dirichlet when $K = 3$. We see that $\breve{\alpha}_0 = \sum_k \breve{\alpha}_k$ controls the strength of the distribution (how peaked it is), and the $\breve{\alpha}_k$ control where the peak occurs. For example, $\text{Dir}(1, 1, 1)$ is a uniform distribution, $\text{Dir}(2, 2, 2)$ is a broad distribution centered at $(1/3, 1/3, 1/3)$, and $\text{Dir}(20, 20, 20)$ is a narrow distribution centered at $(1/3, 1/3, 1/3)$. $\text{Dir}(3, 3, 20)$ is an asymmetric distribution that puts more density in one of the corners. If $\breve{\alpha}_k < 1$ for all k, we get "spikes" at the corners of the simplex. Samples from the distribution when $\breve{\alpha}_k < 1$ will be sparse, as shown in Fig. 4.15.

4.6.3.3 Posterior

We can combine the multinomial likelihood and Dirichlet prior to compute the posterior, as follows:

$$p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})\text{Dir}(\boldsymbol{\theta}|\, \breve{\boldsymbol{\alpha}}) \tag{4.159}$$

$$= \left[\prod_k \theta_k^{N_k}\right] \left[\prod_k \theta_k^{\breve{\alpha}_k - 1}\right] \tag{4.160}$$

$$= \text{Dir}(\boldsymbol{\theta}|\, \breve{\alpha}_1 + N_1, \ldots, \breve{\alpha}_K + N_K) \tag{4.161}$$

$$= \text{Dir}(\boldsymbol{\theta}|\, \widehat{\boldsymbol{\alpha}}) \tag{4.162}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\widehat{\alpha}_k = \breve{\alpha}_k + N_k$ are the parameters of the posterior. So we see that the posterior can be computed by adding the empirical counts to the prior counts.

The posterior mean is given by

$$\overline{\theta}_k = \frac{\widehat{\alpha}_k}{\sum_{k'=1}^{K} \widehat{\alpha}_{k'}} \tag{4.163}$$

The posterior mode, which corresponds to the MAP estimate, is given by

$$\hat{\theta}_k = \frac{\widehat{\alpha}_k - 1}{\sum_{k'=1}^{K} (\widehat{\alpha}_{k'} - 1)} \tag{4.164}$$

If we use $\breve{\alpha}_k = 1$, corresponding to a uniform prior, the MAP becomes the MLE:

$$\hat{\theta}_k = N_k / N \tag{4.165}$$

(See Sec. 4.2.4 for a more direct derivation of this result.)

4.6.3.4 Posterior predictive

The posterior predictive distribution is given by

$$p(y = k|\mathcal{D}) = \int p(y = k|\boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \tag{4.166}$$

$$= \int \theta_k p(\theta_k|\mathcal{D}) d\theta_k = \mathbb{E}\left[\theta_k|\mathcal{D}\right] = \frac{\widehat{\alpha}_k}{\sum_{k'} \widehat{\alpha}_{k'}} \tag{4.167}$$

In other words, the posterior predictive distribution is given by

$$p(y|\mathcal{D}) = \mathrm{Cat}(y|\overline{\boldsymbol{\theta}}) \tag{4.168}$$

where $\overline{\boldsymbol{\theta}} \triangleq \mathbb{E}\left[\boldsymbol{\theta}|\mathcal{D}\right]$ are the posterior mean parameters. If instead we plug-in the MAP estimate, we will suffer from the zero-count problem. The only way to get the same effect as add-one smoothing is to use a MAP estimate with $\breve{\alpha}_c = 2$.

Eq. (4.167) gives the probability of a single future event, conditioned on past observations $\mathbf{y} = (y_1, \ldots, y_N)$. In some cases, we want to know the probability of observing a batch of future data, say $\tilde{\mathbf{y}} = (\tilde{y}_1, \ldots, \tilde{y}_M)$. We can compute this as follows:

$$p(\tilde{\mathbf{y}}|\mathbf{y}) = \frac{p(\tilde{\mathbf{y}}, \mathbf{y})}{p(\mathbf{y})} \tag{4.169}$$

The denominator is the marginal likelihood of the training data, and the numerator is the marginal likelihood of the training and future test data. We discuss how to compute such marginal likelihoods in Sec. 4.6.3.5.

4.6.3.5 Marginal likelihood

By the same reasoning as in Sec. 4.6.2.10, one can show that the marginal likelihood for the Dirichlet-categorical model is given by

$$p(\mathcal{D}) = \frac{B(\mathbf{N}+\boldsymbol{\alpha})}{B(\boldsymbol{\alpha})} \tag{4.170}$$

where

$$B(\boldsymbol{\alpha}) = \frac{\prod_{k=1}^{K}\Gamma(\alpha_k)}{\Gamma(\sum_k \alpha_k)} \tag{4.171}$$

Hence we can rewrite the above result in the following form, which is what is usually presented in the literature:

$$p(\mathcal{D}) = \frac{\Gamma(\sum_k \alpha_k)}{\Gamma(N+\sum_k \alpha_k)} \prod_k \frac{\Gamma(N_k+\alpha_k)}{\Gamma(\alpha_k)} \tag{4.172}$$

4.6.4 The Gaussian-Gaussian model

In this section, we derive the posterior for the parameters of a Gaussian distribution. For simplicity, we assume the variance is known. (The general case is discussed in the sequel to this book, [Mur22], as well as other standard references on Bayesian statistics.)

4.6.4.1 Univariate case

If σ^2 is a known constant, the likelihood for μ has the form

$$p(\mathcal{D}|\mu) \propto \exp\left(-\frac{1}{2\sigma^2}\sum_{n=1}^{N}(y_n-\mu)^2\right) \tag{4.173}$$

One can show that the conjugate prior is another Gaussian, $\mathcal{N}(\mu|\breve{m},\breve{\tau}^2)$. Applying Bayes' rule for Gaussians (Eq. (3.36)), we find that the corresponding posterior is given by

$$p(\mu|\mathcal{D},\sigma^2) = \mathcal{N}(\mu|\hat{m},\hat{\tau}^2) \tag{4.174}$$

$$\hat{\tau}^2 = \frac{1}{\frac{N}{\sigma^2}+\frac{1}{\breve{\tau}^2}} = \frac{\sigma^2\,\breve{\tau}^2}{N\,\breve{\tau}^2+\sigma^2} \tag{4.175}$$

$$\hat{m} = \hat{\tau}^2\left(\frac{\breve{m}}{\breve{\tau}^2}+\frac{N\overline{y}}{\sigma^2}\right) = \frac{\sigma^2}{N\,\breve{\tau}^2+\sigma^2}\,\breve{m} + \frac{N\,\breve{\tau}^2}{N\,\breve{\tau}^2+\sigma^2}\overline{y} \tag{4.176}$$

where $\overline{y} \triangleq \frac{1}{N}\sum_{n=1}^{N} y_n$ is the empirical mean.

This result is easier to understand if we work in terms of the precision parameters, which are just inverse variances. Specifically, let $\kappa = 1/\sigma^2$ be the observation precision, and $\breve{\lambda} = 1/\breve{\tau}^2$ be the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

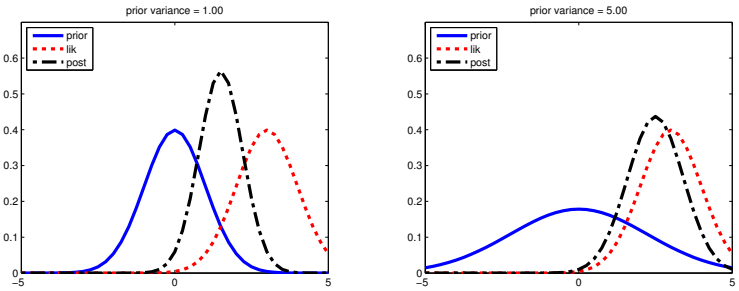


Figure 4.16: Inferring the mean of a univariate Gaussian with known σ^2. (a) Using strong prior, $p(\mu) = \mathcal{N}(\mu|0,1)$. (b) Using weak prior, $p(\mu) = \mathcal{N}(\mu|0,5)$. Generated by code at figures.probml.ai/book1/4.16.

precision of the prior. We can then rewrite the posterior as follows:

$$p(\mu|\mathcal{D},\kappa) = \mathcal{N}(\mu|\, \widehat{m}, \widehat{\lambda}^{-1}) \tag{4.177}$$

$$\widehat{\lambda} = \breve{\lambda} + N\kappa \tag{4.178}$$

$$\widehat{m} = \frac{N\kappa\overline{y} + \breve{\lambda}\breve{m}}{\widehat{\lambda}} = \frac{N\kappa}{N\kappa + \breve{\lambda}}\overline{y} + \frac{\breve{\lambda}}{N\kappa + \breve{\lambda}}\,\breve{m} \tag{4.179}$$

These equations are quite intuitive: the posterior precision $\widehat{\lambda}$ is the prior precision $\breve{\lambda}$ plus N units of measurement precision κ. Also, the posterior mean $\widehat{m}$ is a convex combination of the empirical mean $\overline{y}$ and the prior mean $\breve{m}$. This makes it clear that the posterior mean is a compromise between the empirical mean and the prior. If the prior is weak relative to the signal strength ($\breve{\lambda}$ is small relative to κ), we put more weight on the empirical mean. If the prior is strong relative to the signal strength ($\breve{\lambda}$ is large relative to κ), we put more weight on the prior. This is illustrated in Fig. 4.16. Note also that the posterior mean is written in terms of $N\kappa\overline{y}$, so having N measurements each of precision κ is like having one measurement with value $\overline{y}$ and precision $N\kappa$.

Posterior after seeing $N = 1$ examples

To gain further insight into these equations, consider the posterior after seeing a single data point y (so $N = 1$). Then the posterior mean can be written in the following equivalent ways:

$$\widehat{m} = \frac{\breve{\lambda}}{\widehat{\lambda}}\,\breve{m} + \frac{\kappa}{\widehat{\lambda}}y \tag{4.180}$$

$$= \breve{m} + \frac{\kappa}{\widehat{\lambda}}(y - \breve{m}) \tag{4.181}$$

$$= y - \frac{\breve{\lambda}}{\widehat{\lambda}}(y - \breve{m}) \tag{4.182}$$

The first equation is a convex combination of the prior mean and the data. The second equation is the prior mean adjusted towards the data y. The third equation is the data adjusted towards the prior mean; this is called a **shrinkage** estimate. This is easier to see if we define the weight $w = \breve{\lambda}/\widehat{\lambda}$, which is the ratio of the prior to posterior precision. Then we have

$$\widehat{m} = y - w(y - \breve{m}) = (1 - w)y + w\,\breve{m} \tag{4.183}$$

Note that, for a Gaussian, the posterior mean and posterior mode are the same. Thus we can use the above equations to perform MAP estimation. See Exercise 4.2 for a simple example.

Posterior variance

In addition to the posterior mean or mode of μ, we might be interested in the posterior variance, which gives us a measure of confidence in our estimate. The square root of this is called the **standard error of the mean**:

$$\text{se}(\mu) \triangleq \sqrt{\mathbb{V}\left[\mu|\mathcal{D}\right]} \tag{4.184}$$

Suppose we use an uninformative prior for μ by setting $\breve{\lambda}= 0$ (see Sec. 4.6.5.1). In this case, the posterior mean is equal to the MLE, $\widehat{m}= \overline{y}$. Suppose, in addition, that we approximate σ^2 by the **sample variance**

$$s^2 \triangleq \frac{1}{N}\sum_{n=1}^{N}(y_n - \overline{y})^2 \tag{4.185}$$

Hence $\widehat{\lambda}= N\hat{\kappa} = N/s^2$, so the SEM becomes

$$\text{se}(\mu) = \sqrt{\mathbb{V}\left[\mu|\mathcal{D}\right]} = \frac{1}{\sqrt{\widehat{\lambda}}} = \frac{s}{\sqrt{N}} \tag{4.186}$$

Thus we see that the uncertainty in μ is reduced at a rate of $1/\sqrt{N}$.

In addition, we can use the fact that 95% of a Gaussian distribution is contained within 2 standard deviations of the mean to approximate the 95% **credible interval** for μ using

$$I_{.95}(\mu|\mathcal{D}) = \overline{y} \pm 2\frac{s}{\sqrt{N}} \tag{4.187}$$

4.6.4.2 Multivariate case

For D-dimensional data, the likelihood has the form

$$p(\mathcal{D}|\boldsymbol{\mu}) = \prod_{n=1}^{N}\mathcal{N}(y_n|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \tag{4.188}$$

$$= \frac{N}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{\frac{1}{2}}}\exp\left[-\frac{1}{2}\sum_{n=1}^{N}(\mathbf{y}_n - \boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{y}_n - \boldsymbol{\mu})\right] \tag{4.189}$$

$$= \mathcal{N}(\overline{\mathbf{y}}|\boldsymbol{\mu}, \frac{1}{N}\boldsymbol{\Sigma}) \tag{4.190}$$

where $\overline{\mathbf{y}} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{y}_n$. Thus we replace the set of observations with their mean, and scale down the covariance by a factor of N.

For simplicity, we will use a conjugate prior, which in this case is a Gaussian, namely

$$p(\boldsymbol{\mu}) = \mathcal{N}(\boldsymbol{\mu}|\, \breve{\mathbf{m}}, \breve{\mathbf{V}}) \tag{4.191}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

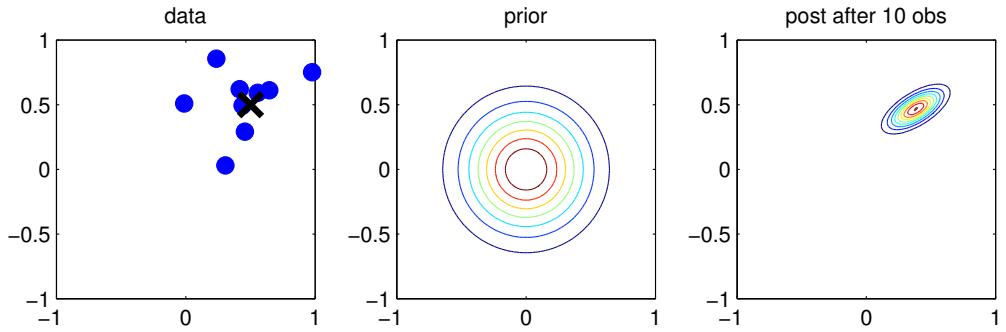


Figure 4.17: Illustration of Bayesian inference for the mean of a 2d Gaussian. (a) The data is generated from $\mathbf{y}_n \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$*, where* $\boldsymbol{\mu} = [0.5, 0.5]^\top$ *and* $\boldsymbol{\Sigma} = 0.1[2, 1; 1, 1]$*). (b) The prior is* $p(\boldsymbol{\mu}) = \mathcal{N}(\boldsymbol{\mu}|\mathbf{0}, 0.1\mathbf{I}_2)$*. (c) We show the posterior after 10 data points have been observed. Generated by code at figures.probml.ai/book1/4.17.*

We can derive a Gaussian posterior for $\boldsymbol{\mu}$ based on the results in Sec. 3.3 We get

$$p(\boldsymbol{\mu}|\mathcal{D}, \boldsymbol{\Sigma}) = \mathcal{N}(\boldsymbol{\mu}|\, \widehat{\mathbf{m}}, \widehat{\mathbf{V}}) \tag{4.192}$$

$$\widehat{\mathbf{V}}^{-1} = \breve{\mathbf{V}}^{-1} + N\boldsymbol{\Sigma}^{-1} \tag{4.193}$$

$$\widehat{\mathbf{m}} = \widehat{\mathbf{V}}\,(\boldsymbol{\Sigma}^{-1}(N\overline{\mathbf{y}}) + \breve{\mathbf{V}}^{-1}\breve{\mathbf{m}}) \tag{4.194}$$

Fig. 4.17 gives a 2d example of these results.

4.6.5 Beyond conjugate priors

We have seen various examples of conjugate priors, all of which have come from the exponential family (see Sec. 12.2). These priors have the advantage of being easy to interpret (in terms of sufficient statistics from a virtual prior dataset), and easy to compute with. However, for most models, there is no prior in the exponential family that is conjugate to the likelihood. Furthermore, even where there is a conjugate prior, the assumption of conjugacy may be too limiting. Therefore in the sections below, we briefly discuss various other kinds of priors.

4.6.5.1 Noninformative priors

When we have little or no domain specific knowledge, it is desirable to use an **uninformative**, **noninformative** or **objective** priors, to "let the data speak for itself". For example, if we want to infer a real valued quantity, such as a location parameter $\mu \in \mathbb{R}$, we can use a **flat prior** $p(\mu) \propto 1$. This can be viewed as an "infinitely wide" Gaussian.

Unfortunately, there is no unique way to define uninformative priors, and they all encode some kind of knowledge. It is therefore better to use the term **diffuse prior**, **minimally informative prior** or **default prior**. See the sequel to this book, [Mur22], for more details.

4.6.5.2 Hierarchical priors

Bayesian models require specifying a prior $p(\boldsymbol{\theta})$ for the parameters. The parameters of the prior are called **hyperparameters**, and will be denoted by $\boldsymbol{\phi}$. If these are unknown, we can put a prior on them; this defines a **hierarchical Bayesian model**, or **multi-level model**, which can visualize like this: $\boldsymbol{\phi} \rightarrow \boldsymbol{\theta} \rightarrow \mathcal{D}$. We assume the prior on the hyper-parameters is fixed (e.g., we may use some kind of minimally informative prior), so the joint distribution has the form

$$p(\boldsymbol{\phi}, \boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\phi})p(\boldsymbol{\theta}|\boldsymbol{\phi})p(\mathcal{D}|\boldsymbol{\theta}) \tag{4.195}$$

The hope is that we can learn the hyperparameters by treating the parameters themselves as datapoints. This is useful when we have multiple related parameters that need to be estimated (e.g., from different subpopulations, or muliple tasks); this provides a learning signal to the top level of the model. See the sequel to this book, [Mur22], for details.

4.6.5.3 Empirical priors

In Sec. 4.6.5.2, we discussed hierarchical Bayes as a way to infer parameters from data. Unfortunately, posterior inference in such models can be computationally challenging. In this section, we discuss a computationally convenient approximation, in which we first compute a point estimate of the hyperparameters, $\hat{\boldsymbol{\phi}}$, and then compute the conditional posterior, $p(\boldsymbol{\theta}|\hat{\boldsymbol{\phi}}, \mathcal{D})$, rather than the joint posterior, $p(\boldsymbol{\theta}, \boldsymbol{\phi}|\mathcal{D})$.

To estimate the hyper-parameters, we can maximize the marginal likelihood:

$$\hat{\boldsymbol{\phi}}_{\text{mml}}(\mathcal{D}) = \operatorname*{argmax}_{\boldsymbol{\phi}} p(\mathcal{D}|\boldsymbol{\phi}) = \operatorname*{argmax}_{\boldsymbol{\phi}} \int p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}|\boldsymbol{\phi})d\boldsymbol{\theta} \tag{4.196}$$

This technique is known as **type II maximum likelihood**, since we are optimizing the hyperparameters, rather than the parameters. Once we have estimated $\hat{\boldsymbol{\phi}}$, we compute the posterior $p(\boldsymbol{\theta}|\hat{\boldsymbol{\phi}}, \mathcal{D})$ in the usual way.

Since we are estimating the prior parameters from data, this approach is **empirical Bayes** (**EB**) [CL96]. This violates the principle that the prior should be chosen independently of the data. However, we can view it as a computationally cheap approximation to inference in the full hierarchical Bayesian model, just as we viewed MAP estimation as an approximation to inference in the one level model $\boldsymbol{\theta} \rightarrow \mathcal{D}$. In fact, we can construct a hierarchy in which the more integrals one performs, the "more Bayesian" one becomes, as shown below.

Method	Definition
Maximum likelihood	$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} p(\mathcal{D}\|\boldsymbol{\theta})$
MAP estimation	$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta}\|\boldsymbol{\phi})$
ML-II (Empirical Bayes)	$\hat{\boldsymbol{\phi}} = \operatorname{argmax}_{\boldsymbol{\phi}} \int p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta}\|\boldsymbol{\phi})d\boldsymbol{\theta}$
MAP-II	$\hat{\boldsymbol{\phi}} = \operatorname{argmax}_{\boldsymbol{\phi}} \int p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta}\|\boldsymbol{\phi})p(\boldsymbol{\phi})d\boldsymbol{\theta}$
Full Bayes	$p(\boldsymbol{\theta}, \boldsymbol{\phi}\|\mathcal{D}) \propto p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta}\|\boldsymbol{\phi})p(\boldsymbol{\phi})$

Note that ML-II is less likely to overfit than "regular" maximum likelihood, because there are typically fewer hyper-parameters $\boldsymbol{\phi}$ than there are parameters $\boldsymbol{\theta}$. See the sequel to this book, [Mur22], for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

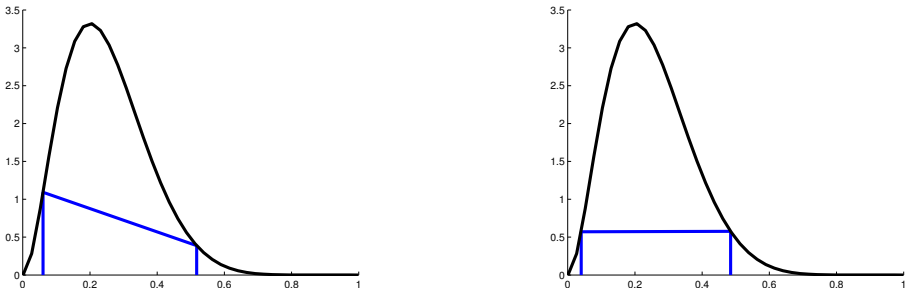


Figure 4.18: (a) Central interval and (b) HPD region for a Beta(3,9) posterior. The CI is (0.06, 0.52) and the HPD is (0.04, 0.48). Adapted from Figure 3.6 of [Hof09]. Generated by code at figures.probml.ai/book1/4.18.

4.6.6 Credible intervals

A posterior distribution is (usually) a high dimensional object that is hard to visualize and work with. A common way to summarize such a distribution is to compute a point estimate, such as the posterior mean or mode, and then to compute a **credible interval**, which quantifies the uncertainty associated with that estimate. (A credible interval is not the same as a confidence interval, which is a concept from frequentist statistics which we discuss in Sec. 4.7.4.)

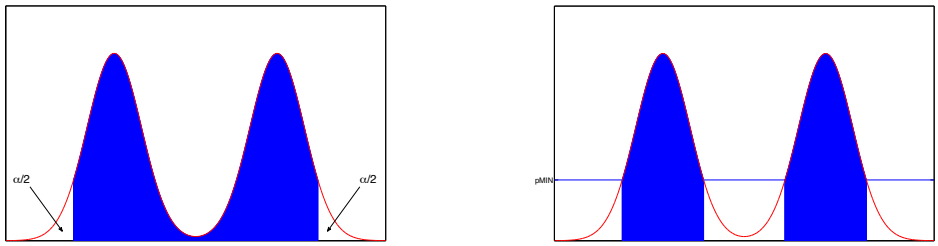
More precisely, we define a $100(1-\alpha)\%$ credible interval to be a (contiguous) region $C = (\ell, u)$ (standing for lower and upper) which contains $1-\alpha$ of the posterior probability mass, i.e.,

$$C_\alpha(\mathcal{D}) = (\ell, u) : P(\ell \leq \theta \leq u|\mathcal{D}) = 1 - \alpha \tag{4.197}$$

There may be many intervals that satisfy Eq. (4.197), so we usually choose one such that there is $(1-\alpha)/2$ mass in each tail; this is called a **central interval**. If the posterior has a known functional form, we can compute the posterior central interval using $\ell = F^{-1}(\alpha/2)$ and $u = F^{-1}(1-\alpha/2)$, where F is the cdf of the posterior, and F^{-1} is the inverse cdf. For example, if the posterior is Gaussian, $p(\theta|\mathcal{D}) = \mathcal{N}(0,1)$, and $\alpha = 0.05$, then we have $\ell = \Phi^{-1}(\alpha/2) = -1.96$, and $u = \Phi^{-1}(1-\alpha/2) = 1.96$, where Φ denotes the cdf of the Gaussian. This is illustrated in Fig. 2.11b. This justifies the common practice of quoting a credible interval in the form of $\mu \pm 2\sigma$, where μ represents the posterior mean, σ represents the posterior standard deviation, and 2 is a good approximation to 1.96.

In general, it is often hard to compute the inverse cdf of the posterior. In this case, a simple alternative is to draw samples from the posterior, and then to use a Monte Carlo approximation to the posterior quantiles: we simply sort the S samples, and find the one that occurs at location α/S along the sorted list. As $S \to \infty$, this converges to the true quantile. See code.probml.ai/book1/beta_credible_int_demo for a demo of this.

A problem with central intervals is that there might be points outside the central interval which have higher probability than points that are inside, as illustratedin Figure 4.18(a). This motivates an alternative quantity known as the **highest posterior density** or **HPD** regionm which is the set of points which have a probability above some threshold. More precisely we find the threshold p^* on



(a) (b)

Figure 4.19: (a) Central interval and (b) HPD region for a hypothetical multimodal posterior. Adapted from Figure 2.2 of [Gel+04]. Generated by code at figures.probml.ai/book1/4.19.

the pdf such that

$$1 - \alpha = \int_{\theta: p(\theta|\mathcal{D}) > p^*} p(\theta|\mathcal{D}) d\theta \tag{4.198}$$

and then define the HPD as

$$C_\alpha(\mathcal{D}) = \{\theta : p(\theta|\mathcal{D}) \geq p^*\} \tag{4.199}$$

In 1d, the HPD region is sometimes called a **highest density interval** or **HDI**. For example, Figure 4.18(b) shows the 95% HDI of a Beta$(3, 9)$ distribution, which is $(0.04, 0.48)$. We see that this is narrower than the central interval, even though it still contains 95% of the mass; furthermore, every point inside of it has higher density than every point outside of it.

For a unimodal distribution, the HDI will be the narrowest interval around the mode containing 95% of the mass. To see this, imagine "water filling" in reverse, where we lower the level until 95% of the mass is revealed, and only 5% is submerged. This gives a simple algorithm for computing HDIs in the 1d case: simply search over points such that the interval contains 95% of the mass and has minimal width. This can be done by 1d numerical optimization if we know the inverse CDF of the distribution, or by search over the sorted data points if we have a bag of samples (see code.probml.ai/book1/betaHPD for some code).

If the posterior is multimodal, the HDI may not even be a connected region: see Figure 4.19(b) for an example. However, summarizing multimodal posteriors is always difficult.

4.6.7 Bayesian machine learning

So far, we have focused on unconditional models of the form $p(\mathbf{y}|\boldsymbol{\theta})$. In supervised machine learning, we use conditional models of the form $p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$. The posterior over the parameters is now $p(\boldsymbol{\theta}|\mathcal{D})$, where $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n) : n = 1 : N\}$. Computing this posterior can be done using the principles we have already discussed. This approach is called **Bayesian machine learning**, since we are "being Bayesian" about the model parameters.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

4.6.7.1 Plugin approximation

Once we have computed the posterior over the parameters, we can compute the posterior predictive distribution over outputs given inputs by marginalizing out the unknown parameters:

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \tag{4.200}$$

Of course, computing this integral is often intractable. A very simple approximation is to assume there is just a single best model, $\hat{\boldsymbol{\theta}}$, such as the MLE. This is equivalent to approximating the posterior as an infinitely narrow, but infinitely tall, "spike" at the chosen value. We can write this as follows:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \delta(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) \tag{4.201}$$

where δ is the Dirac delta function (see Sec. 2.6.5). If we use this approximation, then the predictive distribution can be obtained by simply "plugging in" the point estimate into the likelihood:

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \approx \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \delta(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) d\boldsymbol{\theta} = p(\mathbf{y}|\mathbf{x}, \hat{\boldsymbol{\theta}}) \tag{4.202}$$

This follows from the sifting property of delta functions (Eq. (2.126)).

The approach in Eq. (4.202) is called a **plug-in approximation**. This approach is equivalent to the standard approach used in most of machine learning, in which we first fit the model (i.e. compute a point estimate $\hat{\boldsymbol{\theta}}$) and then use it to make predicitons. However, the standard (plug-in) approach can suffer from overfitting and overconfidence, as we discussed in Sec. 1.2.3. The fully Bayesian approach avoids this by marginalizing out the parameters, but can be expensive. Fortunately, even simple approximations, in which we average over a few plausible parameter values, can improve performance. We give some examples of this below.

4.6.7.2 Example: scalar input, binary output

Suppose we want to perform binary classification, so $y \in \{0, 1\}$. We will use a model of the form

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x} + b)) \tag{4.203}$$

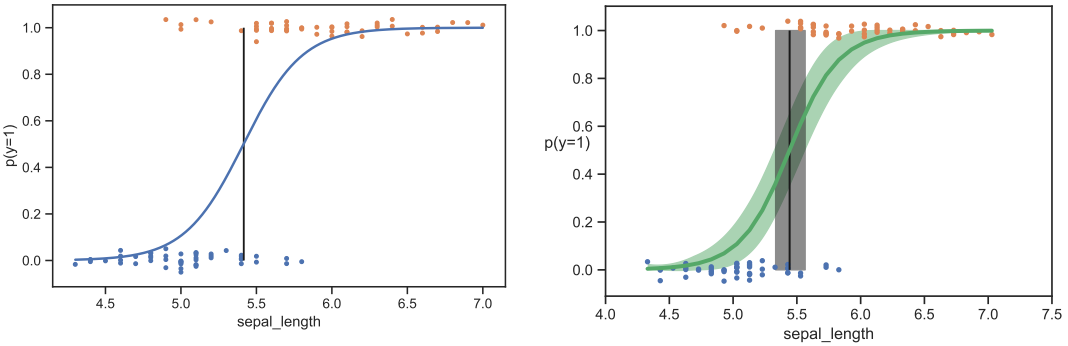
where

$$\sigma(a) \triangleq \frac{e^a}{1 + e^a} \tag{4.204}$$

is the **sigmoid** or **logistic function** which maps $\mathbb{R} \to [0, 1]$, and $\text{Ber}(y|\mu)$ is the Bernoulli distribution with mean μ (see Sec. 2.4 for details). In other words,

$$p(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x} + b}} \tag{4.205}$$

This model is called **logistic regression**. (We discuss this in more detail in Chapter 10.)



(a) (b)

Figure 4.20: (a) Logistic regression for classifying if an Iris flower is versicolor ($y = 1$) or setosa ($y = 0$) using a single input feature x corresponding to sepal length. Labeled points have been (vertically) jittered to avoid overlapping too much. Vertical line is the decision boundary. Generated by code at figures.probml.ai/book1/4.20. (b) Same as (a) but showing posterior distribution. Adapted from Figure 4.4 of [Mar18]. Generated by code at figures.probml.ai/book1/4.20.

Let us apply this model to the task of determining if an iris flower is of type setosa or versicolor, $y_n \in \{0, 1\}$, given information about the sepal length, x_n. (See Sec. 1.2.1.1 for a description of the iris dataset.)

We first fit a 1d logistic regresion model of the following form

$$p(y = 1|x, \boldsymbol{\theta}) = \sigma(b + wx) \tag{4.206}$$

to the dataset $\mathcal{D} = \{(x_n, y_n)\}$ using maximum likelihood estimation. (See Sec. 10.2.3 for details on how to compute the MLE for this model.) Fig. 4.20a shows the plugin approximation to the posterior predictive, $p(y = 1|x, \hat{\boldsymbol{\theta}})$, where $\hat{\boldsymbol{\theta}}$ is the MLE of the parameters. We see that we become more confident that the flower is of type versicolor as the sepal length gets larger, as represented by the sigmoidal (S-shaped) logistic function.

The **decision boundary** is defined to be the input value x^* where $p(y = 1|x^*, \hat{\boldsymbol{\theta}}) = 0.5$. We can solve for this value as follows:

$$\sigma(b + wx^*) = \frac{1}{1 + e^{-(b+wx^*)}} = \frac{1}{2} \tag{4.207}$$

$$b + wx^* = 0 \tag{4.208}$$

$$x^* = -\frac{b}{w} \tag{4.209}$$

From Fig. 4.20a, we see that $x^* \approx 5.5$ cm.

However, the above approach does not model the uncertainty in our estimate of the parameters, and therefore ignores the induced uncertainty in the output probabilities, and the location of the decision boundary. To capture this additional uncertainty, we can use a Bayesian approach to

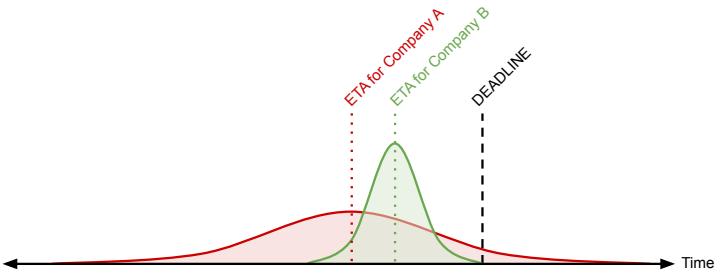


Figure 4.21: Distribution of arrival times for two different shipping companies. ETA is the expected time of arrival. A's distribution has greater uncertainty, and may be too risky. From https://bit.ly/39bc4XL. *Used with kind permission of Brendan Hasz.*

approximate the posterior $p(\boldsymbol{\theta}|\mathcal{D})$. (See Sec. 10.5 for details.) Given this, we can approximate the posterior predictive distribution using a Monte Carlo approximation:

$$p(y=1|x,\mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(y=1|x,\boldsymbol{\theta}^s) \tag{4.210}$$

where $\boldsymbol{\theta}^s \sim p(\boldsymbol{\theta}|\mathcal{D})$ is a posterior sample. Fig. 4.20b plots the mean and 95% credible interval of this function. We see that there is now a range of predicted probabilities for each input. We can also compute a distribution over the location of the decision boundary by using the Monte Carlo approximation

$$p(x^*|\mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} \delta\left(x^* - (-\frac{b^s}{w^s})\right) \tag{4.211}$$

where $(b^s, w^s) = \boldsymbol{\theta}^s$. The 95% credible interval for this distribution is shown by the "fat" vertical line in Fig. 4.20b.

Although carefully modeling our uncertainty may not matter for this application, it can be important in risk-sensitive applications, such as health care and finance, as we discuss in Chapter 5.

4.6.7.3 Example: binary input, scalar output

Now suppose we want to predict the delivery time for a package, $y \in \mathbb{R}$, if shipped by company A vs B. We can encode the company id using a binary feature $x \in \{0, 1\}$, where $x = 0$ means company A and $x = 1$ means company B. We will use the following discriminative model for this problem:

$$p(y|x,\boldsymbol{\theta}) = \mathcal{N}(y|\mu_x, \sigma_x^2) \tag{4.212}$$

where $\mathcal{N}(y|\mu,\sigma^2)$ is the Gaussian distribution

$$\mathcal{N}(y|\mu,\sigma^2) \triangleq \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y-\mu)^2} \tag{4.213}$$

and $\boldsymbol{\theta} = (\mu_0, \mu_1, \sigma_0, \sigma_1)$ are the parameters of the model. We can fit this model using maximum likelihood estimation as we discuss in Sec. 4.2.5; alternatively, we can adopt a Bayesian approach, as we discuss in Sec. 4.6.4.

The advantage of the Bayesian approach is that by capturing uncertainty in the parameters $\boldsymbol{\theta}$, we also capture uncertainty in our forecasts $p(y|x, \mathcal{D})$, whereas using a plug-in approximation $p(y|x, \hat{\boldsymbol{\theta}})$ would underestimate this uncertainty. For example, suppose we have only used each company once, so our training set has the form $\mathcal{D} = \{(x_1 = 0, y_1 = 15), (x_2 = 1, y_2 = 20)\}$. As we show in Sec. 4.2.5, the MLE for the means will be the empirical means, $\hat{\mu}_0 = 15$ and $\hat{\mu}_1 = 20$, but the MLE for the standard deviations will be zero, $\hat{\sigma}_0 = \hat{\sigma}_1 = 0$, since we only have a single sample from each "class". The resulting plug-in prediction will therefore not capture any uncertainty.

To see why modeling the uncertainty is important, consider Fig. 4.21. We see that the expected time of arrival (ETA) for company A is less than for company B; however, the variance of A's distribution is larger, which makes it a risky choice if you want to be confident the package will arrive by the specified deadline. (For more details on how to choose optimal actions in the presence of uncertainty, see Chapter 5.)

Of course, the above example is extreme, because we assumed we only had one example from each delivery company. However, this kind of problem occurs whenever we have few examples of a given kind of input, as can happen whenever the data has a long tail of novel patterns, such as a new combination of words or categorical features.

4.6.7.4 Scaling up

The above examples were both extremely simple, involving 1d input and 1d output, and just 2–4 parameters. Most practical problems involve high dimensional inputs, and sometimes high dimensional outputs, and therefore use models with lots of parameters. Unfortunately, computing the posterior, $p(\boldsymbol{\theta}|\mathcal{D})$, and the posterior predictive, $p(\mathbf{y}|\mathbf{x}, \mathcal{D})$, can be computationally challenging for many models. We discuss this issue in Sec. 4.6.8.

4.6.8 Computational issues

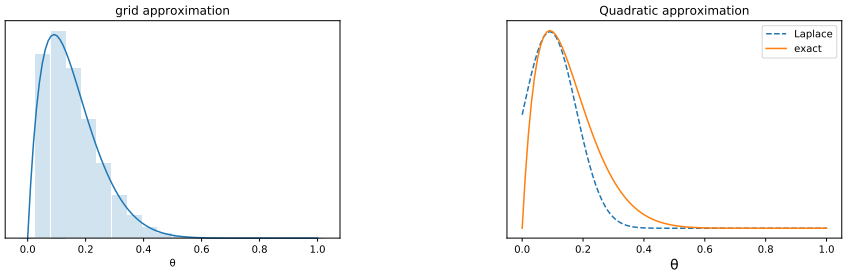
Given a likelihood $p(\mathcal{D}|\boldsymbol{\theta})$ and a prior $p(\boldsymbol{\theta}|\boldsymbol{\phi})$ with (fixed) hyperparameters $\boldsymbol{\phi}$, we can compute the posterior $p(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{\phi})$ using Bayes' rule. However, actually performing this computation is usually intractable, except for simple special cases, such as conjugate models (Sec. 4.6.1), or models where all the latent variables come from a small finite set of possible values. We therefore need to approximate the posterior. There are a large variety of methods for performing **approximate posterior inference**, which trade off accuracy, simplicity, and speed. We briefly discuss some of these algorithms below, but go into more detail in the sequel to this book, [Mur22].

As a running example, we will use the problem of approximating the posterior of a beta-Bernoulli model. Specifically, the goal is to approximate

$$p(\theta|\mathcal{D}, \boldsymbol{\phi}) \propto \left[\prod_{n=1}^{N} \text{Bin}(y_n|\theta)\right] \text{Beta}(\theta|\boldsymbol{\phi}) \tag{4.214}$$

where $\mathcal{D}$ consists of 10 heads and 1 tail (so the total number of observations is $N = 11$). We will use a uniform prior, so $\boldsymbol{\phi} = (1, 1)$. Although we can compute this posterior exactly (see Fig. 4.22), using

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 4.22: Approximating the posterior of a beta-Bernoulli model. (a) Grid approximation using 20 grid points. (b) Laplace approximation. Generated by code at figures.probml.ai/book1/4.22.

the method discussed in Sec. 4.6.2, this serves as a useful pedagogical example since we can compare the approximation to the exact answer. Also, since the target distribution is just 1d, it is easy to visualize the results. (Note, however, that the problem is not completely trivial, since the posterior is highly skewed, due to the use of an imbalanced sample of 10 heads and 1 tail.)

4.6.8.1 Grid approximation

The simplest approach to approximate posterior inference is to partition the space of possible values for the unknowns into a finite set of possibilities, call them $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K$, and then to approximate the posterior by brute-force enumeration, as follows:

$$p(\boldsymbol{\theta} = \boldsymbol{\theta}_k|\mathcal{D}, \boldsymbol{\phi}) \approx \frac{p(\mathcal{D}|\boldsymbol{\theta}_k)p(\boldsymbol{\theta}_k)}{p(\mathcal{D})} = \frac{p(\mathcal{D}|\boldsymbol{\theta}_k)p(\boldsymbol{\theta}_k)}{\sum_{k'=1}^{K} p(\mathcal{D}, \boldsymbol{\theta}_{k'})} \tag{4.215}$$

This is called a **grid approximation**. In Fig. 4.22a, we illustrate this method applied to our 1d problem. We see that it is easily able to capture the skewed posterior. Unfortunately, this approach does not scale to problems in more than 2 or 3 dimensions, because the number of grid points grows exponentially with the number of dimensions.

4.6.8.2 Quadratic (Laplace) approximation

In this section, we discuss a simple way to approximate the posterior using a multivariate Gaussian; this is known as the **Laplace approximation** (see e.g., [TK86; RMC09]), or a **quadratic approximation** [McE20].

To derive this, suppose we write the posterior as follows:

$$p(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{\phi}) = \frac{1}{Z} e^{-\mathcal{E}(\boldsymbol{\theta}|\boldsymbol{\phi})} \tag{4.216}$$

where $\mathcal{E}(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta}, \mathcal{D})$ is called an energy function, and $Z = p(\mathcal{D})$ is the normalization constant. Performing a Taylor series expansion around the mode $\hat{\boldsymbol{\theta}}$ (i.e., the lowest energy state) we get

$$\mathcal{E}(\boldsymbol{\theta}|\boldsymbol{\phi}) \approx \mathcal{E}(\hat{\boldsymbol{\theta}}|\boldsymbol{\phi}) + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \mathbf{g} + \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \mathbf{H}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) \tag{4.217}$$

where $\mathbf{g}$ is the gradient at the mode, and $\mathbf{H}$ is the Hessian. Since $\hat{\boldsymbol{\theta}}$ is the mode, the gradient term is zero. Hence

$$\hat{p}(\boldsymbol{\theta}, \mathcal{D}|\boldsymbol{\phi}) = e^{-\mathcal{E}(\hat{\boldsymbol{\theta}}|\boldsymbol{\phi})} \exp\left[-\frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \mathbf{H}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})\right] \tag{4.218}$$

$$\hat{p}(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{\phi}) = \frac{1}{Z}\hat{p}(\boldsymbol{\theta}, \mathcal{D}|\boldsymbol{\phi}) = \mathcal{N}(\boldsymbol{\theta}|\hat{\boldsymbol{\theta}}, \mathbf{H}^{-1}) \tag{4.219}$$

$$Z = e^{-\mathcal{E}(\hat{\boldsymbol{\theta}}|\boldsymbol{\phi})}(2\pi)^{D/2}|\mathbf{H}|^{-\frac{1}{2}} \tag{4.220}$$

The last line follows from normalization constant of the multivariate Gaussian.

The Laplace approximation is easy to apply, since we can leverage existing optimization algorithms to compute the MAP estimate, and then we just have to compute the Hessian at the mode. (In high dimensional spaces, we can use a diagonal approximation.)

In Fig. 4.22b, we illustrate this method applied to our 1d problem. Unfortunately we see that it is not a particularly good approximation. This is because the posterior is skewed, whereas a Gaussian is symmetric. In addition, the parameter of interest lies in the constrained interval $\theta \in [0, 1]$, whereas the Gaussian assumes an unconstrained space, $\boldsymbol{\theta} \in \mathbb{R}$. Fortunately, we can solve this latter problem by using a change of variable. For example, in this case we can apply the Laplace approximation to $\alpha = \text{logit}(\theta)$. This is a common trick to simplify the job of inference.

4.6.8.3 Variational approximation

In Sec. 4.6.8.2, we discussed the Laplace approximation, which uses an optimization procedure to find the MAP estimate, and then approximates the curvature of the posterior at that point based on the Hessian. In this section, we discuss **variational inference** (**VI**), which is another optimization-based approach to posterior inference, but which has much more modeling flexibility (and thus can give a much more accurate approximation).

VI attempts to approximate an intractable probability distribution, such as $p(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{\phi})$, with one that is tractable, $q(\boldsymbol{\theta})$, so as to minimize some discrepancy D between the distributions:

$$q^* = \underset{q \in \mathcal{Q}}{\text{argmin}}\, D(q, p) \tag{4.221}$$

where $\mathcal{Q}$ is some tractable family of distributions (e.g., multivariate Gaussian). If we define D to be the KL divergence (see Sec. 6.2), then we can derive a lower bound to the log marginal likelihood; this quantity is known as the **evidence lower bound** or **ELBO**. By maximizing th ELBO, we can improve the quality of the posterior approximation. See the sequel to this book, [Mur22], for details.

4.6.8.4 Markov Chain Monte Carlo (MCMC) approximation

Although VI is a fast, optimization-based method, it can give a biased approximation to the posterior, since it is restricted to a specific function form $q \in \mathcal{Q}$. A more flexible approach is to use a non-parametric approximation in terms of a set of samples, $q(\boldsymbol{\theta}) \approx \frac{1}{S}\sum_{s=1}^{S} \delta(\boldsymbol{\theta} - \boldsymbol{\theta}^s)$. This is called a **Monte Carlo approximation** to the posterior. The key issue is how to create the posterior samples $\boldsymbol{\theta}^s \sim p(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{\phi})$ efficiently, without having to evaluate the normalization constant $p(\mathcal{D}|\boldsymbol{\phi}) = \int p(\boldsymbol{\theta}, \mathcal{D}|\boldsymbol{\phi}) d\boldsymbol{\theta}$. A common approach to this problem is known as **Markov chain Monte Carlo** or **MCMC**. If we augment this algorithm with gradient-based information, derived from

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\nabla \log p(\boldsymbol{\theta}, \mathcal{D}|\boldsymbol{\phi})$, we can significantly speed up the method; this is called **Hamiltonian Monte Carlo** or **HMC**. See the sequel to this book, [Mur22], for details.

4.7 Frequentist statistics *

The approach to statistical inference that we described in Sec. 4.6 is called Bayesian statistics. It treats parameters of models just like any other unknown random variable, and applies the rules of probability theory to infer them from data. Attempts have been made to devise approaches to statistical inference that avoid treating parameters like random variables, and which thus avoid the use of priors and Bayes rule. This alternative approach is known as **frequentist statistics**, **classical statistics** or **orthodox statistics**.

The basic idea (formalized in Sec. 4.7.1) is to to represent uncertainty by calculating how a quantity estimated from data (such as a parameter or a predicted label) would change if the data were changed. It is this notion of variation across repeated trials that forms the basis for modeling uncertainty used by the frequentist approach. By contrast, the Bayesian approach views probability in terms of information rather than repeated trials. This allows the Bayesian to compute the probability of one-off events, as we discussed in Sec. 2.1.1. Perhaps more importantly, the Bayesian approach avoids certain paradoxes that plague the frequentist approach (see Sec. 4.7.5 and Sec. 5.7.4). These pathologies led the famous statistician George Box to say:

> I believe that it would be very difficult to persuade an intelligent person that current [frequentist] statistical practice was sensible, but that there would be much less difficulty with an approach via likelihood and Bayes' theorem. — George Box, 1962 (quoted in [Jay76]).

Nevertheless, it is useful to be familiar with frequentist statistics, since it is widely used, and has some key concepts that are useful even for Bayesians [Rub84].

4.7.1 Sampling distributions

In frequentist statistics, uncertainty is not represented by the posterior distribution of a random variable, but instead by the **sampling distribution** of an **estimator**. Let us now define these terms,

As explained in the section on decision theory in Sec. 5.1, an estimator is a decision procedure that specifies what action to take given some observed data. In the context of parameter estimation, where the action space is to return a parameter vector, we will denote this by $\hat{\boldsymbol{\theta}} = \pi(\mathcal{D})$. For example, $\hat{\boldsymbol{\theta}}$ could be the maximum likelihood estimate, the MAP estimate, or the method of moments estimate.

The sampling distribution of an estimator is the distribution of results we would see if we applied the estimator multiple times to different datasets sampled from some distribution; in the context of parameter estimation, it is the distribution of $\hat{\boldsymbol{\theta}}$, viewed as a random variable that depends on the random sample $\mathcal{D}$. In more detail, imagine sampling S different data sets, each of size N, from some true model $p(\mathbf{x}|\boldsymbol{\theta}^*)$ to generate

$$\tilde{\mathcal{D}}^{(s)} = \{\mathbf{x}_n \sim p(\mathbf{x}_n|\boldsymbol{\theta}^*) : n = 1 : N\} \tag{4.222}$$

We denote this by $\mathcal{D}^{(s)} \sim \boldsymbol{\theta}^*$ for brevity. Now apply the estimator to each $\mathcal{D}^{(s)}$ to get a set of estimates, $\{\hat{\boldsymbol{\theta}}(\mathcal{D}^{(s)})\}$. As we let $S \to \infty$, the distribution induced by this set is the sampling distribution of the

estimator. More precisely, we have

$$p(\pi(\tilde{\mathcal{D}}) = \boldsymbol{\theta}|\tilde{\mathcal{D}} \sim \boldsymbol{\theta}^*) \approx \frac{1}{S}\sum_{s=1}^{S} \delta(\boldsymbol{\theta} = \pi(\mathcal{D}^{(s)})) \tag{4.223}$$

In some cases, we can compute this analytically, as we discuss in Sec. 4.7.2, although typically we need to approximate it by Monte Carlo, as we discuss in Sec. 4.7.3.

4.7.2 Gaussian approximation of the sampling distribution of the MLE

The most common estimator is the MLE. When the sample size becomes large, the sampling distribution of the MLE for certain models becomes Gaussian. This is known as the **asymptotic normality** of the sampling distribution. More formally, we have the following result:

Theorem 4.7.1. *If the parameters are identifiable, then*

$$p(\pi(\tilde{\mathcal{D}}) = \hat{\boldsymbol{\theta}}|\tilde{\mathcal{D}} \sim \boldsymbol{\theta}^*) \to \mathcal{N}(\hat{\boldsymbol{\theta}}|\boldsymbol{\theta}^*, (N\mathbf{F}(\boldsymbol{\theta}^*))^{-1}) \tag{4.224}$$

where $\mathbf{F}(\boldsymbol{\theta}^*)$ *is the* **Fisher information matrix**, *defined in Eq.* (4.225).

The Fisher information matrix measures the amount of curvature of the log-likelihood surface at its peak, as we show below.

More formally, the **Fisher information matrix** (**FIM**) is defined to be the covariance of the gradient of the log likelihood (also called the **score function**):

$$\mathbf{F}(\boldsymbol{\theta}) \triangleq \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x}|\boldsymbol{\theta})}\left[\nabla \log p(\mathbf{x}|\boldsymbol{\theta}) \nabla \log p(\mathbf{x}|\boldsymbol{\theta})^\top\right] \tag{4.225}$$

Hnce the (i,j)'th entry has the form

$$F_{ij} = \mathbb{E}_{\mathbf{x}\sim\boldsymbol{\theta}}\left[\left(\frac{\partial}{\partial\theta_i}\log p(\mathbf{x}|\boldsymbol{\theta})\right)\left(\frac{\partial}{\partial\theta_j}\log p(\mathbf{x}|\boldsymbol{\theta})\right)\right] \tag{4.226}$$

One can show the following result.

Theorem 4.7.2. *If* $\log p(\mathbf{x}|\boldsymbol{\theta})$ *is twice differentiable, and under certain regularity conditions, the FIM is equal to the expected Hessian of the NLL, i.e.,*

$$\mathbf{F}(\boldsymbol{\theta})_{ij} \triangleq \mathbb{E}_{\mathbf{x}\sim\boldsymbol{\theta}}\left[\left(\frac{\partial}{\partial\theta_i}\log p(\mathbf{x}|\boldsymbol{\theta})\right)\left(\frac{\partial}{\partial\theta_j}\log p(\mathbf{x}|\boldsymbol{\theta})\right)\right] = -\mathbb{E}_{\mathbf{x}\sim\boldsymbol{\theta}}\left[\frac{\partial^2}{\partial\theta_i\theta_j}\log p(\mathbf{x}|\boldsymbol{\theta})\right] \tag{4.227}$$

Thus we can interpret the FIM as the Hessian of the NLL.

This helps us understand the result in Eq. (4.224): a log-likelihood function with high curvature (large Hessian) will result in a low variance estimate, since the parameters are "well determined" by the data, and hence robust to repeated sampling.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

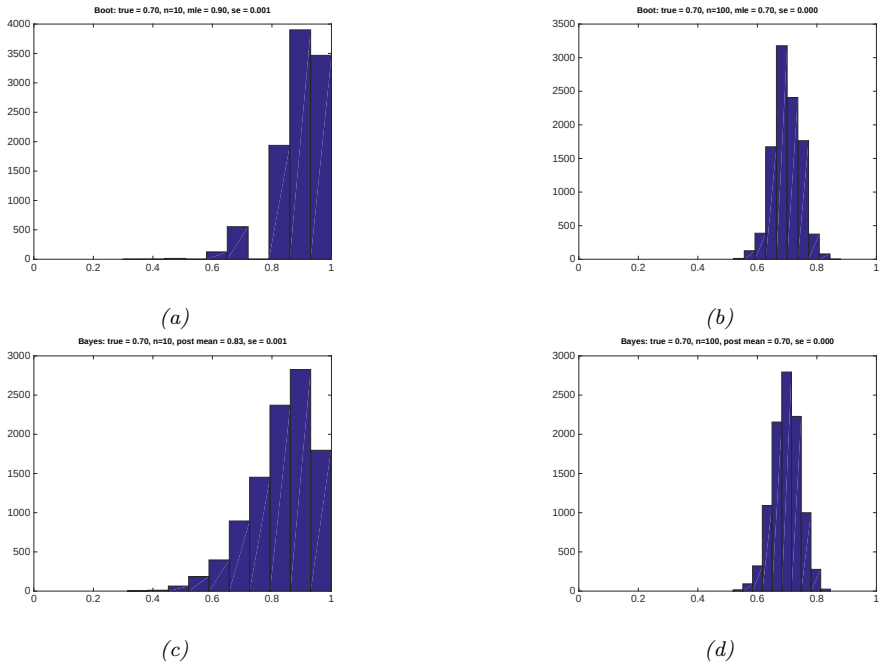


Figure 4.23: Bootstrap (top row) vs Bayes (bottom row). The N data cases were generated from $\text{Ber}(\theta = 0.7)$. *Left column: $N = 10$. Right column: $N = 100$. (a-b) A bootstrap approximation to the sampling distribution of the MLE for a Bernoulli distribution. We show the histogram derived from $B = 10,000$ bootstrap samples. (c-d) Histogram of 10,000 samples from the posterior distribution using a uniform prior. Generated by code at figures.probml.ai/book1/4.23.*

4.7.3 Bootstrap approximation of the sampling distribution of any estimator

In cases where the estimator is a complex function of the data (e.g., not just an MLE), or when the sample size is small, we can approximate its sampling distribution using a Monte Carlo technique known as the **bootstrap**.

The idea is simple. If we knew the true parameters $\boldsymbol{\theta}^*$, we could generate many (say S) fake datasets, each of size N, from the true distribution, using $\tilde{\mathcal{D}}^{(s)} = \{\mathbf{x}_n \sim p(\mathbf{x}_n|\boldsymbol{\theta}^*) : n = 1 : N\}$. We could then compute our estimator from each sample, $\hat{\boldsymbol{\theta}}^s = \pi(\tilde{\mathcal{D}}^{(s)})$ and use the empirical distribution of the resulting $\hat{\boldsymbol{\theta}}^s$ as our estimate of the sampling distribution, as in Eq. (4.223). Since $\boldsymbol{\theta}^*$ is unknown, the idea of the **parametric bootstrap** is to generate each sampled dataset using $\hat{\boldsymbol{\theta}} = \pi(\mathcal{D})$ instead of $\boldsymbol{\theta}^*$, i.e., we use $\tilde{\mathcal{D}}^{(s)} = \{\mathbf{x}_n \sim p(\mathbf{x}_n|\hat{\boldsymbol{\theta}}) : n = 1 : N\}$ in Eq. (4.223). This is a plug-in approximation to the sampling distribution.

An alternative, called the **non-parametric bootstrap**, is to sample N data points from the original data with replacement. This creates a new distribution $\mathcal{D}^{(s)}$ which has the same size as the original. However, the number of unique data points in a bootstrap sample is just $0.632 \times N$, on average. (To see this, note that the probability an item is picked at least once is $(1 - (1 - 1/N)^N)$, which approaches $1 - e^{-1} \approx 0.632$ for large N.)

Fig. 4.23(a-b) shows an example where we compute the sampling distribution of the MLE for a Bernoulli using the parametric bootstrap. (Results using the non-parametric bootstrap are essentially the same.) When $N = 10$, we see that the sampling distribution is asymmetric, and therefore quite far from Gaussian, but when $N = 100$, the distribution looks more Gaussian, as theory suggests (see Sec. 4.7.2).

4.7.3.1 Bootstrap is a "poor man's" posterior

A natural question is: what is the connection between the parameter estimates $\hat{\boldsymbol{\theta}}^s = \pi(\mathcal{D}^{(s)})$ computed by the bootstrap and parameter values sampled from the posterior, $\boldsymbol{\theta}^s \sim p(\cdot|\mathcal{D})$? Conceptually they are quite different. But in the common case that the estimator is MLE and the prior is not very strong, they can be quite similar. For example, Fig. 4.23(c-d) shows an example where we compute the posterior using a uniform Beta(1,1) prior, and then sample from it. We see that the posterior and the sampling distribution are quite similar. So one can think of the bootstrap distribution as a "poor man's" posterior [HTF01, p235].

However, perhaps surprisingly, bootstrap can be slower than posterior sampling. The reason is that the bootstrap has to generate S sampled datasets, and then fit a model to each one. By contrast, in posterior sampling, we only have to "fit" a model once given a single dataset. (Some methods for speeding up the bootstrap when applied to massive data sets are discussed in [Kle+11].)

4.7.4 Confidence intervals

In frequentist statistics, we use the variability induced by the sampling distribution as a way to estimate uncertainty of a parameter estimate. More precisely, we define a $100(1-\alpha)\%$ **confidence interval** for a parameter estimate θ as any interval $I(\tilde{\mathcal{D}}) = (\ell(\tilde{\mathcal{D}}), u(\tilde{\mathcal{D}}))$ derived from a hypothetical dataset $\tilde{\mathcal{D}}$ such that

$$\Pr(\theta \in I(\tilde{\mathcal{D}})|\tilde{\mathcal{D}} \sim \theta) = 1 - \alpha \tag{4.228}$$

It is common to set $\alpha = 0.05$, which yields a 95% CI. This means that, if we repeatedly sampled data, and compute $I(\tilde{\mathcal{D}})$ for each such dataset, then about 95% of such intervals will contain the true parameter θ.

Note, however, that Eq. (4.228) does *not* mean that for any particular dataset that $\theta \in I(\mathcal{D})$ with 95% probability; this is what a Bayesian credible interval computes (Sec. 4.6.6), but is not what a frequentist confidence interval computes. For more details on this important distinction, see Sec. 4.7.5.

Let us put aside such "philosophical" concerns, and discuss how to compute a confidence interval. Suppose that $\hat{\theta}$ is an estimate of the parameter θ. Let θ^* be its true but unknown value. Also, suppose that the sampling distribution of $\Delta = \hat{\theta} - \theta^*$ is known. Let $\underline{\delta}$ and $\overline{\delta}$ denote its $\alpha/2$ and $1 - \alpha/2$ quantiles. Hence

$$\Pr(\underline{\delta} \leq \hat{\theta} - \theta^* \leq \overline{\delta}) = 1 - \alpha \tag{4.229}$$

Rearranging we get

$$\Pr(\hat{\theta} - \overline{\delta} \leq \theta^* \leq \hat{\theta} - \underline{\delta}) = 1 - \alpha \tag{4.230}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

And hence

$$I(\tilde{\mathcal{D}}) = (\hat{\theta}(\tilde{\mathcal{D}}) - \overline{\delta}(\tilde{\mathcal{D}}), \hat{\theta}(\tilde{\mathcal{D}}) + \underline{\delta}(\tilde{\mathcal{D}})) \tag{4.231}$$

is a $100(1-\alpha)\%$ confidence interval.

In some cases, we can analytically compute the distribution of $\Delta = \hat{\theta} - \theta^*$. This can be used to derive exact confidence intervals. However, it is more common to assume a Gaussian approximation to the sampling distribution, as in Sec. 4.7.2. In this case, we have $\sqrt{NF(\hat{\theta})}(\hat{\theta} - \theta^*) \sim \mathcal{N}(0,1)$. Hence we can compute an approximate CI using

$$\hat{\theta} \pm z_{\alpha/2}\hat{\text{se}} \tag{4.232}$$

where $z_{\alpha/2}$ is the $\alpha/2$ quantile of the Gaussian cdf, and $\hat{\text{se}} = 1/\sqrt{NF(\hat{\theta})}$ is the estimated standard error. If we set $\alpha = 0.05$, we have $z_{\alpha/2} = 1.96$, which justifies the common approximation $\hat{\theta} \pm 2\hat{\text{se}}$.

If the Gaussian approximation is not a good one, we can use a bootstrap approximation (see Sec. 4.7.3). In particular, we sample S datasets from $\hat{\theta}(\mathcal{D})$, and apply the estimator to each one to get $\hat{\theta}(\mathcal{D}^{(s)})$; we then use the empirical distribution of $\hat{\theta}(\mathcal{D}) - \hat{\theta}(\mathcal{D}^{(s)})$ as an approximation to the sampling distribution of Δ.

4.7.5 Caution: Confidence intervals are not credible

A 95% frequentist confidence interval for a parameter θ is defined as any interval $I(\tilde{\mathcal{D}})$ such that $\Pr(\theta \in I(\tilde{\mathcal{D}})|\tilde{\mathcal{D}} \sim \theta) = 0.95$, as we explain in Sec. 4.7.4. This does *not* mean that the parameter is 95% likely to live inside this interval given the observed data. That quantity — which is usually what we want to compute — is instead given by the Bayesian credible interval $p(\theta \in I|\mathcal{D})$, as we explain in Sec. 4.6.6. These concepts are quite different: In the frequentist approach, θ is treated as an unknown fixed constant, and the data is treated as random. In the Bayesian approach, we treat the data as fixed (since it is known) and the parameter as random (since it is unknown).

This counter-intuitive definition of confidence intervals can lead to bizarre results. Consider the following example from [Ber85, p11]. Suppose we draw two integers $\mathcal{D} = (y_1, y_2)$ from

$$p(y|\theta) = \begin{cases} 0.5 & \text{if } y = \theta \\ 0.5 & \text{if } y = \theta + 1 \\ 0 & \text{otherwise} \end{cases} \tag{4.233}$$

If $\theta = 39$, we would expect the following outcomes each with probability 0.25:

$$(39,39), (39,40), (40,39), (40,40) \tag{4.234}$$

Let $m = \min(y_1, y_2)$ and define the following interval:

$$[\ell(\mathcal{D}), u(\mathcal{D})] = [m, m] \tag{4.235}$$

For the above samples this yields

$$[39,39], \quad [39,39], \quad [39,39], \quad [40,40] \tag{4.236}$$

Hence Eq. (4.235) is clearly a 75% CI, since 39 is contained in 3/4 of these intervals. However, if we observe $\mathcal{D} = (39, 40)$ then $p(\theta = 39|\mathcal{D}) = 1.0$, so we know that θ must be 39, yet we only have 75% "confidence" in this fact. We see that the CI will "cover" the true parameter 75% of the time, if we compute multiple CIs from different randomly sampled datasets, but if we just have a single observed dataset, and hence a single CI, then the frequentist "coverage" probability can be very misleading.

Another, less contrived, example is as follows. Suppose we want to estimate the parameter θ of a Bernoulli distribution. Let $\overline{y} = \frac{1}{N}\sum_{n=1}^{N} y_n$ be the sample mean. The MLE is $\hat{\theta} = \overline{y}$. An approximate 95% confidence interval for a Bernoulli parameter is $\overline{y} \pm 1.96\sqrt{\overline{y}(1-\overline{y})/N}$ (this is called a **Wald interval** and is based on a Gaussian approximation to the Binomial distribution; compare to Eq. (4.133)). Now consider a single trial, where $N = 1$ and $y_1 = 0$. The MLE is 0, which overfits, as we saw in Sec. 4.5.1. But our 95% confidence interval is also $(0, 0)$, which seems even worse. It can be argued that the above flaw is because we approximated the true sampling distribution with a Gaussian, or because the sample size was too small, or the parameter "too extreme". However, the Wald interval can behave badly even for large N, and non-extreme parameters [BCD01]. By contrast, a Bayesian credible interval with a non-informative Jeffreys prior behaves in the way we would expect.

Several more interesting examples, along with Python code, can be found at [Van14]. See also [Hoe+14; Mor+16; Lyu+20; Cha+19b], who show that many people, including professional statisticians, misunderstand and misuse frequentist confidence intervals in practice, whereas Bayesian credible intervals do not suffer from these problems.

4.7.6 The bias-variance tradeoff

An estimator is a procedure applied to data which returns an estimand. Let $\hat{\boldsymbol{\theta}}()$ be the estimator, and $\hat{\boldsymbol{\theta}}(\mathcal{D})$ be the estimand. In frequentist statistics, we treat the data as a random variable, drawn from some true but unknown distribution, $p^*(\mathcal{D})$; this induces a distribution over the estimand, $p^*(\hat{\boldsymbol{\theta}}(\mathcal{D}))$, known as the sampling distribution (see Sec. 4.7.1). In this section, we discuss two key properties of this distribution, its bias and its variance, which we define below.

4.7.6.1 Bias of an estimator

The **bias** of an estimator is defined as

$$\text{bias}(\hat{\theta}(\cdot)) \triangleq \mathbb{E}\left[\hat{\theta}(\mathcal{D})\right] - \theta^* \tag{4.237}$$

where θ^* is the true parameter value, and the expectation is wrt "nature's distribution" $p(\mathcal{D}|\theta^*)$. If the bias is zero, the estimator is called **unbiased**. For example, the MLE for a Gaussian mean is unbiased:

$$\text{bias}(\hat{\mu}) = \mathbb{E}\left[\overline{x}\right] - \mu = \mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N} x_n\right] - \mu = \frac{N\mu}{N} - \mu = 0 \tag{4.238}$$

where $\overline{x}$ is the sample mean.

However, the MLE for a Gaussian variance, $\sigma^2_{\text{mle}} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \overline{x})^2$, is not an unbiased estimator of σ^2. In fact, one can show (Exercise 4.7) that

$$\mathbb{E}\left[\sigma^2_{\text{mle}}\right] = \frac{N-1}{N}\sigma^2 \tag{4.239}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

so the ML estimator slightly underestimates the variance. Intuitively, this is because we "use up" one of the data points to estimate the mean, so if we have a sample size of 1, we will estimate the variance to be 0. If, however, μ is known, the ML estimator is unbiased (see Exercise 4.8).

Now consider the following estimator

$$\sigma^2_{\text{unb}} \triangleq \frac{1}{N-1}\sum_{n=1}^{N}(x_n - \overline{x})^2 = \frac{N}{N-1}\sigma^2_{\text{mle}} \tag{4.240}$$

This is an unbiased estimator, which we can easily prove as follows:

$$\mathbb{E}\left[\sigma^2_{\text{unb}}\right] = \frac{N}{N-1}\mathbb{E}\left[\sigma^2_{\text{mle}}\right] = \frac{N}{N-1}\frac{N-1}{N}\sigma^2 = \sigma^2 \tag{4.241}$$

4.7.6.2 Variance of an estimator

It seems intuitively reasonable that we want our estimator to be unbiased. However, being unbiased is not enough. For example, suppose we want to estimate the mean of a Gaussian from $\mathcal{D} = \{x_1, \ldots, x_N\}$. The estimator that just looks at the first data point, $\hat{\theta}(\mathcal{D}) = x_1$, is an unbiased estimator, but will generally be further from θ^* than the empirical mean $\overline{x}$ (which is also unbiased). So the variance of an estimator is also important.

We define the variance of an estimator as follows:

$$\mathbb{V}\left[\hat{\theta}\right] \triangleq \mathbb{E}\left[\hat{\theta}^2\right] - \left(\mathbb{E}\left[\hat{\theta}\right]\right)^2 \tag{4.242}$$

where the expectation is taken wrt $p(\mathcal{D}|\theta^*)$. This measures how much our estimate will change as the data changes. We can extend this to a covariance matrix for vector valued estimators.

4.7.6.3 The bias-variance tradeoff

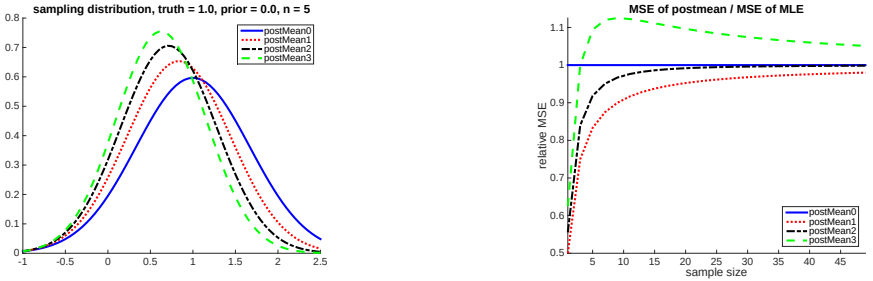
In this section, we discuss a fundamental tradeoff that needs to be made when picking a method for parameter estimation, assuming our goal is to minimize the mean squared error (MSE) of our estimate. Let $\hat{\theta} = \hat{\theta}(\mathcal{D})$ denote the estimate, and $\overline{\theta} = \mathbb{E}\left[\hat{\theta}\right]$ denote the expected value of the estimate (as we vary $\mathcal{D}$). (All expectations and variances are wrt $p(\mathcal{D}|\theta^*)$, but we drop the explicit conditioning for notational brevity.) Then we have

$$\mathbb{E}\left[(\hat{\theta} - \theta^*)^2\right] = \mathbb{E}\left[\left[(\hat{\theta} - \overline{\theta}) + (\overline{\theta} - \theta^*)\right]^2\right] \tag{4.243}$$

$$= \mathbb{E}\left[\left(\hat{\theta} - \overline{\theta}\right)^2\right] + 2(\overline{\theta} - \theta^*)\mathbb{E}\left[\hat{\theta} - \overline{\theta}\right] + (\overline{\theta} - \theta^*)^2 \tag{4.244}$$

$$= \mathbb{E}\left[\left(\hat{\theta} - \overline{\theta}\right)^2\right] + (\overline{\theta} - \theta^*)^2 \tag{4.245}$$

$$= \mathbb{V}\left[\hat{\theta}\right] + \text{bias}^2(\hat{\theta}) \tag{4.246}$$



(a) *(b)*

Figure 4.24: Left: Sampling distribution of the MAP estimate (equivalent to the posterior mean) under a $\mathcal{N}(\theta_0 = 0, \sigma^2/\kappa_0)$ prior with different prior strengths κ_0. (If we set $\kappa = 0$, the MAP estimate reduces to the MLE.) The data is $n = 5$ samples drawn from $\mathcal{N}(\theta^* = 1, \sigma^2 = 1)$. Right: MSE relative to that of the MLE versus sample size. Adapted from Figure 5.6 of [Hof09]. Generated by code at figures.probml.ai/book1/4.24.

In words,

$$\boxed{\text{MSE} = \text{variance} + \text{bias}^2} \tag{4.247}$$

This is called the **bias-variance tradeoff** (see e.g., [GBD92]). What it means is that it might be wise to use a biased estimator, so long as it reduces our variance by more than the square of the bias, assuming our goal is to minimize squared error.

4.7.6.4 Example: MAP estimator for a Gaussian mean

Let us give an example, based on [Hof09, p79]. Suppose we want to estimate the mean of a Gaussian from $\mathbf{x} = (x_1, \ldots, x_N)$. We assume the data is sampled from $x_n \sim \mathcal{N}(\theta^* = 1, \sigma^2)$. An obvious estimate is the MLE. This has a bias of 0 and a variance of

$$\mathbb{V}\left[\overline{x}|\theta^*\right] = \frac{\sigma^2}{N} \tag{4.248}$$

But we could also use a MAP estimate. In Sec. 4.6.4.2, we show that the MAP estimate under a Gaussian prior of the form $\mathcal{N}(\theta_0, \sigma^2/\kappa_0)$ is given by

$$\tilde{x} \triangleq \frac{N}{N+\kappa_0}\overline{x} + \frac{\kappa_0}{N+\kappa_0}\theta_0 = w\overline{x} + (1-w)\theta_0 \tag{4.249}$$

where $0 \leq w \leq 1$ controls how much we trust the MLE compared to our prior. The bias and variance are given by

$$\mathbb{E}\left[\tilde{x}\right] - \theta^* = w\theta^* + (1-w)\theta_0 - \theta^* = (1-w)(\theta_0 - \theta^*) \tag{4.250}$$

$$\mathbb{V}\left[\tilde{x}\right] = w^2\frac{\sigma^2}{N} \tag{4.251}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

So although the MAP estimate is biased (assuming $w < 1$), it has lower variance.

Let us assume that our prior is slightly misspecified, so we use $\theta_0 = 0$, whereas the truth is $\theta^* = 1$. In Fig. 4.24(a), we see that the sampling distribution of the MAP estimate for $\kappa_0 > 0$ is biased away from the truth, but has lower variance (is narrower) than that of the MLE.

In Fig. 4.24(b), we plot $\text{mse}(\tilde{x})/\text{mse}(\overline{x})$ vs N. We see that the MAP estimate has lower MSE than the MLE for $\kappa_0 \in \{1, 2\}$. The case $\kappa_0 = 0$ corresponds to the MLE, and the case $\kappa_0 = 3$ corresponds to a strong prior, which hurts performance because the prior mean is wrong. Thus we see that, provided the prior strength is properly "tuned", a MAP estimate can outperform an ML estimate in terms of minimizing MSE.

4.7.6.5 Example: MAP estimator for linear regression

Another important example of the bias-variance tradeoff arises in ridge regression, which we discuss in Sec. 11.3. In brief, this corresponds to MAP estimation for linear regression under a Gaussian prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$ The zero-mean prior encourages the weights to be small, which reduces overfitting; the precision term, λ, controls the strength of this prior. Setting $\lambda = 0$ results in the MLE; using $\lambda > 0$ results in a biased estimate. To illustrate the effect on the variance, consider a simple example where we fit a 1d ridge regression model using 2 different values of λ. Fig. 4.25 on the left plots each individual fitted curve, and on the right plots the average fitted curve. We see that as we increase the strength of the regularizer, the variance decreases, but the bias increases.

See also Fig. 4.26 where we give a cartoon sketch of the bias variance tradeoff in terms of model complexity.

4.7.6.6 Bias-variance tradeoff for classification

If we use 0-1 loss instead of squared error, the frequentist risk is no longer expressible as squared bias plus variance. In fact, one can show (Exercise 7.2 of [HTF09]) that the bias and variance combine multiplicatively. If the estimate is on the correct side of the decision boundary, then the bias is negative, and decreasing the variance will decrease the misclassification rate. But if the estimate is on the wrong side of the decision boundary, then the bias is positive, so it pays to *increase* the variance [Fri97a]. This little known fact illustrates that the bias-variance tradeoff is not very useful for classification. It is better to focus on expected loss, not directly on bias and variance. We can approximate the expected loss using cross validation, as we discuss in Sec. 4.5.5.

4.8 Exercises

Exercise 4.1 [MLE for the univariate Gaussian]
Show that the MLE for a univariate Gaussian is given by

$$\hat{\mu} = \frac{1}{N}\sum_{n=1}^{N} y_n \tag{4.252}$$

$$\hat{\sigma}^2 = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{\mu})^2 \tag{4.253}$$

Exercise 4.2 [MAP estimation for 1D Gaussians]
(Source: Jaakkola.)

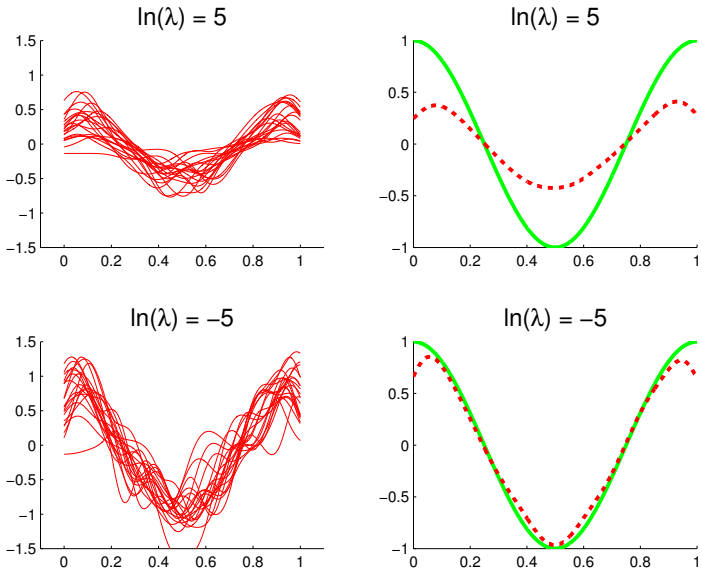


Figure 4.25: Illustration of bias-variance tradeoff for ridge regression. We generate 100 data sets from the true function, shown in solid green. Left: we plot the regularized fit for 20 different data sets. We use linear regression with a Gaussian RBF expansion, with 25 centers evenly spread over the $[0, 1]$ interval. Right: we plot the average of the fits, averaged over all 100 datasets. Top row: strongly regularized: we see that the individual fits are similar to each other (low variance), but the average is far from the truth (high bias). Bottom row: lightly regularized: we see that the individual fits are quite different from each other (high variance), but the average is close to the truth (low bias). Adapted from [Bis06] Figure 3.5. Generated by code at figures.probml.ai/book1/4.25.

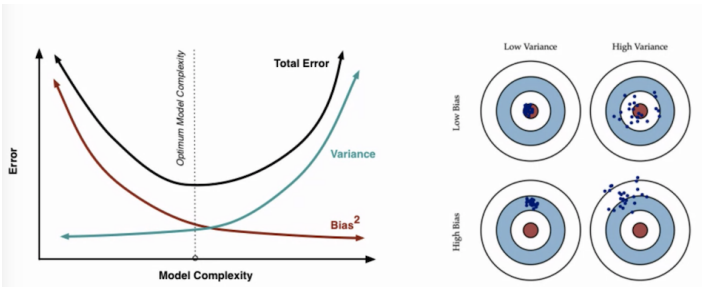


Figure 4.26: Cartoon illustration of the bias variance tradeoff. From http://scott.fortmann-roe.com/docs/BiasVariance.html. Used with kind permission of Scott Fortmann-Roe.

Consider samples $x_1, \ldots, x_n$ from a Gaussian random variable with known variance σ^2 and unknown mean μ. We further assume a prior distribution (also Gaussian) over the mean, $\mu \sim \mathcal{N}(m, s^2)$, with fixed mean m and fixed variance s^2. Thus the only unknown is μ.

a. Calculate the MAP estimate $\hat{\mu}_{MAP}$. You can state the result without proof. Alternatively, with a lot more work, you can compute derivatives of the log posterior, set to zero and solve.
b. Show that as the number of samples n increase, the MAP estimate converges to the maximum likelihood estimate.
c. Suppose n is small and fixed. What does the MAP estimator converge to if we increase the prior variance s^2?
d. Suppose n is small and fixed. What does the MAP estimator converge to if we decrease the prior variance s^2?

Exercise 4.3 [Gaussian posterior credible interval]

(Source: DeGroot.) Let $X \sim \mathcal{N}(\mu, \sigma^2 = 4)$ where μ is unknown but has prior $\mu \sim \mathcal{N}(\mu_0, \sigma_0^2 = 9)$. The posterior after seeing n samples is $\mu \sim \mathcal{N}(\mu_n, \sigma_n^2)$. (This is called a credible interval, and is the Bayesian analog of a confidence interval.) How big does n have to be to ensure

$$p(\ell \leq \mu_n \leq u|D) \geq 0.95 \tag{4.254}$$

where (ℓ, u) is an interval (centered on μ_n) of width 1 and D is the data? Hint: recall that 95% of the probability mass of a Gaussian is within $\pm 1.96\sigma$ of the mean.

Exercise 4.4 [BIC for Gaussians]

(Source: Jaakkola.)

The Bayesian information criterion (BIC) is a penalized log-likelihood function that can be used for model selection. It is defined as

$$BIC = \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{ML}) - \frac{d}{2}\log(N) \tag{4.255}$$

where d is the number of free parameters in the model and N is the number of samples. In this question, we will see how to use this to choose between a full covariance Gaussian and a Gaussian with a diagonal covariance. Obviously a full covariance Gaussian has higher likelihood, but it may not be "worth" the extra parameters if the improvement over a diagonal covariance matrix is too small. So we use the BIC score to choose the model.

We can write

$$\log p(\mathcal{D}|\hat{\boldsymbol{\Sigma}}, \hat{\boldsymbol{\mu}}) = -\frac{N}{2}\mathrm{tr}\left(\hat{\boldsymbol{\Sigma}}^{-1}\hat{\mathbf{S}}\right) - \frac{N}{2}\log(|\hat{\boldsymbol{\Sigma}}|) \tag{4.256}$$

$$\hat{\mathbf{S}} = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T \tag{4.257}$$

where $\hat{\mathbf{S}}$ is the scatter matrix (empirical covariance), the trace of a matrix is the sum of its diagonals, and we have used the trace trick.

a. Derive the BIC score for a Gaussian in D dimensions with full covariance matrix. Simplify your answer as much as possible, exploiting the form of the MLE. Be sure to specify the number of free parameters d.
b. Derive the BIC score for a Gaussian in D dimensions with a *diagonal* covariance matrix. Be sure to specify the number of free parameters d. Hint: for the digaonal case, the ML estimate of $\boldsymbol{\Sigma}$ is the same as $\hat{\boldsymbol{\Sigma}}_{ML}$ except the off-diagonal terms are zero:

$$\hat{\boldsymbol{\Sigma}}_{diag} = \mathrm{diag}(\hat{\boldsymbol{\Sigma}}_{ML}(1,1), \ldots, \hat{\boldsymbol{\Sigma}}_{ML}(D,D)) \tag{4.258}$$

Exercise 4.5 [BIC for a 2d discrete distribution]

(Source: Jaakkola.)

Let $x \in \{0, 1\}$ denote the result of a coin toss ($x = 0$ for tails, $x = 1$ for heads). The coin is potentially biased, so that heads occurs with probability θ_1. Suppose that someone else observes the coin flip and reports to you the outcome, y. But this person is unreliable and only reports the result correctly with probability θ_2; i.e., $p(y|x, \theta_2)$ is given by

	$y=0$	$y=1$
$x=0$	θ_2	$1-\theta_2$
$x=1$	$1-\theta_2$	θ_2

Assume that θ_2 is independent of x and θ_1.

a. Write down the joint probability distribution $p(x, y|\boldsymbol{\theta})$ as a 2×2 table, in terms of $\boldsymbol{\theta} = (\theta_1, \theta_2)$.

b. Suppose have the following dataset: $\mathbf{x} = (1, 1, 0, 1, 1, 0, 0)$, $\mathbf{y} = (1, 0, 0, 0, 1, 0, 1)$. What are the MLEs for θ_1 and θ_2? Justify your answer. Hint: note that the likelihood function factorizes,

$$p(x, y|\boldsymbol{\theta}) = p(y|x, \theta_2)p(x|\theta_1) \tag{4.259}$$

What is $p(\mathcal{D}|\hat{\boldsymbol{\theta}}, M_2)$ where M_2 denotes this 2-parameter model? (You may leave your answer in fractional form if you wish.)

c. Now consider a model with 4 parameters, $\boldsymbol{\theta} = (\theta_{0,0}, \theta_{0,1}, \theta_{1,0}, \theta_{1,1})$, representing $p(x, y|\boldsymbol{\theta}) = \theta_{x,y}$. (Only 3 of these parameters are free to vary, since they must sum to one.) What is the MLE of $\boldsymbol{\theta}$? What is $p(\mathcal{D}|\hat{\boldsymbol{\theta}}, M_4)$ where M_4 denotes this 4-parameter model?

d. Suppose we are not sure which model is correct. We compute the leave-one-out cross validated log likelihood of the 2-parameter model and the 4-parameter model as follows:

$$L(m) = \sum_{i=1}^{n} \log p(x_i, y_i|m, \hat{\theta}(\mathcal{D}_{-i})) \tag{4.260}$$

and $\hat{\theta}(\mathcal{D}_{-i}))$ denotes the MLE computed on $\mathcal{D}$ excluding row i. Which model will CV pick and why? Hint: notice how the table of counts changes when you omit each training case one at a time.

e. Recall that an alternative to CV is to use the BIC score, defined as

$$\text{BIC}(M, \mathcal{D}) \triangleq \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{MLE}) - \frac{\text{dof}(M)}{2} \log N \tag{4.261}$$

where $\text{dof}(M)$ is the number of free parameters in the model, Compute the BIC scores for both models (use log base e). Which model does BIC prefer?

Exercise 4.6 [A mixture of conjugate priors is conjugate]

Consider a mixture prior

$$p(\theta) = \sum_k p(h = k)p(\theta|z = k) \tag{4.262}$$

where each $p(\theta|z = k)$ is conjugate to the likelihood. Prove that this is a conjugate prior.

Exercise 4.7 [ML estimator σ^2_{mle} is biased]

Show that $\hat{\sigma}^2_{MLE} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \hat{\mu})^2$ is a biased estimator of σ^2, i.e., show

$$\mathbf{E}_{\mathrm{X}_1,\ldots,\mathrm{X}_\mathrm{n} \sim \mathcal{N}(\mu,\sigma)}[\hat{\sigma}^2(\mathrm{X}_1, \ldots, \mathrm{X}_\mathrm{n})] \neq \sigma^2$$

Hint: note that $X_1, \ldots, X_N$ are independent, and use the fact that the expectation of a product of independent random variables is the product of the expectations.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 4.8 [Estimation of σ^2 when μ is known]

Suppose we sample $x_1, \ldots, x_N \sim \mathcal{N}(\mu, \sigma^2)$ where μ is a *known* constant. Derive an expression for the MLE for σ^2 in this case. Is it unbiased?

Exercise 4.9 [Variance and MSE of estimators for Gaussian variance]

Prove that the standard error for the MLE for a Gaussian variance is

$$\sqrt{\mathbb{V}\left[\sigma^2_{\text{mle}}\right]} = \sqrt{\frac{2(N-1)}{N^2}}\sigma^2 \tag{4.263}$$

Hint: use the fact that

$$\frac{N-1}{\sigma^2}\sigma^2_{\text{unb}} \sim \chi^2_{N-1}, \tag{4.264}$$

and that $\mathbb{V}\left[\chi^2_{N-1}\right] = 2(N-1)$. Finally, show that $\text{MSE}(\sigma^2_{\text{unb}}) = \frac{2N-1}{N^2}\sigma^4$ and $\text{MSE}(\sigma^2_{\text{mle}}) = \frac{2}{N-1}\sigma^4$.

5 Decision theory

5.1 Bayesian decision theory

Bayesian inference provides the optimal way to update our beliefs about hidden quantities H given observed data $X = x$ by computing the posterior $p(H|x)$. However, at the end of the day, we need to turn our beliefs into **actions** that we can perform in the world. How can we decide which action is best? This is where **Bayesian decision theory** comes in. In this chapter, we give a brief introduction. For more details, see e.g., [DeG70; KWW22].

5.1.1 Basics

In decision theory, we assume the decision maker, or **agent**, has a set of possible actions, $\mathcal{A}$, to choose from. For example, consider the case of a hypothetical doctor treating someone who may have COVID-19. Suppose the actions are to do nothing, or to give the patient an expensive drug with bad side effects, but which can save their life.

Each of these actions has costs and benefits, which will depend on the underlying **state of nature** $H \in \mathcal{H}$. We can encode this information into a **loss function** $\ell(h, a)$, that specifies the loss we incur if we take action $a \in \mathcal{A}$ when the state of nature is $h \in \mathcal{H}$.

For example, suppose the state is defined by the age of the patient (young vs old), and whether they have COVID-19 or not. Note that the age can be observed directly, but the disease state must be inferred from noisy observations, as we discussed in Sec. 2.3. Thus the state is **partially observed**.

Let us assume that the cost of administering a drug is the same, no matter what the state of the patient is. However, the benefits will differ. If the patient is young, we expect them to live a long time, so the cost of not giving the drug if they have COVID-19 is high; but if the patient is old, they have fewer years to live, so the cost of not giving the drug if they have COVID-19 is arguably less (especially in view of the side effects). In medical circles, a common unit of cost is **quality-adjusted life years** or **QALY**. Suppose that the expected QALY for a young person is 60, and for an old person is 10. Let us assume the drug costs the equivalent of 8 QALY, due to induced pain and suffering from side effects. Then we get the loss matrix shown in Table 5.1.

These numbers reflect relative costs and benefits, and will depend on many factors. The numbers can be derived by asking the decision maker about their **preferences** about different possible outcomes. It is a theorem of decision theory that any consistent set of preferences can be converted into an ordinal cost scale (see e.g., https://en.wikipedia.org/wiki/Preference_(economics)).

Once we have specified the loss function, we can compute the **posterior expected loss** or **risk**

State	Nothing	Drugs
No COVID-19, young	0	8
COVID-19, young	60	8
No COVID-19, old	0	8
COVID-19, old	10	8

Table 5.1: Hypothetical loss matrix for a decision maker, where there are 4 states of nature, and 2 possible actions.

test	age	pr(covid)	cost-noop	cost-drugs	action
0	0	0.01	0.84	8.00	0
0	1	0.01	0.14	8.00	0
1	0	0.80	47.73	8.00	1
1	1	0.80	7.95	8.00	0

Table 5.2: Optimal policy for treating COVID-19 patients for each possible observation.

for each possible action:

$$R(a|x) \triangleq \mathbb{E}_{p(h|x)}\left[\ell(h,a)\right] = \sum_{h \in \mathcal{H}} \ell(h,a) p(h|x) \tag{5.1}$$

The **optimal policy** (also called the **Bayes estimator**) specifies what action to take for each possible observation so as to minimize the risk:

$$\pi^*(\mathbf{x}) = \underset{a \in \mathcal{A}}{\operatorname{argmin}}\, \mathbb{E}_{p(h|\mathbf{x})}\left[\ell(h,a)\right] \tag{5.2}$$

An alternative, but equivalent, way of stating this result is as follows. Let us define a **utility function** $U(h,a)$ to be the desirability of each possible action in each possible state. If we set $U(h,a) = -\ell(h,a)$, then the optimal policy is as follows:

$$\pi^*(\mathbf{x}) = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, \mathbb{E}_h\left[U(h,a)\right] \tag{5.3}$$

This is called the **maximum expected utility principle**.

Let us return to our COVID-19 example. The observation $\mathbf{x}$ consists of the age (young or old) and the test result (positive or negative). Using the results from Sec. 2.3.1 on Bayes rule for COVID-19 diagnosis, we can convert the test result into a distribution over disease states (i.e., compute the probability the patient has COVID-19 or not). Given this belief state, and the loss matrix in Table 5.1, we can compute the optimal policy for each possible observation, as shown in Table 5.2.

We see from Table 5.2 that the drug should only be given to young people who test positive. If, however, we reduce the cost of the drug from 8 units to 5, then the optimal policy changes: in this case, we should give the drug to everyone who tests positive. The policy can also change depending on the reliability of the test. For example, if we increase the sensitivity from 0.875 to 0.975, then the probability that someone has COVID-19 if they test positive increases from 0.80 to 0.81, which

changes the optimal policy to be one in which we should administer the drug to everyone who tests positive, even if the drug costs 8 QALY. (See code.probml.ai/book1/dtheory for the code to reproduce this example.)

So far, we have implicitly assumed that the agent is **risk neutral**. This means that their decision is not affected by the degree of certainty in a set of outcomes. For example, such an agent would be indifferent between getting \$50 for sure, of a 50% chance of \$100 or \$0. By contrast, a **risk averse** agent would choose the first. We can generalize the framework of Bayesian decision theory to **risk sensitive** applications, but we do not pursue the matter here. (See e.g., [Cho+15] for details.)

5.1.2 Classification problems

In this section, we use Bayesian decision theory to decide the optimal class label to predict given an observed input $x \in \mathcal{X}$.

5.1.2.1 Zero-one loss

Suppose the states of nature correspond to class labels, so $\mathcal{H} = \mathcal{Y} = \{1, \ldots, C\}$. Furthermore, suppose the actions also correspond to class labels, so $\mathcal{A} = \mathcal{Y}$. In this setting, a very commonly used loss function is the **zero-one loss** $\ell_{01}(y^*, \hat{y})$, defined as follows:

	$\hat{y}=0$	$\hat{y}=1$
$y^*=0$	0	1
$y^*=1$	1	0

(5.4)

We can write this more concisely as follows:

$$\ell_{01}(y^*, \hat{y}) = \mathbb{I}(y^* \neq \hat{y}) \tag{5.5}$$

In this case, the posterior expected loss is

$$R(\hat{y}|\mathbf{x}) = p(\hat{y} \neq y^*|\mathbf{x}) = 1 - p(y^* = \hat{y}|\mathbf{x}) \tag{5.6}$$

Hence the action that minimizes the expected loss is to choose the most probable label:

$$\pi(\mathbf{x}) = \operatorname*{argmax}_{y \in \mathcal{Y}} p(y|\mathbf{x}) \tag{5.7}$$

This corresponds to the **mode** of the posterior distribution, also known as the **maximum a posteriori** or **MAP estimate**

5.1.2.2 Cost-sensitive classification

Consider a binary classification problem where the loss function is $\ell(h, a) = \ell_{a,h}$. Let $p_0 = p(h = 0|x)$ and $p_1 = 1 - p_0$. Thus we should choose label $a = 0$ iff

$$\ell_{00} p_0 + \ell_{01} p_1 < \ell_{10} p_0 + \ell_{11} p_1 \tag{5.8}$$

If $\ell_{00} = \ell_{11} = 0$, this simplifies to

$$p_1 < \frac{\ell_{10}}{\ell_{10} + \ell_{01}} \tag{5.9}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Now suppose $\ell_{01} = c\ell_{10}$, so a false negative costs c times more than a false positive. The decision rule further simplifies to the following: pick $a = 0$ iff $p_1 < 1/(1+c)$. For example, if a false negative costs twice as much as false positive, so $c = 2$, then we use a decision threshold of $1/3$ before declaring a positive.

5.1.2.3 Classification with the "reject" option

In some cases, we may able to say "I don't know" instead of returning an answer that we don't really trust; this is called picking the **reject option** (see e.g., [BW08]). This is particularly important in domains such as medicine and finance where we may be risk averse.

We can formalize the reject option as follows. Suppose the states of nature are $\mathcal{H} = \{1, \ldots, C\}$, and the actions are $\mathcal{A} = \mathcal{H} \cup \{0\}$, where action 0 represents the reject action. Now define the following loss function:

$$\ell(h, a) = \begin{cases} 0 & \text{if } h = a \text{ and } a \in \{1, \ldots, C\} \\ \lambda_r & \text{if } a = 0 \\ \lambda_e & \text{otherwise} \end{cases} \tag{5.10}$$

where λ_r is the cost of the reject action, and λ_e is the cost of a classification error. Exercise 5.1 asks you to show that the optimal action is to pick the reject action if the most probable class has a probability below $\lambda^* = 1 - \frac{\lambda_r}{\lambda_e}$; otherwise you should just pick the most probable class. In other words, the optimal policy is as follows:

$$a^* = \begin{cases} y^* & \text{if } p^* > \lambda^* \\ \text{reject} & \text{otherwise} \end{cases} \tag{5.11}$$

where

$$y^* = \underset{y \in \{1,\ldots,C\}}{\operatorname{argmax}} \; p(y|x) \tag{5.12}$$

$$p^* = p(y^*|x) = \max_{y \in \{1,\ldots,C\}} p(y|x) \tag{5.13}$$

$$\lambda^* = 1 - \frac{\lambda_r}{\lambda_e} \tag{5.14}$$

See Fig. 5.1 for an illustration.

One interesting application of the reject option arises when playing the TV game show Jeopardy. In this game, contestants have to solve various word puzzles and answer a variety of trivia questions, but if they answer incorrectly, they lose money. In 2011, IBM unveiled a computer system called **Watson** which beat the top human Jeopardy champion. Watson uses a variety of interesting techniques [Fer+10], but the most pertinent one for our present discussion is that it contains a module that estimates how confident it is of its answer. The system only chooses to "buzz in" its answer if sufficiently confident it is correct.

For some other methods and applications, see e.g., [Cor+16; GEY19].

5.1.3 ROC curves

In Sec. 5.1.2.2, we showed that we can pick the optimal label in a binary classification problem by thresholding the probability using a value τ, derived from the relative cost of a false positive and

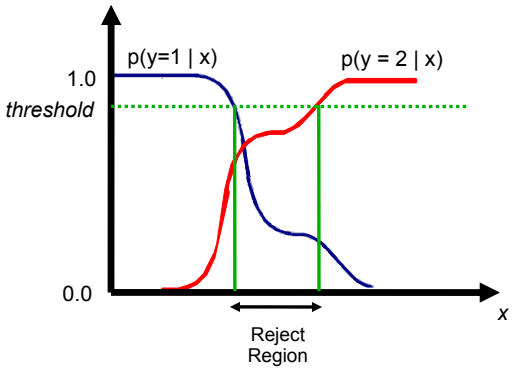


Figure 5.1: For some regions of input space, where the class posteriors are uncertain, we may prefer not to choose class 1 or 2; instead we may prefer the reject option. Adapted from Figure 1.26 of [Bis06].

		Estimate		Row sum
		0	1	
Truth	0	TN	FP	N
	1	FN	TP	P
Col. sum		$\hat{N}$	$\hat{P}$	

Table 5.3: Class confusion matrix for a binary classification problem. TP is the number of true positives, FP is the number of false positives, TN is the number of true negatives, FN is the number of false negatives, P is the true number of positives, $\hat{P}$ is the predicted number of positives, N is the true number of negatives, $\hat{N}$ is the predicted number of negatives.

		Estimate	
		0	1
Truth	0	TN/N=TNR=Spec	FP/N =FPR=Type I = Fallout
	1	FN/P=FNR=Miss=Type II	TP/P=TPR=Sens=Recall

Table 5.4: Class confusion matrix for a binary classification problem normalized per row to get $p(\hat{y}|y)$. Abbreviations: TNR = true negative rate, Spec = specificity, FPR = false positive rate, FNR = false negative rate, Miss = miss rate, TPR = true positive rate, Sens = sensitivity. Note FNR=1-TPR and FPR=1-TNR.

false negative. Instead of picking a single threshold, we can instead consider using a set of different thresholds, and comparing the resulting performance, as we discuss below.

5.1.3.1 Class confusion matrices

For any fixed threshold τ, we consider the following decision rule:

$$\hat{y}_\tau(\mathbf{x}) = \mathbb{I}\left(p(y=1|\mathbf{x}) \geq 1-\tau\right) \tag{5.15}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

		Estimate	
		0	1
Truth	0	TN/$\hat{N}$=NPV	FP/$\hat{P}$=FDR
	1	FN/$\hat{N}$=FOR	TP/$\hat{P}$=Prec=PPV

Table 5.5: Class confusion matrix for a binary classification problem normalized per column to get $p(y|\hat{y})$. Abbreviations: NPV = negative predictive value, FDR = false discovery rate, FOR = false omission rate, PPV = positive predictive value, Prec = precision. Note that FOR=1-NPV and FDR=1-PPV.

We can compute the empirical number of false positives (FP) that arise from using this policy on a set of N labeled examples as follows:

$$FP_\tau = \sum_{n=1}^{N} \mathbb{I}\left(\hat{y}_\tau(\mathbf{x}_n) = 1, y_n = 0\right) \tag{5.16}$$

Similarly, we can compute the empirical number of false negatives (FN), true positives (TP), and true negatives (TN). We can store these results in a 2×2 **class confusion matrix** C, where C_{ij} is the number of times an item with true class label i was (mis)classified as having label j. In the case of binary classification problems, the resulting matrix will look like Table 5.3.

From this table, we can compute $p(\hat{y}|y)$ or $p(y|\hat{y})$, depending on whether we normalize across the rows or columns. We can derive various summary statistics from these distributions, as summarized in Table 5.4 and Table 5.5. For example, the **true positive rate** (TPR), also known as the **sensitivity**, **recall** or **hit rate**, is defined as

$$TPR_\tau = p(\hat{y} = 1|y = 1, \tau) = \frac{TP_\tau}{TP_\tau + FN_\tau} \tag{5.17}$$

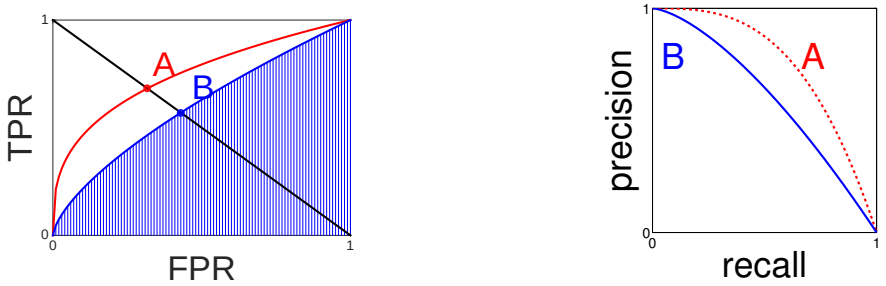
and the **false positive rate** (FPR), also called the **false alarm rate**, or the **type I error rate**, is defined as

$$FPR_\tau = p(\hat{y} = 1|y = 0, \tau) = \frac{FP_\tau}{FP_\tau + TN_\tau} \tag{5.18}$$

We can now plot the TPR vs FPR as an implicit function of τ. This is called a **receiver operating characteristic** or **ROC** curve. See Fig. 5.2(a) for an example.

5.1.3.2 Summarizing ROC curves as a scalar

The quality of a ROC curve is often summarized as a single number using the **area under the curve** or **AUC**. Higher AUC scores are better; the maximum is obviously 1. Another summary statistic that is used is the **equal error rate** or **EER**, also called the **cross-over rate**, defined as the value which satisfies FPR = FNR. Since FNR=1-TPR, we can compute the EER by drawing a line from the top left to the bottom right and seeing where it intersects the ROC curve (see points A and B in Fig. 5.2(a)). Lower EER scores are better; the minimum is obviously 0 (corresponding to the top left corner).



(a) (b)

Figure 5.2: (a) ROC curves for two hypothetical classification systems. The red curve for system A is better than the blue curve for system B. We plot the true positive rate (TPR) vs the false positive rate (FPR) as we vary the threshold τ. We also indicate the equal error rate (EER) with the red and blue dots, and the area under the curve (AUC) for classifier B by the shaded area. Generated by code at figures.probml.ai/book1/5.2. (b) A precision-recall curve for two hypothetical classification systems. The red curve for system A is better than the blue curve for system B. Generated by code at figures.probml.ai/book1/5.2.

5.1.3.3 Class imbalance

In some problems, there is severe **class imbalance**. For example, in information retrieval, the set of negatives (irrelevant items) is usually much larger than the set of positives (relevant items). The ROC curve is unaffected by class imbalance, as the TPR and FPR are fractions within the positives and negatives, respectively. However, the usefulness of an ROC curve may be reduced in such cases, since a large change in the absolute number of false positives will not change the false positive *rate* very much, since FPR is divided by FP+TN (see e.g., [SR15] for discussion). Thus all the "action" happens in the extreme left part of the curve. In such cases, we may choose to use other ways of summarizing the class confusion matrix, such as precision-recall curves, which we discuss in Sec. 5.1.4.

5.1.4 Precision-recall curves

In some problems, the notion of a "negative" is not well-defined. For example, consider detecting objects in images: if the detector works by classifying patches, then the number of patches examined — and hence the number of true negatives — is a parameter of the algorithm, not part of the problem definition. Similarly, information retrieval systems usually get to choose the initial set of candidate items, which are then ranked for relevance; by specifying a cutoff, we can partition this into a positive and negative set, but note that the size of the negative set depends on the total number of items retrieved, which is an algorithm parameter, not part of the problem specification.

In these kinds of situations. we may choose to use a **precision-recall curve** to summarize the performance of our system, as we exlain below, (See e.g., [DG06] for a more detailed discussion of the connection between ROC curves and PR curves.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.1.4.1 Computing precision and recall

The key idea is to replace the FPR with a quantity that is computed just from positives, namely the **precision**:

$$\mathcal{P}(\tau) \triangleq p(y = 1|\hat{y} = 1, \tau) = \frac{TP_\tau}{TP_\tau + FP_\tau} \tag{5.19}$$

The precision measures what fraction of our detections are actually positive. We can compare this to the **recall** (which is the same as the TPR), which measures what fraction of the positives we actually detected:

$$\mathcal{R}(\tau) \triangleq p(\hat{y} = 1|y = 1, \tau) = \frac{TP_\tau}{TP_\tau + FN_\tau} \tag{5.20}$$

If $\hat{y}_n \in \{0, 1\}$ is the predicted label, and $y_n \in \{0, 1\}$ is the true label, we can estimate precision and recall using

$$\mathcal{P}(\tau) = \frac{\sum_n y_n \hat{y}_n}{\sum_n \hat{y}_n} \tag{5.21}$$

$$\mathcal{R}(\tau) = \frac{\sum_n y_n \hat{y}_n}{\sum_n y_n} \tag{5.22}$$

We can now plot the precision vs recall as we vary the threshold τ. See Fig. 5.2(b). Hugging the top right is the best one can do.

5.1.4.2 Summarizing PR curves as a scalar

The PR curve can be summarized as a single number in several ways. First, we can quote the precision for a fixed recall level, such as the precision of the first $K = 10$ entities recalled. This is called the **precision at K** score. Alternatively, we can compute the area under the PR curve. However, it is possible that the precision does not drop monotonically with recall. For example, suppose a classifier has 90% precision at 10% recall, and 96% precision at 20% recall. In this case, rather than measuring the precision *at* a recall of 10%, we should measure the maximum precision we can achieve with *at least* a recall of 10% (which would be 96%). This is called the **interpolated precision**. The average of the interpolated precisions is called the **average precision**; it is equal to the area under the interpolated PR curve, but may not be equal to the area under the raw PR curve.[1] The **mean average precision** or **mAP** is the mean of the AP over a set of different PR curves.

5.1.4.3 F-scores

For a fixed threshold, corresponding to a single point on the PR curve, we can compute a single precision and recall value, which we will denote by $\mathcal{P}$ and $\mathcal{R}$. These are often combined into a single

1. For details, see https://sanchom.wordpress.com/tag/average-precision/.

statistic called the F_β, which weights recall as $\beta > 0$ more important than precision. This is defined as follows:

$$\frac{1}{F_\beta} = \frac{1}{1+\beta^2}\frac{1}{\mathcal{P}} + \frac{\beta^2}{1+\beta^2}\frac{1}{\mathcal{R}} \tag{5.23}$$

or equivalently

$$F_\beta \triangleq (1+\beta^2)\frac{\mathcal{P}\cdot\mathcal{R}}{\beta^2\mathcal{P}+\mathcal{R}} = \frac{(1+\beta^2)TP}{(1+\beta^2)TP+\beta^2 FN+FP} \tag{5.24}$$

If we set $\beta = 1$, we get the harmonic mean of precision and recall:

$$\frac{1}{F_1} = \frac{1}{2}\left(\frac{1}{\mathcal{P}}+\frac{1}{\mathcal{R}}\right) \tag{5.25}$$

$$F_1 = \frac{2}{1/\mathcal{R}+1/\mathcal{P}} = 2\frac{\mathcal{P}\cdot\mathcal{R}}{\mathcal{P}+\mathcal{R}} = \frac{TP}{TP+\frac{1}{2}(FP+FN)} \tag{5.26}$$

To understand why we use the harmonic mean instead of the arithmetic mean, $(\mathcal{P}+\mathcal{R})/2$, consider the following scenario. Suppose we recall all entries, so $\hat{y}_n = 1$ for all n, and $\mathcal{R} = 1$. In this case, the precision $\mathcal{P}$ will be given by the **prevalence**, $p(y=1) = \frac{\sum_n \mathbb{I}(y_n=1)}{N}$. Suppose the prevalence is low, say $p(y=1) = 10^{-4}$. The arithmetic mean of $\mathcal{P}$ and $\mathcal{R}$ is given by $(\mathcal{P}+\mathcal{R})/2 = (10^{-4}+1)/2 \approx 50\%$. By contrast, the harmonic mean of this strategy is only $\frac{2\times 10^{-4}\times 1}{1+10^{-4}} \approx 0.2\%$. In general, the harmonic mean is more conservative, and requires both precision and recall to be high.

Using F_1 score weights precision and recall equally. However, if recall is more important, we may use $\beta = 2$, and if precision is more important, we may use $\beta = 0.5$.

5.1.4.4 Class imbalance

ROC curves are insensitive to class imbalance, but PR curves are not, as noted in [Wil20]. To see this, let the fraction of positives in the dataset be $\pi = P/(P+N)$, and define the ratio $r = P/N = \pi/(1-\pi)$. Let $n = P+N$ be the population size. ROC curves are not affected by changes in r, since the TPR is defined as a ratio within the positive examples, and FPR is defined as a ratio within the negative examples. This means it does not matter which class we define as positive, and which we define as negative.

Now consider PR curves. The precision can be written as

$$\text{Prec} = \frac{TP}{TP+FP} = \frac{P\cdot TPR}{P\cdot TPR + N\cdot FPR} = \frac{TPR}{TPR+\frac{1}{r}FPR} \tag{5.27}$$

Thus Prec $\to 1$ as $\pi \to 1$ and $r \to \infty$, and Prec $\to 0$ as $\pi \to 0$ and $r \to 0$. For example, if we we change from a balanced problem where $r = 0.5$ to an imbalanced problem where $r = 0.1$ (so positives are rarer), the precision at each threshold will drop, and the recall (aka TPR) will stay the same, so the overall PR curve will be lower. Thus if we have multiple binary problems with different prevalences (e.g., object detection of common or rare objects), we should be careful when averaging their precisions [HCD12].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The F-score is also affected by class imbalance. To see this, note that we can rewrite the F-score as follows:

$$\frac{1}{F_\beta} = \frac{1}{1+\beta^2}\frac{1}{\mathcal{P}} + \frac{\beta^2}{1+\beta^2}\frac{1}{\mathcal{R}} \tag{5.28}$$

$$= \frac{1}{1+\beta^2}\frac{TPR + \frac{N}{P}FPR}{TPR} + \frac{\beta^2}{1+\beta^2}\frac{1}{TPR} \tag{5.29}$$

$$F_\beta = \frac{(1+\beta^2)TPR}{TPR + \frac{1}{r}FPR + \beta^2} \tag{5.30}$$

5.1.5 Regression problems

So far, we have considered the case where there are a finite number of actions $\mathcal{A}$ and states of nature $\mathcal{H}$. In this section, we consider the case where the set of actions and states are both equal to the real line, $\mathcal{A} = \mathcal{H} = \mathbb{R}$. We will specify various commonly used loss functions for this case (which can be extended to $\mathbb{R}^D$ by computing the loss elementwise.) The resulting decision rules can be used to compute the optimal parameters for an estimator to return, or the optimal action for a robot to take, etc.

5.1.5.1 L2 loss

The most common loss for continuous states and actions is the ℓ_2 **loss**, also called **squared error** or **quadratic loss**, which is defined as follows:

$$\ell_2(h, a) = (h - a)^2 \tag{5.31}$$

In this case, the risk is given by

$$R(a|\mathbf{x}) = \mathbb{E}\left[(h-a)^2|\mathbf{x}\right] = \mathbb{E}\left[h^2|\mathbf{x}\right] - 2a\mathbb{E}\left[h|\mathbf{x}\right] + a^2 \tag{5.32}$$

The optimal action must satisfy the condition that the derivative of the risk (at that point) is zero (as explained in Chapter 8). Hence the optimal action is to pick the posterior mean:

$$\frac{\partial}{\partial a}R(a|\mathbf{x}) = -2\mathbb{E}\left[h|\mathbf{x}\right] + 2a = 0 \;\Rightarrow\; \pi(\mathbf{x}) = \mathbb{E}\left[h|\mathbf{x}\right] = \int h\, p(h|\mathbf{x})dh \tag{5.33}$$

This is often called the **minimum mean squared error** estimate or **MMSE** estimate.

5.1.5.2 L1 loss

The ℓ_2 loss penalizes deviations from the truth quadratically, and thus is sensitive to **outliers**. A more **robust** alternative is the absolute or ℓ_1 **loss**

$$\ell_1(h, a) = |h - a| \tag{5.34}$$

This is sketched in Fig. 5.3 Exercise 5.4 asks you to show that the optimal estimate is the **posterior median**, i.e., a value a such that $\Pr(h < a|\mathbf{x}) = \Pr(h \geq a|\mathbf{x}) = 0.5$. We can use this for robust regression as discussed in Sec. 11.4.2.

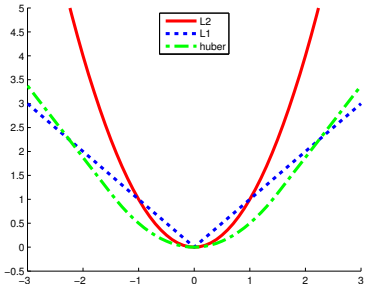


Figure 5.3: Illustration of ℓ_2, ℓ_1, and Huber loss functions with $\delta = 1.5$. Generated by code at figures.probml.ai/book1/5.3.

5.1.5.3 Huber loss

Another robust loss function is the **Huber loss** [Hub64], defined as follows:

$$\ell_\delta(h, a) = \begin{cases} r^2/2 & \text{if } |r| \leq \delta \\ \delta|r| - \delta^2/2 & \text{if } |r| > \delta \end{cases} \tag{5.35}$$

where $r = h - a$. This is equivalent to ℓ_2 for errors that are smaller than δ, and is equivalent to ℓ_1 for larger errors. See Fig. 5.3 for a plot. We can use this for robust regression as discussed in Sec. 11.4.3.

5.1.6 Probabilistic prediction problems

In Sec. 5.1.2, we assumed the set of possible actions was to pick a single class label (or possibly the "reject" or "do not know" action). In Sec. 5.1.5, we assumed the set of possible actions was to pick a real valued scalar. In this section, we assume the set of possible actions is to pick a **probability distribution** over some value of interest. That is, we want to perform **probabilistic prediction** or **probabilistic forecasting**, rather than predicting a specific value. More precisely, we assume the true "state of nature" is a *distribution*, $h = p(Y|x)$, the action is another distribution, $a = q(Y|x)$, and we want to pick q to minimize $\mathbb{E}\left[\ell(p, q)\right]$ for a given x. We discuss various possible loss functions below.

5.1.6.1 KL, cross-entropy and log-loss

A common form of loss functions for comparing two distributions is the **Kullback Leibler divergence**, or **KL divergence**, which is defined as follows:

$$\mathbb{KL}\left(p \| q\right) \triangleq \sum_{y \in \mathcal{Y}} p(y) \log \frac{p(y)}{q(y)} \tag{5.36}$$

(We have assumed the variable y is discrete, for notational simplicity, but this can be generalized to real-valued variables.) In Sec. 6.2, we show that the KL divergence satisfies the following properties: $\mathbb{KL}\left(p \| q\right) \geq 0$ with equality iff $p = q$. Note that it is an asymmetric function of its arguments.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can expand the KL as follows:

$$\mathbb{KL}\left(p\|q\right) = \sum_{y\in\mathcal{Y}} p(y)\log p(y) - \sum_{y\in\mathcal{Y}} p(y)\log q(y) \tag{5.37}$$

$$= -\mathbb{H}(p) + \mathbb{H}(p,q) \tag{5.38}$$

$$\mathbb{H}(p) \triangleq -\sum_{y} p(y)\log p(y) \tag{5.39}$$

$$\mathbb{H}(p,q) \triangleq -\sum_{y} p(y)\log q(y) \tag{5.40}$$

The $\mathbb{H}(p)$ term is known as the **entropy**. This is a measure of uncertainty or variance of p; it is maximal if p is uniform, and is 0 if p is a degenerate or deterministic delta function. Entropy is often used in the field of **information theory**, which is concerned with optimal ways of compressing and communicating data (see Chapter 6). The optimal coding scheme will allocate fewer bits to more frequent symbols (i.e., values of Y for which $p(y)$ is large), and more bits to less frequent symbols. A key result states that the number of bits needed to compress a dataset generated by a distribution p is at least $\mathbb{H}(p)$; the entropy therefore provides a lower bound on the degree to which we can compress data without losing information. The $\mathbb{H}(p,q)$ term is known as the **cross-entropy**. This measures the expected number of bits we need to use to compress a dataset coming from distribution p if we design our code using distribution q. Thus the KL is the extra number of bits we need to use to compress the data due to using the incorrect distribution q. If the KL is zero, it means that we can correctly predict the probabilities of all possible future events, and thus we have learned to predict the future as well as an "oracle" that has access to the true distribution p.

To find the optimal distribution to use when predicting future data, we can minimize $\mathbb{KL}\left(p\|q\right)$. Since $\mathbb{H}(p)$ is a constant wrt q, it can be ignored, and thus we can equivalently minimize the cross-entropy:

$$q^*(Y|x) = \operatorname*{argmin}_q \mathbb{H}(q(Y|x), p(Y|x)) \tag{5.41}$$

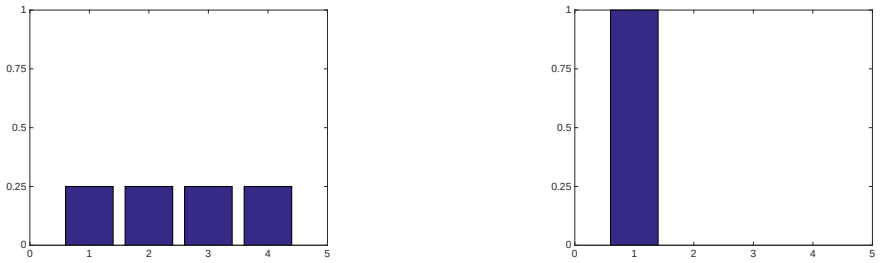
Now consider the special case in which the true state of nature is a degenerate distribution, which puts all its mass on a single outcome, say c, i.e., $h = p(Y|x) = \mathbb{I}\left(Y = c\right)$. This is often called a "**one-hot**" distribution, since it turns "on" the c'th element of the vector, and leaves the other elements "off", as shown in Fig. 5.4. In this case, the cross entropy becomes

$$\mathbb{H}(\delta(Y=c), q) = -\sum_{y\in\mathcal{Y}} \delta(y=c)\log q(y) = -\log q(c) \tag{5.42}$$

This is known as the **log loss** of the predictive distribution q when given target label c.

5.1.6.2 Proper scoring rules

Cross-entropy loss is a very common choice for probabilistic forecasting, but is not the only possible metric. The key property we desire is that the loss function is minimized iff the decision maker picks the distribution q that matches the true distribution q, i.e., $\ell(p,p) \leq \ell(p,q)$, with equality iff $p = q$. Such a loss function ℓ is called a **proper scoring rule** [GR07].



(a) *(b)*

Figure 5.4: Some discrete distributions on the state space $\mathcal{Y} = \{1, 2, 3, 4\}$. (a) A uniform distribution with $p(y = c) = 1/4$. (b) A degenerate (one-hot) distribution that puts all its mass on $y = 1$. Generated by code at figures.probml.ai/book1/5.4.

We can show that cross-entropy loss is a proper scoring rule by virtue of the fact that $\mathbb{KL}(p\|p) \leq \mathbb{KL}(p\|q)$. However, the $\log p(y)/q(y)$ term can be quite sensitive to errors for low probability events [QC+06]. A common alternative is to use the **Brier score** [Bri50], which is defined as follows (for a discrete distribution with C values):

$$\ell(p, q) \triangleq \frac{1}{C} \sum_{c=1}^{C} (q(y = c|x) - p(y = c|x))^2 \tag{5.43}$$

This is just the squared error of the predictive distribution compared to the true distribution, when viewed as vectors. Since it based on squared error, the Brier score is less sensitive to extremely rare or extremely common classes. Fortunately, it is also a proper scoring rule.

5.2 A/B testing *

Suppose you are trying to decide which version of a product is likely to sell more, or which version of a drug is likely to work better. Let us call the versions you are choosing between A and B; sometimes version A is called the **treatment**, and version B is called the **control**. To simplify notation, we will refer to picking version A as choosing action 1, and picking version B as choosing action 0. We will call the resulting outcome from each action that you want to maximize the **reward** (e.g., profit, or lives saved).

A very common approach to such problems is to use an **A/B test**, in which you try both actions out for a while, by randomly assigning a different action to different subsets of the population, and then you measure the results and pick the winner. (This is also sometimes called a "**test and roll**" problem, since you test which method is best, and then roll it out for the rest of the population.)

A key problem in A/B testing is to come up with a decision rule, or policy, for deciding which action is best, after obtaining potentially noisy results during the test phase. Another problem is to choose how many people to assign to the treatment, n_1, and how many to the control, n_0. The fundamental tradeoff is that using larger values of n_1 and n_0 will help you collect more data and hence be more confident in picking the best action, but this incurs an **opportunity cost**, because the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

testing phase involves performing actions that may not result in the highest reward. This is called the **exploration-exploitation tradeoff**, or the **earning while learning dilemma**. In this section, we give a simple Bayesian decision theoretic analysis of this problem, following the presentation of [FB19].[2]

5.2.1 A Bayesian approach

We assume the i'th reward for action j is given by $Y_{ij} \sim \mathcal{N}(\mu_j, \sigma_j^2)$ for $i = 1 : n_j$ and $j = 0 : 1$, where $j = 1$ corresponds to the treatment (action A) and $j = 0$ corresponds to the control (action B). For simplicity, we assume the σ_j^2 are known. For the unknown μ_j parameters (expected reward), we will use Gaussian priors, $\mu_j \sim \mathcal{N}(m_j, \tau_j^2)$. If we assign n_1 people to the treatment and n_0 to the control, then the expected reward in the testing phase is given by

$$\mathbb{E}\left[R_{\text{test}}\right] = n_0 m_0 + n_1 m_1 \tag{5.44}$$

The expected reward in the roll phase depends on the decision rule $\pi(\mathbf{y}_1, \mathbf{y}_0)$, which specifies which action to deploy, where $\mathbf{y}_j = (y_{1j}, \ldots, y_{n_j,j})$ is the data from action j. We derive the optimal policy below, and then discuss some of its properties.

5.2.1.1 Optimal policy

The optimal policy is to choose the action with the greater expected posterior mean reward. Applying Bayes' rule for Gaussians (Eq. (3.36)), we find that the corresponding posterior is given by

$$p(\mu_j|\mathbf{y}_j, n_j) = \mathcal{N}(\mu_j|\, \widehat{m}_j, \widehat{\tau}_j^2) \tag{5.45}$$

$$1/\,\widehat{\tau}_j = n_j/\sigma_j^2 + 1/\tau_j^2 \tag{5.46}$$

$$\widehat{m}_j \,/\, \widehat{\tau}_j = n_j \overline{y}_j/\sigma_j^2 + m_j/\tau_j^2 \tag{5.47}$$

We see that the posterior precision (inverse variance) is a weighted sum of the prior precision plus n_j units of measurement precision. We also see that the posterior precision weighted mean is a sum of the prior precision weighted mean and the measurement precision weighted mean. Hence the optimal policy is

$$\pi^*(\mathbf{y}_1, \mathbf{y}_0) = \begin{cases} 1 & \text{if } \mathbb{E}\left[\mu_1|\mathbf{y}_1\right] \geq \mathbb{E}\left[\mu_0|\mathbf{y}_0\right] \\ 0 & \text{if } \mathbb{E}\left[\mu_1|\mathbf{y}_1\right] < \mathbb{E}\left[\mu_0|\mathbf{y}_0\right] \end{cases} \tag{5.48}$$

where $\pi = 1$ means choose action A, and $\pi = 0$ means choose action B. In the fully symmetric case, where $n_1 = n_0$, $m_1 = m_0$ and $\tau_1 = \tau_0$, we find that the optimal policy is to simply "pick the winner":

$$\pi^*(\mathbf{y}_1, \mathbf{y}_0) = \mathbb{I}\left(\frac{m}{\tau^2} + \frac{\overline{y}_1}{\sigma^2} > \frac{m}{\tau^2} + \frac{\overline{y}_0}{\sigma^2}\right) = \mathbb{I}\left(\overline{y}_1 > \overline{y}_0\right) \tag{5.49}$$

This makes intuitive sense. When the problem is asymmetric, we need to take into account the different sample sizes and prior beliefs.

2. For a similar set of results in the time-discounted setting, see https://chris-said.io/2020/01/10/optimizing-sample-sizes-in-ab-testing-part-I.

5.2.1.2 Expected reward

If the total population size is N, and we cannot reuse people from the testing phase, the expected reward from the roll stage is

$$\mathbb{E}\left[R_{\text{roll}}\right] = \int_{\mu_1}\int_{\mu_0}\int_{\mathbf{y}_1}\int_{\mathbf{y}_0} (N - n_1 - n_0)\left(\pi(\mathbf{y}_1, \mathbf{y}_0)\mu_1 + (1 - \pi(\mathbf{y}_1, \mathbf{y}_0))\mu_0\right) \tag{5.50}$$

$$\times\, p(\mathbf{y}_0)p(\mathbf{y}_1)p(\mu_0)p(\mu_1)d\mathbf{y}_0 d\mathbf{y}_1 d\mu_0 d\mu_1 \tag{5.51}$$

One can show that this equals

$$\mathbb{E}\left[R_{\text{roll}}\right] = (N - n_1 - n_0)\left(m_1 + e\Phi(\frac{e}{v}) + v\phi(\frac{e}{v})\right) \tag{5.52}$$

where ϕ is the Gaussian pdf, Φ is the Gaussian cdf, $e = m_0 - m_1$ and

$$v = \sqrt{\frac{\tau_1^4}{\tau_1^2 + \sigma_1^2/n_1} + \frac{\tau_0^4}{\tau_0^2 + \sigma_0^2/n_0}} \tag{5.53}$$

In the fully symmetric case, where $m_j = m$, $\tau_j = \tau$, $\sigma_j = \sigma$, and $n_j = n$, Eq. (5.52) simplifies to

$$\mathbb{E}\left[R_{\text{roll}}\right] = (N - 2n)\left(m + \frac{\sqrt{2}\tau^2}{\sqrt{\pi}\sqrt{2\tau^2 + \frac{2}{n}\sigma^2}}\right) \tag{5.54}$$

The second term inside the brackets is the incremental gain we expect to see (beyond the prior) per person by virtue of picking the optimal action to deploy. We see that the incremental gain increases with n (because we are more likely to pick the correct action); however, this gain can only be accrued for a smaller number, $N - 2n$, of the population. Not suprisingly, we see that the incremental gain decreases with measurement noise, σ. We also see that the incremental gain increases with prior uncertainty, τ, since there is more to be learned by doing an experiment in this case.

The total expected reward is given by adding Eq. (5.44) and Eq. (5.54):

$$\mathbb{E}\left[R\right] = \mathbb{E}\left[R_{\text{test}}\right] + \mathbb{E}\left[R_{\text{roll}}\right] = Nm + (N - 2n)\left(\frac{\sqrt{2}\tau^2}{\sqrt{\pi}\sqrt{2\tau^2 + \frac{2}{n}\sigma^2}}\right) \tag{5.55}$$

5.2.1.3 Optimal sample size

We can maximize the expected reward in Eq. (5.55) to find the optimal sample size for the testing phase, which (from symmetry) satisfies $n_1^* = n_2^* = n^*$, and from $\frac{d}{dn^*}\mathbb{E}\left[R\right] = 0$ satisfies

$$n^* = \sqrt{\frac{N}{4}u^2 + \left(\frac{3}{4}u^2\right)^2} - \frac{3}{4}u^2 \leq \sqrt{N}\frac{\sigma}{2\tau} \tag{5.56}$$

where $u^2 = \frac{\sigma^2}{\tau^2}$. This we see that the optimal sample size n^* increases as the observation noise σ increases, since we need to collect more data to be confident of the right decision. However, the optimal sample size decreases with τ, since a prior belief that the effect size will be large implies we expect to need less data to reach a confident conclusion.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.2.1.4 Expected error rate

For any given sample size n_1, n_2, we can compute the expected error rate, i.e., the probability that we will pick the wrong action. Since the optimal policy is to pick the action with highest mean (in the symmetric case), the probability of error, conditional on knowin μ_j, is given by

$$\Pr(\pi(\mathbf{y}_1, \mathbf{y}_0) = 1 | \mu_1 < \mu_0) = \Pr(Y_1 - Y_0 > 0 | \mu_1 < \mu_0) = 1 - \Phi\left(\frac{\mu_1 - \mu_0}{\sigma\sqrt{\frac{1}{n_1} + \frac{1}{n_0}}}\right) \tag{5.57}$$

Let $\delta = \mu_1 - \mu_0$ be the treatment effect. When δ is positive and large, the error rate is lower, as is to be expected. When δ is smaller, the error rate is higher, but the consequences on expected reward are less important (i.e., the cost of misclassification is reduced).

The above expression assumed that μ_j are known. Since they are not known, we can compute the expected error rate using $\mathbb{E}\left[\Pr(\pi(\mathbf{y}_1, \mathbf{y}_0) = 1 | \mu_1 < \mu_0)\right]$. By symmetry, the quantity $\mathbb{E}\left[\Pr(\pi(\mathbf{y}_1, \mathbf{y}_0) = 0 | \mu_1 > \mu_0)\right]$ is the same. One can show that both quantities are given by

$$\text{Prob. error} = \frac{1}{4} - \frac{1}{2\pi}\arctan\left(\frac{\sqrt{2}\tau}{\sigma}\sqrt{\frac{n_1 n_0}{n_1 + n_0}}\right) \tag{5.58}$$

As expected, the error rate decreases with the sample size n_1 and n_0, increases with observation noise σ, and decreases with variance of the effect size τ.

5.2.1.5 Regret

Finally, we can compute the **regret**, which is the difference between the expected reward given **perfect information** about the true best action and the expected reward due to our policy. Minimizing regret is equivalent to making the expected reward of our policy equal to the best possible reward (which may be high or low, depending on the problem).

An oracle with perfect information about which μ_J is bigger would pick the highest scoring action, and hence get an expected reward of $N\mathbb{E}\left[\max(\mu_1, \mu_2)\right]$. Since we assume $\mu_j \sim \mathcal{N}(m, \tau^2)$, we have

$$\mathbb{E}\left[R|PI\right] = N\left(m + \frac{\tau}{\sqrt{\pi}}\right) \tag{5.59}$$

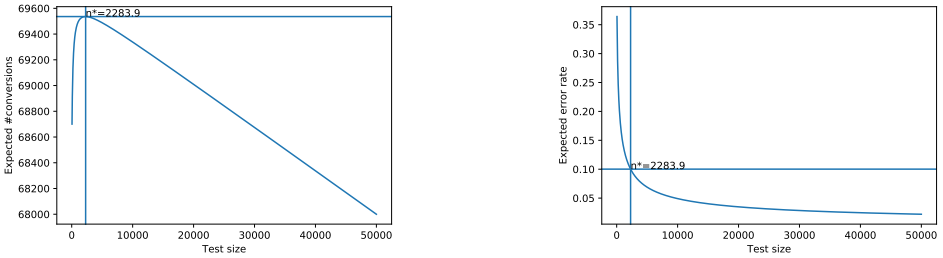
Therefore the regret from the optimal policy is given by

$$\mathbb{E}\left[R|PI\right] - \left(\mathbb{E}\left[R_{\text{test}}|\pi^*\right] + \mathbb{E}\left[R_{\text{roll}}|\pi^*\right]\right) = N\frac{\tau}{\sqrt{\pi}}\left(1 - \frac{\tau}{\sqrt{\tau^2 + \frac{\sigma^2}{n^*}}}\right) + \frac{2n^*\tau^2}{\sqrt{\pi}\sqrt{\tau^2 + \frac{\sigma^2}{n^*}}} \tag{5.60}$$

One can show that the regret is $O(\sqrt{N})$, which is optimal for this problem when using a time horizon (population size) of N [AG13].

5.2.2 Example

In this section, we give a simple example of the above framework. Suppose our goal is to do **website testing**, where have two different versions of a webpage that we want to compare in terms of their



(a) (b)

Figure 5.5: Total expected profit (a) and error rate (b) as a function of the sample size used fo website testing. Generated by code at figures.probml.ai/book1/5.5.

click through rate. The observed data is now binary, $y_{ij} \sim \text{Ber}(\mu_j)$, so it is natural to use a Beta prior, $\mu_j \sim \text{Beta}(\alpha, \beta)$ (see Sec. 4.6.2). However, in this case the optimal sample size and decision rule is harder to compute (see [FB19; Sta+17] for details). As a simple approximation, we can assume $\overline{y}_{ij} \sim \mathcal{N}(\mu_j, \sigma^2)$, where $\mu_j \sim \mathcal{N}(m, \tau^2)$, $m = \frac{\alpha}{\alpha+\beta}$, $\tau^2 = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$, and $\sigma^2 = m(1-m)$.

To set the Gaussian prior, [FB19] used empirical data from about 2000 prior A/B tests. For each test, they observed the number of times the page was served with each of the two variations, as well as the total number of times a user clicked on each version. Given this data, they used a hierarchical Bayesian model to infer $\mu_j \sim \mathcal{N}(m = 0.68, \tau = 0.03)$. This prior implies that the expected effect size is quite small, $\mathbb{E}[|\mu_1 - \mu_0|] = 0.023$. (This is consistent with the results in [Aze+20], who found that most changes made to the Microsoft Bing EXP platform had negligible effect, although there were occasionally some "big hits".)

With this prior, and assuming a population of $N = 100000$, Eq. (5.56) says that the optimal number of trials to run is $n_1^* = n_0^* = 2284$. The expected reward (number of clicks or **conversions**) in the testing phase is $\mathbb{E}[R_{\text{test}}] = 3106$, and in the deployment phase $\mathbb{E}[R_{\text{roll}}] = 66430$, for a total reward of 69536. The expected error rate is 10%.

In Fig. 5.5a, we plot the expected reward vs the size of the test phase n. We see that the reward increases sharply with n to the global maximum at $n^* = 2284$, and then drops off more slowly. This indicates that it is better to have a slightly larger test than one that is too small by the same amount. (However, when using a heavy tailed model, [Aze+20] find that it is better to do lots of smaller tests.)

In Fig. 5.5b, we plot the probability of picking the wrong action vs n. We see that tests that are larger than optimal only reduce this error rate marginally. Consequently, if you want to make the misclassification rate low, you may need a large sample size, particularly if $\mu_1 - \mu_0$ is small, since then it will be hard to detect the true best action. However, it is also less important to identify the best action in this case, since both actions have very similar expected reward. This explains why classical methods for A/B testing based on frequentist statistics, which use hypothesis testing methods to determine if A is better than B (see Sec. 5.7), may often recommend sample sizes that are much larger than necessary. (See [FB19] and references therein for further discussion.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.3 Bandit problems *

This section is co-authored with Lihong Li.

In Sec. 5.2, we discussed A/B testing, in which the decision maker tries two different actions (A and B) a fixed number of times, n_1 and n_0, measures the resulting sequence of rewards, $\mathbf{y}_1$ and $\mathbf{y}_0$, and then picks the best action to use for the rest of time (or the rest of the population) so as to maximize expected reward.

We can obviously generalize this beyond two actions. More importantly, we can generalize this beyond a one-stage decision problem. In particular, suppose we allow the decision maker to try an action a_t, observe the reward r_t, and then decide what to do at time step $t+1$, rather than waiting until $n_1 + n_0$ experiments are finished. This immediate feedback allows for **adaptive policies** that can result in much higher expected reward (lower regret).

We have converted a one-stage decision problem into a **sequential decision problem**. There are many kinds of sequential decision problems, but in this section, we consider the simplest kind, known as a **bandit problem** (see e.g., [BCB12; LS19b; Sli19]).

In a bandit problem, there is an agent (decision maker) which gets to see some **state of nature**, $s_t \in \mathcal{S}$, which it uses to choose an **action** from some **policy** $a_t \sim \pi(s_t)$, and then it finally receives a **reward** sampled from the **environment**, $r_t \sim p_R(s_t, a_t)$. (For future use, we define $R(s,a) = \mathbb{E}[R|s,a]$ as the expected reward.) In the **finite horizon** formulation, the goal is to maximize the expected **cumulative reward**:

$$J \triangleq \sum_{t=1}^{T} \mathbb{E}_{p_R(r_t|s_t,a_t)\pi_t(a_t|s_t)p(s_t)}[r_t] = \sum_{t=1}^{T} \mathbb{E}[r_t] \tag{5.61}$$

where the second expectation without explicit distributions specified is a shorthand when no ambiguity arises. In the **infinite horizon** formulation, where $T = \infty$, the cumulative reward may be infinite. To prevent J from being unbounded, we introduce a **discount factor** $0 < \gamma < 1$, so that

$$J \triangleq \sum_{t=1}^{\infty} \gamma^{t-1} \mathbb{E}[r_t] \tag{5.62}$$

The quantity γ can be interpreted as the probability that the agent is terminated at any moment in time (in which case it will cease to accumulate reward).

We can think of this setup in terms of an agent at a casino who is faced with multiple slot machines, each of which pays out rewards at a different rate. A slot machine is sometimes called a **one-armed bandit**, so a set of such machines is called a **multi-armed bandit** (**MAB**); each different action corresponds to pulling the arm of a different slot machine, but there is just one fixed state (namely being in a given casino). The goal is to quickly figure out which machine pays out the most money, and then to keep playing that one until you become as rich as possible.

5.3.1 Contextual bandits

In the basic bandit problem, the state of nature s_t is fixed, meaning that the world does not change. However, the agent's internal model of the world does change, as it learns about which actions have highest reward. If we allow the state of the environment s_t to vary randomly over time, the model is known as a **contextual bandit**, which is a more flexible model.

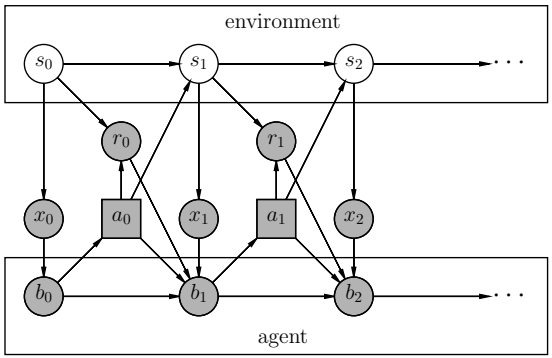


Figure 5.6: Illustration of a partially observable Markov decision process (POMDP) with hidden environment state s_t which generates the observation x_t, controlled by an agent with internal belief state b_t which generates the action a_t. The reward r_t depends on s_t and a_t. Nodes in this graph represent random variables (circles) and decision variables (squares).

For example, consider an **online advertising system**. We can regard the page that the user is currently looking at as the state s_t, and the ad which we choose to show as the action a_t. Since the reward function has the form $R(s_t, a_t)$, we can see that the value of an ad a_t depends on the context s_t. The goal is to maximize the expected reward, which is equivalent to the expected number of times people click on ads; this is known as the **click through rate** or **CTR**. (See e.g., [Gra+10; Li+10; McM+13; Aga+14] for more information about this application.)

Another application of contextual bandits arises in **clinical trials** [VBW15]. In this case, the state s_t are features of the current patient we are treating, and the action a_t is the treatment we choose to give them (e.g., a new drug or a **placebo**). Our goal is to maximize expected reward, i.e., the expected number of people who get cured. (In fact, the goal is often stated in terms of determining which treatment is best, rather than maximizing expected reward; this variant is known as **best-arm identification** [ABM10].)

So far we have assumed the state and reward are sampled from fixed distributions, $s_t \sim p(s_t)$ and $r_t \sim p(r|s_t, a_t)$. This is known as a **stochastic bandit**. It is also possible to allow the reward, and possibly the state, to be chosen in an adversarial manner, where nature tries to minimize the reward of the agent. This is known as an **adversarial bandit**. However, we do not consider this version in this book.

5.3.2 Markov decision processes

We can consider a generalization of the contextual bandit model, in which the next state s_{t+1} depends on s_t and the agent's action a_t; this is called a **Markov decision process** or **MDP**. This defines a **controlled Markov chain**,

$$p(\mathbf{s}_{0:T}|\mathbf{a}_{0:T-1}) = p(s_0) \prod_{t=1}^{T} p(s_t|s_{t-1}, a_{t-1}) \tag{5.63}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $p(s_t|s_{t-1}, a_{t-1})$ is known as the **state transition model**. A further generalization of this assumes that the agent does not see the world state s_t directly, but instead only sees a potentially noisy observation generated from the hidden state, $y_t \sim p(\cdot|s_t, a_t)$; this is called a **partially observable Markov decision process** or **POMDP** (pronounced "pom-dee-pee"). Now the agent's policy is a mapping from all the available data to actions, $a_t \sim \pi(\mathcal{D}_t)$, where $\mathcal{D}_t = (x_0, a_0, r_0, x_1, a_1, r_1, \ldots, x_t)$. See Fig. 5.6 for an illustration. (MDPs are a special case where $x_t = s_t$.) Finding optimal policies for MDPs and POMDPs is studied in the field of **reinforcement learning**.

For more details on MDPs and POMDPs, see e.g., [Koc15]. For more details on RL, see other books such as [Sze10; SB18; Ber19] or the sequel to this book, [Mur22].

5.3.3 Exploration-exploitation tradeoff

The fundamental difficulty in solving bandit problems is known as the **exploration-exploitation tradeoff**. This refers to the fact that the agent needs to try new state/action combinations (this is known as exploration) in order to learn the reward function $R(s, a)$ before it can exploit its knowledge by picking the predicted best action for each state.

One of the simplest approaches to this problem is to use a two-stage strategy. In the first stage, the agent explores, by trying many different actions, and then computes an estimate of the reward function, $\hat{R}(s, a)$. It stops exploring once it is sufficiently confident in all its estimates (as measured by the posterior variance, for example). In the second stage, it exploits its knowledge using the **greedy policy** $\pi(s) = \text{argmax}_a \hat{R}(s, a)$. (This is similar to the A/B testing method in Sec. 5.2.) However, we can often do better by interleaving exploration and exploitation. We discuss various methods for this below.

5.3.4 Optimal solution

In this section, we discuss the optimal solution to the exploration-exploitation tradeoff. We start with a simple example, namely the context-free **Bernoulli bandit**, where $p_R(r|a) = \text{Ber}(r|\mu_a)$, and $\mu_a = p_R(r = 1|a) = R(a)$ is the expected reward for taking action a. Let the estimated reward function have the form $\hat{R}(a; \boldsymbol{\theta}) = \theta_a$, which (in the context-free case) is just a lookup table. The MLE estimate for the reward function at time t, based on the history of observations, $\mathbf{h}_t = \{s_{1:t-1}, a_{1:t-1}, r_{1:t-1}\}$, is given by

$$\hat{\theta}_t(a) = \frac{N_{t,1}(a)}{\max\{N_t(a), 1\}} \tag{5.64}$$

$$N_{t,r}(a) = \sum_{\tau=1}^{t-1} \mathbb{I}(r_t = r)\, \mathbb{I}(a_t = a) \tag{5.65}$$

$$N_t(a) = N_{t,0}(a) + N_{t,1}(a) = \sum_{\tau=1}^{t-1} \mathbb{I}(a_t = a) \tag{5.66}$$

Thus $\hat{\theta}_t(a)$ is just the fraction of times we received a reward of 1 when taking action a.

In order to decide when to stop exploring and start exploiting, we need some notion of confidence in this estimate. The natural approach is to compute the posterior $p(\boldsymbol{\theta}|\mathcal{D}_t)$. If we use a factored beta prior, $p(\boldsymbol{\theta}) = \prod_a \text{Beta}(\theta_a|\alpha, \beta)$, then we can compute the posterior in closed form, as we discuss

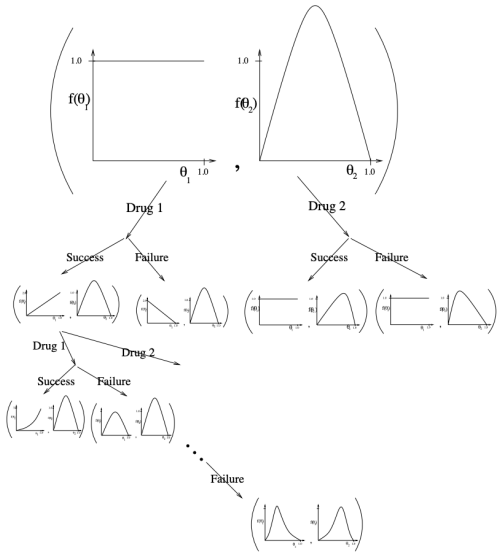


Figure 5.7: Illustration of sequential belief updating for a two-armed beta-Bernoulli bandit, where the arms correspond to different drugs in a clinical trial, and the observed outcomes (rewards) correspond to success or failure of the trial. (Compare to Fig. 4.11, where we display the predictive distribution for a single armed beta-Bernoull model.) From [Sil18]. Used with kind permission of David Silver.

in Sec. 4.6.2. In particular, we find

$$p(\boldsymbol{\theta}|\mathcal{D}_t) = \prod_a \text{Beta}(\theta_a|\alpha + N_{t,1}(a), \beta + N_{t,0}(a)) \tag{5.67}$$

We can use a similar method for a **Gaussian bandit**, where $p_R(r|a) = \mathcal{N}(r|\mu_a, \sigma_a^2)$, using results from Sec. 4.6.4.

In the case of contextual bandits, the problem becomes slightly more complicated. If we assume a conditional Gaussian bandit with a linear reward function, $\mathbb{E}[R|s, a; \boldsymbol{\theta}] = \boldsymbol{\phi}(s, a)^\top \boldsymbol{\theta}$, we can use Bayesian linear regression to compute $p(\boldsymbol{\theta}|\mathcal{D}_t)$ in closed form, as we discuss in Sec. 11.6. If we assume a conditional Bernoulli bandit with a linear-logistic reward function, $\mathbb{E}[R|s, a; \boldsymbol{\theta}] = \sigma(\boldsymbol{\phi}(s, a)^\top \boldsymbol{\theta})$, we can use Bayesian logistic regression to compute $p(\boldsymbol{\theta}|\mathcal{D}_t)$, as we discuss in Sec. 10.5; We can also use nonlinear models, although posterior inference becomes more challenging. Details can be found in the sequel to this book, [Mur22].

In general, let us denote the posterior over the parameters of the reward function by $\mathbf{b}_t = p(\boldsymbol{\theta}|\mathbf{h}_t)$; this is known as othe **belief state** or **information state**. It is a finite sufficient statistic for the history $\mathbf{h}_t$.

It can be verified that the information state can be updated deterministically using Bayes' rule (see Fig. 5.7 for a visualization). An MDP is thus induced, where the state space consists of the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

real-valued vectors $\mathbf{b}_t$, and the transition and reward models are given by

$$p(\mathbf{b}_t|\mathbf{b}_{t-1}, a_t, r_t) = \mathbb{I}\left(\mathbf{b}_t = \text{BayesRule}(\mathbf{b}_{t-1}, a_t, r_t)\right) \tag{5.68}$$

$$p(R|\mathbf{b}_t, a_t = a) = \int p_R(R|\mu_a) p(\mu_a|\mathbf{b}_t) d\mu_a \tag{5.69}$$

This MDP is called the **belief-state MDP**. Since the transition model and reward model are known, we can solve for the optimal MDP policy using dynamic programming. The resulting optimal policy can be represented as a tabular decision tree, similar to Fig. 5.7; this is known as as **Gittins index** [Git89].

Unfortunately, computing the optimal policy for contextual bandits (and MDPs) is much more difficult, so we will need to use other methods. We discuss several options below.

5.3.5 Regret

In the sections below, we will discuss several approximate methods for solving the exploration-exploitation tradeoff. It is useful to define a metric of the degree of suboptimality of these methods. A common approach is to compute the **regret**, which is defined as the difference between the expected reward under the agent's policy and the oracle policy π_*, which knows the true reward function. (Note that the oracle policy will in general be better than the Bayes optimal policy, which we disucssed in Sec. 5.3.4.)

Specifically, let π_t be the agent's policy at time t. Then the **per-step regret** at t is defined as

$$l_t \triangleq \mathbb{E}_{p(s_t)}\left[R(s_t, \pi_*(s_t))\right] - \mathbb{E}_{\pi_t(a_t|s_t)p(s_t)}\left[R(s_t, a_t)\right] \tag{5.70}$$

The **cumulative regret** (**total regret**, or **regret**) at time T is

$$L_T \triangleq \mathbb{E}\left[\sum_{t=1}^{T} l_t\right] \tag{5.71}$$

where the expectation is with respect to randomness in determining π_t, which depends on earlier states, actions and rewards, as well as other potential sources of randomness. By contrast, if we only care about the final performance of the best discovered arm, as in most optimization problems, it is enough to look at the **simple regret** at the last step, namely l_T. This problem is called **pure exploration** [BMS11]. Under the typical assumption that rewards are bounded, L_T is at most linear in T. If the agent's policy converges to the optimal policy as T increases, then the regret is sublinear: $L_T = o(T)$. In general, the slower L_T grows, the more efficient the agent is in trading off exploration and exploitation. To understand its growth rate, it is helpful to consider again the simpler context-free bandit, where $R_* = \text{argmax}_a R(a)$ is the optimal reward. The total regret in the first T steps can be written as

$$L_T = \mathbb{E}\left[\sum_{t=1}^{T} R_* - R(a_t)\right] = \sum_{a \in \mathcal{A}} \mathbb{E}\left[N_{T+1}(a)\right]\left(R_* - R(a)\right) = \sum_{a \in \mathcal{A}} \mathbb{E}\left[N_{T+1}(a)\right] \Delta_a \tag{5.72}$$

where $N_{T+1}(a)$ is the total number of times the agent picks action a, and $\Delta_a = R_* - R(a)$ is the reward **gap**. If the agent under-explores and converges to choosing a suboptimal action (say, $\hat{a}$), then

a linear regret is suffered with a per-step regret of $\Delta_{\hat{a}}$. On the other hand, if the agent over-explores, then $N_t(a)$ will be too large for suboptimal actions, and the agent also suffers a linear regret.

Fortunately, it is possible to achieve sublinear regrets, using some of the methods discussed below. In fact, some of them are optimal in the sense that their regrets are essentially not improvable; that is, they match regret lower bounds. To establish such a lower bound, note that the agent needs to collect enough data to distinguish different reward distributions, before identifying the optimal action. Typically, the deviation of the reward estimate from the true reward decays at the rate of $1/\sqrt{N}$, where N is the sample size (see e.g., Eq. (4.186)). Therefore, if two reward distributions are similar, distinguishing them becomes harder and requires more samples. (For example, consider the case of two arms with Gaussian rewards with slightly different means and large variance, as shown in Fig. 5.8.) A fundamental result is proved by [LR85] for the asymptotic regret (under certain mild assumptions not given here):

$$\liminf_{T\to\infty} L_T \geq \log T \sum_{a:\Delta_a>0} \frac{\Delta_a}{\mathbb{KL}\left(p_R(a)\|p_R(a_*)\right)} \tag{5.73}$$

Thus, we see that the best we can achieve is logarithmic growth in the total regret. Similar lower bounds have also been obtained for various bandits variants.

5.3.6 Upper confidence bounds (UCB)

A particularly effective principle to address the exploration-exploitation tradeoff is know as "**optimism in the face of uncertainty**". The principle selects actions greedily, based on optimistic estimates of their rewards. The most important class of strategies with this principle are collectively called **upper confidence bound** or **UCB** methods.

To use a UCB strategy, the agent maintains an optimistic reward function estimate $\tilde{R}_t$, so that $\tilde{R}_t(s_t, a) \geq R(s_t, a)$ for all a with high probability, and then chooses the greedy action accordingly:

$$a_t = \operatorname*{argmax}_a \tilde{R}_t(s_t, a) \tag{5.74}$$

UCB can be viewed a form of exploration bonus, where the optimistic estimate encourages exploration. Typically, the amount of optimism, $\tilde{R}_t - R$, decreases over time so that the agent gradually reduces exploration. With properly constructed optimistic reward estimates, the UCB strategy has been shown to achieve near-optimal regret in many variants of bandits [LS19b].

The optimistic function $\tilde{R}$ can be obtained in different ways, sometimes in closed forms. One approach is to use a **concentration inequality** [BLM16] to derive a high-probability upper bound of the estimation error: $|\hat{R}_t(s, a) - R_t(s, a)| \leq \delta_t(s, a)$, where $\hat{R}_t$ is a usual estimate of R (often the MLE), and δ_t is a properly selected function. An optimistic reward is then obtained by setting $\tilde{R}_t(s, a) = \hat{R}_t(s, a) + \delta_t(s, a)$.

As an example, consider again the context-free Bernoulli bandit, $R(a) \sim \text{Ber}(\mu(a))$. The MLE $\hat{R}_t(a) = \hat{\mu}_t(a)$ is given by Eq. (5.64). (For simplicity we will assume $N_t(a) > 0$ for all a.) Then the **Chernoff-Hoeffding inequality** [BLM16] leads to $\delta_t(a) = c/\sqrt{N_t(a)}$ for some proper constant c, so

$$\tilde{R}_t(a) = \hat{\mu}_t(a) + \frac{c}{\sqrt{N_t(a)}} \tag{5.75}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

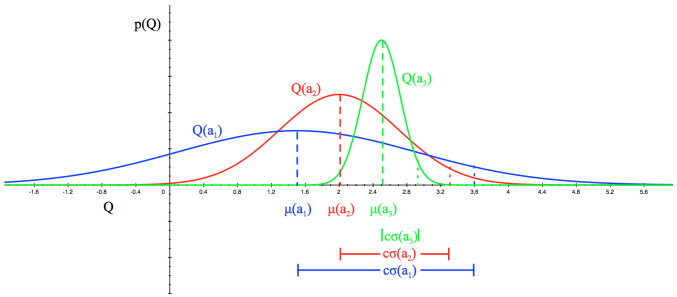


Figure 5.8: Illustration of the reward distribution $Q(a)$ for 3 different actions, and the corresponding lower and upper confidence bounds. From A figure from [Sil18]. Used with kind permission of David Silver.

We may also derive $\tilde{R}$ from Bayesian inference. If we use a beta prior, we can compute the posterior in closed form, as shown in Eq. (5.67). The posterior mean is $\hat{\mu}_t(a) = \mathbb{E}[\mu(a)|\mathcal{D}_t] = \frac{\alpha_t}{\alpha_t+\beta_t}$. From Eq. (4.133), the posterior standard deviation is approximately

$$\hat{\sigma}_t(a) = \sqrt{\mathbb{V}[\mu(a)|\mathcal{D}_t]} \approx \sqrt{\frac{\hat{\mu}_t(a)(1-\hat{\mu}_t(a))}{N_t(a)}} \tag{5.76}$$

We then pick the action

$$a_t = \underset{a}{\operatorname{argmax}}\, \hat{\mu}_t(a) + c\hat{\sigma}_t(a) \tag{5.77}$$

for some constant c that controls how greedy the policy is. We see that this is similar to the frequentist method based on concentration inequalities.

Fig. 5.8 illustrates the UCB principle. We assume there are 3 actions, and we represent $p(R(a)|\mathcal{D}_t)$ using a Gaussian. We show the posterior means μ_a with a vertical dotted line, and the scaled posterior standard deviations $c\sigma(a)$ as a horizontal solid line.

5.3.7 Thompson sampling

Suppose at each step we define the policy to choose action a with probability p, where p is the probability that a is the optimal action:

$$\pi_t(a|s_t, \mathbf{h}_t) = \Pr(a = a_*|s_t, \mathbf{h}_t) = \int \mathbb{I}\left(a = \underset{a'}{\operatorname{argmax}}\, R(s_t, a'; \boldsymbol{\theta})\right) p(\boldsymbol{\theta}|\mathbf{h}_t) d\boldsymbol{\theta} \tag{5.78}$$

This is known as **probability matching** [Sco10]. If the posterior is uncertain, the agent will sample many different actions, resulting in exploration. As the uncertainty decreases, it will start to exploit its knowledge.

We can compute the expression in Eq. (5.78) by using a single Monte Carlo sample $\tilde{\boldsymbol{\theta}}_t \sim p(\boldsymbol{\theta}|\mathbf{h}_t)$. We then plug in this parameter into our reward model, and greedily pick the best action:

$$a_t = \underset{a'}{\operatorname{argmax}}\, R(s_t, a'; \tilde{\boldsymbol{\theta}}_t) \tag{5.79}$$

This will choose actions with exactly the desired probability, since

$$p(a = a_t) = \Pr(a = \operatorname*{argmax}_{a'} R(s_t, a'; \tilde{\boldsymbol{\theta}}_t)) \tag{5.80}$$

$$= \int \mathbb{I}\left(a = \operatorname*{argmax}_{a'} R(s_t, a'; \tilde{\boldsymbol{\theta}}_t)\right) p(\tilde{\boldsymbol{\theta}}_t | \mathbf{h}_t) \tag{5.81}$$

$$= \pi_t(a | s_t, \mathbf{h}_t) \tag{5.82}$$

Thus we see that sampling a reward function and then greedily maximizing the sample correctly implements the desired probability matching rule. This approach is called **Thompson sampling** [Tho33]. Despite its simplicity, this approach can be shown to achieve optimal (logarithmic) regret (see e.g., [Rus+18] for a survey). In addition, it is very easy to implement, and hence is widely used in practice [Gra+10; Sco10; CL11].

5.3.8 Simple heuristics

We have discussed some optimal solutions to the exploration-exploitation tradeoff. However, some of these are difficult to apply, particularly in the context of MDPs. We finish by briefly discussing several simple heuristics that are commonly used.

5.3.8.1 ϵ-greedy

A common approach is to use an **ϵ-greedy** policy π_ϵ, parameterized by $\epsilon \in [0, 1]$. In this case, we pick the greedy action wrt the current model, $a_t = \operatorname{argmax}_a \hat{R}_t(s_t, a)$ with probability $1 - \epsilon$, and a random action with probability ϵ. This rule ensures the agent's continual exploration of all state-action combinations. Unfortunately, this heuristic can be shown to be suboptimal, since it explores every action with at least a constant probability $\epsilon / |\mathcal{A}|$.

5.3.8.2 Boltzmann exploration

A source of inefficiency in the ϵ-greedy rule is that exploration occurs uniformly over all actions. The **Boltzmann policy** can be more efficient, by assigning higher probabilities to explore more promising actions: $\pi_\tau(a|s) = \frac{\exp(\hat{R}_t(s_t,a)/\tau)}{\sum_{a'} \exp(\hat{R}_t(s_t,a')/\tau)}$, where $\tau > 0$ is a temperature parameter that controls how entropic the distribution is (see Fig. 2.9). As τ gets close to 0, π_τ becomes close to a greedy policy. On the other hand, higher values of τ will make $\pi(a|s)$ more uniform, and encourage more exploration. Its action selection probabilities can be much "smoother" with respect to changes in the reward estimates than ϵ-greedy, as illustrated in Table 5.6.

5.3.8.3 Exploration bonuses

Exploration bonus is another simple yet sometimes effective heuristic. The idea is to add a bonus term, $b_t(s_t, a)$, to the reward estimate of action a, and then choose a greedy action accordingly: $a_t = \operatorname{argmax}_a \hat{R}_t(s_t, a) + b_t(s_t, a)$. Typically, the bonus term is large for an action a if it has not been chosen many times (i.e., $N_t(a)$ is small), and decays to 0 as $N_t(a)$ becomes large. These bonuses therefore encourage exploration of rarely selected actions. They are also related to the idea of **intrinisic motivation**, in which the agent performs actions in order to learn about the world, even if there is no external reward (see e.g., [AMH19] for a review).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\hat{R}(s,a_1)$	$\hat{R}(s,a_2)$	$\pi_\epsilon(a\|s_1)$	$\pi_\epsilon(a\|s_2)$	$\pi_\tau(a\|s_1)$	$\pi_\tau(a\|s_2)$
1.00	9.00	0.05	0.95	0.00	1.00
4.00	6.00	0.05	0.95	0.12	0.88
4.90	5.10	0.05	0.95	0.45	0.55
5.05	4.95	0.95	0.05	0.53	0.48
7.00	3.00	0.95	0.05	0.98	0.02
8.00	2.00	0.95	0.05	1.00	0.00

Table 5.6: Comparison of ϵ-greedy policy (with $\epsilon = 0.1$) and Boltzmann policy (with $\tau = 1$) for a simple MDP with 6 states and 2 actions. Adapted from Table 4.1 of [GK19].

Bayes factor $BF(1,0)$	Interpretation
$BF < \frac{1}{100}$	Decisive evidence for M_0
$BF < \frac{1}{10}$	Strong evidence for M_0
$\frac{1}{10} < BF < \frac{1}{3}$	Moderate evidence for M_0
$\frac{1}{3} < BF < 1$	Weak evidence for M_0
$1 < BF < 3$	Weak evidence for M_1
$3 < BF < 10$	Moderate evidence for M_1
$BF > 10$	Strong evidence for M_1
$BF > 100$	Decisive evidence for M_1

Table 5.7: Jeffreys scale of evidence for interpreting Bayes factors.

5.4 Bayesian hypothesis testing

Suppose we have two hypotheses or models, commonly called the **null hypothesis**, M_0, and the **alternative hypothesis**, M_1, and we want to know which one is more likely to be true. This is called **hypothesis testing**.

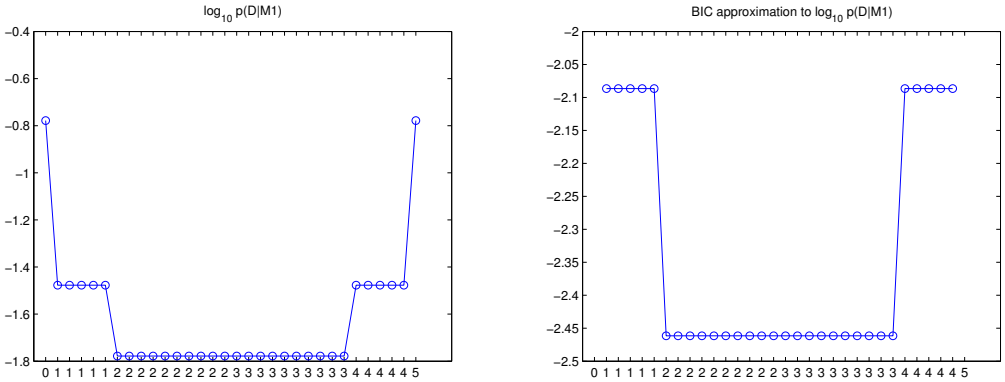
If we use 0-1 loss, the optimal decision is to pick the alternative hypothesis iff $p(M_1|\mathcal{D}) > p(M_0|\mathcal{D})$, or equivalently, if $p(M_1|\mathcal{D})/p(M_0|\mathcal{D}) > 1$. If we use a uniform prior, $p(M_0) = p(M_1) = 0.5$, the decision rule becomes: select M_1 iff $p(\mathcal{D}|M_1)/p(\mathcal{D}|M_0) > 1$. This quantity, which is the ratio of marginal likelihoods of the two models, is known as the **Bayes factor**:

$$B_{1,0} \triangleq \frac{p(\mathcal{D}|M_1)}{p(\mathcal{D}|M_0)} \tag{5.83}$$

This is like a **likelihood ratio**, except we integrate out the parameters, which allows us to compare models of different complexity, due to the Bayesian Occam's razor effect explained in Sec. 5.4.3.

If $B_{1,0} > 1$ then we prefer model 1, otherwise we prefer model 0. Of course, it might be that $B_{1,0}$ is only slightly greater than 1. In that case, we are not very confident that model 1 is better. Jeffreys [Jef61] proposed a scale of evidence for interpreting the magnitude of a Bayes factor, which is shown in Table 5.7. This is a Bayesian alternative to the frequentist concept of a p-value (see Sec. 5.7.3).

We give a worked example of how to compute Bayes factors in Sec. 5.4.1.



(a) (b)

Figure 5.9: (a) Log marginal likelihood vs number of heads for the coin tossing example. (b) BIC approximation. (The vertical scale is arbitrary, since we are holding N fixed.) Generated by code at figures.probml.ai/book1/5.9.

5.4.1 Example: Testing if a coin is fair

As an example, suppose we observe some coin tosses, and want to decide if the data was generated by a fair coin, $\theta = 0.5$, or a potentially biased coin, where θ could be any value in $[0, 1]$. Let us denote the first model by M_0 and the second model by M_1. The marginal likelihood under M_0 is simply

$$p(\mathcal{D}|M_0) = \left(\frac{1}{2}\right)^N \tag{5.84}$$

where N is the number of coin tosses. From Eq. (4.148), the marginal likelihood under M_1, using a Beta prior, is

$$p(\mathcal{D}|M_1) = \int p(\mathcal{D}|\theta)p(\theta)d\theta = \frac{B(\alpha_1 + N_1, \alpha_0 + N_0)}{B(\alpha_1, \alpha_0)} \tag{5.85}$$

We plot $\log p(\mathcal{D}|M_1)$ vs the number of heads N_1 in Fig. 5.9(a), assuming $N = 5$ and a uniform prior, $\alpha_1 = \alpha_0 = 1$. (The shape of the curve is not very sensitive to α_1 and α_0, as long as the prior is symmetric, so $\alpha_0 = \alpha_1$.) If we observe 2 or 3 heads, the unbiased coin hypothesis M_0 is more likely than M_1, since M_0 is a simpler model (it has no free parameters) — it would be a suspicious coincidence if the coin were biased but happened to produce almost exactly 50/50 heads/tails. However, as the counts become more extreme, we favor the biased coin hypothesis. Note that, if we plot the log Bayes factor, $\log B_{1,0}$, it will have exactly the same shape, since $\log p(\mathcal{D}|M_0)$ is a constant.

5.4.2 Bayesian model selection

Now suppose we have a set $\mathcal{M}$ of more than 2 models, and we want to pick the most likely. This is called **model selection**. We can view this as a decision theory problem, where the action space

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

requires choosing one model, $m \in \mathcal{M}$. If we have a 0-1 loss, the optimal action is to pick the most probable model:

$$\hat{m} = \underset{m \in \mathcal{M}}{\operatorname{argmax}}\, p(m|\mathcal{D}) \tag{5.86}$$

where

$$p(m|\mathcal{D}) = \frac{p(\mathcal{D}|m)p(m)}{\sum_{m \in \mathcal{M}} p(\mathcal{D}|m)p(m)} \tag{5.87}$$

is the posterior over models. If the prior over models is uniform, $p(m) = 1/|\mathcal{M}|$, then the MAP model is given by

$$\hat{m} = \underset{m \in \mathcal{M}}{\operatorname{argmax}}\, p(\mathcal{D}|m) \tag{5.88}$$

The quantity $p(\mathcal{D}|m)$ is given by

$$p(\mathcal{D}|m) = \int p(\mathcal{D}|\boldsymbol{\theta}, m)p(\boldsymbol{\theta}|m)d\boldsymbol{\theta} \tag{5.89}$$

This is known as the **marginal likelihood**, or the **evidence** for model m. Intuitively, it is the likelihood of the data averaged over all possible parameter values, weighted by the prior $p(\boldsymbol{\theta}|m)$. If all settings of $\boldsymbol{\theta}$ assign high probability to the data, then this is probably a good model.

5.4.2.1 Example: polynomial regression

As an example of Bayesian model selection, we will consider polynomial regression in 1d. Fig. 5.10 shows the posterior over three different models, corresponding to polynomials of degrees 1, 2 and 3 fit to $N = 5$ data points. We use a uniform prior over models, and use empirical Bayes to estimate the prior over the regression weights (see Sec. 11.6.3). We then compute the evidence for each model (see Sec. 11.6 for details on how to do this). We see that there is not enough data to justify a complex model, so the MAP model is $m = 1$. Fig. 5.11 shows the analogous plot for $N = 30$ data points. Now we see that the MAP model is $m = 2$; the larger sample size means we can safely pick a more complex model.

5.4.3 Occam's razor

Consider two models, a simple one, m_1, and a more complex one, m_2. Suppose that both can explain the data by suitably optimizing their parameters, i.e., for which $p(\mathcal{D}|\hat{\boldsymbol{\theta}}_1, m_1)$ and $p(\mathcal{D}|\hat{\boldsymbol{\theta}}_2, m_2)$ are both large. Intuitively we should prefer m_1, since it is simpler and just as good as m_2. This principle is known as **Occam's razor**.

Let us now see how ranking models based on their marginal likelihood, which involves averaging the likelihood wrt the prior, will give rise to this behavior. The complex model will put less prior probability on the "good" parameters that explain the data, $\hat{\boldsymbol{\theta}}_2$, since the prior must integrate to 1.0 over the entire parameter space. Thus it will take averages in parts of parameter space with low likelihood. By contrast, the simpler model has fewer parameters, so the prior is concentrated

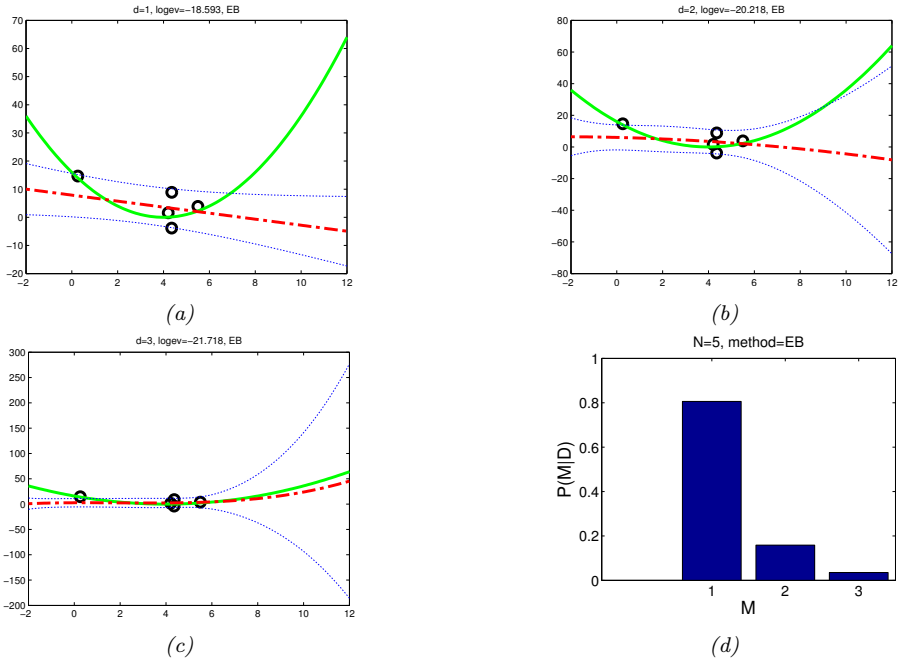


Figure 5.10: Ilustration of Bayesian model selection for polynomial regression. (a-c) We fit polynomials of degrees 1, 2 and 3 fit to $N = 5$ data points. The solid green curve is the true function, the dashed red curve is the prediction (dotted blue lines represent $\pm\sigma$ around the mean). (d) We plot the posterior over models, $p(m|\mathcal{D})$, assuming a uniform prior $p(m) \propto 1$. Adapted from a figure by Zoubin Ghahramani. Generated by code at figures.probml.ai/book1/5.10

over a smaller volume; thus its averages will mostly be in the good part of parameter space, near $\hat{\boldsymbol{\theta}}_1$. Hence we see that the marginal likelihood will prefer the simpler model. This is called the **Bayesian Occam's razor** effect [Mac95; MG05].

Another way to understand the Bayesian Occam's razor effect is to compare the relative predictive abilities of simple and complex models. Since probabilities must sum to one, we have $\sum_{\mathcal{D}'} p(\mathcal{D}'|m) = 1$, where the sum is over all possible datasets. Complex models, which can predict many things, must spread their predicted probability mass thinly, and hence will not obtain as large a probability for any given data set as simpler models. This is sometimes called the **conservation of probability mass** principle, and is illustrated in Fig. 5.12. On the horizontal axis we plot all possible data sets in order of increasing complexity (measured in some abstract sense). On the vertical axis we plot the predictions of 3 possible models: a simple one, M_1; a medium one, M_2; and a complex one, M_3. We also indicate the actually observed data $\mathcal{D}_0$ by a vertical line. Model 1 is too simple and assigns low probability to $\mathcal{D}_0$. Model 3 also assigns $\mathcal{D}_0$ relatively low probability, because it can predict many data sets, and hence it spreads its probability quite widely and thinly. Model 2 is "just right": it predicts the observed data with a reasonable degree of confidence, but does not predict too many other things. Hence model 2 is the most probable model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

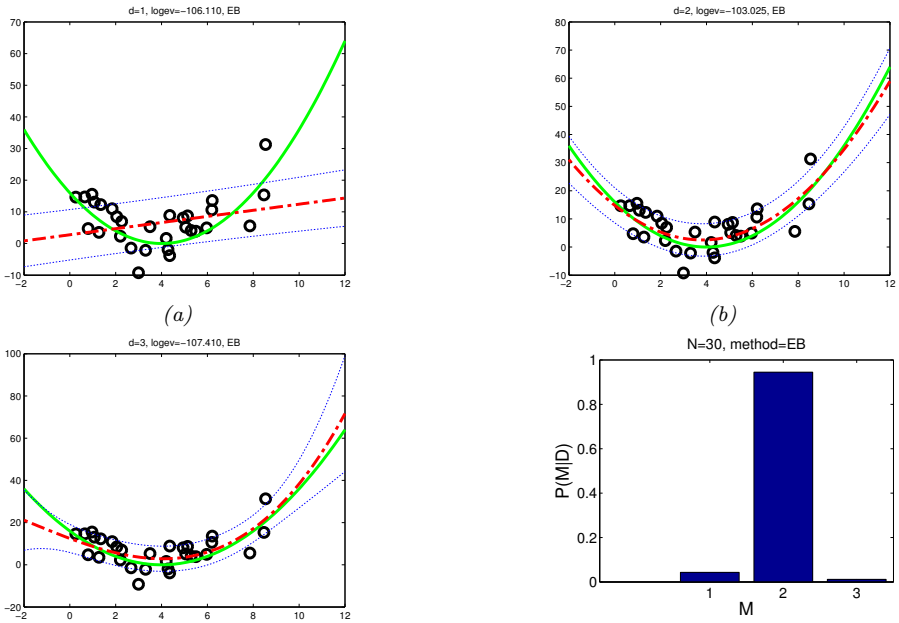


Figure 5.11: Same as Fig. 5.10 except now $N = 30$. Generated by code at figures.probml.ai/book1/5.11

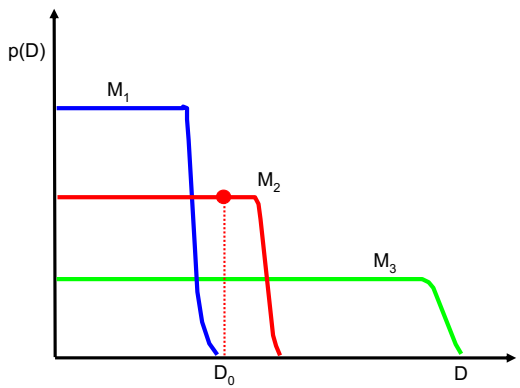


Figure 5.12: A schematic illustration of the Bayesian Occam's razor. The broad (green) curve corresponds to a complex model, the narrow (blue) curve to a simple model, and the middle (red) curve is just right. Adapted from Figure 3.13 of [Bis06]. See also [MG05, Figure 2] for a similar plot produced on real data.

5.4.4 Connection between cross validation and marginal likelihood

We have seen how the marginal likelihood helps us choose models of the "right" complexity. In non-Bayesian approaches to model selection, it is standard to use cross validation (Sec. 4.5.5) for this purpose.

It turns out that the marginal likelihood is closely related to the leave-one-out cross-validation (LOO-CV) estimate, as we now show. We start with the marginal likelihood, which we write in sequential form as follows:

$$p(\mathcal{D}|m) = \prod_{n=1}^{N} p(y_n|y_{1:n-1}, \mathbf{x}_{1:N}, m) = \prod_{n=1}^{N} p(y_n|\mathbf{x}_n, \mathcal{D}_{1:n-1}, m) \tag{5.90}$$

where

$$p(y|\mathbf{x}, \mathcal{D}_{1:n-1}, m) = \int p(y|\mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}_{1:n-1}, m) d\boldsymbol{\theta} \tag{5.91}$$

Suppose we use a plugin approximation to the above distribution to get

$$p(y|\mathbf{x}, \mathcal{D}_{1:n-1}, m) \approx \int p(y|\mathbf{x}, \boldsymbol{\theta}) \delta(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_m(\mathcal{D}_{1:n-1})) d\boldsymbol{\theta} = p(y|\mathbf{x}, \hat{\boldsymbol{\theta}}_m(\mathcal{D}_{1:n-1})) \tag{5.92}$$

Then we get

$$\log p(\mathcal{D}|m) \approx \sum_{n=1}^{N} \log p(y_n|\mathbf{x}_n, \hat{\boldsymbol{\theta}}_m(\mathcal{D}_{1:n-1})) \tag{5.93}$$

This is very similar to a leave-one-out cross-validation estimate of the likelihood, except it is evaluated sequentially. A complex model will overfit the "early" examples and will then predict the remaining ones poorly, and thus will get low marginal likelihood as well as low cross-validation score. See [FH20] for further discussion.

5.4.4.1 Example: ridge regression

In this section, we demonstrate empirically the similarity between the validation loss and the marginal likelihood. We consider a family of ridge regression models with different priors. The likelihood is $p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \beta^{-1})$ and the prior is $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^{-1}\mathbf{I})$. We assume the observation variance $\sigma^2 = 1/\beta$ is known. Let $\lambda = \alpha/\beta$ index the different model families. In Sec. 4.5.5.2, we used cross validation to estimate the optimal λ. In this section, we pick the value of λ with the maximum marginal likelihood. More precisely, we evaluate the log marginal likelihood (evidence) of the model as in Sec. 11.6.3.1. In Fig. 4.7b, we plot the negative log marginal likelihood $\mathcal{L}(\lambda)$ vs $\log(\lambda)$. We see that it has roughly the same shape as the CV curve. In particular, their minima occur for the same value of λ.

Furthermore, since $\mathcal{L}(\lambda)$ is a continuous function of λ (unlike the CV estimate), we can use gradient-based optimization to find $\hat{\lambda}$. This can be easily extended to the case where we have many hyperparameters. See Sec. 11.6.3 for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.4.5 Information criteria *

The marginal likelihood can be difficult to compute, since it requires marginalizing over the entire parameter space. Furthermore, the result can be quite sensitive to the choice of prior. In this section, we discuss some other related metrics for model selection that incorporate a complexity penalty, and which therefore do not require the use of a validation set. These are known as **information criteria**.

The basic idea is to start with the **deviance**, which is two times the negative log likelihood:

$$\text{deviance}(m) \triangleq -2 \sum_{n=1}^{N} \log p(y_n|\hat{\boldsymbol{\theta}}_m, m) \tag{5.94}$$

(The factor of 2 is for historical reasons, since it cancels out the factor of $1/2$ that arises with a Gaussian likelihood.) We know that selecting the model with smallest deviance will result in overfitting, so we add some form of complexity penalty to get

$$J(m) = -2L(m) + C(m) \tag{5.95}$$

where $L(m)$ is the log likelihood, and $C(m)$ is the complexity penalty for model m. We discuss different choices for L and C below. See e.g., [GHV14] for further details.

5.4.5.1 The Bayesian information criterion (BIC)

The **Bayesian information criterion** or **BIC** [Sch78] uses a complexity term of the form

$$C(m) = \log(N) \times \text{dof}(m) \tag{5.96}$$

where N is the size of the training set and "dof" stands for **degrees of freedom** of the model. If we interpret the dof as the number of parameters in the model, then we can derive the BIC as the log marginal likelihood of a Gaussian approximation to the posterior, as we discuss in Sec. 4.6.8.2.

In particular, from Eq. (4.220), we have

$$\log p(\mathcal{D}) \approx \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{\text{map}}) + \log p(\hat{\boldsymbol{\theta}}_{\text{map}}) - \frac{1}{2} \log |\mathbf{H}| \tag{5.97}$$

where $\mathbf{H}$ is the Hessian of the negative log joint $\log p(\mathcal{D}, \boldsymbol{\theta})$ evaluated at the MAP estimate $\hat{\boldsymbol{\theta}}_{\text{map}}$. We see that Eq. (5.97) is the log likelihood plus some penalty terms. If we have a uniform prior, $p(\boldsymbol{\theta}) \propto 1$, we can drop the prior term, and replace the MAP estimate with the MLE, $\hat{\boldsymbol{\theta}}$, yielding

$$\log p(\mathcal{D}) \approx \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}) - \frac{1}{2} \log |\mathbf{H}| \tag{5.98}$$

We now focus on approximating the $\log |\mathbf{H}|$ term, which is sometimes called the **Occam factor**, since it is a measure of model complexity (volume of the posterior distribution). We have $\mathbf{H} = \sum_{i=1}^{N} \mathbf{H}_i$, where $\mathbf{H}_i = \nabla\nabla \log p(\mathcal{D}_i|\boldsymbol{\theta})$. Let us approximate each $\mathbf{H}_i$ by a fixed matrix $\hat{\mathbf{H}}$. Then we have

$$\log |\mathbf{H}| = \log |N\hat{\mathbf{H}}| = \log(N^D|\hat{\mathbf{H}}|) = D \log N + \log |\hat{\mathbf{H}}| \tag{5.99}$$

where $D = \dim(\boldsymbol{\theta})$ and we have assumed $\mathbf{H}$ is full rank. We can drop the $\log|\hat{\mathbf{H}}|$ term, since it is independent of N, and thus will get overwhelmed by the likelihood. Putting all the pieces together, we recover the BIC score

$$\log p(\mathcal{D}) \approx \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}) - \frac{D}{2}\log N \tag{5.100}$$

We can then multiply this by -2 to match Eq. (5.95).

5.4.5.2 Akaike information criterion

The **Akaike information criterion** [Aka74] corresponds to using a complexity term of the form

$$C(m) = 2 \times \text{dof}(m) \tag{5.101}$$

This penalizes complex models less heavily than BIC. It can be derived from a frequentist perspective.

5.4.5.3 Widely applicable information criterion (WAIC)

The main problem with AIC and BIC is that it can be hard to compute the degrees of a freedom of a model, since most parameters are highly correlated and not uniquely identifiable from the likelihood. In particular, if the mapping from parameters to the likelihood is not one-to-one, then the model known as a **singular statistical model**, since the corresponding Fisher information matrix (Sec. 4.7.2), and hence the Hessian $\mathbf{H}$ above, may be singular (have determinant 0). An alternative criterion that works even in the singular case is known as the **widely applicable information criterion** (WAIC), also known as the **Watanabe–Akaike information criterion** [Wat10; Wat13].

In the WAIC, we replace the deviance with average log pointwise predictive density (LPPD) to get

$$\text{LPPD}(m) = \sum_{n=1}^{N} \log \mathbb{E}\left[p(y_n|\mathcal{D}, m)\right] \approx \sum_{n=1}^{N} \log\left(\frac{1}{S}\sum_{s=1}^{S} p(y_n|\boldsymbol{\theta}_s, m)\right) \tag{5.102}$$

where $\boldsymbol{\theta}_s \sim p(\boldsymbol{\theta}|\mathcal{D}, m)$ is a posterior sample. This marginalizes out the parameters. In addition, WAIC approximates the complexity term using

$$C(m) = \sum_{n=1}^{N} \log \mathbb{V}\left[p(y_n|\mathcal{D}, m)\right] \approx \sum_{n=1}^{N} \log\left(\mathbb{V}\{p(y_n|\boldsymbol{\theta}_s, m) : s = 1 : S\}\right) \tag{5.103}$$

The final criterion is

$$\text{WAIC}(m) = -2\text{LPPD}(m) + 2C(m) \tag{5.104}$$

See Fig. 21.18 for an example, where we use this method to select the number of components K in a Gaussian mixture model.

5.4.5.4 Minimum description length (MDL)

We can think about the problem of scoring different models in terms of information theory (Chapter 6). The goal is for the sender to communicate the data to the receiver. First the sender needs to specify

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

which model m to use; this takes $C(m) = -\log p(m)$ bits (see Sec. 6.1). Then the receiver can fit the model, by computing $\hat{\boldsymbol{\theta}}_m$, and can thus approximately reconstruct the data. To perfectly reconstruct the data, the sender needs to send the residual errors that cannot be explained by the model; this takes $-L(m) = -\sum_n \log p(y_n|\hat{\boldsymbol{\theta}}_m, m)$ bits. The total cost is $J(m) = -L(m) + C(m)$. Choosing the model which minimizes $J(m)$ is known as the **minimum description length** or **MDL** principle. See e.g., [HY01] for details.

5.5 Frequentist decision theory

In this section, we discuss **frequentist decision theory**. This is similar to Bayesian decision theory, discussed in Sec. 5.1, but differs in that there is no prior, and hence no posterior, over the unknown state of nature. Consequently we cannot define the risk as the posterior expected loss. We will consider other definitions in Sec. 5.5.1.

5.5.1 Computing the risk of an estimator

We define the frequentist **risk** of an estimator π given an unknown state of nature $\boldsymbol{\theta}$ to be the expected loss when applying that estimator to data $\mathbf{x}$ sampled from the likelihood function $p(\mathbf{x}|\boldsymbol{\theta})$:

$$R(\boldsymbol{\theta}, \pi) \triangleq \mathbb{E}_{p(\mathbf{x}|\boldsymbol{\theta})}\left[\ell(\boldsymbol{\theta}, \pi(\mathbf{x}))\right] \tag{5.105}$$

We give an example of this in Sec. 5.5.1.1.

5.5.1.1 Example

Let us give an example, based on [BS94]. Consider the problem of estimating the mean of a Gaussian. We assume the data is sampled from $x_n \sim \mathcal{N}(\theta^*, \sigma^2 = 1)$. If we use quadratic loss, $\ell_2(\theta, \hat{\theta}) = (\theta - \hat{\theta})^2$. the corresponding risk function is the MSE.

We now consider 5 different estimators for computing θ:

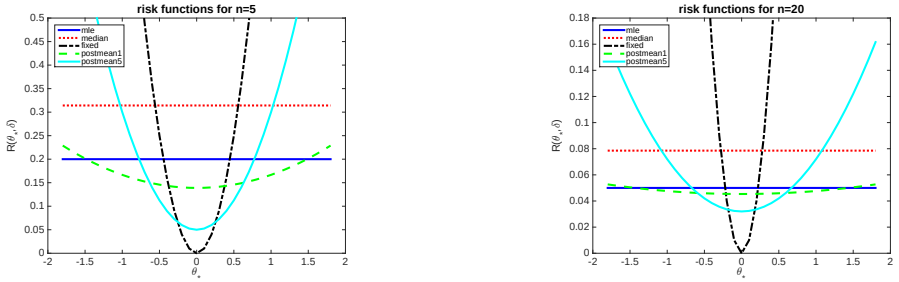
- $\pi_1(\mathcal{D}) = \overline{x}$, the sample mean
- $\pi_2(\mathcal{D}) = \text{median}(\mathcal{D})$, the sample median
- $\pi_3(\mathcal{D}) = \theta_0$, a fixed value
- $\pi_\kappa(\mathcal{D})$, the posterior mean under a $\mathcal{N}(\theta|\theta_0, \sigma^2/\kappa)$ prior:

$$\pi_\kappa(\mathcal{D}) = \frac{N}{N+\kappa}\overline{x} + \frac{\kappa}{N+\kappa}\theta_0 = w\overline{x} + (1-w)\theta_0 \tag{5.106}$$

For π_κ, we use $\theta_0 = 0$, and consider a weak prior, $\kappa = 1$, and a stronger prior, $\kappa = 5$.

Let $\hat{\theta} = \hat{\theta}(\mathbf{x}) = \pi(\mathbf{x})$ be the estimated parameter. The risk of this estimator is given by the MSE. In Sec. 4.7.6.3, we show that the MSE can be decomposed into squared bias plus variance:

$$\text{MSE}(\hat{\theta}|\theta^*) = \mathbb{V}\left[\hat{\theta}\right] + \text{bias}^2(\hat{\theta}) \tag{5.107}$$



(a) (b)

Figure 5.13: Risk functions for estimating the mean of a Gaussian. Each curve represents $R(\hat{\theta}_i(\cdot), \theta^*)$ plotted vs θ^*, where i indexes the estimator. Each estimator is applied to N samples from $\mathcal{N}(\theta^*, \sigma^2 = 1)$. The dark blue horizontal line is the sample mean (MLE); the red line horizontal line is the sample median; the black curved line is the estimator $\hat{\theta} = \theta_0 = 0$; the green curved line is the posterior mean when $\kappa = 1$; the light blue curved line is the posterior mean when $\kappa = 5$. (a) $N = 5$ samples. (b) $N = 20$ samples. Adapted from Figure B.1 of [BS94]. Generated by code at figures.probml.ai/book1/5.13.

where the bias is defined as $\text{bias}(\hat{\theta}) = \mathbb{E}\left[\hat{\theta} - \theta^*\right]$. We now use this expression to derive the risk for each estimator.

π_1 is the sample mean. This is unbiased, so its risk is

$$\text{MSE}(\pi_1|\theta^*) = \mathbb{V}\left[\overline{x}\right] = \frac{\sigma^2}{N} \tag{5.108}$$

π_2 is the sample median. This is also unbiased. Furthermore, one can show that its variance is approximately $\pi/(2N)$, so the risk is

$$\text{MSE}(\pi_2|\theta^*) = \frac{\pi}{2N} \tag{5.109}$$

π_3 returns the constant θ_0, so its bias is $(\theta^* - \theta_0)$ and its variance is zero. Hence the risk is

$$\text{MSE}(\pi_3|\theta^*) = (\theta^* - \theta_0)^2 \tag{5.110}$$

Finally, π_4 is the posterior mean under a Gaussian prior. We can derive its MSE as follows:

$$\text{MSE}(\pi_\kappa|\theta^*) = \mathbb{E}\left[(w\overline{x} + (1-w)\theta_0 - \theta^*)^2\right] \tag{5.111}$$

$$= \mathbb{E}\left[(w(\overline{x} - \theta^*) + (1-w)(\theta_0 - \theta^*))^2\right] \tag{5.112}$$

$$= w^2\frac{\sigma^2}{N} + (1-w)^2(\theta_0 - \theta^*)^2 \tag{5.113}$$

$$= \frac{1}{(N+\kappa)^2}\left(N\sigma^2 + \kappa^2(\theta_0 - \theta^*)^2\right) \tag{5.114}$$

These functions are plotted in Fig. 5.13 for $N \in \{5, 20\}$. We see that in general, the best estimator depends on the value of θ^*, which is unknown. If θ^* is very close to θ_0, then π_3 (which just predicts

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

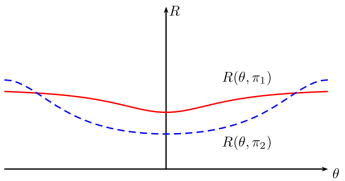


Figure 5.14: Risk functions for two decision procedures, π_1 and π_2. Since π_1 has lower worst case risk, it is the minimax estimator, even though π_2 has lower risk for most values of θ. Thus minimax estimators are overly conservative.

θ_0) is best. If θ^* is within some reasonable range around θ_0, then the posterior mean, which combines the prior guess of θ_0 with the actual data, is best. If θ^* is far from θ_0, the MLE is best.

5.5.1.2 Bayes risk

In general, the true state of nature $\boldsymbol{\theta}$ that generates the data $\mathbf{x}$ is unknown, so we cannot compute the risk given in Eq. (5.105). One solution to this is to assume a prior ρ for $\boldsymbol{\theta}$, and then average it out. This gives us the **Bayes risk**, also called the **integrated risk**:

$$R(\rho, \pi) \triangleq \mathbb{E}_{\rho(\boldsymbol{\theta})}\left[R(\boldsymbol{\theta}, \pi)\right] = \int d\boldsymbol{\theta}\, d\mathbf{x}\, \rho(\boldsymbol{\theta}) p(\mathbf{x}|\boldsymbol{\theta}) \ell(\boldsymbol{\theta}, \pi(\mathbf{x})) \tag{5.115}$$

A decision rule that minimizes the Bayes risk is known as a **Bayes estimator**. This is equivalent to the optimal policy recommended by Bayesian decision theory in Eq. (5.2) since

$$\pi(\mathbf{x}) = \operatorname*{argmin}_a \int d\boldsymbol{\theta}\, \rho(\boldsymbol{\theta}) p(\mathbf{x}|\boldsymbol{\theta}) \ell(\boldsymbol{\theta}, a) = \operatorname*{argmin}_a \int d\boldsymbol{\theta}\, p(\boldsymbol{\theta}|\mathbf{x}) \ell(\boldsymbol{\theta}, a) \tag{5.116}$$

Hence we see that the picking the optimal action on a case-by-case basis (as in the Bayesian approach) is optimal on average (as in the frequentist approach). In other words, the Bayesian approach provides a good way of achieving frequentist goals. See [BS94, p448] for further discussion of this point.

5.5.1.3 Maximum risk

Of course the use of a prior might seem undesirable in the context of frequentist statistics. We can therefore define the **maximum risk** as follows:

$$R_{\max}(\pi) \triangleq \sup_{\boldsymbol{\theta}} R(\boldsymbol{\theta}, \pi) \tag{5.117}$$

A decision rule that minimizes the maximum risk is called a **minimax estimator**, and is denoted π_{MM}. For example, in Fig. 5.14, we see that π_1 has lower worst-case risk than π_2, ranging over all possible values of $\boldsymbol{\theta}$, so it is the minimax estimator.

Minimax estimators have a certain appeal. However, computing them can be hard. And furthermore, they are very pessimistic. In fact, one can show that all minimax estimators are equivalent to Bayes

estimators under a **least favorable prior**.[3] In most statistical situations (excluding game theoretic ones), assuming nature is an adversary is not a reasonable assumption.

5.5.2 Consistent estimators

Suppose we have a dataset $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$ where the samples $\mathbf{x}_n \in \mathcal{X}$ are generated from a distribution $p(\mathbf{x}|\boldsymbol{\theta}^*)$, where $\boldsymbol{\theta}^* \in \Theta$ is the true parameter. Furthermore, suppose the parameters are **identifiable**, meaning that $p(\mathcal{D}|\boldsymbol{\theta}) = p(\mathcal{D}|\boldsymbol{\theta}')$ iff $\boldsymbol{\theta} = \boldsymbol{\theta}'$ for any dataset $\mathcal{D}$. Then we say that an estimator $\pi : \mathcal{X}^N \to \Theta$ is a **consistent estimator** if $\hat{\boldsymbol{\theta}}(\mathcal{D}) \to \boldsymbol{\theta}^*$ as $N \to \infty$ (where the arrow denotes convergence in probability). In other words, the procedure π recovers the true parameter (or a subset of it) in the limit of infinite data. This is equivalent to minimizing the 0-1 loss, $\mathcal{L}(\boldsymbol{\theta}^*, \hat{\boldsymbol{\theta}}) = \mathbb{I}\left(\boldsymbol{\theta}^* \neq \hat{\boldsymbol{\theta}}\right)$. An example of a consistent estimator is the maximum likelihood estimator (MLE).

Note that an estimator can be unbiased but not consistent. For example, consider the estimator $\pi(\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}) = \mathbf{x}_N$. This is an unbiased estimator of the mean, since $\mathbb{E}[\pi(\mathcal{D})] = \mathbb{E}[\mathbf{x}]$. But the sampling distribution of $\pi(\mathcal{D})$ does not converge to a fixed value, so it cannot converge to the point $\boldsymbol{\theta}^*$.

Although consistency is a desirable property, it is of somewhat limited usefulness in practice since most real datasets do not come from our chosen model family (i.e., there is no $\boldsymbol{\theta}^*$ such that $p(\cdot|\boldsymbol{\theta}^*)$ generates the observed data $\mathcal{D}$). In practice, it is more useful to find estimators that minimize some discrepancy measure between the empirical distribution $p_D(\mathbf{x}|\mathcal{D})$ and the estimated distribution $p(\mathbf{x}|\hat{\boldsymbol{\theta}})$. If we use KL divergence as our discrepancy measure, our estimate becomes the MLE.

5.5.3 Admissible estimators

In Fig. 5.13, we see that the sample median (dotted red line) always has higher risk than the sample mean (solid black line), at least under the Gaussian likelihood model. We therefore say that the sample mean **dominates** the sample median as an estimator.

We say that π_1 **dominates** π_2 if $R(\boldsymbol{\theta}, \pi_1) \leq R(\boldsymbol{\theta}, \pi_2)$ for all $\boldsymbol{\theta}$. The domination is said to be strict if the inequality is strict for some $\boldsymbol{\theta}^*$. An estimator is said to be **admissible** if it is not strictly dominated by any other estimator.

In Fig. 5.13, we see that the sample median (dotted red line) always has higher risk than the sample mean (solid black line). Therefore the sample median is not an admissible estimator for the mean. However, this conclusion only applies to the Gaussian likelihood model. If the true model for $p(x|\theta)$ is something heavy-tailed, like a Laplace or Student distribution, or a Gaussian mixture, then the sample median is likely to have lower risk, since it is more robust to outliers.

More surprisingly, one can show that the sample mean is not always an admissible estimator either, even under a Gaussian likelihood model with squared error loss (this is known as **Stein's paradox** [Ste56]).

It is natural to restrict our attention to admissable estimators. [Wal47] showed the following important result (known as the **complete class theorem**), which essentially says that this restricts us to Bayesian estimators.

3. In the context of bandit algorithms (Sec. 5.3), one can similarly show that the worst-case regret that an optimal Bayesian algorithm can suffer is the same as the minimax regret, as shown in [LS19c].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Theorem 5.5.1 (Wald)**.** *Every admissable frequentist decision rule is a Bayes decision rule with respect to some, possibly improper, prior distribution.*

However, the concept of admissibility is of somewhat limited usefulness. For example, it is easy to construct admissible but "silly" estimators, as we show in the following example.

Theorem 5.5.2. *Let $X \sim \mathcal{N}(\theta, 1)$, and consider estimating θ under squared loss. Let $\pi_1(x) = \theta_0$, a constant* independent of the data*. This is an admissible estimator.*

Proof. Suppose not. Then there is some other estimator π_2 with smaller risk, so $R(\theta^*, \pi_2) \leq R(\theta^*, \pi_1)$, where the inequality must be strict for some θ^*. Consider the risk at $\theta^* = \theta_0$. We have $R(\theta_0, \pi_1) = 0$, and

$$R(\theta_0, \pi_2) = \int (\pi_2(x) - \theta_0)^2 p(x|\theta_0) dx \tag{5.118}$$

Since $0 \leq R(\theta^*, \pi_2) \leq R(\theta^*, \pi_1)$ for all θ^*, and $R(\theta_0, \pi_1) = 0$, we have $R(\theta_0, \pi_2) = 0$ and hence $\pi_2(x) = \theta_0 = \pi_1(x)$. Thus the only way π_2 can avoid having higher risk than π_1 at θ_0 is by being equal to π_1. Hence there is no other estimator π_2 with strictly lower risk, so π_2 is admissible. □

Note that the estimator $\pi_1(x) = \theta_0$ is equivalent to a Bayes decision rule with respect to a prior distribution concentrated on θ_0 with certainty, and that this is why it is silly.

5.6 Empirical risk minimization

In this section, we consider how to apply frequentist decision theory in the context of supervised learning.

5.6.1 Empirical risk

In standard accounts of frequentist decision theory used in statistics textbooks, there is a single unknown "state of nature", corresponding to the unknown parameters $\boldsymbol{\theta}^*$ of some model, and we define the risk as in Eq. (5.105), namely $R(\pi, \boldsymbol{\theta}^*) = \mathbb{E}_{p(\mathcal{D}|\boldsymbol{\theta}^*)}[\ell(\boldsymbol{\theta}^*, \pi(\mathcal{D}))]$.

In supervised learning, we have a different unknown state of nature (namely the output y) for each input $\mathbf{x}$, and our estimator π is a prediction function $\hat{y} = f(\mathbf{x})$, and the state of nature is the true distribution $p^*(\mathbf{x}, \mathbf{y})$. Thus the risk of an estimator as follows:

$$R(f, p^*) = R(f) \triangleq \mathbb{E}_{p^*(\mathbf{x})p^*(y|\mathbf{x})}[\ell(\mathbf{y}, f(\mathbf{x})] \tag{5.119}$$

This is called the **population risk**, since the expectations are taken wrt the true joint distribution $p^*(\mathbf{x}, \mathbf{y})$. Of course, p^* is unknown, but we can approximate it using the empirical distribution with N samples:

$$p_D(\mathbf{x}, \mathbf{y}|\mathcal{D}) \triangleq \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}_n, \mathbf{y}_n) \in \mathcal{D}} \delta(\mathbf{x} - \mathbf{x}_n)\delta(\mathbf{y} - \mathbf{y}_n) \tag{5.120}$$

where $p_D(\mathbf{x}, \mathbf{y}) = p_{\text{train}}(\mathbf{x}, \mathbf{y})$. Plugging this in gives us the **empirical risk**:

$$R(f, \mathcal{D}) \triangleq \mathbb{E}_{p_D(\mathbf{x},\mathbf{y})}[\ell(\mathbf{y}, f(\mathbf{x})] = \frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{y}_n, f(\mathbf{x}_n)) \tag{5.121}$$

Note that $R(f, \mathcal{D})$ is a random variable, since it depends of the training set.

A natural way to choose the predictor is to use

$$\hat{f}_{\text{ERM}} = \underset{f \in \mathcal{H}}{\text{argmin}}\, R(f, \mathcal{D}) = \underset{f \in \mathcal{H}}{\text{argmin}}\, \frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{y}_n, f(\mathbf{x}_n)) \tag{5.122}$$

where we optimize over a specific **hypothesis space** $\mathcal{H}$ of functions. This is called **empirical risk minimization** (**ERM**).

5.6.1.1 Approximation error vs estimation error

In this section, we analyze the theoretical performance of functions that are fit using the ERM principle. Let $f^{**} = \text{argmin}_f R(f)$ be the function that achieves the minimal possible population risk, where we optimize over all possible functions. Of course, we cannot consider all possible functions, so let us also define $f^* = \text{argmin}_{f \in \mathcal{H}} R(f)$ to be the best function in our hypothesis space, $\mathcal{H}$. Unfortunately we cannot compute f^*, since we cannot compute the population risk, so let us finally define the prediction function that minimizes the empirical risk in our hypothesis space:

$$f_N^* = \underset{f \in \mathcal{H}}{\text{argmin}}\, R(f, \mathcal{D}) = \underset{f \in \mathcal{H}}{\text{argmin}}\, \mathbb{E}_{p_{\text{train}}} [\ell(\mathbf{y}, f(\mathbf{x}))] \tag{5.123}$$

One can show [BB08] that the risk of our chosen predictor compared to the best possible predictor can be decomposed into two terms, as follows:

$$\mathbb{E}_{p^*} [R(f_N^*) - R(f^{**})] = \underbrace{R(f^*) - R(f^{**})}_{\mathcal{E}_{\text{app}}(\mathcal{H})} + \underbrace{\mathbb{E}_{p^*} [R(f_N^*) - R(f^*)]}_{\mathcal{E}_{\text{est}}(\mathcal{H}, N)} \tag{5.124}$$

The first term, $\mathcal{E}_{\text{app}}(\mathcal{H})$, is the **approximation error**, which measures how closely $\mathcal{H}$ can model the true optimal function f^{**}. The second term, $\mathcal{E}_{\text{est}}(\mathcal{H}, N)$, is the **estimation error** or **generalization error**, which measures the difference in estimated risks due to having a finite training set. We can approximate this by the difference between the training set error and the test set error, using two empirical distributions drawn from p^*:

$$\mathbb{E}_{p^*} [R(f_N^*) - R(f^*)] \approx \mathbb{E}_{p_{\text{train}}} [\ell(\mathbf{y}, f_N^*(\mathbf{x}))] - \mathbb{E}_{p_{\text{test}}} [\ell(\mathbf{y}, f_N^*(\mathbf{x}))] \tag{5.125}$$

This difference is often called the **generalization gap**.

We can decrease the approximation error by using a more expressive family of functions $\mathcal{H}$, but this usually increases the generalization error, due to overfitting. We discuss solutions to this tradeoff below.

5.6.1.2 Regularized risk

To avoid the chance of overfitting, it is common to add a complexity penalty to the objective function, giving us the **regularized empirical risk**:

$$R_\lambda(f, \mathcal{D}) = R(f, \mathcal{D}) + \lambda C(f) \tag{5.126}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $C(f)$ measures the complexity of the prediction function $f(\mathbf{x};\boldsymbol{\theta})$ and $\lambda \geq 0$, which is known as a **hyperparameter**, controls the strength of the complexity penalty. (We discuss how to pick λ in Sec. 5.6.2.)

In practice, we usually work with parametric functions, and apply the regularizer to the parameters themselves. This yields the following form of the objective:

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = R(\boldsymbol{\theta}, \mathcal{D}) + \lambda C(\boldsymbol{\theta}) \tag{5.127}$$

Note that, if the loss function is log loss, and the regularizer is a negative log prior, the regularized risk is given by

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = -\frac{1}{N}\sum_{n=1}^{N} \log p(\mathbf{y}_n|\mathbf{x}_n, \boldsymbol{\theta}) - \lambda \log p(\boldsymbol{\theta}) \tag{5.128}$$

Minimizing this is equivalent to MAP estimation.

5.6.2 Structural risk

A natural way to estimate the hyperparameters is to minimize for the lowest achievable empirical risk:

$$\hat{\lambda} = \operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R_\lambda(\boldsymbol{\theta}, \mathcal{D}) \tag{5.129}$$

(This is an example of **bilevel optimization**, also called **nested optimization**.) Unfortunately, this technique will not work, since it will always pick the least amount of regularization, i.e., $\hat{\lambda} = 0$. To see this, note that

$$\operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = \operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R(\boldsymbol{\theta}, \mathcal{D}) + \lambda C(\boldsymbol{\theta}) \tag{5.130}$$

which is minimized by setting $\lambda = 0$ The problem is that the empirical risk underestimates the population risk, resulting in overfitting when we choose λ. This is called **optimism of the training error**.

If we knew the regularized population risk $R_\lambda(\boldsymbol{\theta})$, instead of the regularized empirical risk $R_\lambda(\boldsymbol{\theta}, \mathcal{D})$, we could use it to pick a model of the right complexity (e.g., value of λ). This is known as **structural risk minimization** [Vap98]. There are two main ways to estimate the population risk for a given model (value of λ), namely cross-validation (Sec. 5.6.3), and statistical learning theory (Sec. 5.6.4), which we discuss below.

5.6.3 Cross-validation

In this section, we discuss a simple way to estimate the population risk for a supervised learning setup. We simply partition the dataset into two, the part used for training the model, and a second part, called the **validation set** or **holdout set**, used for assessing the risk. We can fit the model on the training set, and use its performance on the validation set as an approximation to the population risk.

To explain the method in more detail, we need some notation. First we make the dependence of the empirical risk on the dataset more explicit as follows:

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \ell(\mathbf{y}, f(\mathbf{x};\boldsymbol{\theta})) + \lambda C(\boldsymbol{\theta}) \tag{5.131}$$

Let us also define $\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}) = \text{argmin}_{\boldsymbol{\theta}} R_\lambda(\mathcal{D}, \boldsymbol{\theta})$. Finally, let $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{valid}}$ be a partition of $\mathcal{D}$. (Often we use about 80% of the data for the training set, and 20% for the validation set.)

For each model λ, we fit it to the training set to get $\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}})$. We then use the unregularized empirical risk on the validation set as an estimate of the population risk. This is known as the **validation risk**:

$$R_\lambda^{\text{val}} \triangleq R_0(\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}}), \mathcal{D}_{\text{valid}}) \tag{5.132}$$

Note that we use different data to train and evaluate the model.

The above technique can work very well. However, if the number of training cases is small, this technique runs into problems, because the model won't have enough data to train on, and we won't have enough data to make a reliable estimate of the future performance.

A simple but popular solution to this is to use **cross validation** (**CV**). The idea is as follows: we split the training data into K **folds**; then, for each fold $k \in \{1, \ldots, K\}$, we train on all the folds but the k'th, and test on the k'th, in a round-robin fashion, as sketched in Fig. 4.6. Formally, we have

$$R_\lambda^{\text{cv}} \triangleq \frac{1}{K} \sum_{k=1}^{K} R_0(\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{-k}), \mathcal{D}_k) \tag{5.133}$$

where $\mathcal{D}_k$ is the data in the k'th fold, and $\mathcal{D}_{-k}$ is all the other data. This is called the **cross-validated risk**. Fig. 4.6 illustrates this procedure for $K = 5$. If we set $K = N$, we get a method known as **leave-one-out cross-validation**, since we always train on $N - 1$ items and test on the remaining one.

We can use the CV estimate as an objective inside of an optimization routine to pick the optimal hyperparameter, $\hat{\lambda} = \text{argmin}_\lambda R_\lambda^{\text{cv}}$. Finally we combine all the available data (training and validation), and re-estimate the model parameters using $\hat{\boldsymbol{\theta}} = \text{argmin}_{\boldsymbol{\theta}} R_{\hat{\lambda}}(\boldsymbol{\theta}, \mathcal{D})$.

5.6.4 Statistical learning theory *

The principal problem with cross validation is that it is slow, since we have to fit the model multiple times. This motivates the desire to compute analytic approximations or bounds on the population risk. This is the studied in the field of **statistical learning theory** (SLT) (see e.g., [Vap98]).

More precisely, the goal of SLT is to upper bound the generalization error with a certain probability. If the bound is satisfied, then we can be confident that a hypothesis that is chosen by minimizing the empirical risk will have low population risk. In the case of binary classifiers, this means the hypothesis will make the correct predictions; in this case we say it is **probably approximately correct**, and that the hypothesis class is **PAC learnable** (see e.g., [KV94] for details).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.6.4.1 Bounding the generalization error

In this section, we establish conditions under which we can prove that a hypothesis class is PAC learnable. Let us initially consider the case where the hypothesis space is finite, with size $\dim(\mathcal{H}) = |\mathcal{H}|$. In other words, we are selecting a hypothesis from a finite list, rather than optimizing real-valued parameters. In this case, we can prove the following.

Theorem 5.6.1. *For any data distribution p^*, and any dataset $\mathcal{D}$ of size N drawn from p^*, the probability that the generalization error of a binary classifier will be more than ϵ, in the worst case, is upper bounded as follows:*

$$P\left(\max_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) \leq 2\dim(\mathcal{H})e^{-2N\epsilon^2} \tag{5.134}$$

where $R(h,\mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left(f(\mathbf{x}_i) \neq y_i^*\right)$ is the empirical risk, and $R(h) = \mathbb{E}\left[\mathbb{I}\left(f(\mathbf{x}) \neq y^*\right)\right]$ is the population risk.

Proof. Before we prove this, we introduce two useful results. First, **Hoeffding's inequality**, which states that if $E_1, \ldots, E_N \sim \text{Ber}(\theta)$, then, for any $\epsilon > 0$,

$$P(|\overline{E} - \theta| > \epsilon) \leq 2e^{-2N\epsilon^2} \tag{5.135}$$

where $\overline{E} = \frac{1}{N}\sum_{i=1}^{N} E_i$ is the empirical error rate, and θ is the true error rate. Second, the **union bound**, which says that if $A_1, \ldots, A_d$ are a set of events, then $P(\cup_{i=1}^{d} A_i) \leq \sum_{i=1}^{d} P(A_i)$. Using these results, we have

$$P\left(\max_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) = P\left(\bigcup_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) \tag{5.136}$$

$$\leq \sum_{h\in\mathcal{H}} P\left(|R(h) - R(h,\mathcal{D})| > \epsilon\right) \tag{5.137}$$

$$\leq \sum_{h\in\mathcal{H}} 2e^{-2N\epsilon^2} = 2\dim(\mathcal{H})e^{-2N\epsilon^2} \tag{5.138}$$

□

Ths bound tells us that the optimism of the training error increases with $\dim(\mathcal{H})$ but decreases with $N = |\mathcal{D}|$, as is to be expected.

5.6.4.2 VC dimension

If the hypothesis space $\mathcal{H}$ is infinite (e.g., we have real-valued parameters), we cannot use $\dim(\mathcal{H}) = |\mathcal{H}|$. Instead, we can use a quantity called the **VC dimension** of the hypothesis class, named after Vapnik and Chervonenkis; this measures the degrees of freedom (effective number of parameters) of the hypothesis class. See e.g., [Vap98] for the details.

Unfortunately, it is hard to compute the VC dimension for many interesting models, and the upper bounds are usually very loose, making this approach of limited practical value. However, various other, more practical, estimates of generalization error have recently been devised, especially for DNNs, such as [Jia+20].

(a) (b)

Figure 5.15: (a) Illustration of the Neyman-Pearson hypothesis testing paradigm. Generated by code at figures.probml.ai/book1/5.15. (b) Two hypothetical two-sided power curves. B dominates A. Adapted from Figure 6.3.5 of [LM86]. Generated by code at figures.probml.ai/book1/5.15.

5.7 Frequentist hypothesis testing *

Suppose we have two hypotheses, known as the **null hypothesis** H_0 and an **alternative hypothesis** H_1, and we want to choose the one we think is correct on the basis of a dataset $\mathcal{D}$. We could use a Bayesian approach and compute the Bayes factor $p(H_0|\mathcal{D})/p(H_1|\mathcal{D})$, as we discussed in Sec. 5.4. However, this requires integrating over all possible parameterizations of the models H_0 and H_1, which can be computationally difficult, and which can be sensitive to the choice of prior. In this section, we consider a frequentist approach to the problem.

5.7.1 Likelihood ratio test

If we use 0-1 loss, and assume $p(H_0) = p(H_1)$, then the optimal decision rule is to accept H_0 iff $\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} > 1$. This is called the **likelihood ratio test**. We give some examples of this below.

5.7.1.1 Example: comparing Gaussian means

Suppose we are interested in testing whether some data comes from a Gaussian with mean μ_0 or from a Gaussian with mean μ_1. (We assume a known shared variance σ^2.) This is illustrated in Fig. 5.15a, where we plot $p(x|H_0)$ and $p(x|H_1)$. We can derive the likelihood ratio as follows:

$$\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} = \frac{\exp\left(-\frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n-\mu_0)^2\right)}{\exp\left(-\frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n-\mu_1)^2\right)} \tag{5.139}$$

$$= \exp\left(\frac{1}{2\sigma^2}(2N\overline{x}(\mu_0-\mu_1)+N\mu_1^2-N\mu_0^2)\right) \tag{5.140}$$

We see that this ratio only depends on the observed data via its mean, $\overline{x}$. This is an example of a **test statistic** $\tau(\mathcal{D})$, which is a scalar sufficient statistic for hypothesis testing. From Fig. 5.15a, we can see that $\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} > 1$ iff $\overline{x} < x^*$, where x^* is the point where the two pdf's intersect (we are assuming this point is unique).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.7.1.2 Simple vs compound hypotheses

In Sec. 5.7.1.1, the parameters for the null and alternative hypotheses were either fully specified (μ_0 and μ_1) or shared (σ^2). This is called a **simple hypothesis** test. In general, a hypothesis might not fully specify all the parameters; this is called a **compound hypothesis**. In this case, we should integrate out these unknown parameters, as in the Bayesian approach, since a hypothesis with more parameters will always have higher likelihood. As an approximation, we can "maximize them out", which gives us the maximum likelihood ratio test:

$$\frac{p(H_0|\mathcal{D})}{p(H_1|\mathcal{D})} = \frac{\int_{\boldsymbol{\theta}\in H_0} p(\boldsymbol{\theta}) p_{\boldsymbol{\theta}}(\mathcal{D})}{\int_{\boldsymbol{\theta}\in H_1} p(\boldsymbol{\theta}) p_{\boldsymbol{\theta}}(\mathcal{D})} \approx \frac{\max_{\boldsymbol{\theta}\in H_0} p_{\boldsymbol{\theta}}(\mathcal{D})}{\max_{\boldsymbol{\theta}\in H_1} p_{\boldsymbol{\theta}}(\mathcal{D})} \tag{5.141}$$

5.7.2 Null hypothesis significance testing (NHST)

Rather than assuming 0-1 loss, it is conventional to design the decision rule so that it has a **type I error rate** (the probability of accidently rejecting the null hypothesis H_0) of α. (See Sec. 5.1.3 for details on error rates of binary decision rules.) The error rate α is called the **significance** of the test. Hence the overall approach is called **null hypothesis significance testing** or **NHST**.

In our Gaussian mean example, we see from Fig. 5.15a that the type I error rate is the vertical shaded blue area:

$$\alpha(\mu_0) = p(\text{reject } H_0 | H_0 \text{ is true}) \tag{5.142}$$

$$= p(\overline{X}(\tilde{\mathcal{D}}) > x^* | \tilde{\mathcal{D}} \sim H_0) \tag{5.143}$$

$$= p\left(\frac{\overline{X} - \mu_0}{\sigma/\sqrt{N}} > \frac{x^* - \mu_0}{\sigma/\sqrt{N}}\right) \tag{5.144}$$

Hence $x^* = z_\alpha \sigma/\sqrt{N} + \mu_0$, where z_α is the upper α quantile of the standard Normal.

The type II error rate is the probability we accidently accept the null when the alternative is true:

$$\beta(\mu_1) = p(\text{type II error}) = p(\text{accept } H_0 | H_1 \text{ is true}) = p(\tau(\tilde{\mathcal{D}}) < \tau^* | \tilde{\mathcal{D}} \sim H_1) \tag{5.145}$$

This is shown by the horizontal shaded red area in Fig. 5.15a. We define the **power** of a test as $1 - \beta(\mu_1)$; this is the probability that we reject H_0 given that H_1 is true. In other words, it is the ability to correctly recognize that the null hypothesis is wrong. Clearly the least power occurs if $\mu_1 = \mu_0$ (so the curves overlap); in this case, we have $1 - \beta(\mu_1) = \alpha(\mu_0)$. As μ_1 and μ_0 become further apart, the power approaches 1 (because the shaded red area gets smaller, $\beta \to 0$). If we have two tests, A and B, where power$(B) \geq$ power(A) for the same type I error rate, we say B **dominates** A. See Fig. 5.15b. A test with highest power under H_1 amongst all tests with significance level α is called a **most powerful test**. It turns out that the likelihood ratio test is a most powerful test, a result known as the **Neyman-Pearson lemma**.

5.7.3 p-values

When we reject H_0 we often say the result is **statistically significant** at level α. However, the result may be statistically significant but not practically significant, depending on how far from the decision boundary the test statistic is.

Rather than arbitrarily declaring a result as significant or not, it is preferable to quote the **p-value**. This is defined as the probability, under the null hypothesis, of observing a test statistic that is as large or larger than that actually observed:

$$\text{pval}(\tau(\mathcal{D})) \triangleq \Pr(\tau(\tilde{\mathcal{D}}) \geq \tau(\mathcal{D})|\tilde{\mathcal{D}} \sim H_0) \tag{5.146}$$

In other words, $\text{pval}(\tau_{\text{obs}}) \triangleq \Pr(\tau_{\text{null}} \geq \tau_{\text{obs}})$, where $\tau_{\text{obs}} = \tau(\mathcal{D})$ and $\tau_{\text{null}} = \tau(\tilde{\mathcal{D}})$, where $\tilde{\mathcal{D}} \sim H_0$ is hypothetical future data. To see the connection with hypothesis testing, suppose we pick a decision threshold t^* such that $\Pr(\tau(\tilde{\mathcal{D}}) \geq t^*|H_0) = \alpha$. If we set $t^* = \tau(\mathcal{D})$, then $\alpha = \text{pval}(\tau(\mathcal{D}))$.

Thus if we only accept hypotheses where the p-value is less than $\alpha = 0.05$, then 95% of the time we will correctly reject the null hypothesis. However, this does *not* mean that the alternative hypothesis H_1 is true with probability 0.95. Indeed, even most scientists misinterpret p-values.[4] The quantity that most people want to compute is the Bayesian posterior $p(H_1|\mathcal{D}) = 0.95$. For more on this important distinction, see Sec. 5.7.4.

5.7.4 p-values considered harmful

A p-value is often interpreted as the likelihood of the data under the null hypothesis, so small values are interpreted to mean that H_0 is unlikely, and therefore that H_1 is likely. The reasoning is roughly as follows:

> If H_0 is true, then this test statistic would probably not occur. This statistic did occur. Therefore H_0 is probably false.

However, this is invalid reasoning. To see why, consider the following example (from [Coh94]):

> If a person is an American, then he is probably not a member of Congress. This person is a member of Congress. Therefore he is probably not an American.

This is obviously fallacious reasoning. By contrast, the following logical argument is valid reasoning:

> If a person is a Martian, then he is not a member of Congress. This person is a member of Congress. Therefore he is not a Martian.

The difference between these two cases is that the Martian example is using **deduction**, that is, reasoning forward from logical definitions to their consequences. More precisely, this example uses a rule from logic called **modus tollens**, in which we start out with a definition of the form $P \Rightarrow Q$; when we observe $\neg Q$, we can conclude $\neg P$. By contrast, the American example concerns **induction**, that is, reasoning backwards from observed evidence to probable (but not necessarily true) causes using statistical regularities, not logical definitions.

To perform induction, we need to use probabilistic inference (as explained in detail in [Jay03]). In particular, to compute the probability of the null hypothesis, we should use Bayes rule, as follows:

$$p(H_0|\mathcal{D}) = \frac{p(\mathcal{D}|H_0)p(H_0)}{p(\mathcal{D}|H_0)p(H_0) + p(\mathcal{D}|H_1)p(H_1)} \tag{5.147}$$

4. See e.g., https://fivethirtyeight.com/features/not-even-scientists-can-easily-explain-p-values/.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

	Ineffective	Effective	
"Not significant"	171	4	175
"Significant"	9	16	25
	180	20	200

Table 5.8: Some statistics of a hypothetical clinical trial. Source: [SAM04, p74].

If the prior is uniform, so $p(H_0) = p(H_1) = 0.5$, this can be rewritten in terms of the **likelihood ratio** $LR = p(\mathcal{D}|H_0)/p(\mathcal{D}|H_1)$ as follows:

$$p(H_0|\mathcal{D}) = \frac{LR}{LR+1} \tag{5.148}$$

In the American Congress example, $\mathcal{D}$ is the observation that the person is a member of Congress. The null hypothesis H_0 is that the person is American, and the alternative hypothesis H_1 is that the person is not American. We assume that $p(\mathcal{D}|H_0)$ is low, since most Americans are not members of Congress. However, $p(\mathcal{D}|H_1)$ is also low — in fact, in this example, it is 0, since only Americans can be members of Congress. Hence $LR = \infty$, so $p(H_0|\mathcal{D}) = 1.0$, as intuition suggests. Note, however, that NHST ignores $p(\mathcal{D}|H_1)$ as well as the prior $p(H_0)$, so it gives the wrong results — not just in this problem, but in many problems. Indeed, [SBB01] show that even if the p-value is as low as 0.05, the posterior probability of H_0 can be as high as 30% or more.

In general there can be huge differences between p-values and $p(H_0|\mathcal{D})$. In particular, [SBB01] show that even if the p-value is as low as 0.05, the posterior probability of H_0 can be as high as 30% or more, even with a uniform prior.

Consider this concrete example from [SAM04, p74]. Suppose 200 clinical trials are carried out for some drug. Suppose we perform a statistical test of whether the drug has a significant effect or not. The test has a type I error rate of $\alpha = 0.05$ and a type II error rate of $\beta = 0.2$. The resulting data is shown in Table 5.8.

We can compute the probability that the drug is not effective, given that the result is supposedly "significant", as follows:

$$p(H_0|\text{'significant'}) = \frac{p(\text{'significant'}|H_0)p(H_0)}{p(\text{'significant'}|H_0)p(H_0) + p(\text{'significant'}|H_1)p(H_1)} \tag{5.149}$$

$$= \frac{p(\text{type I error})p(H_0)}{p(\text{type I error})p(H_0) + (1 - p(\text{type II error}))p(H_1)} \tag{5.150}$$

$$= \frac{\alpha p(H_0)}{\alpha p(H_0) + (1-\beta)p(H_1)} \tag{5.151}$$

If we have prior knowledge, based on past experience, that most (say 90%) drugs are ineffective, then we find $p(H_0|\text{'significant'}) = 0.36$, which is much more than the 5% probability people usually associate with a p-value of $\alpha = 0.05$.

Thus we should distrust claims of statistical significance if they violate our prior knowledge.

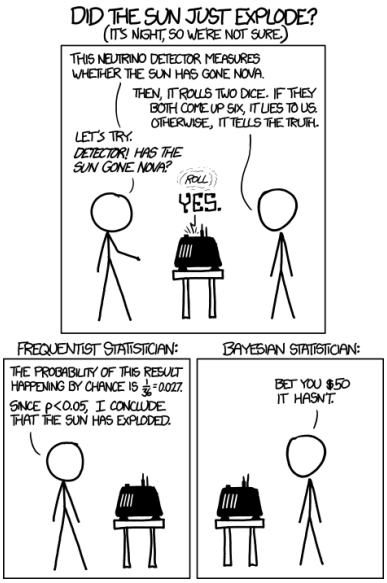


Figure 5.16: Cartoon illustrating the difference between frequentists and Bayesians. (The $p < 0.05$ comment is explained in Sec. 5.7.4. The betting comment is a reference to the Dutch book theorem, which essentially proves that the Bayesian approach to gambling (and other decision theory problems) is optimal, as explained in e.g., [Háj08].) From https://xkcd.com/1132/. Used with kind permission of Rundall Munroe (author of xkcd).

5.7.5 Why isn't everyone a Bayesian?

In Sec. 4.7.5 and Sec. 5.7.4, we have seen that inference based on frequentist principles can exhibit various forms of counter-intuitive behavior that can sometimes contradict common sense reason, as has been pointed out in multiple articles (see e.g., [Mat98; MS11; Kru13; Gel16; Hoe+14; Lyu+20; Cha+19b]).

The fundamental reason is that frequentist inference violates the **likelihood principle** [BW88], which says that inference should be based on the likelihood of the observed data, not on hypothetical future data that you have not observed. Bayes obviously satisfies the likelihood principle, and consequently does not suffer from these pathologies.

Given these fundamental flaws of frequentist statistics, and the fact that Bayesian methods do not have such flaws, an obvious question to ask is: "Why isn't everyone a Bayesian?" The (frequentist) statistician Bradley Efron wrote a paper with exactly this title [Efr86]. His short paper is well worth reading for anyone interested in this topic. Below we quote his opening section:

> The title is a reasonable question to ask on at least two counts. First of all, everyone used to be a Bayesian. Laplace wholeheartedly endorsed Bayes's formulation of the inference problem, and most 19th-century scientists followed suit. This included Gauss, whose statistical work is usually presented in frequentist terms.
>
> A second and more important point is the cogency of the Bayesian argument. Modern

> statisticians, following the lead of Savage and de Finetti, have advanced powerful theoretical arguments for preferring Bayesian inference. A byproduct of this work is a disturbing catalogue of inconsistencies in the frequentist point of view.
>
> Nevertheless, everyone is not a Bayesian. The current era (1986) is the first century in which statistics has been widely used for scientific reporting, and in fact, 20th-century statistics is mainly non-Bayesian. However, Lindley (1975) predicts a change for the 21st century.

Time will tell whether Lindley was right. However, the trends seem to be going in this direction. For example, in 2016 the American Statistical Association published an editorial warning about the use of p-values and NHST [WL16], and in 2018, the journal *Nature* published a similar article [AGM19].

Traditionally, computation has been a barrier to using Bayesian methods, but this is less of an issue these days, due to faster computers and better algorithms (which we will discuss in the sequel to this book, [Mur22]). Another, more fundamental, concern is that the Bayesian approach is only as correct as its modeling assumptions. However, this criticism also applies to frequentist methods, since the sampling distribution of an estimator must be derived using assumptions about the data generating mechanism. Fortunately, in both cases, we can check these assumptions empirically using cross validation (Sec. 4.5.5), calibration, and Bayesian model checking. Indeed, Donald Rubin wrote a paper [Rub84] called "Bayesianly Justifiable and Relevant Frequency Calculations for the Applied Statistician". In it, he writes

> The applied statistician should be Bayesian in principle and calibrated to the real world in practice. [They] should attempt to use specifications that lead to approximately calibrated procedures under reasonable deviations from [their assumptions]. [They] should avoid models that are contradicted by observed data in relevant ways — frequency calculations for hypothetical replications can model a model's adequacy and help to suggest more appropriate models.

See the sequel to this book, [Mur22], for details.

5.8 Exercises

Exercise 5.1 [Reject option in classifiers]

(Source: [DHS01, Q2.13].) In many classification problems one has the option either of assigning $\mathbf{x}$ to class j or, if you are too uncertain, of choosing the **reject option**. If the cost for rejects is less than the cost of falsely classifying the object, it may be the optimal action. Let α_i mean you choose action i, for $i = 1 : C+1$, where C is the number of classes and $C+1$ is the reject action. Let $Y = j$ be the true (but unknown) **state of nature**. Define the loss function as follows

$$\lambda(\alpha_i|Y=j) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, \ldots, C\} \\ \lambda_r & \text{if } i = C+1 \\ \lambda_s & \text{otherwise} \end{cases} \tag{5.152}$$

In otherwords, you incur 0 loss if you correctly classify, you incur λ_r loss (cost) if you choose the reject option, and you incur λ_s loss (cost) if you make a substitution error (misclassification).

a. Show that the minimum risk is obtained if we decide $Y = j$ if $p(Y = j|\mathbf{x}) \geq p(Y = k|\mathbf{x})$ for all k (i.e., j is the most probable class) *and* if $p(Y = j|\mathbf{x}) \geq 1 - \frac{\lambda_r}{\lambda_s}$; otherwise we decide to reject.

b. Describe qualitatively what happens as λ_r/λ_s is increased from 0 to 1 (i.e., the relative cost of rejection increases).

Exercise 5.2 [Newsvendor problem]

Consider the following classic problem in decision theory / economics. Suppose you are trying to decide how much quantity Q of some product (e.g., newspapers) to buy to maximize your profits. The optimal amount will depend on how much demand D you think there is for your product, as well as its cost to you C and its selling price P. Suppose D is unknown but has pdf $f(D)$ and cdf $F(D)$. We can evaluate the expected profit by considering two cases: if $D > Q$, then we sell all Q items, and make profit $\pi = (P - C)Q$; but if $D < Q$, we only sell D items, at profit $(P - C)D$, but have wasted $C(Q - D)$ on the unsold items. So the expected profit if we buy quantity Q is

$$E\pi(Q) = \int_Q^\infty (P - C)Qf(D)dD + \int_0^Q (P - C)Df(D)dD - \int_0^Q C(Q - D)f(D)dD \tag{5.153}$$

Simplify this expression, and then take derivatives wrt Q to show that the optimal quantity Q^* (which maximizes the expected profit) satisfies

$$F(Q^*) = \frac{P - C}{P} \tag{5.154}$$

Exercise 5.3 [Bayes factors and ROC curves]

Let $B = p(D|H_1)/p(D|H_0)$ be the Bayes factor in favor of model 1. Suppose we plot two ROC curves, one computed by thresholding B, and the other computed by thresholding $p(H_1|D)$. Will they be the same or different? Explain why.

Exercise 5.4 [Posterior median is optimal estimate under L1 loss]

Prove that the posterior median is the optimal estimate under L1 loss.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6 Information theory

In this chapter, we introduce a few basic concepts from the field of **information theory**. More details can be found in other books such as [Mac03; CT06], as well as the sequel to this book, [Mur22].

6.1 Entropy

The **entropy** of a probability distribution can be interpreted as a measure of uncertainty, or lack of predictability, associated with a random variable drawn from a given distribution, as we explain below.

We can also use entropy to define the **information content** of a data source. For example, suppose we observe a sequence of symbols $X_n \sim p$ generated from distribution p. If p has high entropy, it will be hard to predict the value of each osbervation X_n. Hence we say that the dataset $\mathcal{D} = (X_1, \ldots, X_n)$ has high information content. By contrast, if p is a degenerate distribution with 0 entropy (the minimal value), then every X_n will be the same, so $\mathcal{D}$ does not contain much information. (All of this can be formalized in terms of data compression, as we discuss in the sequel to this book.)

6.1.1 Entropy for discrete random variables

The entropy of a discrete random variable X with distribution p over K states is defined by

$$\mathbb{H}(X) \triangleq -\sum_{k=1}^{K} p(X=k) \log_2 p(X=k) = -\mathbb{E}_X[\log p(X)] \tag{6.1}$$

(Note that we use the notation $\mathbb{H}(X)$ to denote the entropy of the rv with distribution p, just as people write $\mathbb{V}[X]$ to mean the variance of the distribution associated with X; we could alternatively write $\mathbb{H}(p)$.) Usually we use log base 2, in which case the units are called **bits** (short for binary digits). For example, if $X \in \{1, \ldots, 5\}$ with histogram distribution $p = [0.25, 0.25, 0.2, 0.15, 0.15]$, we find $H = 2.29$ bits. If we use log base e, the units are called **nats**.

The discrete distribution with **maximum entropy** is the uniform distribution. Hence for a K-ary random variable, the entropy is maximized if $p(x=k) = 1/K$; in this case, $\mathbb{H}(X) = \log_2 K$. To see this, note that

$$\mathbb{H}(X) = -\sum_{k=1}^{K} \frac{1}{K} \log(1/K) = -\log(1/K) = \log(K) \tag{6.2}$$

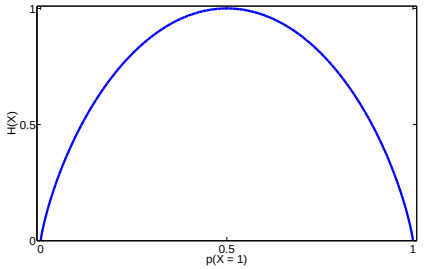
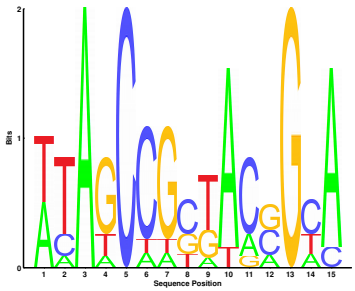


Figure 6.1: Entropy of a Bernoulli random variable as a function of θ. The maximum entropy is $\log_2 2 = 1$. Generated by code at figures.probml.ai/book1/6.1.

a t a g c c g g t a c g g c a
t t a g c t g c a a c c g c a
t c a g c c a c t a g a g c a
a t a a c c g c g a c c g c a
t t a g c c g c t a a g g t a
t a a g c c t c g t a c g t a
t t a g c c g t t a c g g c c
a t a t c c g g t a c a g t a
a t a g c a g g t a c c g a a
a c a t c c g t g a c g g a a

(a)



(b)

Figure 6.2: (a) Some aligned DNA sequences. Each row is a sequence, each column is a location within the sequence. (b) The corresponding **position weight matrix** *represented as a sequence logo. Each column represents a probablity distribution over the alphabet $\{A, C, G, T\}$ for the corresponding location in the sequence. The size of the letter is proportional to the probability. The height of column t is given by $2 - H_t$, where $0 \leq H_t \leq 2$ is the entropy (in bits) of the distribution $\mathbf{p}_t$. Thus deterministic distributions (with an entropy of 0, corresponding to highly conserved locations) have height 2, and uniform distributions (with an entropy of 2) have height 0. Generated by code at figures.probml.ai/book1/6.2.*

Conversely, the distribution with minimum entropy (which is zero) is any delta-function that puts all its mass on one state. Such a distribution has no uncertainty.

For the special case of binary random variables, $X \in \{0, 1\}$, we can write $p(X = 1) = \theta$ and $p(X = 0) = 1 - \theta$. Hence the entropy becomes

$$\mathbb{H}(X) = -[p(X = 1) \log_2 p(X = 1) + p(X = 0) \log_2 p(X = 0)] \tag{6.3}$$

$$= -[\theta \log_2 \theta + (1 - \theta) \log_2 (1 - \theta)] \tag{6.4}$$

This is called the **binary entropy function**, and is also written $\mathbb{H}(\theta)$. We plot this in Fig. 6.1. We see that the maximum value of 1 bit occurs when the distribution is uniform, $\theta = 0.5$. A fair coin requires a single yes/no question to determine its state.

As an interesting application of entropy, consider the problem of representing **DNA sequence motifs**, which is a distribution over short DNA strings. We can estimate this distribution by aligning

a set of DNA sequences (e.g., from different species), and then estimating the empirical distribution of each possible nucleotide from the 4 letter alphabet $X \sim \{A, C, G, T\}$ at each location t in the ith sequence as follows:

$$\mathbf{N}_t = \left(\sum_{i=1}^{N} \mathbb{I}\left(X_{it} = A\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = C\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = G\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = T\right) \right) \tag{6.5}$$

$$\hat{\boldsymbol{\theta}}_t = \mathbf{N}_t / N, \tag{6.6}$$

This $\mathbf{N}_t$ is a length four vector counting the number of times each letter appears at each location amongst the set of sequences. This $\hat{\boldsymbol{\theta}}_t$ distribution is known as a motif. We can also compute the most probable letter in each location; this is called the **consensus sequence**.

One way to visually summarize the data is by using a **sequence logo**, as shown in Fig. 6.2(b). We plot the letters A, C, G and T, with the most probable letter on the top; the height of the t'th bar is defined to be $2 - H_t$, where H_t is the entropy of $\hat{\boldsymbol{\theta}}_t$ (note that 2 is the maximum possible entropy for a distribution over 4 letters). Thus tall bars correspond to nearly deterministic distributions, which are the locations that are conserved by evolution (e.g., because they are part of a gene coding region). In this example, we see that column 13 is all G's, and hence has height 2.

Estimating the entropy of a random variable with many possible states requires estimating its distribution, which can require a lot of data. For example, imagine if X represents the identity of a word in an English document. Since there is a long tail of rare words, and since new words are invented all the time, it can be difficult to reliably estimate $p(X)$ and hence $\mathbb{H}(X)$. For one possible solution to this problem, see [VV13].

6.1.2 Cross entropy

The **cross entropy** between distribution p and q is defined by

$$\mathbb{H}(p, q) \triangleq -\sum_{k=1}^{K} p_k \log q_k \tag{6.7}$$

One can show that the cross entropy is the expected number of bits needed to compress some data samples drawn from distribution p using a code based on distribution q. This can be minimized by setting $q = p$, in which case the expected number of bits of the optimal code is $\mathbb{H}(p, p) = \mathbb{H}(p)$ — this is known as **Shannon's source coding theorem** (see e.g., [CT06]).

6.1.3 Joint entropy

The joint entropy of two random variables X and Y is defined as

$$\mathbb{H}(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y) \tag{6.8}$$

For example, consider choosing an integer from 1 to 8, $n \in \{1, \ldots, 8\}$. Let $X(n) = 1$ if n is even, and $Y(n) = 1$ if n is prime:

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

n	1	2	3	4	5	6	7	8
X	0	1	0	1	0	1	0	1
Y	0	1	1	0	1	0	1	0

The joint distribution is

$p(X,Y)$	$Y=0$	$Y=1$
$X=0$	$\frac{1}{8}$	$\frac{3}{8}$
$X=1$	$\frac{3}{8}$	$\frac{1}{8}$

so the joint entropy is given by

$$\mathbb{H}(X,Y) = -\left[\frac{1}{8}\log_2\frac{1}{8} + \frac{3}{8}\log_2\frac{3}{8} + \frac{3}{8}\log_2\frac{3}{8} + \frac{1}{8}\log_2\frac{1}{8}\right] = 1.81 \text{ bits} \tag{6.9}$$

Clearly the marginal probabilities are uniform: $p(X=1) = p(X=0) = p(Y=0) = p(Y=1) = 0.5$, so $\mathbb{H}(X) = \mathbb{H}(Y) = 1$. Hence $\mathbb{H}(X,Y) = 1.81$ bits $< \mathbb{H}(X) + \mathbb{H}(Y) = 2$ bits. In fact, this upper bound on the joint entropy holds in general. If X and Y are independent, then $\mathbb{H}(X,Y) = \mathbb{H}(X) + \mathbb{H}(Y)$, so the bound is tight. This makes intuitive sense: when the parts are correlated in some way, it reduces the "degrees of freedom" of the system, and hence reduces the overall entropy.

What is the lower bound on $\mathbb{H}(X,Y)$? If Y is a deterministic function of X, then $\mathbb{H}(X,Y) = \mathbb{H}(X)$. So

$$\mathbb{H}(X,Y) \geq \max\{\mathbb{H}(X), \mathbb{H}(Y)\} \geq 0 \tag{6.10}$$

Intuitively this says combining variables together does not make the entropy go down: you cannot reduce uncertainty merely by adding more unknowns to the problem, you need to observe some data, a topic we discuss in Sec. 6.1.4.

We can extend the definition of joint entropy from two variables to n in the obvious way.

6.1.4 Conditional entropy

The **conditional entropy** of Y given X is the uncertainty we have in Y after seeing X, averaged over possible values for X:

$$\mathbb{H}(Y|X) \triangleq \mathbb{E}_{p(X)}\left[\mathbb{H}(p(Y|X))\right] \tag{6.11}$$

$$= \sum_x p(x)\,\mathbb{H}(p(Y|X=x)) = -\sum_x p(x)\sum_y p(y|x)\log p(y|x) \tag{6.12}$$

$$= -\sum_{x,y} p(x,y)\log p(y|x) = -\sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)} \tag{6.13}$$

$$= -\sum_{x,y} p(x,y)\log p(x,y) - \sum_x p(x)\log\frac{1}{p(x)} \tag{6.14}$$

$$= \mathbb{H}(X,Y) - \mathbb{H}(X) \tag{6.15}$$

If Y is a deterministic function of X, then knowing X completely determines Y, so $\mathbb{H}(Y|X) = 0$. If X and Y are independent, knowing X tells us nothing about Y and $\mathbb{H}(Y|X) = \mathbb{H}(Y)$. Since $\mathbb{H}(X,Y) \leq \mathbb{H}(Y) + \mathbb{H}(X)$, we have

$$\mathbb{H}(Y|X) \leq \mathbb{H}(Y) \tag{6.16}$$

with equality iff X and Y are independent. This shows that, on average, conditioning on data never increases one's uncertainty. The caveat "on average" is necessary because for any *particular* observation (value of X), one may get more "confused" (i.e., $\mathbb{H}(Y|x) > \mathbb{H}(Y)$). However, in expectation, looking at the data is a good thing to do. (See also Sec. 6.3.8.)

We can rewrite Eq. (6.15) as follows:

$$\mathbb{H}(X_1, X_2) = \mathbb{H}(X_1) + \mathbb{H}(X_2|X_1) \tag{6.17}$$

This can be generalized to get the **chain rule for entropy**:

$$\mathbb{H}(X_1, X_2, \ldots, X_n) = \sum_{i=1}^{n} \mathbb{H}(X_i|X_1, \ldots, X_{i-1}) \tag{6.18}$$

6.1.5 Perplexity

The **perplexity** of a discrete probability distribution p is defined as

$$\text{perplexity}(p) \triangleq 2^{\mathbb{H}(p)} \tag{6.19}$$

This is often interpreted as a measure of predictability. For example, suppose p is a uniform distribution over K states. In this case, the perplexity is K. Obviously the lower bound on perplexity is $2^0 = 1$, which will be achieved if the distribution can perfectly predict outcomes.

Now suppose we have an empirical distribution based on data $\mathcal{D}$:

$$p_D(x|\mathcal{D}) = \frac{1}{N} \sum_{n=1}^{N} \delta_{x_n}(x) \tag{6.20}$$

We can measure how well p predicts $\mathcal{D}$ by computing

$$\text{perplexity}(p_D, p) \triangleq 2^{\mathbb{H}(p_D, p)} \tag{6.21}$$

Perplexity is often used to evaluate the quality of statistical language models, which is a generative model for sequences of tokens. Suppose the data is a single long document x of length N, and suppose p is a simple unigram model. In this case, the cross entropy term is given by

$$H = -\frac{1}{N} \sum_{n=1}^{N} \log p(x_n) \tag{6.22}$$

and hence the perplexity is given by

$$\text{perplexity}(p_D, p) = 2^H = \sqrt[N]{\prod_{n=1}^{N} \frac{1}{p(x_n)}} \tag{6.23}$$

This is sometimes called the **exponentiated cross entropy**. We see that this is the geometric mean of the inverse predictive probabilities.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

In the case of language models, we usually condition on previous words when predicting the next word. For example, in a bigram model, we use a second order Markov model of the form $p(x_i|x_{i-1})$. We define the **branching factor** of a language model as the number of possible words that can follow any given word. We can thus interpret the perplexity as the weighted average branching factor. For example, suppose the model predicts that each word is equally likely, regardless of context, so $p(x_i|x_{i-1}) = 1/K$. Then the perplexity is $((1/K)^N)^{-1/N} = K$. If some symbols are more likely than others, and the model correctly reflects this, its perplexity will be lower than K. However, as we show in Sec. 6.2, we have $\mathbb{H}(p^*) \leq \mathbb{H}(p^*, p)$, so we can never reduce the perplexity below the entropy of the underlying stochastic process p^*.

See [JM08, p96] for further discussion of perplexity and its uses in language models.

6.1.6 Differential entropy for continuous random variables *

If X is a continuous random variable with pdf $p(x)$, we define the **differential entropy** as

$$h(X) \triangleq -\int_{\mathcal{X}} dx\, p(x) \log p(x) \tag{6.24}$$

assuming this integral exists. For example, suppose $X \sim U(0, a)$. Then

$$h(X) = -\int_0^a dx\, \frac{1}{a} \log \frac{1}{a} = \log a \tag{6.25}$$

Note that, unlike the discrete case, *differential entropy can be negative*. This is because pdf's can be bigger than 1. For example if $X \sim U(0, 1/8)$, we have $h(X) = \log_2(1/8) = -3$.

One way to understand differential entropy is to realize that all real-valued quantities can only be represented to finite precision. It can be shown [CT91, p228] that the entropy of an n-bit quantization of a continuous random variable X is approximately $h(X) + n$. For example, suppose $X \sim U(0, \frac{1}{8})$. Then in a binary representation of X, the first 3 bits to the right of the binary point must be 0 (since the number is $\leq 1/8$). So to describe X to n bits of accuracy only requires $n - 3$ bits, which agrees with $h(X) = -3$ calculated above.

6.1.6.1 Example: Entropy of a Gaussian

The entropy of a d-dimensional Gaussian is

$$h(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})) = \frac{1}{2} \ln |2\pi e \boldsymbol{\Sigma}| = \frac{1}{2} \ln[(2\pi e)^d |\boldsymbol{\Sigma}|] = \frac{d}{2} + \frac{d}{2} \ln(2\pi) + \frac{1}{2} \ln |\boldsymbol{\Sigma}| \tag{6.26}$$

In the 1d case, this becomes

$$h(\mathcal{N}(\mu, \sigma^2)) = \frac{1}{2} \ln \left[2\pi e \sigma^2\right] \tag{6.27}$$

6.1.6.2 Connection with variance

The entropy of a Gaussian increases monotonically as the variance increases. However, this is not always the case. For example, consider a mixture of two 1d Gaussians centered at -1 and +1. As we

move the means further apart, say to -10 and +10, the variance increases (since the average distance from the overall mean gets larger). However, the entropy remains more or less the same, since we are still uncertain about where a sample might fall, even if we know that it will be near -10 or +10.((The exact entropy of a GMM is hard to compute, but a method to compute upper and lower bounds is presented in [Hub+08].)

6.1.6.3 Discretization

In general, computing the differential entropy for a continuous random variable can be difficult. A simple approximation is to **discretize** or **quantize** the variables. There are various methods for this (see e.g., [DKS95; KK06] for a summary), but a simple approach is to bin the distribution based on its empirical quantiles. The critical question is how many bins to use [LM04]. Scott [Sco79] suggested the following heuristic:

$$B = N^{1/3} \frac{\max(\mathcal{D}) - \min(\mathcal{D})}{3.5\sigma(\mathcal{D})} \tag{6.28}$$

where $\sigma(\mathcal{D})$ is the empirical standard deviation of the data, and $N = |\mathcal{D}|$ is the number of datapoints in the empirical distribution. However, the technique of discretization does not scale well if X is a multi-dimensional random vector, due to the curse of dimensionality.

6.2 Relative entropy (KL divergence) *

Given two distributions p and q, it is often useful to define a **distance metric** to measure how "close" or "similar" they are. In fact, we will be more general and consider a **divergence measure** $D(p, q)$ which quantifies how far q is from p, without requiring that D be a metric. More precisely, we say that D is a divergence if $D(p, q) \geq 0$ with equality iff $p = q$, whereas a metric also requires that D be symmetric and satisfy the **triangle inequality**, $D(p, r) \leq D(p, q) + D(q, r)$. There are many possible divergence measures we can use. In this section, we focus on the **Kullback-Leibler divergence** or **KL divergence**, also known as the **information gain** or **relative entropy**, between two distributions p and q.

6.2.1 Definition

For discrete distributions, the KL divergence is defined as follows:

$$\mathbb{KL}\left(p \| q\right) \triangleq \sum_{k=1}^{K} p_k \log \frac{p_k}{q_k} \tag{6.29}$$

This naturally extends to continuous distributions as well:

$$\mathbb{KL}\left(p \| q\right) \triangleq \int dx\, p(x) \log \frac{p(x)}{q(x)} \tag{6.30}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.2.2 Interpretation

We can rewrite the KL as follows:

$$\mathbb{KL}\left(p\|q\right) = \underbrace{\sum_{k=1}^{K} p_k \log p_k}_{-\mathbb{H}(p)} \underbrace{- \sum_{k=1}^{K} p_k \log q_k}_{\mathbb{H}(p,q)} \tag{6.31}$$

We recognize the first term as the negative entropy, and the second term as the cross entropy. Thus we can interpret the KL divergence as the "extra number of bits" you need to pay when compressing data samples from p using the incorrect distribution q as the basis of your coding scheme.

There are various other interpretations of KL divergence. See the sequel to this book, [Mur22], for more information.

6.2.3 Example: KL divergence between two Gaussians

For example, one can show that the KL divergence between two multivariate Gaussian distributions is given by

$$\begin{aligned}&\mathbb{KL}\left(\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)\|\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)\right) \\ &= \frac{1}{2}\left[\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - D + \log\left(\frac{\det(\boldsymbol{\Sigma}_2)}{\det(\boldsymbol{\Sigma}_1)}\right)\right]\end{aligned} \tag{6.32}$$

In the scalar case, this becomes

$$\mathbb{KL}\left(\mathcal{N}(x|\mu_1, \sigma_1)\|\mathcal{N}(x|\mu_2, \sigma_2)\right) = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \tag{6.33}$$

6.2.4 Non-negativity of KL

In this section, we prove that the KL divergence is always non-negative.

To do this, we use **Jensen's inequality**. This states that, for any convex function f, we have that

$$f(\sum_{i=1}^{n} \lambda_i \mathbf{x}_i) \leq \sum_{i=1}^{n} \lambda_i f(\mathbf{x}_i) \tag{6.34}$$

where $\lambda_i \geq 0$ and $\sum_{i=1}^{n} \lambda_i = 1$. In words, this result says that f of the average is less than the average of the f's. This is clearly true for $n = 2$, since a convex function curves up above a straight line connecting the two end points (see Sec. 8.1.3). To prove for general n, we can use induction.

For example, if $f(x) = \log(x)$, which is a concave function, we have

$$\log(\mathbb{E}_x g(x)) \geq \mathbb{E}_x \log(g(x)) \tag{6.35}$$

We use this result below.

Theorem 6.2.1. *(Information inequality)* $\mathbb{KL}\left(p\|q\right) \geq 0$ *with equality iff* $p = q$.

Proof. We now prove the theorem following [CT06, p28]. Let $A = \{x : p(x) > 0\}$ be the support of $p(x)$. Using the concavity of the log function and Jensen's inequality (Sec. 6.2.4), we have that

$$-\mathbb{KL}\left(p\|q\right) = -\sum_{x\in A} p(x)\log\frac{p(x)}{q(x)} = \sum_{x\in A} p(x)\log\frac{q(x)}{p(x)} \tag{6.36}$$

$$\leq \log\sum_{x\in A} p(x)\frac{q(x)}{p(x)} = \log\sum_{x\in A} q(x) \tag{6.37}$$

$$\leq \log\sum_{x\in\mathcal{X}} q(x) = \log 1 = 0 \tag{6.38}$$

Since $\log(x)$ is a strictly concave function ($-\log(x)$ is convex), we have equality in Eq. (6.37) iff $p(x) = cq(x)$ for some c that tracks the fraction of the whole space $\mathcal{X}$ contained in A. We have equality in Eq. (6.38) iff $\sum_{x\in A} q(x) = \sum_{x\in\mathcal{X}} q(x) = 1$, which implies $c = 1$. Hence $\mathbb{KL}\left(p\|q\right) = 0$ iff $p(x) = q(x)$ for all x. □

This theorem has many important implications, as we will see throughout the book. For example, we can show that the uniform distribution is the one that maximizes the entropy:

Corollary 6.2.1. *(Uniform distribution maximizes the entropy)* $\mathbb{H}\left(X\right) \leq \log|\mathcal{X}|$, *where* $|\mathcal{X}|$ *is the number of states for* X, *with equality iff* $p(x)$ *is uniform.*

Proof. Let $u(x) = 1/|\mathcal{X}|$. Then

$$0 \leq \mathbb{KL}\left(p\|u\right) = \sum_{x} p(x)\log\frac{p(x)}{u(x)} = \log|\mathcal{X}| - \mathbb{H}\left(X\right) \tag{6.39}$$

□

6.2.5 KL divergence and MLE

Suppose we want to find the distribution q that is as close as possible to p, as measured by KL divergence:

$$q^* = \arg\min_q \mathbb{KL}\left(p\|q\right) = \arg\min_q \int p(x)\log p(x)dx - \int p(x)\log q(x)dx \tag{6.40}$$

Now suppose p is the empirical distribution, which puts a probability atom on the observed training data and zero mass everywhere else:

$$p_D(x) = \frac{1}{N}\sum_{n=1}^{N}\delta(x - x_n) \tag{6.41}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Using the sifting property of delta functions we get

$$\mathbb{KL}\left(p_D \| q\right) = -\int p_D(x) \log q(x) dx + C \tag{6.42}$$

$$= -\int \left[\frac{1}{N}\sum_n \delta(x - x_n)\right] \log q(x) dx + C \tag{6.43}$$

$$= -\frac{1}{N}\sum_n \log q(x_n) + C \tag{6.44}$$

where $C = \int p(x) \log p(x)$ is a constant independent of q. This is called the **cross entropy** objective, and is equal to the average negative log likelihood of q on the training set. Thus we see that minimizing KL divergence to the empirical distribution is equivalent to maximizing likelihood.

This perspective points out the flaw with likelihood-based training, namely that it puts too much weight on the training set. In most applications, we do not really believe that the empirical distribution is a good representation of the true distribution, since it just puts "spikes" on a finite set of points, and zero density everywhere else. Even if the dataset is large (say 1M images), the universe from which the data is sampled is usually even larger (e.g., the set of "all natural images" is much larger than 1M). We could smooth the empirical distribution using kernel density estimation (Sec. 16.3), but that would require a similar kernel on the space of images. An alternative, algorithmic approach is to use **data augmentation**, which is a way of perturbing the observed data samples in way that we believe reflects plausible "natural variation". Applying MLE on this augmented dataset often yields superior results, especially when fitting models with many parameters [Xie+19a].

6.2.6 Forward vs reverse KL

Suppose we want to approximate a distribution p using a simpler distribution q. We can do this by minimizing $\mathbb{KL}\left(q \| p\right)$ or $\mathbb{KL}\left(p \| q\right)$. This gives rise to different behavior, as we discuss below.

First we consider the **forwards KL**, also called the **inclusive KL**, defined by

$$\mathbb{KL}\left(p \| q\right) = \int p(x) \log \frac{p(x)}{q(x)} dx \tag{6.45}$$

Minimizing this wrt q is known as an **M-projection** or **moment projection**.

We can gain an understanding of the optimal q by considering inputs x for which $p(x) > 0$ but $q(x) = 0$. In this case, the term $\log p(x)/q(x)$ will be infinite. Thus minimizing the KL will force q to include all the areas of space for which p has non-zero probability. Put another way, q will be **zero-avoiding** or **mode-covering**, and will typically over-estimate the support of p. Fig. 6.3(a) illustrates mode covering where p is a bimodal distribution but q is unimodal.

Now consider the **reverse KL**, also called the **exclusive KL**:

$$\mathbb{KL}\left(q \| p\right) = \int q(x) \log \frac{q(x)}{p(x)} dx \tag{6.46}$$

Minimizing this wrt q is known as an **I-projection** or **information projection**.

We can gain an understanding of the optimal q by consider inputs x for which $p(x) = 0$ but $q(x) > 0$. In this case, the term $\log q(x)/p(x)$ will be infinite. Thus minimizing the exclusive KL will

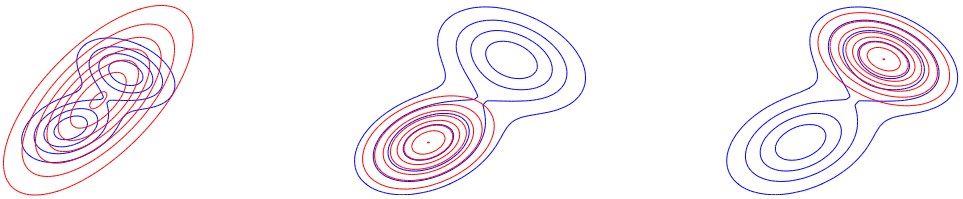


Figure 6.3: Illustrating forwards vs reverse KL on a bimodal distribution. The blue curves are the contours of the true distribution p. The red curves are the contours of the unimodal approximation q. (a) Minimizing forwards KL, $\mathbb{KL}(p\|q)$, wrt q causes q to "cover" p. (b-c) Minimizing reverse KL, $\mathbb{KL}(q\|p)$ wrt q causes q to "lock onto" one of the two modes of p. Adapted from Figure 10.3 of [Bis06]. Generated by code at figures.probml.ai/book1/6.3.

force q to exclude all the areas of space for which p has zero probability. One way to do this is for q to put probability mass in very few parts of space; this is called **zero-forcing** or **mode-seeking** behavior. In this case, q will typically under-estimate the support of p. We illustrate mode seeking when p is bimodal but q is unimodal in Fig. 6.3(b-c).

6.3 Mutual information *

The KL divergence gave us a way to measure how similar two distributions were. How should we measure how dependant two random variables are? One thing we could do is turn the question of measuring the dependence of two random variables into a question about the similarity of their distributions. This gives rise to the notion of **mutual information** (MI) between two random variables, which we define below.

6.3.1 Definition

The mutual information between rv's X and Y is defined as follows:

$$\mathbb{I}(X;Y) \triangleq \mathbb{KL}(p(x,y)\|p(x)p(y)) = \sum_{y\in Y}\sum_{x\in X} p(x,y)\log\frac{p(x,y)}{p(x)\,p(y)} \tag{6.47}$$

(We write $\mathbb{I}(X;Y)$ instead of $\mathbb{I}(X,Y)$, in case X and/or Y represent sets of variables; for example, we can write $\mathbb{I}(X;Y,Z)$ to represent the MI between X and (Y,Z).) For continuous random variables, we just replace sums with integrals.

It is easy to see that MI is always non-negative, even for continuous random variables, since

$$\mathbb{I}(X;Y) = \mathbb{KL}(p(x,y)\|p(x)p(y)) \geq 0 \tag{6.48}$$

We achieve the bound of 0 iff $p(x,y) = p(x)p(y)$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.3.2 Interpretation

Knowing that the mutual information is a KL divergence between the joint and factored marginal distributions tells is that the MI measures the information gain if we update from a model that treats the two variables as independent $p(x)p(y)$ to one that models their true joint density $p(x, y)$.

To gain further insight into the meaning of MI, it helps to re-express it in terms of joint and conditional entropies, as follows:

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X) \tag{6.49}$$

Thus we can interpret the MI between X and Y as the reduction in uncertainty about X after observing Y, or, by symmetry, the reduction in uncertainty about Y after observing X. Incidentally, this result gives an alternative proof that conditioning, on average, reduces entropy. In particular, we have $0 \leq \mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y)$, and hence $\mathbb{H}(X|Y) \leq \mathbb{H}(X)$.

We can also obtain a different interpretation. One can show that

$$\mathbb{I}(X;Y) = \mathbb{H}(X,Y) - \mathbb{H}(X|Y) - \mathbb{H}(Y|X) \tag{6.50}$$

Finally, one can show that

$$\mathbb{I}(X;Y) = \mathbb{H}(X) + \mathbb{H}(Y) - \mathbb{H}(X,Y) \tag{6.51}$$

See Fig. 6.4 for a summary of these equations in terms of an **information diagram**. (Formally, this is a signed measure mapping set expressions to their information-theoretic counterparts [Yeu91].)

6.3.3 Example

As an example, let us reconsider the example concerning prime and even numbers from Sec. 6.1.3. Recall that $\mathbb{H}(X) = \mathbb{H}(Y) = 1$. The conditional distribution $p(Y|X)$ is given by normalizing each row:

	Y=0	Y=1
X=0	$\frac{1}{4}$	$\frac{3}{4}$
X=1	$\frac{3}{4}$	$\frac{1}{4}$

Hence the conditional entropy is

$$\mathbb{H}(Y|X) = -\left[\frac{1}{8}\log_2\frac{1}{4} + \frac{3}{8}\log_2\frac{3}{4} + \frac{3}{8}\log_2\frac{3}{4} + \frac{1}{8}\log_2\frac{1}{4}\right] = 0.81 \text{ bits} \tag{6.52}$$

and the mutual information is

$$\mathbb{I}(X;Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X) = (1 - 0.81) \text{ bits} = 0.19 \text{ bits} \tag{6.53}$$

You can easily verify that

$$\mathbb{H}(X,Y) = \mathbb{H}(X|Y) + \mathbb{I}(X;Y) + \mathbb{H}(Y|X) \tag{6.54}$$

$$= (0.81 + 0.19 + 0.81) \text{ bits} = 1.81 \text{ bits} \tag{6.55}$$

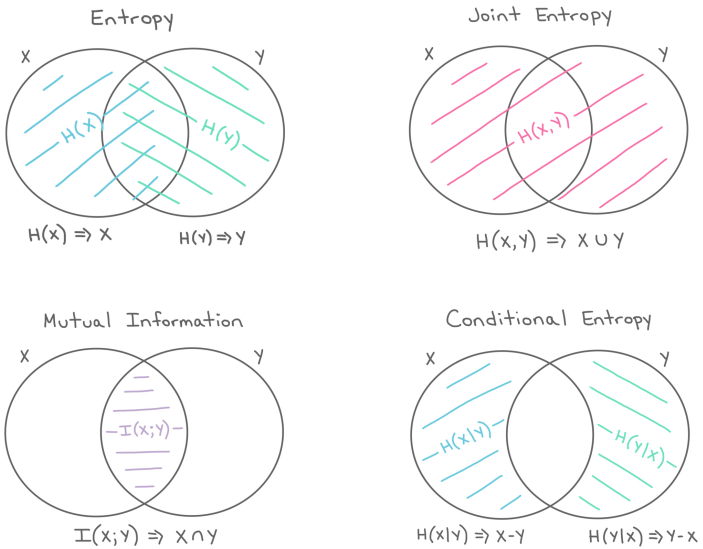


Figure 6.4: The marginal entropy, joint entropy, conditional entropy and mutual information represented as information diagrams. Used with kind permission of Katie Everett.

6.3.4 Conditional mutual information

We can define the **conditional mutual information** in the obvious way

$$\begin{aligned}
\mathbb{I}(X;Y|Z) &\triangleq \mathbb{E}_{p(Z)}\left[\mathbb{I}(X;Y)|Z\right] && (6.56)\\
&= \mathbb{E}_{p(x,y,z)}\left[\log \frac{p(x,y|z)}{p(x|z)p(y|z)}\right] && (6.57)\\
&= \mathbb{H}(X|Z) + \mathbb{H}(Y|Z) - \mathbb{H}(X,Y|Z) && (6.58)\\
&= \mathbb{H}(X|Z) - \mathbb{H}(X|Y,Z) = \mathbb{H}(Y|Z) - \mathbb{H}(Y|X,Z) && (6.59)\\
&= \mathbb{H}(X,Z) + \mathbb{H}(Y,Z) - \mathbb{H}(Z) - \mathbb{H}(X,Y,Z) && (6.60)\\
&= \mathbb{I}(Y;X,Z) - \mathbb{I}(Y;Z) && (6.61)
\end{aligned}$$

The last equation tells us that the conditional MI is the extra (residual) information that X tells us about Y, excluding what we already knew about Y given Z alone.

We can rewrite Eq. (6.61) as follows:

$$\mathbb{I}(Z,Y;X) = \mathbb{I}(Z;X) + \mathbb{I}(Y;X|Z) \tag{6.62}$$

Generalizing to N variables, we get the **chain rule for mutual information**:

$$\mathbb{I}(Z_1,\ldots,Z_N;X) = \sum_{n=1}^{N} \mathbb{I}(Z_n;X|Z_1,\ldots,Z_{n-1}) \tag{6.63}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.3.5 MI as a "generalized correlation coefficient"

Suppose that (x, y) are jointly Gaussian:

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \sigma^2 \end{pmatrix}\right) \tag{6.64}$$

We now show how to compute the mutual information between X and Y.

Using Eq. (6.26), we find that the entropy is

$$h(X, Y) = \frac{1}{2} \log\left[(2\pi e)^2 \det \Sigma\right] = \frac{1}{2} \log\left[(2\pi e)^2 \sigma^4 (1 - \rho^2)\right] \tag{6.65}$$

Since X and Y are individually normal with variance σ^2, we have

$$h(X) = h(Y) = \frac{1}{2} \log\left[2\pi e \sigma^2\right] \tag{6.66}$$

Hence

$$I(X, Y) = h(X) + h(Y) - h(X, Y) \tag{6.67}$$

$$= \log[2\pi e\sigma^2] - \frac{1}{2} \log[(2\pi e)^2 \sigma^4 (1 - \rho^2)] \tag{6.68}$$

$$= \frac{1}{2} \log[(2\pi e\sigma^2)^2] - \frac{1}{2} \log[(2\pi e\sigma^2)^2 (1 - \rho^2)] \tag{6.69}$$

$$= \frac{1}{2} \log \frac{1}{1 - \rho^2} = -\frac{1}{2} \log[1 - \rho^2] \tag{6.70}$$

We now discuss some interesting special cases.

1. $\rho = 1$. In this case, $X = Y$, and $I(X, Y) = \infty$, which makes sense. Observing Y tells us an infinite amount of information about X (as we know its real value exactly).
2. $\rho = 0$. In this case, X and Y are independent, and $I(X, Y) = 0$, which makes sense. Observing Y tells us nothing about X.
3. $\rho = -1$. In this case, $X = -Y$, and $I(X, Y) = \infty$, which again makes sense. Observing Y allows us to predict X to infinite precision.

Now consider the case where X and Y are scalar, but not jointly Gaussian. In general it can be difficult to compute the mutual information between continuous random variables, because we have to estimate the joint density $p(X, Y)$. For scalar variables, a simple approximation is to **discretize** or **quantize** them, by dividing the ranges of each variable into bins, and computing how many values fall in each histogram bin [Sco79]. We can then easily compute the MI using the empirical pmf.

Unfortunately, the number of bins used, and the location of the bin boundaries, can have a significant effect on the results. One way to avoid this is to use K-nearest neighbor distances to estimate densities in a non-parametric, adaptive way. This is the basis of the **KSG estimator** for MI proposed in [KSG04]. This is implemented in the sklearn.feature_selection.mutual_info_regression function. For papers related to this estimator, see [GOV18; HN19].

6.3.6 Normalized mutual information

For some applications, it is useful to have a normalized measure of dependence, between 0 and 1. We now discuss one way to construct such a measure.

First, note that

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) \leq \mathbb{H}(X) \tag{6.71}$$

$$= \mathbb{H}(Y) - \mathbb{H}(Y|X) \leq \mathbb{H}(Y) \tag{6.72}$$

so

$$0 \leq \mathbb{I}(X;Y) \leq \min(\mathbb{H}(X), \mathbb{H}(Y)) \tag{6.73}$$

Therefore we can define the **normalized mutual information** as follows:

$$NMI(X,Y) = \frac{\mathbb{I}(X;Y)}{\min(\mathbb{H}(X), \mathbb{H}(Y))} \leq 1 \tag{6.74}$$

This normalized mutual information ranges from 0 to 1. When $NMI(X,Y) = 0$, we have $\mathbb{I}(X;Y) = 0$, so X and Y are independent. When $NMI(X,Y) = 1$, and $\mathbb{H}(X) < \mathbb{H}(Y)$, we have

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(X) \implies \mathbb{H}(X|Y) = 0 \tag{6.75}$$

and so X is a deterministic function of Y. For example, suppose X is a discrete random variable with pmf $[0.5, 0.25, 0.25]$. We have $MI(X,X) = 1.5$ (using log base 2), and $H(X) = 1.5$, so the normalized MI is 1, as is to be expected.

For continuous random variables, it is harder to normalize the mutual information, because of the need to estimate the differential entropy, which is sensitive to the level of quantization. See Sec. 6.3.7 for further discussion.

6.3.7 Maximal information coefficient

As we discussed in Sec. 6.3.6, it it useful to have a normalized estimate of the mutual information, but this can be tricky to compute for real-valued data. One approach, known as the maximal information coefficient (MIC) [Res+11], is to define the following quantity:

$$\text{MIC}(X,Y) = \max_G \frac{\mathbb{I}((X,Y)|_G)}{\log ||G||} \tag{6.76}$$

where G is the set of 2d grids, and $(X,Y)|_G$ represents a discretization of the variables onto this grid, and $||G||$ is $\min(G_x, G_y)$, where G_x is the number of grid cells in the x direction, and G_y is the number of grid cells in the y direction. (The maximum grid resolution depends on the sample size n; they suggest restricting grids so that $G_x G_y \leq B(n)$, where $B(n) = n^\alpha$, where $\alpha = 0.6$.) The denominator is the entropy of a uniform joint distribution; dividing by this ensures $0 \leq \text{MIC} \leq 1$.

The intuition behind this statistic is the following: if there is a relationship between X and Y, then there should be some discrete gridding of the 2d input space that captures this. Since we don't know the correct grid to use, MIC searches over different grid resolutions (e.g., 2x2, 2x3, etc), as well as over locations of the grid boundaries. Given a grid, it is easy to quantize the data and compute

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

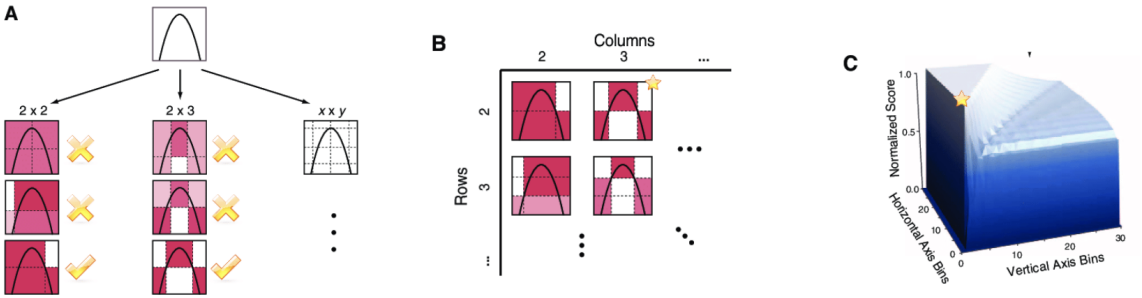


Figure 6.5: Illustration of how the maximal information coefficient (MIC) is computed. (a) We search over different grid resolutions, and grid cell locations, and compute the MI for each. (b) For each grid resolution (k, l), we define set $M(k, l)$ to be the maximum MI for any grid of that size, normalized by $\log(\min(k, l))$. *(c) We visualize the matrix* $\mathbf{M}$. *The maximum entry (denoted by a star) is defined to be the MIC. From Figure 1 of [Res+11]. Used with kind permission of David Reshef.*

MI. We define the **characteristic matrix** $M(k, l)$ to be the maximum MI achievable by any grid of size (k, l), normalized by $\log(\min(k, l))$. The MIC is then the maximum entry in this matrix, $\max_{kl \le B(n)} M(k, l)$. See Fig. 6.5 for a visualization of this process.

In [Res+11], they show that this quantity exhibits a property known as **equitability**, which means that it gives similar scores to equally noisy relationships, regardless of the type of relationship (e.g., linear, non-linear, non-functional).

In [Res+16], they present an improved estimator, called **MICe**, which is more efficient to compute, and only requires optimizing over 1d grids, which can be done in $O(n)$ time using dynamic programming. They also present another quantity, called **TICe** (total information content), that has higher power to detect relationships from small sample sizes, but lower equitability. This is defined to be $\sum_{kl \le B(n)} M(k, l)$. They recommend using TICe to screen a large number of candidate relationships, and then using MICe to quantify the strength of the relationship. For an efficient implementation of both of these metrics, see [Alb+18].

We can interpret MIC of 0 to mean there is no relationship between the variables, and 1 to represent a noise-free relationship of any form. This is illustrated in Fig. 6.6. Unlike correlation coefficients, MIC is not restricted to finding linear relationships. For this reason, the MIC has been called "a correlation for the 21st century" [Spe11].

In Fig. 6.7, we give a more interesting example, from [Res+11]. The data consists of 357 variables measuring a variety of social, economic, health and political indicators, collected by the World Health Organization (WHO). On the left of the figure, we see the correlation coefficient (CC) plotted against the MIC for all 63,546 variable pairs. On the right of the figure, we see scatter plots for particular pairs of variables, which we now discuss:

- The point marked C (near 0,0 on the plot) has a low CC and a low MIC. The corresponding scatter plot makes it clear that there is no relationship between these two variables (percentage of lives lost to injury and density of dentists in the population).
- The points marked D and H have high CC (in absolute value) and high MIC, because they

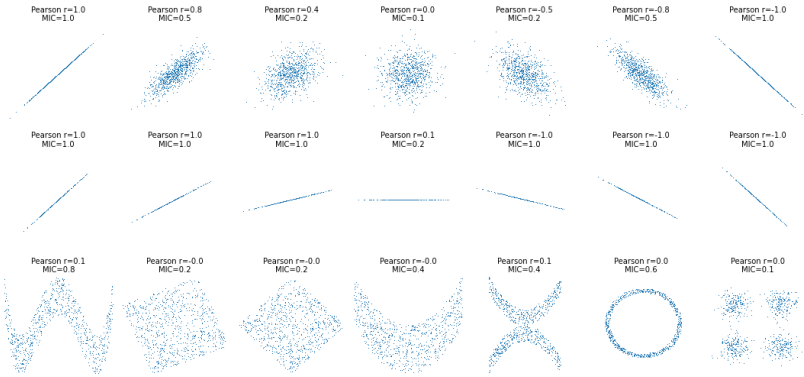


Figure 6.6: Plots of some 2d distributions and the corresponding estmate of correlation coefficient R^2 and the maximal information coefficient (MIC). Compare to Fig. 3.1. Generated by code at figures.probml.ai/book1/6.6.

represent nearly linear relationships.

- The points marked E, F, and G have low CC but high MIC. This is because they correspond to non-linear (and sometimes, as in the case of E and F, non-functional, i.e., one-to-many) relationships between the variables.

6.3.8 Data processing inequality

Suppose we have an unknown variable X, and we observe a noisy function of it, call it Y. If we process the noisy observations in some way to create a new variable Z, it should be intuitively obvious that we cannot increase the amount of information we have about the unknown quantity, X. This is known as the **data processing inequality**. We now state this more formally, and then prove it.

Theorem 6.3.1. *Suppose $X \to Y \to Z$ forms a Markov chain, so that $X \perp Z|Y$. Then $\mathbb{I}(X;Y) \geq \mathbb{I}(X;Z)$.*

Proof. By the chain rule for mutual information (Eq. (6.62)), we can expand the mutual information in two different ways:

$$\mathbb{I}(X;Y,Z) = \mathbb{I}(X;Z) + \mathbb{I}(X;Y|Z) \tag{6.77}$$
$$= \mathbb{I}(X;Y) + \mathbb{I}(X;Z|Y) \tag{6.78}$$

Since $X \perp Z|Y$, we have $\mathbb{I}(X;Z|Y) = 0$, so

$$\mathbb{I}(X;Z) + \mathbb{I}(X;Y|Z) = \mathbb{I}(X;Y) \tag{6.79}$$

Since $\mathbb{I}(X;Y|Z) \geq 0$, we have $\mathbb{I}(X;Y) \geq \mathbb{I}(X;Z)$. Similarly one can prove that $\mathbb{I}(Y;Z) \geq \mathbb{I}(X;Z)$. □

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

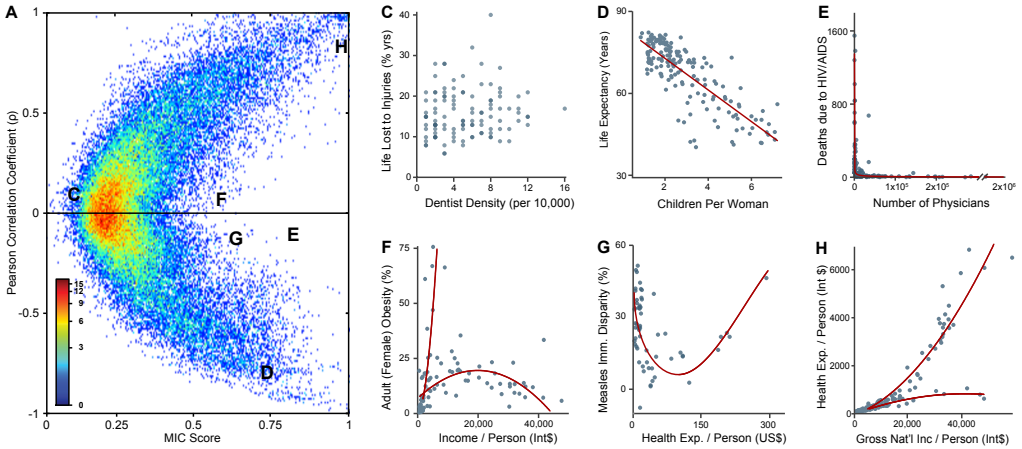


Figure 6.7: Left: Correlation coefficient vs maximal information criterion (MIC) for all pairwise relationships in the WHO data. Right: scatter plots of certain pairs of variables. The red lines are non-parametric smoothing regressions fit separately to each trend. From Figure 4 of [Res+11]. Used with kind permission of David Reshef.

6.3.9 Sufficient Statistics

An important consequence of the DPI is the following. Suppose we have the chain $\theta \to \mathcal{D} \to s(\mathcal{D})$. Then

$$\mathbb{I}(\theta; s(\mathcal{D})) \leq \mathbb{I}(\theta; \mathcal{D}) \tag{6.80}$$

If this holds with equality, then we say that $s(\mathcal{D})$ is a **sufficient statistic** of the data $\mathcal{D}$ for the purposes of inferring θ. In this case, we can equivalently write $\theta \to s(\mathcal{D}) \to \mathcal{D}$, since we can reconstruct the data from knowing $s(\mathcal{D})$ just as accurately as from knowing θ.

An example of a sufficient statistic is the data itself, $s(\mathcal{D}) = \mathcal{D}$, but this is not very useful, since it doesn't summarize the data at all. Hence we define a **minimal sufficient statistic** $s(\mathcal{D})$ as one which is sufficient, and which contains no extra information about θ; thus $s(\mathcal{D})$ maximally compresses the data $\mathcal{D}$ without losing information which is relevant to predicting θ. More formally, we say s is a *minimal* sufficient statistic for $\mathcal{D}$ if for all sufficient statistics $s'(\mathcal{D})$ there is some function f such that $s(\mathcal{D}) = f(s'(\mathcal{D}))$. We can summarize the situation as follows:

$$\theta \to s(\mathcal{D}) \to s'(\mathcal{D}) \to \mathcal{D} \tag{6.81}$$

Here $s'(\mathcal{D})$ takes $s(\mathcal{D})$ and adds redundant information to it, thus creating a one-to-many mapping.

For example, a minimal sufficient statistic for a set of N Bernoulli trials is simply N and $N_1 = \sum_n \mathbb{I}(X_n = 1)$, i.e., the number of successes. In other words, we don't need to keep track of the entire sequence of heads and tails and their ordering, we only need to keep track of the total number of heads and tails. Similarly, for inferring the mean of a Gaussian distribution with known variance we only need to know the empirical mean and number of samples.

6.3.10 Fano's inequality *

A common method for **feature selection** is to pick input features X_d which have high mutual information with the response variable Y. Below we justify why this is a reasonable thing to do. In particular, we state a result, known as **Fano's inequality**, which bounds the probability of misclassification (for any method) in terms of the mutual information between the features X and the class label Y.

Theorem 6.3.2. *(Fano's inequality) Consider an estimator $\hat{Y} = f(X)$ such that $Y \to X \to \hat{Y}$ forms a Markov chain. Let E be the event $\hat{Y} \neq Y$, indicating that an error occured, and let $P_e = P(Y \neq \hat{Y})$ be the probability of error. Then we have*

$$\mathbb{H}(Y|X) \leq \mathbb{H}\left(Y|\hat{Y}\right) \leq \mathbb{H}(E) + P_e \log |\mathcal{Y}| \tag{6.82}$$

Since $\mathbb{H}(E) \leq 1$, as we saw in Fig. 6.1, we can weaken this result to get

$$1 + P_e \log |\mathcal{Y}| \geq \mathbb{H}(Y|X) \tag{6.83}$$

and hence

$$P_e \geq \frac{\mathbb{H}(Y|X) - 1}{\log |\mathcal{Y}|} \tag{6.84}$$

Thus minimizing $\mathbb{H}(Y|X)$ (which can be done by maximizing $\mathbb{I}(X;Y)$) will also minimize the lower bound on P_e.

Proof. (From [CT06, p38].) Using the chain rule for entropy, we have

$$\mathbb{H}\left(E, Y|\hat{Y}\right) = \mathbb{H}\left(Y|\hat{Y}\right) + \underbrace{\mathbb{H}\left(E|Y, \hat{Y}\right)}_{=0} \tag{6.85}$$

$$= \mathbb{H}\left(E|\hat{Y}\right) + \mathbb{H}\left(Y|E, \hat{Y}\right) \tag{6.86}$$

Since conditioning reduces entropy (see Sec. 6.2.4), we have $\mathbb{H}\left(E|\hat{Y}\right) \leq \mathbb{H}(E)$. The final term can be bounded as follows:

$$\mathbb{H}\left(Y|E, \hat{Y}\right) = P(E=0)\,\mathbb{H}\left(Y|\hat{Y}, E=0\right) + P(E=1)\,\mathbb{H}\left(Y|\hat{Y}, E=1\right) \tag{6.87}$$

$$\leq (1 - P_e)0 + P_e \log |\mathcal{Y}| \tag{6.88}$$

Hence

$$\mathbb{H}\left(Y|\hat{Y}\right) \leq \underbrace{\mathbb{H}\left(E|\hat{Y}\right)}_{\leq \mathbb{H}(E)} + \underbrace{\mathbb{H}\left(Y|E, \hat{Y}\right)}_{P_e \log |\mathcal{Y}|} \tag{6.89}$$

Finally, by the data processing inequality, we have $\mathbb{I}(Y;\hat{Y}) \leq \mathbb{I}(Y;X)$, so $\mathbb{H}(Y|X) \leq \mathbb{H}\left(Y|\hat{Y}\right)$, which establishes Eq. (6.82). □

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.4 Exercises

Exercise 6.1 [Expressing mutual information in terms of entropies]
Prove the following identities:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) \tag{6.90}$$

and

$$H(X,Y) = H(X|Y) + H(Y|X) + I(X;Y) \tag{6.91}$$

Exercise 6.2 [Relationship between $D(p||q)$ and χ^2 statistic]
(Source: [CT91, Q12.2].)
Show that, if $p(x) \approx q(x)$, then

$$\mathbb{KL}\left(p||q\right) \approx \frac{1}{2}\chi^2 \tag{6.92}$$

where

$$\chi^2 = \sum_x \frac{(p(x) - q(x))^2}{q(x)} \tag{6.93}$$

Hint: write

$$p(x) = \Delta(x) + q(x) \tag{6.94}$$

$$\frac{p(x)}{q(x)} = 1 + \frac{\Delta(x)}{q(x)} \tag{6.95}$$

and use the Taylor series expansion for $\log(1+x)$.

$$\log(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} \cdots \tag{6.96}$$

for $-1 < x \leq 1$.

Exercise 6.3 [Fun with entropies]
(Source: Mackay.)
Consider the joint distribution $p(X,Y)$

		x			
		1	2	3	4
y	1	1/8	1/16	1/32	1/32
	2	1/16	1/8	1/32	1/32
	3	1/16	1/16	1/16	1/16
	4	1/4	0	0	0

a. What is the joint entropy $H(X,Y)$?
b. What are the marginal entropies $H(X)$ and $H(Y)$?
c. The entropy of X conditioned on a specific value of y is defined as

$$H(X|Y=y) = -\sum_x p(x|y) \log p(x|y) \tag{6.97}$$

Compute $H(X|y)$ for each value of y. Does the posterior entropy on X ever increase given an observation of Y?

d. The conditional entropy is defined as

$$H(X|Y) = \sum_y p(y) H(X|Y=y) \tag{6.98}$$

Compute this. Does the posterior entropy on X increase or decrease when averaged over the possible values of Y?

e. What is the mutual information between X and Y?

Exercise 6.4 [Forwards vs reverse KL divergence]

(Source: Exercise 33.7 of [Mac03].) Consider a factored approximation $q(x, y) = q(x)q(y)$ to a joint distribution $p(x, y)$. Show that to minimize the forwards KL $\mathbb{KL}(p\|q)$ we should set $q(x) = p(x)$ and $q(y) = p(y)$, i.e., the optimal approximation is a product of marginals

Now consider the following joint distribution, where the rows represent y and the columns x.

	1	2	3	4
1	1/8	1/8	0	0
2	1/8	1/8	0	0
3	0	0	1/4	0
4	0	0	0	1/4

Show that the reverse KL $\mathbb{KL}(q\|p)$ for this p has three distinct minima. Identify those minima and evaluate $\mathbb{KL}(q\|p)$ at each of them. What is the value of $\mathbb{KL}(q\|p)$ if we set $q(x, y) = p(x)p(y)$?

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7 Linear algebra

This chapter is co-authored with Zico Kolter.

7.1 Introduction

Linear algebra is the study of matrices and vectors. In this chapter, we summarize the key material that we will need throughout the book. Much more information can be found in other sources, such as [Str09; Kle13; Mol04; TB97; Axl15; Tho17; Agg20].

7.1.1 Notation

In this section, we define some notation.

7.1.1.1 Vectors

A **vector** $\mathbf{x} \in \mathbb{R}^n$ is a list of n numbers, usually written as a **column vector**

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \tag{7.1}$$

The vector of all ones is denoted $\mathbf{1}$. The vector of all zeros is denoted $\mathbf{0}$.
The **unit vector** $\mathbf{e}_i$ is a vector of all 0's, except entry i, which has value 1:

$$\mathbf{e}_i = (0, \ldots, 0, 1, 0, \ldots, 0) \tag{7.2}$$

This is also called a **one-hot vector**.

7.1.1.2 Matrices

A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ with m rows and n columns is a 2d array of numbers, arranged as follows:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}. \tag{7.3}$$

If $m = n$, the matrix is said to be **square**.

We use the notation A_{ij} or $A_{i,j}$ to denote the entry of $\mathbf{A}$ in the ith row and jth column. We use the notation $\mathbf{A}_{i,:}$ to denote the i'th row and $\mathbf{A}_{:,j}$ to denote the j'th column. We treat all vectors as column vectors by default (so $\mathbf{A}_{i,:}$ is viewed as a column vector with n entries). We use bold upper case letters to denote matrices, bold lower case letters to denote vectors, and non-bold letters to denote scalars.

We can view a matrix as a set of columns stacked along the horizontal axis:

$$\mathbf{A} = \begin{bmatrix} | & | & & | \\ \mathbf{A}_{:,1} & \mathbf{A}_{:,2} & \cdots & \mathbf{A}_{:,n} \\ | & | & & | \end{bmatrix}. \tag{7.4}$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{:,1}, \mathbf{A}_{:,2}, \ldots, \mathbf{A}_{:,n}] \tag{7.5}$$

We can also view a matrix as a set of rows stacked along the vertical axis:

$$\mathbf{A} = \begin{bmatrix} — & \mathbf{A}_{1,:}^\top & — \\ — & \mathbf{A}_{2,:}^\top & — \\ & \vdots & \\ — & \mathbf{A}_{m,:}^\top & — \end{bmatrix}. \tag{7.6}$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{1,:}; \mathbf{A}_{2,:}; \ldots; \mathbf{A}_{m,:}] \tag{7.7}$$

(Note the use of a semicolon.)

The **transpose** of a matrix results from "flipping" the rows and columns. Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, its transpose, written $\mathbf{A}^\top \in \mathbb{R}^{n \times m}$, is defined as

$$(\mathbf{A}^\top)_{ij} = A_{ji} \tag{7.8}$$

The following properties of transposes are easily verified:

$$(\mathbf{A}^\top)^\top = \mathbf{A} \tag{7.9}$$

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top \tag{7.10}$$

$$(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top \tag{7.11}$$

If a square matrix satisfies $\mathbf{A} = \mathbf{A}^\top$, it is called **symmetric**. We denote the set of all symmetric matrices of size n as $\mathbb{S}^n$.

7.1.1.3 Tensors

A **tensor** (in machine learning terminology) is just a generalization of a 2d array to more than 2 dimensions, as illustrated in Fig. 7.1. For example, the entries of a 3d tensor are denoted by A_{ijk}.

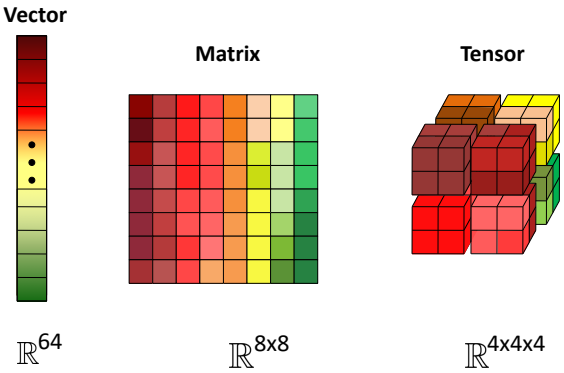


Figure 7.1: Illustration of a 1d vector, 2d matrix, and 3d tensor. The colors are used to represent individual entries of the vector; this list of numbers can also be stored in a 2d matrix, as shown. (In this example, the matrix is layed out in column-major order, which is the opposite of that used by Python.) We can also reshape the vector into a 3d tensor, as shown.

Figure 7.2: Illustration of (a) row-major vs (b) column-major order. From https://commons.wikimedia.org/wiki/File:Row_and_column_major_order.svg. Used with kind permission of Wikipedia author Cmglee.

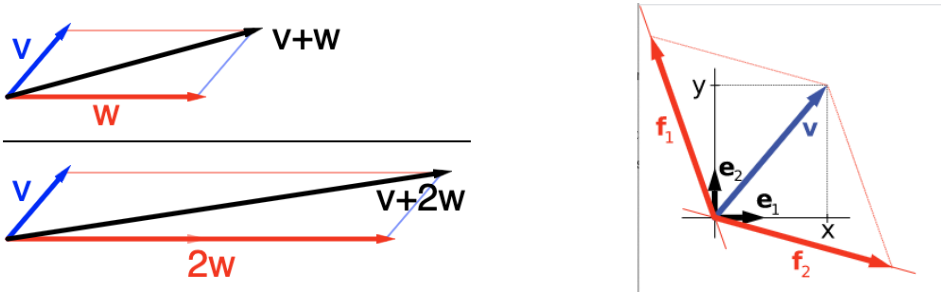
The number of dimensions is known as the **order** or **rank** of the tensor.[1] In mathematics, tensors can be viewed as a way to define multilinear maps, just as matrices can be used to define linear functions, although we will not need to use this interpretation.

We can **reshape** a matrix into a vector by stacking its columns on top of each other, as shown in Fig. 7.1. This is denoted by

$$\text{vec}(\mathbf{A}) = [\mathbf{A}_{:,1}; \cdots; \mathbf{A}_{:,n}] \in \mathbb{R}^{mn \times 1} \tag{7.12}$$

Conversely, we can reshape a vector into a matrix. There are two choices for how to do this, known as **row-major order** (used by languages such as Python and C++) and **column-major order** (used by languages such as Julia, Matlab, R and Fortran). See Fig. 7.2 for an illustration of the difference.

1. Note, however, that the rank of a 2d matrix is a different concept, as discussed in Sec. 7.1.4.3.



(a) (b)

Figure 7.3: (a) Top: A vector $\mathbf{v}$ *(blue) is added to another vector* $\mathbf{w}$ *(red). Bottom:* $\mathbf{w}$ *is stretched by a factor of 2, yielding the sum* $\mathbf{v} + 2\mathbf{w}$*. From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikipedia author IkamusumeFan (b) A vector* $\mathbf{v}$ *in* $\mathbb{R}^2$ *(blue) expressed in terms of different bases: using the standard basis of* $\mathbb{R}^2$*,* $\mathbf{v} = x\mathbf{e}_1 + y\mathbf{e}_2$ *(black), and using a different, non-orthogonal basis:* $\mathbf{v} = \mathbf{f}_1 + \mathbf{f}_2$ *(red). From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikiepdia author Jakob.scholbach*

7.1.2 Vector spaces

In this section, we discuss some fundamental concepts in linear algebra.

7.1.2.1 Vector addition and scaling

We can view a vector $\mathbf{x} \in \mathbb{R}^n$ as defining a point in n-dimensional Euclidean space. A **vector space** is a collection of such vectors, which can be added to together, and scaled by **scalars** (1-dimensional numbers), in order to create new points. These operations are defined to operate elementwise, in the obvious way, namely $\mathbf{x} + \mathbf{y} = (x_1 + y_1, \ldots, x_n + y_n)$ and $c\mathbf{x} = (cx_1, \ldots, cx_n)$, where $c \in \mathbb{R}$. See Fig. 7.3a for an illustration.

7.1.2.2 Linear independence, spans and basis sets

A set of vectors $\{\mathbf{x}_1, \mathbf{x}_2, \ldots \mathbf{x}_n\}$ is said to be **(linearly) independent** if no vector can be represented as a linear combination of the remaining vectors. Conversely, a vector which *can* be represented as a linear combination of the remaining vectors is said to be **(linearly) dependent**. For example, if

$$\mathbf{x}_n = \sum_{i=1}^{n-1} \alpha_i \mathbf{x}_i \tag{7.13}$$

for some $\{\alpha_1, \ldots, \alpha_{n-1}\}$ then $\mathbf{x}_n$ is dependent on $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n-1}\}$; otherwise, it is independent of $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n-1}\}$.

The **span** of a set of vectors $\{\mathbf{x}_1, \mathbf{x}_2, \ldots \mathbf{x}_n\}$ is the set of all vectors that can be expressed as a linear combination of $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$. That is,

$$\text{span}(\{\mathbf{x}_1, \ldots \mathbf{x}_n\}) \triangleq \left\{ \mathbf{v} : \mathbf{v} = \sum_{i=1}^{n} \alpha_i \mathbf{x}_i, \quad \alpha_i \in \mathbb{R} \right\}. \tag{7.14}$$

It can be shown that if $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ is a set of n linearly independent vectors, where each $\mathbf{x}_i \in \mathbb{R}^n$, then $\text{span}(\{\mathbf{x}_1, \ldots \mathbf{x}_n\}) = \mathbb{R}^n$. In other words, *any* vector $\mathbf{v} \in \mathbb{R}^n$ can be written as a linear combination of $\mathbf{x}_1$ through $\mathbf{x}_n$.

A **basis** $\mathcal{B}$ is a set of linearly independent vectors that spans the whole space, meaning that $\text{span}(\mathcal{B}) = \mathbb{R}^n$. There are often multiple bases to choose from, as illustrated in Fig. 7.3b. The **standard basis** uses the **coordinate vectors** $\mathbf{e}_1 = (1, 0, \ldots, 0)$, up to $\mathbf{e}_n = (0, 0, \ldots, 0, 1)$. This lets us translate back and forth between viewing a vector in $\mathbb{R}^2$ as an either an "arrow in the plane", rooted at the origin, or as an ordered list of numbers (corresponding to the coefficients for each basis vector).

7.1.2.3 Linear maps and matrices

A **linear map** or **linear transformation** is any function $f : \mathcal{V} \to \mathcal{W}$ such that $f(\mathbf{v} + \mathbf{w}) = f(\mathbf{v}) + f(\mathbf{w})$ and $f(a\ \mathbf{v}) = a\ f(\mathbf{v})$ for all $\mathbf{v}, \mathbf{w} \in \mathcal{V}$. Once the basis of $\mathcal{V}$ is chosen, a linear map $f : \mathcal{V} \to \mathcal{W}$ is completely determined by specifying the images of the basis vectors, because any element of $\mathcal{V}$ can be expressed is expessed uniquely as a linear combination of them.

Suppose $\mathcal{V} = \mathbb{R}^n$ and $\mathcal{W} = \mathbb{R}^m$. We can compute $f(\mathbf{v}_i) \in \mathbb{R}^m$ for each basis vector in $\mathcal{V}$, and store these along the rows of an $m \times n$ matrix $\mathbf{A}$. We can then compute $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^m$ for any $\mathbf{x} \in \mathbb{R}^n$ as follows:

$$\mathbf{y} = \left(\sum_{j=1}^{n} a_{1j} x_j, \ldots, \sum_{j=1}^{n} a_{mj} x_j \right) \tag{7.15}$$

This corresponds to multiplying the vector $\mathbf{x}$ by the matrix $\mathbf{A}$:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{7.16}$$

See Sec. 7.2 for more details.

If the function is invertible, we can write

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{y} \tag{7.17}$$

See Sec. 7.3 for details.

7.1.2.4 Range and nullspace of a matrix

Suppose we view a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as a set of m vectors in $\mathbb{R}^n$. The **range** (sometimes also called the **column space**) of this matrix is the span of the columns of $\mathbf{A}$. In other words,

$$\text{range}(\mathbf{A}) \triangleq \{\mathbf{v} \in \mathbb{R}^m : \mathbf{v} = \mathbf{A}\mathbf{x}, \mathbf{x} \in \mathbb{R}^n\}. \tag{7.18}$$

This can be thought of as the set of vectors that can be "reached" or "generated" by $\mathbf{A}$; it is a subspace of $\mathbb{R}^m$ whose dimensionality is given by the rank of $\mathbf{A}$ (see Sec. 7.1.4.3). The **nullspace** of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the set of all vectors that get mapped to the null vector when multiplied by $\mathbf{A}$, i.e.,

$$\text{nullspace}(\mathbf{A}) \triangleq \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\}. \tag{7.19}$$

The span of the rows of $\mathbf{A}$ is the complement to the nullspace of $\mathbf{A}$.

See Fig. 7.4 for an illustration of the range and nullspace of a matrix. We shall discuss how to compute the range and nullspace of a matrix numerically in Sec. 7.5.4 below.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

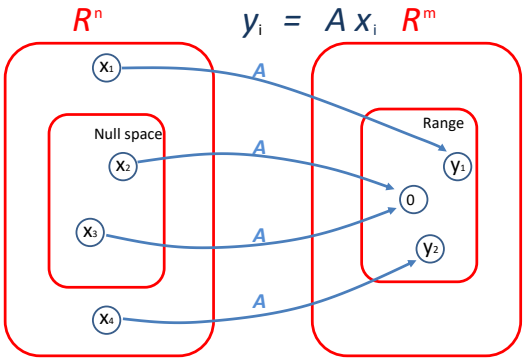


Figure 7.4: Visualization of the nullspace and range of an $m \times n$ matrix $\mathbf{A}$. Here $\mathbf{y}_1 = \mathbf{A}\mathbf{x}_1$, so $\mathbf{y}_1$ is in the range (is reachable); similarly for $\mathbf{y}_2$. Also $\mathbf{A}\mathbf{x}_3 = \mathbf{0}$, so $\mathbf{x}_3$ is in the nullspace (gets mapped to 0); similarly for $\mathbf{x}_4$.

7.1.2.5 Linear projection

The **projection** of a vector $\mathbf{y} \in \mathbb{R}^m$ onto the span of $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ (here we assume $\mathbf{x}_i \in \mathbb{R}^m$) is the vector $\mathbf{v} \in \text{span}(\{\mathbf{x}_1, \ldots \mathbf{x}_n\})$, such that $\mathbf{v}$ is as close as possible to $\mathbf{y}$, as measured by the Euclidean norm $\|\mathbf{v} - \mathbf{y}\|_2$. We denote the projection as $\text{Proj}(\mathbf{y}; \{\mathbf{x}_1, \ldots, \mathbf{x}_n\})$ and can define it formally as

$$\text{Proj}(\mathbf{y}; \{\mathbf{x}_1, \ldots \mathbf{x}_n\}) = \text{argmin}_{\mathbf{v} \in \text{span}(\{\mathbf{x}_1, \ldots, \mathbf{x}_n\})} \|\mathbf{y} - \mathbf{v}\|_2. \tag{7.20}$$

Given a (full rank) matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m > n$, we can define the projection of a vector $\mathbf{y} \in \mathbb{R}^m$ onto the range of $\mathbf{A}$ is as follows:

$$\text{Proj}(\mathbf{y}; \mathbf{A}) = \text{argmin}_{\mathbf{v} \in \mathcal{R}(\mathbf{A})} \|\mathbf{v} - \mathbf{y}\|_2 = \mathbf{A}(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{y} \ . \tag{7.21}$$

These are the same as the normal equations from Sec. 11.2.2.2.

7.1.3 Norms of a vector and matrix

In this section, we discuss ways of measuring the "size" of a vector and matrix.

7.1.3.1 Vector norms

A **norm** of a vector $\|\mathbf{x}\|$ is, informally, a measure of the "length" of the vector. More formally, a norm is any function $f : \mathbb{R}^n \to \mathbb{R}$ that satisfies 4 properties:

- For all $\mathbf{x} \in \mathbb{R}^n$, $f(\mathbf{x}) \geq 0$ (non-negativity).
- $f(\mathbf{x}) = 0$ if and only if $\mathbf{x} = \mathbf{0}$ (definiteness).
- For all $\mathbf{x} \in \mathbb{R}^n$, $t \in \mathbb{R}$, $f(t\mathbf{x}) = |t| f(x)$ (absolute value homogeneity).
- For all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (triangle inequality).

Consider the following common examples:

p-norm $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$, for $p \geq 1$.

2-norm $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$, also called Euclidean norm. Note that $\|\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{x}$.

1-norm $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$.

Max-norm $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

0-norm $\|\mathbf{x}\|_0 = \sum_{i=1}^n \mathbb{I}(|x_i| > 0)$. This is a **pseudo norm**, since it does not satisfy the triangle inequality or homogeneity. It counts the number of non-zero elements in $\mathbf{x}$. If we define $0^0 = 0$, we can write this as $\|\mathbf{x}\|_0 = \sum_{i=1}^n x_i^0$.

7.1.3.2 Matrix norms

Suppose we think of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as a defining a linear function $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$. We define the **induced norm** of $\mathbf{A}$ as the maximum amount by which f can lengthen any unit-norm input:

$$||\mathbf{A}||_p = \max_{\mathbf{x} \neq \mathbf{0}} \frac{||\mathbf{A}\mathbf{x}||_p}{||\mathbf{x}||_p} = \max_{||\mathbf{x}||=1} ||\mathbf{A}\mathbf{x}||_p \tag{7.22}$$

Typically $p = 2$, in which case

$$||\mathbf{A}||_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})} = \max_i \sigma_i \tag{7.23}$$

where σ_i is the i'th singular value.

The **nuclear norm**, also called the **trace norm**, is defined as

$$||\mathbf{A}||_* = \mathrm{tr}(\sqrt{\mathbf{A}^\top \mathbf{A}}) = \sum_i \sigma_i \tag{7.24}$$

where $\sqrt{\mathbf{A}^\top}$ is the matrix square root. Since the singular values are always non-negative, we have

$$||\mathbf{A}||_* = \sum_i |\sigma_i| = ||\boldsymbol{\sigma}||_1 \tag{7.25}$$

Using this as a regularizer encourages many singular values to become zero, resulting in a low rank matrix. More generally, we can define the **Schatten p-norm** as

$$||\mathbf{A}||_p = \left(\sum_i \sigma_i^p(\mathbf{A}) \right)^{1/p} \tag{7.26}$$

If we think of a matrix as a vector, we can define the matrix norm in terms of a vector norm, $||\mathbf{A}|| = ||\mathrm{vec}(\mathbf{A})||$. If the vector norm is the 2-norm, the corresponding matrix norm is the **Frobenius norm**:

$$||\mathbf{A}||_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\mathrm{tr}(\mathbf{A}^\top \mathbf{A})} = ||\mathrm{vec}(\mathbf{A})||_2 \tag{7.27}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

If $\mathbf{A}$ is expensive to evaluate, but $\mathbf{Av}$ is cheap (for a random vector $\mathbf{v}$), we can create a stochastic approximation to the Frobenius norm by using the Hutchinson trace estimator from Eq. (7.37) as follows:

$$||\mathbf{A}||_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A}) = \mathbb{E}\left[\mathbf{v}^\top \mathbf{A}\mathbf{A}^\top \mathbf{v}\right] = \mathbb{E}\left[||\mathbf{Av}||_2^2\right] \tag{7.28}$$

where $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

7.1.4 Properties of a matrix

In this section, we discuss various scalar properties of matrices.

7.1.4.1 Trace of a square matrix

The **trace** of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, denoted $\text{tr}(\mathbf{A})$, is the sum of diagonal elements in the matrix:

$$\text{tr}(\mathbf{A}) \triangleq \sum_{i=1}^{n} A_{ii}. \tag{7.29}$$

The trace has the following properties, where $c \in \mathbb{R}$ is a scalar, and $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ are square matrices:

$$\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}^\top) \tag{7.30}$$

$$\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B}) \tag{7.31}$$

$$\text{tr}(c\mathbf{A}) = c\,\text{tr}(\mathbf{A}) \tag{7.32}$$

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) \tag{7.33}$$

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \tag{7.34}$$

We also have the following important **cyclic permutation property**: For $\mathbf{A}, \mathbf{B}, \mathbf{C}$ such that $\mathbf{ABC}$ is square,

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB}) \tag{7.35}$$

From this, we can derive the **trace trick**, which rewrites the scalar inner product $\mathbf{x}^\top \mathbf{Ax}$ as follows

$$\mathbf{x}^\top \mathbf{Ax} = \text{tr}(\mathbf{x}^\top \mathbf{Ax}) = \text{tr}(\mathbf{x}\mathbf{x}^\top \mathbf{A}) \tag{7.36}$$

In some cases, it may be expensive to evaluate the matrix $\mathbf{A}$, but we may be able to cheaply evaluate matrix-vector products $\mathbf{Av}$. Suppose $\mathbf{v}$ is a random vector such that $\mathbb{E}\left[\mathbf{v}\mathbf{v}^\top\right] = \mathbf{I}$. In this case, we can create a Monte Carlo approximation to $\text{tr}(\mathbf{A})$ using the following identity:

$$\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}\mathbb{E}\left[\mathbf{v}\mathbf{v}^\top\right]) = \mathbb{E}\left[\text{tr}(\mathbf{A}\mathbf{v}\mathbf{v}^\top)\right] = \mathbb{E}\left[\text{tr}(\mathbf{v}^\top \mathbf{Av})\right] \tag{7.37}$$

This is called the **Hutchinson trace estimator** [Hut90].

7.1.4.2 Determinant of a square matrix

The **determinant** of a square matrix, denoted $\det(\mathbf{A})$ or $|\mathbf{A}|$, is a measure of how much it changes a unit volume when viewed as a linear transformation. (The formal definition is rather complex and is not needed here.)

The determinant operator satisfies these properties, where $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$

$$|\mathbf{A}| = |\mathbf{A}^\top| \tag{7.38}$$
$$|c\mathbf{A}| = c^n |\mathbf{A}| \tag{7.39}$$
$$|\mathbf{AB}| = |\mathbf{A}||\mathbf{B}| \tag{7.40}$$
$$|\mathbf{A}| = 0 \text{ iff } \mathbf{A} \text{ is singular} \tag{7.41}$$
$$|\mathbf{A}^{-1}| = 1/|\mathbf{A}| \text{ if } \mathbf{A} \text{ is not singular} \tag{7.42}$$
$$|\mathbf{A}| = \prod_{i=1}^{n} \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \tag{7.43}$$
$$\log |\mathbf{A}| = \mathrm{tr}(\log \mathbf{A}) \tag{7.44}$$

7.1.4.3 Rank of a matrix

The **column rank** of a matrix $\mathbf{A}$ is the dimension of the space spanned by its columns, and the **row rank** is the dimension of the space spanned by its rows. It is a basic fact of linear algebra (that can be shown using the SVD, discussed in Sec. 7.5) that for any matrix $\mathbf{A}$, $\mathrm{columnrank}(\mathbf{A}) = \mathrm{rowrank}(\mathbf{A})$, and so this quantity is simply referred to as the **rank** of $\mathbf{A}$, denoted as $\mathrm{rank}(\mathbf{A})$. The following are some basic properties of the rank:

- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathrm{rank}(\mathbf{A}) \leq \min(m, n)$. If $\mathrm{rank}(\mathbf{A}) = \min(m, n)$, then $\mathbf{A}$ is said to be **full rank**, otherwise it is called **rank deficient**.
- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathrm{rank}(\mathbf{A}) = \mathrm{rank}(\mathbf{A}^\top) = \mathrm{rank}(\mathbf{A}^\top \mathbf{A}) = \mathrm{rank}(\mathbf{A}\mathbf{A}^\top)$.
- For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$, $\mathrm{rank}(\mathbf{AB}) \leq \min(\mathrm{rank}(\mathbf{A}), \mathrm{rank}(\mathbf{B}))$.
- For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$, $\mathrm{rank}(\mathbf{A} + \mathbf{B}) \leq \mathrm{rank}(\mathbf{A}) + \mathrm{rank}(\mathbf{B})$.

One can show that a square matrix is invertible iff it is full rank.

7.1.4.4 Condition numbers

The **condition number** of a matrix $\mathbf{A}$ is a measure of how numerically stable any computations involving $\mathbf{A}$ will be. It is defined as follows:

$$\kappa(\mathbf{A}) \triangleq ||\mathbf{A}|| \cdot ||\mathbf{A}^{-1}|| \tag{7.45}$$

where $||\mathbf{A}||$ is the norm of the matrix. We can show that $\kappa(\mathbf{A}) \geq 1$. (The condition number depends on which norm we use; we will assume the ℓ_2-norm unless stated otherwise.)

We say $\mathbf{A}$ is **well-conditioned** if $\kappa(\mathbf{A})$ is small (close to 1), and **ill-conditioned** if $\kappa(\mathbf{A})$ is large. A large condition number means $\mathbf{A}$ is nearly singular. This is a better measure of nearness to singularity

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

than the size of the determinant. For example, suppose $\mathbf{A} = 0.1\mathbf{I}_{100\times 100}$. Then $\det(\mathbf{A}) = 10^{-100}$, which suggests $\mathbf{A}$ is nearly singular, but $\kappa(\mathbf{A}) = 1$, which means $\mathbf{A}$ is well-conditioned, reflecting the fact that $\mathbf{Ax}$ simply scales the entries of $\mathbf{x}$ by 0.1.

To get a better understanding of condition numbers, consider the linear system of equations $\mathbf{Ax} = \mathbf{b}$. If $\mathbf{A}$ is non-singular, the unique solution is $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. Suppose we change $\mathbf{b}$ to $\mathbf{b} + \Delta\mathbf{b}$; what effect will that have on $\mathbf{x}$? The new solution must satisify

$$\mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b} \tag{7.46}$$

where

$$\Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b} \tag{7.47}$$

We say that $\mathbf{A}$ is well-conditioned if a small $\Delta\mathbf{b}$ results in a small $\Delta\mathbf{x}$; otherwise we say that $\mathbf{A}$ is ill-conditioned.

For example, suppose

$$\mathbf{A} = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 + 10^{-10} & 1 - 10^{-10} \end{pmatrix}, \quad \mathbf{A}^{-1} = \begin{pmatrix} 1 - 10^{10} & 10^{10} \\ 1 + 10^{10} & -10^{10} \end{pmatrix} \tag{7.48}$$

The solution for $\mathbf{b} = (1, 1)$ is $\mathbf{x} = (1, 1)$. If we change $\mathbf{b}$ by $\Delta\mathbf{b}$, the solution changes to

$$\Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b} = \begin{pmatrix} \Delta b_1 - 10^{10}(\Delta b_1 - \Delta b_2) \\ \Delta b_1 + 10^{10}(\Delta b_1 - \Delta b_2) \end{pmatrix} \tag{7.49}$$

So a small change in $\mathbf{b}$ can lead to an extremely large change in $\mathbf{x}$, because $\mathbf{A}$ is ill-conditioned ($\kappa(\mathbf{A}) = 2 \times 10^{10}$).

In the case of the ℓ_2-norm, the condition number is equal to the ratio of the largest to smallest singular values (defined in Sec. 7.5); furthermore, the singular values are the square roots of the eigenvalues:

$$\kappa(\mathbf{A}) = \sigma_{max}/\sigma_{min} = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}} \tag{7.50}$$

We can gain further insight into condition numbers by considering a quadratic objective function $f(\mathbf{x}) = \mathbf{x}^\top\mathbf{Ax}$. If we plot the level set of this function, it will be elliptical, as shown in Sec. 7.4.4. As we increase the condition number of $\mathbf{A}$, the ellipses become more and more elongated along certain directions, corresponding to a very narrow valley in function space. If $\kappa = 1$ (the minimum possible value), the level set will be circular.

7.1.5 Special types of matrices

In this section, we will list some common kinds of matrices with various forms of structure.

7.1.5.1 Diagonal matrix

A **diagonal matrix** is a matrix where all non-diagonal elements are 0. This is typically denoted $\mathbf{D} = \text{diag}(d_1, d_2, \ldots, d_n)$, with

$$\mathbf{D} = \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix} \tag{7.51}$$

The **identity matrix**, denoted $\mathbf{I} \in \mathbb{R}^{n \times n}$, is a square matrix with ones on the diagonal and zeros everywhere else, $\mathbf{I} = \text{diag}(1, 1, \ldots, 1)$. It has the property that for all $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\mathbf{AI} = \mathbf{A} = \mathbf{IA} \tag{7.52}$$

where the size of $\mathbf{I}$ is determined by the dimensions of $\mathbf{A}$ so that matrix multiplication is possible.

We can extract the diagonal vector from a matrix using $\mathbf{d} = \text{diag}(\mathbf{D})$. We can convert a vector into a diagonal matrix by writing $\mathbf{D} = \text{diag}(\mathbf{d})$.

A **block diagonal** matrix is one which contains matrices on its main diagonal, and is 0 everywhere else, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix} \tag{7.53}$$

A **band-diagonal matrix** only has non-zero entries along the diagonal, and on k sides of the diagonal, where k is the bandwidth. For example, a **tridiagonal** 6×6 matrix looks like this:

$$\begin{bmatrix} A_{11} & A_{12} & 0 & \cdots & \cdots & 0 \\ A_{21} & A_{22} & A_{23} & \ddots & \ddots & \vdots \\ 0 & A_{32} & A_{33} & A_{34} & \ddots & \vdots \\ \vdots & \ddots & A_{43} & A_{44} & A_{45} & 0 \\ \vdots & \ddots & \ddots & A_{54} & A_{55} & A_{56} \\ 0 & \cdots & \cdots & 0 & A_{65} & A_{66} \end{bmatrix} \tag{7.54}$$

7.1.5.2 Triangular matrices

An **upper triangular matrix** only has non-zero entries on and above the diagonal. A **lower triangular matrix** only has non-zero entries on and below the diagonal.

Triangular matrices have the useful property that the diagonal entries of $\mathbf{A}$ are the eigenvalues of $\mathbf{A}$, and hence the determinant is the product of diagonal entries: $\det(\mathbf{A}) = \prod_i A_{ii}$.

7.1.5.3 Positive definite matrices

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, the scalar value $\mathbf{x}^\top \mathbf{A}\mathbf{x}$ is called a **quadratic form**. Written explicitly, we see that

$$\mathbf{x}^\top \mathbf{A}\mathbf{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ij} x_i x_j \ . \tag{7.55}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Note that,

$$\mathbf{x}^\top \mathbf{A}\mathbf{x} = (\mathbf{x}^\top \mathbf{A}\mathbf{x})^\top = \mathbf{x}^\top \mathbf{A}^\top \mathbf{x} = \mathbf{x}^\top (\frac{1}{2}\mathbf{A} + \frac{1}{2}\mathbf{A}^\top)\mathbf{x} \tag{7.56}$$

For this reason, we often implicitly assume that the matrices appearing in a quadratic form are symmetric.

We give the following definitions:

- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **positive definite** iff for all non-zero vectors $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{A}\mathbf{x} > 0$. This is usually denoted $\mathbf{A} \succ 0$ (or just $\mathbf{A} > 0$). If it is possible that $\mathbf{x}^\top \mathbf{A}\mathbf{x} = 0$, we say the matrix is **positive semidefinite** or **psd**. We denote the set of all positive definite matrices by $\mathbb{S}^n_{++}$.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **negative definite**, denoted $\mathbf{A} \prec 0$ (or just $\mathbf{A} < 0$) iff for all non-zero $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{A}\mathbf{x} < 0$. If it is possible that $\mathbf{x}^\top \mathbf{A}\mathbf{x} = 0$, we say the matrix is **negative semidefinite**.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **indefinite**, if it is neither positive semidefinite nor negative semidefinite — i.e., if there exists $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$ such that $\mathbf{x}_1^\top \mathbf{A}\mathbf{x}_1 > 0$ and $\mathbf{x}_2^\top \mathbf{A}\mathbf{x}_2 < 0$.

It should be obvious that if $\mathbf{A}$ is positive definite, then $-\mathbf{A}$ is negative definite and vice versa. Likewise, if $\mathbf{A}$ is positive semidefinite then $-\mathbf{A}$ is negative semidefinite and vice versa. If $\mathbf{A}$ is indefinite, then so is $-\mathbf{A}$. It can also be shown that positive definite and negative definite matrices are always invertible.

In Sec. 7.4.3.1, we show that a matrix is positive definite iff its eigenvalues are positive. Note that if all elements of $\mathbf{A}$ are positive, it does not mean $\mathbf{A}$ is necessarily positive definite. For example, $\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 3 & 2 \end{pmatrix}$ is not positive definite. Conversely, a positive definite matrix can have negative entries e.g., $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$

A sufficient condition for a (real, symmetric) matrix to be positive definite is that it is **diagonally dominant**, i.e., if in every row of the matrix, the magnitude of the diagonal entry in that row is larger than the sum of the magnitudes of all the other (non-diagonal) entries in that row. More precisely,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i \tag{7.57}$$

In 2d, any real, symmmetric 2×2 matrix $\begin{pmatrix} a & b \\ b & d \end{pmatrix}$ is positive definite iff $a > 0$, $d > 0$ and $ad > b^2$.

Finally, there is one type of positive definite matrix that comes up frequently, and so deserves some special mention. Given any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (not necessarily symmetric or even square), the **Gram matrix** $\mathbf{G} = \mathbf{A}^\top \mathbf{A}$ is always positive semidefinite. Further, if $m \geq n$ (and we assume for convenience that $\mathbf{A}$ is full rank), then $\mathbf{G} = \mathbf{A}^\top \mathbf{A}$ is positive definite.

7.1.5.4 Orthogonal matrices

Two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ are **orthogonal** if $\mathbf{x}^\top \mathbf{y} = 0$. A vector $\mathbf{x} \in \mathbb{R}^n$ is **normalized** if $\|\mathbf{x}\|_2 = 1$. A set of vectors that is pairwise orthogonal and normalized is called **orthonormal**. A square matrix

$\mathbf{U} \in \mathbb{R}^{n \times n}$ is **orthogonal** if all its columns are orthonormal. (Note the different meaning of the term orthogonal when talking about vectors versus matrices.) If the entries of $\mathbf{U}$ are complex valued, we use the term **unitary** instead of orthogonal.

It follows immediately from the definition of orthogonality and normality that $\mathbf{U}$ is orthogonal iff

$$\mathbf{U}^\top \mathbf{U} = \mathbf{I} = \mathbf{U}\mathbf{U}^\top. \tag{7.58}$$

In other words, the inverse of an orthogonal matrix is its transpose. Note that if $\mathbf{U}$ is not square — i.e., $\mathbf{U} \in \mathbb{R}^{m \times n}$, $n < m$ — but its columns are still orthonormal, then $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$, but $\mathbf{U}\mathbf{U}^\top \neq I$. We generally only use the term orthogonal to describe the previous case, where $\mathbf{U}$ is square.

An example of an orthogonal matrix is a **rotation matrix** (see Exercise 7.1). For example, a rotation in 3d by angle α about the z axis is given by

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.59}$$

If $\alpha = 45°$, this becomes

$$\mathbf{R}(45) = \begin{pmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.60}$$

where $\frac{1}{\sqrt{2}} = 0.7071$. We see that $\mathbf{R}(-\alpha) = \mathbf{R}(\alpha)^{-1} = \mathbf{R}(\alpha)^\top$, so this is an orthogonal matrix.

One nice property of orthogonal matrices is that operating on a vector with an orthogonal matrix will not change its Euclidean norm, i.e.,

$$\|\mathbf{U}\mathbf{x}\|_2 = \|\mathbf{x}\|_2 \tag{7.61}$$

for any nonzero $\mathbf{x} \in \mathbb{R}^n$, and orthogonal $\mathbf{U} \in \mathbb{R}^{n \times n}$.

Similarly, one can show that the angle between two vectors is preserved after they are transformed by an orthogonal matrix. The cosine of the angle between $\mathbf{x}$ and $\mathbf{y}$ is given by

$$\cos(\alpha(\mathbf{x}, \mathbf{y})) = \frac{\mathbf{x}^\top \mathbf{y}}{||\mathbf{x}||||\mathbf{y}||} \tag{7.62}$$

so

$$\cos(\alpha(\mathbf{U}\mathbf{x}, \mathbf{U}\mathbf{y})) = \frac{(\mathbf{U}\mathbf{x})^\top (\mathbf{U}\mathbf{y})}{||\mathbf{U}\mathbf{x}||||\mathbf{U}\mathbf{y}||} = \frac{\mathbf{x}^\top \mathbf{y}}{||\mathbf{x}||||\mathbf{y}||} = \cos(\alpha(\mathbf{x}, \mathbf{y})) \tag{7.63}$$

In summary, transformations by orthogonal matrices are generalizations of rotations (if $\det(\mathbf{U}) = 1$) and reflections (if $\det(\mathbf{U}) = -1$), since they preserve lengths and angles.

Note that there is technique called Gram Schmidt orthogonalization which is a way to make any square matrix orthogonal, but we will not cover it here.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.2 Matrix multiplication

The product of two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is the matrix

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}, \tag{7.64}$$

where

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}. \tag{7.65}$$

Note that in order for the matrix product to exist, the number of columns in $\mathbf{A}$ must equal the number of rows in $\mathbf{B}$.

Matrix multiplication generally takes $O(mnp)$ time, although faster methods exist. In addition, specialized hardware, such as GPUs and TPUs, can be leveraged to speed up matrix multiplication significantly, by performing operations across the rows (or columns) in parallel.

It is useful to know a few basic properties of matrix multiplication:

- Matrix multiplication is **associative**: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
- Matrix multiplication is **distributive**: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.
- Matrix multiplication is, in general, *not* **commutative**; that is, it can be the case that $\mathbf{AB} \neq \mathbf{BA}$.

(In each of the above cases, we are assuming that the dimensions match.)

There are many important special cases of matrix multiplication, as we discuss below.

7.2.1 Vector-Vector Products

Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the quantity $\mathbf{x}^\top \mathbf{y}$, called the **inner product**, **dot product** or **scalar product** of the vectors, is a real number given by

$$\langle \mathbf{x}, \mathbf{y} \rangle \triangleq \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{n} x_i y_i. \tag{7.66}$$

Note that it is always the case that $\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}$.

Given vectors $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$ (they no longer have to be the same size), $\mathbf{x}\mathbf{y}^\top$ is called the **outer product** of the vectors. It is a matrix whose entries are given by $(\mathbf{x}\mathbf{y}^\top)_{ij} = x_i y_j$, i.e.,

$$\mathbf{x}\mathbf{y}^\top \in \mathbb{R}^{m \times n} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_m y_1 & x_m y_2 & \cdots & x_m y_n \end{bmatrix}. \tag{7.67}$$

7.2.2 Matrix-Vector Products

Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, their product is a vector $\mathbf{y} = \mathbf{Ax} \in \mathbb{R}^m$. There are a couple ways of looking at matrix-vector multiplication, and we will look at them both.

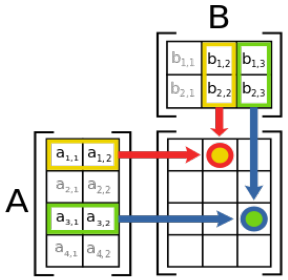


Figure 7.5: Illustration of matrix multiplication. From https://en.wikipedia.org/wiki/Matrix_multiplication. Used with kind permission of Wikipedia author Bilou.

If we write $\mathbf{A}$ by rows, then we can express $\mathbf{y} = \mathbf{A}\mathbf{x}$ as follows:

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} — & \mathbf{a}_1^\top & — \\ — & \mathbf{a}_2^\top & — \\ & \vdots & \\ — & \mathbf{a}_m^\top & — \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}. \tag{7.68}$$

In other words, the ith entry of $\mathbf{y}$ is equal to the inner product of the ith *row* of $\mathbf{A}$ and $\mathbf{x}$, $y_i = \mathbf{a}_i^\top \mathbf{x}$.

Alternatively, let's write $\mathbf{A}$ in column form. In this case we see that

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} | \\ \mathbf{a}_1 \\ | \end{bmatrix} x_1 + \begin{bmatrix} | \\ \mathbf{a}_2 \\ | \end{bmatrix} x_2 + \ldots + \begin{bmatrix} | \\ \mathbf{a}_n \\ | \end{bmatrix} x_n. \tag{7.69}$$

In other words, $\mathbf{y}$ is a **linear combination** of the *columns* of $\mathbf{A}$, where the coefficients of the linear combination are given by the entries of $\mathbf{x}$. We can view the columns of $\mathbf{A}$ as a set of **basis vectors** defining a **linear subspace**. We can contstruct vectors in this subspace by taking linear combinations of the basis vectors. See Sec. 7.1.2 for details.

7.2.3 Matrix-Matrix Products

Below we look at four different (but, of course, equivalent) ways of viewing the matrix-matrix multiplication $\mathbf{C} = \mathbf{A}\mathbf{B}$.

First we can view matrix-matrix multiplication as a set of vector-vector products. The most obvious viewpoint, which follows immediately from the definition, is that the i, j entry of $\mathbf{C}$ is equal to the inner product of the ith row of $\mathbf{A}$ and the jth column of $\mathbf{B}$. Symbolically, this looks like the following,

$$\mathbf{C} = \mathbf{A}\mathbf{B} = \begin{bmatrix} — & \mathbf{a}_1^\top & — \\ — & \mathbf{a}_2^\top & — \\ & \vdots & \\ — & \mathbf{a}_m^\top & — \end{bmatrix} \begin{bmatrix} | & | & & | \\ \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{b}_1 & \mathbf{a}_1^\top \mathbf{b}_2 & \cdots & \mathbf{a}_1^\top \mathbf{b}_p \\ \mathbf{a}_2^\top \mathbf{b}_1 & \mathbf{a}_2^\top \mathbf{b}_2 & \cdots & \mathbf{a}_2^\top \mathbf{b}_p \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_m^\top \mathbf{b}_1 & \mathbf{a}_m^\top \mathbf{b}_2 & \cdots & \mathbf{a}_m^\top \mathbf{b}_p \end{bmatrix}. \tag{7.70}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Remember that since $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, $\mathbf{a}_i \in \mathbb{R}^n$ and $\mathbf{b}_j \in \mathbb{R}^n$, so these inner products all make sense. This is the most "natural" representation when we represent $\mathbf{A}$ by rows and $\mathbf{B}$ by columns. See Fig. 7.5 for an illustration.

Alternatively, we can represent $\mathbf{A}$ by columns, and $\mathbf{B}$ by rows, which leads to the interpretation of $\mathbf{AB}$ as a sum of outer products. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} — & \mathbf{b}_1^\top & — \\ — & \mathbf{b}_2^\top & — \\ & \vdots & \\ — & \mathbf{b}_n^\top & — \end{bmatrix} = \sum_{i=1}^{n} \mathbf{a}_i \mathbf{b}_i^\top \ . \tag{7.71}$$

Put another way, $\mathbf{AB}$ is equal to the sum, over all i, of the outer product of the ith column of $\mathbf{A}$ and the ith row of $\mathbf{B}$. Since, in this case, $\mathbf{a}_i \in \mathbb{R}^m$ and $\mathbf{b}_i \in \mathbb{R}^p$, the dimension of the outer product $\mathbf{a}_i \mathbf{b}_i^\top$ is $m \times p$, which coincides with the dimension of $\mathbf{C}$.

We can also view matrix-matrix multiplication as a set of matrix-vector products. Specifically, if we represent $\mathbf{B}$ by columns, we can view the columns of $\mathbf{C}$ as matrix-vector products between $\mathbf{A}$ and the columns of $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \mathbf{A} \begin{bmatrix} | & | & & | \\ \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ \mathbf{A}\mathbf{b}_1 & \mathbf{A}\mathbf{b}_2 & \cdots & \mathbf{A}\mathbf{b}_p \\ | & | & & | \end{bmatrix} . \tag{7.72}$$

Here the ith column of $\mathbf{C}$ is given by the matrix-vector product with the vector on the right, $\mathbf{c}_i = \mathbf{A}\mathbf{b}_i$. These matrix-vector products can in turn be interpreted using both viewpoints given in the previous subsection.

Finally, we have the analogous viewpoint, where we represent $\mathbf{A}$ by rows, and view the rows of $\mathbf{C}$ as the matrix-vector product between the rows of $\mathbf{A}$ and the matrix $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} — & \mathbf{a}_1^\top & — \\ — & \mathbf{a}_2^\top & — \\ & \vdots & \\ — & \mathbf{a}_m^\top & — \end{bmatrix} \mathbf{B} = \begin{bmatrix} — & \mathbf{a}_1^\top \mathbf{B} & — \\ — & \mathbf{a}_2^\top \mathbf{B} & — \\ & \vdots & \\ — & \mathbf{a}_m^\top \mathbf{B} & — \end{bmatrix} . \tag{7.73}$$

Here the ith row of $\mathbf{C}$ is given by the matrix-vector product with the vector on the left, $\mathbf{c}_i^\top = \mathbf{a}_i^\top \mathbf{B}$.

It may seem like overkill to dissect matrix multiplication to such a large degree, especially when all these viewpoints follow immediately from the initial definition we gave (in about a line of math) at the beginning of this section. However, virtually all of linear algebra deals with matrix multiplications of some kind, and it is worthwhile to spend some time trying to develop an intuitive understanding of the viewpoints presented here.

Finally, a word on notation. We write $\mathbf{A}^2$ as shorthand for $\mathbf{AA}$, which is the matrix product. To denote elementwise squaring of the elements of a matrix, we write $\mathbf{A}^{\odot 2} = [A_{ij}^2]$. (If $\mathbf{A}$ is diagonal, then $\mathbf{A}^2 = \mathbf{A}^{\odot 2}$.)

We can also define the inverse of $\mathbf{A}^2$ using the **matrix square root**: we say $\mathbf{A} = \sqrt{\mathbf{M}}$ if $\mathbf{A}^2 = \mathbf{M}$. To denote elementwise square root of the elements of a matrix, we write $[\sqrt{M_{ij}}]$.

7.2.4 Application: manipulating data matrices

As an application of the above results, consider the case where $\mathbf{X}$ is the $N \times D$ design matrix, whose rows are the data cases. There are various common preprocessing operations that we apply to this matrix, which we summarize below. (Writing these operations in matrix form is useful because it is notationally compact, and it allows us to implement the methods quickly using fast matrix code.)

7.2.4.1 Summing slices of the matrix

Suppose $\mathbf{X}$ is an $N \times D$ matrix. We can sum across the rows by premultiplying by a $1 \times N$ matrix of ones to create a $1 \times D$ matrix:

$$\mathbf{1}_N^\top \mathbf{X} = \begin{pmatrix} \sum_n x_{n1} & \cdots & \sum_n x_{nD} \end{pmatrix} \tag{7.74}$$

Hence the mean of the datavectors is given by

$$\overline{\mathbf{x}}^\top = \frac{1}{N} \mathbf{1}_N^\top \mathbf{X} \tag{7.75}$$

We can sum across the columns by postmultiplying by a $D \times 1$ matrix of ones to create a $N \times 1$ matrix:

$$\mathbf{X}\mathbf{1}_D = \begin{pmatrix} \sum_d x_{1d} \\ \vdots \\ \sum_d x_{Nd} \end{pmatrix} \tag{7.76}$$

We can sum all entries in a matrix by pre and post multiplying by a vector of 1s:

$$\mathbf{1}_N^\top \mathbf{X} \mathbf{1}_D = \sum_{ij} X_{ij} \tag{7.77}$$

Hence the overall mean is given by

$$\overline{x} = \frac{1}{ND} \mathbf{1}_N^\top \mathbf{X} \mathbf{1}_D \tag{7.78}$$

7.2.4.2 Scaling rows and columns of a matrix

We often want to scale rows or columns of a data matrix (e.g., to standardize them). We now show how to write this in matrix notation.

If we pre-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\mathbf{s})$, where $\mathbf{s}$ is an N-vector, then we just scale each row of $\mathbf{X}$ by the corresponding scale factor in $\mathbf{s}$:

$$\text{diag}(\mathbf{s})\mathbf{X} = \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_N \end{pmatrix} \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_1 x_{N,1} \\ & \ddots & \\ s_N x_{1,D} & \cdots & s_N x_{N,D} \end{pmatrix} \tag{7.79}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

If we post-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\mathbf{s})$, where $\mathbf{s}$ is a D-vector, then we just scale each column of $\mathbf{X}$ by the corresponding element in $\mathbf{s}$.

$$\mathbf{X}\text{diag}(\mathbf{s}) = \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_D \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_D x_{1,D} \\ & \ddots & \\ s_1 x_{N,1} & \cdots & s_D x_{N,D} \end{pmatrix} \tag{7.80}$$

Thus we can rewrite the standardization operation from Sec. 10.2.8 in matrix form as follows:

$$\text{standardize}(\mathbf{X}) = (\mathbf{X} - \mathbf{1}_N \boldsymbol{\mu}^T)\text{diag}(\boldsymbol{\sigma})^{-1} \tag{7.81}$$

where $\boldsymbol{\mu} = \overline{\mathbf{x}}$ is the empirical mean, and $\boldsymbol{\sigma}$ is a vector of the empirical standard deviations.

7.2.4.3 Sum of squares and scatter matrix

The **sum of squares matrix** is $D \times D$ matrix defined by

$$\mathbf{S}_0 \triangleq \mathbf{X}^\top \mathbf{X} = \sum_{n=1}^{N} \mathbf{x}_n \mathbf{x}_n^\top = \sum_{n=1}^{N} \begin{pmatrix} x_{n,1}^2 & \cdots & x_{n,1} x_{n,D} \\ & \ddots & \\ x_{n,D} x_{n,1} & \cdots & x_{n,D}^2 \end{pmatrix} \tag{7.82}$$

The **scatter matrix** is a $D \times D$ matrix defined by

$$\mathbf{S}_{\overline{\mathbf{x}}} \triangleq \sum_{n=1}^{N} (\mathbf{x}_n - \overline{\mathbf{x}})(\mathbf{x}_n - \overline{\mathbf{x}})^\top = \left(\sum_n \mathbf{x}_n \mathbf{x}_n^\top \right) - N \overline{\mathbf{x}} \overline{\mathbf{x}}^\top \tag{7.83}$$

We see that this is the sum of squares matrix applied to the mean-centered data. More precisely, define $\tilde{\mathbf{X}}$ to be a version of $\mathbf{X}$ where we subtract the mean $\overline{\mathbf{x}} = \frac{1}{N} \mathbf{X}^\top \mathbf{1}_N$ off every row. Hence we can compute the centered data matrix using

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \overline{\mathbf{x}}^\top = \mathbf{X} - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^\top \mathbf{X} = \mathbf{C}_N \mathbf{X} \tag{7.84}$$

where

$$\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^\top \tag{7.85}$$

is the **centering matrix**. The scatter matrix can now be computed as follows:

$$\mathbf{S}_{\overline{\mathbf{x}}} = \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \mathbf{X}^\top \mathbf{C}_N^\top \mathbf{C}_N \mathbf{X} = \mathbf{X}^\top \mathbf{C}_N \mathbf{X} \tag{7.86}$$

where we exploited the fact that $\mathbf{C}_N$ is symmetric and idempotent, i.e., $\mathbf{C}_N^k = \mathbf{C}_N$ for $k = 1, 2, \ldots$ (since once we subtract the mean, subtracting it again has no effect).

7.2.4.4 Gram matrix

The $N \times N$ matrix $\mathbf{X}\mathbf{X}^\top$ is a matrix of inner products called the **Gram matrix**:

$$\mathbf{K} \triangleq \mathbf{X}\mathbf{X}^\top = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \cdots & \mathbf{x}_1^\top \mathbf{x}_N \\ & \ddots & \\ \mathbf{x}_n^\top \mathbf{x}_1 & \cdots & \mathbf{x}_N^\top \mathbf{x}_N \end{pmatrix} \tag{7.87}$$

Sometimes we want to compute the inner products of the mean-centered data vectors, $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top$. However, if we are working with a feature similarity matrix instead of raw features, we will only have access to $\mathbf{K}$, not $\mathbf{X}$. (We will see examples of this in Sec. 20.4.4 and Sec. 20.4.6.) Fortunately, we can compute $\tilde{\mathbf{K}}$ from $\mathbf{K}$ using the **double centering trick**:

$$\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top = \mathbf{C}_N \mathbf{K} \mathbf{C}_N = \mathbf{K} - \frac{\mathbf{1}_N}{N}\mathbf{K} - \mathbf{K}\frac{\mathbf{1}_N}{N} + \frac{\mathbf{1}_N}{N}\mathbf{K}\frac{\mathbf{1}_N}{N} \tag{7.88}$$

This subtracts the row means and column means from $\mathbf{K}$, and adds back the global mean that gets subtracted twice, so that both row means and column means of $\tilde{\mathbf{K}}$ are equal to zero.

To see why Eq. (7.88) is true, consider the scalar form:

$$\tilde{K}_{ij} = \tilde{\mathbf{x}}_i^\top \tilde{\mathbf{x}}_j = (\mathbf{x}_i - \frac{1}{N}\sum_{k=1}^N \mathbf{x}_k)(\mathbf{x}_j - \frac{1}{N}\sum_{l=1}^N \mathbf{x}_l) \tag{7.89}$$

$$= \mathbf{x}_i^\top \mathbf{x}_j - \frac{1}{N}\sum_{k=1}^N \mathbf{x}_i^\top \mathbf{x}_k - \frac{1}{N}\sum_{k=1}^N \mathbf{x}_j^\top \mathbf{x}_k + \frac{1}{N^2}\sum_{k=1}^N \sum_{l=1}^N \mathbf{x}_k^\top \mathbf{x}_l \tag{7.90}$$

7.2.4.5 Distance matrix

Let $\mathbf{X}$ be $N_x \times D$ datamatrix, and $\mathbf{Y}$ be another $N_y \times D$ datamatrix. We can compute the squared pairwise distances between these using

$$\mathbf{D}_{ij} = (\mathbf{x}_i - \mathbf{y}_j)^\top (\mathbf{x}_i - \mathbf{y}_j) = ||\mathbf{x}_i||^2 - 2\mathbf{x}_i^\top \mathbf{y}_j + ||\mathbf{y}_j||^2 \tag{7.91}$$

Let us now write this in matrix form. Let $\hat{\mathbf{x}} = [||\mathbf{x}_1||^2; \cdots; ||\mathbf{x}_{N_x}||^2] = \text{diag}(\mathbf{X}\mathbf{X}^\top)$ be a vector where each element is the squared norm of the examples in $\mathbf{X}$, and define $\hat{\mathbf{y}}$ similarly. Then we have

$$\mathbf{D} = \hat{\mathbf{x}}\mathbf{1}_{N_y}^\top - 2\mathbf{X}\mathbf{Y}^\top + \mathbf{1}_{N_x}\hat{\mathbf{y}}^\top \tag{7.92}$$

In the case that $\mathbf{X} = \mathbf{Y}$, we have

$$\mathbf{D} = \hat{\mathbf{x}}\mathbf{1}_N^\top - 2\mathbf{X}\mathbf{X}^\top + \mathbf{1}_N\hat{\mathbf{x}}^\top \tag{7.93}$$

This vectorized computation is often much faster than using for loops.

7.2.5 Kronecker products *

If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is a $p \times q$ matrix, then the **Kronecker product** $\mathbf{A} \otimes \mathbf{B}$ is the $mp \times nq$ block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \tag{7.94}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

For example,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \otimes \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{11}b_{12} & a_{11}b_{13} & a_{12}b_{11} & a_{12}b_{12} & a_{12}b_{13} \\ a_{11}b_{21} & a_{11}b_{22} & a_{11}b_{23} & a_{12}b_{21} & a_{12}b_{22} & a_{12}b_{23} \\ a_{21}b_{11} & a_{21}b_{12} & a_{21}b_{13} & a_{22}b_{11} & a_{22}b_{12} & a_{22}b_{13} \\ a_{21}b_{21} & a_{21}b_{22} & a_{21}b_{23} & a_{22}b_{21} & a_{22}b_{22} & a_{22}b_{23} \\ a_{31}b_{11} & a_{31}b_{12} & a_{31}b_{13} & a_{32}b_{11} & a_{32}b_{12} & a_{32}b_{13} \\ a_{31}b_{21} & a_{31}b_{22} & a_{31}b_{23} & a_{32}b_{21} & a_{32}b_{22} & a_{32}b_{23} \end{bmatrix} \tag{7.95}$$

Here are some useful identities:

$$(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1} \tag{7.96}$$

$$(\mathbf{A} \otimes \mathbf{B})\text{vec}(\mathbf{C}) = \text{vec}(\mathbf{B}\mathbf{C}\mathbf{A}^\top) \tag{7.97}$$

See [Loa00] for a list of other useful properties.

7.2.6 Einstein summation *

Einstein summation, or **einsum** for short, is a notational shortcut for working with tensors. The convention was introduced by Einstein [Ein16, sec 5], who later joked to a friend, "I have made a great discovery in mathematics; I have suppressed the summation sign every time that the summation must be made over an index which occurs twice..." [Pai05, p.216]. For example, instead of writing matrix multiplication as $C_{ij} = \sum_k A_{ik}B_{kj}$, we can just write it as $C_{ij} = A_{ik}B_{kj}$, where we drop the $\sum_k$.

As a more complex example, suppose we have a 3d tensor S_{ntk} where n indexes examples in the batch, t indexes locations in the sequence, and k indexes words in a one-hot representation. Let W_{kd} be an embedding matrix that maps sparse one-hot vectors $\mathbb{R}^k$ to dense vectors in $\mathbb{R}^d$. We can convert the batch of sequences of one-hots to a batch of sequences of embeddings as follows:

$$E_{ntd} = \sum_k S_{ntk}W_{kd} \tag{7.98}$$

We can compute the sum of the embedding vectors for each sequence (to get a global representation of each bag of words) as follows:

$$E_{nd} = \sum_k \sum_t S_{ntk}W_{kd} \tag{7.99}$$

Finally we can pass each sequence's vector representation through another linear transform V_{dc} to map to the logits over a classifier with c labels:

$$L_{nc} = \sum_d E_{nd}V_{dc} = \sum_d \sum_k \sum_t S_{ntk}W_{kd}V_{dc} \tag{7.100}$$

In einsum notation, we write $L_{nc} = S_{ntk}W_{kd}V_{dc}$. We sum over k and d because those indices occur twice on the RHS. We sum over t because that index does not occur on the LHS.

Einsum is implemented in NumPy, Tensorflow, PyTorch, etc. What makes it particularly useful is that it can perform the relevant tensor multiplications in complex expressions in an optimal order,

so as to minimize time and intermediate memory allocation.[2] The library is best illustrated by the examples in code.probml.ai/book1/einsum_demo.

Note that the speed of einsum depends on the order in which the operations are performed, which depends on the shapes of the relevant arguments. The optimal ordering minimizes the treewidth of the resulting computation graph, as explained in [GASG18]. In general, the time to compute the optimal ordering is exponential in the number of arguments, so it is common to use a greedy approximation. However, if we expect to repeat the same calculation many times, using tensors of the same shape but potentially different content, we can compute the optimal ordering once and reuse it multiple times.

7.3 Matrix inversion

In this section, we discuss how to invert different kinds of matrces.

7.3.1 The inverse of a square matrix

The **inverse** of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is denoted $\mathbf{A}^{-1}$, and is the unique matrix such that

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} = \mathbf{A}\mathbf{A}^{-1}. \tag{7.101}$$

Note that $\mathbf{A}^{-1}$ exists if and only if $\det(\mathbf{A}) \neq 0$. If $\det(\mathbf{A}) = 0$, it is called a **singular** matrix.

The following are properties of the inverse; all assume that $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ are non-singular:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \tag{7.102}$$

$$(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \tag{7.103}$$

$$(\mathbf{A}^{-1})^{\top} = (\mathbf{A}^{\top})^{-1} \triangleq \mathbf{A}^{-T} \tag{7.104}$$

For the case of a 2×2 matrix, the expression for $\mathbf{A}^{-1}$ is simple enough to give explicitly. We have

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \tag{7.105}$$

For a block diagonal matrix, the inverse is obtained by simply inverting each block separately, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{pmatrix} \tag{7.106}$$

7.3.2 Schur complements *

In this section, we review some useful results concerning block structured matrices.

Theorem 7.3.1 (Inverse of a partitioned matrix)**.** *Consider a general partitioned matrix*

$$\mathbf{M} = \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \tag{7.107}$$

2. These optimizations are implemented in the **opt-einsum** library [GASG18]. Its core functionality is included in NumPy and JAX `einsum` functions, provided you set `optimize=True` parameter.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where we assume $\mathbf{E}$ *and* $\mathbf{H}$ *are invertible. We have*

$$\mathbf{M}^{-1} = \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \tag{7.108}$$

$$= \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \tag{7.109}$$

where

$$\mathbf{M}/\mathbf{H} \triangleq \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} \tag{7.110}$$

$$\mathbf{M}/\mathbf{E} \triangleq \mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F} \tag{7.111}$$

We say that $\mathbf{M}/\mathbf{H}$ *is the* **Schur complement** *of* $\mathbf{M}$ *wrt* $\mathbf{H}$*, and* $\mathbf{M}/\mathbf{E}$ *is the Schur complement of* $\mathbf{M}$ *wrt* $\mathbf{E}$*. (The reason for this notation will be explained in Eq. (7.132).)*

Eq. (7.108) and Eq. (7.109) are called the **partitioned inverse formulae**.

Proof. If we could block diagonalize $\mathbf{M}$, it would be easier to invert. To zero out the top right block of $\mathbf{M}$ we can pre-multiply as follows

$$\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \tag{7.112}$$

Similarly, to zero out the bottom left we can post-multiply as follows

$$\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix} \tag{7.113}$$

Putting it all together we get

$$\underbrace{\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}}_{\mathbf{M}} \underbrace{\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix}}_{\mathbf{Z}} = \underbrace{\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix}}_{\mathbf{W}} \tag{7.114}$$

Taking the inverse of both sides yields

$$\mathbf{Z}^{-1}\mathbf{M}^{-1}\mathbf{X}^{-1} = \mathbf{W}^{-1} \tag{7.115}$$

$$\mathbf{M}^{-1} = \mathbf{Z}\mathbf{W}^{-1}\mathbf{X} \tag{7.116}$$

Substituting in the definitions we get

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \tag{7.117}$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \tag{7.118}$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \tag{7.119}$$

Alternatively, we could have decomposed the matrix $\mathbf{M}$ in terms of $\mathbf{E}$ and $\mathbf{M}/\mathbf{E} = (\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})$, yielding

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \tag{7.120}$$

□

7.3.3 The matrix inversion lemma *

Equating the top left block of the first matrix in Eq. (7.118) with the top left block of the matrix in Eq. (7.119)

$$(\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G})^{-1} = \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})^{-1}\mathbf{G}\mathbf{E}^{-1} \tag{7.121}$$

This is known as the **matrix inversion lemma** or the **Sherman-Morrison-Woodbury formula**.

A typical application in machine learning is the following. Let $\mathbf{X}$ be an $N \times D$ data matrix, and $\mathbf{\Sigma}$ be $N \times N$ diagonal matrix. Then we have (using the substitutions $\mathbf{E} = \mathbf{\Sigma}$, $\mathbf{F} = \mathbf{G}^\top = \mathbf{X}$, and $\mathbf{H}^{-1} = -\mathbf{I}$) the following result:

$$(\mathbf{\Sigma} + \mathbf{X}\mathbf{X}^\top)^{-1} = \mathbf{\Sigma}^{-1} - \mathbf{\Sigma}^{-1}\mathbf{X}(\mathbf{I} + \mathbf{X}^\top\mathbf{\Sigma}^{-1}\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{\Sigma}^{-1} \tag{7.122}$$

The LHS takes $O(N^3)$ time to compute, the RHS takes time $O(D^3)$ to compute.

Another application concerns computing a **rank one update** of an inverse matrix. Let $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = \mathbf{u}$, $\mathbf{G} = \mathbf{v}^\top$, and $H = -1$. Then we have

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^\top)^{-1} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{u}(-1 - \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u})^{-1}\mathbf{v}^\top\mathbf{A}^{-1} \tag{7.123}$$

$$= \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^\top\mathbf{A}^{-1}}{1 + \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u}} \tag{7.124}$$

This is known as the **Sherman-Morrison formula**.

7.3.4 Matrix determinant lemma *

We now use the above results to derive an efficient way to compute the determinant of a block-structured matrix.

From Eq. (7.114), we have

$$|\mathbf{X}||\mathbf{M}||\mathbf{Z}| = |\mathbf{W}| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \tag{7.125}$$

$$\left|\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}\right| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \tag{7.126}$$

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| \tag{7.127}$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}|}{|\mathbf{H}|} \tag{7.128}$$

So we can see that $\mathbf{M}/\mathbf{H}$ acts somewhat like a division operator.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Furthermore, we have

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| = |\mathbf{M}/\mathbf{E}||\mathbf{E}| \tag{7.129}$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}/\mathbf{E}||\mathbf{E}|}{|\mathbf{H}|} \tag{7.130}$$

$$|\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}| = |\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F}||\mathbf{H}^{-1}||\mathbf{E}| \tag{7.131}$$

Hence (setting $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = -\mathbf{u}$, $\mathbf{G} = \mathbf{v}^\top$, $\mathbf{H} = 1$) we have

$$|\mathbf{A} + \mathbf{u}\mathbf{v}^\top| = (1 + \mathbf{v}^\top \mathbf{A}^{-1}\mathbf{u})|\mathbf{A}| \tag{7.132}$$

This is known as the **matrix determinant lemma**.

7.4 Eigenvalue decomposition (EVD)

In this section, we review some standard material on the **eigenvalue decomposition** or **EVD** of square (real-valued) matrices.

7.4.1 Basics

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, we say that $\lambda \in \mathbb{R}$ is an **eigenvalue** of $\mathbf{A}$ and $\mathbf{u} \in \mathbb{R}^n$ is the corresponding **eigenvector** if

$$\mathbf{A}\mathbf{u} = \lambda\mathbf{u}, \quad \mathbf{u} \neq 0 \ . \tag{7.133}$$

Intuitively, this definition means that multiplying $\mathbf{A}$ by the vector $\mathbf{u}$ results in a new vector that points in the same direction as $\mathbf{u}$, but is scaled by a factor λ. For example, if $\mathbf{A}$ is a rotation matrix, then $\mathbf{u}$ is the axis of rotation and $\lambda = 1$.

Note that for any eigenvector $\mathbf{u} \in \mathbb{R}^n$, and scalar $c \in \mathbb{R}$,

$$\mathbf{A}(c\mathbf{u}) = c\mathbf{A}\mathbf{u} = c\lambda\mathbf{u} = \lambda(c\mathbf{u}) \tag{7.134}$$

Hence $c\mathbf{u}$ is also an eigenvector. For this reason when we talk about "the" eigenvector associated with λ, we usually assume that the eigenvector is normalized to have length 1 (this still creates some ambiguity, since $\mathbf{u}$ and $-\mathbf{u}$ will both be eigenvectors, but we will have to live with this).

We can rewrite the equation above to state that $(\lambda, \mathbf{x})$ is an eigenvalue-eigenvector pair of $\mathbf{A}$ if

$$(\lambda\mathbf{I} - \mathbf{A})\mathbf{u} = \mathbf{0}, \quad \mathbf{u} \neq 0 \ . \tag{7.135}$$

Now $(\lambda\mathbf{I} - \mathbf{A})\mathbf{u} = \mathbf{0}$ has a non-zero solution to $\mathbf{u}$ if and only if $(\lambda\mathbf{I} - \mathbf{A})$ has a non-empty nullspace, which is only the case if $(\lambda\mathbf{I} - \mathbf{A})$ is singular, i.e.,

$$\det(\lambda\mathbf{I} - \mathbf{A}) = 0 \ . \tag{7.136}$$

This is called the **characteristic equation** of $\mathbf{A}$. (See Exercise 7.2.) The n solutions of this equation are the n (possibly complex-valued) eigenvalues λ_i, and $\mathbf{u}_i$ are the corresponding eigenvectors. It is standard to sort the eigenvectors in order of their eigenvalues, with the largest magnitude ones first.

The following are properties of eigenvalues and eigenvectors.

- The trace of a matrix is equal to the sum of its eigenvalues,

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i \ . \tag{7.137}$$

- The determinant of $\mathbf{A}$ is equal to the product of its eigenvalues,

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i \ . \tag{7.138}$$

- The rank of $\mathbf{A}$ is equal to the number of non-zero eigenvalues of $\mathbf{A}$.
- If $\mathbf{A}$ is non-singular then $1/\lambda_i$ is an eigenvalue of $\mathbf{A}^{-1}$ with associated eigenvector $\mathbf{u}_i$, i.e., $\mathbf{A}^{-1}\mathbf{u}_i = (1/\lambda_i)\mathbf{u}_i$.
- The eigenvalues of a diagonal or triangular matrix are just the diagonal entries.

7.4.2 Diagonalization

We can write all the eigenvector equations simultaneously as

$$\mathbf{AU} = \mathbf{U}\boldsymbol{\Lambda} \tag{7.139}$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ are the eigenvectors of $\mathbf{A}$ and $\boldsymbol{\Lambda}$ is a diagonal matrix whose entries are the eigenvalues of $\mathbf{A}$, i.e.,

$$\mathbf{U} \in \mathbb{R}^{n \times n} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \\ | & | & & | \end{bmatrix}, \ \boldsymbol{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_n) \ . \tag{7.140}$$

If the eigenvectors of $\mathbf{A}$ are linearly independent, then the matrix $\mathbf{U}$ will be invertible, so

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^{-1}. \tag{7.141}$$

A matrix that can be written in this form is called **diagonalizable**.

7.4.3 Eigenvalues and eigenvectors of symmetric matrices

When $\mathbf{A}$ is real and symmetric, it can be shown that all the eigenvalues are real, and the eigenvectors are **orthonormal**, i.e., $\mathbf{u}_i^\top \mathbf{u}_j = 0$ if $i \neq j$, and $\mathbf{u}_i^\top \mathbf{u}_i = 1$, where $\mathbf{u}_i$ are the eigenvectors. In matrix form, this becomes $\mathbf{U}^\top\mathbf{U} = \mathbf{U}\mathbf{U}^\top = \mathbf{I}$; hence we see that $\mathbf{U}$ is an orthogonal matrix.

We can therefore represent $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top = \begin{pmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_1^\top & - \\ - & \mathbf{u}_2^\top & - \\ & \vdots & \\ - & \mathbf{u}_n^\top & - \end{pmatrix} \tag{7.142}$$

$$= \lambda_1 \begin{pmatrix} | \\ \mathbf{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_1^\top & - \end{pmatrix} + \cdots + \lambda_n \begin{pmatrix} | \\ \mathbf{u}_n \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{u}_n^\top & - \end{pmatrix} = \sum_{i=1}^{n} \lambda_i \mathbf{u}_i \mathbf{u}_i^\top \tag{7.143}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Thus multiplying by any symmetric matrix $\mathbf{A}$ can be interpreted as multiplying by a rotation matrix $\mathbf{U}^\top$, a scaling matrix $\mathbf{\Lambda}$, followed by an inverse rotation $\mathbf{U}$.

Once we have diagonalized a matrix, it is easy to invert. Since $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, where $\mathbf{U}^\top = \mathbf{U}^{-1}$, we have

$$\mathbf{A}^{-1} = \mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^\top = \sum_{i=1}^{d} \frac{1}{\lambda_i}\mathbf{u}_i\mathbf{u}_i^\top \tag{7.144}$$

This corresponds to rotating, unscaling, and then rotating back.

7.4.3.1 Checking for positive definiteness

We can also use the diagonalization property to show that a symmetric matrix is positive definite iff all its eigenvalues are positive. To see this, note that

$$\mathbf{x}^\top\mathbf{A}\mathbf{x} = \mathbf{x}^\top\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top\mathbf{x} = \mathbf{y}^\top\mathbf{\Lambda}\mathbf{y} = \sum_{i=1}^{n}\lambda_i y_i^2 \tag{7.145}$$

where $\mathbf{y} = \mathbf{U}^\top\mathbf{x}$. Because y_i^2 is always nonnegative, the sign of this expression depends entirely on the λ_i's. If all $\lambda_i > 0$, then the matrix is positive definite; if all $\lambda_i \geq 0$, it is positive semidefinite. Likewise, if all $\lambda_i < 0$ or $\lambda_i \leq 0$, then $\mathbf{A}$ is negative definite or negative semidefinite respectively. Finally, if $\mathbf{A}$ has both positive and negative eigenvalues, it is indefinite.

7.4.4 Geometry of quadratic forms

A **quadratic form** is a function that can be written as

$$f(\mathbf{x}) = \mathbf{x}^\top\mathbf{A}\mathbf{x} \tag{7.146}$$

where $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{A}$ is a positive definite, symmetric n-by-n matrix. Let $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ be a diagonalization of $\mathbf{A}$ (see Sec. 7.4.3). Hence we can write

$$f(\mathbf{x}) = \mathbf{x}^\top\mathbf{A}\mathbf{x} = \mathbf{x}^\top\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top\mathbf{x} = \mathbf{y}^\top\mathbf{\Lambda}\mathbf{y} = \sum_{i=1}^{n}\lambda_i y_i^2 \tag{7.147}$$

where $y_i = \mathbf{x}^\top\mathbf{u}_i$ and $\lambda_i > 0$ (since $\mathbf{A}$ is positive definite). The level sets of $f(\mathbf{x})$ define hyper-ellipsoids. For example, in 2d, we have

$$\lambda_1 y_1^2 + \lambda_2 y_2^2 = r \tag{7.148}$$

which is the equation of a 2d ellipse. This is illustrated in Fig. 7.6. The eigenvectors determine the orientation of the ellipse, and the eigenvalues determine how elongated it is.

7.4.5 Standardizing and whitening data

Suppose we have a dataset $\mathbf{X} \in \mathbb{R}^{N\times D}$. It is common to preprocess the data so that each column has zero mean and unit variance. This is called standardizing the data, as we discussed in Sec. 10.2.8.

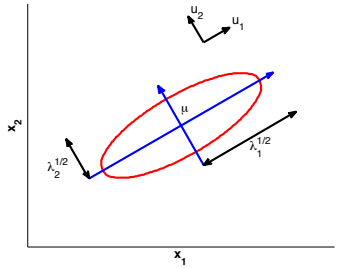


Figure 7.6: Visualization of a level set of the quadratic form $(\mathbf{x}-\boldsymbol{\mu})^\top \mathbf{A}(\mathbf{x}-\boldsymbol{\mu})$ in 2d. The major and minor axes of the ellipse are defined by the first two eigenvectors of $\mathbf{A}$, namely $\mathbf{u}_1$ and $\mathbf{u}_2$. Adapted from Figure 2.7 of [Bis06]. Generated by code at figures.probml.ai/book1/7.6.

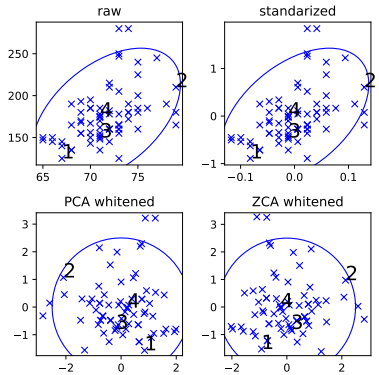


Figure 7.7: (a) Height/weight data. (b) Standardized. (c) PCA Whitening. (d) ZCA whitening. Numbers refer to the first 4 datapoints, but there are 73 datapoints in total. Generated by code at figures.probml.ai/book1/7.7.

Although standardizing forces the variance to be 1, it does not remove correlation between the columns. To do that, we must **whiten** the data. To define this, let the empirical covariance matrix be $\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{X}^\top\mathbf{X}$, and let $\boldsymbol{\Sigma} = \mathbf{EDE}^\top$ be its diagonalization. Equivalently, let $[\mathbf{U}, \mathbf{S}, \mathbf{V}]$ be the SVD of $\mathbf{X}$ (so $\mathbf{E} = \mathbf{V}$ and $\mathbf{D} = \mathbf{S}^2$, as we discuss in Sec. 20.1.3.3.) Now define

$$\mathbf{W}_{pca} = \mathbf{D}^{-\frac{1}{2}}\mathbf{E}^\top \tag{7.149}$$

This is called the **PCA whitening** matrix. (We discuss PCA in Sec. 20.1.) Let $\mathbf{y} = \mathbf{W}_{pca}\mathbf{x}$ be a transformed vector. We can check that its covariance is white as follows:

$$\text{Cov}\left[\mathbf{y}\right] = \mathbf{W}\mathbb{E}\left[\mathbf{x}\mathbf{x}^\top\right]\mathbf{W}^\top = \mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^\top = (\mathbf{D}^{-\frac{1}{2}}\mathbf{E}^\top)(\mathbf{EDE}^\top)(\mathbf{ED}^{-\frac{1}{2}}) = \mathbf{I} \tag{7.150}$$

The whitening matrix is not unique, since any rotation of it, $\mathbf{W} = \mathbf{RW}_{pca}$, will still maintain the whitening property, i.e., $\mathbf{W}^\top\mathbf{W} = \boldsymbol{\Sigma}^{-1}$. For example, if we take $\mathbf{R} = \mathbf{E}$, we get

$$\mathbf{W}_{zca} = \mathbf{ED}^{-\frac{1}{2}}\mathbf{E}^\top = \boldsymbol{\Sigma}^{-\frac{1}{2}} = \mathbf{VS}^{-1}\mathbf{V}^\top \tag{7.151}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is called **Mahalanobis whitening** or **ZCA**. (ZCA stands for "zero-phase component analysis", and was introduced in [BS97].) The advantage of ZCA whitening over PCA whitening is that the resulting transformed data is as close as possible to the original data (in the least squares sense) [Amo17]. This is illustrated in Fig. 7.7. When applied to images, the ZCA transformed data vectors still look like images. This is useful when the method is used inside a deep learning system [KH09].

7.4.6 Power method

We now describe a simple iterative method for computing the eigenvector corresponding to the largest eigenvalue of a real, symmetric matrix; this is called the **power method**. This can be useful when the matrix is very large but sparse. For example, it used by Google's **PageRank** to compute the stationary distribution of the transition matrix of the world wide web (a matrix of size about 3 billion by 3 billion!). In Sec. 7.4.7, we will see how to use this method to compute subsequent eigenvectors and values.

Let $\mathbf{A}$ be a matrix with orthonormal eigenvectors $\mathbf{u}_i$ and eigenvalues $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m| \geq 0$, so $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$. Let $\mathbf{v}_{(0)}$ be an arbitrary vector in the range of $\mathbf{A}$, so $\mathbf{A}\mathbf{x} = \mathbf{v}_{(0)}$ for some $\mathbf{x}$. Hence we can write $\mathbf{v}_{(0)}$ as

$$\mathbf{v}_0 = \mathbf{U}(\mathbf{\Lambda}\mathbf{U}^\top \mathbf{x}) = a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m \tag{7.152}$$

for some constants a_i. We can now repeatedly multiply $\mathbf{v}$ by $\mathbf{A}$ and renormalize:

$$\mathbf{v}_t \propto \mathbf{A}\mathbf{v}_{t-1} \tag{7.153}$$

(We normalize at each iteration for numerical stability.)

Since $\mathbf{v}_t$ is a multiple of $\mathbf{A}^t\mathbf{v}_0$, we have

$$\mathbf{v}_t \propto a_1\lambda_1^t\mathbf{u}_1 + a_2\lambda_2^t\mathbf{u}_2 + \cdots + a_m\lambda_m^t\mathbf{u}_m \tag{7.154}$$

$$= \lambda_1^t\left(a_1\mathbf{u}_1 + a_1(\lambda_2/\lambda_1)^t\mathbf{u}_2 + \cdots + a_m(\lambda_m/\lambda_1)^t\mathbf{u}_m\right) \tag{7.155}$$

$$\rightarrow \lambda_1^t a_1\mathbf{u}_1 \tag{7.156}$$

since $\frac{|\lambda_k|}{|\lambda_1|} < 1$ for $k > 1$ (assuming the eigenvalues are sorted in descending order). So we see that this converges to $\mathbf{u}_1$, although not very quickly (the error is reduced by approximately $|\lambda_2/\lambda_1|$ at each iteration). The only requirement is that the initial guess satisfy $\mathbf{v}_0^\top\mathbf{u}_1 \neq 0$, which will be true for a random $\mathbf{v}_0$ with high probability.

We now discuss how to compute the corresponding eigenvalue, λ_1. Define the **Rayleigh quotient** to be

$$R(\mathbf{A}, \mathbf{x}) \triangleq \frac{\mathbf{x}^\top\mathbf{A}\mathbf{x}}{\mathbf{x}^\top\mathbf{x}} \tag{7.157}$$

Hence

$$R(\mathbf{A}, \mathbf{u}_i) = \frac{\mathbf{u}_i^\top\mathbf{A}\mathbf{u}_i}{\mathbf{u}_i^\top\mathbf{u}_i} = \frac{\lambda_i\mathbf{u}_i^\top\mathbf{u}_i}{\mathbf{u}_i^\top\mathbf{u}_i} = \lambda_i \tag{7.158}$$

Thus we can easily compute λ_1 from $\mathbf{u}_1$ and $\mathbf{A}$. See code.probml.ai/book1/power_method_demo for a demo.

7.4.7 Deflation

Suppose we have computed the first eigenvector and value $\mathbf{u}_1, \lambda_1$ by the power method. We now describe how to compute subsequent eigenvectors and values. Since the eigenvectors are orthonormal, and the eigenvalues are real, we can project out the $\mathbf{u}_1$ component from the matrix as follows:

$$\mathbf{A}^{(2)} = (\mathbf{I} - \mathbf{u}_1\mathbf{u}_1^\top)\mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \mathbf{u}_1\mathbf{u}_1^\top\mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \lambda_1\mathbf{u}_1\mathbf{u}_1^\top \tag{7.159}$$

This is called matrix **deflation**. We can then apply the power method to $\mathbf{A}^{(2)}$, which will find the largest eigenvector/value in the subspace orthogonal to $\mathbf{u}_1$.

In Sec. 20.1.2, we show that the optimal estimate $\hat{\mathbf{W}}$ for the PCA model (described in Sec. 20.1) is given by the first K eigenvectors of the empirical covariance matrix. Hence deflation can be used to implement PCA. It can also be modified to implement sparse PCA [Mac09].

7.4.8 Eigenvectors optimize quadratic forms

We can use matrix calculus to solve an optimization problem in a way that leads directly to eigenvalue/eigenvector analysis. Consider the following, equality constrained optimization problem:

$$\max_{\mathbf{x}\in\mathbb{R}^n} \ \mathbf{x}^\top\mathbf{A}\mathbf{x} \quad \text{subject to } \|\mathbf{x}\|_2^2 = 1 \tag{7.160}$$

for a symmetric matrix $\mathbf{A} \in \mathbb{S}^n$. A standard way of solving optimization problems with equality constraints is by forming the Lagrangian, an objective function that includes the equality constraints (see Sec. 8.5.1). The Lagrangian in this case can be given by

$$\mathcal{L}(\mathbf{x}, \lambda) = \mathbf{x}^\top\mathbf{A}\mathbf{x} + \lambda(1 - \mathbf{x}^\top\mathbf{x}) \tag{7.161}$$

where λ is called the Lagrange multiplier associated with the equality constraint. It can be established that for x^* to be a optimal point to the problem, the gradient of the Lagrangian has to be zero at x^* (this is not the only condition, but it is required). That is,

$$\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, \lambda) = 2\mathbf{A}^\top\mathbf{x} - 2\lambda\mathbf{x} = \mathbf{0}. \tag{7.162}$$

Notice that this is just the linear equation $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$. This shows that the only points which can possibly maximize (or minimize) $\mathbf{x}^\top\mathbf{A}\mathbf{x}$ assuming $\mathbf{x}^\top\mathbf{x} = 1$ are the eigenvectors of $\mathbf{A}$.

7.5 Singular value decomposition (SVD)

We now discuss the SVD, which generalizes EVD to rectangular matrices.

7.5.1 Basics

Any (real) $m \times n$ matrix $\mathbf{A}$ can be decomposed as

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top = \sigma_1 \begin{pmatrix} | \\ \mathbf{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_1^\top & - \end{pmatrix} + \cdots + \sigma_r \begin{pmatrix} | \\ \mathbf{u}_r \\ | \end{pmatrix} \begin{pmatrix} - & \mathbf{v}_r^\top & - \end{pmatrix} \tag{7.163}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

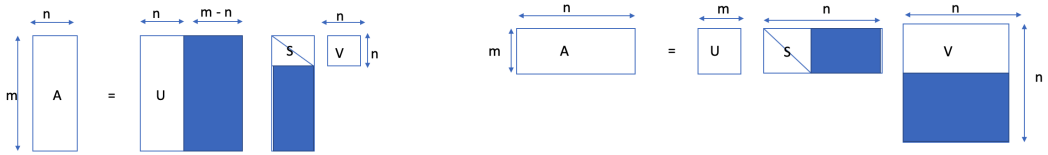


Figure 7.8: SVD decomposition of a matrix, $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. The shaded parts of each matrix are not computed in the economy-sized version. (a) Tall skinny matrix. (b) Short wide matrix.

where $\mathbf{U}$ is an $m \times m$ whose columns are orthornormal (so $\mathbf{U}^\top\mathbf{U} = \mathbf{I}_m$), $\mathbf{V}$ is $n \times n$ matrix whose rows and columns are orthonormal (so $\mathbf{V}^\top\mathbf{V} = \mathbf{V}\mathbf{V}^\top = \mathbf{I}_n$), and $\mathbf{S}$ is a $m \times n$ matrix containing the $r = \min(m, n)$ **singular values** $\sigma_i \geq 0$ on the main diagonal, with 0s filling the rest of the matrix. The columns of $\mathbf{U}$ are the left **singular vectors**, and the columns of $\mathbf{V}$ are the right singular vectors. This is called the **singular value decomposition** or **SVD** of the matrix. See Fig. 7.8 for an example.

As is apparent from Fig. 7.8a, if $m > n$, there are at most n singular values, so the last $m - n$ columns of $\mathbf{U}$ are irrelevant (since they will be multiplied by 0). The **economy sized SVD**, also called a **thin SVD**, avoids computing these unnecessary elements. In other words, if we write the $\mathbf{U}$ matrix as $\mathbf{U} = [\mathbf{U}_1, \mathbf{U}_2]$, we only compute $\mathbf{U}_1$. Fig. 7.8b shows the opposite case, where $m < n$, where we represent $\mathbf{V} = [\mathbf{V}_1; \mathbf{V}_2]$, and only compute $\mathbf{V}_1$.

The cost of computing the SVD is $O(\min(mn^2, m^2n))$. Details on how it works can be found in standard linear algebra textbooks.

7.5.2 Connection between SVD and EVD

If $\mathbf{A}$ is real, symmetric and positive definite, then the singular values are equal to the eigenvalues, and the left and right singular vectors are equal to the eigenvectors (up to a sign change):

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top = \mathbf{U}\mathbf{S}\mathbf{U}^\top = \mathbf{U}\mathbf{S}\mathbf{U}^{-1} \tag{7.164}$$

Note, however, that NumPy always returns the singular values in decreasing order, whereas the eigenvalues need not necessarily be sorted.

In general, for an arbitrary real matrix $\mathbf{A}$, if $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$, we have

$$\mathbf{A}^\top\mathbf{A} = \mathbf{V}\mathbf{S}^\top\mathbf{U}^\top\ \mathbf{U}\mathbf{S}\mathbf{V}^\top = \mathbf{V}(\mathbf{S}^\top\mathbf{S})\mathbf{V}^\top \tag{7.165}$$

Hence

$$(\mathbf{A}^\top\mathbf{A})\mathbf{V} = \mathbf{V}\mathbf{D}_n \tag{7.166}$$

so the eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are equal to $\mathbf{V}$, the right singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}^\top\mathbf{A}$ are equal to $\mathbf{D}_n = \mathbf{S}^\top\mathbf{S}$, which is an $n \times n$ diagonal matrix containing the squared singular values. Similarly

$$\mathbf{A}\mathbf{A}^\top = \mathbf{U}\mathbf{S}\mathbf{V}^\top\ \mathbf{V}\mathbf{S}^\top\mathbf{U}^\top = \mathbf{U}(\mathbf{S}\mathbf{S}^\top)\mathbf{U}^\top \tag{7.167}$$

$$(\mathbf{A}\mathbf{A}^\top)\mathbf{U} = \mathbf{U}\mathbf{D}_m \tag{7.168}$$

so the eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are equal to $\mathbf{U}$, the left singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}\mathbf{A}^\top$ are equal to $\mathbf{D}_m = \mathbf{S}\mathbf{S}^\top$, which is an $m \times m$ diagonal matrix containing the squared singular values. In summary,

$$\mathbf{U} = \text{evec}(\mathbf{A}\mathbf{A}^\top),\ \mathbf{V} = \text{evec}(\mathbf{A}^\top\mathbf{A}),\ \mathbf{D}_m = \text{eval}(\mathbf{A}\mathbf{A}^\top), \mathbf{D}_n = \text{eval}(\mathbf{A}^\top\mathbf{A}) \tag{7.169}$$

If we just use the computed (non-zero) parts in the economy-sized SVD, then we can define

$$\mathbf{D} = \mathbf{S}^2 = \mathbf{S}^\top\mathbf{S} = \mathbf{S}\mathbf{S}^\top \tag{7.170}$$

Note also that an EVD does not always exist, even for square $\mathbf{A}$, whereas an SVD always exists.

7.5.3 Pseudo inverse

The **Moore-Penrose pseudo-inverse** of $\mathbf{A}$, pseudo inverse denoted $\mathbf{A}^\dagger$, is defined as the unique matrix that satisfies the following 4 properties:

$$\mathbf{A}\mathbf{A}^\dagger\mathbf{A} = \mathbf{A},\ \mathbf{A}^\dagger\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}^\dagger,\ (\mathbf{A}\mathbf{A}^\dagger)^\top = \mathbf{A}\mathbf{A}^\dagger,\ (\mathbf{A}^\dagger\mathbf{A})^\top = \mathbf{A}^\dagger\mathbf{A} \tag{7.171}$$

If $\mathbf{A}$ is square and non-singular, then $\mathbf{A}^\dagger = \mathbf{A}^{-1}$.

If $m > n$ (tall, skinny) and the columns of $\mathbf{A}$ are linearly independent (so $\mathbf{A}$ is full rank), then

$$\mathbf{A}^\dagger = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top \tag{7.172}$$

which is the same expression as arises in the normal equations (see Sec. 11.2.2.1). In this case, $\mathbf{A}^\dagger$ is a left inverse of $\mathbf{A}$ because

$$\mathbf{A}^\dagger\mathbf{A} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{A} = \mathbf{I} \tag{7.173}$$

but is not a right inverse because

$$\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top \tag{7.174}$$

only has rank n, and so cannot be the $m \times m$ identity matrix.

If $m < n$ (short, fat) and the rows of $\mathbf{A}$ are linearly independent (so $\mathbf{A}$ is full rank), then the pseudo inverse is

$$\mathbf{A}^\dagger = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1} \tag{7.175}$$

In this case, $\mathbf{A}^\dagger$ is a right inverse of $\mathbf{A}$.

We can compute the pseudo inverse using the SVD decomposition $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. In particular, one can show that

$$\mathbf{A}^\dagger = \mathbf{V}[\text{diag}(1/\sigma_1, \cdots, 1/\sigma_r, 0, \cdots, 0)]\mathbf{U}^\top \tag{7.176}$$

where r is the rank of the matrix. Thus $\mathbf{A}^\dagger$ acts just like a matrix inverse for non-square matrices:

$$\mathbf{A}^\dagger = \mathbf{A}^{-1} = (\mathbf{U}\mathbf{S}\mathbf{V}^\top)^{-1} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top \tag{7.177}$$

where we define $\mathbf{S}^{-1} = \text{diag}(\sigma_1^{-1}, \ldots, \sigma_r^{-1}, 0, \ldots, 0)$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.5.4 SVD and the range and null space of a matrix *

In this section, we show that the left and right singular vectors form an orthonormal basis for the range and null space.

From Eq. (7.163) we have

$$\mathbf{A}\mathbf{x} = \sum_{j:\sigma_j>0} \sigma_j(\mathbf{v}_j^\top \mathbf{x})\mathbf{u}_j = \sum_{j=1}^{r} \sigma_j(\mathbf{v}_j^\top \mathbf{x})\mathbf{u}_j \tag{7.178}$$

where r is the rank of $\mathbf{A}$. Thus any $\mathbf{A}\mathbf{x}$ can be written as a linear combination of the left singular vectors $\mathbf{u}_1, \ldots, \mathbf{u}_r$, so the range of $\mathbf{A}$ is given by

$$\text{range}(\mathbf{A}) = \text{span}\left(\{\mathbf{u}_j : \sigma_j > 0\}\right) \tag{7.179}$$

with dimension r.

To find a basis for the null space, let us now define a second vector $\mathbf{y} \in \mathbb{R}^n$ that is a linear combination solely of the right singular vectors for the zero singular values,

$$\mathbf{y} = \sum_{j:\sigma_j=0} c_j \mathbf{v}_j = \sum_{j=r+1}^{n} c_j \mathbf{v}_j \tag{7.180}$$

Since the $\mathbf{v}_j$'s are orthonormal, we have

$$\mathbf{A}\mathbf{y} = \mathbf{U}\begin{pmatrix} \sigma_1 \mathbf{v}_1^\top \mathbf{y} \\ \vdots \\ \sigma_r \mathbf{v}_r^\top \mathbf{y} \\ \sigma_{r+1}\mathbf{v}_{r+1}^\top \mathbf{y} \\ \vdots \\ \sigma_n \mathbf{v}_n^\top \mathbf{y} \end{pmatrix} = \mathbf{U}\begin{pmatrix} \sigma_1 0 \\ \vdots \\ \sigma_r 0 \\ 0\mathbf{v}_{r+1}^\top \mathbf{y} \\ \vdots \\ 0\mathbf{v}_n^\top \mathbf{y} \end{pmatrix} = \mathbf{U}\mathbf{0} = \mathbf{0} \tag{7.181}$$

Hence the right singular vectors form an orthonormal basis for the null space:

$$\text{nullspace}(\mathbf{A}) = \text{span}\left(\{\mathbf{v}_j : \sigma_j = 0\}\right) \tag{7.182}$$

with dimension $n - r$. We see that

$$\dim(\text{range}(\mathbf{A})) + \dim(\text{nullspace}(\mathbf{A})) = r + (n - r) = n \tag{7.183}$$

In words, this is often written as

$$\text{rank} + \text{nullity} = n \tag{7.184}$$

This is called the **rank-nullity theorem**. It follows from this that the rank of a matrix is the number of nonzero singular values.

(a) (b) (c) (d)

Figure 7.9: Low rank approximations to an image. Top left: The original image is of size 200×320*, so has rank 200. Subsequent images have ranks 2, 5, and 20. Generated by code at figures.probml.ai/book1/7.9.*

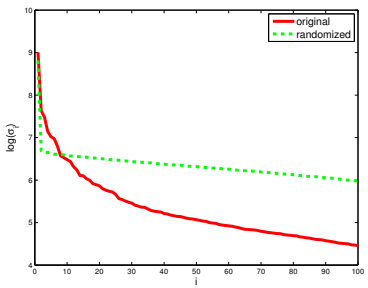


Figure 7.10: First 100 log singular values for the clown image (solid red line), and for a data matrix obtained by randomly shuffling the pixels (dotted green line). Generated by code at figures.probml.ai/book1/7.10. Adapted from Figure 14.24 of [HTF09].

7.5.5 Truncated SVD

Let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the SVD of $\mathbf{A}$, and let $\hat{\mathbf{A}}_K = \mathbf{U}_K\mathbf{S}_K\mathbf{V}_K^\top$, where we use the first K columns of $\mathbf{U}$ and $\mathbf{V}$. This can be shown to be the optimal rank K approximation, in the sense that it minimizes $||\mathbf{A} - \hat{A}_K||_F^2$.

If $K = r = \text{rank}(\mathbf{A})$, there is no error introduced by this decomposition. But if $K < r$, we incur some error. This is called a **truncated SVD**. If the singular values die off quickly, as is typical in natural data (see e.g., Fig. 7.10), the error will be small. The total number of parameters needed to represent an $N \times D$ matrix using a rank K approximation is

$$NK + KD + K = K(N + D + 1) \tag{7.185}$$

As an example, consider the 200×320 pixel image in Fig. 7.9(top left). This has 64,000 numbers in

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

it. We see that a rank 20 approximation, with only $(200 + 320 + 1) \times 20 = 10,420$ numbers is a very good approximation.

One can show that the error in this approximation is given by

$$||\mathbf{A} - \hat{\mathbf{A}}||_F = \sum_{k=K+1}^{r} \sigma_k \tag{7.186}$$

where σ_k is the k'th singular value of $\mathbf{A}$. Furthermore, one can show that the SVD offers the best rank K approximation to a matrix (best in the sense of minimizing the above Frobenius norm).

7.6 Other matrix decompositions *

In this section, we briefly review some other useful matrix decompositions.

7.6.1 LU factorization

We can factorize any square matrix $\mathbf{A}$ into a product of a lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$. For example,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}. \tag{7.187}$$

In general we may need to permute the entries in the matrix before creating this decomposition. To see this, suppose $a_{11} = 0$. Since $a_{11} = l_{11}u_{11}$, this means either l_{11} or u_{11} or both must be zero, but that would imply $\mathbf{L}$ or $\mathbf{U}$ are singular. To avoid this, the first step of the algorithm can simply reorder the rows so that the first element is nonzero. This is repeated for subsequent steps. We can denote this process by

$$\mathbf{PA} = \mathbf{LU} \tag{7.188}$$

where $\mathbf{P}$ is a permutation matrix, i.e., a square binary matrix where $P_{ij} = 1$ if row j gets permuted to row i. This is called **partial pivoting**.

7.6.2 QR decomposition

Suppose we have $\mathbf{A} \in \mathbb{R}^{m \times n}$ representing a set of linearly independent basis vectors (so $m \geq n$), and we want to find a series of orthonormal vectors $\mathbf{q}_1, \mathbf{q}_2, \ldots$ that span the successive subspaces of $\text{span}(\mathbf{a}_1)$, $\text{span}(\mathbf{a}_1, \mathbf{a}_2)$, etc. In other words, we want to find vectors $\mathbf{q}_j$ and coefficients r_{ij} such that

$$\begin{pmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \\ | & | & & | \end{pmatrix} = \begin{pmatrix} | & | & & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \cdots & \mathbf{q}_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \\ & & & r_{nn} \end{pmatrix} \tag{7.189}$$

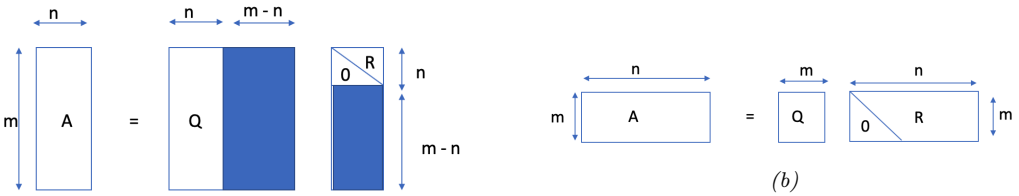


Figure 7.11: Illustration of QR decomposition, $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ and $\mathbf{R}$ is upper triangular. (a) Tall, skinny matrix. The shaded parts are not computed in the economy-sized version, since they are not needed. (b) Short, wide matrix.

We can write this

$$\mathbf{a}_1 = r_{11}\mathbf{q}_1 \tag{7.190}$$

$$\mathbf{a}_2 = r_{12}\mathbf{q}_1 + r_{22}\mathbf{q}_2 \tag{7.191}$$

$$\vdots$$

$$\mathbf{a}_n = r_{1n}\mathbf{q}_1 + \cdots + r_{nn}\mathbf{q}_n \tag{7.192}$$

so we see $\mathbf{q}_1$ spans the space of $\mathbf{a}_1$, and $\mathbf{q}_1$ and $\mathbf{q}_2$ span the space of $\{\mathbf{a}_1, \mathbf{a}_2\}$, etc.

In matrix notation, we have

$$\mathbf{A} = \hat{\mathbf{Q}}\hat{\mathbf{R}} \tag{7.193}$$

where $\hat{\mathbf{Q}}$ is $m \times n$ with orthonormal columns and $\hat{\mathbf{R}}$ is $n \times n$ and upper triangular. This is called a **reduced QR** or **economy sized QR** factorization of $\mathbf{A}$; see Fig. 7.11.

A full QR factorization appends an additional $m - n$ orthonormal columns to $\hat{\mathbf{Q}}$ so it becomes a square, orthogonal matrix $\mathbf{Q}$, which satisfies $\mathbf{QQ}^\top = \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. Also, we append rows made of zero to $\hat{\mathbf{R}}$ so it becomes an $m \times n$ matrix that is still upper triangular, called $\mathbf{R}$: see Fig. 7.11. The zero entries in $\mathbf{R}$ "kill off" the new columns in $\mathbf{Q}$, so the result is the same as $\hat{\mathbf{Q}}\hat{\mathbf{R}}$.

QR decomposition is commonly used to solve systems of linear equations, as we discuss in Sec. 11.2.2.3.

7.6.3 Cholesky decomposition

Any symmetric positive definite matrix can be factorized as $\mathbf{A} = \mathbf{R}^\top \mathbf{R}$, where $\mathbf{R}$ is upper triangular with real, positive diagonal elements. (This can also be written as $\mathbf{A} = \mathbf{LL}^\top$, where $\mathbf{L} = \mathbf{R}^\top$ is lower triangular.) This is called a **Cholesky factorization** or **matrix square root**. In NumPy, this is implemented by `np.linalg.cholesky`. The computational complexity of this operation is $O(V^3)$, where V is the number of variables, but can be less for sparse matrices. Below we give some applications of this factorization.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

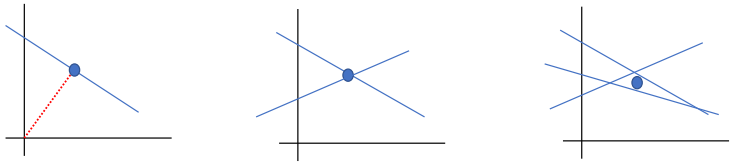


Figure 7.12: Solution of a set of m linear equations in $n = 2$ variables. (a) $m = 1 < n$ so the system is underdetermined. We show the minimal norm solution as a blue circle. (The dotted red line is orthogonal to the line, and its length is the distance to the origin.) (b) $m = n = 2$, so there is a unique solution. (c) $m = 3 > n$, so there is no unique solution. We show the least squares solution.

7.6.3.1 Application: Sampling from an MVN

The Cholesky decomposition of a covariance matrix can be used to sample from a multivariate Gaussian. Let $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$. We first sample $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, which is easy because it just requires sampling from d separate 1d Gaussians. We then set $\mathbf{y} = \mathbf{L}\mathbf{x} + \boldsymbol{\mu}$. This is valid since

$$\text{Cov}\left[\mathbf{y}\right] = \mathbf{L}\text{Cov}\left[\mathbf{x}\right]\mathbf{L}^\top = \mathbf{L}\,\mathbf{I}\,\mathbf{L}^\top = \boldsymbol{\Sigma} \tag{7.194}$$

See code.probml.ai/book1/cholesky_demo for some code.

7.7 Solving systems of linear equations *

An important application of linear algebra is the study of systems of linear equations. For example, consider the following set of 3 equations:

$$3x_1 + 2x_2 - x_3 = 1 \tag{7.195}$$
$$2x_1 - 2x_2 + 4x_3 = -2 \tag{7.196}$$
$$-x_1 + \frac{1}{2}x_2 - x_3 = 0 \tag{7.197}$$

We can represent this in matrix-vector form as follows:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{7.198}$$

where

$$\mathbf{A} = \begin{pmatrix} 3 & 2 & -1 \\ 2 & -2 & 4 \\ -1 & \frac{1}{2} & -1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ -2 \\ 0 \end{pmatrix} \tag{7.199}$$

The solution is $\mathbf{x} = [1, -2, -2]$.

In general, if we have m equations and n unknowns, then $\mathbf{A}$ will be a $m \times n$ matrix, and $\mathbf{b}$ will be a $m \times 1$ vector. If $m = n$ (and $\mathbf{A}$ is full rank), there is a single unique solution. If $m < n$, the system is **underdetermined**, so there is not a unique solution. If $m > n$, the system is **overdetermined**, since there are more constraints than unknowns, and not all the lines intersect at the same point. See Fig. 7.12 for an illustration. We discuss how to compute solutions in each of these cases below.

7.7.1 Solving square systems

In the case where $m = n$, we can solve for $\mathbf{x}$ by computing an LU decomposition, $\mathbf{A} = \mathbf{LU}$, and then proceeding as follows:

$$\mathbf{Ax} = \mathbf{b} \tag{7.200}$$
$$\mathbf{LUx} = \mathbf{b} \tag{7.201}$$
$$\mathbf{Ux} = \mathbf{L}^{-1}\mathbf{b} \triangleq \mathbf{y} \tag{7.202}$$
$$\mathbf{x} = \mathbf{U}^{-1}\mathbf{y} \tag{7.203}$$

The crucial point is that $\mathbf{L}$ and $\mathbf{U}$ are both triangular matrices, so we can avoid taking matrix inverses, and use a method known as **backsubstitution** instead.

In particular, we can solve $\mathbf{y} = \mathbf{L}^{-1}\mathbf{b}$ without taking inverses as follows. First we write

$$\begin{pmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ & & \ddots & \\ L_{n1} & L_{n2} & \cdots & L_{nn} \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} \tag{7.204}$$

Hence we can first solve $L_{nn}y_n = b_n$ to find y_n. We then substitute this in to solve

$$L_{n-1,n-1}y_{n-1} + L_{n-1,n}y_n = b_{n-1} \tag{7.205}$$

for y_{n-1}, etc. This process is often denoted by the **backslash operator**, $\mathbf{y} = \mathbf{L} \setminus \mathbf{b}$. Once we have $\mathbf{y}$, we can solve $\mathbf{x} = \mathbf{U}^{-1}\mathbf{y}$ using backsubstitution in a similar manner.

7.7.2 Solving underconstrained systems (least norm estimation)

In this section, we consider the underconstrained setting, where $m < n$.[3] We assume the rows are linearly independent, so $\mathbf{A}$ is full rank.

When $m < n$, there are multiple possible solutions, which have the form

$$\{\mathbf{x} : \mathbf{Ax} = \mathbf{b}\} = \{\mathbf{x}_p + \mathbf{z} : \mathbf{z} \in \text{nullspace}(\mathbf{A})\} \tag{7.206}$$

where $\mathbf{x}_p$ is any particular solution. It is standard to pick the particular solution with minimal ℓ_2 norm, i.e.,

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\text{argmin}} \, ||\mathbf{x}||_2^2 \;\; \text{s.t.} \;\; \mathbf{Ax} = \mathbf{b} \tag{7.207}$$

We can compute the minimal norm solution using the *right* pseudo inverse:

$$\mathbf{x}_{\text{pinv}} = \mathbf{A}^\top(\mathbf{AA}^\top)^{-1}\mathbf{b} \tag{7.208}$$

To see this, suppose $\mathbf{x}$ is some other solution, so $\mathbf{Ax} = \mathbf{b}$, and $\mathbf{A}(\mathbf{x} - \mathbf{x}_{\text{pinv}}) = \mathbf{0}$. Thus

$$(\mathbf{x} - \mathbf{x}_{\text{pinv}})^\top \mathbf{x}_{\text{pinv}} = (\mathbf{x} - \mathbf{x}_{\text{pinv}})^\top \mathbf{A}^\top(\mathbf{AA}^\top)^{-1}\mathbf{b} = (\mathbf{A}(\mathbf{x} - \mathbf{x}_{\text{pinv}}))^\top(\mathbf{AA}^\top)^{-1}\mathbf{b} = 0 \tag{7.209}$$

3. Our presentation is based in part on lecture notes by Stephen Boyd at http://ee263.stanford.edu/lectures/min-norm.pdf.

and hence $(\mathbf{x} - \mathbf{x}_{\text{pinv}}) \perp \mathbf{x}_{\text{pinv}}$. By **Pythagoras's theorem**, the norm of $\mathbf{x}$ is

$$||\mathbf{x}||^2 = ||\mathbf{x}_{\text{pinv}} + \mathbf{x} - \mathbf{x}_{\text{pinv}}||^2 = ||\mathbf{x}_{\text{pinv}}||^2 + ||\mathbf{x} - \mathbf{x}_{\text{pinv}}||^2 \geq ||\mathbf{x}_{\text{pinv}}||^2 \tag{7.210}$$

Thus any solution apart from $\mathbf{x}_{\text{pinv}}$ has larger norm.

We can also solve the constrained optimization problem in Eq. (7.207) by minimizing the following unconstrained objective

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{x}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{A}\mathbf{x} - \mathbf{b}) \tag{7.211}$$

From Sec. 8.5.1, the optimality conditions are

$$\nabla_{\mathbf{x}} \mathcal{L} = 2\mathbf{x} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}, \ \nabla_{\boldsymbol{\lambda}} \mathcal{L} = \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0} \tag{7.212}$$

From the first condition we have $\mathbf{x} = -\mathbf{A}^\top \boldsymbol{\lambda}/2$. Subsituting into the second we get

$$\mathbf{A}\mathbf{x} = -\frac{1}{2}\mathbf{A}\mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{b} \tag{7.213}$$

which implies $\boldsymbol{\lambda} = -2(\mathbf{A}\mathbf{A}^\top)^{-1}\mathbf{b}$. Hence $\mathbf{x} = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}\mathbf{b}$, which is the right pseudo inverse solution.

There is an interesting connection to regularized least squares. Let $J_1 = ||\mathbf{A}\mathbf{x} - \mathbf{b}||_2^2$ and $J_2 = ||\mathbf{x}||_2^2$. The least-norm solution minimizes J_2 with $J_1 = 0$. Now consider the following weighted sum objective

$$\mathcal{L}(\mathbf{x}) = J_2 + \lambda J_1 = ||\mathbf{x}||_2^2 + \lambda ||\mathbf{b} - \mathbf{A}\mathbf{x}||_2^2 \tag{7.214}$$

Since this is a convex problem, the minimum of the Lagrangian occurs when the gradient is zero, so

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}) = 2\mathbf{x} - 2\lambda \mathbf{A}^\top (\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{0} \implies \mathbf{x}(\mathbf{I} + \lambda \mathbf{A}^\top \mathbf{A}) = \lambda \mathbf{A}^\top \mathbf{b} \tag{7.215}$$

As $\lambda \to \infty$, we get

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} \tag{7.216}$$

where $(\mathbf{A}^\top \mathbf{A})^{-1}\mathbf{A}^\top$ is the *left* pseudoinverse of $\mathbf{A}$. Alternatively (but equivalently) we can put weight μ on the ℓ_2 regularizer $||\mathbf{x}||_2^2$ and let that go to zero. From the above we have

$$(\mu \mathbf{I} + \mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \to \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \tag{7.217}$$

So the regularized least squares solution, in the limit of no regularization, converges to the minimum norm solution.

7.7.3 Solving overconstrained systems (least squares estimation)

If $m > n$, we have an overdetermined solution, which typically does not have an exact solution, but we will try to find the solution that gets as close as possible to satisfying all of the constraints

specified by $\mathbf{Ax} = \mathbf{b}$. We can do this by minimizing the following cost function, known as the **least squares objective**:

$$f(\mathbf{x}) = \frac{1}{2}||\mathbf{Ax} - \mathbf{b}||_2^2 \tag{7.218}$$

Using matrix calculus results from Sec. 7.8 we have that the gradient is given by

$$\mathbf{g}(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = \mathbf{A}^\top \mathbf{Ax} - \mathbf{A}^\top \mathbf{b} \tag{7.219}$$

The optimum can be found by solving $\mathbf{g}(\mathbf{x}) = \mathbf{0}$. This gives

$$\mathbf{A}^\top \mathbf{Ax} = \mathbf{A}^\top \mathbf{b} \tag{7.220}$$

These are known as the **normal equations**, since, at the optimal solution, $\mathbf{b} - \mathbf{Ax}$ is normal (orthogonal) to the range of $\mathbf{A}$, as we explain in Sec. 11.2.2.2. The corresponding solution $\hat{\mathbf{x}}$ is the **ordinary least squares** (**OLS**) solution, which is given by

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} \tag{7.221}$$

The quantity $\mathbf{A}^\dagger = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ is the (left) pseudo inverse of the (non-square) matrix $\mathbf{A}$ (see Sec. 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\mathbf{x}) = \frac{\partial^2}{\partial \mathbf{x}^2} f(\mathbf{x}) = \mathbf{A}^\top \mathbf{A} \tag{7.222}$$

If $\mathbf{A}$ is full rank (so the columns of $\mathbf{A}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\mathbf{v} > \mathbf{0}$, we have

$$\mathbf{v}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v} = (\mathbf{Av})^\top (\mathbf{Av}) = ||\mathbf{Av}||^2 > 0 \tag{7.223}$$

Hence in the full rank case, the least squares objective has a unique global minimum.

7.8 Matrix calculus

The topic of **calculus** concerns computing "rates of change" of functions as we vary their inputs. It is of vital importance to machine learning, as well as almost every other numerical discipline. In this section, we review some standard results. In some cases, we use some concepts and notation from matrix algebra, which we cover in Chapter 7. For more details on these results from a deep learning perspective, see [PH18].

7.8.1 Derivatives

Consider a scalar-argument function $f : \mathbb{R} \to \mathbb{R}$. We define its **derivative** at a point a to be the quantity

$$f'(x) \triangleq \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{7.224}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

assuming the limit exists. This measures how quickly the output changes when we move a small distance in input space away from x (i.e., the "rate of change" of the function). We can interpret $f'(x)$ as the slope of the tangent line at $f(x)$, and hence

$$f(x+h) \approx f(x) + f'(x)h \tag{7.225}$$

for small h.

We can compute a **finite difference** approximation to the derivative by using a finite step size h, as follows:

$$f'(x) \equiv \underbrace{\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h\to 0} \frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h\to 0} \frac{f(x)-f(x-h)}{h}}_{\text{backward difference}} \tag{7.226}$$

The smaller the step size h, the better the estimate, although if h is too small, there can be errors due to numerical cancellation.

We can think of differentiation as an operator that maps functions to functions, $D(f) = f'$, where $f'(x)$ computes the derivative at x (assuming the derivative exists at that point). The use of the prime symbol f' to denote the derivative is called **Lagrange notation**. The second derivative function, which measures how quickly the gradient is changing, is denoted by f''. The n'th derivative function is denoted $f^{(n)}$.

Alternatively, we can use **Leibniz notation**, in which we denote the function by $y = f(x)$, and its derivative by $\frac{dy}{dx}$ or $\frac{d}{dx}f(x)$. To denote the evaluation of the derivative at a point a, we write $\left.\frac{df}{dx}\right|_{x=a}$.

7.8.2 Gradients

We can extend the notion of derivatives to handle vector-argument functions, $f : \mathbb{R}^n \to \mathbb{R}$, by defining the **partial derivative** of f with respect to x_i to be

$$\frac{\partial f}{\partial x_i} = \lim_{h\to 0} \frac{f(\mathbf{x}+h\mathbf{e}_i) - f(\mathbf{x})}{h} \tag{7.227}$$

where $\mathbf{e}_i$ is the i'th unit vector.

The **gradient** of a function at a point $\mathbf{x}$ is the vector of its partial derivatives:

$$\mathbf{g} = \frac{\partial f}{\partial \mathbf{x}} = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \tag{7.228}$$

To emphasize the point at which the gradient is evaluated, we can write

$$\mathbf{g}(\mathbf{x}^*) \triangleq \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\mathbf{x}^*} \tag{7.229}$$

We see that the operator ∇ (pronounced "nabla") maps a function $f : \mathbb{R}^n \to \mathbb{R}$ to another function $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^n$. Since $\mathbf{g}()$ is a vector-valued function, it is known as a **vector field**. By contrast, the derivative function f' is a **scalar field**.

7.8.3 Directional derivative

The **directional derivative** measures how much the function $f : \mathbb{R}^n \to \mathbb{R}$ changes along a direction $\mathbf{v}$ in space. It is defined as follows

$$D_{\mathbf{v}} f(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h} \tag{7.230}$$

We can approximate this numerically using 2 function calls to f, regardless of n. By contrast, a numerical approximation to the standard gradient vector takes $n + 1$ calls (or $2n$ if using central differences).

Note that the directional derivative along $\mathbf{v}$ is the scalar product of the gradient $\mathbf{g}$ and the vector $\mathbf{v}$:

$$D_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{v} \tag{7.231}$$

7.8.4 Total derivative *

Suppose that some of the arguments to the function depend on each other. Concretely, suppose the function has the form $f(t, x(t), y(t))$. We define the **total derivative** of f wrt t as follows:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt} \tag{7.232}$$

If we multiply both sides by the differential dt, we get the **total differential**

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy \tag{7.233}$$

This measures how much f changes when we change t, both via the direct effect of t on f, but also indirectly, via the effects of t on x and y.

7.8.5 Jacobian

Consider a function that maps a vector to another vector, $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$. The **Jacobian matrix** of this function is an $m \times n$ matrix of partial derivatives:

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}^\top} \triangleq \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{pmatrix} \tag{7.234}$$

Note that we lay out the results in the same orientation as the output $\mathbf{f}$; this is sometimes called numerator layout or the Jacobian formulation.[4]

4. For a much more detailed discussion of notation, see https://en.wikipedia.org/wiki/Matrix_calculus.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.8.5.1 Multiplying Jacobians and vectors

The **Jacobian vector product** or **JVP** is defined to be the operation that corresponds to right-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{J_f}(\mathbf{x})\mathbf{v} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{pmatrix} \mathbf{v} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \mathbf{v} \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \mathbf{v} \end{pmatrix} \tag{7.235}$$

So we can see that we can approximate this numerically using just 2 calls to $\mathbf{f}$.

The **vector Jacobian product** or **VJP** is defined to be the operation that corresponds to left-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\mathbf{u} \in \mathbb{R}^m$:

$$\mathbf{u}^\top \mathbf{J_f}(\mathbf{x}) = \mathbf{u}^\top \left(\frac{\partial \mathbf{f}}{\partial x_1}, \cdots, \frac{\partial \mathbf{f}}{\partial x_n} \right) = \left(\mathbf{u} \cdot \frac{\partial \mathbf{f}}{\partial x_1}, \cdots, \mathbf{u} \cdot \frac{\partial \mathbf{f}}{\partial x_n} \right) \tag{7.236}$$

The JVP is more efficient if $m \geq n$, and the VJP is more efficient if $m \leq n$. See Sec. 13.3 for details on how this can be used to perform automatic differentiation in a computation graph such as a DNN.

7.8.5.2 Jacobian of a composition

Sometimes it is useful to take the Jacobian of the composition of two functions. Let $h(\mathbf{x}) = g(f(\mathbf{x}))$. By the chain rule of calculus, we have

$$\mathbf{J}_h(\mathbf{x}) = \mathbf{J}_g(f(\mathbf{x}))\mathbf{J}_f(\mathbf{x}) \tag{7.237}$$

For example, suppose $f : \mathbb{R} \to \mathbb{R}^2$ and $g : \mathbb{R}^2 \to \mathbb{R}^2$. We have

$$\frac{\partial \mathbf{g}}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x} g_1(f_1(x), f_2(x)) \\ \frac{\partial}{\partial x} g_2(f_1(x), f_2(x)) \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial f_1}\frac{\partial f_1}{\partial x} + \frac{\partial g_1}{\partial f_2}\frac{\partial f_2}{\partial x} \\ \frac{\partial g_2}{\partial f_1}\frac{\partial f_1}{\partial x} + \frac{\partial g_2}{\partial f_2}\frac{\partial f_2}{\partial x} \end{pmatrix} \tag{7.238}$$

$$= \frac{\partial \mathbf{g}}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial x} = \begin{pmatrix} \frac{\partial g_1}{\partial f_1} & \frac{\partial g_1}{\partial f_2} \\ \frac{\partial g_2}{\partial f_1} & \frac{\partial g_2}{\partial f_2} \end{pmatrix} \begin{pmatrix} \frac{\partial f_1}{\partial x} \\ \frac{\partial f_2}{\partial x} \end{pmatrix} \tag{7.239}$$

7.8.6 Hessian

For a function $f : \mathbb{R}^n \to \mathbb{R}$ that is twice differentiable, we define the **Hessian matrix** as the (symmetric) $n \times n$ matrix of second partial derivatives:

$$\mathbf{H}_f = \frac{\partial^2 f}{\partial \mathbf{x}^2} = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ & \vdots & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix} \tag{7.240}$$

We see that the Hessian is the Jacobian of the gradient.

7.8.7 Gradients of commonly used functions

In this section, we list without proof the gradients of certain widely used functions.

7.8.7.1 Functions that map scalars to scalars

Consider a differentiable function $f : \mathbb{R} \to \mathbb{R}$. Here are some useful identities from scalar calculus, which you should already be familiar with.

$$\frac{d}{dx} cx^n = cnx^{n-1} \tag{7.241}$$

$$\frac{d}{dx} \log(x) = 1/x \tag{7.242}$$

$$\frac{d}{dx} \exp(x) = \exp(x) \tag{7.243}$$

$$\frac{d}{dx} [f(x) + g(x)] = \frac{df(x)}{dx} + \frac{dg(x)}{dx} \tag{7.244}$$

$$\frac{d}{dx} [f(x)g(x)] = f(x)\frac{dg(x)}{dx} + g(x)\frac{df(x)}{dx} \tag{7.245}$$

$$\frac{d}{dx} f(u(x)) = \frac{du}{dx}\frac{df(u)}{du} \tag{7.246}$$

Eq. (7.246) is known as the **chain rule of calculus**.

7.8.7.2 Functions that map vectors to scalars

Consider a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$. Here are some useful identities:[5]

$$\frac{\partial(\mathbf{a}^\top \mathbf{x})}{\partial \mathbf{x}} = \mathbf{a} \tag{7.247}$$

$$\frac{\partial(\mathbf{b}^\top \mathbf{A}\mathbf{x})}{\partial \mathbf{x}} = \mathbf{A}^\top \mathbf{b} \tag{7.248}$$

$$\frac{\partial(\mathbf{x}^\top \mathbf{A}\mathbf{x})}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x} \tag{7.249}$$

It is fairly easy to prove these identities by expanding out the quadratic form, and applying scalar calculus.

7.8.7.3 Functions that map matrices to scalars

Consider a function $f : \mathbb{R}^{m \times n} \to \mathbb{R}$ which maps a matrix to a scalar. We are using the following natural layout for the derivative matrix:

$$\frac{\partial f}{\partial \mathbf{X}} = \begin{pmatrix} \frac{\partial f}{\partial x_{11}} & \cdots & \frac{\partial f}{\partial x_{1n}} \\ & \vdots & \\ \frac{\partial f}{\partial x_{m1}} & \cdots & \frac{\partial f}{\partial x_{mn}} \end{pmatrix} \tag{7.250}$$

Below are some useful identities.

5. Some of the identities are taken from the list at http://www.cs.nyu.edu/~roweis/notes/matrixid.pdf.

Identities involving quadratic forms

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}(\mathbf{a}^\top \mathbf{X}\mathbf{b}) = \mathbf{a}\mathbf{b}^\top \tag{7.251}$$

$$\frac{\partial}{\partial \mathbf{X}}(\mathbf{a}^\top \mathbf{X}^\top \mathbf{b}) = \mathbf{b}\mathbf{a}^\top \tag{7.252}$$

Identities involving matrix trace

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}\text{tr}(\mathbf{A}\mathbf{X}\mathbf{B}) = \mathbf{A}^\top \mathbf{B}^\top \tag{7.253}$$

$$\frac{\partial}{\partial \mathbf{X}}\text{tr}(\mathbf{X}^\top \mathbf{A}) = \mathbf{A} \tag{7.254}$$

$$\frac{\partial}{\partial \mathbf{X}}\text{tr}(\mathbf{X}^{-1}\mathbf{A}) = -\mathbf{X}^{-\top}\mathbf{A}^\top \mathbf{X}^{-\top} \tag{7.255}$$

$$\frac{\partial}{\partial \mathbf{X}}\text{tr}(\mathbf{X}^\top \mathbf{A}\mathbf{X}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{X} \tag{7.256}$$

Identities involving matrix determinant

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}\det(\mathbf{A}\mathbf{X}\mathbf{B}) = \det(\mathbf{A}\mathbf{X}\mathbf{B})\mathbf{X}^{-\top} \tag{7.257}$$

$$\frac{\partial}{\partial \mathbf{X}}\ln(\det(\mathbf{X})) = \mathbf{X}^{-\top} \tag{7.258}$$

7.8.8 Functional derivative notation *

This section was written by Roy Frostig

The partial differentiation notation introduced in previous sections often relies on associating function arguments with names. For instance, given a function $f : \mathbb{R}^2 \to \mathbb{R}$, the partial derivative of f with respect to its first scalar argument, at a point $\mathbf{a} = (a_1, a_2)$, might be written:

$$\left.\frac{\partial f}{\partial x_1}\right|_{\mathbf{x}=\mathbf{a}} \tag{7.259}$$

This notation is not entirely self-contained. It refers to a name $\mathbf{x} = (x_1, x_2)$, implicit or inferred from context, suggesting the argument of f. An alternative expression is:

$$\frac{\partial}{\partial a_1} f(a_1, a_2) \tag{7.260}$$

where now a_1 serves both as an argument name (or a symbol in an expression) and as a particular evaluation point. Tracking names can become an increasingly complicated endeavor as we compose many functions together, each possibly taking several arguments.

A **functional** notation instead defines derivatives as operators on functions. If a function has multiple arguments, they are identified by position rather than by name, alleviating the need for auxiliary variable definitions. The following definitions draw on those in Spivak's *Calculus on Manifolds* [Spi71] and in Sussman and Wisdom's *Functional Differential Geometry* [SW13]. These and other texts are recommended for a more formal treatment, and for a more mathematically general view.

Here we will simply try to recover concepts from previous subsections and to generalize slightly. We also set up for Sec. 13.3.5, in which we consider algorithms that compute derivatives of functions common in machine learning models and numerical programs. These functions, namely, are equivalent to directed acyclic graphs (DAGs) over basic primitive operations—termed **computation graphs** elsewhere in this book.

The derivative operator. For a function $f : \mathbb{R}^n \to \mathbb{R}^m$, denote its **derivative function**:

$$\partial f : \mathbb{R}^n \to \mathbb{R}^{m \times n} . \tag{7.261}$$

The function ∂f maps a point $\mathbf{x} \in \mathbb{R}^n$ to the Jacobian matrix of all partial derivatives evaluated at $\mathbf{x}$. With this notation, we can recover the Jacobian as:

$$\mathbf{J}_f(\mathbf{x}) = \partial f(\mathbf{x}) \tag{7.262}$$

for all $\mathbf{x} \in \mathbb{R}^n$. In particular, when $m = 1$, we recover the gradient $\nabla f(\mathbf{x})^\top = \partial f(\mathbf{x})$. Indeed, Sec. 7.8.2 already notes that the notation ∇ can be viewed as an operator. When $n = m = 1$, the Jacobian is a scalar, and ∂f is the familiar derivative f'.

Any matrix $\mathbf{M}$ is associated with a linear map $\mathbf{v} \mapsto \mathbf{M}\mathbf{v}$. We can view the Jacobian more generally that way and say that the derivative function above maps a point to a linear function:

$$\partial f : \mathbb{R}^n \to \underbrace{(\mathbb{R}^n \to \mathbb{R}^m)}_{\text{linear}} . \tag{7.263}$$

We will sometimes refer to the argument to ∂f as the **linearization point** for the Jacobian.

Thinking about maps rather than matrices is somewhat optional, but can help us define higher-order derivatives, as we proceed to do below. It also suggests how Jacobians are commonly written in code. When we consider $\partial f(\mathbf{x})$ in a program for a fixed $\mathbf{x}$, we often implement it as a function that carries out multiplication by the Jacobian matrix, i.e. $\mathbf{v} \mapsto \partial f(\mathbf{x})\mathbf{v}$, instead of explicitly representing it as a matrix of numbers in memory. Going a step further, we often implement ∂f entirely as a single function over any linearization point $\mathbf{x}$ and vector $\mathbf{v}$, i.e. as $(\mathbf{x}, \mathbf{v}) \mapsto \partial f(\mathbf{x})\mathbf{v}$. As a toy example with scalars, consider the cosine:

$$(x, v) \mapsto \partial \cos(x)v = -v \sin(x) . \tag{7.264}$$

If we express this at once in code, we can, say, avoid computing $\sin(x)$ whenever $v = 0$.[6]

To take another derivative, we write:

$$\partial^2 f : \mathbb{R}^n \to \underbrace{(\mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}^m)}_{\text{bilinear}} \tag{7.265}$$

6. This example ignores that such an optimization might be done (best) by a compiler. Then again, for more complex examples, implementing $(\mathbf{x}, \mathbf{v}) \mapsto \partial f(\mathbf{x})\mathbf{v}$ as a single subroutine can help guide compiler optimizations all the same.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\partial^2 f(\mathbf{x})$ is a bilinear map, and hence corresponds to an array $\mathbb{R}^{m \times n \times n}$. Extending further:

$$\partial^k f : \mathbb{R}^n \to (\underbrace{\mathbb{R}^n \times \cdots \times \mathbb{R}^n}_{k \text{ times}} \to \mathbb{R}^m) \tag{7.266}$$

where $\partial^k f(\mathbf{x})$ is a multilinear or k-linear map, and corresponds to an array $\mathbb{R}^{m \times n \times \cdots \times n}$.

With $m = 1$, the Hessian is indeed $\mathbf{H}_f(\mathbf{x}) = \partial^2 f(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^n$. Still, even when $m > 1$, arbitrary higher-order derivatives can be written by composing the ∂ operator. Although Jacobians and Hessians suffice to make sense of many machine learning techniques, arbitrary higher-order derivatives can enter the picture as well—for example, when computing something as basic as a function's Taylor series expansion [Kel+20].

The rest of this section returns to the matrix view for Jacobians.

Multiple inputs. Now consider a function of two arguments:

$$g : \mathbb{R}^{n_1} \times \mathbb{R}^{n_2} \to \mathbb{R}^m \,. \tag{7.267}$$

For our purposes, a product domain such as $\mathbb{R}^{n_1} \times \mathbb{R}^{n_2}$ mainly serves to suggest a convenient partitioning of a function's input. It is equivalent to $\mathbb{R}^{n_1+n_2}$, a single input. The latter tells us how the derivative functions of g ought to look, based on previous definitions, and we will swap between the two views with little warning. Multiple inputs tend to arise, again, in the context of programs and DAGs: many functions in code are written to take in multiple arguments, and many basic operations (such as $+$) do the same.

In the context of multiple inputs, we can denote by $\partial_i g$ the derivative function with respect to the i'th argument:

$$\partial_1 g : \mathbb{R}^{n_1} \times \mathbb{R}^{n_2} \to \mathbb{R}^{m \times n_1} \,, \text{ and} \tag{7.268}$$

$$\partial_2 g : \mathbb{R}^{n_1} \times \mathbb{R}^{n_2} \to \mathbb{R}^{m \times n_2} \,. \tag{7.269}$$

The function $\partial_1 g$ maps a pair of points $\mathbf{x} \in \mathbb{R}^{n_1}$ and $\mathbf{y} \in \mathbb{R}^{n_2}$ to the matrix of all partial derivatives of g with respect to its first argument, evaluated at $(\mathbf{x}, \mathbf{y})$. We take ∂g with no subscript to simply mean the concatenation of $\partial_1 g$ and $\partial_2 g$:

$$\partial g : \mathbb{R}^{n_1} \times \mathbb{R}^{n_2} \to \mathbb{R}^{m \times (n_1+n_2)} \,, \text{ where} \tag{7.270}$$

$$\partial g(\mathbf{x}, \mathbf{y}) = \begin{pmatrix} \partial_1 g(\mathbf{x}, \mathbf{y}) & \partial_2 g(\mathbf{x}, \mathbf{y}) \end{pmatrix} \,. \tag{7.271}$$

This convention will simplify our chain rule statement below. It also recovers the Jacobian $\mathbf{J}_g = \partial g$ as before. When $n_1 = n_2 = m = 1$, both sub-matrices are scalar, and $\partial g_1(x, y)$ recovers the partial derivative that might otherwise be written:

$$\frac{\partial}{\partial x} g(x, y) \,. \tag{7.272}$$

However, the expression ∂g_1 bears a meaning on its own (as a function) whereas the expression $\frac{\partial g}{\partial x}$ may be ambiguous without further context. Again composing operators lets us write higher-order derivatives. For instance, $\partial_2 \partial_1 g(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{m \times n_1 \times n_2}$, and if $m = 1$:

$$\mathbf{H}_g(\mathbf{x}, \mathbf{y}) = \begin{pmatrix} \partial_1 \partial_1 g(\mathbf{x}, \mathbf{y}) & \partial_1 \partial_2 g(\mathbf{x}, \mathbf{y}) \\ \partial_2 \partial_1 g(\mathbf{x}, \mathbf{y}) & \partial_2 \partial_2 g(\mathbf{x}, \mathbf{y}) \end{pmatrix} \,. \tag{7.273}$$

Composition and fan-out. If $f = g \circ h$ for some $h : \mathbb{R}^n \to \mathbb{R}^p$ and $g : \mathbb{R}^p \to \mathbb{R}^m$, then the chain rule is:

$$\partial f(\mathbf{x}) = \partial g(h(\mathbf{x}))\, \partial h(\mathbf{x}) \text{ for all } \mathbf{x} \in \mathbb{R}^n . \tag{7.274}$$

How does this interact with our notation for multi-argument functions? For one, it can lead us to consider expressions with **fan-out**, where several sub-expressions are functions of the same input. For instance, assume two functions $a : \mathbb{R}^n \to \mathbb{R}^{m_1}$ and $b : \mathbb{R}^n \to \mathbb{R}^{m_2}$, and that:

$$f(\mathbf{x}) = g(a(\mathbf{x}), b(\mathbf{x})) \tag{7.275}$$

for some function g. Abbreviating $h(\mathbf{x}) = (a(\mathbf{x}), b(\mathbf{x}))$ so that $f(\mathbf{x}) = g(h(\mathbf{x}))$, Eq. (7.274) tells us that:

$$\partial f(\mathbf{x}) = \partial g(h(\mathbf{x}))\, \partial h(\mathbf{x}) \tag{7.276}$$

$$= \partial_1 g(a(\mathbf{x}), b(\mathbf{x}))\, \partial a(\mathbf{x}) + \partial_2 g(a(\mathbf{x}), b(\mathbf{x}))\, \partial b(\mathbf{x}) . \tag{7.277}$$

It also follows from the above that if:

$$f(\mathbf{x}, \mathbf{y}) = g(u(\mathbf{x}), v(\mathbf{y})) \tag{7.278}$$

for some functions u and v—in other words, if we write multiple arguments but exhibit no fan-out—then:

$$\partial_1 f(\mathbf{x}, \mathbf{y}) = \partial_1 g(u(\mathbf{x}), v(\mathbf{y}))\, \partial u(\mathbf{x}) , \text{ and} \tag{7.279}$$

$$\partial_2 f(\mathbf{x}, \mathbf{y}) = \partial_2 g(u(\mathbf{x}), v(\mathbf{y}))\, \partial v(\mathbf{y}) . \tag{7.280}$$

Composition and fan-out rules for derivatives are what let us break down a complex derivative calculation into simpler ones. This is what automatic differentiation techniques rely on when tackling the large computations that accompany modern machine learning models (the topic of Sec. 13.3.5).

7.9 Exercises

Exercise 7.1 [Orthogonal matrices]

a. A rotation in 3d by angle α about the z axis is given by the following matrix:

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.281}$$

Prove that $\mathbf{R}$ is an orthogonal matrix, i.e., $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, for any α.

b. What is the only eigenvector $\mathbf{v}$ of $\mathbf{R}$ with an eigenvalue of 1.0 and of unit norm (i.e., $||\mathbf{v}||^2 = 1$)? (Your answer should be the same for any α.) Hint: think about the geometrical interpretation of eigenvectors.

Exercise 7.2 [Eigenvectors by hand]

Find the eigenvalues and eigenvectors of the following matrix

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \tag{7.282}$$

Compute your result by hand and check it with Python.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8 Optimization

Parts of this chapter were written by Frederik Kunstner, Si Yi Meng, Aaron Mishkin, Sharan Vaswani, and Mark Schmidt.

8.1 Introduction

We saw in Chapter 4 that the core problem in machine learning is parameter estimation (aka model fitting). This requires solving an **optimization problem**, where we try to find the values for a set of variables $\boldsymbol{\theta} \in \Theta$, that minimize a scalar-valued **loss function** or **cost function** $\mathcal{L} : \Theta \to \mathbb{R}$:

$$\boldsymbol{\theta}^* \in \operatorname*{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta}) \tag{8.1}$$

We will assume that the **parameter space** is given by $\Theta \subseteq \mathbb{R}^D$, where D is the number of variables being optimized over. Thus we are focusing on **continuous optimization**, rather than **discrete optimization**.

If we want to *maximize* a **score function** or **reward function** $R(\boldsymbol{\theta})$, we can equivalently minimize $\mathcal{L}(\boldsymbol{\theta}) = -R(\boldsymbol{\theta})$. We will use the term **objective function** to refer generically to a function we want to maximize or minimize. An algorithm that can find an optimum of an objective function is often called a **solver**.

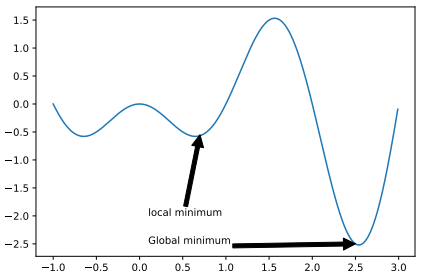
In the rest of this chapter, we discuss different kinds of solvers for different kinds of objective functions, with a focus on methods used in the machine learning community. For more details on optimization, please consult some of the many excellent textbooks, such as [KW19b; BV04; NW06; Ber15; Ber16] as well as various review articles, such as [BCN18; Sun+19b; PPS18].

8.1.1 Local vs global optimization

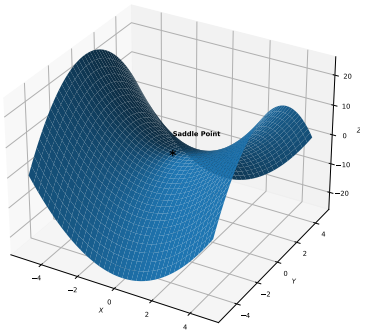
A point that satisfies Eq. (8.1) is called a **global optimum**. Finding such a point is called **global optimization**.

In general, finding global optima is computationally intractable [Neu04]. In such cases, we will just try to find a **local optimum**. For continuous problems, this is defined to be a point $\boldsymbol{\theta}^*$ which has lower (or equal) cost than "nearby" points. Formally, we say $\boldsymbol{\theta}^*$ is a **local minimum** if

$$\exists \delta > 0, \ \forall \boldsymbol{\theta} \in \Theta \ \text{ s.t. } \ ||\boldsymbol{\theta} - \boldsymbol{\theta}^*|| < \delta, \ \mathcal{L}(\boldsymbol{\theta}^*) \leq \mathcal{L}(\boldsymbol{\theta}) \tag{8.2}$$



(a)



(b)

Figure 8.1: (a) Illustration of local and global minimum in 1d. Generated by code at figures.probml.ai/book1/8.1. (b) Illustration of a saddle point in 2d. Generated by code at figures.probml.ai/book1/8.1.

A local minimum could be surrounded by other local minima with the same objective value; this is known as a **flat local minimum**. A point is said to be a **strict local minimum** if its cost is strictly lower than those of neighboring points:

$$\exists \delta > 0,\ \forall \boldsymbol{\theta} \in \Theta, \boldsymbol{\theta} \neq \boldsymbol{\theta}^* : ||\boldsymbol{\theta} - \boldsymbol{\theta}^*|| < \delta,\ \mathcal{L}(\boldsymbol{\theta}^*) < \mathcal{L}(\boldsymbol{\theta}) \tag{8.3}$$

We can define a (strict) **local maximum** analogously. See Fig. 8.1a for an illustration.

A final note on terminology; if an algorithm is guaranteed to converge to a stationary point from any starting point, it is called **globally convergent**. However, this does not mean (rather confusingly) that it will converge to a global optimum; instead, it just means it will converge to some stationary point.

8.1.1.1 Optimality conditions for local vs global optima

For continuous, twice differentiable functions, we can precisely characterize the points which correspond to local minima. Let $\mathbf{g}(\boldsymbol{\theta}) = \nabla \mathcal{L}(\boldsymbol{\theta})$ be the gradient vector, and $\mathbf{H}(\boldsymbol{\theta}) = \nabla^2 \mathcal{L}(\boldsymbol{\theta})$ be the Hessian matrix. (See Sec. 7.8 for a refresher on these concepts, if necessary.) Consider a point $\boldsymbol{\theta}^* \in \mathbb{R}^D$, and let $\mathbf{g}^* = \mathbf{g}(\boldsymbol{\theta})|_{\boldsymbol{\theta}^*}$ be the gradient at that point, and $\mathbf{H}^* = \mathbf{H}(\boldsymbol{\theta})|_{\boldsymbol{\theta}^*}$ be the corresponding Hessian. One can show that the following conditions characterize every local minimum:

- Necessary condition: If $\boldsymbol{\theta}^*$ is a local minimum, then we must have $\mathbf{g}^* = \mathbf{0}$ (i.e., $\boldsymbol{\theta}^*$ must be a **stationary point**), and $\mathbf{H}^*$ must be positive semi-definite.
- Sufficient condition: If $\mathbf{g}^* = \mathbf{0}$ and $\mathbf{H}^*$ is positive definite, then $\boldsymbol{\theta}^*$ is a local optimum.

To see why the first condition is necessary, suppose we were at a point $\boldsymbol{\theta}^*$ at which the gradient is non-zero: at such a point, we could decrease the function by following the negative gradient a small distance, so this would not be optimal. So the gradient must be zero. (In the case of nonsmooth

functions, the necessary condition is that the zero is a local subgradient at the minimum.) To see why a zero gradient is not sufficient, note that the stationary point could be a local minimum, maximum or **saddle point**, which is a point where some directions point downhill, and some uphill (see Fig. 8.1b). More precisely, at a saddle point, the eigenvalues of the Hessian will be both positive and negative. However, if the Hessian at a point is positive semi-definite, then some directions may point uphill, while others are flat. This means that the objective function can not be decreased in the neighbourhood of this point, implying that it is necessarily a local minimum. Moreover, if the Hessian is strictly positive definite, then we are at the bottom of a "bowl", and all directions point uphill, which is sufficient for this to be a minimum.

8.1.2 Constrained vs unconstrained optimization

In **unconstrained optimization**, we define the optimization task as finding any value in the parameter space Θ that minimizes the loss. However, we often have a set of **constraints** on the allowable values. It is standard to partition the set of constraints $\mathcal{C}$ into **inequality constraints**, $g_j(\boldsymbol{\theta}) \leq 0$ for $j \in \mathcal{I}$ and **equality constraints**, $h_k(\boldsymbol{\theta}) = 0$ for $k \in \mathcal{E}$. For example, we can represent a sum-to-one constraint as an equality constraint $h(\boldsymbol{\theta}) = (1 - \sum_{i=1}^{D} \theta_i) = 0$, and we can represent a non-negativity constraint on the parameters by using D inequality constraints of the form $g_i(\boldsymbol{\theta}) = -\theta_i \leq 0$.

We define the **feasible set** as the subset of the parameter space that satisfies the constraints:

$$\mathcal{C} = \{\boldsymbol{\theta} : g_j(\boldsymbol{\theta}) \leq 0 : j \in \mathcal{I}, h_k(\boldsymbol{\theta}) = 0 : k \in \mathcal{E}\} \subseteq \mathbb{R}^D \tag{8.4}$$

Our **constrained optimization** problem now becomes

$$\boldsymbol{\theta}^* \in \operatorname*{argmin}_{\boldsymbol{\theta} \in \mathcal{C}} \mathcal{L}(\boldsymbol{\theta}) \tag{8.5}$$

If $\mathcal{C} = \mathbb{R}^D$, it is called **unconstrained optimization**.

The addition of constraints can change the number of optima of a function. For example, a function that was previously unbounded (and hence had no well-defined global maximum or minimum) can "acquire" multiple maxima or minima when we add constraints, as illustrated in Fig. 8.2. However, if we add too many constraints, we may find that the feasible set becomes empty. The task of finding any point (regardless of its cost) in the feasible set is called a **feasibility problem**; this can be a hard subproblem in itself.

A common strategy for solving constrained problems is to create penalty terms that measure how much we violate each constraint. We then add these terms to the objective and solve an unconstrained optimization problem. The **Lagrangian** is a special case of such a combined objective (see Sec. 8.5 for details).

8.1.3 Convex vs nonconvex optimization

In **convex optimization**, we require the objective to be a convex function defined over a convex set (see Sec. 8.1.3 for definitions of these terms). In such problems, every local minimum is also a global minimum. Thus many models are designed so that their training objectives are convex.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

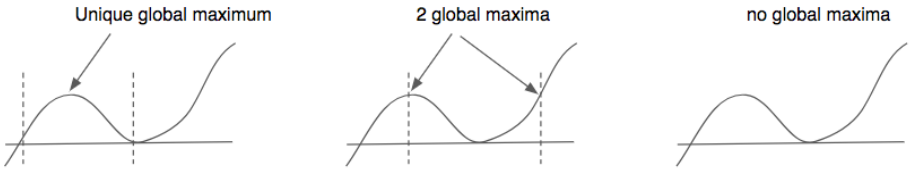


Figure 8.2: Illustration of constrained maximization of a nonconvex 1d function. The area between the dotted vertical lines represents the feasible set. (a) There is a unique global maximum since the function is concave within the support of the feasible set. (b) There are two global maxima, both occuring at the boundary of the feasible set. (c) In the unconstrained case, this function has no global maximum, since it is unbounded.

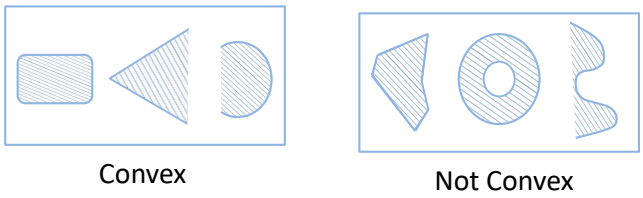


Figure 8.3: Illustration of some convex and non-convex sets.

8.1.3.1 Convex sets

We say $\mathcal{S}$ is a **convex set** if, for any $\mathbf{x}, \mathbf{x}' \in \mathcal{S}$, we have

$$\lambda \mathbf{x} + (1-\lambda)\mathbf{x}' \in \mathcal{S}, \quad \forall\, \lambda \in [0,1] \tag{8.6}$$

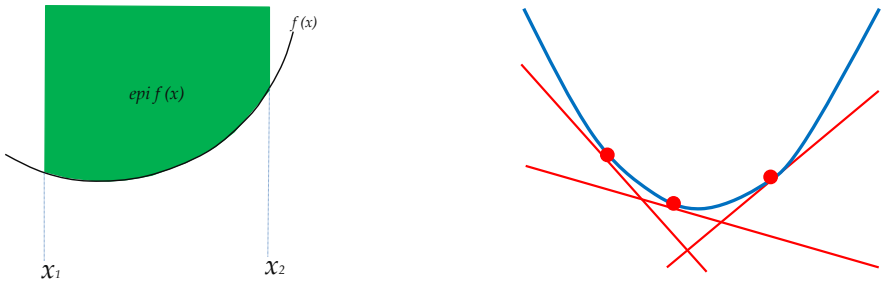
That is, if we draw a line from $\mathbf{x}$ to $\mathbf{x}'$, all points on the line lie inside the set. See Fig. 8.3 for some illustrations of convex and non-convex sets.

8.1.3.2 Convex functions

We say f is a **convex function** if its **epigraph** (the set of points above the function, illustrated in Fig. 8.4a) defines a convex set. Equivalently, a function $f(\mathbf{x})$ is called convex if it is defined on a convex set and if, for any $\mathbf{x}, \mathbf{y} \in \mathcal{S}$, and for any $0 \leq \lambda \leq 1$, we have

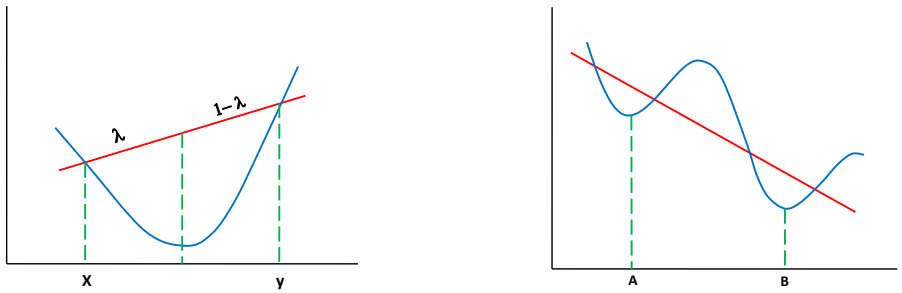
$$f(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}) \tag{8.7}$$

See Fig. 8.5(a) for a 1d example of a convex function. A function is called **strictly convex** if the inequality is strict. A function $f(\mathbf{x})$ is **concave** if $-f(\mathbf{x})$ is convex, and **strictly concave** if $-f(\mathbf{x})$ is strictly convex. See Fig. 8.5(b) for a 1d example of a function that is neither convex nor concave.



(a) (b)

Figure 8.4: (a) Illustration of the epigraph of a function. (b) For a convex function $f(x)$, its epipgraph can be represented as the intersection of half-spaces defined by linear lower bounds derived from the **conjugate function** $f^*(\lambda) = \max_x \lambda x - f(x)$.



(a) (b)

Figure 8.5: (a) Illustration of a convex function. We see that the chord joining $(x, f(x))$ to $(y, f(y))$ lies above the function. (b) A function that is neither convex nor concave. **A** *is a local minimum,* **B** *is a global minimum.*

Here are some examples of 1d convex functions:

$$\begin{aligned} x^2 \\ e^{ax} \\ -\log x \\ x^a,\ a > 1, x > 0 \\ |x|^a,\ a \geq 1 \\ x \log x,\ x > 0 \end{aligned}$$

8.1.3.3 Characterization of convex functions

Intuitively, a convex function is shaped like a bowl. Formally, one can prove the following important result:

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

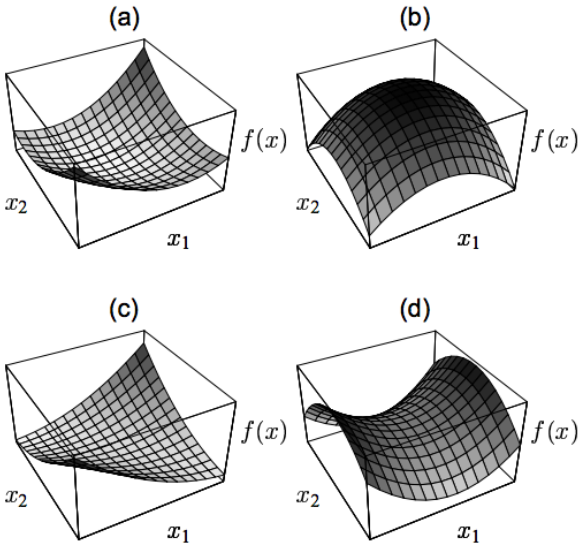


Figure 8.6: The quadratic form $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$ in 2d. (a) $\mathbf{A}$ is positive definite, so f is convex. (b) $\mathbf{A}$ is negative definite, so f is concave. (c) $\mathbf{A}$ is positive semidefinite but singular, so f is convex, but not strictly. Notice the valley of constant height in the middle. (d) $\mathbf{A}$ is indefinite, so f is neither convex nor concave. The stationary point in the middle of the surface is a saddle point. From Figure 5 of [She94].

Theorem 8.1.1. *Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is twice differentiable over its domain. Then f is convex iff $\mathbf{H} = \nabla^2 f(\mathbf{x})$ is positive semi definite (Sec. 7.1.5.3) for all $\mathbf{x} \in \text{dom}(f)$. Furthermore, f is strictly convex if $\mathbf{H}$ is positive definite.*

For example, consider the quadratic form

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x} \tag{8.8}$$

This is convex if $\mathbf{A}$ is positive semi definite, and is strictly convex if $\mathbf{A}$ is positive definite. It is neither convex nor concave if $\mathbf{A}$ has eigenvalues of mixed sign. See Fig. 8.6.

8.1.3.4 Strongly convex functions

We say a function f is **strongly convex** with parameter $m > 0$ if the following holds for all $\mathbf{x}$, $\mathbf{y}$ in f's domain:

$$(\nabla f(\mathbf{x}) - \nabla f(\mathbf{y}))^\top (\mathbf{x} - \mathbf{y}) \geq m||\mathbf{x} - \mathbf{y}||_2^2 \tag{8.9}$$

A strongly convex function is also strictly convex, but not vice versa.

If the function f is twice continuously differentiable, then it is strongly convex with parameter m if and only if $\nabla^2 f(\mathbf{x}) \succeq m\mathbf{I}$ for all $\mathbf{x}$ in the domain, where $\mathbf{I}$ is the identity and $\nabla^2 f$ is the Hessian matrix, and the inequality $\succeq$ means that $\nabla^2 f(\mathbf{x}) - m\mathbf{I}$ is positive semi-definite. This is equivalent

(a) (b)

Figure 8.7: (a) Smooth 1d function. (b) Non-smooth 1d function. (There is a discontinuity at the origin.) From https://scipy-lectures.org/advanced/mathematical_optimization/index.html#smooth-and-non-smooth-problems. Used with kind permission of Gael Varoquaux

to requiring that the minimum eigenvalue of $\nabla^2 f(\mathbf{x})$ be at least m for all $\mathbf{x}$. If the domain is just the real line, then $\nabla^2 f(x)$ is just the second derivative $f''(x)$, so the condition becomes $f''(x) \geq m$. If $m = 0$, then this means the Hessian is positive semidefinite (or if the domain is the real line, it means that $f''(x) \geq 0$), which implies the function is convex, and perhaps strictly convex, but not strongly convex.

The distinction between convex, strictly convex, and strongly convex is rather subtle. To better understand this, consider the case where f is twice continuously differentiable and the domain is the real line. Then we can characterize the differences as follows:

- f is convex if and only if $f''(x) \geq 0$ for all x.
- f is strictly convex if $f''(x) > 0$ for all x (note: this is sufficient, but not necessary).
- f is strongly convex if and only if $f''(x) \geq m > 0$ for all x.

Note that it can be shown that a function f is strongly convex with parameter m iff the function

$$J(\mathbf{x}) = f(\mathbf{x}) - \frac{m}{2}||\mathbf{x}||^2 \tag{8.10}$$

is convex.

8.1.4 Smooth vs nonsmooth optimization

In **smooth optimization**, the objective and constraints are continuously differentiable functions. For smooth functions, we can quantify the degree of smoothness using the **Lipschitz constant**. In the 1d case, this is defined as any constant $L \geq 0$ such that, for all real x_1 and x_2, we have

$$|f(x_1) - f(x_2)| \leq L|x_1 - x_2| \tag{8.11}$$

This is illustrated in Fig. 8.8: for a given constant L, the function output cannot change by more than L if we change the function input by 1 unit. This can be generalized to vector inputs using a suitable norm.

In **nonsmooth optimization**, there are at least some points where the gradient of the objective function or the constraints is not well-defined. See Fig. 8.7 for an example. In some optimization

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

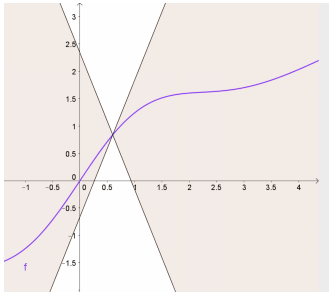


Figure 8.8: For a Lipschitz continuous function f, there exists a double cone (white) whose origin can be moved along the graph of f so that the whole graph always stays outside the double cone. From https://en.wikipedia.org/wiki/Lipschitz_continuity. Used with kind permission of Wikipedia author Taschee.

problems, we can partition the objective into a part that only contains smooth terms, and a part that contains the nonsmooth terms:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.12}$$

where $\mathcal{L}_s$ is smooth (differentiable), and $\mathcal{L}_r$ is nonsmooth ("rough"). This is often referred to as a **composite objective**. In machine learning applications, $\mathcal{L}_s$ is usually the training set loss, and $\mathcal{L}_r$ is a regularizer, such as the ℓ_1 norm of $\boldsymbol{\theta}$. This composite structure can be exploited by various algorithms.

8.1.4.1 Subgradients

In this section, we generalize the notion of a derivative to work with functions which have local discontinuities. In particular, for a convex function of several variables, $f : \mathbb{R}^n \to \mathbb{R}$, we say that $\mathbf{g} \in \mathbb{R}^n$ is a **subgradient** of f at $\mathbf{x} \in \text{dom}(f)$ if for all vectors $\mathbf{z} \in \text{dom}(f)$,

$$f(\mathbf{z}) \geq f(\mathbf{x}) + \mathbf{g}^\top(\mathbf{z} - \mathbf{x}) \tag{8.13}$$

Note that a subgradient can exist even when f is not differentiable at a point, as shown in Fig. 8.9.

A function f is called **subdifferentiable** at $\mathbf{x}$ if there is at least one subgradient at $\mathbf{x}$. The set of such subgradients is called the **subdifferential** of f at $\mathbf{x}$, and is denoted $\partial f(\mathbf{x})$.

For example, consider the absolute value function $f(x) = |x|$. Its subdifferential is given by

$$\partial f(x) = \begin{cases} \{-1\} & \text{if } x < 0 \\ [-1, 1] & \text{if } x = 0 \\ \{+1\} & \text{if } x > 0 \end{cases} \tag{8.14}$$

where the notation $[-1, 1]$ means any value between -1 and 1 inclusive. See Fig. 8.10 for an illustration.

8.2 First-order methods

In this section, we consider iterative optimization methods that leverage **first order** derivatives of the objective function, i.e., they compute which directions point "downhill", but they ignore curvature

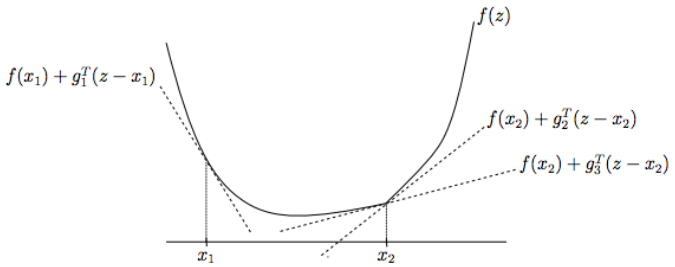


Figure 8.9: Illustration of subgradients. At $\mathbf{x}_1$, the convex function f is differentiable, and $\mathbf{g}_1$ (which is the derivative of f at $\mathbf{x}_1$) is the unique subgradient at $\mathbf{x}_1$. At the point $\mathbf{x}_2$, f is not differentiable, because of the "kink". However, there are many subgradients at this point, of which two are shown. From Figure 1 of [BD17]. Used with kind permission of Stephen Boyd.

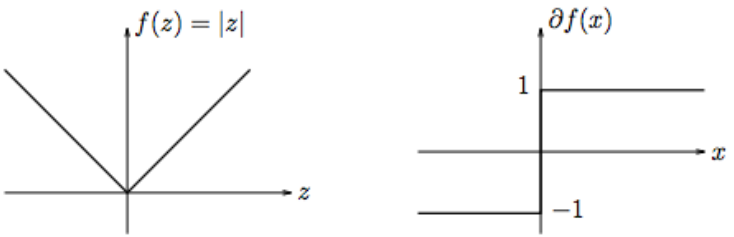


Figure 8.10: The absolute value function (left) and its subdifferential (right). From Figure 3 of [BD17]. Used with kind permission of Stephen Boyd.

information. All of these algorithms require that the user specify a starting point $\boldsymbol{\theta}_0$. Then at each iteration t, they perform an update of the following form:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \eta_t \mathbf{d}_t \tag{8.15}$$

where η_t is known as the **step size** or **learning rate**, and $\mathbf{d}_t$ is a **descent direction**, such as the negative of the **gradient**, given by $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$. These update steps are continued until the method reaches a stationary point, where the gradient is zero.

8.2.1 Descent direction

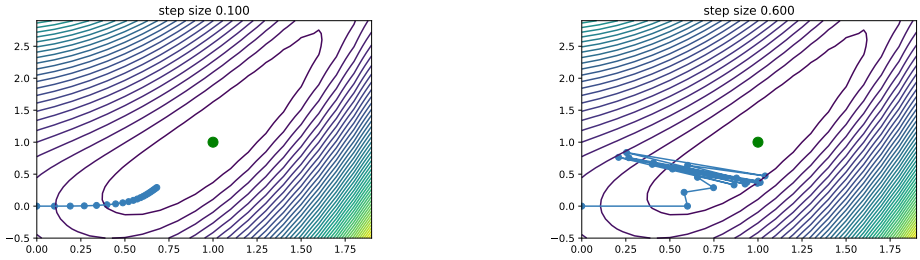
We say that a direction $\mathbf{d}$ is a **descent direction** if there is a small enough (but nonzero) amount η we can move in direction $\mathbf{d}$ and be guaranteed to decrease the function value. Formally, we require that there exists an $\eta_{\max} > 0$ such that

$$\mathcal{L}(\boldsymbol{\theta} + \eta \mathbf{d}) < \mathcal{L}(\boldsymbol{\theta}) \tag{8.16}$$

for all $0 < \eta < \eta_{\max}$. The gradient at the current iterate,

$$\mathbf{g}_t \triangleq \nabla \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t} = \nabla \mathcal{L}(\boldsymbol{\theta}_t) = \mathbf{g}(\boldsymbol{\theta}_t) \tag{8.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.11: Steepest descent on a simple convex function, starting from $(0, 0)$*, for 20 steps, using a fixed step size. The global minimum is at* $(1, 1)$*. (a)* $\eta = 0.1$*. (b)* $\eta = 0.6$*. Generated by code at figures.probml.ai/book1/8.11.*

points in the direction of maximal increase in f, so the negative gradient is a descent direction. It can be shown that any direction $\mathbf{d}$ is also a descent direction if the angle θ between $\mathbf{d}$ and $-\mathbf{g}_t$ is less than 90 degrees and satisfies

$$\mathbf{d}^\top \mathbf{g}_t = ||\mathbf{d}||\ ||\mathbf{g}_t||\ \cos(\theta) < 0 \tag{8.18}$$

It seems that the best choice would be to pick $\mathbf{d}_t = -\mathbf{g}_t$. This is known as the direction of **steepest descent**. However, this can be quite slow. We consider faster versions later.

8.2.2 Step size (learning rate)

In machine learning, the sequence of step sizes $\{\eta_t\}$ is called the **learning rate schedule**. There are several widely used methods for picking this, some of which we discuss below. (See also Sec. 8.4.3, where we discuss schedules for stochastic optimization.)

8.2.2.1 Constant step size

The simplest method is to use a constant step size, $\eta_t = \eta$. However, if it is too large, the method may fail to converge, and if it is too small, the method will converge but very slowly.

For example, consider the convex function

$$\mathcal{L}(\boldsymbol{\theta}) = 0.5(\theta_1^2 - \theta_2)^2 + 0.5(\theta_1 - 1)^2 \tag{8.19}$$

Let us pick as our descent direction $\mathbf{d}_t = -\mathbf{g}_t$. Fig. 8.11 shows what happens if we use this descent direction with a fixed step size, starting from $(0, 0)$. In Fig. 8.11(a), we use a small step size of $\eta = 0.1$; we see that the iterates move slowly along the valley. In Fig. 8.11(b), we use a larger step size $\eta = 0.6$; we see that the iterates start oscillating up and down the sides of the valley and never converge to the optimum, even though this is a convex problem.

In some cases, we can derive a theoretical upper bound on the maximum step size we can use. For example, consider a quadratic objective, $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} + \mathbf{b}^\top \boldsymbol{\theta} + c$ with $\mathbf{A} \succeq \mathbf{0}$. One can show that

steepest descent will have global convergence iff the step size satisfies

$$\eta < \frac{2}{\lambda_{\max}(\mathbf{A})} \tag{8.20}$$

where $\lambda_{\max}(\mathbf{A})$ is the largest eigenvalue of $\mathbf{A}$. The intuitive reason for this can be understood by thinking of a ball rolling down a valley. We want to make sure it doesn't take a step that is larger than the slope of the steepest direction, which is what the largest eigenvalue measures (see Sec. 3.2.2).

More generally, setting $\eta < 2/L$, where L is the Lipschitz constant of the gradient (Sec. 8.1.4), ensures convergence. Since this constant is generally unknown, we usually need to adapt the step size, as we discuss below.

8.2.2.2 Line search

The optimal step size can be found by finding the value that maximally decreases the objective along the chosen direction by solving the 1d minimization problem

$$\eta_t = \operatorname*{argmin}_{\eta>0} \phi_t(\eta) = \operatorname*{argmin}_{\eta>0} \mathcal{L}(\boldsymbol{\theta}_t + \eta \mathbf{d}_t) \tag{8.21}$$

This is known as **line search**, since we are searching along the line defined by $\mathbf{d}_t$.

If the loss is convex, this subproblem is also convex, because $\phi_t(\eta) = \mathcal{L}(\boldsymbol{\theta}_t + \eta \mathbf{d}_t)$ is a convex function of an affine function of η, for fixed $\boldsymbol{\theta}_t$ and $\mathbf{d}_t$. For example, consider the quadratic loss

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} + \mathbf{b}^\top \boldsymbol{\theta} + c \tag{8.22}$$

Computing the derivative of ϕ gives

$$\frac{d\phi(\eta)}{d\eta} = \frac{d}{d\eta}\left[\frac{1}{2}(\boldsymbol{\theta} + \eta\mathbf{d})^\top \mathbf{A}(\boldsymbol{\theta} + \eta\mathbf{d}) + \mathbf{b}^\top(\boldsymbol{\theta} + \eta\mathbf{d}) + c\right] \tag{8.23}$$

$$= \mathbf{d}^\top \mathbf{A}(\boldsymbol{\theta} + \eta\mathbf{d}) + \mathbf{d}^\top \mathbf{b} \tag{8.24}$$

$$= \mathbf{d}^\top(\mathbf{A}\boldsymbol{\theta} + \mathbf{b}) + \eta \mathbf{d}^\top \mathbf{A}\mathbf{d} \tag{8.25}$$

Solving for $\frac{d\phi(\eta)}{d\eta} = 0$ gives

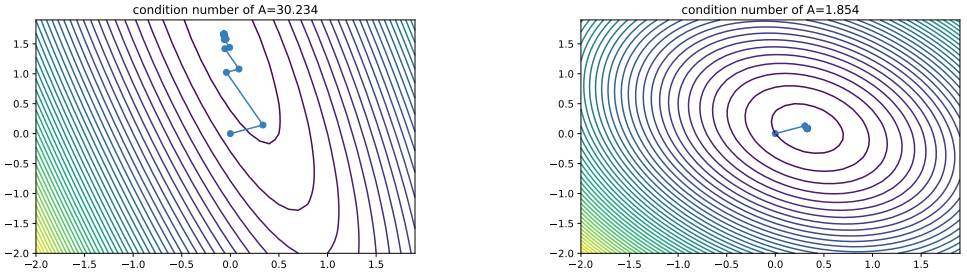
$$\eta = -\frac{\mathbf{d}^\top(\mathbf{A}\boldsymbol{\theta} + \mathbf{b})}{\mathbf{d}^\top \mathbf{A}\mathbf{d}} \tag{8.26}$$

Using the optimal step size is known as **exact line search**. However, it is not usually necessary to be so precise. There are several methods, such as the **Armijo backtracking method**, that try to ensure sufficient reduction in the objective function without spending too much time trying to solve Eq. (8.21). In particular, we can start with the current stepsize (or some maximum value), and then reduce it by a factor $0 < \beta < 1$ at each step until we satisfy the following condition, known as the **Armijo-Goldstein** test:

$$\mathcal{L}(\boldsymbol{\theta}_t + \eta\mathbf{d}_t) \leq \mathcal{L}(\boldsymbol{\theta}_t) + c\eta \mathbf{d}_t^\top \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{8.27}$$

where $c \in [0, 1]$ is a constant, typically $c = 10^{-4}$. In practice, the initialization of the line-search and how to backtrack can significantly affect performance. See [NW06, Sec 3.1] for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.12: Illustration of the effect of condition number κ on the convergence speed of steepest descent with exact line searches. (a) Large κ. (b) Small κ. Generated by code at figures.probml.ai/book1/8.12.

8.2.3 Convergence rates

We want to find optimization algorithms that converge quickly to a (local) optimum. For certain convex problems, with a gradient with bounded Lipschitz constant, one can show that gradient descent converges at a **linear rate**. This means that there exists a number $0 < \mu < 1$ such that

$$|\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)| \leq \mu |\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)| \tag{8.28}$$

Here μ is called the **rate of convergence**.

For some simple problems, we can derive the convergence rate explicitly, For example, consider a quadratic objective $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} + \mathbf{b}^\top \boldsymbol{\theta} + c$ with $\mathbf{A} \succ 0$. Suppose we use steepest descent with exact line search. One can show (see e.g., [Ber15]) that the convergence rate is given by

$$\mu = \left(\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \right)^2 \tag{8.29}$$

where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{A}$ and $\lambda_{\min}$ is the smallest eigenvalue. We can rewrite this as $\mu = \left(\frac{\kappa-1}{\kappa+1}\right)^2$, where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number of $\mathbf{A}$. Intuitively, the condition number measures how "skewed" the space is, in the sense of being far from a symmetrical "bowl". (See Sec. 7.1.4.4 for more information on condition numbers.)

Fig. 8.12 illustrates the effect of the condition number on the convergence rate. On the left we show an example where $\mathbf{A} = [20, 5; 5, 2]$, $\mathbf{b} = [-14; -6]$ and $c = 10$, so $\kappa(\mathbf{A}) = 30.234$. On the right we show an example where $\mathbf{A} = [20, 5; 5, 16]$, $\mathbf{b} = [-14; -6]$ and $c = 10$, so $\kappa(\mathbf{A}) = 1.8541$. We see that steepest descent converges much more quickly for the problem with the smaller condition number.

In the more general case of non-quadratic functions, the objective will often be locally quadratic around a local optimum. Hence the convergence rate depends on the condition number of the Hessian, $\kappa(\mathbf{H})$, at that point. We can often improve the convergence speed by optimizing a surrogate objective (or model) at each step which has a Hessian that is close to the Hessian of the objective function as we discuss in Sec. 8.3.

Although line search works well, we see from Fig. 8.12 that the path of steepest descent with an exact line-search exhibits a characteristic **zig-zag** behavior, which is inefficient. This problem can be overcome using a method called **conjugate gradient** descent (see e.g., [She94]).

8.2.4 Momentum methods

Gradient descent can move very slowly along flat regions of the loss landscape, as we illustrated in Fig. 8.11. We discuss some solutions to this below.

8.2.4.1 Momentum

One simple heuristic, known as the **heavy ball** or **momentum** method [Ber99], is to move faster along directions that were previously good, and to slow down along directions where the gradient has suddenly changed, just like a ball rolling downhill. This can be implemented as follows:

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_{t-1} \tag{8.30}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \mathbf{m}_t \tag{8.31}$$

where $\mathbf{m}_t$ is the momentum (mass times velocity) and $0 < \beta < 1$. A typical value of β is 0.9. For $\beta = 0$, the method reduces to gradient descent.

We see that $\mathbf{m}_t$ is like an exponentially weighted moving average of the past gradients (see Sec. 4.4.2.2):

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_{t-1} = \beta^2 \mathbf{m}_{t-2} + \beta \mathbf{g}_{t-2} + \mathbf{g}_{t-1} = \cdots = \sum_{\tau=0}^{t-1} \beta^\tau \mathbf{g}_{t-\tau-1} \tag{8.32}$$

If all the past gradients are a constant, say $\mathbf{g}$, this simplifies to

$$\mathbf{m}_t = \mathbf{g} \sum_{\tau=0}^{t-1} \beta^\tau \tag{8.33}$$

The scaling factor is a geometric series, whose infinite sum is given by

$$1 + \beta + \beta^2 + \cdots = \sum_{i=0}^{\infty} \beta^i = \frac{1}{1-\beta} \tag{8.34}$$

Thus in the limit, we multiply the gradient by $1/(1-\beta)$. For example, if $\beta = 0.9$, we scale the gradient up by 10.

Since we update the parameters using the gradient average $\mathbf{m}_{t-1}$, rather than just the most recent gradient, $\mathbf{g}_{t-1}$, we see that past gradients can exhibit some influence on the present. Furthermore, when momentum is combined with SGD, discussed in Sec. 8.4, we will see that it can simulate the effects of a larger minibatch, without the computational cost.

8.2.4.2 Nesterov momentum

One problem with the standard momentum method is that it may not slow down enough at the bottom of a valley, causing oscillation. The **Nesterov accelerated gradient** method of [Nes04] instead modifies the gradient descent to include an extrapolation step, as follows:

$$\tilde{\boldsymbol{\theta}}_{t+1} = \boldsymbol{\theta}_t + \beta_t(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}) \tag{8.35}$$

$$\boldsymbol{\theta}_{t+1} = \tilde{\boldsymbol{\theta}}_{t+1} - \eta_t \nabla \mathcal{L}(\tilde{\boldsymbol{\theta}}_{t+1}) \tag{8.36}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

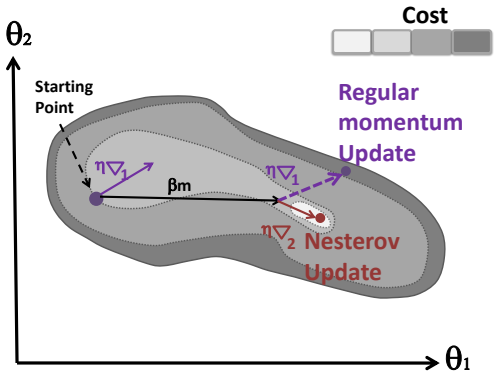


Figure 8.13: Illustration of the Nesterov update. Adapted from Figure 11.6 of [Gér19].

This is essentially a form of one-step "look ahead", that can reduce the amount of oscillation, as illustrated in Fig. 8.13.

Nesterov accelerated gradient can also be rewritten in the same format as standard momentum. In this case, the momentum term is updated using the gradient at the predicted new location,

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t - \eta_t \nabla \mathcal{L}(\boldsymbol{\theta}_t + \beta \mathbf{m}_t) \tag{8.37}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \mathbf{m}_{t+1} \tag{8.38}$$

This explains why the Nesterov accelerated gradient method is sometimes called Nesterov momentum. It also shows how this method can be faster than standard momentum: the momentum vector is already roughly pointing in the right direction, so measuring the gradient at the new location, $\boldsymbol{\theta}_t + \beta \mathbf{m}_t$, rather than the current location, $\boldsymbol{\theta}_t$, can be more accurate.

The Nesterov accelerated gradient method is provably faster than steepest descent for convex functions when β and η_t are chosen appropriately. It is called "accelerated" because of this improved convergence rate, which is optimal for gradient-based methods using only first-order information when the objective function is convex and has Lipschitz-continuous gradients. In practice, however, using Nesterov momentum can be slower than steepest descent, and can even unstable if β or η_t are misspecified.

8.3 Second-order methods

Optimization algorithms that only use the gradient are called **first-order** methods. They have the advantage that the gradient is cheap to compute and to store, but they do not model the curvature of the space, and hence they can be slow to converge, as we have seen in Fig. 8.12. **Second-order** optimization methods incorporate curvature in various ways (e.g., via the Hessian), which may yield faster convergence. We discuss some of these methods below.

8.3.1 Newton's method

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{H}_t^{-1} \mathbf{g}_t \tag{8.39}$$

where

$$\mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t} = \nabla^2 \mathcal{L}(\boldsymbol{\theta}_t) = \mathbf{H}(\boldsymbol{\theta}_t) \tag{8.40}$$

is assumed to be positive-definite to ensure the update is well-defined. The pseudo-code for Newton's method is given in Algorithm 1. The intuition for why this is faster than gradient descent is that the matrix inverse $\mathbf{H}^{-1}$ "undoes" any skew in the local curvature, converting a topology like Fig. 8.12a to one like Fig. 8.12b.

Algorithm 1: Newton's method for minimizing a function

Initialize $\boldsymbol{\theta}_0$;
for $t = 1, 2, \ldots$ *until convergence* **do**
  Evaluate $\mathbf{g}_t = \nabla \mathcal{L}(\boldsymbol{\theta}_t)$;
  Evaluate $\mathbf{H}_t = \nabla^2 \mathcal{L}(\boldsymbol{\theta}_t)$;
  Solve $\mathbf{H}_t \mathbf{d}_t = -\mathbf{g}_t$ for $\mathbf{d}_t$;
  Use line search to find stepsize η_t along $\mathbf{d}_t$;
  $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \eta_t \mathbf{d}_t$;

This algorithm can be derived as follows. Consider making a second-order Taylor series approximation of $\mathcal{L}(\boldsymbol{\theta})$ around $\boldsymbol{\theta}_t$:

$$\mathcal{L}_{\text{quad}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}_t) + \mathbf{g}_t^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_t) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_t)^\top \mathbf{H}_t (\boldsymbol{\theta} - \boldsymbol{\theta}_t) \tag{8.41}$$

The minimum of $\mathcal{L}_{\text{quad}}$ is at

$$\boldsymbol{\theta} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \mathbf{g}_t \tag{8.42}$$

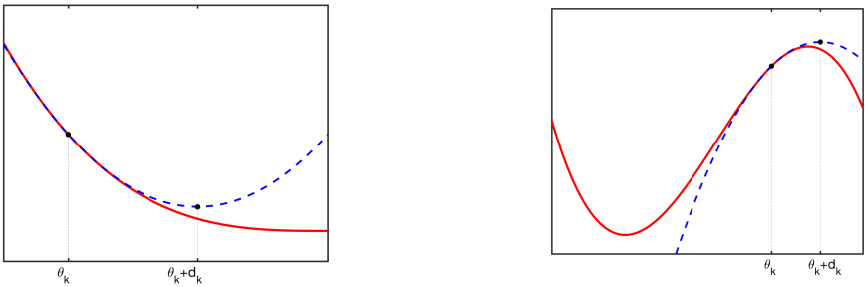
So if the quadratic approximation is a good one, we should pick $\mathbf{d}_t = -\mathbf{H}_t^{-1} \mathbf{g}_t$ as our descent direction. See Fig. 8.14(a) for an illustration. Note that, in a "pure" Newton method, we use $\eta_t = 1$ as our stepsize. However, we can also use linesearch to find the best stepsize; this tends to be more robust as using $\eta_t = 1$ may not always converge globally.

If we apply this method to linear regression, we get to the optimum in one step, since (as we shown in Sec. 11.2.2.1) we have $\mathbf{H} = \mathbf{X}^\top \mathbf{X}$ and $\mathbf{g} = \mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}$, so the Newton update becomes

$$\mathbf{w}_1 = \mathbf{w}_0 - \mathbf{H}^{-1} \mathbf{g} = \mathbf{w}_0 - (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X} \mathbf{w}_0 - \mathbf{X}^\top \mathbf{y}) = \mathbf{w}_0 - \mathbf{w}_0 + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{8.43}$$

which is the OLS estimate. However, when we apply this method to logistic regression, it may take multiple iterations to converge to the global optimum, as we discuss n Sec. 10.2.6.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.14: Illustration of Newton's method for minimizing a 1d function. (a) The solid curve is the function $\mathcal{L}(x)$. The dotted line $\mathcal{L}_{\text{quad}}(\theta)$ is its second order approximation at θ_t. The Newton step d_t is what must be added to θ_t to get to the minimum of $\mathcal{L}_{\text{quad}}(\theta)$. Adapted from Figure 13.4 of [Van06]. Generated by code at figures.probml.ai/book1/8.14. (b) Illustration of Newton's method applied to a nonconvex function. We fit a quadratic function around the current point θ_t and move to its stationary point, $\theta_{t+1} = \theta_t + d_t$. Unfortunately, this takes us near a local maximum of f, not minimum. This means we need to be careful about the extent of our quadratic approximation. Adapted from Figure 13.11 of [Van06]. Generated by code at figures.probml.ai/book1/8.14.

8.3.2 BFGS and other quasi-Newton methods

Quasi-Newton methods, sometimes called **variable metric** methods, iteratively build up an approximation to the Hessian using information gleaned from the gradient vector at each step. The most common method is called **BFGS** (named after its simultaneous inventors, Broyden, Fletcher, Goldfarb and Shanno), which updates the approximation to the Hessian $\mathbf{B}_t \approx \mathbf{H}_t$ as follows:

$$\mathbf{B}_{t+1} = \mathbf{B}_t + \frac{\mathbf{y}_t \mathbf{y}_t^\top}{\mathbf{y}_t^\top \mathbf{s}_t} - \frac{(\mathbf{B}_t \mathbf{s}_t)(\mathbf{B}_t \mathbf{s}_t)^\top}{\mathbf{s}_t^\top \mathbf{B}_t \mathbf{s}_t} \tag{8.44}$$

$$\mathbf{s}_t = \boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1} \tag{8.45}$$

$$\mathbf{y}_t = \mathbf{g}_t - \mathbf{g}_{t-1} \tag{8.46}$$

This is a rank-two update to the matrix. If $\mathbf{B}_0$ is positive-definite, and the step size η is chosen via line search satisfying both the Armijo condition in Eq. (8.27) and the following curvature condition

$$\nabla \mathcal{L}(\boldsymbol{\theta}_t + \eta \mathbf{d}_t) \geq c_2 \eta \mathbf{d}_t^\top \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{8.47}$$

then $\mathbf{B}_{t+1}$ will remain positive definite. The constant c_2 is chosen within $(c, 1)$ where c is the tunable parameter in Eq. (8.27). The two step size conditions are together known as the **Wolfe conditions**. We typically start with a diagonal approximation, $\mathbf{B}_0 = \mathbf{I}$. Thus BFGS can be thought of as a "diagonal plus low-rank" approximation to the Hessian.

Alternatively, BFGS can iteratively update an approximation to the inverse Hessian, $\mathbf{C}_t \approx \mathbf{H}_t^{-1}$, as follows:

$$\mathbf{C}_{t+1} = \left(\mathbf{I} - \frac{\mathbf{s}_t \mathbf{y}_t^\top}{\mathbf{y}_t^\top \mathbf{s}_t}\right) \mathbf{C}_t \left(\mathbf{I} - \frac{\mathbf{y}_t \mathbf{s}_t^\top}{\mathbf{y}_t^\top \mathbf{s}_t}\right) + \frac{\mathbf{s}_t \mathbf{s}_t^\top}{\mathbf{y}_t^\top \mathbf{s}_t} \tag{8.48}$$

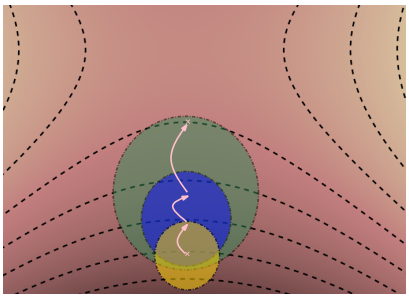


Figure 8.15: Illustration of the trust region approach. The dashed lines represents contours of the original nonconvex objective. The circles represent successive quadratic approximations. From Figure 4.2 of [Pas14]. Used with kind permission of Razvan Pascanu.

Since storing the Hessian approximation still takes $O(D^2)$ space, for very large problems, one can use **limited memory BFGS**, or **L-BFGS**, where we control the rank of the approximation by only using the M most recent $(\mathbf{s}_t, \mathbf{y}_t)$ pairs while ignoring older information. Rather than storing $\mathbf{B}_t$ explicitly, we just store these vectors in memory, and then approximate $\mathbf{H}_t^{-1}\mathbf{g}_t$ by performing a sequence of inner products with the stored $\mathbf{s}_t$ and $\mathbf{y}_t$ vectors. The storage requirements are therefore $O(MD)$. Typically choosing M to be between 5–20 suffices for good performance [NW06, p177].

Note that sklearn uses LBFGS as its default solver for logistic regression.[1]

8.3.3 Trust region methods

If the objective function is nonconvex, then the Hessian $\mathbf{H}_t$ may not be positive definite, so $\mathbf{d}_t = -\mathbf{H}_t^{-1}\mathbf{g}_t$ may not be a descent direction. This is illustrated in 1d in Fig. 8.14(b), which shows that Newton's method can end up in a local maximum rather than a local minimum.

In general, any time the quadratic approximation made by Newton's method becomes invalid, we are in trouble. However, there is usually a local region around the current iterate where we can safely approximate the objective by a quadratic. Let us call this region $\mathcal{R}_t$, and let us call $M(\boldsymbol{\delta})$ the model (or approximation) to the objective, where $\boldsymbol{\delta} = \boldsymbol{\theta} - \boldsymbol{\theta}_t$. Then at each step we can solve

$$\boldsymbol{\delta}^* = \underset{\boldsymbol{\delta} \in \mathcal{R}_t}{\operatorname{argmin}}\, M_t(\boldsymbol{\delta}) \tag{8.49}$$

This is called **trust-region optimization**. (This can be seen as the "opposite" of line search, in the sense that we pick a distance we want to travel, determined by $\mathcal{R}_t$, and then solve for the optimal direction, rather than picking the direction and then solving for the optimal distance.)

We usually assume that $M_t(\boldsymbol{\delta})$ is a quadratic approximation:

$$M_t(\boldsymbol{\delta}) = \mathcal{L}(\boldsymbol{\theta}_t) + \mathbf{g}_t^\top \boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top \mathbf{H}_t \boldsymbol{\delta} \tag{8.50}$$

where $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$ is the gradient, and $\mathbf{H}_t = \nabla^2_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$ is the Hessian. Furthermore, it is common to assume that $\mathcal{R}_t$ is a ball of radius r, i.e., $\mathcal{R}_t = \{\boldsymbol{\delta} : ||\boldsymbol{\delta}||_2 \leq r\}$. Using this, we can convert

1. See https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

the constrained problem into an unconstrained one as follows:

$$\boldsymbol{\delta}^* = \underset{\boldsymbol{\delta}}{\operatorname{argmin}}\, M(\boldsymbol{\delta}) + \lambda||\boldsymbol{\delta}||_2^2 = \underset{\boldsymbol{\delta}}{\operatorname{argmin}}\, \mathbf{g}^\top\boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top(\mathbf{H}+\lambda\mathbf{I})\boldsymbol{\delta} \tag{8.51}$$

for some Lagrange multiplier $\lambda > 0$ which depends on the radius r (see Sec. 8.5.1 for a discussion of Lagrange multipliers). We can solve this using

$$\boldsymbol{\delta} = -(\mathbf{H}+\lambda\mathbf{I})^{-1}\mathbf{g} \tag{8.52}$$

This is called **Tikhonov damping** or **Tikhonov regularization**. See Fig. 8.15 for an illustration.

Note that adding a sufficiently large $\lambda\mathbf{I}$ to $\mathbf{H}$ ensures the resulting matrix is always positive definite. As $\lambda \to 0$, this trust method reduces to Newton's method, but for λ large enough, it will make all the negative eigenvalues positive (and all the 0 eigenvalues become equal to λ).

8.3.4 Natural gradient descent *

In this section, we discuss **natural gradient descent** (**NGD**) [Ama98], which is a second order method for optimizing the parameters of (conditional) probability distributions $p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})$. The key idea is to compute parameter updates by measuring distances between the induced distributions, rather than comparing parameter values directly.

For example, when comparing two Gaussians, it is not very meaningful to compute Euclidean distance in parameter space, $||\boldsymbol{\theta}-\boldsymbol{\theta}'||_2$, where $\boldsymbol{\theta} = (\mu, \sigma)$, since the significance (in terms of prediction) of a difference of size δ between μ and μ' depends on the value of σ. This is illustrated in Fig. 8.16(a-b), which shows two univariate Gaussian distributions (dotted and solid lines) whose means differ by δ. In Fig. 8.16(a), they share the same small variance σ^2, whereas in Fig. 8.16(b), they share the same large variance. It is clear that the value of δ matters much more (in terms of the effect on the distribution) when the variance is small. Thus we see that the two parameters interact with each other, which the Euclidean distance cannot capture. This problem gets much worse when we consider more complex models, such as deep neural networks (see e.g., [SSE18; ZMG19]). By modeling such correlations, NGD can sometimes converge much faster than other gradient methods.

Algorithmically, the basic idea in NGD is to use the Fisher information matrix (Sec. 4.7.2) as a preconditioning matrix, i.e., we perform updates of the following form:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{F}(\boldsymbol{\theta}_t)^{-1}\mathbf{g}_t \tag{8.53}$$

The term

$$\mathbf{F}(\boldsymbol{\theta}_t)^{-1}\mathbf{g}_t = \mathbf{F}(\boldsymbol{\theta}_t)^{-1}\nabla\mathcal{L}(\boldsymbol{\theta}_t) \triangleq \tilde{\nabla}\mathcal{L}(\boldsymbol{\theta}_t) \tag{8.54}$$

is called the **natural gradient**, where $\mathbf{F}$ is the (averaged) Fisher information matrix (see Sec. 4.7.2), given by

$$\mathbf{F}(\boldsymbol{\theta}) = \mathbb{E}_{p_D(\mathbf{x})}\left[\mathbf{F}_{\mathbf{x}}(\boldsymbol{\theta})\right] \tag{8.55}$$

$$\mathbf{F}_{\mathbf{x}}(\boldsymbol{\theta}) = -\mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\left[\nabla^2 \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\right] = \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\left[(\nabla \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}))(\nabla \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}))^\top\right] \tag{8.56}$$

The intuition for why NGD is a good idea is based on the following approximation:

$$\mathbb{KL}\left(p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}) \| p_{\boldsymbol{\theta}+\boldsymbol{\delta}}(\mathbf{y}|\mathbf{x})\right) \approx \frac{1}{2}\boldsymbol{\delta}^\top\mathbf{F}_{\mathbf{x}}\boldsymbol{\delta} \tag{8.57}$$

(a) (b)

Figure 8.16: Changing the mean of a Gaussian by a fixed amount (from solid to dotted curve) can have more impact when the (shared) variance is small (as in a) compared to when the variance is large (as in b). Hence the impact (in terms of prediction accuracy) of a change to μ depends on where the optimizer is in (μ, σ) space. From Figure 3 of [Hon+10], reproduced from [Val00]. Used with kind permission of Antti Honkela.

where $\boldsymbol{\delta} = \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t$. Thus we see that the divergence in the output distribution is a quadratic function of $\boldsymbol{\delta}$, where $\mathbf{F}_{\mathbf{x}}$ represents the curvature term. Thus we see that NGD is a kind of trust region method.

8.3.4.1 Example

In this section, we consider a simple example of NGD from [SD12]. We will use a 2D Gaussian with an unusual, highly coupled parameterization:

$$p(\mathbf{y}; \boldsymbol{\theta}) = \frac{1}{2\pi} \exp\left[-\frac{1}{2}\left(y_1 - \left[3\theta_1 + \frac{1}{3}\theta_2 \right] \right)^2 - \frac{1}{2}\left(y_2 - \frac{1}{3}\theta_1 \right)^2 \right] \tag{8.58}$$

The objective is the cross-entropy loss:

$$\mathcal{L}(\boldsymbol{\theta}) = -\mathbb{E}_{p^*(\mathbf{y})} \left[\log p(\mathbf{y}; \boldsymbol{\theta}) \right] \tag{8.59}$$

The gradient of this objective is given by

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \begin{pmatrix} \mathbb{E}_{p^*(\mathbf{x})} \left[3(y_1 - [3\theta_1 + \frac{1}{3}\theta_2]) + \frac{1}{3}(y_2 - \frac{1}{3}\theta_1) \right] \\ \mathbb{E}_{p^*(\mathbf{x})} \left[\frac{1}{3}(y_1 - [3\theta_1 + \frac{1}{3}\theta_2]) \right] \end{pmatrix} \tag{8.60}$$

Suppose that $p^*(\mathbf{y}) = p(\mathbf{y}; [0, 0])$. Then the Fisher matrix is a constant matrix, given by

$$\mathbf{F} = \begin{pmatrix} 3^2 + \frac{1}{3^2} & 1 \\ 1 & \frac{1}{3^2} \end{pmatrix} \tag{8.61}$$

Fig. 8.17 compares steepest descent in $\boldsymbol{\theta}$ space with the natural gradient method, which is equivalent to steepest descent in a "whitened" parameter space given by $\boldsymbol{\phi} = \mathbf{F}^{\frac{1}{2}} \boldsymbol{\theta}$ (see Sec. 7.4.5 for details). Both methods start at $\boldsymbol{\theta} = (1, -1)$. The global optimum is at $\boldsymbol{\theta} = (0, 0)$. We see that the NG method (blue dots) converges much faster to this optimum and takes the shortest path, whereas steepest descent takes a very circuitous route. We also see that the gradient field in the whitened parameter space is more "spherical", which makes descent much simpler and faster.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

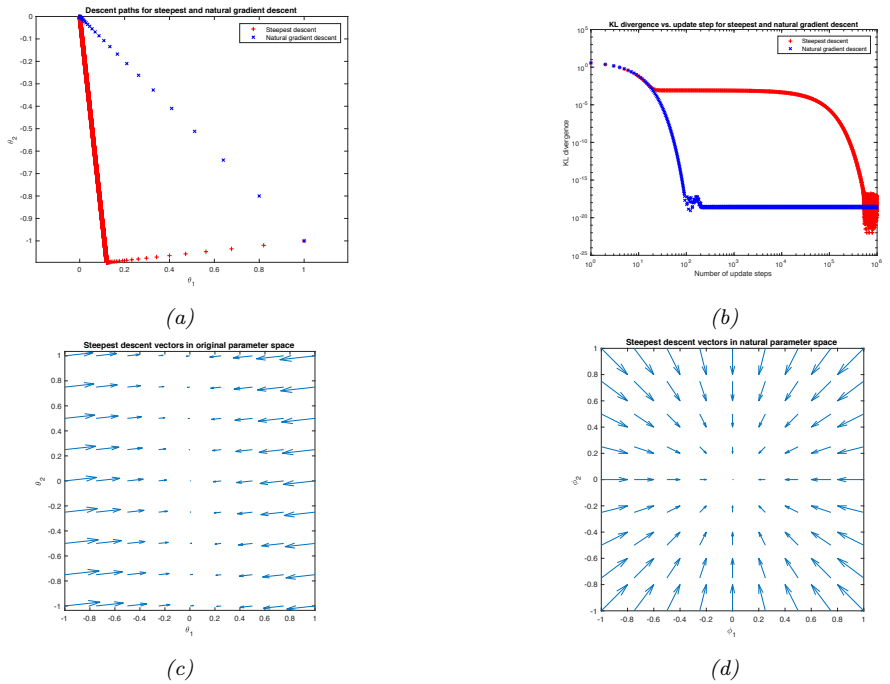


Figure 8.17: Illustration of the benefits of natural gradient vs steepest descent on a 2D problem. (a) Trajectories of the two methods in parameter space (red = steepest descent, blue = NG). They both start at $(1, -1)$, bottom right location. (b) Objective vs number of iterations. (c) Gradient field in the $\boldsymbol{\theta}$ parameter space. (d) Gradient field in the whitened $\boldsymbol{\phi} = \mathbf{F}^{\frac{1}{2}}\boldsymbol{\theta}$ parameter space used by NG. Generated by code at figures.probml.ai/book1/8.17.

8.3.4.2 Approximating the FIM

Unfortunately, computing the FIM is in general intractable. However, a variety of approximations to this matrix have been proposed for high dimensional problems, such as the **KFAC** (Kronecker-Factored approximate curvature) method from [MG15].

A simpler approach is to approximate the FIM by replacing the model's distribution with the empirical distribution. In particular, define $p_D(\mathbf{x}, \mathbf{y}) = \frac{1}{N}\sum_{n=1}^{N} \delta(\mathbf{x} - \mathbf{x}_n)\delta(\mathbf{y} - \mathbf{y}_n)$, $p_D(\mathbf{x}) = \frac{1}{N}\sum_{n=1}^{N} \delta(\mathbf{x} - \mathbf{x}_n)$ and $p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{y}) = p_D(\mathbf{x})p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$. Then we can compute the **empirical Fisher** [Mar16] as follows:

$$\mathbf{F} = \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{x},\mathbf{y})}\left[\nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})^\top\right] \tag{8.62}$$

$$\approx \mathbb{E}_{p_D(\mathbf{x},\mathbf{y})}\left[\nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})^\top\right] \tag{8.63}$$

$$= \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \nabla \log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})^\top \tag{8.64}$$

This approximation is widely used, since it is simple to compute. Unfortunately, it does not work as well as the true Fisher [KBH19; Tho+19]. For it to be a good approximation, we need to already be

close to the minimum, to have $p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{y}) \approx p_D(\mathbf{x}, \mathbf{y})$. Using the outer-product of gradients leads to a large or small preconditioner when the gradients are large or small, regardless of curvature. For example, for a linear regression model, the FIM is constant but the squared gradients are not; using the approximation leads to small steps when the gradient is large and large steps when the gradient is small.

8.4 Stochastic gradient descent

In this section, we consider **stochastic optimization**, where the goal is to minimize the average value of a function:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{z})}\left[\mathcal{L}(\boldsymbol{\theta}, \mathbf{z})\right] \tag{8.65}$$

where $\mathbf{z}$ is a random input to the objective. This could be a "noise" term, coming from the environment, or it could be a training example drawn randomly from the training set, as we explain below.

At each iteration, we assume we observe $\mathcal{L}_t(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}, \mathbf{z}_t)$, where $\mathbf{z}_t \sim q$. We also assume a way to compute an unbiased estimate of the gradient of $\mathcal{L}$. If the distribution $q(\mathbf{z})$ is independent of the parameters we are optimizing, we can use $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_t(\boldsymbol{\theta}_t)$. In this case, The resulting algorithm can be written as follows:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}(\boldsymbol{\theta}_t, \mathbf{z}_t) = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t \tag{8.66}$$

This method is known as **stochastic gradient descent** or **SGD**. As long as the gradient estimate is unbiased, then this method will converge to a stationary point, providing we decay the step size η_t at a certain rate, as we discuss in Sec. 8.4.3.

8.4.1 Application to finite sum problems

SGD is very widely used in machine learning. To see why, recall from Sec. 4.3 that many model fitting procedures are based on empirical risk minimization, which involve minimizing the following loss:

$$\mathcal{L}(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t)) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_n(\boldsymbol{\theta}_t) \tag{8.67}$$

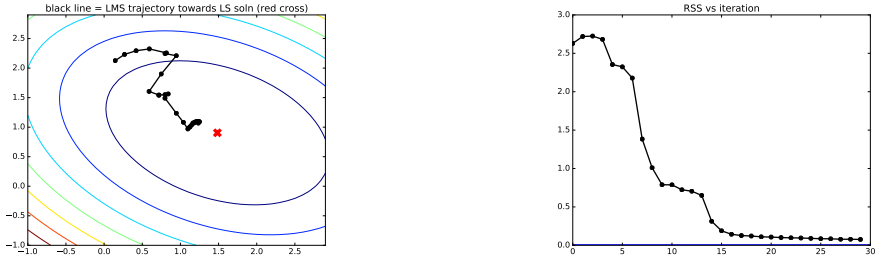
This is called a **finite sum problem**. The gradient of this objective has the form

$$\mathbf{g}_t = \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t)) \tag{8.68}$$

This requires summing over all N training examples, and thus can be slow if N is large. Fortunately we can approximate this by sampling a **minibatch** of $B \ll N$ samples to get

$$\mathbf{g}_t \approx \frac{1}{|\mathcal{B}_t|} \sum_{n \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{|\mathcal{B}_t|} \sum_{n \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{y}_n, f(\mathbf{x}_n; \boldsymbol{\theta}_t)) \tag{8.69}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.18: Illustration of the LMS algorithm. Left: we start from $\boldsymbol{\theta} = (-0.5, 2)$ and slowly converging to the least squares solution of $\hat{\boldsymbol{\theta}} = (1.45, 0.93)$ (red cross). Right: plot of objective function over time. Note that it does not decrease monotonically. Generated by code at figures.probml.ai/book1/8.18.

where $\mathcal{B}_t$ is a set of randomly chosen examples to use at iteration t.[2] This is an unbiased approximation to the empirical average in Eq. (8.68). Hence we can safely use this with SGD.

Although the theoretical rate of convergence of SGD is slower than batch GD (in particular, SGD has a sublinear convergence rate), in practice SGD is often faster, since the per-step time is much lower [BB08; BB11]. To see why SGD can make faster progress than full batch GD, suppose we have a dataset consisting of a single example duplicated K times. Batch training will be (at least) K times slower than SGD, since it will waste time computing the gradient for the repeated examples. Even if there are no duplicates, batch training can be wasteful, since early on in training the parameters are not well estimated, so it is not worth carefully evaluating the gradient.

8.4.2 Example: SGD for fitting linear regression

In this section, we show how to use SGD to fit a linear regression model. Recall from Sec. 4.2.7 that the objective has the form

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2N} \sum_{n=1}^{N} (\mathbf{x}_n^\top \boldsymbol{\theta} - y_n)^2 = \frac{1}{2N} ||\mathbf{X}\boldsymbol{\theta} - \mathbf{y}||_2^2 \tag{8.70}$$

The gradient is

$$\mathbf{g}_t = \frac{1}{N} \sum_{n=1}^{N} (\boldsymbol{\theta}_t^\top \mathbf{x}_n - y_n)\mathbf{x}_n \tag{8.71}$$

Now consider using SGD with a minibatch size of $B = 1$. The update becomes

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t (\boldsymbol{\theta}_t^\top \mathbf{x}_n - y_n)\mathbf{x}_n \tag{8.72}$$

2. In practice we usually sample $\mathcal{B}_t$ without replacement. However, once we reach the end of the dataset (i.e., after a single training **epoch**), we can perform a random shuffling of the examples, to ensure that each minibatch on the next epoch is different from the last. This version of SGD is analyzed in [HS19].

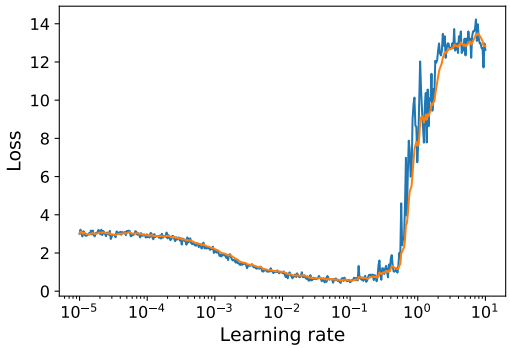


Figure 8.19: Loss vs learning rate (horizontal axis). Training loss vs learning rate for a small MLP fit to FashionMNIST using vanilla SGD. (Raw loss in blue, EWMA smoothed version in orange). Generated by code at figures.probml.ai/book1/8.19.

where $n = n(t)$ is the index of the example chosen at iteration t. The overall algorithm is called the **least mean squares** (**LMS**) algorithm, and is also known as the **delta rule**, or the **Widrow-Hoff rule**.

Fig. 8.18 shows the results of applying this algorithm to the data shown in Fig. 11.2. We start at $\boldsymbol{\theta} = (-0.5, 2)$ and converge (in the sense that $||\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}||_2^2$ drops below a threshold of 10^{-2}) in about 26 iterations. Note that SGD (and hence LMS) may require multiple passes through the data to find the optimum.

8.4.3 Choosing the step size (learning rate)

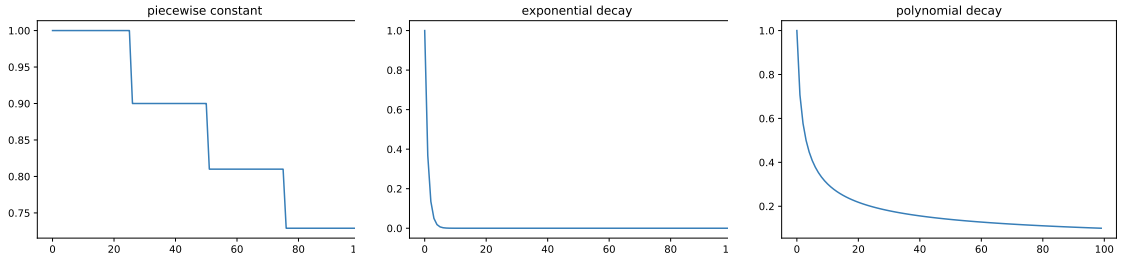
When using SGD, we need to be careful in how we choose the learning rate in order to achieve convergence. For example, in Fig. 8.19 we plot the loss vs the learning rate when we apply SGD to a deep neural network classifier (see Chapter 13 for details). We see a U-shaped curve, where overly small learning rates result in underfitting, and overly large learning results result in instability of the model (c.f., Fig. 8.11(b)); in both cases, we fail to converge to a local optimum.

One heuristic for choosing a good learning rate, proposed in [Smi18], is to start with a small learning rate and gradually increase it, evaluating performance using a small number of minibatches. We then make a plot like the one in Fig. 8.19, and pick the learning rate with the lowest loss. (In practice, it is better to pick a rate that is slightly smaller than (i.e., to the left of) the one with the lowest loss, to ensure stability.)

Rather than choosing a single constant learning rate, we can use a **learning rate schedule**, in which we adjust the step size over time. Theoretically, a sufficient condition for SGD to achieve convergence is if the learning rate schedule satisfies the **Robbins-Monro conditions**:

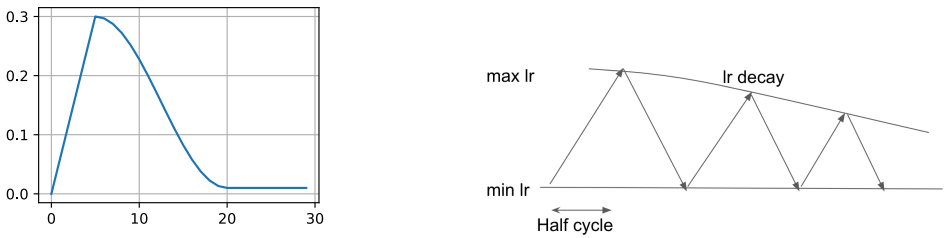
$$\eta_t \to 0, \ \frac{\sum_{t=1}^{\infty} \eta_t^2}{\sum_{t=1}^{\infty} \eta_t} \to 0 \tag{8.73}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 8.20: Illustration of some common learning rate schedules. (a) Piecewise constant. (b) Exponential decay. (c) Polynomial decay. Generated by code at figures.probml.ai/book1/8.20.



(a) (b)

Figure 8.21: (a) Linear warm-up followed by cosine cool-down. (b) Cyclical learning rate schedule.

Some common examples of learning rate schedules are listed below:

$$\eta_t = \eta_i \text{ if } t_i \leq t \leq t_{i+1} \quad \text{piecewise constant} \tag{8.74}$$

$$\eta_t = \eta_0 e^{-\lambda t} \text{ exponential decay} \tag{8.75}$$

$$\eta_t = \eta_0 (\beta t + 1)^{-\alpha} \text{ polynomial decay} \tag{8.76}$$

In the piecewise constant schedule, t_i are a set of time points at which we adjust the learning rate to a specified value. For example, we may set $\eta_i = \eta_0 \gamma^i$, which reduces the initial learning rate by a factor of γ for each threshold (or milestone) that we pass. Figure 8.20a illustrates this for $\eta_0 = 1$ and $\gamma = 0.9$. This is called **step decay**. Sometimes the thresholds times are computed adaptively, by estimating when the train or validation loss has plateaud; this is called **reduce-on-plateau**. Exponential decay is typically too fast, as illustrated in Fig. 8.20b. A common choice is polynomial decay, with $\alpha = 0.5$ and $\beta = 1$, as illustrated in Fig. 8.20c; this corresponds to a **square-root schedule**, $\eta_t = \eta_0 \frac{1}{\sqrt{t+1}}$.

In the deep learning community, another common schedule is to quickly increase the learning rate and then gradually decrease it again, as shown in Fig. 8.21a. This is called **learning rate warmup**. The motivation for this is the following: initially the parameters may be in a part of the loss landscape that is poorly conditioned, so a large step size will "bounce around" too much (c.f.,

Fig. 8.11(b)) and fail to make progress downhill. However, with a slow learning rate, the algorithm can discover flatter regions of space, where a larger step size can be used. Once there, fast progress can be made. However, to ensure convergence to a point, we must reduce the learning rate to 0. See [Got+19] for more details.

It is also possible to increase and decrease the learning rate multiple times, in a cyclical fashion. This is called a **cyclical learning rate** [Smi18], and was popularized by the `fast.ai` course. See Fig. 8.21b for an illustration using triangular shapes. The motivation behind this approach is to escape local minima. The minimum and maximum learning rates can be found based on the initial "dry run" described above, and the half-cycle can be chosen based on how many restarts you want to do with your training budget. A related approach, known as **stochastic gradient descent with warm restarts**, was proposed in [LH17]; they proposed storing all the checkpoints visited after each cool down, and using all of them as members of a model ensemble. (See Sec. 18.2 for a discussion of ensemble learning.)

An alternative to using heuristics for estimating the learning rate is to use line search (Sec. 8.2.2.2). This is tricky when using SGD, because the noisy gradients make the computation of the Armijo condition difficult [CS20]. However, [Vas+19] show that it can be made to work if the variance of the gradient noise goes to zero over time. This can happen if the model is sufficiently flexible that it can perfectly interpolate the training set.

8.4.4 Iterate averaging

The parameter estimates produced by SGD can be very unstable over time. To reduce the variance of the estimate, we can compute the average using

$$\overline{\boldsymbol{\theta}}_t = \frac{1}{t}\sum_{i=1}^{t}\boldsymbol{\theta}_i = \frac{1}{t}\boldsymbol{\theta}_t + \frac{t-1}{t}\overline{\boldsymbol{\theta}}_{t-1} \tag{8.77}$$

where $\boldsymbol{\theta}_t$ are the usual SGD iterates. This is called **iterate averaging** or **Polyak-Ruppert averaging** [Rup88].

In [PJ92], they prove that the estimate $\overline{\boldsymbol{\theta}}_t$ achieves the best possible asymptotic convergence rate among SGD algorithms, matching that of variants using second-order information, such as Hessians.

This averaging can also have statistical benefits. For example, in [NR18], they prove that, in the case of linear regression, this method is equivalent to ℓ_2 regularization (i.e., ridge regression).

Rather than an exponential moving average of SGD iterates, **Stochastic Weight Averaging** (SWA) [Izm+18] uses an *equal* average in conjunction with a modified learning rate schedule. In contrast to standard Polyak-Ruppert averaging, which was motivated for faster convergence rates, SWA exploits the flatness in objectives used to train deep neural networks, to find solutions which provide better generalization.

8.4.5 Variance reduction *

In this section, we discuss various ways to reduce the variance in SGD. In some cases, this can improve the theoretical convergence rate from sublinear to linear (i.e., the same as full-batch gradient descent) [SLRB17; JZ13; DBLJ14]. These methods reduces the variance of the gradients, rather than the parameters themselves and are designed to work for finite sum problems.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.4.5.1 SVRG

The basic idea of **stochastic variance reduced gradient** (**SVRG**) [JZ13] is to use a control variate, in which we estimate a baseline value of the gradient based on the full batch, which we then use to compare the stochastic gradients to.

More precisely, ever so often (e.g., once per epoch), we compute the full gradient at a "snapshot" of the model parameters $\tilde{\boldsymbol{\theta}}$; the corresponding "exact" gradient is therefore $\nabla \mathcal{L}(\tilde{\boldsymbol{\theta}})$. At step t, we compute the usual stochastic gradient at the current parameters, $\nabla \mathcal{L}_t(\boldsymbol{\theta}_t)$, but also at the snapshot parameters, $\nabla \mathcal{L}_t(\tilde{\boldsymbol{\theta}})$, which we use as a baseline. We can then use the following improved gradient estimate

$$\mathbf{g}_t = \nabla \mathcal{L}_t(\boldsymbol{\theta}_t) - \nabla \mathcal{L}_t(\tilde{\boldsymbol{\theta}}) + \nabla \mathcal{L}(\tilde{\boldsymbol{\theta}}) \tag{8.78}$$

to compute $\boldsymbol{\theta}_{t+1}$. This is unbiased because $\mathbb{E}\left[\nabla \mathcal{L}_t(\tilde{\boldsymbol{\theta}})\right] = \nabla \mathcal{L}(\tilde{\boldsymbol{\theta}})$. Furthermore, the update only involves two gradient computations, since we can compute $\nabla \mathcal{L}(\tilde{\boldsymbol{\theta}})$ once per epoch. At the end of the epoch, we update the snapshot parameters, $\tilde{\boldsymbol{\theta}}$, based on the most recent value of $\boldsymbol{\theta}_t$, or a running average of the iterates, and update the expected baseline. (We can compute snapshots less often, but then the baseline will not be correlated with the objective and can hurt performance, as shown in [DB18].)

Iterations of SVRG are computationally faster than those of full-batch GD, but SVRG can still match the theoretical convergence rate of GD.

8.4.5.2 SAGA

In this section, we describe the **stochastic averaged gradient accelerated** (**SAGA**) algorithm of [DBLJ14]. Unlike SVRG, it only requires one full batch gradient computation, at the start of the algorithm. However, it "pays" for this saving in time by using more memory. In particular, it must store N gradient vectors. This enables the method to maintain an approximation of the global gradient by removing the old local gradient from the overall sum and replacing it with the new local gradient. This is called an **aggregated gradient** method.

More precisely, we first initialize by computing $\mathbf{g}_n^{\text{local}} = \nabla \mathcal{L}_n(\boldsymbol{\theta}_0)$ for all n, and the average, $\mathbf{g}^{\text{avg}} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{g}_n^{\text{local}}$. Then, at iteration t, we use the gradient estimate

$$\mathbf{g}_t = \nabla \mathcal{L}_n(\boldsymbol{\theta}_t) - \mathbf{g}_n^{\text{local}} + \mathbf{g}^{\text{avg}} \tag{8.79}$$

where $n \sim \text{Unif}\{1, \ldots, N\}$ is the example index sampled at iteration t. We then update $\mathbf{g}_n^{\text{local}} = \nabla \mathcal{L}_n(\boldsymbol{\theta}_t)$ and $\mathbf{g}^{\text{avg}}$ by replacing the old $\mathbf{g}_n^{\text{local}}$ by its new value.

This has an advantage over SVRG since it only has to do one full batch sweep at the start. (In fact, the initial sweep is not necessary, since we can compute $\mathbf{g}^{\text{avg}}$ "lazily", by only incorporating gradients we have seen so far.) The downside is the large extra memory cost. However, if the features (and hence gradients) are sparse, the memory cost can be reasonable. Indeed, the SAGA algorithm is recommended for use in the sklearn logistic regression code when N is large and $\mathbf{x}$ is sparse.[3]

3. See https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

8.4.5.3 Application to deep learning

Variance reduction methods are widely used for fitting ML models with convex objectives, such as linear models. However, there are various difficulties associated with using SVRG with conventional deep learning training practices. For example, the use of batch normalization (Sec. 14.2.4.1), data augmentation (Sec. 19.1) and dropout (Sec. 13.5.4) all break the assumptions of the method, since the loss will differ randomly in ways that depend not just on the parameters and the data index n. For more details, see e.g., [DB18; Arn+19].

8.4.6 Preconditioned SGD

In this section, we consider **preconditioned SGD**, which involves the following update:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{M}_t^{-1} \mathbf{g}_t, \tag{8.80}$$

where $\mathbf{M}_t$ is a **preconditioning matrix**, or simply the **preconditioner**, typically chosen to be positive-definite. Unfortunately the noise in the gradient estimates make it difficult to reliably estimate the Hessian, which makes it difficult to use the methods from Sec. 8.3. In addition, it is expensive to solve for the update direction with a full preconditioning matrix. Therefore most practitioners use a diagonal preconditioner $\mathbf{M}_t$. Such preconditioners do not necessarily use second-order information, but often result in speedups compared to vanilla SGD.

8.4.6.1 AdaGrad

The **AdaGrad** (short for "adaptive gradient") method of [DHS11] was originally designed for optimizing convex objectives where many elements of the gradient vector are zero; these might correspond to features that are rarely present in the input, such as rare words. The update has the following form

$$\theta_{t+1,d} = \theta_{t,d} - \eta_t \frac{1}{\sqrt{s_{t,d} + \epsilon}} g_{t,d} \tag{8.81}$$

where

$$s_{t,d} = \sum_{t=1}^{t} g_{t,d}^2 \tag{8.82}$$

is the sum of the squared gradients and $\epsilon > 0$ is a small term to avoid dividing by zero. Equivalently we can write the update in vector form as follows:

$$\Delta \boldsymbol{\theta}_t = -\eta_t \frac{1}{\sqrt{\mathbf{s}_t + \epsilon}} \mathbf{g}_t \tag{8.83}$$

where the square root and division is performed elementwise. Viewed as preconditioned SGD, this is equivalent to taking $\mathbf{M}_t = \text{diag}(\mathbf{s}_t + \boldsymbol{\epsilon})^{1/2}$. This is an example of an **adaptive learning rate**; the overall stepsize η_t still needs to be chosen, but the results are less sensitive to it compared to vanilla GD. In particular, we usually fix $\eta_t = \eta_0$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.4.6.2 RMSProp and AdaDelta

A defining feature of AdaGrad is that the term in the denominator gets larger over time, so the effective learning rate drops. While it is necessary to ensure convergence, it might hurt performance as the denominator gets large too fast.

An alternative is to use an exponentially weighted moving average (EWMA, Sec. 4.4.2.2) of the past squared gradients, rather than their sum:

$$s_{t+1,d} = \beta s_{t,d} + (1-\beta) g_{t,d}^2 \tag{8.84}$$

In practice we usually use $\beta \sim 0.9$, which puts more weight on recent examples. In this case,

$$\sqrt{s_{t,d}} \approx \text{RMS}(\mathbf{g}_{1:t,d}) = \sqrt{\frac{1}{t}\sum_{\tau=1}^{t} g_{\tau,d}^2} \tag{8.85}$$

where RMS stands for "root mean squared". Hence this method, (which is based on the earlier **RPROP** method of [RB93]) is known as **RMSProp** [Hin14]. The overall update of RMSProp is

$$\Delta\boldsymbol{\theta}_t = -\eta_t \frac{1}{\sqrt{\mathbf{s}_t + \epsilon}} \mathbf{g}_t. \tag{8.86}$$

The **AdaDelta** method was independently introduced in [Zei12], and is similar to RMSprop. However, in addition to accumulating an EWMA of the gradients in $\hat{\mathbf{s}}$, it also keeps an EWMA of the updates $\boldsymbol{\delta}_t$ to obtain an update of the form

$$\Delta\boldsymbol{\theta}_t = -\eta_t \frac{\sqrt{\boldsymbol{\delta}_{t-1} + \epsilon}}{\sqrt{\mathbf{s}_t + \epsilon}} \mathbf{g}_t \tag{8.87}$$

where

$$\boldsymbol{\delta}_t = \beta \boldsymbol{\delta}_{t-1} + (1-\beta)(\Delta\boldsymbol{\theta}_t)^2 \tag{8.88}$$

and $\mathbf{s}_t$ is the same as in RMSProp. This has the advantage that the "units" of the numerator and denominator cancel, so we are just elementwise-multiplying the gradient by a scalar. This eliminates the need to tune the learning rate η_t, which means one can simply set $\eta_t = 1$, although popular implementations of AdaDelta still keep η_t as a tunable hyperparameter. However, since these adaptive learning rates need not decrease with time (unless we choose η_t to explicitly do so), these methods are not guaranteed to converge to a solution.

8.4.6.3 Adam

It is possible to combine RMSProp with momentum. In particular, let us compute an EWMA of the gradients (as in momentum) and squared gradients (as in RMSProp)

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \tag{8.89}$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \tag{8.90}$$

We then perform the following update:

$$\Delta\boldsymbol{\theta}_t = -\eta_t \frac{1}{\sqrt{\mathbf{s}_t} + \epsilon}\mathbf{m}_t \tag{8.91}$$

The resulting method is known as **Adam**, which stands for "adaptive moment estimation" [KB15].

The standard values for the various constants are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-6}$. (If we set $\beta_1 = 0$ and no bias-correction, we recover RMSProp, which does not use momentum.) For the overall learning rate, it is common to use a fixed value such as $\eta_t = 0.001$. Again, as the adaptive learning rate may not decrease over time, convergence is not guaranteed (see Sec. 8.4.6.4).

If we initialize with $\mathbf{m}_0 = \mathbf{s}_0 = \mathbf{0}$, then initial estimates will be biased towards small values. The authors therefore recommend using the bias-corrected moments, which increase the values early in the optimization process. These estimates are given by

$$\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t) \tag{8.92}$$

$$\hat{\mathbf{s}}_t = \mathbf{s}_t/(1-\beta_2^t) \tag{8.93}$$

The advantage of bias-correction is shown in Fig. 4.3.

8.4.6.4 Issues with adaptive learning rates

When using diagonal scaling methods, the overall learning rate is determined by $\eta_0\mathbf{M}_t^{-1}$, which changes with time. Hence these methods are often called **adaptive learning rate** methods. However, they still require setting the base learning rate η_0.

Since the EWMA methods are typically used in the stochastic setting where the gradient estimates are noisy, their learning rate adaptation can result in non-convergence even on convex problems [RKK18]. Various solutions to this problem have been proposed, including AMSGrad [RKK18], Padam [CG18; Zho+18], and Yogi [Zah+18]. For example, the Yogi update modifies Adam by replacing

$$\mathbf{s}_t = \beta_2\mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 = \mathbf{s}_{t-1} + (1-\beta_2)(\mathbf{g}_t^2 - \mathbf{s}_{t-1}) \tag{8.94}$$

with

$$\mathbf{s}_t = \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \odot \operatorname{sgn}(\mathbf{g}_t^2 - \mathbf{s}_{t-1}) \tag{8.95}$$

8.5 Constrained optimization

In this section, we consider the following **constrained optimization problem**:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}\in\mathcal{C}} \mathcal{L}(\boldsymbol{\theta}) \tag{8.96}$$

where the feasible set, or constraint set, is

$$\mathcal{C} = \{\boldsymbol{\theta} \in \mathbb{R}^D : h_i(\boldsymbol{\theta}) = 0, i \in \mathcal{E},\ g_j(\boldsymbol{\theta}) \le 0, j \in \mathcal{I}\} \tag{8.97}$$

where $\mathcal{E}$ is the set of **equality constraints**, and $\mathcal{I}$ is the set of **inequality constraints**, and $\mathcal{C} = \mathcal{I} \cap \mathcal{E}$ is the set of all constraints. (We define the constraints in this way, rather than as Boolean

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

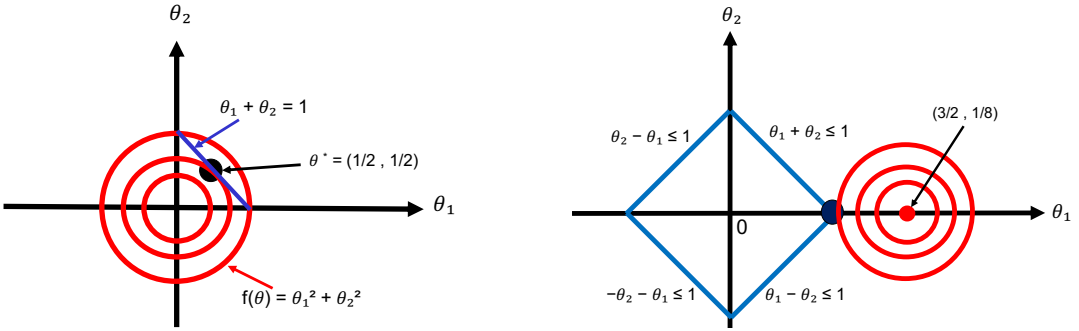


Figure 8.22: Illustration of some constrained optimization problems. Red contours are the level sets of the objective function $\mathcal{L}(\boldsymbol{\theta})$. Optimal constrained solution is the black dot, (a) Blue line is the equality constraint $h(\boldsymbol{\theta}) = 0$. (b) Blue lines denote the inequality constraints $|\theta_1| + |\theta_2| \leq 1$. (Compare to Fig. 11.10 (left).)

membership of some set, so it gives us a measure of how far from feasible any given proposed solution is.)

For example, suppose we have a quadratic objective, $\mathcal{L}(\boldsymbol{\theta}) = \theta_1^2 + \theta_2^2$, subject to a linear equality constraint, $h(\boldsymbol{\theta}) = 1 - \theta_1 - \theta_2 = 0$. Fig. 8.22(a) plots the level sets of f, as well as the constraint surface. What we are trying to do is find the point $\boldsymbol{\theta}^*$ that lives on the line, but which is closest to the origin. It is clear from the geometry that the optimal solution is $\boldsymbol{\theta} = (0.5, 0.5)$, indicated by the solid black dot.

In the following sections, we briefly describe some of the theory and algorithms underlying constrained optimization. More details can be found in other books, such as [BV04; NW06; Ber15; Ber16].

8.5.1 Lagrange multipliers

In this section, we discuss how to solve equality contrained optimization problems. We initially assume that we have just one equality constraint, $h(\boldsymbol{\theta}) = 0$.

First note that for any point on the constraint surface, $\nabla h(\boldsymbol{\theta})$ will be orthogonal to the constraint surface. To see why, consider another point nearby, $\boldsymbol{\theta} + \boldsymbol{\epsilon}$, that also lies on the surface. If we make a first-order Taylor expansion around $\boldsymbol{\theta}$ we have

$$h(\boldsymbol{\theta} + \boldsymbol{\epsilon}) \approx h(\boldsymbol{\theta}) + \boldsymbol{\epsilon}^\top \nabla h(\boldsymbol{\theta}) \tag{8.98}$$

Since both $\boldsymbol{\theta}$ and $\boldsymbol{\theta} + \boldsymbol{\epsilon}$ are on the constraint surface, we must have $h(\boldsymbol{\theta}) = h(\boldsymbol{\theta} + \boldsymbol{\epsilon})$ and hence $\boldsymbol{\epsilon}^\top \nabla h(\boldsymbol{\theta}) \approx 0$. Since $\boldsymbol{\epsilon}$ is parallel to the constraint surface, $\nabla h(\boldsymbol{\theta})$ must be perpendicular to it.

We seek a point $\boldsymbol{\theta}^*$ on the constraint surface such that $\mathcal{L}(\boldsymbol{\theta})$ is minimized. We just showed that it must satisfy the condition that $\nabla h(\boldsymbol{\theta}^*)$ is orthogonal to the constraint surface. In addition, such a point must have the property that $\nabla \mathcal{L}(\boldsymbol{\theta})$ is also orthogonal to the constraint surface, as otherwise we could decrease $\mathcal{L}(\boldsymbol{\theta})$ by moving a short distance along the constraint surface. Since both $\nabla h(\boldsymbol{\theta})$ and $\nabla \mathcal{L}(\boldsymbol{\theta})$ are orthogonal to the constraint surface at $\boldsymbol{\theta}^*$, they must be parallel (or anti-parallel) to

each other. Hence there must exist a constant $\lambda^* \in \mathbb{R}$ such that

$$\nabla \mathcal{L}(\boldsymbol{\theta}^*) = \lambda^* \nabla h(\boldsymbol{\theta}^*) \tag{8.99}$$

(We cannot just equate the gradient vectors, since they may have different magnitudes.) The constant λ^* is called a **Lagrange multiplier**, and can be positive, negative, or zero. This latter case occurs when $\nabla f(\boldsymbol{\theta}^*) = 0$.

We can convert Eq. (8.99) into an objective, known as the **Lagrangian**, that we should minimize:

$$\mathcal{L}(\boldsymbol{\theta}, \lambda) \triangleq \mathcal{L}(\boldsymbol{\theta}) - \lambda h(\boldsymbol{\theta}) \tag{8.100}$$

At a stationary point of the Lagrangian, we have

$$\nabla_{\boldsymbol{\theta},\lambda} \mathcal{L}(\boldsymbol{\theta}, \lambda) = \mathbf{0} \iff \lambda \nabla_{\boldsymbol{\theta}} h(\boldsymbol{\theta}) = \nabla \mathcal{L}(\boldsymbol{\theta}),\ h(\boldsymbol{\theta}) = 0 \tag{8.101}$$

This is called a **critical point**, and satisfies the original constraint $h(\boldsymbol{\theta}) = 0$ and Eq. (8.99).

If we have $m > 1$ constraints, we can form a new constraint function by addition, as follows:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}) = \mathcal{L}(\boldsymbol{\theta}) - \sum_{j=1}^{m} \lambda_j h_j(\boldsymbol{\theta}) \tag{8.102}$$

We now have $D+m$ equations in $D+m$ unknowns and we can use standard unconstrained optimization methods to find a stationary point. We give some examples below.

8.5.1.1 Example: 2d Quadratic objective with one linear equality constraint

Consider minimizing $\mathcal{L}(\boldsymbol{\theta}) = \theta_1^2 + \theta_2^2$ subject to the constraint that $\theta_1 + \theta_2 = 1$.

(This is the problem illustrated in Fig. 8.22(a).) The Lagrangian is

$$L(\theta_1, \theta_2, \lambda) = \theta_1^2 + \theta_2^2 - 1 - \lambda(\theta_1 + \theta_2 - 1) \tag{8.103}$$

We have the following conditions for a stationary point:

$$\frac{\partial}{\partial \theta_1} L(\theta_1, \theta_2, \lambda) = 2\theta_1 + \lambda = 0 \tag{8.104}$$

$$\frac{\partial}{\partial \theta_2} L(\theta_1, \theta_2, \lambda) = 2\theta_2 + \lambda = 0 \tag{8.105}$$

$$\frac{\partial}{\partial \lambda} L(\theta_1, \theta_2, \lambda) = \theta_1 + \theta_2 - 1 = 0 \tag{8.106}$$

From Equations 8.104 and 8.105 we find $2\theta_1 = -\lambda = 2\theta_2$, so $\theta_1 = \theta_2$. Also, from Eq. (8.106), we find $2\theta_1 = 1$. So $\boldsymbol{\theta}^* = (0.5, 0.5)$, as we claimed earlier. Furthermore, this is the global minimum since the objective is convex and the constraint is affine.

8.5.2 The KKT conditions

In this section, we generalize the concept of Lagrange multipliers to additionally handle inequality constraints.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

First consider the case where we have a single inequality constraint $g(\boldsymbol{\theta}) \leq 0$. To find the optimum, one approach would be to consider an unconstrained problem where we add the penalty as an infinite step function:

$$\hat{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \infty \mathbb{I}(g(\boldsymbol{\theta}) > 0) \tag{8.107}$$

However, this is a discontinuous function that is hard to optimize.

Instead, we create a lower bound of the form $\mu g(\boldsymbol{\theta})$, where $\mu \geq 0$. This gives us the following Lagrangian:

$$\mathcal{L}(\boldsymbol{\theta}, \mu) = \mathcal{L}(\boldsymbol{\theta}) + \mu g(\boldsymbol{\theta}) \tag{8.108}$$

Note that the step function can be recovered using

$$\hat{\mathcal{L}}(\boldsymbol{\theta}) = \max_{\mu \geq 0} \mathcal{L}(\boldsymbol{\theta}, \mu) = \begin{cases} \infty & \text{if } g(\boldsymbol{\theta}) > 0, \\ \mathcal{L}(\boldsymbol{\theta}) & \text{otherwise} \end{cases} \tag{8.109}$$

Thus our optimization problem becomes

$$\min_{\boldsymbol{\theta}} \max_{\mu \geq 0} \mathcal{L}(\boldsymbol{\theta}, \mu) \tag{8.110}$$

Now consider the general case where we have multiple inequality constraints, $\mathbf{g}(\boldsymbol{\theta}) \leq \mathbf{0}$, and multiple equality constraints, $\mathbf{h}(\boldsymbol{\theta}) = \mathbf{0}$. The **generalized Lagrangian** becomes

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \mathcal{L}(\boldsymbol{\theta}) - \sum_i \mu_i g_i(\boldsymbol{\theta}) - \sum_j \lambda_j h_j(\boldsymbol{\theta}) \tag{8.111}$$

(We are free to change $-\lambda_j h_j$ to $+\lambda_j h_j$ since the sign is arbitrary.) Our optimization problem becomes

$$\min_{\boldsymbol{\theta}} \max_{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \tag{8.112}$$

When $\mathcal{L}$, g and h are convex and satisfy a technical property called Slater's condition (see [BV04, Sec.5.2.3]) all critical points of this problem must satisfy the following criteria.

- All constraints are satisfied (this is called **feasibility**):

$$\mathbf{g}(\boldsymbol{\theta}) \leq \mathbf{0}, \ \mathbf{h}(\boldsymbol{\theta}) = \mathbf{0} \tag{8.113}$$

- The solution is a stationary point:

$$\nabla \mathcal{L}(\boldsymbol{\theta}^*) - \sum_i \mu_i \nabla g_i(\boldsymbol{\theta}^*) - \sum_j \lambda_i \nabla h_j(\boldsymbol{\theta}^*) = \mathbf{0} \tag{8.114}$$

- The penalty for the inequality constraint points in the right direction (this is called **dual feasibility**):

$$\boldsymbol{\mu} \geq \mathbf{0} \tag{8.115}$$

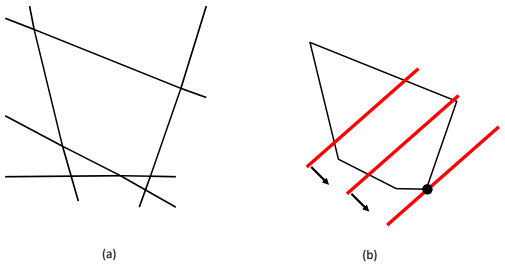


Figure 8.23: (a) A convex polytope in 2d defined by the intersection of linear constraints. (b) Depiction of the feasible set as well as the linear objective function. The red line is a level set of the objective, and the arrow indicates the direction in which it is improving. We see that the optimal solution lies at a vertex of the polytope.

- The Lagrange multipliers pick up any slack in the inactive constraints, i.e., either $\mu_i = 0$ or $g_i(\boldsymbol{\theta}^*) = 0$, so

$$\boldsymbol{\mu} \odot \mathbf{g} = \mathbf{0} \tag{8.116}$$

  This is called **complementary slackness**.

To see why the last condition holds, consider (for simplicity) the case of a single inequality constraint, $g(\boldsymbol{\theta}) \leq 0$. Either it is **active**, meaning $g(\boldsymbol{\theta}) = 0$, or it is inactive, meaning $g(\boldsymbol{\theta}) < 0$. In the active case, the solution lies on the constraint boundary, and $g(\boldsymbol{\theta}) = 0$ becomes an equality constraint; then we have $\nabla \mathcal{L} = \mu \nabla g$ for some constant $\mu \neq 0$, because of Eq. (8.99). In the inactive case, the solution is not on the constraint boundary; we still have $\nabla \mathcal{L} = \mu \nabla g$, but now $\mu = 0$.

These are called called the **Karush-Kuhn-Tucker** (**KKT**) conditions. If $\mathcal{L}$ is a convex function, and the constraints define a convex set, the KKT conditions are sufficient for (global) optimality, as well as necessary.

8.5.3 Linear programming

Consider optimizing a linear function subject to linear constraints. When written in **standard form**, this can be represented as

$$\min_{\boldsymbol{\theta}} \mathbf{c}^\top \boldsymbol{\theta} \qquad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \mathbf{b}, \ \boldsymbol{\theta} \geq 0 \tag{8.117}$$

The feasible set defines a convex **polytope**, which is a convex set defined as the intersection of half spaces. See Fig. 8.23(a) for a 2d example. Fig. 8.23(b) shows a linear cost function that decreases as we move to the bottom right. We see that the lowest point that is in the feasible set is a vertex. In fact, it can be proved that the optimum point always occurs at a vertex of the polytope, assuming the solution is unique. If there are multiple solutions, the line will be parallel to a face. There may also be no optima inside the feasible set; in this case, the problem is said to be infeasible.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.5.3.1 The simplex algorithm

It can be shown that the optima of an LP occur at vertices of the polytope defining the feasible set (see Fig. 8.23(b) for an example). The **simplex algorithm** solves LPs by moving from vertex to vertex, each time seeking the edge which most improves the objective.

In the worst-case scenario, the simplex algorithm can take time exponential in D, although in practice it is usually very efficient. There are also various polynomial-time algorithms, such as the interior point method, although these are often slower in practice.

8.5.3.2 Applications

There are many applications of linear programming in science, engineering and business. It is also useful in some machine learning problems. For example, Sec. 11.4.2.1 shows how to use it to solve robust linear regression. It is also useful for state estimation in graphical models (see e.g., [SGJ11]).

8.5.4 Quadratic programming

Consider minimizing a quadratic objective subject to linear equality and inequality constraints. This kind of problem is known as a **quadratic program** or **QP**, and can be written as follows:

$$\min_{\boldsymbol{\theta}} \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{H}\boldsymbol{\theta} + \mathbf{c}^\top \boldsymbol{\theta} \quad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \mathbf{b},\ \mathbf{A}_{eq}\boldsymbol{\theta} = \mathbf{b}_{eq} \tag{8.118}$$

If $\mathbf{H}$ is positive semidefinite, then this is a convex optimization problem.

8.5.4.1 Example: 2d quadratic objective with linear inequality constraints

As a concrete example, suppose we want to minimize

$$\mathcal{L}(\boldsymbol{\theta}) = (\theta_1 - \frac{3}{2})^2 + (\theta_2 - \frac{1}{8})^2 = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{H}\boldsymbol{\theta} + \mathbf{c}^\top \boldsymbol{\theta} + \text{const} \tag{8.119}$$

where $\mathbf{H} = 2\mathbf{I}$ and $\mathbf{c} = -(3, 1/4)$, subject to

$$|\theta_1| + |\theta_2| \leq 1 \tag{8.120}$$

See Fig. 8.22(b) for an illustration.

We can rewrite the constraints as

$$\theta_1 + \theta_2 \leq 1,\ \ \theta_1 - \theta_2 \leq 1,\ \ -\theta_1 + \theta_2 \leq 1,\ \ -\theta_1 - \theta_2 \leq 1 \tag{8.121}$$

which we can write more compactly as

$$\mathbf{A}\boldsymbol{\theta} - \mathbf{b} \leq \mathbf{0} \tag{8.122}$$

where $\mathbf{b} = \mathbf{1}$ and

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 1 \\ -1 & -1 \end{pmatrix} \tag{8.123}$$

This is now in the standard QP form.

From the geometry of the problem, shown in Fig. 8.22(b), we see that the constraints corresponding to the two left faces of the diamond) are inactive (since we are trying to get as close to the center of the circle as possible, which is outside of, and to the right of, the constrained feasible region). Denoting $g_i(\boldsymbol{\theta})$ as the inequality constraint corresponding to row i of $\mathbf{A}$, this means $g_3(\boldsymbol{\theta}^*) > 0$ and $g_4(\boldsymbol{\theta}^*) > 0$, and hence, by complementarity, $\mu_3^* = \mu_4^* = 0$. We can therefore remove these inactive constraints.

From the KKT conditions we know that

$$\mathbf{H}\boldsymbol{\theta} + \mathbf{c} + \mathbf{A}^\top \boldsymbol{\mu} = \mathbf{0} \tag{8.124}$$

Using these for the actively constrained subproblem, we get

$$\begin{pmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \end{pmatrix} \begin{pmatrix} \theta_1 \\ \theta_2 \\ \mu_1 \\ \mu_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 1/4 \\ 1 \\ 1 \end{pmatrix} \tag{8.125}$$

Hence the solution is

$$\boldsymbol{\theta}_* = (1, 0)^\top, \boldsymbol{\mu}_* = (0.625, 0.375, 0, 0)^\top \tag{8.126}$$

Notice that the optimal value of $\boldsymbol{\theta}$ occurs at one of the vertices of the ℓ_1 "ball" (the diamond shape).

8.5.4.2 Applications

There are several applications of quadratic programming in ML. In Sec. 11.5, we show how to use it for sparse linear regression. And in Sec. 17.5, we show how to use it for SVMs (support vector machines).

8.5.5 Mixed integer linear programming *

Integer linear programming or **ILP** corresponds to minimizing a linear objective, subject to linear constraints, where the optimization variables are discrete integers instead of reals. In standard form, the problem is as follows:

$$\min_{\boldsymbol{\theta}} \mathbf{c}^\top \boldsymbol{\theta} \quad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \mathbf{b}, \ \boldsymbol{\theta} \geq 0, \boldsymbol{\theta} \in \mathbb{Z}^D \tag{8.127}$$

where $\mathbb{Z}$ is the set of integers. If some of the optimization variables are real-valued, it is called a **mixed ILP**, often called a **MIP** for short. (If all of the variables are real-valued, it becomes a standard LP.)

MIPs have a large number of applications, such as in vehicle routing, scheduling and packing. They are also useful for some ML applications, such as formally verifying the behavior of certain kinds of deep neural networks [And+18], and proving robustness properties of DNNs to adversarial (worst-case) perturbations [TXT19].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.6 Proximal gradient method *

We are often interested in optimizing an objective of the form

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.128}$$

where $\mathcal{L}_s$ is differentiable (smooth), and $\mathcal{L}_r$ is convex but not necessarily differentiable (i.e., it may be non-smooth or "rough"). For example, $\mathcal{L}_s$ might be the negative log likelihood (NLL), and $\mathcal{L}_r$ might be an indicator function that is infinite if a constraint is violated (see Sec. 8.6.1), or $\mathcal{L}_r$ might be the ℓ_1 norm of some parameters (see Sec. 8.6.2), or $\mathcal{L}_r$ might measure how far the parameters are from a set of allowed quantized values (see Sec. 8.6.3).

One way to tackle such problems is to use the **proximal gradient method** (see e.g., [PB+14; PSW15]). Roughly speaking, this takes a step of size η in the direction of $\mathcal{L}_s$, and then projects the resulting update into a space that respects $\mathcal{L}_r$. More precisely, the update is as follows

$$\boldsymbol{\theta}_{t+1} = \text{prox}_{\eta_t \mathcal{L}_r}(\boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\boldsymbol{\theta}_t)) \tag{8.129}$$

where $\text{prox}_{\eta f}(\boldsymbol{\theta})$ is the **proximal operator** of $\mathcal{L}_r$ (scaled by η) evaluated at $\boldsymbol{\theta}$:

$$\text{prox}_{\eta \mathcal{L}_r}(\boldsymbol{\theta}) \triangleq \underset{\mathbf{z}}{\text{argmin}} \left(\mathcal{L}_r(\mathbf{z}) + \frac{1}{2\eta} ||\mathbf{z} - \boldsymbol{\theta}||_2^2 \right) \tag{8.130}$$

(The factor of $\frac{1}{2}$ is an arbitrary convention.) We can rewrite the proximal operator as solving a constrained opimtization problem, as follows:

$$\text{prox}_{\eta \mathcal{L}_r}(\boldsymbol{\theta}) = \underset{\mathbf{z}}{\text{argmin}}\, \mathcal{L}_r(\mathbf{z}) \quad \text{s.t.} \quad ||\mathbf{z} - \boldsymbol{\theta}||_2 \leq \rho \tag{8.131}$$

where the bound ρ depends on the scaling factor η. Thus we see that the proximal projection minimizes the function while staying close to (i.e., proximal to) the current iterate. We give some examples below.

8.6.1 Projected gradient descent

Suppose we want to solve the problem

$$\underset{\boldsymbol{\theta}}{\text{argmin}}\, \mathcal{L}_s(\boldsymbol{\theta}) \quad \text{s.t.} \quad \boldsymbol{\theta} \in \mathcal{C} \tag{8.132}$$

where $\mathcal{C}$ is a convex set. For example, we may have the **box constraints** $\mathcal{C} = \{\boldsymbol{\theta} : \mathbf{l} \leq \boldsymbol{\theta} \leq \mathbf{u}\}$, where we specify lower and upper bounds on each element. These bounds can be infinite for certain elements if we don't want to constrain values along that dimension. For example, if we just want to ensure the parameters are non-negative, we set $l_d = 0$ and $u_d = \infty$ for each dimension d.

We can convert the constrained optimization problem into an unconstrained one by adding a **penalty term** to the original objective:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.133}$$

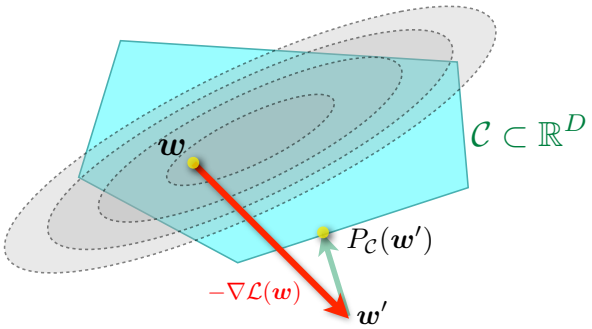


Figure 8.24: Illustration of projected gradient descent. $\mathbf{w}$ is the current parameter estimate, $\mathbf{w}'$ is the update after a gradient step, and $P_{\mathcal{C}}(\mathbf{w}')$ projects this onto the constraint set $\mathcal{C}$. From https://bit.ly/3eJ3BhZ Used with kind permission of Martin Jaggi.

where $\mathcal{L}_r(\boldsymbol{\theta})$ is the indicator function for the convex set $\mathcal{C}$, i.e.,

$$\mathcal{L}_r(\boldsymbol{\theta}) = I_{\mathcal{C}}(\boldsymbol{\theta}) = \begin{cases} 0 & \text{if } \boldsymbol{\theta} \in \mathcal{C} \\ \infty & \text{if } \boldsymbol{\theta} \notin \mathcal{C} \end{cases} \tag{8.134}$$

We can use proximal gradient descent to solve Eq. (8.133). The proximal operator for the indicator function is equivalent to projection onto the set $\mathcal{C}$:

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}' \in \mathcal{C}}{\text{argmin}} \, ||\boldsymbol{\theta}' - \boldsymbol{\theta}||_2 \tag{8.135}$$

This method is known as **projected gradient descent**. See Fig. 8.24 for an illustration.

For example, consider the box constraints $\mathcal{C} = \{\boldsymbol{\theta} : \mathbf{l} \leq \boldsymbol{\theta} \leq \mathbf{u}\}$. The projection operator in this case can be computed elementwise by simply thresholding at the boundaries:

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta})_d = \begin{cases} l_d & \text{if } \theta_d \leq l_d \\ x_d & \text{if } l_d \leq \theta_d \leq u_d \\ u_d & \text{if } \theta_d \geq u_d \end{cases} \tag{8.136}$$

For example, if we want to ensure all elements are non-negative, we can use

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \boldsymbol{\theta}_+ = [\max(\theta_1, 0), \ldots, \max(\theta_D, 0)] \tag{8.137}$$

See Sec. 11.5.9.2 for an application of this method to sparse linear regression.

8.6.2 Proximal operator for ℓ_1-norm regularizer

Consider a linear predictor of the form $f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{d=1}^{D} \theta_d x_d$. If we have $\theta_d = 0$ for any dimension d, we ignore the corresponding feature x_d. This is a form of **feature selection**, which can be

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

useful both as a way to reduce overfitting as well as way to improve model interpretability. We can encourage weights to be zero (and not just small) by penalizing the ℓ_1 norm,

$$||\boldsymbol{\theta}||_1 = \sum_{d=1}^{D} |\theta_d| \tag{8.138}$$

This is called a **sparsity inducing regularizer**.

To see why this induces sparsity, consider two possible parameter vectors, one which is sparse, $\boldsymbol{\theta} = (1, 0)$, and one which is non-sparse, $\boldsymbol{\theta}' = (1/\sqrt{2}, 1/\sqrt{2})$. Both have the same ℓ_2 norm

$$||(1,0)||_2^2 = ||(1/\sqrt{2}, 1/\sqrt{2})||_2^2 = 1 \tag{8.139}$$

Hence ℓ_2 regularization (Sec. 4.5.3) will not favor the sparse solution over the dense solution. However, when using ℓ_1 regularization, the sparse solution is cheaper, since

$$||(1,0)||_1 = 1 < ||(1/\sqrt{2}, 1/\sqrt{2})||_1 = \sqrt{2} \tag{8.140}$$

See Sec. 11.5 for more details on sparse regression.

If we combine this regularizer with our smooth loss, we get

$$\mathcal{L}(\boldsymbol{\theta}) = \text{NLL}(\boldsymbol{\theta}) + \lambda||\boldsymbol{\theta}||_1 \tag{8.141}$$

We can optimize this objective using proximal gradient descent. The key question is how to compute the prox operator for the function $f(\boldsymbol{\theta}) = ||\boldsymbol{\theta}||_1$. Since this function decomposes over dimensions d, the proximal projection can be computed componentwise. From Eq. (8.130), with $\eta = 1$, we have

$$\text{prox}_{\lambda f}(\theta) = \underset{\theta_0}{\text{argmin}}\, |\theta_0| + \frac{1}{2\lambda}(z-\theta)^2 = \underset{\theta_0}{\text{argmin}}\, \lambda|\theta_0| + \frac{1}{2}(z-\theta)^2 \tag{8.142}$$

In Sec. 11.5.3, we show that the solution to this is given by

$$\text{prox}_{\lambda f}(\theta) = \begin{cases} \theta - \lambda & \text{if } \theta \geq \lambda \\ 0 & \text{if } |\theta| \leq \lambda \\ \theta + \lambda & \text{if } \theta \leq -\lambda \end{cases} \tag{8.143}$$

This is known as the **soft thresholding operator**, since values less than λ in absolute value are set to 0 (thresholded), but in a continuous way. Note that soft thresholding can be written more compactly as

$$\text{SoftThreshold}(\theta, \lambda) = \text{sign}(\theta)\,(|\theta| - \lambda)_+ \tag{8.144}$$

where $\theta_+ = \max(\theta, 0)$ is the positive part of θ. In the vector case, we perform this elementwise:

$$\text{SoftThreshold}(\boldsymbol{\theta}, \lambda) = \text{sign}(\boldsymbol{\theta}) \odot (|\boldsymbol{\theta}| - \lambda)_+ \tag{8.145}$$

See Sec. 11.5.9.3 for an application of this method to sparse linear regression.

8.6.3 Proximal operator for quantization

In some applications (e.g., when training deep neural networks to run on memory-limited **edge devices**, such as mobile phones) we want to ensure that the parameters are **quantized**. For example, in the extreme case where each parameter can only be -1 or +1, the state space becomes $\mathcal{C} = \{-1, +1\}^D$.

Let us define a regularizer that measures distance to the nearest quantized version of the parameter vector:

$$\mathcal{L}_r(\boldsymbol{\theta}) = \inf_{\boldsymbol{\theta}_0 \in \mathcal{C}} ||\boldsymbol{\theta} - \boldsymbol{\theta}_0||_1 \tag{8.146}$$

(We could also use the ℓ_2 norm.) In the case of $\mathcal{C} = \{-1, +1\}^D$, this becomes

$$\mathcal{L}_r(\boldsymbol{\theta}) = \sum_{d=1}^{D} \inf_{[\theta_0]_d \in \{\pm 1\}} |\theta_d - [\theta_0]_d| = \sum_{d=1}^{D} \min\{|\theta_d - 1|, |\theta_d + 1|\} = ||\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})||_1 \tag{8.147}$$

Let us define the corresponding quantization operator to be

$$q(\boldsymbol{\theta}) = \text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \text{argmin}\, \mathcal{L}_r(\boldsymbol{\theta}) = \text{sign}(\boldsymbol{\theta}) \tag{8.148}$$

The core difficulty with quantized learning is that quantization is not a differentiable operation. A popular solution to this is to use the **straight-through estimator**, which uses the approximation $\frac{\partial \mathcal{L}}{\partial q(\boldsymbol{\theta})} \approx \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ (see e.g., [Yin+19b]). The corresponding update can be done in two steps: first compute the gradient vector at the quantized version of the current parameters, and then update the unconstrained parameters using this approximate gradient:

$$\tilde{\boldsymbol{\theta}}_t = \text{proj}_{\mathcal{C}}(\boldsymbol{\theta}_t) = q(\boldsymbol{\theta}_t) \tag{8.149}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\tilde{\boldsymbol{\theta}}_t) \tag{8.150}$$

When applied to $\mathcal{C} = \{-1, +1\}^D$, this is known as the **binary connect** method [CBD15].

We can get better results using proximal gradient descent, in which we treat quantization as a regularizer, rather than a hard constraint; this is known as **ProxQuant** [BWL19]. The update becomes

$$\tilde{\boldsymbol{\theta}}_t = \text{prox}_{\lambda \mathcal{L}_r}\left(\boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\boldsymbol{\theta}_t)\right) \tag{8.151}$$

In the case that $\mathcal{C} = \{-1, +1\}^D$, one can show that the proximal operator is a generalization of the soft thresholding operator in Eq. (8.145):

$$\text{prox}_{\lambda \mathcal{L}_r}(\boldsymbol{\theta}) = \text{SoftThreshold}(\boldsymbol{\theta}, \lambda, \text{sign}(\boldsymbol{\theta})) \tag{8.152}$$

$$= \text{sign}(\boldsymbol{\theta}) + \text{sign}(\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})) \odot (|\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})| - \lambda)_+ \tag{8.153}$$

This can be generalized to other forms of quantization; see [Yin+19b] for details.

8.7 Bound optimization *

In this section, we consider a class of algorithms known as **bound optimization** or **MM** algorithms. In the context of minimization, MM stands for **majorize-minimize**. In the context of maximization, MM stands for **minorize-maximize**. We will discuss a special case of MM, known as **expectation maximization** or **EM**, in Sec. 8.7.2.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

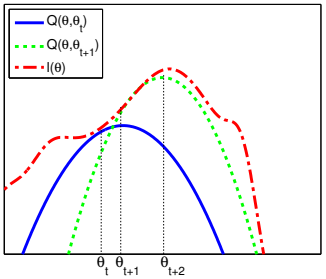


Figure 8.25: Illustration of a bound optimization algorithm. Adapted from Figure 9.14 of [Bis06]. Generated by code at figures.probml.ai/book1/8.25.

8.7.1 The general algorithm

In this section, we give a brief outline of MM methods. (More details can be found in e.g., [HL04; Mai15; SBP17; Nad+19],) To be consistent with the literature, we assume our goal is to *maximize* some function $\text{LL}(\boldsymbol{\theta})$, such as the log likelihood, wrt its parameters $\boldsymbol{\theta}$. The basic approach in MM algorithms is to construct a **surrogate function** $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ which is a tight lowerbound to $\text{LL}(\boldsymbol{\theta})$ such that $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \leq \text{LL}(\boldsymbol{\theta})$ and $Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \text{LL}(\boldsymbol{\theta}^t)$. If these conditions are met, we say that Q minorizes LL. We then perform the following update at each step:

$$\boldsymbol{\theta}^{t+1} = \operatorname*{argmax}_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \tag{8.154}$$

This guarantees us monotonic increases in the original objective:

$$\text{LL}(\boldsymbol{\theta}^{t+1}) \geq Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}^t) \geq Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \text{LL}(\boldsymbol{\theta}^t) \tag{8.155}$$

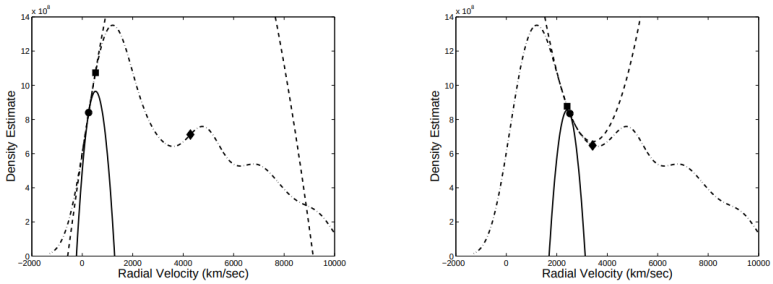
where the first inequality follows since $Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}')$ is a lower bound on $\text{LL}(\boldsymbol{\theta}^{t+1})$ for any $\boldsymbol{\theta}'$; the second inequality follows from Eq. (8.154); and the final equality follows the tightness property. As a consequence of this result, if you do not observe monotonic increase of the objective, you must have an error in your math and/or code. This is a surprisingly powerful debugging tool.

This process is sketched in Fig. 8.25. The dashed red curve is the original function (e.g., the log-likelihood of the observed data). The solid blue curve is the lower bound, evaluated at $\boldsymbol{\theta}^t$; this touches the objective function at $\boldsymbol{\theta}^t$. We then set $\boldsymbol{\theta}^{t+1}$ to the maximum of the lower bound (blue curve), and fit a new bound at that point (dotted green curve). The maximum of this new bound becomes $\boldsymbol{\theta}^{t+2}$, etc.

If Q is a quadratic lower bound, the overall method is similar to Newton's method, which repeatedly fits and then optimizes a quadratic approximation, as shown in Fig. 8.14(a). The difference is that optimizing Q is guaranteed to lead to an improvement in the objective, even if it is not convex, whereas Newton's method may overshoot or lead to a decrease in the objective, as shown in Fig. 8.26, since it is a quadratic approximation and not a bound.

8.7.2 The EM algorithm

In this section, we discuss the **expectation maximization** (**EM**) algorithm [DLR77; MK97], which is an algorithm designed to compute the MLE or MAP parameter estimate for probability models



(a) Overshooting. (b) Seeking the wrong root.

Figure 8.26: The quadratic lower bound of an MM algorithm (solid) and the quadratic approximation of Newton's method (dashed) superimposed on an empirical density esitmate (dotted). The starting point of both algorithms is the circle. The square denotes the outcome of one MM update. The diamond denotes the outcome of one Newton update. (a) Newton's method overshoots the-se, global maximum. (b) Newton's method results in a reduction of the objective. From Figure 4 of [FT05]. Used with kind permission of Carlo Tomasi.

that have **missing data** and/or **hidden variables**. We let $\mathbf{y}_n$ be the visible data for example n, and $\mathbf{z}_n$ be the hidden data.

The basic idea behind EM is to alternate between estimating the hidden variables (or missing values) during the **E step** (expectation step), and then using the fully observed data to compute the MLE during the **M step** (maximization step). Of course, we need to iterate this process, since the expected values depend on the parameters, but the parameters depend on the expected values.

In Sec. 8.7.2.1, we show that EM is an MM algorithm, which implies that this iterative procedure will converge to a local maximum of the log likelihood. The speed of convergence depends on the amount of missing data, which affects the tightness of the bound [XJ96; MD97; SRG03; KKS20].

There are many applications of EM in ML. In this book, we consider several examples:

- In Sec. 8.7.3, we show how to use EM to fit mixture models, such as Gaussian mixtures.
- In Sec. 8.7.4, we show how to use EM to fit a multivariate Gaussian, when we have missing data.
- In Sec. 11.4.1.1, we show to use EM to fit robust linear regression models.

8.7.2.1 Lower bound

The goal of EM is to maximize the log likelihood of the observed data:

$$\text{LL}(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log p(\mathbf{y}_n|\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{\mathbf{z}_n} p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta}) \right] \tag{8.156}$$

where $\mathbf{y}_n$ are the visible variables and $\mathbf{z}_n$ are the hidden variables. Unfortunately this is hard to optimize, since the log cannot be pushed inside the sum.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

EM gets around this problem as follows. First, consider a set of arbitrary distributions $q_n(\mathbf{z}_n)$ over each hidden variable $\mathbf{z}_n$. The observed data log likelihood can be written as follows:

$$\text{LL}(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \frac{p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})}{q_n(\mathbf{z}_n)} \right] \tag{8.157}$$

Using Jensen's inequality (Eq. (6.34)), we can push the log (which is a concave function) inside the expectation to get the following lower bound on the log likelihood:

$$\text{LL}(\boldsymbol{\theta}) \geq \sum_n \sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \log \frac{p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})}{q_n(\mathbf{z}_n)} \tag{8.158}$$

$$= \sum_n \underbrace{\mathbb{E}_{q_n}\left[\log p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})\right] + \mathbb{H}(q_n)}_{\mathcal{E}(\boldsymbol{\theta}, q_n|\mathbf{y}_n)} \tag{8.159}$$

$$= \sum_n \mathcal{E}(\boldsymbol{\theta}, q_n|\mathbf{y}_n) \triangleq \mathcal{E}(\boldsymbol{\theta}, \{q_n\}|\mathcal{D}) \tag{8.160}$$

where $\mathbb{H}(q)$ is the entropy of probability distribution q, and $\mathcal{E}(\boldsymbol{\theta}, \{q_n\}|\mathcal{D})$ is called the **evidence lower bound** or **ELBO**, since it is a lower bound on the log marginal likelihood, $\log p(\mathbf{y}_{1:N}|\boldsymbol{\theta})$, also called the evidence. Optimizing this bound is the basis of variational inference, as we discuss in Sec. 4.6.8.3.

The **variational EM** algorithm alternates between maximizing the lower bound wrt the distributions q_n in the E step, and the model parameters $\boldsymbol{\theta}$ in the M step, as proposed in [NH98]. The original EM algorithm is a special case where the distributions q_n computed in the E step have a specific form, as we discuss below.

8.7.2.2 E step

We see that the lower bound is a sum of N terms, each of which has the following form:

$$\mathcal{E}(\boldsymbol{\theta}, q_n|\mathbf{y}_n) = \sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \log \frac{p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})}{q_n(\mathbf{z}_n)} \tag{8.161}$$

$$= \sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \log \frac{p(\mathbf{z}_n|\mathbf{y}_n, \boldsymbol{\theta}) p(\mathbf{y}_n|\boldsymbol{\theta})}{q_n(\mathbf{z}_n)} \tag{8.162}$$

$$= \sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \log \frac{p(\mathbf{z}_n|\mathbf{y}_n, \boldsymbol{\theta})}{q_n(\mathbf{z}_n)} + \sum_{\mathbf{z}_n} q_n(\mathbf{z}_n) \log p(\mathbf{y}_n|\boldsymbol{\theta}) \tag{8.163}$$

$$= -\mathbb{KL}\left(q_n(\mathbf{z}_n) \| p(\mathbf{z}_n|\mathbf{y}_n, \boldsymbol{\theta})\right) + \log p(\mathbf{y}_n|\boldsymbol{\theta}) \tag{8.164}$$

where $\mathbb{KL}\left(q\|p\right) \triangleq \sum_z q(z) \log \frac{q(z)}{p(z)}$ is the Kullback-Leibler divergence (or KL divergence for short) between probability distributions q and p. We discuss this in more detail in Sec. 6.2, but the key property we need here is that $\mathbb{KL}\left(q\|p\right) \geq 0$ and $\mathbb{KL}\left(q\|p\right) = 0$ iff $q = p$. Hence we can maximize the lower bound $\mathcal{E}(\boldsymbol{\theta}, \{q_n\}|\mathcal{D})$ wrt $\{q_n\}$ by setting each one to $q_n^* = p(\mathbf{z}_n|\mathbf{y}_n, \boldsymbol{\theta})$. This is called the **E step**. This ensures the ELBO is a tight lower bound:

$$\mathcal{E}(\boldsymbol{\theta}, \{q_n^*\}|\mathcal{D}) = \sum_n \log p(\mathbf{y}_n|\boldsymbol{\theta}) = \text{LL}(\boldsymbol{\theta}|\mathcal{D}) \tag{8.165}$$

To see how this connects to bound optimization, let us define

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) = \mathcal{E}(\boldsymbol{\theta}, \{p(\mathbf{z}_n|\mathbf{y}_n; \boldsymbol{\theta}^t)\}) \tag{8.166}$$

Then we have $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \leq \text{LL}(\boldsymbol{\theta})$ and $Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \text{LL}(\boldsymbol{\theta}^t)$, as required.

However, if we cannot compute the posteriors $p(\mathbf{z}_n|\mathbf{y}_n; \boldsymbol{\theta}^t)$ exactly, we can still use an approximate distribution $q(\mathbf{z}_n|\mathbf{y}_n; \boldsymbol{\theta}^t)$; this will yield a non-tight lower-bound on the log-likelihood. This generalized version of EM is known as variational EM [NH98].

8.7.2.3 M step

In the M step, we need to maximize $\mathcal{E}(\boldsymbol{\theta}, \{q_n^t\})$ wrt $\boldsymbol{\theta}$, where the q_n^t are the distributions computed in the E step at iteration t. Since the entropy terms $\mathbb{H}(q_n)$ are constant wrt $\boldsymbol{\theta}$, so we can drop them in the M step. We are left with

$$\text{LL}^t(\boldsymbol{\theta}) = \sum_n \mathbb{E}_{q_n^t(\mathbf{z}_n)} \left[\log p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})\right] \tag{8.167}$$

This is called the **expected complete data log likelihood**. If the joint probability is in the exponential family (Sec. 12.2), we can rewrite this as

$$\text{LL}^t(\boldsymbol{\theta}) = \sum_n \mathbb{E}\left[\mathbf{t}(\mathbf{y}_n, \mathbf{z}_n)^\top \boldsymbol{\theta} - A(\boldsymbol{\theta})\right] = \sum_n (\mathbb{E}\left[\mathbf{t}(\mathbf{y}_n, \mathbf{z}_n)\right]^\top \boldsymbol{\theta} - A(\boldsymbol{\theta})) \tag{8.168}$$

where $\mathbb{E}\left[\mathbf{t}(\mathbf{y}_n, \mathbf{z}_n)\right]$ are called the **expected sufficient statistics**.

In the M step, we maximize the expected complete data log likelihood to get

$$\boldsymbol{\theta}^{t+1} = \arg\max_{\boldsymbol{\theta}} \sum_n \mathbb{E}_{q_n^t} \left[\log p(\mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})\right] \tag{8.169}$$

In the case of the exponential family, the maximization can be solved in closed-form by matching the moments of the expected sufficient statistics (Sec. 12.2.4).

We see from the above that the E step does not in fact need to return the full set of posterior distributions $\{q(\mathbf{z}_n)\}$, but can instead just return the sum of the expected sufficient statistics, $\sum_n \mathbb{E}_{q(\mathbf{z}_n)} \left[\mathbf{t}(\mathbf{y}_n, \mathbf{z}_n)\right]$. This will become clearer in the examples below.

8.7.3 Example: EM for a GMM

In this section, we show how to use the EM algorithm to compute MLE and MAP estimates of the parameters for a Gaussian mixture model (GMM).

8.7.3.1 E step

The E step simply computes the **responsibility** of cluster k for generating data point n, as estimated using the current parameter estimates $\boldsymbol{\theta}^{(t)}$:

$$r_{nk}^{(t)} = p^*(z_n = k|\boldsymbol{\theta}^{(t)}) = \frac{\pi_k^{(t)} p(\mathbf{y}_n|\boldsymbol{\theta}_k^{(t)})}{\sum_{k'} \pi_{k'}^{(t)} p(\mathbf{y}_n|\boldsymbol{\theta}_{k'}^{(t)})} \tag{8.170}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.7.3.2 M step

The M step maximizes the expected complete data log likelihood, given by

$$\text{LL}^t(\boldsymbol{\theta}) = \mathbb{E}\left[\sum_n \log p(z_n|\boldsymbol{\pi}) + \log p(\mathbf{y}_n|z_n,\boldsymbol{\theta})\right] \tag{8.171}$$

$$= \mathbb{E}\left[\sum_n \log\left(\prod_k \pi_k^{z_{nk}}\right) + \log\left(\prod_k \mathcal{N}(\mathbf{y}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)^{z_{nk}}\right)\right] \tag{8.172}$$

$$= \sum_n \sum_k \mathbb{E}\left[z_{nk}\right] \log \pi_k + \sum_n \sum_k \mathbb{E}\left[z_{nk}\right] \log \mathcal{N}(\mathbf{y}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k) \tag{8.173}$$

$$= \sum_n \sum_k r_{nk}^{(t)} \log(\pi_k) - \frac{1}{2}\sum_n \sum_k r_{nk}^{(t)}\left[\log|\boldsymbol{\Sigma}_k| + (\mathbf{y}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{y}_n - \boldsymbol{\mu}_k)\right] + \text{const} \tag{8.174}$$

where $z_{nk} = \mathbb{I}(z_n = k)$ is a one-hot encoding of the categorical value z_n. This objective is just a weighted version of the standard problem of computing the MLEs of an MVN (see Sec. 4.2.6). One can show that the new parameter estimates are given by

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} \mathbf{y}_n}{r_k^{(t)}} \tag{8.175}$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} (\mathbf{y}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{y}_n - \boldsymbol{\mu}_k^{(t+1)})^\top}{r_k^{(t)}}$$

$$= \frac{\sum_n r_{nk}^{(t)} \mathbf{y}_n \mathbf{y}_n^\top}{r_k^{(t)}} - \boldsymbol{\mu}_k^{(t+1)} (\boldsymbol{\mu}_k^{(t+1)})^\top \tag{8.176}$$

where $r_k^{(t)} \triangleq \sum_n r_{nk}^{(t)}$ is the weighted number of points assigned to cluster k. The mean of cluster k is just the weighted average of all points assigned to cluster k, and the covariance is proportional to the weighted empirical scatter matrix.

The M step for the mixture weights is simply a weighted form of the usual MLE:

$$\pi_k^{(t+1)} = \frac{1}{N}\sum_n r_{nk}^{(t)} = \frac{r_k^{(t)}}{N} \tag{8.177}$$

8.7.3.3 Example

An example of the algorithm in action is shown in Fig. 8.27 where we fit some 2d data with a 2 component GMM. The data set, from [Bis06], is derived from measurements of the Old Faithful geyser in Yellowstone National Park. In particular, we plot the time to next eruption in minutes versus the duration of the eruption in minutes. The data was standardized, by removing the mean and dividing by the standard deviation, before processing; this often helps convergence. We start with $\boldsymbol{\mu}_1 = (-1, 1)$, $\boldsymbol{\Sigma}_1 = \mathbf{I}$, $\boldsymbol{\mu}_2 = (1, -1)$, $\boldsymbol{\Sigma}_2 = \mathbf{I}$. We then show the cluster assignments, and corresponding mixture components, at various iterations.

For more details on applying GMMs for clustering, see Sec. 21.4.1.

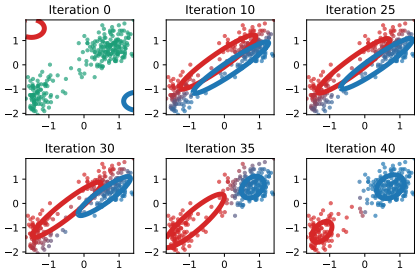
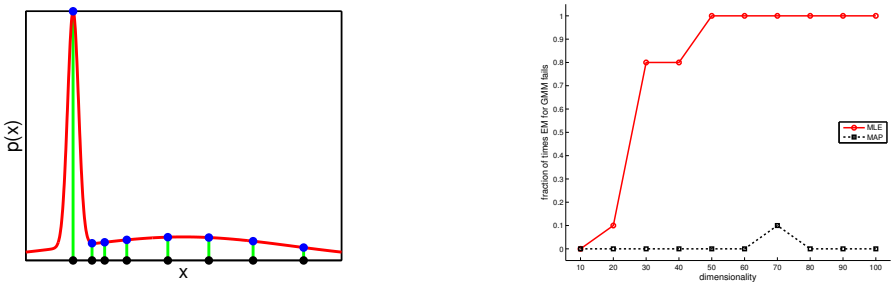


Figure 8.27: Illustration of the EM for a GMM applied to the Old Faithful data. The degree of redness indicates the degree to which the point belongs to the red cluster, and similarly for blue; thus purple points have a roughly 50/50 split in their responsibilities to the two clusters. Adapted from [Bis06] Figure 9.8. Generated by code at figures.probml.ai/book1/8.27.



(a) *(b)*

Figure 8.28: (a) Illustration of how singularities can arise in the likelihood function of GMMs. Here $K = 2$, but the first mixture component is a narrow spike (with $\sigma_1 \approx 0$) centered on a single data point x_1. Adapted from Figure 9.7 of [Bis06]. Generated by code at figures.probml.ai/book1/8.28. (b) Illustration of the benefit of MAP estimation vs ML estimation when fitting a Gaussian mixture model. We plot the fraction of times (out of 5 random trials) each method encounters numerical problems vs the dimensionality of the problem, for $N = 100$ samples. Solid red (upper curve): MLE. Dotted black (lower curve): MAP. Generated by code at figures.probml.ai/book1/8.28.

8.7.3.4 MAP estimation

Computing the MLE of a GMM often suffers from numerical problems and overfitting. To see why, suppose for simplicity that $\boldsymbol{\Sigma}_k = \sigma_k^2 \mathbf{I}$ for all k. It is possible to get an infinite likelihood by assigning one of the centers, say $\boldsymbol{\mu}_k$, to a single data point, say $\mathbf{y}_n$, since then the likelihood of that data point is given by

$$\mathcal{N}(\mathbf{y}_n | \boldsymbol{\mu}_k = \mathbf{y}_n, \sigma_k^2 \mathbf{I}) = \frac{1}{\sqrt{2\pi\sigma_k^2}} e^0 \tag{8.178}$$

Hence we can drive this term to infinity by letting $\sigma_k \to 0$, as shown in Fig. 8.28(a). We call this the "collapsing variance problem".

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

An easy solution to this is to perform MAP estimation. Fortunately, we can still use EM to find this MAP estimate. Our goal is now to maximize the expected complete data log-likelihood plus the log prior:

$$\text{LL}^t(\boldsymbol{\theta}) \left[\sum_n \sum_k r_{nk}^{(t)} \log \pi_{nk} + \sum_n \sum_k r_{nk}^{(t)} \log p(\mathbf{y}_n|\boldsymbol{\theta}_k) \right] + \log p(\boldsymbol{\pi}) + \sum_k \log p(\boldsymbol{\theta}_k) \tag{8.179}$$

Note that the E step remains unchanged, but the M step needs to be modified, as we now explain.

For the prior on the mixture weights, it is natural to use a Dirichlet prior (Sec. 4.6.3.2), $\boldsymbol{\pi} \sim \text{Dir}(\boldsymbol{\alpha})$, since this is conjugate to the categorical distribution. The MAP estimate is given by

$$\tilde{\pi}_k^{(t+1)} = \frac{r_k^{(t)} + \alpha_k - 1}{N + \sum_k \alpha_k - K} \tag{8.180}$$

If we use a uniform prior, $\alpha_k = 1$, this reduces to the MLE.

For the prior on the mixture components, let us consider a conjugate prior of the form

$$p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \text{NIW}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k | \breve{\mathbf{m}}, \breve{\kappa}, \breve{\nu}, \breve{\mathbf{S}}) \tag{8.181}$$

This is called the **Normal-Inverse-Wishart distribution** (see the sequel to this book, [Mur22], for details.) Suppose we set the hyper-parameters for $\boldsymbol{\mu}$ to be $\breve{\kappa}= 0$, so that the $\boldsymbol{\mu}_k$ are unregularized; this the prior will only influence our estimate of $\boldsymbol{\Sigma}_k$. In this case, the MAP estimates are given by

$$\tilde{\boldsymbol{\mu}}_k^{(t+1)} = \hat{\boldsymbol{\mu}}_k^{(t+1)} \tag{8.182}$$

$$\tilde{\boldsymbol{\Sigma}}_k^{(t+1)} = \frac{\breve{\mathbf{S}} + \hat{\boldsymbol{\Sigma}}_k^{(t+1)}}{\breve{\nu} + r_k^{(t)} + D + 2} \tag{8.183}$$

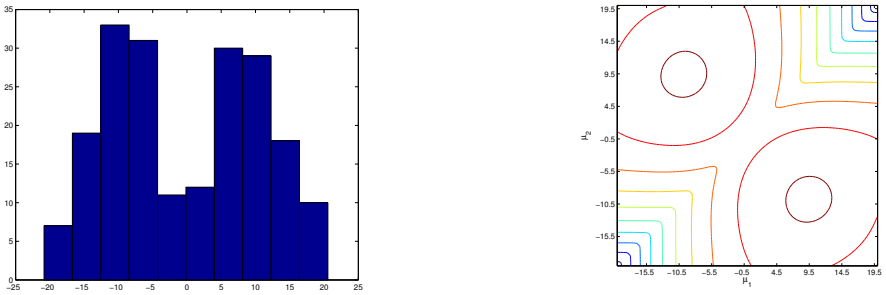
where $\hat{\boldsymbol{\mu}}_k$ is the MLE for $\boldsymbol{\mu}_k$ from Eq. (8.175), and $\hat{\boldsymbol{\Sigma}}_k$ is the MLE for $\boldsymbol{\Sigma}_k$ from Eq. (8.176).

Now we discuss how to set the prior covariance, $\breve{\mathbf{S}}$. One possibility (suggested in [FR07, p163]) is to use

$$\breve{\mathbf{S}}= \frac{1}{K^{1/D}} \text{diag}(s_1^2, \ldots, s_D^2) \tag{8.184}$$

where $s_d = (1/N) \sum_{n=1}^N (x_{nd} - \overline{x}_d)^2$ is the pooled variance for dimension d. The parameter $\breve{\nu}$ controls how strongly we believe this prior. The weakest prior we can use, while still being proper, is to set $\breve{\nu}= D + 2$, so this is a common choice.

We now illustrate the benefits of using MAP estimation instead of ML estimation in the context of GMMs. We apply EM to some synthetic data with $N = 100$ samples in D dimensions, using either ML or MAP estimation. We count the trial as a "failure" if there are numerical issues involving singular matrices. For each dimensionality, we conduct 5 random trials. The results are illustrated in Fig. 8.28(b). We see that as soon as D becomes even moderately large, ML estimation crashes and burns, whereas MAP with an appropriate prior estimation rarely encounters numerical problems.



(a) (b)

Figure 8.29: Left: $N = 200$ data points sampled from a mixture of 2 Gaussians in 1d, with $\pi_k = 0.5$, $\sigma_k = 5$, $\mu_1 = -10$ and $\mu_2 = 10$. Right: Likelihood surface $p(\mathcal{D}|\mu_1, \mu_2)$, with all other parameters set to their true values. We see the two symmetric modes, reflecting the unidentifiability of the parameters. Generated by code at figures.probml.ai/book1/8.29.

8.7.3.5 Nonconvexity of the NLL

The likelihood for a mixture model is given by

$$\text{LL}(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{z_n=1}^{K} p(\mathbf{y}_n, z_n|\boldsymbol{\theta}) \right] \tag{8.185}$$

In general, this will have multiple modes, and hence there will not be a unique global optimum.

Fig. 8.29 illustrates this for a mixture of 2 Gaussians in 1d. We see that there are two equally good global optima, corresponding to two different labelings of the clusters, one in which the left peak corresponds to $z = 1$, and one in which the left peak corresponds to $z = 2$. This is called the **label switching problem**; see Sec. 21.4.1.2 for more details.

The question of how many modes there are in the likelihood function is hard to answer. There are $K!$ possible labelings, but some of the peaks might get merged, depending on how far apart the μ_k are. Nevertheless, there can be an exponential number of modes. Consequently, finding any global optimum is NP-hard [Alo+09; Dri+04]. We will therefore have to be satisfied with finding a local optimum. To find a good local optimum, we can use Kmeans++ (Sec. 21.3.4) to initialize EM.

8.7.4 Example: EM for an MVN with missing data

In Sec. 4.2.6, we explained how to compute the MLE for an MVN (multivariate normal) when we have a fully observed data matrix $\mathbf{Y}$. In this section, we consider the case where we have **missing data** or **partially observed data**. For example, we can think of the entries of $\mathbf{Y}$ as being answers to a survey; some of these answers may be unknown. (See Sec. 1.5.5 for a more general discussion of missing data.)

In this section, we make the missing at random (MAR) assumption, for simplicity. Under the MAR assumption, the log likelihood of the visible data has the form

$$\log p(\mathbf{X}|\boldsymbol{\theta}) = \sum_n \log p(\mathbf{x}_n|\boldsymbol{\theta}) = \sum_n \log \left[\int p(\mathbf{x}_n, \mathbf{z}_n|\boldsymbol{\theta}) d\mathbf{z}_n \right] \tag{8.186}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\mathbf{x}_n$ are the visible variables in case n, $\mathbf{z}_n$ are the hidden variables, and $\mathbf{y}_n = (\mathbf{z}_n, \mathbf{x}_n)$ are all the variables. Unfortunately, this objective is hard to maximize. since we cannot push the log inside the expectation. Fortunately, we can easily apply EM, as we explain below.

8.7.4.1 E step

Suppose we have the parameters $\boldsymbol{\theta}^{t-1}$ from the previous iteration. Then we can compute the expected complete data log likelihood at iteration t as follows:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) = \mathbb{E}\left[\sum_{n=1}^{N} \log \mathcal{N}(\mathbf{y}_n|\boldsymbol{\mu}, \boldsymbol{\Sigma})|\mathcal{D}, \boldsymbol{\theta}^{t-1}\right] \tag{8.187}$$

$$= -\frac{N}{2}\log|2\pi\boldsymbol{\Sigma}| - \frac{1}{2}\sum_n \mathbb{E}\left[(\mathbf{y}_n - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{y}_n - \boldsymbol{\mu})\right] \tag{8.188}$$

$$= -\frac{N}{2}\log|2\pi\boldsymbol{\Sigma}| - \frac{1}{2}\mathrm{tr}(\boldsymbol{\Sigma}^{-1}\sum_n \mathbb{E}\left[(\mathbf{y}_n - \boldsymbol{\mu})(\mathbf{y}_n - \boldsymbol{\mu})^\top\right] \tag{8.189}$$

$$= -\frac{N}{2}\log|\boldsymbol{\Sigma}| - \frac{ND}{2}\log(2\pi) - \frac{1}{2}\mathrm{tr}(\boldsymbol{\Sigma}^{-1}\mathbb{E}\left[\mathbf{S}(\boldsymbol{\mu})\right]) \tag{8.190}$$

where

$$\mathbb{E}\left[\mathbf{S}(\boldsymbol{\mu})\right] \triangleq \sum_n \left(\mathbb{E}\left[\mathbf{y}_n\mathbf{y}_n^\top\right] + \boldsymbol{\mu}\boldsymbol{\mu}^\top - 2\boldsymbol{\mu}\mathbb{E}\left[\mathbf{y}_n\right]^\top\right) \tag{8.191}$$

(We drop the conditioning of the expectation on $\mathcal{D}$ and $\boldsymbol{\theta}^{t-1}$ for brevity.) We see that we need to compute $\sum_n \mathbb{E}\left[\mathbf{y}_n\right]$ and $\sum_n \mathbb{E}\left[\mathbf{y}_n\mathbf{y}_n^\top\right]$; these are the expected sufficient statistics.

To compute these quantities, we use the results from Sec. 3.2.3. We have

$$p(\mathbf{z}_n|\mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_n|\mathbf{m}_n, \mathbf{V}_n) \tag{8.192}$$

$$\mathbf{m}_n \triangleq \boldsymbol{\mu}_h + \boldsymbol{\Sigma}_{hv}\boldsymbol{\Sigma}_{vv}^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_v) \tag{8.193}$$

$$\mathbf{V}_n \triangleq \boldsymbol{\Sigma}_{hh} - \boldsymbol{\Sigma}_{hv}\boldsymbol{\Sigma}_{vv}^{-1}\boldsymbol{\Sigma}_{vh} \tag{8.194}$$

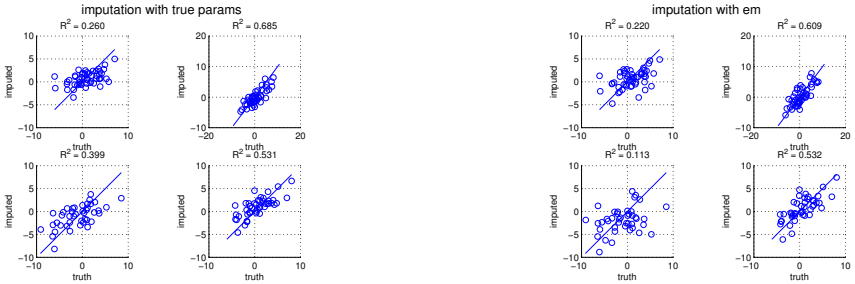
where we partition $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ into blocks based on the hidden and visible indices h and v. Hence the expected sufficient statistics are

$$\mathbb{E}\left[\mathbf{y}_n\right] = (\mathbb{E}\left[\mathbf{z}_n\right]; \mathbf{x}_n) = (\mathbf{m}_n; \mathbf{x}_n) \tag{8.195}$$

To compute $\mathbb{E}\left[\mathbf{y}_n\mathbf{y}_n^\top\right]$, we use the result that $\mathrm{Cov}\left[\mathbf{y}\right] = \mathbb{E}\left[\mathbf{y}\mathbf{y}^\top\right] - \mathbb{E}\left[\mathbf{y}\right]\mathbb{E}\left[\mathbf{y}^\top\right]$. Hence

$$\mathbb{E}\left[\mathbf{y}_n\mathbf{y}_n^\top\right] = \mathbb{E}\left[\begin{pmatrix}\mathbf{z}_n \\ \mathbf{x}_n\end{pmatrix}\begin{pmatrix}\mathbf{z}_n^\top & \mathbf{x}_n^\top\end{pmatrix}\right] = \begin{pmatrix}\mathbb{E}\left[\mathbf{z}_n\mathbf{z}_n^\top\right] & \mathbb{E}\left[\mathbf{z}_n\right]\mathbf{x}_n^\top \\ \mathbf{x}_n\mathbb{E}\left[\mathbf{z}_n\right]^\top & \mathbf{x}_n\mathbf{x}_n^\top\end{pmatrix} \tag{8.196}$$

$$\mathbb{E}\left[\mathbf{z}_n\mathbf{z}_n^\top\right] = \mathbb{E}\left[\mathbf{z}_n\right]\mathbb{E}\left[\mathbf{z}_n\right]^\top + \mathbf{V}_n \tag{8.197}$$



(a) *(b)*

Figure 8.30: Illustration of data imputation. (a) Scatter plot of true values vs imputed values using true parameters. (b) Same as (a), but using parameters estimated with EM. We just show the first four variables, for brevity. Generated by code at figures.probml.ai/book1/8.30.

8.7.4.2 M step

By solving $\nabla Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t-1)}) = \mathbf{0}$, we can show that the M step is equivalent to plugging these ESS into the usual MLE equations to get

$$\boldsymbol{\mu}^t = \frac{1}{N} \sum_n \mathbb{E}\left[\mathbf{y}_n\right] \tag{8.198}$$

$$\boldsymbol{\Sigma}^t = \frac{1}{N} \sum_n \mathbb{E}\left[\mathbf{y}_n \mathbf{y}_n^\top\right] - \boldsymbol{\mu}^t (\boldsymbol{\mu}^t)^\top \tag{8.199}$$

Thus we see that EM is *not* equivalent to simply replacing variables by their expectations and applying the standard MLE formula; that would ignore the posterior variance and would result in an incorrect estimate. Instead we must compute the expectation of the sufficient statistics, and plug that into the usual equation for the MLE.

8.7.4.3 Initialization

To get the algorithm started, we can compute the MLE based on those rows of the data matrix that are fully observed. If there are no such rows, we can just estimate the diagonal terms of $\boldsymbol{\Sigma}$ using the observed marginal statistics. We are then ready to start EM.

8.7.4.4 Example

As an example of this procedure in action, let us reconsider the imputation problem from Sec. 3.2.4, which had $N = 100$ 10-dimensional data cases, with 50% missing data. Let us fit the parameters using EM. Call the resulting parameters $\hat{\boldsymbol{\theta}}$. We can use our model for predictions by computing $\mathbb{E}\left[\mathbf{z}_n | \mathbf{x}_n, \hat{\boldsymbol{\theta}}\right]$. Fig. 8.30 indicates that the results obtained using the learned parameters are almost as good as with the true parameters. Not surprisingly, performance improves with more data, or as the fraction of missing data is reduced.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

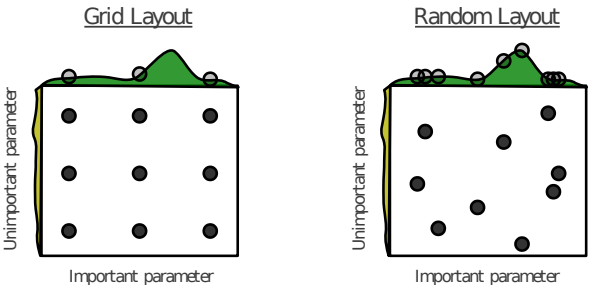


Figure 8.31: Illustration of grid search (left) vs random search (right). From Figure 1 of [BB12]. Used with kind permission of James Bergstra.

8.8 Blackbox and derivative free optimization

In some optimization problems, the objective function is a **blackbox**, meaning that its functional form is unknown. This means we cannot use gradient-based methods to optimize it. Instead, solving such problems require **blackbox optimization** (**BBO**) methods, also called **derivative free optimization** (**DFO**).

In ML, this kind of problem often arises when performing model selection. For example, suppose we have some hyper-parameters, $\boldsymbol{\lambda} \in \boldsymbol{\Lambda}$, which control the type or complexity of a model. We often define the objective function $\mathcal{L}(\boldsymbol{\lambda})$ to be the loss on a validation set (see Sec. 4.5.4). Since the validation loss depends on the optimal model parameters, which are computed using a complex algorithm, this objective function is effectively a blackbox.[4] Furthermore, evaluating this blackbox is expensive, since it requires training the model before computing the validation loss.

8.8.1 Grid search and random search

A simple (and very common) approach is to definite a discrete grid of possible hyper-parameters, and then to exhaustively enumerate all combinations, as shown in Fig. 8.31(left). This is called **grid search**. If some dimensions don't affect the output very much, using a fine grid along that dimension is wasteful, and better results can sometimes be found more quickly using **random search**, as shown in Fig. 8.31(right).

8.8.2 Simulated annealing *

Simulated annealing [KJV83; LA87] is a **stochastic local search** algorithm that attempts to find the global maximum of a black-box function $f(\mathbf{x}) = -\mathcal{E}(\mathbf{x})$, where $\mathcal{E}()$ is known as the "energy function" (note that this can be positive or negative). SA can be used for both discrete and continuous optimization.

4. If the optimal parameters are computed using a gradient-based optimizer, we can "unroll" the gradient steps, to create a deep circuit that maps from the training data to the optimal parameters and hence to the validation loss. We can then optimize through the optimizer (see e.g., [Fra+17]. However, this technique can only be applied in limited settings.

At each step, we sample a new state according to some proposal distribution $\mathbf{x}' \sim q(\cdot|\mathbf{x}_t)$. For real-valued parameters, this is often simply a random walk proposal centered on the current iterate, $\mathbf{x}' = \mathbf{x}_t + \boldsymbol{\epsilon}_{t+1}$, where $\boldsymbol{\epsilon}_{t+1} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$. (The matrix $\boldsymbol{\Sigma}$ is often diagonal, and may be updated over time using the method in [Cor+87].) Having proposed a new state, we compute the acceptance probability

$$\alpha_{t+1} = \exp\left((f(\mathbf{x}') - f(\mathbf{x}_t))/T_t\right) \tag{8.200}$$

where T_t is the temperature of the system. We then accept the new state (i.e., set $\mathbf{x}_{t+1} = \mathbf{x}'$) with probability $\min(1, \alpha_{t+1})$, otherwise we stay in the current state (i.e., set $\mathbf{x}_{t+1} = \mathbf{x}_t$). This means that if the new state has lower energy (is more probable), we will definitely accept it, but if it has higher energy (is less probable), we might still accept, depending on the current temperature. Thus the algorithm allows "downhill" moves in probability space (uphill in energy space), but less frequently as the temperature drops.

Fig. 2.9 gives an example of a function defined on a discrete state space with 3 possible values. At high temperatures, $T \gg 1$, the surface is approximately flat, and hence it is easy to move around (i.e., to avoid local optima). As the temperature cools, the largest peaks become larger, and the smallest peaks disappear. By cooling slowly enough, it is possible to "track" the largest peak, and thus find the global optimum (minimum energy state). This is an example of a **continuation method**.

The rate at which the temperature changes over time is called the **cooling schedule**. It has been shown [Haj88] that if one cools according to a logarithmic schedule, $T_t \propto 1/\log(t+1)$, then the method is guaranteed to find the global optimum under certain assumptions. However, this schedule is often too slow. In practice it is common to use an **exponential cooling schedule** of the form $T_{t+1} = \gamma T_t$, where $\gamma \in (0, 1]$ is the cooling rate. Cooling too quickly means one can get stuck in a local maximum, but cooling too slowly just wastes time. The best cooling schedule is difficult to determine; this is one of the main drawbacks of simulated annealing.

8.8.3 Model-based blackbox optimization *

Many model selection problems (and other blackbox optimization problems) involve a combinatorially large search space. For example, in **neural architecture search**, the task is to pick the best neural net graph structure (see e.g., [EMH19]). In this case, the state space can be viewed as the set of edge labels, $\mathcal{X} = \{0, 1\}^{N \times N}$, where N is the number of nodes, and $x_{ij} = 1$ iff there is an edge from i to j. (Note that not all combinations are feasible, so we may have $\mathcal{L}(\mathbf{x}) = \infty$ for some settings.)

In such exponentially large spaces, random search or blind local search is usually not very effective. It is usually better to try to learn up a *model* of the loss landscape, which can be used to guide subsequent search steps. (This is analogous to building up a quadratic approximation to the loss surface in second order gradient methods.)

This model could be an implicit (non-parametric) model, as used by population-based methods (aka **evolutionary algorithms**, see e.g., [Sör15]), or it could be an explicit (parametric) model. There are many possible choices for parametric model. For example, the **cross-entropy method** (see e.g., [Rub97; RK04; Boe+05]) uses a multivariate Gaussian to represent a distribution over promising points in a real-valued configuration space; **probabilistic model-building genetic algorithms** [Pel08; Bal17] uses generic probability models (such as graphical models) to represent a distribution over promising discrete configurations; **Bayesian optimization** (see e.g., [Sha+16a]) uses regression models (such as Gaussian processes, discussed in Chapter 17) to provide a cheap **surrogate function**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

to the expensive blackbox function; and so on. For more information on blackbox optimization, see e.g, [KW19b; HA17].

8.9 Exercises

Exercise 8.1 [Subderivative of the hinge loss function]

Let $f(x) = (1 - x)_+$ be the hinge loss function, where $(z)_+ = \max(0, z)$. What are $\partial f(0)$, $\partial f(1)$, and $\partial f(2)$?

Exercise 8.2 [EM for the Student distribution]

Derive the EM equations for computing the MLE for a multivariate Student distribution. Consider the case where the dof parameter is known and unknown separately. Hint: write the Student distribution as a scale mixture of Gaussians.

Part II

Linear models

9 Linear discriminant analysis

9.1 Introduction

In this chapter, we consider classification models of the following form:

$$p(y = c|\mathbf{x}; \boldsymbol{\theta}) = \frac{p(\mathbf{x}|y = c; \boldsymbol{\theta})p(y = c; \boldsymbol{\theta})}{\sum_{c'} p(\mathbf{x}|y = c'; \boldsymbol{\theta})p(y = c; \boldsymbol{\theta})} \tag{9.1}$$

The term $p(y = c; \boldsymbol{\theta})$ is the prior over class labels, and the term $p(\mathbf{x}|y = c; \boldsymbol{\theta})$ is called the **class conditional density** for class c.

The overall model is called a **generative classifier**, since it specifies a way to *generate* the features $\mathbf{x}$ for each class c, by sampling from $p(\mathbf{x}|y = c; \boldsymbol{\theta})$. By contrast, a **discriminative classifier** directly models the class posterior $p(y|\mathbf{x}; \boldsymbol{\theta})$. We discuss the pros and cons of these two approaches to classification in Sec. 9.4.

If we choose the class conditional densities in a special way, we will see that the resulting posterior over classes is a linear function of $\mathbf{x}$, i.e., $\log p(y = c|\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}^\top \mathbf{x} + \text{const}$, where $\mathbf{w}$ is derived from $\boldsymbol{\theta}$. Thus the overall method is called **linear discriminant analysis** or **LDA**.[1]

9.2 Gaussian discriminant analysis

In this section, we consider a generative classifier where the class conditional densities are multivariate Gaussians:

$$p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \tag{9.2}$$

The corresponding class posterior therefore has the form

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) \propto \pi_c \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \tag{9.3}$$

where $\pi_c = p(y = c)$ is the prior probability of label c. (Note that we can ignore the normalization constant in the denominator of the posterior, since it is independent of c.) We call this model **Gaussian discriminant analysis** or **GDA**.

1. This term is rather confusing for two reaons. First, LDA is a generative, not discriminative, classifier. Second, LDA also stands for "latent Dirichlet allocation", which is a popular unsupervised generative model for bags of words [BNJ03].

(a) *(b)*

Figure 9.1: (a) Some 2d data from 3 different classes. (b) Fitting 2d Gaussians to each class. Generated by code at figures.probml.ai/book1/9.1.

(a) *(b)*

Figure 9.2: Gaussian discriminant analysis fit to data in Fig. 9.1. (a) Unconstrained covariances induce quadratic decision boundaries. (b) Tied covariances induce linear decision boundaries. Generated by code at figures.probml.ai/book1/9.2.

9.2.1 Quadratic decision boundaries

From Eq. (9.3), we see that the log posterior over class labels is given by

$$\log p(y=c|\mathbf{x},\boldsymbol{\theta}) = \log \pi_c - \frac{1}{2}\log|2\pi\boldsymbol{\Sigma}_c| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}-\boldsymbol{\mu}_c) \tag{9.4}$$

This is called the **discriminant function**. We see that the decision boundary between any two classes, say c and c', will be a quadratic function of $\mathbf{x}$. Hence this is known as **quadratic discriminant analysis** (QDA).

For example, consider the 2d data from 3 different classes in Fig. 9.1a. We fit full covariance Gaussian class-conditionals (using the method explained in Sec. 9.2.4), and plot the results in Fig. 9.1b. We see that the features for the blue class are somewhat correlated, whereas the features for the green class are independent, and the features for the red class are independent and isotropic (spherical covariance). In Fig. 9.2a, we see that the resulting decision boundaries are quadratic functions of $\mathbf{x}$.

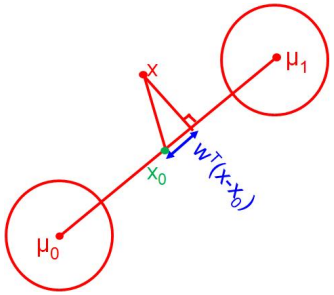


Figure 9.3: Geometry of LDA in the 2 class case where $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \mathbf{I}$.

9.2.2 Linear decision boundaries

Now we consider a special case of Gaussian discriminant analysis in which the covariance matrices are **tied** or **shared** across classes, so $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$. If $\boldsymbol{\Sigma}$ is independent of c, we can simplify Eq. (9.4) as follows:

$$\log p(y=c|\mathbf{x},\boldsymbol{\theta}) = \log \pi_c - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_c) + \text{const} \tag{9.5}$$

$$= \underbrace{\log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c}_{\gamma_c} + \mathbf{x}^\top \underbrace{\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c}_{\boldsymbol{\beta}_c} \underbrace{+\text{const} - \frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}}_{\kappa} \tag{9.6}$$

$$= \gamma_c + \mathbf{x}^\top \boldsymbol{\beta}_c + \kappa \tag{9.7}$$

The final term is independent of c, and hence is an irrelevant additive constant that can be dropped. Hence we see that the discriminant function is a linear function of $\mathbf{x}$, so the decision boundaries will be linear. Hence this method is called **linear discriminant analysis** or **LDA**. See Fig. 9.2b for an example.

9.2.3 The connection between LDA and logistic regression

In this section, we derive an interesting connection between LDA and logistic regression, which we introduced in Sec. 2.5.3. From Eq. (9.7) we can write

$$p(y=c|\mathbf{x},\boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^\top \mathbf{x}+\gamma_c}}{\sum_{c'} e^{\boldsymbol{\beta}_{c'}^\top \mathbf{x}+\gamma_{c'}}} = \frac{e^{\mathbf{w}_c^\top [1,\mathbf{x}]}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top [1,\mathbf{x}]}} \tag{9.8}$$

where $\mathbf{w}_c = [\gamma_c, \boldsymbol{\beta}_c]$. We see that Eq. (9.8) has the same form as the multinomial logistic regression model. The key difference is that in LDA, we first fit the Gaussians (and class prior) to maximize the joint likelihood $p(\mathbf{x}, y|\boldsymbol{\theta})$, as discussed in Sec. 9.2.4, and then we derive $\mathbf{w}$ from $\boldsymbol{\theta}$. By contrast, in logistic regression, we estimate $\mathbf{w}$ directly to maximize the conditional likelihood $p(y|\mathbf{x},\mathbf{w})$. In general, these can give different results (see Exercise 10.3).

To gain further insight into Eq. (9.8), let us consider the binary case. In this case, the posterior is

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

given by

$$p(y=1|\mathbf{x},\boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_1^\top \mathbf{x}+\gamma_1}}{e^{\boldsymbol{\beta}_1^\top \mathbf{x}+\gamma_1} + e^{\boldsymbol{\beta}_0^\top \mathbf{x}+\gamma_0}} = \frac{1}{1+e^{(\boldsymbol{\beta}_0-\boldsymbol{\beta}_1)^\top \mathbf{x}+(\gamma_0-\gamma_1)}} \tag{9.9}$$

$$= \sigma\left((\boldsymbol{\beta}_1-\boldsymbol{\beta}_0)^\top \mathbf{x} + (\gamma_1-\gamma_0)\right) \tag{9.10}$$

where $\sigma(\eta)$ refers to the sigmoid function.

Now

$$\gamma_1-\gamma_0 = -\frac{1}{2}\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_0 + \log(\pi_1/\pi_0) \tag{9.11}$$

$$= -\frac{1}{2}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0) + \log(\pi_1/\pi_0) \tag{9.12}$$

So if we define

$$\mathbf{w} = \boldsymbol{\beta}_1 - \boldsymbol{\beta}_0 = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0) \tag{9.13}$$

$$\mathbf{x}_0 = \frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0) - (\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)\frac{\log(\pi_1/\pi_0)}{(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)} \tag{9.14}$$

then we have $\mathbf{w}^\top \mathbf{x}_0 = -(\gamma_1-\gamma_0)$, and hence

$$p(y=1|\mathbf{x},\boldsymbol{\theta}) = \sigma(\mathbf{w}^\top(\mathbf{x}-\mathbf{x}_0)) \tag{9.15}$$

This has the same form as binary logistic regression. Hence the MAP decision rule is

$$\hat{y}(\mathbf{x}) = 1 \text{ iff } \mathbf{w}^\top \mathbf{x} > c \tag{9.16}$$

where $c = \mathbf{w}^\top \mathbf{x}_0$. If $\pi_0 = \pi_1 = 0.5$, then the threshold simplifies to $c = \frac{1}{2}\mathbf{w}^\top(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)$.

To interpret this equation geometrically, suppose $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}$. In this case, $\mathbf{w} = \sigma^{-2}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)$, which is parallel to a line joining the two centroids, $\boldsymbol{\mu}_0$ and $\boldsymbol{\mu}_1$. So we can classify a point based by projecting it onto this line, and then checking if the projection is closer to $\boldsymbol{\mu}_0$ or $\boldsymbol{\mu}_1$, as illustrated in Fig. 9.3. The question of how close it has to be depends on the prior over classes. If $\pi_1 = \pi_0$, then $\mathbf{x}_0 = \frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)$, which is halfway between the means. If we make $\pi_1 > \pi_0$, we have to be closer to $\boldsymbol{\mu}_0$ than halfway in order to pick class 0. And vice versa if $\pi_0 > \pi_1$. Thus we see that the class prior just changes the decision threshold, but not the overall shape of the decision boundary. (A similar argument applies in the multi-class case.)

9.2.4 Model fitting

We now discuss how to fit a GDA model using maximum likelihood estimation. The likelihood function is as follows

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} \text{Cat}(y_n|\boldsymbol{\pi}) \prod_{c=1}^{C} \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c)^{\mathbb{I}(y_n=c)} \tag{9.17}$$

Hence the log-likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \left[\sum_{n=1}^{N}\sum_{c=1}^{C}\mathbb{I}\left(y_n = c\right)\log \pi_c\right] + \sum_{c=1}^{C}\left[\sum_{n:y_n=c}\log \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right] \tag{9.18}$$

Thus we see that we can optimize $\boldsymbol{\pi}$ and the $(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ terms separately.

From Sec. 4.2.4, we have that the MLE for the class prior is $\hat{\pi}_c = \frac{N_c}{N}$. Using the results from Sec. 4.2.6, we can derive the MLEs for the Gaussians as follows:

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c}\sum_{n:y_n=c}\mathbf{x}_n \tag{9.19}$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c}\sum_{n:y_n=c}(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)^\top \tag{9.20}$$

Unfortunately the MLE for $\hat{\boldsymbol{\Sigma}}_c$ can easily overfit (i.e., the estimate may not be well-conditioned) if N_c is small compared to D, the dimensionality of the input features. We discuss some solutions to this below.

9.2.4.1 Tied covariances

If we force $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$ to be tied, we will get linear decision boundaries, as we have seen. This also usually results in a more reliable parameter estimate, since we can pool all the samples across classes:

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_{c=1}^{C}\sum_{n:y_n=c}(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)^\top \tag{9.21}$$

9.2.4.2 Diagonal covariances

If we force $\boldsymbol{\Sigma}_c$ to be diagonal, we reduce the number of parameters from $O(CD^2)$ to $O(CD)$, which avoids the overfitting problem. However, this loses the ability to capture correlations between the features. (This is known as the naive Bayes assumption, which we discuss further in Sec. 9.3.) Despite this approximation, this approach scales well to high dimensions.

We can further restrict the model capacity by using a shared (tied) diagonal covariace matrix. This is called "diagonal LDA" [BL04].

9.2.4.3 MAP estimation

Forcing the covariance matrix to be diagonal is a rather strong assumption. An alternative approach is to perform MAP estimation of a (shared) full covariance Gaussian, rather than using the MLE. Based on the results of Sec. 4.5.2, we find that the MAP estimate is

$$\hat{\boldsymbol{\Sigma}}_{\text{map}} = \lambda\text{diag}(\hat{\boldsymbol{\Sigma}}_{\text{mle}}) + (1-\lambda)\hat{\boldsymbol{\Sigma}}_{\text{mle}} \tag{9.22}$$

where λ controls the amount of regularization. This technique is known as **regularized discriminant analysis** or RDA [HTF09, p656].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

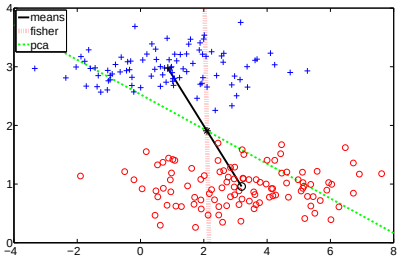


Figure 9.4: Example of Fisher's linear discriminant applied to data in 2d drawn from two classes. Dashed green line = first principal basis vector. Dotted red line = Fisher's linear discriminant vector. Solid black line joins the class-conditional means. Generated by code at figures.probml.ai/book1/9.4.

9.2.5 Nearest centroid classifier

If we assume a uniform prior over classes, we can compute the most probable class label as follows:

$$\hat{y}(\mathbf{x}) = \underset{c}{\operatorname{argmax}} \log p(y=c|\mathbf{x},\boldsymbol{\theta}) = \underset{c}{\operatorname{argmin}} (\mathbf{x}-\boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu}_c) \tag{9.23}$$

This is called the **nearest centroid classifier**, or **nearest class mean classifier** (**NCM**), since we are assigning $\mathbf{x}$ to the class with the closest $\boldsymbol{\mu}_c$, where distance is measured using (squared) Mahalanobis distance.

We can replace this with any other distance metric to get the decision rule

$$\hat{y}(\mathbf{x}) = \underset{c}{\operatorname{argmin}}\, d^2(\mathbf{x}, \boldsymbol{\mu}_c) \tag{9.24}$$

We discuss how to learn distance metrics in Sec. 16.2, but one simple approach is to use

$$d^2(\mathbf{x}, \boldsymbol{\mu}_c) = ||\mathbf{x}-\boldsymbol{\mu}_c||^2_{\mathbf{W}} = (\mathbf{x}-\boldsymbol{\mu}_c)^\top (\mathbf{W}\mathbf{W}^\top)(\mathbf{x}-\boldsymbol{\mu}_c) = ||\mathbf{W}(\mathbf{x}-\boldsymbol{\mu}_c)||^2 \tag{9.25}$$

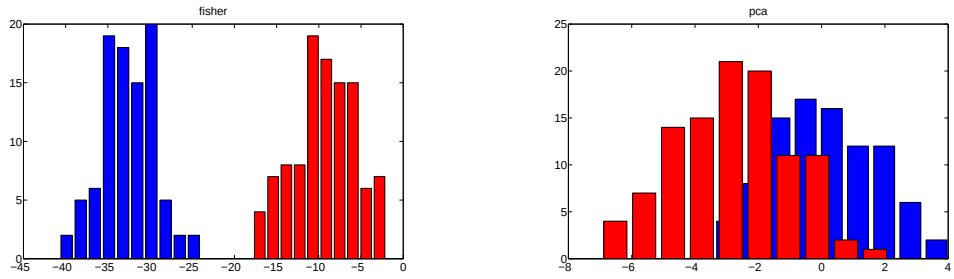
The corresponding class posterior becomes

$$p(y=c|\mathbf{x},\boldsymbol{\mu},\mathbf{W}) = \frac{\exp(-\frac{1}{2}||\mathbf{W}(\mathbf{x}-\boldsymbol{\mu}_c)||_2^2)}{\sum_{c'=1}^{C} \exp(-\frac{1}{2}||\mathbf{W}(\mathbf{x}-\boldsymbol{\mu}_{c'})||_2^2)} \tag{9.26}$$

We can optimize $\mathbf{W}$ using gradient descent applied to the discriminative loss. This is called **nearest class mean metric learning** [Men+12]. The advantage of this technique is that it can be used for **one-shot learning** of new classes, since we just need to see a single labeled prototype $\boldsymbol{\mu}_c$ per class (assuming we have learned a good $\mathbf{W}$ already).

9.2.6 Fisher's linear discriminant analysis *

Discriminant analysis is a generative approach to classification, which requires fitting an MVN to the features. As we have discussed, this can be problematic in high dimensions. An alternative approach is to reduce the dimensionality of the features $\mathbf{x} \in \mathbb{R}^D$ and then fit an MVN to the resulting



(a) (b)

Figure 9.5: Example of Fisher's linear discriminant. (a) Projection of points onto Fisher's vector in Fig. 9.4 shows good class separation. (b) Projection of points onto PCA vector in Fig. 9.4 shows poor class separation. Generated by code at figures.probml.ai/book1/9.5.

low-dimensional features $\mathbf{z} \in \mathbb{R}^K$. The simplest approach is to use a linear projection matrix, $\mathbf{z} = \mathbf{W}\mathbf{x}$, where $\mathbf{W}$ is a $K \times D$ matrix. One approach to finding $\mathbf{W}$ would be to use principal components analysis or PCA (Sec. 20.1). However, PCA is an unsupervised technique that does not take class labels into account. Thus the resulting low dimensional features are not necessarily optimal for classification, as illustrated in Fig. 9.4.

An alternative approach is to use gradient based methods to optimize the log likelihood, derived from the class posterior in the low dimensional space, as we discussed in Sec. 9.2.5.

A third approach (which relies on an eigendecomposition, rather than a gradient-based optimizer) is to find the matrix $\mathbf{W}$ such that the low-dimensional data can be classified as well as possible using a Gaussian class-conditional density model. The assumption of Gaussianity is reasonable since we are computing linear combinations of (potentially non-Gaussian) features. This approach is called **Fisher's linear discriminant analysis**, or **FLDA**.

FLDA is an interesting hybrid of discriminative and generative techniques. The drawback of this technique is that it is restricted to using $K \leq C - 1$ dimensions, regardless of D, for reasons that we will explain below. In the two-class case, this means we are seeking a single vector $\mathbf{w}$ onto which we can project the data. Below we derive the optimal $\mathbf{w}$ in the two-class case. We then generalize to the multi-class case, and finally we give a probabilistic interpretation of this technique.

9.2.6.1 Derivation of the optimal 1d projection

We now derive this optimal direction $\mathbf{w}$, for the two-class case, following the presentation of [Bis06, Sec 4.1.4]. Define the class-conditional means as

$$\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{n:y_n=1} \mathbf{x}_n, \; \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{n:y_n=2} \mathbf{x}_n \tag{9.27}$$

Let $m_k = \mathbf{w}^\top \boldsymbol{\mu}_k$ be the projection of each mean onto the line $\mathbf{w}$. Also, let $z_n = \mathbf{w}^\top \mathbf{x}_n$ be the projection of the data onto the line. The variance of the projected points is proportional to

$$s_k^2 = \sum_{n:y_n=k} (z_n - m_k)^2 \tag{9.28}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The goal is to find $\mathbf{w}$ such that we maximize the distance between the means, $m_2 - m_1$, while also ensuring the projected clusters are "tight", which we can do by minimizing their variance. This suggests the following objective:

$$J(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} \tag{9.29}$$

We can rewrite the right hand side of the above in terms of $\mathbf{w}$ as follows

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}} \tag{9.30}$$

where $\mathbf{S}_B$ is the between-class scatter matrix given by

$$\mathbf{S}_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \tag{9.31}$$

and $\mathbf{S}_W$ is the within-class scatter matrix, given by

$$\mathbf{S}_W = \sum_{n:y_n=1} (\mathbf{x}_n - \boldsymbol{\mu}_1)(\mathbf{x}_n - \boldsymbol{\mu}_1)^\top + \sum_{n:y_n=2} (\mathbf{x}_n - \boldsymbol{\mu}_2)(\mathbf{x}_n - \boldsymbol{\mu}_2)^\top \tag{9.32}$$

To see this, note that

$$\mathbf{w}^\top \mathbf{S}_B \mathbf{w} = \mathbf{w}^\top (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \mathbf{w} = (m_2 - m_1)(m_2 - m_1) \tag{9.33}$$

and

$$\begin{aligned} \mathbf{w}^\top \mathbf{S}_W \mathbf{w} &= \sum_{n:y_n=1} \mathbf{w}^\top (\mathbf{x}_n - \boldsymbol{\mu}_1)(\mathbf{x}_n - \boldsymbol{\mu}_1)^\top \mathbf{w} + \\ &\quad \sum_{n:y_n=2} \mathbf{w}^\top (\mathbf{x}_n - \boldsymbol{\mu}_2)(\mathbf{x}_n - \boldsymbol{\mu}_2)^\top \mathbf{w} \qquad (9.34) \\ &= \sum_{n:y_n=1} (z_n - m_1)^2 + \sum_{n:y_n=2} (z_n - m_2)^2 \qquad (9.35) \end{aligned}$$

Eq. (9.30) is a ratio of two scalars; we can take its derivative with respect to $\mathbf{w}$ and equate to zero. One can show (Exercise 9.1) that $J(\mathbf{w})$ is maximized when

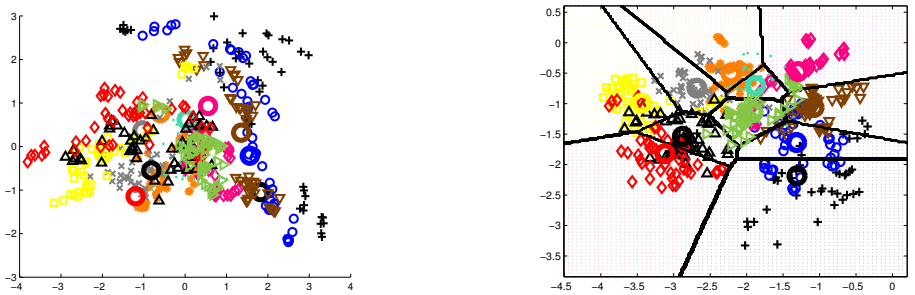
$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w} \tag{9.36}$$

where

$$\lambda = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}} \tag{9.37}$$

Eq. (9.36) is called a **generalized eigenvalue** problem. If $\mathbf{S}_W$ is invertible, we can convert it to a regular eigenvalue problem:

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w} \tag{9.38}$$



(a) (b)

Figure 9.6: (a) PCA projection of vowel data to 2d. (b) FLDA projection of vowel data to 2d. We see there is better class separation in the FLDA case. Adapted from Figure 4.11 of [HTF09]. Generated by code at figures.probml.ai/book1/9.6.

However, in the two class case, there is a simpler solution. In particular, since

$$\mathbf{S}_B\mathbf{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \mathbf{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \tag{9.39}$$

then, from Eq. (9.38) we have

$$\lambda\ \mathbf{w} = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \tag{9.40}$$

$$\mathbf{w} \propto \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \tag{9.41}$$

Since we only care about the directionality, and not the scale factor, we can just set

$$\mathbf{w} = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \tag{9.42}$$

This is the optimal solution in the two-class case. If $\mathbf{S}_W \propto \mathbf{I}$, meaning the pooled covariance matrix is isotropic, then $\mathbf{w}$ is proportional to the vector that joins the class means. This is an intuitively reasonable direction to project onto, as shown in Fig. 9.3.

9.2.6.2 Extension to higher dimensions and multiple classes

We can extend the above idea to multiple classes, and to higher dimensional subspaces, by finding a projection *matrix* $\mathbf{W}$ which maps from D to K. Let $\mathbf{z}_n = \mathbf{W}\mathbf{x}_n$ be the low dimensional projection of the n'th data point. Let $\mathbf{m}_c = \frac{1}{N_c}\sum_{n:y_n=c}\mathbf{z}_n$ be the corresponding mean for the c'th class and $\mathbf{m} = \frac{1}{N}\sum_{c=1}^{C} N_c\mathbf{m}_c$ be the overall mean, both in the low dimensional space. We define the following scatter matrices:

$$\tilde{\mathbf{S}}_W = \sum_{c=1}^{C}\sum_{n:y_n=c}(\mathbf{z}_n - \mathbf{m}_c)(\mathbf{z}_n - \mathbf{m}_c)^\top \tag{9.43}$$

$$\tilde{\mathbf{S}}_B = \sum_{c=1}^{C} N_c(\mathbf{m}_c - \mathbf{m})(\mathbf{m}_c - \mathbf{m})^\top \tag{9.44}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Finally, we define the objective function as maximizing the following:[2]

$$J(\mathbf{W}) = \frac{|\tilde{\mathbf{S}}_B|}{|\tilde{\mathbf{S}}_W|} = \frac{|\mathbf{W}^\top \mathbf{S}_B \mathbf{W}|}{|\mathbf{W}^\top \mathbf{S}_W \mathbf{W}|} \tag{9.45}$$

where $\mathbf{S}_W$ and $\mathbf{S}_B$ are defined in the original high dimensional space in the obvious way (namely using $\mathbf{x}_n$ instead of $\mathbf{z}_n$, $\boldsymbol{\mu}_c$ instead of $\mathbf{m}_c$, and $\boldsymbol{\mu}$ instead of $\mathbf{m}$). The solution can be shown [DHS01] to be $\mathbf{W} = \mathbf{S}_W^{-\frac{1}{2}} \mathbf{U}$, where $\mathbf{U}$ are the K leading eigenvectors of $\mathbf{S}_W^{-\frac{1}{2}} \mathbf{S}_B \mathbf{S}_W^{-\frac{1}{2}}$, assuming $\mathbf{S}_W$ is non-singular. (If it is singular, we can first perform PCA on all the data.)

Fig. 9.6 gives an example of this method applied to some $D = 10$ dimensional speech data, representing $C = 11$ different vowel sounds. We project to $K = 2$ dimensions in order to visualize the data. We see that FLDA gives better class separation than PCA.

Note that FLDA is restricted to finding at most a $K \leq C - 1$ dimensional linear subspace, no matter how large D, because the rank of the between class scatter matrix $\mathbf{S}_B$ is $C - 1$. (The -1 term arises because of the $\boldsymbol{\mu}$ term, which is a linear function of the $\boldsymbol{\mu}_c$.) This is a rather severe restriction which limits the usefulness of FLDA.

9.3 Naive Bayes classifiers

In this section, we discuss a simple generative approach to classification in which we assume the features are conditionally independent given the class label. This is called the **naive Bayes assumption**. The model is called "naive" since we do not expect the features to be independent, even conditional on the class label. However, even if the naive Bayes assumption is not true, it often results in classifiers that work well [DP97]. One reason for this is that the model is quite simple (it only has $O(CD)$ parameters, for C classes and D features), and hence it is relatively immune to overfitting.

More precisely, the naive Bayes assumption corresponds to using a class conditional density of the following form:

$$p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} p(x_d|y = c, \boldsymbol{\theta}_{dc}) \tag{9.46}$$

where $\boldsymbol{\theta}_{dc}$ are the parameters for the class conditional density for class c and feature d. Hence the posterior over class labels is given by

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{p(y = c|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_d|y = c, \boldsymbol{\theta}_{dc})}{\sum_{c'} p(y = c'|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_d|y = c', \boldsymbol{\theta}_{dc'})} \tag{9.47}$$

where π_c is the prior probability of class c, and $\boldsymbol{\theta} = (\boldsymbol{\pi}, \{\boldsymbol{\theta}_{dc}\})$ are all the parameters. This is known as a **naive Bayes classifier** or **NBC**.

9.3.1 Example models

We still need to specify the form of the probability distributions in Eq. (9.46). This depends on what type of feature x_d is. We give some examples below:

2. An alternative criterion that is sometimes used [Fuk90] is $J(\mathbf{W}) = \text{tr}\left\{\tilde{\mathbf{S}}_W^{-1} \tilde{\mathbf{S}}_B\right\} = \text{tr}\left\{(\mathbf{W}\mathbf{S}_W\mathbf{W}^\top)^{-1}(\mathbf{W}\mathbf{S}_B\mathbf{W}^\top)\right\}$.

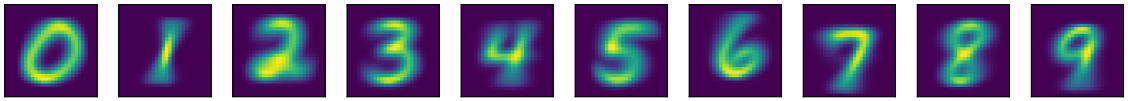


Figure 9.7: Visualization of the Bernoulli class conditional densities for a naive Bayes classifier fit to a binarized version of the MNIST dataset. Generated by code at figures.probml.ai/book1/9.7.

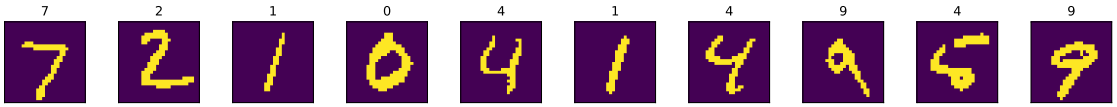


Figure 9.8: Visualization of the predictions made by the model in Fig. 9.7 when applied to the first 10 binarized MNIST test images. The 9'th example is incorrect. Generated by code at figures.probml.ai/book1/9.8.

- In the case of binary features, $x_d \in \{0, 1\}$, we can use the Bernoulli distribution: $p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Ber}(x_d|\theta_{dc})$, where θ_{dc} is the probability that $x_d = 1$ in class c. This is sometimes called the **multivariate Bernoulli naive Bayes** model. For example, Fig. 9.7 shows the estimated parameters for each class when we fit this model to a binarized version of MNIST. This approach does surprisingly well, and has a test set accuracy of 84.3%. (See Fig. 9.8 for some sample predictions.)
- In the case of categorical features, $x_d \in \{1, \ldots, K\}$, we can use the categorical distribution: $p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Cat}(x_d|\boldsymbol{\theta}_{dc})$, where θ_{dck} is the probability that $x_d = k$ given that $y = c$.
- In the case of real-valued features, $x_d \in \mathbb{R}$, we can use the univariate Gaussian distribution: $p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \mathcal{N}(x_d|\mu_{dc}, \sigma_{dc}^2)$, where μ_{dc} is the mean of feature d when the class label is c, and σ_{dc}^2 is its variance. (This is equivalent to Gaussian discriminant analysis using diagonal covariance matrices.)

9.3.2 Model fitting

In this section, we discuss how to fit a naive Bayes classifier using maximum likelihood estimation.

We can write the likelihood as follows:

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} \text{Cat}(y_n|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_{nd}|y_n, \boldsymbol{\theta}_d) \tag{9.48}$$

$$= \prod_{n=1}^{N} \text{Cat}(y_n|\boldsymbol{\pi}) \prod_{d=1}^{D} \prod_{c=1}^{C} p(x_{nd}|\boldsymbol{\theta}_{d,c})^{\mathbb{I}(y_n=c)} \tag{9.49}$$

so the log-likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \left[\sum_{n=1}^{N}\sum_{c=1}^{C} \mathbb{I}\left(y_n = c\right) \log \pi_c\right] + \sum_{c=1}^{C}\sum_{d=1}^{D}\left[\sum_{n:y_n=c} \log p(x_{nd}|\boldsymbol{\theta}_{dc})\right] \tag{9.50}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We see that this decomposes into a term for $\boldsymbol{\pi}$, and CD terms for each $\boldsymbol{\theta}_{dc}$:

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \log p(\mathcal{D}_y|\boldsymbol{\pi}) + \sum_c \sum_d \log p(\mathcal{D}_{dc}|\boldsymbol{\theta}_{dc}) \tag{9.51}$$

where $\mathcal{D}_y = \{y_n : n = 1 : N\}$ are all the labels, and $\mathcal{D}_{dc} = \{x_{nd} : y_n = c\}$ are all the values of feature d for examples from class c. Hence we can estimate these parameters separately.

In Sec. 4.2.4, we show that the MLE for $\boldsymbol{\pi}$ is the vector of empirical counts, $\hat{\pi}_c = \frac{N_c}{N}$. The MLEs for $\boldsymbol{\theta}_{dc}$ depend on the choice of the class conditional density for feature d. We discuss some common choices below.

- In the case of discrete features, we can use a categorical distribution. A straightforward extension of the results in Sec. 4.2.4 gives the following expression for the MLE:

$$\hat{\theta}_{dck} = \frac{N_{dck}}{\sum_{k'=1}^{K} N_{dck'}} = \frac{N_{dck}}{N_c} \tag{9.52}$$

  where $N_{dck} = \sum_{n=1}^{N} \mathbb{I}(x_{nd} = k, y_n = c)$ is the number of times that feature d had value k in examples of class c.

- In the case of binary features, the categorical distribution becomes the Bernoulli, and the MLE becomes

$$\hat{\theta}_{dc} = \frac{N_{dc}}{N_c} \tag{9.53}$$

  which is the empirical fraction of times that feature d is on in examples of class c.

- In the case of real-valued features, we can use a Gaussian distribution. A straightforward extension of the results in Sec. 4.2.5 gives the following expression for the MLE:

$$\hat{\mu}_{dc} = \frac{1}{N_c} \sum_{n:y_n=c} x_{nd} \tag{9.54}$$

$$\hat{\sigma}^2_{dc} = \frac{1}{N_c} \sum_{n:y_n=c} (x_{nd} - \hat{\mu}_{dc})^2 \tag{9.55}$$

  Thus we see that fitting a naive Bayes classifier is extremely simple and efficient.

9.3.3 Bayesian naive Bayes

In this section, we extend our discussion of MLE estimation for naive Bayes classifiers from Sec. 9.3.2 to compute the posterior distribution over the parameters. For simplicity, let us assume we have categorical features, so $p(x_d|\boldsymbol{\theta}_{dc}) = \text{Cat}(x_d|\boldsymbol{\theta}_{dc})$, where $\theta_{dck} = p(x_d = k|y = c)$. In Sec. 4.6.3.2, we show that the conjugate prior for the categorical likelihood is the Dirichlet distribution, $p(\boldsymbol{\theta}_{dc}) = \text{Dir}(\boldsymbol{\theta}_{dc}|\boldsymbol{\beta}_{dc})$, where β_{dck} can be interpereted as a set of "**pseudo counts**", corresponding to counts N_{dck} that come from prior data. Similarly we use a Dirichlet prior for the label frequencies,

$p(\boldsymbol{\pi}) = \text{Dir}(\boldsymbol{\pi}|\boldsymbol{\alpha})$. By using a conjugate prior, we can compute the posterior in closed form, as we explain in Sec. 4.6.3. In particular, we have

$$p(\boldsymbol{\theta}|\mathcal{D}) = \text{Dir}(\boldsymbol{\pi}|\,\widehat{\boldsymbol{\alpha}}) \prod_{d=1}^{D}\prod_{c=1}^{C} \text{Dir}(\boldsymbol{\theta}_{dc}|\,\widehat{\boldsymbol{\beta}}_{dc}) \tag{9.56}$$

where $\widehat{\alpha}_c = \breve{\alpha}_c + N_c$ and $\widehat{\beta}_{dck} = \breve{\beta}_{dck} + N_{dck}$.

Using the results from Sec. 4.6.3.4, we can derive the posterior predictive distribution as follows. The prior over the label is given by $p(y|\mathcal{D}) = \text{Cat}(y|\overline{\boldsymbol{\pi}})$, where $\overline{\pi}_c = \widehat{\alpha}_c \,/\, \sum_{c'} \widehat{\alpha}_{c'}$. For the features, we have $p(x_d = k|y = c, \mathcal{D}) = \overline{\theta}_{dck}$, where

$$\overline{\theta}_{dck} = \frac{\widehat{\beta}_{dck}}{\sum_{k'} \widehat{\beta}_{dck'}} = \frac{\breve{\beta}_{dck} + N_{dck}}{\sum_{k'} \breve{\beta}_{dck'} + N_{dck'}} \tag{9.57}$$

is the posterior mean of the parameters.

If $\breve{\beta}_{dck} = 0$, this reduces to the MLE in Eq. (9.52). By contrast, if we set $\breve{\beta}_{dck} = 1$, we add 1 to all the empirical counts before normalizing. This is called **add-one smoothing** or **Laplace smoothing**. For example, in the binary case, this gives

$$\overline{\theta}_{dc} = \frac{\breve{\beta}_{dc1} + N_{dc1}}{\breve{\beta}_{dc0} + N_{dc0} + \breve{\beta}_{dc1} + N_{dc1}} = \frac{1 + N_{dc1}}{2 + N_c} \tag{9.58}$$

Once we have estimated the parameter posterior, we can compute the predicted distribution over the label as follows:

$$p(y = c|\mathbf{x}, \mathcal{D}) \propto p(y = c|\mathcal{D}) \prod_d p(x_d|y = c, \mathcal{D}) = \overline{\pi}_c \prod_d \prod_k \overline{\theta}_{dck}^{\mathbb{I}(x_d = k)} \tag{9.59}$$

This gives us a fully Bayesian form of naive Bayes, in which we have integrated out all the parameters. (In this case, the predictive distribution can be obtained merely by plugging in the posterior mean parameters.)

9.3.4 The connection between naive Bayes and logistic regression

In this section, we show that the class posterior $p(y|\mathbf{x}, \boldsymbol{\theta})$ for a NBC model has the same form as multinomial logistic regression. For simplicity, we assume that the features are all discrete, and each has K states, although the result holds for arbitrary feature distributions in the exponential family.

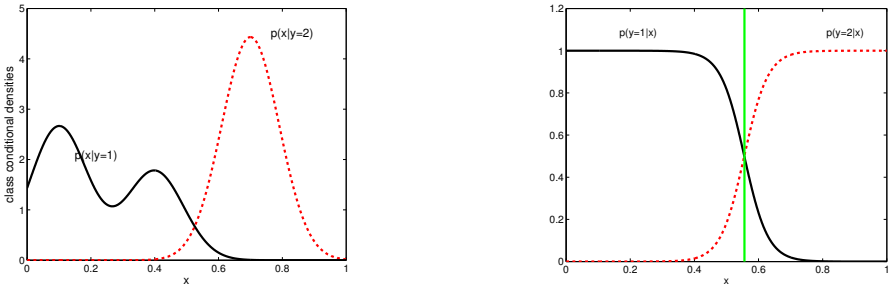
Let $x_{dk} = \mathbb{I}(x_d = k)$, so $\mathbf{x}_d$ is a one-hot encoding of feature d. Then the class conditional density can be written as follows:

$$p(\mathbf{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Cat}(x_d|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D}\prod_{k=1}^{K} \theta_{dck}^{x_{dk}} \tag{9.60}$$

Hence the posterior over classes is given by

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{\pi_c \prod_d \prod_k \theta_{dck}^{x_{dk}}}{\sum_{c'} \pi_{c'} \prod_d \prod_k \theta_{dc'k}^{x_{dk}}} = \frac{\exp[\log \pi_c + \sum_d \sum_k x_{dk} \log \theta_{dck}]}{\sum_{c'} \exp[\log \pi_{c'} + \sum_d \sum_k x_{dk} \log \theta_{dc'k}]} \tag{9.61}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 9.9: The class-conditional densities $p(x|y=c)$ (left) may be more complex than the class posteriors $p(y=c|x)$ (right). Adapted from Figure 1.27 of [Bis06]. Generated by code at figures.probml.ai/book1/9.9.

This can be written as a softmax

$$p(y=c|\mathbf{x},\boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^\top \mathbf{x}+\gamma_c}}{\sum_{c'=1}^{C} e^{\boldsymbol{\beta}_{c'}^\top \mathbf{x}+\gamma_{c'}}} \tag{9.62}$$

by suitably defining $\boldsymbol{\beta}_c$ and γ_c. This has exactly the same form as multinomial logistic regression in Sec. 2.5.3. The difference is that with naive Bayes we optimize the joint likelihood $\prod_n p(y_n, \mathbf{x}_n|\boldsymbol{\theta})$, whereas with logistic regression, we optimize the conditional likelihood $\prod_n p(y_n|\mathbf{x}_n, \boldsymbol{\theta})$. In general, these can give different results (see Exercise 10.3).

9.4 Generative vs discriminative classifiers

A model of the form $p(\mathbf{x}, y) = p(y)p(\mathbf{x}|y)$ is called a **generative classifier**, since it can be used to generate examples $\mathbf{x}$ from each class y. By contrast, a model of the form $p(y|\mathbf{x})$ is called a **discriminative classifier**, since it can only be used to discriminate between different classes. Below we discuss various pros and cons of the generative and discriminative approaches to classification.

9.4.1 Advantages of discriminative classifiers

The main advantages of discriminative classifiers are as follows:

- **Better predictive accuracy**. Discriminative classifiers are often much more accurate than generative classifiers [NJ02]. The reason is that the conditional distribution $p(y|\mathbf{x})$ is often much simpler (and therefore easier to learn) than the joint distribution $p(y, \mathbf{x})$, as illustrated in Fig. 9.9. In particular, discriminative models do not need to "waste effort" modeling the distribution of the input features.
- **Can handle feature preprocessing**. A big advantage of discriminative methods is that they allow us to preprocess the input in arbitrary ways. For example, we can perform a polynomial expansion of the input features, and we can replace a string of words with embedding vectors (see Sec. 20.5). It is often hard to define a generative model on such pre-processed data, since the new features can be correlated in complex ways which are hard to model.

- **Well-calibrated probabilities**. Some generative classifiers, such as naive Bayes (described in Sec. 9.3), make strong independence assumptions which are often not valid. This can result in very extreme posterior class probabilities (very near 0 or 1). Discriminative models, such as logistic regression, are often better calibrated in terms of their probability estimates, although they also sometimes need adjustment (see e.g., [NMC05]).

9.4.2 Advantages of generative classifiers

The main advantages of generative classifiers are as follows:

- **Easy to fit.** Generative classifiers are often very easy to fit. For example, in Sec. 9.3.2, we show how to fit a naive Bayes classifier by simple counting and averaging. By contrast, logistic regression requires solving a convex optimization problem (see Sec. 10.2.3 for the details), and neural nets require solving a non-convex optimization problem, both of which are much slower.
- **Can easily handle missing input features.** Sometimes some of the inputs (components of $\mathbf{x}$) are not observed. In a generative classifier, there is a simple method for dealing with this, as we show in Sec. 1.5.5. However, in a discriminative classifier, there is no principled solution to this problem, since the model assumes that $\mathbf{x}$ is always available to be conditioned on.
- **Can fit classes separately.** In a generative classifier, we estimate the parameters of each class conditional density independently (as we show in Sec. 9.3.2), so we do not have to retrain the model when we add more classes. In contrast, in discriminative models, all the parameters interact, so the whole model must be retrained if we add a new class.
- **Can handle unlabeled training data.** It is easy to use generative models for semi-supervised learning, in which we combine labeled data $\mathcal{D}_{xy} = \{(\mathbf{x}_n, y_n)\}$ and unlabeled data, $\mathcal{D}_x = \{\mathbf{x}_n\}$. However, this is harder to do with discriminative models, since there is no uniquely optimal way to exploit $\mathcal{D}_x$.

9.4.3 Handling missing features

Sometimes we are missing parts of the input $\mathbf{x}$ during training and/or testing. In a generative classifier, we can handle this situation by marginalizing out the missing values. (We assume that the missingness of a feature is not informative about its potential value.) By contrast, when using a discriminative model, there is no unique best way to handle missing inputs, as we discuss in Sec. 1.5.5.

For example, suppose we are missing the value of x_1. We just have to compute

$$p(y=c|\mathbf{x}_{2:D}, \boldsymbol{\theta}) \propto p(y=c|\boldsymbol{\pi}) p(\mathbf{x}_{2:D}|y=c, \boldsymbol{\theta}) \tag{9.63}$$

$$= p(y=c|\boldsymbol{\pi}) \sum_{x_1} p(x_1, \mathbf{x}_{2:D}|y=c, \boldsymbol{\theta}) \tag{9.64}$$

In Gaussian discriminant analysis, we can marginalize out x_1 using the equations from Sec. 3.2.3.

If we make the naive Bayes assumption, things are even easier, since we can just ignore the likelihood term for x_1. This follows because

$$\sum_{x_1} p(x_1, x_{2:D}|y=c, \boldsymbol{\theta}) = \left[\sum_{x_1} p(x_1|\boldsymbol{\theta}_{1c})\right] \prod_{d=2}^{D} p(x_d|\boldsymbol{\theta}_{dc}) = \prod_{d=2}^{D} p(x_d|\boldsymbol{\theta}_{dc}) \tag{9.65}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where we exploited the fact that $\sum_{x_1} p(x_1|y=c, \boldsymbol{\theta}_{1c}) = 1$.

9.5 Exercises

Exercise 9.1 [Derivation of Fisher's linear discriminant]

Show that the maximum of $J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$ is given by $\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$
where $\lambda = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$. Hint: recall that the derivative of a ratio of two scalars is given by $\frac{d}{dx}\frac{f(x)}{g(x)} = \frac{f'g - fg'}{g^2}$, where $f' = \frac{d}{dx} f(x)$ and $g' = \frac{d}{dx} g(x)$. Also, recall that $\frac{d}{d\mathbf{x}} \mathbf{x}^T \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$.

10 Logistic regression

10.1 Introduction

Logistic regression is a widely used discriminative classification model $p(y|\mathbf{x};\boldsymbol{\theta})$, where $\mathbf{x} \in \mathbb{R}^D$ is a fixed-dimensional input vector, $y \in \{1,\ldots,C\}$ is the class label, and $\boldsymbol{\theta}$ are the parameters. If $C = 2$, this is known as **binary logistic regression**, and if $C > 2$, it is known as **multinomial logistic regression**, or alternatively, **multiclass logistic regression**. We give the details below.

10.2 Binary logistic regression

Binary logistic regression corresponds to the following model

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x} + b)) \tag{10.1}$$

where σ is the sigmoid function defined in Sec. 2.4.2. In other words,

$$p(y = 1|\mathbf{x};\boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}} \tag{10.2}$$

where $a = \mathbf{w}^\top \mathbf{x} + b$ is the log-odds, $\log(p/1-p)$, where $p = p(y = 1|\mathbf{x};\boldsymbol{\theta})$, as explained in Sec. 2.4.2. (In ML, the quantity a is usually called the **logit** or the **pre-activation**.)

Sometimes we choose to use the labels $\tilde{y} \in \{-1, +1\}$ instead of $y \in \{0, 1\}$. We can compute the probability of these alternative labels using

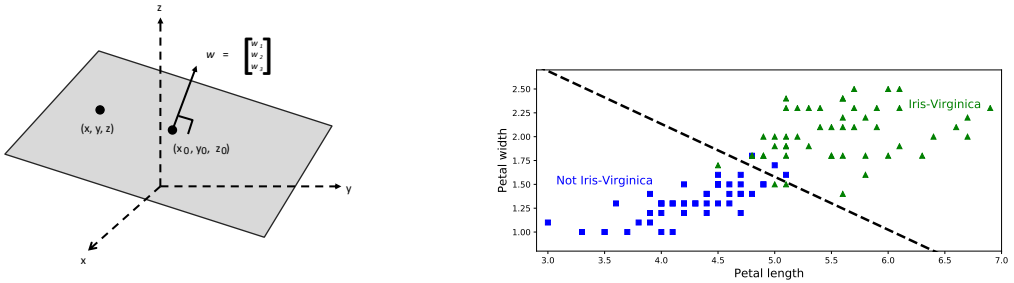
$$p(\tilde{y}|\mathbf{x},\boldsymbol{\theta}) = \sigma(\tilde{y}a) \tag{10.3}$$

since $\sigma(-a) = 1 - \sigma(a)$. This slightly more compact notation is widely used in the ML literature.

10.2.1 Linear classifiers

The sigmoid gives the probability that the class label is $y = 1$. If the loss for misclassifying each class is the same, then the optimal decision rule is to predict $y = 1$ iff class 1 is more likely than class 0, as we explained in Sec. 5.1.2.2. Thus

$$f(\mathbf{x}) = \mathbb{I}\left(p(y = 1|\mathbf{x}) > p(y = 0|\mathbf{x})\right) = \mathbb{I}\left(\log \frac{p(y = 1|\mathbf{x})}{p(y = 0|\mathbf{x})} > 0\right) = \mathbb{I}\left(a > 0\right) \tag{10.4}$$



(a) *(b)*

Figure 10.1: (a) Visualization of a 2d plane in a 3d space with surface normal $\mathbf{w}$ *going through point* $\mathbf{x}_0 = (x_0, y_0, z_0)$*. See text for details. (b) Visualization of optimal linear decision boundary induced by logistic regression on a 2-class, 2-feature version of the iris dataset. Generated by code at figures.probml.ai/book1/10.1. Adapted from Figure 4.24 of [Gér19].*

where $a = \mathbf{w}^\top \mathbf{x} + b$.

Thus we can write the prediction function as follows:

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^\top \mathbf{x} = b + \sum_{d=1}^{D} w_d x_d \tag{10.5}$$

where $\mathbf{w}^\top \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle$ is the inner product between the weight vector $\mathbf{w}$ and the feature vector $\mathbf{x}$. This function defines a linear **hyperplane**, with normal vector $\mathbf{w} \in \mathbb{R}^D$ and an offset $b \in \mathbb{R}$ from the origin.

Eq. (10.5) can be understood by looking at Fig. 10.1a. Here we show a plane in a 3d feature space going through the point $\mathbf{x}_0$ with surface normal $\mathbf{w}$. Points on the surface satisfy $\mathbf{w}^\top(\mathbf{x} - \mathbf{x}_0) = 0$. If we define $b = -\mathbf{w}^\top \mathbf{x}_0$, we can rewrite this as $\mathbf{w}^\top \mathbf{x} + b = 0$. This plane separates 3d space into two **half spaces**. This linear plane is known as a **decision boundary**. If we can perfectly separate the training examples by such a linear boundary (without making any classification errors on the training set), we say the data is **linearly separable**. From Fig. 10.1b, we see that the two-class, two-feature version of the iris dataset is not linearly separable.

In general, there will be uncertainty about the correct class label, so we need to predict a probability distribution over labels, and not just decide which side of the decision boundary we are on. In Fig. 10.2, we plot $p(y = 1|x_1, x_2; \mathbf{w}) = \sigma(w_1 x_1 + w_2 x_2)$ for different weight vectors $\mathbf{w}$. The vector $\mathbf{w}$ defines the orientation of the decision boundary, and its magnitude, $||\mathbf{w}|| = \sqrt{\sum_{d=1}^{D} w_d^2}$, controls the steepness of the sigmoid, and hence the confidence of the predictions.

10.2.2 Nonlinear classifiers

We can often make a problem linearly separable by preprocessing the inputs in a suitable way. In particular, let $\boldsymbol{\phi}(\mathbf{x})$ be a transformed version of the input feature vector. For example, suppose we use $\boldsymbol{\phi}(x_1, x_2) = [1, x_1^2, x_2^2]$, and we let $\mathbf{w} = [-R^2, 1, 1]$. Then $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) = x_1^2 + x_2^2 - R^2$, so the decision boundary (where $f(\mathbf{x}) = 0$) defines a circle with radius R, as shown in Fig. 10.3. The resulting

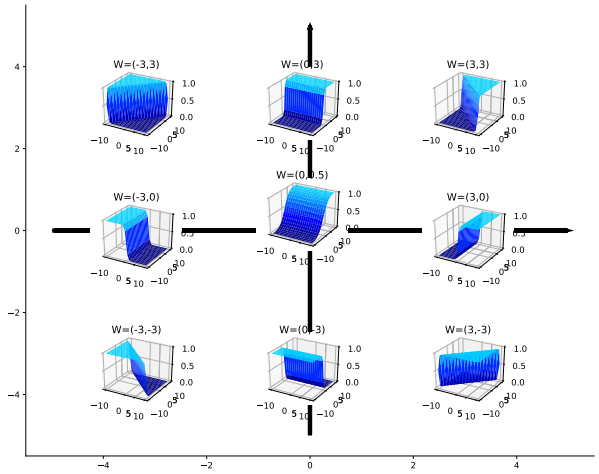


Figure 10.2: Plots of $\sigma(w_1 x_1 + w_2 x_2)$. Here $\mathbf{w} = (w_1, w_2)$ defines the normal to the decision boundary. Points to the right of this have $\sigma(\mathbf{w}^\top \mathbf{x}) > 0.5$, and points to the left have $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$. Adapted from Figure 39.3 of [Mac03]. Generated by code at figures.probml.ai/book1/10.2.

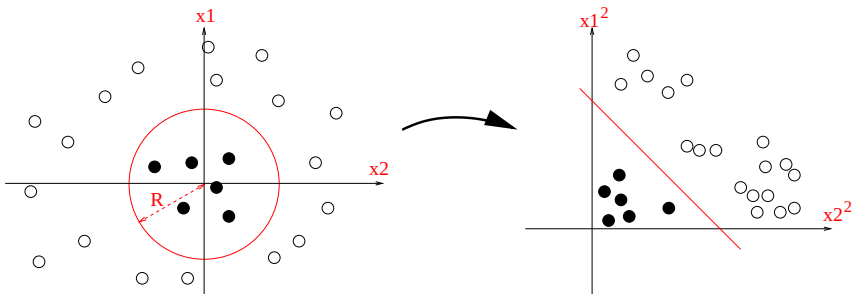


Figure 10.3: Illustration of how we can transform a quadratic decision boundary into a linear one by transforming the features from $\mathbf{x} = (x_1, x_2)$ to $\boldsymbol{\phi}(\mathbf{x}) = (x_1^2, x_2^2)$. Used with kind permission of Jean-Philippe Vert

function f is still linear in the parameters $\mathbf{w}$, which is important for simplifying the learning problem, as we will see in Sec. 10.2.3. However, we can gain even more power by learning the parameters of the feature extractor $\boldsymbol{\phi}(\mathbf{x})$ in addition to linear weights $\mathbf{w}$; we discuss how to do this in Part III.

In Fig. 10.3, we used a quadratic expansion of the features. We can also use a higher order polynomial, as in Sec. 1.2.2.2. In Fig. 1.7, we show the effects of using polynomial expansion up to degree K on a 2d logistic regression problem. As in Fig. 1.7, we see that the model becomes more complex as the number of parameters increases, and eventually results in overfitting. We discuss ways to reduce overfitting in Sec. 10.2.7.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.2.3 Maximum likelihood estimation

In this section, we discuss how to estimate the parameters of a logistic regression model using maximum likelihood estimation.

10.2.3.1 Objective function

The NLL (ignoring the bias term, for notational simplicity) is given by

$$\text{NLL}(\mathbf{w}) = -\log \prod_{n=1}^{N} \text{Ber}(y_n|\mu_n) \tag{10.6}$$

$$= -\sum_{n=1}^{N} \log[\mu_n^{y_n} \times (1-\mu_n)^{1-y_n}] \tag{10.7}$$

$$= -\sum_{n=1}^{N} [y_n \log \mu_n + (1-y_n)\log(1-\mu_n)] \tag{10.8}$$

$$= \sum_{n=1}^{N} \mathbb{H}(y_n, \mu_n) \tag{10.9}$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the log odds, and $\mathbb{H}(y_n, \mu_n)$ is the **binary cross entropy** defined by

$$\mathbb{H}(p, q) = -[p \log q + (1-p)\log(1-q)] \tag{10.10}$$

If we use $\tilde{y}_n \in \{-1, +1\}$ instead of $y_n \in \{0, 1\}$, then we can rewrite this as follows:

$$\text{NLL}(\mathbf{w}) = -\sum_{n=1}^{N} [\mathbb{I}(\tilde{y}_n = 1)\log(\sigma(a_n)) + \mathbb{I}(\tilde{y}_n = -1)\log(\sigma(-a_n))] \tag{10.11}$$

$$= -\sum_{n=1}^{N} \log(\sigma(\tilde{y}_n a_n)) \tag{10.12}$$

$$= \sum_{n=1}^{N} \log(1 + \exp(-\tilde{y}_n a_n)) \tag{10.13}$$

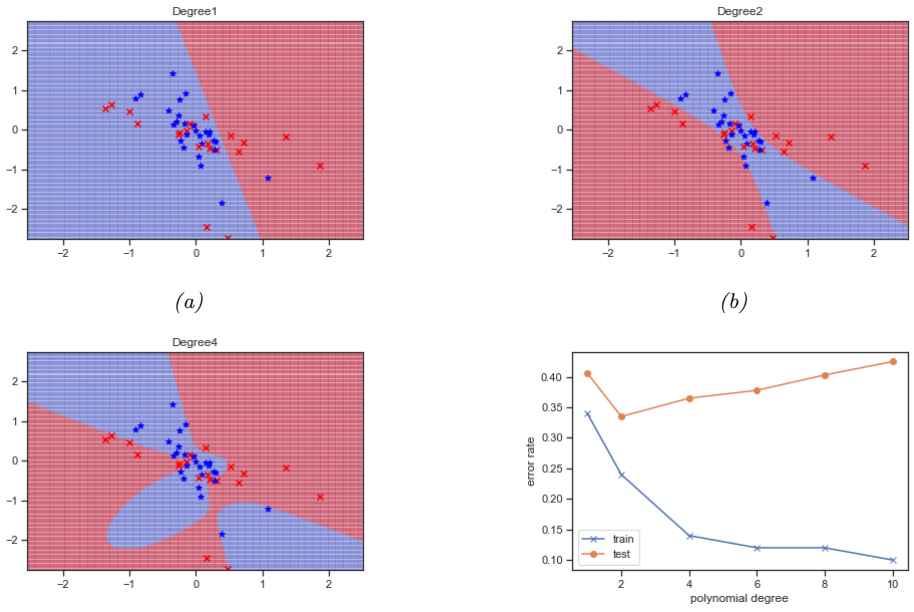
However, in this book, we will mostly use the $y_n \in \{0, 1\}$ notation, since it is easier to generalize to the multiclass case (Sec. 10.3), and makes the connection with cross-entropy easier to see.

10.2.3.2 Optimizing the objective

To find the MLE, we must solve

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) = \mathbf{0} \tag{10.14}$$

We can use any gradient-based optimization algorithm to solve this, such as those we discuss in Chapter 8. We give a specific example in Sec. 10.2.4. But first we must derive the gradient, as we explain below.



(a) *(b)* *(c)* *(d)*

Figure 10.4: Polynomial feature expansion applied to a two-class, two-dimensional logistic regression problem. (a) Degree $K = 1$. (a) Degree $K = 2$. (a) Degree $K = 4$. (d) Train and test error vs degree. Generated by code at figures.probml.ai/book1/10.4.

10.2.3.3 Deriving the gradient

Although we can use automatic differentiation methods (Sec. 13.3) to compute the gradient of the NLL, it is also easy to do explicitly, as we show below. Fortunately the resulting equations will turn out to have a simple and intuitive interpretation, which can be used to derive other methods, as we will see.

To start, note that

$$\frac{d\mu_n}{da_n} = \sigma(a_n)(1 - \sigma(a_n)) \tag{10.15}$$

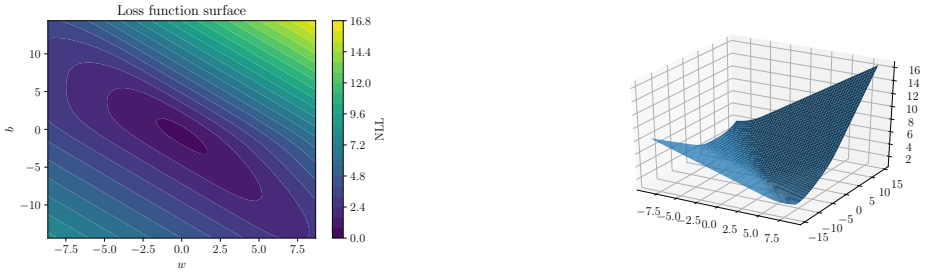
where $a_n = \mathbf{w}^\top \mathbf{x}_n$ and $\mu_n = \sigma(a_n)$. Hence by the chain rule (and the rules of vector calculus, discussed in Sec. 7.8) we have

$$\frac{\partial}{\partial w_d}\mu_n = \frac{\partial}{\partial w_d}\sigma(\mathbf{w}^\top \mathbf{x}_n) = \frac{\partial}{\partial a_n}\sigma(a_n)\frac{\partial a_n}{\partial w_d} = \mu_n(1 - \mu_n)x_{nd} \tag{10.16}$$

The gradient for the bias term can be derived in the same way, by using the input $x_{n0} = 1$ in the above equation. However, we will ignore the bias term for simplicity. Hence

$$\nabla_{\mathbf{w}} \log(\mu_n) = \frac{1}{\mu_n}\nabla_{\mathbf{w}}\mu_n = (1 - \mu_n)\mathbf{x}_n \tag{10.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 10.5: NLL loss surface for binary logistic regression applied to Iris dataset with 1 feature and 1 bias term. The goal is to minimize the function. Generated by code at figures.probml.ai/book1/10.5.

Similarly,

$$\nabla_{\mathbf{w}} \log(1-\mu_n) = \frac{-\mu_n(1-\mu_n)\mathbf{x}_n}{1-\mu_n} = -\mu_n \mathbf{x}_n \tag{10.18}$$

Thus the gradient vector of the NLL is given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = -\sum_{n=1}^{N} [y_n(1-\mu_n)\mathbf{x}_n - (1-y_n)\mu_n \mathbf{x}_n] \tag{10.19}$$

$$= -\sum_{n=1}^{N} [y_n \mathbf{x}_n - y_n \mathbf{x}_n \mu_n - \mathbf{x}_n \mu_n + y_n \mathbf{x}_n \mu_n)] \tag{10.20}$$

$$= \sum_{n=1}^{N} (\mu_n - y_n)\mathbf{x}_n \tag{10.21}$$

$$= \mathbf{1}_N^\top (\text{diag}(\boldsymbol{\mu} - \mathbf{y})\mathbf{X}) \tag{10.22}$$

This has a nice intuitive interpretation as the sum of the differences between the observed label y_n and the prediction μ_n, weighted by the inputs $\mathbf{x}_n$.

10.2.3.4 Deriving the Hessian

Gradient-based optimizers will find a stationary point where $\mathbf{g}(\mathbf{w}) = \mathbf{0}$. This could either be a global optimum or a local optimum. To be sure the stationary point is the global optimum, we must show that the objective is **convex**, for reasons we explain in Sec. 8.1.1.1. Intuitvely this means that the NLL has a **bowl shape**, with a unique lowest point, which is indeed the case, as illustrated in Fig. 10.5b.

More formally, we must prove that the Hessian is positive semi-definite, which we now do. (See Chapter 7 for relevant background information on linear algebra.) Once can show that the Hessian is

given by

$$\mathbf{H}(\mathbf{w})\nabla_{\mathbf{w}}\nabla_{\mathbf{w}}^{\top}\mathrm{NLL}(\mathbf{w}) = \sum_{n=1}^{N}(\mu_n(1-\mu_n)\mathbf{x}_n)\mathbf{x}_n^{\top} = \mathbf{X}^{\top}\mathbf{S}\mathbf{X} \tag{10.23}$$

where

$$\mathbf{S} \triangleq \mathrm{diag}(\mu_1(1-\mu_1), \ldots, \mu_N(1-\mu_N)) \tag{10.24}$$

We see that $\mathbf{H}$ is positive semi definite, since for any nonzero vector $\mathbf{v}$, we have

$$\mathbf{v}^{\top}\mathbf{X}^{\top}\mathbf{S}\mathbf{X}\mathbf{v} = (\mathbf{v}^{\top}\mathbf{X}^{\top}\mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}}\mathbf{X}\mathbf{v}) = ||\mathbf{v}^{\top}\mathbf{X}^{\top}\mathbf{S}^{\frac{1}{2}}||_2^2 \geq 0 \tag{10.25}$$

We can ensure that the NLL is strictly convex by ensuring that $\mu_n > 0$ for all n; we can do this using ℓ_2 regularization, as we discuss in Sec. 10.2.7.

10.2.4 Stochastic gradient descent

Our goal is to solve the following optimization problem

$$\hat{\mathbf{w}} \triangleq \operatorname*{argmin}_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \tag{10.26}$$

where $\mathcal{L}(\mathbf{w})$ is the loss function, in this case the negative log likelihood:

$$\mathrm{NLL}(\mathbf{w}) = -\sum_{n=1}^{N}[y_n \log \mu_n + (1-y_n)\log(1-\mu_n)] \tag{10.27}$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, and $a_n = \mathbf{w}^{\top}\mathbf{x}_n$ is the log odds.

There are many algorithms we could use to compute Eq. (10.26), as we discuss in Chapter 8. Perhaps the simplest is to use stochastic gradient descent (Sec. 8.4). If we use a minibatch of size 1, then we get the following simple update equation:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\mu_n - y_n)\mathbf{x}_n \tag{10.28}$$

where on step t we we sample example $n = n_t$.

Since we know the objective is convex (see Sec. 10.2.3.4), then one can show that this procedure will converge to the global optimum, provided we decay the learning rate at the appropriate rate (see Sec. 8.4.3). We can improve the convergence speed using variance reduction techniques such as SAGA (Sec. 8.4.5.2).

10.2.5 Perceptron algorithm

A **perceptron**, first introduced in [Ros58], is a deterministic binary classifier of the following form:

$$f(\mathbf{x}_n; \boldsymbol{\theta}) = \mathbb{I}\left(\mathbf{w}^{\top}\mathbf{x}_n + b > 0\right) \tag{10.29}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This can be seen to be a limiting case of a binary logistic regression classifier, in which the sigmoid function $\sigma(a)$ is replaced by the Heaviside step function $H(a) \triangleq \mathbb{I}(a > 0)$. See Fig. 2.7 for a comparison of these two functions.

Since the Heaviside function is not differentiable, we cannot use gradient-based optimization methods to fit this model. However, Rosenblatt proposed the **perceptron learning algorithm** instead. The basic idea is to start with random weights, and then iteratively update them whenever the model makes a prediction mistake. More precisely, we update the weights using

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\hat{y}_n - y_n)\mathbf{x}_n \tag{10.30}$$

where $(\mathbf{x}_n, y_n)$ is the labeled example sampled at iteration t, and η_t is the learning rate or step size. (We can set the step size to 1, since the magnitude of the weights does not affect the decision boundary.) See code.probml.ai/book1/perceptron_demo_2d for a simple implementation of this algorithm.

The perceptron update rule in Eq. (10.30) has an intuitive interpretation: if the prediction is correct, no change is made, otherwise we move the weights in a direction so as to make the correct answer more likely. More precisely, if $y_n = 1$ and $\hat{y}_n = 0$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_n$, and if $y_n = 0$ and $\hat{y}_n = 1$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}_n$.

By comparing Eq. (10.30) to Eq. (10.28), we see that the perceptron update rule is equivalent to the SGD update rule for binary logistic regression using the approximation where we replace the soft probabilities $\mu_n = p(y_n = 1|\mathbf{x}_n)$ with hard labels $\hat{y}_n = f(\mathbf{x}_n)$. The advantage of the perceptron method is that we don't need to compute probabilities, which can be useful when the label space is very large. The disadvantage is that the method will only converge when the data is linearly separable [Nov62], whereas SGD for minimizing the NLL for logistic regression will always converge to the globally optimal MLE, even if the data is not linearly separable.

In Sec. 13.2, we will generalize perceptrons to nonlinear functions, thus significantly enhancing their usefulness.

10.2.6 Iteratively reweighted least squares

Gradient descent is a **first order** optimization method, which means it only uses first order gradients to navigate through the loss landscape. This can be slow, especially when some directions of space point steeply downhill, whereas other have a shallower gradient, as is the case in Fig. 10.5a. In such problems, it can be much faster to use a **second order** optimization method, that takes the curvature of the space into account.

We discuss such methods in more detail in Sec. 8.3. Here we just consider a simple second order method that works well for logistic regression. We focus on the full batch setting (so we assume N is small), since it is harder to make second order methods work in the stochastic setting (see e.g., [Byr+16; Liu+18b] for some methods).

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{H}_t^{-1}\mathbf{g}_t \tag{10.31}$$

where

$$\mathbf{H}_t \triangleq \nabla^2\mathcal{L}(\mathbf{w})|_{\mathbf{w}_t} = \nabla^2\mathcal{L}(\mathbf{w}_t) = \mathbf{H}(\mathbf{w}_t) \tag{10.32}$$

is assumed to be positive-definite to ensure the update is well-defined. If the Hessian is exact, we can set the step size to $\eta_t = 1$.

We now apply this method to logistic regression. Recall from Sec. 10.2.3.3 that the gradient and Hessian are given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \sum_{n=1}^{N} (\mu_n - y_n)\mathbf{x}_n \tag{10.33}$$

$$\mathbf{H} = \mathbf{X}^\top \mathbf{S} \mathbf{X} \tag{10.34}$$

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1-\mu_1), \ldots, \mu_N(1-\mu_N)) \tag{10.35}$$

Hence the Newton update has the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1}\mathbf{g}_t \tag{10.36}$$

$$= \mathbf{w}_t + (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t) \tag{10.37}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \left[(\mathbf{X}^\top \mathbf{S}_t \mathbf{X}) \mathbf{w}_t + \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t) \right] \tag{10.38}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top \left[\mathbf{S}_t \mathbf{X} \mathbf{w}_t + \mathbf{y} - \boldsymbol{\mu}_t \right] \tag{10.39}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S}_t \mathbf{z}_t \tag{10.40}$$

where we have defined the **working response** as

$$\mathbf{z}_t \triangleq \mathbf{X}\mathbf{w}_t + \mathbf{S}_t^{-1}(\mathbf{y} - \boldsymbol{\mu}_t) \tag{10.41}$$

and $\mathbf{S}_t = \text{diag}(\mu_{t,n}(1-\mu_{t,n}))$. Since $\mathbf{S}_t$ is a diagonal matrix, we can rewrite the targets in component form as follows:

$$z_{t,n} = \mathbf{w}_t^\top \mathbf{x}_n + \frac{y_n - \mu_{t,n}}{\mu_{t,n}(1-\mu_{t,n})} \tag{10.42}$$

Eq. (10.40) is an example of a weighted least squares problem (Sec. 11.2.2.4), which is a minimizer of

$$\sum_{n=1}^{N} S_{t,n}(z_{t,n} - \mathbf{w}_t^\top \mathbf{x}_n)^2 \tag{10.43}$$

The overall method is therefore known as the **iteratively reweighted least squares** (**IRLS**) algorithm, since at each iteration we solve a weighted least squares problem, where the weight matrix $\mathbf{S}_t$ changes at each iteration. See Algorithm 2 for some pseudocode.

Note that **Fisher scoring** is the same as IRLS except we replace the Hessian of the actual log-likelihood with its expectation, i.e., we use the Fisher information matrix (Sec. 4.7.2) instead of $\mathbf{H}$. Since the Fisher information matrix is independent of the data, it can be precomputed, unlike the Hessian, which must be reevaluated at every iteration. This can be faster for problems with many parameters.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 2: Iteratively reweighted least squares (IRLS)

$\mathbf{w} = \mathbf{0}$;
$w_0 = \log(\overline{y}/(1-\overline{y}))$;
repeat
  for $n = 1 : N$ **do**
    $a_n = w_0 + \mathbf{w}^\top \mathbf{x}_n$;
    $\mu_n = \sigma(a_n)$;
    $s_n = \mu_n(1-\mu_n)$;
    $z_n = \eta_n + \frac{y_n - \mu_n}{s_n}$;
  $\mathbf{S} = \text{diag}(s_{1:N})$;
  $\mathbf{w} = (\mathbf{X}^\top \mathbf{S}\mathbf{X})^{-1}\mathbf{X}^\top \mathbf{S}\mathbf{z}$;
until *converged*;

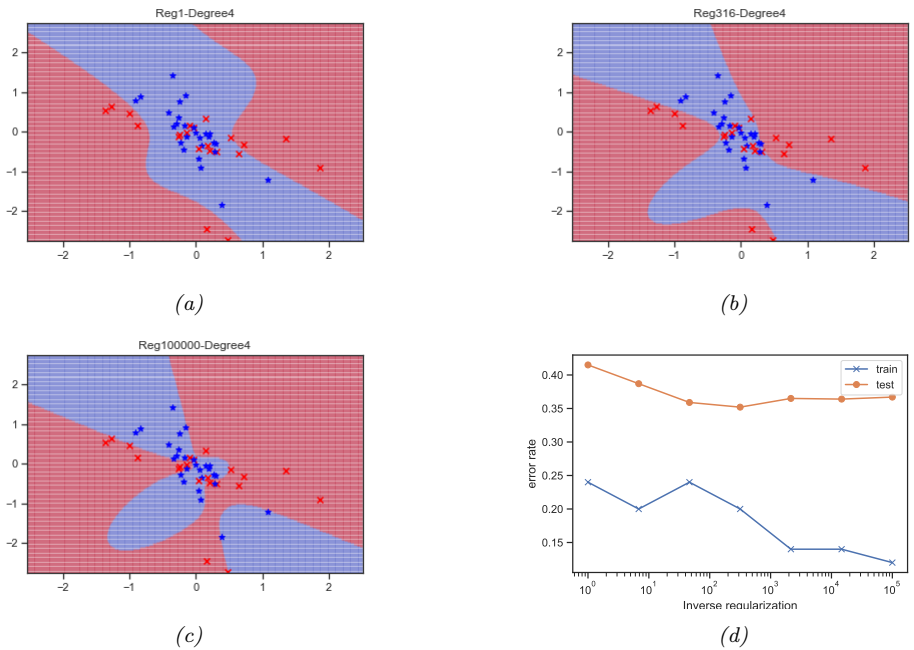


Figure 10.6: Weight decay with variance C applied to two-class, two-dimensional logistic regression problem with a degree 4 polynomial. (a) $C = 1$. (a) $C = 316$. (a) $C = 100,000$. (d) Train and test error vs C. Generated by code at figures.probml.ai/book1/10.6.

10.2.7 MAP estimation

In Fig. 10.4, we saw how logistic regression can overfit when there are too many parameters compared to training examples. This is a consequence of the ability of maximum likelihood to find weights that force the decision boundary to "wiggle" in just the right way so as to curve around the examples. To get this behavior, the weights often need to be set to large values. For example, in Fig. 10.4, when we use degree $K = 1$, we find that the MLE for the two input weights (ignoring the bias) is

$$\hat{\mathbf{w}} = [0.51291712, 0.11866937] \tag{10.44}$$

When we use degree $K = 2$, we get

$$\hat{\mathbf{w}} = [2.27510513, 0.05970325, 11.84198867, 15.40355969, 2.51242311] \tag{10.45}$$

And when $K = 4$, we get

$$\hat{\mathbf{w}} = [-3.07813766, \cdots, -59.03196044, 51.77152431, 10.25054164] \tag{10.46}$$

One way to reduce such overfitting is to prevent the weights from becoming so large. We can do this by using a zero-mean Gaussian prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, C\mathbf{I})$, and then using MAP estimation, as we discussed in Sec. 4.5.3. The new training objective becomes

$$\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda||\mathbf{w}||_2^2 \tag{10.47}$$

where $||\mathbf{w}||_2^2 = \sum_{d=1}^{D} w_d^2$ and $\lambda = 1/C$. This is called ℓ_2 **regularization** or **weight decay**. The larger the value of λ, the more the parameters are penalized for being "large" (deviating from the zero-mean prior), and thus the less flexible the model. See Fig. 10.6 for an illustration.

We can compute the MAP estimate by slightly modifying the input to the above gradient-based optimization algorithms. Specifically, we use the penalized negative log likelihood, and its gradient and Hessian, have the following forms:

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^\top \mathbf{w} \tag{10.48}$$

$$\nabla_{\mathbf{w}} \text{PNLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) + 2\lambda \mathbf{w} \tag{10.49}$$

$$\nabla^2_{\mathbf{w}} \text{PNLL}(\mathbf{w}) = \mathbf{H}(\mathbf{w}) + 2\lambda \mathbf{I} \tag{10.50}$$

where $\mathbf{g}(\mathbf{w})$ is the gradient and $\mathbf{H}(\mathbf{w})$ is the Hessian.

For an interesting exercise related to ℓ_2 regularized logistic regression, see Exercise 10.2.

10.2.8 Standardization

In Sec. 10.2.7, we use an isotropic prior $p(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$ to prevent overfitting. This implicitly encodes the assumption that we expect all weights to be similar in magnitude, which in turn encodes the assumption we expect all input features to be similar in magnitude. However, in many datasets, input features are on different scales. In such cases, it is common to **standardize** the data, to ensure each feature has mean 0 and variance 1. We can do this by subtracting the mean and dividing by

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

the standard deviation of each feature, as follows:

$$\text{standardize}(x_{nd}) = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d} \tag{10.51}$$

$$\hat{\mu}_d = \frac{1}{N}\sum_{n=1}^{N} x_{nd} \tag{10.52}$$

$$\hat{\sigma}_d^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{nd} - \hat{\mu}_d)^2 \tag{10.53}$$

An alternative is to use **min-max scaling**, in which we rescale the inputs so they lie in the interval $[0, 1]$. Both methods ensure the features are comparable in magnitude, which can help with model fitting and inference, even if we don't use MAP estimation. (See Sec. 11.6.1.2 for a discussion of this point.)

10.3 Multinomial logistic regression

Multinomial logistic regression is a discriminative classification model of the following form:

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{x} + \mathbf{b})) \tag{10.54}$$

where $\mathbf{x} \in \mathbb{R}^D$ is the input vector, $y \in \{1, \ldots, C\}$ is the class label, $\mathcal{S}()$ is the softmax function (Sec. 2.5.2), $\mathbf{W}$ is a $C \times D$ weight matrix, $\mathbf{b}$ is C-dimensional bias vector, $\boldsymbol{\theta} = (\mathbf{W}, \mathbf{b})$ are all the parameters. We will henceforth ignore the bias term $\mathbf{b}$; we assume we prepend each $\mathbf{x}$ with a 1, and add $\mathbf{b}$ to the first column of $\mathbf{W}$. Thus $\boldsymbol{\theta} = \mathbf{W}$.

If we let $\mathbf{a} = \mathbf{W}\mathbf{x}$ be the C-dimensional vector of **logits**, then we can rewrite the above as follows:

$$p(y = c|\mathbf{x};\boldsymbol{\theta}) = \frac{e^{a_c}}{\sum_{c'=1}^{C} e^{a_{c'}}} \tag{10.55}$$

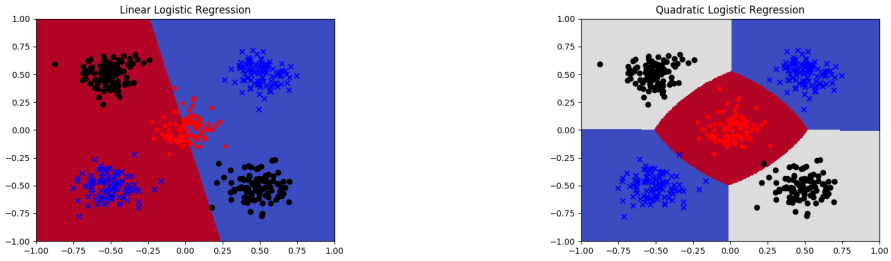
Because of the normalization condition $\sum_{c=1}^{C} p(y_n = c|\mathbf{x}_n;\boldsymbol{\theta}) = 1$, we can set $\mathbf{w}_C = \mathbf{0}$. (For example, in binary logistic regression, where $C = 2$, we only learn a single weight vector.) Therefore the parameters $\boldsymbol{\theta}$ correspond to a weight matrix $\mathbf{w}$ of size $D(C-1)$, where $\mathbf{x}_n \in \mathbb{R}^D$.

Note that this model assumes the labels are mutually exclusive, i.e., there is only one true label. For some applications (e.g., **image tagging**), we want to predict one or more labels for an input; in this case, the output space is the set of subsets of $\{1, \ldots, C\}$. This is called **multi-label classification**, as opposed to **multi-class classification**. This can be viewed as a bit vector, $\mathcal{Y} = \{0, 1\}^C$, where the c'th output is set to 1 if the c'th tag is present. We can tackle this using a modified version of binary logistic regression with multiple outputs:

$$p(\mathbf{y}|\mathbf{x};\boldsymbol{\theta}) = \prod_{c=1}^{C} \text{Ber}(y_c|\sigma(\mathbf{w}_c^\top \mathbf{x})) \tag{10.56}$$

10.3.1 Linear and nonlinear classifiers

Logistic regression computes linear decision boundaries in the input space, as shown in Fig. 10.7(a) for the case where $\mathbf{x} \in \mathbb{R}^2$ and we have $C = 3$ classes. However, we can always transform the inputs



(a) (b)

Figure 10.7: Example of 3-class logistic regression with 2d inputs. (a) Original features. (b) Quadratic features. Generated by code at figures.probml.ai/book1/10.7.

in some way to create nonlinear boundaries. For example, suppose we replace $\mathbf{x} = (x_1, x_2)$ by

$$\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2] \tag{10.57}$$

This lets us create quadratic decision boundaries, as illustrated in Fig. 10.7(b).

10.3.2 Maximum likelihood estimation

In this section, we discuss how to compute the maximum likelihood estimate (MLE) by minimizing the negative log likelihood (NLL).

10.3.2.1 Objective

The NLL is given by

$$\text{NLL}(\boldsymbol{\theta}) = -\log \prod_{n=1}^{N} \prod_{c=1}^{C} \mu_{nc}^{y_{nc}} = -\sum_{n=1}^{N} \sum_{c=1}^{C} y_{nc} \log \mu_{nc} = \sum_{n=1}^{N} \mathbb{H}(\mathbf{y}_n, \boldsymbol{\mu}_n) \tag{10.58}$$

where $\mu_{nc} = p(y_n = c|\mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{S}(f(\mathbf{x}_n; \boldsymbol{\theta}))_c$, $y_{nc} = \mathbb{I}(y_n = c)$ is the one-hot encoding of y_n, and $\mathbb{H}(\mathbf{y}_n, \boldsymbol{\mu}_n)$ is the cross-entropy:

$$\mathbb{H}(\mathbf{p}, \mathbf{q}) = -\sum_{c=1}^{C} p_c \log q_c \tag{10.59}$$

10.3.2.2 Optimizing the objective

To find the optimum, we need to solve $\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{0}$, where $\mathbf{w}$ is a vectorized version of the weight matrix $\mathbf{W}$, and where we are ignoring the bias term for notational simplicity. We can find such a stationary point using any gradient-based optimizer; we give some examples below. But first we derive the gradient and Hessian, and then prove that the objective is convex.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.3.2.3 Deriving the gradient

To derive the gradient of the NLL, we need to use the Jacobian of the softmax function, which is as follows (see Exercise 10.1 for the proof):

$$\frac{\partial \mu_c}{\partial a_j} = \mu_c(\delta_{cj} - \mu_j) \tag{10.60}$$

where $\delta_{cj} = \mathbb{I}(c = j)$. For example, if we have 3 classes, the Jacobian matrix is given by

$$\left[\frac{\partial \mu_c}{\partial a_j}\right]_{cj} = \begin{pmatrix} \mu_1(1-\mu_1) & -\mu_1\mu_2 & -\mu_1\mu_3 \\ -\mu_2\mu_1 & \mu_2(1-\mu_2) & -\mu_2\mu_3 \\ -\mu_3\mu_1 & -\mu_3\mu_2 & \mu_3(1-\mu_3) \end{pmatrix} \tag{10.61}$$

In matrix form, this can be written as

$$\frac{\partial \boldsymbol{\mu}}{\partial \mathbf{a}} = (\boldsymbol{\mu}\mathbf{1}^\top) \odot (\mathbf{I} - \mathbf{1}\boldsymbol{\mu}^\top) \tag{10.62}$$

where $\odot$ is elementwise product, $\boldsymbol{\mu}\mathbf{1}^\top$ copies $\boldsymbol{\mu}$ across each column, and $\mathbf{1}\boldsymbol{\mu}^\top$ copies $\boldsymbol{\mu}$ across each row.

We can now derive the (batch) gradient of the NLL using the chain rule, as follows:

$$\nabla_{\mathbf{w}_j} \text{NLL} = \sum_n \sum_c \frac{\partial \text{NLL}}{\partial \mu_{nc}} \frac{\partial \mu_{nc}}{\partial a_{nj}} \frac{\partial a_{nj}}{\partial \mathbf{w}_j} \tag{10.63}$$

$$= -\sum_n \sum_c \frac{y_{nc}}{\mu_{nc}} \mu_{nc}(\delta_{jc} - \mu_{nj})\mathbf{x}_n \tag{10.64}$$

$$= \sum_n \sum_c y_{nc}(\mu_{nj} - \delta_{jc})\mathbf{x}_n \tag{10.65}$$

$$= \sum_n (\sum_c y_{nc})\mu_{nj}\mathbf{x}_n - \sum_n y_{nj}\mathbf{x}_n \tag{10.66}$$

$$= \sum_n (\mu_{nj} - y_{nj})\mathbf{x}_n \tag{10.67}$$

In matrix notation, we have

$$\mathbf{g}(\mathbf{W}) = \sum_{n=1}^N \mathbf{x}_n(\boldsymbol{\mu}_n - \mathbf{y}_n)^\top \tag{10.68}$$

This has the same form as in the binary logistic regression case, namely an error term times the input $\mathbf{x}_n$, summed over N.

10.3.2.4 Deriving the Hessian

Exercise 10.1 asks you to show that the Hessian of the NLL for multinomial logistic regression is given by

$$\mathbf{H}(\mathbf{W}) = \sum_{n=1}^N (\text{diag}(\boldsymbol{\mu}_n) - \boldsymbol{\mu}_n\boldsymbol{\mu}_n^\top) \otimes (\mathbf{x}_n\mathbf{x}_n^\top) \tag{10.69}$$

where $\mathbf{A} \otimes \mathbf{B}$ is the Kronecker product (Sec. 7.2.5). In other words, the block c, c' submatrix is given by

$$\mathbf{H}_{c,c'}(\mathbf{W}) = \sum_n \mu_{nc}(\delta_{c,c'} - \mu_{n,c'})\mathbf{x}_n\mathbf{x}_n^\top \tag{10.70}$$

For example, if we have 3 features and 2 classes, this becomes

$$\mathbf{H}(\mathbf{W}) = \sum_n \begin{pmatrix} \mu_{n1} - \mu_{n1}^2 & -\mu_{n1}\mu_{n2} \\ -\mu_{n1}\mu_{n2} & \mu_{n2} - \mu_{n2}^2 \end{pmatrix} \otimes \begin{pmatrix} x_{n1}x_{n1} & x_{n1}x_{n2} & x_{n1}x_{n3} \\ x_{n2}x_{n1} & x_{n2}x_{n2} & x_{n2}x_{n3} \\ x_{n3}x_{n1} & x_{n3}x_{n2} & x_{n3}x_{n3} \end{pmatrix} \tag{10.71}$$

$$= \sum_n \begin{pmatrix} (\mu_{n1} - \mu_{n1}^2)\mathbf{X}_n & -\mu_{n1}\mu_{n2}\mathbf{X}_n \\ -\mu_{n1}\mu_{n2}\mathbf{X}_n & (\mu_{n2} - \mu_{n2}^2)\mathbf{X}_n \end{pmatrix} \tag{10.72}$$

where $\mathbf{X}_n = \mathbf{x}_n\mathbf{x}_n^\top$. Exercise 10.1 also asks you to show that this is a positive definite matrix, so the objective is convex.

10.3.3 Gradient-based optimization

It is straightforward to use the gradient in Sec. 10.3.2.3 to derive the SGD algorithm. Similarly, we can use the Hessian in Sec. 10.3.2.4 to derive a second-order optimization method. However, computing the Hessian can be expensive, so it is common to approximate it using quasi-Newton methods, such as limited memory BFGS. (BFGS stands for Broyden, Fletcher, Goldfarb and Shanno.) See Sec. 8.3.2 for details. Another approach, which is similar to IRLS, is described in Sec. 10.3.4.

All of these methods rely on computing the gradient of the log-likelihood, which in turn requires computing normalized probabilities, which can be computed from the logits vector $\mathbf{a} = \mathbf{W}\mathbf{x}$ using

$$p(y = c|\mathbf{x}) = \exp(a_c - \text{lse}(\mathbf{a})) \tag{10.73}$$

where lse is the log-sum-exp function defined in Sec. 2.5.4. For this reason, many software libraries define a version of the cross-entropy loss that takes unnormalized logits as input.

10.3.4 Bound optimization

In this section, we consider an approach for fitting logistic regression using a class of algorithms known as bound optimization, which we describe in Sec. 8.7. The basic idea is to iteratively construct a lower bound on the function you want to maximize, and then to update the bound, so it "pushes up" on the true function. Optimizing the bound is often easier than updating the function directly.

If $\text{LL}(\boldsymbol{\theta})$ is a concave function we want to maximize, then one way to obtain a valid lower bound is to use a bound on its Hessian, i.e., to find a negative definite matrix $\mathbf{B}$ such that $\mathbf{H}(\boldsymbol{\theta}) \succ \mathbf{B}$. In this case, one can show that

$$\text{LL}(\boldsymbol{\theta}) \geq \text{LL}(\boldsymbol{\theta}^t) + (\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\top \mathbf{g}(\boldsymbol{\theta}^t) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\top \mathbf{B}(\boldsymbol{\theta} - \boldsymbol{\theta}^t) \tag{10.74}$$

where $\mathbf{g}(\boldsymbol{\theta}^t) = \nabla \text{LL}(\boldsymbol{\theta}^t)$. Defining $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ as the right-hand-side of Eq. (10.74), the update becomes

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \mathbf{B}^{-1}\mathbf{g}(\boldsymbol{\theta}^t) \tag{10.75}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is similar to a Newton update, except we use $\mathbf{B}$, which is a fixed matrix, rather than $\mathbf{H}(\boldsymbol{\theta}^t)$, which changes at each iteration. This can give us some of the advantages of second order methods at lower computational cost.

Let us now apply this to logistic regression, following [Kri+05], Let $\boldsymbol{\mu}_n(\mathbf{w}) = [p(y_n = 1|\mathbf{x}_n, \mathbf{w}), \ldots, p(y_n = C|\mathbf{x}_n, \mathbf{w})]$ and $\mathbf{y}_n = [\mathbb{I}(y_n = 1), \ldots, \mathbb{I}(y_n = C)]$. We want to *maximize* the log-likelihood, which is as follows:

$$\mathrm{LL}(\mathbf{w}) = \sum_{n=1}^{N}\left[\sum_{c=1}^{C} y_{nc}\mathbf{w}_c^\top \mathbf{x}_n - \log\sum_{c=1}^{C}\exp(\mathbf{w}_c^\top \mathbf{x}_n)\right] \tag{10.76}$$

The gradient is given by the following (see Sec. 10.3.2.3 for details of the derivation):

$$\mathbf{g}(\mathbf{w}) = \sum_{n=1}^{N}(\mathbf{y}_n - \boldsymbol{\mu}_n(\mathbf{w}))\otimes\mathbf{x}_n \tag{10.77}$$

where $\otimes$ denotes kronecker product (which, in this case, is just outer product of the two vectors). The Hessian is given by the following (see Sec. 10.3.2.4 for details of the derivation):

$$\mathbf{H}(\mathbf{w}) = -\sum_{n=1}^{N}(\mathrm{diag}(\boldsymbol{\mu}_n(\mathbf{w})) - \boldsymbol{\mu}_n(\mathbf{w})\boldsymbol{\mu}_n(\mathbf{w})^\top)\otimes(\mathbf{x}_n\mathbf{x}_n^\top) \tag{10.78}$$

We can construct a lower bound on the Hessian, as shown in [Boh92]:

$$\mathbf{H}(\mathbf{w}) \succ -\frac{1}{2}[\mathbf{I} - \mathbf{1}\mathbf{1}^\top/C]\otimes\left(\sum_{n=1}^{N}\mathbf{x}_n\mathbf{x}_n^\top\right) \triangleq \mathbf{B} \tag{10.79}$$

where $\mathbf{I}$ is a C-dimensional identity matrix, and $\mathbf{1}$ is a C-dimensional vector of all 1s.[1] In the binary case, this becomes

$$\mathbf{H}(\mathbf{w}) \succ -\frac{1}{2}\left(1 - \frac{1}{2}\right)\left(\sum_{n=1}^{N}\mathbf{x}_n\mathbf{x}_n^\top\right) = -\frac{1}{4}\mathbf{X}^\top\mathbf{X} \tag{10.80}$$

This follows since $\mu_n \leq 0.5$ so $-(\mu_n - \mu_n^2) \geq -0.25$.

We can use this lower bound to construct an MM algorithm to find the MLE. The update becomes

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \mathbf{B}^{-1}\mathbf{g}(\mathbf{w}^t) \tag{10.81}$$

This iteration can be faster than IRLS (Sec. 10.2.6) since we can precompute $\mathbf{B}^{-1}$ in time independent of N, rather than having to invert the Hessian at each iteration. For example, let us consider the binary case, so $\mathbf{g}^t = \nabla\mathrm{LL}(\mathbf{w}^t) = \mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu}^t)$, where $\boldsymbol{\mu}^t = [p_n(\mathbf{w}^t), (1 - p_n(\mathbf{w}^t))]_{n=1}^N$. The update becomes

$$\mathbf{w}^{t+1} = \mathbf{w}^t - 4(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{g}^t \tag{10.82}$$

Compare this to Eq. (10.37), which has the following form:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \mathbf{H}^{-1}\mathbf{g}(\mathbf{w}^t) = \mathbf{w}^t - (\mathbf{X}^\top\mathbf{S}^t\mathbf{X})^{-1}\mathbf{g}^t \tag{10.83}$$

where $\mathbf{S}^t = \mathrm{diag}(\boldsymbol{\mu}^t \odot (1 - \boldsymbol{\mu}^t))$. We see that Eq. (10.82) is faster to compute, since we can precompute the constant matrix $(\mathbf{X}^\top\mathbf{X})^{-1}$.

1. If we enforce that $\mathbf{w}_C = \mathbf{0}$, we can use $C - 1$ dimensions for these vectors / matrices.

10.3.5 MAP estimation

In Sec. 10.2.7 we discussed the benefits of ℓ_2 regularization for binary logistic regression. These benefits hold also in the multi-class case. However, there is also an additional, and surprising, benefit to do with **identifiability** of the parameters, as pointed out in [HTF09, Ex.18.3]. (We say that the parameters are identifiable if there is a unique value that maximizes the likelihood; equivalently, we require that the NLL be *strictly* convex.)

To see why identifiability is an issue, recall that multiclass logistic regression has the form

$$p(y=c|\mathbf{x},\mathbf{W}) = \frac{\exp(\mathbf{w}_c^T\mathbf{x})}{\sum_{k=1}^{C}\exp(\mathbf{w}_k^T\mathbf{x})} \tag{10.84}$$

where $\mathbf{W}$ is a $C \times C$ weight matrix. We can arbitrarily define $\mathbf{w}_c = \mathbf{0}$ for one of the classes, say $c = C$, since $p(y = C|\mathbf{x},\mathbf{W}) = 1 - \sum_{c=1}^{C-1} p(y = c|\mathbf{x},\mathbf{w})$. In this case, the model has the form

$$p(y=c|\mathbf{x},\mathbf{W}) = \frac{\exp(\mathbf{w}_c^T\mathbf{x})}{1+\sum_{k=1}^{C-1}\exp(\mathbf{w}_k^T\mathbf{x})} \tag{10.85}$$

If we don't "clamp" one of the vectors to some constant value, the parameters will be unidentifiable.

However, suppose we don't clamp $\mathbf{w}_c = \mathbf{0}$, so we are using Equation 10.84, but we add ℓ_2 regularization by optimizing

$$\text{PNLL}(\mathbf{w}) = -\sum_{n=1}^{N}\log p(y_n|\mathbf{x}_n,\mathbf{W}) + \lambda\sum_{c=1}^{C}||\mathbf{w}_c||_2^2 \tag{10.86}$$

At the optimum we have $\sum_{c=1}^{C}\hat{w}_{cj} = 0$ for $j = 1 : D$, so the weights automatically satisfy a sum-to-zero constraint, thus making them uniquely identifiable.

To see why, note that at the optimum we have

$$\nabla\text{NLL}(\mathbf{w}) + 2\lambda\mathbf{w} = \mathbf{0} \tag{10.87}$$

$$\sum_n(\mathbf{y}_n - \boldsymbol{\mu}_n)\otimes\mathbf{x}_n = \lambda\mathbf{w} \tag{10.88}$$

Hence for any feature dimension j we have

$$\lambda\sum_c w_{cj} = \sum_n\sum_c(y_{nc}-\mu_{nc})x_{nj} = \sum_n(\sum_c y_{nc} - \sum_c\mu_{nc})x_{nj} = \sum_n(1-1)x_{nj} = 0 \tag{10.89}$$

Thus if $\lambda > 0$ we have $\sum_c\hat{w}_{cj} = 0$, so the weights will sum to zero across classes for each feature dimension.

10.3.6 Maximum entropy classifiers

Recall that the multinomial logistic regression model can be written as

$$p(y=c|\mathbf{x},\mathbf{W}) = \frac{\exp(\mathbf{w}_c^\top\mathbf{x})}{Z(\mathbf{w},\mathbf{x})} = \frac{\exp(\mathbf{w}_c^\top\mathbf{x})}{\sum_{c'=1}^{C}\exp(\mathbf{w}_{c'}^\top\mathbf{x})} \tag{10.90}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $Z(\mathbf{w}, \mathbf{x}) = \sum_c \exp(\mathbf{w}_c^\top \mathbf{x})$ is the partition function (normalization constant). This uses the same features, but a different weight vector, for every class. There is a slight extension of this model that allows us to use features that are class-dependent. This model can be written as

$$p(y = c|\mathbf{x}, \mathbf{w}) = \frac{1}{Z(\mathbf{w}, \mathbf{x})} \exp(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}, c)) \tag{10.91}$$

where $\boldsymbol{\phi}(\mathbf{x}, c)$ is the feature vector for class c. This is called a **maximum entropy classifer**, or **maxent classifier** for short. (The origin of this term is explained in Sec. 12.2.6.)

Maxent classifiers include multinomial logistic regression as a special case. To see this let $\mathbf{w} = [\mathbf{w}_1, \ldots, \mathbf{w}_C]$, and define the feature vector as follows:

$$\boldsymbol{\phi}(\mathbf{x}, c) = [\mathbf{0}, \ldots, \mathbf{x}, \ldots, \mathbf{0}] \tag{10.92}$$

where $\mathbf{x}$ is embedded in the c'th block, and the remaining blocks are zero. In this case, $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}, c) = \mathbf{w}_c^\top \mathbf{x}$, so we recover multinomial logistic regression.

Maxent classifiers are very widely used in the field of natural language processing. For example, consider the problem of **semantic role labeling**, where we classify a word $\mathbf{x}$ into a semantic role y, such as person, place or thing. We might define (binary) features such as the following:

$$\phi_1(\mathbf{x}, y) = \mathbb{I}(y = \text{person} \wedge \mathbf{x} \text{ occurs after "Mr." or "Mrs"}) \tag{10.93}$$

$$\phi_2(\mathbf{x}, y) = \mathbb{I}(y = \text{person} \wedge \mathbf{x} \text{ is in whitelist of common names}) \tag{10.94}$$

$$\phi_3(\mathbf{x}, y) = \mathbb{I}(y = \text{place} \wedge \mathbf{x} \text{ is in Google maps}) \tag{10.95}$$

$$\vdots$$

We see that the features we use depend on the label.

There are two main ways of creating these features. The first is to manually specify many possibly useful features using various templates, and then use a feature selection algorithm, such as the group lasso method of Sec. 11.5.7. The second is to incrementally add features to the model, using a heuristic feature generation method.

10.3.7 Hierarchical classification

Sometimes the set of possible labels can be structured into a **hierarchy** or **taxonomy**. For example, we might want to predict what kind of an animal is in an image: it could be a dog or a cat; if it is a dog, it could be a golden retriever or a German shepherd, etc. Intuitively, it makes sense to try to predict the most precise label for which we are confident [Den+12], that is, the system should "hedge its bets".

One simple way to achieve this, proposed in [RF17], is as follows. First, create a model with a binary output label for every possible node in the tree. Before training the model, we will use **label smearing**, so that a label is propagated to all of its parents (**hypernyms**). For example, if an image is labeled "golden retriever", we will also label it "dog". If we train a multi-label classifier (which produces a vector $p(\mathbf{y}|\mathbf{x})$ of binary labels) on such smeared data, it will perform hierarchical classification, predicting a set of labels at different levels of abstraction.

However, this method could predict "golden retriever", "cat" and "bird" all with probability 1.0, since the model does not capture the fact that some labels are mutually exclusive. To prevent this,

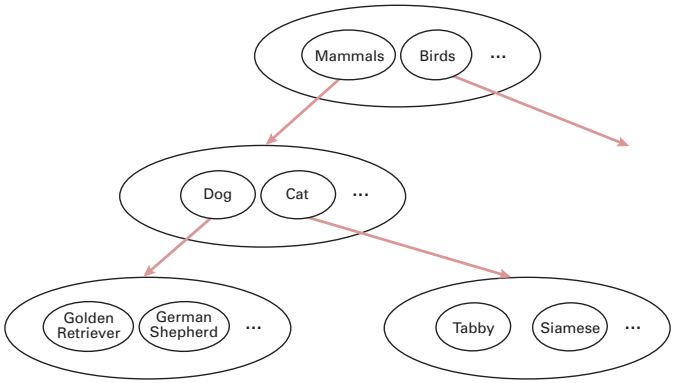


Figure 10.8: A simple example of a label hierarchy. Nodes within the same ellipse have a mutual exclusion relationship between them.

we can add a mutual exclusion constraint between all label nodes which are siblings, as shown in Fig. 10.8. For example, this model enforces that $p(\text{mammal}|\mathbf{x}) + p(\text{bird}|\mathbf{x}) = 1$, since these two labels are children of the root node. We can further partition the mammal probability into dogs and cats, so we have $p(\text{dog}|\mathbf{x}) + p(\text{cat}|\mathbf{x}) = p(\text{mammal}|\mathbf{x})$.

[Den+14; Din+15] generalize the above method by using a conditional graphical model where the graph structure can be more complex than a tree. In addition, they allow for soft constraints between labels, in addition to hard constraints.

10.3.8 Handling large numbers of classes

In this section, we discuss some issues that arise when there are a large number of potential labels, e.g., if the labels correspond to words from a language.

10.3.8.1 Hierarchical softmax

In regular softmax classifiers, computing the normalization constant in the softmax, which is needed to compute the gradient of the log likelihood, takes $O(C)$ time, which can become the bottleneck of C is large. However, if we structure the labels as a tree, we can compute the probability of any label in $O(\log C)$ time, by multiplying the probabilities of each edge on the path from the root to the leaf. For example, consider the tree in Fig. 10.9. We have

$$p(y = \text{I'm}|C) = 0.57 \times 0.68 \times 0.72 = 0.28 \tag{10.96}$$

Thus we replace the "flat" output softmax with a tree-structured sequence of binary classifiers. This is called **hierarchical softmax** [Goo01; MB05].

A good way to structure such a tree is to use Huffman encoding, where the most frequent labels are placed near the top of the tree, as suggested in [Mik+13a]. (For a different appproach, based on clustering the most common labels together, see [Gra+17]. And for yet another approach, based on sampling labels, see [Tit16].)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

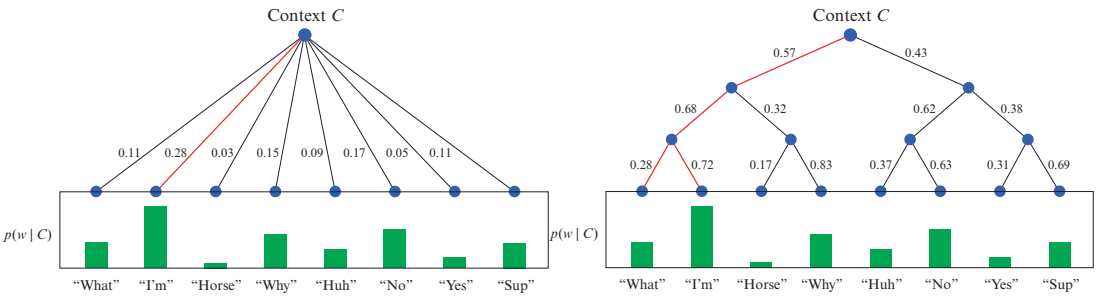


Figure 10.9: A flat and hierarchical softmax model $p(w|C)$, where C are the input features (context) and w is the output label (word). Adapted from https://www.quora.com/What-is-hierarchical-softmax.

10.3.8.2 Class imbalance and the long tail

Another issue that often arises when there are a large number of classes is that for most classes, we may have very few examples. More precisely, if N_c is the number of examples of class c, then the empirical distribution $p(N_1, \ldots, N_C)$ may have a **long tail**. The result is an extreme form of **class imbalance** (see e.g., [ASR15]). Since the rare classes will have a smaller effect on the overall loss than the common classes, the model may "focus its attention" on the common classes.

One method that can help is to set the bias terms $\mathbf{b}$ such that $\mathcal{S}(\mathbf{b})_c = N_c/N$; such a model will match the empirical label prior even when using weights of $\mathbf{w} = \mathbf{0}$. As the weights are adjusted, the model can learn input-dependent deviations from this prior.

Another common approach is to resample the data to make it more balanced, before (or during) training. In particular, suppose we sample a datapoint from class c with probability

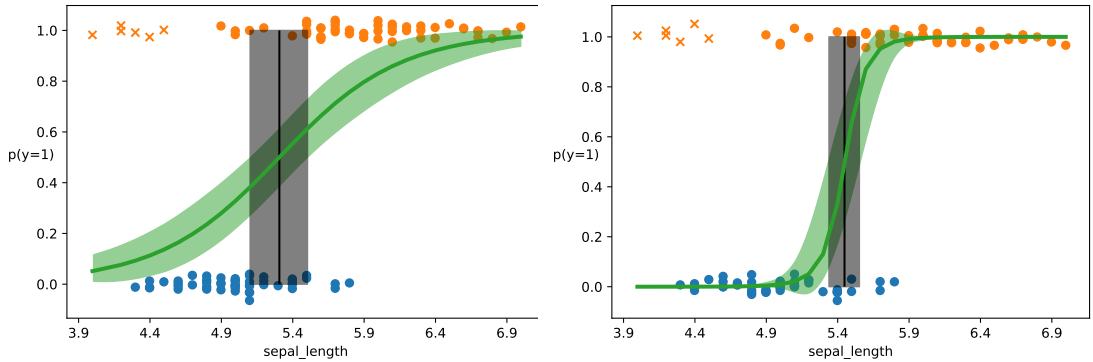
$$p_c = \frac{N_c^q}{\sum_i^C N_i^q} \tag{10.97}$$

If we set $q = 1$, we recover standard **instance-balanced sampling**, where $p_c \propto N_c$; this common classes will be sampled more than rare classes. If we set $q = 0$, we recover **class-balanced sampling**, where $p_c = 1/C$; this can be thought of as first sampling a class uniformly at random, and then sampling an instance of this class. Finally, we can consider other options, such as $q = 0.5$, which is known as **square-root sampling** [Mah+18].

Yet another method that is simple and can easily handle the long tail is to use the **nearest class mean classifier**. This has the form

$$f(\mathbf{x}) = \operatorname*{argmin}_c ||\mathbf{x} - \boldsymbol{\mu}_c||_2^2 \tag{10.98}$$

where $\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n$ is the mean of the features belonging to class c. This induces a softmax posterior, as we discussed in Sec. 9.2.5. We can get much better results if we first use a neural network (see Part III) to learn good features, by training a DNN classifier with cross-entropy loss on the original unbalanced data. We then replace $\mathbf{x}$ with $\boldsymbol{\phi}(\mathbf{x})$ in Eq. (10.98). This simple approach can give very good performance on long-tailed distributions [Kan+20].



(a) (b)

Figure 10.10: (a) Logistic regression on some data with outliers (denoted by x). Training points have been (vertically) jittered to avoid overlapping too much. Vertical line is the decision boundary, and its posterior credible interval. (b) Same as (a) but using robust model, with a mixture likelihood. Adapted from Figure 4.13 of [Mar18]. Generated by code at figures.probml.ai/book1/10.10.

10.4 Robust logistic regression *

Sometimes we have **outliers** in our data, which are often due to labeling errors, also called **label noise**. To prevent the model from being adversely affected by such contamination, we will use **robust logistic regression**. In this section, we discuss some approaches to this problem.

10.4.1 Mixture model for the likelihood

One of the simplest ways to define a robust logistic regression model is to modify the likelihood so that it predicts that each output label y is generated uniformly at random with probability π, and otherwise is generated using the usual conditional model. In the binary case, this becomes

$$p(y|\mathbf{x}) = \pi \text{Ber}(y|0.5) + (1-\pi)\text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x})) \tag{10.99}$$

This approach, of using a mixture model for the observation model to make it robust, can be applied to many different models (e.g., DNNs).

We can fit this model using standard methods, such as SGD or Bayesian inference methods such as MCMC. For example, let us create a "contaminated" version of the 1d, two-class Iris dataset that we discussed in Sec. 4.6.7.2. We will add add 6 examples of class 1 (versicolor) with abnormally low sepal length. In Fig. 10.10a, we show the results of fitting a standard (Bayesian) logistic regression model to this dataset. In Fig. 10.10b, we show the results of fitting the above robust model. In the latter case, we see that the decision boundary is similar to the one we inferred from non-contaminated data, as shown in Fig. 4.20b. We also see that the posterior uncertainty about the decision boundary's location is smaller than when using a non-robust model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.4.2 Bi-tempered loss

In this section, we present an approach to robust logistic regression proposed in [Ami+19].

The first observation is that examples that are far from the decision boundary, but mislabeled, will have undue adverse affect on the model if the loss function is convex [LS10]. This can be overcome by replacing the usual cross entropy loss with a "tempered" version, that uses a temperature parameter $0 \leq t_1 < 1$ to ensure the loss from outliers is bounded. In particular, consider the standard relative entropy loss function:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \mathbb{H}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_c y_c \log \hat{y}_c \tag{10.100}$$

where $\mathbf{y}$ is the true label distribution (often one-hot) and $\hat{\mathbf{y}}$ is the predicted distribution. We define the **tempered cross entropy** loss as follows:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_c \left[y_c (\log_{t_1} y_c - \log_{t_1} \hat{y}_c) - \frac{1}{2-t_1}(y_c^{2-t_1} - \hat{y}_c^{2-t_1}) \right] \tag{10.101}$$

which simplifes to the following when the true distribution $\mathbf{y}$ is one-hot, with all its mass on class c:

$$\mathcal{L}(c, \hat{\mathbf{y}}) = -\log_{t_1} \hat{y}_c - \frac{1}{2-t_1}\left(1 - \sum_{c'=1}^{C} \hat{y}_{c'}^{2-t_1}\right) \tag{10.102}$$

Here $\log_t$ is tempered version of the log function:

$$\log_t(x) \triangleq \frac{1}{1-t}(x^{1-t} - 1) \tag{10.103}$$

This is mononotically increasing and concave, and reduces to the standard (natural) logarithm when $t = 1$. (Similarly, tempered cross entropy reduces to standard cross entropy when $t = 1$.) However, the tempered log function is bounded from below by $-1/(1-t)$ for $0 \leq t < 1$, and hence the cross entropy loss is bounded from above (see Fig. 10.11).

The second observation is that examples that are near the decision boundary, but mislabeled, need to use a transfer function (that maps from activations $\mathbb{R}^C$ to probabilities $[0,1]^C$) that has heavier tails than the softmax, which is based on the exponential, so it can "look past" the neighborhood of the immediate examples. In particular, the standard softmax is defined by

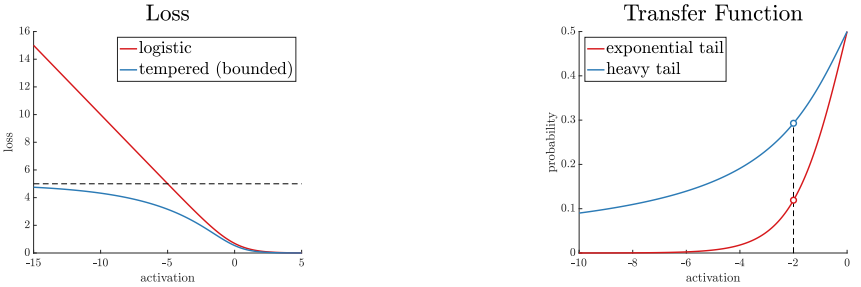
$$\hat{y}_c = \frac{a_c}{\sum_{c'=1}^{C} \exp(a_{c'})} = \exp\left[a_c - \log \sum_{c'=1}^{C} \exp(a_{c'}) \right] \tag{10.104}$$

where $\mathbf{a}$ is the logits vector. We can make a heavy tailed version by using the **tempered softmax**, which uses a temperature parameter $t_2 > 1 > t_1$ as follows:

$$\hat{y}_c = \exp_{t_2}(a_c - \lambda_{t_2}(\mathbf{a})) \tag{10.105}$$

where

$$\exp_t(x) \triangleq [1 + (1-t)x]_+^{1/(1-t)} \tag{10.106}$$



(a) (b)

Figure 10.11: (a) Illustration of logistic and tempered logistic loss with $t_1 = 0.8$. (b) Illustration of sigmoid and tempered sigmoid transfer function with $t_2 = 2.0$. From https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html . Used with kind permission of Ehsan Amid.

is a tempered version of the exponential function. (This reduces to the standard exponental function as $t \to 1$.) In Fig. 10.11(right), we show that the tempered softmax (in the two-class case) has heavier tails, as desired.

All that remains is a way to compute $\lambda_{t_2}(\mathbf{a})$. This must satisfy the following fixed point equation:

$$\sum_{c=1}^{C} \exp_{t_2}(a_c - \lambda(\mathbf{a})) = 1 \tag{10.107}$$

We can solve for λ using binary search, or by using the iterative procedure in Algorithm 3.

Algorithm 3: Iterative algorithm for computing $\lambda(\mathbf{a})$ in Eq. (10.107). From [AWS19].

Input: logits $\mathbf{a}$, temperature $t > 1$;
$\mu := \max(\mathbf{a})$;
$\mathbf{a} := \mathbf{a} - \mu$;
while $\mathbf{a}$ *not converged* **do**
  $Z(\mathbf{a}) := \sum_{c=1}^{C} \exp_t(a_c)$;
  $\mathbf{a} := Z(\mathbf{a})^{1-t}(\mathbf{a} - \mu\mathbf{1})$;
Return $-\log_t \frac{1}{Z(\mathbf{a})} + \mu$

Combining the tempered softmax with the tempered cross entropy results in a method called **bi-tempered logistic regression**. In Fig. 10.12, we show an example of this in 2d. The top row is standard logistic regression, the bottom row is bi-tempered. The first column is clean data. The second column has label noise near the boundary. The robust version uses $t_1 = 1$ (standard cross entropy) but $t_2 = 4$ (tempered softmax with heavy tails). The third column has label noise far from the boundary. The robust version uses $t_1 = 0.2$ (tempered cross entropy with bounded loss) but $t_2 = 1$ (standard softmax). The fourth column has both kinds of noise; in this case, the robust version uses $t_1 = 0.2$ and $t_2 = 4$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

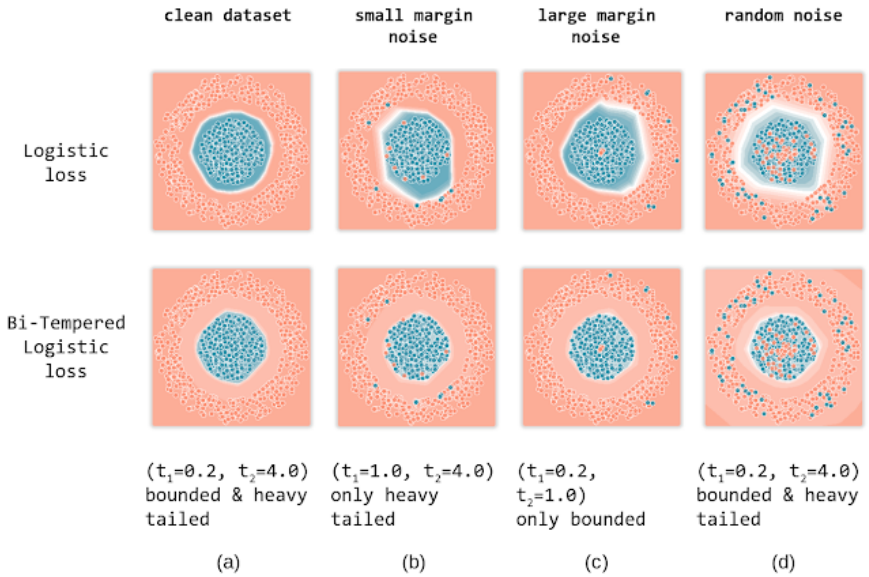


Figure 10.12: Illustration of standard and bi-tempered logistic regression on data with label noise. From https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html . Used with kind permission of Ehsan Amid.

10.5 Bayesian logistic regression *

So far we have focused on point estimates of the parameters, either the MLE or the MAP estimate. However, in some cases we want to compute the posterior, $p(\mathbf{w}|\mathcal{D})$, in order to capture our uncertainty. This can be particularly useful in settings where we have little data, and where choosing the wrong decision may be costly; this arises in online advertising, for example (see Sec. 5.3).

Unlike with linear regression, it is not possible to compute the posterior exactly for a logistic regression model. A wide range of approximate algorithms can be used,. In this section, we use one of the simplest, known as the Laplace approximation (Sec. 4.6.8.2). See the sequel to this book, [Mur22] for more advanced approximations.

10.5.1 Laplace approximation

As we discuss in Sec. 4.6.8.2, the Laplace approximation approximates the posterior using a Gaussian. The mean of the Gaussian is equal to the MAP estimate $\hat{\mathbf{w}}$, and the covariance is equal to the inverse Hessian $\mathbf{H}$ computed at the MAP estimate, i.e., $p(\mathbf{w}|\mathcal{D}) \approx \mathcal{N}(\mathbf{w}|\hat{\mathbf{w}}, \mathbf{H}^{-1})$, We can find the mode using a standard optimization method (see Sec. 10.2.7), and we use can use the results from Sec. 10.2.3.4 to compute the Hessian at the mode.

As an example, consider the data illustrated in Fig. 10.13(a). There are many parameter settings that correspond to lines that perfectly separate the training data; we show 4 example lines. The likelihood surface is shown in Fig. 10.13(b). The diagonal line connects the origin to the point in the

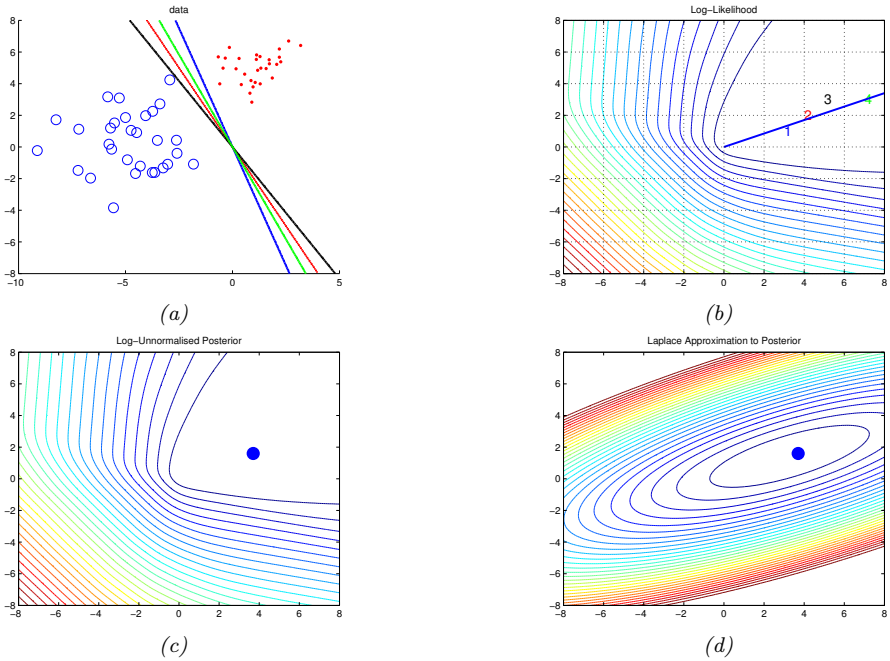


Figure 10.13: (a) Illustration of the data. (b) Log-likelihood for a logistic regression model. The line is drawn from the origin in the direction of the MLE (which is at infinity). The numbers correspond to 4 points in parameter space, corresponding to the lines in (a). (c) Unnormalized log posterior (assuming vague spherical prior). (d) Laplace approximation to posterior. Adapted from a figure by Mark Girolami. Generated by code at figures.probml.ai/book1/10.13.

grid with maximum likelihood, $\hat{\mathbf{w}}_{\text{mle}} = (8.0, 3.4)$. (The unconstrained MLE has $||\mathbf{w}|| = \infty$, as we discussed in Sec. 10.2.7; this point can be obtained by following the diagonal line infinitely far to the right.)

For each decision boundary in Fig. 10.13(a), we plot the corresponding parameter vector in Fig. 10.13(b). These parameters values are $\mathbf{w}_1 = (3, 1)$, $\mathbf{w}_2 = (4, 2)$, $\mathbf{w}_3 = (5, 3)$, and $\mathbf{w}_4 = (7, 3)$. These points all approximately satisfy $\mathbf{w}_i(1)/\mathbf{w}_i(2) \approx \hat{\mathbf{w}}_{\text{mle}}(1)/\hat{\mathbf{w}}_{\text{mle}}(2)$, and hence are close to the orientation of the maximum likelihood decision boundary. The points are ordered by increasing weight norm (3.16, 4.47, 5.83, and 7.62).

To ensure a unique solution, we use a (spherical) Gaussian prior centered at the origin, $\mathcal{N}(\mathbf{w}|\mathbf{0}, \sigma^2\mathbf{I})$. The value of σ^2 controls the strength of the prior. If we set $\sigma^2 = \infty$, we force the MAP estimate to be $\mathbf{w} = \mathbf{0}$; this will result in maximally uncertain predictions, since all points $\mathbf{x}$ will produce a predictive distribution of the form $p(y = 1|\mathbf{x}) = 0.5$. If we set $\sigma^2 = 0$, the MAP estimate becomes the MLE, resulting in minimally uncertain predictions. (In particular, all positively labeled points will have $p(y = 1|\mathbf{x}) = 1.0$, and all negatively labeled points will have $p(y = 1|\mathbf{x}) = 0.0$, since the data is separable.) As a compromise (to make a nice illustration), we pick the value $\sigma^2 = 100$.

Multiplying this prior by the likelihood results in the unnormalized posterior shown in Fig. 10.13(c).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

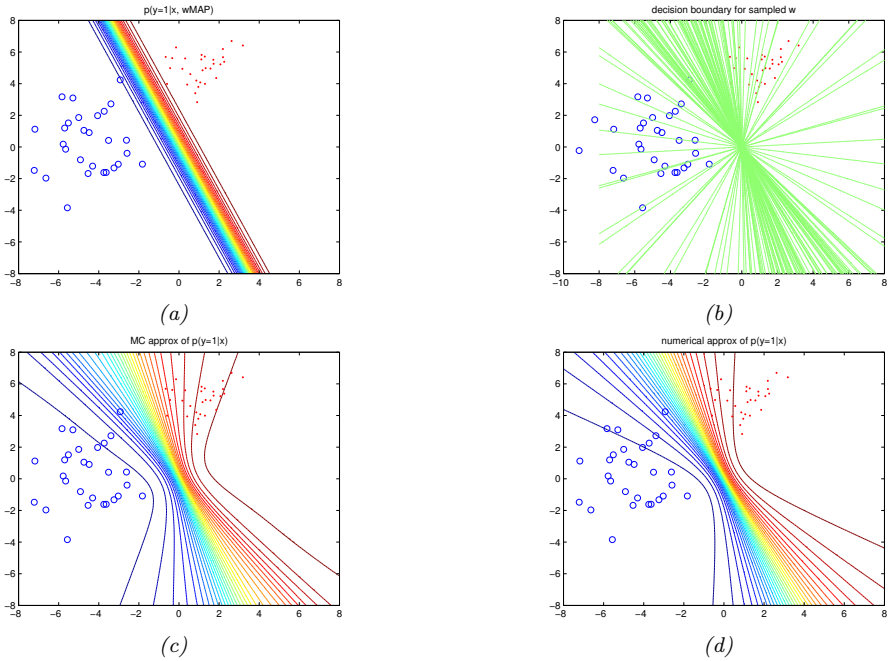


Figure 10.14: Posterior predictive distribution for a logistic regression model in 2d. Top left: contours of $p(y = 1|\mathbf{x}, \hat{\mathbf{w}}_{map})$. Top right: samples from the posterior predictive distribution. Bottom left: Averaging over these samples. Bottom right: moderated output (probit approximation). Adapted from a figure by Mark Girolami. Generated by code at figures.probml.ai/book1/10.14.

The MAP estimate is shown by the blue dot. The Laplace approximation to this posterior is shown in Fig. 10.13(d). We see that it gets the mode correct (by construction), but the shape of the posterior is somewhat distorted. (The southwest-northeast orientation captures uncertainty about the magnitude of $\mathbf{w}$, and the southeast-northwest orientation captures uncertainty about the orientation of the decision boundary.)

In Fig. 10.14, we show contours of the posterior predictive distribution. Fig. 10.14(a) shows the plugin approximation using the MAP estimate. We see that there is no uncertainty about the decision boundary, even though we are generating probabilistic predictions over the labels. Fig. 10.14(b) shows what happens when we plug in samples from the Gaussian posterior. Now we see that there is considerable uncertainty about the orientation of the "best" decision boundary. Fig. 10.14(c) shows the average of these samples. By averaging over multiple predictions, we see that the uncertainty in the decision boundary "splays out" as we move further from the training data. Fig. 10.14(d) shows that the probit approximation gives very similar results to the Monte Carlo approximation.

10.5.2 Approximating the posterior predictive

The posterior $p(\mathbf{w}|\mathcal{D})$ tells us everything we know about the parameters of the model given the data. However, in machine learning applications, the main task of interest is usually to predict an output y

given an input $\mathbf{x}$, rather than to try to understand the parameters of our model. Thus we need to compute the **posterior predictive distribution**

$$p(y|\mathbf{x}, \mathcal{D}) = \int p(y|\mathbf{x}, \mathbf{w}) p(\mathbf{w}|\mathcal{D}) d\mathbf{w} \tag{10.108}$$

As we discussed in Sec. 4.6.7.1, a simple approach to this is to first compute a point estimate $\hat{\mathbf{w}}$ of the parameters, such as the MLE or MAP estimate, and then to ignore all posterior uncertainty, by assuming $p(\mathbf{w}|\mathcal{D}) = \delta(\mathbf{w} - \hat{\mathbf{w}})$. In this case, the above integral reduces to the following plugin approximation:

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) \approx \int p(\mathbf{y}|\mathbf{x}, \mathbf{w}) \delta(\mathbf{w} - \hat{\mathbf{w}}) d\mathbf{w} = p(\mathbf{y}|\mathbf{x}, \hat{\mathbf{w}}) \tag{10.109}$$

However, if we want to compute uncertainty in our predictions, we should use a non-degenerate posterior. It is common to use a Gaussian posterior, as we will see. But we still need to approximate the integral in Eq. (10.108). We discuss some approaches to this below.

10.5.2.1 Monte Carlo approximation

The simplest approach is to use a **Monte Carlo approximation** to the integral. This means we draw S samples from the posterior, $\mathbf{w}_s \sim p(\mathbf{w}|\mathcal{D})$. and then compute

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} \sigma(\mathbf{w}_s^\top \mathbf{x}) \tag{10.110}$$

10.5.2.2 Probit approximation

Although the Monte Carlo approximation is simple, it can be slow, since we need to draw S samples *at test time* for each input $\mathbf{x}$. Fortunately, if $p(\mathbf{w}|\mathcal{D}) = \mathcal{N}(\mathbf{w}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, there is a simple yet accurate deterministic approximation, first suggested in [SL90]. To explain this approximation, we follow the presentation of [Bis06, p219]. The key observation is that the sigmoid function $\sigma(a)$ is similar in shape to the Gaussian cdf (see Sec. 2.6.1) $\Phi(a)$. In particular we have $\sigma(a) \approx \Phi(\lambda a)$, where $\lambda^2 = \pi/8$ ensures the two functions have the same slope at the origin. This is useful since we can integrate a Gaussian cdf wrt a Gaussian pdf exactly:

$$\int \Phi(\lambda a) \mathcal{N}(a|m, v) da = \Phi\left(\frac{m}{(\lambda^{-2} + v)^{\frac{1}{2}}}\right) = \Phi\left(\frac{\lambda m}{(1 + \lambda^2 v)^{\frac{1}{2}}}\right) \approx \sigma(\kappa(v) m) \tag{10.111}$$

where we have defined

$$\kappa(v) \triangleq (1 + \pi v/8)^{-\frac{1}{2}} \tag{10.112}$$

Thus if we define $a = \mathbf{x}^\top \mathbf{w}$, we have

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \sigma(\kappa(v) m) \tag{10.113}$$

$$m = \mathbb{E}[a] = \mathbf{x}^\top \boldsymbol{\mu} \tag{10.114}$$

$$v = \mathbb{V}[a] = \mathbb{V}\left[\mathbf{x}^\top \mathbf{w}\right] = \mathbf{x}^\top \boldsymbol{\Sigma} \mathbf{x} \tag{10.115}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where we used Eq. (2.159) in the last line. Since Φ^{-1} is known as the probit function, we will call this the **probit approximation**.

Using Eq. (10.113) results in predictions that are less extreme (in terms of their confidence) than the plug-in estimate. To see this, note that $0 < \kappa(v) < 1$ and hence $\kappa(v)m < m$, so $\sigma(\kappa(v)m)$ is closer to 0.5 than $\sigma(m)$ is. However, the decision boundary itself will not be affected. To see this, note that the decision boundary is the set of points $\mathbf{x}$ for which $p(y = 1|\mathbf{x}, \mathcal{D}) = 0.5$. This implies $\kappa(v)m = 0$, which implies $m = \overline{\mathbf{w}}^\top \mathbf{x} = 0$; but this is the same as the decision boundary from the plugin estimate. Thus "being Bayesian" doesn't change the misclassification rate (in this case), but it does change the confidence estimates of the model, which can be important, as we illustrate in Sec. 10.5.1.

In the multiclass case, [Nab01, p353] suggests the following extension of the Spiegelhalter method of Sec. 10.5.2.2:

$$p(y = c|\mathbf{x}, \mathcal{D}) \approx \frac{\exp(\kappa(v_c)m_c)}{\sum_{c'} \exp(\kappa(v_{c'})m_{c'})} \tag{10.116}$$

$$m_c = \overline{\mathbf{m}}_c^\top \mathbf{x} \tag{10.117}$$

$$v_c = \mathbf{x}^\top \mathbf{V}_{c,c} \mathbf{x} \tag{10.118}$$

where κ is defined in Eq. (10.112). Unlike the binary case, taking into account posterior covariance gives different predictions than the plug-in approach (see Exercise 3.10.3 of [RW06]).

10.6 Exercises

Exercise 10.1 [Gradient and Hessian of log-likelihood for multinomial logistic regression]

a. Let $\mu_{ik} = \mathcal{S}(\boldsymbol{\eta}_i)_k$, where $\boldsymbol{\eta}_i = \mathbf{w}^T \mathbf{x}_i$. Show that the Jacobian of the softmax is

$$\frac{\partial \mu_{ik}}{\partial \eta_{ij}} = \mu_{ik}(\delta_{kj} - \mu_{ij}) \tag{10.119}$$

where $\delta_{kj} = I(k = j)$.

b. Hence show that the gradient of the NLL is given by

$$\nabla_{\mathbf{w}_c} \ell = \sum_i (y_{ic} - \mu_{ic}) \mathbf{x}_i \tag{10.120}$$

Hint: use the chain rule and the fact that $\sum_c y_{ic} = 1$.

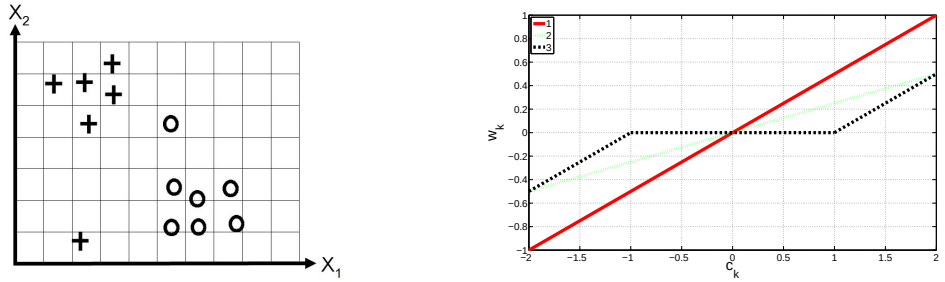
c. Show that the block submatrix of the Hessian for classes c and c' is given by

$$\mathbf{H}_{c,c'} = -\sum_i \mu_{ic}(\delta_{c,c'} - \mu_{i,c'}) \mathbf{x}_i \mathbf{x}_i^T \tag{10.121}$$

Hence show that the Hessian of the NLL is positive definite.

Exercise 10.2 [Regularizing separate terms in 2d logistic regression]

(Source: Jaakkola.)



(a) (b)

Figure 10.15: (a) Data for logistic regression question. (b) Plot of $\hat{w}_k$ vs amount of correlation c_k for three different estimators.

a. Consider the data in Figure 10.15a, where we fit the model $p(y = 1|\mathbf{x}, \mathbf{w}) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$. Suppose we fit the model by maximum likelihood, i.e., we minimize

$$J(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) \tag{10.122}$$

where $\ell(\mathbf{w}, \mathcal{D}_{\text{train}})$ is the log likelihood on the training set. Sketch a possible decision boundary corresponding to $\hat{\mathbf{w}}$. (Copy the figure first (a rough sketch is enough), and then superimpose your answer on your copy, since you will need multiple versions of this figure). Is your answer (decision boundary) unique? How many classification errors does your method make on the training set?

b. Now suppose we regularize only the w_0 parameter, i.e., we minimize

$$J_0(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_0^2 \tag{10.123}$$

Suppose λ is a very large number, so we regularize w_0 all the way to 0, but all other parameters are unregularized. Sketch a possible decision boundary. How many classification errors does your method make on the training set? Hint: consider the behavior of simple linear regression, $w_0 + w_1 x_1 + w_2 x_2$ when $x_1 = x_2 = 0$.

c. Now suppose we heavily regularize only the w_1 parameter, i.e., we minimize

$$J_1(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_1^2 \tag{10.124}$$

Sketch a possible decision boundary. How many classification errors does your method make on the training set?

d. Now suppose we heavily regularize only the w_2 parameter. Sketch a possible decision boundary. How many classification errors does your method make on the training set?

Exercise 10.3 [Logistic regression vs LDA/QDA]

(Source: Jaakkola.) Suppose we train the following binary classifiers via maximum likelihood.

a. GaussI: A generative classifier, where the class-conditional densities are Gaussian, with both covariance matrices set to $\mathbf{I}$ (identity matrix), i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \mathbf{I})$. We assume $p(y)$ is uniform.

b. GaussX: as for GaussI, but the covariance matrices are unconstrained, i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

c. LinLog: A logistic regression model with linear features.

d. QuadLog: A logistic regression model, using linear and quadratic features (i.e., polynomial basis function expansion of degree 2).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

After training we compute the performance of each model M on the training set as follows:

$$L(M) = \frac{1}{n} \sum_{i=1}^{n} \log p(y_i|\mathbf{x}_i, \hat{\boldsymbol{\theta}}, M) \tag{10.125}$$

(Note that this is the *conditional* log-likelihood $p(y|\mathbf{x}, \hat{\boldsymbol{\theta}})$ and not the joint log-likelihood $p(y, \mathbf{x}|\hat{\boldsymbol{\theta}})$.) We now want to compare the performance of each model. We will write $L(M) \leq L(M')$ if model M *must* have lower (or equal) log likelihood (on the training set) than M', for any training set (in other words, M is worse than M', at least as far as training set logprob is concerned). For each of the following model pairs, state whether $L(M) \leq L(M')$, $L(M) \geq L(M')$, or whether no such statement can be made (i.e., M might sometimes be better than M' and sometimes worse); also, for each question, briefly (1-2 sentences) explain why.

a. GaussI, LinLog.
b. GaussX, QuadLog.
c. LinLog, QuadLog.
d. GaussI, QuadLog.
e. Now suppose we measure performance in terms of the average misclassification rate on the training set:

$$R(M) = \frac{1}{n} \sum_{i=1}^{n} I(y_i \neq \hat{y}(\mathbf{x}_i)) \tag{10.126}$$

Is it true in general that $L(M) > L(M')$ implies that $R(M) < R(M')$? Explain why or why not.

11 Linear regression

11.1 Introduction

In this chapter, we discuss **linear regression**, which is a very widely used method for predicting a real-valued output (also called the **dependent variable** or **target**) $y \in \mathbb{R}$, given a vector of real-valued inputs (also called **independent variables** or **explanatory variables**, or **covariates**) $\mathbf{x} \in \mathbb{R}^D$. The key property of the model is that the expected value of the output is assumed to be a linear function of the input, $\mathbb{E}[y|\mathbf{x}] = \mathbf{w}^\top \mathbf{x}$, which makes the model easy to interpret, and easy to fit to data. We discuss nonlinear extensions later in this book.

11.2 Standard linear regression

In this section, we discuss the most common form of linear regression model.

11.2.1 Terminology

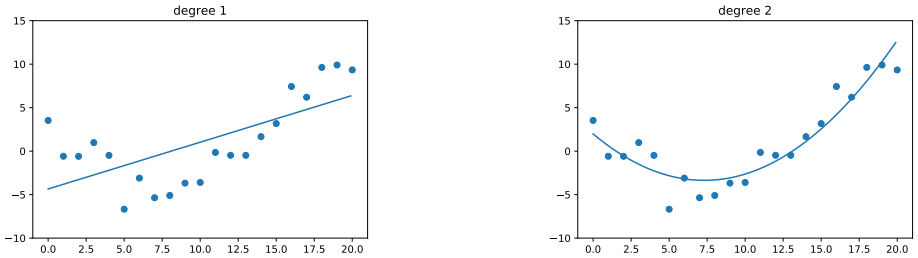
The term "linear regression" usually refers to a model of the following form:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2) \tag{11.1}$$

where $\boldsymbol{\theta} = (w_0, \mathbf{w}, \sigma^2)$ are all the parameters of the model. (In statistics, the parameters w_0 and $\mathbf{w}$ are usually denoted by β_0 and $\boldsymbol{\beta}$.)

The vector of parameters $\mathbf{w}_{1:D}$ are known as the **weights** or **regression coefficients**. Each coefficient w_d specifies the change in the output we expect if we change the corresponding input feature x_d by one unit. For example, suppose x_1 is the age of a person, x_2 is their education level (represented as a continuous number), and y is their income. Thus w_1 corresponds to the increase in income we expect as someone becomes one year older (and hence get more experience), and w_2 corresponds to the increase in income we expect as someone's education level increases by one level. The term w_0 is the **offset** or **bias** term, and specifies the output value if all the inputs are 0. This captures the unconditional mean of the response, $w_0 = \mathbb{E}[y]$, and acts as a baseline. We will usually assume that $\mathbf{x}$ is written as $[1, x_1, \ldots, x_D]$, so we can absorb the offset term w_0 into the weight vector $\mathbf{w}$.

If the input is one-dimensional (so $D = 1$), the model has the form $f(\mathbf{x}; \mathbf{w}) = ax + b$, where $b = w_0$ is the intercept, and $a = w_1$ is the slope. This is called **simple linear regression**. If the input is multi-dimensional, $\mathbf{x} \in \mathbb{R}^D$ where $D > 1$, the method is called **multiple linear regression**. If the



(a) (b)

Figure 11.1: Polynomial of degrees 1 and 2 fit to 21 datapoints. Generated by code at figures.probml.ai/book1/11.1.

output is also multi-dimensional, $\mathbf{y} \in \mathbb{R}^J$, where $J > 1$, it is called **multivariate linear regression**,

$$p(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \prod_{j=1}^{J} \mathcal{N}(y_j|\mathbf{w}_j^\top \mathbf{x}, \sigma_j^2) \tag{11.2}$$

See Exercise 11.1 for a simple numerical example.

In general, a straight line will not provide a good fit to most data sets. However, we can always apply a nonlinear transformation to the input features, by replacing $\mathbf{x}$ with $\boldsymbol{\phi}(\mathbf{x})$ to get

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}), \sigma^2) \tag{11.3}$$

As long as the parameters of the **feature extractor** $\boldsymbol{\phi}$ are fixed, the model remains *linear in the parameters*, even if it is not linear in the inputs. (We discuss ways to learn the feature extractor, and the final linear mapping, in Part III.)

As a simple example of a nonlinear transformation, consider the case of **polynomial regression**, which we introduced in Sec. 1.2.2.2. If the input is 1d, and we use a polynomial expansion of degree d, we get $\boldsymbol{\phi}(x) = [1, x, x^2, \ldots, x^d]$. See Fig. 11.1 for an example.

11.2.2 Least squares estimation

To fit a linear regression model to data, we will minimize the negative log likelihood on the training set. The objective function is given by

$$\text{NLL}(\mathbf{w}, \sigma^2) = -\sum_{n=1}^{N} \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2 \right) \right] \tag{11.4}$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2 + \frac{N}{2} \log(2\pi\sigma^2) \tag{11.5}$$

where we have defined the predicted response $\hat{y}_n \triangleq \mathbf{w}^\top \mathbf{x}_n$. The MLE is the point where $\nabla_{\mathbf{w},\sigma} \text{NLL}(\mathbf{w}, \sigma^2) = \mathbf{0}$. We can first optimize wrt $\mathbf{w}$, and then solve for the optimal σ.

In this section, we just focus on estimating the weights $\mathbf{w}$. In this case, the NLL is equal (up to irrelevant constants) to the **residual sum of squares**, which is given by

$$\text{RSS}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 = \frac{1}{2}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) \tag{11.6}$$

We discuss how to optimize this below.

11.2.2.1 Ordinary least squares

From Eq. (7.249) we can show that the gradient is given by

$$\nabla_{\mathbf{w}}\text{RSS}(\mathbf{w}) = \mathbf{X}^\top\mathbf{X}\mathbf{w} - \mathbf{X}^\top\mathbf{y} \tag{11.7}$$

Setting the gradient to zero and solving gives

$$\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y} \tag{11.8}$$

These are known as the **normal equations**, since, at the optimal solution, $\mathbf{y} - \mathbf{X}\mathbf{w}$ is normal (orthogonal) to the range of $\mathbf{X}$, as we explain in Sec. 11.2.2.2. The corresponding solution $\hat{\mathbf{w}}$ is the **ordinary least squares** (**OLS**) solution, which is given by

$$\hat{\mathbf{w}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{11.9}$$

The quantity $\mathbf{X}^\dagger = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ is the (left) pseudo inverse of the (non-square) matrix $\mathbf{A}$ (see Sec. 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\mathbf{w}) = \frac{\partial^2}{\partial \mathbf{x}^2}\text{RSS}(\mathbf{w}) = \mathbf{X}^\top\mathbf{X} \tag{11.10}$$

If $\mathbf{X}$ is full rank (so the columns of $\mathbf{X}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\mathbf{v} > \mathbf{0}$, we have

$$\mathbf{v}^\top(\mathbf{X}^\top\mathbf{X})\mathbf{v} = (\mathbf{X}\mathbf{v})^\top(\mathbf{X}\mathbf{v}) = ||\mathbf{X}\mathbf{v}||^2 > 0 \tag{11.11}$$

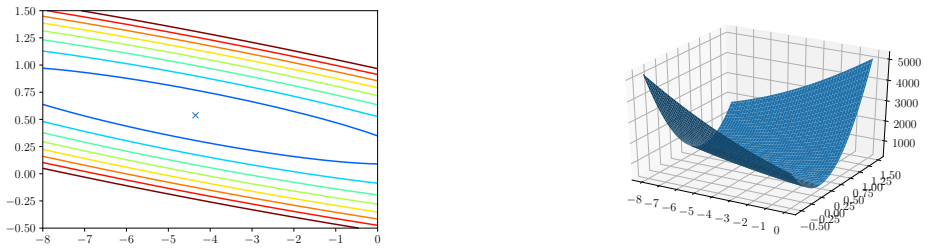
Hence in the full rank case, the least squares objective has a unique global minimum. See Fig. 11.2 for an illustration.

11.2.2.2 Geometric interpretation of least squares

The normal equations have an elegant geometrical interpretation, deriving from Sec. 7.7, as we now explain. We will assume $N > D$, so there are more observations than unknowns. (This is known as an **overdetermined system**.) We seek a vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ that lies in the linear subspace spanned by $\mathbf{X}$ and is as close as possible to $\mathbf{y}$, i.e., we want to find

$$\underset{\hat{\mathbf{y}} \in \text{span}(\{\mathbf{x}_{:,1},\ldots,\mathbf{x}_{:,d}\})}{\text{argmin}} \|\mathbf{y} - \hat{\mathbf{y}}\|_2. \tag{11.12}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.2: (a) Contours of the RSS error surface for the example in Fig. 11.1a. The blue cross represents the MLE. (b) Corresponding surface plot. Generated by code at figures.probml.ai/book1/11.2.

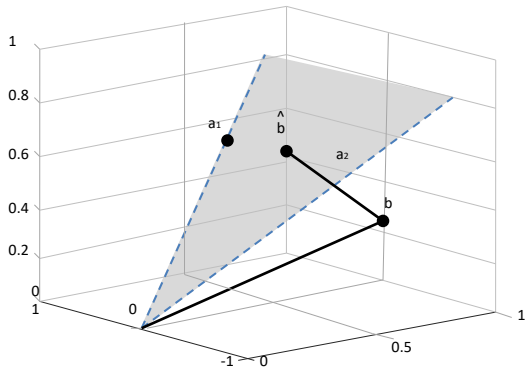


Figure 11.3: Graphical interpretation of least squares for $m = 3$ equations and $n = 2$ unknowns when solving the system $\mathbf{Ax} = \mathbf{b}$. $\mathbf{a}_1$ and $\mathbf{a}_2$ are the columns of $\mathbf{A}$, which define a 2d linear subspace embedded in $\mathbb{R}^3$. The target vector $\mathbf{b}$ is a vector in $\mathbb{R}^3$; its orthogonal projection onto the linear subspace is denoted $\hat{\mathbf{b}}$. The line from $\mathbf{b}$ to $\hat{\mathbf{b}}$ is the vector of residual errors, whose norm we want to minimize.

where $\mathbf{x}_{:,d}$ is the d'th column of $\mathbf{X}$. Since $\hat{\mathbf{y}} \in \text{span}(\mathbf{X})$, there exists some weight vector $\mathbf{w}$ such that

$$\hat{\mathbf{y}} = w_1\mathbf{x}_{:,1} + \cdots + w_n\mathbf{x}_{:,D} = \mathbf{Xw} \tag{11.13}$$

To minimize the norm of the residual, $\mathbf{y} - \hat{\mathbf{y}}$, we want the residual vector to be orthogonal to every column of $\mathbf{X}$. Hence

$$\mathbf{x}_{:,d}^\top(\mathbf{y} - \hat{\mathbf{y}}) = 0 \Rightarrow \mathbf{X}^\top(\mathbf{y} - \mathbf{Xw}) = \mathbf{0} \Rightarrow \mathbf{w} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{11.14}$$

Hence our projected value of $\mathbf{y}$ is given by

$$\hat{\mathbf{y}} = \mathbf{Xw} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{11.15}$$

This corresponds to an **orthogonal projection** of $\mathbf{y}$ onto the column space of $\mathbf{X}$. For example, consider the case where we have $N = 3$ training examples, each of dimensionality $D = 2$. The

training data defines a 2d linear subspace, defined by the 2 columns of $\mathbf{X}$, each of which is a point in 3d. We project $\mathbf{y}$, which is also a point in 3d, onto this 2d subspace, as shown in Fig. 11.3.

The **projection matrix**

$$\text{Proj}(\mathbf{X}) \triangleq \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top \tag{11.16}$$

is sometimes called the **hat matrix**, since $\hat{\mathbf{y}} = \text{Proj}(\mathbf{X})\mathbf{y}$. In the special case that $\mathbf{X} = \mathbf{x}$ is a column vector, the orthogonal projection of $\mathbf{y}$ onto the line $\mathbf{x}$ becomes

$$\text{Proj}(\mathbf{x})\mathbf{y} = \mathbf{x}\,\frac{\mathbf{x}^\top\mathbf{y}}{\mathbf{x}^\top\mathbf{x}} \tag{11.17}$$

11.2.2.3 Algorithmic issues

Recall that the OLS solution is

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger\mathbf{y} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{11.18}$$

However, even if it is theoretically possible to compute the pseudo-inverse by inverting $\mathbf{X}^\top\mathbf{X}$, we should not do so for numerical reasons, since $\mathbf{X}^\top\mathbf{X}$ may be ill conditioned or singular.

A better (and more general) approach is to compute the pseudo-inverse using the SVD. Indeed, if you look at the source code for the function sklearn.linear_model.fit, you will see that it uses the scipy.linalg.lstsq function, which in turns calls DGELSD, which is an SVD-based solver implemented by the LAPACK library, written in Fortran.[1]

However, if $\mathbf{X}$ is tall and skinny (i.e., $N \gg D$), it can be quicker to use QR decomposition (Sec. 7.6.2). To do this, let $\mathbf{X} = \mathbf{QR}$, where $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$. In Sec. 7.7, we show that OLS is equivalent to solving the system of linear equations $\mathbf{Xw} = \mathbf{y}$ in a way that minimizes $||\mathbf{Xw} - \mathbf{y}||_2^2$. (If $N = D$ and $\mathbf{X}$ is full rank, the equations have a unique solution, and the error will be 0.) Using QR decomposition, we can rewrite this system of equations as follows:

$$(\mathbf{QR})\mathbf{w} = \mathbf{y} \tag{11.19}$$

$$\mathbf{Q}^\top\mathbf{QRw} = \mathbf{Q}^\top\mathbf{y} \tag{11.20}$$

$$\mathbf{w} = \mathbf{R}^{-1}(\mathbf{Q}^\top\mathbf{y}) \tag{11.21}$$

Since $\mathbf{R}$ is upper triangular, we can solve this last set of equations using backsubstitution, thus avoiding matrix inversion. See code.probml.ai/book1/linsys_solve_demo for a demo.

An alternative to the use of direct methods based on matrix decomposition (such as SVD and QR) is to use iterative solvers, such as the **conjugate gradient** method (which assumes $\mathbf{X}$ is symmetric positive definite), and the **GMRES** (generalized minimal residual method), that works for general $\mathbf{X}$. (In SciPy, this is implemented by `sparse.linalg.gmres`.) These methods just require the ability to perform matrix-vector multiplications (i.e., an implementation of a **linear operator**), and thus are well-suited to problems where $\mathbf{X}$ is sparse or structured. For details, see e.g., [TB97].

A final important issue is that it is usually essential to **standardize** the input features before fitting the model, to ensure thet are zero mean and unit variance. We can do this using Eq. (10.51).

1. Note that a lot of the "Python" scientific computing stack sits on top of source code that is written in Fortran or C++, for reasons of speed. This makes it hard to change the underlying algorithms. By contrast, the scientific computing libraries in the Julia language are written in Julia itself, aiding clarity without sacrificing speed.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11.2.2.4 Weighted least squares

In some cases, we want to associate a weight with each example. For example, in **heteroskedastic regression**, the variance depends on the input, so the model has the form

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2(\mathbf{x})) = \frac{1}{\sqrt{2\pi\sigma^2(\mathbf{x})}} \exp\left(-\frac{1}{2\sigma^2(\mathbf{x})}(y - \mathbf{w}^\top \mathbf{x})^2\right) \tag{11.22}$$

Thus

$$p(\mathbf{y}|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \boldsymbol{\Lambda}^{-1}) \tag{11.23}$$

where $\boldsymbol{\Lambda} = \text{diag}(1/\sigma^2(\mathbf{x}_n))$. This is known as **weighted linear regression**. One can show that the MLE is given by

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{y} \tag{11.24}$$

This is known as the **weighted least squares** estimate.

11.2.3 Other approaches to computing the MLE

In this section, we discuss other approaches for computing the MLE.

11.2.3.1 Solving for offset and slope separately

Typically we use a model of the form $p(y|\mathbf{x},\boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2)$, where w_0 is an offset or "bias" term. We can compute $(w_0, \mathbf{w})$ at the same time by adding a column of 1s to $\mathbf{X}$, and the computing the MLE as above. Alternatively, we can solve for $\mathbf{w}$ and w_0 separately. (This will be useful later.) In particular, one can show that

$$\hat{\mathbf{w}} = (\mathbf{X}_c^T \mathbf{X}_c)^{-1} \mathbf{X}_c^T \mathbf{y}_c = \left[\sum_{i=1}^N (\mathbf{x}_n - \overline{\mathbf{x}})(\mathbf{x}_n - \overline{\mathbf{x}})^T\right]^{-1} \left[\sum_{i=1}^N (y_n - \overline{y})(\mathbf{x}_n - \overline{\mathbf{x}})\right] \tag{11.25}$$

$$\hat{w}_0 = \frac{1}{N}\sum_n y_n - \frac{1}{N}\sum_n \mathbf{x}_n^T \hat{\mathbf{w}} = \overline{y} - \overline{\mathbf{x}}^T \hat{\mathbf{w}} \tag{11.26}$$

where $\mathbf{X}_c$ is the centered input matrix containing $\mathbf{x}_n^c = \mathbf{x}_n - \overline{\mathbf{x}}$ along its rows, and $\mathbf{y}_c = \mathbf{y} - \overline{\mathbf{y}}$ is the centered output vector. Thus we can first compute $\hat{\mathbf{w}}$ on centered data, and then estimate w_0 using $\overline{y} - \overline{\mathbf{x}}^T \hat{\mathbf{w}}$.

11.2.3.2 Simple linear regression (1d inputs)

In the case of 1d (scalar) inputs, the results from Sec. 11.2.3.1 reduce to the following simple form, which may be familiar from basic statistics classes:

$$\hat{w}_1 = \frac{\sum_n (x_n - \overline{x})(y_n - \overline{y})}{\sum_n (x_n - \overline{x})^2} = \frac{C_{xy}}{C_x^2} \tag{11.27}$$

$$\hat{w}_0 = \overline{y} - \hat{w}_1 \overline{x} = \mathbb{E}[y] - w_1 \mathbb{E}[x] \tag{11.28}$$

where $C_{xy} = \text{Cov}[X, Y]$ and $C_x^2 = \text{Cov}[X, X] = \mathbb{V}[X]$. We will use this result below.

11.2.3.3 Partial regression (2d inputs)

From Eq. (11.27), we can compute the **regression coefficient** of Y on X as follows:

$$R_{YX} \triangleq \frac{\partial}{\partial x}\mathbb{E}\left[Y|X=x\right] = w_1 = \frac{\sigma_{XY}}{\sigma_X^2} \tag{11.29}$$

This is the slope of the linear prediction for Y given X.

Now consider the case where we have 2 inputs, call them X and Z, so $Y = w_0 + w_X X + w_Z Z + \epsilon$, where $\mathbb{E}\left[\epsilon\right] = 0$. The least squares estimate of the coefficients are obtained when ϵ is uncorrelated with each of the regressors, so $\text{Cov}\left[\epsilon, X\right] = \mathbb{E}\left[\epsilon X\right] = 0$ and $\text{Cov}\left[\epsilon, Z\right] = \mathbb{E}\left[\epsilon Z\right] = 0$. Using this, one can show that the optimal coefficients for X and Y are given by

$$w_X = R_{YX\cdot Z} = \frac{\sigma_X^2\sigma_{YZ} - \sigma_{YX}\sigma_{XZ}}{\sigma_Z^2\sigma_X^2 - \sigma_{ZX}^2} \tag{11.30}$$

$$w_Z = R_{YZ\cdot X} = \frac{\sigma_Z^2\sigma_{YX} - \sigma_{YZ}\sigma_{ZY}}{\sigma_Z^2\sigma_X^2 - \sigma_{ZX}^2} \tag{11.31}$$

The term $R_{YX\cdot Z}$ is called the **partial regression coefficient** of Y on X, keeping Z constant. This is equal to

$$R_{YX\cdot Z} = \frac{\partial}{\partial x}\mathbb{E}\left[Y|X=x, Z=z\right] \tag{11.32}$$

Note that this quantity is invariant to the specific value of Z we condition on. Note also that it is possible for R_{YX} to be positive and $R_{YX\cdot Z}$ to be negative, since conditioning on extra variables can always flip the sign of a regression coefficient, as in Simpsons paradox (see Sec. 3.1.5).

11.2.3.4 Recursively computing the MLE

OLS is a batch method for computing the MLE. In some applications, the data arrives in a continual stream, so we want to compute the estimate online, or **recursively**, as we discussed in Sec. 4.4.2. In this section, we show how to do this for the case of simple (1d) linear regession.

Recall from Sec. 11.2.3.2 that the batch MLE for simple linear regression is given by

$$\hat{w}_1 = \frac{\sum_n (x_n - \overline{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_x^2} \tag{11.33}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1\bar{x} \tag{11.34}$$

where $C_{xy} = \text{Cov}\left[X, Y\right]$ and $C_x = \text{Cov}\left[X, X\right] = \mathbb{V}\left[X\right]$.

We now discuss how to compute these results in a recursive fashion. To do this, let us define the following sufficient statistics:

$$\overline{x}^{(n)} = \frac{1}{n}\sum_{i=1}^{n} x_i, \quad \overline{y}^{(n)} = \frac{1}{n}\sum_{i=1}^{n} y_i \tag{11.35}$$

$$C_{xx}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2, \quad C_{xy}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y}), \quad C_{yy}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \overline{y})^2 \tag{11.36}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

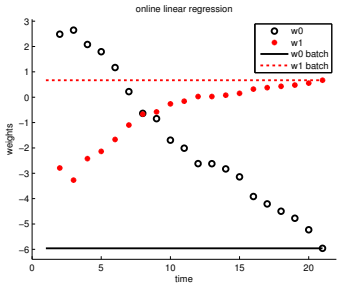


Figure 11.4: Regression coefficients over time for the 1d model in Fig. 1.7a(a). Generated by code at figures.probml.ai/book1/11.4.

We can update the means online using

$$\overline{x}^{(n+1)} = \overline{x}^{(n)} + \frac{1}{n+1}(x_{n+1} - \overline{x}^{(n)}), \;\; \overline{y}^{(n+1)} = \overline{y}^{(n)} + \frac{1}{n+1}(y_{n+1} - \overline{y}^{(n)}) \tag{11.37}$$

To update the covariance terms, let us first rewrite $C_{xy}^{(n)}$ as follows:

$$C_{xy}^{(n)} = \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) + (\sum_{i=1}^{n} \overline{x}^{(n)}\overline{y}^{(n)}) - \overline{x}^{(n)}(\sum_{i=1}^{n} y_i) - \overline{y}^{(n)}(\sum_{i=1}^{n} x_i)\right] \tag{11.38}$$

$$= \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) + n\overline{x}^{(n)}\overline{y}^{(n)} - \overline{x}^{(n)} n\overline{y}^{(n)} - \overline{y}^{(n)} n\overline{x}^{(n)}\right] \tag{11.39}$$

$$= \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) - n\overline{x}^{(n)}\overline{y}^{(n)}\right] \tag{11.40}$$

Hence

$$\sum_{i=1}^{n} x_i y_i = nC_{xy}^{(n)} + n\overline{x}^{(n)}\overline{y}^{(n)} \tag{11.41}$$

and so

$$C_{xy}^{(n+1)} = \frac{1}{n+1}\left[x_{n+1}y_{n+1} + nC_{xy}^{(n)} + n\overline{x}^{(n)}\overline{y}^{(n)} - (n+1)\overline{x}^{(n+1)}\overline{y}^{(n+1)}\right] \tag{11.42}$$

We can derive the update for $C_{xx}^{(n+1)}$ in a similar manner.

See Fig. 11.4 for a simple illustration of these equations in action.

In Sec. 8.4.2, we discuss the least mean squares algorithm, which uses SGD to recursively compute the MLE for linear regression with D-dimensional inputs.

11.2.3.5 Deriving the MLE from a generative perspective

Linear regression is a discriminative model of the form $p(y|\mathbf{x})$. However, we can also use generative models for regression, by analogy to how we use generative models for classification in Chapter 9,

The goal is to compute the conditional expectation

$$f(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}] = \int y\ p(y|\mathbf{x})dy = \frac{\int y\ p(\mathbf{x}, y)dy}{\int p(\mathbf{x}, y)dy} \tag{11.43}$$

Suppose we fit $p(\mathbf{x}, y)$ using an MVN. The MLEs for the parameters of the joint distribution are the empiricial means and covariances (see Sec. 4.2.6 for a proof of this result):

$$\boldsymbol{\mu}_x = \frac{1}{N}\sum_n \mathbf{x}_n \tag{11.44}$$

$$\mu_y = \frac{1}{N}\sum_n y_n \tag{11.45}$$

$$\boldsymbol{\Sigma}_{xx} = \frac{1}{N}\sum_n (\mathbf{x}_n - \overline{\mathbf{x}})(\mathbf{x}_n - \overline{\mathbf{x}})^\top = \frac{1}{N}\mathbf{X}_c^\top \mathbf{X}_c \tag{11.46}$$

$$\boldsymbol{\Sigma}_{xy} = \frac{1}{N}\sum_n (\mathbf{x}_n - \overline{\mathbf{x}})(y_n - \overline{y}) = \frac{1}{N}\mathbf{X}_c^\top \mathbf{y}_c \tag{11.47}$$

Hence from Eq. (3.31), we have

$$\mathbb{E}[y|\mathbf{x}] = \mu_y + \boldsymbol{\Sigma}_{yx}\boldsymbol{\Sigma}_{xx}^{-1}(\mathbf{x} - \boldsymbol{\mu}_x) \tag{11.48}$$

We can rewrite this as $\mathbb{E}[y|\mathbf{x}] = w_0 + \mathbf{w}^\top \mathbf{x}$ by defining

$$w_0 = \mu_y - \mathbf{w}^\top \boldsymbol{\mu}_x = \overline{y} - \mathbf{w}^\top \overline{\mathbf{x}} \tag{11.49}$$

$$\mathbf{w} = \boldsymbol{\Sigma}_{xx}^{-1}\boldsymbol{\Sigma}_{yx}^\top = \left(\mathbf{X}_c^\top \mathbf{X}_c\right)^{-1}\mathbf{X}_c^\top \mathbf{y}_c \tag{11.50}$$

This matches the MLEs for the discriminative model as we showed in Sec. 11.2.3.1. Thus we see that fitting the joint model, and then conditioning it, yields the same result as fitting the conditional model. However, this is only true for Gaussian models (see Sec. 9.4 for further discussion of this point).

11.2.3.6 Deriving the MLE for σ^2

After estimating $\hat{\mathbf{w}}_{\text{mle}}$ using one of the above methods, we can estimate the noise variance. It is easy to show that the MLE is given by

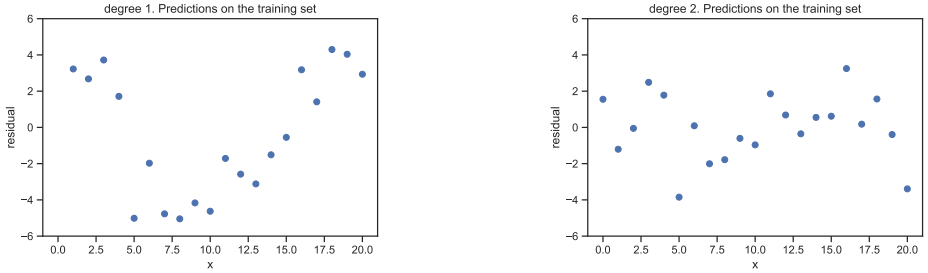
$$\hat{\sigma}^2_{\text{mle}} = \underset{\sigma^2}{\text{argmin}}\, \text{NLL}(\hat{\mathbf{w}}, \sigma^2) = \frac{1}{N}\sum_{n=1}^{N}(y_n - \mathbf{x}_n^\top \hat{\mathbf{w}})^2 \tag{11.51}$$

This is just the MSE of the residuals, which is an intuitive result.

11.2.4 Measuring goodness of fit

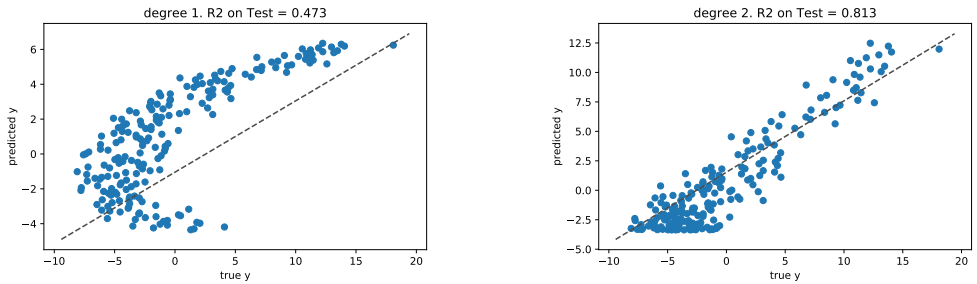
In this section, we discuss some simple ways to assess how well a regression model fits the data (which is known as **goodness of fit**).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.5: Residual plot for polynomial regression of degree 1 and 2 for the functions in Fig. 1.7a(a-b). Generated by code at figures.probml.ai/book1/11.5.



(a) (b)

Figure 11.6: Fit vs actual plots for polynomial regression of degree 1 and 2 for the functions in Fig. 1.7a(a-b). Generated by code at figures.probml.ai/book1/11.6.

11.2.4.1 Residual plots

For 1d inputs, we can check the reasonableness of the model by plotting the residuals, $r_n = y_n - \hat{y}_n$, vs the input x_n. This is called a **residual plot**. The model assumes that the residuals have a $\mathcal{N}(0, \sigma^2)$ distribution, so the residual plot should be a cloud of points more or less equally above and below the horizontal line at 0, without any obvious trends.

As an example, in Fig. 11.5(a), we plot the residuals for the linear model in Fig. 1.7a(a). We see that there is some curved structure to the residuals, indicating a lack of fit. In Fig. 11.5(b), we plot the residuals for the quadratic model in Fig. 1.7a(b). We see a much better fit.

11.2.4.2 Prediction accuracy and R^2

We can measure the quality of a regression model's predictions by plotting $\hat{y}_n$ vs y_n, rather than plotting vs x_n. See Fig. 11.6 for some examples. We can assess the fit quantitatively by computing the RSS (residual sum of squares) on the dataset: $\text{RSS}(\mathbf{w}) = \sum_{n=1}^{N}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2$. A model with lower RSS fits the data better. Another measure that is used is **root mean squared error** or

RMSE:

$$\text{RMSE}(\mathbf{w}) \triangleq \sqrt{\frac{1}{N}\text{RSS}(\mathbf{w})} \tag{11.52}$$

A more interpretable measure can be computed using the **coefficient of determination**, denoted by R^2:

$$R^2 \triangleq 1 - \frac{\sum_{n=1}^{N}(\hat{y}_n - y_n)^2}{\sum_{n=1}^{N}(\overline{y} - y_n)^2} = 1 - \frac{\text{RSS}}{\text{TSS}} \tag{11.53}$$

where $\overline{y} = \frac{1}{N}\sum_{n=1}^{N} y_n$ is the empirical mean of the response, $\text{RSS} = \sum_{n=1}^{N}(y_n - \hat{y}_n)^2$ is the residual sum of squares, and $\text{TSS} = \sum_{n=1}^{N}(y_n - \overline{y})^2$ is the total sum of squares. Thus we see tht R^2 measures the variance in the predictions relative to a simple constant prediction of $\hat{y}_n = \overline{y}$. One can show that $0 \leq R^2 \leq 1$, where larger values imply a greater reduction in variance (better fit). This is illustrated in Fig. 11.6.

11.3 Ridge regression

Maximum likelihood estimation can result in overfitting, as we discussed in Sec. 1.2.2.2. A simple solution to this is to use MAP estimation with a zero-mean Gaussian prior on the weights, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$, as we discused in Sec. 4.5.3. This is called **ridge regression**.

In more detail, we compute the MAP estimate as follows:

$$\hat{\mathbf{w}}_{\text{map}} = \text{argmin}\, \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \frac{1}{2\tau^2}\mathbf{w}^\top\mathbf{w} \tag{11.54}$$

$$= \text{argmin}\, \text{RSS}(\mathbf{w}) + \lambda||\mathbf{w}||_2^2 \tag{11.55}$$

where $\lambda \triangleq \frac{\sigma^2}{\tau^2}$ is proportional to the strength of the prior, and

$$||\mathbf{w}||_2 \triangleq \sqrt{\sum_{d=1}^{D}|w_d|^2} = \sqrt{\mathbf{w}^\top\mathbf{w}} \tag{11.56}$$

is the ℓ_2 norm of the vector $\mathbf{w}$. Thus we are penalizing weights that become too large in magnitude. In general, this technique is called ℓ_2 **regularization** or **weight decay**, and is very widely used. See Fig. 11.7 for an illustration.

Note that we do not penalize the offset term w_0, since that only affects the global mean of the output, and does not contribute to overfitting. See Exercise 11.2.

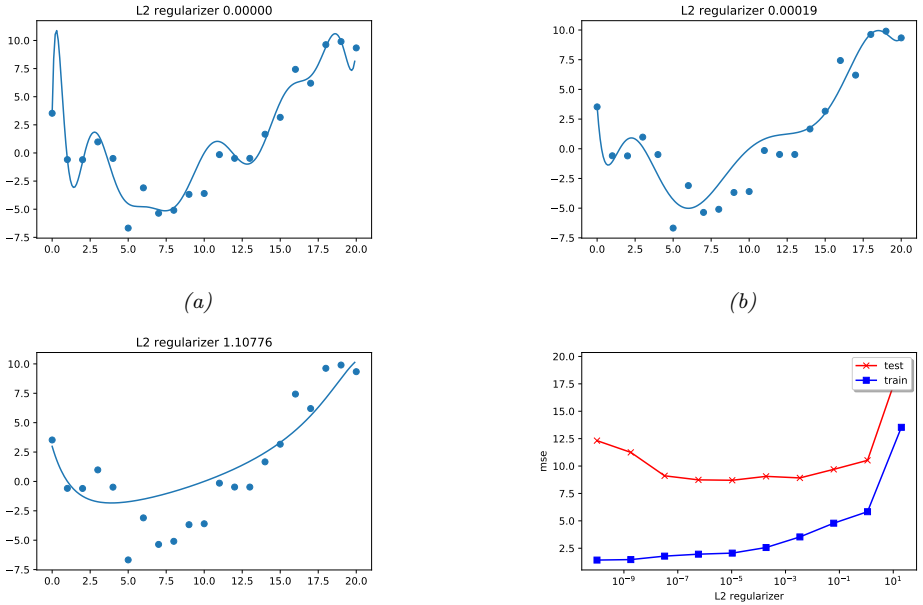
11.3.1 Computing the MAP estimate

In this section, we discuss algorithms for computing the MAP estimate.

The MAP estimate corresponds to minimizing the following penalized objective:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda||\mathbf{w}||_2^2 \tag{11.57}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)* *(c)* *(d)*

Figure 11.7: (a-c) Ridge regression applied to a degree 14 polynomial fit to 21 datapoints. (d) MSE vs strength of regularizer. The degree of regularization increases from left to right, so model complexity decreases from left to right. Generated by code at figures.probml.ai/book1/11.7.

where $\lambda = \sigma^2/\tau^2$ is the strength of the regularizer. The derivative is given by

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2\left(\mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y} + \lambda \mathbf{w}\right) \tag{11.58}$$

and hence

$$\hat{\mathbf{w}}_{\text{map}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y} = (\sum_n \mathbf{x}_n \mathbf{x}_n^\top + \lambda \mathbf{I}_D)^{-1} (\sum_n y_n \mathbf{x}_n) \tag{11.59}$$

11.3.1.1 Solving using QR

Naively computing the primal estimate $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ using matrix inversion is a bad idea, since it can be slow and numerically unstable. In this section, we describe a way to convert the problem to a standard least squares problem, to which we can apply QR decomposition, as discussed in Sec. 11.2.2.3.

We assume the prior has the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \mathbf{\Lambda}^{-1})$, where $\mathbf{\Lambda}$ is the precision matrix. In the case of ridge regression, $\mathbf{\Lambda} = (1/\tau^2)\mathbf{I}$. We can emulalate this prior by adding "virtual data" to the training set to get

$$\tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}} \end{pmatrix}, \quad \tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0}_{D\times 1} \end{pmatrix} \tag{11.60}$$

where $\mathbf{\Lambda} = \sqrt{\mathbf{\Lambda}}\sqrt{\mathbf{\Lambda}}^\top$ is a Cholesky decomposition of $\mathbf{\Lambda}$. We see that $\tilde{\mathbf{X}}$ is $(N+D) \times D$, where the extra rows represent pseudo-data from the prior.

We now show that the RSS on this expanded data is equivalent to penalized RSS on the original data:

$$f(\mathbf{w}) = (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w})^\top(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w}) \tag{11.61}$$

$$= \left(\begin{pmatrix}\mathbf{y}/\sigma \\ \mathbf{0}\end{pmatrix} - \begin{pmatrix}\mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}}\end{pmatrix}\mathbf{w}\right)^\top \left(\begin{pmatrix}\mathbf{y}/\sigma \\ \mathbf{0}\end{pmatrix} - \begin{pmatrix}\mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}}\end{pmatrix}\mathbf{w}\right) \tag{11.62}$$

$$= \begin{pmatrix}\frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\mathbf{\Lambda}}\mathbf{w}\end{pmatrix}^\top \begin{pmatrix}\frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\mathbf{\Lambda}}\mathbf{w}\end{pmatrix} \tag{11.63}$$

$$= \frac{1}{\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + (\sqrt{\mathbf{\Lambda}}\mathbf{w})^\top(\sqrt{\mathbf{\Lambda}}\mathbf{w}) \tag{11.64}$$

$$= \frac{1}{\sigma^2}(\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{w}^\top\mathbf{\Lambda}\mathbf{w} \tag{11.65}$$

Hence the MAP estimate is given by

$$\hat{\mathbf{w}}_{\text{map}} = (\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{y}} \tag{11.66}$$

which can be solved using standard OLS methods. In particular, we can compute the QR decomposition of $\tilde{\mathbf{X}}$, and then proceed as in Sec. 11.2.2.3. This takes $O((N+D)D^2)$ time.

11.3.1.2 Solving using SVD

In this section, we assume $D \gg N$, which is the usual case when using ridge regression. In this case, it is faster to use SVD than QR. To see how this works, let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\top\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\top = \mathbf{U}^\top\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Now let $\mathbf{Z} = \mathbf{U}\mathbf{S}$ be an $N \times N$ matrix. Then we can rewrite the ridge estimate thus:

$$\hat{\mathbf{w}}_{\text{map}} = \mathbf{V}(\mathbf{Z}^\top\mathbf{Z} + \lambda\mathbf{I}_N)^{-1}\mathbf{Z}^\top\mathbf{y} = \mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^\top\mathbf{y} \tag{11.67}$$

In other words, we can replace the D-dimensional vectors $\mathbf{x}_i$ with the N-dimensional vectors $\mathbf{z}_i$ and perform our penalized fit as before. We then transform the N-dimensional solution to the D-dimensional solution by multiplying by $\mathbf{V}$. Geometrically, we are rotating to a new coordinate system in which all but the first N coordinates are zero. This does not affect the solution since the spherical Gaussian prior is rotationally invariant. The overall time is now $O(DN^2)$ operations.

11.3.2 Connection between ridge regression and PCA

In this section, we discuss an interesting connection between ridge regression and PCA (which we describe in Sec. 20.1), in order to gain further insight into why ridge regression works well. Our discussion is based on [HTF09, p66].

Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\top\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\top = \mathbf{U}^\top\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Using Eq. (11.67) we can see that the ridge predictions on the training set are given

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

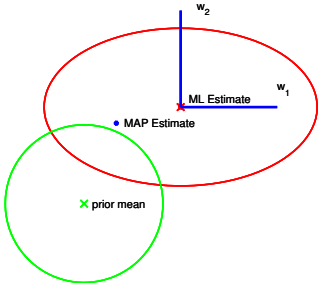


Figure 11.8: Geometry of ridge regression. The likelihood is shown as an ellipse, and the prior is shown as a circle centered on the origin. Adapted from Figure 3.15 of [Bis06]. Generated by code at figures.probml.ai/book1/11.8.

by

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{map}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top \mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^\top \mathbf{y} \tag{11.68}$$

$$= \mathbf{U}\tilde{\mathbf{S}}\mathbf{U}^\top \mathbf{y} = \sum_{j=1}^{D} \mathbf{u}_j \tilde{S}_{jj} \mathbf{u}_j^\top \mathbf{y} \tag{11.69}$$

where

$$\tilde{S}_{jj} \triangleq [\mathbf{S}(\mathbf{S}^2 + \lambda I)^{-1}\mathbf{S}]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \tag{11.70}$$

and σ_j are the singular values of $\mathbf{X}$. Hence

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{map}} = \sum_{j=1}^{D} \mathbf{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \mathbf{u}_j^\top \mathbf{y} \tag{11.71}$$

In contrast, the least squares prediction is

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{mle}} = (\mathbf{U}\mathbf{S}\mathbf{V}^\top)(\mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top \mathbf{y}) = \mathbf{U}\mathbf{U}^\top \mathbf{y} = \sum_{j=1}^{D} \mathbf{u}_j \mathbf{u}_j^\top \mathbf{y} \tag{11.72}$$

If σ_j^2 is small compared to λ, then direction $\mathbf{u}_j$ will not have much effect on the prediction. In view of this, we define the effective number of **degrees of freedom** of the model as follows:

$$\text{dof}(\lambda) = \sum_{j=1}^{D} \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \tag{11.73}$$

When $\lambda = 0$, $\text{dof}(\lambda) = D$, and as $\lambda \to \infty$, $\text{dof}(\lambda) \to 0$.

Let us try to understand why this behavior is desirable. In Sec. 11.6, we show that $\text{Cov}[\mathbf{w}|\mathcal{D}] \propto (\mathbf{X}^\top \mathbf{X})^{-1}$, if we use a uniform prior for $\mathbf{w}$. Thus the directions in which we are most uncertain about

$\mathbf{w}$ are determined by the eigenvectors of $(\mathbf{X}^\top\mathbf{X})^{-1}$ with the largest eigenvalues, as shown in Fig. 7.6; these correspond to the eigenvectors of $\mathbf{X}^\top\mathbf{X}$ with the smallest eigenvalues. In Sec. 7.5.2, we show that the squared singular values σ_j^2 are equal to the eigenvalues of $\mathbf{X}^\top\mathbf{X}$. Hence small singular values σ_j correspond to directions with high posterior variance. It is these directions which ridge shrinks the most.

This process is illustrated in Fig. 11.8. The horizontal w_1 parameter is not-well determined by the data (has high posterior variance), but the vertical w_2 parameter is well-determined. Hence $w_{\text{map}}(2)$ is close to $w_{\text{mle}}(2)$, but $w_{\text{map}}(1)$ is shifted strongly towards the prior mean, which is 0. In this way, ill-determined parameters are reduced in size towards 0. This is called **shrinkage**.

There is a related, but different, technique called **principal components regression**, which is a supervised version of PCA, which we explain in Sec. 20.1. The idea is this: first use PCA to reduce the dimensionality to K dimensions, and then use these low dimensional features as input to regression. However, this technique does not work as well as ridge regression in terms of predictive accuracy [HTF01, p70]. The reason is that in PC regression, only the first K (derived) dimensions are retained, and the remaining $D-K$ dimensions are entirely ignored. By contrast, ridge regression uses a "soft" weighting of all the dimensions.

11.3.3 Choosing the strength of the regularizer

To find the optimal value of λ, we can try a finite number of distinct values, and use cross validation to estimate their expected loss, as discussed in Sec. 4.5.5.2. See Fig. 11.7d for an example.

This approach can be quite expensive if we have many values to choose from. Fortunately, we can often **warm start** the optimization procedure, using the value of $\hat{\mathbf{w}}(\lambda_k)$ as an initializer for $\hat{\mathbf{w}}(\lambda_{k+1})$, where $\lambda_{k+1} < \lambda_k$; in other words, we start with a highly constrained model (strong regularizer), and then gradually relax the constraints (decrease the amount of regularization). The set of parameters $\hat{\mathbf{w}}_k$ that we sweep out in this way is known as the **regularization path**. See Fig. 11.12(a) for an example.

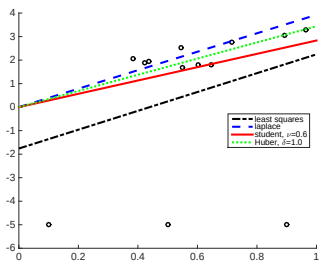
We can also use an empirical Bayes approach to choose λ. In particular, we choose the hyperparameter by computing $\hat{\lambda} = \text{argmax}_\lambda \log p(\mathcal{D}|\lambda)$, where $p(\mathcal{D}|\lambda)$ is the marginal likelihood or evidence. Fig. 4.7b shows that this gives essentially the same result as the CV estimate. However, the Bayesian approach has several advantages: computing $p(\mathcal{D}|\lambda)$ can be done by fitting a single model, whereas CV has to fit the same model K times; and $p(\mathcal{D}|\lambda)$ is a smooth function of λ, so we can use gradient-based optimization instead of discrete search.

11.4 Robust linear regression *

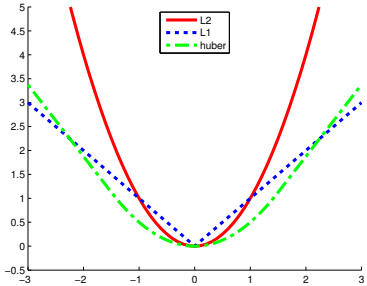
It is very common to model the noise in regression models using a Gaussian distribution with zero mean and constant variance, $r_n \sim \mathcal{N}(0, \sigma^2)$, where $r_n = y_n - \mathbf{w}^\top\mathbf{x}_n$. In this case, maximizing likelihood is equivalent to minimizing the sum of squared residuals, as we have seen. However, if we have **outliers** in our data, this can result in a poor fit, as illustrated in Fig. 11.9(a). (The outliers are the points on the bottom of the figure.) This is because squared error penalizes deviations quadratically, so points far from the line have more effect on the fit than points near to the line.

One way to achieve **robustness** to outliers is to replace the Gaussian distribution for the response variable with a distribution that has **heavy tails**. Such a distribution will assign higher likelihood to outliers, without having to perturb the straight line to "explain" them. We discuss several possible

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a)



(b)

Figure 11.9: (a) Illustration of robust linear regression. Generated by code at figures.probml.ai/book1/11.9. (b) Illustration of ℓ_2, ℓ_1, and Huber loss functions with $\delta = 1.5$. Generated by code at figures.probml.ai/book1/11.9.

Likelihood	Prior	Posterior	Name	Section
Gaussian	Uniform	Point	Least squares	11.2.2
Student	Uniform	Point	Robust regression	11.4.1
Laplace	Uniform	Point	Robust regression	11.4.2
Gaussian	Gaussian	Point	Ridge	11.3
Gaussian	Laplace	Point	Lasso	11.5
Gaussian	Gauss-Gamma	Gauss-Gamma	Bayesian lin. reg	11.6

Table 11.1: Summary of various likelihoods, priors and posteriors used for linear regression. The likelihood refers to the distributional form of $p(y|\mathbf{x}, \mathbf{w}, \sigma^2)$, and the prior refers to the distributional form of $p(\mathbf{w})$. The posterior refers to the distributional form of $p(\mathbf{w}|\mathcal{D})$. "Point" stands for the degenerate distribution $\delta(\mathbf{w} - \hat{\mathbf{w}})$, where $\hat{\mathbf{w}}$ is the MAP estimate. MLE is equivalent to using a point posterior and a uniform prior.

alternative probability distributions for the response variable below; see Table 11.1 for a summary.

11.4.1 Robust regression using the Student t distribution

In Sec. 2.7.1, we discussed the robustness properties of the Student distribution. To use this in a regression context, we can just make the mean be a linear function of the inputs, as proposed in [Zel76]:

$$p(y|\mathbf{x}, \mathbf{w}, \sigma^2, \nu) = \mathcal{T}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2, \nu) \tag{11.74}$$

11.4.1.1 Computing the MLE using EM

We now give a simple iterative algorithm for computing the MLE for linear regression using a Student likelihood. To do this, we will associate a weight with every example, representing the probability it is an outlier. We then compute a weighted least squares estimate. Since the probability of being an outlier depends on the current parameters, we iterate this process until convergence.

More precisely, we will use the EM algorithm, introduced in Sec. 8.7.2, to solve the problem. At first blush it may not be apparent how to do this, since there is no missing data, and there are no

hidden variables. However, it turns out that we can introduce "artificial" hidden variables to make the problem easier to solve; this is a common trick. The key insight is that we can represent the Student distribution as a Gaussian scale mixture, as we discussed in Sec. 3.4.3.1.

We can apply the GSM version of the Student distribution to our problem by associating a latent scale $z_n \in \mathbb{R}_+$ with each example. The complete data log likelihood is therefore given by

$$\log p(\mathbf{y}, \mathbf{z}|\mathbf{X}, \mathbf{w}, \sigma^2, \nu) = \sum_n -\frac{1}{2}\log(2\pi z_n \sigma^2) - \frac{1}{2z_n\sigma^2}(y_i - \mathbf{w}^T\mathbf{x}_i)^2 \tag{11.75}$$

$$+ (\frac{\nu}{2} - 1)\log(z_n) - z_n\frac{\nu}{2} + \text{const} \tag{11.76}$$

Ignoring terms not involving $\mathbf{w}$, and taking expectations, we have

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) = -\sum_n \frac{\lambda_n}{2\sigma^2}(y_n - \mathbf{w}^T\mathbf{x}_n)^2 \tag{11.77}$$

where $\lambda_n^t \triangleq \mathbb{E}\left[1/z_n|y_n, \mathbf{x}_n, \mathbf{w}^t\right]$. We recognize this as a weighted least squares objective, with weight λ_n^t per data point .

We now discuss how to compute these weights. Using the results from Sec. 2.7.5, one can show that

$$p(z_n|y_n, \mathbf{x}_n, \boldsymbol{\theta}) = \text{IG}(\frac{\nu+1}{2}, \frac{\nu+\delta_n}{2}) \tag{11.78}$$

where $\delta_n = \frac{(y_n - \mathbf{x}^T\mathbf{x}_n)^2}{\sigma^2}$ is the standardized residual. Hence

$$\lambda_n = \mathbb{E}\left[1/z_n\right] = \frac{\nu^t + 1}{\nu^t + \delta_n^t} \tag{11.79}$$

So if the residual δ_n^t is large, the point will be given low weight λ_n^t, which makes intuitive sense, since it is probably an outlier.

11.4.2 Robust regression using the Laplace distribution

In Sec. 2.7.3, we noted that the Laplace distribution is also robust to outliers. If we use this as our observation model for regression, we get the following likelihood:

$$p(y|\mathbf{x}, \mathbf{w}, b) = \text{Lap}(y|\mathbf{w}^\top\mathbf{x}, b) \propto \exp(-\frac{1}{b}|y - \mathbf{w}^\top\mathbf{x}|) \tag{11.80}$$

The robustness arises from the use of $|y - \mathbf{w}^\top\mathbf{x}|$ instead of $(y - \mathbf{w}^\top\mathbf{x})^2$. Fig. 11.9(a) gives an example of the method in action.

11.4.2.1 Computing the MLE using linear programming

We can compute the MLE for this model using linear programming. As we explain in Sec. 8.5.3, this is a way to solve a constrained optimization problems of the form

$$\underset{\mathbf{v}}{\text{argmin}}\, \mathbf{c}^\top\mathbf{v} \;\text{ s.t. }\; \mathbf{A}\mathbf{v} \leq \mathbf{b} \tag{11.81}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\mathbf{v} \in \mathbb{R}^n$ is the set of n unknown parameters, $\mathbf{c}^\top \mathbf{v}$ is the linear objective function we want to minimize, and $\mathbf{a}_i^\top \mathbf{v} \leq b_i$ is a set of m linear constraints we must satisfy. To apply this to our problem, let us define $\mathbf{v} = (w_1, \ldots, w_D, e_1, \ldots, e_N) \in \mathbb{R}^{D+N}$, where $e_i = |y_i - \hat{y}_i|$ is the residual error for example i. We want to minimize the sum of the residuals, so we define $\mathbf{c} = (0, \cdots, 0, 1, \cdots, 1) \in \mathbb{R}^{D+N}$, where the first D elements are 0, and the last N elements are 1.

We need to enforce the constraint that $e_i = |\hat{y}_i - y_i|$. In fact it is sufficient to enforce the constraint that $|\mathbf{w}^\top \mathbf{x}_i - y_i| \leq e_i$, since minimizing the sum of the e_i's will "push down" on this constraint and make it tight. Since $|a| \leq b \implies -b \leq a \leq b$, we can encode $|\mathbf{w}^\top \mathbf{x}_i - y_i| \leq e_i$ as two linear constraints:

$$e_i \geq \mathbf{w}^\top \mathbf{x}_i - y_i \tag{11.82}$$

$$e_i \geq -(\mathbf{w}^\top \mathbf{x}_i - y_i) \tag{11.83}$$

We can write Eq. (11.82)–Eq. (11.83) as

$$\big(\mathbf{x}_i, 0, \cdots, 0, -1, 0, \cdots, 0\big)^\top \mathbf{v} \leq y_i \tag{11.84}$$

where the first D entries are filled with $\mathbf{x}_i$, and the -1 is in the $(D+i)$'th entry of the vector. Similarly we can write Eq. (11.83) as

$$\big(-\mathbf{x}_i, 0, \cdots, 0, -1, 0, \cdots, 0\big)^\top \mathbf{v} \leq -y_i \tag{11.85}$$

We can write these constraints in the form $\mathbf{A}\mathbf{v} \leq \mathbf{b}$ by defining $\mathbf{A} \in \mathbb{R}^{2N \times (N+D)}$ as follows:

$$\mathbf{A} = \begin{pmatrix} \mathbf{x}_1 & -1 & 0 & 0 \cdots & 0 \\ -\mathbf{x}_1 & -1 & 0 & 0 \cdots & 0 \\ \mathbf{x}_2 & 0 & -1 & 0 \cdots & 0 \\ -\mathbf{x}_2 & 0 & -1 & 0 \cdots & 0 \\ & & \vdots & & \end{pmatrix} \tag{11.86}$$

and defining $\mathbf{b} \in \mathbb{R}^{2N}$ as

$$\mathbf{b} = \big(y_1, -y_1, y_2, -y_2, \cdots, y_N, -y_N\big) \tag{11.87}$$

11.4.3 Robust regression using Huber loss

An alternative to minimizing the NLL using a Laplace or Student likelihood is to use the **Huber loss**, which is defined as follows:

$$\ell_{\text{huber}}(r, \delta) = \begin{cases} r^2/2 & \text{if } |r| \leq \delta \\ \delta|r| - \delta^2/2 & \text{if } |r| > \delta \end{cases} \tag{11.88}$$

This is equivalent to ℓ_2 for errors that are smaller than δ, and is equivalent to ℓ_1 for larger errors. See Fig. 5.3 for a plot.

The advantage of this loss function is that it is everywhere differentiable. Consequently optimizing the Huber loss is much faster than using the Laplace likelihood, since we can use standard smooth

optimization methods (such as SGD) instead of linear programming. Fig. 11.9 gives an illustration of the Huber loss function in action. The results are qualitatively similiar to the Laplace and Student methods.

The parameter δ, which controls the degree of robustness, is usually set by hand, or by cross-validation. However, [Bar19] shows how to approximate the Huber loss such that we can optimize δ by gradient methods.

11.4.4 Robust regression by randomly or iteratively removing outliers

In the computer vision community, a common approach to robust regression is to use **RANSAC**, which stands for "random sample consensus" [FB81]. This works as follows: we sample a small initial set of points, fit the model to them, identify outliers wrt this model (based on large residuals), remove the outliers, and then refit the model to the inliers. We repeat this for many random initial sets and pick the best model.

A deterministic alternative to RANSAC is the following iterative scheme: intially we assume that all datapoints are inliers, and we fit the model to compute $\hat{\mathbf{w}}_0$; then, for each iteration t, we identify the outlier points as those with large residual under the model $\hat{\mathbf{w}}_t$, remove them, and refit the model to the remaining points to get $\hat{\mathbf{w}}_{t+1}$. Even though this hard thresholding scheme makes the problem nonconvex, this simple scheme can be proved to rapidly converge to the optimal estimate under some reasonable assumptions [Muk+19; Sug+19].

11.5 Lasso regression

In Sec. 11.3, we assumed a Gaussian prior for the regression coefficients when fitting linear regression models. This is often a good choice, since it encourages the parameters to be small, and hence prevents overfitting. However, sometimes we want the parameters to not just be small, but to be exactly zero, i.e., we want $\hat{\mathbf{w}}$ to be **sparse**, so that we minimize the **L0-norm**:

$$||\mathbf{w}||_0 = \sum_{d=1}^{D} \mathbb{I}\left(|w_d| > 0\right) \tag{11.89}$$

This is useful because it can be used to perform **feature selection**. To see this, note that the prediction has the form $f(\mathbf{x}; \mathbf{w}) = \sum_{d=1}^{D} w_d x_d$, so if any $w_d = 0$, we ignore the corresponding feature x_d. (The same idea can be applied to nonlinear models, such as DNNs, by encouraging the first layer weights to be sparse.)

11.5.1 MAP estimation with a Laplace prior (ℓ_1 regularization)

There are many ways to compute such sparse estimates (see e.g., [Bha+19]). In this section we focus on MAP estimation using the Laplace distribution (which we discussed in Sec. 11.4.2) as the prior:

$$p(\mathbf{w}|\lambda) = \prod_{d=1}^{D} \text{Lap}(w_d|0, 1/\lambda) \propto \prod_{d=1}^{D} e^{-\lambda|w_d|} \tag{11.90}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

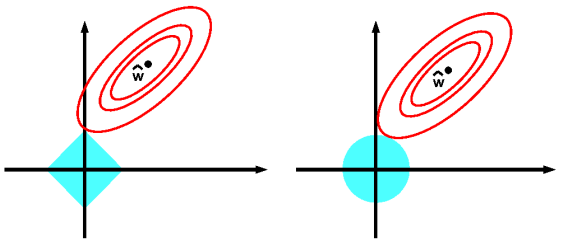


Figure 11.10: Illustration of ℓ_1 (left) vs ℓ_2 (right) regularization of a least squares problem. Adapted from Figure 3.12 of [HTF01].

where λ is the sparsity parameter, and

$$\text{Lap}(w|\mu, b) \triangleq \frac{1}{2b} \exp\left(-\frac{|w-\mu|}{b}\right) \tag{11.91}$$

Here μ is a location parameter and $b > 0$ is a scale parameter. Fig. 2.13 shows that $\text{Lap}(w|0, b)$ puts more density on 0 than $\mathcal{N}(w|0, \sigma^2)$, even when we fix the variance to be the same.

To perform MAP estimation of a linear regression model with this prior, we just have to minimize the following objective:

$$\text{PNLL}(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{w}|\lambda) = ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 + \lambda||\mathbf{w}||_1 \tag{11.92}$$

where $||\mathbf{w}||_1 \triangleq \sum_{d=1}^{D} |w_d|$ is the ℓ_1 norm of $\mathbf{w}$. This method is called **lasso**, which stands for "least absolute shrinkage and selection operator" [Tib96]. (We explain the reason for this name below.) More generally, MAP estimation with a Laplace prior is called **ℓ_1-regularization**.

Note also that we could use other norms for the weight vector. In general, the q-norm is defined as follows:

$$\|\mathbf{w}\|_q = \left(\sum_{d=1}^{D} |w_d|^q\right)^{1/q} \tag{11.93}$$

For $q < 1$, we can get even sparser solutions. In the limit where $q = 0$, we get the **ℓ_0-norm**:

$$\|\mathbf{w}\|_0 = \sum_{d=1}^{D} \mathbb{I}\left(|w_d| > 0\right) \tag{11.94}$$

However, one can show that for any $q < 1$, the problem becomes non-convex (see e.g., [HTW15]). Thus ℓ_1-norm is the tightest **convex relaxation** of the ℓ_0-norm.

11.5.2 Why does ℓ_1 regularization yield sparse solutions?

We now explain why ℓ_1 regularization results in sparse solutions, whereas ℓ_2 regularization does not. We focus on the case of linear regression, although similar arguments hold for other models.

The lasso objective is the following non-smooth objective:

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) + \lambda ||\mathbf{w}||_1 \tag{11.95}$$

This is the Lagrangian for the following quadratic program (see Sec. 8.5.4):

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) \quad \text{s.t.} \quad ||\mathbf{w}||_1 \leq B \tag{11.96}$$

where B is an upper bound on the ℓ_1-norm of the weights: a small (tight) bound B corresponds to a large penalty λ, and vice versa.

Similarly, we can write the ridge regression objective $\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) + \lambda ||\mathbf{w}||_2^2$ in bound constrained form:

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) \quad \text{s.t.} \quad ||\mathbf{w}||_2^2 \leq B \tag{11.97}$$

In Fig. 11.10, we plot the contours of the NLL objective function, as well as the contours of the ℓ_2 and ℓ_1 constraint surfaces. From the theory of constrained optimization (Sec. 8.5) we know that the optimal solution occurs at the point where the lowest level set of the objective function intersects the constraint surface (assuming the constraint is active). It should be geometrically clear that as we relax the constraint B, we "grow" the ℓ_1 "ball" until it meets the objective; the corners of the ball are more likely to intersect the ellipse than one of the sides, especially in high dimensions, because the corners "stick out" more. The corners correspond to sparse solutions, which lie on the coordinate axes. By contrast, when we grow the ℓ_2 ball, it can intersect the objective at any point; there are no "corners", so there is no preference for sparsity.

11.5.3 Hard vs soft thresholding

The lasso objective has the form $\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda ||\mathbf{w}||_1$. One can show (Exercise 11.3) that the gradient for the smooth NLL part is given by

$$\frac{\partial}{\partial w_d} \text{NLL}(\mathbf{w}) = a_d w_d - c_d \tag{11.98}$$

$$a_d = \sum_{n=1}^{N} x_{nd}^2 \tag{11.99}$$

$$c_d = \sum_{n=1}^{N} x_{nd}(y_n - \mathbf{w}_{-d}^\top \mathbf{x}_{n,-d}) \tag{11.100}$$

where $\mathbf{w}_{-d}$ is $\mathbf{w}$ without component d, and similarly $\mathbf{x}_{n,-d}$ is feature vector $\mathbf{x}_n$ without component d. We see that c_d is proportional to the correlation between d'th column of features, $\mathbf{x}_{:,d}$, and the residual error obtained by predicting using all the other features, $\mathbf{r}_{-d} = \mathbf{y} - \mathbf{X}_{:,-d}\mathbf{w}_{-d}$. Hence the magnitude of c_d is an indication of how relevant feature d is for predicting $\mathbf{y}$, relative to the other features and the current parameters. Setting the gradient to 0 gives the optimal update for w_d, keeping all other weights fixed:

$$w_d = c_d / a_d = \frac{\mathbf{x}_{:,d}^\top \mathbf{r}_{-d}}{||\mathbf{x}_{:,d}||_2^2} \tag{11.101}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

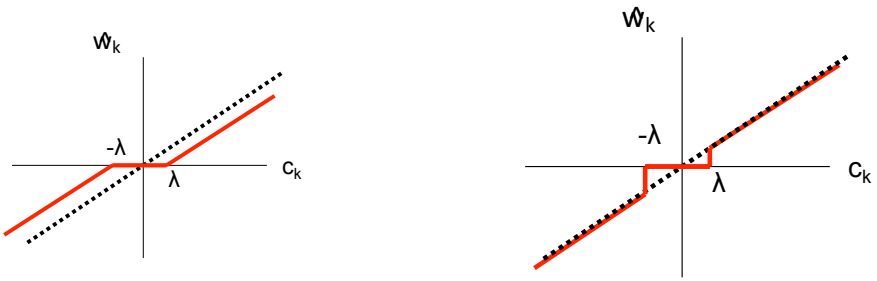


Figure 11.11: Left: soft thresholding. Right: hard thresholding. In both cases, the horizontal axis is the residual error incurred by making predictions using all the coefficients except for w_k, and the vertical axis is the estimated coefficient $\hat{w}_k$ that minimizes this penalized residual. The flat region in the middle is the interval $[-\lambda, +\lambda]$.

The corresponding new prediction for $\mathbf{r}_{-d}$ becomes $\hat{\mathbf{r}}_{-d} = w_d \mathbf{x}_{:,d}$, which is the orthogonal projection of the residual onto the column vector $\mathbf{x}_{:,d}$, consistent with Eq. (11.15).

Now we add in the ℓ_1 term. Unfortunately, the $||\mathbf{w}||_1$ term is not differentiable whenever $w_d = 0$. Fortunately, we can still compute a subgradient at this point. Using Eq. (8.14) we find that

$$\partial_{w_d} \mathcal{L}(\mathbf{w}) = (a_d w_d - c_d) + \lambda \partial_{w_d} ||\mathbf{w}||_1 \tag{11.102}$$

$$= \begin{cases} \{a_d w_d - c_d - \lambda\} & \text{if } w_d < 0 \\ [-c_d - \lambda, -c_d + \lambda] & \text{if } w_d = 0 \\ \{a_d w_d - c_d + \lambda\} & \text{if } w_d > 0 \end{cases} \tag{11.103}$$

Depending on the value of c_d, the solution to $\partial_{w_d} \mathcal{L}(\mathbf{w}) = 0$ can occur at 3 different values of w_d, as follows:

1. If $c_d < -\lambda$, so the feature is strongly negatively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d + \lambda}{a_d} < 0$.
2. If $c_d \in [-\lambda, \lambda]$, so the feature is only weakly correlated with the residual, then the subgradient is zero at $\hat{w}_d = 0$.
3. If $c_d > \lambda$, so the feature is strongly positively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d - \lambda}{a_d} > 0$.

In summary, we have

$$\hat{w}_d(c_d) = \begin{cases} (c_d + \lambda)/a_d & \text{if } c_d < -\lambda \\ 0 & \text{if } c_d \in [-\lambda, \lambda] \\ (c_d - \lambda)/a_d & \text{if } c_d > \lambda \end{cases} \tag{11.104}$$

We can write this as follows:

$$\hat{w}_d = \text{SoftThreshold}(\frac{c_d}{a_d}, \lambda / a_d) \tag{11.105}$$

where

$$\text{SoftThreshold}(x, \delta) \triangleq \text{sign}(x)\,(|x| - \delta)_+ \tag{11.106}$$

and $x_+ = \max(x, 0)$ is the positive part of x. This is called **soft thresholding** (see also Sec. 8.6.2). This is illustrated in Fig. 11.11(a), where we plot $\hat{w}_d$ vs c_d. The dotted black line is the line $w_d = c_d/a_d$ corresponding to the least squares fit. The solid red line, which represents the regularized estimate $\hat{w}_d$, shifts the dotted line down (or up) by λ, except when $-\lambda \leq c_d \leq \lambda$, in which case it sets $w_d = 0$.

By contrast, in Fig. 11.11(b), we illustrate **hard thresholding**. This sets values of w_d to 0 if $-\lambda \leq c_d \leq \lambda$, but it does not shrink the values of w_d outside of this interval. The slope of the soft thresholding line does not coincide with the diagonal, which means that even large coefficients are shrunk towards zero. This is why lasso stands for "least absolute selection *and shrinkage* operator". Consequently, lasso is a biased estimator (see Sec. 4.7.6.1).

A simple solution to the biased estimate problem, known as **debiasing**, is to use a two-stage estimation process: we first estimate the support of the weight vector (i.e., identify which elements are non-zero) using lasso; we then re-estimate the chosen coefficients using least squares. For an example of this in action, see Sec. 11.5.3.

11.5.4 Regularization path

If $\lambda = 0$, we get the OLS solution. which will be dense. As we increase λ, the solution vector $\hat{\mathbf{w}}(\lambda)$ will tend to get sparser. If λ is bigger than some critical value, we get $\hat{\mathbf{w}} = \mathbf{0}$. This critical value is obtained when the gradient of the NLL cancels out with the gradient of the penalty:

$$\lambda_{\max} = \max_d |\nabla_{w_d} \text{NLL}(\mathbf{0})| = \max_d c_d(\mathbf{w} = 0) = \max_d |\mathbf{y}^\top \mathbf{x}_{:,d}| = ||\mathbf{X}^\top \mathbf{y}||_\infty \tag{11.107}$$

Alternatively, we can work with the bound B on the ℓ_1 norm. When $B = 0$, we get $\hat{\mathbf{w}} = \mathbf{0}$. As we increase B, the solution becomes denser. The largest value of B for which any component is zero is given by $B_{\max} = ||\hat{\mathbf{w}}_{\text{mle}}||_1$.

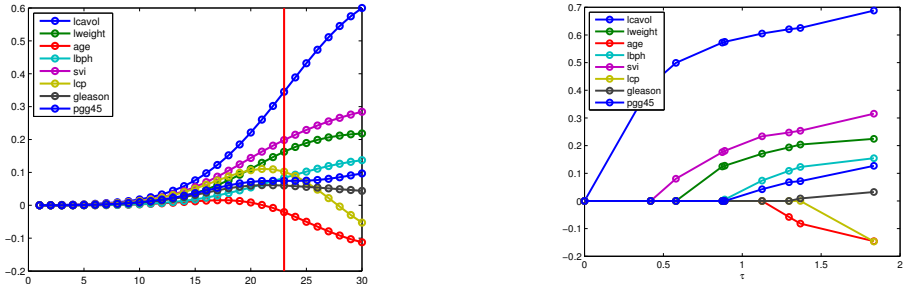
As we increase λ, the solution vector $\hat{\mathbf{w}}$ gets sparser, although not necessarily monotonically. We can plot the values $\hat{w}_d$ vs λ (or vs the bound B) for each feature d; this is known as the **regularization path**. This is illustrated in Fig. 11.12(b), where we apply lasso to the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) On the left, when $B = 0$, all the coefficients are zero. As we increase B, the coefficients gradually "turn on".[2] The analogous result for ridge regression is shown in Fig. 11.12(a). For ridge, we see all coefficients are non-zero (assuming $\lambda > 0$), so the solution is not sparse.

Remarkably, it can be shown that the lasso solution path is a piecewise linear function of λ [Efr+04; GL15]). That is, there are a set of critical values of λ where the active set of non-zero coefficients changes. For values of λ between these critical values, each non-zero coefficient increases or decreases in a linear fashion. This is illustrated in Fig. 11.12(b). Furthermore, one can solve for these critical values analytically [Efr+04]. In Fig. 11.13. we display the actual coefficient values at each of these critical steps along the regularization path (the last line is the least squares solution).

By changing λ from $\lambda_{\max}$ to 0, we can go from a solution in which all the weights are zero to a solution in which all weights are non-zero. Unfortunately, not all subset sizes are achievable using

2. It is common to plot the solution versus the **shrinkage factor**, defined as $s(B) = B/B_{\max}$, rather than against B. This merely affects the scale of the horizontal axis, not the shape of the curves.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 11.12: (a) Profiles of ridge coefficients for the prostate cancer example vs bound B on ℓ_2 norm of $\mathbf{w}$, so small B (large λ) is on the left. The vertical line is the value chosen by 5-fold CV using the 1 standard error rule. Adapted from Figure 3.8 of [HTF09]. Generated by code at figures.probml.ai/book1/11.12. (b) Same as (a) but using ℓ_1 norm of $\mathbf{w}$. The x-axis shows the critical values of $\lambda = 1/B$, where the regularization path is discontinuous. Adapted from Figure 3.10 of [HTF09]. Generated by code at figures.probml.ai/book1/11.12.

0	0	0	0	0	0	0	0
0.4279	0	0	0	0	0	0	0
0.5015	0.0735	0	0	0	0	0	0
0.5610	0.1878	0	0	0.0930	0	0	0
0.5622	0.1890	0	0.0036	0.0963	0	0	0
0.5797	0.2456	0	0.1435	0.2003	0	0	0.0901
0.5864	0.2572	-0.0321	0.1639	0.2082	0	0	0.1066
0.6994	0.2910	-0.1337	0.2062	0.3003	-0.2565	0	0.2452
0.7164	0.2926	-0.1425	0.2120	0.3096	-0.2890	-0.0209	0.2773

Figure 11.13: Values of the coefficients for linear regression model fit to prostate cancer dataset as we vary the strength of the ℓ_1 regularizer. These numbers are plotted in Fig. 11.12(b).

lasso. In particular, one can show that, if $D > N$, the optimal solution can have at most N variables in it, before reaching the complete set corresponding to the OLS solution of minimal ℓ_1 norm. In Sec. 11.5.8, we will see that by using an ℓ_2 regularizer as well as an ℓ_1 regularizer (a method known as the elastic net), we can achieve sparse solutions which contain more variables than training cases. This lets us explore model sizes between N and D.

11.5.5 Comparison of least squares, lasso, ridge and subset selection

In this section, we compare least squares, lasso, ridge and subset selection. For simplicity, we assume all the features of $\mathbf{X}$ are orthonormal, so $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$. In this case, the NLL is given by

$$\text{NLL}(\mathbf{w}) = ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 = \mathbf{y}^\top \mathbf{y} + \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} \tag{11.108}$$

$$= \text{const} + \sum_d w_d^2 - 2\sum_d \sum_n w_d x_{nd} y_n \tag{11.109}$$

so we see this factorizes into a sum of terms, one per dimension. Hence we can write down the MAP and ML estimates analytically for each w_d separately, as given below.

- **MLE** From Eq. (11.102), the OLS solution is given by

$$\hat{w}_d^{\text{mle}} = c_d / a_d = \mathbf{x}_{:d}^\top \mathbf{y} \tag{11.110}$$

where $\mathbf{x}_{:d}$ is the d'th column of $\mathbf{X}$.

- **Ridge** One can show that the ridge estimate is given by

$$\hat{w}_d^{\text{ridge}} = \frac{\hat{w}_d^{\text{mle}}}{1+\lambda} \tag{11.111}$$

- **Lasso** From Eq. (11.105), and using the fact that $\hat{w}_d^{\text{mle}} = c_d/a_d$, we have

$$\hat{w}_d^{\text{lasso}} = \text{sign}(\hat{w}_d^{\text{mle}}) \left(|\hat{w}_d^{\text{mle}}| - \lambda\right)_+ \tag{11.112}$$

This corresponds to soft thresholding, shown in Fig. 11.11(a).

- **Subset selection** If we pick the best K features using subset selection, the parameter estimate is as follows

$$\hat{w}_d^{\text{ss}} = \begin{cases} \hat{w}_d^{\text{mle}} & \text{if rank}(|\hat{w}_d^{\text{mle}}|) \leq K \\ 0 & \text{otherwise} \end{cases} \tag{11.113}$$

where rank refers to the location in the sorted list of weight magnitudes. This corresponds to hard thresholding, shown in Fig. 11.11(b).

We now experimentally compare the prediction performance of these methods on the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) Fig. 11.14 shows the estimated coefficients at the value of λ (or K) chosen by cross-validation; we see that the subset method is the sparsest, then lasso. In terms of predictive performance, all methods are very similar, as can be seen from Fig. 11.15a.

11.5.6 Variable selection consistency

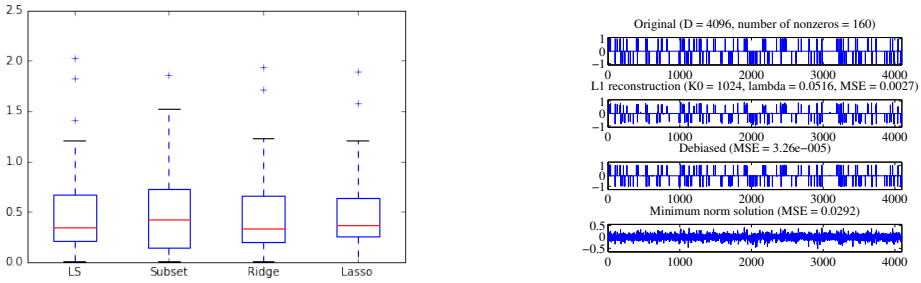
It is common to use ℓ_1 regularization to estimate the set of relevant variables, a process known as **variable selection**. A method that can recover the true set of relevant variables (i.e., the support of $\mathbf{w}^*$) in the $N \to \infty$ limit is called **model selection consistent**. (This is a theoretical notion that assumes the data comes from the model.)

Let us give an example. We first generate a sparse signal $\mathbf{w}^*$ of size $D = 4096$, consisting of 160 randomly placed ± 1 spikes. Next we generate a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 1024$. Finally we generate a noisy observation $\mathbf{y} = \mathbf{X}\mathbf{w}^* + \boldsymbol{\epsilon}$, where $\epsilon_n \sim \mathcal{N}(0, 0.01^2)$. We then estimate $\mathbf{w}$ from $\mathbf{y}$ and $\mathbf{X}$. The original $\mathbf{w}^*$ is shown in the first row of Fig. 11.15b. The second row is the ℓ_1 estimate $\hat{\mathbf{w}}_{L1}$ using $\lambda = 0.1\lambda_{\max}$. We see that this has "spikes" in the right places, so it has correctly identified the relevant variables. However, although we see that $\hat{\mathbf{w}}_{L1}$ has correctly identified the non-zero components, but they are too small, due to shrinkage. In the third row, we show the results of using the debiasing technique discussed in Sec. 11.5.3. This shows that we can recover the original weight vector. By contrast, the final row shows the OLS estimate, which is dense.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Term	OLS	Best Subset	Ridge	Lasso
intercept	2.465	2.477	2.467	2.465
lcalvol	0.676	0.736	0.522	0.548
lweight	0.262	0.315	0.255	0.224
age	-0.141	0.000	-0.089	0.000
lbph	0.209	0.000	0.186	0.129
svi	0.304	0.000	0.259	0.186
lcp	-0.287	0.000	-0.095	0.000
gleason	-0.021	0.000	0.025	0.000
pgg45	0.266	0.000	0.169	0.083
Test error	0.521	0.492	0.487	0.457
Std error	0.176	0.141	0.157	0.146

Figure 11.14: Results of different methods on the prostate cancer data, which has 8 features and 67 training cases. Methods are: OLS = ordinary least squares, Subset = best subset regression, Ridge, Lasso. Rows represent the coefficients; we see that subset regression and lasso give sparse solutions. Bottom row is the mean squared error on the test set (30 cases). Adapted from Table 3.3. of [HTF09]. Generated by code at figures.probml.ai/book1/11.14.



(a) *(b)*

Figure 11.15: (a) Boxplot displaying (absolute value of) prediction errors on the prostate cancer test set for different regression methods. Generated by code at figures.probml.ai/book1/11.15. (b) Example of recovering a sparse signal using lasso. See text for details. Adapted from Figure 1 of [FNW07]. Generated by code at figures.probml.ai/book1/11.15.

Furthermore, it is visually clear that there is no single threshold value we can apply to $\hat{\mathbf{w}}_{\text{mle}}$ to recover the correct sparse weight vector.

To use lasso to perform variable selection, we have to pick λ. It is common to use cross validation to pick the optimal value on the regularization path. However, it is important to note that cross validation is picking a value of λ that results in good predictive accuracy. This is not usually the same value as the one that is likely to recover the "true" model. To see why, recall that ℓ_1 regularization performs selection *and* shrinkage, that is, the chosen coefficients are brought closer to 0. In order to prevent relevant coefficients from being shrunk in this way, cross validation will tend to pick a value of λ that is not too large. Of course, this will result in a less sparse model which contains irrelevant

variables (false positives). Indeed, it was proved in [MB06] that the prediction-optimal value of λ does not result in model selection consistency. However, various extensions to the basic method have been devised that are model selection consistent (see e.g., [BG11; HTW15]).

11.5.7 Group lasso

In standard ℓ_1 regularization, we assume that there is a 1:1 correspondence between parameters and variables, so that if $\hat{w}_d = 0$, we interpret this to mean that variable d is excluded. But in more complex models, there may be many parameters associated with a given variable. In particular, each variable d may have a vector of weights $\mathbf{w}_d$ associated with it, so the overall weight vector has block structure, $\mathbf{w} = [\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_D]$. If we want to exclude variable d, we have to force the whole subvector $\mathbf{w}_d$ to go to zero. This is called **group sparsity**.

11.5.7.1 Applications

Here are some examples where group sparsity is useful:

- Linear regression with categorical inputs: If the d'th variable is categorical with K possible levels, then it will be represented as a one-hot vector of length K (Sec. 1.5.3.1), so to exclude variable d, we have to set the whole vector of incoming weights to 0.
- Multinomial logistic regression: The d'th variable will be associated with C different weights, one per class (Sec. 10.3), so to exclude variable d, we have to set the whole vector of outgoing weights to 0.
- Neural networks: the k'th neuron will have multiple inputs, so if we want to "turn the neuron off", we have to set all the incoming weights to zero. This allows us to use group sparsity to learn neural network structure (for details, see e.g., [GEH19]).
- Multi-task learning: each input feature is associated with C different weights, one per output task. If we want to use a feature for all of the tasks or none of the tasks, we should select weights at the group level [OTJ07].

11.5.7.2 Penalizing the two-norm

To encourage group sparsity, we partition the parameter vector into G groups, $\mathbf{w} = [\mathbf{w}_1, \ldots, \mathbf{w}_G]$. Then we minimize the following objective

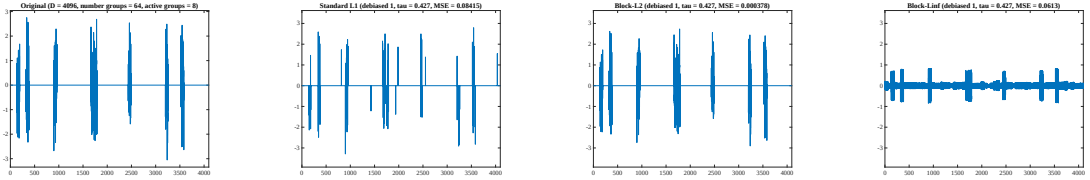
$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \sum_{g=1}^{G} ||\mathbf{w}_g||_2 \tag{11.114}$$

where $||\mathbf{w}_g||_2 = \sqrt{\sum_{d \in g} w_d^2}$ is the 2-norm of the group weight vector. If the NLL is least squares, this method is called **group lasso** [YL06; Kyu+10].

Note that if we had used the sum of the squared 2-norms in Eq. (11.114), then the model would become equivalent to ridge regression, since

$$\sum_{g=1}^{G} ||\mathbf{w}_g||_2^2 = \sum_{g} \sum_{d \in g} w_d^2 = ||\mathbf{w}||_2^2 \tag{11.115}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)* *(c)* *(d)*

Figure 11.16: Illustration of group lasso where the original signal is piecewise Gaussian. (a) Original signal. (b) Vanilla lasso estimate. (c) Group lasso estimate using a ℓ_2 norm on the blocks. (d) Group lasso estimate using an ℓ_∞ norm on the blocks. Adapted from Figures 3-4 of [WNF09]. Generated by code at figures.probml.ai/book1/11.16.

By using the square root, we are penalizing the radius of a ball containing the group's weight vector: the only way for the radius to be small is if all elements are small.

Another way to see why the square root version enforces sparsity at the group level is to consider the gradient of the objective. Suppose there is only one group of two variables, so the penalty has the form $\sqrt{w_1^2 + w_2^2}$. The derivative wrt w_1 is

$$\frac{\partial}{\partial w_1}(w_1^2 + w_2^2)^{\frac{1}{2}} = \frac{w_1}{\sqrt{w_1^2 + w_2^2}} \tag{11.116}$$

If w_2 is close to zero, then the derivative approaches 1, and w_1 is driven to zero as well, with force proportional to λ. If, however, w_2 is large, the derivative approaches 0, and w_1 is free to stay large as well. So all the coefficients in the group will have similar size.

11.5.7.3 Penalizing the infinity norm

A variant of this technique replaces the 2-norm with the infinity-norm [TVW05; ZRY05]:

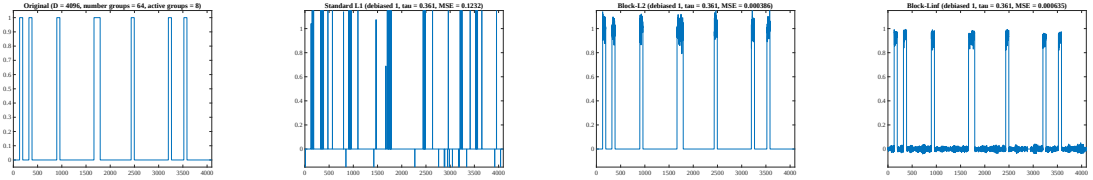
$$||\mathbf{w}_g||_\infty = \max_{d \in g} |w_d| \tag{11.117}$$

It is clear that this will also result in group sparsity, since if the largest element in the group is forced to zero, all the smaller ones will be as well.

11.5.7.4 Example

An illustration of these techniques is shown in Fig. 11.16 and Fig. 11.17. We have a true signal $\mathbf{w}$ of size $D = 2^{12} = 4096$, divided into 64 groups each of size 64. We randomly choose 8 groups of $\mathbf{w}$ and assign them non-zero values. In Fig. 11.16 the values are drawn from a $\mathcal{N}(0, 1)$; in Fig. 11.17, the values are all set to 1. We then sample a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 2^{10} = 1024$. Finally, we generate $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, 10^{-4}\mathbf{I}_N)$. Given this data, we estimate the support of $\mathbf{w}$ using ℓ_1 or group ℓ_1, and then estimate the non-zero values using least squares (debiased estimate).

We see from the figures that group lasso does a much better job than vanilla lasso, since it respects the known group structure. We also see that the ℓ_∞ norm has a tendency to make all the elements



(a) (b) (c) (d)

Figure 11.17: Same as Fig. 11.16, except the original signal is piecewise constant. Generated by code at figures.probml.ai/book1/11.17.

within a block to have similar magnitude. This is appropriate in the second example, but not the first. (The value of λ was the same in all examples, and was chosen by hand.)

11.5.8 Elastic net (ridge and lasso combined)

In group lasso, we need to specify the group structure ahead of time. For some problems, we don't know the group structure, and yet we would still like highly correlated coefficients to be treated as an implicit group. One way to achieve this effect, proposed in [ZH05], is to use the **elastic net**, which is a hybrid between lasso and ridge regression.[3] This corresponds to minimizing the following objective:

$$\mathcal{L}(\mathbf{w}, \lambda_1, \lambda_2) = ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 + \lambda_2 ||\mathbf{w}||_2^2 + \lambda_1 ||\mathbf{w}||_1 \tag{11.118}$$

This penalty function is *strictly convex* (assuming $\lambda_2 > 0$) so there is a unique global minimum, even if $\mathbf{X}$ is not full rank. It can be shown [ZH05] that any strictly convex penalty on $\mathbf{w}$ will exhibit a **grouping effect**, which means that the regression coefficients of highly correlated variables tend to be equal. In particular, if two features are identically equal, so $\mathbf{X}_{:j} = \mathbf{X}_{:k}$, one can show that their estimates are also equal, $\hat{w}_j = \hat{w}_k$. By contrast, with lasso, we may have that $\hat{w}_j = 0$ and $\hat{w}_k \neq 0$ or vice versa, resulting in less stable estimates.

In addition to its soft grouping behavior, elastic net has other advantages. In particular, if $D > N$, the maximum number of non-zero elements that can be selected (excluding the MLE, which has D non-zero elements) is N. By contrast, elastic net can select more than N non-zero variables on its path to the dense estimate, thus exploring more possible subsets of variables.

11.5.9 Optimization algorithms

A large variety of algorithms have been proposed to solve the lasso problem, and other ℓ_1-regularized convex objectives. In this section, we briefly mention some of the most popular methods.

3. It is apparently called the "elastic net" because it is "like a stretchable fishing net that retains 'all the big fish"' [ZH05].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11.5.9.1 Coordinate descent

Sometimes it is hard to optimize all the variables simultaneously, but it easy to optimize them one by one. In particular, we can solve for the j'th coefficient with all the others held fixed as follows:

$$w_j^* = \underset{\eta}{\operatorname{argmin}} \mathcal{L}(\mathbf{w} + \eta \mathbf{e}_j) \tag{11.119}$$

where $\mathbf{e}_j$ is the j'th unit vector. This is called **coordinate descent**. We can either cycle through the coordinates in a deterministic fashion, or we can sample them at random, or we can choose to update the coordinate for which the gradient is steepest.

This method is particularly appealing if each one-dimensional optimization problem can be solved analytically, as is the case for lasso (see Eq. (11.104)). This is known as the **shooting** algorithm [Fu98; WL08]. (The term "shooting" is a reference to cowboy theme inspired by the term "lasso".) See Algorithm 4 for details.

This coordinate descent method has been generalized to the GLM case in [FHT10], and is the basis of the popular **glmnet** software library.

Algorithm 4: Coordinate descent for lasso (aka shooting algorithm)

Initialize $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$;
repeat
  for $d = 1, \ldots, D$ **do**
    $a_d = \sum_{n=1}^N x_{nd}^2$;
    $c_d = \sum_{n=1}^N x_{nd}(y_n - \mathbf{w}^\top \mathbf{x}_n + w_d x_{nd})$;
    $w_d = \text{SoftThreshold}(\frac{c_j}{a_j}, \lambda / a_j)$;
until *converged*;

11.5.9.2 Projected gradient descent

In this section, we convert the non-differentiable ℓ_1 penalty into a smooth regularizer. To do this, we first use the **split variable trick** to define $\mathbf{w} = \mathbf{w}^+ - \mathbf{w}^-$, where $\mathbf{w}^+ = \max\{\mathbf{w}, 0\}$ and $\mathbf{w}^- = -\min\{\mathbf{w}, 0\}$. Now we can replace $||\mathbf{w}||_1$ with $\sum_d (w_d^+ + w_d^-)$. We also have to replace $\text{NLL}(\mathbf{w})$ with $\text{NLL}(\mathbf{w}^+ + \mathbf{w}^-)$. Thus we get the following smooth, but constrained, optimization problem:

$$\min_{\mathbf{w}^+ \geq 0, \mathbf{w}^- \geq 0} \text{NLL}(\mathbf{w}^+ - \mathbf{w}^-) + \lambda \sum_{d=1}^D (w_d^+ + w_d^-) \tag{11.120}$$

We can use projected gradient descent (Sec. 8.6.1) to solve this problem. Specifically, we can enforce the constraint by projecting onto the positive orthant, which we can do using $w_d := \max(w_d, 0)$; this operation is denoted by P_+. Thus the projected gradient update takes the following form:

$$\begin{pmatrix} \mathbf{w}_{t+1}^+ \\ \mathbf{w}_{t+1}^- \end{pmatrix} = P_+ \left(\begin{bmatrix} \mathbf{w}_t^+ - \eta_t \nabla \text{NLL}(\mathbf{w}_t^+ - \mathbf{w}_t^-) - \eta_t \lambda \mathbf{e} \\ \mathbf{w}_t^- + \eta_t \nabla \text{NLL}(\mathbf{w}_t^+ - \mathbf{w}_t^-) - \eta_t \lambda \mathbf{e} \end{bmatrix} \right) \tag{11.121}$$

where $\mathbf{e}$ is the unit vector of all ones.

11.5.9.3 Proximal gradient descent

In Sec. 8.6, we introduced proximal gradient descent, which can be used to optimize smooth functions with non-smooth penalties, such as ℓ_1. In Sec. 8.6.2, we showed that the proximal operator for the ℓ_1 penalty corresponds to soft thresholding. Thus the proximal gradient descent update can be written as

$$\mathbf{w}_{t+1} = \text{SoftThreshold}(\mathbf{w}_t - \eta_t \nabla \text{NLL}(\mathbf{w}_t), \eta_t \lambda) \tag{11.122}$$

where the soft thresholding operator (Eq. (8.144)) is applied elementwise. This is called the **iterative soft thresholding algorithm** or **ISTA** [DDDM04; Don95]. If we combine this with Nesterov acceleration, we get the method known as "fast ISTA" or **FISTA** [BT09], which is widely used to fit sparse linear models.

11.5.9.4 LARS

In this section, we discuss methods that can generate a set of solutions for different values of λ, starting with the empty set, i.e., they compute the full regularization path (Sec. 11.5.4). These algorithms exploit the fact that one can quickly compute $\hat{\mathbf{w}}(\lambda_k)$ from $\hat{\mathbf{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$; this is known as **warm starting**. In fact, even if we only want the solution for a single value of λ, call it λ_*, it can sometimes be computationally more efficient to compute a set of solutions, from $\lambda_{\max}$ down to λ_*, using warm-starting; this is called a **continuation method** or **homotopy** method. This is often much faster than directly "cold-starting" at λ_*; this is particularly true if λ_* is small.

The **LARS** algorithm [Efr+04], which stands for "least angle regression and shrinkage", is an example of a homotopy method for the lasso problem. This can compute $\hat{\mathbf{w}}(\lambda)$ for all possible values of λ in an efficient manner. (A similar algorithm was independently invented in [OPT00b; OPT00a]).

LARS works as follows. It starts with a large value of λ, such that only the variable that is most correlated with the response vector $\mathbf{y}$ is chosen. Then λ is decreased until a second variable is found which has the same correlation (in terms of magnitude) with the current residual as the first variable, where the residual at step k on the path is defined as $\mathbf{r}_k = \mathbf{y} - \mathbf{X}_{:,F_k}\mathbf{w}_k$, where F_k is the current **active set** (cf., Eq. (11.100)). Remarkably, one can solve for this new value of λ analytically, by using a geometric argument (hence the term "least angle"). This allows the algorithm to quickly "jump" to the next point on the regularization path where the active set changes. This repeats until all the variables are added.

It is necessary to allow variables to be removed from the current active set, even as we increase λ, if we want the sequence of solutions to correspond to the regularization path of lasso. If we disallow variable removal, we get a slightly different algorithm called **least angle regression** or **LAR**. LAR is very similar to **greedy forward selection**, and a method known as **least squares boosting** (see e.g., [HTW15]).

11.6 Bayesian linear regression *

We have seen how to compute the MLE and MAP estimate for linear regression models under various priors. In this section, we discuss how to compute the posterior over the parameters, $p(\boldsymbol{\theta}|\mathcal{D})$. For simplicity, we assume the variance is known, so we just want to compute $p(\mathbf{w}|\mathcal{D}, \sigma^2)$. See the sequel to this book, [Mur22], for the general case.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11.6.1 Computing the posterior

To start with, we must pick a prior. As in ridge regression in Sec. 11.3, we will use

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\ \breve{\mathbf{w}}, \breve{\mathbf{\Sigma}}) \tag{11.123}$$

where $\breve{\mathbf{w}}= \mathbf{0}$ is the prior mean and $\breve{\mathbf{\Sigma}}= \tau^2\mathbf{I}_D$ is the prior covariance. We can rewrite the likelihood in terms of an MVN as follows:

$$p(\mathcal{D}|\mathbf{w}, \sigma^2) = \prod_{n=1}^{N} p(y_n|\mathbf{w}^\top\mathbf{x}, \sigma^2) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2\mathbf{I}_N) \tag{11.124}$$

where $\mathbf{I}_N$ is the $N \times N$ identity matrix. We can then use Bayes rule for Gaussians (Eq. (3.36)) to derive the posterior, which is as follows:

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \sigma^2) \propto \mathcal{N}(\mathbf{w}|\mathbf{0}, \tau^2\mathbf{I}_D)\mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2\mathbf{I}_N) = \mathcal{N}(\mathbf{w}|\ \widehat{\mathbf{w}}, \widehat{\Sigma}) \tag{11.125}$$

$$\widehat{\mathbf{w}} \triangleq (\lambda\mathbf{I}_D + \mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \tag{11.126}$$

$$\widehat{\mathbf{\Sigma}} \triangleq \sigma^2(\lambda\mathbf{I}_D + \mathbf{X}^\top\mathbf{X})^{-1} \tag{11.127}$$

$\widehat{\mathbf{w}}$ is the posterior mean, $\widehat{\mathbf{\Sigma}}$ is the posterior covariance, and $\lambda \triangleq \frac{\sigma^2}{\tau^2}$ controls how strong the prior is compared to the observation noise. (Note that $\widehat{\mathbf{\Sigma}}$ is independent of the observed output variables $\mathbf{y}$, a peculiar property of linear Gaussian models.)

11.6.1.1 1d example

Suppose we have a 1d regression model of the form $f(x; \mathbf{w}) = w_0 + w_1x_1$, where the true parameters are $w_0 = -0.3$ and $w_1 = 0.5$. We now perform inference $p(\mathbf{w}|\mathcal{D})$ and visualize the 2d prior and posterior as the size of the training set N increases.

In particular, in Fig. 11.18 (which inspired the front cover of this book), we plot the likelihood, the posterior, and an approximation to the posterior predictive distribution.[4] Each row plots these distributions as we increase the amount of training data, N. We now explain each row:

- In the first row, $N = 0$, so the posterior is the same as the prior. In this case, our predictions are "all over the place", since our prior is essentially uniform.
- In the second row, $N = 1$, so we have seen one data point (the blue circle in the plot in the third column). Our posterior becomes constrained by the corresponding likelihood, and our predictions pass close to the observed data. However, we see that the posterior has a ridge-like shape, reflecting the fact that there are many possible solutions, with different slopes/intercepts. This makes sense since we cannot uniquely infer two parameters (w_0 and w_1) from one observation.
- In the third row, $N = 2$. In this case, the posterior becomes much narrower since we have two constraints from the likelihood. Our predictions about the future are all now closer to the training data.

4. To approximate this, we draw some samples from the posterior, $\mathbf{w}_s \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$, and then plot the line $\mathbb{E}[y|x, \mathbf{w}_s]$, where x ranges over $[-1, 1]$, for each sampled parameter value.

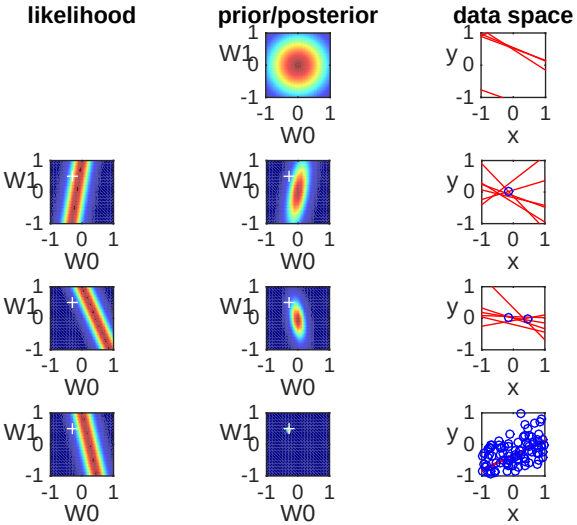


Figure 11.18: Sequential Bayesian inference of the parameters of a linear regression model $p(y|\mathbf{x}) = \mathcal{N}(y|w_0 + w_1 x_1, \sigma^2)$. Left column: likelihood function for current data point. Middle column: posterior given first N data points, $p(w_0, w_1|\mathbf{x}_{1:N}, y_{1:N}, \sigma^2)$. Right column: samples from the current posterior predictive distribution. Row 1: prior distribution ($N = 0$). Row 2: after 1 data point. Row 3: after 2 data points. Row 4: after 100 data points. The white cross in columns 1 and 2 represents the true parameter value; we see that the mode of the posterior rapidly converges to this point. The blue circles in column 3 are the observed data points. Adapted from Figure 3.7 of [Bis06]. Generated by code at figures.probml.ai/book1/11.18.

- In the fourth (last) row, $N = 100$. Now the posterior is essentially a delta function, centered on the true value of $\mathbf{w}_* = (-0.3, 0.5)$, indicated by a white cross in the plots in the first and second columns. The variation in our predictions is due to the inherent Gaussian noise with magnitude σ^2.

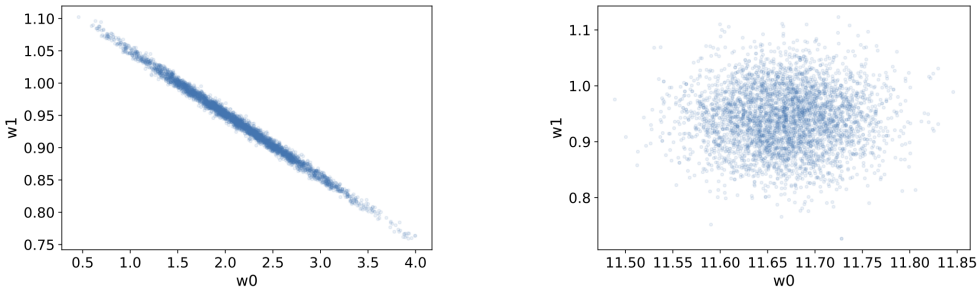
This example illustrates that, as the amount of data increases, the posterior mean estimate, $\widehat{\boldsymbol{\mu}} = \mathbb{E}[\mathbf{w}|\mathcal{D}]$, converges to the true value $\mathbf{w}_*$ that generated the data. We thus say that the Bayesian estimate is a consistent estimator (see Sec. 5.5.2 for more details). We also see that our posterior uncertainty decreases over time. This is what we mean when we say we are "learning" about the parameters as we see more data.

11.6.1.2 Centering the data reduces posterior correlation

The astute reader might notice that the shape of the 2d posterior in Fig. 11.18 is an elongated ellipse (which eventually collapses to a point as $N \to \infty$). This implies that there is a lot of posterior correlation between the two parameters, which can cause computational difficulties.

To understand why this happens, note that each data point induces a likelihood function corresponding to a line which goes through that data point. When we look at all the data together, we see

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.19: Posterior samples of $p(w_0, w_1|\mathcal{D})$ for 1d linear regression model $p(y|x, \boldsymbol{\theta}) = \mathcal{N}(y|w_0 + w_1 x, \sigma^2)$ with a Gaussian prior. (a) Original data. (b) Centered data. Generated by code at figures.probml.ai/book1/11.19

that predictions with maximum likelihood must correspond to lines that go through the mean of the data, $(\overline{x}, \overline{y})$. There are many such lines, but if we increase the slope, we must decrease the intercept. Thus we can think of the set of high probability lines as spinning around the data mean, like a wheel of fortune.[5] This correlation between w_0 and w_1 is why the posterior has the form of a diagonal line. (The Gaussian prior converts this into an elongated ellipse, but the posterior correlation still persists until the sample size causes the posterior to shrink to a point.)

It can be hard to compute such elongated posteriors. One simple solution is to center the input data, i.e., by using $x'_n = x_n - \overline{x}$. Now the lines can pivot around the origin, reducing the posterior correlation between w_0 and w_1. See Fig. 11.19 for an illustration. (We may also choose to divide each x_n by the standard deviation of that feature, as discussed in Sec. 10.2.8.)

Note that we can convert the posterior derived from fitting to the centered data back to the original coordinates by noting that

$$y' = w'_0 + w'_1 x' = w'_0 + w'_1(x - \overline{x}) = (w'_0 - w'_1 \overline{x}) + w'_1 x \tag{11.128}$$

Thus the parameters on the uncentered data are $w_0 = w'_0 - w'_1 \overline{x}$ and $w_1 = w'_1$.

11.6.2 Computing the posterior predictive

We have discussed how to compute our uncertainty about the parameters of the model, $p(\mathbf{w}|\mathcal{D})$. But what about the uncertainty associated with our predictions about future outputs? Using Eq. (3.39), we can show that the posterior predictive distribution at a test point $\mathbf{x}$ is also Gaussian:

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) = \int \mathcal{N}(y|\mathbf{x}^\top \mathbf{w}, \sigma^2) \mathcal{N}(\mathbf{w}|\, \widehat{\boldsymbol{\mu}}, \widehat{\boldsymbol{\Sigma}}) d\mathbf{w} \tag{11.129}$$

$$= \mathcal{N}(y|\, \widehat{\boldsymbol{\mu}}^\top\, \mathbf{x}, \widehat{\sigma}^2\, (\mathbf{x})) \tag{11.130}$$

where $\widehat{\sigma}^2\ (\mathbf{x}) \triangleq \sigma^2 + \mathbf{x}^\top\ \widehat{\boldsymbol{\Sigma}}\ \mathbf{x}$ is the variance of the posterior predictive distribution at point $\mathbf{x}$ after seeing the N training examples. The predicted variance depends on two terms: the variance

5. This analogy is from [Mar18, p96].

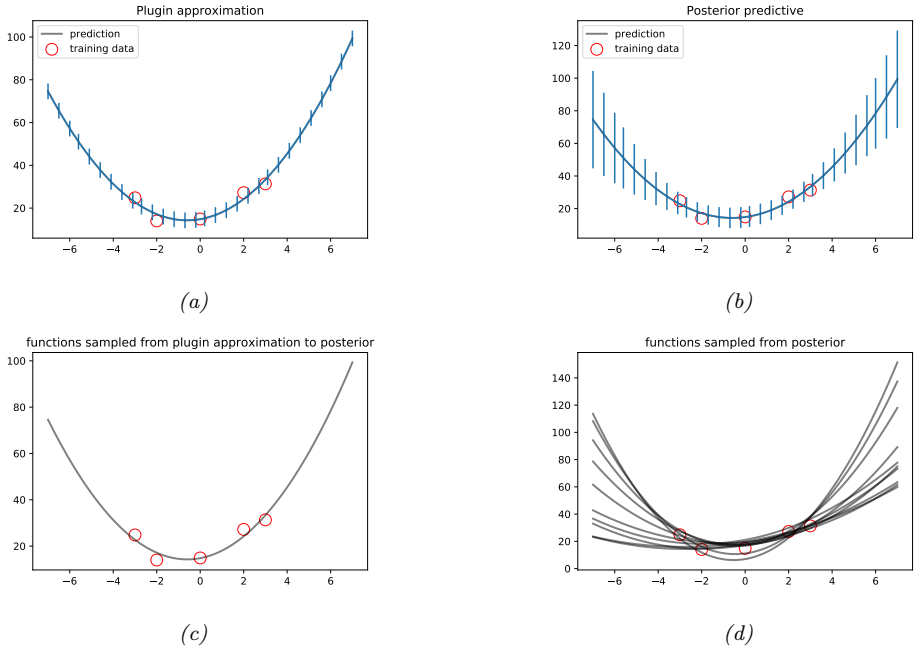


Figure 11.20: (a) Plugin approximation to predictive density (we plug in the MLE of the parameters) when fitting a second degree polynomial to some 1d data. (b) Posterior predictive density, obtained by integrating out the parameters. Black curve is posterior mean, error bars are 2 standard deviations of the posterior predictive density. (c) 10 samples from the plugin approximation to posterior predictive distribution. (d) 10 samples from the true posterior predictive distribution. Generated by code at figures.probml.ai/book1/11.20.

of the observation noise, σ^2, and the variance in the parameters, $\widehat{\mathbf{\Sigma}}$. The latter translates into variance about observations in a way which depends on how close $\mathbf{x}$ is to the training data $\mathcal{D}$. This is illustrated in Fig. 11.20(b), where we see that the error bars get larger as we move away from the training points, representing increased uncertainty. This can be important for certain applications, such as active learning, where we choose where to collect training data (see Sec. 19.4).

In some cases, it is computationally intractable to compute the parameter posterior, $p(\mathbf{w}|\mathcal{D})$. In such cases, we may choose to use a point estimate, $\hat{\mathbf{w}}$, and then to use the plugin approximation. This gives

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) = \int \mathcal{N}(y|\mathbf{x}^\top \mathbf{w}, \sigma^2)\delta(\mathbf{w} - \hat{\mathbf{w}})d\mathbf{w} = p(y|\mathbf{x}^\top \hat{\mathbf{w}}, \sigma^2). \tag{11.131}$$

We see that the posterior predictive variance is constant, and independent of the data, as illustrated in Fig. 11.20(a). If we sample a parameter from this posterior, we will always recover a single function, as shown in Fig. 11.20(c). By contrast, if we sample from the true posterior, $\mathbf{w}_s \sim p(\mathbf{w}|\mathcal{D}, \sigma^2)$, we will get a range of different functions, as shown in Fig. 11.20(d), which more accurately reflects our uncertainty.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11.6.3 Empirical Bayes (Automatic relevancy determination)

In Sec. 11.5, we computed the MAP estimate of the regression parameters for a linear regression model using a sparsity-promoting Laplace prior. In this section, we will use a Gaussian prior, which is not, in general, sparsity promoting. However, we will show that by using empirical Bayes (Sec. 4.6.5.3) to estimate the precisions of this Gaussian prior, the resulting estimate is sparse, for reasons we explain in Sec. 11.6.3.2.

This technique is known as **sparse Bayesian learning** [Tip01] or **automatic relevancy determination** (**ARD**) [Mac95; Nea96]. It was originally developed for neural networks (where sparsity is applied to the first layer weights), but here we apply it to linear models.

11.6.3.1 ARD for linear models

In this section, we explain ARD in more detail, by applying it to linear regression. The likelihood is $p(y|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, 1/\beta)$, where $\beta = 1/\sigma^2$. The prior is $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \mathbf{A}^{-1})$, where $\mathbf{A} = \text{diag}(\boldsymbol{\alpha})$. The marginal likelihood can be computed analytically (using Eq. (3.39)) as follows:

$$p(\mathbf{y}|\mathbf{X}, \boldsymbol{\alpha}, \beta) = \int \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, (1/\beta)\mathbf{I}_N)\mathcal{N}(\mathbf{w}|\mathbf{0}, \mathbf{A}^{-1})d\mathbf{w} \tag{11.132}$$

$$= \mathcal{N}(\mathbf{y}|\mathbf{0}, \beta^{-1}\mathbf{I}_N + \mathbf{X}\mathbf{A}^{-1}\mathbf{X}^\top) \tag{11.133}$$

$$= \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{C}_{\boldsymbol{\alpha}}) \tag{11.134}$$

where $\mathbf{C}_{\boldsymbol{\alpha}} \triangleq \beta^{-1}\mathbf{I}_N + \mathbf{X}\mathbf{A}^{-1}\mathbf{X}^\top$.

The objective is the log marginal likelihood, given by

$$\ell(\boldsymbol{\alpha}, \beta) = -\frac{1}{2}\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\alpha}, \beta) = \log|\mathbf{C}_{\boldsymbol{\alpha}}| + \mathbf{y}^\top \mathbf{C}_{\boldsymbol{\alpha}}^{-1}\mathbf{y} \tag{11.135}$$

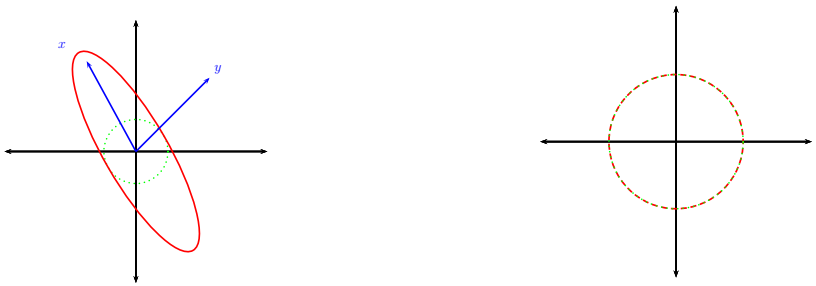
There are various algorithms for optimizing $\ell(\boldsymbol{\alpha}, \beta)$, some of which we discuss in Sec. 11.6.3.3.

ARD can be used as an alternative to ℓ_1 regularization (Sec. 11.5). Although the ARD objective is not convex, it tends to give much sparser results [WW12]. In addition, it can be shown [WRN10] that the ARD objective has many fewer local optima than the ℓ_0-regularized objective, and hence is much easier to optimize.

11.6.3.2 Why does ARD result in a sparse solution?

Once we have estimated $\boldsymbol{\alpha}$ and β, we can compute the posterior over the parameters using Bayes rule for Gaussians, to get $p(\mathbf{w}|\mathcal{D}, \hat{\boldsymbol{\alpha}}, \hat{\beta}) = \mathcal{N}(\mathbf{w}|\,\widehat{\mathbf{w}}, \widehat{\boldsymbol{\Sigma}})$, where $\widehat{\boldsymbol{\Sigma}}^{-1} = \hat{\beta}\mathbf{X}^\top\mathbf{X} + \mathbf{A}$ and $\widehat{\mathbf{w}} = \hat{\beta}\,\widehat{\boldsymbol{\Sigma}}\,\mathbf{X}^\top\mathbf{y}$. If we have $\hat{\alpha}_d \approx \infty$, then $\widehat{w}_d \approx 0$, so the solution vector will be sparse.

We now give an intuitive argument, based on [Tip01], about when such a sparse solution may be optimal. We shall assume $\beta = 1/\sigma^2$ is fixed for simplicity. Consider a 1d linear regression with 2 training examples, so $\mathbf{X} = \mathbf{x} = (x_1, x_2)$, and $\mathbf{y} = (y_1, y_2)$. We can plot $\mathbf{x}$ and $\mathbf{y}$ as vectors in the plane, as shown in Fig. 11.21. Suppose the feature is irrelevant for predicting the response, so $\mathbf{x}$ points in a nearly orthogonal direction to $\mathbf{y}$. Let us see what happens to the marginal likelihood as we change α. The marginal likelihood is given by $p(\mathbf{y}|\mathbf{x}, \alpha, \beta) = \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{C}_\alpha)$, where $\mathbf{C}_\alpha = \frac{1}{\beta}\mathbf{I} + \frac{1}{\alpha}\mathbf{x}\mathbf{x}^\top$. If α is finite, the posterior will be elongated along the direction of $\mathbf{x}$, as in Fig. 11.21(a). However,



(a) (b)

Figure 11.21: Illustration of why ARD results in sparsity. The vector of inputs $\mathbf{x}$ *does not point towards the vector of outputs* $\mathbf{y}$*, so the feature should be removed. (a) For finite* α*, the probability density is spread in directions away from* $\mathbf{y}$*. (b) When* $\alpha = \infty$*, the probability density at* $\mathbf{y}$ *is maximized. Adapted from Figure 8 of [Tip01].*

if $\alpha = \infty$, we have $\mathbf{C}_\alpha = \frac{1}{\beta}\mathbf{I}$, which is spherical, as in Fig. 11.21(b). If $|\mathbf{C}_\alpha|$ is held constant, the latter assigns higher probability density to the observed response vector $\mathbf{y}$, so this is the preferred solution. In other words, the marginal likelihood "punishes" solutions where α_d is small but $\mathbf{X}_{:,d}$ is irrelevant, since these waste probability mass. It is more parsimonious (from the point of view of Bayesian Occam's razor) to eliminate redundant dimensions.

Another way to understand the sparsity properties of ARD is as approximate inference in a hierarchical Bayesian model. In particular, suppose we put a conjugate prior on each precision, $\alpha_d \sim \text{Ga}(a, b)$, and on the observation precision, $\beta \sim \text{Ga}(c, d)$. Since exact inference with a Student prior is intractable, we can use variational Bayes [BT00], with a factored posterior approximation of the form

$$q(\mathbf{w}, \boldsymbol{\alpha}) = q(\mathbf{w})q(\boldsymbol{\alpha}) \approx \mathcal{N}(\mathbf{w}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \prod_d \text{Ga}(\alpha_d|\, \widehat{a}_d, \widehat{b}_d) \tag{11.136}$$

(ARD approximates $q(\boldsymbol{\alpha})$ by a point estimate.) By integrating out $\boldsymbol{\alpha}$, we get the posterior marginal $q(\mathbf{w})$ on the weights, which is given by

$$p(\mathbf{w}|\mathcal{D}) = \int \mathcal{N}(\mathbf{w}|\mathbf{0}, \text{diag}(\boldsymbol{\alpha})^{-1}) \prod_d \text{Ga}(\alpha_d|\, \widehat{a}_d, \widehat{b}) d\boldsymbol{\alpha} \tag{11.137}$$

This is a Gaussian scale mixture, and can be shown to be the same as a multivariate Student distribution (see Sec. 3.4.3.1), with non-diagonal covariance. Note that the Student has a large spike at 0, which intuitively explains why the posterior mean (which, for a Student distribution, is equal to the posterior mode) is sparse.

Finally, we can also view ARD as a MAP estimation problem with a **non-factorial prior** [WN07]. Intuitively, the dependence between the w_j parameters arises, despite the use of a diagonal Gaussian prior, because the prior precision α_j is estimated based after marginalizing out all $\mathbf{w}$, and hence depends on all the features. Interestingly, [WRN10] prove that MAP estimation with non-factorial priors is strictly better than MAP estimation with any possible factorial prior in the following

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

sense: the non-factorial objective always has fewer local minima than factorial objectives, while still satisfying the property that the global optimum of the non-factorial objective corresponds to the global optimum of the ℓ_0 objective — a property that ℓ_1 regularization, which has no local minima, does not enjoy.

11.6.3.3 Algorithms for ARD

There are various algorithms for optimizing $\ell(\boldsymbol{\alpha}, \beta)$. One approach is to use EM, in which we compute $p(\mathbf{w}|\mathcal{D}, \boldsymbol{\alpha})$ in the E step and then maximize $\boldsymbol{\alpha}$ in the M step. In variational Bayes, we infer both $\mathbf{w}$ and $\boldsymbol{\alpha}$ (see [Dru08] for details). In [WN10], they present a method based on iteratively reweighted ℓ_1 estimation.

Recently, [HXW17] showed that the nested iterative computations performed these methods can emulated by a recurrent neural network (Sec. 15.2). Furthermore, by training this model, it is possible to achieve much faster convergence than manually designed optimization algorithms.

11.6.3.4 Relevance vector machines

Suppose we create a linear regression model of the form $p(y|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}), \sigma^2)$, where $\boldsymbol{\phi}(\mathbf{x}) = [\mathcal{K}(\mathbf{x}, \mathbf{x}_1), \ldots, \mathcal{K}(\mathbf{x}, \mathbf{x}_N)]$, where $\mathcal{K}()$ is a kernel function (Sec. 17.2) and $\mathbf{x}_1, \ldots, \mathbf{x}_N$ are the N training points. This is called **kernel basis function expansion**, and transforms the input from $\mathbf{x} \in \mathcal{X}$ to $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^N$. Obviously this model has $O(N)$ parameters, and hence is nonparametric. However, we can use ARD to select a small subset of the exemplars. This technique is called the relevance vector machine (RVM), and will be discussed in Sec. 17.6.1.

11.7 Exercises

Exercise 11.1 [Multi-output linear regression]

(Source: Jaakkola.)

Consider a linear regression model with a 2 dimensional response vector $\mathbf{y}_i \in \mathbb{R}^2$. Suppose we have some binary input data, $x_i \in \{0, 1\}$. The training data is as follows:

x	y
0	$(-1, -1)^T$
0	$(-1, -2)^T$
0	$(-2, -1)^T$
1	$(1, 1)^T$
1	$(1, 2)^T$
1	$(2, 1)^T$

Let us embed each x_i into 2d using the following basis function:

$$\boldsymbol{\phi}(0) = (1, 0)^T, \quad \boldsymbol{\phi}(1) = (0, 1)^T \tag{11.138}$$

The model becomes

$$\hat{\mathbf{y}} = \mathbf{W}^T \boldsymbol{\phi}(x) \tag{11.139}$$

where $\mathbf{W}$ is a 2×2 matrix. Compute the MLE for $\mathbf{W}$ from the above data.

Exercise 11.2 [Centering and ridge regression]
Assume that $\overline{\mathbf{x}} = 0$, so the input data has been centered. Show that the optimizer of

$$J(\mathbf{w}, w_0) = (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1})^T(\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1}) + \lambda\mathbf{w}^T\mathbf{w} \tag{11.140}$$

is

$$\hat{w}_0 = \overline{y} \tag{11.141}$$

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \tag{11.142}$$

Exercise 11.3 [Partial derivative of the RSS]
Let $RSS(\mathbf{w}) = ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2$ be the residual sum of squares.

a. Show that

$$\frac{\partial}{\partial w_k} RSS(\mathbf{w}) = a_k w_k - c_k \tag{11.143}$$

$$a_k = 2\sum_{i=1}^{n} x_{ik}^2 = 2||\mathbf{x}_{:,k}||^2 \tag{11.144}$$

$$c_k = 2\sum_{i=1}^{n} x_{ik}(y_i - \mathbf{w}_{-k}^T\mathbf{x}_{i,-k}) = 2\mathbf{x}_{:,k}^T\mathbf{r}_k \tag{11.145}$$

where $\mathbf{w}_{-k} = \mathbf{w}$ without component k, $\mathbf{x}_{i,-k}$ is $\mathbf{x}_i$ without component k, and $\mathbf{r}_k = \mathbf{y} - \mathbf{w}_{-k}^T\mathbf{x}_{:,-k}$ is the residual due to using all the features except feature k. Hint: Partition the weights into those involving k and those not involving k.

b. Show that if $\frac{\partial}{\partial w_k} RSS(\mathbf{w}) = 0$, then

$$\hat{w}_k = \frac{\mathbf{x}_{:,k}^T\mathbf{r}_k}{||\mathbf{x}_{:,k}||^2} \tag{11.146}$$

Hence when we sequentially add features, the optimal weight for feature k is computed by computing orthogonally projecting $\mathbf{x}_{:,k}$ onto the current residual.

Exercise 11.4 [Reducing elastic net to lasso]
Define

$$J_1(\mathbf{w}) = |\mathbf{y} - \mathbf{X}\mathbf{w}|^2 + \lambda_2|\mathbf{w}|^2 + \lambda_1|\mathbf{w}|_1 \tag{11.147}$$

and

$$J_2(\mathbf{w}) = |\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\tilde{\mathbf{w}}|^2 + c\lambda_1|\mathbf{w}|_1 \tag{11.148}$$

where $c = (1 + \lambda_2)^{-\frac{1}{2}}$ and

$$\tilde{\mathbf{X}} = c\begin{pmatrix}\mathbf{X} \\ \sqrt{\lambda}_2\mathbf{I}_d\end{pmatrix}, \quad \tilde{\mathbf{y}} = \begin{pmatrix}\mathbf{y} \\ \mathbf{0}_{d\times 1}\end{pmatrix} \tag{11.149}$$

Show

$$\text{argmin}\, J_1(\mathbf{w}) = c(\text{argmin}\, J_2(\mathbf{w})) \tag{11.150}$$

i.e.

$$J_1(c\mathbf{w}) = J_2(\mathbf{w}) \tag{11.151}$$

and hence that one can solve an elastic net problem using a lasso solver on modified data.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 11.5 [Shrinkage in linear regression]

(Source: Jaakkola.) Consider performing linear regression with an orthonormal design matrix, so $||\mathbf{x}_{:,k}||_2^2 = 1$ for each column (feature) k, and $\mathbf{x}_{:,k}^T \mathbf{x}_{:,j} = 0$, so we can estimate each parameter w_k separately.

Figure 10.15b plots $\hat{w}_k$ vs $c_k = 2\mathbf{y}^T \mathbf{x}_{:,k}$, the correlation of feature k with the response, for 3 different esimation methods: ordinary least squares (OLS), ridge regression with parameter λ_2, and lasso with parameter λ_1.

a. Unfortunately we forgot to label the plots. Which method does the solid (1), dotted (2) and dashed (3) line correspond to?
b. What is the value of λ_1?
c. What is the value of λ_2?

Exercise 11.6 [EM for mixture of linear regression experts]

Derive the EM equations for fitting a mixture of linear regression experts.

12 Generalized linear models *

12.1 Introduction

In Chapter 10, we discussed logistic regression, which, in the binary case, corresponds to the model $p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x}))$. In Chapter 11, we discuss linear regression, which corresponds to the model $p(y|\mathbf{x}, \mathbf{w}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2)$. These are obviously very similar to each other. In particular, the mean of the output, $\mathbb{E}[y|\mathbf{x}, \mathbf{w}]$, is a linear function of the inputs $\mathbf{x}$ in both cases.

It turns out that there is a broad family of models with this property, known as **generalized linear models** or **GLMs** [MN89]. We explain these in Sec. 12.3. But first we need to introduce the exponential family, which are a core building block of GLMs (and many other kinds of model).

12.2 The exponential family

In this section, we define the **exponential family**, which includes many common probability distributions. The exponential family plays a crucial role in statistics and machine learning, for various reasons, including the following:

- The exponential family is the unique family of distributions that has maximum entropy (and hence makes the least set of assumptions) subject to some user-chosen constraints, as discussed in Sec. 12.2.6.
- The exponential family is at the core of GLMs, as discussed in Sec. 12.3.
- The exponential family is at the core of variational inference, as discussed in the sequel to this book, [Mur22].
- Under certain regularity conditions, the exponential family is the only family of distributions with finite-sized sufficient statistics, as discussed in Sec. 12.2.4.
- All members of the exponential family have a **conjugate prior** [DY79], which simplifies Bayesian inference of the parameters, as discussed in Sec. 4.6.1.

12.2.1 Definition

Consider a family of probability distributions parameterized by $\boldsymbol{\eta} \in \mathbb{R}^K$ with fixed support over $\mathcal{Y}^D \subseteq \mathbb{R}^D$. We say that the distribution $p(\mathbf{y}|\boldsymbol{\eta})$ is in the **exponential family** if its density can be

written in the following way:

$$p(\mathbf{y}|\boldsymbol{\eta}) \triangleq \frac{1}{Z(\boldsymbol{\eta})} h(\mathbf{y}) \exp[\boldsymbol{\eta}^\top \mathbf{t}(\mathbf{y})] = h(\mathbf{y}) \exp[\boldsymbol{\eta}^\top \mathbf{t}(\mathbf{y}) - A(\boldsymbol{\eta})] \tag{12.1}$$

where $h(\mathbf{y})$ is a scaling constant (also known as the **base measure**, often 1), $\mathbf{t}(\mathbf{y}) \in \mathbb{R}^K$ are the **sufficient statistics**, $\boldsymbol{\eta}$ are the **natural parameters** or **canonical parameters**, $Z(\boldsymbol{\eta})$ is a normalization constant known as the **partition function**, and $A(\boldsymbol{\eta}) = \log Z(\boldsymbol{\eta})$ is the **log partition function**. One can show that A is a concave function over the concave set $\Omega \triangleq \{\boldsymbol{\eta} \in \mathbb{R}^K : A(\boldsymbol{\eta}) < \infty\}$.

It is convenient if the natural parameters are independent of each other. Formally, we say that an exponential family is **minimal** if there is no $\boldsymbol{\eta} \in \mathbb{R}^K \setminus \{0\}$ such that $\boldsymbol{\eta}^\top \mathbf{t}(\mathbf{y}) = 0$. This last condition can be violated in the case of multinomial distributions, because of the sum to one constraint on the parameters; however, it is easy to reparameterize the distribution using $K - 1$ independent parameters, as we show below.

Eq. (12.1) can be generalized by defining $\boldsymbol{\eta} = f(\boldsymbol{\phi})$, where $\boldsymbol{\phi}$ is some other, possibly smaller, set of parameters. In this case, the distribution has the form

$$p(\mathbf{y}|\boldsymbol{\phi}) = h(\mathbf{y}) \exp[f(\boldsymbol{\phi})^\top \mathbf{t}(\mathbf{y}) - A(f(\boldsymbol{\phi}))] \tag{12.2}$$

If the mapping from $\boldsymbol{\phi}$ to $\boldsymbol{\eta}$ is nonlinear, we call this a **curved exponential family**. If $\boldsymbol{\eta} = f(\boldsymbol{\phi}) = \boldsymbol{\phi}$, the model is said to be in **canonical form**. If, in addition, $\mathbf{t}(\mathbf{y}) = \mathbf{y}$, we say this is a **natural exponential family** or **NEF**. In this case, it can be written as

$$p(\mathbf{y}|\boldsymbol{\eta}) = h(\mathbf{y}) \exp[\boldsymbol{\eta}^\top \mathbf{y} - A(\boldsymbol{\eta})] \tag{12.3}$$

12.2.2 Examples

In this section, we consider some common examples of distributions in the exponential family. Each corresponds to a different way of defining $h(\mathbf{y})$ and $\mathbf{t}(\mathbf{y})$ (since Z and hence A is derived from knowing h and $\mathbf{t}$).

12.2.2.1 Bernoulli distribution

The Bernoulli distribution can be written in exponential family form as follows:

$$\begin{aligned} \text{Ber}(y|\mu) &= \mu^y (1-\mu)^{1-y} && (12.4)\\ &= \exp[y \log(\mu) + (1-y)\log(1-\mu)] && (12.5)\\ &= \exp[\mathbf{t}(y)^\top \boldsymbol{\eta}] && (12.6) \end{aligned}$$

where $\mathbf{t}(y) = [\mathbb{I}(y=1), \mathbb{I}(y=0)]$, $\boldsymbol{\eta} = [\log(\mu), \log(1-\mu)]$, and μ is the mean parameter. However, this is an **over-complete representation** since there is a linear dependendence between the features. We can see this as follows:

$$\mathbf{1}^\top \mathbf{t}(y) = \mathbb{I}(y=0) + \mathbb{I}(y=1) = 1 \tag{12.7}$$

If the representation is overcomplete, $\boldsymbol{\eta}$ is not uniquely identifiable. It is common to use a **minimal representation**, which means there is a unique $\boldsymbol{\eta}$ associated with the distribution. In this case, we

can just define

$$\text{Ber}(y|\mu) = \exp\left[y \log\left(\frac{\mu}{1-\mu}\right) + \log(1-\mu)\right] \tag{12.8}$$

We can put this into exponential family form by defining

$$\eta = \log\left(\frac{\mu}{1-\mu}\right) \tag{12.9}$$
$$\mathbf{t}(y) = y \tag{12.10}$$
$$A(\eta) = -\log(1-\mu) = \log(1+e^{\eta}) \tag{12.11}$$
$$h(y) = 1 \tag{12.12}$$

We can recover the mean parameter μ from the canonical parameter η using

$$\mu = \sigma(\eta) = \frac{1}{1+e^{-\eta}} \tag{12.13}$$

which we recognize as the logistic (sigmoid) function.

12.2.2.2 Categorical distribution

We can represent the discrete distribution with K categories as follows (where $y_k = \mathbb{I}(y=k)$):

$$\text{Cat}(y|\boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{y_k} = \exp\left[\sum_{k=1}^{K} y_k \log \mu_k\right] \tag{12.14}$$
$$= \exp\left[\sum_{k=1}^{K-1} y_k \log \mu_k + \left(1 - \sum_{k=1}^{K-1} y_k\right) \log(1 - \sum_{k=1}^{K-1} \mu_k)\right] \tag{12.15}$$
$$= \exp\left[\sum_{k=1}^{K-1} y_k \log\left(\frac{\mu_k}{1-\sum_{j=1}^{K-1}\mu_j}\right) + \log(1 - \sum_{k=1}^{K-1} \mu_k)\right] \tag{12.16}$$
$$= \exp\left[\sum_{k=1}^{K-1} y_k \log\left(\frac{\mu_k}{\mu_K}\right) + \log \mu_K\right] \tag{12.17}$$

where $\mu_K = 1 - \sum_{k=1}^{K-1} \mu_k$. We can write this in exponential family form as follows:

$$\text{Cat}(y|\boldsymbol{\eta}) = \exp(\boldsymbol{\eta}^\top \mathbf{t}(\mathbf{y}) - A(\boldsymbol{\eta})) \tag{12.18}$$
$$\boldsymbol{\eta} = [\log\frac{\mu_1}{\mu_K}, \ldots, \log\frac{\mu_{K-1}}{\mu_K}] \tag{12.19}$$
$$A(\boldsymbol{\eta}) = -\log(\mu_K) \tag{12.20}$$
$$\mathbf{t}(y) = [\mathbb{I}(y=1), \ldots, \mathbb{I}(y=K-1)] \tag{12.21}$$
$$h(y) = 1 \tag{12.22}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can recover the mean parameters from the canonical parameters using

$$\mu_k = \frac{e^{\eta_k}}{1 + \sum_{j=1}^{K-1} e^{\eta_j}} \tag{12.23}$$

If we define $\eta_K = 0$, we can rewrite this as follows:

$$\mu_k = \frac{e^{\eta_k}}{\sum_{j=1}^{K} e^{\eta_j}} \tag{12.24}$$

for $k = 1 : K$. Hence $\boldsymbol{\mu} = \mathcal{S}(\boldsymbol{\eta})$, where $\mathcal{S}$ is the softmax or multinomial logit function in Eq. (1.8). From this, we find

$$\mu_K = 1 - \frac{\sum_{k=1}^{K-1} e^{\eta_k}}{1 + \sum_{k=1}^{K-1} e^{\eta_k}} = \frac{1}{1 + \sum_{k=1}^{K-1} e^{\eta_k}} \tag{12.25}$$

and hence

$$A(\boldsymbol{\eta}) = -\log(\mu_K) = \log\left(\sum_{k=1}^{K} e^{\eta_k}\right) \tag{12.26}$$

12.2.2.3 Univariate Gaussian

The univariate Gaussian is usually written as follows:

$$\mathcal{N}(y|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{\frac{1}{2}}} \exp[-\frac{1}{2\sigma^2}(y-\mu)^2] \tag{12.27}$$

$$= \frac{1}{(2\pi)^{\frac{1}{2}}} \exp[\frac{\mu}{\sigma^2}y - \frac{1}{2\sigma^2}y^2 - \frac{1}{2\sigma^2}\mu^2 - \log\sigma] \tag{12.28}$$

We can put this in exponential family form by defining

$$\boldsymbol{\eta} = \begin{pmatrix} \mu/\sigma^2 \\ -\frac{1}{2\sigma^2} \end{pmatrix} \tag{12.29}$$

$$\mathbf{t}(y) = \begin{pmatrix} y \\ y^2 \end{pmatrix} \tag{12.30}$$

$$A(\boldsymbol{\eta}) = \frac{\mu^2}{2\sigma^2} + \log\sigma = \frac{-\eta_1^2}{4\eta_2} - \frac{1}{2}\log(-2\eta_2) \tag{12.31}$$

$$h(y) = \frac{1}{\sqrt{2\pi}} \tag{12.32}$$

We define the **moment parameters** as the mean of the sufficient statistics vector, $\mathbf{m} = \mathbb{E}[\mathbf{t}(\mathbf{y})]$. We see that for a univariate Gaussian, we have $\mathbf{m} = [\mu, \mu^2 + \sigma^2]$.

If we fix $\sigma^2 = 1$, we can write this as a natural exponential family, by defining

$$\eta = \mu \tag{12.33}$$
$$\mathbf{t}(y) = y \tag{12.34}$$
$$A(\mu) = \frac{\mu^2}{2\sigma^2} + \log \sigma = \frac{\mu^2}{2} \tag{12.35}$$
$$h(y) = \frac{1}{\sqrt{2\pi}} \exp[-\frac{y^2}{2}] \tag{12.36}$$

12.2.2.4 Multivariate Gaussian

It is common to parameterize the multivariate normal (MVN) in terms of the mean vector $\boldsymbol{\mu}$ and the covariance matrix $\boldsymbol{\Sigma}$. The corresponding pdf is given by

$$\mathcal{N}(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}\sqrt{\det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}\mathbf{y}^\top \boldsymbol{\Sigma}^{-1}\mathbf{y} + \mathbf{y}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} - \frac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right) \tag{12.37}$$
$$= c \exp\left(\mathbf{y}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} - \frac{1}{2}\mathbf{y}^\top \boldsymbol{\Sigma}^{-1}\mathbf{y}\right) \tag{12.38}$$
$$c \triangleq \frac{\exp(-\frac{1}{2}\boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu})}{(2\pi)^{D/2}\sqrt{\det(\boldsymbol{\Sigma})}} \tag{12.39}$$

However, we can also represent the Gaussian using **canonical parameters** or **natural parameters**. In particular, we define the **precision matrix** $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$, and the precision-weighted mean, $\boldsymbol{\xi} \triangleq \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}$. The pdf for the MVN in **canonical form** (also called **information form**) is then giving by the following:

$$\mathcal{N}_c(\mathbf{y}|\boldsymbol{\xi}, \boldsymbol{\Lambda}) \triangleq c' \exp\left(\mathbf{y}^\top \boldsymbol{\xi} - \frac{1}{2}\mathbf{y}^\top \boldsymbol{\Lambda}\mathbf{y}\right) \tag{12.40}$$
$$c' \triangleq \frac{\exp(-\frac{1}{2}\boldsymbol{\xi}^\top \boldsymbol{\Lambda}^{-1}\boldsymbol{\xi})}{(2\pi)^{D/2}\sqrt{\det(\boldsymbol{\Lambda}^{-1})}} \tag{12.41}$$

where we use the notation $\mathcal{N}_c()$ to distinguish it from the standard parameterization $\mathcal{N}()$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can convert this to exponential family notation as follows:

$$\mathcal{N}_c(\mathbf{y}|\boldsymbol{\xi},\boldsymbol{\Lambda}) = \underbrace{(2\pi)^{-D/2}}_{h(\mathbf{y})} \underbrace{\exp\left[\frac{1}{2}\log|\boldsymbol{\Lambda}| - \frac{1}{2}\boldsymbol{\xi}^\top\boldsymbol{\Lambda}^{-1}\boldsymbol{\xi}\right]}_{g(\boldsymbol{\eta})} \exp\left[-\frac{1}{2}\mathbf{y}^\top\boldsymbol{\Lambda}\mathbf{y} + \mathbf{y}^\top\boldsymbol{\xi}\right] \tag{12.42}$$

$$= h(\mathbf{y})g(\boldsymbol{\eta})\exp\left[-\frac{1}{2}\mathbf{y}^\top\boldsymbol{\Lambda}\mathbf{y} + \mathbf{y}^\top\boldsymbol{\xi}\right] \tag{12.43}$$

$$= h(\mathbf{y})g(\boldsymbol{\eta})\exp\left[-\frac{1}{2}(\sum_{ij} y_i y_j \Lambda_{ij}) + \mathbf{y}^\top\boldsymbol{\xi}\right] \tag{12.44}$$

$$= h(\mathbf{y})g(\boldsymbol{\eta})\exp\left[-\frac{1}{2}\mathrm{vec}(\boldsymbol{\Lambda})^\top\mathrm{vec}(\mathbf{y}\mathbf{y}^\top) + \mathbf{y}^\top\boldsymbol{\xi}\right] \tag{12.45}$$

$$= h(\mathbf{y})\exp\left[\boldsymbol{\eta}^\top\mathbf{t}(\mathbf{y}) - A(\boldsymbol{\eta})\right] \tag{12.46}$$

where

$$h(\mathbf{y}) = (2\pi)^{-D/2} \tag{12.47}$$

$$\boldsymbol{\eta} = [\boldsymbol{\xi}; -\frac{1}{2}\mathrm{vec}(\boldsymbol{\Lambda})] = [\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}; -\frac{1}{2}\mathrm{vec}(\boldsymbol{\Sigma}^{-1})] \tag{12.48}$$

$$\mathbf{t}(\mathbf{y}) = [\mathbf{y}; \mathrm{vec}(\mathbf{y}\mathbf{y}^\top)] \tag{12.49}$$

$$A(\boldsymbol{\eta}) = -\log g(\boldsymbol{\eta}) = -\frac{1}{2}\log|\boldsymbol{\Lambda}| + \frac{1}{2}\boldsymbol{\xi}^\top\boldsymbol{\Lambda}^{-1}\boldsymbol{\xi} \tag{12.50}$$

From this, we see that the mean (moment) parameters are given by

$$\mathbf{m} = \mathbb{E}[\mathbf{t}(\mathbf{y})] = [\boldsymbol{\mu}; \boldsymbol{\mu}\boldsymbol{\mu}^\top + \boldsymbol{\Sigma}] \tag{12.51}$$

(Note that the above is not a minimal representation, since $\boldsymbol{\Lambda}$ is a symmetric matrix. We can convert to minimal form by working with the upper or lower half of each matrix.)

In Sec. 3.2.3, we gave the equations for marginalization and conditioning of an MVN in moment form. It is also possible to derive the marginalization and conditioning formulas in information for (see Exercise 3.12). We find

$$p(\mathbf{y}_2) = \mathcal{N}_c(\mathbf{y}_2|\boldsymbol{\xi}_2 - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\xi}_1, \boldsymbol{\Lambda}_{22} - \boldsymbol{\Lambda}_{21}\boldsymbol{\Lambda}_{11}^{-1}\boldsymbol{\Lambda}_{12}) \tag{12.52}$$

$$p(\mathbf{y}_1|\mathbf{y}_2) = \mathcal{N}_c(\mathbf{y}_1|\boldsymbol{\xi}_1 - \boldsymbol{\Lambda}_{12}\mathbf{y}_2, \boldsymbol{\Lambda}_{11}) \tag{12.53}$$

Thus we see that marginalization is easier in moment form, and conditioning is easier in information form.

12.2.2.5 Non-examples

Not all distributions of interest belong to the exponential family. For example, the Student distribution (Sec. 2.7.1) does not belong, since its pdf (Eq. (2.127)) does not have the required form. (However, there is a generalization, known as the **ϕ-exponential family** [Nau04; Tsa88] which does include the Student distribution.)

As a more subtle example, consider the uniform distribution, $Y \sim \text{Unif}(\theta_1, \theta_2)$. The pdf has the form

$$p(y|\boldsymbol{\theta}) = \frac{1}{\theta_2 - \theta_1} \mathbb{I}(\theta_1 \leq y \leq \theta_2) \tag{12.54}$$

It is tempting to think this is in the exponential family, with $h(y) = 1$, $\mathbf{t}(y) = \mathbf{0}$, and $Z(\boldsymbol{\theta}) = \theta_2 - \theta_1$. However, the support of this distribution (i.e., the set of values $\mathcal{Y} = \{y : p(y) > 0\}$ depends on the parameters $\boldsymbol{\theta}$, which violates an assumption of the exponential family.

12.2.3 Log partition function is cumulant generating function

The first and second **cumulants** of a distribution are its mean $\mathbb{E}[Y]$ and variance $\mathbb{V}[Y]$, whereas the first and second moments are $\mathbb{E}[Y]$ and $\mathbb{E}[Y^2]$. We can also compute higher order cumulants (and moments). An important property of the exponential family is that derivatives of the log partition function can be used to generate all the **cumulants** of the sufficient statistics In particular, the first and second cumulants are given by

$$\nabla_{\boldsymbol{\eta}} A(\boldsymbol{\eta}) = \mathbb{E}[\mathbf{t}(\mathbf{y})] \tag{12.55}$$

$$\nabla^2 A(\boldsymbol{\eta}) = \text{Cov}[\mathbf{t}(\mathbf{y})] \tag{12.56}$$

We prove this result below.

From the above result, we see that the Hessian of A is positive definite, and hence A is a convex function (see Sec. 8.1.3). This will be important when we talk about parameter estimation in Sec. 12.2.4.

12.2.3.1 Derivation of the mean

To prove the result, we focus on the 1d case. For the first derivative we have

$$\frac{dA}{d\eta} = \frac{d}{d\eta}\left(\log \int \exp(\eta t(y)) h(y) dy\right) \tag{12.57}$$

$$= \frac{\frac{d}{d\eta}\int \exp(\eta t(y)) h(y) dy}{\int \exp(\eta t(y)) h(y) dy} \tag{12.58}$$

$$= \frac{\int t(y) \exp(\eta t(y)) h(y) dy}{\exp(A(\eta))} \tag{12.59}$$

$$= \int t(y) \exp(\eta t(y) - A(\eta)) h(y) dy \tag{12.60}$$

$$= \int t(y) p(y) dy = \mathbb{E}[t(y)] \tag{12.61}$$

For example, consider the Bernoulli distribution. We have $A(\eta) = \log(1 + e^{\eta})$, so the mean is given by

$$\frac{dA}{d\eta} = \frac{e^{\eta}}{1 + e^{\eta}} = \frac{1}{1 + e^{-\eta}} = \sigma(\eta) = \mu \tag{12.62}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

12.2.3.2 Derivation of the variance

For the second derivative we have

$$\frac{d^2A}{d\eta^2} = \frac{d}{d\eta}\int t(y)\exp(\eta t(y) - A(\eta))h(y)dy \tag{12.63}$$

$$= \int t(y)\exp\left(\eta t(y) - A(\eta)\right)h(y)(t(y) - A'(\eta))dy \tag{12.64}$$

$$= \int t(y)p(y)(t(y) - A'(\eta))dy \tag{12.65}$$

$$= \int t^2(y)p(y)dy - A'(\eta)\int t(y)p(y)dy \tag{12.66}$$

$$= \mathbb{E}\left[t^2(Y)\right] - \mathbb{E}\left[t(y)\right]^2 = \mathbb{V}\left[t(y)\right] \tag{12.67}$$

where we used the fact that $A'(\eta) = \frac{dA}{d\eta} = \mathbb{E}\left[t(y)\right]$. For example, for the Bernoulli distribution we have

$$\frac{d^2A}{d\eta^2} = \frac{d}{d\eta}(1+e^{-\eta})^{-1} = (1+e^{-\eta})^{-2}e^{-\eta} \tag{12.68}$$

$$= \frac{e^{-\eta}}{1+e^{-\eta}}\frac{1}{1+e^{-\eta}} = \frac{1}{e^{\eta}+1}\frac{1}{1+e^{-\eta}} = (1-\mu)\mu \tag{12.69}$$

12.2.4 MLE for the exponential family

The likelihood of an exponential family model has the form

$$p(\mathcal{D}|\boldsymbol{\eta}) = \left[\prod_{n=1}^{N} h(\mathbf{y}_n)\right]\exp\left(\boldsymbol{\eta}^\top[\sum_{n=1}^{N}\mathbf{t}(\mathbf{y}_n)] - NA(\boldsymbol{\eta})\right) \propto \exp\left[\boldsymbol{\eta}^\top\mathbf{t}(\mathcal{D}) - NA(\boldsymbol{\eta})\right] \tag{12.70}$$

where $\mathbf{t}(\mathcal{D})$ are the sufficient statistics:

$$\mathbf{t}(\mathcal{D}) = [\sum_{n=1}^{N}\mathbf{t}_1(\mathbf{y}_n), \ldots, \sum_{n=1}^{N}\mathbf{t}_K(\mathbf{y}_n)] \tag{12.71}$$

For example, for the Bernoulli model we have $\mathbf{t}(\mathcal{D}) = [\sum_n \mathbb{I}\left(y_n = 1\right)]$, and for the univariate Gaussian, we have $\mathbf{t}(\mathcal{D}) = [\sum_n y_n, \sum_n y_n^2]$.

The **Pitman-Koopman-Darmois theorem** states that, under certain regularity conditions, the exponential family is the only family of distributions with finite sufficient statistics. (Here, finite means a size independent of the size of the data set.) In other words, for an exponential family with natural parameters $\boldsymbol{\eta}$, we have

$$p(\mathcal{D}|\boldsymbol{\eta}) = p(\mathbf{t}(\mathcal{D})|\boldsymbol{\eta}) \tag{12.72}$$

We now show how to use this result to compute the MLE. The log likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\eta}) = \boldsymbol{\eta}^\top\mathbf{t}(\mathcal{D}) - NA(\boldsymbol{\eta}) + \text{const} \tag{12.73}$$

Since $-A(\boldsymbol{\eta})$ is concave in $\boldsymbol{\eta}$, and $\boldsymbol{\eta}^\top \mathbf{t}(\mathcal{D})$ is linear in $\boldsymbol{\eta}$, we see that the log likelihood is concave, and hence has a unique global maximum. To derive this maximum, we use the fact (shown in Sec. 12.2.3) that the derivative of the log partition function yields the expected value of the sufficient statistic vector:

$$\nabla_{\boldsymbol{\eta}} \log p(\mathcal{D}|\boldsymbol{\eta}) = \nabla_{\boldsymbol{\eta}} \boldsymbol{\eta}^\top \mathbf{t}(\mathcal{D}) - N \nabla_{\boldsymbol{\eta}} A(\boldsymbol{\eta}) = \mathbf{t}(\mathcal{D}) - N \mathbb{E}\left[\mathbf{t}(\mathbf{y})\right] \tag{12.74}$$

Setting this gradient to zero, we see that at the MLE, the empirical average of the sufficient statistics must equal the model's theoretical expected sufficient statistics, i.e., $\hat{\boldsymbol{\eta}}$ must satisfy

$$\mathbb{E}\left[\mathbf{t}(\mathbf{y})\right] = \frac{1}{N} \sum_{n=1}^{N} \mathbf{t}(\mathbf{y}_n) \tag{12.75}$$

This is called **moment matching**. For example, in the Bernoulli distribution, we have $\mathbf{t}(y) = \mathbb{I}(Y = 1)$, so the MLE satisfies

$$\mathbb{E}\left[\mathbf{t}(y)\right] = p(Y = 1) = \mu = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(y_n = 1) \tag{12.76}$$

12.2.5 Exponential dispersion family

In this section, we consider a slight extension of the natural exponential family known as the **exponential dispersion family**. This will be useful when we discuss GLMs in Sec. 12.3. For a scalar variable, this has the form

$$p(y|\eta, \sigma^2) = h(y, \sigma^2) \exp\left[\frac{\eta y - A(\eta)}{\sigma^2}\right] \tag{12.77}$$

Here σ^2 is called the **dispersion parameter**. For fixed σ^2, this is a natural exponential family.

12.2.6 Maximum entropy derivation of the exponential family

Suppose we want to find a distribution $p(\mathbf{x})$ to describe some data, where all we know are the expected values (F_k) of certain features or functions $f_k(\mathbf{x})$:

$$\int d\mathbf{x}\, p(\mathbf{x}) f_k(\mathbf{x}) = F_k \tag{12.78}$$

For example, f_1 might compute x, f_2 might compute x^2, making F_1 the empirical mean and F_2 the empirical second moment. Our prior belief in the distribution is $q(x)$.

To formalize what we mean by "least number of assumptions", we will search for the distribution that is as close as possible to our prior, in the sense of KL divergence (Sec. 6.2), while satisfying our constraints. If our prior is uniform, minimizing the KL is equivalent to maximizing the entropy.

To minimize KL subject to the constraints in Eq. (12.78), and the constraint that $p(\mathbf{x}) \geq 0$ and $\sum_{\mathbf{x}} p(\mathbf{x}) = 1$, we need to use Lagrange multipliers (see Sec. 8.5.1). The Lagrangian is given by

$$J(p, \boldsymbol{\lambda}) = -\sum_{\mathbf{x}} p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x})} + \lambda_0 \left(1 - \sum_{\mathbf{x}} p(\mathbf{x})\right) + \sum_k \lambda_k \left(F_k - \sum_{\mathbf{x}} p(\mathbf{x}) f_k(\mathbf{x})\right) \tag{12.79}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can use the calculus of variations to take derivatives wrt the function p, but we will adopt a simpler approach and treat $\mathbf{p}$ as a fixed length vector (since we are assuming that $\mathbf{x}$ is discrete). Then we have

$$\frac{\partial J}{\partial p_c} = -1 - \log \frac{p(x=c)}{q(x=c)} - \lambda_0 - \sum_k \lambda_k f_k(x=c) \tag{12.80}$$

Setting $\frac{\partial J}{\partial p_c} = 0$ for each c yields

$$p(\mathbf{x}) = \frac{q(\mathbf{x})}{Z} \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \tag{12.81}$$

where we have defined $Z \triangleq e^{1+\lambda_0}$. Using the sum-to-one constraint, we have

$$1 = \sum_{\mathbf{x}} p(\mathbf{x}) = \frac{1}{Z} \sum_{\mathbf{x}} q(\mathbf{x}) \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \tag{12.82}$$

Hence the normalization constant is given by

$$Z = \sum_{\mathbf{x}} q(\mathbf{x}) \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \tag{12.83}$$

This has exactly the form of the exponential family, where $\mathbf{f}(\mathbf{x})$ is the vector of sufficient statistics, $-\boldsymbol{\lambda}$ are the natural parameters, and $q(\mathbf{x})$ is our base measure.

If we use a uniform prior, $q(\mathbf{x}) \propto 1$, minimizing the KL divergence is equivalent to maximizing the entropy (see Sec. 6.1 for details on entropy). The result is called a **maximum entropy model**.

For example, if the features are $f_1(x) = x$ and $f_2(x) = x^2$, and we want to match the first and second moments, we get the Gaussian disribution.

12.3 Generalized linear models (GLMs)

A **generalized linear model** or **GLM** is a conditional version of an exponential dispersion family distribution, in which the natural parameters are a linear function of the input. More precisely, the model has the following form:

$$p(y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = \exp\left[\frac{y_n \eta_n - A(\eta_n)}{\sigma^2} + \log h(y_n, \sigma^2)\right] \tag{12.84}$$

where $\eta_n \triangleq \mathbf{w}^\top \mathbf{x}_n$ is the (input dependent) natural parameter, $A(\eta_n)$ is the log normalizer, $\mathbf{t}(y) = y$ is the sufficient statistic, and σ^2 is the dispersion term. Based on the results in Sec. 12.2.3, we can show that the mean and variance of the response variable are as follows:

$$\mu_n \triangleq \mathbb{E}\left[y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2\right] = A'(\eta_n) \triangleq g^{-1}\eta_n) \tag{12.85}$$

$$\mathbb{V}\left[y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2\right] = A''(\eta_n)\sigma^2 \tag{12.86}$$

We will denote the mapping from the linear inputs to the mean of the output using $\mu_n = g^{-1}(\eta_n)$, where the function g is known as the **link function**, and g^{-1} is known as the **mean function**.

12.3.1 Examples

In this section, we give some examples of widely used GLMs.

12.3.1.1 Linear regression

Recall that linear regression has the form

$$p(y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{1}{2\sigma^2}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2) \tag{12.87}$$

Hence

$$\log p(y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = -\frac{1}{2\sigma^2}(y_n - \eta_n)^2 - \frac{1}{2}\log(2\pi\sigma^2) \tag{12.88}$$

where $\eta_n = \mathbf{w}^\top \mathbf{x}_n$. We can write this in GLM form as follows:

$$\log p(y_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = \frac{y_n\eta_n - \frac{\eta_n^2}{2}}{\sigma^2} - \frac{1}{2}\left(\frac{y_n^2}{\sigma^2} + \log(2\pi\sigma^2)\right) \tag{12.89}$$

We see that $A(\eta_n) = \eta_n^2/2$ and hence

$$\mathbb{E}[y_n] = \eta_n = \mathbf{w}^\top \mathbf{x}_n \tag{12.90}$$

$$\mathbb{V}[y_n] = \sigma^2 \tag{12.91}$$

12.3.1.2 Binomial regression

If the response variable is the number of successes in N_n trials, $y_n \in \{0, \ldots, N_n\}$, we can use **binomial regression**, which is defined by

$$p(y_n|\mathbf{x}_n, N_n, \mathbf{w}) = \text{Bin}(y_n|\sigma(\mathbf{w}^\top \mathbf{x}_n), N_n) \tag{12.92}$$

We see that binary logistic regression is the special case when $N_n = 1$.

The log pdf is given by

$$\log p(y_n|\mathbf{x}_n, N_n, \mathbf{w}) = y_n \log \mu_n + (N_n - y_n)\log(1 - \mu_n) + \log\binom{N_n}{y_n} \tag{12.93}$$

$$= y_n \log(\frac{\mu_n}{1 - \mu_n}) + N_n \log(1 - \mu_n) + \log\binom{N_n}{y_n} \tag{12.94}$$

where $\mu_n = \sigma(\eta_n)$. To rewrite this in GLM form, let us define

$$\eta_n \triangleq \log\left[\frac{\mu_n}{(1 - \mu_n)}\right] = \log\left[\frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}_n}} \frac{1 + e^{-\mathbf{w}^\top \mathbf{x}_n}}{e^{-\mathbf{w}^\top \mathbf{x}_n}}\right] = \log \frac{1}{e^{-\mathbf{w}^\top \mathbf{x}_n}} = \mathbf{w}^\top \mathbf{x}_n \tag{12.95}$$

Hence we can write binomial regression in GLM form as follows

$$\log p(y_n|\mathbf{x}_n, N_n, \mathbf{w}) = y_n\eta_n - A(\eta_n) + h(y_n) \tag{12.96}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $h(y_n) = \log \binom{N_n}{y_n}$ and

$$A(\eta_n) = -N_n \log(1-\mu_n) = N_n \log(1+e^{\eta_n}) \tag{12.97}$$

Hence

$$\mathbb{E}[y_n] = \frac{dA}{d\eta_n} = \frac{N_n e^{\eta_n}}{1+e^{\eta_n}} = \frac{N_n}{1+e^{-\eta_n}} = N_n \mu_n \tag{12.98}$$

and

$$\mathbb{V}[y_n] = \frac{d^2 A}{d\eta_n^2} = N_n \mu_n (1-\mu_n) \tag{12.99}$$

12.3.1.3 Poisson regression

If the response variable is an integer count, $y_n \in \{0, 1, \ldots\}$, we can use **Poisson regression**, which is defined by

$$p(y_n|\mathbf{x}_n, \mathbf{w}) = \text{Poi}(y_n|\exp(\mathbf{w}^\top \mathbf{x}_n)) \tag{12.100}$$

where

$$\text{Poi}(y|\mu) = e^{-\mu} \frac{\mu^y}{y!} \tag{12.101}$$

is the Poisson distribution. Poisson regression is widely used in bio-statistical applications, where y_n might represent the number of diseases of a given person or place, or the number of reads at a genomic location in a high-throughput sequencing context (see e.g., [Kua+09]).

The log pdf is given by

$$\log p(y_n|\mathbf{x}_n, \mathbf{w}) = y_n \log \mu_n - \mu_n - \log(y_n!) \tag{12.102}$$

where $\mu_n = \exp(\mathbf{w}^\top \mathbf{x}_n)$. Hence in GLM form we have

$$\log p(y_n|\mathbf{x}_n, \mathbf{w}) = y_n \eta_n - A(\eta_n) + h(y_n) \tag{12.103}$$

where $\eta_n = \log(\mu_n) = \mathbf{w}^\top \mathbf{x}_n$, $A(\eta_n) = \mu_n = e^{\eta_n}$, and $h(y_n) = -\log(y_n!)$. Hence

$$\mathbb{E}[y_n] = \frac{dA}{d\eta_n} = e^{\eta_n} = \mu_n \tag{12.104}$$

and

$$\mathbb{V}[y_n] = \frac{d^2 A}{d\eta_n^2} = e^{\eta_n} = \mu_n \tag{12.105}$$

12.3.2 Maximum likelihood estimation

GLMs can be fit using similar methods to those that we used to fit logistic regression. In particular, the negative log-likelihood has the following form (ignoring constant terms):

$$\text{NLL}(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) = -\frac{1}{\sigma^2}\sum_{n=1}^{N}\ell_n \tag{12.106}$$

where

$$\ell_n \triangleq \eta_n y_n - A(\eta_n) \tag{12.107}$$

where $\eta_n = \mathbf{w}^\top \mathbf{x}_n$. For notational simplicity, we will assume $\sigma^2 = 1$.

We can compute the gradient for a single term as follows:

$$\mathbf{g}_n \triangleq \frac{\partial \ell_n}{\partial \mathbf{w}} = \frac{\partial \ell_n}{\partial \eta_n}\frac{\partial \eta_n}{\partial \mathbf{w}} = (y_n - A'(\eta_n))\mathbf{x}_n = (y_n - \mu_n)\mathbf{x}_n \tag{12.108}$$

where $\mu_n = f(\mathbf{w}^\top \mathbf{x})$, and f is the inverse link function that maps from canonical parameters to mean parameters. For example, in the case of logistic regression, $f(\eta_n) = \sigma(\eta_n)$, so we recover Eq. (10.22). This gradient expression can be used inside SGD, or some other gradient method, in the obvious way.

The Hessian is given by

$$\mathbf{H} = \frac{\partial^2}{\partial \mathbf{w}\partial \mathbf{w}^\top}\text{NLL}(\mathbf{w}) = -\sum_{n=1}^{N}\frac{\partial \mathbf{g}_n}{\partial \mathbf{w}^\top} \tag{12.109}$$

where

$$\frac{\partial \mathbf{g}_n}{\partial \mathbf{w}^\top} = \frac{\partial \mathbf{g}_n}{\partial \mu_n}\frac{\partial \mu_n}{\partial \mathbf{w}^\top} = -\mathbf{x}_n f'(\mathbf{w}^\top \mathbf{x}_n)\mathbf{x}_n^\top \tag{12.110}$$

Hence

$$\mathbf{H} = \sum_{n=1}^{N} f'(\eta_n)\mathbf{x}_n \mathbf{x}_n^\top \tag{12.111}$$

For example, in the case of logistic regression, $f(\eta_n) = \sigma(\eta_n)$, and $f'(\eta_n) = \sigma(\eta_n)(1 - \sigma(\eta_n))$, so we recover Eq. (10.23). In general, we see that the Hessian is positive definite, since $f'(\eta_n) > 0$; hence the negative log likelihood is convex, so the MLE for a GLM is unique (assuming $f(\eta_n) > 0$ for all n).

12.3.3 GLMs with non-canonical link functions

We have see how the mean parameters of the output distribution are given by $\mu = g^{-1}(\eta)$, where the function g is the link function. There are several choices for this function, as we now discuss.

The **canonical link function** g satisfies the property that $\theta = g(\mu)$, where θ are the canonical (natural) parameters. Hence

$$\theta = g(\mu) = g(g^{-1}(\eta)) = \eta \tag{12.112}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is what we have assumed so far. For example, for the Bernoulli distribution, the canonical parameter is the log-odds $\theta = \log(\mu/(1-\mu))$, which is given by the logit transform

$$\theta = g(\mu) = \text{logit}(\mu) = \log\left(\frac{\mu}{1-\mu}\right) \tag{12.113}$$

The inverse of this is the sigmoid or logistic funciton $\mu = \sigma(\theta) = 1/(1+e^{-\theta})$.

However, we are free to use other kinds of link function. For example, in Sec. 12.4 we use

$$\eta = g(\mu) = \Phi^{-1}(\mu) \tag{12.114}$$

This is known as the **probit link function**.

Another link function that is sometimes used for binary responses is the **complementary log-log** function

$$\eta = g(\mu) = \log(-\log(1-\mu)) \tag{12.115}$$

This is used in applications where we either observe 0 events (denoted by $y = 0$) or one or more (denoted by $y = 1$), where events are assumed to be governed by a Poisson distribution with rate λ. Let E be the number of events. The Poisson assumption means $p(E = 0) = \exp(-\lambda)$ and hence

$$p(y = 0) = (1-\mu) = p(E = 0) = \exp(-\lambda) \tag{12.116}$$

Thus $\lambda = -\log(1-\mu)$. When λ is a function of covariates, we need to ensure it is positive, so we use $\lambda = e^{\eta}$, and hence

$$\eta = \log(\lambda) = \log(-\log(1-\mu)) \tag{12.117}$$

12.4 Probit regression

In this section, we discuss **probit regression**, which is similiar to binary logistic regression except it uses $\mu_n = \Phi(a_n)$ instead of $\mu_n = \sigma(a_n)$ as the mean function, where Φ is the cdf of the standard normal, and $a_n = \mathbf{w}^\top \mathbf{x}_n$. The corresponding link function is therefore $a_n = g(\mu_n) = \Phi^{-1}(\mu_n)$; the inverse of the Gaussian cdf is known as the **probit function**.

The Gaussian cdf Φ is very similar to the logistic function, as shown in Fig. 12.1. Thus probit regression and "regular" logistic regression behave very similarly. However, probit regression has some advantages. In particular, it has a simple interpretation as a latent variable model (see Sec. 12.4.1), which arises from the field of **choice theory** as studied in economics (see e.g., [Koo03]). This also simplifies the task of Bayesian parameter inference.

12.4.1 Latent variable interpretation

We can interpret $a_n = \mathbf{w}^\top \mathbf{x}_n$ as a factor that is proportional to how likely a person is respond positively (generate $y_n = 1$) given input $\mathbf{x}_n$. However, typically there are other unobserved factors that influence someone's response. Let us model these hidden factors by Gaussian noise, $\epsilon_n \sim \mathcal{N}(0, 1)$. Let the combined preference for positive outcomes be represented by the latent variable $z_n = \mathbf{w}^\top \mathbf{x}_n + \epsilon_n$.

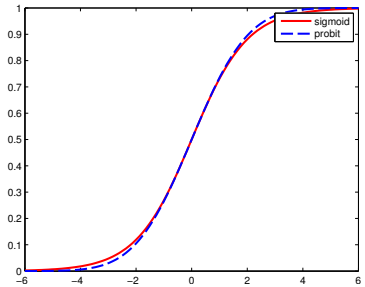


Figure 12.1: The logistic (sigmoid) function $\sigma(x)$ in solid red, with the Gaussian cdf function $\Phi(\lambda x)$ in dotted blue superimposed. Here $\lambda = \sqrt{\pi/8}$, which was chosen so that the derivatives of the two curves match at $x = 0$. Adapted from Figure 4.9 of [Bis06]. Generated by code at figures.probml.ai/book1/12.1.

We assume that the person will pick the positive label iff this latent factor is positive rather than negative, i.e.,

$$y_n = \mathbb{I}(z_n \geq 0) \tag{12.118}$$

When we marginalize out z_n, we recover the probit model:

$$p(y_n = 1|\mathbf{x}_n, \mathbf{w}) = \int \mathbb{I}(z_n \geq 0)\mathcal{N}(z_n|\mathbf{w}^\top \mathbf{x}_n, 1)dz_n \tag{12.119}$$

$$= p(\mathbf{w}^\top \mathbf{x}_n + \epsilon_n \geq 0) = p(\epsilon_n \geq -\mathbf{w}^\top \mathbf{x}_n) \tag{12.120}$$

$$= 1 - \Phi(-\mathbf{w}^\top \mathbf{x}_n) = \Phi(\mathbf{w}^\top \mathbf{x}_n) \tag{12.121}$$

Thus we can think of probit regression as a threshold function applied to noisy input.

We can interpret logistic regression in the same way. However, in that case the noise term ϵ_n comes from a **logistic distribution**, defined as follows:

$$f(y|\mu, s) \triangleq \frac{e^{-\frac{y-\mu}{s}}}{s(1 + e^{-\frac{y-\mu}{s}})^2} \tag{12.122}$$

The cdf of this distribution is given by

$$F(y|\mu, s) = \frac{1}{1 + e^{-\frac{y-\mu}{s}}} \tag{12.123}$$

It is clear that if we use logistic noise with $\mu = 0$ and $s = 1$ we recover logistic regression. However, it is computationally easier to deal with Gaussian noise, as we show below.

12.4.2 Maximum likelihood estimation

In this section, we discuss some methods for fitting probit regression using MLE.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

12.4.2.1 MLE using SGD

We can find the MLE for probit regression using standard gradient methods. Let $\mu_n = \mathbf{w}^\top \mathbf{x}_n$, and let $\tilde{y}_n \in \{-1, +1\}$. Then the gradient of the log-likelihood for a single example n is given by

$$\mathbf{g}_n \triangleq \frac{d}{d\mathbf{w}} \log p(\tilde{y}_n|\mathbf{w}^\top \mathbf{x}_n) = \frac{d\mu_n}{d\mathbf{w}} \frac{d}{d\mu_n} \log p(\tilde{y}_n|\mathbf{w}^\top \mathbf{x}_n) = \mathbf{x}_n \frac{\tilde{y}_n \phi(\mu_n)}{\Phi(\tilde{y}_n \mu_n)} \tag{12.124}$$

where ϕ is the standard normal pdf, and Φ is its cdf. Similarly, the Hessian for a single case is given by

$$\mathbf{H}_n = \frac{d}{d\mathbf{w}^2} \log p(\tilde{y}_n|\mathbf{w}^\top \mathbf{x}_n) = -\mathbf{x}_n \left(\frac{\phi(\mu_n)^2}{\Phi(\tilde{y}_n \mu_n)^2} + \frac{\tilde{y}_n \mu_n \phi(\mu_n)}{\Phi(\tilde{y}_n \mu_n)} \right) \mathbf{x}_n^\top \tag{12.125}$$

This can be passed to any gradient-based optimizer.

12.4.2.2 MLE using EM

We can use the latent variable interpretation of probit regression to derive an elegant EM algorithm for fitting the model. The complete data log likelihood has the following form, assuming a $\mathcal{N}(\mathbf{0}, \mathbf{V}_0)$ prior on $\mathbf{w}$:

$$\ell(\mathbf{z}, \mathbf{w}|\mathbf{V}_0) = \log p(\mathbf{y}|\mathbf{z}) + \log \mathcal{N}(\mathbf{z}|\mathbf{X}\mathbf{w}, \mathbf{I}) + \log \mathcal{N}(\mathbf{w}|\mathbf{0}, \mathbf{V}_0) \tag{12.126}$$

$$= \sum_n \log p(y_n|z_n) - \frac{1}{2}(\mathbf{z} - \mathbf{X}\mathbf{w})^\top (\mathbf{z} - \mathbf{X}\mathbf{w}) - \frac{1}{2}\mathbf{w}^\top \mathbf{V}_0^{-1} \mathbf{w} \tag{12.127}$$

The posterior in the E step is a **truncated Gaussian**:

$$p(z_n|y_n, \mathbf{x}_n, \mathbf{w}) = \begin{cases} \mathcal{N}(z_n|\mathbf{w}^\top \mathbf{x}_n, 1)\mathbb{I}(z_n > 0) & \text{if } y_n = 1 \\ \mathcal{N}(z_n|\mathbf{w}^\top \mathbf{x}_n, 1)\mathbb{I}(z_n < 0) & \text{if } y_n = 0 \end{cases} \tag{12.128}$$

In Eq. (12.127), we see that $\mathbf{w}$ only depends linearly on $\mathbf{z}$, so we just need to compute $\mathbb{E}[z_n|y_n, \mathbf{x}_n, \mathbf{w}]$, so we just need to compute the posterior mean. One can show that this is given by

$$\mathbb{E}[z_n|\mathbf{w}, \mathbf{x}_n] = \begin{cases} \mu_n + \frac{\phi(\mu_n)}{1-\Phi(-\mu_n)} = \mu_n + \frac{\phi(\mu_n)}{\Phi(\mu_n)} & \text{if } y_n = 1 \\ \mu_n - \frac{\phi(\mu_n)}{\Phi(-\mu_n)} = \mu_n - \frac{\phi(\mu_n)}{1-\Phi(\mu_i)} & \text{if } y_n = 0 \end{cases} \tag{12.129}$$

where $\mu_n = \mathbf{w}^\top \mathbf{x}_n$.

In the M step, we estimate $\mathbf{w}$ using ridge regression, where $\boldsymbol{\mu} = \mathbb{E}[\mathbf{z}]$ is the output we are trying to predict. Specifically, we have

$$\hat{\mathbf{w}} = (\mathbf{V}_0^{-1} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\mu} \tag{12.130}$$

The EM algorithm is simple, but can be much slower than direct gradient methods, as illustrated in Fig. 12.2. This is because the posterior entropy in the E step is quite high, since we only observe that z is positive or negative, but are given no information from the likelihood about its magnitude. Using a stronger regularizer can help speed convergence, because it constrains the range of plausible z values. In addition, one can use various speedup tricks, such as data augmentation [DM01].

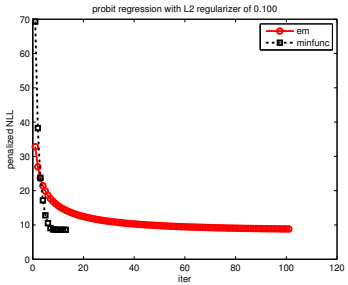


Figure 12.2: Fitting a probit regression model in 2d using a quasi-Newton method or EM. Generated by code at figures.probml.ai/book1/12.2.

12.4.3 Ordinal probit regression *

One advantage of the latent variable interpretation of probit regression is that it is easy to extend to the case where the response variable is ordered in some way, such as the outputs low, medium and high. This is called **ordinal regression**. The basic idea is as follows. If there are C output values, we introduce $C+1$ thresholds γ_j and set

$$y_n = j \quad \text{if} \quad \gamma_{j-1} < z_n \leq \gamma_j \tag{12.131}$$

where $\gamma_0 \leq \cdots \leq \gamma_C$. For identifiability reasons, we set $\gamma_0 = -\infty$, $\gamma_1 = 0$ and $\gamma_C = \infty$. For example, if $C = 2$, this reduces to the standard binary probit model, whereby $z_n < 0$ produces $y_n = 0$ and $z_n \geq 0$ produces $y_n = 1$. If $C = 3$, we partition the real line into 3 intervals: $(-\infty, 0]$, $(0, \gamma_2]$, (γ_2, ∞). We can vary the parameter γ_2 to ensure the right relative amount of probability mass falls in each interval, so as to match the empirical frequencies of each class label. See e.g., [AC93] for further details.

Finding the MLEs for this model is a bit trickier than for binary probit regression, since we need to optimize for $\mathbf{w}$ and $\boldsymbol{\gamma}$, and the latter must obey an ordering constraint. See e.g., [KL09] for an approach based on EM. It is also possible to derive a simple Gibbs sampling algorithm for this model (see e.g., [Hof09, p216]).

12.4.4 Multinomial probit models *

Now consider the case where the response variable can take on C unordered categorical values, $y_n \in \{1, \ldots, C\}$. The **multinomial probit** model is defined as follows:

$$z_{nc} = \mathbf{w}_c^\top \mathbf{x}_{nc} + \epsilon_{nc} \tag{12.132}$$

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \tag{12.133}$$

$$y_n = \arg\max_c z_{nc} \tag{12.134}$$

See e.g., [DE04; GR06; Sco09; FSF10] for more details on the model and its connection to multinomial logistic regression.

If instead of setting $y_i = \text{argmax}_c\, z_{ic}$ we use $y_{ic} = \mathbb{I}(z_{ic} > 0)$, we get a model known as **multivariate probit**, which is one way to model C correlated binary outcomes (see e.g., [TMD12]).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Part III

Deep neural networks

13 Neural networks for unstructured data

13.1 Introduction

In Part II, we discussed linear models for regression and classification. In particular, in Chapter 10, we discussed logistic regression, which, in the binary case, corresponds to the model $p(y|\mathbf{x}, \mathbf{w}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x}))$, and in the multiclass case corresponds to the model $p(y|\mathbf{x}, \mathbf{w}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{x}))$. In Chapter 11, we discussed linear regression, which corresponds to the model $p(y|\mathbf{x}, \mathbf{w}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2)$. And in Chapter 12, we discussed generalized linear models, which have the form

$$p(\mathbf{y}|\mathbf{x}; \boldsymbol{\theta}) = p(\mathbf{y}|g^{-1}(f(\mathbf{x}; \boldsymbol{\theta})) \tag{13.1}$$

where $p(\mathbf{y}|\boldsymbol{\mu})$ is an exponential family distribution with mean parameters $\boldsymbol{\mu}$, $g^{-1}()$ is the inverse link function corresponding to that distribution, and

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{13.2}$$

is a linear (affine) transformation of the input, where $\mathbf{W}$ are called the **weights** and $\mathbf{b}$ are called the **biases**.

The assumption of linearity is very restrictive. A simple way of increasing the flexibility of such models is to perform a feature transformation, by replacing $\mathbf{x}$ with $\boldsymbol{\phi}(\mathbf{x})$. For example, we can use a polynomial transform, which in 1d is given by $\boldsymbol{\phi}(x) = [1, x, x^2, x^3, \ldots]$, as we discussed in Sec. 1.2.2.2. This is sometimes called **basis function expansion**. The model now becomes

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}\boldsymbol{\phi}(\mathbf{x}) + \mathbf{b} \tag{13.3}$$

This is still linear in the parameters $\boldsymbol{\theta} = (\mathbf{W}, \mathbf{b})$, which makes model fitting easy. However, having to specify the feature transformation by hand is very limiting.

A natural extension is to endow the feature extractor with its own parameters, $\boldsymbol{\theta}'$, to get

$$f(\mathbf{x}; \boldsymbol{\theta}, \boldsymbol{\theta}') = \mathbf{W}\boldsymbol{\phi}(\mathbf{x}; \boldsymbol{\theta}') + \mathbf{b} \tag{13.4}$$

We can obviously repeat this process recursively, to create more and more complex functions. If we compose L functions, we get

$$f(\mathbf{x}; \boldsymbol{\theta}) = f_L(f_{L-1}(\cdots(f_1(\mathbf{x}))\cdots)) \tag{13.5}$$

where $f_\ell(\mathbf{x}) = f(\mathbf{x}; \boldsymbol{\theta}_\ell)$ is the function at layer ℓ. This is the key idea behind **deep neural networks** or **DNN**s.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Table 13.1: Truth table for the XOR (exclusive OR) function, $y = x_1 \veebar x_2$.

The term "DNN" actually encompases a larger family of models, in which we compose differentiable functions into any kind of DAG (directed acyclic graph), mapping input to output. Eq. (13.5) is the simplest example where the DAG is a chain. This is known as a **feedforward neural network** (**FFNN**) or **multilayer perceptron** (**MLP**).

An MLP assumes that the input is a fixed-dimensional vector, say $\mathbf{x} \in \mathbb{R}^D$. We call such data "**unstructured data**", since we do not make any assumptions about the form of the input. However, this makes the model difficult to apply to inputs that have variable size or shape. In Chapter 14, we discuss **convolutional neural networks** (**CNN**), which are designed to work with variable-sized images; in Chapter 15, we discuss **recurrent neural networks** (**RNN**), which are designed to work with variable-sized sequences; and in Chapter 23, we discuss **graph neural networks** (**GNN**), which are designed to work with variable-sized graphs. For even more information on DNNs, see various other books, such as [Zha+20; Cho21; Gér19; GBC16]. Some practical details can be found in the supplementary material to this book, available at `probml.ai`. There are also many online courses with even more information, such as `d2l.ai` (Dive Into Deep Learning), `fast.ai`, `fullstackdeeplearning.com`, etc.

13.2 Multilayer perceptrons (MLPs)

In Sec. 10.2.5, we explained that a **perceptron** is a deterministic version of logistic regression. Specifically, it is a mapping of the following form:

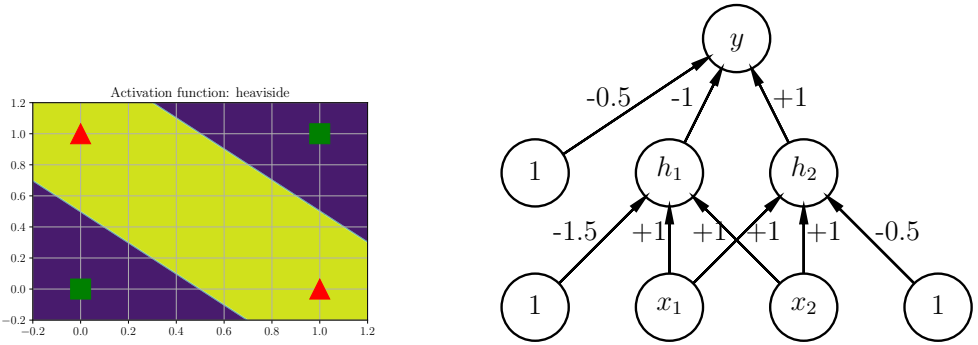
$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbb{I}\left(\mathbf{w}^\top \mathbf{x} + b \geq 0\right) = H(\mathbf{w}^\top \mathbf{x} + b) \tag{13.6}$$

where $H(a)$ is the **heaviside step function**, also known as a **linear threshold function**. Since the decision boundaries represented by perceptrons are linear, they are very limited in what they can represent. In 1969, Marvin Minsky and Seymour Papert published a famour book called "Perceptrons" [MP69] in which they gave numerous examples of pattern recognition problems which perceptrons cannot solve. We give a specific example below, before discussing how to solve the problem.

13.2.1 The XOR problem

One of the most famous examples from the "Perceptrons" book is the **XOR problem**. Here the goal is to learn a function that computes the exclusive OR of its two binary inputs. The truth table for this function is given in Table 13.1. We visualize this function in Fig. 13.1a. It is clear that the data is not linearly separable, so a perceptron cannot represent this mapping.

However, we can overcome this problem by stacking multiple perceptrons on top of each other. This is called a **multilayer perceptron** (**MLP**). For example, to solve the XOR problem, we can



(a) (b)

Figure 13.1: (a) Illustration of the fact that the XOR function is not linearly separable, but can be separated by the two layer model using Heaviside activation functions. Adapted from Figure 10.6 of [Gér19]. Generated by code at figures.probml.ai/book1/13.1. (b) A neural net with one hidden layer, whose weights have been manually constructed to implement the XOR function. h_1 is the AND function and h_2 is the OR function. The bias terms are implemented using weights from constant nodes with the value 1.

use the MLP shown in Fig. 13.1b. This consists of 3 perceptrons, denoted h_1, h_2 and y. The nodes marked x are inputs, and the nodes marked 1 are constant terms. The nodes h_1 and h_2 are called **hidden units**, since their values are not observed in the training data.

The first hidden unit computes $h_1 = x_1 \wedge x_2$ by using appropriately set weights. (Here $\wedge$ is the AND operation.) In particular, it has inputs from x_1 and x_2, both weighted by 1.0, but has a bias term of -1.5 (this is implemented by a "wire" with weight -1.5 coming from a dummy node whose value is fixed to 1). Thus h_1 will fire iff x_1 and x_2 are both on, since then

$$\mathbf{w}_1^\top \mathbf{x} - b_1 = [1.0, 1.0]^\top [1, 1] - 1.5 = 0.5 > 0 \tag{13.7}$$

Similarly, the second hidden unit computes $h_2 = x_1 \vee x_2$, where $\vee$ is the OR operation, and the third computes the output $y = \overline{h_1} \wedge h_2$, where $\overline{h} = \neg h$ is the NOT (logical negation) operation. Thus y computes

$$y = f(x_1, x_2) = \overline{(x_1 \wedge x_2)} \wedge (x_1 \vee x_2) \tag{13.8}$$

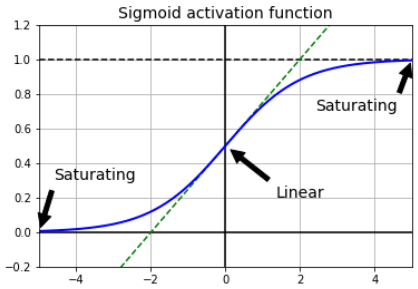
This is equivalent to the XOR function.

By generalizing this example, we can show that an MLP can represent any logical function. However, we obviously want to avoid having to specify the weights and biases by hand. In the rest of this chapter, we discuss ways to learn these parameters from data.

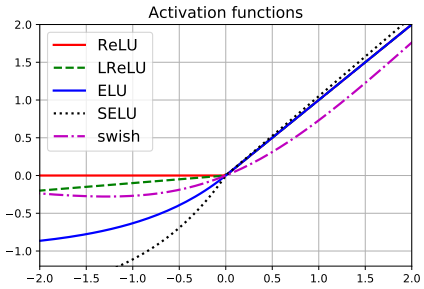
13.2.2 Differentiable MLPs

The MLP we discussed in Sec. 13.2.1 was defined as a stack of perceptrons, each of which involved the non-differentiable Heaviside function. This makes such models very difficult to train, which is why

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a)



(b)

Figure 13.2: (a) Illustration of how the sigmoid function is linear for inputs near 0, but saturates for large positive and negative inputs. Adapted from 11.1 of [Gér19]. (b) Plots of some popular non-saturating activation functions. Generated by code at figures.probml.ai/book1/13.2.

they were never widely used. However, suppose we replace the Heaviside function $H : \mathbb{R} \to \{0, 1\}$ with a differentiable **activation function** $\varphi : \mathbb{R} \to \mathbb{R}$. More precisely, we define the hidden units $\mathbf{z}_l$ at each layer l to be a linear transformation of the hidden units at the previous layer passed elementwise through this activation function:

$$\mathbf{z}_l = f_l(\mathbf{z}_{l-1}) = \varphi_l\left(\mathbf{b}_l + \mathbf{W}_l \mathbf{z}_{l-1}\right) \tag{13.9}$$

or, in scalar form,

$$z_{kl} = \varphi_l\left(b_{kl} + \sum_{j=1}^{K_{l-1}} w_{jkl} z_{jl-1}\right) \tag{13.10}$$

The quantity that is passed to the activation function is called the **pre-activations**:

$$\mathbf{a}_l = \mathbf{b}_l + \mathbf{W}_l \mathbf{z}_{l-1} \tag{13.11}$$

so $\mathbf{z}_l = \varphi_l(\mathbf{a}_l)$.

If we now compose L of these functions together, as in Eq. (13.5), then we can compute the gradient of the output wrt the parameters in each layer using the chain rule, also known as **backpropagation**, as we explain in Sec. 13.3. (This is true for any kind of differentiable activation function, although some kinds work better than others, as we discuss in Sec. 13.2.3.) We can then pass the gradient to an optimizer, and thus minimize some training objective, as we discuss in Sec. 13.4. For this reason, the term "MLP" almost always refers to this differentiable form of the model, rather than the historical version with non-differentiable linear threshold units.

13.2.3 Activation functions

We are free to use any kind of differentiable activation function we like at each layer. However, if we use a *linear* activation function, $\varphi_\ell(a) = c_\ell a$, then the whole model reduces to a regular linear model.

To see this, note that Eq. (13.5) becomes

$$f(\mathbf{x};\boldsymbol{\theta}) = \mathbf{W}_L c_L(\mathbf{W}_{L-1} c_{L-1}(\cdots(\mathbf{W}_1\mathbf{x})\cdots)) \propto \mathbf{W}_L\mathbf{W}_{L-1}\cdots\mathbf{W}_1\mathbf{x} = \mathbf{W}'\mathbf{x} \tag{13.12}$$

where we dropped the bias terms for notational simplicity. For this reason, it is important to use nonlinear activation functions.

In the early days of neural networks, a common choice was to use a sigmoid (logistic) function, which can be seen as a smooth approximation to the Heaviside function used in a perceptron. However, as shown in Fig. 13.2a, the sigmoid function **saturates** at 1 for large positive inputs, and at 0 for large negative inputs. (The tanh function has a similar shape, but saturates at -1 and +1.) In these regimes, the gradient of the output wrt the input will be close to zero, so any gradient signal from higher layers will not be able to propagate back to earlier layers. This is called the **vanishing gradient problem**, and it makes it hard to train the model using gradient descent (see Sec. 13.4.2 for details).

One of the keys to being able to train very deep models is to use **non-saturating activation functions**. Several different functions have been proposed (see Fig. 13.2b for a plot, and Sec. 13.4.3 for details). The most common is **rectified linear unit** or **ReLU**, proposed in [GBB11; KSH12]. This is defined as

$$\text{ReLU}(a) = \max(a, 0) = a\mathbb{I}(a > 0) \tag{13.13}$$

The ReLU function simply "turns off" negative inputs, and passes positive inputs unchanged.

13.2.4 Example models

MLPs can be used to perform classification and regression for many kinds of data. We give some examples below.

13.2.4.1 MLP for classifying 2d data into 2 categories

Figure 13.3 gives an illustration of an MLP with two hidden layers applied to a 2d input vector, correspondonding to points in the plane, coming from two concentric circles. This model has the following form:

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \text{Ber}(y|\sigma(a_3)) \tag{13.14}$$

$$a_3 = \mathbf{w}_3^\top \mathbf{z}_2 + b_3 \tag{13.15}$$

$$\mathbf{z}_2 = \varphi(\mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2) \tag{13.16}$$

$$\mathbf{z}_1 = \varphi(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \tag{13.17}$$

Here a_3 is the final logit score, which is converted to a probability via the sigmoid (logistic) function. The value a_3 is computed by taking a linear combination of the 2 hidden units in layer 2, using $a_3 = \mathbf{w}_3^\top \mathbf{z}_2 + b_3$. In turn, layer 2 is computed by taking a nonlinear combination of the 4 hidden units in layer 1, using $\mathbf{z}_2 = \varphi(\mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2)$. Finally, layer 1 is computed by taking a nonlinear combination of the 2 input units, using $\mathbf{z}_1 = \varphi(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$. By adjusting the parameters, $\boldsymbol{\theta} = (\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{w}_3, b_3)$, to minimize the negative log likelihood, we can fit the training data very well, despite the highly nonlinear nature of the decision boundary. (You can find an interactive version of this figure at http://playground.tensorflow.org.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

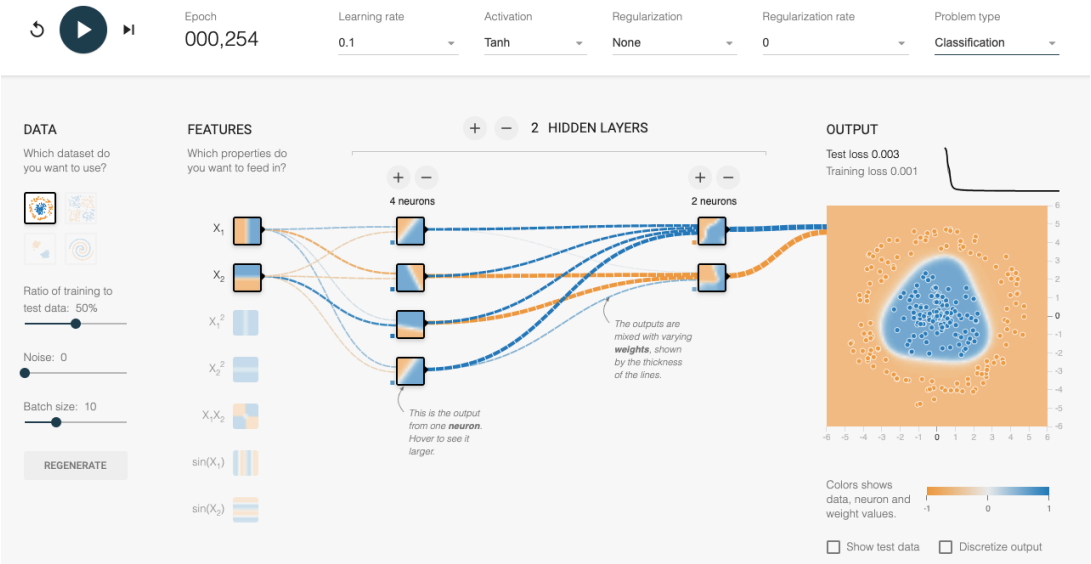


Figure 13.3: An MLP with 2 hidden layers applied to a set of 2d points from 2 classes, shown in the top left corner. The visualizations associated with each hidden unit show the decision boundary at that part of the network. The final output is shown on the right. The input is $\mathbf{x} \in \mathbb{R}^2$, the first layer activations are $\mathbf{z}_1 \in \mathbb{R}^4$, the second layer activations are $\mathbf{z}_2 \in \mathbb{R}^2$, and the final logit is $a_3 \in \mathbb{R}$, which is converted to a probability using the sigmoid function. This is a screenshot from the interactive demo at `http://playground.tensorflow.org`.

13.2.4.2 MLP for image classification

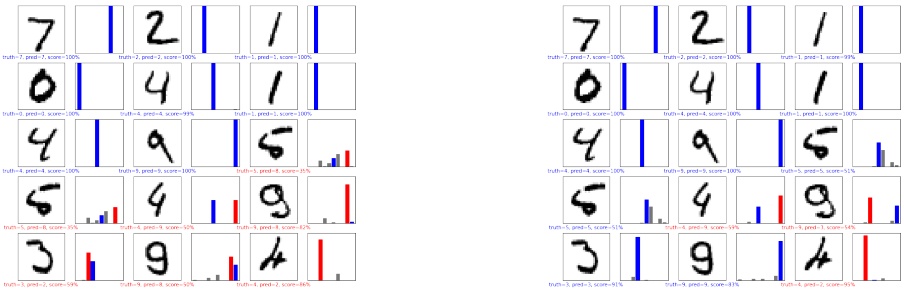
To apply an MLP to image classification, we need to "**flatten**" the 2d input into 1d vector. We can then use a feedforward architecture similar to the one described in Sec. 13.2.4.1. For example, consider building an MLP to classifiy MNIST digits (Sec. 3.4.2). These are $28 \times 28 = 784$-dimensional. If we use 2 hidden layers with 128 units each, followed by a final 10 way softmax layer, we get the model shown in Fig. 13.4.

We show some predictions from this model in Fig. 13.5. We train it for just two "**epochs**" (passes over the dataset), but already the model is doing quite well, with a test set accuracy of 97.1%. Furthermore, the errors seem sensible, e.g., 9 is mistaken as a 3. Training for more epochs can further improve test accuracy.

In Chapter 14 we discuss a different kind of model, called a convolutional neural network, which is better suited to images. This gets even better performance and uses fewer parameters, by exploiting prior knowledge about the spatial structure of images. By contrast, the MLP is invariant to a permutation of its inputs. Put another way, we could randomly shuffle the pixels and we would get the same result (assuming we use the same random permutation for all inputs).

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 128)               100480
_________________________________________________________________
dense_1 (Dense)              (None, 128)               16512
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0
```

Figure 13.4: Structure of the MLP used for MNIST classification. Note that $100,480 = (784+1) \times 128$*, and* $16,512 = (128+1) \times 128$*. Generated by code at figures.probml.ai/book1/13.4.*



(a) *(b)*

Figure 13.5: Results of applying an MLP (with 2 hidden layers with 128 units and 1 output layer with 10 units) to some MNIST images (cherry picked to include some errors). Red is incorrect, blue is correct. (a) After 1 epoch of training. (b) After 2 epochs. Generated by code at figures.probml.ai/book1/13.5.

13.2.4.3 MLP for sentiment analysis of movie reviews

The "MNIST of text classification" is arguably the **IMDB movie review dataset** from [Maa+11]. (IMDB stands for "internet movie database".) This contains 25k labeled examples for training, and 25k for testing. Each example has a binary label, representing a positive or negative rating. This task is known as (binary) **sentiment analysis**. For example, here are the first two examples from the training set:

```
1. this film was just brilliant casting location scenery story direction everyone's
really suited the part they played robert <UNK> is an amazing actor ...

2. big hair big boobs bad music and a giant safety pin these are the words to best
describe this terrible movie i love cheesy horror movies and i've seen hundreds...
```

Not surprisingly, the first example is labeled positive and the second negative.

We can design an MLP to perform sentiment analysis as follows. Suppose the input is a sequence of T tokens, $\mathbf{x}_{1:T}$, where $\mathbf{x}_t$ is a one-hot vector of size V, where V is the vocabulary size. We treat this as an unordered bag of words (Sec. 1.5.4.1). The first layer of the model is a $E \times V$ embedding

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, None, 16)          160000
_________________________________________________________________
global_average_pooling1d (Gl (None, 16)                0
_________________________________________________________________
dense (Dense)                (None, 16)                272
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 17
=================================================================
Total params: 160,289
Trainable params: 160,289
Non-trainable params: 0
_________________________________________________________________
```

Figure 13.6: Structure of the MLP used for IMDB classification. We use a vocabulary size of $V = 1000$, an embedding size of $E = 16$, and a hidden layer of size 16. The embedding matrix $\mathbf{W}_1$ has size $10,000 \times 16$, the hidden layer (labeled "dense") has a weight matrix $\mathbf{W}_2$ of size 16×16 and bias $\mathbf{b}_2$ of size 16 (note that $16 \times 16 + 16 = 272$), and the final layer (labeled "dense_1") has a weight vector $\mathbf{w}_3$ of size 16 and a bias b_3 of size 1. The global average pooling layer has no free parameters. Generated by code at figures.probml.ai/book1/13.6.

matrix $\mathbf{W}_1$, which converts each sparse V-dimensional vector to a dense E-dimensional embedding $\mathbf{e}_t = \mathbf{W}_1 \mathbf{x}_t$ (see Sec. 20.5 for more details on word embeddings). Next we convert this sequence $T \times D$ of embeddings into a fixed-sized vector using **global average pooling** to compute $\bar{\mathbf{e}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{e}_t$. We then pass this through a nonlinear hidden layer to compute the K-dimensional vector $\mathbf{h}$, which then gets passed into the final linear logistic layer. In summary, the model is as follows:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}_3^\top \mathbf{h} + b_3)) \tag{13.18}$$

$$\mathbf{h} = \varphi(\mathbf{W}_2 \bar{\mathbf{e}} + \mathbf{b}_2) \tag{13.19}$$

$$\bar{\mathbf{e}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{e}_t \tag{13.20}$$

$$\mathbf{e}_t = \mathbf{W}_1 \mathbf{x}_t \tag{13.21}$$

If we use a vocabulary size of $V = 1000$, an embedding size of $E = 16$, and a hidden layer of size 16, we get the model shown in Fig. 13.6. This gets an accuracy of 86% on the validation set.

We see that most of the parameters are in the embedding matrix, which can result in overfitting. Fortunately, we can perform unsupervised pre-training of word embedding models, as we discuss in Sec. 20.5, and then we just have to fine-tune the output layers for this specific labeled task.

13.2.4.4 MLP for heteroskedastic regression

We can also use MLPs for regression. Fig. 13.7 shows how we can make a model for heteroskedastic nonlinear regression. (The term "heteroskedastic" just means that the predicted output variance is input-dependent, as discussed in Sec. 2.6.3.) This function has two outputs which compute $f_\mu(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}, \boldsymbol{\theta}]$ and $f_\sigma(\mathbf{x}) = \sqrt{\mathbb{V}[y|\mathbf{x}, \boldsymbol{\theta}]}$. We can share most of the layers (and hence parameters) between these two functions by using a common "**backbone**" and two output "**heads**", as shown in Fig. 13.7. For the μ head, we use a linear activation, $\varphi(a) = a$. For the σ head, we use a softplus activation, $\varphi(a) = \sigma_+(a) = \log(1 + e^a)$. If we use linear heads and a nonlinear backbone, the overall

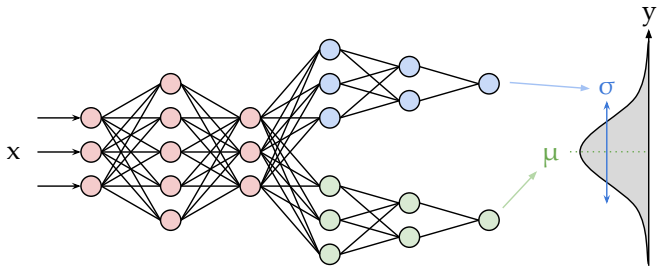
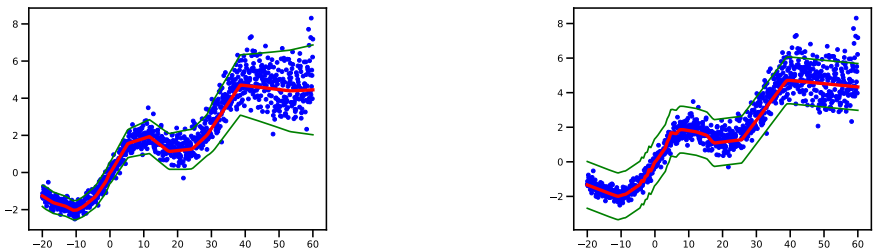


Figure 13.7: Illustration of an MLP with a shared "backbone" and two output "heads", one for predicting the mean and one for predicting the variance. From https://brendanhasz.github.io/2019/07/23/bayesian-density-net.html. Used with kind permission of Brendan Hasz.



(a) *(b)*

Figure 13.8: Illustration of predictions from an MLP fit using MLE to a 1d regression dataset with growing noise. (a) Output variance is input-dependent, as in Fig. 13.7. (b) Mean is computed using same model as in (a), but output variance is treated as a fixed parameter σ^2, which is estimated by MLE after training, as in Sec. 11.2.3.6. Generated by code at figures.probml.ai/book1/13.8.

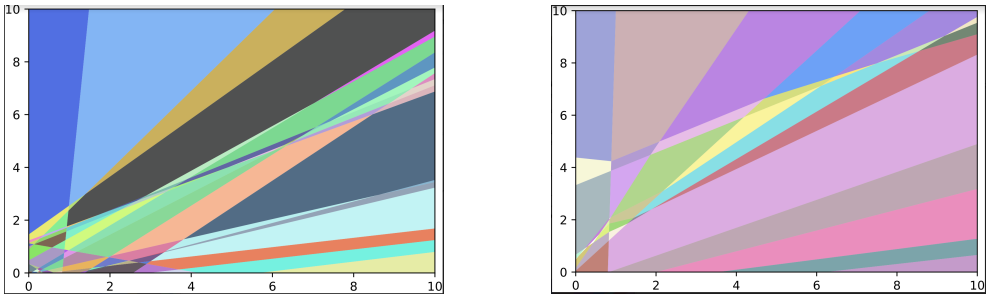
model is given by

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}\left(y|\mathbf{w}_{\boldsymbol{\mu}}^\top f(\mathbf{x}; \mathbf{w}_{\text{shared}}), \sigma_+(\mathbf{w}_{\boldsymbol{\sigma}}^\top f(\mathbf{x}; \mathbf{w}_{\text{shared}}))\right) \tag{13.22}$$

Fig. 13.8 shows the advantage of this kind of model on a dataset where the mean grows linearly over time, with seasonal oscillations, and the variance increases quadratically. (This is a simple example of a **stochastic volatility model**; it can be used to model financial data, as well as the global temperature of the earth, which (due to climate change) is increasing in mean *and* in variance.) We see that a regression model where the output variance σ^2 is treated as a fixed (input-independent) parameter will sometimes be underconfident, since it needs to adjust to the overall noise level, and cannot adapt to the noise level at each point in input space.

13.2.5 The importance of depth

One can show that an MLP *with one hidden layer* is a **universal function approximator**, meaning it can model any suitably smooth function, given enough hidden units, to any desired level of accuracy



(a) *(b)*

Figure 13.9: A decomposition of $\mathbb{R}^2$ into a finite set of linear decision regions produced by an MLP with ReLU *activations with (a) one hidden layer of 25 hidden units and (b) two hidden layers. From Figure 1 of [HAB19]. Used with kind permission of Maksym Andriuschenko.*

[HSW89; Cyb89; Hor91]. Intuitively, the reason for this is that each hidden unit can specify a half plane, and a sufficiently large combination of these can "carve up" any region of space, to which we can associate any response (this is easiest to see when using piecewise linear activation functions, as shown in Fig. 13.9).

However, various arguments, both experimental and theoretical (e.g., [Has87; Mon+14; Rag+17; Pog+17]), have shown that deep networks work better than shallow ones. The reason is that later layers can leverage the features that are learned by earlier layers; that is, the function is defined in a **compositional** or **hierarchical** way. For example, suppose we want to classify DNA strings, and the positive class is associated with the regular expression `*AA??CGCG??AA*`. Although we could fit this with a single hidden layer model, intuitively it will be easier to learn if the model first learns to detect the AA and CG "motifs" using the hidden units in layer 1, and then uses these features to define a simple linear classifier in layer 2, analogously to how we solved the XOR problem in Sec. 13.2.1.

13.2.5.1 The "deep learning revolution"

Although the ideas behind DNNs date back several decades, it was not until the 2010s that they started to become very widely used. The first area to adopt these methods was the field of automatic speech recognition (ASR), based on breakthrough results in [Dah+11]. This approach rapidly became the standard paradigm, and was widely adopted in academia and industry [Hin+12].

However, the moment that got the most attention was when [KSH12] showed that deep CNNs could significantly improve performance on the challenging ImageNet image classification benchmark, reducing the error rate from 26% to 16% in a single year (see Fig. 1.14b); this was a huge jump compared to the previous rate of progress of about 2% reduction per year.

The "explosion" in the usage of DNNs has several contributing factors. One is the availability of cheap **GPUs** (graphics processing units); these were originally developed to speed up image rendering for video games, but they can also massively reduce the time it takes to fit large CNNs, which involve similar kinds of matrix-vector computations. Another is the growth in large labeled datasets, which enables us to fit complex function approximators with many parameters without

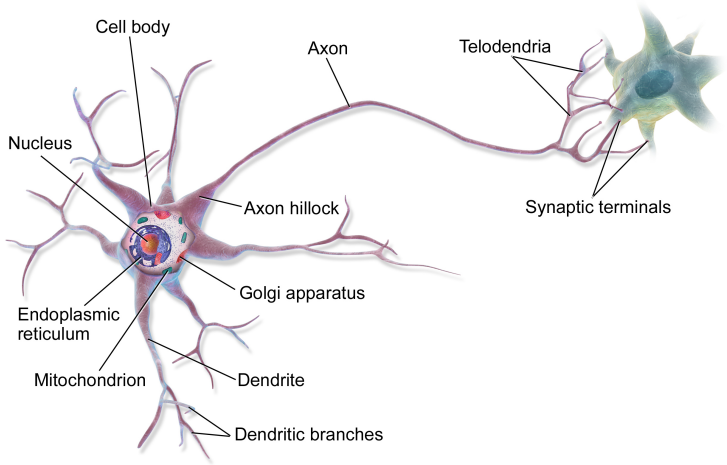


Figure 13.10: Illustration of two neurons connected together in a "circuit". The output axon of the left neuron makes a synaptic connection with the dendrites of the cell on the right Electrical charges, in the form of ion flows, allow the cells to communicate. From https://en.wikipedia.org/wiki/Neuron. Used with kind permission of Wikipedia author BruceBlaus.

overfitting. (For example, ImageNet has 1.3M labeled images, and is used to fit models that have millions of parameters.) Indeed, if deep learning systems are viewed as "rockets", then large datasets have been called the fuel.[1]

Motivated by the outstanding empirical success of DNNs, various companies started to become interested in this technology. This had led to the development of high quality open-source software libraries, such as Tensorflow (made by Google), PyTorch (made by Facebook), and MXNet (made by Amazon). These libraries support automatic differentiation (see Sec. 13.3) and scalable gradient-based optimization (see Sec. 8.4) of complex differentiable functions. We will use some of these libraries in various places throughout the book to implement a variety of models, not just DNNs.[2]

More details on the history of the "deep learning revolution" can be found in e.g., [Sej18].

13.2.6 Connections with biology

In this section, we discuss the connections between the kinds of neural networks we have discussed above, known as **artificial neural networks** or **ANN**s, and real neural networks. The details on how real biological brains work are quite complex (see e.g., [Kan+12]), but we can give a simple "cartoon".

We start by considering a model of a single neuron. To a first approximation, we can say that whether neuron k fires, denoted by $h_k \in \{0, 1\}$, depends on the activity of its inputs, denoted by

1. This popular analogy is due to Andrew Ng, who mentioned it in a keynote talk at the GPU Technology Conference (GTC) in 2015. His slides are available at https://bit.ly/38RTxzH.
2. Note, however, that some have argued (see e.g., [BI19]) that current libraries are too inflexible, and put too much emphasis on methods based on dense matrix-vector multiplication, as opposed to more general algorithmic primitives.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

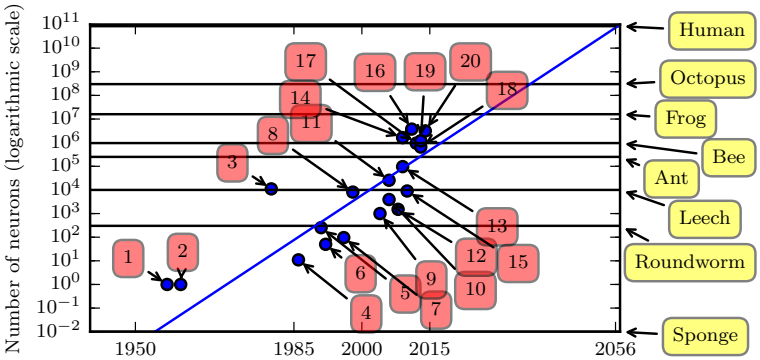


Figure 13.11: Plot of neural network sizes over time. Models 1, 2, 3 and 4 correspond to the perceptron [Ros58], the adaptive linear unit [WH60] the neocognitron [Fuk80], and the first MLP trained by backprop [RHW86]. From Figure 1.11 of [GBC16]. Used with kind permission of Ian Goodfellow.

$\mathbf{x} \in \mathbb{R}^D$, as well as the strength of the incoming connections, which we denote by $\mathbf{w}_k \in \mathbb{R}^D$. We can compute a weighted sum of the inputs using $a_k = \mathbf{w}_k^\top \mathbf{x}$. These weights can be viewed as "wires" connecting the inputs x_d to neuron h_k; these are analogous to **dendrites** in a real neuron (see Fig. 13.10). This weighted sum is then compared to a threshold, b_k, and if the activation exceeds the threshold, the neuron fires; this is analogous to the neuron emitting an electrical output or **action potential**. Thus we can model the behavior of the neuron using $h_k(\mathbf{x}) = H(\mathbf{w}_k^\top \mathbf{x} - b_k)$, where $H(a) = \mathbb{I}(a > 0)$ is the Heaviside function. The is called the **McCulloch-Pitts model** of a neuron, and was proposed in 1943 [MP43].

We can combine multiple such neurons together to make an ANN. The result has sometimes been viewed as a model of the brain. However, ANNs differs from biological brains in many ways, including the following:

- Most ANNs use backpropagation to modify the strength of their connections (see Sec. 13.3). However, real brains do not use backprop, since there is no way to send information backwards along an axon [Ben+15b; BS16; KH19]. Instead, they use local update rules for adjusting synaptic strengths.
- Most ANNs are strictly feedforward, but real brains have many feedback connections. It is believed that this feedback acts like a prior, which can be combined with bottom up likelihoods from the sensory system to compute a posterior over hidden states of the world, which can then be used for optimal decision making (see e.g., [Doy+07]).
- Most ANNs use simplified neurons consisting of a weighted sum passed through a nonlinearity, but real biological neurons have complex dendritic tree structure (see Fig. 13.10), with complex spatio-temporal dynamics.
- Most ANNs are smaller in size and number of connections than biological brains (see Fig. 13.11). Of course, ANNs are getting larger every week, fueled by various new **hardware accelerators**, such as GPUs and **TPUs** (**tensor processing units**), etc. However, even if ANNs match

biological brains in terms of number of units, the comparison is misleading since the processing capability of a biological neuron is much higher than an artificial neuron (see point above).

- Most ANNs are designed to model a single function, such as mapping an image to a label, or a sequence of words to another sequence of words. By contrast, biological brains are very complex systems, composed of multiple specialized interacting modules, which implement different kinds of functions or behaviors such as perception, control, memory, language, etc (see e.g., [Sha88; Kan+12]).

Of course, there are efforts to make realistic models of biological brains (e.g., the **Blue Brain Project** [Mar06; Yon19]). However, an interesting question is whether studying the brain at this level of detail is useful for "solving AI". It is commonly believed that the low level details of biological brains do not matter if our goal is to build "intelligent machines", just as aeroplanes do not flap their wings. However, presumably "AIs" will follow similar "laws of intelligence" to intelligent biological agents, just as planes and birds follow the same laws of aerodynamics.

Unfortunately, we do not yet know what the "laws of intelligence" are, or indeed if there even are such laws. In this book we make the assumption that any intelligent agent should follow the basic principles of information processing and Bayesian decision theory, which is known to be the optimal way to make decisions under uncertainty (see e.g., [KWW22]).

Of course, biological agents are subject to many constraints (e.g., computational, ecological) which often require algorithmic "shortcuts" to the optimal solution; this can explain many of the **heuristics** that people use in everyday reasoning [KST82; GTA00; Gri20]. As the tasks we want our machines to solve become harder, we may be able to gain insights from other areas of neuroscience and cognitive science (see e.g., [MWK16; Has+17; Lak+17]).

13.3 Backpropagation

This section is coauthored with Mathieu Blondel.

In this section, we describe the famous **backpropagation algorithm**, which can be used to compute the gradient of a loss function applied to the output of the network wrt the parameters in each layer. This gradient can then be passed to a gradient-based optimization algorithm, as we discuss in Sec. 13.4.

The backpropagation algorithm was originally discovered in [BH69], and independently in [Wer74]. However, it was [RHW86] that brought the algorithm to the attention of the "mainstream" ML community. See the wikipedia page[3] for more historical details.

We initially assume the computation graph is a simple linear chain of stacked layers, as in an MLP. In this case, backprop is equivalent to repeated applications of the chain rule of calculus (see Eq. (7.246)). However, the method can be generalized to arbitrary directed acyclic graphs (DAGs), as we discuss in Sec. 13.3.4. This general procedure is often called **automatic differentiation** or **autodiff**.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

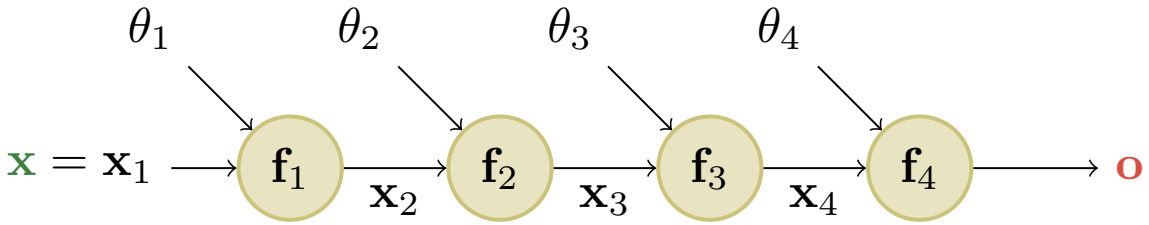


Figure 13.12: A simple linear-chain feedforward model with 4 layers. Here $\mathbf{x}$ *is the input and* $\mathbf{o}$ *is the output. From [Blo20].*

13.3.1 Forward vs reverse mode differentiation

Consider a mapping of the form $\mathbf{o} = \mathbf{f}(\mathbf{x})$, where $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{o} \in \mathbb{R}^m$. We assume that $\mathbf{f}$ is defined as a composition of functions:

$$\mathbf{f} = \mathbf{f}_4 \circ \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1 \tag{13.23}$$

where $\mathbf{f}_1 : \mathbb{R}^n \to \mathbb{R}^{m_1}$, $\mathbf{f}_2 : \mathbb{R}^{m_1} \to \mathbb{R}^{m_2}$, $\mathbf{f}_3 : \mathbb{R}^{m_2} \to \mathbb{R}^{m_3}$, and $\mathbf{f}_4 : \mathbb{R}^{m_3} \to \mathbb{R}^m$. The intermediate steps needed to compute $\mathbf{o} = \mathbf{f}(\mathbf{x})$ are $\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x})$, $\mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2)$, $\mathbf{x}_4 = \mathbf{f}_3(\mathbf{x}_3)$, and $\mathbf{o} = \mathbf{f}_4(\mathbf{x}_4)$.

We can compute the Jacobian $\mathbf{J}_\mathbf{f}(\mathbf{x}) = \frac{\partial \mathbf{o}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$ using the chain rule:

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_4}\frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3}\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\partial \mathbf{x}} = \frac{\partial \mathbf{f}_4(\mathbf{x}_4)}{\partial \mathbf{x}_4}\frac{\partial \mathbf{f}_3(\mathbf{x}_3)}{\partial \mathbf{x}_3}\frac{\partial \mathbf{f}_2(\mathbf{x}_2)}{\partial \mathbf{x}_2}\frac{\partial \mathbf{f}_1(\mathbf{x})}{\partial \mathbf{x}} \tag{13.24}$$

$$= \mathbf{J}_{\mathbf{f}_4}(\mathbf{x}_4)\mathbf{J}_{\mathbf{f}_3}(\mathbf{x}_3)\mathbf{J}_{\mathbf{f}_2}(\mathbf{x}_2)\mathbf{J}_{\mathbf{f}_1}(\mathbf{x}) \tag{13.25}$$

We now discuss how to compute the Jacobian $\mathbf{J}_\mathbf{f}(\mathbf{x})$ efficiently. Recall that

$$\mathbf{J}_\mathbf{f}(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{pmatrix} = \left(\frac{\partial \mathbf{f}}{\partial x_1}, \cdots, \frac{\partial \mathbf{f}}{\partial x_n}\right) \in \mathbb{R}^{m \times n} \tag{13.26}$$

where $\nabla f_i(\mathbf{x})^\top \in \mathbb{R}^{1 \times n}$ is the i'th row (for $i = 1 : m$) and $\frac{\partial \mathbf{f}}{\partial x_j} \in \mathbb{R}^m$ is the j'th column (for $j = 1 : n$). Note that, in our notation, when $m = 1$, the gradient, denoted $\nabla \mathbf{f}(\mathbf{x})$, has the same shape as $\mathbf{x}$. It is therefore a column vector, while $\mathbf{J}_\mathbf{f}(\mathbf{x})$ is a row vector. In this case, we therefore technically have $\nabla \mathbf{f}(\mathbf{x}) = \mathbf{J}_\mathbf{f}(\mathbf{x})^\top$.

We can extract the i'th row from $\mathbf{J}_\mathbf{f}(\mathbf{x})$ by using a vector Jacobian product (VJP) of the form $\mathbf{e}_i^\top \mathbf{J}_\mathbf{f}(\mathbf{x})$, where $\mathbf{e}_i \in \mathbb{R}^m$ is the unit basis vector. Similarly, we can extract the j'th column from $\mathbf{J}_\mathbf{f}(\mathbf{x})$ by using a Jacobian vector product (JVP) of the form $\mathbf{J}_\mathbf{f}(\mathbf{x})\mathbf{e}_j$, where $\mathbf{e}_j \in \mathbb{R}^n$. This shows that the computation of $\mathbf{J}_\mathbf{f}(\mathbf{x})$ reduces to either n JVPs or m VJPs.

If $n < m$, it is more efficient to compute $\mathbf{J}_\mathbf{f}(\mathbf{x})$ for each column $j = 1 : n$ by using JVPs in a right-to-left manner. The right multiplication with a column vector $\mathbf{v}$ is

$$\mathbf{J}_\mathbf{f}(\mathbf{x})\mathbf{v} = \underbrace{\mathbf{J}_{\mathbf{f}_4}(\mathbf{x}_4)}_{m \times m_3}\underbrace{\mathbf{J}_{\mathbf{f}_3}(\mathbf{x}_3)}_{m_3 \times m_2}\underbrace{\mathbf{J}_{\mathbf{f}_2}(\mathbf{x}_2)}_{m_2 \times m_1}\underbrace{\mathbf{J}_{\mathbf{f}_1}(\mathbf{x}_1)}_{m_1 \times n}\underbrace{\mathbf{v}}_{n \times 1} \tag{13.27}$$

3. https://en.wikipedia.org/wiki/Backpropagation#History

This can be computed using **forward mode differentiation**; see Algorithm 5 for the pseudocode. Assuming $m = 1$ and $n = m_1 = m_2 = m_3$, the cost of computing $\mathbf{J}_\mathbf{f}(\mathbf{x})$ is $O(n^3)$.

Algorithm 5: Foward mode differentiation

$\mathbf{x}_1 := \mathbf{x}$
$\mathbf{v}_j := \mathbf{e}_j \in \mathbb{R}^n$ for $j = 1 : n$
for $k = 1 : K$ **do**
  $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$
  $\mathbf{v}_j := \mathbf{J}_{\mathbf{f}_k}(\mathbf{x}_k)\mathbf{v}_j$ for $j = 1 : n$
Return $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_\mathbf{f}(\mathbf{x})]_{:,j} = \mathbf{v}_j$ for $j = 1 : n$

If $n > m$ (e.g., if the output is a scalar), it is more efficient to compute $\mathbf{J}_\mathbf{f}(\mathbf{x})$ for each row $i = 1 : m$ by using VJPs in a left-to-right manner. The left multiplication with a row vector $\mathbf{u}^\top$ is

$$\mathbf{u}^\top \mathbf{J}_\mathbf{f}(\mathbf{x}) = \underbrace{\mathbf{u}^\top}_{1\times m} \underbrace{\mathbf{J}_{\mathbf{f}_4}(\mathbf{x}_4)}_{m\times m_3} \underbrace{\mathbf{J}_{\mathbf{f}_3}(\mathbf{x}_3)}_{m_3\times m_2} \underbrace{\mathbf{J}_{\mathbf{f}_2}(\mathbf{x}_2)}_{m_2\times m_1} \underbrace{\mathbf{J}_{\mathbf{f}_1}(\mathbf{x}_1)}_{m_1\times n} \tag{13.28}$$

This can be done using **reverse mode differentiation**; see Algorithm 6 for the pseudocode. Assuming $m = 1$ and $n = m_1 = m_2 = m_3$, the cost of computing $\mathbf{J}_\mathbf{f}(\mathbf{x})$ is $O(n^2)$.

Algorithm 6: Reverse mode differentiation

$\mathbf{x}_1 := \mathbf{x}$
for $k = 1 : K$ **do**
  $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$
$\mathbf{u}_i := \mathbf{e}_i \in \mathbb{R}^m$ for $i = 1 : m$
for $k = K : 1$ **do**
  $\mathbf{u}_i^\top := \mathbf{u}_i^\top \mathbf{J}_{\mathbf{f}_k}(\mathbf{x}_k)$ for $i = 1 : m$
Return $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_\mathbf{f}(\mathbf{x})]_{i,:} = \mathbf{u}_i^\top$ for $i = 1 : m$

Both Algorithms 5 and 6 can be adapted to compute JVPs and VJPs against *any* collection of input vectors, by accepting $\{\mathbf{v}_j\}_{j=1,\ldots,n}$ and $\{\mathbf{u}_i\}_{i=1,\ldots,m}$ as respective inputs. Initializing these vectors to the standard basis is useful specifically for producing the complete Jacobian as output.

13.3.2 Reverse mode differentiation for multilayer perceptrons

In the previous section, we considered a simple linear-chain feedforward model where each layer does not have any learnable parameters. In this section, each layer can now have (optional) parameters $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_4$. See Fig. 13.12 for an illustration. We focus on the case where the mapping has the form $\mathcal{L} : \mathbb{R}^n \to \mathbb{R}$, so the output is a scalar. For example, consider ℓ_2 loss for a MLP with one hidden layer:

$$\mathcal{L}((\mathbf{x}, \mathbf{y}), \boldsymbol{\theta}) = \frac{1}{2}||\mathbf{y} - \mathbf{W}_2\varphi(\mathbf{W}_1\mathbf{x})||_2^2 \tag{13.29}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

we can represent this as the following feedforward model:

$$\mathcal{L} = \mathbf{f}_4 \circ \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1 \tag{13.30}$$
$$\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x}, \boldsymbol{\theta}_1) = \mathbf{W}_1\mathbf{x} \tag{13.31}$$
$$\mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2, \emptyset) = \varphi(\mathbf{x}_2) \tag{13.32}$$
$$\mathbf{x}_4 = \mathbf{f}_3(\mathbf{x}_3, \boldsymbol{\theta}_3) = \mathbf{W}_2\mathbf{x}_3 \tag{13.33}$$
$$\mathcal{L} = \mathbf{f}_4(\mathbf{x}_4, \mathbf{y}) = \frac{1}{2}||\mathbf{x}_4 - \mathbf{y}||^2 \tag{13.34}$$

We use the notation $\mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)$ to denote the function at layer k, where $\mathbf{x}_k$ is the previous output and $\boldsymbol{\theta}_k$ are the optional parameters for this layer.

In this example, the final layer returns a scalar, since it corresponds to a loss function $\mathcal{L} \in \mathbb{R}$. Therefore it is more efficient to use reverse mode differentation to compute the gradient vectors.

We first discuss how to compute the gradient of the scalar output wrt the parameters in each layer. We can compute $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_4}$ directly, using vector calculus. For the intermediate terms, we can use the chain rule to get

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4}\frac{\partial \mathbf{x}_4}{\partial \boldsymbol{\theta}_3} \tag{13.35}$$
$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4}\frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3}\frac{\partial \mathbf{x}_3}{\partial \boldsymbol{\theta}_2} \tag{13.36}$$
$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4}\frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3}\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\partial \boldsymbol{\theta}_1} \tag{13.37}$$

where each $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_k} = (\nabla_{\boldsymbol{\theta}_k}\mathcal{L})^\top$ is a d_k-dimensional gradient row vector, where d_k is the number of parameters in layer k. We see that these can be computed recursively, by multiplying the gradient row vector at layer k by the Jacobian $\frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_{k-1}}$ which is an $n_k \times n_{k-1}$ matrix, where n_k is the number of hidden units in layer k. See Algorithm 7 for the pseudocode.

This algorithm computes the gradient of the loss wrt the parameters at each layer. It also computes the gradient of the loss wrt the input, $\nabla_{\mathbf{x}}\mathcal{L} \in \mathbb{R}^n$, where n is the dimensionality of the input. This latter quantity is not needed for parameter learning, but can be useful for generating inputs to a model (see Sec. 14.6 for some applications).

All that remains is to specify how to compute the vector Jacobian product (VJP) of all supported layers. The details of this depend on the form of the function at each layer. We discuss some examples below.

13.3.3 Vector-Jacobian product for common layers

Recall that the Jacobian for a layer of the form $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$. is defined by

$$\mathbf{J}_{\mathbf{f}}(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{pmatrix} = \left(\frac{\partial \mathbf{f}}{\partial x_1}, \cdots, \frac{\partial \mathbf{f}}{\partial x_n}\right) \in \mathbb{R}^{m \times n} \tag{13.38}$$

where $\nabla f_i(\mathbf{x})^\top \in \mathbb{R}^n$ is the i'th row (for $i = 1 : m$) and $\frac{\partial \mathbf{f}}{\partial x_j} \in \mathbb{R}^m$ is the j'th column (for $j = 1 : n$). In this section, we describe how to compute the VJP $\mathbf{u}^\top \mathbf{J}_{\mathbf{f}}(\mathbf{x})$ for common layers.

Algorithm 7: Backpropagation for an MLP with K layers

// Forward pass
$\mathbf{x}_1 := \mathbf{x}$
for $k = 1 : K$ **do**
  $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)$
// Backward pass
$\mathbf{u}_{K+1} := 1$
for $k = K : 1$ **do**
  $\mathbf{g}_k := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \boldsymbol{\theta}_k}$
  $\mathbf{u}_k^\top := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \mathbf{x}_k}$
// Output
Return $\mathcal{L} = \mathbf{x}_{K+1}$, $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{u}_1$, $\{\nabla_{\boldsymbol{\theta}_k} \mathcal{L} = \mathbf{g}_k : k = 1 : K\}$

13.3.3.1 Cross entropy layer

Consider a cross-entropy loss layer taking logits $\mathbf{x}$ and target labels $\mathbf{y}$ as input, and returning a scalar:

$$z = f(\mathbf{x}) = \text{CrossEntropyWithLogits}(\mathbf{y}, \mathbf{x}) = -\sum_c y_c \log(\mathcal{S}(\mathbf{x})_c) = -\sum_c y_c \log p_c \tag{13.39}$$

where $\mathbf{p} = \mathcal{S}(\mathbf{x}) = \frac{e^{x_c}}{\sum_{c'=1}^C e^{x_{c'}}}$ are the predicted class probabilites, and $\mathbf{y}$ is the target one-hot encoding. (So both $\mathbf{p}$ and $\mathbf{y}$ live in the C-dimensional probability simplex.) The Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial z}{\partial \mathbf{x}} = (\mathbf{p} - \mathbf{y})^\top \in \mathbb{R}^{1 \times C} \tag{13.40}$$

To see this, assume the target label is class c. We have

$$z = f(\mathbf{x}) = -\log(p_c) = -\log\left(\frac{e^{x_c}}{\sum_j e^{x_j}}\right) = \log\left(\sum_j e^{x_j}\right) - x_c \tag{13.41}$$

Hence

$$\frac{\partial z}{\partial x_i} = \frac{\partial}{\partial x_i} \log \sum_j e^{x_j} - \frac{\partial}{\partial x_i} x_c = \frac{e^{x_i}}{\sum_j e^{x_j}} - \frac{\partial}{\partial x_i} x_c = p_i - \mathbb{I}(i = c) \tag{13.42}$$

If we define $\mathbf{y} = [\mathbb{I}(i = c)]$, we recover Eq. (13.40). Note that the Jacobian of this layer is a row vector, since the output is a scalar. The VJP is simply $\mathbf{u}^\top \mathbf{J}$, for any $\mathbf{u} \in \mathbb{R}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

13.3.3.2 Elementwise nonlinearity

Consider a layer that applies an elementwise nonlinearity, $\mathbf{z} = \mathbf{f}(\mathbf{x}) = \varphi(\mathbf{x})$, so $z_i = \varphi(x_i)$. The (i, j) element of the Jacobian is given by

$$\frac{\partial z_i}{\partial x_j} = \begin{cases} \varphi'(x_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{13.43}$$

where $\varphi'(a) = \frac{d}{da}\varphi(a)$. In other words, the Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \text{diag}(\varphi'(\mathbf{x})) \tag{13.44}$$

For an arbitrary vector $\mathbf{u}$, we can compute $\mathbf{u}^\top \mathbf{J}$ by elementwise multiplication of the diagonal elements of $\mathbf{J}$ with $\mathbf{u}$. For example, if

$$\varphi(a) = \text{ReLU}(a) = \max(a, 0) \tag{13.45}$$

we have

$$\varphi'(a) = \begin{cases} 0 & a < 0 \\ 1 & a > 0 \end{cases} \tag{13.46}$$

The subderivative (Sec. 8.1.4.1) at $a = 0$ is any value in $[0, 1]$. It is often taken to be 0. Hence

$$\text{ReLU}'(a) = H(a) \tag{13.47}$$

where H is the Heaviside step function.

13.3.3.3 Linear layer

Now consider a linear layer, $\mathbf{z} = \mathbf{f}(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x}$, where $\mathbf{W} \in \mathbb{R}^{m \times n}$, so $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{z} \in \mathbb{R}^m$. We can compute the Jacobian wrt the input vector, $\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$, as follows. Note that

$$z_i = \sum_{k=1}^{n} W_{ik} x_k \tag{13.48}$$

So the (i, j) entry of the Jacobian will be

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_{k=1}^{n} W_{ik} x_k = \sum_{k=1}^{n} W_{ik} \frac{\partial}{\partial x_j} x_k = W_{ij} \tag{13.49}$$

since $\frac{\partial}{\partial x_j} x_k = \mathbb{I}(k = j)$. Hence the Jacobian wrt the input is

$$\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{W} \tag{13.50}$$

The VJP between $\mathbf{u}^\top \in \mathbb{R}^{1\times m}$ and $\mathbf{J} \in \mathbb{R}^{m\times n}$ is

$$\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{u}^\top \mathbf{W} \in \mathbb{R}^{1\times n} \tag{13.51}$$

Now consider the Jacobian wrt the weight matrix, $\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$. This can be represented as a $m \times (m \times n)$ matrix, which is complex to deal with. So instead, let us focus on taking the gradient wrt a single weight, W_{ij}. This is easier to compute, since $\frac{\partial \mathbf{z}}{\partial W_{ij}}$ is a vector. To compute this, note that

$$z_k = \sum_{l=1}^{m} W_{kl} x_l \tag{13.52}$$

$$\frac{\partial z_k}{\partial W_{ij}} = \sum_{l=1}^{m} x_l \frac{\partial}{\partial W_{ij}} W_{kl} = \sum_{l=1}^{m} x_l \mathbb{I}\left(i = k \text{ and } j = l\right) \tag{13.53}$$

Hence

$$\frac{\partial \mathbf{z}}{\partial W_{ij}} = \begin{pmatrix} 0 & \cdots & 0 & x_j & 0 & \cdots & 0 \end{pmatrix}^\top \tag{13.54}$$

where the non-zero entry occurs in location i. The VJP between $\mathbf{u}^\top \in \mathbb{R}^{1\times m}$ and $\frac{\partial \mathbf{z}}{\partial \mathbf{W}} \in \mathbb{R}^{m\times(m\times n)}$ can be represented as a matrix of shape $1 \times (m \times n)$. Note that

$$\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial W_{ij}} = \sum_{k=1}^{m} u_k \frac{\partial z_k}{\partial W_{ij}} = u_i x_j \tag{13.55}$$

Therefore

$$\left[\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial \mathbf{W}}\right]_{1,:} = \mathbf{u}\mathbf{x}^\top \in \mathbb{R}^{m\times n} \tag{13.56}$$

13.3.3.4 Putting it altogether

For an exercise that puts this altogether, see Exercise 13.1.

13.3.4 Computation graphs

MLPs are a simple kind of DNN in which each layer feeds directly into the next, forming a chain structure, as shown in Fig. 13.12. However, modern DNNs can combine differentiable components in much more complex ways, to create a **computation graph**, analogous to how programmers combine elementary functions to make more complex ones. (Indeed, some have suggested that "deep learning" be called "**differentiable programming**".) The only restriction is that the resulting computation graph corresponds to a **directed ayclic graph** (**DAG**), where each node is a differentiable function of all its inputs.

For example, consider the function

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_1}} \tag{13.57}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

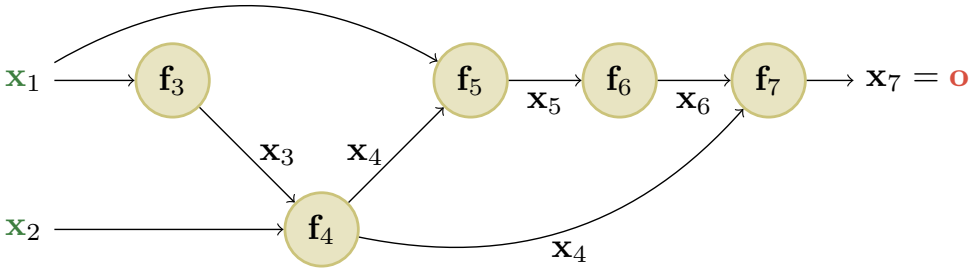


Figure 13.13: An example of a computation graph with 2 (scalar) inputs and 1 (scalar) output. From [Blo20].

We can compute this using the DAG in Fig. 13.13, with the following intermediate functions:

$$x_3 = f_3(x_1) = e^{x_1} \tag{13.58}$$
$$x_4 = f_4(x_2, x_3) = x_2 x_3 \tag{13.59}$$
$$x_5 = f_5(x_1, x_4) = x_1 + x_4 \tag{13.60}$$
$$x_6 = f_6(x_5) = \sqrt{x_5} \tag{13.61}$$
$$x_7 = f_7(x_4, x_6) = x_4 x_6 \tag{13.62}$$

Note that we have numbered the nodes in topological order (parents before children). During the backward pass, since the graph is no longer a chain, we may need to sum gradients along multiple paths. For example, since x_4 influences x_5 and x_7, we have

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_4} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_5}\frac{\partial \mathbf{x}_5}{\partial \mathbf{x}_4} + \frac{\partial \mathbf{o}}{\partial \mathbf{x}_7}\frac{\partial \mathbf{x}_7}{\partial \mathbf{x}_4} \tag{13.63}$$

We can avoid repeated computation by working in reverse topological order. For example,

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_7} = \frac{\partial \mathbf{x}_7}{\partial \mathbf{x}_7} = \mathbf{I}_m \tag{13.64}$$
$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_6} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_7}\frac{\partial \mathbf{x}_7}{\partial \mathbf{x}_6} \tag{13.65}$$
$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_5} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_6}\frac{\partial \mathbf{x}_6}{\partial \mathbf{x}_5} \tag{13.66}$$
$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_4} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_5}\frac{\partial \mathbf{x}_5}{\partial \mathbf{x}_4} + \frac{\partial \mathbf{o}}{\partial \mathbf{x}_7}\frac{\partial \mathbf{x}_7}{\partial \mathbf{x}_4} \tag{13.67}$$

In general, we use

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_j} = \sum_{k \in \text{children}(j)} \frac{\partial \mathbf{o}}{\partial \mathbf{x}_k}\frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_j} \tag{13.68}$$

where the sum is over all all children k of node j, as shown in Fig. 13.14. The $\frac{\partial \mathbf{o}}{\partial \mathbf{x}_k}$ gradient vector has already been computed for each child k; this quantity is called the **adjoint**. This gets multiplied by the Jacobian $\frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_j}$ of each child.

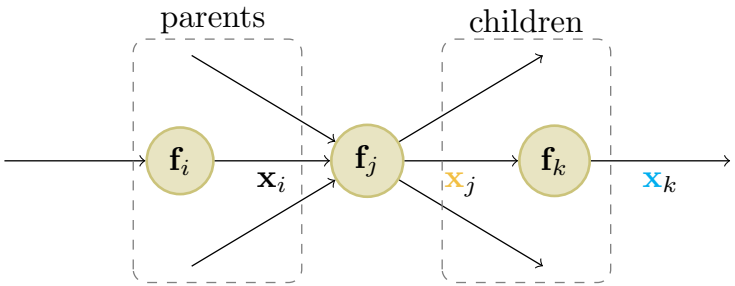


Figure 13.14: Notation for automatic differentiation at node j in a computation graph. From [Blo20].

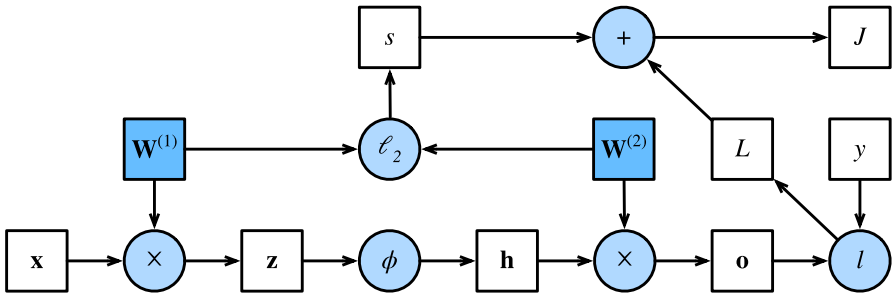


Figure 13.15: Computation graph for an MLP with input $\mathbf{x}$, hidden layer $\mathbf{h}$, output $\mathbf{o}$, loss function $L = \ell(\mathbf{o}, y)$, an ℓ_2 regularizer s on the weights, and total loss $J = L + s$. From Figure 4.7.1 of [Zha+20]. Used with kind permission of Aston Zhang.

The computation graph can be computed ahead of time, by using an API to define a **static graph**. (This is how Tensorflow 1 worked.) Alternatively, the graph can be computed "**just in time**", by **tracing** the execution of the function on an input argument. (This is how Tensorflow eager mode works, as well as JAX and PyTorch.) The latter approach makes it easier to work with a **dynamic graph**, whose shape can change depending on the values computed by the function.

Fig. 13.15 shows a computation graph corresponding to an MLP with one hidden layer with weight decay. More precisely, the model computes the linear pre-activations $\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x}$, the hidden activations $\mathbf{h} = \phi(\mathbf{z})$, the linear outputs $\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}$, the loss $L = \ell(\mathbf{o}, y)$, the regularizer $s = \frac{\lambda}{2}(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2)$, and the total loss $J = L + s$.

13.3.5 Automatic differentiation in functional form *

This section was written by Roy Frostig

Sec. 13.3.1 defined the forward- and reverse-mode diferentiation algorithms. These operate on chain-structured computations, carrying out Jacobian-vector products (JVPs) and vector-Jacobian products (VJPs) respectively. This section generalizes both procedures from chains to DAGs, and then discusses extentions to certain functional programs more generally. It relies heavily on the notation and setup in Sec. 7.8.8.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

What does the operator notation of Sec. 7.8.8 reveal about forward and reverse modes? Consider an example function $f : \mathbb{R}^n \to \mathbb{R}^m$ given by a chain composition of three others:

$$f = c \circ b \circ a \,. \tag{13.69}$$

By the chain rule:

$$\partial f(\mathbf{x}) = \partial c(b(a(\mathbf{x}))) \; \partial b(a(\mathbf{x})) \; \partial a(\mathbf{x}) \,. \tag{13.70}$$

A JVP against a vector $\mathbf{v} \in \mathbb{R}^n$ is therefore:

$$\partial f(\mathbf{x})\mathbf{v} = \partial c(b(a(\mathbf{x}))) \; [\partial b(a(\mathbf{x})) \; [\partial a(\mathbf{x})[\mathbf{v}]]] \;. \tag{13.71}$$

The above expression is bracketed to highlight a right-to-left multiplication order that corresponds to Algorithm 5 (forward mode). In order to carry out this multiplication, it makes sense to compute prefixes of the original chain:

$$\mathbf{x}, \; a(\mathbf{x}), \; b(a(\mathbf{x})) \tag{13.72}$$

alongside the partial JVPs, because each is then immediately used as the linearization point for the subsequent Jacobian:

$$\partial a(\mathbf{x}), \; \partial b(a(\mathbf{x})), \; \partial c(b(a(\mathbf{x}))) \,. \tag{13.73}$$

By contrast, transposing Eq. (13.70) shows that a VJP against a vector $\mathbf{u} \in \mathbb{R}^m$ is:

$$\partial f(\mathbf{x})^\top \mathbf{u} = \partial a(\mathbf{x})^\top \partial b(a(\mathbf{x}))^\top \partial c(b(a(\mathbf{x})))^\top \mathbf{u} \,. \tag{13.74}$$

Algorithm 6 (reverse mode) corresponds to a right-to-left multiplication of Eq. (13.74). But this time, transposition has reversed the Jacobians relative to their order in Eq. (13.70). To handle this, the algorithm computes the original chain prefixes ($\mathbf{x}$, $a(\mathbf{x})$, $b(a(\mathbf{x}))$) upfront first. It then reads them in reverse to form each subsequent linearization point:

$$\partial c(b(a(\mathbf{x})))^\top, \; \partial b(a(\mathbf{x}))^\top, \; \partial a(\mathbf{x})^\top \,. \tag{13.75}$$

Because Algorithm 6 stores all chain prefixes before its backward traversal, it differs from Algorithm 5 in that it requires an additional amount of memory that, in general, grows linearly in the chain's depth. There are ways to combat this memory requirement in special-case scenarios, such as when the chained operations are each reversible [MDA15; Gom+17; KKL20]. One can also trade off memory for computation by discarding some prefixes and re-computing them as needed.

13.3.5.1 From chains to circuits

To set up for this section, we will distinguish between (i) **input nodes** of a computation DAG, which stand in for a function's arguments, and (ii) **primitive nodes**, each of which is labeled by a primitive operation. We assume that input nodes have no incoming edges and (without loss of generality) exactly one outgoing edge each, and that the DAG has exactly one sink node. Such a DAG describes a circuit composed of primitives, i.e. a function from the input nodes to the sink.

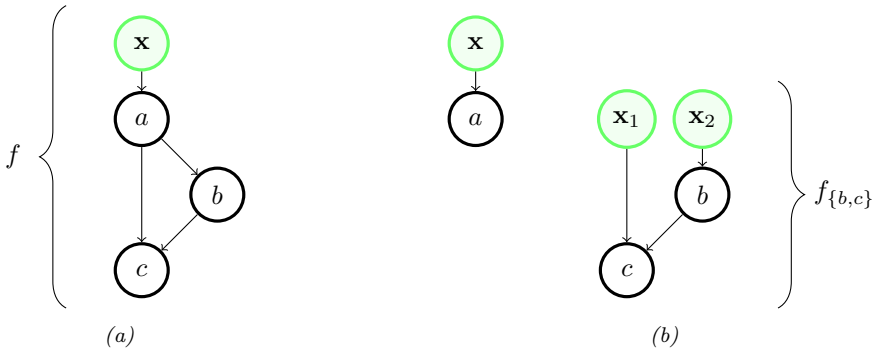


Figure 13.16: A circuit for a function f over three primitives, and its decomposition into two circuits without fan-out. Input nodes are drawn in green.

Jabobian products decompose along chains thanks to the aptly-named chain rule. When moving from chains to DAGs (as in Sec. 13.3.4), do we need some sort of "DAG rule," or a new pair of algorithms, in order to decompose our calculation along the graph? DAGs introduce two new features: **fan-in** and **fan-out**. In graphical terms, fan-in simply refers to multiple edges incoming to a node, and fan-out refers to multiple edges outgoing.

What do these mean in functional terms? Fan-in happens when a primitive operation accepts multiple arguments. We observed in Sec. 7.8.8 that multiple arguments can be treated as one, and how the chain rule then applies. Fan-out requires slightly more care, specifically for reverse-mode differentiation. Why does it entail summation, as in Eq. (13.68)?

The gist of an answer can be illustrated with a small example. Consider the circuit in Fig. 13.16a. The operation a precedes b and c topologically, with an outgoing edge to each of both. We can cut a away from $\{b, c\}$ to produce two new circuits, shown in Fig. 13.16b. The first corresponds to a and the second corresponds to the remaining computation, given by:

$$f_{\{b,c\}}(\mathbf{x}_1, \mathbf{x}_2) = c(\mathbf{x}_1, b(\mathbf{x}_2))\,. \tag{13.76}$$

We can recover the complete function f from a and $f_{\{b,c\}}$ with the help of a function dup given by:

$$\text{dup}(\mathbf{x}) = (\mathbf{x}, \mathbf{x}) \equiv \begin{pmatrix} I \\ I \end{pmatrix} \mathbf{x} \tag{13.77}$$

so that f can be written as a chain composition:

$$f = f_{\{b,c\}} \circ \text{dup} \circ a\,. \tag{13.78}$$

The circuit for $f_{\{b,c\}}$ contains no fan-out, and composition rules such as Eq. (7.278) tell us its derivatives in terms of b, c, and their derivatives, all based on the chain rule. Meanwhile, the chain rule applied to Eq. (13.78) says that:

$$\partial f(\mathbf{x}) = \partial f_{\{b,c\}}(\text{dup}(a(\mathbf{x}))) \, \partial \,\text{dup}(a(\mathbf{x})) \, \partial a(\mathbf{x}) \tag{13.79}$$

$$= \partial f_{\{b,c\}}(a(\mathbf{x}), a(\mathbf{x})) \begin{pmatrix} I \\ I \end{pmatrix} \partial a(\mathbf{x})\,. \tag{13.80}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The above expression suggests calculating a JVP of f at $\mathbf{x}$ against a vector $\mathbf{v}$ by right-to-left multiplication. It is similar to the JVP calculation suggested by Eq. (13.71), but with a *duplication* operation $\begin{pmatrix} I & I \end{pmatrix}^\top$ in the middle that arises from the Jacobian of dup.

Transposing the Jacobian of f shows that a VJP against a vector $\mathbf{u}$ is:

$$\partial f(\mathbf{x})^\top \mathbf{u} = \partial a(\mathbf{x})^\top \begin{pmatrix} I & I \end{pmatrix} \partial f_{\{b,c\}}(a(\mathbf{x}), a(\mathbf{x}))^\top \mathbf{u}\,. \tag{13.81}$$

Considering right-to-left multiplication, this too is similar to the VJP calculation suggested by Eq. (13.74), but with a *summation* operation $\begin{pmatrix} I & I \end{pmatrix}$ in the middle that arises from the *transposed* Jacobian of dup. More generally, extending reverse mode to handle fan-out amounts to processing operations in topological order—first forward and then in reverse—and summing partial VJPs along multiple outgoing edges.

Algorithm 8: Foward-mode circuit differentiation (JVP)

input: $f : \mathbb{R}^n \to \mathbb{R}^m$ composing $f_1, \ldots, f_T$ in topological order, where f_1 is identity
input: linearization point $\mathbf{x} \in \mathbb{R}^n$ and vector $\mathbf{v} \in \mathbb{R}^n$
$\mathbf{x}_1, \mathbf{v}_1 := \mathbf{x}, \mathbf{v}$
for $t := 2, \ldots, T$ **do**
  let $[q_1, \ldots, q_r] = \text{parents}(t)$
  $\mathbf{x}_t := f_t(\mathbf{x}_{q_1}, \ldots, \mathbf{x}_{q_r})$
  $\mathbf{v}_t := \sum_{i=1}^{r} \partial_i f_t(\mathbf{x}_{q_1}, \ldots, \mathbf{x}_{q_r}) \mathbf{v}_{q_i}$
output: $\mathbf{x}_T$, equal to $f(\mathbf{x})$
output: $\mathbf{v}_T$, equal to $\partial f(\mathbf{x})\mathbf{v}$

Algorithm 9: Reverse-mode circuit differentiation (VJP)

input: $f : \mathbb{R}^n \to \mathbb{R}^m$ composing $f_1, \ldots, f_T$ in topological order, where f_1, f_T are identity
input: linearization point $\mathbf{x} \in \mathbb{R}^n$ and vector $\mathbf{u} \in \mathbb{R}^m$
$\mathbf{x}_1 := \mathbf{x}$
for $t := 2, \ldots, T$ **do**
  let $[q_1, \ldots, q_r] = \text{parents}(t)$
  $\mathbf{x}_t := f_t(\mathbf{x}_{q_1}, \ldots, \mathbf{x}_{q_r})$
$\mathbf{u}_{(T-1)\to T} := \mathbf{u}$
for $t := T-1, \ldots, 2$ **do**
  let $[q_1, \ldots, q_r] = \text{parents}(t)$
  $\mathbf{u}'_t := \sum_{c \in \text{children}(t)} \mathbf{u}_{t\to c}$
  $\mathbf{u}_{q_i \to t} := \partial_i f_t(\mathbf{x}_{q_1}, \ldots, \mathbf{x}_{q_r})^\top \mathbf{u}'_t \quad \text{for } i = 1, \ldots, r$
output: $\mathbf{x}_T$, equal to $f(\mathbf{x})$
output: $\mathbf{u}_{1\to 2}$, equal to $\partial f(\mathbf{x})^\top \mathbf{u}$

Algorithms 8 and 9 give a complete description of forward- and reverse-mode differentiation on DAGs. For brevity they assume a single argument to the entire circuit function. Nodes are indexed

$1, \ldots, T$. The first is the input node associated and the remaining $T-1$ are labeled by their operation $f_2, \ldots, f_T$. We take f_1 to be the identity. For each t, if f_t takes k arguments, let parents(t) be the ordered list of k indices of its parent nodes (possibly containing duplicates, due to fan-out), and let children(t) be the indices of its children (again possibly duplicate). Algorithm 9 takes a few more conventions: that f_T is the identity, that node T has $T-1$ as its only parent, and that the child of node 1 is node 2.

Fan-out is a feature of *graphs*, but arguably not an essential feature of *functions*. One can always remove all fan-out from a circuit representation by duplicating nodes. Our motivation for fan-out is to allow for an efficient representation—and, in turn, efficient memory use in Algorithms 8 and 9.

13.3.5.2 From circuits to programs

Representing a function's internal structure as a DAG can seem natural in some domains such as neural network modeling (where the language of graphs has indeed been historically common). In broader context, a far more widespread—and potentially more expressive—way to describe a function from primitives is by writing it in a **formal language**, i.e. as a function in a programming language such as Python (or Haskell, or even a variant of the lambda calculus).[4]

The expressive power of a circuit representation depends on what primitive operations are allowed, the same can be said of a simple programming language's primitives, so in comparing them we should assume a fixed collection of primitives (including, say, elementwise arithmetic operations, array contractions, convolutions, slicing, concatenation, and so forth). Among other reasons, a formal language might then be more natural than a DAG representation when accounting for control flow constructs such as conditionals, loops, and reductions.

In languages with a means of name or variable binding, multiple appearances of the same variable are analogous to fan-out in a DAG. Fig. 13.16a corresponds to a function f that we could write in a functional language as:

```
f(x) =
  let ax = a(x)
   in c(ax, b(ax))
```

in which `ax` indeed appears twice after it is bound.

As a language is made more expressive, we might ask whether—and how efficiently—we can automatically differentiate functions denoted in the language. Understanding this interaction between language capacity and automatic differentiability is an ongoing topic of computer science research [AP19; Vyt+19]. Functional languages that are intentionally restricted—for instance disallowing functions that take functions as arguments—have already proven effective in widely-used tools. Systems such as JAX, Dex, and others are designed around pure functional programming models, and internally rely on such functional representations for differentiation [Sha+19; FJL18; Bra+18; Mac+19; Dex; Fro+21].

4. In Python, what the language calls a "function" does not always describe a pure function of the arguments listed in its syntactic definition; its behavior may rely on side effects or global state, as allowed by the language. Here, we specifically mean a Python function that is pure and functional. JAX's documentation details this restriction [Bra+18].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

13.4 Training neural networks

In this section, we discuss how to fit DNNs to data. The standard approach is to use maximum likelihood estimation, by minimizing the NLL:

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(\mathbf{y}_n|\mathbf{x}_n;\boldsymbol{\theta}) \tag{13.82}$$

It is also common to add a regularizer (such as the negative log prior), as we discuss in Sec. 13.5.

In principle we can just use the backprop algorithm (Sec. 13.3) to compute the gradient of this loss and pass it to an off-the-shelf optimizer, such as those discussed in Chapter 8. (The Adam optimizer of Sec. 8.4.6.3 is a popular choice, due to its ability to scale to large datasets (by virtue of being an SGD-type algorithm), and to converge fairly quikcly (by virtue of using diagonal preconditioning and momentum).) However, in practice this may not work well. In this section, we discuss various problems that may arise, as well as some solutions. For more details on the practicalities of training DNNs, see various other books, such as [HG20; Zha+20; Gér19].

In addition to practical issues, there are important theoretical issues. In particular, we note that the DNN loss is not a convex objective, so in general we will not be able to find the global optimum. Nevertheless, SGD can often find suprisingly good solutions. The research into why this is the case is still being conducted; see [Bah+20] for a recent review of some of this work.

13.4.1 Tuning the learning rate

It is important to tune the learning rate (step size), to ensure convergence to a good solution. We discuss this issue in Sec. 8.4.3.

13.4.2 Vanishing and exploding gradients

When training very deep models, the gradient tends to become either very small (this is called the **vanishing gradient problem**) or very large (this is called the **exploding gradient probem**), because the error signal is being passed through a series of layers which either amplify or diminish it [Hoc+01]. (Similar problems arise in RNNs on long sequences, as we explain in Sec. 15.2.6.)

To explain the problem in more detail, consider the gradient of the loss wrt a node at layer l:

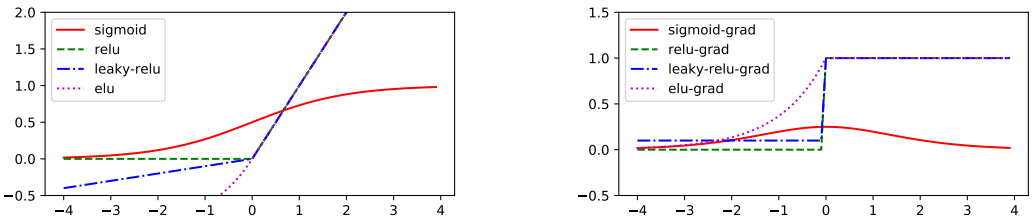
$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l} = \mathbf{J}_l \mathbf{g}_{l+1} \tag{13.83}$$

where $\mathbf{J}_l = \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l}$ is the Jacobian matrix, and $\mathbf{g}_{l+1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}}$ is the gradient at the next layer. If $\mathbf{J}_l$ is constant across layers, it is clear that the contribution of the gradient from the final layer, $\mathbf{g}_L$, to layer l will be $\mathbf{J}^{L-l}\mathbf{g}_L$. Thus the behavior of the system depends on the eigenvectors of $\mathbf{J}$.

Although $\mathbf{J}$ is a real-valued matrix, it is not (in general) symmetric, so its eigenvalues and eigenvectors can be complex-valued, with the imaginary components corresponding to oscillatory behavior. Let λ be the **spectral radius** of $\mathbf{J}$, which is the maximum of the absolute values of the eigenvalues. If this is greater than 1, the gradient can explode; it this is less than 1, the gradient can vanish. (Similarly, the spectral radius of $\mathbf{W}$, connecting $\mathbf{z}_l$ to $\mathbf{z}_{l+1}$, determines the stability of the dynamical system when run in forwards mode.)

Name	Definition	Range	Reference
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$[0, 1]$	
Hyperbolic tangent	$\tanh(a) = 2\sigma(2a) - 1$	$[-1, 1]$	
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$	[GBB11]
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$	[GBB11; KSH12]
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$	[MHN13]
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$	[CUH16]
Swish	$a\sigma(a)$	$[-\infty, \infty]$	[RZL17]

Table 13.2: List of some popular activation functions for neural networks.



(a) *(b)*

Figure 13.17: (a) Some popular activation functions. (b) Plot of their gradients. Generated by code at figures.probml.ai/book1/13.17.

The exploding gradient problem can be ameliorated by **gradient clipping**, in which we cap the magnitude of the gradient if it becomes too large, i.e., we use

$$\mathbf{g}' = \min(1, \frac{c}{||\mathbf{g}||})\mathbf{g} \tag{13.84}$$

This way, the norm of $\mathbf{g}'$ can never exceed c, but the vector is always in the same direction as $\mathbf{g}$.

However, the vanishing gradient problem is more difficult to solve. There are various solutions, such as the following:

- Modify the the activation functions at each layer to prevent the gradient from becoming too large or too small; see Sec. 13.4.3.
- Modify the architecture so that the updates are additive rather than multiplicative; see Sec. 13.4.4.
- Modify the architecture to standardize the activations at each layer, so that the distribution of activations over the dataset remains constant during training; see Sec. 14.2.4.1.
- Carefully choose the initial values of the parameters; see Sec. 13.4.5.

13.4.3 Non-saturating activation functions

In Sec. 13.2.3, we mentioned that the sigmoid activation function saturates at 0 for large negative inputs, and at 1 for large positive inputs. It turns out that the gradient signal in these regimes is 0,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

preventing backpropagation from working.

To see why the gradient vanishes, consider a layer which computes $\mathbf{z} = \sigma(\mathbf{W}\mathbf{x})$, where

$$\varphi(a) = \sigma(a) = \frac{1}{1 + \exp(-a)} \tag{13.85}$$

If the weights are initialized to be large (positive or negative), then it becomes very easy for $\mathbf{a} = \mathbf{W}\mathbf{x}$ to take on large values, and hence for $\mathbf{z}$ to saturate near $\mathbf{0}$ or $\mathbf{1}$, since the sigmoid saturates, as shown in Fig. 13.17a. Now let us consider the gradient of the loss wrt the inputs $\mathbf{x}$ (from an earlier layer) and the parameters $\mathbf{W}$. The derivative of the activation function is given by

$$\varphi'(a) = \sigma(a)(1 - \sigma(a)) \tag{13.86}$$

See Fig. 13.17b for a plot. In Sec. 13.3.3, we show that the gradient of the loss wrt the inputs is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \boldsymbol{\delta}^\top \mathbf{W} = \mathbf{W}^\top \mathbf{z}(1 - \mathbf{z}) \tag{13.87}$$

and the gradient of the loss wrt the parameters is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \boldsymbol{\delta}\mathbf{x}^\top = \mathbf{z}(1 - \mathbf{z})\mathbf{x}^\top \tag{13.88}$$

Hence, if $\mathbf{z}$ is near 0 or 1, the gradients will go to 0.

One of the keys to being able to train very deep models is to use **non-saturating activation functions**. Several different functions have been proposed (see Table 13.2 for a summary). The most common is **rectified linear unit** or **ReLU**, proposed in [GBB11; KSH12]. This is defined as

$$\text{ReLU}(a) = \max(a, 0) = a\mathbb{I}(a > 0) \tag{13.89}$$

The ReLU function simply "turns off" negative inputs, and passes positive inputs unchanged. The gradient has the following form:

$$\text{ReLU}'(a) = \mathbb{I}(a > 0) \tag{13.90}$$

See Fig. 13.17b for a plot. Now suppose we use this in a layer to compute $\mathbf{z} = \text{ReLU}(\mathbf{W}\mathbf{x})$. In Sec. 13.3.3, we show that the gradient wrt the inptus has the form

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^\top \mathbb{I}(\mathbf{z} > \mathbf{0}) \tag{13.91}$$

and wrt the parameters has the form

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbb{I}(\mathbf{z} > \mathbf{0})\, \mathbf{x}^\top \tag{13.92}$$

Hence the gradient will not vanish, as long a $\mathbf{z}$ is positive.

Unfortunately, if the weights are initialized to be large and negative, then it becomes very easy for (some components of) $\mathbf{a} = \mathbf{W}\mathbf{x}$ to take on large negative values, and hence for $\mathbf{z}$ to go to 0. This will cause the gradient for the weights to go to 0. The algorithm will never be able to escape this

situation, so the hidden units (components of $\mathbf{z}$) will stay permanently off. This is called the "**dead** ReLU" problem [Lu+19].

This can be solved by using non-saturating variants of ReLU. One alternate is the **leaky ReLU**, proposed in [MHN13]. This is defined as

$$\text{LReLU}(a; \alpha) = \max(\alpha a, a) \tag{13.93}$$

where $0 < \alpha < 1$. The slope of this function is 1 for positive inputs, and α for negative inputs, thus ensuring there is some signal passed back to earlier layers, even when the input is negative. If we allow the parameter α to be learned, rather than fixed, the leaky ReLU is called **parametric ReLU** [He+15].

Another popular choice is the **ELU**, proposed in [CUH16]. This is defined by

$$\text{ELU}(a; \alpha) = \begin{cases} \alpha(e^a - 1) & \text{if } a \leq 0 \\ a & \text{if } a > 0 \end{cases} \tag{13.94}$$

This has the advantage over leaky ReLU of being a smooth function.

A slight variant of ELU, known as **SELU** (self-normalizing ELU), was proposed in [Kla+17]. This has the form

$$\text{SELU}(a; \alpha, \lambda) = \lambda \text{ELU}(a; \alpha) \tag{13.95}$$

Suprisingly, they prove that by setting α and and λ to carefully chosen values, this activation function is guaranteed to ensure that the output of each layer is standardized (provided the input is also standardized), even without the use of techniques such as batchnorm (Sec. 14.2.4.1). This can help with model fitting.

As an alternative to manually discovering good activation functions, we can use blackbox optimization methods to search over the space of functional forms. Such an approach was used in [RZL17], where they discovered a function they call **swish** that seems to do well on some image classification benchmarks. It is defined by

$$\text{swish}(a; \beta) = a\sigma(\beta a) \tag{13.96}$$

See Fig. 13.2b for a plot.

13.4.4 Residual connections

One solution to the vanishing gradient problem for DNNs is to use a **residual network** or **ResNet** [He+16a]. This is a feedforward model in which each layer has the form of a **residual block**, defined by

$$\mathcal{F}'_l(\mathbf{x}) = \mathcal{F}_l(\mathbf{x}) + \mathbf{x} \tag{13.97}$$

where $\mathcal{F}_l$ is a standard shallow nonlinear mapping (e.g., linear-activation-linear). The inner $\mathcal{F}_l$ function computes the residual term or delta that needs to be added to the input $\mathbf{x}$ to generate the desired output; it is often easier to learn to generate a small perturbation to the input than to

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

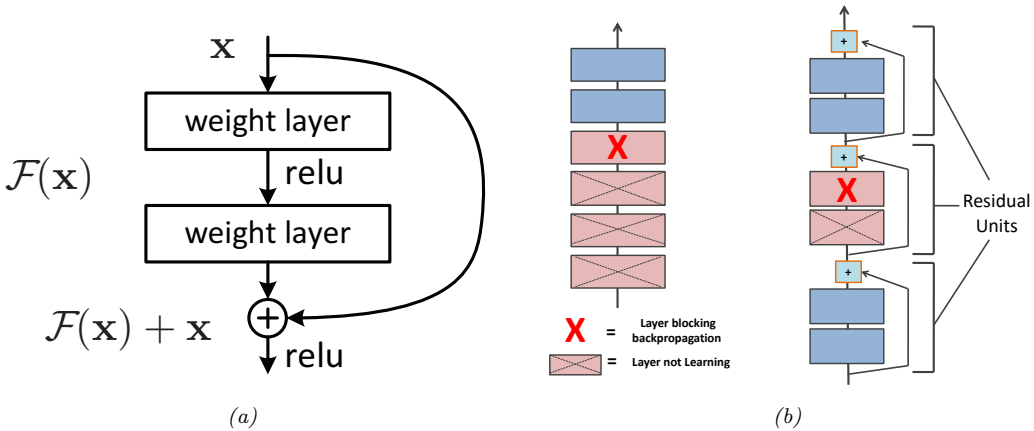


Figure 13.18: (a) Illustration of a residual block. (b) Illustration of why adding residual connections can help when training a very deep model. Adapted from Figure 14.16 of [Gér19].

directly predict the output. (Residual connections are usually used in conjunction with CNNs, as discussed in Sec. 14.3.4, but can also be used in MLPs.)

A model with residual connections has the same number of parameters as a model without residual connections, but it is earier to train. The reason is that gradients can flow directly from the output to earlier layers, as sketched in Fig. 13.18b. To see this, note that the activations at the output layer can be derived in terms of any previous layer l using

$$\mathbf{z}_L = \mathbf{z}_l + \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{z}_i; \boldsymbol{\theta}_i). \tag{13.98}$$

We can therefore compute the gradient of the loss wrt the parameters of the l'th layer as follows:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_l} = \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} \tag{13.99}$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \frac{\partial \mathbf{z}_L}{\partial \mathbf{z}_l} \tag{13.100}$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \left(1 + \sum_{i=l}^{L-1} \frac{\partial f(\mathbf{z}_i; \boldsymbol{\theta}_i)}{\partial \mathbf{z}_l} \right) \tag{13.101}$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} + \text{otherterms} \tag{13.102}$$

Thus we see that the gradient at layer l depends directly on the gradient at layer L in a way that is independent of the depth of the network.

13.4.5 Parameter initialization

Since the objective function for DNN training is non-convex, the way that we we initialize the parameters of a DNN can play a big role on what kind of solution we end up with, as well as how

easy the function is to train (i.e., how well information can flow forwards and backwards though the model). In the rest of this section, we present some common heuristic methods that are used for initializing parameters.

13.4.5.1 Heuristic initialization schemes

In [GB10], they show that sampling parameters from a standard normal with fixed variance can result in exploding activations or gradients. To see why, consider a linear unit with no activation function given by $o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j$; suppose $w_{ij} \sim \mathcal{N}(0, \sigma^2)$, and $\mathbb{E}[x_j] = 0$ and $\mathbb{V}[x_j] = \gamma^2$, where we assume x_j are independent of w_{ij}. The mean and variance of the output is given by

$$\mathbb{E}[o_i] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij} x_j] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}]\,\mathbb{E}[x_j] = 0 \tag{13.103}$$

$$\mathbb{V}[o_i] = \mathbb{E}\left[o_i^2\right] - (\mathbb{E}[o_i])^2 = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}\left[w_{ij}^2 x_j^2\right] - 0 = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}\left[w_{ij}^2\right]\mathbb{E}\left[x_j^2\right] = n_{\text{in}}\sigma^2\gamma^2 \tag{13.104}$$

To keep the output variance from blowing up, we need to ensure $n_{\text{in}}\sigma^2 = 1$ (or some other constant), where n_{in} is the **fan-in** of a unit (number of incoming connections).

Now consider the backwards pass. By analogous reasoning, we see that the variance of the gradients can blow up unless $n_{\text{out}}\sigma^2 = 1$, where n_{out} is the **fan-ou** of a unit (number of outgoing connections). To satisfy both requirements at once, we set $\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1$, or equivalently

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}} \tag{13.105}$$

Thisis known as **Xavier initialization** or **Glorot initialization**, named after the first author of [GB10].

A special case arises if we use $\sigma^2 = 1/n_{\text{in}}$; this is known as **LeCun initialization**, named after Yann LeCun, who proposed it in the 1990s. This is equivalent to Glorot initialization when $n_{\text{in}} = n_{\text{out}}$. If we use $\sigma^2 = 2/n_{\text{in}}$, the method is called **He initialization**, named after Ximing He, who proposed it in [He+15].

Note that it is not necessary to use a Gaussian distribution. Indeed, the above derivation just worked in terms of the first two moments (mean and variance), and made no assumptions about Gaussianity. For example, suppose we sample weights from a uniform distribution, $w_{ij} \sim \text{Unif}(-a, a)$. The mean is 0, and the variance is $\sigma^2 = a^2/3$. Hence we should set $a = \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}$.

Although the above derivation assumes a linear output unit, the technique works well empirically even for nonlinear units. The best choice of initialization method depends on which activation function you use. For linear, tanh, logistic, and softmax, Glorot is recommended. For ReLU and variants, He is recommended. For SELU, LeCun is recommended. See [Gér19] for more heuristics.

13.4.5.2 Data-driven initializations

We can also adopt a data-driven approach to parameter initialization. For example, [MM16] proposed a simple but effective scheme known as **layer-sequential unit-variance** (LSUV) initialization, which works as follows. First we initialize the weights of each (fully connected or convolutional)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

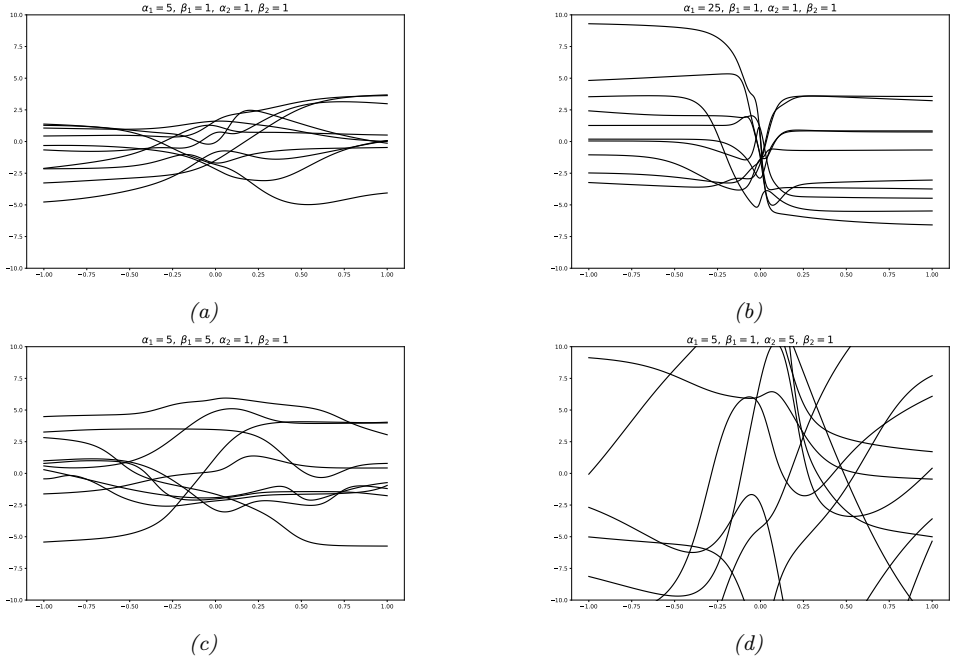


Figure 13.19: The effects of changing the hyperparameters on an MLP with one hidden layer. (a) Random functions sampled from a Gaussian prior with hyperparameters $\alpha_1 = 5$, $\beta_1 = 1$, $\alpha_2 = 1$, $\beta_2 = 1$. (b) Increasing α_1 by factor of 5. (c) Increasing β_1 by factor of 5. (d) Inreasing α_2 by factor of 5. Generated by code at figures.probml.ai/book1/13.19

layer with orthonormal matrices, as proposed in [SMG14]. (This can be achieved by drawing from $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, reshaping to $\mathbf{w}$ to a matrix $\mathbf{W}$, and then computing an orthonormal basis using QR or SVD decomposition.) Then, for each layer l, we compute the variance v_l of the activations across a minibatch; we then rescale using $\mathbf{W}_l := \mathbf{W}_l / \sqrt{v_l}$. This scheme can be viewed as an orthonormal initialization combined with batch normalization performed only on the first mini-batch. This is faster than full batch normalization, but can sometimes work just as well.

13.4.5.3 A function space view

To understand the effect of parameter initialization, we visualize the corresponding function that the network defines. Concretely, consider an MLP with L hidden layers and a linear output:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}_L(\cdots \varphi(\mathbf{W}_2 \varphi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)) + \mathbf{b}_L \tag{13.106}$$

We will use the following factored Gaussian prior for the parameters:

$$\mathbf{W}_\ell \sim \mathcal{N}(\mathbf{0}, \alpha_\ell^2 \mathbf{I}), \ \mathbf{b}_\ell \sim \mathcal{N}(\mathbf{0}, \beta_\ell^2 \mathbf{I}) \tag{13.107}$$

We can reparameterize this distribution as

$$\mathbf{W}_\ell = \alpha_\ell \boldsymbol{\eta}_\ell, \boldsymbol{\eta}_\ell \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \mathbf{b}_\ell = \beta_\ell \boldsymbol{\epsilon}_\ell, \boldsymbol{\epsilon}_\ell \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{13.108}$$

Hence every setting of the prior hyperparameters specifies the following random function:

$$f(\mathbf{x};\boldsymbol{\alpha},\boldsymbol{\beta}) = \alpha_L \boldsymbol{\eta}_L(\cdots\varphi(\alpha_1\boldsymbol{\eta}_1\mathbf{x}+\beta_1\boldsymbol{\epsilon}_1)) + \beta_L\boldsymbol{\epsilon}_L \tag{13.109}$$

To get a feeling for the effect of these hyperparameters, we can sample MLP parameters from this prior and plot the resulting random functions. We use a sigmoid nonlinearity, so $\varphi(a) = \sigma(a)$. We consider $L = 2$ layers, so $\mathbf{W}_1$ are the input-to-hidden weights, and $\mathbf{W}_2$ are the hidden-to-output weights. We assume the input and output are scalars, so we are generating random nonlinear mappings $f : \mathbb{R} \to \mathbb{R}$.

Fig. 13.19(a) shows some sampled functions where $\alpha_1 = 5$, $\beta_1 = 1$, $\alpha_2 = 1$, $\beta_2 = 1$ in the top left In Fig. 13.19(b) we increasing α_1; this allows the first layer weights to get bigger, making the sigmoid-like shape of the functions steeper (compare to Fig. 10.2). In Fig. 13.19(c), we increase β_1; this allows the first layer biases to get bigger, which allows the center of the sigmoid to shift left and right more, away from the origin. In Fig. 13.19(d), we increase α_2; this allows the second layer linear weights to get bigger, making the functions more "wiggly" (greater sensitivity to change in the input, and hence larger dynamic range).

The above results are specific to the case of sigmoidal activation functions. ReLU units can behave differently. For example, [WI20, App. E] show that for MLPs with ReLU units, if we set $\beta_\ell = 0$, so the bias terms are all zero, the effect of changing α_ℓ is just to rescale the output. To see this, note that Eq. (13.109) simplifies to

$$f(\mathbf{x};\boldsymbol{\alpha},\boldsymbol{\beta}=\mathbf{0}) = \alpha_L\boldsymbol{\eta}_L(\cdots\varphi(\alpha_1\boldsymbol{\eta}_1\mathbf{x})) = \alpha_L\cdots\alpha_1\boldsymbol{\eta}_L(\cdots\varphi(\boldsymbol{\eta}_1\mathbf{x})) \tag{13.110}$$

$$= \alpha_L\cdots\alpha_1 f(\mathbf{x};(\boldsymbol{\alpha}=\mathbf{1},\boldsymbol{\beta}=\mathbf{0})) \tag{13.111}$$

where we used the fact that for ReLU, $\varphi(\alpha z) = \alpha\varphi(z)$ for any positive α, and $\varphi(\alpha z) = 0$ for any negative α (since the pre-activation $z \geq 0$). In general, it is the ratio of α and β that matters for determining what happens to input signals as they propagate forwards and backwards through a randomly initialized model; for details, see e.g., [Bah+20].

We see from the above that different distributions over the parameters of a DNN correspond to different *distributions over functions*. Thus initializing the model at random is like sampling a point from this prior. In the limit of infinitely widely neural networks, we can derive this prior distribution analytically: this is known as a **neural network Gaussian process**, and is explained in the sequel to this book, [Mur22].

13.4.6 Multi-GPU training

It can be quite slow to train large models on large datasets. One way to speed this process up is to use specialized hardware, such as **graphics processing units** (**GPUs**), which are very efficient at performing matrix-matrix multiplication. If we have multiple GPUs, we can sometimes further speed things up. There are two main approaches: **model parallelism**, in which we partition the model between machines, and **data parallelism**, in which each machine has its own copy of the model, and applies it to a different set of data.

Model parallelism can be quite complicated, since it requires tight communication between machines to ensure they compute the correct answer. We will not discuss this further. Data parallelism is generally much simpler, since it is **embarassingly parallel**. To use this to speed up training, at each training step t, we do the following: 1) we partition the minibatch across the K machines

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

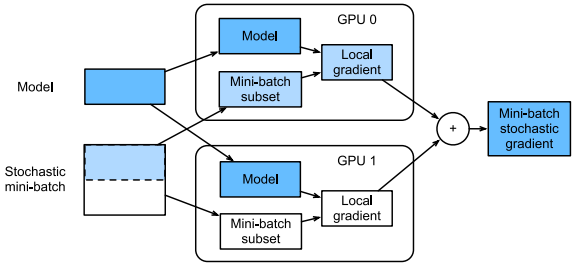


Figure 13.20: Calculation of minibatch stochastic gradient using data parallelism and two GPUs. From Figure 12.5.2 of [Zha+20]. Used with kind permission of Aston Zhang.

to get $\mathcal{D}_t^k$; 2) each machine k computes its own gradient, $\mathbf{g}_t^k = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_t^k)$; 3) we collect all the local gradients on a central machine (e.g., device 0) and sum them using $\mathbf{g}_t = \sum_{k=1}^K \mathbf{g}_t^k$; 4) we broadcast the summed gradient back to all devices, so $\tilde{\mathbf{g}}_t^k = \mathbf{g}_t$; 5) each machine updates its own copy of the parameters using $\boldsymbol{\theta}_t^k := \boldsymbol{\theta}_t^k - \eta_t \tilde{\mathbf{g}}_t^k$. See Fig. 13.20 for an illustration, and code.probml.ai/book1/multi_gpu_training_torch for some sample code.

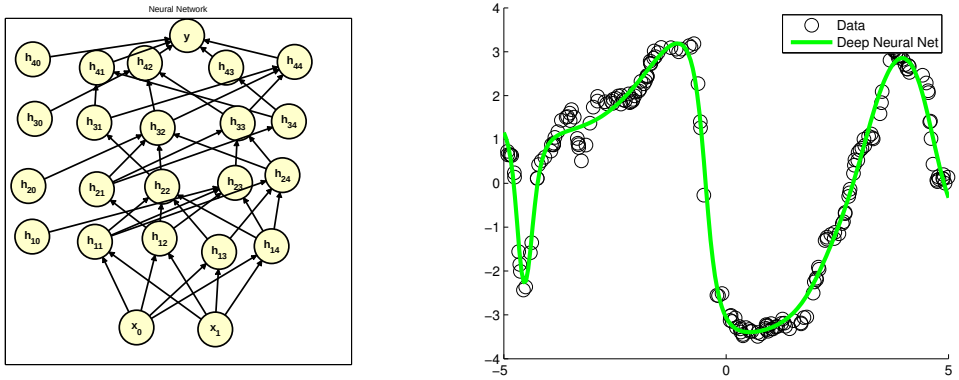
Note that steps 3 and 4 are usually combined into one atomic step; this is known as an **all-reduce** operation (where we use sum to reduce the set of (gradient) vectors into one). If each machine blocks until receiving the centrally aggregated gradient, $\mathbf{g}_t$, the method is known as **synchronous training**. This will give the same results as training with one machine (with a larger batchsize), only faster. If we let each machine update its parameters using its own local gradient estimate, and not wait for the broadcast to/from the other machines, the method is called **asynchronous training**. This is not guaranteed to work, since the different machines may get out of step, and hence will be updating different versions of the parameters; this approach has therefore been called **hogwild training** [Niu+11]. However, if the updates are sparse, so each machine "touches" a different part of the parameter vector, one can prove that hogwild training behaves like standard synchronous SGD.

13.5 Regularization

In Sec. 13.4 we discussed computational issues associated with training (large) neural networks. In this section, we discuss statistical issues. In particular, we focus on ways to avoid overfitting. This is crucial, since large neural networks can easily have millions of parameters.

13.5.1 Early stopping

Perhaps the simplest way to prevent overfitting is called **early stopping**, which refers to the heuristic of stopping the training procedure when the error on the validation set first starts to increase (see Fig. 4.8 for an example). This method works because we are restricting the ability of the optimization algorithm to transfer information from the training examples to the parameters, as explained in [AS19].



(a) (b)

Figure 13.21: (a) A deep but sparse neural network. The connections are pruned using ℓ_1 regularization. At each level, nodes numbered 0 are clamped to 1, so their outgoing weights correspond to the offset/bias terms. (b) Predictions made by the model on the training set. Generated by code at figures.probml.ai/book1/13.21.

13.5.2 Weight decay

A common approach to reduce overfitting is to impose a prior on the parameters, and then use MAP estimation. It is standard to use a Gaussian prior for the weights $\mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^2\mathbf{I})$ and biases, $\mathcal{N}(\mathbf{b}|\mathbf{0}, \beta^2\mathbf{I})$. (See Sec. 13.4.5.3 for a discussion of this prior.) This is equivalent to ℓ_2 regularization of the objective. In the neural networks literature, this is called **weight decay**, since it encourages small weights, and hence simpler models, as in ridge regression (Sec. 11.3).

13.5.3 Sparse DNNs

Since there are many weights in a neural network, it is often helpful to encourage sparsity. This allows us to perform **model compression**, which can save memory and time. To do this, we can use ℓ_1 regularization (as in Sec. 11.5), or ARD (as in Sec. 11.6.3), or several other methods. (see [Hoe+21] for a recent review.)

As a simple example, Fig. 13.21 shows a 5 layer MLP which has been fit to some 1d regression data using an ℓ_1 regularizer on the weights. We see that the resulting graph topology is sparse. Of course, many other approaches to sparse estimation are possible

Despite the intuitive appeal of sparse topology, in practice these methods are not widely used, since modern GPUs are optimized for *dense* matrix multiplication, and there are few computational benefits to sparse weight matrices. However, if we use methods that encourage *group* sparsity, we can prune out whole layers of the model. This results in *block sparse* weight matrices, which can result in speedups and memory savings (see e.g., [Sca+17; Wen+16; MAV17; LUW17]).

13.5.4 Dropout

Suppose that we randomly (on a per-example basis) turn off all the outgoing connections from each neuron with probability p, as illustrated in Fig. 13.22. This technique is known as **dropout** [Sri+14].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

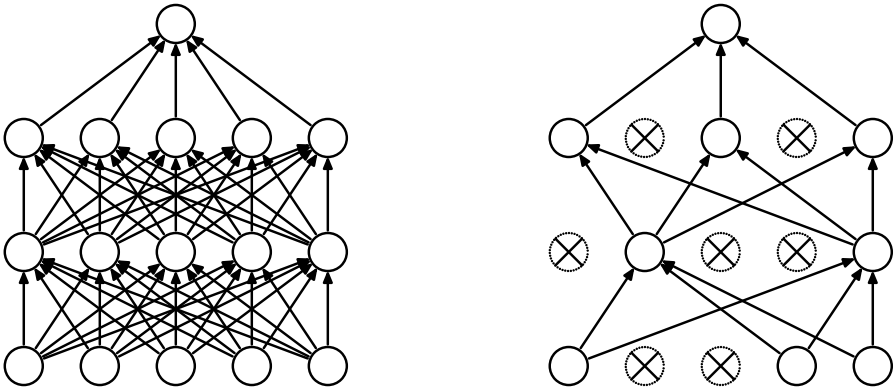


Figure 13.22: Illustration of dropout. (a) A standard neural net with 2 hidden layers. (b) An example of a thinned net produced by applying dropout with $p_0 = 0.5$. Units that have been dropped out are marked with an x. From Figure 1 of [Sri+14]. Used with kind permission of Geoff Hinton.

Dropout can dramatically reduce overfitting and is very widely used. Intuitively, the reason dropout works well is that it prevents complex co-adaptation of the hidden units. In other words, each unit must learn to perform well even if some of the other units are missing at random. This prevents the units from learning complex, but fragile, dependencies on each other.[5] A more formal explanation, in terms of Gaussian scale mixture priors, can be found in [NHLS19].

We can view dropout as estimating a noisy version of the weights, $\theta_{lij} = w_{lij}\epsilon_{li}$, where $\epsilon_{li} \sim \text{Ber}(1-p)$ is a Bernoulli noise term. (So if we sample $\epsilon_{li} = 0$, then all of the weights going out of unit i in layer $l-1$ into any j in layer l will be set to 0.) At test time, we usually turn th enoise off; this is equivalent to setting $\epsilon_{li} = 1$. To derive the corresponding estimate for the non-noisy weights, we should use $w_{lij} = \theta_{lij}/\mathbb{E}[\epsilon_{li}]$, so the weights have the same expectation at test time as they did during training. For Bernoulli noise, we have $\mathbb{E}[\epsilon] = 1-p$, so we should divide by $1-p$ before making predictions.

We can, however, use dropout at test time if we wish. The result is an **ensemble** of networks, each with slightly different sparse graph structures. This is called **Monte Carlo dropout** [GG16; KG17], and has the form

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(\mathbf{y}|\mathbf{x}, \hat{\mathbf{W}}\epsilon^s + \hat{\mathbf{b}}) \tag{13.112}$$

where S is the number of samples, and we write $\hat{\mathbf{W}}\epsilon^s$ to indicate that we are multiplying all the estimated weight matrices by a sampled noise vector. This can sometimes provide a good approximation to the Bayesian posterior predictive distribution $p(\mathbf{y}|\mathbf{x}, \mathcal{D})$, especially if the noise rate is optimized [GHK17].

5. Geoff Hinton, who invented dropout, said he was inspired by a talk on sexual reproduction, which encourages genes to be individually useful (or at most depend on a small number of other genes), even when combined with random other genes.

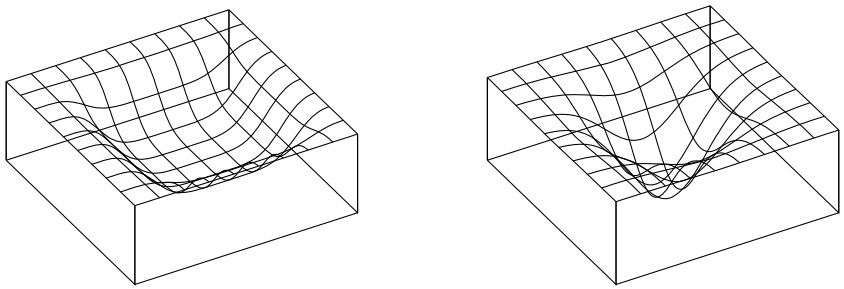


Figure 13.23: Flat vs sharp minima. From Figures 1 and 2 of [HS97a]. Used with kind permission of Jürgen Schmidhuber.

13.5.5 Bayesian neural networks

Modern DNNs are usually trained using a (penalized) maximum likelihood objective to find a single setting of parameters. However, with large models, there are often many more parameters than data points, so there may be multiple possible models which fit the training data equally well, yet which generalize in different ways. It is often useful to capture the induced uncertainty in the posterior predictive distribution. This can be done by *marginalizing out* the parameters by computing

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}) d\boldsymbol{\theta} \tag{13.113}$$

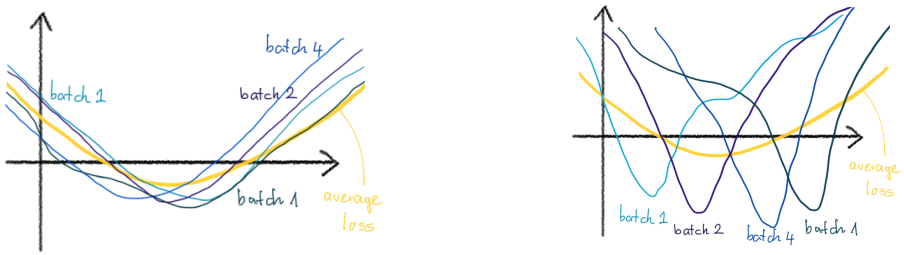
The result is known as a **Bayesian neural network** or **BNN**. It can be thought of as an infinite ensemble of differently weight neural networks. By marginalizing out the parameters, we can avoid overfitting [Mac95]. Bayesian marginalization is challenging for large neural networks, but also can lead to significant performance gains [WI20]. For more details on the topic of **Bayesian deep learning**, see the sequel to this book, [Mur22].

13.5.6 Regularization effects of (stochastic) gradient descent *

Some optimization methods (in particular, second-order batch methods) are able to find "needles in haystacks", corresponding to narrow but deep "holes" in the loss landscape, corresponding to parameter settings with very low loss. These are known as **sharp minima**, see Fig. 13.23(right). From the point of view of minimizing the empirical loss, the optimizer has done a good job. However, such solutions generally correspond to a model that has overfit the data. It is better to find points that correspond to **flat minima**, as shown in Fig. 13.23(left); such solutions are more robust and generalize better. To see why, note that flat minima correspond to regions in parameter space where there is a lot of posterior uncertainty, and hence samples from this region are less able to precisely memorize irrelevant details about the training set [AS17]. SGD often finds such flat minima by virtue of the addition of noise, which prevents it from "entering" narrow regions of the loss landscape (see e.g., [SL18]). This is called **implicit regularization**. (It is also possible to explicitly encourage SGD to find such flat minima, using **sharpness aware minimization** [For+21].)

Of course, the loss landscape depends not just on the parameter values, but also on the data. Since we usually cannot afford to do full-batch gradient descent, we will get a set of loss curves, one per

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 13.24: Each curve shows how the loss varies across parameter values for a given minibatch. (a) A stable local minimum. (b) An unstable local minimum. From https://www.inference.vc/notes-on-the-origin-of-implicit-regularization-in-stochastic-gradient-descent/. Used with kind permission of Ferenc Huszar

minibatch. If each one of these curves corresponds to a wide basin, as shown in Fig. 13.24a, we are at a point in parameter space that is robust to perturbations, and will likely generalize well. However, if the overall wide basin is the result of averaging over many different narrow basisns, as shown in Fig. 13.24b, the resulting estimate will likely generalize less well.

This can be formalized using the analysis in [Smi+21; BD21]. Specifically, they consider continuous time gradient flow which approximates the behavior of (S)GD. In [BD21], they consider full-batch GD, and show that the flow has the form $\dot{\mathbf{w}} = -\nabla_{\mathbf{w}} \tilde{\mathcal{L}}_{GD}(\mathbf{w})$, where

$$\tilde{\mathcal{L}}_{GD}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\epsilon}{4}||\nabla \mathcal{L}(\mathbf{w})||^2 \tag{13.114}$$

where $\mathcal{L}(\mathbf{w})$ is the original loss, ϵ is the learning rate, and the second term is an implicit regularization term that penalizes solutions with large gradients (high curvature).

In [Smi+21], they extend this analysis to the SGD case. They show that the flow has the form $\dot{\mathbf{w}} = -\nabla_{\mathbf{w}} \tilde{\mathcal{L}}_{SGD}(\mathbf{w})$, where

$$\tilde{\mathcal{L}}_{SGD}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\epsilon}{4} \sum_{k=1}^{m} ||\nabla \mathcal{L}_k(\mathbf{w})||^2 \tag{13.115}$$

where m is the number of minibatches, and $\mathcal{L}_k(\mathbf{w})$ is the loss on the k'th such minibatch. Comparing this to the full-batch GD loss, we see

$$\tilde{\mathcal{L}}_{SGD}(\mathbf{w}) = \tilde{\mathcal{L}}_{GD}(\mathbf{w}) + \frac{\epsilon}{4} \sum_{k=1}^{m} ||\nabla \mathcal{L}_k(\mathbf{w}) - \mathcal{L}(\mathbf{w})||^2 \tag{13.116}$$

The second term estimates the variance of the minibatch gradients, which is a measure of stability, and hence of generalization ability.

The above analysis shows that SGD not only has computational advantages (since it is faster than full-batch GD or second-order methods), but also statistical advantages.

13.6 Other kinds of feedforward networks

13.6.1 Radial basis function networks

Consider a 1 layer neural net where the hidden layer is given by the feature vector

$$\boldsymbol{\phi}(\mathbf{x}) = [\mathcal{K}(\mathbf{x}, \boldsymbol{\mu}_1), \ldots, \mathcal{K}(\mathbf{x}, \boldsymbol{\mu}_K)] \tag{13.117}$$

where $\boldsymbol{\mu}_k \in \mathcal{X}$ are a set of K **centroids** or **exemplars**, and $\mathcal{K}(\mathbf{x}, \boldsymbol{\mu}) \geq 0$ is a **kernel function**. We describe kernel functions in detail in Sec. 17.2. Here we just give an example, namely the **Gaussian kernel**

$$\mathcal{K}_{\text{gauss}}(\mathbf{x}, \mathbf{c}) \triangleq \exp\left(-\frac{1}{2\sigma^2}||\mathbf{c} - \mathbf{x}||_2^2\right) \tag{13.118}$$

The parameter σ is known as the **bandwidth** of the kernel. Note that this kernel is shift invariant, meaning it is only a function of the distance $r = ||\mathbf{x} - \mathbf{c}||_2$, so we can equivalently write this as

$$\mathcal{K}_{\text{gauss}}(r) \triangleq \exp\left(-\frac{1}{2\sigma^2}r^2\right) \tag{13.119}$$

This is therefore called a **radial basis function kernel** or **RBF kernel**.

A 1 layer neural net in which we use Eq. (13.117) as the hidden layer, with RBF kernels, is called an **RBF network** [BL88]. This has the form

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = p(y|\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})) \tag{13.120}$$

where $\boldsymbol{\theta} = (\boldsymbol{\mu}, \mathbf{w})$. If the centroids $\boldsymbol{\mu}$ are fixed, we can solve for the optimal weights $\mathbf{w}$ using (regularized) least squares, as discussed in Chapter 11. If the centroids are unknown, we can estimate them by using an unsupervised clustering method, such as K-means (Sec. 21.3). Alternatively, we can associate one centroid per data point in the training set, to get $\boldsymbol{\mu}_n = \mathbf{x}_n$, where now $K = N$. This is an example of a **non-parametric model**, since the number of parameters grows (in this case linearly) with the amount of data, and is not independent of N. If $K = N$, the model can perfectly interpolate the data, and hence may overfit. However, by ensuring that the output weight vector $\mathbf{w}$ is sparse, the model will only use a finite subset of the input examples; this is called a **sparse kernel machine**, and will be discussed in more detail in Sec. 17.6.1 and Sec. 17.5. Another way to avoid overfitting is to adopt a Bayesian approach, by integrating out the weights $\mathbf{w}$; this gives rise to a model called a **Gaussian process**, which will be discussed in more detail in Sec. 17.3.

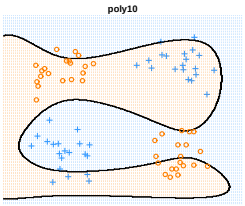
13.6.1.1 RBF network for regression

We can use RBF networks for regression by defining $p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}), \sigma^2)$. For example, Figure 13.26 shows a 1d data set fit with $K = 10$ uniformly spaced RBF prototypes, but with the bandwidth ranging from small to large. Small values lead to very wiggly functions, since the predicted function value will only be non-zero for points $\mathbf{x}$ that are close to one of the prototypes $\boldsymbol{\mu}_k$. If the bandwidth is very large, the design matrix reduces to a constant matrix of 1's, since each point is equally close to every prototype; hence the corresponding function is just a straight line.

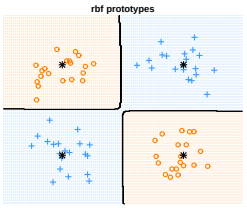
Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

(a)



(b)



(c)

Figure 13.25: (a) xor truth table. (b) Fitting a linear logistic regression classifier using degree 10 polynomial expansion. (c) Same model, but using an RBF kernel with centroids specified by the 4 black crosses. Generated by code at figures.probml.ai/book1/13.25.

13.6.1.2 RBF network for classification

We can use RBF networks for binary classification by defining $p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}))$. As an example, consider the data coming from the exclusive or function. This is a binary-valued function of two binary inputs. Its truth table is shown in Figure 13.25(a). In Figure 13.1(b), we have shown some data labeled by the xor function, but we have **jittered** the points to make the picture clearer.[6] We see we cannot separate the data even using a degree 10 polynomial. However, using an RBF kernel and just 4 prototypes easily solves the problem as shown in Figure 13.1(c).

13.6.2 Mixtures of experts

When considering regression problems, it is common to assume a unimodal output distribution, such as a Gaussian or Student distribution, where the mean and variance is some function of the input, i.e.,

$$p(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}|f_\mu(\mathbf{x}), \text{diag}(\sigma_+(f_\sigma(\mathbf{x})))) \tag{13.121}$$

where the f functions may be MLPs (possibly with some shared hidden units, as in Fig. 13.7. However, this will not work well for **one-to-many functions**, in which each input can have multiple possible outputs.

Fig. 13.27a gives a simple example of such a function. We see that in the middle of the plot there are certain x values for which there are two equally probable y values. There are many real world problems of this form, e.g., 3d pose prediction of a person from a single image [Bo+08], colorization of a black and white image [Gua+17], predicting future frames of a video sequence [VT17], etc. Any model which is trained to maximize likelihood using a unimodal output density — even if the model is a flexible nonlinear model, such as neural network — will work poorly on one-to-many functions such as these, since it will just produce a blurry average output.

To prevent this problem of regression to the mean, we can use a **conditional mixture model**. That is, we assume the output is a weighted mixture of K different outputs, corresponding to different

6. Jittering is a common visualization trick in statistics, wherein points in a plot/display that would otherwise land on top of each other are dispersed with uniform additive noise.

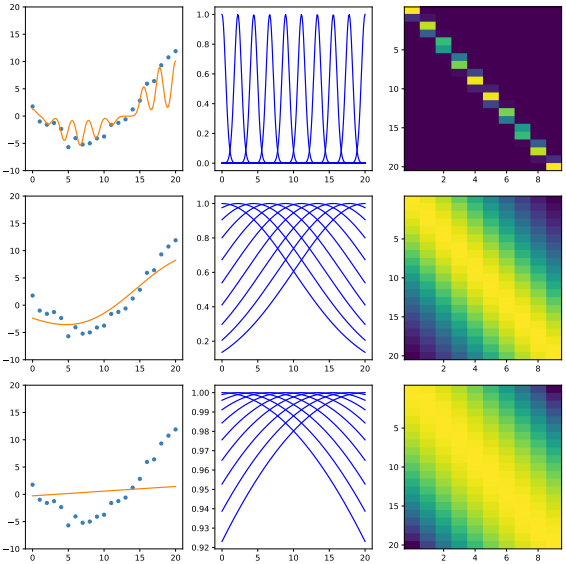


Figure 13.26: Linear regression using 10 equally spaced RBF basis functions in 1d. Left column: fitted function. Middle column: basis functions evaluated on a grid. Right column: design matrix. Top to bottom we show different bandwidths for the kernel function: $\sigma = 0.5, 10, 50$. Generated by code at figures.probml.ai/book1/13.26.

modes of the output distribution for each input $\mathbf{x}$. In the Gaussian case, this becomes

$$p(\mathbf{y}|\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{y}|\mathbf{x}, z=k)p(z=k|\mathbf{x}) \tag{13.122}$$

$$p(\mathbf{y}|\mathbf{x}, z=k) = \mathcal{N}(\mathbf{y}|f_{\mu,k}(\mathbf{x}), \text{diag}(f_{\sigma,k}(\mathbf{x}))) \tag{13.123}$$

$$p(z=k|\mathbf{x}) = \text{Cat}(z|\mathcal{S}(f_z(\mathbf{x}))) \tag{13.124}$$

Here $f_{\mu,k}$ predicts the mean of the k'th Gaussian, $f_{\sigma,k}$ predicts its variance terms, and f_z predicts which mixture component to use. This model is called a **mixture of experts** (**MoE**) [Jac+91; JJ94; YWG12; ME14]. The idea is that the k'th submodel $p(\mathbf{y}|\mathbf{x}, z=k)$ is considered to be an "expert" in a certain region of input space. The function $p(z=k|\mathbf{x})$ is called a **gating function**, and decides which expert to use, depending on the input values. By picking the most likely expert for a given input $\mathbf{x}$, we can "activate" just a subset of the model. This is an example of **conditional computation**, since we decide what expert to run based on the results of earlier computations from the gating network [Sha+17].

We can train this model using SGD, or using the EM algorithm (see Sec. 8.7.3 for details on the latter method).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

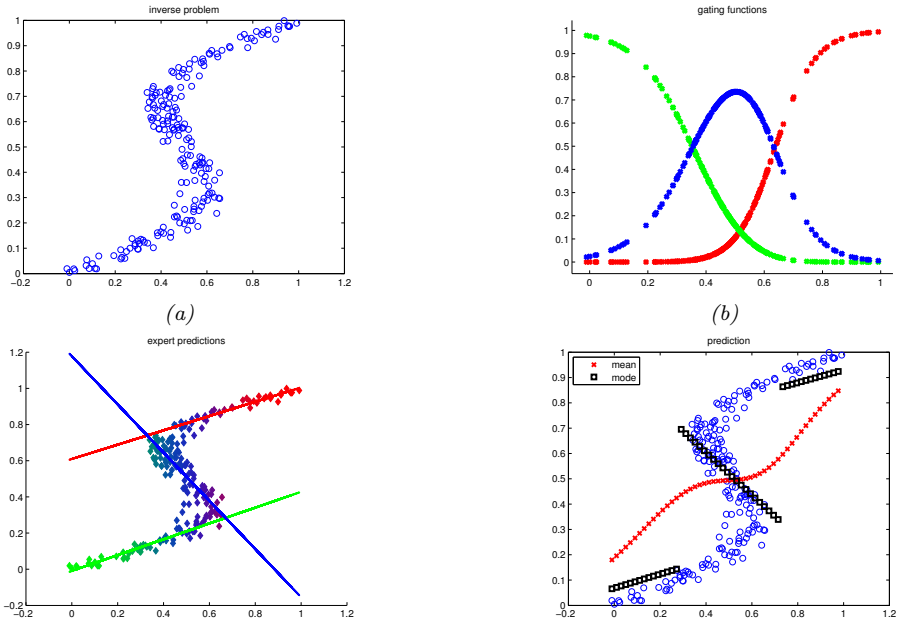


Figure 13.27: (a) Some data from a one-to-many function. (b) The responsibilities of each expert for the input domain. (c) Prediction of each expert. (d) Overeall prediction. Mean is red cross, mode is black square. Adapted from Figures 5.20 and 5.21 of [Bis06]. Generated by code at figures.probml.ai/book1/13.27.

13.6.2.1 Mixture of linear experts

In this section, we consider a simple example in which we use linear regression experts and a linear classification gating function, i.e., the model has the form:

$$p(y|\mathbf{x}, z = k, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}_k^\top \mathbf{x}, \sigma_k^2) \tag{13.125}$$

$$p(z|\mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(z|\mathcal{S}(\mathbf{V}\mathbf{x})) \tag{13.126}$$

The individual weighting term $p(z = k|\mathbf{x})$ is called the **responsibility** for expert k for input $\mathbf{x}$. In Fig. 13.27b, we see how the gating networks softly partitions the input space amongst the $K = 3$ experts.

Each expert $p(y|\mathbf{x}, z = k)$ corresponds to a linear regression model with different parameters. These are shown in Fig. 13.27c.

If we take a weighted combination of the experts as our output, we get the red curve in Fig. 13.27a, which is clearly is a bad predictor. If instead we only predict using the most active expert (i.e., the one with the highest responsibility), we get the discontinuous black curve, which is a much better predictor.

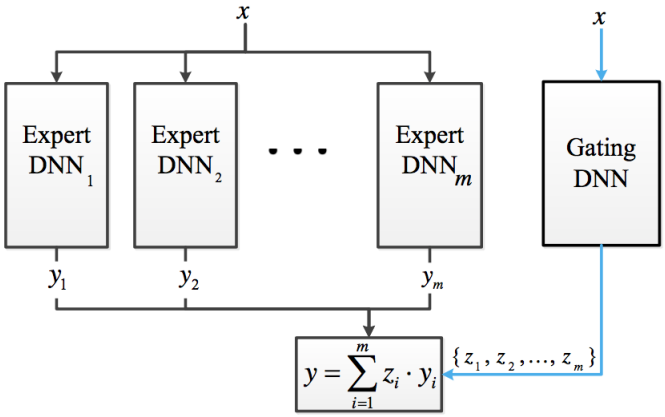


Figure 13.28: Deep MOE with m experts, represented as a neural network. From Figure 1 of [CGG17]. Used with kind permission of Jacob Goldberger.

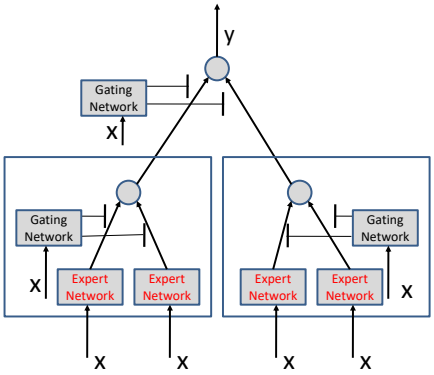


Figure 13.29: A 2-level hierarchical mixture of experts as a neural network. The top gating network chooses between the left and right expert, shown by the large boxes; the left and right experts themselves choose between their left and right sub-experts.

13.6.2.2 Mixture density networks

The gating function and experts can be be any kind of conditional probabilistic model, not just a linear model. If we make them both DNNs, then resulting model is called a **mixture density network** (**MDN**) [Bis94; ZS14] or a **deep mixture of experts** [CGG17]. See Fig. 13.28 for a sketch of the model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

13.6.2.3 Hierarchical MOEs

If each expert is itself an MoE model, the resulting model is called a **hierarchical mixture of experts** [JJ94]. See Fig. 13.29 for an illustration of such a model with a two level hierarchy.

An HME with L levels can be thought of as a "soft" decision tree of depth L, where each example is passed through every branch of the tree, and the final prediction is a weighted average. (We discuss decision trees in Sec. 18.1.)

13.7 Exercises

Exercise 13.1 [Backpropagation for a MLP]

(Based on an exercise by Kevin Clark.)

Consider the following classification MLP with one hidden layer:

$$\mathbf{x} = \text{input} \in \mathbb{R}^D \tag{13.127}$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^K \tag{13.128}$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) \in \mathbb{R}^K \tag{13.129}$$

$$\mathbf{a} = \mathbf{V}\mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^C \tag{13.130}$$

$$\mathcal{L} = \text{CrossEntropy}(\mathbf{y}, \mathcal{S}(\mathbf{a})) \in \mathbb{R} \tag{13.131}$$

where $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{b}_1 \in \mathbb{R}^K$, $\mathbf{W} \in \mathbb{R}^{K \times D}$, $\mathbf{b}_2 \in \mathbb{R}^C$, $\mathbf{V} \in \mathbb{R}^{C \times K}$, where D is the size of the input, K is the number of hidden units, and C is the number of classes. Show that the gradients for the parameters and input are as follows:

$$\nabla_{\mathbf{V}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{V}}\right]_{1,:} = \mathbf{u}_2 \mathbf{h}^\top \in \mathbb{R}^{C \times K} \tag{13.132}$$

$$\nabla_{\mathbf{b}_2} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2}\right)^\top = \mathbf{u}_2 \in \mathbb{R}^C \tag{13.133}$$

$$\nabla_{\mathbf{W}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{W}}\right]_{1,:} = \mathbf{u}_1 \mathbf{x}^\top \in \mathbb{R}^{K \times D} \tag{13.134}$$

$$\nabla_{\mathbf{b}_1} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1}\right)^\top = \mathbf{u}_1 \in \mathbb{R}^K \tag{13.135}$$

$$\nabla_{\mathbf{x}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{x}}\right)^\top = \mathbf{W}^\top \mathbf{u}_1 \in \mathbb{R}^D \tag{13.136}$$

where the gradients of the loss wrt the two layers (logit and hidden) are given by the following:

$$\mathbf{u}_2 = \nabla_{\mathbf{a}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}}\right)^\top = (\mathbf{p} - \mathbf{y}) \in \mathbb{R}^C \tag{13.137}$$

$$\mathbf{u}_1 = \nabla_{\mathbf{z}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}}\right)^\top = (\mathbf{V}^\top \mathbf{u}_2) \odot H(\mathbf{z}) \in \mathbb{R}^K \tag{13.138}$$

with H is the Heaviside function. Note that, in our notation, the gradient (which has the same shape as the variable with respect to which we differentiate) is equal to the Jacobian's transpose when the variable is a vector and to the first slice of the Jacobian when the variable is a matrix.

14 Neural networks for images

14.1 Introduction

In Chapter 13, we discussed multilayered perceptrons (MLPs) as a way to learn functions mapping "unstructured" input vectors $\mathbf{x} \in \mathbb{R}^D$ to outputs. In this chapter, we extend this to the case where the input $\mathbf{x}$ has 2d spatial structure. (Similar ideas apply to 1d temporal structure, or 3d spatio-temporal structure.)

To see why it is not a good idea to apply MLPs directly to image data, recall that the core operation in an MLP at each hidden layer is computing the activations $\mathbf{z} = \varphi(\mathbf{W}\mathbf{x})$, where $\mathbf{x}$ is the input to a layer, $\mathbf{W}$ are the weights, and $\varphi()$ is the nonlinear activation function. Thus the j'th element of the hidden layer has value $z_j = \varphi(\mathbf{w}_j^\top \mathbf{x})$. We can think of this inner product operation as comparing the input $\mathbf{x}$ to a learned template or pattern $\mathbf{w}_j$; if the match is good (large positive inner product), the activation of that unit will be large (assuming a ReLU nonlinearity), signalling that the j'th pattern is present in the input.

However, this does not work well if the input is a variable-sized image, $\mathbf{x} \in \mathbb{R}^{WHC}$, where W is the width, H is the height, and C is the number of input **channels** (e.g., $C = 3$ for RGB color). The problem is that we would need to learn a different-sized weight matrix $\mathbf{W}$ for every size of input image. In addition, even if the input was fixed size, the number of parameters needed would be prohibitive for reasonably sized images, since the weight matrix would have size $(W \times H \times C) \times D$, where D is the number of outputs (hidden units). The final problem is that a pattern that occurs in one location may not be recognized when it occurs in a different location — that is, the model may not exhibit **translation invariance** — because the weights are not shared across locations (see Fig. 14.1).

To solve these problems, we will use **convolutional neural networks** (**CNN**s), in which we replace matrix multiplication with a convolution operation. We explain this in detail in Sec. 14.2, but the basic idea is to divide the input into overlapping 2d **image patches**, and to compare each patch with a set of small weight matrices, or **filters**, which represent parts of an object; this is illustrated in Fig. 14.2. We can think of this as a form of **template matching**. We will learn these templates from data, as we explain below. Because the templates are small (often just 3x3 or 5x5), the number of parameters is significantly reduced. And because we use convolution to do the template matching, instead of matrix multiplication, the model will be translationally invariant. This is useful for tasks such as image classification, where the goal is to classify if an object is present, regardless of its location.

CNNs have many other applications besides image classification, as we will discuss later in this chapter. They can also be applied to 1d inputs (see Sec. 15.3) and 3d inputs; however, we mostly

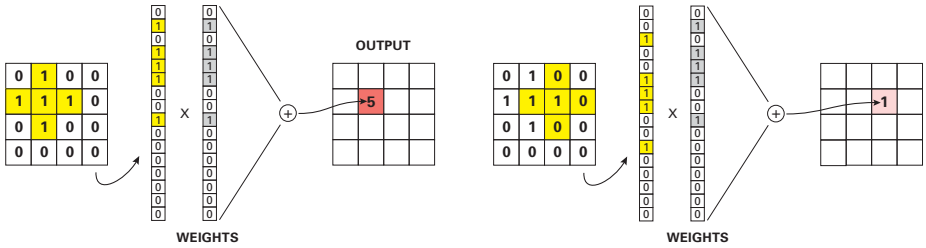


Figure 14.1: Detecting patterns in 2d images using unstructured MLPs does not work well, because the method is not translation invariant. We can design a weight vector to act as a **matched filter** *for detecting the desired cross-shape. This will give a strong response of 5 if the object is on the left, but a weak response of 1 if the object is shifted over to the right. Adapted from Figure 7.16 of [SAV20].*

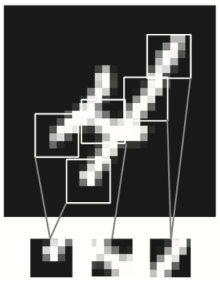


Figure 14.2: We can classify a digit by looking for certain discriminative features (image templates) occuring in the correct (relative) locations. From Figure 5.1 of [Cho17]. Used with kind permission of Francois Chollet.

focus on the 2d case in this chapter.

14.2 Common layers

In this section, we discuss the basics of CNNs.

14.2.1 Convolutional layers

We start by describing the basics of convolution in 1d, and then in 2d, and then describe how they are used as a key component of CNNs.

14.2.1.1 Convolution in 1d

The **convolution** between two functions, say $f, g : \mathbb{R}^D \to \mathbb{R}$, is defined as

$$[f \circledast g](\mathbf{z}) = \int_{\mathbb{R}^D} f(\mathbf{u}) g(\mathbf{z} - \mathbf{u}) d\mathbf{u} \tag{14.1}$$

Now suppose we replace the functions with finite-length vectors, which we can think of as functions defined on a finite set of points. For example, suppose f is evaluated at the points $\{-L, -L +$

-	-	1	2	3	4	-	-	
7	6	5	-	-	-	-	-	$z_0 = x_0 w_0 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_0 w_1 + x_1 w_0 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_2 w_2 + x_3 w_1 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_3 w_2 = 28$

Figure 14.3: Discrete convolution of $\mathbf{x} = [1, 2, 3, 4]$ *with* $\mathbf{w} = [5, 6, 7]$ *to yield* $\mathbf{z} = [5, 16, 34, 52, 45, 28]$. *We see that this operation consists of "flipping"* $\mathbf{w}$ *and then "dragging" it over* $\mathbf{x}$, *multiplying elementwise, and adding up the results.*

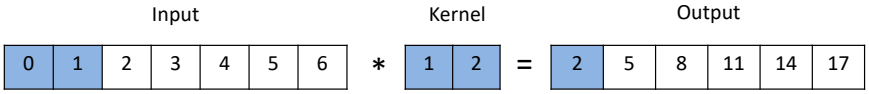


Figure 14.4: 1d cross correlation. Adapted from Figure 15.3.2 of [Zha+20]. Used with kind permission of Aston Zhang.

$1, \ldots, 0, 1, \ldots, L\}$ to yield the weight vector (also called a **filter** or **kernel**) $w_{-L} = f(-L)$ up to $w_L = f(L)$. Now let g be evaluated at points $\{-N, \ldots, N\}$ to yield the feature vector $x_{-N} = g(-N)$ up to $x_N = g(N)$. Then the above equation becomes

$$[\mathbf{w} \circledast \mathbf{x}](i) = w_{-L} x_{i+L} + \cdots + w_{-1} x_{i+1} + w_0 x_i + w_1 x_{i-1} + \cdots + w_L x_{i-L} \tag{14.2}$$

(We discuss boundary conditions (edge effects) later on.) We see that we "flip" the weight vector $\mathbf{w}$ (since indices of $\mathbf{w}$ are reversed), and then "drag" it over the $\mathbf{x}$ vector, summing up the local windows at each point, as illustrated in Fig. 14.3.

There is a very closely related operation, in which we do not flip $\mathbf{w}$ first:

$$[\mathbf{w} * \mathbf{x}](i) = w_{-L} x_{i-L} + \cdots + w_{-1} x_{i-1} + w_0 x_i + w_1 x_{i+1} + \cdots + w_L x_{i+L} \tag{14.3}$$

This is called **cross correlation**; If the weight vector is symmetric, as is often the case, then cross correlation and convolution are the same. In the deep learning literature, the term "convolution" is usually used to mean cross correlation; we will follow this convention.

We can also evaluate the weights $\mathbf{w}$ on domain $\{0, 1, \ldots, L-1\}$ and the features $\mathbf{x}$ on domain $\{0, 1, \ldots, N-1\}$, to eliminate negative indices. Then the above equation becomes

$$[\mathbf{w} \circledast \mathbf{x}](i) = \sum_{u=0}^{L-1} w_u x_{i+u} \tag{14.4}$$

See Fig. 14.4 for an example.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

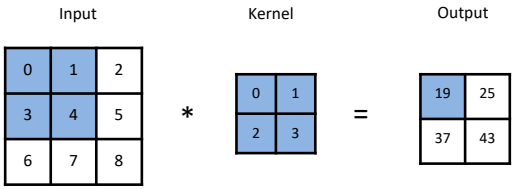


Figure 14.5: Illustration of 2d cross correlation. Generated by code at figures.probml.ai/book1/14.5. Adapted from Figure 6.2.1 of [Zha+20].

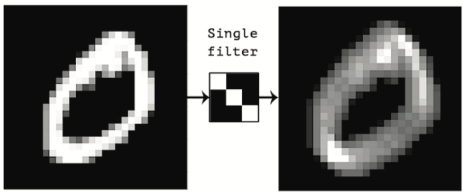


Figure 14.6: Convolving a 2d image (left) with a 3×3 filter (middle) produces a 2d response map (right). The bright spots of the response map correspond to locations in the image which contain diagonal lines sloping down and to the right. From Figure 5.3 of [Cho17]. Used with kind permission of Francois Chollet.

14.2.1.2 Convolution in 2d

In 2d, Eq. (14.4) becomes

$$[\mathbf{W} \circledast \mathbf{X}](i,j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u,j+v} \tag{14.5}$$

where the 2d filter $\mathbf{W}$ has size $H \times W$. For example, consider convolving a 3×3 input $\mathbf{X}$ with a 2×2 kernel $\mathbf{W}$ to compute a 2×2 output $\mathbf{Y}$:

$$\mathbf{Y} = \begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \circledast \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix} \tag{14.6}$$

$$= \begin{pmatrix} (w_1 x_1 + w_2 x_2 + w_3 x_4 + x_4 x_5) & (w_1 x_2 + w_2 x_3 + w_3 x_5 + x_4 x_6) \\ (w_1 x_4 + w_2 x_5 + w_3 x_7 + x_4 x_8) & (w_1 x_5 + w_2 x_6 + w_3 x_8 + x_4 x_9) \end{pmatrix} \tag{14.7}$$

See Fig. 14.5 for a visualization of this process.

We can think of 2d convolution as **template matching**, since the output at a point (i,j) will be large if the corresponding image patch centered on (i,j) is similar to $\mathbf{W}$. If the template $\mathbf{W}$ corresponds to an oriented edge, then convolving with it will cause the output **heat map** to "light up" in regions that contain edges that match that orientation, as shown in Fig. 14.6. More generally, we can think of convolution as a form of **feature detection**. The resulting output $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X}$ is therefore called a **feature map**.

14.2.1.3 Convolution as matrix-vector multiplication

Since convolution is a linear operator, we can represent it by matrix multiplication. For example, consider Eq. (14.7). We can rewrite this as matrix-vector mutiplication by flattening the 2d matrix $\mathbf{X}$ into a 1d vector $\mathbf{x}$, and multiplying by a Toeplitz-like matrix $\mathbf{C}$ derived from the kernel $\mathbf{W}$, as follows:

$$\mathbf{y} = \mathbf{C}\mathbf{x} = \left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array}\right) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix} \tag{14.8}$$

$$= \begin{pmatrix} w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5 \\ w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6 \\ w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8 \\ w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9 \end{pmatrix} \tag{14.9}$$

We can recover the 2×2 output by reshaping the 4×1 vector $\mathbf{y}$ back to $\mathbf{Y}$. (See code.probml.ai/book1/conv2d_torch for a demo.)

Thus we see that CNNs are like MLPs where the weight matrices have a special sparse structure, and the elements are tied across spatial locations. This implements the idea of translation invariance, and massively reduces the number of parameters compared to a weight matrix in a standard fully connected or dense layer, as used in MLPs.

14.2.1.4 Boundary conditions and padding

In Eq. (14.7), we saw that convolving a 3×3 image with a 2×2 filter resulted in a 2×2 output. In general, convolving a $f_h \times f_w$ filter over an image of size $x_h \times x_w$ produces an output of size $(x_h - f_h + 1) \times (x_w - f_w + 1)$; this is called **valid convolution**, since we only apply the filter to "valid" parts of the input, i.e., we don't let it "slide off the ends". If we want the output to have the same size as the input, we can use **zero-padding**, which means we add a border of 0s to the image, as illustrated in Fig. 14.7. This is called **same convolution**.

In general, if the input has size $x_h \times x_w$, we use a kernel of size $f_h \times f_w$, we use zero padding on each side of size p_h and p_w, then the output has the following size [DV16]:

$$(x_h + 2p_h - f_h + 1) \times (x_w + 2p_w - f_w + 1) \tag{14.10}$$

For example, consider Fig. 14.8a. We have $p = 1$, $f = 3$, $x_h = 5$ and $x_w = 7$, so the output has size

$$(5 + 2 - 3 + 1) \times (7 + 2 - 3 + 1) = 5 \times 7 \tag{14.11}$$

If we set $2p = f - 1$, then the output will have the same size as the input.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

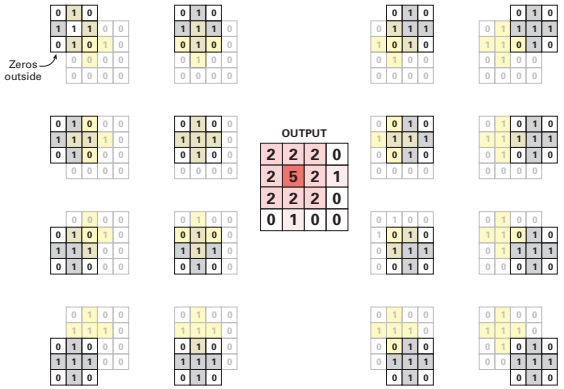


Figure 14.7: Same-convolution (using zero-padding) ensures the output is the same size as the input. Adapted from Figure 8.3 of [SAV20].

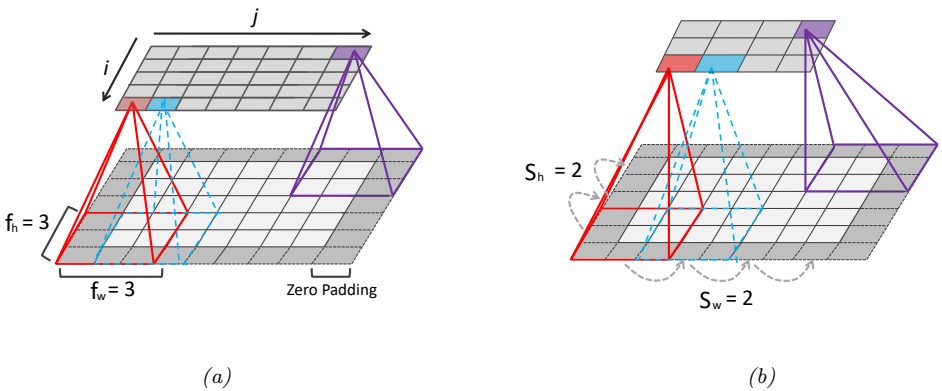


Figure 14.8: Illustration of padding and strides in 2d convolution. (a) We apply "same convolution" to a 5×7 input (with zero padding) using a 3×3 filter to create a 5×7 output. (b) Now we use a stride of 2, so the output has size 3×4. Adapted from Figures 14.3–14.4 of [Gér19].

14.2.1.5 Strided convolution

Since each output pixel is generated by a weighted combination of inputs in its **receptive field** (based on the size of the filter), neighoring outputs will be very similar in value, since their inputs are overlapping. We can reduce this redundancy (and speedup computation) by skipping every s'th input. This is called **strided convolution**. This is illustrated in Fig. 14.8b, where we convolve a 5×7 image with a 3×3 filter with stride 2 to get a 3×4 output.

In general, if the input has size $x_h \times x_w$, we use a kernel of size $f_h \times f_w$, we use zero padding on each side of size p_h and p_w, and we use strides of size s_h and s_w, then the output has the following

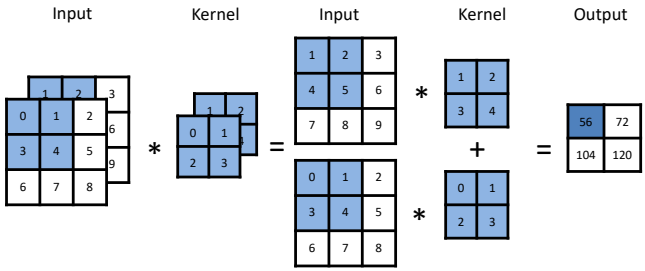


Figure 14.9: Illustration of 2d convolution applied to an input with 2 channels. Generated by code at figures.probml.ai/book1/14.9. Adapted from Figure 6.4.1 of [Zha+20].

size [DV16]:

$$\left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor \tag{14.12}$$

For example, consider Fig. 14.8b, where we set the stride to $s = 2$. Now the output is smaller than the input.

$$\left(\lfloor \frac{5+2-3+2}{2} \rfloor, \lfloor \frac{7+2-3+2}{2} \rfloor \right) = \left(\lfloor \frac{6}{2} \rfloor, \lfloor \frac{4}{1} \rfloor \right) = 3 \times 4 \tag{14.13}$$

14.2.1.6 Multiple input and output channels

In Fig. 14.6, the input was a gray-scale image. In general, the input will have multiple **channels** (e.g., RGB, or hyper-spectral bands for satellite images). We can extend the definition of convolution to this case by defining a kernel for each input channel; thus now $\mathbf{W}$ is a 3d weight matrix or **tensor**. We compute the output by convolving channel c of the input with kernel $\mathbf{W}_{:,:,c}$, and then summing over channels:

$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,sj+v,c} w_{u,v,c} \tag{14.14}$$

where s is the stride (which we assume is the same for both height and width, for simplicity), and b is the bias term. This is illustrated in Fig. 14.9.

Each weight matrix can detect a single kind of feature. We typically want to detect multiple kinds of features, as illustrated in Fig. 14.2. We can do this by making $\mathbf{W}$ into a 4d weight matrix. The filter to detect feature type d in input channel c is stored in $\mathbf{W}_{:,:,c,d}$. We extend the definition of convolution to this case as follows:

$$z_{i,j,d} = b_d + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,sj+v,c} w_{u,v,c,d} \tag{14.15}$$

This is illustrated in Fig. 14.10. Each vertical cylindrical column denotes the set of output features at a given location, $\mathbf{z}_{i,j,1:D}$; this is sometimes called a **hypercolumn**. Each element is a different

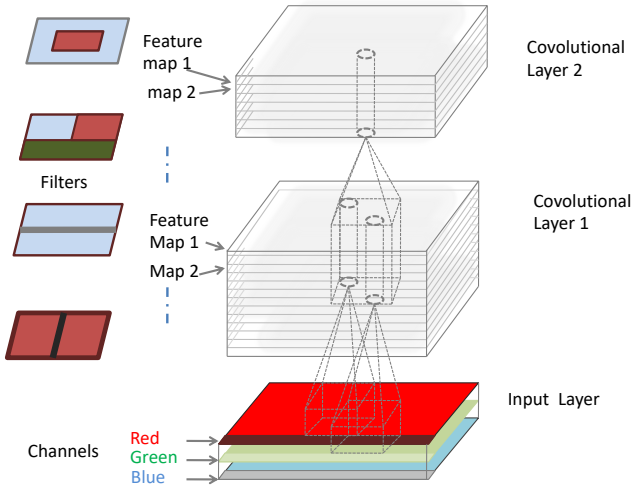


Figure 14.10: Illustration of a CNN with 2 convolutional layers. The input has 3 color channels. The feature maps at internal layers have multiple channels. The cylinders correspond to hypercolumns, which are feature vectors at a certain location. Adapted from Figure 14.6 of [Gér19].

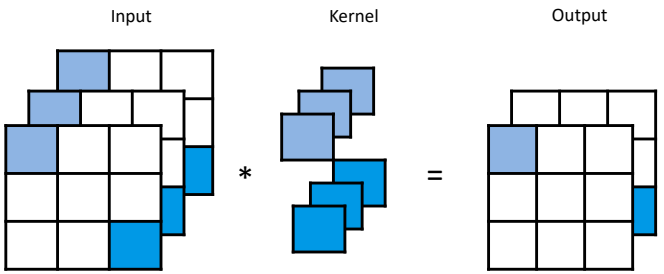


Figure 14.11: Mapping 3 channels to 2 using convolution with a filter of size $1 \times 1 \times 3 \times 2$. *Adapted from Figure 6.4.2 of [Zha+20].*

weighted combination of the C features in the receptive field of each of the feature maps in the layer below.[1]

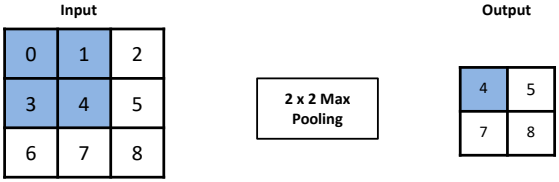


Figure 14.12: Illustration of maxpooling with a 2x2 filter and a stride of 1. Adapted from Figure 6.5.1 of [Zha+20].

14.2.1.7 1×1 (pointwise) convolution

Sometimes we just want to take a weighted combination of the features at a given location, rather than across locations. This can be done using using **1x1 convolution**, also called **pointwise convolution**. This changes the number of channels from C to D, without changing the spatial dimensionality:

$$z_{i,j,d} = b_d + \sum_{c=0}^{C-1} x_{i,j,c} w_{0,0,c,d} \tag{14.16}$$

This can be thought of as a single layer MLP applied to each feature column in parallel.

14.2.2 Pooling layers

Convolution will preserve information about the location of input features (modulo reduced resolution), a property known as **equivariance**. In some case we want to be invariant to the location. For example, when performing image classification, we may just want to know if an object of interest (e.g., a face) is present anywhere in the image.

One simple way to achieve this is called **max pooling**, which just computes the maximum over its incoming values, as illustrated in Fig. 14.12. An alternative is to use **average pooling**, which replaces the max by the mean. In either case, the output neuron has the same response no matter where the input pattern occurs within its receptive field. (Note that we apply pooling to each feature channel independently.)

If we average over all the locations in a feature map, the method is called **global average pooling**. Thus we can convert a $H \times W \times D$ feature map into a $1 \times 1 \times D$ dimensional feature map; this can be reshaped to a D-dimensional vector, which can be passed into a fully connected layer to map it to a C-dimensional vector before passing into a softmax output. The use of global average pooling means we can apply the classifier to an image of any size, since the final feature map will always be converted to a fixed D-dimensional vector before being mapped to a distribution over the C classes.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

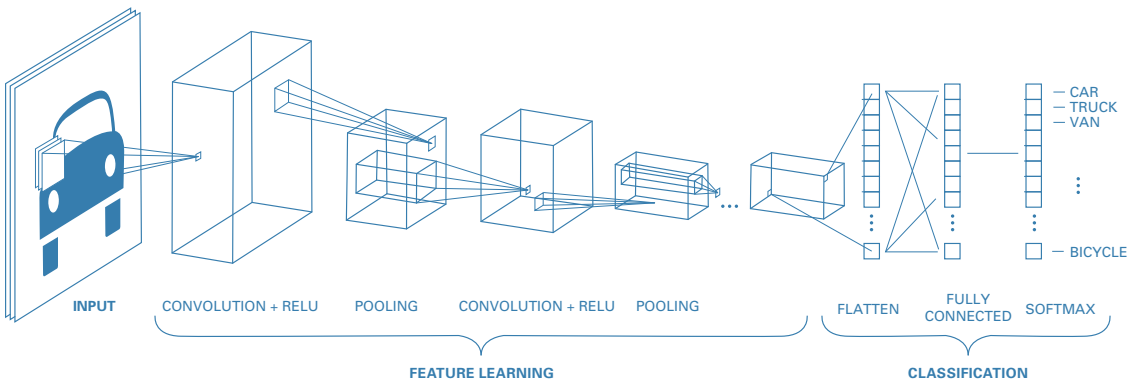


Figure 14.13: A simple CNN for classifying images. Adapted from https://blog.floydhub.com/building-your-first-convnet/.

14.2.3 Putting it altogether

A common design pattern is to create a CNN by alternating convolutional layers with max pooling layers, followed by a final linear classification layer at the end. This is illustrated in Fig. 14.13. (We omit normalization layers in this example, since the model is quite shallow.) This design pattern first appeared in Fukushima's **neocognitron** [Fuk75], and was inspired by Hubel and Wiesel's model of simple and complex cells in the human visual cortex [HW62]. In 1998 Yann LeCun used a similar design in his eponynous **LeNet** model [LeC+98], which used backpropagation and SGD to esitmate the parameters. This design pattern continues to be popular in neurally-inspired models of visual object recognition [RP99], as well as various practical applications (see Sec. 14.3 and Sec. 14.5).

14.2.4 Normalization layers

The basic design in Fig. 14.13 works well for shallow CNNs, but it can be difficult to scale it to deeper models, due to problems with vanishing or exploding gradients, as explained in Sec. 13.4.2. A common solution to this problem is to add extra layers to the model, to standardize the statistics of the hidden units (i.e., to ensure they are zero mean and unit variance), just like we do to the inputs of many models. We discuss various kinds of **normalization layer** below.

14.2.4.1 Batch normalization

The most popular normalization layer called **batch normalization** (**BN**) [IS15]. This ensures the distribution of the activations within a layer is zero mean and unit variance, when averaged across the samples in a minibatch. More precisely, we replace the activation vector $\mathbf{z}_n$ (or sometimes the

1. In Tensorflow, a filter for 2d CNNs has shape (H, W, C, D), and a minibatch of feature maps has shape (batch-size, image-height, image-width, image-channels); this is called **NHWC** format. Other systems use different data layouts.

pre-activation vector $\mathbf{a}_n$) for example n (in some layer) with $\tilde{\mathbf{z}}_n$, which is computed as follows:

$$\tilde{\mathbf{z}}_n = \boldsymbol{\gamma} \odot \hat{\mathbf{z}}_n + \boldsymbol{\beta} \tag{14.17}$$

$$\hat{\mathbf{z}}_n = \frac{\mathbf{z}_n - \boldsymbol{\mu}_{\mathcal{B}}}{\sqrt{\boldsymbol{\sigma}^2_{\mathcal{B}} + \epsilon}} \tag{14.18}$$

$$\boldsymbol{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} \mathbf{z} \tag{14.19}$$

$$\boldsymbol{\sigma}^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} (\mathbf{z} - \boldsymbol{\mu}_{\mathcal{B}})^2 \tag{14.20}$$

where $\mathcal{B}$ is the minibatch containing example n, $\boldsymbol{\mu}_{\mathcal{B}}$ is the mean of the activations for this batch[2], $\boldsymbol{\sigma}^2_{\mathcal{B}}$ is the corresponding variance, $\hat{\mathbf{z}}_n$ is the standardized activation vector, $\tilde{\mathbf{z}}_n$ is the shifted and scaled version (the output of the BN layer), $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ are learnable parameters for this layer, and $\epsilon > 0$ is a small constant. Since this transformation is differentiable, we can easily pass gradients back to the input of the layer and to the BN parameters $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$.

When applied to the input layer, batch normalization is equivalent to the usual standardization procedure we discussed in Sec. 10.2.8. Note that the mean and variance for the input layer can be computed once, since the data is static. However, the empirical means and variances of the internal layers keep changing, as the parameters adapt. (This is sometimes called "**internal covariate shift**".) This is why we need to recompute $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}^2$ on each minibatch.

At test time, we may have a single input, so we cannot compute batch statistics. The standard solution to this is as follows: after training, compute $\boldsymbol{\mu}_l$ and $\boldsymbol{\sigma}^2_l$ for layer l across all the examples in the training set (i.e. using the full batch), and then "freeze" these parameters, and add them to the list of other parameters for the layer, namely $\boldsymbol{\beta}_l$ and $\boldsymbol{\gamma}_l$. At test time, we then use these frozen training values for $\boldsymbol{\mu}_l$ and $\boldsymbol{\sigma}^2_l$, rather than computing statistics from the test batch. Thus when using a model with BN, we need to specify if we are using it for inference or training. (See batchnorm_torch.ipynb for some sample code.)

For speed, we can combine a frozen batch norm layer with the previous layer. In particular suppose the previous layer computes $\mathbf{XW} + \mathbf{b}$; combining this with BN gives $\boldsymbol{\gamma} \odot (\mathbf{XW} + \mathbf{b} - \boldsymbol{\mu})/\boldsymbol{\sigma} + \boldsymbol{\beta}$. If we define $\mathbf{W}' = \boldsymbol{\gamma} \odot \mathbf{W}/\boldsymbol{\sigma}$ and $\mathbf{b}' = \boldsymbol{\gamma} \odot (\mathbf{b} - \boldsymbol{\mu})/\boldsymbol{\sigma} + \boldsymbol{\beta}$, then we can write the combined layers as $\mathbf{XW}' + \mathbf{b}'$. This is called **fused batchnorm**. Similar tricks can be developed to speed up BN during training [Jun+19].

The benefits of batch normalization (in terms of training speed and stability) can be quite dramatic, especially for deep CNNs. The exact reasons for this are still unclear, but BN seems to make the optimization landscape significantly smoother [San+18b]. It also reduces the sensitivity to the learning rate [ALL18]. In addition to computational advantages, it has statistical advantages. In particular, BN acts like a regularizer; indeed it can be shown to be equivalent to a form of approximate Bayesian inference [TAS18; Luo+19].

However, the reliance on a minibatch of data causes several problems. In particular, it can result in unstable estimates of the parameters when training with small batch sizes, although a more

2. When applied to a convolutional layer, we average across spatial locations and across examples, but not across channels (so the length of $\boldsymbol{\mu}$ is the number of channels). When applied to a fully connected layer, we just average across examples (so the length of $\boldsymbol{\mu}$ is the width of the layer).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

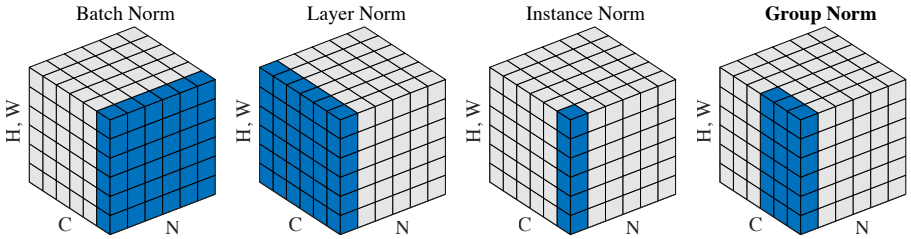


Figure 14.14: Illustration of different activation normalization methods for a CNN. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels. Left to right: batch norm, layer norm, instance norm, and group norm (with 2 groups of 3 channels). From Figure 2 of [WH18]. Used with kind permission of Kaiming He.

recent version of the method, known as **batch renormalization** [Iof17], partially addresses this. We discuss some other alternatives to batch norm below.

14.2.4.2 Other kinds of normalization layer

In Sec. 14.2.4.1 we discussed **batch normalization**, which standardizes all the activations within a given feature channel to be zero mean and unit variance. This can significantly help with training, and allow for a larger learning rate. (See code.probml.ai/book1/batchnorm_torch for some sample code.)

Although batch normalization works well, it struggles when the batch size is small, since the estimated mean and variance parameters can be unreliable. One solution is to compute the mean and variance by pooling statistics across other dimensions of the tensor, but not across examples in the batch. More precisely, let z_i refer to the i'th element of a tensor; in the case of 2d images, the index i has 4 components, indicating batch, channel, height and width: $i = (i_N, i_C, i_H, i_W)$. We compute the mean and standard deviation for each index z_i as follows:

$$\mu_i = \frac{1}{|\mathcal{S}_i|} \sum_{k \in \mathcal{S}_i} z_k, \; \sigma_i = \sqrt{\frac{1}{|\mathcal{S}_i|} \sum_{k \in \mathcal{S}_i} (z_k - \mu_i)^2 + \epsilon} \tag{14.21}$$

where $\mathcal{S}_i$ is the set of elements we average over. We then compute $\hat{z}_i = (z_i - \mu_i)/\sigma_i$ and $\tilde{z}_i = \gamma_c \hat{x}_i + \beta_c$, where c is the channel corresponding to index i.

In batch norm, we pool over batch, height, width, so $\mathcal{S}_i$ is the set of all location in the tensor that match the channel index of i. To avoid problems with small batches, we can instead pool over channel, height and width, but match on the batch index. This is known as **layer normalization** [BKH16]. (See code.probml.ai/book1/layer_norm_torch for some sample code.) Alternatively, we can have separate normalization parameters for each example in the batch and for each channel. This is known as **instance normalization** [UVL16].

A natural generalization of the above methods is known as **group normalization** [WH18], where we pool over all locations whose channel is in the same group as i's. This is illustrated in Fig. 14.14. Layer normalization is a special case in which there is a single group, containing all the channels.

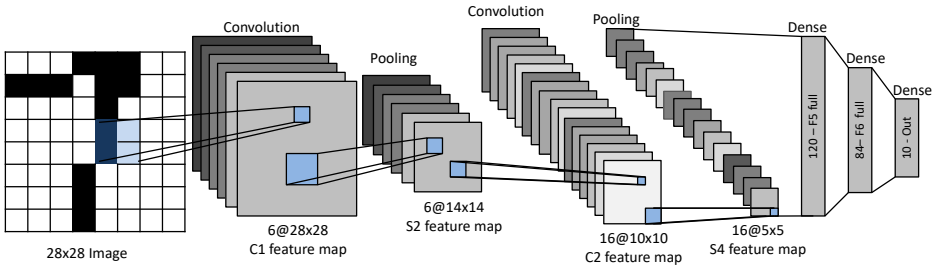


Figure 14.15: LeNet5, a convolutional neural net for classifying handwritten digits. From Figure 6.6.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Instance normalization is a special case in which there are C groups, one per channel. In [WH18], they show experimentally that it can be better (in terms of training speed, as well as training and test accuracies) to use groups that are larger than individual channels, but smaller than all the channels.

14.2.4.3 Normalizer-free networks

Recently, [Bro+21] have proposed a method called **normalizer-free networks**, which is a way to train deep residual networks without using batchnorm or any other form of normalization layer. The key is to replace it with adaptive gradient clipping, as an alternative way to avoid training instabilities. That is, we use Eq. (13.84), but adapt the clipping strength dynamically. The resulting model is faster train, and more accurate, than other competitive models trained with batchnorm.

14.3 Common architectures for image classification

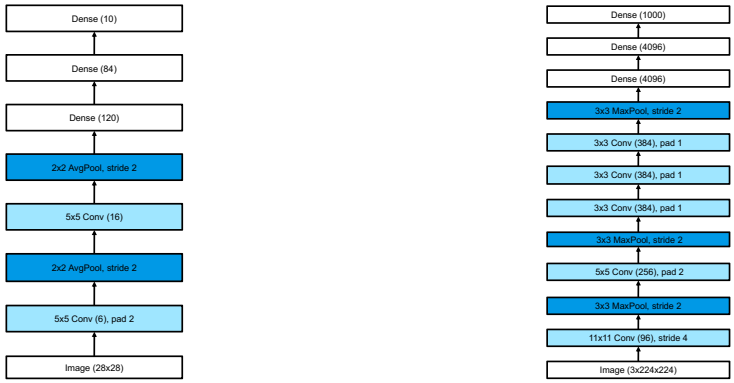
It is common to use CNNs to perform image classification, which is the task of estimating the function $f : \mathbb{R}^{H \times W \times K} \to \{0, 1\}^C$, where K is the number of input channels (e.g., $K = 3$ for RGB images), and C is the number of class labels.

In this section, we briefly review various CNNs that have been developed over the years to solve image classification tasks. See e.g., [Gu+18b] for a more extensive review of image classifiers.

14.3.1 LeNet

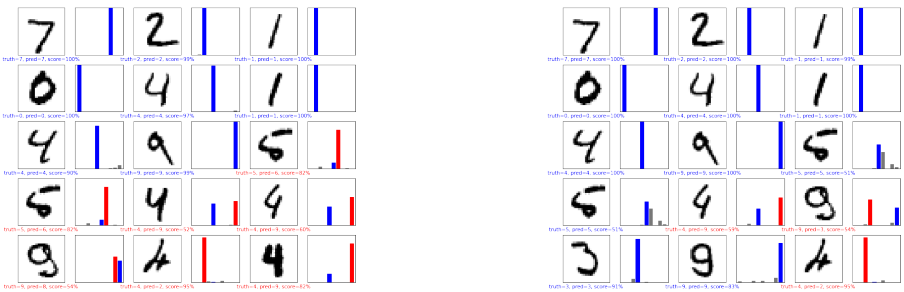
One of the earliest CNNs, created in 1998, is known as **LeNet** [LeC+98], named after its creator, Yann LeCun. It was designed to classify images of handwritten digits, and was trained on the MNIST dataset introduced in Sec. 3.4.2. The model is shown in Fig. 14.15. (Aee also Fig. 14.16a for a more compact representation of the model.) Some predictions of this model are shown in Fig. 14.17. After just 1 epoch, the test accuracy is already 98.8%. By contrast, the MLP in Sec. 13.2.4.2 had an accuracy of 95.9% after 1 epoch. More rounds of training can further increase accuracy to a point where performance is indistinguishable from label noise. (See code.probml.ai/book1/lenet_torch for some sample code.)

Of course, classifying isolated digits is of limited applicability: in the real world, people usually



(a) *(b)*

Figure 14.16: (a) LeNet5. We assume the input has size $1 \times 28 \times 28$, as is the case for MNIST. From Figure 6.6.2 of [Zha+20]. Used with kind permission of Aston Zhang. (b) AlexNet. We assume the input has size $3 \times 224 \times 224$, as is the case for (cropped and rescaled) images from ImageNet. From Figure 7.1.2 of [Zha+20]. Used with kind permission of Aston Zhang.



(a) *(b)*

Figure 14.17: Results of applying a CNN to some MNIST images (cherry picked to include some errors). Red is incorrect, blue is correct. (a) After 1 epoch of training. (b) After 2 epochs. Generated by code at figures.probml.ai/book1/14.17.

write strings of digits or other letters. This requires both segmentation and classification. LeCun and colleagues devised a way to combine convolutional neural networks with a model similar to a conditional random field to solve this problem. The system was deployed by the US postal service. See [LeC+98] for a more detailed account of the system.

14.3.2 AlexNet

Although CNNs have been around for many years, it was not until the paper of [KSH12] in 2012 that mainstream computer vision researchers paid attention to them. In that paper, the authors showed how to reduce the (top 5) error rate on the ImageNet challenge (Sec. 1.5.1.2) from the previous best

of 26% to 15%, which was a dramatic improvement. This model became known as **AlexNet** model, named after its creator, Alex Krizhevsky.

Fig. 14.16b(b) shows the architecture. It is very similar to LeNet, shown in Fig. 14.16a, with the following differences: it is deeper (8 layers of adjustable parameters (i.e., excluding the pooling layers) instead of 5); it uses ReLU nonlinearities instead of tanh (see Sec. 13.2.3 for why this is important); it uses dropout (Sec. 13.5.4) for regularization instead of weight decay; and it stacks several convolutional layers on top of each other, rather than strictly alternating between convolution and pooling. Stacking multiple convolutional layers together has the advantage that the receptive fields become larger as the output of one layer is fed into another (for example, three 3×3 filters in a row will have a receptive field size of 7×7). This is better than using a single layer with a larger receptive field, since the multiple layers also have nonlinearities in between. Also, three 3×3 filters have fewer parameters than one 7×7.

Note that AlexNet has 60M free parameters (which is much more than the 1M labeled examples), mostly due to the three fully connected layers at the output. Fitting this model relied on using two GPUs (due to limited memory of GPUs at that time), and is widely considered an engineering *tour de force*.[3] Fig. 1.14a shows some predictions made by the model on some images from ImageNet.

14.3.3 GoogLeNet (Inception)

Google who developed a model known as **GoogLeNet** [Sze+15b]. (The name is a pun on Google and LeNet.) The main difference from earlier models is that GoogLeNet used a new kind of block, known as an **inception block**[4], that employs multiple parallel pathways, each of which has a convolutional filter of a different size. See Fig. 14.18 for an illustration. This lets the model learn what the optimal filter size should be at each level. The overall model consists of 9 inception blocks followed by global average pooling. See Fig. 14.19 for an illustration. Since this model first came out, various extensions were proposed; details can be found in [IS15; Sze+15a; SIV17].

14.3.4 ResNet

The winner of the 2015 ImageNet classification challenge was a team at Microsoft, who proposed a model known as **ResNet** [He+16a]. The key idea is to replace $\mathbf{x}_{l+1} = \mathcal{F}_l(\mathbf{x}_l)$ with

$$\mathbf{x}_{l+1} = \varphi(\mathbf{x}_l + \mathcal{F}_l(\mathbf{x}_l)) \tag{14.22}$$

This is known as a **residual block**, since $\mathcal{F}_l$ only needs to learn the residual, or difference, between input and output of this layer, which is a simpler task. In [He+16a], $\mathcal{F}$ has the form conv-BN-relu-conv-BN, where conv is a convolutional layer, and BN is a batch norm layer (Sec. 14.2.4.1). See Fig. 14.20(left) for an illustration.

We can ensure the spatial dimensions of the output $\mathcal{F}_l(\mathbf{x}_l)$ of the convolutional layer match those of the input $\mathbf{x}_l$ by using padding. However, if we want to allow for the output of the convolutional

3. The 3 authors of the paper (Alex Krizhevsky, Ilya Sutskever and Geoff Hinton) were subsequently hired by Google, although Ilya left in 2015, and Alex left in 2017. For more historical details, see https://en.wikipedia.org/wiki/AlexNet. Note that AlexNet was not the first CNN implemented on a GPU; that honor goes to a group at Microsoft [CPS06], who got a 4x speedup over CPUs, and then [Cir+11], who got a 60x speedup.

4. This term comes from the movie *Inception*, in which the phrase "We need to go deeper" was uttered. This became a popular meme in 2014.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

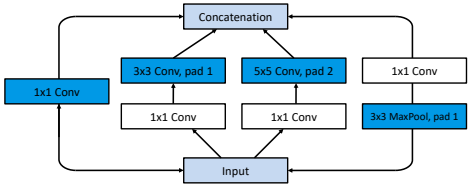


Figure 14.18: Inception module. The 1×1 convolutional layers reduce the number of channels, keeping the spatial dimensions the same. The parallel pathways through convolutions of different sizes allows the model to learn which filter size to use for each layer. The final depth concatenation block combines the outputs of all the different pathways (which all have the same spatial size). From Figure 7.4.1 of [Zha+20]. Used with kind permission of Aston Zhang.

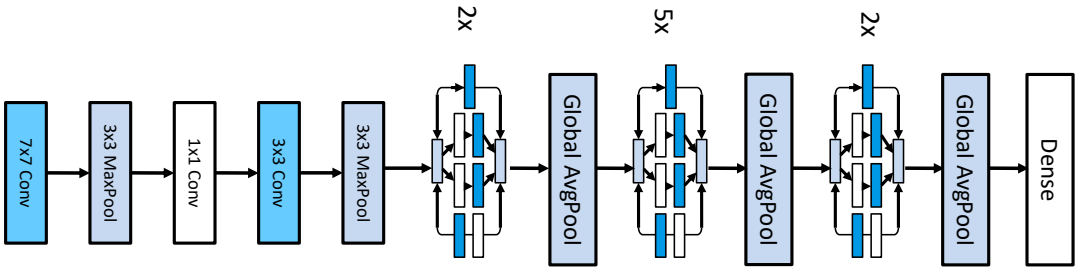


Figure 14.19: GoogLeNet (slightly simplified from the original). Input is on the left. From Figure 7.4.2 of [Zha+20]. Used with kind permission of Aston Zhang.

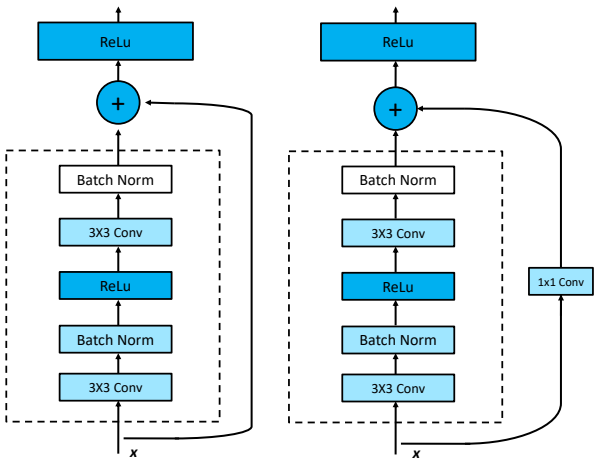


Figure 14.20: A residual block for a CNN. Left: standard version. Right: version with 1x1 convolution, to allow a change in the number of channels between the input to the block and the output. From Figure 7.6.3 of [Zha+20]. Used with kind permission of Aston Zhang

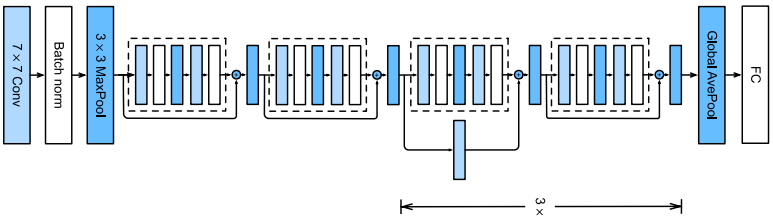


Figure 14.21: The ResNet-18 architecture. Each dotted module is a residual block shown in Fig. 14.20. From Figure 7.6.4 of [Zha+20]. Used with kind permission of Aston Zhang

layer to have a different number of channels, we need to add 1×1 convolution to the skip connection on $\mathbf{x}_l$. See Fig. 14.20(right) for an illustration.

The use of residual blocks allows us to train very deep models. The reason this is possible is that gradient can flow directly from the output to earlier layers, via the skip connections, for reasons explained in Sec. 13.4.4.

In [He+16a] trained a 152 layer ResNet on ImageNet. However, it is common to use shallower models. For example, Fig. 14.21 shows the **ResNet-18** architecture, which has 18 trainable layers: there are 2 3x3 conv layers in each residual block, and there are 8 such blocks, with an initial 7x7 conv (stride 2) and a final fully connected layer. Symbolically, we can define the model as follows:

```
(Conv : BN : Max) : (R : R) : (R'  : R)  : (R'  : R)  : (R'  : R) : Avg : FC
```

where R is a residual block, R' is a residual block with skip connection (due to the change in the number of channels) with stride 2, FC is fully connected (dense) layer, and : denotes concatenation. (See code.probml.ai/book1/resnet_torch for some example code that defines the model more precisely.) Note that the input size gets reduced spatially by a factor of $2^5 = 32$ (factor of 2 for each R' block, plus the initial Conv-7x7(2) and Max-pool), so a 224x224 images becomes a 7x7 image before going into the global average pooling layer.

An alternative to using a very deep model is to use a very "wide" model, with lots of feature channels per layer. This is the idea behind the **wide resnet** model [ZK16], which is quite popular.

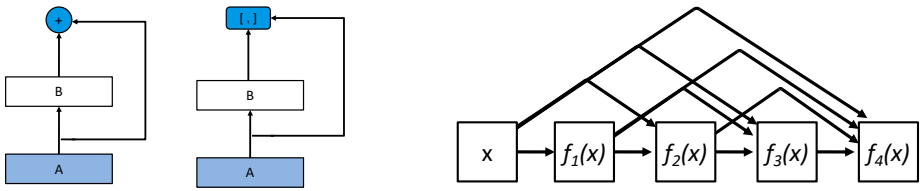
In [He+16b], they showed how a small modification of the above scheme allows us to train models with up to 1001 layers. The key insight is that the signal on the skip connections is still being attentuated due to the use of the nonlinear activation function after the addition step, $\mathbf{x}_{l+1} = \varphi(\mathbf{x}_l + \mathcal{F}(\mathbf{x}_l))$. They showed that it is better to use

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \varphi(\mathcal{F}_l(\mathbf{x}_l)) \tag{14.23}$$

This is called a **preactivation resnet** or **PreResnet** for short. Now it is very easy for the network to learn the identity function at a given layer: if we use ReLU activations, we just need to ensure that $\mathcal{F}_l(\mathbf{x}_l) = \mathbf{0}$, which we can do by setting the weights and biases to 0.

14.3.5 DenseNet

In a residual net, we add the output of each function to its input. An alternative approach would be to concatenate the output with the input, as illustrated in Fig. 14.22a. If we stack a series of such



(a) (b)

Figure 14.22: (a) Left: a residual block adds the output to the input. Right: a densenet block concatenates the output with the input. (b) Illustration of a densenet. From Figures 7.7.1–7.7.2 of [Zha+20]. Used with kind permission of Aston Zhang.

blocks, we can get an architecture similar to Fig. 14.22b. This is known as a **DenseNets** [Hua+17a], since each layer densely depends on all previous layers. Thus the overall model is computing a function of the form

$$\mathbf{x} \rightarrow [\mathbf{x}, f_1(\mathbf{x}), f_2(\mathbf{x}, f_1(\mathbf{x})), f_3(\mathbf{x}, f_1(\mathbf{x}), f_2(\mathbf{x}, f_1(\mathbf{x}))), \ldots] \tag{14.24}$$

The dense connectivity increases the number of parameters, since the channels get stacked depthwise. We can compensate for this by adding 1×1 convolution layers in between. We can also add pooling layers with a stride of 2 to reduce the spatial resolution. (See code.probml.ai/book1/densenet_torch for some sample code.)

DenseNets can perform better than ResNets, since all previously computed features are directly accessible to the output layer. However, they can be more computationally expensive.

14.3.6 Neural architecture search

We have seen how many CNNs are fairly similar in their design, and simply rearrange various building blocks (such as convolutional or pooling layers) in different topologies, and with different parameter settings (e.g., stride or number of channels). We can automate this design process using blackbox (derivative free) optimization methods to find architectures that minimize the validation loss. This is called **Auto-ML**; in the context of neural nets, it is called **neural architecture search** (**NAS**).

When performing NAS, we can optimize for multiple objectives at the same time,m such as accuracy, model size, training or inference speed, etc (this is how **EfficientNetv2** is created [TL21]). The main challenge arises due to the expense of computing the objective (since it requires training each candidate point in model space). One way to reduce the number of calls to the objective function is to use Bayesian optimization (see e.g., [WNS19]). Another approach is to create differentiable approximations to the loss (see e.g., [LSY19; Wan+21]). The field of NAS is very large and still growing. See [EMH19] for a more thorough review.

14.4 Other forms of convolution *

We discussed the basics of convolution in Sec. 14.2. In this section, we discuss some extensions, which are needed for applications such as image segmentation and image generation.

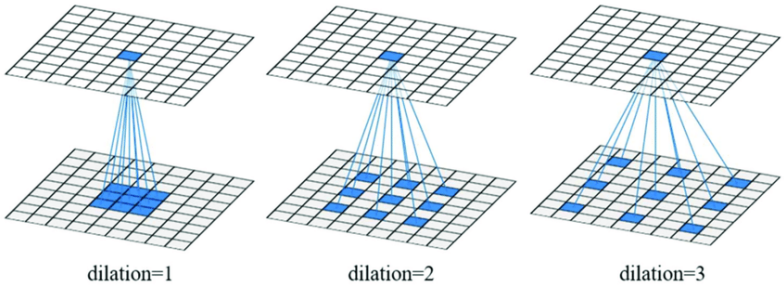


Figure 14.23: Dilated convolution with a 3x3 filter using rate 1, 2 and 3. From Figure 1 of [Cui+19]. Used with kind permission of Ximin Cui.

14.4.1 Dilated convolution

Convolution is an operation that combines the pixel values in a local neighborhood. By using striding, and stacking many layers of convolution together, we can enlarge the receptive field of each neuron, which is the region of input space that each neuron responds to. However, we would need many layers to give each neuron enough context to cover the entire image (unless we used very large filters, which would be slow and require too many parameters).

As an alternative, we can use **convolution with holes** [Mal99], sometimes known by the French term **à trous algorithm**, and recently renamed **dilated convolution** [YK16]. This method simply takes every r'th input element when performing convolution, where r is known as the **rate** or **dilation factor**. For example, in 1d, convolving with filter $\mathbf{w}$ using rate $r = 2$ is equivalent to regular convolution using the filter $\tilde{\mathbf{w}} = [w_1, 0, w_2, 0, w_3]$, where we have inserted 0s to expand the receptive field (hence the term "convolution with holes"). This allows us to get the benefit of increased receptive fields without increasing the number of parameters or the amount of compute. See Fig. 14.23 for an illustration.

More precisely, dilated convolution in 2d is defined as follows:

$$z_{i,j,d} = b_d + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{i+ru,j+rv,c} w_{u,v,c,d} \tag{14.25}$$

where we assume the same rate r for both height and width, for simplicity. Compare this to Eq. (14.15), where the stride parameter uses $x_{si+u,sj+v,c}$.

14.4.2 Transposed convolution

In convolution, we reduce from a large input $\mathbf{X}$ to a small output $\mathbf{Y}$ by taking a weighted combination of the input pixels and the convolutional kernel $\mathbf{K}$. This is easiest to explain in code:

```
def conv(X, K):
    h, w = K.shape
    Y = zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
```

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

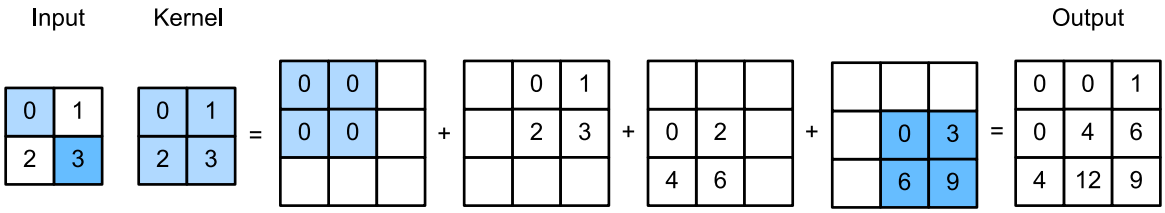


Figure 14.24: Transposed convolution with 2x2 kernel. From Figure 13.10.1 of [Zha+20]. Used with kind permission of Aston Zhang.

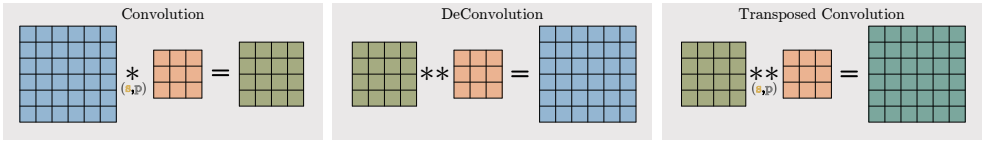


Figure 14.25: Convolution, deconvolution and transposed convolution. Here s is the stride and p is the padding. From https://towardsdatascience.com/what-is-transposed-convolutional-layer-40e5e6e31c11 . Used with kind permission of Aqeel Anwar.

```
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i + h, j:j + w] * K).sum()
    return Y
```

In **transposed convolution**, we do the opposite, in order to produce a larger output from a smaller input:

```
def trans_conv(X, K):
    h, w = K.shape
    Y = zeros((X.shape[0] + h - 1, X.shape[1] + w - 1))
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            Y[i:i + h, j:j + w] += X[i, j] * K
    return Y
```

This is equivalent to padding the input image with $(h-1, w-1)$ 0s (on the bottom right), where (h, w) is the kernel size, then placing a weighted copy of the kernel on each one of the input locations, where the weight is the corresponding pixel value, and then adding up. This process is illustrated in Fig. 14.24. We can think of the kernel as a "stencil" that is used to generate the output, modulated by the weights in the input.

The term "transposed convolution" comes from the interpretation of convolution as matrix multiplication, which we discussed in Sec. 14.2.1.3. If $\mathbf{W}$ is the matrix derived from kernel $\mathbf{K}$ using the process illustrated in Eq. (14.9), then one can show that $\mathbf{Y} = \text{transposed-conv}(\mathbf{X}, \mathbf{K})$ is equivalent to $\mathbf{Y} = \text{reshape}(\mathbf{W}^\top \text{vec}(\mathbf{X}))$. See code.probml.ai/book1/transposed_conv_torch for a demo.

Note that transposed convolution is also sometimes called **deconvolution**, but this is an incorrect usage of the term: deconvolution is the process of "undoing" the effect of convolution with a known

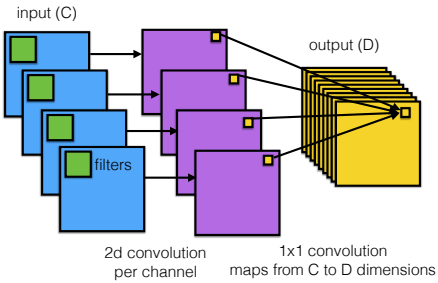


Figure 14.26: Depthwise separable convolutions: each of the C input channels undergoes a 2d convolution to produce C output channels, which get combined pointwise (via 1x1 convolution) to produce D output channels. From https://bit.ly/2L9fm2o. Used with kind permission of Eugenio Culurciello.

filter, such as a blur filter, to recover the original input, as illustrated in Fig. 14.25.

14.4.3 Depthwise separable convolution

Standard convolution uses a filter of size $H \times W \times C \times D$, which requires a lot of data to learn and a lot of time to compute with. A simplification, known as **depthwise separable convolution**, first convolves each input channel by a corresponding 2d filter, and then maps these C channels to D channels using 1×1 convolution:

$$z_{i,j,d} = b_d + w'_{c,d} \left(\sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{i+u,j+v,c} w_{u,v,c} \right) \tag{14.26}$$

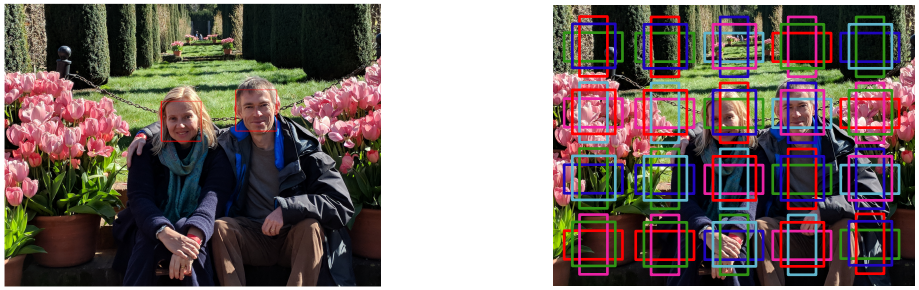
See Fig. 14.26 for an illustration.

To see the advantage of this, let us consider a simple numerical example.[5] Regular convolution of a $12 \times 12 \times 3$ input with a $5 \times 5 \times 3 \times 256$ filter gives a $8 \times 8 \times 256$ output (assuming valid convolution: 12-5+1=8), as illustrated in Fig. 14.13. With separable convolution, we start with $12 \times 12 \times 3$ input, convolve with a $5 \times 5 \times 1 \times 1$ filter (across space but not channels) to get $8 \times 8 \times 3$, then pointwise convolve (across channels but not space) with a $1 \times 1 \times 3 \times 256$ filter to get a $8 \times 8 \times 256$ output. So the output has the same size as before, but we used many fewer parameters to define the layer, and used much less compute. For this reason, separable convolution is often used in lightweight CNN models, such as the **MobileNet** model [How+17; San+18a] and other **edge devices**.

14.5 Solving other discriminative vision tasks with CNNs

In this section, we briefly discuss how to tackle various other vision tasks using CNNs. Each task also introduces a new architectural innovation to the library of basic building blocks we have already seen. More details on CNNs for computer vision can be found in e.g., [Bro19].

5. This example is from https://bit.ly/2Uj64Vo by Chi-Feng Wang.



(a) (b)

Figure 14.27: (a) Illustration of face detection, a special case of object detection. (Photo of author and his wife Margaret, taken at Filoli in California in Feburary, 2018. Image processed by Jonathan Huang using SSD face model.) (b) Illustration of anchor boxes. Adapted from [Zha+20, Sec 12.5].

14.5.1 Image tagging

Image classification associates a single label with the whole image, i.e., the outputs are assumed to be mutually exclusive. In many problems, there may be multiple objects present, and we want to label all of them. This is known as **image tagging**, and is an application of multi-label prediction. In this case, we define the output space as $\mathcal{Y} = \{0, 1\}^C$, where C is the number of tag types. Since the output bits are independent (given the image), we should replace the final softmax with a set of C logistic units.

Users of social media sites like **Instagram** often create hash tags for their images; this therefore provides a "free" way of creating large supervised datasets. Of course, many tags may be quite sparsely used, and their meaning may not be well-defined visually. (For example, someone may take a photo of themselves after they get a COVID test and tag the image "#covid"; however, visually it just looks like any other image of a person.) Thus this kind of user-generated labeling is usually considered quite noisy. However, it can be useful for "pre-training", as discussed in [Mah+18].

Finally, it is worth noting that image tagging is often a much more sensible objective than image classification, since many images have multiple objects in them, and it can be hard to know which one we should be labeling. Indeed, Andrej Karpathy, who created the "human performance benchmark" on ImageNet, noted the following:[6]

> Both [CNNs] and humans struggle with images that contain multiple ImageNet classes (usually many more than five), with little indication of which object is the focus of the image. This error is only present in the classification setting, since every image is constrained to have exactly one correct label. In total, we attribute *16% of human errors* to this category.

14.5.2 Object detection

In some cases, we want to produce a variable number of outputs, corresponding to a variable number of objects of interest that may be present in the image. (This is an example of an **open world**

6. Source: https://bit.ly/3cFbALk

problem, with an unknown number of objects.)

A canonical example of this is **object detection**, in which we must return a set of **bounding boxes** representing the locations of objects of interest, together with their class labels. A special case of this is **face detection**, where there is only one class of interest. This is illustrated in Fig. 14.27a.[7]

The simplest way to tackle such detection problems is to convert it into a closed world problem, in which there is a finite number of possible locations (and orientations) any object can be in. These candidate locations are known as **anchor boxes**. We can create boxes at multiple locations, scales and aspect ratios, as illustrated in Fig. 14.27b. For each box, we train the system to predict what category of object it contains (if any); we can also perform regression to predict the offset of the object location from the center of the anchor. (These residual regression terms allow sub-grid spatial localization.)

Abstractly, we are learning a function of the form

$$f_{\boldsymbol{\theta}} : \mathbb{R}^{H \times W \times K} \rightarrow [0,1]^{A \times A} \times \{1, \ldots, C\}^{A \times A} \times (\mathbb{R}^4)^{A \times A} \tag{14.27}$$

where K is the number of input channels, A is the number of anchor boxes in each dimension, and C is the number of object types (class labels). For each box location (i, j), we predict three outputs: an object presence probability, $p_{ij} \in [0, 1]$, an object category, $y_{ij} \in \{1, \ldots, C\}$, and two 2d offset vectors, $\boldsymbol{\delta}_{ij} \in \mathbb{R}^4$, which can be added to the centroid of the box to get the top left and bottom right corners.

Several models of this type have been proposed, including the **single shot detector** model of [Liu+16], and the **YOLO** (you only look once) model of [Red+16]. Many other methods for object detection have been proposed over the years. These models make different tradeoffs between speed, accuracy, simplicity, etc. See [Hua+17b] for an empirical comparison, and [Zha+18] for a more recent review.

14.5.3 Instance segmentation

In object detection, we predict a label abd bounding box for each object. In **instance segmentation**, the goal is to predict the label and 2d shape mask of each object instance in the image, as illustrated in Fig. 14.28. This can be done by applying a semantic segmentation model to each detected box, which has to label each pixel as foreground or background. (See Sec. 14.5.4 for more details on semantic segmentation.)

14.5.4 Semantic segmentation

In **semantic segmentation**, we have to predict a class label $y_i \in \{1, \ldots, C\}$ for each pixel, where the classes may represent things like sky, road, car, etc. In contrast to instance segmentation, which we discussed in Sec. 14.5.3, all car pixels get the same label, so semantic segmentation does not differentiate between objects. We can combine semantic segmentation of "stuff" (like sky, road) and instance segmentation of "things" (like car, person) into a coherent framework called "**panoptic segmentation**" [Kir+19].

7. Note that face detection is different from **face recognition**, which is a classification task that tries to predict the identity of a person from a set or "**gallery**" of possible people. Face recognition is usually solved by applying the classifier to all the patches that are detected as containing faces.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

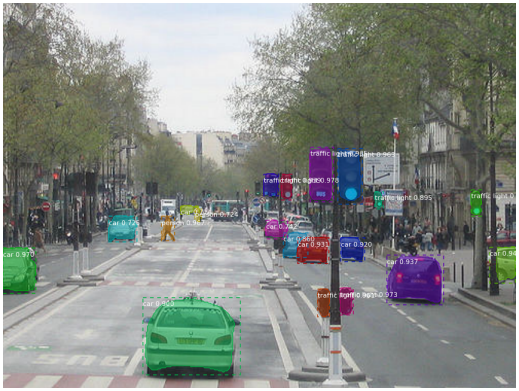


Figure 14.28: Illustration of object detection and instance segmentation using Mask R-CNN. From https://github.com/matterport/Mask_RCNN. Used with kind permission of Waleed Abdulla.

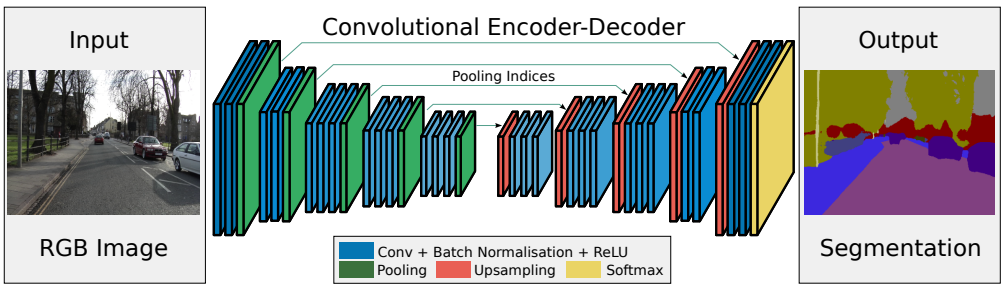


Figure 14.29: Illustration of an **encoder-decoder** *(aka* **U-net***) CNN for semantic segmentation The encoder uses convolution (which downsamples), and the decoder uses transposed convolution (which upsamples). From Figure 1 of [BKC17]. Used with kind permission of Alex Kendall.*

A common way to tackle semantic segmentation is to use an **encoder-decoder** architecture, as illustrated in Fig. 14.29. The encoder uses standard convolution to map the input into a small 2d bottleneck, which captures high level properties of the input at a coarse spatial resolution. (This typically uses a technique called dilated convolution that we explain in Sec. 14.4.1, to capture a large field of view, i.e., more context.) The decoder maps the small 2d bottleneck back to a full-sized output image using a technique called transposed convolution that we explain in Sec. 14.4.2. Since the bottleneck loses information, we can also add skip connections from input layers to output layers. The overall structure resembles the letter U, so this is also known as a **U-net** [RFB15].

A similar encoder-decoder architecture can be used for other **dense prediction** or **image-to-image** tasks, such as **depth prediction** (predict the distance from the camera, $z_i \in \mathbb{R}$, for each pixel i), **surface normal prediction** (predict the orientation of the surface, $\mathbf{z}_i \in \mathbb{R}^3$, at each image patch), etc. We can of course train one model to solve all of these tasks simultaneously, using multiple output heads, as illustrated in Fig. 14.30. (See e.g., [Kok17] for details.)

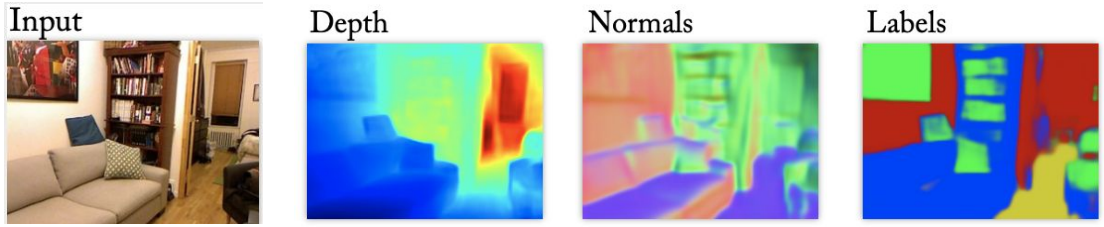


Figure 14.30: Illustration of a multi-task dense prediction problem. From Based on Figure 1 of [EF15]. Used with kind permission of Rob Fergus.

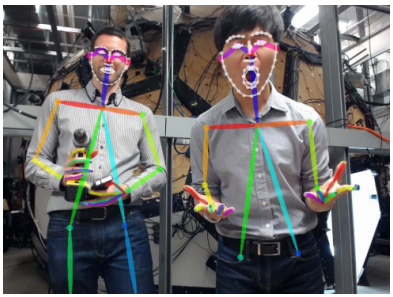


Figure 14.31: Illustration of keypoint detection for body, hands and face using the OpenPose system. From Figure 8 of [Cao+18]. Used with kind permission of Yaser Sheikh.

14.5.5 Human pose estimation

We can train an object detector to detect people, and to predict their 2d shape, as represented by a mask. However, we can also train the model to predict the location of a fixed set of skeletal keypoints, e.g., the location of the head or hands. This is called **human pose estimation**. See Fig. 14.31 for an example. There are several techniques for this, e.g., **PersonLab** [Pap+18] and **OpenPose** [Cao+18]. See [Bab19] for a recent review.

We can also predict 3d properties of each detected object. The main limitation is the ability to collect enough labeled training data, since it is difficult for human annotators to label things in 3d. However, we can use **computer graphics** engines to create simulated images with infinite ground truth 3d annotations (see e.g., [GNK18]).

14.6 Generating images by inverting CNNs *

A CNN trained for image classification is a discriminative model of the form $p(y|\mathbf{x})$, which takes as input an image, and returns as output a probability distribution over C class labels. In this section we discuss how to "invert" this model, by converting it intp a (conditional) **generative image model** of the form $p(\mathbf{x}|y)$. This will allow us to generate images that belong to a specific class. (We discuss more pricipled approaches to creating generative models for images in the sequel to this book, [Mur22].)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

14.6.1 Converting a trained classifier into a generative model

We can define a joint distribution over images and labels using $p(\mathbf{x}, y) = p(\mathbf{x})p(y|\mathbf{x})$, where $p(y|\mathbf{x})$ is the CNN classifier, and $p(\mathbf{x})$ is some prior over images. If we then clamp the class label to a specific value, we can create a conditional generative model using $p(\mathbf{x}|y = c) \propto p(\mathbf{x})p(y = c|\mathbf{x})$. Note that the discriminative classifier $p(y|\mathbf{x})$ was trained to "throw away" information, so $p(y|\mathbf{x})$ is not an invertible function. Thus the prior term $p(\mathbf{x})$ will play an important role in regularizing this process, as we see in Sec. 14.6.2.

One way to sample from this model is to use the Metropolis Hastings algorithm (Sec. 4.6.8.4), treating $\mathcal{E}_c(\mathbf{x}) = \log p(y = c|\mathbf{x}) + \log p(\mathbf{x})$ as the energy function. Since gradient information is available, we can use a proposal of the form $q(\mathbf{x}'|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \epsilon\mathbf{I})$, where $\boldsymbol{\mu}(\mathbf{x}) = \mathbf{x} + \frac{\epsilon}{2}\nabla \log \mathcal{E}_c(\mathbf{x})$. This is called the **Metropolis-adjusted Langevin algorithm** (MALA). As an approximation, we can ignore the rejection step, and accept every proposal. This is called the **unadjusted Langevin algorithm**. In addition, we can scale the gradient of the log prior and log likelihood indepenendently. Thus we get an update over the space of images that looks like a noisy version of SGD, except we take derivatives wrt the input pixels (using Eq. (13.51)), instead of the parameters:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(y = c|\mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(\mathbf{0}, \epsilon_3^2\mathbf{I}) \tag{14.28}$$

We can interpret each term in this equation as follows: the ϵ_1 term ensures the image is plausible under the prior, the ϵ_2 term ensures the image is plausible under the likelihood, and the ϵ_3 term is a noise term, in order to generate diverse samples. If we set $\epsilon_3 = 0$, the method becomes a deterministic algorithm to (approximately) generate the "most likely image" for this class.

14.6.2 Image priors

In this section, we discuss various kinds of image priors that we can use to regularize the ill-posed problem of inverting a classifier. These priors, together with the image that we start the optimization from, will determine the kinds of outputs that we generate.

14.6.2.1 Gaussian prior

Just specifying the class label is not enough information to specify the kind of images we want. We also need a prior $p(\mathbf{x})$ over what constitutes a "plausible" image. The prior can have a large effect on the quality of the resulting image, as we show below.

Arguably the simplest prior is $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\mathbf{0}, \mathbf{I})$, as suggested in [SVZ14]. (This assumes the image pixels have been centered.) This can prevent pixels from taking on extreme values. In this case, the update due to the prior term has the form

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}_t) = \nabla_{\mathbf{x}} - \frac{1}{2}||\mathbf{x}_t - \mathbf{0}||_2^2 = -\mathbf{x}_t \tag{14.29}$$

Thus the overall update (assuming $\epsilon_2 = 1$ and $\epsilon_3 = 0$) has the form

$$\mathbf{x}_{t+1} = (1 - \epsilon_1)\mathbf{x}_t + \frac{\partial \log p(y = c|\mathbf{x}_t)}{\partial \mathbf{x}_t} \tag{14.30}$$

See Fig. 14.32 for some samples generated by this method.

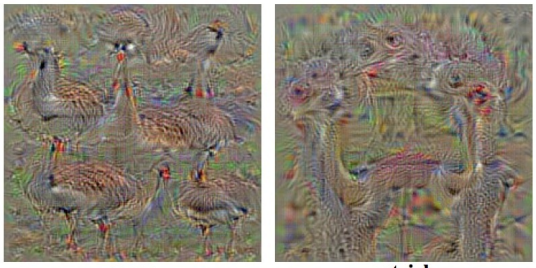


Figure 14.32: Images that maximize the probability of ImageNet classes "goose" and "ostrich" under a simple Gaussian prior. From `http://yosinski.com/deepvis`. *Used with kind permission of Jeff Clune.*

14.6.2.2 Total variation (TV) prior

We can generate slightly more realistic looking images if we use additional regularizers. [MV15; MV16] suggested computing the **total variation** or **TV** norm of the image. This is equal to the integral of the per-pixel gradients, which can be approximated as follows:

$$\text{TV}(\mathbf{x}) = \sum_{ijk} (x_{ijk} - x_{i+1,j,k})^2 + (x_{ijk} - x_{i,j+1,k})^2 \tag{14.31}$$

where x_{ijk} is the pixel value in row i, column j and channel k (for RGB images). We can rewrite this in terms of the horizontal and vertical **Sobel edge detector** applied to each channel:

$$\text{TV}(\mathbf{x}) = \sum_{k} ||\mathbf{H}(\mathbf{x}_{:,:,k})||_F^2 + ||\mathbf{V}(\mathbf{x}_{:,:,k})||_F^2 \tag{14.32}$$

See Fig. 14.33 for an illustration of these edge detectors. Using $p(\mathbf{x}) \propto \exp(-\text{TV}(\mathbf{x}))$ discourages images from having high frequency artefacts. In [Yos+15], they use Gaussian blur instead of TV norm, but this has a similar effect.

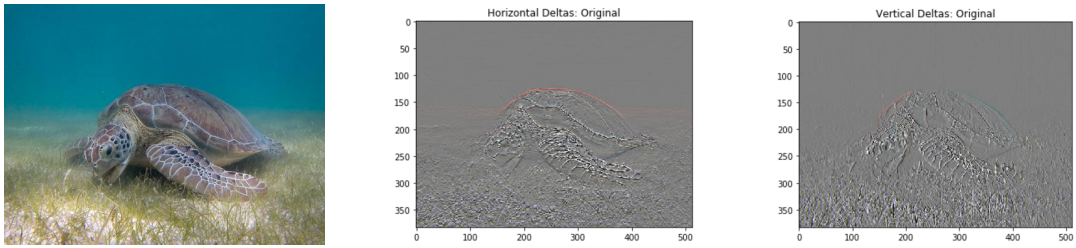
In Fig. 14.34 we show some results of optimizing $\log p(y = c, \mathbf{x})$ using a TV prior and a CNN likelihood for different class labels c starting from random noise.

14.6.3 Visualizing the features learned by a CNN

It is interesting to ask what the "neurons" in a CNN are learning. One way to do this is to start with a random image, and then to optimize the input pixels so as to maximize the average activation of a particular neuron. This is called **activation maximization** (AM), and uses the same technique as in Sec. 14.6.1 but fixes an internal node to a specific value, rather than clamping the output class label.

Fig. 14.35 illustrates the output of this method (with the TV prior) when applied to the AlexNet CNN trained on Imagenet classification. We see that, as the depth increases, neurons are learning to recognize simple edges/blobs, then texture patterns, then object parts, and finally whole objects. This is believed to be roughly similar to the hierarchical structure of the visual cortex (see e.g., [Kan+12]).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 14.33: Illustration of total variation norm. (a) Input image: a green sea turtle (Used with kind permission of Wikimedia author P. Lindgren). (b) Horizontal deltas. (c) Vertical deltas. Generated by code at figures.probml.ai/book1/14.33.

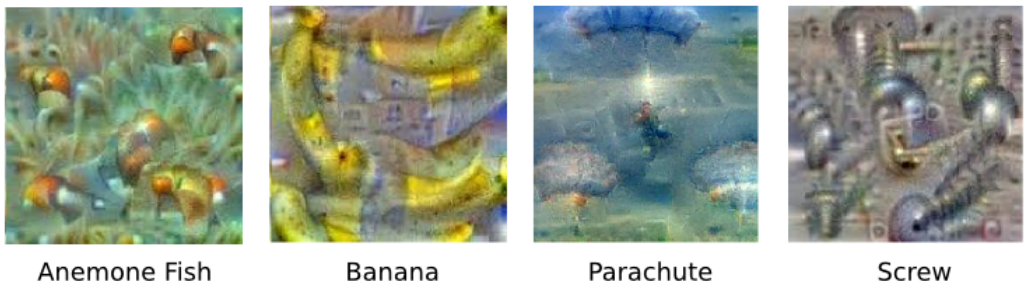


Figure 14.34: Images that maximize the probability of certain ImageNet classes under a TV prior. From https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html . Used with kind permission of Alexander Mordvintsev.

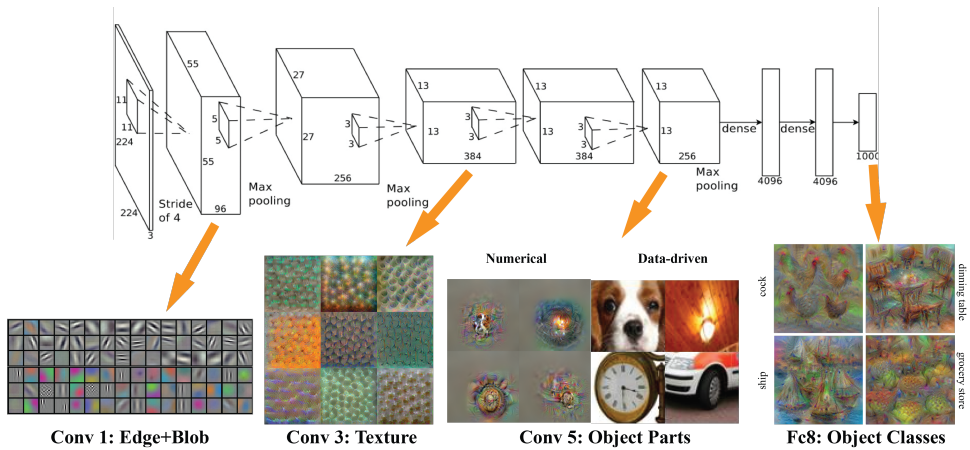
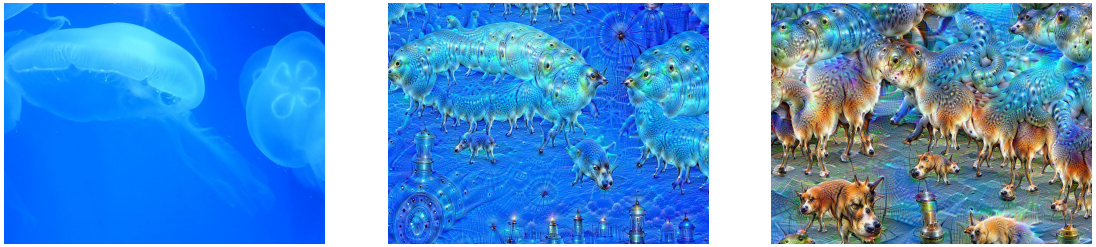


Figure 14.35: We visualize "optimal stimuli" for neurons in layers Conv 1, 3, 5 and fc8 in the AlexNet architecture, trained on the ImageNet dataset. For Conv5, we also show retrieved real images (under the column "data driven") that produce similar activations. Based on the method in [MV16]. Used with kind permission of Donglai Wei.



(a) (b) (c)

Figure 14.36: Illustration of DeepDream. The CNN is an Inception classifier trained on ImageNet. (a) Starting image of an Aurelia aurita (also called moon jelly). (b) Image generated after 10 iterations. (c) Image generated after 50 iterations. From https://en.wikipedia.org/wiki/DeepDream. Used with kind permission of Wikipedia author Martin Thoma.

An alternative to optimizing in pixel space is to search the training set for images that maximally activate a given neuron. This is illustrated in Fig. 14.35 for the Conv5 layer.

For more information on feature visualization see e.g., [OMS17].

14.6.4 Deep Dream

So far we have focused on generating images which maximize the class lable or some other neuron of interest. In this section we tackle a more artistic application, in which we want to generate versions of an input image that emphasize certain features.

To do this, we view our pre-trained image classifier as a feature extractor. Based on the results in Sec. 14.6.3, we know the activity of neurons in different layers correspond to different kinds of features in the image. Suppose we are interested in "amplifying" features from layers $l \in \mathcal{L}$. We can do this be defining an energy or loss function of the form $\mathcal{L}(\mathbf{x}) = \sum_{l \in \mathcal{L}} \overline{\phi}_l(\mathbf{x})$, where $\boldsymbol{\phi}_l(\mathbf{x})$ is the average over the features in layer l, $\overline{\phi}_l = \frac{1}{HWC} \sum_{hwc} \boldsymbol{\phi}_{lhwc}(\mathbf{x})$. We can now use gradient descent to optimize this energy. The resulting process is called **DeepDream** [MOT15], since the model amplifies features that were only hinted at in the original image and then creates images with more and more of them.[8]

Fig. 14.36 shows an example. We start with an image of a jellyfish, which we pass into a CNN that was trained to classify ImageNet images. After several iterations, we generate some image which is a hybrid of the input and the kinds of "hallucinations" we saw in Fig. 14.32; these hallucinations involve dog parts, since ImageNet has so many kinds of dogs in its label set. See [Tho16] for details, and https://deepdreamgenerator.com for a fun web-based demo.

14.6.5 Neural style transfer

The DeepDream system in Fig. 14.36 shows one way that CNNs can be used to create "art". However, it is rather creepy. In this section, we discuss a related approach that gives the user more control. In particular, the user has to specify a reference "style image" $\mathbf{x}_s$ and "content image" $\mathbf{x}_c$. The system

8. The method was originally called **Inceptionism**, since it uses the inception CNN (Sec. 14.3.3).

(a) (b) (c)

Figure 14.37: Example output from a neural style transfer system (a) Content image: a green sea turtle (Used with kind permission of Wikimedia author P. Lindgren). (b) Style image: a painting by Wassily Kandinsky called "Composition 7". (c) Output of neural style generation. Generated by code at figures.probml.ai/book1/14.37.

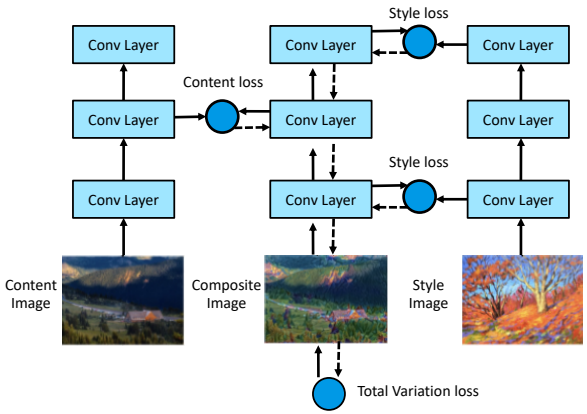


Figure 14.38: Illustration of how neural style transfer works. Adapted from Figure 12.12.2 of [Zha+20].

will then try to generate a new image $\mathbf{x}$ that "re-renders" $\mathbf{x}_c$ in the style of $\mathbf{x}_s$. This is called **neural style transfer**, and is illustrated in Fig. 14.37. This was first proposed in [GEB16], and there are now many papers on this topic; see [Jin+17] for a recent review.

14.6.5.1 How it works

Style transfer works by optimizing the following energy function:

$$\mathcal{L}(\mathbf{x}|\mathbf{x}_s, \mathbf{x}_c) = \lambda_{TV}\mathcal{L}_{\text{TV}}(\mathbf{x}) + \lambda_c\mathcal{L}_{\text{content}}(\mathbf{x}, \mathbf{x}_c) + \lambda_s\mathcal{L}_{\text{style}}(\mathbf{x}, \mathbf{x}_s) \tag{14.33}$$

See Fig. 14.38 for a high level illustration.

The first term in Eq. (14.33) is the total variation prior discussed in Sec. 14.6.2.2. The second term measures how similar $\mathbf{x}$ is to $\mathbf{x}_c$ by comparing feature maps of a pre-trained CNN $\boldsymbol{\phi}(\mathbf{x})$ in the

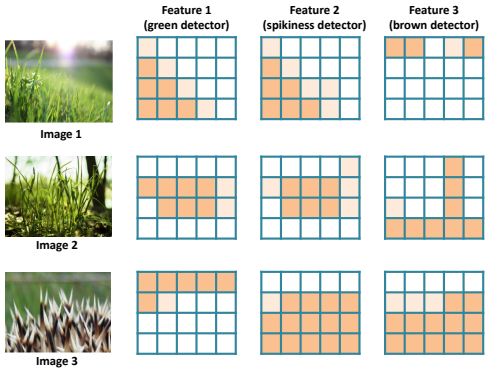


Figure 14.39: Schematic representation of 3 kinds of feature maps for 3 different input images. Adapted from Figure 5.16 of [Fos19].

relevant "content layer" l:

$$\mathcal{L}_{\text{content}}(\mathbf{x}, \mathbf{x}_c) = \frac{1}{C_\ell H_\ell W_\ell} ||\boldsymbol{\phi}_\ell(\mathbf{x}) - \boldsymbol{\phi}_\ell(\mathbf{x}_c)||_2^2 \tag{14.34}$$

Finally we have to define the style term. We can interpret visual style as the statistical distribution of certain kinds of image features. The location of these features in the image may not matter, but their co-occurence does. This is illustrated in Fig. 14.39. It is clear (to a human) that image 1 is more similar in style to image 2 than to image 3. Intuitively this is because both image 1 and image 2 have spiky green patches in them, whereas image 3 has spiky things that are not green.

To capture the co-occurence statistics we compute the **Gram matrix** for an image using feature maps from a specific layer ℓ:

$$G_\ell(\mathbf{x})_{c,d} = \frac{1}{H_\ell W_\ell} \sum_{h=1}^{H_\ell} \sum_{w=1}^{W_\ell} \boldsymbol{\phi}_\ell(\mathbf{x})_{h,w,c} \, \boldsymbol{\phi}_\ell(\mathbf{x})_{h,w,d} \tag{14.35}$$

The Gram matrix is a $C_\ell \times C_\ell$ matrix which is proportional to the uncentered covariance of the C_ℓ-dimensional feature vectors sampled over each of the $H_\ell W_\ell$ locations.

Given this, we define the style loss for layer ℓ as follows:

$$\mathcal{L}_{\text{style}}^\ell(\mathbf{x}, \mathbf{x}_s) = ||\mathbf{G}_\ell(\mathbf{x}) - \mathbf{G}_\ell(\mathbf{x}_s)||_F^2 \tag{14.36}$$

Finally, we define the overall style loss as a sum over the losses for a set $\mathcal{S}$ of layers:

$$\mathcal{L}_{\text{style}}(\mathbf{x}, \mathbf{x}_s) = \sum_{\ell \in \mathcal{S}} \mathcal{L}_{\text{style}}^\ell(\mathbf{x}, \mathbf{x}_s) \tag{14.37}$$

For example, in Fig. 14.38, we compute the style loss at layers 1 and 3. (Lower layers will capture visual texture, and higher layers will capture object layout.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

14.6.5.2 Speeding up the method

In [GEB16], they used L-BFGS (Sec. 8.3.2) to optimize Eq. (14.33), starting from white noise. We can get faster results if use an optimizer such as Adam instead of BFGS, and initialize from the content image instead of white noise. Nevertheless, running an optimizer for every new style and content image is slow. Several papers (see e.g., [JAFF16; Uly+16; UVL16; LW16]) have proposed to train a neural network to directly predict the outcome of this optimization, rather than solving it for each new image pair. (This can be viewed as a form of amortized optimization.) In particular, for every style image $\mathbf{x}_s$, we fit a model f_s such that $f_s(\mathbf{x}_c) = \text{argmin}_{\mathbf{x}} \mathcal{L}(\mathbf{x}|\mathbf{x}_s, \mathbf{x}_c)$. We can then apply this model to new content images without having to reoptimize.

More recently, [DSK16] has shown how it is possible to train a single network that takes as input both the content and a discrete representation s of the style, and then produces $f(\mathbf{x}_c, s) = \text{argmin}_{\mathbf{x}} \mathcal{L}(\mathbf{x}|s, \mathbf{x}_c)$ as the output. This avoids the need to train a separate network for every style image. The key idea is to standardize the features at a given layer using scale and shift parameters that are style specific. In particular, we use the following **conditional instance normalization** transformation:

$$\text{CIN}(\phi(\mathbf{x}_c), s) = \gamma_s \left(\frac{\phi(\mathbf{x}_c) - \mu(\phi(\mathbf{x}_c))}{\sigma(\phi(\mathbf{x}_c))} \right) + \beta_s \tag{14.38}$$

where $\mu(\phi(\mathbf{x}_c))$ is the mean of the features in a given layer, $\sigma(\phi(\mathbf{x}_c))$ is the standard deviation, and β_s and γ_s are parameters for style type s. (See Sec. 14.2.4.2 for more details on instance normalization.) Surprisingly, this simple trick is enough to capture many kinds of styles.

The drawback of the above technique is that it only works for a fixed number of discrete styles. [HB17] proposed to generalize this by replacing the constants β_s and γ_s by the output of another CNN, which takes an arbitrary style image $\mathbf{x}_s$ as input. That is, in Eq. (14.38), we set $\beta_s = f_{\boldsymbol{\beta}}(\phi(\mathbf{x}_s))$ and $\gamma_s = f_{\boldsymbol{\gamma}}(\phi(\mathbf{x}_s))$, and we learn the parameters $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ along with all the other parameters. The model becomes

$$\text{AIN}(\phi(\mathbf{x}_c), \phi(\mathbf{x}_s)) = f_\gamma(\phi(\mathbf{x}_s)) \left(\frac{\phi(\mathbf{x}_c) - \mu(\phi(\mathbf{x}_c))}{\sigma(\phi(\mathbf{x}_c))} \right) + f_\beta(\phi(\mathbf{x}_s)) \tag{14.39}$$

They call their method **adaptive instance normalization**.

14.7 Adversarial Examples *

This section is coauthored with Justin Gilmer.

In Sec. 14.6.1 we discussed ways to generate images that are most likely to activate a given neuron, such as the output label $y = c$, by solving

$$\mathbf{x}^* = \underset{\mathbf{x}'}{\text{argmax}} \left[\log p(y = c|\mathbf{x}'; \boldsymbol{\theta}) + \log p(\mathbf{x}') \right] \tag{14.40}$$

where $p(\mathbf{x}')$ is a prior term to regularize the inversion. In this section, we discuss how to modify this approach to generate an image that is *least likely* to generate the true or original label $y = c$. Instead of using a prior on "plausible images", we impose the constraint that $\mathbf{x}^*$ should remain "perceptually similar" to the original input $\mathbf{x}$. This gives rise to the following objective:

$$\mathbf{x}_{\text{adv}} = \underset{\mathbf{x}' \in \Delta(\mathbf{x})}{\text{argmin}} \log p(y = c|\mathbf{x}') \tag{14.41}$$

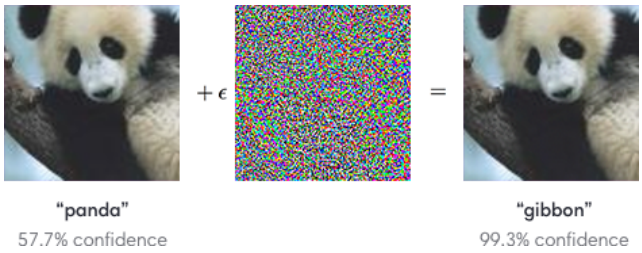


Figure 14.40: Example of an adversarial attack on an image classifier. Left column: original image which is correctly classified. Middle column: small amount of structured noise which is added to the input (magnitude of noise is magnified by 10×). Right column: new image, which is confidently misclassified as a "gibbon", even though it looks just like the original "panda" image. Here $\epsilon = 0.007$ From Figure 1 of [GSS15]. Used with kind permission of Ian Goodfellow.

where $\Delta(\mathbf{x})$ is the set of images that are "similar" to $\mathbf{x}$ (we discuss different notions of similarity below).

Eq. (14.41) is an example of an **adversarial attack**. We illustrate this in Fig. 14.40. The input image $\mathbf{x}$ is on the left, and is predicted to be a panda with probability 57%. By adding a tiny amount of carefully chosen noise (shown in the middle) to the input, we generate the **adversarial image** $\mathbf{x}_{\text{adv}}$ on the right: this "looks like" the input, but is now classified as a gibbon with probability 99%.

The ability to create adversarial images was first noted in [Sze+14]. It is suprisingly easy to create such examples, which seems paradoxical, given the fact that modern classifiers seemt to work so well on normal inputs (see e.g., Fig. 1.14b which shows ImageNet classification error over time), and the perturbed images "look" the same to humans. We explain this paradox in Sec. 14.7.5.

The existence of adversarial images also raises security concerns. For example, [Sha+16b] showed they could force a face recognition system to misclassify person A as person B, merely by asking person A to wear a pair of sunglasses with a special pattern on them, and [Eyk+18] show that is possible to attach small "adverarial stickers" to traffic signs to classify stop signs as speed limit signs. (However, to guarantee robust security, we need to defend against much stronger "**threat models**", not just imperceptible tweaks, as we discuss in Sec. 14.7.3.)

Below we briefly discuss how to create adversarial attacks, why they occur, and how we can try to defend against them. We focus on the case of deep neural nets for images, although it is important to note that many other kinds of models (including logistic regression and generative models) can also suffer from adversarial attacks. Furthermore, this is not restricted to the image domain, but occurs with many kinds of high dimensional inputs. For example, [Li+19] contains an audio attack and [Dal+04; Jia+19] contains a text attack. More details on adversarial examples can be found in e.g., [Wiy+19; Yua+19].

14.7.1 Whitebox (gradient-based) attacks

To create an adversarial example, we must find a "small" perturbation $\boldsymbol{\delta}$ to add to the input $\mathbf{x}$ to create $\mathbf{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta}$ so that $f(\mathbf{x}_{\text{adv}}) = y'$, where $f()$ is the classifier, and y' is the label we want to force the system to output. This is known as a **targeted attack**. Alternatively, we may just want to find a perturbation that causes the current predicted label to change from its current value to any

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

other value, so that $f(\mathbf{x}+\boldsymbol{\delta}) \neq f(\mathbf{x})$, which is known as **untargeted attack**.

In general, we define the objective for the adversary as *maximizing* the following loss:

$$\mathbf{x}_{\text{adv}} = \underset{\mathbf{x}' \in \Delta(\mathbf{x})}{\text{argmax}}\, \mathcal{L}(\mathbf{x}', y; \boldsymbol{\theta}) \tag{14.42}$$

where y is the true label. For the untargeted case, we can define $\mathcal{L}(\mathbf{x}', y; \boldsymbol{\theta}) = -\log p(y|\mathbf{x}')$, so we minimize the probability of the true label; and for the targeted case, we can define $\mathcal{L}(\mathbf{x}', y; \boldsymbol{\theta}) = \log p(y'|\mathbf{x}')$, where we maximize the probability of the desired label $y' \neq y$.

To define what we mean by "small" perturbation, we impose the constraint that $\mathbf{x}_{\text{adv}} \in \Delta(\mathbf{x})$, which is the set of "perceptually similar" images to the input $\mathbf{x}$. Most of the literature has focused on a simplistic setting in which the adversary is restricted to making bounded l_p perturbations of a clean input $\mathbf{x}$, that is

$$\Delta(\mathbf{x}) = \{\mathbf{x}' : ||\mathbf{x}' - \mathbf{x}||_p < \epsilon\} \tag{14.43}$$

Typically people assume $p = 1$ or $p = 0$. We will discuss more realistic threat models in Sec. 14.7.3.

In this section, we assume that the attacker knows the model parameters $\boldsymbol{\theta}$; this is called a **whitebox attack**, and lets us use gradient based optimization methods. We relax this assumption in Sec. 14.7.2.)

To solve the optimization problem in Eq. (14.42), we can use any kind of constrained optimization method. In [Sze+14] they used bound-constrained BFGS (Sec. 8.3.2). [GSS15] proposed the more efficient **fast gradient sign** (**FGS**) method, which performs iterative updates of the form

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \boldsymbol{\delta}_t \tag{14.44}$$

$$\boldsymbol{\delta}_t = \epsilon\, \text{sign}(\nabla_{\mathbf{x}} \log p(y'|\mathbf{x}, \boldsymbol{\theta})|_{\mathbf{x}_t}) \tag{14.45}$$

where $\epsilon > 0$ is a small learning rate. (Note that this gradient is with respect to the input pixels, as in Sec. 14.6.1, not the model parameters.) Fig. 14.40 gives an example of this process.

More recently, [Mad+18] proposed the more powerful **projected gradient descent** (**PGD**) attack; this can be thought of as an iterated version of FGS. There is no "best" variant of PGD for solving 14.42. Instead, what matters more is the implementation details, e.g. how many steps are used, the step size, and the exact form of the loss. To avoid local minima, we may use random restarts, choosing random points in the constraint space Δ to initialize the optimization. The algorithm should be carefully tuned to the specific problem, and the loss should be monitored to check for optimization issues. For best practices, see [Car+19].

14.7.2 Blackbox (gradient-free) attacks

In this section, we no longer assume that the adversary knows the parameters $\boldsymbol{\theta}$ of the predictive model f. This is known as a **black box attack**. In such cases, we must use derivative-free optimization methods (see Sec. 8.8).

Evolutionary algorithms (EA) are one class of DFO solvers. These were used in [NYC15] to create blackbox attacks. Fig. 14.41 shows some images that were generated by applying an EA to a random noise image. These are known as **fooling images**, as opposed to adversarial images, since they are not visually realistic. Fig. 14.42 shows some fooling images that were generated by applying EA to

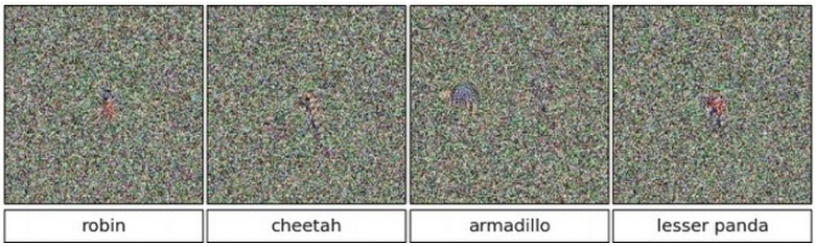


Figure 14.41: Images that look like random noise but which cause the CNN to confidently predict a specific class. From Figure 1 of [NYC15]. Used with kind permission of Jeff Clune.

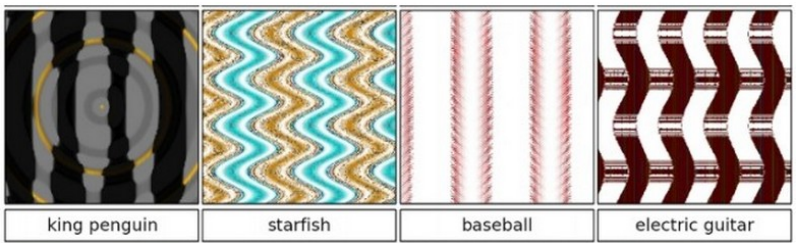


Figure 14.42: Synthetic images that cause the CNN to confidently predict a specific class. From Figure 1 of [NYC15]. Used with kind permission of Jeff Clune.

the parameters of a compositional pattern-producing network [Sta07].[9] By suitably perturbing the CPPN parameters, it is possible to generate structured images with high fitness (classifier score), but which do not look like natural images [Aue12].

In [SVK19], they used differential evolution to attack images by modifying a single pixel. This is equivalent to bounding the ℓ_0 norm of the perturbation, so that $||\mathbf{x}_{\text{adv}} - \mathbf{x}||_0 = 1$.

In [Pap+17], they learned a differentiable surrogate model of the blackbox, by just querying its predictions y for different inputs $\mathbf{x}$. They then used gradient-based methods to generate adversarial attacks on their surrogate model, and then showed that these attacks transferred to the real model. In this way, they were able to attack various the image classification APIs of various cloud service providers, including Google, Amazon and MetaMind.

14.7.3 Real world adversarial attacks

Typically, the space of possible adversarial inputs Δ can be quite large, and will be difficult to exactly define mathematically as it will depend on semantics of the input based on the attacker's goals [BR18]. Consider for example of the **content constrained** threat model discussed in [Gil+18a]. One instance of this threat model involves image spam, where the attacker wishes to upload an image attachment in an email that will not be classified as spam by a detection model. In this case Δ is

9. A CPPN is a set of elementary functions (such as linear, sine, sigmoid, and Gaussian) which can be composed in order to specify the mapping from each coordinate to the desired color value. CPPN was originally developed as a way to encode abstract properties such as symmetry and repetition, which are often seen during biological development.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Figure 14.43: An adversarially modified image to evade spam detectors. The image is constructed from scratch, and does not involve applying a small perturbation to any given image. This is an illustrative example of how large the space of possible adversarial inputs Δ can be when the attacker has full control over the input. From [Big+11]. Used with kind permission of Battista Biggio.

incredibly large as it consists of all possible images which contain some semantic concept the attacker wishes to upload (in this case an advertisement). To explore Δ, spammers can utilize different fonts, word orientations or add random objects to the background as is the case of the adversarial example in Figure 14.43 (see [Big+11] for more examples). Of course, optimization based methods may still be used here to explore parts of Δ. However, in practice it may be preferable to design an adversarial input by hand as this can be significantly easier to execute with only limited-query black-box access to the underlying classifier.

14.7.4 Defenses based on robust optimization

As discussed in Section 14.7.3, securing a system against adversarial inputs in more general threat models seems extraordinarily difficult, due to the vast space of possible adversarial inputs Δ. However, there is a line of research focused on producing models which are invariant to perturbations within a small constraint set $\Delta(\mathbf{x})$, with a focus on l_p-robustness where $\Delta(\mathbf{x}) = \{\mathbf{x}' : ||\mathbf{x} - \mathbf{x}'||_p < \epsilon\}$. Although solving this toy threat model has little application to security settings, enforcing smoothness priors have in some cases improved robustness to random image corruptions [SHS], lead to models which transfer better [Sal+20], and can bias models towards different features in the data [Yin+19a].

Perhaps the most straightforward method for improving l_p-robustness is to directly optimize for it though **robust optimization** [BTEGN09], also known as **adversarial training** [GSS15]. We define the **adversarial risk** to be

$$\min_{\theta} \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim p(\mathbf{x},\mathbf{y})} \left[\max_{\mathbf{x}' \in \Delta(\mathbf{x})} L(\mathbf{x}', \mathbf{y}; \theta) \right] \tag{14.46}$$

The min max formulation in equation 14.46 poses unique challenges from an optimization perspective—it requires solving both the non-concave inner maximazation and the non-convex outer minimization problems. Even worse, the inner max is NP-hard to solve in general [Kat+17]. However, in practice it

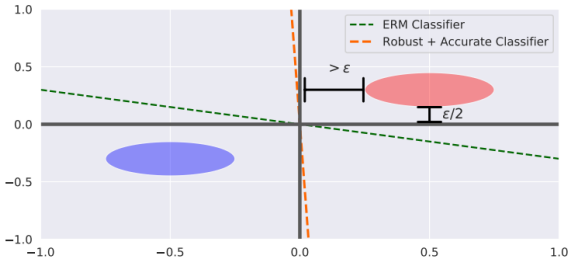


Figure 14.44: Illustration of robust vs non-robust features for separating two classes, represented by Gaussian blobs. The green line is the decision boundary induced by empirical risk minimization (ERM). The red line is the decision boundary induced by robust training — it ignores the vertical feature x_2, which is less robust than the horizontal feature x_1. From Figure 14 of [Ily+19]. Used with kind permission of Andrew Ilyas.

may be sufficient to compute the gradient of the outer objective $\nabla_\theta L(\mathbf{x}_{\text{adv}}, \mathbf{y}, ; \theta)$ at an approximately maximal point in the inner problem $\mathbf{x}_{\text{adv}} \approx \text{argmax}_{\mathbf{x}} L(\mathbf{x}_{\text{adv}}, \mathbf{y}; \theta)$ [Mad+18]. Currently, best practice is to approximate the inner problem using a few steps of PGD.

Other methods seek to **certify** that a model is robust within a given region $\Delta(x)$. One method for certification uses randomized smoothing [CRK19]—a technique for converting a model robust to random noise into a model which is provably robust to bounded worst-case perturbations in the l_2-metric. Another class of methods applies specifically for networks with ReLU activations, leveraging the property that the model is locally linear, and that certifying in region defined by linear constraints reduces to solving a series of linear programs, for which standard solvers can be applied [WK18].

14.7.5 Why models have adversarial examples

The existence of adversarial inputs is paradoxical, since modern classifiers seem to do so well on normal inputs. The basic reason that classifiers can get fooled in this way is that many datasets contain features which are discriminative but not robust to small perturbations [Ily+19]. This is illustrated in Fig. 14.44. The data is drawn from a mixture of 2 Gaussians in 2d, where the mixture component depends on the class label (c.f., Gaussian discriminant analysis, Sec. 9.2). We see that there are many (linear) decision boundaries that achieve perfect accuracy on the test set. However, some are more robust to perturbations than others. In high dimensions, it is easy for a model to find features that "happen" to work well, but which are not robust to small changes.

The existence of adversarial examples is best viewed as a symptom of the more general problem of lack of robustness to **distribution shift**. If a model's accuracy drops on some shifted distribution of inputs $q(\mathbf{x})$ that differs from the training distribution $p(\mathbf{x})$, then the model will necessarily be vunerable to an adversarial attack: if errors exist, there must be a nearest such error. Furthermore, if the input distribution is high dimensional, then we should expect the nearest error to be significantly closer than errors which are sampled randomly from some out-of-distribution $q(\mathbf{x})$.

Consider, for example when $p(\mathbf{x})$ is a distribution of clean images and $q(\mathbf{x})$ applies additive Gaussian noise to the inputs at test time. In Fig. 14.45, we see that a small amount of such (non-adversarial)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

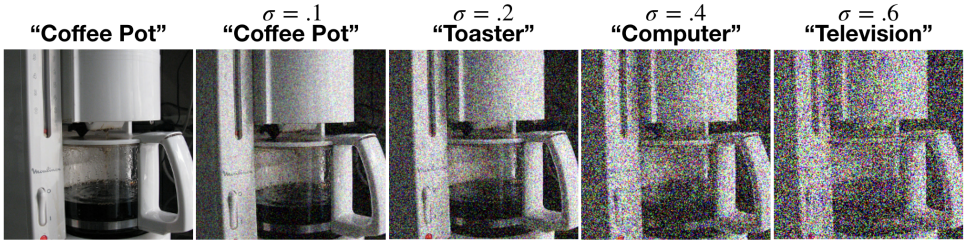
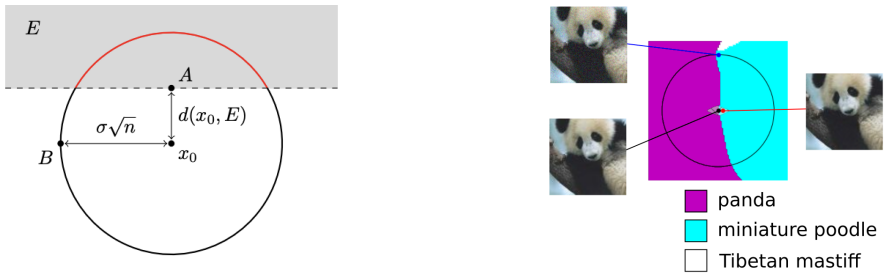


Figure 14.45: Effect of Gaussian noise of increasing magnitude on an image classifier. (The model is a ResNet-50 CNN (Sec. 14.3.4) trained on ImageNet with added Gaussian noise of magnitude $\sigma = 0.6$.) From Figure 23 of [For+19]. Used with kind permission of Justin Gilmer.



(a) (b)

Figure 14.46: (a) When the input dimension n is large and the decision boundary is locally linear, even a small error rate in random noise will imply the existence of small adversarial perturbations. Here, $d(\mathbf{x}_0, E)$ denotes the distance from a clean input $\mathbf{x}_0$ to an adversarial example (A) while the distance from $\mathbf{x}_0$ to a random sample $N(0; \sigma^2 I)$ (B) will be approximately $\sigma\sqrt{n}$. As $n \to \infty$ the ratio of $d(x_0, A)$ to $d(\mathbf{x}_0, B)$ goes to 0. (b) A 2d slice of the InceptionV3 decision boundary through three points: a clean image (black), an adversarial example (red), and an error in random noise (blue). The adversarial example and the error in noise lie in the same region of the error set which is misclassified as "miniature poodle", which closely resembles a halfspace as in Figure (a). Used with kind permission of Justin Gilmer.

noise can significantly effect the output of a high quality classifier.

A cartoon illustration of what is going on is shown in Figure 14.46a, where $\mathbf{x}_0$ is the clean input image, B is an image corrupted by Gaussian noise, and A is an adversarial image. If we assume a linear decision boundary, then the error set E is a half space a certain distance from $\mathbf{x}_0$. We can relate the distance to the decision boundary $d(\mathbf{x}_0, E)$ with the error rate in noise at some input $\mathbf{x}_0$, denoted by $\mu = \mathbb{P}_{\delta \sim N(0,\sigma I)}[\mathbf{x}_0 + \delta \in E]$. With a linear decision boundary the relationship between these two quantities is determined by

$$d(\mathbf{x}_0, E) = -\sigma \Phi^{-1}(\mu) \tag{14.47}$$

where Φ^{-1} denotes the inverse cdf of the gaussian distribution. When the input dimension is large, this distance will be significantly smaller than the distance to a randomly sampled noisy image $\mathbf{x}_0 + \delta$ for $\delta \sim N(0, \sigma I)$, as the noise term will with high propbability have norm $||\delta||_2 \approx \sigma\sqrt{d}$. As a concrete

example consider the ImageNet dataset, where $d = 224 \times 224 \times 3$ and suppose we set $\sigma = .2$. Then if the error rate in noise is just $\mu = .01$, equation 14.47 will imply that $d(\mathbf{x}_0, E) = .5$. Thus the distance to an adversarial example will be more than 100 times closer than the distance to a typical noisy images, which will be $\sigma\sqrt{d} \approx 77.6$. This phenomenon of small volume error sets being close to most points in a data distribution $p(\mathbf{x})$ is called **concentration of measure**, and is a property common among many high dimensional data distributions [MDM19; Gil+18b].

In summary, although the existence of adversarial examples is often discussed as an unexpected phenomenon, there is nothing special about the existence of worst-case errors for ML classifiers—they will always exist as long as errors exist.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

15 Neural networks for sequences

15.1 Introduction

In this chapter, we discuss various kinds of neural networks for sequences. We will consider the case where the input is a sequence, the output is a sequence, or both are sequences. Such models have many applications, such as machine translation, speech recognition, text classification, image captioning, etc. Our presentation borrows from parts of [Zha+20], which should be consulted for more details.

15.2 Recurrent neural networks (RNNs)

A **recurrent neural network** or **RNN** is a neural network which maps from an input space of sequences to an output space of sequences in a stateful way. That is, the prediction of output $\mathbf{y}_t$ depends not only on the input $\mathbf{x}_t$, but also on the hidden state of the system, $\mathbf{h}_t$, which gets updated over time, as the sequence is processed. Such models can be used for sequence generation, sequence classification, and sequence translation, as we explain below.

15.2.1 Vec2Seq (sequence generation)

In this section, we discuss how to learn functions of the form $f_{\boldsymbol{\theta}} : \mathbb{R}^D \to \mathbb{R}^{N_\infty C}$, where D is the size of the input vector, and the output is an arbitrary-length sequence of vectors, each of size C. (Note that words are discrete tokens, but can be converted to real-valued vectors as we discuss in Sec. 1.5.4.) We call these **vec2seq** models, since they map a vector to a sequence.

The output sequence $\mathbf{y}_{1:T}$ is generated one token at a time. At each step we sample $\tilde{y}_t$ from the hidden state $\mathbf{h}_t$ of the model, and then "feed it back in" to the model to get the new state $\mathbf{h}_{t+1}$ (which also depends on the input $\mathbf{x}$). See Fig. 15.1 for an illustration. In this way the model defines a conditional generative model of the form $p(\mathbf{y}_{1:T}|\mathbf{x})$, which captures dependencies between the output tokens. We explain this in more detail below.

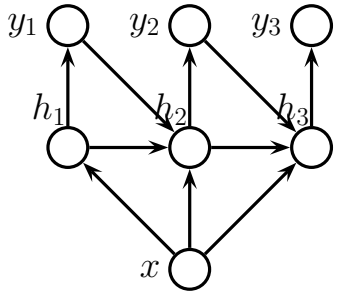


Figure 15.1: Recurrent neural network (RNN) for generating a variable length output sequence $\mathbf{y}_{1:T}$ given an optional fixed length input vector $\mathbf{x}$.

15.2.1.1 Models

For notational simplicity, let T be the length of the output (with the understanding that this is chosen dynamically). The RNN then corresponds to the following conditional generative model:

$$p(\mathbf{y}_{1:T}|\mathbf{x}) = \sum_{\mathbf{h}_{1:T}} p(\mathbf{y}_{1:T}, \mathbf{h}_{1:T}|\mathbf{x}) = \sum_{\mathbf{h}_{1:T}} \prod_{t=1}^{T} p(\mathbf{y}_t|\mathbf{h}_t) p(\mathbf{h}_t|\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) \tag{15.1}$$

where $\mathbf{h}_t$ is the hidden state, and where we define $p(\mathbf{h}_1|\mathbf{h}_0, \mathbf{y}_0, \mathbf{x}) = p(\mathbf{h}_1|\mathbf{x})$ as the initial hidden state distribution (often deterministic).

The output distribution is usually given by

$$p(\mathbf{y}_t|\mathbf{h}_t) = \text{Cat}(\mathbf{y}_t|\mathcal{S}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)) \tag{15.2}$$

where $\mathbf{W}_{hy}$ are the hidden-to-output weights, and $\mathbf{b}_y$ is the bias term. However, for real-valued outputs, we can use

$$p(\mathbf{y}_t|\mathbf{h}_t) = \mathcal{N}(\mathbf{y}_t|\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \sigma^2\mathbf{I}) \tag{15.3}$$

We assume the hidden state is computed deterministically as follows:

$$p(\mathbf{h}_t|\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) = \mathbb{I}\left(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x})\right) \tag{15.4}$$

for some deterministic function f. The update function f is usually given by

$$\mathbf{h}_t = \varphi(\mathbf{W}_{xh}[\mathbf{x}; \mathbf{y}_{t-1}] + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{15.5}$$

where $\mathbf{W}_{hh}$ are the hidden-to-hidden weights, $\mathbf{W}_{xh}$ are the input-to-hidden weights, and $\mathbf{b}_h$ are the bias terms. See Fig. 15.1 for an illustration, and code.probml.ai/book1/rnn_torch for some code.

Note that $\mathbf{y}_t$ depends on $\mathbf{h}_t$, which depends on $\mathbf{y}_{t-1}$, which depends on $\mathbf{h}_{t-1}$, and so on. Thus $\mathbf{y}_t$ implicitly depends on all past observations (as well as the optional fixed input $\mathbf{x}$). Thus an RNN overcomes the limitations of standard Markov models, in that they can have unbounded memory. This makes RNNs theoretically as powerful as a **Turing machine** [SS95; PMB19]. In practice,

the githa some thong the time traveller held in his hand was a glitteringmetallic framework scarcely larger than a small clock and verydelicately made there was ivory in it and the latter than s bettyre tat howhong s ie time thave ler simk you a dimensions le ghat dionthat shall travel indifferently in any direction of space and timeas the driver determinesfilby contented himself with laughterbut i have experimental verification said the time travellerit would be remarkably convenient for the histo

Figure 15.2: Example output of length 500 generated from a character level RNN when given the prefix "the". We use greedy decoding, in which the most likely character at each step is computed, and then fed back into the model. The model is trained on the book The Time Machine *by H. G. Wells. Generated by code at figures.probml.ai/book1/15.2.*

however, the memory length is determined by the size of the latent state and the strength of the parameters; see Sec. 15.2.7 for further discussion of this point.

When we generate from an RNN, we sample from $\tilde{\mathbf{y}}_t \sim p(\mathbf{y}_t|\mathbf{h}_t)$, and then "feed in" the sampled value into the hidden state, to deterministically compute $\mathbf{h}_{t+1} = f(\mathbf{h}_t, \tilde{\mathbf{y}}_t, \mathbf{x})$, from which we sample $\tilde{\mathbf{y}}_{t+1} \sim p(\mathbf{y}_{t+1}|\mathbf{h}_{t+1})$, etc. Thus the only stochasticity in the system comes from the noise in the observation (output) model, which is fed back to the system in each step. (However, there is a variant, known as a **variational RNN** [Chu+15], that adds stochasticity to the dynamics of $\mathbf{h}_t$ independent of the observation noise.)

15.2.1.2 Applications

RNNs can be used to generate sequences unconditionally (by setting $\mathbf{x} = \emptyset$) or conditionally on $\mathbf{x}$. Unconditional sequence generation is often called **language modeling**; this refers to learning joint probability distributions over sequences of discrete tokens, i.e., models of the form $p(y_1, \ldots, y_T)$. (See also Sec. 3.5.1.2, where we discuss using Markov chains for language modeling.)

Figure 15.2 shows a sequence generated from a simple RNN trained on the book *The Time Machine* by H. G. Wells. (This is a short science fiction book, with just 32,000 words and 170k characters.) We see that the generated sequence looks plausible, even though it is not very meaningful. By using more sophisticated RNN models (such as those that we discuss in Sec. 15.2.7.1 and Sec. 15.2.7.2), and by training on more data, we can create RNNs that give state-of-the-art performance on the language modeling task [CNB17]. (In the language modeling community, performance is usually measured by perplexity, which is just the exponential of the average per-token negative log likelihood; see Sec. 6.1.5 for more information.)

We can also make the generated sequence depend on some kind of input vector $\mathbf{x}$. For example, consider the task of **image captioning**: in this case, $\mathbf{x}$ is some embedding of the image computed by a CNN, as illustrated in Fig. 15.3. (In this example, $D = H \times W \times 3$ is the size of the image and C is the size of the vocabulary.) See e.g., [Hos+18] for a review of image captioning methods.

It is also possible to use RNNs to generate sequences of real-valued feature vectors, such as pen strokes for hand-written characters [Gra13] and hand-drawn shapes [HE18]. This can also be useful for time series forecasting real-value sequences.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

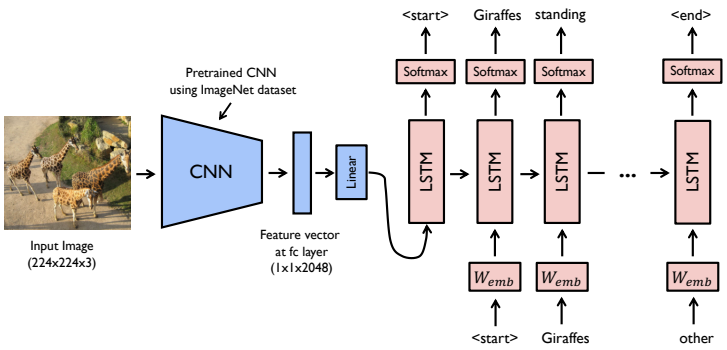


Figure 15.3: Illustration of a CNN-RNN model for image captioning. The pink boxes labeled "LSTM" refer to a specific kind of RNN that we discuss in Sec. 15.2.7.2. The pink boxes labeled W_{emb} refer to embedding matrices for the (sampled) one-hot tokens, so that the input to the model is a real-valued vector. From `https://bit.ly/2FKnqHm`. *Used with kind permission of Yunjey Choi.*

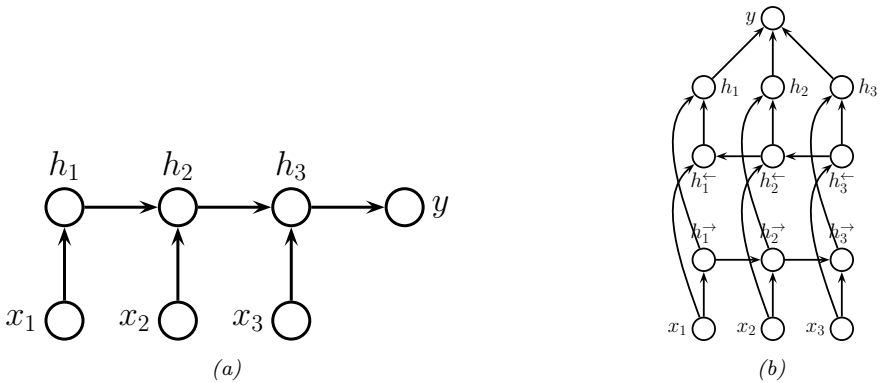


Figure 15.4: (a) RNN for sequence classification. (b) Bi-directional RNN for sequence classification.

15.2.2 Seq2Vec (sequence classification)

In this section, we assume we have a single fixed-length output vector $\mathbf{y}$ we want to predict, given a variable length sequence as input. Thus we want to learn a function of the form $f_{\boldsymbol{\theta}} : \mathbb{R}^{TD} \to \mathbb{R}^C$. We call this a **seq2vec** model. We will focus on the case where the output is a class label, $y \in \{1, \ldots, C\}$, for notational simplicity.

The simplest approach is to use the final state of the RNN as input to the classifier:

$$p(y|\mathbf{x}_{1:T}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{h}_T)) \tag{15.6}$$

See Fig. 15.4a for an illustration.

We can often get better results if we let the hidden states of the RNN depend on the past and future context. To do this, we create two RNNs, one which recursively computes hidden states in the

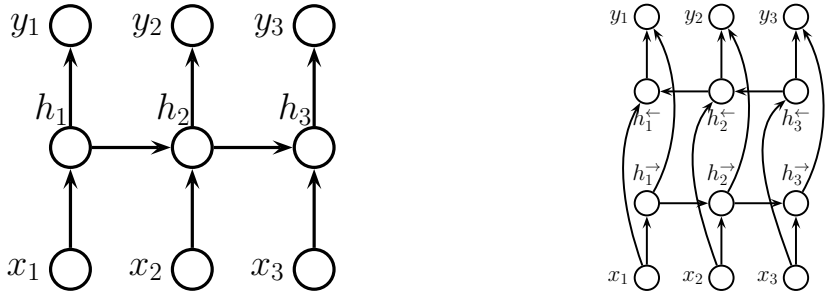


Figure 15.5: (a) RNN for transforming a sequence to another, aligned sequence. (b) Bi-directional RNN for the same task.

forwards direction, and one which recursively computes hidden states in the backwards direction. This is called a **bidirectional RNN** [SP97]. More precisely, the model is defined as follows:

$$\mathbf{h}_t^{\rightarrow} = \varphi(\mathbf{W}_{xh}^{\rightarrow}\mathbf{x}_t + \mathbf{W}_{hh}^{\rightarrow}\mathbf{h}_{t-1}^{\rightarrow} + \mathbf{b}_h^{\rightarrow}) \tag{15.7}$$

$$\mathbf{h}_t^{\leftarrow} = \varphi(\mathbf{W}_{xh}^{\leftarrow}\mathbf{x}_t + \mathbf{W}_{hh}^{\leftarrow}\mathbf{h}_{t+1}^{\leftarrow} + \mathbf{b}_h^{\leftarrow}) \tag{15.8}$$

We can then define $\mathbf{h}_t = [\mathbf{h}_t^{\rightarrow}, \mathbf{h}_t^{\leftarrow}]$ to be the representation of the state at time t, taking into account past and future information. Finally we average pool over these hidden states to get the final classifier:

$$p(y|\mathbf{x}_{1:T}) = \text{Cat}(y|\mathbf{W}\mathcal{S}(\overline{\mathbf{h}})) \tag{15.9}$$

$$\overline{\mathbf{h}} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{h}_t \tag{15.10}$$

See Fig. 15.4b for an illustration, and code.probml.ai/book1/rnn_sentiment_torch for some code. (This is similar to the 1d CNN text classifier1 in Sec. 15.3.1.)

15.2.3 Seq2Seq (sequence translation)

In this section, we consider learning functions of the form $f_{\boldsymbol{\theta}} : \mathbb{R}^{TD} \rightarrow \mathbb{R}^{T'C}$. We consider two cases: one in which $T' = T$, so the input and output sequences have the same length (and hence are aligned), and one in which $T' \neq T$, so the input and output sequences have different lengths. This is called a **seq2seq** problem.

15.2.3.1 Aligned case

In this section, we consider the case where the input and output sequences are aligned. We can also think of it as **dense sequence labeling**, since we predict one label per location. It is straightforward to modify an RNN to solve this task, as shown in Fig. 15.5a. This corresponds to

$$p(\mathbf{y}_{1:T}|\mathbf{x}_{1:T}) = \sum_{\mathbf{h}_{1:T}}\prod_{t=1}^{T} p(\mathbf{y}_t|\mathbf{h}_t)\mathbb{I}\left(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)\right) \tag{15.11}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

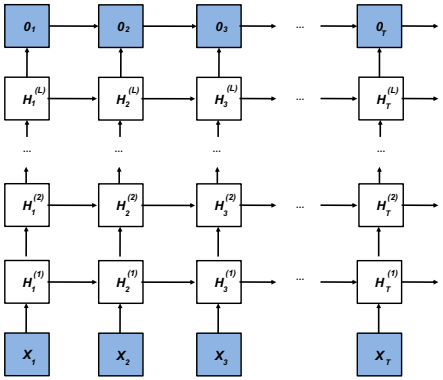


Figure 15.6: Illustration of a deep RNN. Adapted from Figure 9.3.1 of [Zha+20].

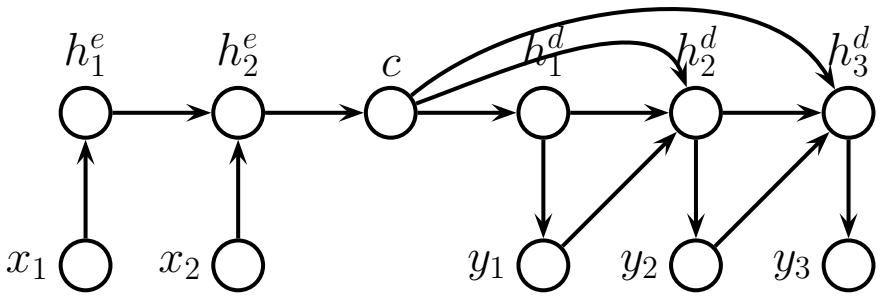


Figure 15.7: Encoder-decoder RNN architecture for mapping sequence $\mathbf{x}_{1:T}$ to sequence $\mathbf{y}_{1:T'}$.

where we define $\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1) = f_0(\mathbf{x}_1)$ to be the initial state.

Note that $\mathbf{y}_t$ depends on $\mathbf{h}_t$ which only depends on the past inputs, $\mathbf{x}_{1:t}$. We can get better results if we let the decoder look into the "future" of $\mathbf{x}$ as well as the past, by using a bidirectional RNN, as shown in Fig. 15.5b.

We can create more expressive models by stacking multiple hidden chains on top of each other, as shown in Fig. 15.6, The hidden units for layer l at time t are computed using

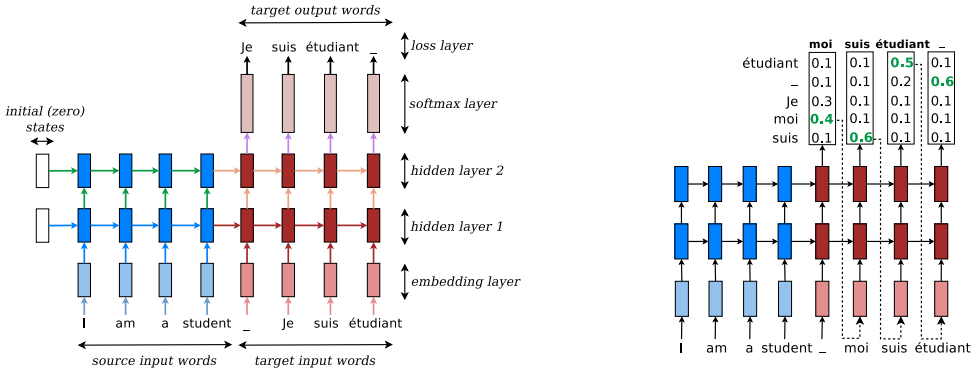
$$\mathbf{H}_t^l = \varphi_l(\mathbf{H}_t^{l-1}\mathbf{W}_{xh}^l + \mathbf{H}_{t-1}^l\mathbf{W}_{hh}^l + \mathbf{b}_h^l) \tag{15.12}$$

The output is given by

$$\mathbf{O}_t = \mathbf{H}_t^L\mathbf{W}_{ho} + \mathbf{b}_o \tag{15.13}$$

15.2.3.2 Unaligned case

In this section, we discuss how to learn a mapping from one sequence of length T to another of length T'. We first encode the input sequence to get the context vector $\mathbf{c} = f_e(\mathbf{x}_{1:T})$, using the last state of an RNN (or average pooling over a biRNN). We then generate the output sequence using an RNN



(a) (b)

Figure 15.8: (a) Illustration of a seq2seq model for translating English to French. The - character represents the end of a sentence. From Figure 2.4 of [Luo16]. Used with kind permission of Minh-Thang Luong. (b) Illustration of greedy decoding. The most likely French word at each step is highlighted in green, and then fed in as input to the next step of the decoder. From Figure 2.5 of [Luo16]. Used with kind permission of Minh-Thang Luong.

decoder $\mathbf{y}_{1:T'} = f_d(\mathbf{c})$. This is called an **encoder-decoder architecture** [SVL14; Cho+14b]. See Fig. 15.7 for an illustration.

An important application of this is **machine translation**. When this is tackled using RNNs, it is called **neural machine translation** (as opposed to the older approach called **statistical machine translation**, that did not use neural networks). See Fig. 15.8a for the basic idea, and code.probml.ai/book1/nmt_torch for some code which has more details. For a review of the NMT literature, see [Luo16; Neu17].

15.2.4 Teacher forcing

When training a language model, the likelihood of a sequence of words $w_1, v_2, \ldots, w_T$, is given by

$$p(\mathbf{w}_{1:T}) = \prod_{t=1}^{T} p(w_t|\mathbf{w}_{1:t-1}) \tag{15.14}$$

In an RNN, we therefore set the input to $x_t = w_{t-1}$ and the output to $y_t = w_t$. Note that we condition on the *ground truth* labels from the past, $\mathbf{w}_{1:t-1}$, not labels generated from the model. This is called **teacher forcing**, since the teacher's values are "force fed" into the model as input at each step (i.e., x_t is set to w_{t-1}).

Unfortunately, teacher forcing can sometimes result in models that perform poorly at test time. The reason is that the model has only ever been trained on inputs that are "correct", so it may not know what to do if, at test time, it encounters an input sequence $\mathbf{w}_{t-1}$ generated from the previous step that deviates from what it saw in training.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

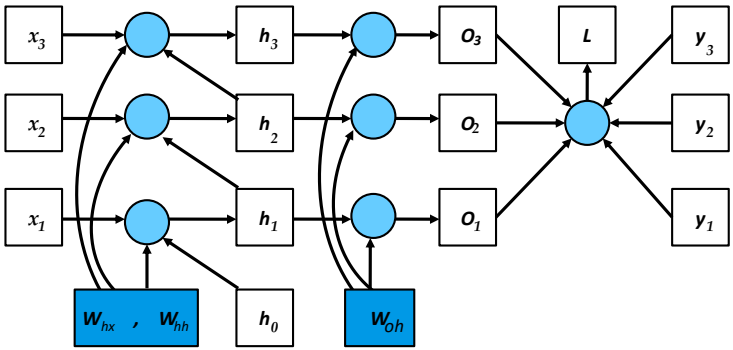


Figure 15.9: An RNN unrolled (vertically) for 3 time steps, with the target output sequence and loss node shown explicitly. From Figure 8.7.2 of [Zha+20]. Used with kind permission of Aston Zhang.

A common solution to this is known as **scheduled sampling** [Ben+15a]. This starts off using teacher forcing, but at random time steps, feeds in in samples from the model instead; the fraction of time this happens is gradually increased.

An alternative solution is to use other kinds of models where MLE training works better, such as 1d CNNs (Sec. 15.3) and transformers (Sec. 15.5).

15.2.5 Backpropagation through time

We can compute the maximum likelihood estimate of the parameters for an RNN by solving $\boldsymbol{\theta}^* = \text{argmax}_{\boldsymbol{\theta}} \, p(\mathbf{y}_{1:T}|\mathbf{x}_{1:T}, \boldsymbol{\theta})$, where we have assumed a single training sequence for notational simplicity. To compute the MLE, we have to compute gradients of the loss wrt the parameters. To do this, we can unroll the computation graph, as shown in Fig. 15.9, and then apply the backpropagation algorithm. This is called **backpropagation through time** (BPTT) [Wer90].

More precisely, consider the following model:

$$\mathbf{h}_t = \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} \tag{15.15}$$

$$\mathbf{o}_t = \mathbf{W}_{ho}\mathbf{h}_t \tag{15.16}$$

where $\mathbf{o}_t$ are the output logits, and where we drop the bias terms for notational simplicity. We assume y_y are the true target labels for each time step, so we define the loss to be

$$L = \frac{1}{T}\sum_{t=1}^{T} \ell(y_t, \mathbf{o}_t) \tag{15.17}$$

We need to compute the derivatives $\frac{\partial L}{\partial \mathbf{W}_{hx}}$, $\frac{\partial L}{\partial \mathbf{W}_{hh}}$, and $\frac{\partial L}{\partial \mathbf{W}_{ho}}$. The latter term is easy, since it is local to each time step. However, the first two terms depend on the hidden state, and thus require working backwards in time.

To see this, let us simplify the notation by defining

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{-t1}, \mathbf{w}_h) \tag{15.18}$$

$$\mathbf{o}_t = g(\mathbf{h}_t, \mathbf{w}_o) \tag{15.19}$$

where $\mathbf{w}_h$ is the flattened version of $\mathbf{W}_{hh}$ and $\mathbf{W}_{hx}$ stacked together. We focus on computing $\frac{\partial L}{\partial \mathbf{w}_h}$. By the chain rule, we have

$$\frac{\partial L}{\partial \mathbf{w}_h} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial \ell(y_t, \mathbf{o}_t)}{\partial \mathbf{w}_h} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial \ell(y_t, \mathbf{o}_t)}{\partial \mathbf{o}_t} \frac{\partial g(\mathbf{h}_t, \mathbf{w}_o)}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} \tag{15.20}$$

We can expand the last term as follows:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{w}_h} \tag{15.21}$$

If we expand this recursively, we find

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^{t} \frac{\partial f(\mathbf{x}_j, \mathbf{h}_{j-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial f(\mathbf{x}_i, \mathbf{h}_{i-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} \tag{15.22}$$

Unfortunately, this takes $O(T)$ time to compute per time step, for a total of $O(T^2)$ overall. It is therefore standard to truncate the sum to the most recent K terms. It is possible to adaptively pick a suitable truncation parameter K [AFF19]; however, it is usually set equal to the length of the subsequence in the current minibatch.

When using truncated BPTT, we can train the model with batches of short sequences, usually created by extracting non-overlapping subsequences (windows) from the original sequence. If the previous subsequence ends at time $t-1$, and the current subsequence starts at time t, we can "carry over" the hidden state of the RNN across batch updates during training. However, if the subsequences are not ordered, we need to reset the hidden state. See code.probml.ai/book1/rnn_torch for some sample code that illustrates these details.

15.2.6 Vanishing and exploding gradients

Unforunately, the activations in an RNN can decay or explode as we go forwards in time, since we multiply by the weight matrix $\mathbf{W}_{hh}$ at each time step. Similarly, the gradients in an RNN can decay or explode as we go backwards in time, since we multiply the Jacobians at each time step (see Sec. 13.4.2 for details). A simple heuristic is to use gradient clipping (Eq. (13.84)). More sophisticated methods attempt to control the spectral radius λ of the forward mapping, $\mathbf{W}_{hh}$, as well as the backwards mapping, given by the Jacobian $\mathbf{J}_{hh}$.

The simplest way to control the spectral radius is to randomly initialize $\mathbf{W}_{hh}$ in such a way as to ensure $\lambda \approx 1$, and then keep it fixed (i.e., we do not learn $\mathbf{W}_{hh}$). In this case, only the output matrix $\mathbf{W}_{ho}$ needs to be learned, resulting in a convex optimization problem. This is called an **echo state network** [JH04]. A closely related approach, known as a **liquid state machine** [MNM02], uses binary-valued (spiking) neurons instead of real-valued neurons. A generic term for both ESNs

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

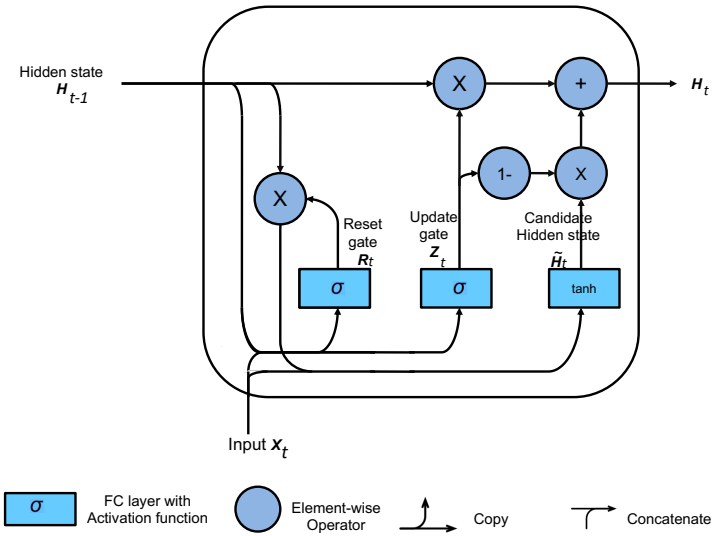


Figure 15.10: Illustration of a GRU. Adapted from Figure 9.1.3 of [Zha+20].

and LSMs is **reservoir computing** [LJ09]. Another approach to this problem is use constrained optimization to ensure the $\mathbf{W}_{hh}$ matrix remains orthogonal [Vor+17].

An alternative to explicitly controlling the spectral radius is to modify the RNN architecture itself, to use additive rather than multiplicative updates to the hidden states, as we discuss in Sec. 15.2.7. This significantly improves training stability.

15.2.7 Gating and long term memory

RNNs with enough hidden units can in principle remember inputs from long in the past. However, in practice "vanilla" RNNs fail to do this because of the vanishing gradient problem (Sec. 13.4.2). In this section we give a solution to this in which we update the hidden state in an additive way, similar to a residual net (Sec. 14.3.4).

15.2.7.1 Gated recurrent units (GRU)

In this section, we discuss models which use **gated recurrent units** (**GRU**), as proposed in [Cho+14b]. The key idea is to learn when to update the hidden state, by using a gating unit. This can be used to selectively "remember" important pieces of information when they are first seen. The model can also learn when to reset the hidden state, and thus forget things that are no longer useful.

To explain the model in more detail, we present it in two steps, following the presentation of [Zha+20, Sec 8.8]. We assume $\mathbf{X}_t$ is a $N \times D$ matrix, where N is the batch size, and D is the vocabulary size. Similarly, $\mathbf{H}_t$ is a $N \times H$ matrix, where H is the number of hidden inits at time t.

The **reset gate** $\mathbf{R}_t \in \mathbb{R}^{N \times H}$ and **update gate** $\mathbf{Z}_t \in \mathbb{R}^{N \times H}$ are computed using

$$\mathbf{R}_t = \sigma(\mathbf{W}_{xr}\mathbf{X}_t + \mathbf{H}_{t-1}\mathbf{W}_{hr} + \mathbf{b}_r) \tag{15.23}$$

$$\mathbf{Z}_t = \sigma(\mathbf{W}_{xz}\mathbf{X}_t + \mathbf{H}_{t-1}\mathbf{W}_{hz} + \mathbf{b}_z) \tag{15.24}$$

Note that each element of $\mathbf{R}_t$ and $\mathbf{Z}_t$ is in $[0, 1]$, because of the sigmoid function.

Given this, we we define a "candidate" next state vector using

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1})\mathbf{W}_{hh} + \mathbf{b}_h) \tag{15.25}$$

This combines the old memories that are not reset (computed using $\mathbf{R}_t \odot \mathbf{H}_{t-1}$) with the new inputs $\mathbf{X}_t$. We pass the resulting linear combination through a tanh function to ensure the hidden units remain in the interval $(-1, 1)$. If the entries of the reset gate $\mathbf{R}_t$ are close to 1, we recover the standard RNN update rule. If the entries are close to 0, the model acts more like an MLP applied to $\mathbf{X}_t$. Thus the reset gate can capture new, short-term information.

Once we have computed the candidate new state, the model computes the actual new state by using the dimensions from the candidate state $\tilde{\mathbf{H}}_t$ chosen by the update gate, $1 - \mathbf{z}_t$), and keeping the remaining dimensions at their old values of $\mathbf{H}_{t-1}$:

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t \tag{15.26}$$

When $Z_{td} = 1$, we pass $H_{t-1,d}$ through unchanged, and ignore $\mathbf{X}_t$. Thus the update gate can capture long-term dependencies.

See Fig. 15.10 for an illustration of the overall architecture, and code.probml.ai/book1/gru_torch for some sample code.

15.2.7.2 Long short term memory (LSTM)

In this section, we discuss the **long short term memory** (**LSTM**) model of [HS97b], which is a more sophisticated version of the GRU (and pre-dates it by almost 20 years).

The basic idea is to augment the hidden state $\mathbf{h}_t$ with a **memory cell** $\mathbf{c}_t$. We need three gates to controll this cell: the **output gate** $\mathbf{O}_t$ determines what gets read out; the **input gate** $\mathbf{I}_t$ determines what gets read in; and the **forget gate** $\mathbf{F}_t$ determes when we should reset the cell. These gates are computed as follows:

$$\mathbf{O}_t = \sigma(\mathbf{X}_t\mathbf{X}_{xo} + \mathbf{H}_{t-1}\mathbf{W}_{ho} + \mathbf{b}_o) \tag{15.27}$$

$$\mathbf{I}_t = \sigma(\mathbf{X}_t\mathbf{X}_{xi} + \mathbf{H}_{t-1}\mathbf{W}_{hi} + \mathbf{b}_i) \tag{15.28}$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t\mathbf{X}_{xf} + \mathbf{H}_{t-1}\mathbf{W}_{hf} + \mathbf{b}_f) \tag{15.29}$$

We then compute a candidate cell state:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{xc} + \mathbf{H}_{t-1}\mathbf{W}_{hc} + \mathbf{b}_c) \tag{15.30}$$

The actual update to the cell is either the candidate cell (if the input gate is on) or the old cell (if the not-forget gate is on):

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \tag{15.31}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

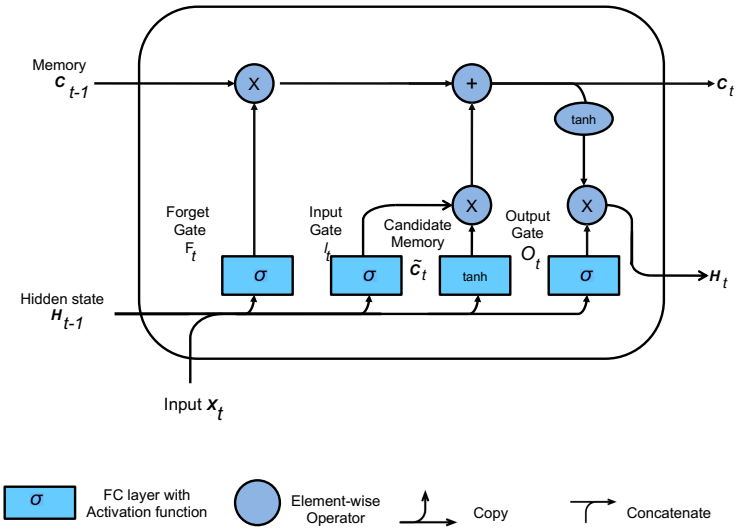


Figure 15.11: Illustration of an LSTM. Adapted from Figure 9.2.4 of [Zha+20].

If $\mathbf{F}_t = 1$ and $\mathbf{I}_t = 0$, this can remember long term memories.[1]

Finally, we compute the hidden state to be a transformed version of the cell, provided the output gate is on:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \tag{15.32}$$

Note that $\mathbf{H}_t$ is used as the output of the unit as well as the hidden state for the next time step. This lets the model remember what it has just output (short-term memory), whereas the cell $\mathbf{C}_t$ acts as a long-term memory. See Fig. 15.11 for an illustration of the overall model, and code.probml.ai/book1/lstm_torch for some sample code.

Sometimes we add **peephole connections**, where we pass the cell state as an additional input to the gates. Many other variants have been proposed. In fact, [JZS15] used genetic algorithms to test over 10,000 different architectures. Some of these worked better than LSTMs or GRUs, but in general, LSTMs seemed to do consistently well across most tasks. Similar conclusions were reached in [Gre+17]. More recently, [ZL17] used an RNN controller to generate strings which specify RNN architectures, and then trained the controller using reinforcement learning. This resulted in a novel cell structure that outperformed LSTM. However, it is rather complex and has not been adopted by the community.

1. One important detail pointed out in [JZS15] is that we need to initialize the bias term for the forget gate $\mathbf{b}_f$ to be large, so the sigmoid is close to 1. This ensures that information can easily pass through the $\mathbf{C}$ chain over time. Without this trick, performance is often much worse.

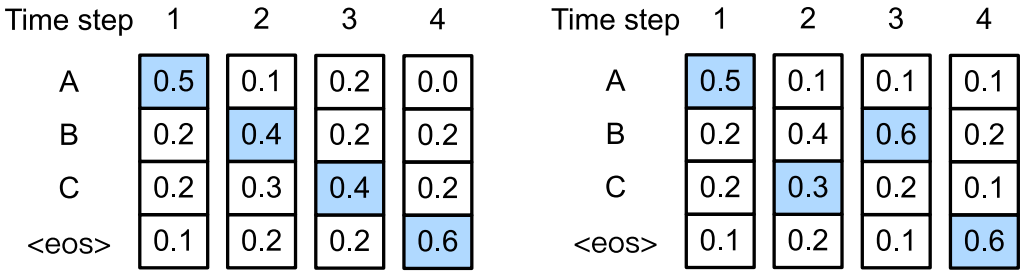


Figure 15.12: Conditional probabilities of generating each token at each step for two different sequences. From Figures 9.8.1–9.8.2 of [Zha+20]. Used with kind permission of Aston Zhang.

15.2.8 Beam search

The simplest way to generate from an RNN is to use **greedy decoding**, in which we compute $\hat{y}_t = \text{argmax}_y \, p(y_t = y|\hat{\mathbf{y}}_{1:t}, \mathbf{x})$ at each step. We can repeat this process until we generate the end-of-sentence token. See Fig. 15.8b for an illustration of this method applied to NMT.

Unfortunately greedy decoding will not generate the MAP sequence, which is defined by $\mathbf{y}^*_{1:T} = \text{argmax}_{\mathbf{y}_{1:T}} \, p(\mathbf{y}_{1:T}|\mathbf{x})$. The reason is that the locally optimal symbol at step t might not be on the globally optimal path.

As an example, consider Fig. 15.12a. We greedily pick the MAP symbol at step 1, which is A. Conditional on this, suppose we have $p(y_2|y_1 = A) = [0.1, 0.4, 0.3, 0.3]$, as shown. We greedily pick the MAP symbol from this, which is B. Conditional on this, suppose we have $p(y_3|y_1 = A, y_2 = B) = [0.2, 0.2, 0.4, 0.2]$, as shown. We greedily pick the MAP symbol from this, which is C. Conditional on this, suppose we have $p(y_4|y_1 = A, y_2 = B, y_3 = C) = [0.0, 0.2, 0.2, 0.6]$, as shown. We greedily pick the MAP symbol from this, which is eos (end of sentence), so we stop generating. The overall probability of the generated sequence is $0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$.

Now consider Fig. 15.12b. At step 2, suppose we pick the second most probable token, namely C. Conditional on this, suppose we have $p(y_3|y_1 = A, y_2 = C) = [0.1, 0.6, 0.2, 0.1]$, as shown. We greedily pick the MAP symbol from this, which is B. Conditional on this, suppose we have $p(y_4|y_1 = A, y_2 = C, y_3 = B) = [0.0, 0.2, 0.2, 0.6]$, as shown. We greedily pick the MAP symbol from this, which is eos (end of sentence), so we stop generating. The overall probability of the generated sequence is $0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$. So by being less greedy, we found a sequence with overall higher likelihood.

For hidden Markov models, we can use an algorithm called **Viterbi decoding** (which is an example of **dynamic programming**) to compute the globally optimal sequence in $O(TV^2)$ time, where V is the number of words in the vocabulary. (See [Mur22] for details.) But for RNNs, computing the global optimum takes $O(V^T)$, since the hidden state is not a sufficient statistic for the data.

Beam search is a much faster heuristic method. In this approach, we compute the top K candidate outputs at each step; we then expand each one in all V possible ways, to generate VK candidates, from which we pick the top K again. This process is illustrated in Fig. 15.13.

If we are able to devise a **heuristic function** to approximate the overall probability of any partial

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

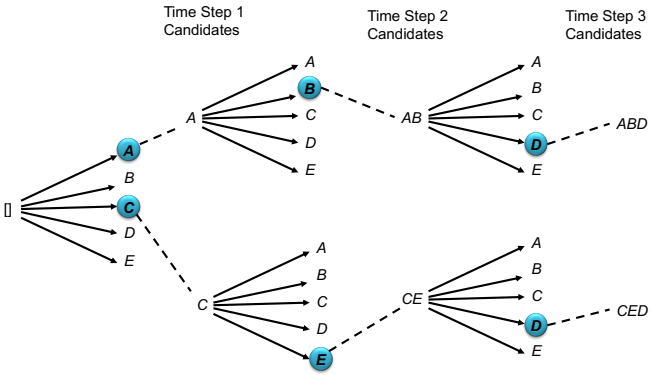


Figure 15.13: Illustration of beam search using a beam of size $K = 2$. The vocabulary is $\mathcal{Y} = \{A, B, C, D, E\}$, with size $V = 5$. We assume the top 2 symbols at step 1 are A,C. At step 2, we evaluate $p(y_1 = A, y_2 = y)$ and $p(y_1 = C, y_2 = y)$ for each $y \in \mathcal{Y}$. This takes $O(KV)$ time. We then pick the top 2 partial paths, which are $(y_1 = A, y_2 = B)$ and $(y_1 = C, y_2 = E)$, and continue in the obvious way. Adapted from Figure 9.8.3 of [Zha+20].

path, then we can use this to guide the search; this is the basis of A^* **search** and **Monte Carlo tree search**. This has been applied to (conditional) language generation in [Kum+18].

15.3 1d CNNs

Convolutional neural networks (Chapter 14) compute a function of some local neighborhood for each input using tied weights, and return an output. They are usually used for 2d inputs, but can also be applied in the 1d case, as we discuss below. They are an interesting alternative to RNNs that are much easier to train, because they don't have to maintain long term hidden state.

15.3.1 1d CNNs for sequence classification

In this section, we discuss the use of 1d CNNs for learning a mapping from variable-length sequences to a fixed length output, i.e., a function of the form $f_{\boldsymbol{\theta}} : \mathbb{R}^{DT} \to \mathbb{R}^C$, where T is the length of the input, D is the number of features per input, and C is the size of the output vector (e.g., class logits).

A basic 1d convolution operation applied to a 1d sequence is shown in Fig. 14.4. Typically the input sequence will have $D > 1$ input channels (feature dimensions). In this case, we can convolve each channel separately and add up the result, using a different 1d filter (kernel) for each input channel to get $z_i = \sum_d \mathbf{x}_{i-k:i+k,d}^\top \mathbf{w}_d$, where k is size of the 1d receptive field, and $\mathbf{w}_d$ is the filter for input channel d. This produces a 1d vector $\mathbf{z} \in \mathbb{R}^T$ encoding the input (ignoring boundary effects). We can create a vector representation for each location using a different weight vector for each output channel c to get $z_{ic} = \sum_d \mathbf{x}_{i-k:i+k,d}^\top \mathbf{w}_{d,c}$. This implements a mapping from TD to TC. To reduce this to a fixed sized vector, $\mathbf{z} \in \mathbb{R}^C$, we can use max-pooling over time to get $z_c = \max_i z_{ic}$. We can then pass this into a softmax layer.

In [Kim14], they applied this model to sequence classification. The idea is to embed each word

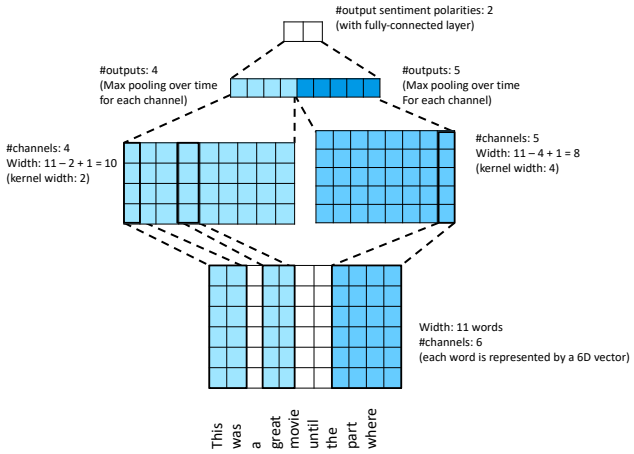


Figure 15.14: Illustration of the TextCNN model for binary sentiment classification. Adapted from Figure 15.3.5 of [Zha+20].

using an embedding layer, and then to compute various features using 1d kernels of different widths, to capture patterns of different length scales. We then apply max pooling over time, and concatenate the results, and pass to a fully connected layer. See Fig. 15.14 for an illustration, and code.probml.ai/book1/cnn1d_sentiment_torch for some code.

15.3.2 Causal 1d CNNs for sequence generation

To use 1d CNNs in a generative setting, we must convert them to a **causal CNN**, in which each output variable only depends on previously generated variables. (This is also called a **convolutional Markov model**.) In particular, we define the model as follows:

$$p(\mathbf{y}|\mathbf{x}) = \prod_{t=1}^{T} p(y_t|\mathbf{x}_{1:y-m}, \mathbf{x}) \tag{15.33}$$

where m is the memory length of the model, $\mathbf{x}$ is some optional conditioning information, T is the length of the output, and $p(y_t|\mathbf{y}_{1:t-m}, \mathbf{x})$ is implemented using **causal convolution**. This is like regular 1d convolution except we "mask out" future inputs, so that y_t only depends on the past values, rather than past and future values. In order to capture long-range dependencies, we can use dilated convolution (Sec. 14.4.1), as illustrated in Fig. 15.15.

This model has been successfully used to create a state of the art **text to speech** (TTS) synthesis system known as **wavenet** [oor+16]. In particular, they stack 10 causal 1d convolutional layers with dilation rates $1, 2, 4, \ldots, 256, 512$ to get a convolutional block with an effective receptive field of 1024. (They left-padded the input sequences with a number of zeros equal to the dilation rate before every layer, so that every layer has the same length.) They then repeat this block 3 times to compute deeper features.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

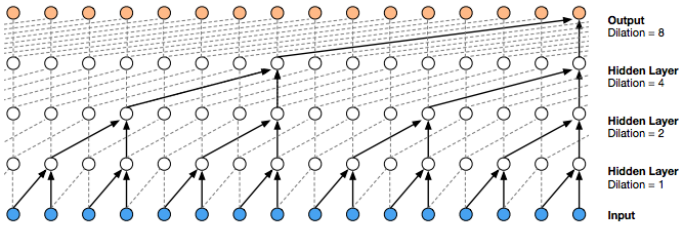


Figure 15.15: Illustration of the wavenet model using dilated (atrous) convolutions, with dilation factors of 1, 2, 4 and 8. From Figure 3 of [oor+16]. Used with kind permission of Aaron van den Oord.

In wavenet, the conditioning information $\mathbf{x}$ is a set of linguistic features derived from an input sequence of words; the model then generates raw audio using the above model. It is also possible to create a fully end-to-end approach, which starts with raw words rather than linguistic features (see [Wan+17]).

Although wavenet produces high quality speech, it is too slow for use in production systems. However, it can be "distilled" into a parallel generative model [Oor+18]. We discuss these kinds of parallel generative models in the sequel to this book, [Mur22].

15.4 Attention

In all of the neural networks we have considered so far, the hidden activations are a linear combination of the input activations, followed by a nonlinearity: $z = \varphi(\mathbf{w}^\top \mathbf{v})$, where $\mathbf{v} \in \mathbb{R}^v$ are the hidden feature vectors, and $\mathbf{w} \in \mathbb{R}^v$ are a fixed set of weights that are learned on a training set.

However, we can imagine a more flexible model in which we have a set of m feature vectors or **values** $\mathbf{V} \in \mathbb{R}^{m \times v}$, and the model dynamically decides (in an input dependenent way) which one to use, based on how similar the input **query** vector $\mathbf{q} \in \mathbb{R}^q$ is to a set of m **keys** $\mathbf{K} \in \mathbb{R}^{m \times k}$. If $\mathbf{q}$ is most similar to key i, then we use value $\mathbf{v}_i$. This is the basic idea behind **attention mechanisms**. This idea was originally developed for sequence models, and we will therefore explain it in this context. However, it can be more generally applied. Our presentation in the following sections is based on [Zha+20, Chap 10.].

15.4.1 Attention as soft dictionary lookup

We can think of attention as a dictionary lookup, in which we compare the query $\mathbf{q}$ to each key $\mathbf{k}_i$, and then retrieve the corresponding value $\mathbf{v}_i$. To make this lookup operation differentiable, instead of retrieving a single value $\mathbf{v}_i$, we compute a convex combination of the values, as follows:

$$\text{Attn}(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)) = \text{Attn}(\mathbf{q}, \{(\mathbf{k}_i, \mathbf{v}_i)\}) = \sum_{i=1}^{m} \alpha_i(\mathbf{q}, \{\mathbf{k}_i\}) \mathbf{v}_i \in \mathbb{R}^v \tag{15.34}$$

where $\alpha_{1:m}(\mathbf{q}, \{\mathbf{k}_i\})\}$ are a set of **attention weights** that satisfy $0 \leq \alpha_i(\mathbf{q}, \{\mathbf{k}_i\}) \leq 1$ for each i and $\sum_i \alpha_i(\mathbf{q}, \{\mathbf{k}_i\}) = 1$.

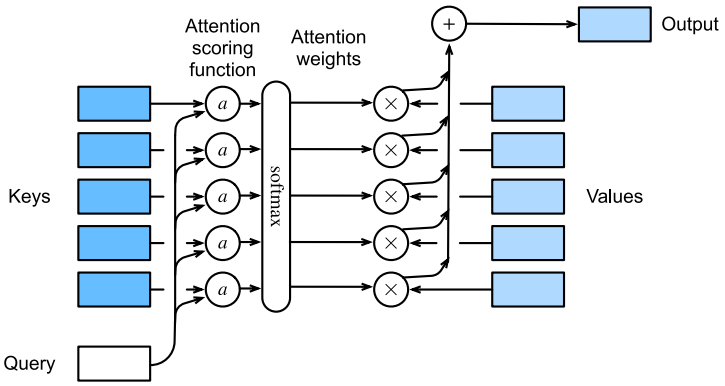


Figure 15.16: Attention computes a weighted average of a set of values, where the weights are derived by comparing the query vector to a set of keys. From Figure 10.3.1 of [Zha+20]. Used with kind permission of Aston Zhang.

The attention weights can be computed from an **attention score** function $a(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}$, that computes the similarity of query $\mathbf{q}$ to key $\mathbf{k}_i$. We will discuss several such score function below. Given the scores, we can compute the attention weights using the softmax function:

$$\alpha_i(\mathbf{q}, \{\mathbf{k}_i\}) = \mathcal{S}_i([a(\mathbf{q}, \mathbf{k}_1), \ldots, a(\mathbf{q}, \mathbf{k}_m)]) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^{m} \exp(a(\mathbf{q}, \mathbf{k}_j))} \tag{15.35}$$

See Fig. 15.16 for an illustration.

In some cases, we want to restrict attention to a subset of the dictionary, corresponding to valid entries. For example, we might want to pad sequences to a fixed length (for efficient minibatching), in which case we should "mask out" the padded locations. This is called **masked attention**. We can implement this efficient by setting the attention score for the masked entries to a large negative number, such as 10^6, so that the corresponding softmax weights will be 0. (This is analogous to causal convolution, discussed in Sec. 15.3.2.)

15.4.2 Kernel regression as non-parametric attention

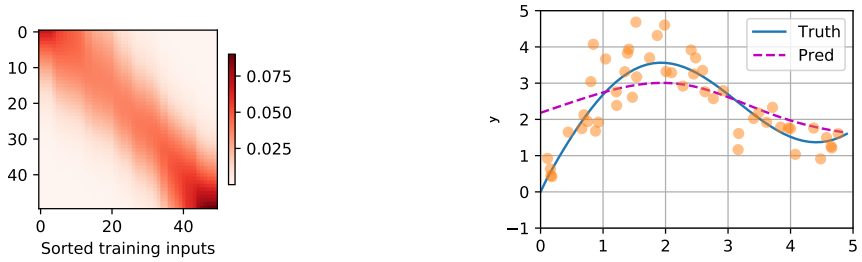
In Sec. 16.3.5, we discuss kernel regression, which is a nonparametric model of the form

$$f(x) = \sum_{i=1}^{n} \alpha_i(x, \{x_i\}) y_i \tag{15.36}$$

where $\alpha_i(x, \{x_i\}) \geq 0$ measures the normalized similarity of test input x to training input x_i. This similarity measure is usually computed by defining the attention score in terms of a a density kernel, such as the Gaussian:

$$\mathcal{K}_\sigma(u) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}u^2} \tag{15.37}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 15.17: Kernel regression in 1d. (a) Kernel weight matrix. (b) Resulting predictions on a dense grid of test points. Generated by code at figures.probml.ai/book1/15.17.

where σ is called the bandwidth. We then define $a(x, x_i) = \mathcal{K}_\sigma(x - x_i)$.

Because the scores are normalized, we can drop the $\frac{1}{\sqrt{2\pi\sigma^2}}$ term. In addition, to maintain notation consistency with [Zha+20, Ch. 10], we rewrite the term inside the exponential as follows:

$$\mathcal{K}(x; w) = \exp(-\frac{w^2}{2}u^2) \tag{15.38}$$

Plugging this in to Eq. (15.36), we get

$$f(x) = \sum_{i=1}^{n} \alpha_i(x, \{x_i\})y_i \tag{15.39}$$

$$= \sum_{i=1}^{n} \frac{\exp(-\frac{1}{2}((x - x_i)w)^2}{\sum_{j=1}^{n} \exp(-\frac{1}{2}((x - x_j)w)^2} y_i \tag{15.40}$$

$$= \sum_{i=1}^{N} \mathcal{S}_i(\{-\frac{1}{2}((x - x_j)w)^2\})y_i \tag{15.41}$$

We can interpret this as a form of nonparametric attention, where the queries are the test points x, the keys are the training inputs x_i, and the values are the training labels y_i.

If we set $w = 1$, the resulting attention matrix $A_{ji} = \alpha_i(x_j^*, \{x_i\})$ for test input j is shown in Fig. 15.17a. The resulting predicted curve is shown in Fig. 15.17b.

The size of the diagonal band in Fig. 15.17a, and hence the sparsity of the attention mechanism, dependends on the parameter w. If we increase w, corresponding to reducing the kernel bandwidth, the band will get narrower, but the model will start to overfit.

15.4.3 Parametric attention

In Sec. 15.4.2, we defined the attention score in terms of the Gaussian kernel, comparing a scalar query (test point) to each of the scalar values in the training set. This does not scale well to large training sets, or high-dimensional inputs. We will therefore turn our attention to parametric models,

where we have a fixed set of keys and values, and where we compare queries and keys in a learned embedding space.

There are several ways to do this. In the general case, the query $\mathbf{q} \in \mathbb{R}^q$ and the key $\mathbf{k} \in \mathbb{R}^k$ may have different sizes. To compare them, we can map them to a common embedding space of size h by computing $\mathbf{W}_q\mathbf{q}$ and $\mathbf{W}_k\mathbf{k}$. where $\mathbf{W}_q \in \mathbb{R}^{h\times q}$ and $\mathbf{W}_k \in \mathbb{R}^{h\times k}$. We can them pass these into an MLP to get the following **additive attention** scoring function:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}) \in \mathbb{R} \tag{15.42}$$

A more computationally efficient approach is to assume the queries and keys both have length d, so we can compute $\mathbf{q}^\top\mathbf{k}$ directly. If we assume these are independent random variables with 0 mean and unit variance, the mean of their inner product is 0, and the variance is d. (This follows from Eq. (2.33) and Eq. (2.38).) To ensure the variance of the inner product remains 1 regardless of the size of the inputs, it is standard to divide by $\sqrt{d}$. This gives rise to the **scaled dot-product attention**:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top\mathbf{k}/\sqrt{d} \in \mathbb{R} \tag{15.43}$$

In practice, we usually deal with minibatches of n vectors at a time. Let the corresponding matrices of queries, keys and values be denoted by $\mathbf{Q} \in \mathbb{R}^{n\times d}$, $\mathbf{K} \in \mathbb{R}^{m\times d}$, $\mathbf{V} \in \mathbb{R}^{m\times v}$. Then we can compute the attention-weighted outputs as follows:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathcal{S}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}})\mathbf{V} \in \mathbb{R}^{n\times v} \tag{15.44}$$

where the softmax fucntion $\mathcal{S}$ is applied row-wise. See code.probml.ai/book1/attention_torch for some sample code.

15.4.4 Seq2Seq with attention

Recall the seq2seq model from Sec. 15.2.3. This uses an RNN decoder of the form $\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{c})$, where $\mathbf{c}$ is a fixed-length context vector, representing the encoding of the input. Usually we set $\mathbf{c} = \mathbf{h}_T^e$, which is the final state of the encoder RNN (or we use a bidirectional RNN with average pooling). However, for tasks such as machine translation, this can result in poor performance, since the output does not have access to the input words themselves. We can avoid this bottleneck by allowing the output words to directly "look at" the input words. But which inputs should it look at? After all, word order is not always preserved across languages (e.g., German often puts verbs at the end of a sentence), so we need to infer the **alignment** between source and target.

We can solve this problem (in a differentiable way) by using (soft) **attention**, as first proposed in [BCB15; LPM15]. In particular, we can replace the fixed context vector $\mathbf{c}$ in the decoder with a dynamic context vector $\mathbf{c}_t$ computed as follows:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_i(\mathbf{h}_{t-1}^d, \{\mathbf{h}_i^e\})\mathbf{h}_i \tag{15.45}$$

This uses attention where the query is the hidden state of the decoder at the previous step, $\mathbf{h}_{t-1}^d$, the keys are all the hidden states from the encoder, and the values are also the hidden states from

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

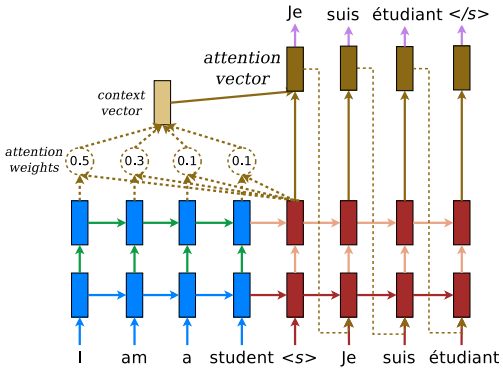
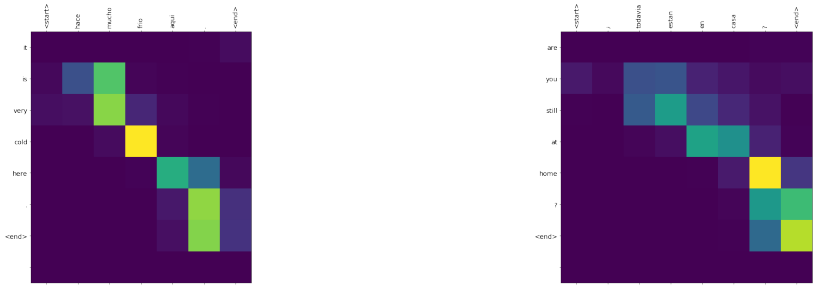


Figure 15.18: Illustration of seq2seq with attention for English to French translation. Used with kind permission of Minh-Thang Luong.



(a) *(b)*

Figure 15.19: Illustration of the attention heatmaps generated while translating two sentences from Spanish to English. (a) Input is "hace mucho frio aqui.", output is "it is very cold here.". (b) Input is "¿todavia estan en casa?", output is "are you still at home?". Note that when generating the output token "home", the model should attend to the input token "casa", but in fact it seems to attend to the input token "?". Generated by code at figures.probml.ai/book1/15.19.

the encoder. (When the RNN has multiple hidden layers, we usually take the top layer from the encoder, as the keys and values, and the top layer of the decoder as the query.) This context vector is concatenated with the input vector of the decoder, $\mathbf{y}_{t-1}$, and fed into $\mathbf{h}_t^d$. See Fig. 15.18 for an illustration of the overall model.

We can train this model in the usual way on sentence pairs, and then use it to perform machibe translation. (See code.probml.ai/book1/nmt_attention_torch for some sample code.) We can also visualize the attention weights computed at each step of decoding, to get an idea of which parts of the input the model thinks are most relevant for generating the corresponding output. Some examples are shown in Fig. 15.19.

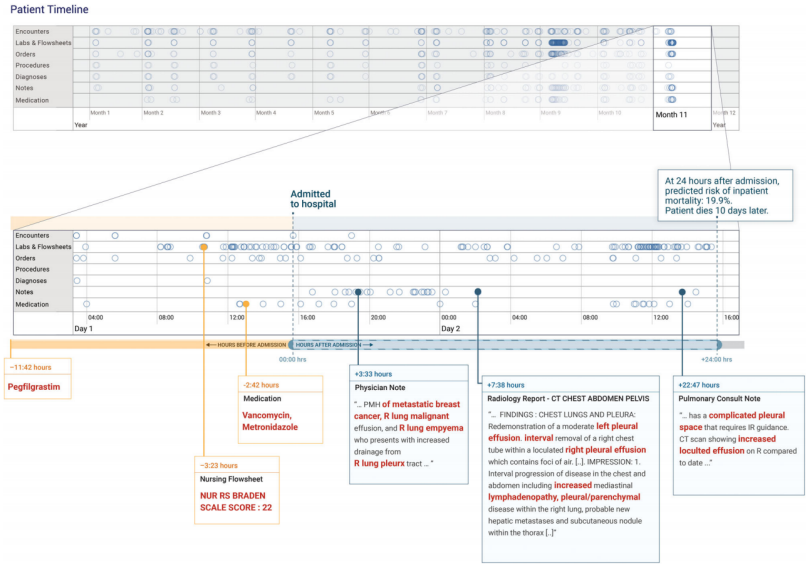


Figure 15.20: Example of an electronic health record. In this example, 24h after admission to the hospital, the RNN classifier predicts the risk of death as 19.9%; the patient ultimately died 10 days after admission. The "relevant" keywords from the input clinical notes are shown in red, as identified by an attention mechanism. From Figure 3 of [Raj+18]. Used with kind permission of Alvin Rakomar.

15.4.5 Seq2vec with attention (text classification)

We can also use attention with sequence classifiers. For example [Raj+18] apply an RNN classifier to the problem of predicting if a patient will die or not. The input is a set of **electronic health records**, which is a time series containing structured data, as well as unstructured text (clinical notes). Attention is useful for identifying "relevant" parts of the input, as illustrated in Fig. 15.20.

15.4.6 Seq+Seq2Vec with attention (text pair classification)

Suppose we see the sentence "A person on a horse jumps over a log" (call this the **premise**) and then we later read "A person is outdoors on a horse" (call this the **hypothesis**). We may reasonably say that the premise **entails** the hypothesis, meaning that the hypothesis is more likely given the premise.[2] Now suppose the hypothesis is "A person is at a diner ordering an omelette". In this case, we would say that the premise **contradicts** the hypothesis, since the hypothesis is less likely given the premise. Finally, suppose the hypothesis is "A person is training his horse for a competition". In this case, we see that the relationship between premise and hypothesis is **neutral**, since the hypothesis may or may not follow from the premise. The task of classifying a sentence pair into these three categories is known as **textual entailment** or "**natural language inference**". A standard

2. Note that the premise does not logically imply the hypothesis, since the person could be horse-back riding indoors, but generally people ride horses outdoors. Also, we are assuming the phrase "a person" refers to the same person in the two sentences.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

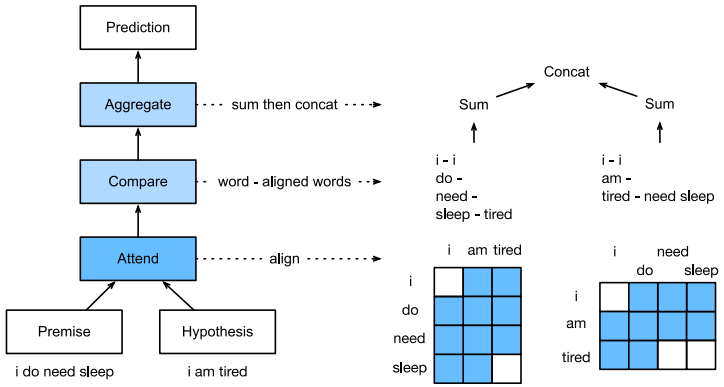


Figure 15.21: Illustration of sentence pair entailment classification using an MLP with attention to align the premise ("I do need sleep") with the hypothesis ("I am tired"). White squares denote active attention weights, blue squares are inactive. (We are assuming hard 0/1 attention for simplicity.) From Figure 15.5.2 of [Zha+20]. Used with kind permission of Aston Zhang.

benchmark in this area is the **Stanford Natural Language Inference** or **SNLI** corpus [Bow+15]. This consists of 550,000 labeled sentence pairs.

An interesting solution to this classification problem was presented in [Par+16]; at the time, it was the state of the art on the SNLI dataset. The overall approach is sketched in Fig. 15.21. Let $\mathbf{A} = (\mathbf{a}_1, \ldots, \mathbf{a}_m)$ be the premise and $\mathbf{B} = (\mathbf{b}_1, \ldots, \mathbf{b}_n)$ be the hypothesis, where $\mathbf{a}_i, \mathbf{b}_j \in \mathbb{R}^E$ are embedding vectors for the words. The model has 3 steps. First, each word in the premise, $\mathbf{a}_i$, attends to each word in the hypothesis, $\mathbf{b}_j$, to compute an attention weight

$$e_{ij} = f(\mathbf{a}_i)^\top f(\mathbf{b}_j) \tag{15.46}$$

where $f : \mathbb{R}^E \to \mathbb{R}^D$ is an MLP; we then compute a weighted average of the matching words in the hypothesis,

$$\boldsymbol{\beta}_i = \sum_{j=1}^{n} \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp(e_{ik})} \mathbf{b}_j \tag{15.47}$$

Next, we compare $\mathbf{a}_i$ with $\boldsymbol{\beta}_i$ by mapping their concatenation to a hidden space using an MLP $g : \mathbb{R}^{2E} \to \mathbb{R}^H$:

$$\mathbf{v}_{A,i} = g([\mathbf{a}_i, \boldsymbol{\beta}_i]), \; i = 1, \ldots, m \tag{15.48}$$

Finally, we aggregate over the comparisons to get an overall similarity of premise to hypothesis:

$$\mathbf{v}_A = \sum_{i=1}^{m} \mathbf{v}_{A,i} \tag{15.49}$$

(a) (b)

Figure 15.22: Image captioning using attention. (a) Soft attention. Generates "a woman is throwing a frisbee in a park". (b) Hard attention. Generates "a man and a woman playing frisbee in a field". From Figure 6 of [Xu+15]. Used with kind permission of Kelvin Xu.

We can similarly compare the hypothesis to the premise using

$$\boldsymbol{\alpha}_j = \sum_{i=1}^{m} \frac{\exp(e_{ij})}{\sum_{k=1}^{m} \exp(e_{kj})} \mathbf{a}_i \tag{15.50}$$

$$\mathbf{v}_{B,j} = g([\mathbf{b}_j, \boldsymbol{\alpha}_j]), \; j = 1, \ldots, n \tag{15.51}$$

$$\mathbf{v}_B = \sum_{j=1}^{n} \mathbf{v}_{B,j} \tag{15.52}$$

At the end, we classify the output using another MLP $h : \mathbb{R}^{2H} \to \mathbb{R}^3$:

$$\hat{y} = h([\mathbf{v}_A, \mathbf{v}_B]) \tag{15.53}$$

See code.probml.ai/book1/entailment_attention_mlp_torch for some sample code.

We can modify this model to learn other kinds of mappings from sentence pairs to output labels. For example, in the **semantic textual similarity** task, the goal is to predict how semantically related two input sentences are. A standard dataset for this is the **STS Benchmark** [Cer+17], where relatedness ranges from 0 (meaning unrelated) to 5 (meaning maximally related).

15.4.7 Soft vs hard attention

If we force the attention heatmap to be sparse, so that each output can only attend to one input location instead of a weighted combination of all of them, the method is called **hard attention**. We compare these two approaches for an image captioning problem in Fig. 15.22b. Unfortunately, hard attention results in a nondifferentiable training objective, and requires methods such as reinforcement learning to fit the model. See [Xu+15] for the details.

It seems from the above examples that these attention heatmaps can "explain" why the model generates a given output. However, the interpretability of attention is controversial (see e.g., [JW19; WP19; SS19; Bru+19] for discussion).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

15.5 Transformers

The **transformer** model [Vas+17] is a seq2seq model which uses attention in the encoder as well as the decoder, thus eliminating the need for RNNs, as we explain below. Transformers have been used for many (conditional) sequence generation tasks, such as machine translation [Vas+17], constituency parsing [Vas+17], music generation [Hua+18], protein sequence generation [Mad+20; Cho+20b], abstractive text summarization [Zha+19a], image generation [Par+18] (treating the image as a rasterized 1d sequence), etc.

The transformer is a rather complex model that uses several new kinds of building blocks or layers. We introduce these new blocks below, and then discuss how to put them altogether.

15.5.1 Self-attention

In Sec. 15.4.4 we showed how the decoder of an RNN could use attention to the input sequence in order to capture contexual embeddings of each input. However, rather than the decoder attending to the encoder, we can modify the model so the encoder attends to itself. This is called **self attention**.

In more detail, given a sequence of input tokens $\mathbf{x}_1, \ldots, \mathbf{x}_n$, where $\mathbf{x}_i \in \mathbb{R}^d$, self-attention can generate a sequence of outputs of the same size using

$$\mathbf{y}_i = \text{Attn}(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \ldots, (\mathbf{x}_n, \mathbf{x}_n)) \tag{15.54}$$

where the query is $\mathbf{x}_i$, and the keys and values are all the (valid) inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n$.

To use this in a decoder, we can set $\mathbf{x}_i = \mathbf{y}_{i-1}$, and $n = i - 1$, so all the previously generated outputs are available. At training time, all the output are already known, so we can evaluate the above function in parallel, overcoming the sequential bottleneck of using RNNs.

In addition to improved speed, self-attention can give improved representations of context. As an example, consider translating the English sentences "The animal didn't cross the street because it was too *tired*" and "The animal didn't cross the street because it was too *wide*" into French. To generate a pronoun of the correct gender in French, we need to know what "it" refers to (this is called **coreference resolution**). In the first case, the word "it" refers to the animal. In the second case, the word "it" now refers to the street.

Fig. 15.23 illustrates how self attention applied to the English sentence is able to resolve this ambiguity. In the first sentence, the representation for "it" depends on the earlier representations of "animal", whereas in the latter, it depends on the earlier representations of "street".

15.5.2 Multi-headed attention

If we think of an attention matrix as like a kernel matrix (as discussed in Sec. 15.4.2), it is natural to want to use multiple attention matrices, to capture different notions of similarity. This is the basic idea behind **multi-headed attention** (MHA). In more detail, query a given $\mathbf{q} \in \mathbb{R}^{d_q}$, keys $\mathbf{k}_j \in \mathbb{R}^{d_k}$, and values $\mathbf{v}_j \in \mathbb{R}^{d_v}$, we define the i'th attention head to be

$$\mathbf{h}_i = \text{Attn}(\mathbf{W}_i^{(q)}\mathbf{q}, \{\mathbf{W}_i^{(k)}\mathbf{k}_j, \mathbf{W}_i^{(v)}\mathbf{v}_j\}) \in \mathbb{R}^{p_v} \tag{15.55}$$

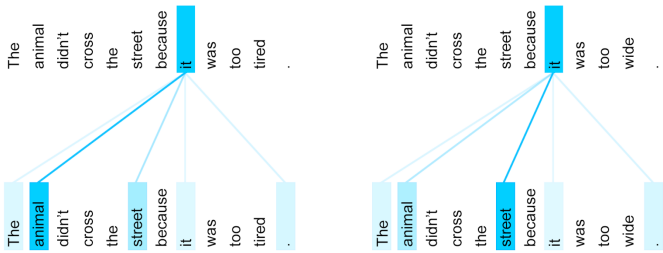


Figure 15.23: Illustration of how encoder self-attention for the word "it" differs depending on the input context. From https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html *. Used with kind permission of Jakob Uszkoreit.*

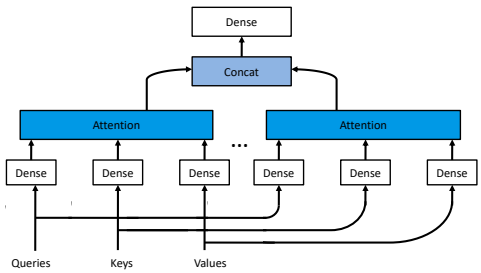


Figure 15.24: Multi-head attention. Adapted from Figure 9.3.3 of [Zha+20].

where $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$, and $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$ are projection matrices. We then stack the h heads together, and project to $\mathbb{R}^{p_o}$ using

$$\mathbf{h} = \text{MHA}(\mathbf{q}, \{\mathbf{k}_j, \mathbf{v}_j\}) = \mathbf{W}_o \begin{pmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{pmatrix} \in \mathbb{R}^{p_o} \tag{15.56}$$

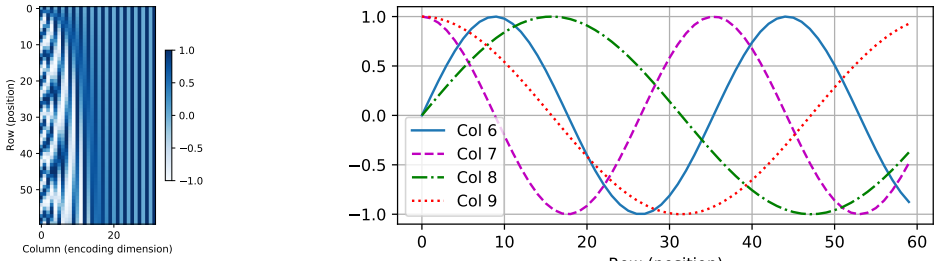
where $\mathbf{h}_i$ is defined in Eq. (15.55), and $\mathbf{W}_o \in \mathbb{R}^{p_o \times h p_v}$. If we set $p_q h = p_k h = p_v h = p_o$, we can compute all the output heads in parallel, See code.probml.ai/book1/multi_head_attention for some sample code.

15.5.3 Positional encoding

The performance of "vanilla" self-attention can be low, since attention is permutation invariant, and hence ignores the input word ordering. To overcome this, we can concatenate the word embeddings with a **positional embedding**, so that the model knows what order the words occur in.

One way to do this is to represent each position by an integer. However, neural networks cannot natively handle integers. To overcome this, we can encode the integer in binary form. For example, if we assume the sequence length is $n = 3$, we get the following sequence of $d = 3$-dimensional bit

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 15.25: (a) Positional encoding matrix for a sequence of length $n = 60$ and an embedding dimension of size $d = 32$. (b) Basis functions for columsn 6 to 9. Generated by code at figures.probml.ai/book1/15.25.

vectors for each location: 000, 001, 010, 011, 100, 101, 110, 111. We see that the right most index toggles the fastest (has highest frequency), whereas the left most index (most significant bit) toggles the slowest. (We could of course change this, so that the left most bit toggles fastest.) We can represent this as a position matrix $\mathbf{P} \in \mathbb{R}^{n \times d}$.

We can think of the above representation as using a set of basis functions (corresponding to powers of 2), where the coefficients are 0 or 1. We can obtain a more compact code by using a different set of basis functions, and real-valued weights. [Vas+17] propose to use a sinusoidal basis, as follows:

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/d}}\right), \; p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/d}}\right), \tag{15.57}$$

where $C = 10,000$ corresponds to some maximum sequence length. For example, if $d = 4$, the i't row is

$$\mathbf{p}_i = [\sin(\frac{i}{C^{0/4}}), \cos(\frac{i}{C^{0/4}}), \sin(\frac{i}{C^{2/4}}), \cos(\frac{i}{C^{2/4}})] \tag{15.58}$$

Fig. 15.25a shows the corresponding position matrix for $n = 60$ and $d = 32$. In this case, the left-most columns toggle fastest. We see that each row has a real-valued "fingerprint" representing its location in the sequence. Fig. 15.25b shows some of the basis functions (column vectors) for dimensions 6 to 9.

The advantage of this representation is two-fold. First, it can be computed for arbitrary length inputs (up to $T \leq C$), unlike a learned mapping from integers to vectors. Second, nearby locations have similar encodings, which provides a useful form of inductive bias. In particular, we have $\mathbf{p}_{t+\phi} = f(\mathbf{p}_t)$, where f is a linear transformation. To see this, note that

$$\begin{pmatrix} \sin(\omega_k(t+\phi)) \\ \cos(\omega_k(t+\phi)) \end{pmatrix} = \begin{pmatrix} \sin(\omega_k t)\cos(\omega_k \phi) + \cos(\omega_k t)\sin(\omega_k \phi) \\ \cos(\omega_k t)\cos(\omega_k \phi) - \sin(\omega t)\sin(\omega_k \phi) \end{pmatrix} \tag{15.59}$$

$$= \begin{pmatrix} \cos(\omega_k \phi) & \sin(\omega_k \phi) \\ -\sin(\omega_k \phi) & \cos(\omega_k \phi) \end{pmatrix} \begin{pmatrix} \sin(\omega_k t) \\ \cos(\omega_k t) \end{pmatrix} \tag{15.60}$$

Once we have computed the positional emebddings $\mathbf{P}$, we need to combine them with the original word embeddings $\mathbf{X}$ using $\mathbf{X} + \mathbf{P}$.

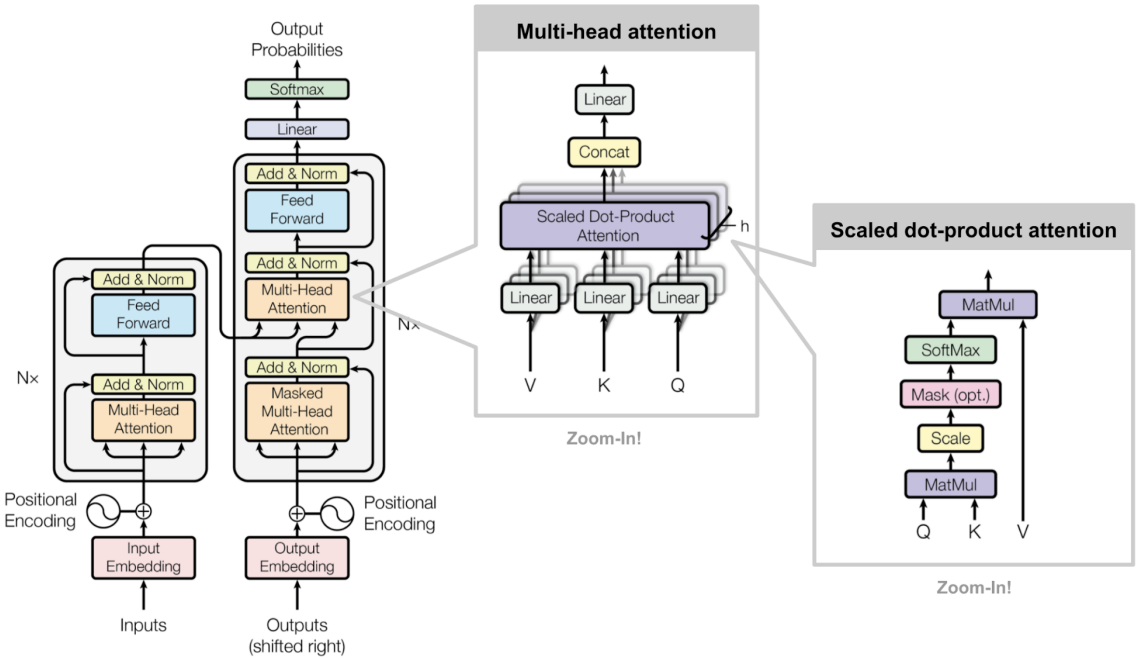


Figure 15.26: The transformer. From [Wen18]. Used with kind permission of Lilian Weng. Adapted from Figures 1–2 of [Vas+17].

15.5.4 Putting it altogether

A transformer is a seq2seq model that uses self-attention for the encoder and decoder rather than an RNN. The encoder uses a series of encoder blocks, each of which uses multi-headed attention (Sec. 15.5.2), residual connections (Sec. 13.4.4), and layer normalization (Sec. 14.2.4.2). More precisely, the encoder block $\mathbf{E} = \text{EB}(\mathbf{X})$ can be defined as follows:

$$\mathbf{Z} = \text{LN}(\text{MHA}(\mathbf{X}, \mathbf{X}, \mathbf{X}) + \mathbf{X}) \tag{15.61}$$

$$\mathbf{E} = \text{LN}(\text{FF}(\mathbf{Z}) + \mathbf{Z}) \tag{15.62}$$

where FF is a position-wise feed-forward network applied to every location in the sequence in parallel, $\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ is multi-head attention, and $\text{LN}(\mathbf{Z})$ is layer-normalization. The overall encoder $\mathbf{E} = \text{Encoder}(\mathbf{X})$ is defined by applying positional encoding to the embedding of the input sequence, following by N copies of the encoder block:

$$\mathbf{Z} = \text{POS}(\text{Embed}(\mathbf{X})) \tag{15.63}$$

$$\mathbf{E} = \text{EB}^N(\mathbf{Z}) \tag{15.64}$$

See the LHS of Fig. 15.26 for an illustration.

The decoder has a somewhat more complex structure. It is given access to the encoder via another multi-head attention block. But it is also given access to previously generated outputs: these

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Layer type	Complexity	Sequential ops.	Max. path length
Self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$

Table 15.1: Comparison of the transformer with other neural sequential generative models. n is the sequence length, d is the dimensionality of the input features, and k is the kernel size for convolution. Based on Table 1 of [Vas+17].

are shifted, and then combined with a positional embedding, and then fed into a masked (causal) multi-head attention model. Finally the output distribution over tokens at each location is computed in parallel.

In more detail, the decoder block $\mathbf{D} = \text{DB}(\mathbf{X}, \mathbf{E})$ is defined as follows:

$$\mathbf{Z} = \text{LN}(\text{MHA}(\mathbf{X}, \mathbf{X}, \mathbf{X}) + \mathbf{X}) \tag{15.65}$$

$$\mathbf{Z}' = \text{LN}(\text{MHA}(\mathbf{Z}, \mathbf{E}, \mathbf{E}) + \mathbf{Z}) \tag{15.66}$$

$$\mathbf{D} = \text{LN}(\text{FF}(\mathbf{Z}') + \mathbf{Z}') \tag{15.67}$$

The overall decoder $\mathbf{Y} = \text{Decoder}(\mathbf{X}, \mathbf{E})$ is defined by N copies of the decoder block:

$$\mathbf{Z} = \text{POS}(\text{Embed}(\mathbf{X})) \tag{15.68}$$

$$\mathbf{Y} = \text{DB}^N(\mathbf{Z}, \mathbf{E}) \tag{15.69}$$

See the RHS of Fig. 15.26 for an illustration.

During training time, all the inputs $\mathbf{X}$ to the decoder are known in advance, since they are derived from embedding the lagged target output sequence. During inference (test) time, we need to decode sequentially, and use masked attention, where we feed the generated output into the embedding layer, and add it to the set of keys/values that can be attended to. (We initialize by feeding in the start-of-sequence token.) See code.probml.ai/book1/transformers_torch for some sample code, and [Rus18] for a detailed tutorial on this model.

15.5.5 Comparing transformers, CNNs and RNNs

In Fig. 15.27, we visually compare three different architectures for mapping a sequence $\mathbf{x}_{1:n}$ to another sequence $\mathbf{y}_{1:n}$: a 1d CNN, an RNN, and an attention-based model. Each model makes different tradeoffs in terms of speed and expressive power, where the latter can be quantified in terms of the maximum path length between any two inputs. See Table 15.1 for a summary.

For a 1d CNN with kernel size k and d feature channels, the time to compute the output is $O(knd^2)$, which can be done in parallel. We need a stack of n/k layers, or $\log_k(n)$ if we use dilated convolution, to ensure all pairs can communicate. For example, in Fig. 15.27, we see that x_1 and x_5 are initially 5 apart, and then 3 apart in layer 1, and then connected in layer 2.

For an RNN, the computational complexity is $O(nd^2)$, for a hidden state of size d, since we have to perform matrix-vector multiplication at each step. This is an inherently sequential operation. The maximum path length is $O(n)$.

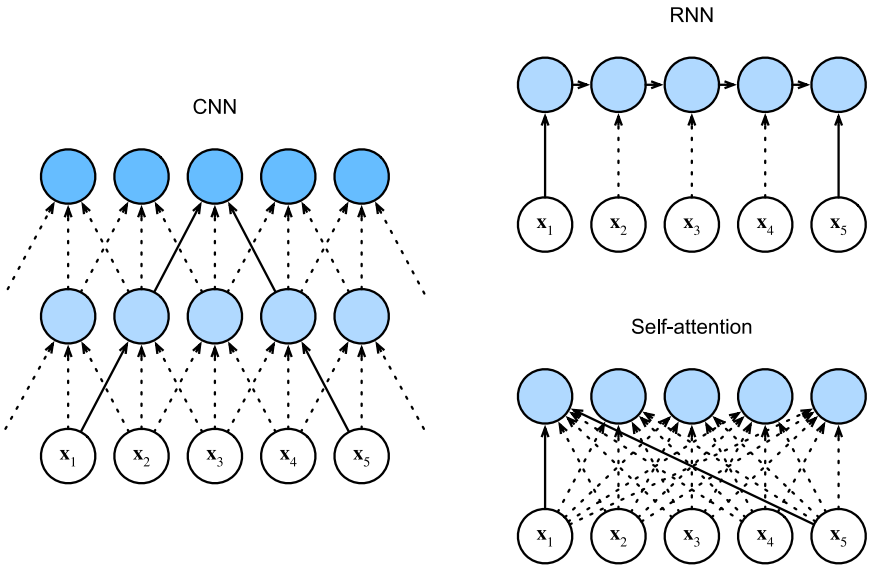


Figure 15.27: Comparison of (1d) CNNs, RNNs and self-attention models. From Figure 10.6.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Finally, for self-attention models, every output is directly connected to every input, so the maximum path length is $O(1)$. However, the computational cost is $O(n^2 d)$. For short sequences, we typically have $n \ll d$, so this is fine. For longer sequences, we discuss various fast versions of attention in Sec. 15.6.

15.5.6 Transformers for images *

In addition to computational issues, we note that transformers, RNNs and CNNs have different inductive bias. For example, CNNs encode a notion of locality and equivariance (due to weight tying), as well as invariance (due to pooling), which can be useful for tasks such as image classification. Surprisingly, however, it has been found that transformers can also do well at image classification, at least if trained on enough data.

In particular, [Dos+21] present the **ViT** model (vision transformer), that chops the input up into 16x16 patches, projects each patch into an embedding space, and then passes this set of embeddings $\mathbf{x}_{1:T}$ to a transformer, analogous to the way word embeddings are passed to a transformer. The input is also prepended with a special [CLASS] embedding, $\mathbf{x}_0$. The output of the transformer is a set of encodings $\mathbf{e}_{0:T}$; the model maps $\mathbf{e}_0$ to the target class label y, and is trained in a supervised way. See Fig. 15.28 for an illustration.

After supervised pretraining, the model is fine-tuned on various downstream classification tasks, an approach known as transfer learning (see Sec. 19.2 for more details). When trained on "small" datasets such as ImageNet (which has 1k classes and 1.3M images), they find that they cannot outperform a pretrained CNN ResNet model (Sec. 14.3.4) known as **BiT** (big transfer) [Kol+20].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

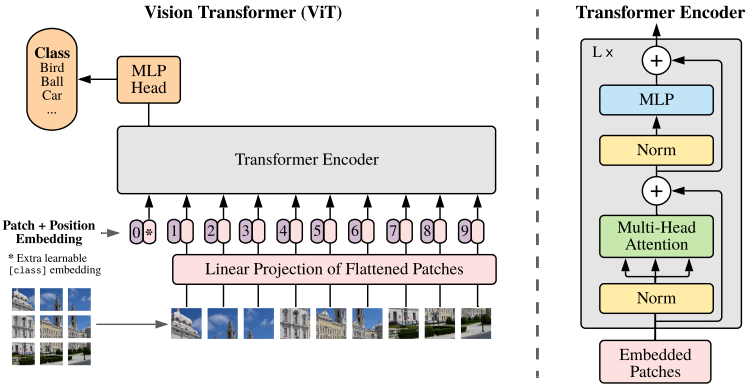


*Figure 15.28: The Vision Transformer (ViT) model. This treats an image as a set of input patches. The input is prepended with the special CLASS embedding vector (denoted by *) in location 0. The class label for the image is derived by applying softmax to the final ouput encoding at location 0. From Figure 1 of [Dos+21]. Used with kind permission of Alexey Dosovitskiy*

However, when trained on larger datasets, such as ImageNet-21k (with 21k classes and 14M images), or the Google-internal JFT dataset (with 18k classes and 303M images), they find that ViT does better than BiT at transfer learning. It is also cheaper to train than ResNet at this scale. (Hwoever, training is still expensive: the large ViT model on ImageNet-21k takes 30 days on a cloud TPUv3 with 8 cores!)

15.6 Efficient transformers *

This section is written by Krzysztof Choromanski.

Regular transformers take $O(N^2)$ time and space complexity, for a sequence of length N, which makes them impractical to apply to long sequences. In the past few years, researchers have proposed several more efficient variants of transformers to bypass this difficulty. In this section, we give a brief survey of some of these methods (see Fig. 15.29 for a summary). For more details, see e.g., [Tay+20a; Tay+20b]).

15.6.1 Fixed non-learnable localized attention patterns

The simplest modification of the attention mechanism is to constrain it to a fixed non-learnable localized window, in other words restrict each token to attend only to a pre-selected set of other tokens. If for instance, each sequence is chunked into K blocks, each of length $\frac{N}{K}$, and attention is conducted only within a block, then space/time complexity is reduced from $O(N^2)$ to $\frac{N^2}{K}$. For $K \gg 1$ this constitutes substantial overall computational improvements. Such an approach is applied in particular in [Qiu+19a; Par+18]. The attention patterns do not need to be in the form of blocks. Other approaches involve strided/"dilated windows" or hybrid patterns, where several fixed attention patterns are combined together [Chi+19b; BPC20].

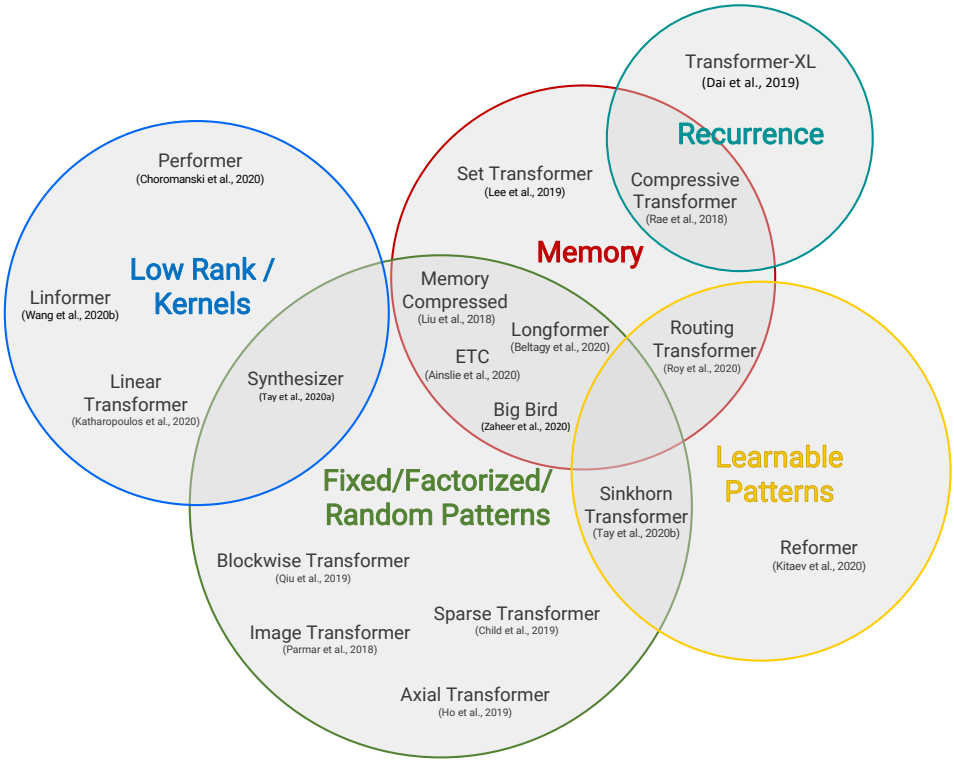


Figure 15.29: Venn diagram presenting the taxonomy of different efficient transformer architectures. From [Tay+20a]. Used with kind permission of Yi Tay.

15.6.2 Learnable sparse attention patterns

A natural extension of the above approach is to allow the above compact patterns to be learned. The attention is still restricted to pairs of tokens within a single partition of some partitioning of the set of all the tokens, but now those partitionings are trained. In this class of methods we can distinguish two main approaches: based on hashing and clustering. In the hashing scenario all tokens are hashed and thus different partitions correspond to different hashing-buckets. This is the case for instance for the **Reformer** architecture [KKL20], where locality sensitive hashing (LSH) is applied. That leads to time complexity $O(NM^2 \log(M))$ of the attention module, where M stands for the dimenionality of tokens' embeddings.

Hashing approaches require the set of queries to be identical to the set of keys. Furthermore, the number of hashes needed for precise partitioning (which in the above expression is treated as a constant) can be a large constant. In the clustering approach, tokens are clustered using standard clustering algorithms such as K-means (Sec. 21.3); this is known as the "clustering transformer" [Roy+20]. As in the block-case, if K equal-size clusters are used then space complexity of the attention module is reduced to $O(\frac{N^2}{K})$. In practice K is often taken to be of order $K = \Theta(\sqrt{N})$, yet

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

imposing that the clusters be similar in size is in practice difficult.

15.6.3 Memory and recurrence methods

In some approaches, a side memory module can access several tokens simultaneously. This method is often instantiated in the form of a *global memory* algorithm as used in [Lee+19; Zah+20].

Another approach is to connect differet local blocks via recurrence. A flagship example of this approach is the class of Transformer-XL methods [Dai+19].

15.6.4 Low-rank and kernel methods

In this section, we discuss methods that approximate attention using low rank matrices. In [She+18; Kat+20] they approximate the attention matrix $\mathbf{A}$ directly by a low rank matrix, so that

$$A_{ij} = \boldsymbol{\phi}(\mathbf{q}_i)^\top \boldsymbol{\phi}(\mathbf{k}_j) \tag{15.70}$$

where $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^M$ is some finite-dimensional vector with $M < D$. One can leverage this structure to compute $\mathbf{AV}$ in $O(N)$ time. Unfortunately, for softmax attention, the $\mathbf{A}$ is not low rank.

In **Linformer** [Wan+20a], they instead transform the keys and values via random Gaussian projections. They then apply the theory of the Johnson-Lindenstrauss Transform [AL13] to approximate softmax attention in this lower dimensional space.

In **Performer** [Cho+20a; Cho+20b], they show that the attention matrix can be computed using a (positive definite) kernel function. We define kernel functions in Sec. 17.2, but the basic idea is that $\mathcal{K}(\mathbf{q}, \mathbf{k}) \geq 0$ is some measure of similarity between $\mathbf{q} \in \mathbb{R}^D$ and $\mathbf{k} \in \mathbb{R}^D$. For example, the Gaussian kernel, also called the radial basis function kernel, has the form

$$\mathcal{K}_{\text{gauss}}(\mathbf{q}, \mathbf{k}) = \exp\left(-\frac{1}{2\sigma^2}||\mathbf{q} - \mathbf{k}||_2^2\right) \tag{15.71}$$

To see how this can be used to compute an attention matrix, note that [Cho+20a] show the following:

$$A_{i,j} = \exp(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{D}}) = \exp(\frac{-\|\mathbf{q}_i - \mathbf{k}_j\|_2^2}{2\sqrt{D}}) \times \exp(\frac{\|\mathbf{q}_i\|_2^2}{2\sqrt{D}}) \times \exp(\frac{\|\mathbf{k}_j\|_2^2}{2\sqrt{D}}). \tag{15.72}$$

The first term in the above expression is equal to $\mathcal{K}_{\text{gauss}}(\mathbf{q}_i D^{-1/4}, \mathbf{k}_i D^{-1/4})$ with $\sigma = 1$, and the other two terms are just independent scaling factors.

So far we have not gained anything computationally. However, we will show in Sec. 17.4.3 that the Gaussian kernel can be written as the expectation of a set of random features:

$$\mathcal{K}_{\text{gauss}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}\left[\boldsymbol{\eta}(\mathbf{x})^\top \boldsymbol{\eta}(\mathbf{y})\right] \tag{15.73}$$

where $\boldsymbol{\eta}(\mathbf{x}) \in \mathbb{R}^M$ is a random feature vector derived from $\mathbf{x}$, either based on trigonometric functions Eq. (17.69) or exponential functions Eq. (17.70). (The latter has the advantage that all the features are positive, which gives much better results [Cho+20b].) Therefore for the regular softmax attention, $A_{i,j}$ can be rewritten as

$$A_{i,j} = \mathbb{E}[\boldsymbol{\phi}(\mathbf{q}_i)^\top \boldsymbol{\phi}(\mathbf{k}_j)] \tag{15.74}$$

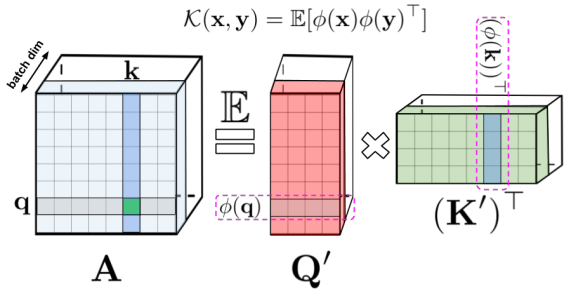


Figure 15.30: Attention matrix $\mathbf{A}$ rewritten as a product of two lower rank matrices $\mathbf{Q}'$ and $(\mathbf{K}')^\top$ with random feature maps $\boldsymbol{\phi}(\mathbf{q}_i) \in \mathbb{R}^M$ and $\boldsymbol{\phi}(\mathbf{v}_k) \in \mathbb{R}^M$ for the corresponding queries/keys stored in the rows/columns. Used with kind permission of Krzysztof Choromanski.

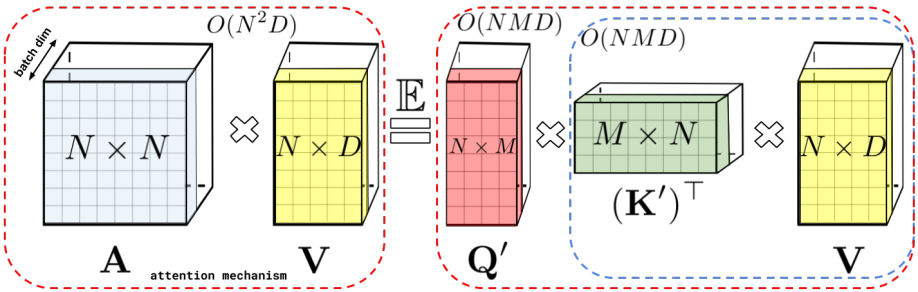


Figure 15.31: Decomposition of the attention matrix $\mathbf{A}$ can be leveraged to improve attention computations via matrix associativity property. To compute $\mathbf{AV}$, we first calculate $\mathbf{G} = (\mathbf{k}')^\top \mathbf{V}$ and then $\mathbf{q}'\mathbf{G}$, resulting in linear in N space and time complexity. Used with kind permission of Krzysztof Choromanski.

where $\boldsymbol{\phi}$ is defined as:

$$\boldsymbol{\phi}(\mathbf{x}) \triangleq \exp\left(\frac{\|\mathbf{x}\|_2^2}{2\sqrt{D}}\right) \boldsymbol{\eta}\left(\frac{\mathbf{x}}{D^{\frac{1}{4}}}\right). \tag{15.75}$$

We can write the full attention matrix as follows

$$\mathbf{A} = \mathbb{E}[\mathbf{Q}'(\mathbf{K}')^\top] \tag{15.76}$$

where $\mathbf{Q}', \mathbf{K}' \in \mathbb{R}^{N \times M}$ have rows encoding random feature maps corresponding to the queries and keys. (Note that we can get better performance if we ensure these random features are orthogonal, see [Cho+20a] for the details.) See Fig. 15.30 for an illustration.

We can create an approximation to $\mathbf{A}$ by using a single sample of the random features $\boldsymbol{\phi}(\mathbf{q}_i)$ and $\boldsymbol{\phi}(\mathbf{k}_j)$, and using a small value of M, say $M = O(D \log(D))$. We can then approximate the entire attention operator in $O(N)$ time using

$$\widehat{\text{attention}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{diag}^{-1}(\mathbf{Q}'((\mathbf{K}')^\top \mathbf{1}_N))(\mathbf{Q}'((\mathbf{K}')^\top \mathbf{V})) \tag{15.77}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

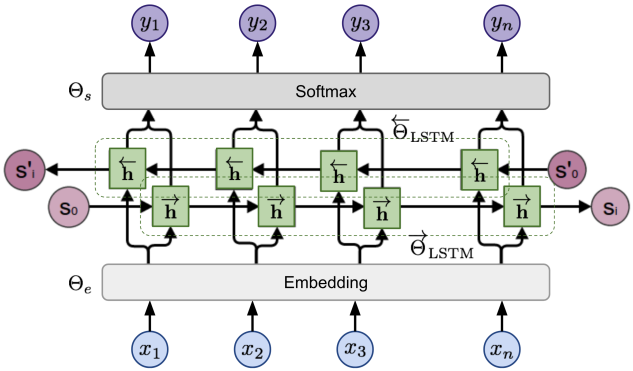


Figure 15.32: Illustration of ELMo bidirectional language model. Here $y_t = x_{t+1}$ when acting as the target for the forwards LSTM, and $y_t = x_{t-1}$ for the backwards LSTM. (We add bos *and* eos *sentinels to handle the edge cases.) From [Wen19]. Used with kind permission of Lilian Weng.*

This can be shown to be an unbiased approximation to the exact softmax attention operator. See Fig. 15.31 for an illustration. (For details on how to generalize this to masked (causal) attention, see [Cho+20a].)

15.7 Language models and unsupervised representation learning

We have discussed how RNNs and autoregressive (decoder-only) transformers can be used as **language models**, which are generative sequence models of the form $p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t|\mathbf{x}_{1:t-1})$, where each x_t is a discrete token, such as a word or wordpiece. (See Sec. 1.5.4 for a discussion of text preprocessing methods.) The latent state of these models can then be used as a continuous vector representation of the text. That is, instead of using the one-hot vector $\mathbf{x}_t$, or a learned embedding of it (such as those discussed in Sec. 20.5), we use the hidden state $\mathbf{h}_t$, which depends on all the previous words in the sentence. These vectors can then be used as **contextual word embeddings**, for purposes such as text classification or seq2seq tasks (see e.g. [LKB20] for a review). The advantage of this approach is that we can **pre-train** the language model in an unsupervised way, on a large corpus of text, and then we can **fine-tune** the model in a supervised way on a small labeled task-specific dataset. (This general approach is called **transfer learning**, see Sec. 19.2 for details.)

If our primary goal is to compute useful representations for transfer learning, as opposed to generating text, we can replace the generative sequence model with non-causal models that can compute a representation of a sentence, but cannot generate it. These models have the advantage that now the hidden state $\mathbf{h}_t$ can depend on the past, $\mathbf{y}_{1:t-1}$, present $\mathbf{y}_t$, and future, $\mathbf{y}_{t+1:T}$. This can sometimes result in better representations, since it takes into account more context.

In the sections below, we briefly discuss some unsupervised models for representation learning on text, using both causal and non-causal models.

15.7.1 ELMo

In [Pet+18], they present a method called **ELMo**, which is short for "Embeddings from Language Model". The basic idea is to fit two RNN language models, one left-to-right, and one right-to-left, and then to combine their hidden state representations to come up with an embedding for each word. Unlike a biRNN (Sec. 15.2.2), which needs an input-output pair, ELMo is trained in an unsupervised way, to minimize the negative log likelihood of the input sentence $\mathbf{x}_{1:T}$:

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{t=1}^{T} [\log p(x_t|\mathbf{x}_{1:t-1}; \boldsymbol{\theta}_e, \boldsymbol{\theta}^{\rightarrow}, \boldsymbol{\theta}_s) + \log p(x_t|\mathbf{x}_{t+1:T}; \boldsymbol{\theta}_e, \boldsymbol{\theta}^{\leftarrow}, \boldsymbol{\theta}_s)] \tag{15.78}$$

where $\boldsymbol{\theta}_e$ are the shared parameters of the embedding layer, $\boldsymbol{\theta}_s$ are the shared parameters of the softmax output layer, and $\boldsymbol{\theta}^{\rightarrow}$ and $\boldsymbol{\theta}^{\leftarrow}$ are the parameters of the two RNN models. (They use LSTM RNNs, described in Sec. 15.2.7.2.) See Fig. 15.32 for an illustration.

After training, we define the contextual representation $\mathbf{r}_t = [\mathbf{e}_t, \mathbf{h}^{\rightarrow}_{t,1:L}, \mathbf{h}^{\leftarrow}_{t,1:L}]$, where L is the number of layers in the LSTM. We then learn a task-specific set of linear weights to map this to the final context-specific embedding of each token: $\mathbf{r}^j_t = \mathbf{r}_t^{\top} \mathbf{w}^j$, where j is the task id. If we are performing a syntactic task like **part-of-speech** (**POS**) tagging (i.e., labeling each word as a noun, verb, adjective, etc), then the task will learn to put more weight on lower layers. If we are performing a semantic task like **word sense disambiguation** (**WSD**), then the task will learn to put more weight on higher layers. In both cases, we only need a small amount of task-specific labeled data, since we are just learning a single weight vector, to map from $\mathbf{r}_{1:T}$ to the target labels $\mathbf{y}_{1:T}$.

15.7.2 BERT

In this section, we describe the **BERT** model (Bidirectional Encoder Representations from Transformers) of [Dev+19]. Like ELMo, this is a non-causal model, that can be used to create representations of text, but not to generate text. In particular, it uses a transformer model to map a modified version of a sequence back to the unmodified form. The modified input at location t omits all words except for the t'th, and the task is to predict the missing word. This is called the **fill-in-the-blank** or **cloze** task.

15.7.2.1 Masked language model task

More precisely, the model is trained to minimize the negative log **pseudo-likelihood**:

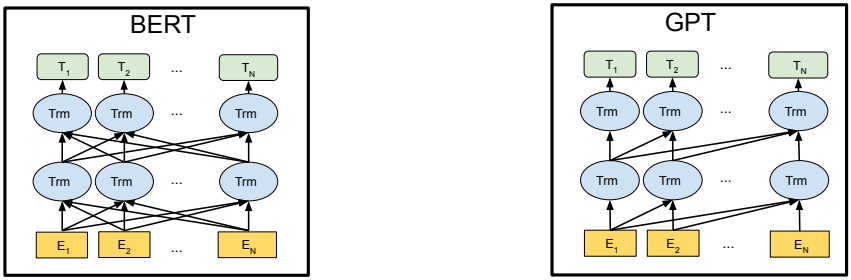
$$\mathcal{L} = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}} \mathbb{E}_{\mathbf{m}} \sum_{i \in \mathbf{m}} -\log p(x_i|\mathbf{x}_{-\mathbf{m}}) \tag{15.79}$$

where $\mathbf{m}$ is a random binary mask. For example, if we train the model on transcripts from cooking videos, we might create a training sentence of the form

```
Let's make [MASK] chicken! [SEP] It [MASK] great with orange sauce.
```

where [SEP] is a separator token inserted between two sentences. The desired target labels for the masked words are "some" and "tastes". (This example is from [Sun+19a].)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 15.33: Illustration of (a) BERT and (b) GPT. E_t is the embedding vector for the input token at location t, and T_t is the output target to be predicted. From Figure 3 of [Dev+19]. Used with kind permission of Ming-Wei Chang.

The conditional probability is given by applying a softmax to the final layer hidden vector at location i:

$$p(x_i|\hat{\mathbf{x}}) = \frac{\exp(\mathbf{h}(\hat{\mathbf{x}})_i^\top \mathbf{e}(x_i))}{\sum_{x'} \exp(\mathbf{h}(\hat{\mathbf{x}})_i^\top \mathbf{e}(x'))} \tag{15.80}$$

where $\hat{\mathbf{x}} = \mathbf{x}_{-\mathbf{m}}$ is the masked input sentence, and $\mathbf{e}(x)$ is the embedding for token x. This is used to compute the loss at the masked locations; this is therefore called a **masked language model**. (This is similar to a denoising autoencoder, Sec. 20.3.2). See Fig. 15.33a for an illustration of the model.

15.7.2.2 Next sentence prediction task

In addition to the masked language model objective, the original BERT paper added an additional objective, in which the model is trained to classify if one sentence follows another. More precisely, the model is fed as input

$$\text{CLS } A_1\ A_2; \ldots\ A_m;\ \text{SEP } B_1\ B_2; \ldots; B_n\ \text{SEP} \tag{15.81}$$

where SEP is a special separator token, and CLS is a special token marking the class. If sentence B follows A in the original text, we set the target label to $y = 1$, but if B is a randomly chosen sentence, we set the target label to $y = 0$. This is called the **next sentence prediction** task. This kind of pre-training can be useful for sentence-pair classification tasks, such as textual entailment or textual similarity, which we discussed in Sec. 15.4.6. (Note that this kind of pre-training is considered unsupervised, or self-supervised, since the target labels are automatically generated.)

When performing next sentence prediction, the input to the model is specified using 3 different embeddings: one per token, one for each segment label (sentence A or B), and one per location (using a learned positional embedding). These are then added. See Fig. 15.34 for an illustration. BERT then uses a transformer encoder to learn a mapping from this input embedding sequence to an output embedding sequence, which gets decoded into word labels (for the masked locations) or a class label (for the CLS location).

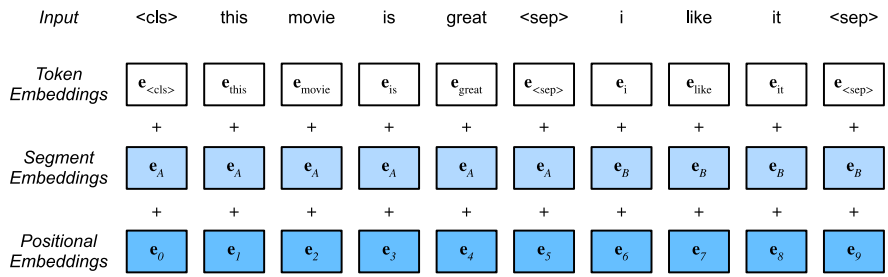


Figure 15.34: Illustration of how a pair of input sequences, denoted A and B, are encoded before feeding to BERT. From Figure 14.8.2 of [Zha+20]. Used with kind permission of Aston Zhang.

15.7.2.3 Fine-tuning BERT for NLP applications

After pre-training BERT in an unsupervised way, we can use it for various downtream tasks by performing supervised fine-tuning. (See Sec. 19.2 for more background on such transfer learning methods.) Figure 15.35 illustrates how we can modify a BERT model to perform different tasks, by simply adding one or more new output heads to the final hidden layer. See code.probml.ai/book1/bert_torch for some sample code.

In Fig. 15.35(a), we show how we can tackle single sentence classification (e.g., sentiment analysis): we simply take the feature vector associated with the dummy CLS token and feed it into an MLP. Since each output attends to all inputs, this hidden vector will summarize the entire sentence. The MLP then learns to map this to the desired label space.

In Fig. 15.35(b), we show how we can tackle sentence-pair classification (e.g., textual entailment, as discussed in Sec. 15.4.6): we just feed in the two input sentences, formatted as in Eq. (15.81), and then classify the CLS token.

In Fig. 15.35(c), we show how we can tackle single sentence tagging, in which we associate a label or tag with each word, instead of just the entire sentence. A common application of this is part of speech tagging, in which we annotate each words a noun, verb, adjective, etc. Another application of this is **noun phrase chunking**, also called **shallow parsing**, in which we must annotate the span of each noun phrase. The span is encoded using the **BIO** notation, in which B is the beginning of an entity, I-x is for inside, and O is for outside any entity. For example, consider the following sentence:

```
B       I       O    O     O          B   I         O    B   I   I
British Airways rose after announcing its withdrawl from the UAI deal
```

We see that there are 3 noun phrases, "British Airways", "its withdrawl" and "the UAI deal". (We require that the B, I and O labels occur in order, so this a prior constraint that can be included in the model.)

We can also associate types with each noun phrase, for example distinguishing person, location, organization, and other. Thus the label space becomes {B-Per, I-Per, B-Loc, I-Loc, B-Org, I-Org, Outside }. This is called **named entity recognition**, and is a key step in **information extraction**. For example, consider the following sentence:

```
BP  IP    O     O     O  BL  IL    BP    O      O   O       O
```

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

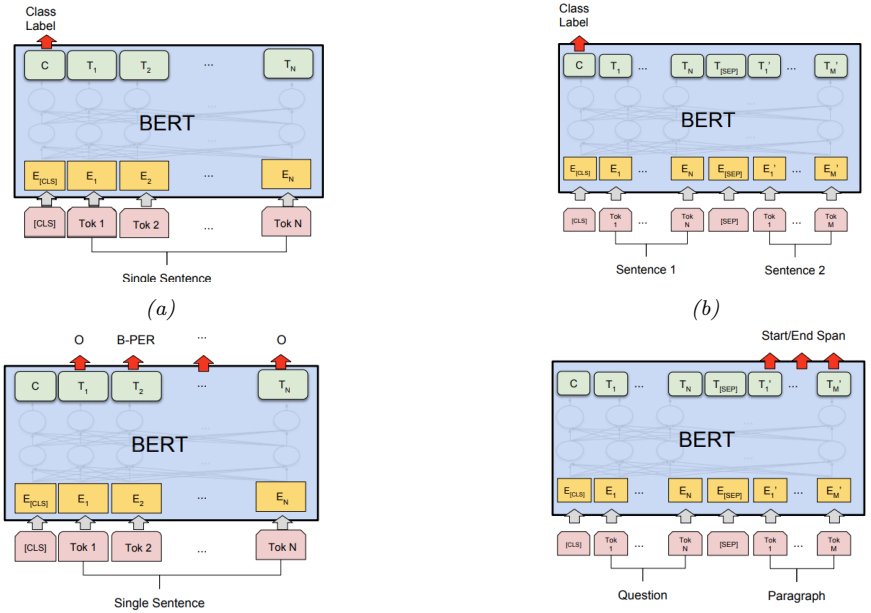


Figure 15.35: Illustration of how BERT can be used for different kinds of supervised NLP tasks. (a) Single sentence classification (e.g., sentiment analysis); (b) Sentence-pair classification (e.g., textual entailment); (d) Single sentence tagging (e.g., shallow parsing); (d) Question answering. From Figure 4 of [Dev+19]. Used with kind permission of Ming-Wei Chang.

```
Mrs Green spoke today in New York. Green chairs the finance committee.
```

From this, we infer that the first sentence has two named entities, namely "Mrs Green" (of type Person) and "New York" (of type Location). The second sentence mentions another person, "Green", that most likely is the same as the first person, although this across-sentence entity resolution is not part of the basic NER task.

Finally, in Fig. 15.35(d), we show how we can tackle **question answering**. Here the first input sentence is the question, the second is the background text, and the output is required to specifying the start and end locations of the relevant part of the background that contains the answer (see Fig. 1.15). The start location s and end location e are computed by applying 2 different MLPs to a pooled version of the output encodings for the background text; the output of the MLPs is a softmax over all locations. At test time, we can extract the span (i, j) which maximizes the sum of scores $s_i + e_j$ for $i \leq j$.

BERT achieves state of the art performance on many NLP tasks. Interestingly, [TDP19] shows that BERT implicitly rediscovers the standard NLP pipeline, in which different layers perform tasks such as part of speech (POS) tagging, parsing, named entity relationship (NER) detection, semantic role labeling (SRL), coreference resolution, etc. More details on NLP can be found in [JM20].

15.7.3 GPT

In [Rad+18], they propose a model called **GPT**, which is short for "Generative Pre-training Transformer". This is a causal (generative) model, that uses a masked transformer as the decoder. See Fig. 15.33b for an illustration.

In the original GPT paper, they jointly optimize on a large unlabeled dataset, and a small labeled dataset. In the classification setting, the loss is given by $\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{LM}}$, where $\mathcal{L}_{\text{cls}} = -\sum_{(\mathbf{x},y)\in\mathcal{D}_L} \log p(y|\mathbf{x})$ is the classification loss on the labeled data, and $\mathcal{L}_{\text{LM}} = -\sum_{\mathbf{x}\in\mathcal{D}_U} \sum_t p(x_t|\mathbf{x}_{1:t-1})$ is the language modeling loss on the unlabeled data.

In [Rad+19], they propose **GPT-2**, which is a larger version of GPT, trained on a large web corpus called **WebText**. They also eliminate any task-specific training, and instead just train it as a language model. More recently, OpenAI released **GPT-3** [Bro+20], which is an even larger version of GPT-2, but based on the same principles. An open-source version of the model is available at https://huggingface.co/EleutherAI, which was trained on an 800GB English-language web corpus called "**The Pile**" [Gao+20].

15.7.3.1 Applications of GPT

GPT can generate text given an initial input **prompt**. The prompt can specify a task; if the generated response fulfills the task "out of the box", we say the model is performing **zero-shot task transfer** (see Sec. 19.6 for details).

For example, to perform **abstractive summarization** of some input text $\mathbf{x}_{1:T}$ (as opposed to **extractive summarization**, which just selects a subset of the input words), we sample from $p(\mathbf{x}_{T+1:T+100}|[\mathbf{x}_{1:T}; \text{TL;DR}])$, where **TL;DR** is a special token added to the end of the input text, which tells the system the user wants a summary. TL;DR stands for "too long; didn't read" and frequently occurs in webtext followed by a human-created summary. By adding this token to the input, the user hopes to "trigger" the transformer decoder into a state in which it enters summarization mode. (A better way to tell the model what task to perform is to train it on input-output pairs, as discussed in Sec. 15.7.4.)

GPT has also been applied to image modeling. For example, [Che+20a] present a model called **iGPT**, which converts an image into a sequence of 32x32 patches, encodes each patch into a discrete symbol using vector quantization, and then trains the model in the same way as a language model. After training, the learned representation of the image (set of patches) is average pooled, and used as input to a supervised downstream classification task, an approach known as transfer learning (see Sec. 19.2). Pretraining in this way performs surprisingly well, despite the fact that transformers do not have any spatial inductive bias (see Sec. 15.5.6), and despite the fact that the input is reduced to a coarse resolution. (However, this may reflect the simplicity of the downstream classification tasks, rather than the ability of the model to learn "interesting structure" about the world.) To scale this method to higher resolution images, it is necessary to use some of the techniques discussed in Sec. 15.6 to avoid the $O(D^2)$ cost. For example, [Chi+19b] good results on generating a 64×64-sized version of ImageNet using **sparse transformers**. (This requires generating sequences of length $D = 64 \times 64 \times 3 = 12,288$.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

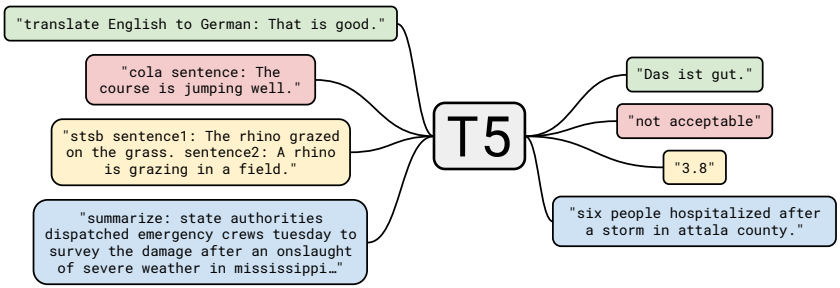


*Figure 15.36: Illustration of how the T5 model ("Text-to-text Transfer Transformer") can be used to perform multiple NLP tasks, such as translating English to German; determining if a sentence is linguistic valid or not (***CoLA** *stands for "Corpus of Linguistic Acceptability"); determining the degree of semantic similarity (***STSB** *stands for "Semantic Textual Similarity Benchmark"); and abstractive summarization. From Figure 1 of [Raf+19]. Used with kind permission of Colin Raffel.*

15.7.4 T5

Many models are trained in an unsupervised way, and then fine-tuned on specific tasks. It is also possible to train a single model to perform multiple tasks, by telling the system what task to perform as part of the input sentence, and then training it as a seq2seq model, as illustrated in Fig. 15.36. This is the approach used in **T5** [Raf+19], which stands for "Text-to-text Transfer Transformer". The model is a standard seq2seq transformer, that is pretrained on unsupervised $(\mathbf{x}', \mathbf{x}'')$ pairs, where $\mathbf{x}'$ is a masked version of $\mathbf{x}$ and $\mathbf{x}''$ are the missing tokens that need to be predicted, and then fine-tuned on multiple supervised $(\mathbf{x}, \mathbf{y})$ pairs.

The unsupervised data comes from **C4**, or the "Colossal Clean Crawled Corpus", a 750GB corpus of web text. This is used for pretraining using a BERT-like denoising objective. For example, the sentence $\mathbf{x}$ ="Thank you for inviting me to your party last week" may get converted to the input $\mathbf{x}'$ = "Thank you <X> me to your party <Y> week" and the output (target) $\mathbf{x}''$ = "<X> for inviting <Y> last <EOS>", where $< X >$ and $< Y >$ are tokens that are unique to this example. The supervised datasets are manually created, and are taken from the literature. This approach is currently the state-of-the art on many NLP tasks.

15.7.5 Discussion

Giant language models, such as BERT and GPT-3, have recently generated a lot of interest, and have even made their way into the mainstream media.[3] However, there is some doubt about whether such systems "understand" language in any meaningful way, beyond just rearranging word patterns seen in their massive training sets. For example, [NK19] show that the ability of BERT to perform almost as well as humans on the the Argument Reasoning Comprehension Task is "entirely accounted for by exploitation of spurious statistical cues in the dataset". By slightly tweaking the dataset, performance can be reduced to chance levels. For other criticisms of such models, see e.g., [BK20; Mar20].

3. See e.g., https://www.nytimes.com/2020/11/24/science/artificial-intelligence-ai-gpt3.html.

Part IV

Nonparametric models

16 Exemplar-based methods

So far in this book, we have mostly focused on **parametric models**, either unconditional $p(\mathbf{y}|\boldsymbol{\theta})$ or conditional $p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is a fixed-dimensional vector of parameters. The parameters are estimated from a variable-sized dataset, $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n) : n = 1 : N\}$, but after model fitting, the data is thrown away.

In this section we consider various kinds of **nonparametric models**, that keep the training data around. Thus the effective number of parameters of the model can grow with $|\mathcal{D}|$. We focus on models that can be defined in terms of the **similarity** between a test input, $\mathbf{x}$, and each of the training inputs, $\mathbf{x}_n$. Alternatively, we can define the models in terms of a dissimilarity or distance function $d(\mathbf{x}, \mathbf{x}_n)$. Since the models keep the training examples around at test time, we call them **exemplar-based models**. (This approach is also called **instance-based learning** [AKA91], or **memory-based learning**.)

16.1 K nearest neighbor (KNN) classification

In this section, we discuss one of the simplest kind of classifier, known as the **K nearest neighbor** (**KNN**) classifier. The idea is as follows: to classify a new input $\mathbf{x}$, we find the K closest examples to $\mathbf{x}$ in the training set, denoted $N_K(\mathbf{x}, \mathcal{D})$, and then look at their labels, to derive a distribution over the outputs for the local region around $\mathbf{x}$. More precisely, we compute

$$p(y = c|\mathbf{x}, \mathcal{D}) = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c) \tag{16.1}$$

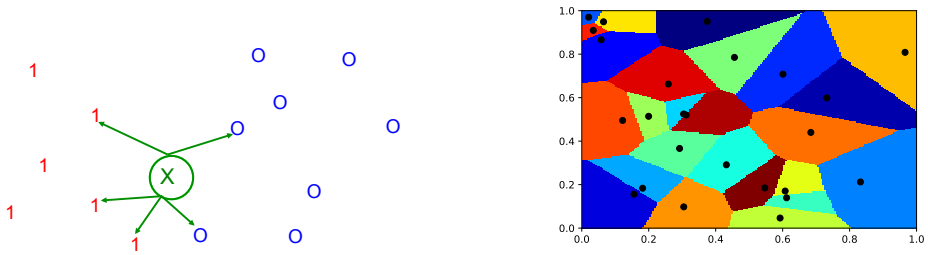
We can then return this distribution, or the majority label.

The two main parameters in the model are the size of the neighborhood, K, and the distance metric $d(\mathbf{x}, \mathbf{x}')$. For the latter, it is common to use the **Mahalanobis distance**

$$d_{\mathbf{M}}(\mathbf{x}, \boldsymbol{\mu}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{M} (\mathbf{x} - \boldsymbol{\mu})} \tag{16.2}$$

where $\mathbf{M}$ is a positive definite matrix. If $\mathbf{M} = \mathbf{I}$, this reduces to Euclidean distance. We discuss how to learn the distance metric in Sec. 16.2.

Despite the simplicity of KNN classifiers, it can be shown that this approach becomes within a factor of 2 of the Bayes error (which measures the performance of the best possible classifier) if $N \to \infty$ [CH67; CD14]. (Of course the convergence rate to this optimal performance may be poor in practice, for reasons we discuss in Sec. 16.1.2.)



(a) *(b)*

Figure 16.1: (a) Illustration of a K-nearest neighbors classifier in 2d for $K = 5$. The nearest neighbors of test point $\mathbf{x}$ have labels $\{1, 1, 1, 0, 0\}$, so we predict $p(y = 1|\mathbf{x}, \mathcal{D}) = 3/5$. (b) Illustration of the Voronoi tesselation induced by 1-NN. Adapted from Figure 4.13 of [DHS01]. Generated by code at figures.probml.ai/book1/16.1.

16.1.1 Example

We illustrate the KNN classifier in 2d in Fig. 16.1(a) for $K = 5$. The test point is marked as an "x". 3 of the 5 nearest neighbors have label 1, and 2 of the 5 have label 0. Hence we predict $p(y = 1|\mathbf{x}, \mathcal{D}) = 3/5 = 0.6$.

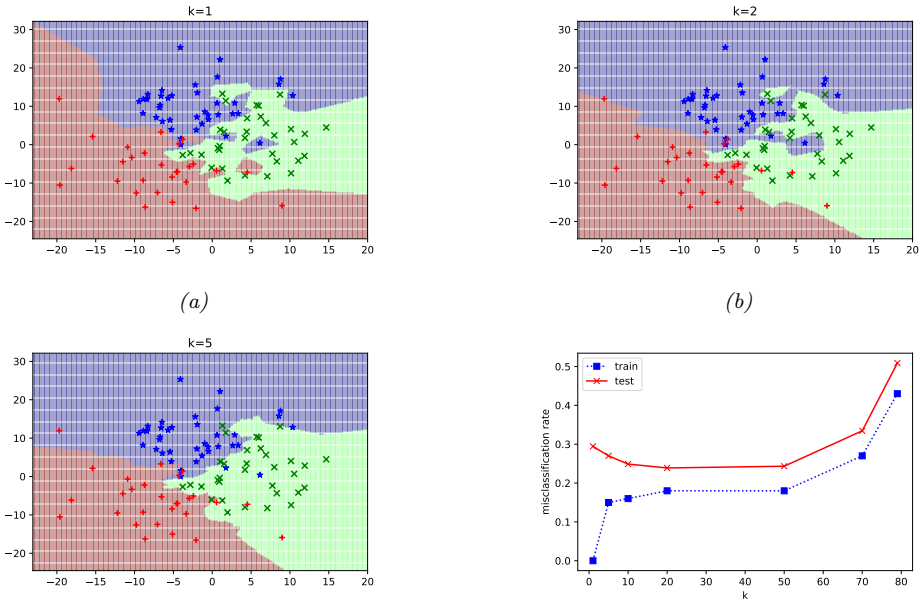
If we use $K = 1$, we just return the label of the nearest neighbor, so the predictive distribution becomes a delta function. A KNN classifier with $K = 1$ induces a **Voronoi tessellation** of the points (see Fig. 16.1(b)). This is a partition of space which associates a region $V(\mathbf{x}_n)$ with each point $\mathbf{x}_n$ in such a way that all points in $V(\mathbf{x}_n)$ are closer to $\mathbf{x}_n$ than to any other point. Within each cell, the predicted label is the label of the corresponding training point. Thus the training error will be 0 when $K = 1$. However, such a model is usually overfitting the training set, as we show below.

Fig. 16.2 gives an example of KNN applied to a 2d dataset, in which we have three classes. We see how, with $K = 1$, the method makes zero errors on the training set. As K increases, the decision boundaries become smoother (since we are averaging over larger neighborhoods), so the training error increases, as we start to underfit. This is shown in Fig. 16.2(d). The test error shows the usual U-shaped curve.

16.1.2 The curse of dimensionality

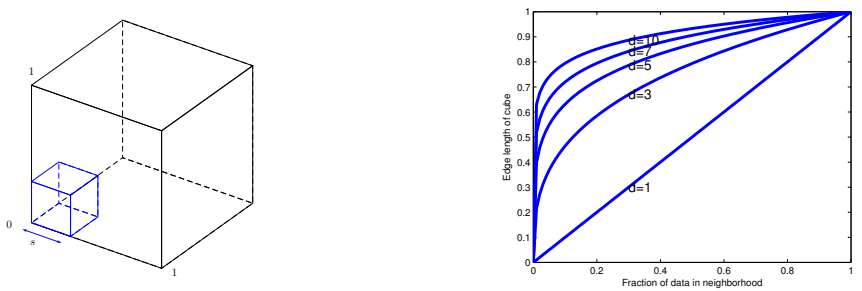
The main statistical problem with KNN classifiers is that they do not work well with high dimensional inputs, due to the **curse of dimensionality**.

The basic problem is that the volume of space grows exponentially fast with dimension, so you might have to look quite far away in space to find your nearest neighbor. To make this more precise, consider this example from [HTF09, p22]. Suppose we apply a KNN classifier to data where the inputs are uniformly distributed in the D-dimensional unit cube. Suppose we estimate the density of class labels around a test point $\mathbf{x}$ by "growing" a hyper-cube around $\mathbf{x}$ until it contains a desired fraction p of the data points. The expected edge length of this cube will be $e_D(s) \triangleq p^{1/D}$; this function is plotted in Fig. 16.3(b). If $D = 10$, and we want to base our estimate on 10% of the data, we have $e_{10}(0.1) = 0.8$, so we need to extend the cube 80% along each dimension around $\mathbf{x}$. Even if



(a) (b) (c) (d)

Figure 16.2: Decision boundaries induced by a KNN classifier. (a) $K = 1$. (b) $K = 2$. (c) $K = 5$. (d) Train and test error vs K. Generated by code at figures.probml.ai/book1/16.2.



(a) (b)

Figure 16.3: Illustration of the curse of dimensionality. (a) We embed a small cube of side s inside a larger unit cube. (b) We plot the edge length of a cube needed to cover a given volume of the unit cube as a function of the number of dimensions. Adapted from Figure 2.6 from [HTF09]. Generated by code at figures.probml.ai/book1/16.3.

we only use 1% of the data, we find $e_{10}(0.01) = 0.63$. Since the range of the data is only 0 to 1 along each dimension, we see that the method is no longer very local, despite the name "nearest neighbor". The trouble with looking at neighbors that are so far away is that they may not be good predictors about the behavior of the function at a given point.

There are two main solutions to the curse: make some assumptions about the form of the function (i.e., use a parametric model), and/or use a metric that only cares about a subset of the dimensions (see Sec. 16.2).

16.1.3 Reducing the speed and memory requirements

KNN classifiers store all the training data. This is obviously very wasteful of space. Various heuristic pruning techniques have been proposed to remove points that do not affect the decision boundaries, see e.g., [WM00]. In Sec. 17.6, we discuss a more principled approach based on a sparsity promoting prior; the resulting method is called a sparse kernel machine, and only keeps a subset of the most useful exemplars.

In terms of running time, the challenge is to find the K nearest neighbors in less than $O(N)$ time, where N is the size of the training set. Finding exact nearest neighbors is computationally intractable when the dimensionality of the space goes above about 10 dimensions, so most methods focus on finding the approximate nearest neighbors. There are two main classes of techniques, based on partitioning space into regions, or using hashing.

For partitioning methods, one can either use some kind of **k-d tree**, which divides space into axis-parallel regions, or some kind of clustering method, which uses anchor points. For hashing methods, **locality sensitive hashing** (**LSH**) [GIM99] is widely used, although more recent methods learn the hashing function from data (see e.g., [Wan+15]). See [LRU14] for a good introduction to hashing methods.

An open-source library called **FAISS**, for efficient exact and approximate nearest neighbor search (and K-means clustering) of dense vectors, is available at `https://github.com/facebookresearch/faiss`, and described in [JDJ17].

16.1.4 Open set recognition

> Ask not what this is called, ask what this is like. — Moshe Bar.[Bar09]

In all of the classification problems we have considered so far, we have assumed that the set of classes $\mathcal{C}$ is fixed. (This is an example of the **closed world assumption**, which assumes there is a fixed number of (types of) things.) However, many real world problems involve test samples that come from new categories. This is called **open set recognition**, as we discuss below.

16.1.4.1 Online learning, OOD detection and open set recognition

For example, suppose we train a face recognition system to predict the identity of a person from a fixed set or **gallery** of face images. Let $\mathcal{D}_t = \{(\mathbf{x}_n, y_n) : \mathbf{x}_n \in \mathcal{X}, y_n \in \mathcal{C}_t, n = 1 : N_t\}$ be the labeled dataset at time t, where $\mathcal{X}$ is the set of (face) images, and $\mathcal{C}_t = \{1, \ldots, C_t\}$ is the set of people known to the system at time t (where $C_t \leq t$). At test time, the system may encounter a new person that it has not seen before. Let $\mathbf{x}_{t+1}$ be this new image, and $y_{t+1} = C_{t+1}$ be its new label. The system needs to recognize that the input is from a new category, and not accidently classify it with a label from $\mathcal{C}_t$.

This is called **novelty detection**. In this case, the input is being generated from the distribution $p(\mathbf{x}|y = C_{t+1})$, where $C_{t+1} \notin \mathcal{C}_t$ is the new "class label". Detecting that $\mathbf{x}_{t+1}$ is from a novel class may be hard if the appearance of this new image is similar to the appearance of any of the existing images in $\mathcal{D}_t$.

If the system is succesful at detecting that $\mathbf{x}_{t+1}$ is novel, then it may ask for the id of this new instance, call it C_{t+1}. It can then add the labeled pair $(\mathbf{x}_{t+1}, C_{t+1})$ to the dataset to create $\mathcal{D}_{t+1}$, and can grow the set of unique classes by adding C_{t+1} to $\mathcal{C}_t$ (c.f., [JK13]). This is called **incremental learning**, **online learning**, **life-long learning**, or **continual learning**. At future time points, the system may encounter an image sampled from $p(\mathbf{x}|y = c)$, where c is an existing class, or where c is a new class, or the image may be sampled from some entirely different kind of distribution $p'(\mathbf{x})$ unrelated to faces (e.g., someone uploads a photo of their dog). (Detecting this latter kind of event is called **out-of-distribution** or **OOD** detection.)

In this online setting, we often only get a few (sometimes just one) example of each class. Prediction in this setting is known as **few-shot classification**, and is discussed in more detail in Sec. 19.6. KNN classifiers are well-suited to this task. For example, we can just store all the instances of each class in a gallery of examples, as we explained above. At time $t + 1$, when we get input $\mathbf{x}_{t+1}$, rather than predicting a label for $\mathbf{x}_{t+1}$ by comparing it to some parametric model for each class, we just find the example in the gallery that is nearest (most similar) to $\mathbf{x}_{t+1}$, call it $\mathbf{x}'$. We then need to determine if $\mathbf{x}'$ and $\mathbf{x}_{t+1}$ are sufficiently similar to constitute a match. (In the context of person classification, this is known as **person re-identification** or **face verification**, see e.g., [WSH16]).) If there is no match, we can declare the input to be novel or OOD.

The key ingredient for all of the above problems is the (dis)similarity metric between inputs. We discuss ways to learn this in Sec. 16.2.

16.1.4.2 Other open world problems

The problem of open-set recognition, and incremental learning, are just examples of problems that require the **open world assumption** c.f., [Rus15]. There are many other examples of such problems.

For example, consider the problem of **entity resolution**, called **entity linking**. In this problem, we need to determine if different strings (e.g., "John Smith" and "Jon Smith") refer to the same entity or not. See e.g. [SHF15] for details.

Another important application is in **multi-object tracking**. For example, when a radar system detects a new "blip", is it due to an existing missile that is being tracked, or is it a new objective that has entered the airspace? An elegant mathematical framework for dealing with such problems, known as **random finite sets**, is described in [Mah07; Mah13; Vo+15].

16.2 Learning distance metrics

Being able to compute the "semantic distance" between a pair of points, $d(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^+$ for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$, or equivalently their similarity $s(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^+$, is of crucial importance to tasks such as nearest neighbor classification (Sec. 16.1), self-supervised learning (Sec. 19.2.3.3), similarity-based clustering (Sec. 21.5), content-based retrieval, visual tracking, etc.

When the input space is $\mathcal{X} = \mathbb{R}^D$, the most common distance metric is the Mahalanobis distance

$$d_{\mathbf{M}}(\mathbf{x}, \mathbf{x}') = \sqrt{(\mathbf{x} - \mathbf{x}')^\top \mathbf{M} (\mathbf{x} - \mathbf{x}')} \tag{16.3}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We discuss some methods to learn the matrix $\mathbf{M}$ in Sec. 16.2.1. For high dimensional inputs, or structured inputs, it is better to first learn an embedding $\mathbf{e} = f(\mathbf{x})$, and then to compute distances in embedding space. When f is a DNN, this is called **deep metric learning**; we discuss this in Sec. 16.2.2.

16.2.1 Linear and convex methods

In this section, we discuss some methods that try to learn the Mahalanobis distance matrix $\mathbf{M}$, either directly (as a convex problem), or indirectly via a linear projection. For other approaches to metirc learning, see e.g., [Kul13; Kim19] for more details.

16.2.1.1 Large margin nearest neighbors

In [WS09], they propose to learn the Mahalanobis matrix $\mathbf{M}$ so that the resulting distance metric works well when used by a nearest neighbor classifier. The resulting method is called **large margin nearest neighbor** or **LMNN**.

This works as follows. For each example data point i, let N_i be a set of **target neighbors**; these are usually chosen to be the set of K points with the same class label that are closest in Euclidean distance. We now optimize $\mathbf{M}$ so that we minimize the distance between each point i and all of its target neighbors $j \in N_i$:

$$\mathcal{L}_{\text{pull}}(\mathbf{M}) = \sum_{i=1}^{N} \sum_{j \in N_i} d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j)^2 \tag{16.4}$$

We also want to ensure that examples with incorrect labels are far away. To do this, we ensure that each example i is closer (by some margin $m \geq 0$) to its target neighbors j than to other points l with different labels (so-called **impostors**). We can do this by minimizing

$$\mathcal{L}_{\text{push}}(\mathbf{M}) = \sum_{i=1}^{N} \sum_{j \in N_i} \sum_{l=1}^{N} \mathbb{I}\left(y_i \neq y_l\right) \left[m + d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j)^2 - d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_l)^2\right]_+ \tag{16.5}$$

where $[z]_+ = \max(z, 0)$ is the hinge loss function (Sec. 4.3.2). The overall objective is $\mathcal{L}(\mathbf{M}) = (1-\lambda)\mathcal{L}_{\text{pull}}(\mathbf{M}) + \lambda \mathcal{L}_{\text{push}}(\mathbf{M})$, where $0 < \lambda < 1$. This is a convex function defined over a convex set, which can be minimized using **semidefinite programming**. Alternatively, we can parameterize the problem using $\mathbf{M} = \mathbf{W}^\top \mathbf{W}$, and then minimize wrt $\mathbf{W}$ using unconstrained gradient methods. This is no longer convex, but allows us to use a low-dimensional mapping $\mathbf{W}$.

For large datasets, we need to tackle the $O(N^3)$ cost of computing Eq. (16.5). We discuss some speedup tricks in Sec. 16.2.5.

16.2.1.2 Neighborhood components analysis

Another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^\top \mathbf{W}$ is known as **neighborhood components analysis** or **NCA** [Gol+05]. This defines the probability that sample $\mathbf{x}_i$ has $\mathbf{x}_j$ as its nearest neighbor using the linear softmax function

$$p_{ij}^{\mathbf{W}} = \frac{\exp(-||\mathbf{W}\mathbf{x}_i - \mathbf{W}\mathbf{x}_j||_2^2)}{\sum_{l \neq i} \exp(-||\mathbf{W}\mathbf{x}_i - \mathbf{W}\mathbf{x}_l||_2^2)} \tag{16.6}$$

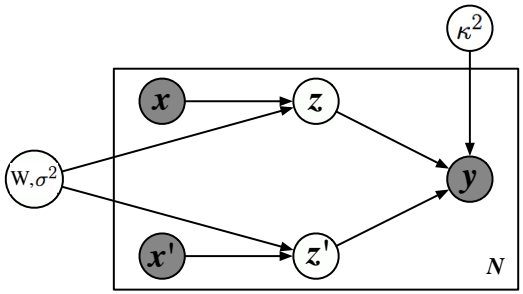


Figure 16.4: Illustration of latent coincidence analysis (LCA) as a directed graphical model. The inputs $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^D$ are mapped into Gaussian latent variables $\mathbf{z}, \mathbf{z}' \in \mathbb{R}^L$ via a linear mapping $\mathbf{W}$. If the two latent points coincide (within length scale κ) then we set the similarity label to $y = 1$, otherwise we set it to $y = 0$. From Figure 1 of [DS12]. Used with kind permission of Lawrence Saul.

(This is a supervised version of stochastic neighborhood embeddings discussed in Sec. 20.4.10.1.) The expected number of correctly classified examples according for a 1NN classifier using distance $\mathbf{W}$ is given by $J(\mathbf{W}) = \sum_{i=1}^{N} \sum_{j \neq i: y_j = y_i} p_{ij}^{\mathbf{W}}$. Let $\mathcal{L}(\mathbf{W}) = 1 - J(\mathbf{W})/N$ be the leave one out error. We can minimize $\mathcal{L}$ wrt $\mathbf{W}$ using gradient methods.

16.2.1.3 Latent coincidence analysis

Yet another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^\top \mathbf{W}$ is known as **latent coincidence analysis** or **LCA** [DS12]. This defines a conditional latent variable model for mapping a pair of inputs, $\mathbf{x}$ and $\mathbf{x}'$, to a label $y \in \{0, 1\}$, which specifies if the inputs are similar (e.g., have same class label) or dissimilar. Each input $\mathbf{x} \in \mathbb{R}^D$ is mapped to a low dimensional latent point $\mathbf{z} \in \mathbb{R}^L$ using a stochastic mapping $p(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\mathbf{W}\mathbf{x}, \sigma^2 \mathbf{I})$, and $p(\mathbf{z}'|\mathbf{x}') = \mathcal{N}(\mathbf{z}'|\mathbf{W}\mathbf{x}', \sigma^2 \mathbf{I})$. (Compare this to factor analysis, discussed in Sec. 20.2.) We then define the probability that the two inputs are similar using $p(y = 1|\mathbf{z}, \mathbf{z}') = \exp(-\frac{1}{2\kappa^2}||\mathbf{z} - \mathbf{z}'||)$. See Fig. 16.4 for an illustration of the modeling assumptions.

We can maximize the log marginal likelihood $\ell(\mathbf{W}, \sigma^2, \kappa^2) = \sum_n \log p(y_n|\mathbf{x}_n, \mathbf{x}'_n)$ using the EM algorithm (Sec. 8.7.2). (We can set $\kappa = 1$ WLOG, since it just changes the scale of $\mathbf{W}$.) More precisely, in the E step, we compute the posterior $p(\mathbf{z}, \mathbf{z}'|\mathbf{x}, \mathbf{x}', y)$ (which can be done in closed form), and in the M step, we solve a weighted least squares problem (c.f., Sec. 13.6.2). EM will monotonically increase the objective, and does not need step size adjustment, unlike the gradient based methods used in NCA (Sec. 16.2.1.2). (It is also possible to use variational Bayes (Sec. 4.6.8.3) to fit this model, as well as various sparse and nonlinear extensions, as discussed in [ZMY19].)

16.2.2 Deep metric learning

When measuring the distance between high-dimensional or structured inputs, it is very useful to first learn an embedding to a lower dimensional "semantic" space, where distances are more meaningful, and less subject to the curse of dimensionality (Sec. 16.1.2). Let $\mathbf{e} = f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}^L$ be an embedding of the input that preserves the "relevant" semantic aspects of the input, and let $\hat{\mathbf{e}} = \mathbf{e}/||\mathbf{e}||_2$ be the ℓ_2-normalized version. This ensures that all points lie on a hyper-sphere. We can then measure the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

distance between two points using the normalized Euclidean distance

$$d(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta}) = ||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j||_2^2 \tag{16.7}$$

where smaller values means more similar, or the cosine similarity

$$d(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta}) = \hat{\mathbf{e}}_i^\top \hat{\mathbf{e}}_j \tag{16.8}$$

where larger values means more similar. (Cosine similarity measures the angle between the two vectors, as illustrated in Fig. 20.43.) These quantities are related via

$$||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j||_2^2 = (\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j)^\top (\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j) = 2 - 2\hat{\mathbf{e}}_i^\top \hat{\mathbf{e}}_j \tag{16.9}$$

This overall approach is called **deep metric learning** or DML.

The basic idea in DML is to learn the embedding function such that similar examples are closer than dissimilar examples. More precisely, we assume we have a labeled dataset, $\mathcal{D} = \{(\mathbf{x}_i, y_i) : i = 1 : N\}$, from which we can derive a set of similar pairs, $\mathcal{S} = \{(i, j) : y_i = y_j\}$. If $(i, j) \in \mathcal{S}$ but $(i, k) \notin S$, then we assume that $\mathbf{x}_i$ and $\mathbf{x}_j$ should be close in embedding space, whereas $\mathbf{x}_i$ and $\mathbf{x}_k$ should be far. We discuss various ways to enforce this property below. Note that these methods also work when we do not have class labels, provided we have some other way of defining similar pairs. For example, in Sec. 19.2.3.3, we discuss self-supervised approaches to representation learning, that automatically create semantically similar pairs, and learn embeddings to force these pairs to be closer than unrelated pairs.

Before discussing DML in more detail, it is worth mentioning that many recent approaches to DML are not as good as they claim to be, as pointed out in [MBL20; Rot+20]. (The claims in some of these papers are often invalid due to improper experimental comparisons, a common flaw in contemporary ML research, as discussed in e.g., [BLV19; LS19d].) We therefore focus on (slightly) older and simpler methods, that tend to be more robust.

16.2.3 Classification losses

Suppose we have labeled data with C classes. Then we can fit a classification model in $O(NC)$ time, and then reuse the hidden features as an embedding function. (It is common to use the second-to-last layer, since it generalizes better to new classes than the final layer.) This approach is simple and scalable. However, it only learns to embed examples on the correct side of a decision boundary, which does not necessarily result in similar examples being placed close together and dissimilar examples being placed far apart. In addition, this method cannot be used if we do not have labeled training data.

16.2.4 Ranking losses

In this section, we consider minimizing **ranking loss**, to ensure that similar examples are closer than dissimilar examples. Most of these methods do not need class labels (although we sometimes assume that labels exist as a notationally simple way to define similarity).

(a) (b)

Figure 16.5: Networks for deep metric learning. (a) Siamese network. (b) Triplet network. Adapted from Figure 5 of [KB19].

16.2.4.1 Pairwise (contrastive) loss and Siamese networks

One of the earliest approaches to representation learning from similar/dissimilar pairs was based on minimizing the following **contrastive loss** [CHL05]:

$$\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_i, \mathbf{x}_j) = \mathbb{I}\left(y_i = y_j\right) d(\mathbf{x}_i, \mathbf{x}_j)^2 + \mathbb{I}\left(y_i \neq y_j\right) [m - d(\mathbf{x}_i, \mathbf{x}_j)]_+^2 \tag{16.10}$$

where $[z]_+ = \max(0, z)$ is the hinge loss and $, > 0$ is a margin parameter. Intuitively, we want to force positive pairs (with the same label) to be close, and negative pairs (with different labels) to be further apart than some minimal safety margin. We minimize this loss over all pairs of data. Naively this takes $O(N^2)$ time; see Sec. 16.2.5 for some speedups.

Note that we use the same feature extractor $\mathbf{f}(\cdot; \boldsymbol{\theta})$ for both inputs, $\mathbf{x}_i$ and $\mathbf{x}_j$. when computing the distance, as illustrated in Fig. 16.5a. The resulting network is therefore called a **Siamese network** (named after Siamese twins).

16.2.4.2 Triplet loss

One disadvantage of pairwise losses is that the optimization of the positive pairs is independent of the negative pairs, which can make their magnitudes incomparable. A solution to this is to use the **triplet loss** [SKP15]. This is defined as follows. For each example i (known as an **anchor**), we find a similar (positive) example $\mathbf{x}_i^+$ and a dissimilar (negative) example $\mathbf{x}_i^-$. We then minimize the following loss, averaged overall all triples:

$$\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_i, \mathbf{x}_i^+, \mathbf{x}_i^-) = [d_{\boldsymbol{\theta}}(\mathbf{x}_i, \mathbf{x}_i^+)^2 - d_{\boldsymbol{\theta}}(\mathbf{x}_i, \mathbf{x}_i^-)^2 + m]_+ \tag{16.11}$$

Intuitively this says we want the distance from the anchor to the positive to be less (by some safety margin m) than the distance from the anchor to the negative. We can compute the triplet loss using a triplet network as shown in Fig. 16.5b.

Naively minimizing triplet loss takes $O(N^3)$ time. In practice we compute the loss on a minibatch (chosen so that there is at least one similar and one dissimilar example for the anchor point, often taken to be the first entry in the minibatch). Nevertheless the method can be slow, We discuss some speedups in Sec. 16.2.5.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

16.2.4.3 N-pairs loss

One problem with the triplet loss is that each anchor is only compared to one negative example at a time. This might not provide a strong enough learning signal. One solution to this is to create a multi-class classification problem in which we create a set of $N-1$ negatives and 1 positive for every anchor. This is called the **N-pairs loss** [Soh16]. More precisely, we define the following loss for each set:

$$\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}_i, \mathbf{x}_j^+, \{\mathbf{x}_k^-\}_{k=1}^{N-1}) = -\log\left(1 + \sum_{k=1}^{N-1} \exp(\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_k^-) - \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_j^+))\right) \tag{16.12}$$

$$= -\log \frac{\exp(\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_k^-))}{\exp(\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_j^+)) + \sum_{k=1}^{N-1} \exp(\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_k^-))} \tag{16.13}$$

Note that the N-pairs loss is the same as the **InfoNCE** loss used in the CPC paper [OLV18]. In [Che+20b], they propose a version where the scale the similarities by a temperature term; they call this the **NT-Xent** (normalized temperature-scaled cross-entropy) loss. We can view the temperature parameter as scaling the radius of the hypersphere on which the data lives.

When $N = 2$, the loss reduces to the logistic loss

$$\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}, \mathbf{x}^+, \mathbf{x}_i) = -\log\left(1 + \exp(\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x})^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i) - \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x})^\top \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}^+))\right) \tag{16.14}$$

Compare this to the margin loss used by triplet learning (when $m = 1$):

$$\mathcal{L}(\boldsymbol{\theta}; \mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = -\max\left(0, \hat{\mathbf{e}}(\mathbf{x})^\top \hat{\mathbf{e}}(\mathbf{x}^-) - \hat{\mathbf{e}}(\mathbf{x})^\top \hat{\mathbf{e}}(\mathbf{x}^+) + 1\right) \tag{16.15}$$

See Fig. 4.2 for a comparison of these two functions.

16.2.5 Speeding up ranking loss optimization

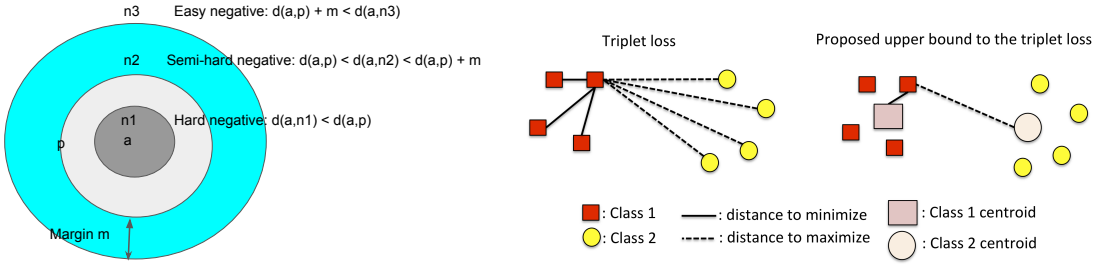
The main disadvantage of ranking loss is the $O(N^2)$ or $O(N^3)$ cost of computing the loss function, due to the need to compare all pairs or triples of examples. In this section, we discuss various speedup tricks.

16.2.5.1 Mining techniques

A key insight is that we don't need to consider all negative examples for each anchor, since most will be uniformative (i.e., will incur zero loss). Instead we can focus attention on negative examples which are closer to the anchor than its nearest positive example. These are called **hard negatives**, and are particularly useful for speeding up triplet loss.

More precisely, if a is an anchor and p is its nearest positive example, we say that n is a hard negative (for a) if $d(\mathbf{x}_a, \mathbf{x}_n) < d(\mathbf{x}_a, \mathbf{x}_p)$ and $y_n \neq y_a$. Sometimes an anchor may not have any hard negatives. We can therefore increase the pool of candidates by considering **semi-hard negatives**, for which

$$d(\mathbf{x}_a, \mathbf{x}_p) < d(\mathbf{x}_a, \mathbf{x}_n) < d(\mathbf{x}_a, \mathbf{x}_p) + m \tag{16.16}$$



(a) *(b)*

Figure 16.6: Speeding up triplet loss minimization. (a) Illustration of hard vs easy negatives. Here a is the anchor point, p is a positive point, and n_i are negative points. Adapted from Figure 4 of [KB19]. (b) Standard triplet loss would take $8 \times 3 \times 4 = 96$ calculations, whereas using a proxy loss (with one proxy per class) takes $8 \times 2 = 16$ calculations. From Figure 1 of [Do+19]. Used with kind permission of Gustavo Cerneiro.

where $m > 0$ is a margin parameter. See Fig. 16.6a for an illustration. This is the technique used by Google's **FaceNet** model [SKP15], which learns an embedding function for faces, so it can cluster similar looking faces together, to which the user can attach a name.

In practice, the hard negatives are usually chosen from within the minibatch. This therefore requires large batch sizes to ensure sufficient diversity. Alternatively, we can have a separate process that continually updates the set of candidate hard negatives, as the distance measure evolves during training.

16.2.5.2 Proxy methods

Triplet loss minimization is expensive even with hard negative mining (Sec. 16.2.5.1). Ideally we can find a method that is $O(N)$ time, just like classification loss.

One such method, proposed in [MA+17], measures the distance between each anchor and a set of P **proxies** that represent each class, rather than directly measuring distance between examples. These proxies need to be updated online as the distance metric evolves during learning. The overall procedure takes $O(NP^2)$ time, where $P \sim C$.

More recently, [Qia+19] propoosed to represent each class with multiple prototypes, while still achieving linear time complexity, using a **soft triple** loss.

16.2.5.3 Optimizing an upper bound

[Do+19] proposed a simple and fast method for optimizing the triplet loss. The key idea is to define one *fixed* proxy or centroid per class, and then to use distance to the proxy as an upper bound on the triplet loss.

More precisely, consider a simplified form of the triplet loss, without the margin term:

$$\ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) = ||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j|| - ||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_k|| \tag{16.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\hat{\mathbf{e}}_i = \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{x}_i)$, etc. Using the triangle inequality we have

$$||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j|| \leq ||\hat{\mathbf{e}}_i - \mathbf{c}_{y_i}|| + ||\hat{\mathbf{e}}_j - \mathbf{c}_{y_i}|| \tag{16.18}$$

$$||\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_k|| \geq ||\hat{\mathbf{e}}_i - \mathbf{c}_{y_k}|| - ||\hat{\mathbf{e}}_k - \mathbf{c}_{y_k}|| \tag{16.19}$$

Hence

$$\ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \leq \ell_u(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \triangleq ||\hat{\mathbf{e}}_i - \mathbf{c}_{y_i}|| - ||\hat{\mathbf{e}}_i - \mathbf{c}_{y_k}|| + ||\hat{\mathbf{e}}_j - \mathbf{c}_{y_i}|| + ||\hat{\mathbf{e}}_k - \mathbf{c}_{y_k}|| \tag{16.20}$$

We can use this to derive a tractable upper bound on the triplet loss as follows:

$$\mathcal{L}_t(\mathcal{D}, \mathcal{S}) = \sum_{(i,j)\in\mathcal{S},(i,k)\notin\mathcal{S},i,j,k\in\{1,\ldots,N\}} \ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \leq \sum_{(i,j)\in\mathcal{S},(i,k)\notin\mathcal{S},i,j,k\in\{1,\ldots,N\}} \ell_u(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \tag{16.21}$$

$$= C' \sum_{i=1}^{N} \left(||\mathbf{x}_i - \mathbf{c}_{y_i}|| - \frac{1}{3(C-1)} \sum_{m=1, m\neq y_i}^{C} ||\mathbf{x}_i - \mathbf{c}_m|| \right) \triangleq \mathcal{L}_u(\mathcal{D}, \mathcal{S}) \tag{16.22}$$

where $C' = 3(C-1)(\frac{N}{C} - 1)\frac{N}{C}$ is a constant. It is clear that $\mathcal{L}_u$ can be computed in $O(NC)$ time. See Fig. 16.6b for an illustration.

In [Do+19], they show that $0 \leq \mathcal{L}_t - \mathcal{L}_u \leq \frac{N^3}{C^2}K$, where K is some constant that depends on the spread of the centroids. To ensure the bound is tight, the centroids should be as far from each other as possible, and the distances between them should be as similar as possible. An easy way to ensure is to define the $\mathbf{c}_m$ vectors to be one-hot vectors, one per class. These vectors already have unit norm, and are orthogonal to each other. The distance between each pair of centroids is $\sqrt{2}$, which ensures the upper bound is fairly tight.

The downside of this approach is that it assumes the embedding layer is $L = C$ dimensional. There are two solutions to this. First, after training, we can add a linear projection layer to map from C to $L \neq C$, or we can take the second-to-last layer of the embedding network. The second approach is to sample a large number of points on the L-dimensional unit hyper-sphere (which we can do by sampling from the standard normal, and then normalizing [Mar72]), and then running K-means clustering (Sec. 21.3) with $K = C$. In the experiments reported in [Do+19], these two approaches give similar results.

Interestingly, in [Rot+20], they show that increasing $\pi_{\text{intra}}/\pi_{\text{inter}}$ results in improved downstream performance on various retrieval tasks, where

$$\pi_{\text{intra}} = \frac{1}{Z_{\text{intra}}} \sum_{c=1}^{C} \sum_{i\neq j: y_i = y_j = c} d(\mathbf{x}_i, \mathbf{x}_j) \tag{16.23}$$

is the average intra-class distance, and

$$\pi_{\text{inter}} = \frac{1}{Z_{\text{inter}}} \sum_{c=1}^{C} \sum_{c'=1}^{C} d(\boldsymbol{\mu}_c, \boldsymbol{\mu}_{c'}) \tag{16.24}$$

is the average inter-class distance, where $\boldsymbol{\mu}_c = \frac{1}{Z_c} \sum_{i:y_i=c} \hat{\mathbf{e}}_i$ is the mean embedding for examples from class c. This suggests that we should not only keep the centroids far apart (in order to maximize the numerator), but we should also prevent examples from getting too close to their centroids (in order to minimize the denominator); this latter term is not captured in the method of [Do+19].

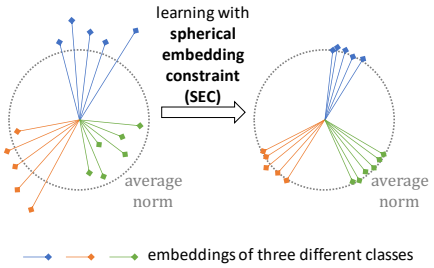


Figure 16.7: Adding spherical embedding constraint to a deep metric learning method. Used with kind permission of Dingyi Zhang.

16.2.6 Other training tricks for DML

Besides the speedup tricks in Sec. 16.2.5, there are a lot of other details that are important to get right in order to ensure good DML performance. Many of these details are discussed in [MBL20; Rot+20]. Here we just briefly mention a few.

One important issue is how the minibatches are created. In classification problems (at least with balanced classes), selecting examples at random from the training set is usually sufficient. However, for DML, we need to ensure that each example has some other examples in the minibatch that are similar to it, as well as some others that are dissimilar to it. One approach is to use hard mining techniques (Sec. 16.2.5.1). Another idea is to use coreset methods applied to previously learned embeddings to select a diverse minibatch at each step [Sin+20]. However, [Rot+20] show that the following simple strategy also works well for creating each batch: pick B/n classes, and then pick N_c examples randomly from each class, where B is the batch size, and $N_c = 2$ is a tuning parameter.

Another important issue is avoiding overfitting. Since most datasets used in the DML literature are small, it is standard to use an image classifier, such as GoogLeNet (Sec. 14.3.3) or ResNet (Sec. 14.3.4), which has been pre-trained on ImageNet, and then to fine-tune the model using the DML loss. (See Sec. 19.2 for more details on this kind of transfer learning.) In addition, it is standard to use data augmentation (see Sec. 19.1). (Indeed, with some self-supervised learning methods, data aug is the only way to create similar pairs.)

In [ZLZ20], they propose to add a **spherical embedding constraint** (SEC), which is an additional batchwise regularization term, which encourages all the examples to have the same norm. That is, the regularizer is just the empirical variance of the norms of the (unnormalized) embeddings in that batch. See Fig. 16.7 for an illustration. This regularizer can be added to any of the existing DML losses to modestly improve training speed and stability, as well as final performance, analogously to how batchnorm (Sec. 14.2.4.1) is used.

16.3 Kernel density estimation (KDE)

In this section, we consider a form of non-parametric density esimation known as **kernel density estimation** or **KDE**. This is a form of generative model, since it defines a probability distribution $p(\mathbf{x})$ that can be evaluated pointwise, and which can be sampled from to generate new data.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

16.3.1 Density kernels

Before explaining KDE, we must define what we mean by a "kernel". This term has several different meanings in machine learning and statistics.[1] In this section, we use a specific kind of kernel which we refer to as a **density kernel**. This is a function $\mathcal{K} : \mathbb{R} \to \mathbb{R}_+$ such that $\int \mathcal{K}(x)dx = 1$ and $\mathcal{K}(-x) = \mathcal{K}(x)$. This latter symmetry property implies the $\int x\mathcal{K}(x)dx = 0$, and hence

$$\int x\mathcal{K}(x - x_n)dx = x_n \tag{16.25}$$

A simple example of such a kernel is the **boxcar kernel**, which is the uniform distribution within the unit interval around the origin:

$$\mathcal{K}(x) \triangleq 0.5\mathbb{I}\left(|x| \leq 1\right) \tag{16.26}$$

Another example is the **Gaussian kernel**:

$$\mathcal{K}(x) = \frac{1}{(2\pi)^{\frac{1}{2}}} e^{-x^2/2} \tag{16.27}$$

We can control the width of the kernel by introducing a **bandwidth** parameter h:

$$\mathcal{K}_h(x) \triangleq \frac{1}{h}\mathcal{K}(\frac{x}{h}) \tag{16.28}$$

We can generalize to vector valued inputs by defining a **radial basis function** or **RBF** kernel:

$$\mathcal{K}_h(\mathbf{x}) \propto \mathcal{K}_h(||\mathbf{x}||) \tag{16.29}$$

In the case of the Gaussian kernel, this becomes

$$\mathcal{K}_h(\mathbf{x}) = \frac{1}{h^D(2\pi)^{D/2}} \prod_{d=1}^{D} \exp(-\frac{1}{2h^2}x_d^2) \tag{16.30}$$

Although Gaussian kernels are popular, they have unbounded support. Some alternative kernels, which have compact support (which can be computationally faster), are listed in Table 16.1. See Fig. 16.8 for a plot of these kernel functions.

16.3.2 Parzen window density estimator

To explain how to use kernels to define a nonparametric density estimate, recall the form of the Gaussian mixture model from Sec. 3.4.1. If we assume a fixed spherical Gaussian covariance and uniform mixture weights, we get

$$p(\mathbf{x}|\boldsymbol{\theta}) = \frac{1}{K}\sum_{k=1}^{K} \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \sigma^2\mathbf{I}) \tag{16.31}$$

1. For a good blog post on this, see https://francisbach.com/cursed-kernels/.

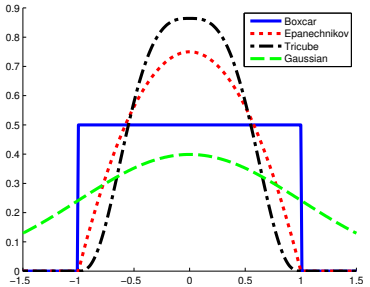


Figure 16.8: A comparison of some popular normalized kernels. Generated by code at figures.probml.ai/book1/16.8.

Name	Definition	Compact	Smooth	Boundaries
Gaussian	$\mathcal{K}(x) = (2\pi)^{-\frac{1}{2}} e^{-x^2/2}$	0	1	1
Boxcar	$\mathcal{K}(x) = \frac{1}{2}\mathbb{I}(\lvert x\rvert \leq 1)$	1	0	0
Epanechnikov kernel	$\mathcal{K}(x) = \frac{3}{4}(1-x^2)\mathbb{I}(\lvert x\rvert \leq 1)$	1	1	0
Tri-cube kernel	$\mathcal{K}(x) = \frac{70}{81}(1-\lvert x\rvert^3)^3\mathbb{I}(\lvert x\rvert \leq 1)$	1	1	1

Table 16.1: List of some popular normalized kernels in 1d. Compact=1 means the function is non-zero for a finite range of inputs. Smooth=1 means the function is differentiable over the range of its support. Boundaries=1 means the function is also differentiable at the boundaries of its support.

One problem with this model is that it requires specifying the number K of clusters, as well as their locations $\boldsymbol{\mu}_k$. An alternative to estimating these parameters is to allocate one cluster center per data point. In this case, the model becomes

$$p(\mathbf{x}|\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{N}(\mathbf{x}|\mathbf{x}_n, \sigma^2\mathbf{I}) \tag{16.32}$$

We can generalize Eq. (16.32) by writing

$$p(\mathbf{x}|\mathcal{D}) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \tag{16.33}$$

where $\mathcal{K}_h$ is a density kernel. This is called a **Parzen window density estimator**, or **kernel density estimator** (**KDE**).

The advantage over a parametric model is that no model fitting is required (except for choosing h, discussed in Sec. 16.3.3), and there is no need to pick the number of cluster centers. The disadvantage is that the model takes a lot of memory (you need to store all the data) and a lot of time to evaluate.

Fig. 16.9 illustrates KDE in 1d for two kinds of kernel. On the top, we use a boxcar kernel; the resulting model just counts how many data points land within an interval of size h around each x_n to get a piecewise constant density. On the bottom, we use a Gaussian kernel, which results in a smoother density.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

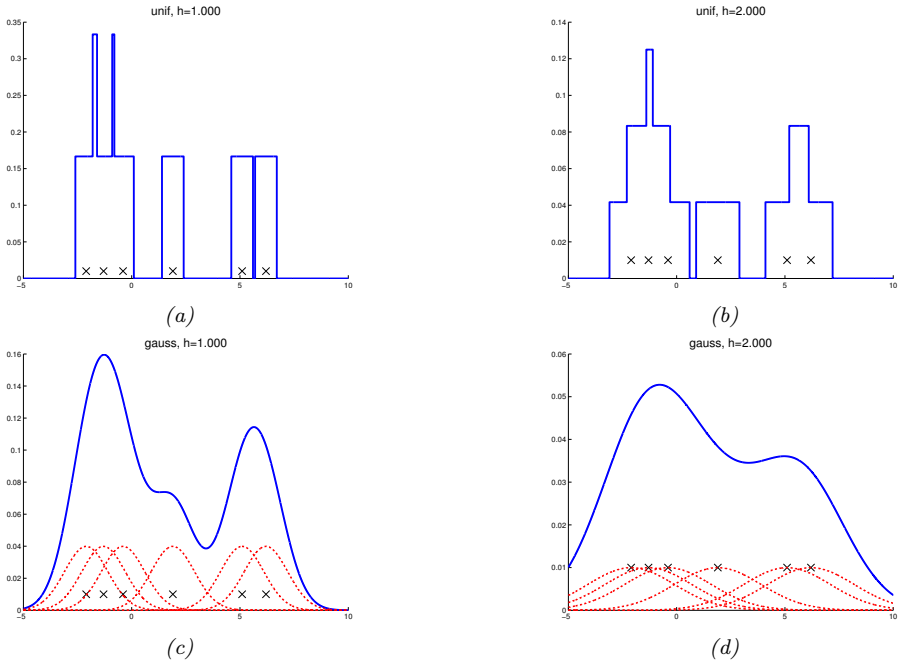


Figure 16.9: A nonparametric (Parzen) density estimator in 1d estimated from 6 data points, denoted by x. Top row: uniform kernel. Bottom row: Gaussian kernel. Left column: bandwidth parameter $h = 1$. Right column: bandwidth parameter $h = 2$. Adapted from http://en.wikipedia.org/wiki/Kernel_density_estimation. *Generated by code at figures.probml.ai/book1/16.9.*

16.3.3 How to choose the bandwidth parameter

We see from Fig. 16.9 that the bandwidth parameter h has a large effect on the learned distribution. We can view this as controlling the complexity of the model.

In the case of 1d data, where the "true" data generating distribution is assumed to be a Gaussian, one can show [BA97a] that the optimal bandwidth for a Gaussian kernel (from the point of view of minimizing frequentist risk) is given by $h = \sigma \left(\frac{4}{3N}\right)^{1/5}$. We can compute a robust approximation to the standard deviation by first computing the **median absolute deviation**, $\text{median}(|\mathbf{x} - \text{median}(\mathbf{x})|)$, and then using $\hat{\sigma} = 1.4826$ MAD. If we have D dimensions, we can estimate h_d separately for each dimension, and then set $h = (\prod_{d=1}^{D} h_d)^{1/D}$.

16.3.4 From KDE to KNN classification

In Sec. 16.1, we discussed the K nearest neighbor classifier as a heuristic approach to classification. Interestingly, we can derive it as a generative classifier in which the class conditional densities $p(\mathbf{x}|y = c)$ are modeled using KDE. Rather than using a fixed bandwidth and counting how many data points fall within the hyper-cube centered on a datapoint, we will we allow the bandwidth or volume to be different for each data point. Specifically, we will "grow" a volume around $\mathbf{x}$ until we

encounter K data points, regardless of their class label. This is called a **balloon kernel density estimator** [TS92]. Let the resulting volume have size $V(\mathbf{x})$ (this was previously h^D), and let there be $N_c(\mathbf{x})$ examples from class c in this volume. Then we can estimate the class conditional density as follows:

$$p(\mathbf{x}|y=c,\mathcal{D}) = \frac{N_c(\mathbf{x})}{N_c V(\mathbf{x})} \tag{16.34}$$

where N_c is the total number of examples in class c in the whole data set. If we take the class prior to be $p(y=c) = N_c/N$, then the class posterior is given by

$$p(y=c|\mathbf{x},\mathcal{D}) = \frac{\frac{N_c(\mathbf{x})}{N_c V(\mathbf{x})}\frac{N_c}{N}}{\sum_{c'}\frac{N_{c'}(\mathbf{x})}{N_{c'}V(\mathbf{x})}\frac{N_{c'}}{N}} = \frac{N_c(\mathbf{x})}{\sum_{c'}N_{c'}(\mathbf{x})} = \frac{N_c(\mathbf{x})}{K} = \frac{1}{K}\sum_{n\in N_K(\mathbf{x},\mathcal{D})}\mathbb{I}(y_n=c) \tag{16.35}$$

where we used the fact that $\sum_c N_c(\mathbf{x}) = K$, since we choose a total of K points (regardless of class) around every point. This matches 16.1.

16.3.5 Kernel regression

Just as KDE can be used for generative classifiers (see Sec. 16.1), it can also be used for generative models for regression, as we discuss below.

16.3.5.1 Nadaraya-Watson estimator for the mean

In regression, our goal is to compute the conditional expectation

$$\mathbb{E}[y|\mathbf{x},\mathcal{D}] = \int y\, p(y|\mathbf{x},\mathcal{D})dy = \frac{\int y\, p(\mathbf{x},y|\mathcal{D})dy}{\int p(\mathbf{x},y|\mathcal{D})dy} \tag{16.36}$$

If we use an MVN for $p(y,\mathbf{x}|\mathcal{D})$, we derive a result which is equivalent to linear regression, as we showed in Sec. 11.2.3.5. However, the assumption that $p(y,\mathbf{x}|\mathcal{D})$ is Gaussian is rather limiting. We can use KDE to more accurately approximate the joint density $p(\mathbf{x},y|\mathcal{D})$ as follows:

$$p(y,\mathbf{x}|\mathcal{D}) \approx \frac{1}{N}\sum_{n=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)\mathcal{K}_h(y-y_n) \tag{16.37}$$

Hence

$$\mathbb{E}[y|\mathbf{x},\mathcal{D}] = \frac{\frac{1}{N}\sum_{n=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)\int y\mathcal{K}_h(y-y_n)dy}{\frac{1}{N}\sum_{n=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)\int \mathcal{K}_h(y-y_n)dy} \tag{16.38}$$

We can simplify the numerator using the fact that $\int y\mathcal{K}_h(y-y_n)dy = y_n$ (from Eq. (16.25)). We can simplify the denominator using the fact that density kernels integrate to one, i.e., $\int \mathcal{K}_h(y-y_n)dy = 1$. Thus

$$\mathbb{E}[y|\mathbf{x},\mathcal{D}] = \frac{\sum_{n=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)y_n}{\sum_{n=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)} = \sum_{n=1}^{N}y_n w_n(\mathbf{x}) \tag{16.39}$$

$$w_n(\mathbf{x}) \triangleq \frac{\mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)}{\sum_{n'=1}^{N}\mathcal{K}_h(\mathbf{x}-\mathbf{x}_{n'})} \tag{16.40}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

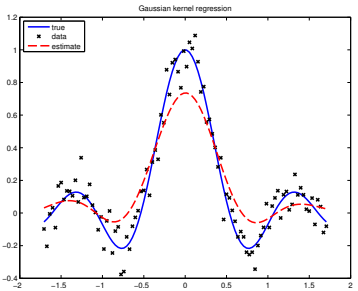


Figure 16.10: An example of kernel regression in 1d using a Gaussian kernel. Generated by code at figures.probml.ai/book1/16.10.

We see that the prediction is just a weighted sum of the outputs at the training points, where the weights depend on how similar $\mathbf{x}$ is to the stored training points. This method is called **kernel regression**, **kernel smoothing**, or the **Nadaraya-Watson** (N-W) model. See Fig. 16.10 for an example, where we use a Gaussian kernel.

In Sec. 17.3.3, we discuss the connection between kernel regression and Gaussian process regression.

16.3.5.2 Estimator for the variance

Sometimes it is useful to compute the predictive variance, as well as the predictive mean. We can do this by noting that

$$\mathbb{V}\left[y|\mathbf{x},\mathcal{D}\right] = \mathbb{E}\left[y^2|\mathbf{x},\mathcal{D}\right] - \mu(\mathbf{x})^2 \tag{16.41}$$

where $\mu(\mathbf{x}) = \mathbb{E}\left[y|\mathbf{x},\mathcal{D}\right]$ is the N-W estimate. If we use a Gaussian kernel with variance σ^2, we can compute $\mathbb{E}\left[y^2|\mathbf{x},\mathcal{D}\right]$ as follows:

$$\mathbb{E}\left[y^2|\mathbf{x},\mathcal{D}\right] = \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)\int y^2\mathcal{K}_h(y-y_n)dy}{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)\int \mathcal{K}_h(y-y_n)dy} \tag{16.42}$$

$$= \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)(\sigma^2+y_n^2)}{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x}-\mathbf{x}_n)} \tag{16.43}$$

where we used the fact that

$$\int y^2\mathcal{N}(y|y_n,\sigma^2)dy = \sigma^2 + y_n^2 \tag{16.44}$$

Combining Eq. (16.43) with Eq. (16.41) gives

$$\mathbb{V}\left[y|\mathbf{x},\mathcal{D}\right] = \sigma^2 + \sum_{n=1}^N w_n(\mathbf{x})y_n^2 - \mu(\mathbf{x})^2 \tag{16.45}$$

This matches Eqn. 8 of [BA10] (modulo the initial σ^2 term).

16.3.5.3 Locally weighted regression

We can drop the normalization term from Eq. (16.39) to get

$$\mu(\mathbf{x}) = \sum_{n=1}^{N} y_n \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \tag{16.46}$$

This is just a weighted sum of the observed responses, where the weights depend on how similar the test input $\mathbf{x}$ is to the training points $\mathbf{x}_n$.

Rather than just interpolating the stored responses y_n, we can fit a locally linear model around each training point:

$$\mu(\mathbf{x}) = \min_{\boldsymbol{\beta}} \sum_{n=1}^{N} [y_n - \boldsymbol{\beta}^\top \boldsymbol{\phi}(\mathbf{x}_n)]^2 \, \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \tag{16.47}$$

where $\boldsymbol{\phi}(\mathbf{x}) = [1, \mathbf{x}]$. This is called **locally linear regression** (LRR) or **locally-weighted scatterplot smoothing**, and is commonly known by the acronym **LOWESS** or **LOESS** [CD88]. This is often used when annotating scatter plots with local trend lines.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17 Kernel methods

In this chapter, we consider **nonparametric methods** for regression and classification. Such methods do not assume a fixed parametric form for the prediction function, but instead try to estimate the function itself (rather than the parameters) directly from data. The key idea is that we observe the function value at a fixed set of N points, namely $y_n = f(\mathbf{x}_n)$ for $n = 1 : N$, where f is the unknown function, so to predict the function value at a new point, say $\mathbf{x}_*$, we just have to compare how "similar" $\mathbf{x}_*$ is to each of the N training points, $\{\mathbf{x}_n\}$, and then we can predict that $f(\mathbf{x}_*)$ is some weighted combination of the $\{f(\mathbf{x}_n)\}$ values. Thus we may need to "remember" the entire training set, $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$, in order to make predictions at test time — we cannot "compress" $\mathcal{D}$ into a fixed-sized parameter vector.

The weights that are used for prediction are determined by the similarity between $\mathbf{x}_*$ and each $\mathbf{x}_n$, which is computed using a special kind of function known as kernel function, $\mathcal{K}(\mathbf{x}_n, \mathbf{x}_*) \geq 0$, which we explain in Sec. 17.2. This approach is similar to RBF networks (Sec. 13.6.1), except we use the datapoints themselves as centroids.

In Sec. 17.3, we discuss a Bayesian approach called Gaussian processes, which allows us to use the kernel to define a prior and posterior over functions. Alternatively we can use this kernel to define a point estimate of the function using a method called Support Vector Machines, which we explain in Sec. 17.5. But first, in Sec. 17.1, we consider a simpler setting in which the inputs are points on the real line, and the similarity is just Euclidean distance.

17.1 Inferring functions from data

Suppose we want to estimate a 1d function, defined on the real line, $f : \mathbb{R} \to \mathbb{R}$. For simplicity, we assume the domain of the function is $[0, 1]$, and, following [CS07, p135], we will discretize this into a set of D equally spaced points, $\mathcal{X} = \{x_1, \ldots, x_D\}$. Let $f_j = f(x_j)$ be the unknown function value at input $x_j \in \mathcal{X}$. We assume that we observe the function value at a set of N points, $y_n = f(x_n)$, for $n \in \mathcal{V}$, where $\mathcal{V} \subseteq \mathcal{X}$ is the set of observed locations; let $\mathcal{D} = \{(x_n, y_n)\}$ be the observed data. The values $f_j = f(x_j)$ for $j \in \mathcal{H}$ are unknown or hidden, where $\mathcal{H} = \mathcal{X} \setminus \mathcal{V}$ is the set of $D - N$ locations for which we do not have observed data.

Our goal is to infer the unkown function values at test locations $\mathcal{H}$ given the observed function values at the training locations $\mathcal{V}$, i.e. we want to compute $p(\mathbf{f}|\mathcal{D})$, where $\mathbf{f} = (f_1, \ldots, f_D)$. We discuss how to do this below. The key trick is that, by approximating the function by a list of function values on a grid of D points, we can perform inference with random vectors, instead of random functions. We generalize this to arbitrary input domains in Sec. 17.3.

17.1.1 Smoothness prior

Computing $p(f|\mathcal{D})$ is an ill-posed problem, since there are many functions that can fit the data. However, it is often reasonable to assume that the unknown function is smooth. In this section, following [CS07, p135], we approximate the function by a list of function values on a grid of D points, so we can perform inference with random vectors, instead of random functions. We generalize this to arbitrary input domains in Sec. 17.3.

We can encode our smoothness prior by assuming that f_j is an average of its neighbors, f_{j-1} and f_{j+1}, plus some Gaussian noise:

$$f_j = \frac{1}{2}(f_{j-1} + f_{j+1}) + \epsilon_j, \quad 2 \le j \le D-1 \tag{17.1}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, (1/\lambda)\mathbf{I})$. The precision term λ controls how much we think the function will vary: a large λ corresponds to a belief that the function is very smooth (differences are close to the mean value of 0), whereas a small λ corresponds to a belief that the function is quite "wiggly". In vector form, the above equation can be written as follows: $\mathbf{Lf} = \boldsymbol{\epsilon}$, where $\mathbf{L}$ is the $(D-2) \times D$ second order **finite difference matrix**

$$\mathbf{L} = \frac{1}{2}\begin{pmatrix} -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & & \ddots & & & \\ & & & -1 & 2 & -1 \end{pmatrix} \tag{17.2}$$

Since $\mathbf{Lf} = \boldsymbol{\epsilon}$, we have $\mathbf{f} = (\mathbf{L}^\top\mathbf{L})^{-1}\mathbf{L}^\top\boldsymbol{\epsilon}$. The mean of this is $\mathbb{E}[\mathbf{f}] = \mathbf{0}$, and the covariance is

$$\text{Cov}[\mathbf{f}] = \left[(\mathbf{L}^\top\mathbf{L})^{-1}\mathbf{L}^\top\right] \text{Cov}[\boldsymbol{\epsilon}] \left[\mathbf{L}(\mathbf{L}^\top\mathbf{L})^{-1}\right] = \lambda^{-1}(\mathbf{L}^\top\mathbf{L})^{-1} \tag{17.3}$$

and hence

$$p(\mathbf{f}) = \mathcal{N}(\mathbf{f}|\mathbf{0}, (\lambda\mathbf{L}^\top\mathbf{L})^{-1}) \propto \exp\left(-\frac{\lambda}{2}||\mathbf{Lx}||_2^2\right) \tag{17.4}$$

We will henceforth assume we have scaled $\mathbf{L}$ by $\sqrt{\lambda}$ so we can ignore the λ term, and just write $\boldsymbol{\Lambda} = \mathbf{L}^\top\mathbf{L}$ for the precision matrix.

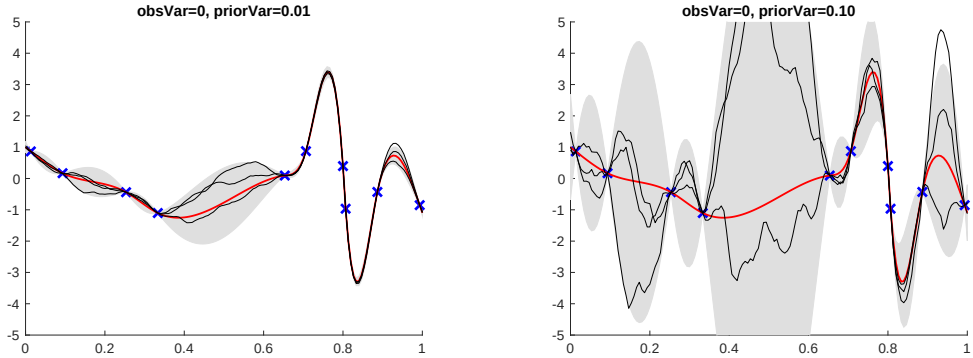
Note that although $\mathbf{f}$ is D-dimensional, the precision matrix $\boldsymbol{\Lambda}$ only has rank $D-2$. Thus this is an improper prior, known as an intrinsic Gaussian random field (see [RH05] for details). However, providing we observe $N \ge 2$ data points, the posterior will be proper.

17.1.2 Inference from noise-free observations

In this section, we assume that we have N noise-free observations of the random vector $\mathbf{f}$ at locations $x_1, \ldots, x_N$, so $f_n = y_n$ for $n \in \mathcal{V}$. In this case, we want our predictions for $f(x)$ to perfectly match the observations y at the observed x locations; in other words, we want to **interpolate** the data.

Let $\mathbf{f}_2$ be these N observed function values, and let $\mathbf{f}_1$ be the remaining $D-N$ unknown function values. Without loss of generality, assume that the unknown variables are ordered first, then the known variables. Then we can partition the $\mathbf{L}$ matrix as follows:

$$\mathbf{L} = [\mathbf{L}_1,\ \mathbf{L}_2],\ \mathbf{L}_1 \in \mathbb{R}^{(D-2)\times(D-N)},\ \mathbf{L}_2 \in \mathbb{R}^{(D-2)\times(N)} \tag{17.5}$$



(a) (b)

Figure 17.1: Interpolating a function from noise-free data using a smoothness prior with prior precision λ. Blue crosses are observations. Red line is posterior mean, thin black lines are posterior samples. Shaded gray area is the pointwise 95% marginal credible interval for $f(x_j)$, i.e., $\mu_j \pm 2\sqrt{\Sigma_{1|2,jj}}$. (a) $\lambda = 1$. (b) $\lambda = 0.1$. Adapted from Figure 7.1 of [CS07]. Generated by code at figures.probml.ai/book1/17.1.

We can also partition the precision matrix of the joint distribution:

$$\mathbf{\Lambda} = \mathbf{L}^\top \mathbf{L} = \begin{pmatrix} \mathbf{\Lambda}_{11} & \mathbf{\Lambda}_{12} \\ \mathbf{\Lambda}_{21} & \mathbf{\Lambda}_{22} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_1^\top \mathbf{L}_1 & \mathbf{L}_1^\top \mathbf{L}_2 \\ \mathbf{L}_2^\top \mathbf{L}_1 & \mathbf{L}_2^\top \mathbf{L}_2 \end{pmatrix} \tag{17.6}$$

Using Eq. (3.31), we can write the posterior distribution over the unknown function vaues as follows:

$$p(\mathbf{f}_1|\mathbf{f}_2) = p(\mathbf{f}_1|\mathbf{f}_2) = \mathcal{N}(\mathbf{f}_1|\boldsymbol{\mu}_{1|2}, \mathbf{\Sigma}_{1|2}) \tag{17.7}$$

$$\boldsymbol{\mu}_{1|2} = -\mathbf{\Lambda}_{11}^{-1}\mathbf{\Lambda}_{12}\mathbf{f}_2 = -\mathbf{L}_1^{-1}\mathbf{L}_2\mathbf{f}_2 \tag{17.8}$$

$$\mathbf{\Sigma}_{1|2} = \mathbf{\Lambda}_{11}^{-1} \tag{17.9}$$

Note that we can compute the mean by solving the following system of linear equations: $\mathbf{L}_1\boldsymbol{\mu}_{1|2} = -\mathbf{L}_2\mathbf{f}_2$. This is efficient since $\mathbf{L}_1$ is tridiagonal.

Fig. 17.1 gives an illustration of this process in action. We see that the posterior mean $\boldsymbol{\mu}_{1|2}$ equals the observed data at the specified points, and smoothly interpolates in between, as desired. When the variance of the prior is small, the posterior samples are smooth, and the uncertainty is low. When the variance of the prior is larger, the posterior samples are more wiggly, and the uncertainty becomes larger.

We can interpret this in terms of a spring system. Suppose we have $D-1$ equal length springs connected in a line, each with a "stiffness" of λ, connecting D equally spaced points. We "clamp" N of these points to observed values, and let the other $D-N$ "slide" freely up or down, so they exert no force on the springs they are connected to. The posterior mean of the function takes on a shape that corresponds to the lowest energy state of this spring system. Samples from the posterior corresond to perturbing the system by "shaking" it a little bit; this will cause the springs to vibrate or oscillate. Clearly the stiffer springs (with larger prior precision $\lambda 0$, and hence smaller prior variance) will vibrate less than the less stiff springs. This explains the difference in the shaded regions in Fig. 17.1.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17.1.3 Inference from noisy observations

We can generalize the above approach to compute $p(\mathbf{f}|\mathbf{y})$, where $y_n = f_n + \epsilon_n$ is a noisy observation of the function value $f_n = f(\mathbf{x}_n)$. However, the algebra is a bit messier. We therefore defer discussion of this to the general case in Sec. 17.3.2.

17.2 Mercer kernels

In Sec. 17.1, we saw how we could encode prior knowledge about the smoothness of a function by specifying that $f(x_i)$ and $f(x_j)$ should be similar if x_i and x_j were nearest neighbors on the 1d input grid. To generalize this to arbitrary input domains, we introduce the notion of a **kernel function**.

The word "kernel" has many different meanings in mathematics, including density kernels (Sec. 16.3.1), transition kernels of a Markov chain, (Sec. 3.5.1.2), and convolutional kernels (Sec. 14.1). Here we consider a **Mercer kernel**, also called a **positive definite kernel**. This is any symmetric function $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^+$ such that

$$\sum_{i=1}^{N}\sum_{j=1}^{N} \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)c_i c_j \geq 0 \tag{17.10}$$

for any set of N (unique) points $\mathbf{x}_i \in \mathcal{X}$, and any choice of numbers $c_i \in \mathbb{R}$. (We assume $\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) > 0$, so that we can only achieve equality in the above equation if $c_i = 0$ for all i.)

Another way to understand this condition is the following. Given a set of N datapoints, let us define the **Gram matrix** as the following $N \times N$ similarity matrix:

$$\mathbf{K} = \begin{pmatrix} \mathcal{K}(\mathbf{x}_1, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_1, \mathbf{x}_N) \\ & \vdots & \\ \mathcal{K}(\mathbf{x}_N, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix} \tag{17.11}$$

We say that $\mathcal{K}$ is a Mercer kernel iff the Gram matrix is positive definite for any set of (distinct) inputs $\{\mathbf{x}_i\}_{i=1}^N$.

The most widely used kernel for real-valued inputs is the **squared exponential kernel** (SE kernel), also called the **exponentiated quadratic**, **Gaussian kernel RBF kernel**. It is defined by

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{||\mathbf{x} - \mathbf{x}'||}{2\ell^2}\right) \tag{17.12}$$

Here ℓ corresponds to the length scale of the kernel, i.e., the distance over which we expect differences to matter. This is known as the **bandwidth** parameter. The RBF kernel measures similarity between two vectors in $\mathbb{R}^D$ using (scaled) Euclidean distance. In Sec. 17.2.2, we will discuss several other kinds of kenrel.

In Sec. 17.3, we show how to use kernels to define priors and posteriors over functions. The basic idea is this: if $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ is large, meaning the inputs are similar, then we expect the output of the function to be similar as well, so $f(\mathbf{x}) \approx f(\mathbf{x}')$. More precisely, information we learn about $f(\mathbf{x})$ will help us predict $f(\mathbf{x}')$ for all $\mathbf{x}'$ which are correlated with $\mathbf{x}$, and hence for which $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ is large.

17.2.1 Mercer's theorem

Recall from Sec. 7.4 that any positive definite matrix $\mathbf{K}$ can be represented using an eigendecomposition of the form $\mathbf{K} = \mathbf{U}^\top \mathbf{\Lambda} \mathbf{U}$, where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues $\lambda_i > 0$, and $\mathbf{U}$ is a matrix containing the eigenvectors. Now consider element (i, j) of $\mathbf{K}$:

$$k_{ij} = (\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i})^\top (\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:j}) \tag{17.13}$$

where $\mathbf{U}_{:i}$ is the i'th column of $\mathbf{U}$. If we define $\boldsymbol{\phi}(\mathbf{x}_i) = \mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i}$, then we can write

$$k_{ij} = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j) = \sum_m \phi_m(\mathbf{x}_i) \phi_m(\mathbf{x}_j) \tag{17.14}$$

Thus we see that the entries in the kernel matrix can be computed by performing an inner product of some feature vectors that are implicitly defined by the eigenvectors of the kernel matrix. This idea can be generalized to apply to kernel functions, not just kernel matrices; this result is known as **Mercer's theorem**.

For example, consider the **quadratic kernel** $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle^2$. In 2d, we have

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = (x_1 x_1' + x_2 x_2')^2 = x_1^2 (x_1')^2 + 2 x_1 x_2 x_1' x_2' + x_2^2 (x_2')^2 \tag{17.15}$$

We can write this as $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x})$ if we define $\boldsymbol{\phi}(x_1, x_2) = [x_1^2, \sqrt{2} x_1 x_2, x_2^2] \in \mathbb{R}^3$. So we embed the 2d inputs $\mathbf{x}$ into a 3d feature space $\boldsymbol{\phi}(\mathbf{x})$.

Now consider the RBF kernel. In this case, the corresponding feature representation is infinite dimensional (see Sec. 17.4.3 for details). However, by working with kernel functions, we can avoid having to deal with infinite dimensional vectors.

17.2.2 Some popular Mercer kernels

In the sections below, we describe some popular Mercer kernels. More details can be found at [Wil14] and https://www.cs.toronto.edu/~duvenaud/cookbook/.

17.2.2.1 Stationary kernels for real-valued vectors

For real-valued inputs, $\mathcal{X} = \mathbb{R}^D$, it is common to use **stationary kernels**, which are functions of the form $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}(||\mathbf{x} - \mathbf{x}'||)$; thus the value only depends on the elementwise difference between the inputs. The RBF kernel is a stationary kernel. We give some other examples below.

ARD kernel

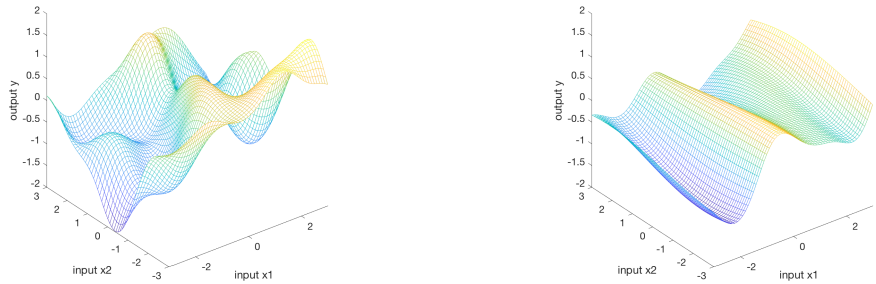
We can generalize the RBF kernel by replacing Euclidean distance with Mahalanobis distance, as follows:

$$\mathcal{K}(\mathbf{r}) = \sigma^2 \exp\left(-\frac{1}{2} \mathbf{r}^\top \mathbf{\Sigma}^{-1} \mathbf{r}\right) \tag{17.16}$$

where $\mathbf{r} = \mathbf{x} - \mathbf{x}'$. If $\mathbf{\Sigma}$ is diagonal, this can be written as

$$\mathcal{K}(\mathbf{r}; \boldsymbol{\ell}, \sigma^2) = \sigma^2 \exp\left(-\frac{1}{2} \sum_{d=1}^D \frac{1}{\ell_d^2} r_d^2\right) = \prod_{d=1}^D \mathcal{K}(r_d; \ell_d, \sigma^{2/d}) \tag{17.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.2: Function samples from a GP with an ARD kernel. (a) $\ell_1 = \ell_2 = 1$. Both dimensions contribute to the response. (b) $\ell_1 = 1$, $\ell_2 = 5$. The second dimension is essentially ignored. Adapted from Figure 5.1 of [RW06]. Generated by code at figures.probml.ai/book1/17.2.

where

$$\mathcal{K}(r; \ell, \tau^2) = \tau^2 \exp\left(-\frac{1}{2}\frac{1}{\ell^2}r^2\right) \tag{17.18}$$

We can interpret σ^2 as the overall variance, and ℓ_d as defining the **characteristic length scale** of dimension d. If d is an irrelevant input dimensions, we can set $\ell_d = \infty$, so the corresponding dimension will be ignored. This is known as **automatic relevancy determination** or **ARD** (Sec. 11.6.3). Hence the corresponding kernel is called the **ARD kernel**. See Fig. 17.2 for an illustration of some 2d functions sampled from a GP using this prior.

Matern kernels

The SE kernel gives rise to functions that are infinitely differentiable, and therefore are very smooth. For many applications, it is better to use the **Matern kernel**, which gives rise to "rougher" functions, which can better model local "wiggles" without having to make the overall length scale very small.

The Matern kernel has the following form:

$$\mathcal{K}(r; \nu, \ell) = \frac{2^{1-\nu}}{\Gamma(\nu)}\left(\frac{\sqrt{2\nu}r}{\ell}\right)^{\nu} K_\nu\left(\frac{\sqrt{2\nu}r}{\ell}\right) \tag{17.19}$$

where K_ν is a modified Bessel function and ℓ is the length scale. Functions sampled from this GP are k-times differentiable iff $\nu > k$. As $\nu \to \infty$, this approaches the SE kernel.

(a) (b)

Figure 17.3: Functions sampled from a GP with a Matern kernel. (a) $\nu = 5/2$. (b) $\nu = 1/2$. Generated by code at figures.probml.ai/book1/17.3.

(a) Periodic kernel. (b) Cosine kernel.

Figure 17.4: Functions sampled from a GP using various stationary periodic kernels. Generated by code at figures.probml.ai/book1/17.4.

For values $\nu \in \{\frac{1}{2}, \frac{3}{2}, \frac{5}{2}\}$, the function simplifies as follows:

$$\mathcal{K}(r; \frac{1}{2}, \ell) = \exp(-\frac{r}{\ell}) \tag{17.20}$$

$$\mathcal{K}(r; \frac{3}{2}, \ell) = \left(1 + \frac{\sqrt{3}r}{\ell}\right) \exp\left(-\frac{\sqrt{3}r}{\ell}\right) \tag{17.21}$$

$$\mathcal{K}(r; \frac{5}{2}, \ell) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right) \tag{17.22}$$

The value $\nu = \frac{1}{2}$ corresponds to the **Ornstein-Uhlenbeck process**, which describes the velocity of a particle undergoing Brownian motion. The corresponding function is continuous but not differentiable, and hence is very "jagged". See Fig. 17.3b for an illustration.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Periodic kernels

The **periodic kernel** captures repeating structure, and has the form

$$\mathcal{K}_{\text{per}}(r;\ell,p) = \exp\left(-\frac{2}{\ell^2}\sin^2(\pi\frac{r}{p})\right) \tag{17.23}$$

where p is the period. See Fig. 17.4a for an illustration.

In the limit that $\ell \to \infty$, we get the **cosine kernel**:

$$\mathcal{K}(r;p) = \cos\left(2\pi\frac{r}{p}\right) \tag{17.24}$$

See Fig. 17.4b for an illustration.

17.2.2.2 Making new kernels from old

Given two valid kernels $\mathcal{K}_1(\mathbf{x},\mathbf{x}')$ and $\mathcal{K}_2(\mathbf{x},\mathbf{x}')$, we can create a new kernel using any of the following methods:

$$\mathcal{K}(\mathbf{x},\mathbf{x}') = c\mathcal{K}_1(\mathbf{x},\mathbf{x}'), \text{ for any constant } c > 0 \tag{17.25}$$
$$\mathcal{K}(\mathbf{x},\mathbf{x}') = f(\mathbf{x})\mathcal{K}_1(\mathbf{x},\mathbf{x}')f(\mathbf{x}'), \text{ for any function } f \tag{17.26}$$
$$\mathcal{K}(\mathbf{x},\mathbf{x}') = q(\mathcal{K}_1(\mathbf{x},\mathbf{x}')) \text{ for any function polynomial } q \text{ with nonneg. coef.} \tag{17.27}$$
$$\mathcal{K}(\mathbf{x},\mathbf{x}') = \exp(\mathcal{K}_1(\mathbf{x},\mathbf{x}')) \tag{17.28}$$
$$\mathcal{K}(\mathbf{x},\mathbf{x}') = \mathbf{x}^\top\mathbf{A}\mathbf{x}', \text{ for any psd matrix } \mathbf{A} \tag{17.29}$$

For example, suppose we start with the linear kernel $\mathcal{K}(\mathbf{x},\mathbf{x}') = \mathbf{x}\mathbf{x}'$. We know this is a valid Mercer kernel, since the corresponding Gram matrix is just the (scaled) covariance matrix of the data. From the above rules, we can see that the polynomial kernel $\mathcal{K}(\mathbf{x},\mathbf{x}') = (\mathbf{x}^\top\mathbf{x}')^M$ is a valid Mercer kernel. This contains all monomials of order M. For example, if $M = 2$ and the inputs are 2d, we have

$$(\mathbf{x}^\top\mathbf{x}')^2 = (x_1x_1' + x_2x_2')^2 = (x_1x_1')^2 + (x_2x_2)^2 + (x_1x_1')(x_2x_2') \tag{17.30}$$

We can generalize this to contain all terms up to degree M by using the kernel $\mathcal{K}(\mathbf{x},\mathbf{x}') = (\mathbf{x}^\top\mathbf{x}' + c)^M$. For example, if $M = 2$ and the inputs are 2d, we have

$$\begin{aligned}(\mathbf{x}^\top\mathbf{x}' + 1)^2 = {}& (x_1x_1')^2 + (x_1x_1')(x_2x_2') + (x_1x_1') \\ &+ (x_2x_2)(x_1x_1') + (x_2x_2')^2 + (x_2x_2') \\ &+ (x_1x_1') + (x_2x_2') + 1\end{aligned} \tag{17.31}$$

We can also use the above rules to establish that the Gaussian kernel is a valid kernel. To see this, note that

$$||\mathbf{x} - \mathbf{x}'||^2 = \mathbf{x}^\top\mathbf{x} + (\mathbf{x}')^\top\mathbf{x}' - 2\mathbf{x}^\top\mathbf{x}' \tag{17.32}$$

and hence

$$\mathcal{K}(\mathbf{x},\mathbf{x}') = \exp(-||\mathbf{x} - \mathbf{x}'||^2/2\sigma^2) = \exp(-\mathbf{x}^\top\mathbf{x}/2\sigma^2)\exp(\mathbf{x}^\top\mathbf{x}'/\sigma^2)\exp(-(\mathbf{x}')^\top\mathbf{x}'/2\sigma^2) \tag{17.33}$$

is a valid kernel.

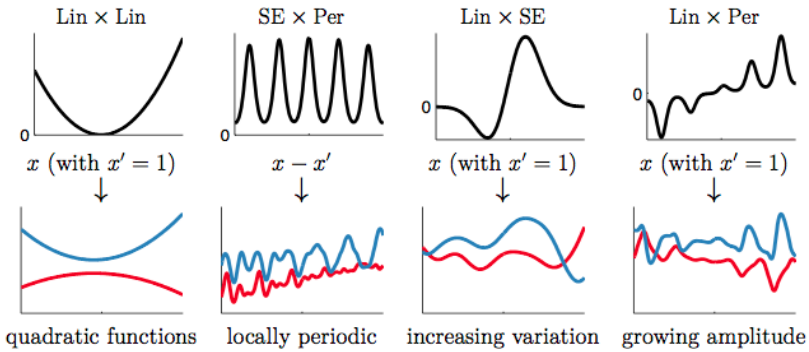


Figure 17.5: Examples of 1d structures obtained by multiplying elementary kernels. Top row shows $\mathcal{K}(x, x' = 1)$. Bottom row shows some functions sampled from $GP(f|0, \mathcal{K})$. From Figure 2.2 of [Duv14]. Used with kind permission of David Duvenaud.

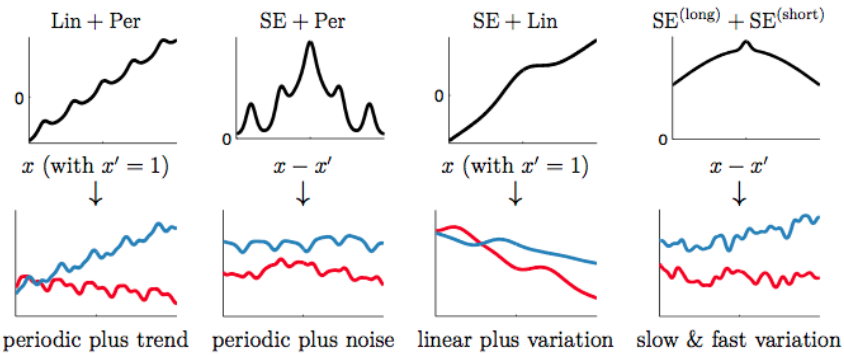


Figure 17.6: Examples of 1d structures obtained by adding elementary kernels. Here $\text{SE}^{(\text{short})}$ and $\text{SE}^{(\text{long})}$ are two SE kernels with different length scales. From Figure 2.4 of [Duv14]. Used with kind permission of David Duvenaud.

17.2.2.3 Combining kernels by addition and multiplication

We can also combine kernels using addition or multiplication:

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') + \mathcal{K}_2(\mathbf{x}, \mathbf{x}') \tag{17.34}$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') \times \mathcal{K}_2(\mathbf{x}, \mathbf{x}') \tag{17.35}$$

Multiplying two positive-definite kernels together always results in another positive definite kernel. This is a way to get a conjunction of the individual properties of each kernel, as illustrated in Fig. 17.5.

In addition, adding two positive-definite kernels together always results in another positive definite kernel. This is a way to get a disjunction of the individual properties of each kernel, as illustrated in Fig. 17.6.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17.2.2.4 Kernels for structured inputs

Kernels are particularly useful when the inputs are structured objects, such as strings and graphs, since it is often hard to "featurize" variable-sized inputs For example, we can define a **string kernel** which compares strings in terms of the number of n-grams they have in common [Lod+02; BC17].

We can also define kernels on graphs [KJM19]. For example, the **random walk kernel** conceptually performs random walks on two graphs simultaneously, and then counts the number of paths that were produced by both walks. This can be computed efficiently as discussed in [Vis+10]. For more details on graph kernels, see [KJM19].

For a review of kernels on structured objects, see e.g., [Gär03].

17.3 Gaussian processes

In this section, we discuss **Gaussian processes**, which is a way to define distributions over functions of the form $f : \mathcal{X} \to \mathbb{R}$, where $\mathcal{X}$ is any domain. The key assumption is that the function values at a set of $M > 0$ inputs, $\mathbf{f} = [f(\mathbf{x}_1), \ldots, f(\mathbf{x}_M)]$, is jointly Gaussian, with mean $(\boldsymbol{\mu} = m(\mathbf{x}_1), \ldots, m(\mathbf{x}_M))$ and covariance $\boldsymbol{\Sigma}_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$, where m is a mean function and $\mathcal{K}$ is a positive definite (Mercer) kernel. Since we assume this holds for any $M > 0$, this includes the case where $M = N+1$, containing N training points $\mathbf{x}_n$ and 1 test point $\mathbf{x}_*$. Thus we can infer $f(\mathbf{x}_*)$ from knowledge of $f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)$ by manipulating the joint Gaussian distribution $p(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_N), f(\mathbf{x}_*))$, as we explain below. This generalizes the approach from Sec. 17.1 to work with an arbitrary number of points from an arbitrary input domain. We can also extend this to work with the case where we observe noisy functions of $f(\mathbf{x}_n)$, such as in regression or classification problems.

17.3.1 Noise-free observations

Suppose we observe a training set $\mathcal{D} = \{(\mathbf{x}_n, y_n) : n = 1 : N\}$, where $y_n = f(\mathbf{x}_n)$ is the noise-free observation of the function evaluated at $\mathbf{x}_n$. If we ask the GP to predict $f(\mathbf{x})$ for a value of $\mathbf{x}$ that it has already seen, we want the GP to return the answer $f(\mathbf{x})$ with no uncertainty. In other words, it should act as an **interpolator** of the training data.

Now we consider the case of predicting the outputs for new inputs that may not be in $\mathcal{D}$. Specifically, given a test set $\mathbf{X}_*$ of size $N_* \times D$, we want to predict the function outputs $\mathbf{f}_* = [f(\mathbf{x}_1), \ldots, f(\mathbf{x}_{N_*})]$. By definition of the GP, the joint distribution $p(\mathbf{f}_X, \mathbf{f}_* | \mathbf{X}, \mathbf{X}_*)$ has the following form

$$\begin{pmatrix} \mathbf{f}_X \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}_{X,X} & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\top & \mathbf{K}_{*,*} \end{pmatrix} \right) \tag{17.36}$$

where $\boldsymbol{\mu}_X = [m(\mathbf{x}_1), \ldots, m(\mathbf{x}_N)]$, $\boldsymbol{\mu}_* = [m(\mathbf{x}_1^*), \ldots, m(\mathbf{x}_{N_*}^*)]$, $\mathbf{K}_{X,X} = \mathcal{K}(\mathbf{X}, \mathbf{X})$ is $N \times N$, $\mathbf{K}_{X,*} = \mathcal{K}(\mathbf{X}, \mathbf{X}_*)$ is $N \times N_*$, and $\mathbf{K}_{*,*} = \mathcal{K}(\mathbf{X}_*, \mathbf{X}_*)$ is $N_* \times N_*$. See Fig. 17.7 for an illustration. By the standard rules for conditioning Gaussians (Sec. 3.2.3), the posterior has the following form

$$p(\mathbf{f}_* | \mathbf{X}_*, \mathcal{D}) = \mathcal{N}(\mathbf{f}_* | \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \tag{17.37}$$

$$\boldsymbol{\mu}_* = m(\mathbf{X}_*) + \mathbf{K}_{X,*}^\top \mathbf{K}_{X,X}^{-1} (\mathbf{f}_X - m(\mathbf{X})) \tag{17.38}$$

$$\boldsymbol{\Sigma}_* = \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^\top \mathbf{K}_{X,X}^{-1} \mathbf{K}_{X,*} \tag{17.39}$$

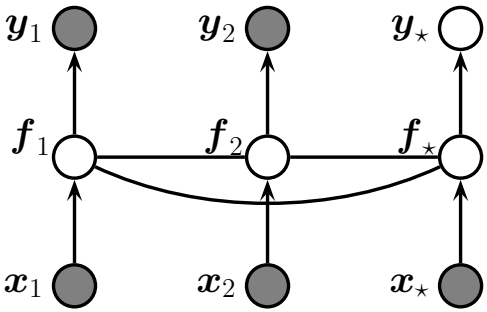
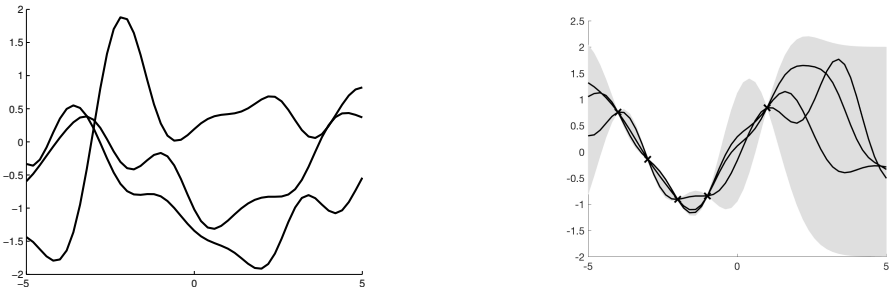


Figure 17.7: A Gaussian process for 2 training points, $\mathbf{x}_1$ and $\mathbf{x}_2$, and 1 testing point, $\mathbf{x}_*$, represented as a graphical model representing $p(\mathbf{y}, \mathbf{f}_X|\mathbf{X}) = \mathcal{N}(\mathbf{f}_X|m(\mathbf{X}), \mathcal{K}(\mathbf{X}))\prod_i p(y_i|f_i)$. The hidden nodes $f_i = f(\mathbf{x}_i)$ represent the value of the function at each of the data points. These hidden nodes are fully interconnected by undirected edges, forming a Gaussian graphical model; the edge strengths represent the covariance terms $\Sigma_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$. If the test point $\mathbf{x}_*$ is similar to the training points $\mathbf{x}_1$ and $\mathbf{x}_2$, then the value of the hidden function f_* will be similar to f_1 and f_2, and hence the predicted output y_* will be similar to the training values y_1 and y_2.



(a) *(b)*

Figure 17.8: Left: some functions sampled from a GP prior with squared exponential kernel. Right: some samples from a GP posterior, after conditioning on 5 noise-free observations. The shaded area represents $\mathbb{E}[f(\mathbf{x})] \pm 2\text{std}[f(\mathbf{x})]$. Adapted from Figure 2.2 of [RW06]. Generated by code at figures.probml.ai/book1/17.8.

This process is illustrated in Fig. 17.8. On the left we show some samples from the prior, $p(f)$, where we use an RBF kernel (Sec. 17.2) and a zero mean function. On the right, we show samples from the posterior, $p(f|\mathcal{D})$. We see that the model perfectly interpolates the training data, and that the predictive uncertainty increases as we move further away from the observed data.

17.3.2 Noisy observations

Now let us consider the case where what we observe is a noisy version of the underlying function, $y_n = f(\mathbf{x}_n) + \epsilon_n$, where $\epsilon_n \sim \mathcal{N}(0, \sigma_y^2)$. In this case, the model is not required to interpolate the data,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

but it must come "close" to the observed data. The covariance of the observed noisy responses is

$$\text{Cov}\left[y_i, y_j\right] = \text{Cov}\left[f_i, f_j\right] + \text{Cov}\left[\epsilon_i, \epsilon_j\right] = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) + \sigma_y^2 \delta_{ij} \tag{17.40}$$

where $\delta_{ij} = \mathbb{I}\left(i = j\right)$. In other words

$$\text{Cov}\left[\mathbf{y}|\mathbf{X}\right] = \mathbf{K}_{X,X} + \sigma_y^2 \mathbf{I}_N \triangleq \hat{\mathbf{K}}_{X,X} \tag{17.41}$$

The joint density of the observed data and the latent, noise-free function on the test points is given by

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \hat{\mathbf{K}}_{X,X} & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\top & \mathbf{K}_{*,*} \end{pmatrix} \right) \tag{17.42}$$

Hence the posterior predictive density at a set of test points $\mathbf{X}_*$ is

$$p(\mathbf{f}_*|\mathcal{D}, \mathbf{X}_*) = \mathcal{N}(\mathbf{f}_*|\boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \tag{17.43}$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\mu}_* + \mathbf{K}_{X,*}^\top \hat{\mathbf{K}}_{X,X}^{-1}(\mathbf{y} - \boldsymbol{\mu}_X) \tag{17.44}$$

$$\boldsymbol{\Sigma}_{*|X} = \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^\top \hat{\mathbf{K}}_{X,X}^{-1} \mathbf{K}_{X,*} \tag{17.45}$$

In the case of a single test input, this simplifies as follows

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_*|m_* + \mathbf{k}_*^\top \hat{\mathbf{K}}_{X,X}^{-1}(\mathbf{y} - \boldsymbol{\mu}_X),\ k_{**} - \mathbf{k}_*^\top \hat{\mathbf{K}}_{X,X}^{-1} \mathbf{k}_*) \tag{17.46}$$

where $\mathbf{k}_* = [\mathcal{K}(\mathbf{x}_*, \mathbf{x}_1), \ldots, \mathcal{K}(\mathbf{x}_*, \mathbf{x}_N)]$ and $k_{**} = \mathcal{K}(\mathbf{x}_*, \mathbf{x}_*)$. If the mean function is zero, we can write the posterior mean as follows:

$$\mu_{*|X} = \mathbf{k}_*^\top (\hat{\mathbf{K}}_{X,X}^{-1} \mathbf{y}) \triangleq \mathbf{k}_*^\top \boldsymbol{\alpha} = \sum_{n=1}^N \mathcal{K}(\mathbf{x}_*, \mathbf{x}_n) \alpha_n \tag{17.47}$$

This is identical to the predictions from kernel ridge regression in Eq. (17.117).

17.3.3 Comparison to kernel regression

In Sec. 16.3.5, we discussed kernel regression, which is a generative approach to regression in which we approximate $p(y, \mathbf{x})$ using kernel density estimation. In particular, Eq. (16.39) gives us

$$\mathbb{E}\left[y|\mathbf{x}, \mathcal{D}\right] = \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) y_n}{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n)} = \sum_{n=1}^N y_n w_n(\mathbf{x}) \tag{17.48}$$

$$w_n(\mathbf{x}) \triangleq \frac{\mathcal{K}_h(\mathbf{x} - \mathbf{x}_n)}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} \tag{17.49}$$

This is very similar to Eq. (17.47). However, there are a few important differences. Firstly, in a GP, we use a positive definite (Mercer) kernel instead of a density kernel; Mercer kernels can be defined on structured objects, such as strings and graphs, which is harder to do for density kernels.

Second, a GP is an interpolator (at least when $\sigma^2 = 0$), so $\mathbb{E}[y|\mathbf{x}_n, \mathcal{D}] = y_n$. By contrast, kernel regression is not an interpolator (although it can be made into one by iteratively fitting the residuals, as in [KJ16]). Third, a GP is a Bayesian method, which means we can estimate hyperparameters (of the kernel) by maximizing the marginal likelihood; by contrast, in kernel regression we must use cross-validation to estimate the kernel parameters, such as the bandwidth. Fourth, computing the weights w_n for kernel regression takes $O(N)$ time, where $N = |\mathcal{D}|$, whereas computing the weights α_n for GP regression takes $O(N^3)$ time (although there are approximation methods that can reduce this to $O(NM^2)$, as we discuss in Sec. 17.4).

17.3.4 Weight space vs function space

In this section, we show how Bayesian linear regression is a special case of a GP.

Consider the linear regression model $y = f(\mathbf{x}) + \epsilon$, where $f(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$ and $\epsilon \sim \mathcal{N}(0, \sigma_y^2)$. If we use a Gaussian prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \boldsymbol{\Sigma}_w)$, then the posterior is as follows (see Sec. 11.6.1 for the derivation):

$$p(\mathbf{w}|\mathcal{D}) = \mathcal{N}(\mathbf{w}|\frac{1}{\sigma_y^2}\mathbf{A}^{-1}\boldsymbol{\Phi}^T\mathbf{y}, \mathbf{A}^{-1}) \tag{17.50}$$

where $\boldsymbol{\Phi}$ is the $N \times D$ design matrix, and

$$\mathbf{A} = \sigma_y^{-2}\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \boldsymbol{\Sigma}_w^{-1} \tag{17.51}$$

The posterior predictive distribution for $f_* = f(\mathbf{x}_*)$ is therefore

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_*|\frac{1}{\sigma_y^2}\boldsymbol{\phi}_*^\top\mathbf{A}^{-1}\boldsymbol{\Phi}^\top\mathbf{y},\ \boldsymbol{\phi}_*^\top\mathbf{A}^{-1}\boldsymbol{\phi}_*) \tag{17.52}$$

where $\boldsymbol{\phi}_* = \boldsymbol{\phi}(\mathbf{x}_*)$. This views the problem of inference and prediction in **weight space**.

We now show that this is equivalent to the predictions made by a GP using a kernel of the form $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top\boldsymbol{\Sigma}_w\boldsymbol{\phi}(\mathbf{x}')$. To see this, let $\mathbf{K} = \boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\top$, $\mathbf{k}_* = \boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_*$, and $k_{**} = \boldsymbol{\phi}_*^\top\boldsymbol{\Sigma}_w\boldsymbol{\phi}_*$. Using this notation, and the matrix inversion lemma, we can rewrite Eq. (17.52) as follows

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_*|\boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \tag{17.53}$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\phi}_*^\top\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\top(\mathbf{K} + \sigma_y^2\mathbf{I})^{-1}\mathbf{y} = \mathbf{k}_*^\top\hat{\mathbf{K}}_{X,X}^{-1}\mathbf{y} \tag{17.54}$$

$$\boldsymbol{\Sigma}_{*|X} = \boldsymbol{\phi}_*^\top\boldsymbol{\Sigma}_w\boldsymbol{\phi}_* - \boldsymbol{\phi}_*^\top\boldsymbol{\Sigma}_w\boldsymbol{\Phi}^\top(\mathbf{K} + \sigma_y^2\mathbf{I})^{-1}\boldsymbol{\Phi}\boldsymbol{\Sigma}_w\boldsymbol{\phi}_* = k_{**} - \mathbf{k}_*^\top\hat{\mathbf{K}}_{X,X}^{-1}\mathbf{k}_* \tag{17.55}$$

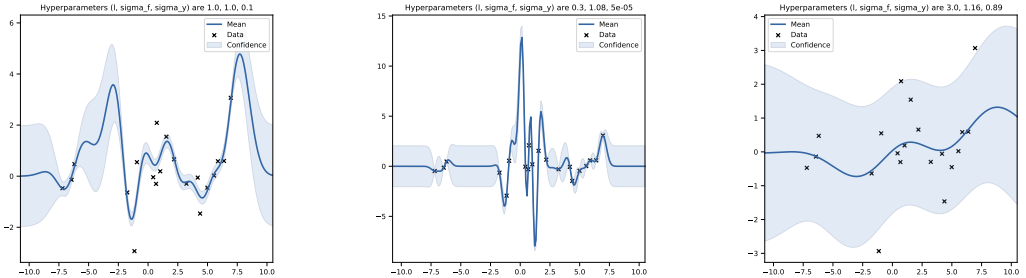
which matches the results in Eq. (17.46), assuming $m(\mathbf{x}) = 0$. (Non-zero mean can be captured by adding a constant feature with value 1 to $\boldsymbol{\phi}(\mathbf{x})$.)

Thus we can derive a GP from Bayesian linear regression. Note, however, that linear regression assumes $\boldsymbol{\phi}(\mathbf{x})$ is a finite length vector, whereas a GP allows us to work directly in terms of kernels, which may correspond to infinite length feature vectors (see Sec. 17.2.1). That is, a GP works in **function space**.

17.3.5 Numerical issues

In this section, we discuss computational and numerical issues which arise when implementing the above equations. For notational simplicity, we assume the prior mean is zero, $m(\mathbf{x}) = 0$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 17.9: Some 1d GPs with SE kernels but different hyper-parameters fit to 20 noisy observations. The hyper-parameters $(\ell, \sigma_f, \sigma_y)$ are as follows: (a) (1,1,0.1) (b) (0.3, 1.08, 0.00005), (c) (3.0, 1.16, 0.89). Adapted from Figure 2.5 of [RW06]. Generated by code at figures.probml.ai/book1/17.9.

The posterior predictive mean is given by $\mu_* = \mathbf{k}_*^\top \hat{\mathbf{K}}_{X,X}^{-1} \mathbf{y}$. For reasons of numerical stability, it is unwise to directly invert $\hat{\mathbf{K}}_{X,X}$. A more robust alternative is to compute a Cholesky decomposition, $\hat{\mathbf{K}}_{X,X} = \mathbf{L}\mathbf{L}^\top$, which takes $O(N^3)$ time. Then we compute $\boldsymbol{\alpha} = \mathbf{L}^\top \setminus (\mathbf{L} \setminus \mathbf{y})$, where we have used the backslash operator to represent backsubstitution (Sec. 7.7.1). Given this, we can compute the posterior mean for each test case in $O(N)$ time using

$$\mu_* = \mathbf{k}_*^\top \hat{\mathbf{K}}_{X,X}^{-1} \mathbf{y} = \mathbf{k}_*^\top \mathbf{L}^{-\top} (\mathbf{L}^{-1} \mathbf{y}) = \mathbf{k}_*^\top \boldsymbol{\alpha} \tag{17.56}$$

We can compute the variance in $O(N^2)$ time for each test case using

$$\sigma_*^2 = k_{**} - \mathbf{k}_*^\top \mathbf{L}^{-T} \mathbf{L}^{-1} \mathbf{k}_* = k_{**} - \mathbf{v}^\top \mathbf{v} \tag{17.57}$$

where $\mathbf{v} = \mathbf{L} \setminus \mathbf{k}_*$.

Finally, the log marginal likelihood (needed for kernel learning, Sec. 17.3.6) can be computed using

$$\log p(\mathbf{y}|\mathbf{X}) = -\frac{1}{2}\mathbf{y}^\top \boldsymbol{\alpha} - \sum_{n=1}^{N} \log L_{nn} - \frac{N}{2} \log(2\pi) \tag{17.58}$$

17.3.6 Estimating the kernel

Most kernels have some free parameters, which can have a large effect on the predictions from the model. For example, suppose we are performing 1d regression using a GP with an RBF kernel of the form

$$\mathcal{K}(x_p, x_q) = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2}(x_p - x_q)^2\right) \tag{17.59}$$

Here ℓ is the horizontal scale over which the function changes, σ_f^2 controls the vertical scale of the function. We assume observation noise with variance σ_y^2.

We sampled 20 observations from an MVN with a covariance given by $\boldsymbol{\Sigma} = \mathcal{K}(x_i, x_j)$ for a grid of points $\{x_i\}$, and added observation noise of value σ_y. We then fit this data using a GP with the

same kernel, but with a range of hyperparmameters. Fig. 17.9 illustrates the effects of changing these parameters. In Fig. 17.9(a), we use $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$, and the result is a good fit. In Fig. 17.9(b), we reduce the length scale to $\ell = 0.3$ (the other parameters were optimized by maximum (marginal) likelihood, a technique we discuss below); now the function looks more "wiggly". Also, the uncertainty goes up faster, since the effective distance from the training points increases more rapidly. In Fig. 17.9(c), we increase the length scale to $\ell = 3$; now the function looks smoother.

17.3.6.1 Empirical Bayes

To estimate the kernel parameters $\boldsymbol{\theta}$ (sometimes called hyperparameters), we could use exhaustive search over a discrete grid of values, with validation loss as an objective, but this can be quite slow. (This is the approach used by nonprobabilistic methods, such as SVMs (Sec. 17.5) to tune kernels.) Here we consider an empirical Bayes approach (Sec. 4.6.5.3), which will allow us to use gradient-based optimization methods, which are much faster. In particular, we will maximize the marginal likelihood

$$p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = \int p(\mathbf{y}|\mathbf{f}, \mathbf{X}) p(\mathbf{f}|\mathbf{X}, \boldsymbol{\theta}) d\mathbf{f} \tag{17.60}$$

(The reason it is called the marginal likelihood, rather than just likelihood, is because we have marginalized out the latent Gaussian vector $\mathbf{f}$.)

For notational simplicity, we assume the mean function is 0. Since $p(\mathbf{f}|\mathbf{X}) = \mathcal{N}(\mathbf{f}|\mathbf{0}, \mathbf{K})$, and $p(\mathbf{y}|\mathbf{f}) = \prod_{n=1}^{N} \mathcal{N}(y_n|f_n, \sigma_y^2)$, the marginal likelihood is given by

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = \log \mathcal{N}(\mathbf{y}|\mathbf{0}, \hat{\mathbf{K}}_{X,X}) = -\frac{1}{2}\mathbf{y}\hat{\mathbf{K}}_{X,X}^{-1}\mathbf{y} - \frac{1}{2}\log|\hat{\mathbf{K}}_{X,X}| - \frac{N}{2}\log(2\pi) \tag{17.61}$$

where the dependence of $\hat{\mathbf{K}}_{X,X} = \mathbf{K}_{X,X} + \sigma_2^2 \mathbf{I}_N$ on $\boldsymbol{\theta}$ is implicit. The first term is a data fit term, the second term is a model complexity term, and the third term is just a constant. To understand the tradeoff between the first two terms, consider a SE kernel in 1D, as we vary the length scale ℓ and hold σ_y^2 fixed. For short length scales, the fit will be good, so $\mathbf{y}^\top \hat{\mathbf{K}}_{X,X}^{-1}\mathbf{y}$ will be small. However, the model complexity will be high: $\mathbf{K}$ will be almost diagonal, (as in Fig. 13.26, top right), since most points will not be considered "near" any others, so the $\log|\hat{\mathbf{K}}_{X,X}|$ term will be large. For long length scales, the fit will be poor but the model complexity will be low: $\mathbf{K}$ will be almost all 1's, (as in Fig. 13.26, bottom right), so $\log|\hat{\mathbf{K}}_{X,X}|$ will be small.

We now discuss how to maximize the marginal likelihood. One can show that

$$\frac{\partial}{\partial \theta_j} \log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = \frac{1}{2}\mathbf{y}^\top \hat{\mathbf{K}}_{X,X}^{-1} \frac{\partial \hat{\mathbf{K}}_{X,X}}{\partial \theta_j} \hat{\mathbf{K}}_{X,X}^{-1}\mathbf{y} - \frac{1}{2}\mathrm{tr}(\hat{\mathbf{K}}_{X,X}^{-1} \frac{\partial \hat{\mathbf{K}}_{X,X}}{\partial \theta_j}) \tag{17.62}$$

$$= \frac{1}{2}\mathrm{tr}\left((\boldsymbol{\alpha}\boldsymbol{\alpha}^\top - \hat{\mathbf{K}}_{X,X}^{-1}) \frac{\partial \hat{\mathbf{K}}_{X,X}}{\partial \theta_j}\right) \tag{17.63}$$

where $\boldsymbol{\alpha} = \hat{\mathbf{K}}_{X,X}^{-1}\mathbf{y}$. It takes $O(N^3)$ time to compute $\hat{\mathbf{K}}_{X,X}^{-1}$, and then $O(N^2)$ time per hyper-parameter to compute the gradient.

The form of $\frac{\partial \hat{\mathbf{K}}_{X,X}}{\partial \theta_j}$ depends on the form of the kernel, and which parameter we are taking derivatives with respect to. Often we have constraints on the hyper-parameters, such as $\sigma_y^2 \geq 0$. In this case, we can define $\theta = \log(\sigma_y^2)$, and then use the chain rule.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

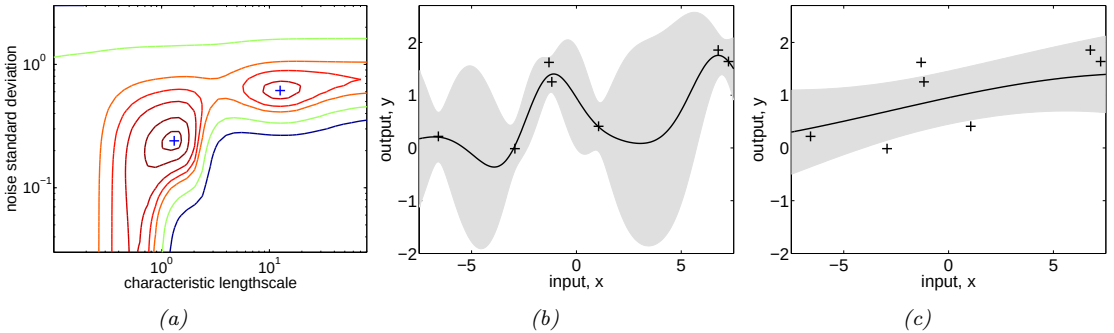


Figure 17.10: Illustration of local minima in the marginal likelihood surface. (a) We plot the log marginal likelihood vs kernel length scale ℓ and observation noise σ_y, for fixed signal level $\sigma_f = 1$, using the 7 data points shown in panels b and c. (b) The function corresponding to the lower left local minimum, $(\ell, \sigma_y) \approx (1, 0.2)$. This is quite "wiggly" and has low noise. (c) The function corresponding to the top right local minimum, $(\ell, \sigma_y) \approx (10, 0.8)$. This is quite smooth and has high noise. The data was generated using $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$. From Figure 5.5 of [RW06]. Generated by code at figures.probml.ai/book1/17.10.

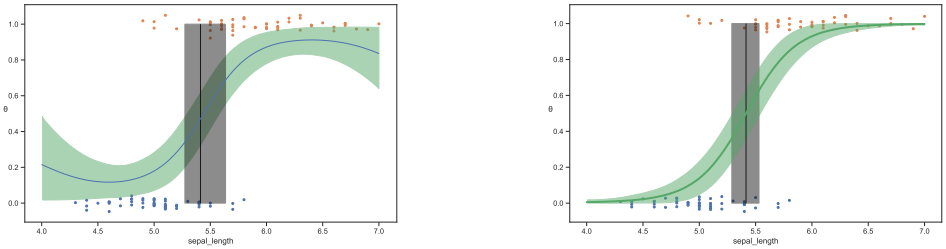
Given an expression for the log marginal likelihood and its derivative, we can estimate the kernel parameters using any standard gradient-based optimizer. However, since the objective is not convex, local minima can be a problem, as we illustrate below, so we may need to use multiple restarts.

As an example, consider the RBF in Eq. (17.59) with $\sigma_f^2 = 1$. In Fig. 17.10(a), we plot $\log p(\mathbf{y}|\mathbf{X}, \ell, \sigma_y^2)$ (where $\mathbf{X}$ and $\mathbf{y}$ are the 7 data points shown in panels b and c) as we vary ℓ and σ_y^2. The two local optima are indicated by $+$. The bottom left optimum corresponds to a low-noise, short-length scale solution (shown in panel b). The top right optimum corresponds to a high-noise, long-length scale solution (shown in panel c). With only 7 data points, there is not enough evidence to confidently decide which is more reasonable, although the more complex model (panel b) has a marginal likelihood that is about 60% higher than the simpler model (panel c). With more data, the more complex model would become even more preferred.

Fig. 17.10 illustrates some other interesting (and typical) features. The region where $\sigma_y^2 \approx 1$ (top of panel a) corresponds to the case where the noise is very high; in this regime, the marginal likelihood is insensitive to the length scale (indicated by the horizontal contours), since all the data is explained as noise. The region where $\ell \approx 0.5$ (left hand side of panel a) corresponds to the case where the length scale is very short; in this regime, the marginal likelihood is insensitive to the noise level (indicated by the vertical contours), since the data is perfectly interpolated. Neither of these regions would be chosen by a good optimizer.

17.3.6.2 Bayesian inference

When we have a small number of datapoints (e.g., when using GPs for Bayesian optimization), using a point estimate of the kernel parameters can give poor results [Bul11; WF14]. In such cases, we may wish to approximate the posterior over the kernel parameters. Several methods can be used. For example, [MA10] shows how to use slice sampling, [Hen+15] shows how to use Hamiltonian Monte Carlo, and [BBV11] shows how to use sequential Monte Carlo.



(a) (b)

Figure 17.11: GP classifier for a binary classification problem on Iris flowers (setosa vs versicolor) using a single input feature (sepal length). The fat vertical line is the credible interval for the decision boundary. (a) SE kernel. (b) SE plus linear kernel. Adapted from Figures 7.11–7.12 of [Mar18]. Generated by code at figures.probml.ai/book1/17.11.

17.3.7 GPs for classification

So far, we have focused on GPs for regression using Gaussian likelihoods. In this case, the posterior is also a GP, and all computation can be performed analytically. However, if the likelihood is non-Gaussian, such as the Bernoulli likelihood for binary classification, we can no longer compute the posterior exactly.

There are various approximations we can make. some of which we discuss in the sequel to this book, [Mur22]. In this section, we use the Hamiltonian Monte Carlo method (Sec. 4.6.8.4), both for the latent Gaussian function $\mathbf{f}$ as well as the kernel hyperparameters $\boldsymbol{\theta}$. The basic idea is to specify the negative log joint

$$-\mathcal{E}(\mathbf{f}, \boldsymbol{\theta}) = \log p(\mathbf{f}, \boldsymbol{\theta}|\mathbf{X}, \mathbf{y}) = \log \mathcal{N}(\mathbf{f}|\mathbf{0}, \mathbf{K}(\mathbf{X}, \mathbf{X})) + \sum_{n=1}^{N} \log \text{Ber}(y_n|f_n(\mathbf{x}_n)) + \log p(\boldsymbol{\theta}) \tag{17.64}$$

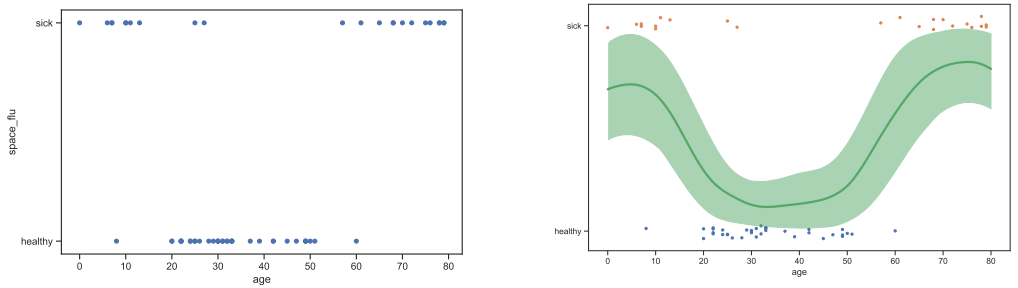
We then use autograd to compute $\nabla_{\mathbf{f}} \mathcal{E}(\mathbf{f}, \boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}} \mathcal{E}(\mathbf{f}, \boldsymbol{\theta})$, and use these gradients as inputs to a Gaussian proposal distribution.

Let us consider a 1d example from [Mar18]. This is similar to the Bayesian logistic regression example from Fig. 4.20, where the goal is to classify iris flowers as being setosa or versicolor, $y_n \in \{0, 1\}$, given information about the sepal length, x_n. We will use an SE kernel with length scale ℓ. We put a $\text{Ga}(2, 0.5)$ prior on ℓ.

Fig. 17.11a shows the results using the SE kernel. This is similar to the results of linear logistic regression (see Fig. 4.20), except that at the edges (away from the data), the probability curves towards 0.5. This is because the prior mean function is $m(x) = 0$, and $\sigma(0) = 0.5$. We can eliminate this artefact by using a more flexible kernel, which encodes the prior knowledge that we expect the output to be monotonically increasing or decreasing in the input. We can do this using a **linear kernel**,

$$\mathcal{K}(x, x') = (x - c)(x' - c) \tag{17.65}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.12: (a) Fictitious "space flu" binary classification problem. (b) Fit from a GP with SE kernel. Adapted from Figures 7.13–7.14 of [Mar18]. Generated by code at figures.probml.ai/book1/17.12.

We can scale and add this to the SE kernel to get

$$\mathcal{K}(x, x') = \tau(x - c)(x' - c) + \exp\left[-\frac{(x - x')^2}{2\ell^2}\right] \tag{17.66}$$

The results are shown in Fig. 17.11b, and look more reasonable.

One might wonder why we bothered to use a GP, when the results are no better than a simple linear logistic regresion model. The reason is that the GP is much more flexible, and makes fewer a priori assumptions, beyond smoothness. For example, suppose the data looked like Fig. 17.12a. In this case, a linear logistic regression model could not fit the data. We could in principle use a neural network, but it may not work well since we only have 60 data points. However, GPs are well designed to handle the small sample setting. In Fig. 17.12b, we show the results of fitting a GP with an SE kernel to this data. The results look reasonable.

17.3.8 Connections with deep learning

It turns out that there are many interesting connections and similarities between GPs and deep neural networks. For example, one can show that an RBF network with one hidden layer, which is **infinitely wide**, is equivalent to a GP with an RBF kernel. (This follows from the fact that the RBF kernel can be expressed as the inner product of an infinite number of features.) More generally, we can interpret many kinds of DNNs as GPs using a specific kind of kernel derived from the architecture of the model. See the sequel to this book, [Mur22], for details.

17.4 Scaling GPs to large datasets

The main disadvantage of GPs (and other kernel methods, such as SVMs, which we discuss in Sec. 17.5) is that inverting the $N \times N$ kernel matrix takes $O(N^3)$ time, making the method too slow for big datasets. Many different approximate schemes have been proposed to speedup GPs (see e.g., [Liu+18a] for a review). In this section, we briefly mention some of them. For more details, see the sequel to this book, [Mur22].

17.4.1 Sparse (inducing-point) approximations

A simple approach to speeding up GP inference is to use less data. A better approach is to try to "summarize" the N training points $\mathbf{X}$ into $M \ll N$ **inducing points** or **pseudo inputs** $\mathbf{Z}$. This lets us replace $p(\mathbf{f}|\mathbf{f}_X)$ with $p(\mathbf{f}|\mathbf{f}_Z)$, where $\mathbf{f}_X = \{f(\mathbf{x}) : \mathbf{x} in \mathbf{X}\}$ is the vector of observed function values at the training points, and $\mathbf{f}_Z = \{f(\mathbf{x}) : \mathbf{x} in \mathbf{Z}\}$ is the vector of estimated function values at the inducing points. By optimizing $(\mathbf{Z}, \mathbf{f}_Z)$ we can learn to "compress" the training data $(\mathbf{X}, \mathbf{f}_X)$ into a "bottleneck" $(\mathbf{Z}, \mathbf{f}_Z)$, thus speeding up computation from $O(N^3)$ to $O(M^3)$. This is called a **sparse GP**. This whole process can be made rigorous using the framework of variational inference. For details, see the sequel to this book, [Mur22].

17.4.2 Exploiting parallelization and kernel matrix structure

It takes $O(N^3)$ time to compute the Cholesky decomposition of $\mathbf{K}_{X,X}$, which is needed to solve the linear system $\hat{\mathbf{K}}_{X,X}\boldsymbol{\alpha} = \mathbf{y}$ and to compute $|\mathbf{K}_{X,X}|$, where $\hat{\mathbf{K}}_{X,X} = \mathbf{K}_{X,X} + \sigma^2\mathbf{I}_N$. An alternative to Cholesky decomposition is to use linear algebra methods, often called **Krylov subspace methods**, which are based just on **matrix vector multiplication** or **MVM**. These approaches are often much faster, since they can naturally exploit structure in the kernel matrix. Moreover, even if the kernel matrix does not have special structure, matrix multiplies are trivial to parallelize, and can thus be greatly accelerated by GPUs, unlike Cholesky based methods which are largely sequential. This is the basis of the popular **GPyTorch** package [Gar+18]. For more details, see the sequel to this book, [Mur22].

17.4.3 Random feature approximation

Although the power of kernels resides in the ability to avoid working with featurized representations of the inputs, such kernelized methods take $O(N^3)$ time, in order to invert the Gram matrix $\mathbf{K}$. This can make it difficult to use such methods on large scale data. Fortunately, we can approximate the feature map for many (shift invariant) kernels using a randomly chosen finite set of M basis functions, thus reducing the cost to $O(NM + M^3)$. We briefly discuss this idea below. For more details, see e.g., [Liu+20].

17.4.3.1 Random features for RBF kernel

We will focus on the case of the Gaussian RBF kernel. One can show that

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') \approx \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}') \tag{17.67}$$

where the (real-valued) feature vector is given by

$$\boldsymbol{\phi}(\mathbf{x}) \triangleq \frac{1}{\sqrt{T}}[(\sin(\boldsymbol{\omega}_1^\top \mathbf{x}), ..., \sin(\boldsymbol{\omega}_T^\top \mathbf{x}), \cos(\boldsymbol{\omega}_1^\top \mathbf{x}), ..., \cos(\boldsymbol{\omega}_T^\top \mathbf{x}))] \tag{17.68}$$

$$= \frac{1}{\sqrt{T}}[\sin(\boldsymbol{\Omega}\mathbf{x}), \cos(\boldsymbol{\Omega}\mathbf{x})] \tag{17.69}$$

where $T = M/2$, and $\boldsymbol{\Omega} \in \mathbb{R}^{T \times D}$ is a random Gaussian matrix, where the entries are sampled iid from $\mathcal{N}(0, 1/\sigma^2)$, where σ is the kernel bandwidth. The bias of the approximation decreases as we

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

increase M. In practice, we use a finite M, and compute a single sample Monte Carlo approximation to the expectation by drawing a single random matrix. The representation in Eq. (17.69) are called **random Fourier features** (**RFF**) [RR08] or "weighted sums of random kitchen sinks" [RR09].

We can also use positive random features, rather than trigonometric random features, which can be preferable in some applications, such as attention Sec. 15.6.4. In particular, we can use

$$\boldsymbol{\phi}(\mathbf{x}) \triangleq e^{-||\mathbf{x}||^2/2} \frac{1}{\sqrt{M}} \left[(\exp(\boldsymbol{\omega}_1^\top \mathbf{x}), \cdots, (\exp(\boldsymbol{\omega}_M^\top \mathbf{x})\right] \tag{17.70}$$

where $\boldsymbol{\omega}_m$ are sampled as before. For details, see [Cho+20b].

Regardless of whether we use trigonometric or positive features, we can obtain a lower variance estimate by ensuring that the rows of $\mathbf{Z}$ are random but orthogonal; these are called **orthogonal random features**. Such sampling can be conducted efficiently via Gram-Schmidt orthogonalization of the unstructured Gaussian matrices [Yu+16], or several approximations that are even faster (see [CRW17; Cho+19]).

17.4.3.2 Fastfood approximation

Unfortunately, storing the random matrix $\boldsymbol{\Omega}$ takes $O(DM)$ space, and computing $\boldsymbol{\Omega}\mathbf{x}$ takes $O(DM)$ time, where D is the input dimensionality, and M is the number of random features. This can be prohibitive if $M \gg D$, which it may need to be in order to get any benefits over using the original set of features. Fortunately, we can use the **fast Hadamard transform** to reduce the memory from $O(MD)$ to $O(M)$, and reduce the time from $O(MD)$ to $O(M \log D)$. This approach has been called **fastfood** [LSS13], a reference to the original term "kitchen sinks".

17.4.3.3 Extreme learning machines

We can use the random features approximation to the kernel to convert a GP into a linear model of the form

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}\boldsymbol{\phi}(\mathbf{x}) = \mathbf{W}h(\mathbf{Z}\mathbf{x}) \tag{17.71}$$

where $h(a) = \sqrt{1/M}[\sin(a), \cos(a)]$ for RBF kernels. This is equivalent to a one-layer MLP with random (and fixed) input-to-hidden weights. When $M > N$, this corresponds to an over-parameterized model, which can perfectly interpolate the training data.

In [Cur+17], they apply this method to fit a logistic regression model of the form $f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top h(\hat{\mathbf{Z}}\mathbf{x}) + \mathbf{b}$ using SGD; they call the resulting method "**McKernel**". We can also optimize $\mathbf{Z}$ as well as $\mathbf{W}$, as discussed in [Alb+17], although now the problem is no longer convex.

Alternatively, we can use $M < N$, but stack many such random nonlinear layers together, and just optimize the output weights. This has been called an **extreme learning machine** or **ELM** (see e.g., [Hua14]), although this work is controversial.[1]

1. The controversy has arisen because the inventor Guang-Bin Huang has been accused of not citing related prior work, such as the equivalent approach based on random feature approximations to kernels. For details, see https://en.wikipedia.org/wiki/Extreme_learning_machine#Controversy.

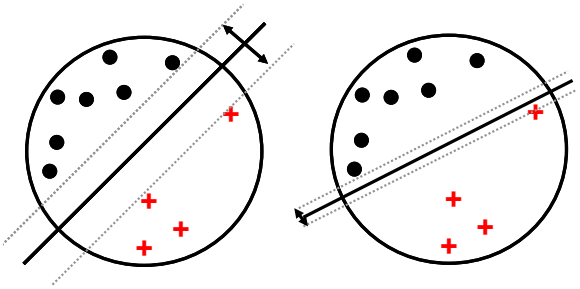


Figure 17.13: Illustration of the large margin principle. Left: a separating hyper-plane with large margin. Right: a separating hyper-plane with small margin.

17.5 Support vector machines (SVMs)

In this section, we discuss a form of (non-probabilistic) predictors for classification and regression problems which have the form

$$f(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i \mathcal{K}(\mathbf{x}, \mathbf{x}_i) \tag{17.72}$$

By adding suitable constraints, we can ensure that many of the α_i coefficients are 0, so that predictions at test time only depend on a subset of the training points. The surving points are called "**support vectors**", and the resulting model is called a **support vector machine** or **SVM**. We give a brief summary below. More details, can be found in e.g., [VGS97; SS01],

17.5.1 Large margin classifiers

Consider a binary classifier of the form $h(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$, where the decision boundary is given by the following linear function:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 \tag{17.73}$$

(In the SVM literature, it is common to assume the class labels are -1 and $+1$, rather than 0 and 1. To avoid confusion, we denote such target labels by $\tilde{y}$ rather than y.) There may be many lines that separate the data. However, intuitively we would like to pick the one that has maximum **margin**, which is the distance of the closest point to the decision boundary, since this will give us the most robust solution. This idea is illustrated in Fig. 17.13: the solution on the left has larger margin than the one on the right, and intuitively is is better, since it will be less sensitive to perturbations of the data.

How can we compute such a **large margin classifier**? First we need to derive an expression for the distance of a point to the decision boundary. Referring to Fig. 17.14(a), we see that

$$\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{||\mathbf{w}||} \tag{17.74}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

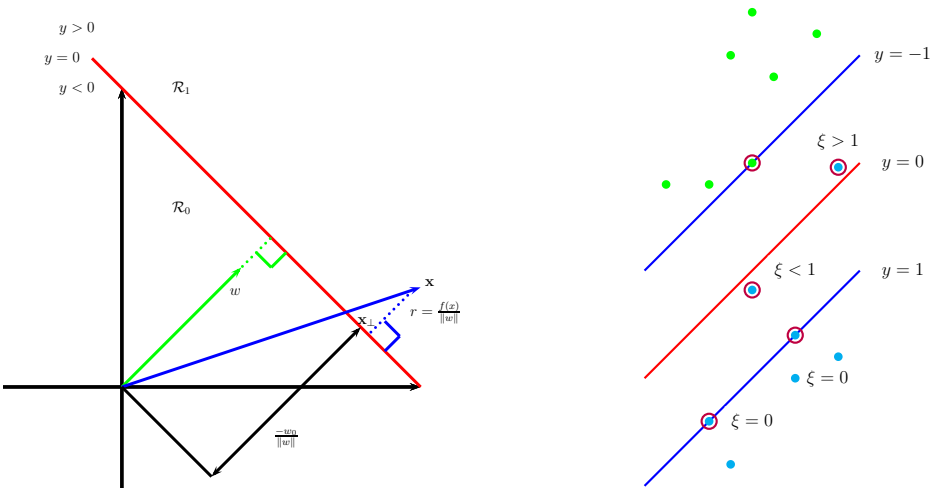


Figure 17.14: (a) Illustration of the geometry of a linear decision boundary in 2d. A point $\mathbf{x}$ *is classified as belonging in decision region* $\mathcal{R}_1$ *if* $f(\mathbf{x}) > 0$*, otherwise it belongs in decision region* $\mathcal{R}_0$*;* $\mathbf{w}$ *is a vector which is perpendicular to the decision boundary. The term* w_0 *controls the distance of the decision boundary from the origin.* $\mathbf{x}_\perp$ *is the orthogonal projection of* $\mathbf{x}$ *onto the boundary. The signed distance of* $\mathbf{x}$ *from the dboundary is given by* $f(\mathbf{x})/||\mathbf{w}||$*. Adapted from Figure 4.1 of [Bis06]. (b) Points with circles around them are support vectors, and have dual variables* $\alpha_n > 0$*. In the soft margin case, we associate a slack variable* ξ_n *with each example. If* $0 < \xi_n < 1$*, the point is inside the margin, but on the correct side of the decision boundary. If* $\xi_n > 1$*, the point is on the wrong side of the boundary. Adapted from Figure 7.3 of [Bis06].*

where r is the distance of $\mathbf{x}$ from the decision boundary whose normal vector is $\mathbf{w}$, and $\mathbf{x}_\perp$ is the orthogonal projection of $\mathbf{x}$ onto this boundary.

We would like to maximize r, so we need to express it as a function of $\mathbf{w}$. First, note that

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = (\mathbf{w}^\top \mathbf{x}_\perp + w_0) + r\frac{\mathbf{w}^\top \mathbf{w}}{||\mathbf{w}||} = (\mathbf{w}^\top \mathbf{x}_\perp + w_0) + r||\mathbf{w}|| \tag{17.75}$$

Since $0 = f(\mathbf{x}_\perp) = \mathbf{w}^\top \mathbf{x}_\perp + w_0$, we have $f(\mathbf{x}) = r||\mathbf{w}||$ and hence $r = \frac{f(\mathbf{x})}{||\mathbf{w}||}$.

Since we want to ensure each point is on the correct side of the boundary, we also require $f(\mathbf{x}_n)\tilde{y}_n > 0$. We want to maximize the distance of the closest point, so our final objective becomes

$$\max_{\mathbf{w}, w_0} \frac{1}{||\mathbf{w}||} \min_{n=1}^{N} \left[\tilde{y}_n(\mathbf{w}^\top \mathbf{x}_n + w_0)\right] \tag{17.76}$$

Note that by rescaling the parameters using $\mathbf{w} \to k\mathbf{w}$ and $w_0 \to kw_0$, we do not change the distance of any point to the boundary, since the k factor cancels out when we divide by $||\mathbf{w}||$. Therefore let us define the scale factor such that $\tilde{y}_n f_n = 1$ for the point that is closest to the decision boundary. Hence we require $\tilde{y}_n f_n \geq 1$ for all n. Finally, note that maximizing $1/||\mathbf{w}||$ is equivalent to minimizing

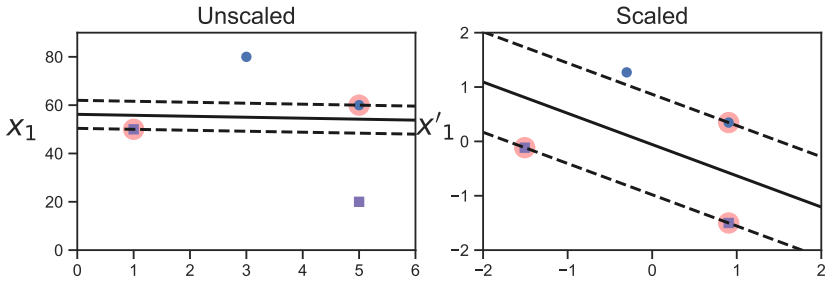


Figure 17.15: Illustration of the benefits of scaling the input features before computing a max margin classifier. Adapted from Figure 5.2 of [Gér19]. Generated by code at figures.probml.ai/book1/17.15.

$||\mathbf{w}||^2$. Thus we get the new objective

$$\min_{\mathbf{w},w_0} \frac{1}{2}||\mathbf{w}||^2 \quad \text{s.t.} \quad \tilde{y}_n(\mathbf{w}^\top \mathbf{x}_n + w_0) \geq 1, n = 1 : N \tag{17.77}$$

(The factor of $\frac{1}{2}$ is added for convenience and doesn't affect the optimal parameters.) The constraint says that we want all points to be on the correct side of the decision boundary with a margin of at least 1.

Note that it is important to scale the input variables before using an SVM, otherwise the margin measures distance of a point to the boundary using all input dimensions equally. See Fig. 17.15 for an illustration.

17.5.2 The dual problem

The objective in Eq. (17.77) is a standard quadratic programming problem (Sec. 8.5.4), since we have a quadratic objective subject to linear constraints. This has $N + D + 1$ variables subject to N constraints, and is known as a **primal problem**.

In convex optimization, for every primal problem we can derive a **dual problem**. Let $\boldsymbol{\alpha} \in \mathbb{R}^N$ be the dual variables, corresponding to Lagrange multipliers that enforce the N inequality constraints. The generalized Lagrangian is given below (see Sec. 8.5.2 for relevant background information on constrained optimization):

$$\mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} - \sum_{n=1}^{N} \alpha_n(\tilde{y}_n(\mathbf{w}^\top \mathbf{x}_n + w_0) - 1) \tag{17.78}$$

To optimize this, we must find a stationary point that satisfies

$$(\hat{\mathbf{w}}, \hat{w}_0, \hat{\boldsymbol{\alpha}}) = \min_{\mathbf{w},w_0} \max_{\boldsymbol{\alpha}} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) \tag{17.79}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can do this by computing the partial derivatives wrt $\mathbf{w}$ and w_0 and setting to zero. We have

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = \mathbf{w} - \sum_{n=1}^{N} \alpha_n \tilde{y}_n \mathbf{x}_n \tag{17.80}$$

$$\frac{\partial}{\partial w_0} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = -\sum_{n=1}^{N} \alpha_n \tilde{y}_n \tag{17.81}$$

and hence

$$\hat{\mathbf{w}} = \sum_{n=1}^{N} \hat{\alpha}_n \tilde{y}_n \mathbf{x}_n \tag{17.82}$$

$$0 = \sum_{n=1}^{N} \hat{\alpha}_n \tilde{y}_n \tag{17.83}$$

Plugging these into the Lagrangian yields the following

$$\mathcal{L}(\hat{\mathbf{w}}, \hat{w}_0, \boldsymbol{\alpha}) = \frac{1}{2} \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - \sum_{n=1}^{N} \alpha_n \tilde{y}_n \hat{\mathbf{w}}^\top \mathbf{x}_n - \sum_{n=1}^{N} \alpha_n \tilde{y}_n w_0 + \sum_{n=1}^{N} \alpha_n \tag{17.84}$$

$$= \frac{1}{2} \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - 0 + \sum_{n=1}^{N} \alpha_n \tag{17.85}$$

$$= -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j + \sum_{n=1}^{N} \alpha_n \tag{17.86}$$

This is called the **dual form** of the objective. We want to maximize this wrt $\boldsymbol{\alpha}$ subject to the constraints that $\sum_{n=1}^{N} \alpha_n \tilde{y}_n = 0$ and $0 \leq \alpha_n$ for $n = 1 : N$.

The above objective is a quadratic problem in N variables. Standard QP solvers take $O(N^3)$ time. However, specialized algorithms, which avoid the use of generic QP solvers, have been developed for this problem, such as the **sequential minimal optimization** or **SMO** algorithm [Pla98], which takes $O(N)$ to $O(N^2)$ time.

Since this is a convex objective, the solution must satisfy the KKT conditions (Sec. 8.5.2), which tell us tha the following properties hold:

$$\alpha_n \geq 0 \tag{17.87}$$

$$\tilde{y}_n f(\mathbf{x}_n) - 1 \geq 0 \tag{17.88}$$

$$\alpha_n (\tilde{y}_n f(\mathbf{x}_n) - 1) = 0 \tag{17.89}$$

Hence either $\alpha_n = 0$ (in which case example n is ignored when computing $\hat{\mathbf{w}}$) or the constraint $\tilde{y}_n(\hat{\mathbf{w}}^\top \mathbf{x}_n + \hat{w}_0) = 1$ is active. This latter condition means that example n lies on the decision boundary; these points are are known as the **support vectors**, as shown in Fig. 17.14(b). We denote the set of support vectors by $\mathcal{S}$.

To perform prediction, we use

$$f(\mathbf{x}; \hat{\mathbf{w}}, \hat{w}_0) = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathbf{x}_n^\top \mathbf{x} + \hat{w}_0 \tag{17.90}$$

To solve for $\hat{w}_0$ we can use the fact that for any support vector, we have $\tilde{y}_n f(\mathbf{x}; \hat{\mathbf{w}}, \hat{w}_0) = 1$. Multiplying both sides by $\tilde{y}_n$, and exploiting the fact that $\tilde{y}_n^2 = 1$, we get $\hat{w}_0 = \tilde{y}_n - \hat{\mathbf{w}}^\top \mathbf{x}_n$. In practice we get better results by averaging over all the support vectors to get

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \hat{\mathbf{w}}^\top \mathbf{x}_n) = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \mathbf{x}_m^\top \mathbf{x}_n) \tag{17.91}$$

17.5.3 Soft margin classifiers

If the data is not linearly separable, there will be no feasible solution in which $\tilde{y}_n f_n \geq 1$ for all n. We therefore introduce **slack variables** $\xi_n \geq 0$ and replace the hard constraints that $\tilde{y}_n f_n \geq 0$ with the **soft margin constraints** that $\tilde{y}_n f_n \geq 1 - \xi_n$. The new objective becomes

$$\min_{\mathbf{w}, w_0, \boldsymbol{\xi}} \frac{1}{2} ||\mathbf{w}||^2 + C \sum_{n=1}^N \xi_n \quad \text{s.t.} \quad \xi_n \geq 0, \quad \tilde{y}_n (\mathbf{x}_n^\top \mathbf{w} + w_0) \geq 1 - \xi_n \tag{17.92}$$

where $C \geq 0$ is a hyper parameter controlling how many points we allow to violate the margin constraint. (If $C = \infty$, we recover the unregularized, hard-margin classifier.)

The corresponding Lagrangian for the soft margin classifier becomes

$$\mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu}) = \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n (\tilde{y}_n (\mathbf{w}^\top \mathbf{x}_n + w_0) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n \tag{17.93}$$

where $\alpha_n \geq 0$ and $\mu_n \geq 0$ are the Lagrange multipliers. Optimizing out $\mathbf{w}$, w_0 and $\boldsymbol{\xi}$ gives the dual form

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j \tag{17.94}$$

This is identical to the hard margin case; however, the constraints are different. In particular, the KKT conditions imply

$$0 \leq \alpha_n \leq C \tag{17.95}$$

$$\sum_{n=1}^N \alpha_n \tilde{y}_n = 0 \tag{17.96}$$

If $\alpha_n = 0$, the point is ignored. If $0 < \alpha_n < C$ then $\xi_n = 0$, so the point lies on the margin. If $\alpha_n = C$, the point can lie inside the margin, and can either be correctly classified if $\xi_n \leq 1$, or misclassified if $\xi_n > 1$. See Fig. 17.14(b) for an illustration. Hence $\sum_n \xi_n$ is an upper bound on the number of misclassified points.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

As before, the bias term can be computed using

$$\hat{w}_0 = \frac{1}{|\mathcal{M}|} \sum_{n \in \mathcal{M}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \mathbf{x}_m^\top \mathbf{x}_n) \tag{17.97}$$

where $\mathcal{M}$ is the set of points having $0 < \alpha_n < C$.

There is an alternative formulation of the soft margin SVM known as the **ν-SVM classifier** [Sch+00]. This involves maximizing

$$\mathcal{L}(\boldsymbol{\alpha}) = -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j \tag{17.98}$$

subject to the constraints that

$$0 \leq \alpha_n \leq 1/N \tag{17.99}$$

$$\sum_{n=1}^{N} \alpha_n \tilde{y}_n = 0 \tag{17.100}$$

$$\sum_{n=1}^{M} \alpha_n \geq \nu \tag{17.101}$$

This has the advantage that the parameter ν, which replaces C, can be interpreted as an upper bound on the fraction of **margin errors** (points for which $\xi_n > 0$), as well as a lower bound on the number of support vectors.

17.5.4 The kernel trick

So far we have converted the large margin binary classification problem into a dual problem in N unknowns ($\boldsymbol{\alpha}$) which (in general) takes $O(N^3)$ time to solve, which can be slow. However, the principal benefit of the dual problem is that we can replace all inner product operations $\mathbf{x}^\top \mathbf{x}'$ with a call to a positive definite (Mercer) kernel function, $\mathcal{K}(\mathbf{x}, \mathbf{x}')$. This is called the **kernel trick**.

In particular, we can rewrite the prediction function in Eq. (17.90) as follows:

$$f(\mathbf{x}) = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathbf{x}_n^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) + \hat{w}_0 \tag{17.102}$$

We also need to kernelize the bias term. This can be done by kernelizing Eq. (17.91) as follows:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - (\sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \mathbf{x}_j)^\top \mathbf{x}_i \right) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - (\sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \mathcal{K}(\mathbf{x}_j, \mathbf{x}_i) \right) \tag{17.103}$$

The kernel trick allows us to avoid having to deal with an explicit feature representation of our data, and allows us to easily apply classifiers to structured objects, such as strings and graphs.

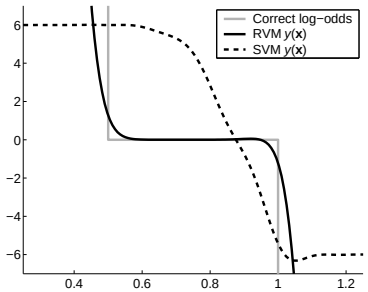


Figure 17.16: Log-odds vs x for 3 different methods. Adapted from Figure 10 of [Tip01]. Used with kind permission of Mike Tipping.

17.5.5 Converting SVM outputs into probabilitues

An SVM classifier produces a hard-labeling, $\hat{y}(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$. However, we often want a measure of confidence in our prediction. One heuristic approach is to interpret $f(\mathbf{x})$ as the log-odds ratio, $\log \frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})}$. We can then convert the output of an SVM to a probability using

$$p(y = 1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(af(\mathbf{x}) + b) \tag{17.104}$$

where a, b can be estimated by maximum likelihood on a separate validation set. (Using the training set to estimate a and b leads to severe overfitting.) This technique was first proposed in [Pla00], and is known as **Platt scaling**.

However, the resulting probabilities are not particularly well calibrated, since there is nothing in the SVM training procedure that justifies interpreting $f(\mathbf{x})$ as a log-odds ratio. To illustrate this, consider an example from [Tip01]. Suppose we have 1d data where $p(x|y = 0) = \text{Unif}(0, 1)$ and $p(x|y = 1) = \text{Unif}(0.5, 1.5)$. Since the class-conditional distributions overlap in the $[0.5, 1]$ range, the log-odds of class 1 over class 0 should be zero in this region, and infinite outside this region. We sampled 1000 points from the model, and then fit a probabilistic kernel classifier (an RVM, described in Sec. 17.6.1) and an SVM with a Gaussian kernel of width 0.1. Both models can perfectly capture the decision boundary, and achieve a generalization error of 25%, which is Bayes optimal in this problem. The probabilistic output from the RVM is a good approximation to the true log-odds, but this is not the case for the SVM, as shown in Fig. 17.16.

17.5.6 Connection with logistic regression

We have see that data points that are on the correct side of the decision boundary have $\xi_n = 0$; for the others, we have $\xi_n = 1 - \tilde{y}_n f(\mathbf{x}_n)$. Therefore we can rewrite the objective in Eq. (17.92) as follows:

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^{N} \ell_{\text{hinge}}(\tilde{y}_n, f(\mathbf{x}_n))) + \lambda|\mathbf{w}||^2 \tag{17.105}$$

where $\lambda = (2C)^{-1}$ and $\ell_{\text{hinge}}(y, \eta)$ is the **hinge loss** function defined by

$$\ell_{\text{hinge}}(\tilde{y}, \eta) = \max(0, 1 - \tilde{y}\eta) \tag{17.106}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

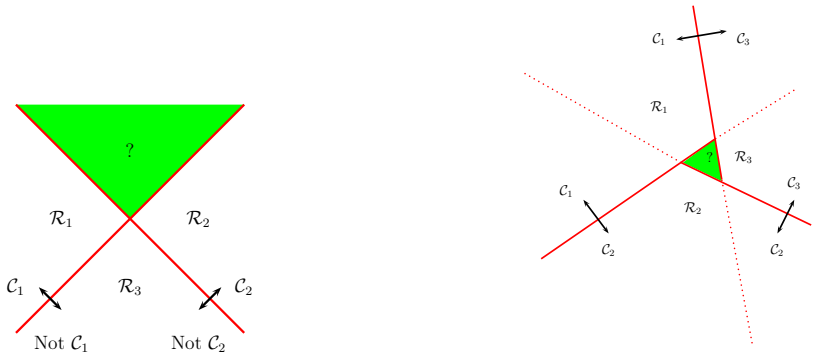


Figure 17.17: (a) The one-versus-rest approach. The green region is predicted to be both class 1 and class 2. (b) The one-versus-one approach. The label of the green region is ambiguous. Adapted from Figure 4.2 of [Bis06].

As we see from Fig. 4.2, this is a convex, piecewise differentiable upper bound to the 0-1 loss, that has the shape of a partially open door hinge.

By contrast, (penalized) logistic regression optimizes

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^{N} \ell_{ll}(\tilde{y}_n, f(\mathbf{x}_n))) + \lambda |\mathbf{w}||^2 \tag{17.107}$$

where the **log loss** is given by

$$\ell_{ll}(\tilde{y}, \eta) = -\log p(y|\eta) = \log(1 + e^{-\tilde{y}\eta}) \tag{17.108}$$

This is also plotted in Fig. 4.2. We see that it is similar to the hinge loss, but with two important differences. First the hinge loss is piecewise linear, so we cannot use regular gradient methods to optimize it. (We can, however, compute the subgradient at $\tilde{y}\eta = 1$.) Second, the hinge loss has a region where it is strictly 0; this results in sparse estimates.

We see that both functions are convex upper bounds on the 0-1 loss, which is given by

$$\ell_{01}(\tilde{y}, \hat{y}) = \mathbb{I}(\tilde{y} \neq \hat{y}) = \mathbb{I}(\tilde{y}\,\hat{y} < 0) \tag{17.109}$$

These upper bounds are easier to optimize and can be viewed as surrogates for the 0-1 loss. See Sec. 4.3.2 for details.

17.5.7 Multi-class classification with SVMs

SVMs are inherently a binary classifier. One way to convert them to a multi-class classification model is to train C binary classifiers, where the data from class c is treated as positive, and the data from all the other classes is treated as negative. We then use the rule $\hat{y}(\mathbf{x}) = \arg\max_c f_c(\mathbf{x})$ to

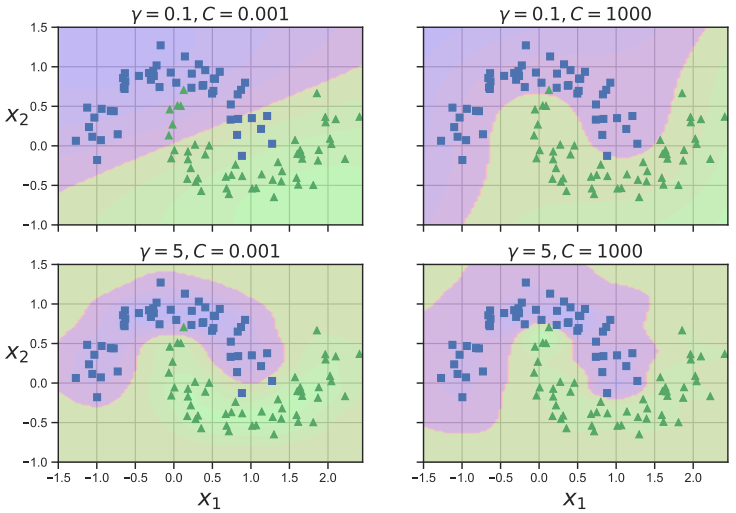


Figure 17.18: SVM classifier with RBF kernel with precision γ and regularizer C applied to two moons data. Adapted from Figure 5.9 of [Gér19]. Generated by code at figures.probml.ai/book1/17.18.

predict the final label, where $f_c(\mathbf{x}) = \log \frac{p(c=1|\mathbf{x})}{p(c=0|\mathbf{x})}$ is the score given by classifier c. This is known as the **one-versus-the-rest** approach (also called **one-vs-all**).

Unfortunately, this approach has several problems. First, it can result in regions of input space which are ambiguously labeled. For example, the green region at the top of Fig. 17.17(a) is predicted to be both class 2 and class 1. A second problem is that the magnitude of the f_c's scores are not calibrated with each other, so it is hard to compare them. Finally, each binary subproblem is likely to suffer from the class imbalance problem (Sec. 10.3.8.2). For example, suppose we have 10 equally represented classes. When training f_1, we will have 10% positive examples and 90% negative examples, which can hurt performance.

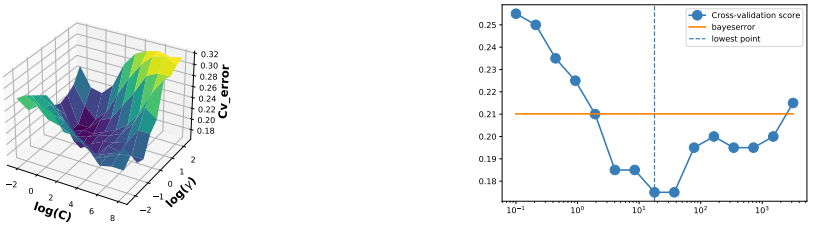
Another approach is to use the **one-versus-one** or OVO approach, also called **all pairs**, in which we train $C(C-1)/2$ classifiers to discriminate all pairs $f_{c,c'}$. We then classify a point into the class which has the highest number of votes. However, this can also result in ambiguities, as shown in Fig. 17.17(b). Also, this requires fitting $O(C^2)$ models.

17.5.8 How to choose the regularizer C

SVMs require that you specify the kernel function and the parameter C. Typically C is chosen by cross-validation. Note, however, that C interacts quite strongly with the kernel parameters. For example, suppose we are using an RBF kernel with precision $\gamma = \frac{1}{2\sigma^2}$. If γ is large, corresponding to narrow kernels, we may need heavy regularization, and hence small C. If γ is small, a larger value of C should be used. So we see that γ and C are tightly coupled, as illustraed in Fig. 17.18.

The authors of libsvm [HCL09] recommend using CV over a 2d grid with values $C \in \{2^{-5}, 2^{-3}, \ldots, 2^{15}\}$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 17.19: (a) A cross validation estimate of the 0-1 error for an SVM classifier with RBF kernel with different precisions $\gamma = 1/(2\sigma^2)$ and different regularizer $\lambda = 1/C$, applied to a synthetic data set drawn from a mixture of 2 Gaussians. (b) A slice through this surface for $\gamma = 5$ The red dotted line is the Bayes optimal error, computed using Bayes rule applied to the model used to generate the data. Adapted from Figure 12.6 of [HTF09]. Generated by code at figures.probml.ai/book1/17.19.

and $\gamma \in \{2^{-15}, 2^{-13}, \ldots, 2^3\}$. See Fig. 17.19 which shows the CV estimate of the 0-1 risk as a function of C and γ.

To choose C efficiently, one can develop a path following algorithm in the spirit of lars (Sec. 11.5.4). The basic idea is to start with C small, so that the margin is wide, and hence all points are inside of it and have $\alpha_i = 1$. By slowly increasing C, a small set of points will move from inside the margin to outside, and their α_n values will change from 1 to 0, as they cease to be support vectors. When C is maximal, the margin becomes empty, and no support vectors remain. See [Has+04] for the details.

17.5.9 Kernel ridge regression

Recall the equation for ridge regression from Eq. (11.57):

$$\hat{\mathbf{w}}_{\text{map}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y} = (\sum_n \mathbf{x}_n \mathbf{x}_n^\top + \lambda \mathbf{I}_D)^{-1} (\sum_n \tilde{y}_n \mathbf{x}_n) \tag{17.110}$$

Using the matrix inversion lemma (Sec. 7.3.3, we can rewrite the ridge estimate as follows

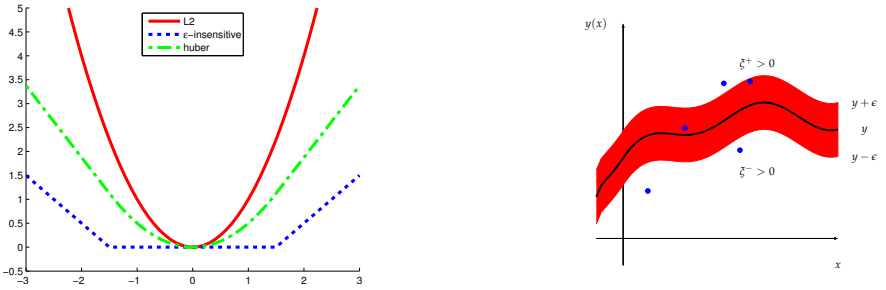
$$\mathbf{w} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1} \mathbf{y} = \sum_n \mathbf{x}_n ((\sum_n \mathbf{x}_n^\top \mathbf{x}_n + \lambda \mathbf{I}_N)^{-1} \mathbf{y})_n \tag{17.111}$$

Let us define the following **dual variables**:

$$\boldsymbol{\alpha} \triangleq (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_N)^{-1} \mathbf{y} = (\sum_n \mathbf{x}_n^\top \mathbf{x}_n + \lambda \mathbf{I}_N)^{-1} \mathbf{y} \tag{17.112}$$

Then we can rewrite the **primal variables** as follows

$$\mathbf{w} = \mathbf{X}^\top \boldsymbol{\alpha} = \sum_{n=1}^{N} \alpha_n \mathbf{x}_n \tag{17.113}$$



(a) (b)

Figure 17.20: (a) Illustration of ℓ_2, Huber and ϵ-insensitive loss functions, where $\epsilon = 1.5$. Generated by code at figures.probml.ai/book1/17.20. (b) Illustration of the ϵ-tube used in SVM regression. Points above the tube have $\xi_i^+ > 0$ and $\xi_i^- = 0$. Points below the tube have $\xi_i^+ = 0$ and $\xi_i^- > 0$. Points inside the tube have $\xi_i^+ = \xi_i^- = 0$. Adapted from Figure 7.7 of [Bis06].

This tells us that the solution vector is just a linear sum of the N training vectors. When we plug this in at test time to compute the predictive mean, we get

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \mathbf{x} = \sum_{n=1}^{N} \alpha_n \mathbf{x}_n^\top \mathbf{x} \tag{17.114}$$

We can then use the kernel trick to rewrite this as

$$f(\mathbf{x}; \mathbf{w}) = \sum_{n=1}^{N} \alpha_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) \tag{17.115}$$

where

$$\boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{y} \tag{17.116}$$

In other words,

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{k}^\top (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{y} \tag{17.117}$$

where $\mathbf{k} = [\mathcal{K}(\mathbf{x}, \mathbf{x}_1), \ldots, \mathcal{K}(\mathbf{x}, \mathbf{x}_N)]$. This is called **kernel ridge regression**. (See Sec. 17.7.5 for more discussion of this method.)

The trouble with the above approach is that the solution vector $\boldsymbol{\alpha}$ is not sparse, so predictions at test time will take $O(N)$ time. We discuss a solution to this in Sec. 17.5.10.

17.5.10 SVMs for regression

Consider the following ℓ_2-regularized ERM problem:

$$J(\mathbf{w}, \lambda) = \lambda ||\mathbf{w}||^2 + \sum_{n=1}^{N} \ell(\tilde{y}_n, \hat{y}_n) \tag{17.118}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\hat{y}_n = \mathbf{w}^\top \mathbf{x}_n + w_0$. If we use the quadratic loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$, where $y, \hat{y} \in \mathbb{R}$, we recover ridge regression (Sec. 11.3). If we then apply the kernel trick, we recover kernel ridge regression (Sec. 17.5.9).

The problem with kernel ridge regression is that the solution depends on all N training points, which makes it computationally intractable. However, by changing the loss function, we can make the optimal set of basis function coefficients, $\boldsymbol{\alpha}^*$, be sparse, as we show below.

In particular, consider the following variant of the Huber loss function (Sec. 5.1.5.3) called the **epsilon insensitive loss function**:

$$L_\epsilon(y, \hat{y}) \triangleq \begin{cases} 0 & \text{if } |y - \hat{y}| < \epsilon \\ |y - \hat{y}| - \epsilon & \text{otherwise} \end{cases} \tag{17.119}$$

This means that any point lying inside an ϵ-**tube** around the prediction is not penalized, as in Fig. 17.20.

The corresponding objective function is usually written in the following form

$$J = \frac{1}{2}||\mathbf{w}||^2 + C \sum_{n=1}^{N} L_\epsilon(\tilde{y}_n, \hat{y}_n) \tag{17.120}$$

where $\hat{y}_n = f(\mathbf{x}_n) = \mathbf{w}^\top \mathbf{x}_n + w_0$ and $C = 1/\lambda$ is a regularization constant. This objective is convex and unconstrained, but not differentiable, because of the absolute value function in the loss term. As in Sec. 11.5.9, where we discussed the lasso problem, there are several possible algorithms we could use. One popular approach is to formulate the problem as a constrained optimization problem. In particular, we introduce **slack variables** to represent the degree to which each point lies outside the tube:

$$\tilde{y}_n \leq f(\mathbf{x}_n) + \epsilon + \xi_n^+ \tag{17.121}$$

$$\tilde{y}_n \geq f(\mathbf{x}_n) - \epsilon - \xi_n^- \tag{17.122}$$

Given this, we can rewrite the objective as follows:

$$J = \frac{1}{2}||\mathbf{w}||^2 + C \sum_{n=1}^{N} (\xi_n^+ + \xi_n^-) \tag{17.123}$$

This is a quadratic function of $\mathbf{w}$, and must be minimized subject to the linear constraints in Equations 17.121-17.122, as well as the positivity constraints $\xi_n^+ \geq 0$ and $\xi_n^- \geq 0$. This is a standard quadratic program in $2N + D + 1$ variables.

By forming the Lagrangian and optimizing, as we did above, one can show that the optimal solution has the following form

$$\hat{\mathbf{w}} = \sum_n \alpha_n \mathbf{x}_n \tag{17.124}$$

where $\alpha_n \geq 0$ are the dual variables. (See e.g., [SS02] for details.) Fortunately, the $\boldsymbol{\alpha}$ vector is sparse, meaning that many of its entries are equal to 0. This is because the loss doesn't care about errors which are smaller than ϵ. The degree of sparsity is controlled by C and ϵ.

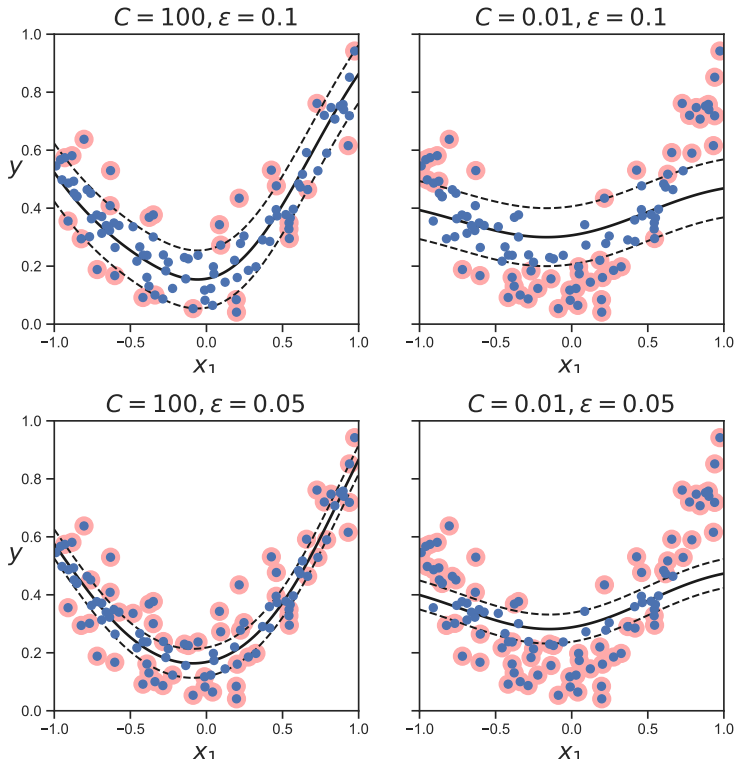


Figure 17.21: Illustration of support vector regression. Adapted from Figure 5.11 of [Gér19]. Generated by code at figures.probml.ai/book1/17.21.

The $\mathbf{x}_n$ for which $\alpha_n > 0$ are called the **support vectors**; these are points for which the errors lie on or outside the ϵ tube. These are the only training examples we need to keep for prediction at test time, since

$$f(\mathbf{x}) = \hat{w}_0 + \hat{\mathbf{w}}^\top \mathbf{x} = \hat{w}_0 + \sum_{n:\alpha_n > 0} \alpha_n \mathbf{x}_n^\top \mathbf{x} \tag{17.125}$$

Finally, we can use the kernel trick to get

$$f(\mathbf{x}) = \hat{w}_0 + \sum_{n:\alpha_n > 0} \alpha_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) \tag{17.126}$$

This overall technique is called **support vector machine regression** or **SVM regression** for short, and was first proposed in [VGS97].

In Fig. 17.21, we give an example where we use an RBF kernel with $\gamma = 1$. When C is small, the model is heavily regularized; when C is large, the model is less regularized and can fit the data better. We also see that when ϵ is small, the tube is smaller, so there are more support vectors.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

17.6 Sparse vector machines

GPs are very flexible models, but incur an $O(N)$ time cost at prediction time, which can be prohibitive. SVMs solve that problem by estimating a sparse weight vector. However, SVMs do not give calibrated probabilistic outputs.

We can get the best of both worlds by using parametric models, where the feature vector is defined using basis functions centered on each of the training points, as follows:

$$\boldsymbol{\phi}(\mathbf{x}) = [\mathcal{K}(\mathbf{x}, \mathbf{x}_1), \ldots, \mathcal{K}(\mathbf{x}, \mathbf{x}_N)] \tag{17.127}$$

where $\mathcal{K}$ is any similarity kernel, not necessarily a Mercer kernel. Eq. (17.127) maps $\mathbf{x} \in \mathcal{X}$ into $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^N$. We can plug this new feature vector into any discriminative model, such as logistic regression. Since we have $D = N$ parameters, we need to use some kind of regularization, to prevent overfitting. If we fit such a model using ℓ_2 regularization (which we will call **L2VM**, for ℓ_2-vector machine), the result often has good predictive performance, but the weight vector $\mathbf{w}$ will be dense, and will depend on all N training points. A natural solution is to impose a sparsity-promoting prior on $\mathbf{w}$, so that not all the exemplars need to be kept. We call such methods **sparse vector machines**.

17.6.1 Relevance vector machines (RVMs)

The simplest way to ensure $\mathbf{w}$ is sparse is to use ℓ_1 regularization, as in Sec. 11.5. We call this **L1VM** or **Laplace vector machine**, since this approach is equivalent to using MAP estimation with a Laplace prior for $\mathbf{w}$.

However, sometimes ℓ_1 regularization does not result in a sufficient level of sparsity for a given level of accuracy. An alternative approach is based on the use of **ARD** or **automatic relevancy determination**, which uses type II maximum likelihood (aka empirical Bayes) to estimate a sparse weight vector. We explain the basic idea (for a linear regression model) in Sec. 11.6.3. For a linear classifier, we can use an approximate EM algorithm, in which we use the Laplace approximation in the E step to compute $p(\mathbf{w}|\mathcal{D}, \boldsymbol{\alpha})$, and then we optimize the marginal likelihood wrt $\boldsymbol{\alpha}$ in the M step (see [Bis06, Sec 7.2] for details). If we apply this technique to a feature vector defined in terms of kernels, as in Eq. (17.127), we get a method called the **relevance vector machine** or **RVM** [Tip01; TF03].

17.6.2 Comparison of sparse and dense kernel methods

In Fig. 17.22, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a binary classification problem in 2d. For simplicity, λ was chosen by hand for L2VM and L1VM; for RVMs, the parameters are estimated using empirical Bayes; and for the SVM, we use CV to pick $C = 1/\lambda$, since SVM performance is very sensitive to this parameter (see Sec. 17.5.8). We see that all the methods give similar performance. However, RVM is the sparsest (and hence fastest at test time), then L1VM, and then SVM. RVM is also the fastest to train. This is despite the fact that the RVM code is in Matlab and the SVM code is in the C language. The reason is that CV (needed for the SVM) requires fitting multiple models, which is slow, whereas empirical Bayes only has to fit a single model.

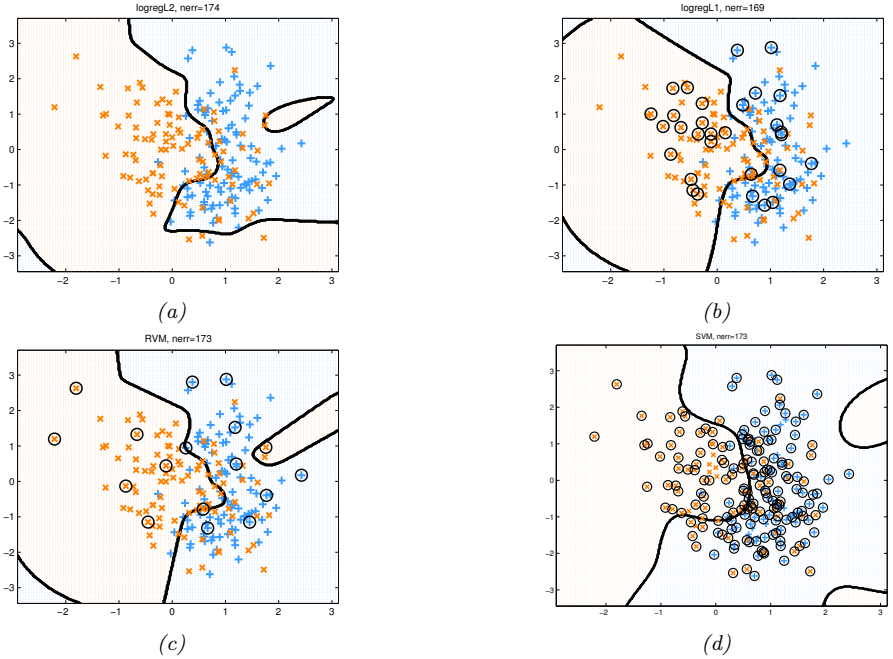


Figure 17.22: Example of non-linear binary classification using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM with $\lambda = 5$. (b) L1VM with $\lambda = 1$. (c) RVM. (d) SVM with $C = 1/\lambda$ chosen by cross validation. Black circles denote the support vectors. Generated by code at figures.probml.ai/book1/17.22.

Method	Opt. $\mathbf{w}$	Opt. kernel	Sparse	Prob.	Multiclass	Non-Mercer	Section
SVM	Convex	CV	Yes	No	Indirectly	No	17.5
L2VM	Convex	EB	No	Yes	Yes	Yes	17.6.1
L1VM	Convex	CV	Yes	Yes	Yes	Yes	17.6.1
RVM	Not convex	EB	Yes	Yes	Yes	Yes	17.6.1
GP	N/A	EB	No	Yes	Yes	No	17.3.7

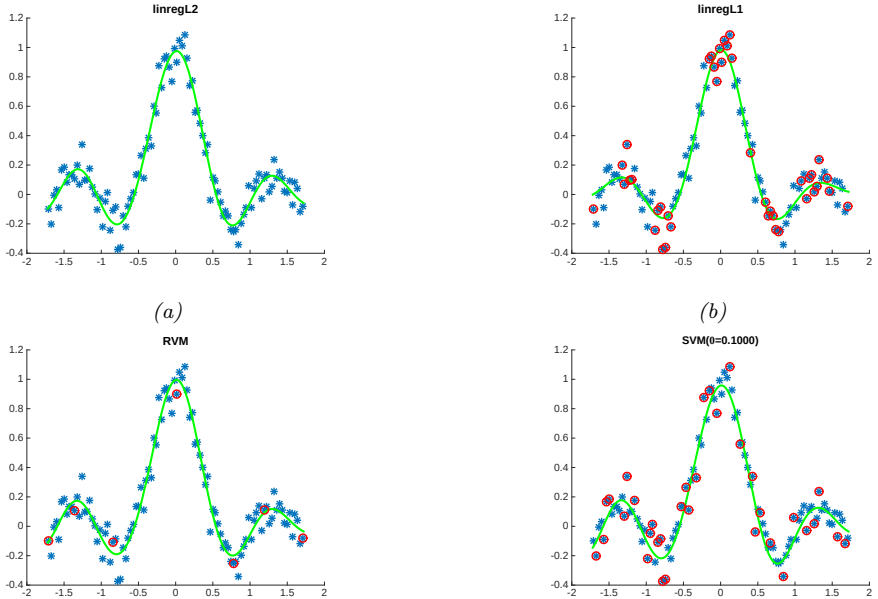
Table 17.1: Comparison of various kernel based classifiers. EB = empirical Bayes, CV = cross validation. See text for details.

In Fig. 17.23, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a 1d regression problem. Again, we see that predictions are quite similar, but RVM is the sparsest, then L1VM, then SVM. This is further illustrated in Fig. 17.24.

We have now discussed several different methods for classification and regression based on kernels, namely GPs, SVMs, L2VMs1, L1VMs, and RVMs. We summarize these different methods in Table 17.1. The columns of this table have the following meaning:

- Optimize $\mathbf{w}$: a key question is whether the objective $\mathcal{L}(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{w})$ is convex or not. L2VM, L1VM and SVMs have convex objectives. RVMs do not. GPs are Bayesian methods

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)* *(c)* *(d)*

Figure 17.23: Example of kernel based regression on the noisy sinc function using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM with $\lambda = 0.5$. (b) L1VM with $\lambda = 0.5$. (c) RVM. (d) SVM regression with $C = 1/\lambda$. and $\epsilon = 0.1$ (the default for SVMlight). Red circles denote the retained training exemplars. Generated by code at figures.probml.ai/book1/17.23.

that integrate out the weights $\mathbf{w}$.

- Optimize kernel: all the methods require that we "tune" the kernel parameters, such as the bandwidth of the RBF kernel, as well as the level of regularization. For methods based on Gaussian priors, including L2VM, RVMs and GPs, we can use efficient gradient based optimizers to maximize the marginal likelihood. For SVMs and L1VMs, we must use cross validation, which is slower (see Sec. 17.5.8).
- Sparse: L1VM, RVMs and SVMs are sparse kernel methods, in that they only use a subset of the training examples. GPs and L2VM are not sparse: they use all the training examples. The principle advantage of sparsity is that prediction at test time is usually faster. However, this usually results in overconfidence in the predictions.
- Probabilistic: All the methods except for SVMs produce probabilistic output of the form $p(y|\mathbf{x})$. SVMs produce a "confidence" value that can be converted to a probability, but such probabilities are usually very poorly calibrated (see Sec. 17.5.5).
- Multiclass: All the methods except for SVMs naturally work in the multiclass setting, by using a categorical distribution instead of a Bernoulli. The SVM can be made into a multiclass classifier,

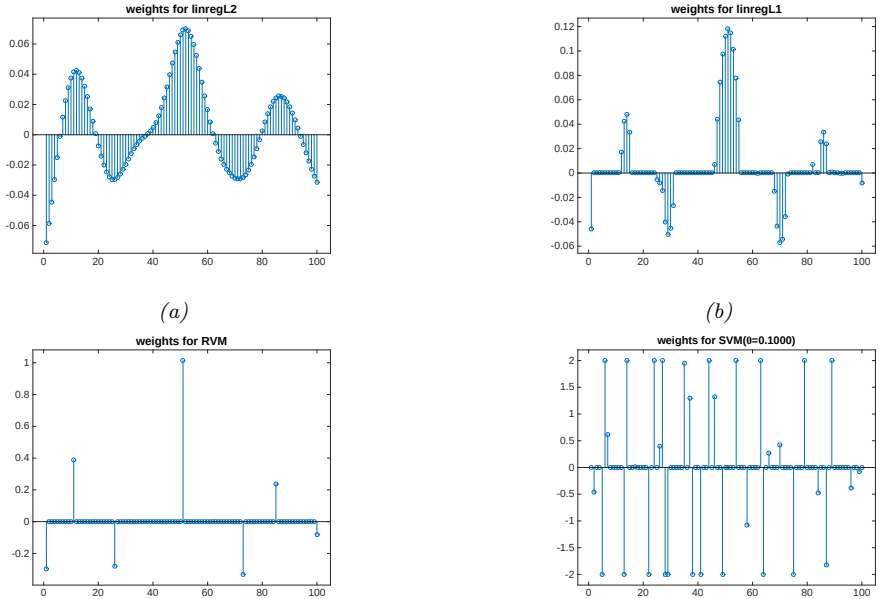


Figure 17.24: Coefficient vectors of length $N = 100$ for the models in Fig. 17.23. Generated by code at figures.probml.ai/book1/17.24.

but there are various difficulties with this approach, as discussed in Sec. 17.5.7.

- Mercer kernel: SVMs and GPs require that the kernel is positive definite; the other techniques do not, since the kernel function in Eq. (17.127) can be an arbitrary function of two inputs.

17.7 Optimizing in function space *

We can view GPs as a way of performing Bayesian inference over the space of functions $f : \mathcal{X} \to \mathbb{R}$. We might also be interested in MAP estimation in the space of functions, or, more generally, penalized or regularized point estimation of functions. In this section, we introduce the mathematical machinery needed to perform this task. We will see that this generalizes the ideas behind SVMs. More details can be found in [SS01; STC04; Dau04; SG12; Kan+18].

17.7.1 Functional analysis

In this section, we introduce some basis concepts from the area of **functional analysis**. This extends ideas from Sec. 7.1.2 to the case of *vector spaces of functions*.

Every element of a (real-valued) function space $\mathcal{F}$ is a function of the form $f : \mathcal{X} \to \mathbb{R}$, and elements (functions) can be added and scalar multiplied as if they were vectors. That is, if $f \in \mathcal{F}$ and $g \in \mathcal{F}$, then $\alpha f + \beta g \in \mathcal{F}$ for $\alpha, \beta \in \mathbb{R}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can also define an **inner product** for $\mathcal{F}$, which is a mapping $\langle f, g\rangle \in \mathbb{R}$ which satisfies the following:

$$\langle \alpha f_1 + \beta f_2, g\rangle = \alpha\langle f_1, g\rangle + \beta\langle f_2, g\rangle \tag{17.128}$$
$$\langle f, g\rangle = \langle g, f\rangle \tag{17.129}$$
$$\langle f, f\rangle \geq 0 \tag{17.130}$$
$$\langle f, f\rangle = 0 \text{ iff } f(\mathbf{x}) = 0 \text{ for all } \mathbf{x} \in \mathcal{X} \tag{17.131}$$

We define the norm of a function using

$$||f|| \triangleq \sqrt{\langle f, f\rangle} \tag{17.132}$$

We say that $f, g \in \mathcal{F}$ are orthogonal to each other if $\langle f, g\rangle = 0$. We say that a set of functiions $\{f_i, i \in \mathcal{I}\}$ is an orthonormal system if the norm of every f_i is 1, and the $\{f_i\}$ are orthogonal. Hence the $\{f_i\}$ are linearly independent. An orthonormal system is called an **orthonormal basis** if there no other (non-zero) $f \in \mathcal{F}$ that is orthogonal to all the $\{f_i\}$.

17.7.2 Hilbert space

A function space $\mathcal{H}$ with an inner product operator is called a **Hilbert space**. (We aso require that the function space be complete, which means that every Cauchy sequence of functions $f_i \in \mathcal{H}$ has a limit that is also in $\mathcal{H}$.)

The most common Hilbert space is the space known as L^2. To define this, we need to specify a **measure** μ on the input space $\mathcal{X}$; this is a function that assigns any (suitable) subset A of $\mathcal{X}$ to a positive number, such as its volume. This can be defined in terms of the density function $w : \mathcal{X} \to \mathbb{R}$, as follows:

$$\mu(A) = \int_A w(x)dx \tag{17.133}$$

Thus we have $\mu(dx) = w(x)dx$. We can now define $L^2(\mathcal{X}, \mu)$ to be the space of functions $f : \mathcal{X} \to \mathbb{R}$ that satisfy

$$\int_{\mathcal{X}} f(x)^2 w(x)dx < \infty \tag{17.134}$$

This is known as the set of **square-integrable functions**. This space has an inner product defined by

$$\langle f, g\rangle = \int_{\mathcal{X}} f(x)g(x)w(x)dx \tag{17.135}$$

A Hilbert space with an orthonormal basis $\{f_i\}$ is similar to a Euclidean vector space. In particular, any element $f \in \mathcal{H}$ can be written as a unique linear combination of basis vectors:

$$f = \sum_i \langle f, f_i\rangle f_i \tag{17.136}$$

17.7.3 Reproducing Kernel Hilbert Space

Let $\mathcal{H}$ be the space $L^2(\mathcal{X})$ of functions $\mathcal{X} \to \mathbb{R}$. For each $x \in \mathcal{X}$, let $\delta_x : \mathcal{H} \to \mathbb{R}$ be the **evaluation functional** that takes as input a function $f \in \mathcal{H}$ and evaluates it at x, i.e., $\delta_x(f) = f(x)$. We say that a Hilbert space $\mathcal{H}$ is a **Reproducing Kernel Hilbert Space** or **RKHS** if for each δ_x there is a unique element $\mathcal{K}_x \in \mathcal{H}$ such that for every $f \in \mathcal{H}$

$$\delta_x f = f(x) = \langle f, \mathcal{K}_x \rangle \tag{17.137}$$

Now consider the function $f = \mathcal{K}_y \in \mathcal{H}$ evaluated at x. We have

$$\delta_x \mathcal{K}_y = \mathcal{K}_y(x) = \langle \mathcal{K}_y, \mathcal{K}_x \rangle \triangleq \mathcal{K}(x, y) \tag{17.138}$$

This is called the **reproducing property** and $\mathcal{K}(x, y)$ is called the **RKHS kernel**. Clearly the kernel function is symmetric. One can also show that it is positive semidefinite, in the sense that we can represent it as

$$\mathcal{K}(x, y) = \sum_{j \geq 1} \lambda_j \psi_j(x) \psi_j(y) \tag{17.139}$$

where λ_j is a countable sequence of non-negative numbers decreasing to 0, and the ψ_j are orthonormal functions in $L^2(\mathcal{X})$. This result is known as **Mercer's theorem**.

Mercer's theorem tells us that every RKHS has a corresponding positive definite kernel function. The converse of this is known as the **Moore-Aronszajn theorem**. More precisely, this says that for every psd kernel function $\mathcal{K} : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, there exists a unique RKHS of functions $f : \mathcal{X} \to \mathbb{R}$ with reproducing kernel $\mathcal{K}$.

17.7.4 Representer theorem

In this section, we consider the application of RKHS to the problem of (regularized) empirical risk minimization. In particular, we state and prove the **representer theorem** [KW70; SHS01], which generalizes the result of Eq. (17.102).

Theorem 17.7.1 (Representer theorem)**.** *Consider the following regularized empirical risk minimization problem:*

$$f^* = \operatorname*{argmin}_{f \in \mathcal{H}_\mathcal{K}} \mathcal{L}(f) \tag{17.140}$$

$$\mathcal{L}(f) = \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(x_n)) + R(\|f\|) \tag{17.141}$$

where $\mathcal{H}_\mathcal{K}$ is an RKHS with kernel $\mathcal{K}$, $L(y, \hat{y}) \in \mathbb{R}$ is a loss function, $R(c) \in \mathbb{R}$ is a strictly monotonic penalty function, and

$$||f||_\mathcal{H} = \sqrt{\langle f, f \rangle_\mathcal{H}} \tag{17.142}$$

is the norm of the function. Then we have

$$f^*(x) = \sum_{i=1}^{N} \alpha_i \mathcal{K}(x, x_i) \tag{17.143}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\alpha_i \in \mathbb{R}$ are some coefficients that depend on the training data $\{(x_n, y_n)\}$.

Proof. Let us define the feature map $\Phi(x) = \mathcal{K}(\cdot, x)$. Given any $x_1, \ldots, x_N$, we can use orthogonal projection to decompose any $f \in \mathcal{H}_\mathcal{K}$ into a sum of two functions, one lying in the span of $\{\Phi(x_1), \ldots, \Phi(x_N)\}$, and the other lying in the orthogonal complement; let this latter function be denoted by $v(x)$. Thus

$$f = \sum_{i=1}^{N} \alpha_i \Phi(x_i) + v \tag{17.144}$$

where $\langle v, \Phi(x_i)\rangle = 0$ for all i. Using this result, together with the reproducing property, we have

$$f(x_j) = \langle \sum_{i=1}^{N} \alpha_i \Phi(x_i) + v, \Phi(x_j)\rangle = \sum_{i=1}^{N} \alpha_i \langle \Phi(x_i), \Phi(x_j)\rangle \tag{17.145}$$

Since f is independent of v, we see that the first term in $\mathcal{L}(f)$ is independent of v. Now let us consider the second (regularization) term in $\mathcal{L}(f)$. Since v is orthogonal to $\sum_{i=1}^{N} \alpha_i \Phi(x_i)$ and R is strictly monotonic, we have

$$R(\|f\|) = R\left(\| \sum_{i=1}^{N} \alpha_i \Phi(x_i) + v\| \right) \tag{17.146}$$

$$= R\left(\sqrt{\| \sum_{i=1}^{N} \alpha_i \Phi(x_i)\|^2 + \|v\|^2} \right) \tag{17.147}$$

$$\geq R\left(\| \sum_{i=1}^{N} \alpha_i \Phi(x_i)\| \right) \tag{17.148}$$

Thus we can minimize the second term of $\mathcal{L}(f)$ by setting $v = 0$. Hence we can minimize the overall regularized loss by using a solution of the form

$$f^*(\cdot) = \sum_{i=1}^{N} \alpha_i \Phi(x_i) = \sum_{i=1}^{N} \alpha_i \mathcal{K}(\cdot, x_i) \tag{17.149}$$

□

We can generalize this to allow some parts of the function (e.g., the bias term) to not be penalized. In particular, suppose $f \in \mathcal{F}$ can be written as $f(\mathbf{x}) = h(\mathbf{x}) + h_0(\mathbf{x})$, where h is an RKHS $\mathcal{H}$ with kernel $\mathcal{K}$, and h_0 is another linear space of real-valued functions $\mathcal{H}_0$ on $\mathcal{X}$. We define our modified objective to be

$$f^* = \underset{f \in \mathcal{H} \oplus \mathcal{H}_0}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, f(\mathbf{x}_n)) + \lambda ||h||_\mathcal{H}^2 \tag{17.150}$$

where the penality is not applied to h_0. One can show that the optimal solution has the form

$$f^*(\mathbf{x}) = \sum_{n=1}^{N} \alpha_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) + \sum_{m=1}^{M} \eta_m q_m(\mathbf{x}) \tag{17.151}$$

where $\{q_1, \ldots, q_M\}$ is a basis of $\mathcal{H}_0$. (See e.g., [Kro+19, p231] for the proof, which is similar to the one we gave above.)

We give an example in Sec. 17.7.5.

17.7.5 Kernel ridge regression revisited

Consider the function space version of ridge regression:

$$\min_{f \in \mathcal{H} \oplus \mathcal{H}_0} \frac{1}{N} \sum_{n=1}^{N} (y_n - f(\mathbf{x}_n))^2 + \lambda ||h||_{\mathcal{H}}^2 \tag{17.152}$$

where $f(\mathbf{x}) = h(\mathbf{x}) + h_0(\mathbf{x})$. From the representer theorem, this is equivalent to solving

$$\min_{\boldsymbol{\alpha} \in \mathbb{R}^N, \boldsymbol{\eta} \in \mathbb{R}^M} \frac{1}{N} ||\mathbf{y} - (\mathbf{K}\boldsymbol{\alpha} + \mathbf{Q}\boldsymbol{\eta})||^2 + \lambda \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha} \tag{17.153}$$

where $\mathbf{K}$ is $N \times N$ Gram matrix, and $\mathbf{Q}$ is the $N \times M$ matrix with entries $[q_m(\mathbf{x}_n)]$.

This is a convex objective which can be solved by differentating wrt $\boldsymbol{\alpha}$ and $\boldsymbol{\eta}$ and equating to 0. This gives the following system of $N + M$ linear equations:

$$\begin{pmatrix} \mathbf{K}\mathbf{K}^\top + N\lambda \mathbf{K} & \mathbf{K}\mathbf{Q} \\ \mathbf{Q}^\top \mathbf{K}^\top & \mathbf{Q}^\top \mathbf{Q} \end{pmatrix} \begin{pmatrix} \boldsymbol{\alpha} \\ \boldsymbol{\eta} \end{pmatrix} = \begin{pmatrix} \mathbf{K}^\top \\ \mathbf{Q}^\top \end{pmatrix} \mathbf{y} \tag{17.154}$$

As long as $\mathbf{Q}$ is full column rank, the minimizing function is unique. For example, if we set $\mathcal{H}_0$ to be the space of constant functions, we can span it using $q_1(\mathbf{x}) = 1$, so $\mathbf{Q} = \mathbf{1}_N$. Alternatively, we can ignore the offset term, and use $q_1(\mathbf{x}) = 0$. Then we can drop the $\mathbf{Q}$ term to get

$$(\mathbf{K}\mathbf{K}^\top + N\lambda \mathbf{K})\boldsymbol{\alpha} = \mathbf{K}^\top \mathbf{y} \tag{17.155}$$

$$\mathbf{K}(\mathbf{K} + N\lambda \mathbf{I})\boldsymbol{\alpha} = \mathbf{K}\mathbf{y} \tag{17.156}$$

$$\boldsymbol{\alpha} = (\mathbf{K} + N\lambda \mathbf{I})^{-1} \mathbf{y} \tag{17.157}$$

This recovers the kernel ridge regression result discussed in Sec. 17.5.9.

17.8 Exercises

Exercise 17.1 [Fitting an SVM classifier by hand]

(Source: Jaakkola.) Consider a dataset with 2 points in 1d: $(x_1 = 0, y_1 = -1)$ and $(x_2 = \sqrt{2}, y_2 = 1)$. Consider mapping each point to 3d using the feature vector $\boldsymbol{\phi}(x) = [1, \sqrt{2}x, x^2]^T$. (This is equivalent to using a second order polynomial kernel.) The max margin classifier has the form

$$\min ||\mathbf{w}||^2 \quad \text{s.t.} \tag{17.158}$$

$$y_1(\mathbf{w}^T \phi(\mathbf{x}_1) + w_0) \geq 1 \tag{17.159}$$

$$y_2(\mathbf{w}^T \phi(\mathbf{x}_2) + w_0) \geq 1 \tag{17.160}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

a. Write down a vector that is parallel to the optimal vector $\mathbf{w}$. Hint: recall from Figure 7.8 (12Apr10 version) that $\mathbf{w}$ is perpendicular to the decision boundary between the two points in the 3d feature space.

b. What is the value of the margin that is achieved by this $\mathbf{w}$? Hint: recall that the margin is the distance from each support vector to the decision boundary. Hint 2: think about the geometry of 2 points in space, with a line separating one from the other.

c. Solve for $\mathbf{w}$, using the fact the margin is equal to $1/||\mathbf{w}||$.

d. Solve for w_0 using your value for $\mathbf{w}$ and Equations 17.158 to 17.160. Hint: the points will be on the decision boundary, so the inequalities will be tight.

e. Write down the form of the discriminant function $f(x) = w_0 + \mathbf{w}^T \boldsymbol{\phi}(x)$ as an explicit function of x.

18 Trees, forests, bagging and boosting

18.1 Classification and regression trees (CART)

Classification and regression trees or **CART** models [BFO84], also called **decision trees** [Qui86; Qui93], are defined by recursively partitioning the input space, and defining a local model in each resulting region of input space. The overall model can be represented by a tree, with one leaf per region, as we explain below.

18.1.1 Model definition

We start by considering regression trees, where all inputs are real-valued. The tree consists of a set of nested decision rules. At each node i, the feature dimension d_i of the input vector $\mathbf{x}$ is compared to a threshold value t_i, and the input is then passed down to the left or right branch, depending on whether it is above or below threshold. At the leaves of the tree, the model specifies the predicted output for any input that falls into that part of the input space.

For example, consider the regression tree in Fig. 18.1(a). The first node asks if x_1 is less than some threshold t_1. If yes, we then ask if x_2 is less than some other threshold t_2. If yes, we enter the bottom left leaf node. This corresponds to the region of space defined by

$$R_1 = \{\mathbf{x} : x_1 \leq t_1, x_2 \leq t_2\} \tag{18.1}$$

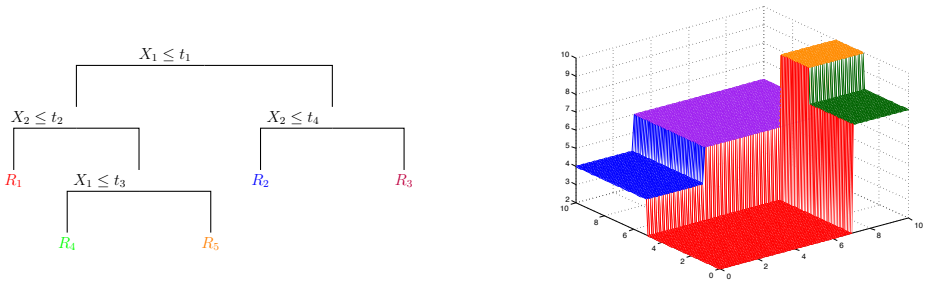
We can associate this region with the predicted output computing using Other branches of the tree define different regions in terms of **axis parallel splits**. The overall result is that we partition the 2d input space into 5 regions, as shown in Figure 18.1(b).[1] We can now associate a mean response with each of these regions, resulting in the piecewise constant surface shown in Figure 18.1(b). For example, the output for region 1 can be estimated using

$$w_1 = \frac{\sum_{n=1}^{N} y_n \mathbb{I}\left(\mathbf{x}_n \in R_1\right)}{\sum_{n=1}^{N} \mathbb{I}\left(\mathbf{x}_n \in R_1\right)} \tag{18.2}$$

Formally, a regression tree can be defined by

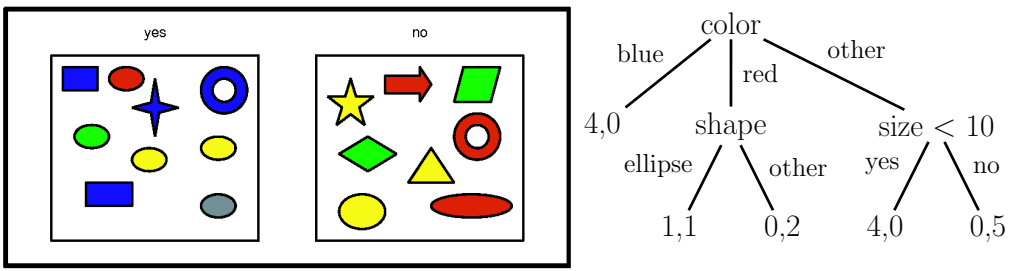
$$f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{j=1}^{J} w_j \mathbb{I}\left(\mathbf{x} \in R_j\right) \tag{18.3}$$

1. By using enough splits (i.e., deep enough trees), we can make a piecewise linear approximationt to decision boundaries with more complex shapes, but it may require a lot of data to fit such a model.



(a) (b)

Figure 18.1: A simple regression tree on two inputs. Adapted from Figure 9.2 of [HTF09]. Generated by code at figures.probml.ai/book1/18.1.



(a) (b)

Figure 18.2: (a) A set of shapes with corresponding binary labels. The features are: color (values "blue", "red", "other"), shape (values "ellipse", "other"), and size (real-valued). (b) A hypothetical classification tree fitted to this data. A leaf labeled as (n_1, n_0) means that there are n_1 positive examples that fall into this partition, and n_0 negative examples.

where R_j is the region specified by the j'th leaf node, w_j is the predicted output for that node, and $\boldsymbol{\theta} = \{(R_j, w_j) : j = 1 : J\}$, where J is the number of nodes. The regions themselves are defined by the feature dimensions that are used in each split, and the corresponding thresholds, on the path from the root to the leaf. For example, in Figure 18.1(a), we have $R_1 = [(d_1 \leq t_1), (d_2 \leq t_2)]$, $R_2 = [(d_1 \leq t_1), (d_2 > t_2), (d_3 \leq t_3)]$, etc. (For categorical inputs, we can define the splits based on comparing feature d_i to each of the possible values for that feature, rather than comparing to a numeric threshold.) We discuss how to learn these regions in Sec. 18.1.2.

For classification problems, the leaves contain a distribution over the class labels, rather than just the mean response. See Fig. 18.2 for an example of a classification tree.

18.1.2 Model fitting

To fit the model, we need to minimize the following loss:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{n=1}^{N} \ell(y_n, f(\mathbf{x}_n; \boldsymbol{\theta})) = \sum_{j=1}^{J} \sum_{\mathbf{x}_n \in R_j} \ell(y_i, w_j) \tag{18.4}$$

Unfortunately, this is not differentiable, because of the need to learn the discrete tree structure. Indeed, finding the optimal partitioning of the data is NP-complete [HR76]. The standard practice is to use a greedy procedure, in which we iteratively grow the tree one node at a time. This approach is used by CART [BFO84], **C4.5** [Qui93], and **ID3** [Qui86], which are three popular implementations of the method.

The idea is as follows. Suppose we are at node i, and let $\mathcal{D}_i = \{(\mathbf{x}_n, y_n) \in N_i\}$ be the set of examples that reach this node. Let $\mathcal{D}_i^L(j,t) = \{(\mathbf{x}_n, y_n) \in N_i : y_{n,j} \leq t\}$ and $\mathcal{D}_i^R(j,t) = \{(\mathbf{x}_n, y_n) \in N_i : y_{n,j} > t\}$ be a partition of these examples into left and right subtrees. (For categorical features, we use $\mathcal{D}_i^L(j,t) = \{(\mathbf{x}_n, y_n) \in N_i : y_{n,j} = t\}$ and $\mathcal{D}_i^R(j,t) = \{(\mathbf{x}_n, y_n) \in N_i : y_{n,j} \neq t\}$.) We choose the best feature j_i to split on, and the best value for that feature, t_i, as follows:

$$(j_i, t_i) = \arg \min_{j \in \{1,\ldots,D\}} \min_{t \in \mathcal{T}_j} \frac{|\mathcal{D}_i^L(j,t)|}{|\mathcal{D}|} c(\mathcal{D}_i^L(j,t)) + \frac{|\mathcal{D}_i^R(j,t)|}{|\mathcal{D}|} c(\mathcal{D}_i^R(j,t)) \tag{18.5}$$

where the cost function $c()$ is defined below.

The set of possible thresholds $\mathcal{T}_j$ for feature j can be obtained by sorting the unique values of $\{x_{nj}\}$. For example, if feature 1 has the values $\{4.5, -12, 72, -12\}$, then we set $\mathcal{T}_1 = \{-12, 4.5, 72\}$.

In the case of categorical inputs, suppose feature j has K_j possible values. Then we check if the feature is equal to each of those values or not. This defines a set of K_j possible binary splits. Alternatively, we could allow for a multi-way split, with K_k branches, as in Fig. 18.2. However, this may cause **data fragmentation**, in which too little data might "fall" into each subtree, resulting in overfitting. Therefore it is more common to use binary splits.

We now discuss the cost function $c(\mathcal{D}_i)$ which is used to evaluate the cost of node i. For regression, we can use the mean squared error

$$\text{cost}(\mathcal{D}_i) = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}_i} (y_n - \overline{y})^2 \tag{18.6}$$

where $\overline{y} = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}_i} y_n$ is the mean of the response variable for examples reaching node i.

For classification, we first compute the empirical distribution over class labels for this node:

$$\hat{\pi}_{ic} = \frac{1}{|\mathcal{D}_i|} \sum_{n \in \mathcal{D}_i} \mathbb{I}(y_n = c) \tag{18.7}$$

Given this, we can then compute the **Gini index**

$$G_i = \sum_{c=1}^{C} \hat{\pi}_{ic}(1 - \hat{\pi}_{ic}) = \sum_c \hat{\pi}_{ic} - \sum_c \hat{\pi}_{ic}^2 = 1 - \sum_c \hat{\pi}_{ic}^2 \tag{18.8}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is the expected error rate. To see this, note that $\hat{\pi}_{ic}$ is the probability a random entry in the leaf belongs to class c, and $1 - \hat{\pi}_{ic}$ is the probability it would be misclassified.

Alternatively we can define cost as the entropy or **deviance** of the node:

$$H_i = \mathbb{H}(\hat{\boldsymbol{\pi}}_i) = -\sum_{c=1}^{C} \hat{\pi}_{ic} \log \hat{\pi}_{ic} \tag{18.9}$$

A node that is **pure** (i.e., only has examples of one class) will have 0 entropy.

Given one of the above cost functions, we can use Eq. (18.5) to pick the best feature, and best threshold at each node. We then partition the data, and call the fitting algorithm recursively on each subset of the data.

18.1.3 Regularization

If we let the tree become deep enough, it can achieve 0 error on the training set (assuming no label noise), by partioning the input space into sufficiently small regions where the output is constant. However, this will typically result in overfitting. To prevent this, there are two main approaches. The first is to stop the tree growing process according to some heuristic, such as having too few examples at a node, or reaching a maximum depth. The second approach is to grow the tree to its maximum depth, where no more splits are possible, and then to **prune** it back, by merging split subtrees back into their parent (see e.g., [BA97b]). This can partially overcome the greedy nature of top-down tree growing. (For example, consider applying the top-down approach to the xor data in Fig. 13.1: the algorithm would never make any splits, since each feature on its own has no predictive power.) However, forward growing and backward pruning is slower than the greedy top-down approach.

18.1.4 Handling missing input features

In general, it is hard for discriminative models, such as neural networks, to handle missing input features, as we discussed in Sec. 1.5.5. However, for trees, there are some simple heuristics that can work well.

The standard heuristic for handling missing inputs in decision trees is to look for a series of "backup" variables, which can induce a similar partition to the chosen variable at any given split; these can be used in case the chosen variable is unobserved at test time. These are called **surrogate splits**. This method finds highly correlated features, and can be thought of as learning a local joint model of the input. This has the advantage over a generative model of not modeling the entire joint distribution of inputs, but it has the disadvantage of being entirely ad hoc. A simpler approach, applicable to categorical variables, is to code "missing" as a new value, and then to treat the data as fully observed.

18.1.5 Pros and cons

Tree models are popular for several reasons:

- They are easy to interpret.
- They can easily handle mixed discrete and continuous inputs.

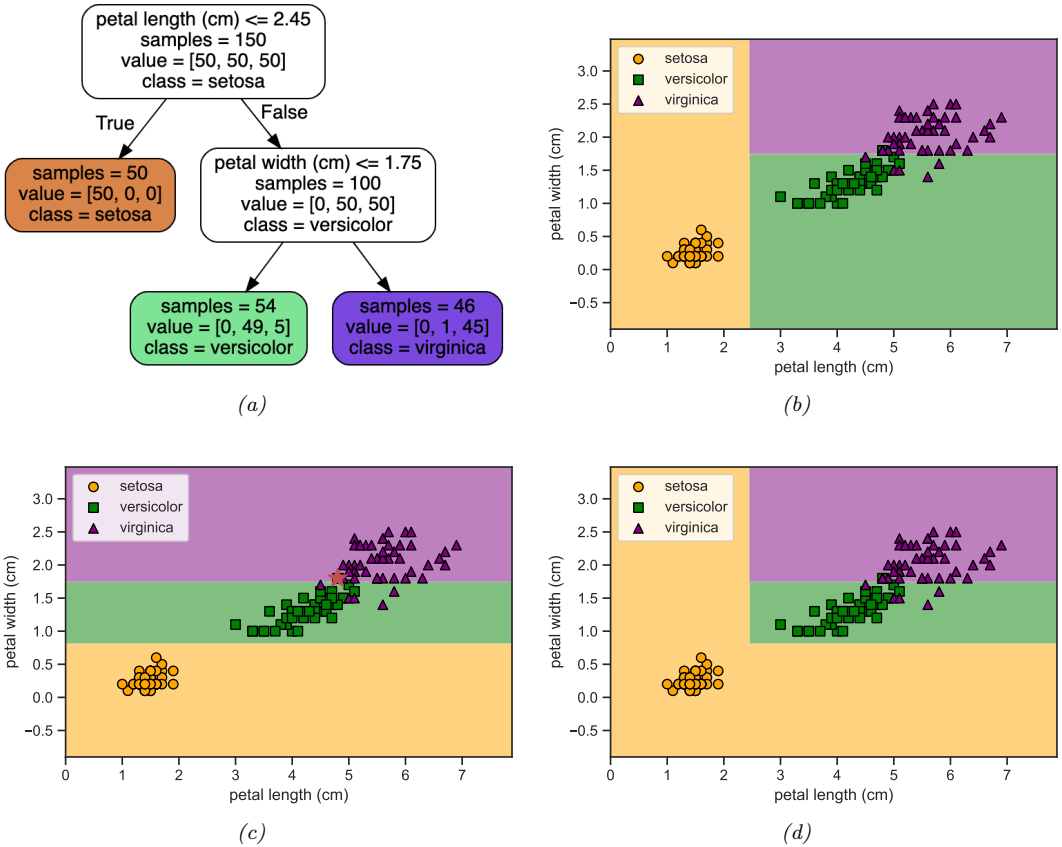


Figure 18.3: (a) A decision tree of depth 2 fit to the iris data, using just the petal length and petal width features. Leaf nodes are color coded according to the majority class. The number of training samples that pass from the root to each node is shown inside each box, as well as how many of these values fall into each class. This can be normalized to get a distribution over class labels for each node. (b) Decision surface induced by (a). (c) Fit to data where we omit a single data point (shown by red star). (d) Ensemble of the two models in (b) and (c). Generated by code at figures.probml.ai/book1/18.3

- They are insensitive to monotone transformations of the inputs (because the split points are based on ranking the data points), so there is no need to standardize the data.
- They perform automatic variable selection.
- They are relatively robust to outliers.
- They are fast to fit, and scale well to large data sets.
- They can handle missing input features.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

However, tree models also have some disadvantages. The primary one is that they do not predict very accurately compared to other kinds of model. This is in part due to the greedy nature of the tree construction algorithm.

A related problem is that trees are **unstable**: small changes to the input data can have large effects on the structure of the tree, due to the hierarchical nature of the tree-growing process, causing errors at the top to affect the rest of the tree. For example, consider the tree in Fig. 18.3b. Omitting even a single data point from the training set can result in a dramatically different decision surface, as shown in Fig. 18.3c, due to the use of axis parallel splits. (Omitting features can also cause instability.) In Sec. 18.3 and Sec. 18.4, we will turn this instability into a virtue.

18.2 Ensemble learning

In Sec. 18.1, we saw that decision trees can be quite unstable, in the sense that their predictions might vary a lot if the training data is perturbed. In other words, decision trees are a high variance estimator. A simple way to reduce variance is to average multiple models. This is called **ensemble learning**. The result model has the form

$$f(y|\mathbf{x}) = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} f_m(y|\mathbf{x}) \tag{18.10}$$

where f_m is the m'th base model. The ensemble will have similar bias to the base models, but lower variance, generally resulting in improved overall performance (see Sec. 4.7.6.3 for details on the bias-variance tradeoff).

Averaging is a sensible way to combine predictions from regression models. For classifiers, it can sometimes be better to take a majority vote of the outputs. (This is sometimes called a **committee method**.) To see why this can help, suppose each base model is a binary classifier with an accuracy of θ, and suppose class 1 is the correct class. Let $Y_m \in \{0, 1\}$ be the prediction for the m'th model, and let $S = \sum_{m=1}^{M} Y_m$ be the number of votes for class 1. We define the final predictor to be the majaority vote, i.e., class 1 if $S > M/2$ and class 0 otherwise. The probability that the ensemble will pick class 1 is

$$p = \Pr(S > M/2) = 1 - B(M/2, M, \theta) \tag{18.11}$$

where $B(x, M, \theta)$ is the cdf of the binomial distribution with parameters M and θ evaluated at x. For $\theta = 0.51$ and $M = 1000$, we get $p = 0.73$ and with $M = 10,000$ we get $p = 0.97$.

The performance of the voting approach is dramatically improved, because we assumed each predictor made independent errors. In practice, their mistakes may be correlated, but as long as we ensemble sufficiently diverse models, we can still come out ahead.

18.2.1 Stacking

An alternative to using an unweighted average or majority vote is to learn how to combine the base models, by using

$$f(y|\mathbf{x}) = \sum_{m \in \mathcal{M}} w_m f_m(y|\mathbf{x}) \tag{18.12}$$

This is called **stacking**, which stands for "stacked generalization" [Wol92]. Note that the combination weights used by stacking need to be trained on a separate dataset, otherwise they would put all their mass on the best performing base model.

18.2.2 Ensembling is not Bayes model averaging

It is worth noting that an ensemble of models is not the same as using Bayes model averaging over models (Sec. 4.6), as pointed out in [Min00]. An ensemble considers a larger hypothesis class of the form

$$p(y|\mathbf{x}, \mathbf{w}, \boldsymbol{\theta}) = \sum_{m \in \mathcal{M}} w_m p(y|\mathbf{x}, \boldsymbol{\theta}_m) \tag{18.13}$$

whereas BMA uses

$$p(y|\mathbf{x}, \mathcal{D}) = \sum_{m \in \mathcal{M}} p(m|\mathcal{D}) p(y|\mathbf{x}, m, \mathcal{D}) \tag{18.14}$$

The key difference is that in the case of BMA, the weights $p(m|\mathcal{D})$ sum to one, and in the limit of infinite data, only a single model will be chosen (namely the MAP model). By contrast, the ensemble weights w_m are arbitrary, and don't collapses in this way to a single model.

18.3 Bagging

In this section, we discuss **bagging** [Bre96], which stands for "bootstrap aggregating". This is a simple form of ensemble learning in which we fit M different base models to different randomly sampled versions of the data; this encourages the different models to make diverse predictions. The datasets are sampled with replacement (a technique known as bootstrap sampling, Sec. 4.7.3), so a given example may appear multiple times, until we have a total of N examples per model (where N is the number of original data points).

The disadvantage of bootstrap is that each base model only sees, on average, 63% of the unique input examples. To see why, note that the chance that a single item will not be selected from a set of size N in any of N draws is $(1 - 1/N)^N$. In the limit of large N, this becomes $e^{-1} \approx 0.37$, which means only $1 - 0.37 = 0.63$ of the data points will be selected.

The 37% of the training instances that are not used by a given base model are called **out-of-bag instances** (oob). We can use the predicted performance of the base model on these oob instances as an estimate of test set performance. This provides a useful alternative to cross validation.

The main advantage of bootstrap is that it prevents the ensemble from relying too much on any individual training example, which enhances robustness and generalization [Gra04]. For example, comparing Fig. 18.3b and Fig. 18.3c, we see that omitting a single example from the training set can have a large impact on the decision tree that we learn (even though the tree growing algorithm is otherwise deterministic). By averaging the predictions from both of these models, we get the more reasonable prediction model in Fig. 18.3d. This advantage generally increases with the size of the ensemble, as shown in Fig. 18.4. (Of course, larger ensembles take more memory and more time.)

Bagging does not always improve performance. In particular, it relies on the base models being unstable estimators, so that omitting some of the data significantly changes the resulting model fit.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

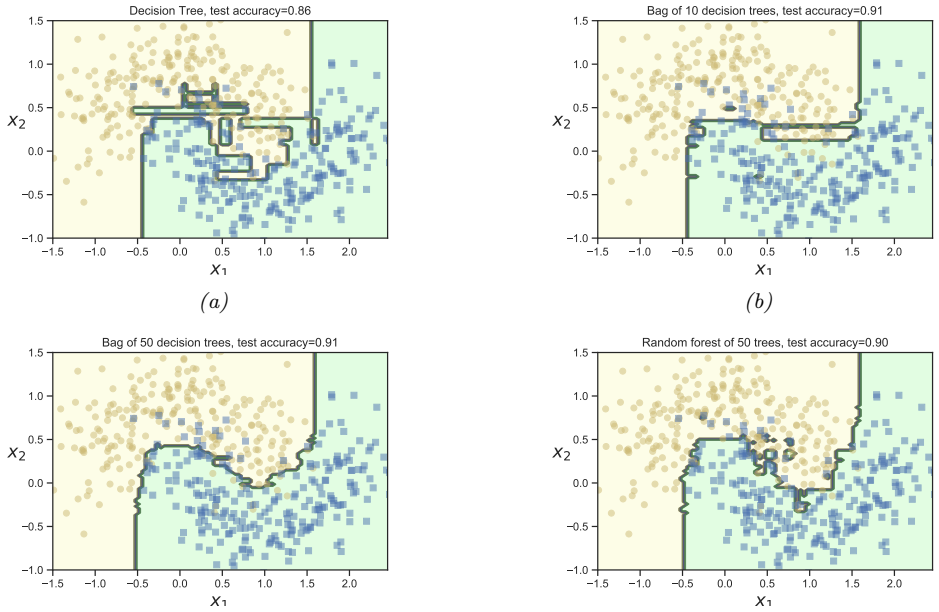


Figure 18.4: (a) A single decision tree. (b-c) Bagging ensemble of 10 and 50 trees. (d) Random forest of 50 trees. Adapted from Figure 7.5 of [Gér19]. Generated by code at figures.probml.ai/book1/18.4.

This is the case for decision trees, but not for other models, such as nearest neighbor classifiers. For neural networks, the story is more mixed. They can be unstable wrt their training set. On the other hand, deep networks will underperform if they only see 63% of the data, so bagged DNNs do not usually work well [NTL20].

18.4 Random forests

Bagging relies on the assumption that re-running the same learning algorithm on different subsets of the data will result in sufficiently diverse base models. The technique known as **random forests** [Bre01] tries to decorrelate the base learners even further by learning trees based on a randomly chosen subset of input variables (at each node of the tree), as well as a randomly chosen subset of data cases. It does this by modifying Eq. (18.5) so the the feature split dimension j is optimized over a random subset of the features, $S_i \subset \{1, \ldots, D\}$. For example, consider the email spam dataset [HTF09, p301]. This dataset contains 4601 email messages, each of which is classified as spam (1) or non-spam (0). The data was open sourced by George Forman from Hewlett-Packard (HP) Labs.

There are 57 quantitative (real-valued) predictors, as follows:

- 48 features corresponding to the percentage of words in the email that match a given word, such as "remove" or "labs".

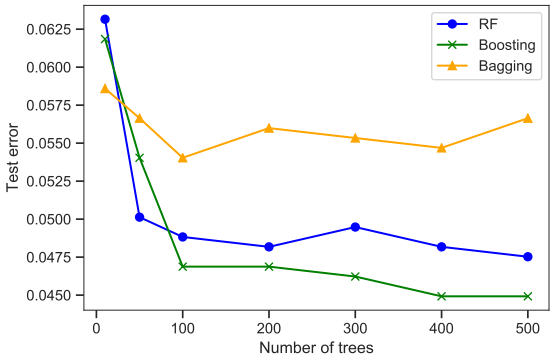


Figure 18.5: Preditive accuracy vs size of tree ensemble for bagging, random forests and gradient boosting with log loss. Adapted from Figure 15.1 of [HTF09]. Generated by code at figures.probml.ai/book1/18.5.

- 6 features corresponding to the percentage of characters in the email that match a given character, namely `; . [ ! $ #`
- 3 features corresponding to the average length, max length, and sum of lengths of uninterrupted sequences of capital letters. (These features are called CAPAVE, CAPMAX and CAPTOT.)

Fig. 18.5 shows that random forests work much better than bagged decision trees, because many imput features are irrelevant. (We also see that a method called "boosting", discussed in Sec. 18.5, works even better; however, this requires sequentially fitting trees, whereas random forests can be fit in parallel.)

18.5 Boosting

Ensembles of trees, whether fit by bagging or the random forest algorithm, corresponding to a model of the form

$$f(\mathbf{x};\boldsymbol{\theta}) = \sum_{m=1}^{M} \beta_m F_m(\mathbf{x};\boldsymbol{\theta}_m) \tag{18.15}$$

where F_m is the m'th tree, and β_m is the corresponding weight, often set to $\beta_m = 1$. We can generalize this by allowing the F_m functions to be general function approximators, such as neural networks, not just trees. The result is called an **additive model** [HTF09]. We can think of this as a linear model with **adaptive basis functions**. The goal, as usual, is to minimize the empirical loss (with an optional regularizer):

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i)) \tag{18.16}$$

Boosting [Sch90; FS96] is an algorithm for sequentially fitting additive models where each F_m is a binary classifier that returns $F_m \in \{-1, +1\}$. In particular, we first F_1 on the original data, and

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

then we weight the data samples by the errors made by F_1, so misclassified examples get more weight. Next we fit F_2 to this weighted data set. We keep repeating this process until we have fit the desired number M of components. (M is a hyper-parameter that controls the complexity of the overall model, and can be chosen by monitoring performance on a validation set, and using early stopping.)

It can be shown that, as long as each F_m has an accuracy that is better than chance (even on the weighted dataset), then the final ensemble of classifiers will have higher accuracy than any given component. That is, if F_m is a **weak learner** (so its accuracy is only slightly better than 50%), then we can boost its performance using the above procedure so that the final f becomes a **strong learner**. (See e.g., [SF12] for more details on the learning theory approach to boosting.)

Note that boosting reduces the bias of the strong learner, by fitting trees that depend on each other, whereas bagging and RF reduce the variance by fitting independent trees. In many cases, boosting can work better. See Fig. 18.5 for an example.

The original boosting algorithm focused on binary classification with a particular loss function that we will explain in Sec. 18.5.3, and was derived from the PAC learning theory framework (see Sec. 5.6.4). In the rest of this section, we focus on a more statistical version of boosting, due to [FHT00; Fri01], which works with arbitrary loss functions, making the method suitable for regression, multi-class classification, ranking, etc. Our presentation is based on [HTF09, ch10] and [BH07], which should be consulted for further details.

18.5.1 Forward stagewise additive modeling

In this section, we discuss **forward stagewise additive modeling**, in which we sequentially optimize the objective in Eq. (18.16) for general (differentiable) loss functions, where f is an additive model as in Equation 18.15. That is, at iteration m, we compute

$$(\beta_m, \boldsymbol{\theta}_m) = \underset{\beta, \boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^{N} \ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \boldsymbol{\theta})) \tag{18.17}$$

We then set

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F(\mathbf{x}_i; \boldsymbol{\theta}_m) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}_i) \tag{18.18}$$

(Note that we do not adjust the parameters of previously added models.) The details on how to perform this optimization step depend on the loss function that we choose, and (in some cases) on the form of the weak learner F, as we discuss below.

18.5.2 Quadratic loss and least squares boosting

Suppose we use squared error loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$. In this case, the i'th term in the objective at step m becomes

$$\ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \boldsymbol{\theta})) = (y_i - f_{m-1}(\mathbf{x}_i) - \beta F(\mathbf{x}_i; \boldsymbol{\theta}))^2 = (r_{im} - \beta F(\mathbf{x}_i; \boldsymbol{\theta}))^2 \tag{18.19}$$

where $r_{im} = y_i - f_{m-1}(\mathbf{x}_i)$ is the residual of the current model on the i'th observation. We can minimize the above objective by simply setting $\beta = 1$, and fitting F to the residual errors. This is called **least squares boosting** [BY03].

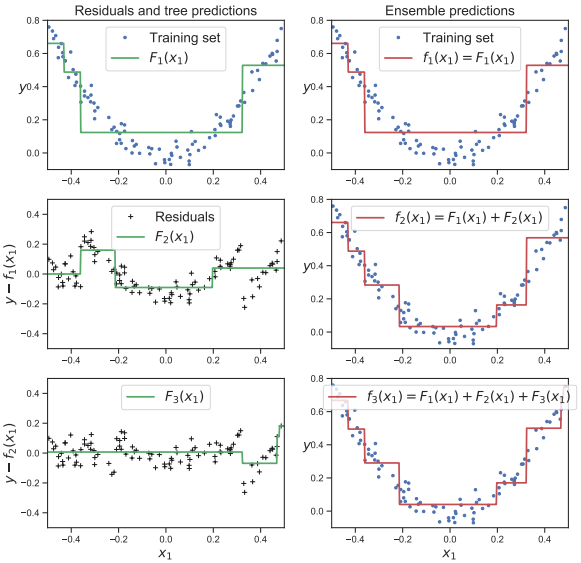


Figure 18.6: Illustration of boosting using a regression tree of depth 2 applied to a 1d dataset. Adapted from Figure 7.9 of [Gér19]. Generated by code at figures.probml.ai/book1/18.6

We give an example of this process in Fig. 18.6, where we use a regression tree of depth 2 as the weak learner. On the left, we show the result of fitting the weak learner to the residuals, and on the right, we show the current strong learner. We see how each new weak learner that is added to the ensemble corrects the errors made by earlier versions of the model.

18.5.3 Exponential loss and AdaBoost

Suppose we are interested in binary classification, i.e., predicting $\tilde{y}_i \in \{-1, +1\}$. Let us assume the weak learner computes

$$p(y = 1|\mathbf{x}) = \frac{e^{F(\mathbf{x})}}{e^{-F(\mathbf{x})} + e^{F(\mathbf{x})}} = \frac{1}{1 + e^{-2F(\mathbf{x})}} \tag{18.20}$$

so $F(\mathbf{x})$ returns half the log odds. We know from Eq. (10.13) that the negative log likelihood is given by

$$\ell(\tilde{y}, F(\mathbf{x})) = \log(1 + e^{-2\tilde{y}F(\mathbf{x})}) \tag{18.21}$$

We can minimize this by ensuring that the **margin** $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$ is as large as possible. We see from Fig. 18.7 that the log loss is a smooth upper bound on the 0-1 loss. We also see that it penalizes negative margins more heavily than positive ones, as desired (since positive margins are already correctly classified).

However, we can also use other loss functions. In this section, we consider the **exponential loss**

$$\ell(\tilde{y}, F(\mathbf{x})) = \exp(-\tilde{y}F(\mathbf{x})) \tag{18.22}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

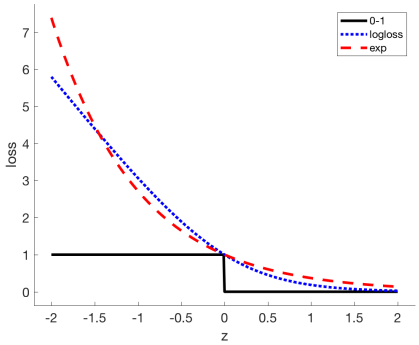


Figure 18.7: Illustration of various loss functions for binary classification. The horizontal axis is the margin $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$, the vertical axis is the loss. The log loss uses log base 2. Generated by code at figures.probml.ai/book1/18.7.

We see from Fig. 18.7 that this is also a smooth upper bound on the 0-1 loss. In the population setting (with infinite sample size), the optimal solution to the exponential loss is the same as for log loss. To see this, we can just set the derivative of the expected loss (for each $\mathbf{x}$) to zero:

$$\frac{\partial}{\partial F(\mathbf{x})}\mathbb{E}\left[e^{-\tilde{y}f(\mathbf{x})}|\mathbf{x}\right] = \frac{\partial}{\partial F(\mathbf{x})}[p(\tilde{y}=1|\mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y}=-1|\mathbf{x})e^{F(\mathbf{x})}] \tag{18.23}$$

$$= -p(\tilde{y}=1|\mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y}=-1|\mathbf{x})e^{F(\mathbf{x})} \tag{18.24}$$

$$= 0 \Rightarrow \frac{p(\tilde{y}=1|\mathbf{x})}{p(\tilde{y}=-1|\mathbf{x})} = e^{2F(\mathbf{x})} \tag{18.25}$$

However, it turns out that the exponential loss is easier to optimize in the boosting setting, as we show below. (We consider the log loss case in Sec. 18.5.4.) We now discuss how to solve for the m'th weak learner, F_m, when we use exponential loss. At step m we have to minimize

$$L_m(F) = \sum_{i=1}^{N} \exp[-\tilde{y}_i(f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i))] = \sum_{i=1}^{N} \omega_{i,m} \exp(-\beta \tilde{y}_i F(\mathbf{x}_i)) \tag{18.26}$$

where $\omega_{i,m} \triangleq \exp(-\tilde{y}_i f_{m-1}(\mathbf{x}_i))$ is a weight applied to datacase i, and $\tilde{y}_i \in \{-1, +1\}$. We can rewrite this objective as follows:

$$L_m = e^{-\beta} \sum_{\tilde{y}_i = F(\mathbf{x}_i)} \omega_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq F(\mathbf{x}_i)} \omega_{i,m} \tag{18.27}$$

$$= (e^{\beta} - e^{-\beta}) \sum_{i=1}^{N} \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\mathbf{x}_i)) + e^{-\beta} \sum_{i=1}^{N} \omega_{i,m} \tag{18.28}$$

Consequently the optimal function to add is

$$F_m = \underset{F}{\operatorname{argmin}} \sum_{i=1}^{N} \omega_{i,m} \mathbb{I}\left(\tilde{y}_i \neq F(\mathbf{x}_i)\right) \tag{18.29}$$

This can be found by applying the weak learner to a weighted version of the dataset, with weights $\omega_{i,m}$.

All that remains is to solve for the size of the update, β. Subsituting F_m into L_m and solving for β we find

$$\beta_m = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m} \tag{18.30}$$

where

$$\text{err}_m = \frac{\sum_{i=1}^{N} \omega_i \mathbb{I}\left(\tilde{y}_i \neq F_m(\mathbf{x}_i)\right)}{\sum_{i=1}^{N} \omega_{i,m}} \tag{18.31}$$

Therefore overall update becomes

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}) \tag{18.32}$$

After updating the strong learner, we need to recompute the weights for the next iteration, as follows:

$$\omega_{i,m+1} = e^{-\tilde{y}_i f_m(\mathbf{x}_i)} = e^{-\tilde{y}_i f_{m-1}(\mathbf{x}_i) - \tilde{y}_i \beta_m F_m(\mathbf{x}_i)} = \omega_{i,m} e^{-\tilde{y}_i \beta_m F_m(\mathbf{x}_i)} \tag{18.33}$$

If $\tilde{y}_i = F_m(\mathbf{x}_i)$, then $\tilde{y}_i F_m(\mathbf{x}_i) = 1$, and if $\tilde{y}_i \neq F_m(\mathbf{x}_i)$, then $\tilde{y}_i F_m(\mathbf{x}_i) = -1$. Hence $-\tilde{y}_i F_m(\mathbf{x}_i) - 2\mathbb{I}\left(\tilde{y}_i \neq F_m(\mathbf{x}_o)\right) - 1$, so the update becomes

$$\omega_{i,m+1} = \omega_{i,m} e^{\beta_m (2\mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i)) - 1)} = \omega_{i,m} e^{2\beta_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))} e^{-\beta_m} \tag{18.34}$$

Since the $e^{-\beta_m}$ is constant across all examples, it can be dropped. If we then define $\alpha_m = 2\beta_m$, the update becomes

$$\omega_{i,m+1} = \begin{cases} \omega_{i,m} e^{\alpha_m} & \text{if } \tilde{y}_i \neq F_m(\mathbf{x}_i) \\ \omega_{i,m} & \text{otherwise} \end{cases} \tag{18.35}$$

Thus we see that we exponentially increase weights of misclassified examples. The resulting algorithm shown in Algorithm 10, and is known as **Adaboost.M1** [FS96].[2]

A multiclass generalization of exponential loss, and an adaboost-like algorithm to minimize it, known as **SAMME** (stagewise additive modeling using a multiclass exponential loss function), is described in [Has+09]. This is implemented in scikit learn (the AdaBoostClassifier class).

2. In [FHT00], this is called **discrete AdaBoost**, since it assumes that the base classifier F_m returns a binary class label. If F_m returns a probability instead, a modified algorithm, known as **real AdaBoost**, can be used. See [FHT00] for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 10: Adaboost.M1, for binary classification with exponential loss

$\omega_i = 1/N$;
for $m = 1 : M$ **do**
  Fit a classifier $F_m(\mathbf{x})$ to the training set using weights $\mathbf{w}$;
  Compute $\text{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))}{\sum_{i=1}^N \omega_{i,m})}$;
  Compute $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$;
  Set $\omega_i \leftarrow \omega_i \exp[\alpha_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))]$;
Return $f(\mathbf{x}) = \text{sgn}\left[\sum_{m=1}^M \alpha_m F_m(\mathbf{x})\right]$;

18.5.4 LogitBoost

The trouble with exponential loss is that it puts a lot of weight on misclassified examples, as is apparent from the exponential blowup on the left hand side of Figure 18.7. This makes the method very sensitive to outliers (mislabeled examples). In addition, $e^{-\tilde{y}f}$ is not the logarithm of any pmf for binary variables $\tilde{y} \in \{-1, +1\}$; consequently we cannot recover probability estimates from $f(\mathbf{x})$.

A natural alternative is to use log loss insteadm as we discussed in Sec. 18.5.3. This only punishes mistakes linearly, as is clear from Figure 18.7. Furthermore, it means that we will be able to extract probabilities from the final learned function, using

$$p(y = 1|\mathbf{x}) = \frac{e^{f(\mathbf{x})}}{e^{-f(\mathbf{x})} + e^{f(\mathbf{x})}} = \frac{1}{1 + e^{-2f(\mathbf{x})}} \tag{18.36}$$

The goal is to minimze the expected log-loss, given by

$$L_m(F) = \sum_{i=1}^N \log\left[1 + \exp\left(-2\tilde{y}_i(f_{m-1}(\mathbf{x}) + F(\mathbf{x}_i))\right)\right] \tag{18.37}$$

By performing a Newton upate on this objective (similar to IRLS), one can derive the algorithm shown in Algorithm 11. This is known as **logitBoost** [FHT00]. The key subroutine is the ability of the weak learner F to solve a weighted least squares problem. This method can be generalized to the multi-class setting, as explained in [FHT00].

18.5.5 Gradient boosting

Rather than deriving new versions of boosting for every different loss function, it is possible to derive a generic version, known as **gradient boosting** [Fri01; Mas+00]. To explain this, imagine solving $\hat{\mathbf{f}} = \text{argmin}_{\mathbf{f}} \mathcal{L}(\mathbf{f})$ by performing gradient descent in the space of functions. Since functions are infinite dimensional objects, we will represent them by their values on the training set, $\mathbf{f} = (f(\mathbf{x}_1), \ldots, f(\mathbf{x}_N))$. At step m, let $\mathbf{g}_m$ be the gradient of $\mathcal{L}(\mathbf{f})$ evaluated at $\mathbf{f} = \mathbf{f}_{m-1}$:

$$g_{im} = \left[\frac{\partial \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)}\right]_{f=f_{m-1}} \tag{18.38}$$

Algorithm 11: LogitBoost, for binary classification with log-loss

$\omega_i = 1/N$, $\pi_i = 1/2$;
for $m = 1 : M$ **do**
  Compute the working response $z_i = \frac{y_i^* - \pi_i}{\pi_i(1-\pi_i)}$;
  Compute the weights $\omega_i = \pi_i(1-\pi_i)$;
  $F_m = \text{argmin}_F \sum_{i=1}^N \omega_i (z_i - F(\mathbf{x}_i))^2$;
  Update $f(\mathbf{x}) \leftarrow f(\mathbf{x}) + \frac{1}{2} F_m(\mathbf{x})$;
  Compute $\pi_i = 1/(1 + \exp(-2f(\mathbf{x}_i)))$;
Return $f(\mathbf{x}) = \text{sgn}\left[\sum_{m=1}^M F_m(\mathbf{x})\right]$;

Name	Loss	$-\partial \ell(y_i, f(\mathbf{x}_i))/\partial f(\mathbf{x}_i)$
Squared error	$\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$	$y_i - f(\mathbf{x}_i)$
Absolute error	$\lvert y_i - f(\mathbf{x}_i)\rvert$	$\text{sgn}(y_i - f(\mathbf{x}_i))$
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$
Binary Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$
Multiclass logloss	$-\sum_c y_{ic} \log \pi_{ic}$	$y_{ic} - \pi_{ic}$

Table 18.1: Some commonly used loss functions, their gradients, and their population minimizers F^*. For binary classification problems, we assume $\tilde{y}_i \in \{-1, +1\}$, and $\pi_i = \sigma(2f(\mathbf{x}_i))$. For regression problems, we assume $y_i \in \mathbb{R}$. Adapted from [HTF09, p360] and [BH07, p483].

Gradients of some common loss functions are given in Table 18.1. We then make the update

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \beta_m \mathbf{g}_m \tag{18.39}$$

where β_m is the step length, chosen by

$$\beta_m = \underset{\beta}{\text{argmin}}\, \mathcal{L}(\mathbf{f}_{m-1} - \beta \mathbf{g}_m) \tag{18.40}$$

In its current form, this is not much use, since it only optimizes f at a fixed set of N points, so we do not learn a function that can generalize. However, we can modify the algorithm by fitting a weak learner to approximate the negative gradient signal. That is, we use this update

$$F_m = \underset{F}{\text{argmin}} \sum_{i=1}^N (-g_{im} - F(\mathbf{x}_i))^2 \tag{18.41}$$

The overall algorithm is summarized in Algorithm 12. We have omitted the line search step for β_m, which is not strictly necessary, as argued in [BH07]. However, we have introduced a learning rate or **shrinkage factor** $0 < \nu \leq 1$, to control the size of the updates, for regularization purposes.

If we apply this algorithm using squared loss, we recover L2Boosting, since $-g_{im} = y_i - f_{m-1}(\mathbf{x}_i)$ is just the residual error. We can also apply this algorithm to other loss functions, such as absolute loss or Huber loss (Sec. 5.1.5.3), which is useful for robust regression problems.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 12: Gradient boosting

Initialize $f_0(\mathbf{x}) = \text{argmin}_F \sum_{i=1}^N L(y_i, F(\mathbf{x}_i))$;
for $m = 1 : M$ **do**
  Compute the gradient residual using $r_{im} = -\left[\frac{\partial L(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)}\right]_{f(\mathbf{x}_i)=f_{m-1}(\mathbf{x}_i)}$
  Use the weak learner to compute $F_m = \text{argmin}_F \sum_{i=1}^N (r_{im} - F(\mathbf{x}_i)^2$;
  Update $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$;
Return $f(\mathbf{x}) = f_M(\mathbf{x})$

For classification, we can use log-loss. In this case, we get an algorithm known as **BinomialBoost** [BH07]. The advantage of this over LogitBoost is that it does not need to be able to do weighted fitting: it just applies any black-box regression model to the gradient vector. To apply this to multi-class classification, we can fit C separate regression trees, using the pseudo residual of the form

$$-g_{icm} = \frac{\partial \ell(y_i, f_{1m}(\mathbf{x}_i), \ldots, f_{Cm}(\mathbf{x}_i))}{\partial f_{cm}(\mathbf{x}_i)} = \mathbb{I}(y_i = c) - \pi_{ic} \tag{18.42}$$

Although the trees are fit separately, their predictions are combined via a softmax transform

$$p(y = c|\mathbf{x}) = \frac{e^{f_c(\mathbf{x})}}{\sum_{c'=1}^C e^{f_{c'}(\mathbf{x})}} \tag{18.43}$$

When we have large datasets, we can use a stochastic variant in which we subsample (without replacement) a random fraction of the data to pass to the regression tree at each iteration. This is called **stochastic gradient boosting** [Fri99]. Not only is it faster, but it can also generalize better, because subsampling the data is a form of regularization

18.5.5.1 Gradient tree boosting

In practice, gradient boosting nearly always assumes that the weak learner is a regression tree, which is a model of the form

$$F_m(\mathbf{x}) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \tag{18.44}$$

where w_{jm} is the predicted output for region R_{jm}. (In general, w_{jm} could be a vector.) This combination is called **gradient boosted regression trees**, or **gradient tree boosting**. (A related version is known as **MART**, which stands for "multivariate additive regression trees" [FM03].)

To use this in gradient boosting, we first find good regions R_{jm} for tree m using standard regression tree learning (see Sec. 18.1) on the residuals; we then (re)solve for the weights of each leaf by solving

$$\hat{w}_{jm} = \underset{w}{\text{argmin}} \sum_{\mathbf{x}_i \in R_{jm}} \ell(y_i, f_{m-1}(\mathbf{x}_i) + w) \tag{18.45}$$

For squared error (as used by gradient boosting), the optimal weight $\hat{w}_{jm}$ is the just the mean of the residuals in that leaf.

18.5.5.2 XGBoost

XGBoost (https://github.com/dmlc/xgboost), which stands for "extreme gradient boosting", is a very efficient and widely used implementation of gradient boosted trees, that adds a few more improvements beyond the description in Sec. 18.5.5.1. The details can be found in [CG16], but in brief, the extensions are as follows: it adds a regularizer on the tree complexity, it uses a second order approximation of the loss (from [FHT00]) instead of just a linear approximation, it samples features at internal nodes (as in random forests), and it uses various computer science methods (such as handling out-of-core computation for large datasets) to ensure scalability.[3]

In more detail, XGBoost optimizes the following regularized objective

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i)) + \Omega(f) \tag{18.46}$$

where

$$\Omega(f) = \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2 \tag{18.47}$$

is the regularizer, where J is the number of leaves, and $\gamma \geq 0$ and $\lambda \geq 0$ are regularization coefficients. At the m'th step, the loss is given by

$$\mathcal{L}_m(F_m) = \sum_{i=1}^{N} \ell(y_i, f_{m-1}(\mathbf{x}_i) + F_m(\mathbf{x}_i)) + \Omega(F_m) + \text{const} \tag{18.48}$$

We can compute a second order Taylor expansion of this as follows:

$$\mathcal{L}_m(F_m) \approx \sum_{i=1}^{N} \left[\ell(y_i, f_{m-1}(\mathbf{x}_i)) + g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \Omega(F_m) + \text{const} \tag{18.49}$$

where h_{im} is the Hessian

$$h_{im} = \left[\frac{\partial^2 \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)^2} \right]_{f=f_{m-1}} \tag{18.50}$$

In the case of regression trees, we have $F(\mathbf{x}) = w_{q(\mathbf{x})}$, where $q : \mathbb{R}^D \to \{1, \ldots, J\}$ specifies which leaf node $\mathbf{x}$ belongs to, and $\mathbf{w} \in \mathbb{R}^J$ are the leaf weights. Hence we can rewrite Eq. (18.49) as follows, dropping terms that are independent of F_m:

$$\mathcal{L}_m(q, \mathbf{w}) \approx \sum_{i=1}^{N} \left[g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \gamma J + \frac{1}{2}\lambda \sum_{j=1} J w_j^2 \tag{18.51}$$

$$= \sum_{j=1}^{J} \left[(\sum_{i \in I_j} g_{im}) w_j + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2 \right] + \gamma J \tag{18.52}$$

3. Some other popular gradient boosted trees packages are **CatBoost** (https://catboost.ai/) and **LightGBM** (https://github.com/Microsoft/LightGBM).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $I_j = \{i : q(\mathbf{x}_i) = j\}$ is the set of indices of data points assigned to the j'th leaf.

Let us define $G_{jm} = \sum_{i \in I_j} g_{im}$ and $H_{jm} = \sum_{i \in I_j} h_{im}$. Then the above simplifies to

$$\mathcal{L}_m(q, \mathbf{w}) = \sum_{j=1}^{J} \left[G_{jm} w_j + \frac{1}{2}(H_{jm} + \lambda) w_j^2 \right] + \gamma J \tag{18.53}$$

This is a quadratic in each w_jm so the optimal weights are given by

$$w_j^* = -\frac{G_{jm}}{H_{jm} + \lambda} \tag{18.54}$$

The loss for evaluating different tree structures q then becomes

$$\mathcal{L}_m(q, \mathbf{w}^*) = -\frac{1}{2} \sum_{j=1}^{J} \frac{G_{jm}^2}{H_{jm} + \lambda} + \gamma J \tag{18.55}$$

We can greedily optimize this using a recursive node splitting procedure, as in Sec. 18.1. Specifically, for a given leaf j, we consider splitting it into a left and right half, $I = I_L \cup I_R$. We can compute the gain (reduction in loss) of such a split as follows:

$$\text{gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{(H_L + H_R) + \lambda} \right] - \gamma \tag{18.56}$$

where $G_L = \sum_{i \in I_L} g_{im}$, $G_R = \sum_{i \in I_R} g_{im}$, $H_L = \sum_{i \in I_L} h_{im}$, and $H_R = \sum_{i \in I_R} h_{im}$. Thus we see that it is not worth splitting a node if the gain is less than γ.

A fast approximation for evaluating this objective, that does not require sorting the features (for choosing the optimal threshold to split on), is described in [CG16].

18.6 Interpreting tree ensembles

Trees are popular because they are interpretable. Unfortunately, ensembles of trees (whether in the form of bagging, random forests, or boosting) lose that property. Fortunately, there are some simple methods we can use to interpret what function has been learned.

18.6.1 Feature importance

For a single decision tree T , [BFO84] proposed the following measure for **feature importance** of feature k:

$$R_k(T) = \sum_{j=1}^{J-1} G_j \mathbb{I}\left(v_j = k\right) \tag{18.57}$$

where the sum is over all non-leaf (internal) nodes, G_j is the gain in accuracy (reduction in cost) at node j, and $v_j = k$ if node j uses feature k. We can get a more reliable estimate by averaging over all trees in the ensemble:

$$R_k = \frac{1}{M} \sum_{m=1}^{M} R_k(T_m) \tag{18.58}$$

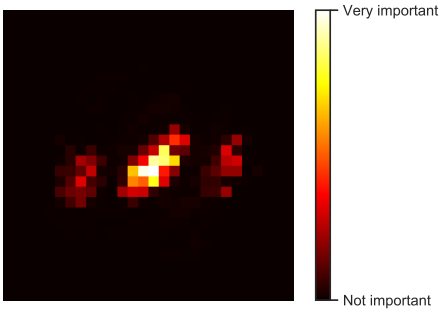


Figure 18.8: Feature importance of a random forest classifier trained to distinguish MNIST digits from classes 0 and 8. Adapted from Figure 7.6 of [Gér19]. Generated by code at figures.probml.ai/book1/18.8

After computing these scores, we can normalize them so the largest value is 100%. We give some examples below.

Fig. 18.8 gives an example of estimating feature importance for a classifier trained to distinguish MNIST digits from classes 0 and 8. We see that it focuses on the parts of the image that differ between these classes.

In Fig. 18.9a, we plot the relative importance of each of the features for the spam dataset (Sec. 18.4). Not surprisingly, we find that the most important features are the words "george" (the name of the recipient) and "hp" (the company he worked for), as well as the characters ! and $. (Note it can be the presence or absence of these features that is informative.)

18.6.2 Partial dependency plots

After we have identified the most relevant input features, we can try to assess the impact they have on the output. A **partial dependency plot** for feature k is a plot of

$$\overline{f}_k(x_k) = \frac{1}{N}\sum_{n=1}^{N} f(\mathbf{x}_{n,-k}, x_k) \tag{18.59}$$

vs x_k. Thus we marginalize out all features except k. In the case of a binary classifier, we can convert this to log odds, $\log p(y=1|x_k)/p(y=0|x_k)$, before plotting. We illustrate this for our spam example in Fig. 18.9b for 4 different features. We see that as the frequency of ! and "remove" increases, so does the probability of spam. Conversely, as the frequency of "edu" or "hp" increases, the probability of spam decreases.

We can also try to capture interaction effects between features j and k by computing

$$\overline{f}_{jk}(x_j, x_k) = \frac{1}{N}\sum_{n=1}^{N} f(\mathbf{x}_{n,-jk}, x_j, x_k) \tag{18.60}$$

We illustrate this for our spam example in Fig. 18.9c for hp and !. We see that higher frequency of ! makes it more likely to be spam, but much more so if the word "hp" is missing.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

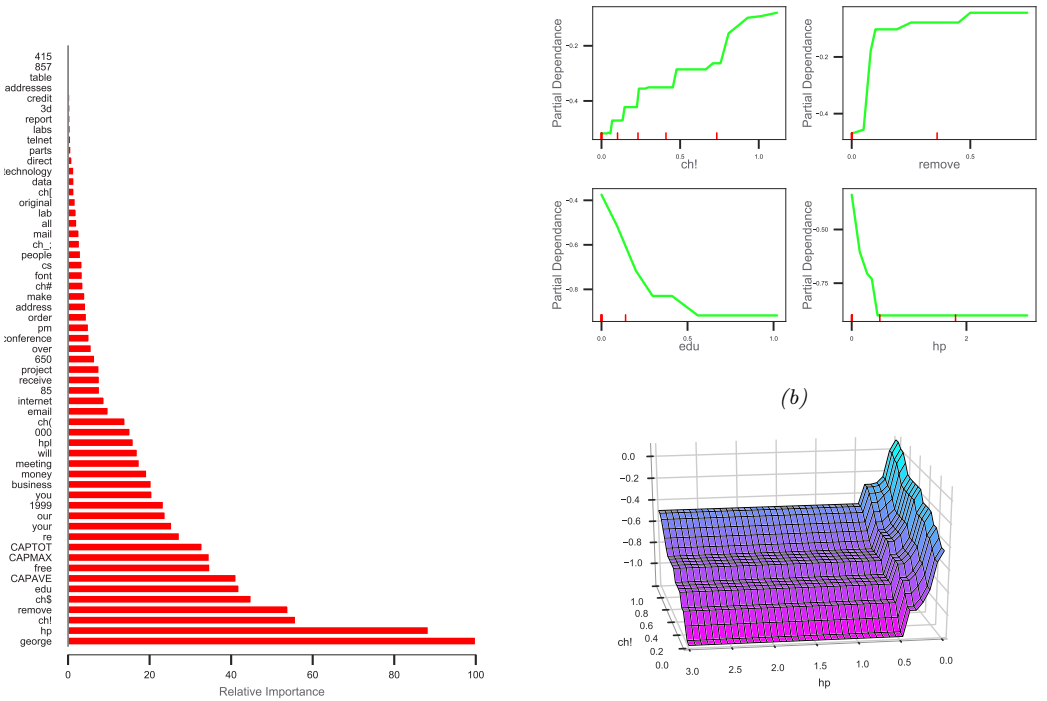


Figure 18.9: (a) Relative feature importance for the spam classification problem. Adapted from Figure 10.6 of [HTF09]. (b) Partial dependence of log-odds of the spam class for 4 important predictors. The red ticks at the base of the plot are deciles of the empirical distribution for this feature. (c) Joint partial dependence of log-odds on the features hp and !. Adapted from Figure 10.6–10.8 of [HTF09]. Generated by code at figures.probml.ai/book1/18.9.

Part V

Beyond supervised learning

19 Learning with fewer labeled examples

Many ML models, especially neural networks, often have many more parameters than we have labeled training examples. For example, a ResNet CNN (Sec. 14.3.4) with 50 layers has 23 million parameters. Transformer models (Sec. 15.5) can be even bigger. Of course these parameters are highly correlated, so they are not independent "degrees of freedom". Nevertheless, such big models are slow to train and, more importantly, they may easily overfit. This is particularly a problem when you do not have a large labeled training set. In this chapter, we discuss some ways to tackle this issue, beyond the generic regularization techniques we discussed in Sec. 13.5 such as early stopping, weight decay and dropout.

19.1 Data augmentation

Suppose we just have a single small labeled dataset. In some cases, we may be able to create artificially modified versions of the input vectors, which capture the kinds of variations we expect to see at test time, while keeping the original labels unchanged. This is called **data augmentation**.[1] We give some examples below, and then discuss why this approach works.

19.1.1 Examples

For image classification tasks, standard data augmentation methods include random crops, zooms, and mirror image flips, as illustrated in Fig. 19.1. [GVZ16] gives a more sophisticated example, where they render text characters onto an image in a realistic way, thereby creating a very large dataset of text "in the wild". They used this to train a state of the art visual text localization and reading system. Other examples of data augmentation include artifically adding background noise to clean speech signals, and artificially replacing characters or words at random in text documents.

If we afford to train and test the model many times using different versions of the data, we can learn which augmentations work best, using blackbox optimization methods such as RL (see e.g., [Cub+19]) or Bayesian optimization (see e.g., [Lim+19]); this is called **AutoAugment**. We can also learn to combine multiple augmentations together; this is called **AugMix** [Hen+20].

1. The term "data augmentation" is also used in statistics to mean the addition of auxiliary latent variables to a model in order to speed up convergence of posterior inference algorithms [DM01].

Figure 19.1: Illustration of random crops and zooms of a image images. Generated by code at figures.probml.ai/book1/19.1.

19.1.2 Theoretical justification

Data augmentation often significantly improves performance (predictive accuracy, robustness, etc). At first this might seem like we are getting something for nothing, since we have not provided additional data. However, the data augmentation mechanism can be viewed as a way to algorithmically inject prior knowledge.

To see this, reall that in standard ERM training, we minimize the empirical risk

$$R(f) = \int \ell(f(\mathbf{x}), y) p^*(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \tag{19.1}$$

where we approximate $p^*(\mathbf{x}, \mathbf{y})$ by the empirical distribution

$$p_D(\mathbf{x}, \mathbf{y}) = \frac{1}{N} \sum_{n=1}^{N} \delta(\mathbf{x} - \mathbf{x}_n) \delta(\mathbf{y} - \mathbf{y}_n) \tag{19.2}$$

We can think of data augmentation as replacing the empirical distribution with the following algorithmically smoothed distribution

$$p_D(\mathbf{x}, \mathbf{y}|A) = \frac{1}{N} \sum_{n=1}^{N} p(\mathbf{x}|\mathbf{x}_n, A) \delta(\mathbf{y} - \mathbf{y}_n) \tag{19.3}$$

where A is the data augmentation algorithm, which generates a sample $\mathbf{x}$ from a training point $\mathbf{x}_n$, such that the label ("semantics") is not changed. (A very simple example would be a Gaussian kernel, $p(\mathbf{x}|\mathbf{x}_n, A) = \mathcal{N}(\mathbf{x}|\mathbf{x}_n, \sigma^2 \mathbf{I})$.) This has been called **vicinal risk minimization** [Cha+01], since we are minimizing the risk in the vicinity of each training point $\mathbf{x}$. For more details on this perspective, see [Zha+17b; CDL19; Dao+19].

19.2 Transfer learning

This section is coauthored with Colin Raffel.

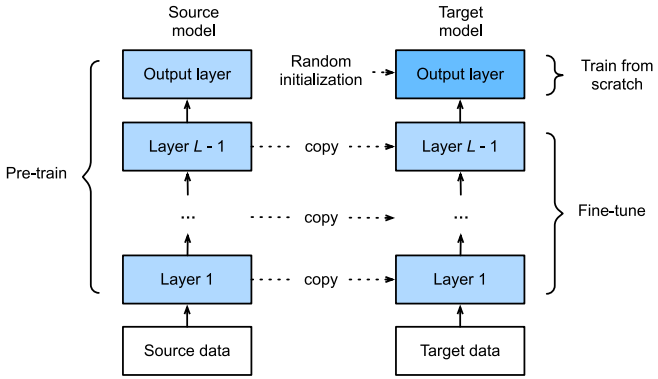


Figure 19.2: Illustration of fine-tuning, where we freeze all the layers except for the last, which is optimized on the target dataset. From Figure 13.2.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Many data-poor tasks have some high-level structural similarity to other data-rich tasks. For example, consider the task of **fine-grained visual classification** of endangered bird species. Given that endangered birds are by definition rare, it is unlikely that a large quantity of diverse labeled images of these birds exist. However, birds bear many structural similarities across species - for example, most birds have wings, feathers, beaks, claws, etc. We therefore might expect that first training a model on a large dataset of non-endangered bird species and then continuing to train it on a small dataset of endangered species could produce better performance than training on the small dataset alone.

This is called **transfer learning**, since we are transferring information from one dataset to another, via a shared set of parameters. More precisely, we first perform a **pre-training phase**, in which we train a model with parameters $\boldsymbol{\theta}$ on a large **source dataset** $\mathcal{D}_p$; this may be labeled or unlabeled. We then perform a second **fine-tuning phase** on the small labeled **target dataset** $\mathcal{D}_q$ of interest. (The fact that the pre-training and fine-tuning tasks differ means that transfer learning is slightly different from semi-supervised learning, which we discuss in Sec. 19.3.) We discuss these two phases in more detail below, but for more information, see e.g., [Tan+18; Zhu+19] for recent surveys.

19.2.1 Fine-tuning

Suppose, for now, that we already have a pretrained classifier, $p(y|\mathbf{x}, \boldsymbol{\theta}_p)$, such as a CNN, that works well for inputs $\mathbf{x} \in \mathcal{X}_p$ (e.g. natural images) and outputs $y \in \mathcal{Y}_p$ (e.g., ImageNet labels), where the data comes from a distribution $p(\mathbf{x}, y)$ similar to the one used in training. Now we want to create a new model $q(y|\mathbf{x}, \boldsymbol{\theta}_q)$ that works well for inputs $\mathbf{x} \in \mathcal{X}_q$ (e.g. bird images) and outputs $y \in \mathcal{Y}_q$ (e.g., fine-grained bird labels), where the data comes from a distribution $q(\mathbf{x}, y)$ which may be different from p.

We will assume that the set of possible inputs is the same, so $\mathcal{X}_q \approx \mathcal{X}_p$ (e.g., both are RGB images), or that we can easily transform inputs from domain p to domain q (e.g., we can convert an RGB image to grayscale by dropping the chrominance channels and just keeping luminance). (If this is not the case, then we may need to use a method called domain adaptation, that modifies models to map

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

between modalities, as discussed in Sec. 19.2.4.)

However, the output domains are usually different, i.e., $\mathcal{Y}_q \neq \mathcal{Y}_p$. For example, $\mathcal{Y}_p$ might be Imagenet labels and $\mathcal{Y}_q$ might be medical labels (e.g., types of diabetic retinopathy [Arc+19]). In this case, we need to "translate" the output of the pre-trained model to the new domain. This is easy to do with neural networks: we simply "chop off" the final layer of the original model, and add a new "head" to model the new class labels, as illustrated in Fig. 19.2. For example, suppose $p(y|\mathbf{x}, \boldsymbol{\theta}_p) = \mathcal{S}(y|\mathbf{W}_2\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_2)$, where $\boldsymbol{\theta}_p = (\mathbf{W}_2, \mathbf{b}_2, \boldsymbol{\theta}_1)$. Then we can construct $q(y|\boldsymbol{\theta}_q) = \mathcal{S}(y|\mathbf{W}_3^\top \mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_3)$, where $\boldsymbol{\theta}_q = (\mathbf{W}_3, \mathbf{b}_3, \boldsymbol{\theta}_1)$ and $\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1)$ is the shared nonlinear feature extractor.

After performing this "model surgery", we can fine-tune the new model with parameters $\boldsymbol{\theta}_q = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$, where $\boldsymbol{\theta}_1$ parameterizes the feature extractor, and $\boldsymbol{\theta}_3$ parameters the final linear layer that maps features to the new set of labels. If we treat $\boldsymbol{\theta}_1$ as "**frozen parameters**", then the resulting model $q(y|\mathbf{x}, \boldsymbol{\theta}_q)$ is linear in its parameters, so we have a convex optimization problem for which many simple and efficient fitting methods exist (see Part II). This is particularly helpful in the long-tail setting, where some classes are very rare [Kan+20]. However, a linear "decoder" may be too limiting, so we can also allow $\boldsymbol{\theta}_1$ to be fine-tuned as well, but using a lower learning rate, to prevent the values moving too far from the values estimated on $\mathcal{D}_p$.

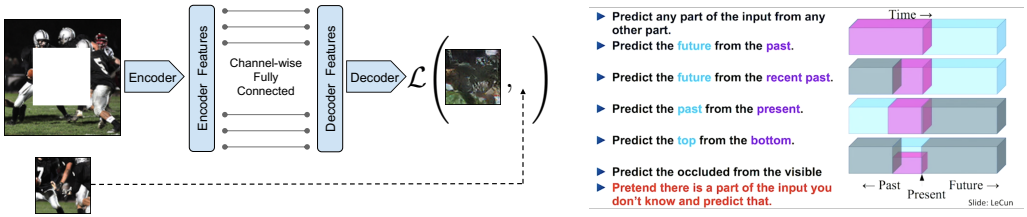
19.2.2 Supervised pre-training

The pre-training task may be supervised or unsupervised; the main requirements are that it can teach the model basic structure about the problem domain and that it is sufficiently similar to the downstream fine-tuning task. The notion of task similarity is not rigorously defined, but in practice the domain of the pre-training task is often more broad than that of the fine-tuning task (e.g., pre-train on all bird species and fine-tune on endangered ones).

The most straightforward form of transfer learning is the case where a large labeled dataset is suitable for pre-training. For example, it is very common to use the ImageNet dataset (Sec. 1.5.1.2) to pretrain CNNs, which can then be used for an a variety of downstream tasks and datasets (see e.g., [Kol+19]). Imagenet has 1.28 million natural images, each associated with a label from one of 1,000 classes. The classes constitute a wide variety of different concepts, including animals, foods, buildings, musical instruments, clothing, and so on. The images themselves are diverse in the sense that they contain objects from many angles and in many sizes with a wide variety of backgrounds. This diversity and scale may partially explain why it has become a de-facto pre-training task for transfer learning in computer vision. (See code.probml.ai/book1/finetune_cnn_torch for some example code.)

However, Imagenet pre-training has been shown to be less helpful when the domain of the fine-tuning task is quite different from natural images (e.g. medical images [Rag+19]). And in some cases where it is helpful (e.g., training object detection systems), it seems to be more of a speedup trick (by warm-starting optimization at a good point) rather than something that is essential, in the sense that one can achieve comparable performance on the downstream task when training from scratch, if done for long enough [HGD19].

Supervised pre-training is somewhat less common in non-vision applications. One notable exception is to pre-train on natural language inference data (i.e. whether a sentence implies or contradicts another) to learn vector representations of sentences [Con+17], though this approach has largely been supplanted by unsupervised methods (Sec. 19.2.3). Another non-vision application of transfer learning is to pre-train a speech recognition on a large English-labeled corpus before fine-tuning on



(a) (b)

Figure 19.3: (a) Context encoder for self-supervised learning. From [Pat+16]. Used with kind permission of Deepak Pathak. (b) Some other proxy tasks for self-supervised learning. From [LeC18]. Used with kind permission of Yann LeCun.

low-resource languages [Ard+20].

19.2.3 Unsupervised pre-training (self-supervised learning)

It is increasingly common to use **unsupervised pre-training**, because unlabeled data is often easy to acquire, e.g., unlabeled images or text documents from the web.

For a short period of time it was common to pre-train deep neural networks using an unsupervised objective (e.g., reconstruction error, as discussed in Sec. 20.3) over the labeled dataset (i.e. ignoring the labels) before proceeding with standard supervised training [HOT06; Vin+10b; Erh+10]. While this technique is also called unsupervised pre-training, it differs from the form of pre-training for transfer learning we discuss in this section, which uses a (large) unlabeled dataset for pre-training before fine-tuning on a different (smaller) labeled dataset.

Pre-training tasks that use unlabeled data are often called **self-supervised** rather than unsupervised. This term is used because the labels are created by the algorithm, rather than being provided externally by a human, as in standard supervised learning. Both supervised and self-supervised learning are discriminative tasks, since they require predicting outputs given inputs. By contrast, other unsupervised approaches, such as some of those discussed in Chapter 20, are generative, since they predict outputs unconditionally.

There are many different self-supervised learning heuristics that have been tried (see e.g., [GR18; JT19; Ren19] for a review). We can identify at least three main broad groups, which we discuss below.

19.2.3.1 Imputation tasks

One approach to semi-supervised learning is to solve **imputation tasks**. In this approacg, we partition the input vector $\mathbf{x}$ into two parts, $\mathbf{x} = (\mathbf{x}_h, \mathbf{x}_v)$, and then try to predict the hidden part $\mathbf{x}_h$ given the remaining visible part, $\mathbf{x}_v$, using a model of the form $\hat{\mathbf{x}}_h = f(\mathbf{x}_v, \mathbf{x}_h = \mathbf{0})$. We can think of this as a "**fill-in-the-blank**" task; in the NLP community, this is called a **cloze task**. See Fig. 19.3 for some visual examples, and Sec. 15.7.2 for some NLP examples.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

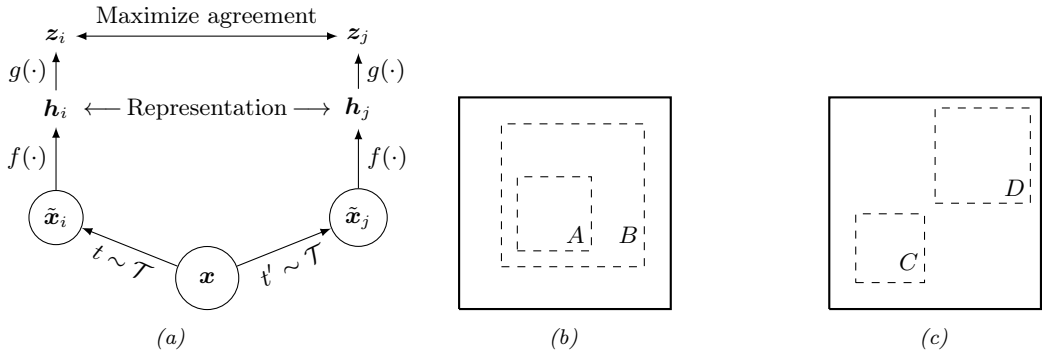


Figure 19.4: (a) Illustration of SimCLR training. $\mathcal{T}$ is a set of stochastic semantics-preserving transformations (data augmentations). (b-c) Illustration of the benefit of random crops. Solid rectangles represent the original image, dashed rectangles are random crops. On the left, the model is forced to predict the local view A from the global view B (and vice versa). On the right, the model is forced to predict the appearance of adjacent views (C,D). From Figures 2–3 of [Che+20c]. Used with kind permission of Ting Chen.

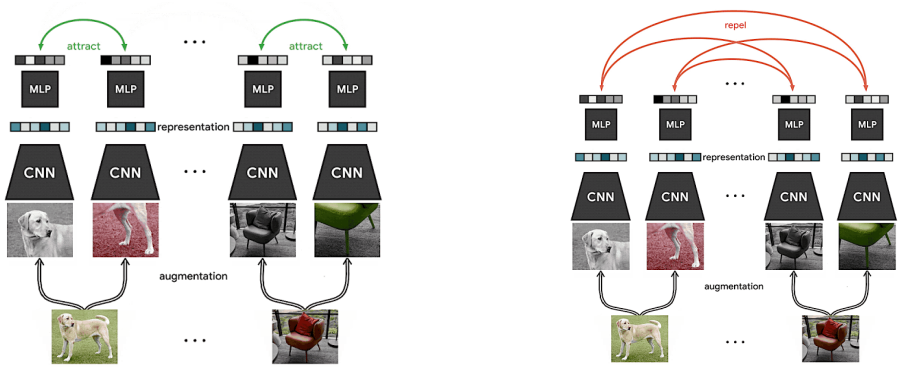
19.2.3.2 Proxy tasks

Another approach to SSL is to solve **proxy tasks**, also called **pretext tasks**. In this setup, we create pairs of inputs, $(\mathbf{x}_1, \mathbf{x}_2)$, and then train a Siamese network classifier (Fig. 16.5a) of the form $p(y|\mathbf{x}_1, \mathbf{x}_2) = p(y|f(r(\mathbf{x}_1), r(\mathbf{x}_2))$, where $r(\mathbf{x})$ is some function that performs "**representation learning**" [BCV13], and y is some label that captures the relationship between $\mathbf{x}_1$ and $\mathbf{x}_2$. For example, if $\mathbf{x}_1$ is an image patch, and $\mathbf{x}_2$ is a random rotation of $\mathbf{x}_1$ that we create, we can try to predict the rotation angle y [GSK18].

19.2.3.3 Contrastive tasks

The currently most popular approach to self-supervised learning is to use various kinds of **contrastive tasks**. The basic idea is to create pairs of examples that are semantically similar to each other, using data augmentation methods (Sec. 19.1), and then to ensure that the distance between their representations is closer (in embedding space) than the distance between two unrelated examples. This is exactly the same idea that is used in deep metric learning (Sec. 16.2.2) — the only difference is that the algorithm creates its own similar pairs, rather than relying on an externally provided measure of similarity, such as labels.

As an example of such an approach, consider the **SimCLR** (simple contrastive learning of visual representations) method of [Che+20c; Che+20d], which has shown state of the art performance on transfer learning and semi-supervised learning. The basic idea is as follows. Each input $\mathbf{x} \in \mathbb{R}^D$ is converted to two augmented "views" $\mathbf{x}'$, $\mathbf{x}''$, which are "semantically equivalent". (For example, if $\mathbf{x}$ is an image, these could be small perturbations to the image, such as random crops, as discussed in Sec. 19.1.) In addition, we sample "negative" examples $\mathbf{y}_1, \ldots, \mathbf{y}_n$ from the dataset which represent "semantically different" images (in practice, these are the other examples in the minibatch). Next we define some feature mapping $f : \mathbb{R}^D \to \mathbb{R}^E$, where D is the size of the input, and E is the size of the embedding. We then try to maximize the similarity of the similar views, while minimizing the



(a) (b)

Figure 19.5: Visualization of SimCLR training. Each input image in the minibatch is randomly modified in two different ways (using cropping (followed by resize), flipping, and color distortion), and then fed into a Siamese network. The embeddings (final layer) for each pair derived from the same image is forced to be close, whereas the embeddings for all other pairs are forced to be far. From `https://ai.googleblog.com/2020/04/advancing-self-supervised-and-semi.html`. *Used with kind permission of Ting Chen.*

similarity of the different views:

$$J = f(\mathbf{x}')^\top f(\mathbf{x}'') - \log \sum_{i=1}^{n} \exp\left[f(\mathbf{y}_i)^\top f(\mathbf{x}'')\right] \tag{19.4}$$

See Fig. 19.4a for an illustration. (In practice, we use cosine similarity, so we ℓ_2-normalize the representations producted by f before taking inner products.)

Interestingly, we can interpret this as a form of conditional **energy based model** of the form

$$p(\mathbf{x}'|\mathbf{x}'') = \frac{\exp[-E(\mathbf{x}'|\mathbf{x}'')]}{Z(\mathbf{x}'')} \tag{19.5}$$

where $E(\mathbf{x}'|\mathbf{x}'') = -f(\mathbf{x}')^\top f(\mathbf{x}'')$ is the energy, and

$$Z(\mathbf{x}'') = \int \exp[-E(\mathbf{y}|\mathbf{x}'')]d\mathbf{y} \tag{19.6}$$

is the normalization constant, known as the **partition function**. The conditional log likelihood under this model has the form

$$\log p(\mathbf{x}'|\mathbf{x}'') = -f(\mathbf{x}')^\top f(\mathbf{x}'') - \log \int \exp[f(\mathbf{y})^\top f(\mathbf{x}'')]d\mathbf{y} \tag{19.7}$$

The only difference from Eq. (19.4) is that we replace the integral with a Monte Carlo upper bound derived from the negative samples. Thus we can think of contrastive learning as approximate maximum likelihood estimation of a conditional energy based generative model [Gra+20]. More details on such models can be found in the sequel to this book, [Mur22].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

A critical ingredient to the success of SimCLR is the choice of data augmentation methods. By using random cropping, they can force the model to predict local views from global views, as well as to predict adjacent views of the same image (see Fig. 19.4). After cropping, all images are resized back to the same size. In addition, they randomly flip the image some fraction of the time.[2]

SimCLR relies on large batch training, in order to ensure a sufficiently diverse set of negatives. When this is not possible, we can use a memory bank of past (negative) embeddings, which can be updated using exponential moving averaging (Sec. 4.4.2.2). This is known as **momentum contrastive learning** or **MoCo** [He+20].

19.2.4 Domain adaptation

Consider a problem in which we have inputs from different domains, such as a **source domain** $\mathcal{X}_s$ and **target domain** $\mathcal{X}_t$, but a common set of output labels, $\mathcal{Y}$. (This is the "dual" of transfer learning, since the input domains are different, but the output domains the same.) For example, the domains might be images from a computer graphics system and real images, or product reviews and movie reviews. We assume we do not have labeled examples from the target domain. Our goal is to fit the model on the source domain, and then modify its parameters so it works on the target domain. This is called (unsupervised) **domain adaptation** (see e.g., [KL19] for a review).

A common approach to this problem is to train the source classifier in such a way that it cannot distinguish whether the input is coming from the source or target distribution; in this case, it will only be able to use features that are common to both domains. This is called **domain adversarial learning** [Gan+16]. More formally, let $d_n \in \{s, t\}$ be a label that specifies if the data example n comes from domain s or t. We want to optimize

$$\min_{\boldsymbol{\phi}} \max_{\boldsymbol{\theta}} \frac{1}{N_s + N_t} \sum_{n \in \mathcal{D}_s, \mathcal{D}_t} \ell(d_n, f_{\boldsymbol{\theta}}(\mathbf{x}_n)) + \frac{1}{N_s} \sum_{m \in \mathcal{D}_s} \ell(y_m, g_{\boldsymbol{\phi}}(f_{\boldsymbol{\theta}}(\mathbf{x}_m))) \tag{19.8}$$

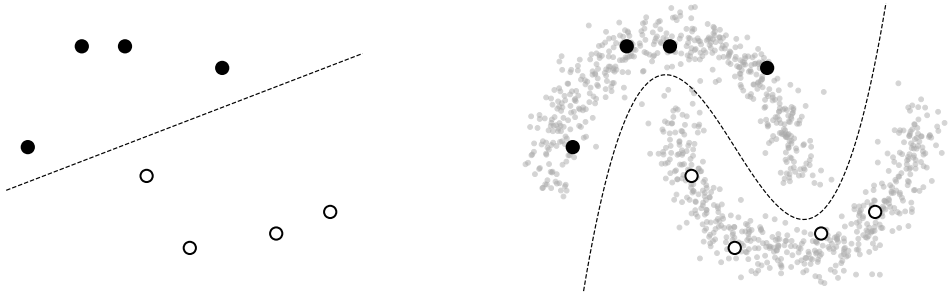
where $N_s = |\mathcal{D}_s|$, $N_t = |\mathcal{D}_t|$, f maps $\mathcal{X}_s \cup \mathcal{X}_t \to \mathcal{H}$, and g maps $\mathcal{H} \to \mathcal{Y}_t$. The objective in Eq. (19.8) minimizes the loss on the desired task of classifying y, but *maximizes* the loss on the auxiliary task of classifying the source domain d. This can be implemented by the **gradient sign reversal** trick, and is related to GANs (generative adversarial networks). See e.g., [Csu17; Wu+19] for some other approaches to domain adaptation.

19.3 Semi-supervised learning

This section is co-authored with Colin Raffel.

Many recent successful applications of machine learning are in the supervised learning setting, where a large dataset of labeled examples are available for training a model. However, in many practical applications it is expensive to obtain this labeled data. Consider the case of automatic speech recognition: Modern datasets contain thousands of hours of audio recordings [Pan+15; Ard+20]. The

2. It turns out that distinguishing positive crops (from the same image) from negative crops (from different images) is often easy to do just based on color histograms. To prevent this kind of "cheating", they also apply a random color distortion, thus cutting off this "short circuit". The combination of random cropping and color distortion is found to work better than either method alone.



(a) (b)

Figure 19.6: Illustration of the benefits of semi-supervised learning for a binary classification problem. Labeled points from each class are shown as black and white circles respectively. (a) Decision boundary we might learn given only unlabeled data. (b) Decision boundary we might learn if we also had a lot of unlabeled data points, shown as smaller grey circles.

process of annotating the words spoken in a recording is many times slower than realtime, potentially resulting in a long (and costly) annotation process. To make matters worse, in some applications data must be labeled by an expert (such as a doctor in medical applications) which can further increase costs.

Semi-supervised learning can alleviate the need for labeled data by taking advantage of unlabeled data. The general goal of semi-supervised learning is to allow the model to learn the high-level structure of the data distribution from unlabeled data and only rely on the labeled data for learning the fine-grained details of a given task. Whereas in standard supervised learning we assume that we have access to samples from the joint distribution of data and labels $\mathbf{x}, y \sim p(\mathbf{x}, y)$, semi-supervised learning assumes that we additionally have access to samples from the marginal distribution of $\mathbf{x}$, namely $\mathbf{x} \sim p(\mathbf{x})$. Further, it is generally assumed that we have many more of these unlabeled samples since they are typically cheaper to obtain. Continuing the example of automatic speech recognition, it is often much cheaper to simply record people talking (which would produce unlabeled data) than it is to transcribe recorded speech. Semi-supervised learning is a good fit for the scenario where a large amount of unlabeled data has been collected and the practitioner would like to avoid having to label all of it.

19.3.1 Self-training and pseudo-labeling

An early and straightforward approach to semi-supervised learning is **self-training** [Scu65; Agr70; McL75]. The basic idea behind self-training is to use the model itself to infer predictions on unlabeled data, and then treat these predictions as labels for subsequent training. Self-training has endured as a semi-supervised learning method because of its simplicity and general applicability; i.e. it is applicable to any model that can generate predictions for the unlabeled data. Recently, it has become common to refer to this approach as "**pseudo-labeling**" [Lee13] because the inferred labels for unlabeled data are only "pseudo-correct" in comparison with the true, ground-truth targets used in supervised learning.

Algorithmically, self-training typically follows one of the following two procedures. In the first

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

approach, pseudo-labels are first predicted for the entire collection of unlabeled data and the model is re-trained (possibly from scratch) to convergence on the combination of the labeled and (pseudo-labeled) unlabeled data. Then, the unlabeled data is re-labeled by the model and the process repeats itself until a suitable solution is found. The second approach instead continually generates predictions on randomly-chosen batches of unlabeled data and immediately trains the model against these pseudo-labels. Both approaches are currently common in practice; the first "offline" variant has been shown to be particularly successful when leveraging giant collections of unlabeled data [Yal+19; Xie+20] whereas the "online" approach is often used as one component of more sophisticated semi-supervised learning methods [Soh+20]. Neither variant is fundamentally better than the other. Offline self-training can result in training the model on "stale" pseudo-labels, since they are only updated each time the model converges. However, online pseudo-labeling can incur larger computational costs since it involves constantly "re-labeling" unlabeled data.

Self-training can suffer from an obvious problem: If the model generates incorrect predictions for unlabeled data and then is re-trained on these incorrect predictions, it can become progressively worse and worse at the intended classification task until it eventually learns a totally invalid solution. This issue has been dubbed **confirmation bias** [TV17] because the model is continually confirming its own (incorrect) bias about the decision rule.

A common way to mitigate confirmation bias is to use a "selection metric" [RHS05] which heuristically tries to only retain pseudo-labels that are correct. For example, assuming that a model outputs probabilities for each possible class, a frequently-used selection metric is to only retain pseudo-labels whose largest class probability is above a threshold [Yar95; RHS05]. If the model's class probability estimates are well-calibrated, then this selection metric will only retain labels that are highly likely to be correct (according to the model, at least). More sophisticated selection metrics can be designed according to the problem domain.

19.3.2 Entropy minimization

Self-training has the implicit effect of encouraging the model to output low-entropy (i.e. high-confidence) predictions. This effect is most apparent in the online setting with a cross-entropy loss, where the model minimizes the following loss function $\mathcal{L}$ on unlabeled data:

$$\mathcal{L} = -\max_c \log p_\theta(y = c|\mathbf{x}) \tag{19.9}$$

where $p_\theta(y|\mathbf{x})$ is the model's class probability distribution given input $\mathbf{x}$. This function is minimized when the model assigns all of its class probability to a single class c^*, i.e. $p(y = c^*|\mathbf{x}) = 1$ and $p(y \neq c^*)|\mathbf{x}) = 0$.

A closely-related semi-supervised learning method is **entropy minimization** [GB05], which minimizes the following loss function:

$$\mathcal{L} = -\sum_{c=1}^{C} p_\theta(y = c|\mathbf{x}) \log p(y = c|\mathbf{x}) \tag{19.10}$$

Note that this function is also minimized when the model assigns all of its class probability to a single class. We can make the entropy-minimization loss in Eq. (19.10) equivalent to the online self-training loss in Eq. (19.9) by replacing the first $p_\theta(y = c|\mathbf{x})$ term with a "one-hot" vector that

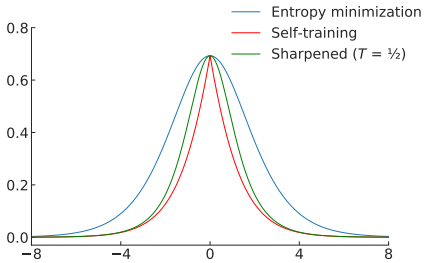


Figure 19.7: Comparison of the entropy minimization, self-training, and "sharpened" entropy minimization loss functions for a binary classification problem.

assigns a probability of 1 for the class that was assigned the highest probability. In other words, online self-training minimizes the cross-entropy between the model's output and the "hard" target $\arg\max p_\theta(y|\mathbf{x})$, whereas entropy minimization uses the the "soft" target $p_\theta(y|\mathbf{x})$. One way to trade off between these two extremes is to adjust the "temperature" of the target distribution by raising each probability to the power of $1/T$ and renormalizing; this is the basis of the **mixmatch** method of [Ber+19a; Ber+19b; Xie+19b]. At $T = 1$, this is equivalent to entropy minimization; as $T \to 0$, it becomes hard online self-training. A comparison of these loss functions is shown in Fig. 19.7.

19.3.2.1 The cluster assumption

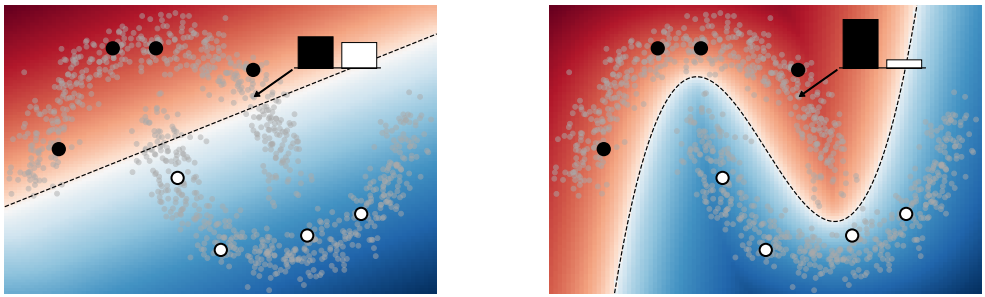
Why is entropy minimization a good idea? A basic assumption of many semi-supervised learning methods is that the decision boundary between classes should fall in a low-density region of the data manifold. This effectively assumes that the data corresponding to different classes are clustered together. A good decision boundary, therefore, should not pass through clusters; it should simply separate them. Semi-supervised learning methods that make the "**cluster assumption**" can be thought of as using unlabeled data to estimate the shape of the data manifold and moving the decision boundary away from it.

Entropy minimization is one such method. To see why, first assume that the decision boundary between two classes is "smooth", i.e. the model does not abrubtly change its class prediction anywhere in its domain. This is true in practice for simple and/or regularized models. In this case, if the decision boundary passes through a high-density region of data, it will by necessity produce high-entropy predictions for some samples from the data distribution. Entropy minimization will therefore encourage the model to place its decision boundary in low-density regions of the input space to avoid transitioning from one class to another in a region of space where data may be sampled. A visualization of this behavior is shown in Fig. 19.8.

19.3.2.2 Input-output mutual information

An alternative justification for the entropy minimization objective was proposed by Bridle, Heading, and MacKay [BHM92], where it was shown that it naturally arises from maximizing the mutual information (Sec. 6.3) between the data and the label (i.e. the input and output of a model). Denoting

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 19.8: Visualization demonstrating how entropy minimization enforces the cluster assumption. The classifier assigns a higher probability to class 1 (black dots) or 2 (white dots) in red or blue regions respectively. The predicted class probabilities for one particular unlabeled datapoint is shown in the bar plot. In (a), the decision boundary passes through high-density regions of data, so the classifier is forced to output high-entropy predictions. In (b), the classifier avoids high-density regions and is able to assign low-entropy predictions to most of the unlabeled data.

$\mathbf{x}$ as the input and y as the target, the input-output mutual information can be written as

$$\mathcal{I}(y;\mathbf{x}) = \iint p(y,\mathbf{x}) \log \frac{p(y,\mathbf{x})}{p(y)p(\mathbf{x})} dy d\mathbf{x} \tag{19.11}$$

$$= \iint p(y|\mathbf{x})p(\mathbf{x}) \log \frac{p(y,\mathbf{x})}{p(y)p(\mathbf{x})} dy d\mathbf{x} \tag{19.12}$$

$$= \int p(\mathbf{x}) d\mathbf{x} \int p(y|\mathbf{x}) \log \frac{p(y|\mathbf{x})}{p(y)} dy \tag{19.13}$$

$$= \int p(\mathbf{x}) d\mathbf{x} \int p(y|\mathbf{x}) \log \frac{p(y|\mathbf{x})}{\int p(\mathbf{x})p(y|\mathbf{x}) d\mathbf{x}} dy \tag{19.14}$$

Note that the first integral is equivalent to taking an expectation over $\mathbf{x}$, and the second integral is equivalent to summing over all possible values of the class y. Using these relations, we obtain

$$\mathcal{I}(y;\mathbf{x}) = \mathbb{E}_{\mathbf{x}} \left[\sum_{i=1}^{L} p(y_i|\mathbf{x}) \log \frac{p(y_i|\mathbf{x})}{\mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})]} \right] \tag{19.15}$$

$$= \mathbb{E}_{\mathbf{x}} \left[\sum_{i=1}^{L} p(y_i|\mathbf{x}) \log p(y_i|\mathbf{x}) \right] - \mathbb{E}_{\mathbf{x}} \left[\sum_{i=1}^{L} p(y_i|\mathbf{x}) \log \mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})] \right] \tag{19.16}$$

$$= \mathbb{E}_{\mathbf{x}} \left[\sum_{i=1}^{L} p(y_i|\mathbf{x}) \log p(y_i|\mathbf{x}) \right] - \sum_{i=1}^{L} \mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})] \log \mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})]] \tag{19.17}$$

Since we had initially sought to *maximize* the mutual information, and we typically *minimize* loss functions, we can convert this to a suitable loss function by negating it:

$$\mathcal{I}(y;\mathbf{x}) = -\mathbb{E}_{\mathbf{x}}\left[\sum_{i=1}^{L} p(y_i|\mathbf{x}) \log p(y_i|\mathbf{x})\right] + \sum_{i=1}^{L} \mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})] \log \mathbb{E}_{\mathbf{x}}[p(y_i|\mathbf{x})]] \tag{19.18}$$

The first term is exactly the entropy minimization objective in expectation. The second term specifies that we should maximize the entropy of the expected class prediction, i.e. the average class prediction over our training set. This encourages the model to predict each possible class with equal probability, which is only appropriate when we know a priori that all classes are equally likely.

19.3.3 Co-training

Co-training [BM98] is also similar to self-training, but makes an additional assumption that there are two complementary "views" (i.e. independent sets of features) of the data, both of which can be used separately to train a reasonable model. After training two models separately on each view, unlabeled data is classified by each model to obtain candidate pseudo-labels. If a particular pseudo-label receives a low-entropy prediction (indicating high confidence) from one model and a high-entropy prediction (indicating low confidence) from the other, then that pseudo-labeled datapoint is added to the training set for the low-confidence model. Then, the process is repeated with the new, larger training datasets. The procedure of only retaining pseudo-labels when one of the models is confident ideally builds up the training sets with correctly-labeled data.

Co-training makes the strong assumption that there are two informative-but-independent views of the data, which may not be true for many problems. The **Tri-Training** algorithm [ZL05] circumvents this issue by instead using *three* models that are first trained on independently-sampled (with replacement) subsets of the labeled data. Ideally, initially training on different collections of labeled data results in models that do not always agree on their predictions. Then, pseudo-labels are generated for the unlabeled data independently by each of the three models. For a given unlabeled datapoint, if two of the models agree on the pseudo-label, it is added to the training set for the third model. This can be seen as a selection metric, because it only retains pseudo-labels where two (differently initialized) models agree on the correct label. The models are then re-trained on the combination of the labeled data and the new pseudo-labels, and the whole process is repeated iteratively.

19.3.4 Label propagation on graphs

If two datapoints are "similar" in some meaningful way, we might expect that they share a label. This idea has been referred to as the **manifold assumption**. **Label propagation** is a semi-supervised learning technique that leverages the manifold assumption to assign labels to unlabeled data. Label propagation first constructs a graph where the nodes are the data examples and the edge weights represent the degree of similarity. The node labels are known for nodes corresponding to labeled data but are unknown for unlabeled data. Label propagation then propagates the known labels across edges of the graph in such a way that there is minimal disagreement in the labels of a given node's neighbors. This provides label guesses for the unlabeled data, which can then be used in the usual way for supervised training of a model.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

More specifically, the basic label propagation algorithm [ZG02] proceeds as follows: First, let $w_{i,j}$ denote a non-negative edge weight between $\mathbf{x}_i$ and $\mathbf{x}_j$ that provides a measure of similarity for the two (labeled or unlabeled) datapoints. Assuming that we have M labeled datapoints and N unlabeled datapoints, define the $(M+N) \times (M+N)$ transition matrix $\mathbf{T}$ as having entries

$$\mathbf{T}_{i,j} = \frac{w_{i,j}}{\sum_k w_{k,j}} \tag{19.19}$$

$\mathbf{T}_{i,j}$ represents the probability of propagating the label for node j to node i. Further, define the $(M+N) \times C$ label matrix $\mathbf{Y}$, where C is the number of possible classes. The ith row of $\mathbf{Y}$ represents the class probability distribution of datapoint i. Then, repeat the following steps until the values in $\mathbf{Y}$ do not change significantly: First, use the transition matrix $\mathbf{T}$ to propagate labels in $\mathbf{Y}$ by setting $\mathbf{Y} \leftarrow \mathbf{TY}$. Then, re-normalize the rows of Y by setting $\mathbf{Y}_{i,c} \leftarrow \mathbf{Y}_{i,c} / \sum_k \mathbf{Y}_{k,c}$. Finally, replace the rows of $\mathbf{Y}$ corresponding to labeled datapoints with their one-hot representation (i.e. $\mathbf{Y}_{i,c} = 1$ if datapoint i has ground-truth label c and 0 otherwise). After convergence, guessed labels are chosen based on the highest class probability for each datapoint in $\mathbf{Y}$.

This algorithm iteratively uses the similarity of datapoints (encoded in the weights used to construct the transition matrix) to propagate information from the (fixed) labels onto the unlabeled data. At each iteration, the label distribution for a given datapoint is computed as the weighted average of the label distributions for all of its connected datapoints, where the weighting corresponds to the edge weights in $\mathbf{T}$. It can be shown that this procedure converges to a single fixed point, whose computational cost mainly involves the inversion of the matrix of unlabled-to-unlabled transition probabilities [ZG02].

The overall approach can be seen as a form of **transductive learning**, since it is learning to predict labels for a fixed unlabeled dataset, rather than learning a model that generalizes. However, given the induced labeling. we can perform **inductive learning** in the usual way.

The success of label propagation depends heavily on the notion of similarity used to construct the weights between different nodes (datapoints). For simple data, measuring the Euclidean distance between datapoints can be sufficient. However, for complex and high-dimensional data the Euclidean distance might not meaningfully reflect the likelihood that two datapoints share the same class. The similarity weights can also be set arbitrarily according to problem-specific knowledge. For a few examples of different ways of constructing the similarity graph, see Zhu [Zhu05, chapter 3]. For some recent papers that use this approach in conjunction with deep learning, see e.g., [BRR18; Isc+19].

19.3.5 Consistency regularization

Consistency regularization leverages the simple idea that perturbing a given datapoint (or the model itself) should not cause the model's output to change dramatically. Since measuring consistency in this way only makes use of the model's outputs (and not ground-truth labels), it is readily applicable to unlabeled data and therefore can be used to create appropriate loss functions for semi-supervised learning. This idea was first proposed under the framework of "learning with pseudo-ensembles" [BAP14], with similar variants following soon thereafter [LA16; SJT16].

In its most general form, both the model $p_\theta(y|\mathbf{x})$ and the transformations applied to the input can be stochastic. For example, in computer vision problems we may transform the input by using data augmentation like randomly rotating or adding noise the input image, and the network may include stochastic components like dropout (Sec. 13.5.4) or weight noise [Gra11]. A common and

simple form of consistency regularization first samples $\mathbf{x}' \sim q(\mathbf{x}'|\mathbf{x})$ (where $q(\mathbf{x}'|\mathbf{x})$ is the distribution induced by the stochastic input transformations) and then minimizes the loss $\|p_\theta(y|\mathbf{x}) - p_\theta(y|\mathbf{x}')\|^2$. In practice, the first term $p_\theta(y|\mathbf{x})$ is typically treated as fixed (i.e. gradients are not propagated through it). In the semi-supervised setting, the combined loss function over a batch of labeled data $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_M, y_M)$ and unlabeled data $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$ is

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{M} \log p_\theta(y = y_i|\mathbf{x}_i) + \lambda \sum_{j=1}^{N} \|p_\theta(y|\mathbf{x}_j) - p_\theta(y|\mathbf{x}'_j)\|^2 \tag{19.20}$$

where λ is a scalar hyperparameter that balances the importance of the loss on unlabeled data and, for simplicity, we write $\mathbf{x}'_j$ to denote a sample drawn from $q(\mathbf{x}'|\mathbf{x}_j)$.

The basic form of consistency regularization in Eq. (19.20) reveals many design choices that impact the success of this semi-supervised learning approach. First, the value chosen for the λ hyperparameter is important. If it is too large, then the model may not give enough weight to learning the supervised task and will instead start to reinforce its own bad predictions (as with confirmation bias in self-training). Since the model is often poor at the start of training before it has been trained on much labeled data, it is common in practice to initialize set λ to zero and increase its value over the course of training.

A second important consideration are the random transformations applied to the input, i.e., $q(\mathbf{x}'|\mathbf{x})$. Generally speaking, these transformations should be designed so that they do not change the label of $\mathbf{x}$. As mentioned above, a common choice is to use domain-specific data augmentations. It has recently been shown that using strong data augmentations that heavily corrupt the input (but, arguably, still do not change the label) can produce particularly strong results [Xie+19b; Ber+19b; Soh+20].

The use of data augmentation requires expert knowledge to determine what kinds of transformations are label-preserving and appropriate for a given problem. An alternative technique, called **virtual adversarial training** (VAT), instead transforms the input using an analytically-found perturbation designed to maximally change the model's output (similar to adversarial examples, see Sec. 14.7). Specifically, VAT computes a perturbation $\boldsymbol{\delta}$ that approximates $\boldsymbol{\delta} = \text{argmax}_{\boldsymbol{\delta}} \mathbb{KL}\left(p_\theta(y|\mathbf{x}) \| p_\theta(y|\mathbf{x} + \boldsymbol{\delta})\right)$. The approximation is done by sampling $\mathbf{d}$ from a multivariate Gaussian distribution, initializing $\boldsymbol{\delta} = \mathbf{d}$, and then setting

$$\boldsymbol{\delta} \leftarrow \nabla_{\boldsymbol{\delta}} \mathbb{KL}\left(p_\theta(y|\mathbf{x}) \| p_\theta(y|\mathbf{x} + \boldsymbol{\delta})\right)|_{\boldsymbol{\delta}=\xi\mathbf{d}} \tag{19.21}$$

where ξ is a small constant, typically 10^{-6}. VAT then sets

$$\mathbf{x}' = \mathbf{x} + \epsilon \frac{\boldsymbol{\delta}}{\|\boldsymbol{\delta}\|_2} \tag{19.22}$$

and proceeds as usual with consistency regularization (as in Eq. (19.20)), where ϵ is a scalar hyperparameter that sets the L2-norm of the perturbation applied to $\mathbf{x}$.

Consistency regularization can also profoundly affect the geometry properties of the training objective, and the trajectory of SGD, such that performance can particularly benefit from non-standard training procedures. For example, the Euclidean distances between weights at different training epochs is significantly larger for objectives that use consistency regularization. Athiwaratkun, Izmailov, and Wilson [AIW19] show that a variant of **stochastic weight averaging** (SWA) [Izm+18]

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

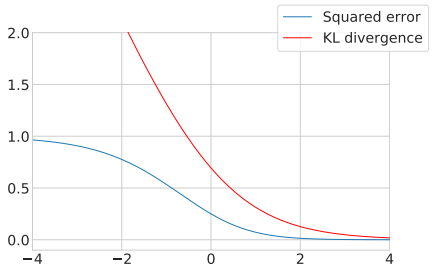


Figure 19.9: Comparison of the squared error and KL divergence lossses for a consistency regularization. This visualization is for a binary classification problem where it is assumed that the model's output for the unperturbed input is 1. The figure plots the loss incurred for a particular value of the logit (i.e. the pre-activation fed into the output sigmoid nonlinearity) for the perturbed input. As the logit grows towards infinity, the model predicts a class label of 1 (in agreement with the prediction for the unperturbed input); as it grows towards negative infinity, the model predictions class 0. The squared error loss saturates (and has zero gradients) when the model predicts one class or the other with high probability, but the KL divergence grows without bound as the model predicts class 0 with more and more confidence.

can achieve state-of-the-art performance on semi-supervised learning tasks by exploiting the geometric properties of consistency regularization.

A final consideration when using consistency regularization is the function used to measure the difference between the network's output with and without perturbations. Equation (19.20) uses the squared L2 distance (also referred to as the Brier score), which is a common choice [SJT16; TV17; LA16; Ber+19a]. It is also common to use the KL divergence $\mathbb{KL}\left(p_\theta(y|\mathbf{x})\|p_\theta(y|\mathbf{x}')\right.$ in analogy with the cross-entropy loss (i.e. KL divergence between ground-truth label and prediction) used for labeled examples [Miy+18; Ber+19b; Xie+19b]. The gradient of the squared-error loss approaches zero as the model's predictions on the perturbed and unperturbed input differ more and more, assuming the model uses a softmax nonlinearity on its output. Using the squared-error loss therefore has a possible advantage that the model is not updated when its predictions are very unstable. However, the KL divergence has the same scale as the cross-entropy loss used for labeled data, which makes for more intuitive tuning of the unlabeled loss hyperparameter λ. A comparison of the two loss functions is shown in Fig. 19.9.

19.3.6 Deep generative models *

Generative models provide a natural way of making use of unlabeled data through learning a model of the marginal distribution by minimizing $\mathcal{L}_U = -\sum_n \log p_{\boldsymbol{\theta}}(\mathbf{x}_n)$. Various approaches have leveraged generative models for semi-supervised by developing ways to use the model of $p_{\boldsymbol{\theta}}(\mathbf{x}_n)$ to help produce a better supervised model.

19.3.6.1 Variational autoencoders

In Sec. 20.3.5, we describe the variational autoencoder (VAE), which defines a probabilistic model of the joint distribution of data $\mathbf{x}$ and latent variables $\mathbf{z}$. Data is assumed to be generated by first

sampling $\mathbf{z} \sim p(\mathbf{z})$ and then sampling $\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$. For learning, the VAE uses an encoder $\mathbf{q}_\lambda(\mathbf{z}|\mathbf{x})$ to approximate the posterior and a decoder $p_\theta(\mathbf{x}|\mathbf{z})$ to approximate the likelihood. The encoder and decoder are typically deep neural networks. The parameters of the encoder and decoder can be jointly trained by maximizing the evidence lower bound (ELBO) of data.

The marginal distribution of latent variables $p(\mathbf{z})$ is often chosen to be a simple distribution like a diagonal-covariance Gaussian. In practice, this can make the latent variables $\mathbf{z}$ more amenable to downstream classification thanks to the facts that $\mathbf{z}$ is typically lower-dimensional than $\mathbf{x}$, that $\mathbf{z}$ is constructed via cascaded nonlinear transformations, and that the dimensions of the latent variables are designed to be independent. In other words, the latent variables can provide a (learned) representation where data may be more easily separable. In [Kin+14], this approach is called **M1** and it is indeed shown that the latent variables can be used to train stronger models when labels are scarce. (The general idea of unsupervised learning of representations to help with downstream classification tasks is described further in Sec. 19.2.3.)

An alternative approach to leveraging VAEs, also proposed in [Kin+14] and called **M2**, has the form

$$p_{\boldsymbol{\theta}}(\mathbf{x}, y) = p_{\boldsymbol{\theta}}(y)p_{\boldsymbol{\theta}}(\mathbf{x}|y) = p_{\boldsymbol{\theta}}(y) \int p_{\boldsymbol{\theta}}(\mathbf{x}|y, \mathbf{z})p_{\boldsymbol{\theta}}(\mathbf{z})d\mathbf{z} \tag{19.23}$$

where $\mathbf{z}$ is a latent variable, $p_{\boldsymbol{\theta}}(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$ is the latent prior, $p_{\boldsymbol{\theta}}(y) = \text{Cat}(y|\boldsymbol{\pi})$ the label prior, and $p_{\boldsymbol{\theta}}(\mathbf{x}|y, \mathbf{z}) = p(\mathbf{x}|f_{\boldsymbol{\theta}}(y, \mathbf{z}))$ is the likelihood, such as a Gaussian, with parameters computed by f (a deep neural network). The main innovation of this approach is to assume that data is generated according to both a latent class variable y as well as the continuous latent variable $\mathbf{z}$. The class variable y is observed for labeled data and unobserved for unlabled data.

To compute the likelihood for labeled data, $p_{\boldsymbol{\theta}}(\mathbf{x}, y)$, we need to marginalize over $\mathbf{z}$, which can do approximately by using an inference network of the form $q_{\boldsymbol{\omega}}(\mathbf{z}|y, \mathbf{x}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_{\boldsymbol{\omega}}(y, \mathbf{x}), \text{diag}(\sigma^2_{\boldsymbol{\omega}}(\mathbf{x}))$. We then use the following variational lower bound

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}, y) \geq \mathbb{E}_{q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x},y)} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|y, \mathbf{z}) + \log p_{\boldsymbol{\theta}}(y) + \log p_{\boldsymbol{\theta}}(\mathbf{z}) - \log q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}, y)\right] = -\mathcal{L}(\mathbf{x}, y) \tag{19.24}$$

as is standard for VAEs (see Sec. 20.3.5). The only difference is that we observe two kinds of data: $\mathbf{x}$ and y.

To compute the likelihood for unlabeled data, $p_{\boldsymbol{\theta}}(\mathbf{x})$, we need to marginalize over $\mathbf{z}$ and y, which can do approximately by using an inference network of the form

$$q_{\boldsymbol{\omega}}(\mathbf{z}, y|\mathbf{x}) = q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})q_{\boldsymbol{\omega}}(y|\mathbf{x}) \tag{19.25}$$

$$q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_{\boldsymbol{\omega}}(\mathbf{x}), \text{diag}(\sigma^2_{\boldsymbol{\omega}}(\mathbf{x})) \tag{19.26}$$

$$q_{\boldsymbol{\omega}}(y|\mathbf{x}) = \text{Cat}(y|\boldsymbol{\pi}_{\boldsymbol{\omega}}(\mathbf{x})) \tag{19.27}$$

Note that $q_{\boldsymbol{\omega}}(y|\mathbf{x})$ acts like a discriminative classifier, that imputes the missing labels. We then use the following variational lower bound:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \mathbb{E}_{q_{\boldsymbol{\omega}}(\mathbf{z},y|\mathbf{x})} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|y, \mathbf{z}) + \log p_{\boldsymbol{\theta}}(y) + \log p_{\boldsymbol{\theta}}(\mathbf{z}) - \log q_{\boldsymbol{\omega}}(\mathbf{z}, y|\mathbf{x})\right] \tag{19.28}$$

$$= -\sum_y q_{\boldsymbol{\omega}}(y|\mathbf{x})\mathcal{L}(\mathbf{x}, y) + \mathbb{H}\left(q_{\boldsymbol{\omega}}(y|\mathbf{x})\right) = -\mathcal{U}(\mathbf{x}) \tag{19.29}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Note that the discriminative classifier $q_{\boldsymbol{\omega}}(y|\mathbf{x})$ is only used to compute the loglikelihood of the unlabeled data, which is undesirable. We can therefore add an extra classification loss on the supervised data, to get the following overall objective function:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}_L}\left[\mathcal{L}(\mathbf{x},y)\right] + \mathbb{E}_{\mathbf{x}\sim\mathcal{D}_U}\left[\mathcal{U}(\mathbf{x})\right] + \alpha\mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}_L}\left[-\log q_{\boldsymbol{\omega}}(y|\mathbf{x})\right] \tag{19.30}$$

where α is a hyperparameter that controls the relative weight of generative and discriminative learning.

Of course, the probablistic model used in M2 is just one of many ways to decompose the dependencies between the observed data, the class labels, and the continuous latent variables. There are also many ways other than variational inference to perform approximate inference. The best technique will be problem dependent, but overall the main advantage of the generative approach is that we can incorporate domain knowledge. For example, we can model the missing data mechanism, since the absence of a label may be informative about the underlying data (e.g., people may be reluctant to answer a survey question about their health if they are unwell).

19.3.6.2 Generative adversarial networks

Generative adversarial networks (GANs) (described in more detail in the sequel to this book, [Mur22]) are a popular class of generative models that learn an implicit model of the data distribution. They consist of a generator network, which maps samples from a simple latent distribution to the data space, and a critic network, which attempts to distinguish between the outputs of the generator and samples from the true data distribution. The generator is trained to generate samples that the critic classifies as "real".

Since standard GANs do not produce a learned latent representation of a given datapoint and do not learn an explicit model of the data distribution, we cannot use the same approaches as were used for VAEs. Instead, semi-supervised learning with GANs is typically done by modifying the critic so that it outputs either a class label or "fake" instead of simply classifying real vs. fake [Sal+16; Ode16]. For labeled real data, the critic is trained to output the appropriate class label, and for unlabeled real data, it is trained to raise the probability of any of the class labels. As with standard GAN training, the critic is trained to classify outputs from the generator as fake and the generator is trained to fool the critic.

In more detail, let $p_\theta(y|\mathbf{x})$ denote the critic with $C+1$ outputs corresponding to C classes plus a "fake" class, and let $G(\mathbf{z})$ denote the generator which takes as input samples from the prior distribution $p(\mathbf{z})$. Let us assume that we are using the standard cross-entropy GAN loss as originally proposed in [Goo+14]. Then the critic's loss is

$$-\mathbb{E}_{\mathbf{x},y\sim p(\mathbf{x},y)}\log p_\theta(y|\mathbf{x}) - \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}\log[1-p_\theta(y=C+1|\mathbf{x})] - \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\log p_\theta(y=C+1|G(\mathbf{z})) \tag{19.31}$$

This tries to maximize the probability of the correct class for the labeled examples, to minimize the probability of the fake class for real unlabeled examples, and to maximize the probability of the take class for generated examples. The generator's loss is simpler, namely

$$\mathbb{E}_{z\sim p(\mathbf{z})}\log p_\theta(y=C+1|G(\mathbf{z})) \tag{19.32}$$

A diagram visualizing the semi-supervised GAN framework is shown in Fig. 19.10.

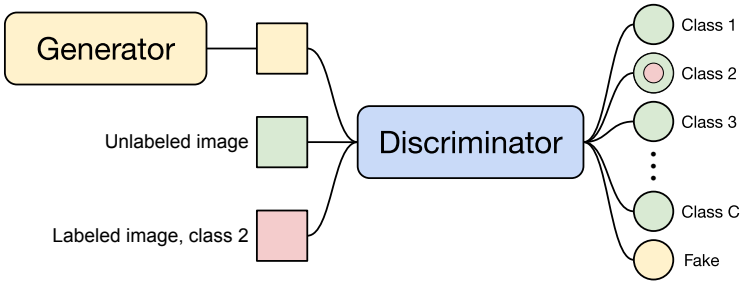


Figure 19.10: Diagram of the semi-supervised GAN framework. The discriminator is trained to output the class of labeled datapoints (red), a "fake" label for outputs from the generator (yellow), and any label for unlabeled data (green).

19.3.6.3 Normalizing flows

Normalizing flows (described in more detail in the sequel to this book, [Mur22]) are a tractable way to define deep generative models. More precisely, they define an invertible mapping $f_\theta : \mathcal{X} \to \mathcal{Z}$, with parameters θ, from the data space $\mathcal{X}$ to the latent space $\mathcal{Z}$. The density in data space can be written starting from the density in the latent space using the change of variables formula:

$$p(x) = p(f(x)) \cdot \left| \det \left(\frac{\partial f}{\partial x} \right) \right|. \tag{19.33}$$

We can extend this to semi-supervised learning, as proposed in [Izm+20]. For class labels $y \in \{1 \dots \mathcal{C}\}$, we can specify the latent distribution, conditioned on a label k, as Gaussian with mean μ_k and covariance Σ_k: $p(z|y=k) = \mathcal{N}(z|\mu_k, \Sigma_k)$. The marginal distribution of z is then a Gaussian mixture. The likelihood for labeled data is then

$$p_{\mathcal{X}}(x|y=k) = \mathcal{N}\left(f(x)|\mu_k, \Sigma_k\right) \cdot \left| \det \left(\frac{\partial f}{\partial x} \right) \right|, \tag{19.34}$$

and the likelihood for data with unknown label is $p(x) = \sum_k p(x|y=k)p(y=k)$.

For semi-supervised learning we can then maximize the joint likelihood of the labeled $\mathcal{D}_\ell$ and unlabeled data $\mathcal{D}_u$:

$$p(\mathcal{D}_\ell, \mathcal{D}_u|\theta) = \prod_{(x_i,y_i)\in\mathcal{D}_\ell} p(x_i, y_i) \prod_{x_j \in \mathcal{D}_u} p(x_j), \tag{19.35}$$

over the parameters θ of the bijective function f, which learns a density model for a Bayes classifier.

Given a test point x, the model predictive distribution is given by

$$p_{\mathcal{X}}(y|x) = \frac{p(x|y)p(y)}{p(x)} = \frac{\mathcal{N}\left(f(x)|\mu_y, \Sigma_y\right)}{\sum_{k=1}^{\mathcal{C}} \mathcal{N}(f(x)|\mu_k, \Sigma_k)}. \tag{19.36}$$

We can make predictions for a test point x with the Bayes decision rule $y = \arg\max_{c\in\{1,\dots,\mathcal{C}\}} p(y=c|x)$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

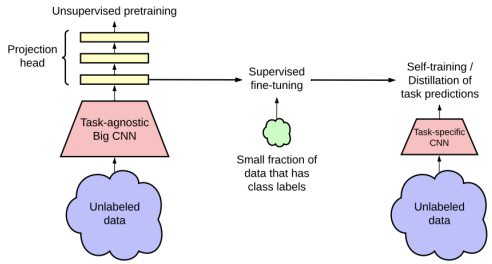


Figure 19.11: Combinng self-supervised learning on unlabeled data (left), supervised fine-tuning (middle), and self-training on pseudo-labeled data (right). From Figure 3 of [Che+20d]. Used with kind permission of Ting Chen

19.3.7 Combining self-supervised and semi-supervised learning

It is possible to combine self-supervised and semi-supervised learning. For example, [Che+20d] using SimCLR (Sec. 19.2.3.3) to perform self-supervised representation learning on the unlabeled data, they then fine-tune this representation on a small labeled dataset (as in transfer learning, Sec. 19.2), and finally, they apply the trained model back to the original unlabeled dataset, and distill the predictions from this teacher model T into a student model S. (**Knowledge distillation** is the name given to the approach of training one model on the predictions of another, as originally proposed in [HVD14].) That is, after fine-tuning T, they train S by minimizing

$$\mathcal{L}(T) = -\sum_{\mathbf{x}_i \in \mathcal{D}} \left[\sum_y p^T(y|\mathbf{x}_i;\tau) \log p^S(y|\mathbf{x}_i;\tau) \right] \tag{19.37}$$

where $\tau > 0$ is a temperature parameter applied to the softmax output, which is used to perform **label smoothing**. If S has the same form as T, this is known as **self-training**, as discussed in Sec. 19.3.1. However, normally the student S is smaller than the teacher T. (For example, T might be a high capacity model, and S is a lightweight version that runs on a phone.) See Fig. 19.11 for an illustration of the overall approach.

19.4 Active learning

In **active learning**, the goal is to identify the true predictive mapping $y = f(\mathbf{x})$ by querying as few $(\mathbf{x}, y)$ points as possible. There are three main variants. In **query synthesis**, the algorithm gets to choose any input $\mathbf{x}$, and can ask for its corresponding output $y = f(\mathbf{x})$. In **pool-based active learning**, there is a large, but fixed, set of unlabeled data points, and the algorithm gets to ask for a label for one or more of these points. Finally, in **stream-based active learning**, the incoming data is arriving continuously, and the algorithm must choose whether it wants to request a label for the current input or not.

There are various closely related problems. In **Bayesian optimization** (Sec. 8.8.3), the goal is to estimate the location of the global optimum $\mathbf{x}^* = \text{argmin}_{\mathbf{x}} f(\mathbf{x})$ in as few queries as possible; typically we fit a surrogate (response surface) model to the intermediate $(\mathbf{x}, y)$ queries, to decide

which question to ask next. In **experiment design**, the goal is to infer a parameter vector of some model, using carefully chosen data samples $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, i.e. we want to estimate $p(\boldsymbol{\theta}|\mathcal{D})$ using as little data as possible. (This can be thought of as an unsupervised, or generalized, form of active learning.)

In this section, we give a brief review of the pool based approach to active learning. For more details, see e.g., [Set12] for a review.

19.4.1 Decision-theoretic approach

In the decision theoretic approach to active learning, proposed in [KHB07; RM01], we define the utility of querying $\mathbf{x}$ in terms of the **value of information**. In particular, we define the utility of issuing query $\mathbf{x}$ as

$$U(\mathbf{x}) \triangleq \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\min_a R(a|\mathcal{D}) - R(a|\mathcal{D},(\mathbf{x},y))\right] \tag{19.38}$$

where $R(a|\mathcal{D}) = \mathbb{E}_{p(\theta|\mathcal{D})}\left[\ell(\theta,a)\right]$ is the posterior expected loss of taking some future action a given the data $\mathcal{D}$ observed so far. Unfortunately, evaluating $U(\mathbf{x})$ for each $\mathbf{x}$ is quite expensive, since for each possible response y we might observe, we have to update our beliefs given $(\mathbf{x}, y)$ to see what affect it might have on our future decisions (similar to look ahead search technique applied to belief states).

19.4.2 Information-theoretic approach

In the information theoretic approach to active supervised learning, we avoid using task-specific loss functions, and instead focus on learning our model as well as we can. In particular, [Lin56] proposed to define the utility of querying $\mathbf{x}$ in terms of **information gain** about the parameters $\boldsymbol{\theta}$, i.e., the reduction in entropy:

$$U(\mathbf{x}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D},\mathbf{x},y)\right)\right] \tag{19.39}$$

(Note that the first term is a constant wrt $\mathbf{x}$, but we include it for later convenience.) Exercise 19.1 asks you to show that this objective is identical to the expected change in the posterior over the parameters which is given by

$$U'(\mathbf{x}) \triangleq \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\mathbb{KL}\left(p(\boldsymbol{\theta}|\mathcal{D},\mathbf{x},y) \| p(\boldsymbol{\theta}|\mathcal{D})\right)\right] \tag{19.40}$$

Using symmetry of the mutual information, we can rewrite Eq. (19.39) as follows:

$$\begin{aligned} U(\mathbf{x}) &= \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D},\mathbf{x},y)\right)\right] && (19.41)\\ &= \mathbb{I}(\boldsymbol{\theta}, y|\mathcal{D},\mathbf{x}) && (19.42)\\ &= \mathbb{H}\left(p(y|\mathbf{x},\mathcal{D})\right) - \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}\left[\mathbb{H}\left(p(y|\mathbf{x},\boldsymbol{\theta})\right)\right] && (19.43) \end{aligned}$$

The advantage of this approach is that we now only have to reason about the uncertainty of the predictive distribution over outputs y, not over the parameters $\boldsymbol{\theta}$.

Eq. (19.43) has an interesting interpretation. The first term prefers examples $\mathbf{x}$ for which there is uncertainty in the predicted label. Just using this as a selection criterion is called **maximum**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

entropy sampling [SW87]. However, this can have problems with examples which are inherently ambiguous or mislabeled. The second term in Eq. (19.43) will discourage such behavior, since it prefers examples $\mathbf{x}$ for which the predicted label is fairly certain once we know $\boldsymbol{\theta}$; this will avoid picking inherently hard-to-predict examples. In other words, Eq. (19.43) will select examples $\mathbf{x}$ for which the model makes confident predictions which are highly diverse. This approach has therefore been called **Bayesian active learning by disagreement** or **BALD** [Hou+12].

This method can be used to train classifiers for other domains where expert labels are hard to acquire, such as medical images or astronomical images [Wal+20].

19.4.3 Batch active learning

So far, we have assumed a greedy or **myopic** strategy, in which we select a single example $\mathbf{x}$, as if it were the last datapoint to be selected. But sometimes we have a budget to collect a set of B samples, call them $(\mathbf{X}, \mathbf{Y})$. In this case, the information gain criterion becomes $U(\mathbf{X}) = \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(\mathbf{Y}|\mathbf{X},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathbf{Y},\mathbf{X},\mathcal{D})\right)\right]$. Unfortunately, optimizing this is NP-hard in the horizon length B [KLQ95; KG05].

Fortunately, under certain conditions, the greedy strategy is near-optimal, as we now explain. First note that, for any given $\mathbf{X}$, the information gain function $f(\mathbf{Y}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{H}\left(p(\boldsymbol{\theta}|\mathbf{Y},\mathbf{X},\mathcal{D})\right)$ maps a set of labels $\mathbf{Y}$ to a scalar. It is clear that $f(\emptyset) = 0$, and that f is non-decreasing, meaning $f(Y^{\text{large}}) \geq f(Y^{\text{small}})$, due to the "more information never hurts" principle. Furthermore, [KG05] proved that f is **submodular**. As a consequence is that a sequential greedy approach is within a constant factor of optimal. If we combine this greedy technique with the BALD objective, we get a method called **BatchBALD** [KAG19].

19.5 Meta-learning *

The field of **meta learning**, also called **learning to learn** [TP97], is concerned with learning multiple related functions (often called "**tasks**"). For example, suppose we have a set of J related datasets, $\{\mathcal{D}^j : j = 1 : J\}$, where $\mathcal{D}^j = \{(\mathbf{x}_n^j, y_n^j) : n = 1 : N_j\}$, $\mathbf{x}_n^j \sim p(\mathbf{x})$, and $y_n^j = f^j(\mathbf{x}_n^j)$. (For example, $p(\mathbf{x})$ could be a distribution over images, f^1 could be a function mapping images to dog breeds, f^2 could be a function mapping images to car types, etc.) We can apply this inner algorithm to generate a labeled set $\mathcal{D}_{\text{meta}} = \{(\mathcal{D}^j, f^j) : j = 1 : J\}$, which we can use to learn an outer or "meta" algorithm or function $\hat{M}$ that *maps a dataset to a prediction function*, $\hat{f}^j = \hat{M}(\mathcal{D}^j)$ (c.f., [Min99]). By contrast, conventional supervised learning learns a function $\hat{f}$ that maps a single example to a label, $\hat{y}_n = \hat{f}(\mathbf{x}_n)$.

There are many other approaches to meta-learning (see e.g., [Van18] for a survey). In Sec. 19.5.1, we discuss one popular approach, known as **MAML**. There are also many applications of meta-learning, such as few-shot learning (see Sec. 19.6).

19.5.1 Model-agnostic meta-learning (MAML)

A natural approach to meta learning is to use a hierarchical Bayesian model, as illustrated in Fig. 19.12. The parameters for each task $\boldsymbol{\theta}_j$ are assumed to come from a common prior, $p(\boldsymbol{\theta}_j|\boldsymbol{\phi})$, which can be used to help pool statistical strength from multiple data-poor problems.

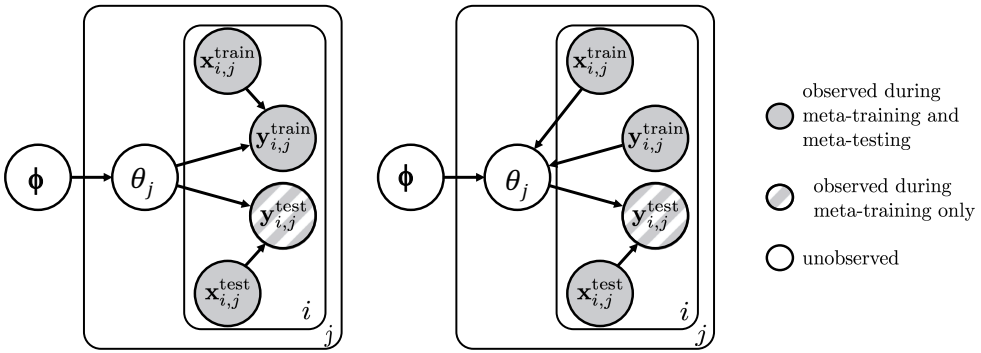


Figure 19.12: Graphical model corresponding to MAML. Left: generative model. Right: During meta-training, each of the task parameters $\boldsymbol{\theta}_j$'s are updated using their local datasets. The indices j are over tasks (meta datasets), and i are over instances within each task. Solid shaded nodes are always observed; semi-shaded (striped) nodes are only observed during meta training time (i.e., not at test time). From Figure 1 of [FXL18]. Used with kind permission of Chelsea Finn.

We could perform Bayesian inference for the task parameters $\boldsymbol{\theta}_j$ and the shared hyper-parameters $\boldsymbol{\phi}$, but a more efficient approach is to use the following empirical Bayes (Sec. 4.6.5.3) approximation:

$$\boldsymbol{\phi}^* = \operatorname*{argmax}_{\boldsymbol{\phi}} \frac{1}{J} \sum_{j=1}^{J} \log p(\mathcal{D}^j_{\text{valid}} | \hat{\boldsymbol{\theta}}_j(\boldsymbol{\phi}, \mathcal{D}^j_{\text{train}})) \tag{19.44}$$

where $\hat{\boldsymbol{\theta}}_j = \hat{\boldsymbol{\theta}}(\boldsymbol{\phi}, \mathcal{D}^j_{\text{train}})$ is a point estimate of the parameters for task j based on $\mathcal{D}^j_{\text{train}}$ and prior $\boldsymbol{\phi}$, and where we use a cross-validation approximation to the marginal likelihood (Sec. 5.4.4).

To compute the point estimate of the task parameters, $\hat{\boldsymbol{\theta}}_j$, we use K steps of a gradient ascent procedure starting at $\boldsymbol{\phi}$ with a learning rate of η. This can be shown to be equivalent to an approximate MAP estimate using a Gaussian prior centered at $\boldsymbol{\phi}$, where the strength of the prior is controlled by the number of gradient steps [San96; Gra+18]. (This is an example of **fast adapation** of the task specific weights starting from the shared prior $\boldsymbol{\phi}$.)

Thus we see that learning the prior $\boldsymbol{\phi}$ is equivalent to learning a good initializer for task-specific learning. This is known as **model-agnostic meta-learning** or **MAML** [FAL17].

19.6 Few-shot learning *

People can learn to predict from very few labeled examples. This is called **few-shot learning**. In the extreme in which the person or system learns from a single example of each class, this is called **one-shot learning**, and if no labeled examples are given, it is called **zero-shot learning**.

A common way to evaluate methods for FSL is to use **C-way N-shot classification**, in which the system is expected to learn to classify C classes using just N training examples of each class. Typically N and C are very small, e.g., Fig. 19.13 illustrates the case where we have $C = 3$ classes,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

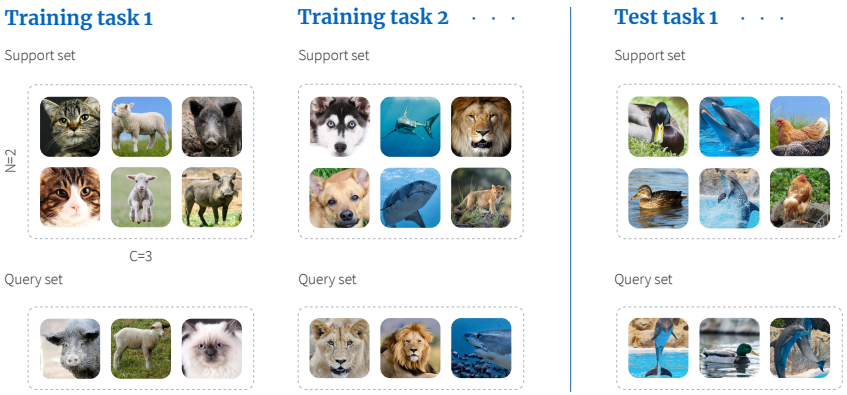


Figure 19.13: Illustration of meta-learning for few-shot learning. Here, each task is a 3-way-2-shot classification problem because each training task contains a support set with three classes, each with two examples. From https://www.borealisai.com/en/blog/tutorial-2-few-shot-learning-and-meta-learning-i. Copyright (2019) Borealis AI. Used with kind permission of Simon Prince and April Cooper.

each with $N = 2$ examples. Since the amount of data from the new domain (here, ducks, dolphins and hens) is so small, we cannot expect to learn from scratch. Therefore we turn to meta-learning.

During training, the meta-algorithm M trains on a labeled support set from group j, returns a predictor f^j, which is then evaluated on a disjoint query set also from group j. We optimize M over all J groups. Finally we can apply M to our new labeled support set to get f^{test}, which is applied to the query set from the test domain. This is illustrated in Fig. 19.13. We see that there is no overlap between the classes in the two training tasks ({cat, lamb, pig} and {dog, shark, lion}) and those in the test task ({duck, dolphin, hen}). Thus the algorithm M must learn to predict image classes in general rather than any particular set of labels.

There are many approaches to few-shot learning. We discuss one such method in Sec. 19.6.1. For more methods, see e.g., [Wan+20b].

19.6.1 Matching networks

One approach is to few shot learning is learn a distance metric on some other dataset, and then to use $d_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{x}')$ inside of a nearest neighbor classifier. Essentially this defines a semi-parametric model of the form $p_{\boldsymbol{\theta}}(y|\mathbf{x}, \mathcal{S})$, where $\mathcal{S}$ is the small labeled dataset (known as the support set), and $\boldsymbol{\theta}$ are the parameters of the distance function. This approach is widely used for **fine-grained classification** tasks, where there are many different visually similar categories, such as face images from a gallery, or product images from a catalog.

An extension of this approach is to learn a function of the form

$$p_{\boldsymbol{\theta}}(y|\mathbf{x}, \mathcal{S}) = \mathbb{I}\left(y = \sum_{n \in \mathcal{S}} a_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{x}_n; \mathcal{S}) y_n\right) \tag{19.45}$$

where $a_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{x}_n; \mathcal{S}) \in \mathbb{R}^+$ is some kind of adaptive similarity kernel. For example, we can use an

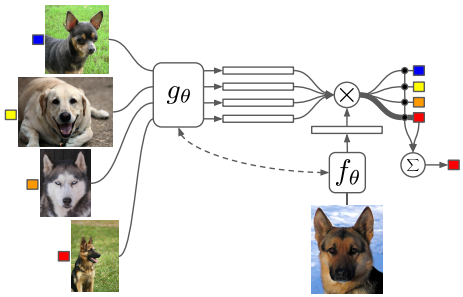


Figure 19.14: Illustration of a matching network for one-shot learning. From Figure 1 of [Vin+16]. Used with kind permission of Oriol Vinyals.

attention kernel of the form

$$a(\mathbf{x}, \mathbf{x}_n; \mathcal{S}) = \frac{\exp(c(f(\mathbf{x}), g(\mathbf{x}_n)))}{\sum_{n'=1}^{N} \exp(c(f(\mathbf{x}), g(\mathbf{x}_{n'})))} \tag{19.46}$$

where $c(\mathbf{u}, \mathbf{v})$ is the cosine distance. (We can make f and g be the same function if we want.) Intuitively, the attention kernel will compare $\mathbf{x}$ to $\mathbf{x}_n$ in the context of all the labeled examples, which provides an implicit signal about which feature dimensions are relevant. (We discuss attention mechanisms in more detail in Sec. 15.4.) This is called a **matching network** [Vin+16]. See Fig. 19.14 for an illustration.

We can train the f and g functions using multiple small datasets, as in meta-learning (Sec. 19.5). More precisely, let $\mathcal{D}$ be a large labeled dataset (e.g., ImageNet), and let $p(\mathcal{L})$ be a distribution over its labels. We create a task by sampling a small set of labels (say 25), $\mathcal{L} \sim p(\mathcal{L})$, and then sampling a small support set of examples from $\mathcal{D}$ with those labels, $\mathcal{S} \sim \mathcal{L}$, and finally sampling a small test set with those same labels, $\mathcal{T} \sim \mathcal{L}$. We then train the model to predict the test labels given the support set, i.e., we optimize the following objective:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \mathbb{E}_{\mathcal{L} \sim p(\mathcal{L})} \left[\mathbb{E}_{\mathcal{S} \sim \mathcal{L}, \mathcal{T} \sim \mathcal{L}} \left[\sum_{(\mathbf{x}, y) \in \mathcal{T}} \log p_{\boldsymbol{\theta}}(y | \mathbf{x}, \mathcal{S}) \right] \right] \tag{19.47}$$

After training, we freeze $\boldsymbol{\theta}$, and apply Eq. (19.45) to a test support set $\mathcal{S}$.

19.7 Exercises

Exercise 19.1 [Information gain equations]

Consider the following two objectives for evaluating the utility of querying a datapoint $\mathbf{x}$ in an active learning setting:

$$U(\mathbf{x}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D}, \mathbf{x}, y)\right)\right] \tag{19.48}$$

$$U'(\mathbf{x}) \triangleq \mathbb{E}_{p(y|\mathbf{x},\mathcal{D})}\left[\mathbb{KL}\left(p(\boldsymbol{\theta}|\mathcal{D}, \mathbf{x}, y) \| p(\boldsymbol{\theta}|\mathcal{D})\right)\right] \tag{19.49}$$

Prove that these are equal.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20 Dimensionality reduction

A common form of unsupervised learning is **dimensionality reduction**, in which we learn a mapping from the high-dimensional visible space, $\mathbf{x} \in \mathbb{R}^D$, to a low-dimensional latent space, $\mathbf{z} \in \mathbb{R}^L$. This mapping can either be a parametric model $\mathbf{z} = f(\mathbf{x}; \boldsymbol{\theta})$ which can be applied to any input, or it can be a nonparametric mapping where we compute an **embedding** $\mathbf{z}_n$ for each input $\mathbf{x}_n$ in the data set, but not for any other points. This latter approach is mostly used for data visualization, whereas the former approach can also be used as a preprocessing step for other kinds of learning algorithms. For example, we might first reduce the dimensionality by learning a mapping from $\mathbf{x}$ to $\mathbf{z}$, and then learn a simple linear classifier on this embedding, by mapping $\mathbf{z}$ to y.

20.1 Principal components analysis (PCA)

The simplest and most widely used form of dimensionality reduction is **principal components analysis** or **PCA**. The basic idea is to find a linear and orthogonal projection of the high dimensional data $\mathbf{x} \in \mathbb{R}^D$ to a low dimensional subspace $\mathbf{z} \in \mathbb{R}^L$, such that the low dimensional representation is a "good approximation" to the original data, in the following sense: if we project or **encode** $\mathbf{x}$ to get $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$, and then unproject or **decode** $\mathbf{z}$ to get $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z}$, then we want $\hat{\mathbf{x}}$ to be close to $\mathbf{x}$ in ℓ_2 distance. In particular, we can define the following **reconstruction error** or **distortion**:

$$\mathcal{L}(\mathbf{W}) \triangleq \frac{1}{N} \sum_{n=1}^{N} ||\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})||_2^2 \tag{20.1}$$

where the encode and decoding stages are both linear maps, as we explain below.

In Sec. 20.1.2, we show that we can minimize this objective by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \overline{\mathbf{x}})(\mathbf{x}_n - \overline{\mathbf{x}})^\top = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c \tag{20.2}$$

where $\mathbf{X}_c$ is a centered version of the $N \times D$ design matrix. In Sec. 20.2.2, we show that this is equivalent to maximizing the likelihood of a latent linear Gaussian model known as probabilistic PCA.

20.1.1 Examples

Before giving the details, we start by showing some examples.

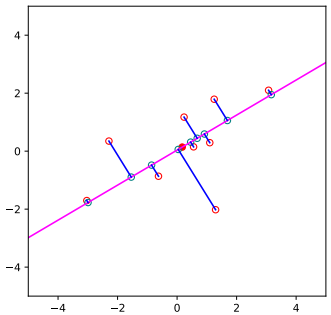


Figure 20.1: An illustration of PCA where we project from 2d to 1d. Circles are the original data points, crosses are the reconstructions. The red star is the data mean. Generated by code at figures.probml.ai/book1/20.1.

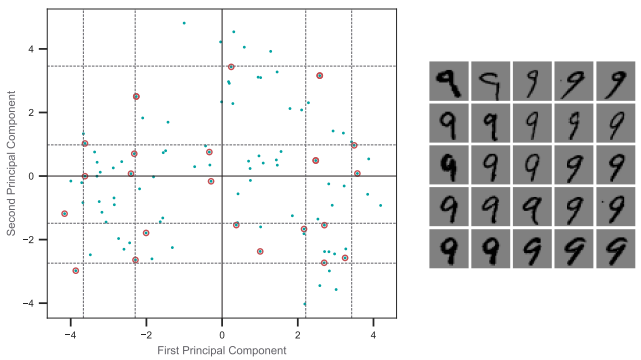
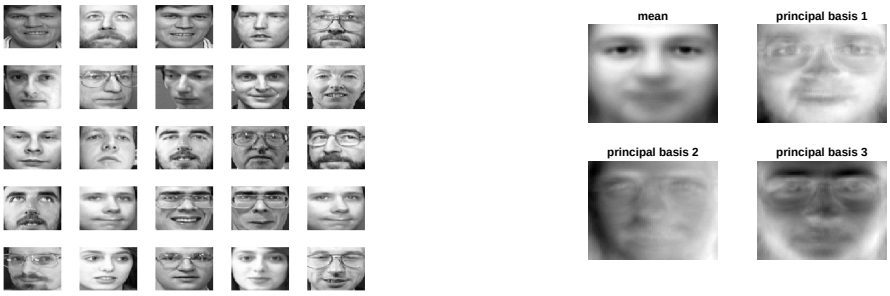


Figure 20.2: An illustration of PCA applied to MNIST digits from class 9. Grid points are at the 5, 25, 50, 75, 95 % quantiles of the data distribution along each dimension. The circled points are the closest projected images to the vertices of the grid. Adapted from Figure 14.23 of [HTF09]. Generated by code at figures.probml.ai/book1/20.2.

Fig. 20.1 shows a very simple example, where we project 2d data to a 1d line. This direction captures most of the variation in the data.

In Fig. 20.2, we show what happens when we project some MNIST images of the digit 9 down to 2d. Although the inputs are high dimensional (specifically $28 \times 28 = 784$ dimensional), the number of "effective degrees of freedom" is much less, since the pixels are correlated, and many digits look similar. Therefore we can represent each image as a point in a low dimensional linear space.

In general, it can be hard to interpet the latent dimensions to which the data is projected. However, by looking at several projected points along a given direction, and the examples from which they are derived, we see that the first principal component (horizontal direction) seems to capture the orientation of the digit, and the second component (vertical direction) seems to capture line thickness.



(a) (b)

Figure 20.3: a) Some randomly chosen 64×64 *pixel images from the Olivetti face database. (b) The mean and the first three PCA components represented as images. Generated by code at figures.probml.ai/book1/20.3.*

In Fig. 20.3, we show PCA applied to another image dataset, knwon as the Olivetti face dataset, which is a set of 64×64 grayscale images. We project these to a 3d subspace. The resulting basis vectors (columns of the projection matrix $\mathbf{W}$) are shown as images in in Fig. 20.3b; these are known as **eigenfaces** [Tur13], for reasons that will be explained in Sec. 20.1.2. We see that the main modes of variation in the data are related to overall lighting, and then differences in the eyebrow region of the face. If we use enough dimensions (but fewer than the 4096 we started with), we can use the represenation $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$ as input to a nearest-neighbor classifier to perform face recognition; this is faster and more reliable than working in pixel space [MWP98].

20.1.2 Derivation of the algorithm

Suppose we have an (unlabeled) dataset $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$, where $\mathbf{x}_n \in \mathbb{R}^D$. We can represent this as an $N \times D$ data matrix $\mathbf{X}$. We will assume $\overline{\mathbf{x}} = \frac{1}{N}\sum_{n=1}^N \mathbf{x}_n = \mathbf{0}$, which we can ensure by centering the data.

We would like to approximate each $\mathbf{x}_n$ by a low dimensional representation, $\mathbf{z}_n \in \mathbb{R}^L$. We assume that each $\mathbf{x}_n$ can be "explained" in terms of a weighted combination of basis functions $\mathbf{w}_1, \ldots, \mathbf{w}_L$, where each $\mathbf{w}_k \in \mathbb{R}^D$, and where the weights are given by $\mathbf{z}_n \in \mathbb{R}^L$, i.e., we assume $\mathbf{x}_n \approx \sum_{k=1}^L z_{nk}\mathbf{w}_k$. The vector $\mathbf{z}_n$ is the low dimensional representation of $\mathbf{x}_n$, and is known as the **latent vector**, since it consists of latent or "hidden" values that are not observed in the data. The collection of these latent variables are called the **latent factors**.

We can measure the error produced by this approximation as follows:

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N}||\mathbf{X} - \mathbf{Z}\mathbf{W}^\top||_F^2 = \frac{1}{N}||\mathbf{X}^\top - \mathbf{W}\mathbf{Z}^\top||_F^2 = \frac{1}{N}\sum_{n=1}^N ||\mathbf{x}_n - \mathbf{W}\mathbf{z}_n||^2 \tag{20.3}$$

where the rows of $\mathbf{Z}$ contain the low dimension versions of the rows of $\mathbf{X}$. This is known as the (average) **reconstruction error**, since we are approximating each $\mathbf{x}_n$ by $\hat{\mathbf{x}}_n = \mathbf{W}\mathbf{z}_n$.

We want to minimize this subject to the constraint that $\mathbf{W}$ is an orthogonal matrix. Below we show that the optimal solution is obtained by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.1.2.1 Base case

Let us start by estimating the best 1d solution, $\mathbf{w}_1 \in \mathbb{R}^D$. We will find the remaining basis vectors $\mathbf{w}_2$, $\mathbf{w}_3$, etc. later.

Let the coefficients for each of the data points associated with the first basis vector be denoted by $\tilde{\mathbf{z}}_1 = [z_{11}, \ldots, z_{N1}] \in \mathbb{R}^N$. The reconstruction error is given by

$$\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N}\sum_{n=1}^{N} ||\mathbf{x}_n - z_{n1}\mathbf{w}_1||^2 = \frac{1}{N}\sum_{n=1}^{N} (\mathbf{x}_n - z_{n1}\mathbf{w}_1)^\top (\mathbf{x}_n - z_{n1}\mathbf{w}_1) \tag{20.4}$$

$$= \frac{1}{N}\sum_{n=1}^{N} [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1}\mathbf{w}_1^\top \mathbf{x}_n + z_{n1}^2 \mathbf{w}_1^\top \mathbf{w}_1] \tag{20.5}$$

$$= \frac{1}{N}\sum_{n=1}^{N} [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1}\mathbf{w}_1^\top \mathbf{x}_i + z_{n1}^2] \tag{20.6}$$

since $\mathbf{w}_1^\top \mathbf{w}_1 = 1$ (by the orthonormality assumption). Taking derivatives wrt z_{n1} and equating to zero gives

$$\frac{\partial}{\partial z_{n1}} \mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N}[-2\mathbf{w}_1^\top \mathbf{x}_n + 2z_{n1}] = 0 \Rightarrow z_{n1} = \mathbf{w}_1^\top \mathbf{x}_n \tag{20.7}$$

So the optimal embedding is obtained by orthogonally projecting the data onto $\mathbf{w}_1$ (see Fig. 20.1(a)). Plugging this back in gives the loss for the weights:

$$\mathcal{L}(\mathbf{w}_1) = \mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1^*(\mathbf{w}_1)) = \frac{1}{N}\sum_{n=1}^{N} [\mathbf{x}_n^\top \mathbf{x}_n - z_{n1}^2] = \text{const} - \frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 \tag{20.8}$$

To solve for $\mathbf{w}_1$, note that

$$\mathcal{L}(\mathbf{w}_1) = -\frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 = -\frac{1}{N}\sum_{n=1}^{N} \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 = -\mathbf{w}_1^\top \hat{\mathbf{\Sigma}} \mathbf{w}_1 \tag{20.9}$$

where $\mathbf{\Sigma}$ is the empirical covariance matrix (since we assumed the data is centered). We can trivially optimize this by letting $||\mathbf{w}_1|| \to \infty$, so we impose the constraint $||\mathbf{w}_1|| = 1$ and instead optimize

$$\tilde{\mathcal{L}}(\mathbf{w}_1) = \mathbf{w}_1^\top \hat{\mathbf{\Sigma}} \mathbf{w}_1 + \lambda_1(\mathbf{w}_1^\top \mathbf{w}_1 - 1) \tag{20.10}$$

where λ_1 is a Lagrange multiplier (see Sec. 8.5.1). Taking derivatives and equating to zero we have

$$\frac{\partial}{\partial \mathbf{w}_1} \tilde{\mathcal{L}}(\mathbf{w}_1) = 2\hat{\mathbf{\Sigma}}\mathbf{w}_1 - 2\lambda_1 \mathbf{w}_1 = 0 \tag{20.11}$$

$$\hat{\mathbf{\Sigma}}\mathbf{w}_1 = \lambda_1 \mathbf{w}_1 \tag{20.12}$$

Hence the optimal direction onto which we should project the data is an eigenvector of the covariance matrix. Left multiplying by $\mathbf{w}_1^\top$ (and using $\mathbf{w}_1^\top \mathbf{w}_1 = 1$) we find

$$\mathbf{w}_1^\top \hat{\mathbf{\Sigma}} \mathbf{w}_1 = \lambda_1 \tag{20.13}$$

Since we want to maximize this quantity (minimize the loss), we pick the eigenvector which corresponds to the largest eigenvalue.

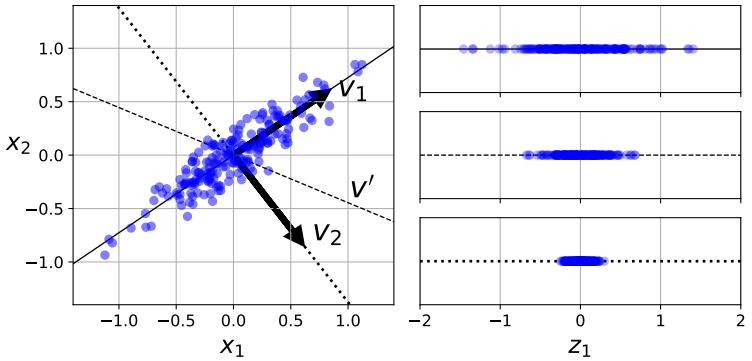


Figure 20.4: Illustration of the variance of the points projected onto different 1d vectors. v_1 is the first principal component, which maximizes the variance of the projection. v_2 is the second principal component which is direction orthogonal to v_1. Finally v' is some other vector in between v_1 and v_2. Adapted from Figure 8.7 of [Gér19]. Generated by code at figures.probml.ai/book1/20.4

20.1.2.2 Optimal weight vector maximizes the variance of the projected data

Before continuing, we make an interesting observation. Since the data has been centered, we have

$$\mathbb{E}\left[z_{n1}\right] = \mathbb{E}\left[\mathbf{x}_n^\top \mathbf{w}_1\right] = \mathbb{E}\left[\mathbf{x}_n\right]^\top \mathbf{w}_1 = 0 \tag{20.14}$$

Hence variance of the projected data is given by

$$\mathbb{V}\left[\tilde{\mathbf{z}}_1\right] = \mathbb{E}\left[\tilde{\mathbf{z}}_1^2\right] - \left(\mathbb{E}\left[\tilde{\mathbf{z}}_1\right]\right)^2 = \frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 - 0 = -\mathcal{L}(\mathbf{w}_1) + \text{const} \tag{20.15}$$

From this, we see that *minimizing* the reconstruction error is equivalent to *maximizing* the variance of the projected data:

$$\arg\min_{\mathbf{w}_1} \mathcal{L}(\mathbf{w}_1) = \arg\max_{\mathbf{w}_1} \mathbb{V}\left[\tilde{\mathbf{z}}_1(\mathbf{w}_1)\right] \tag{20.16}$$

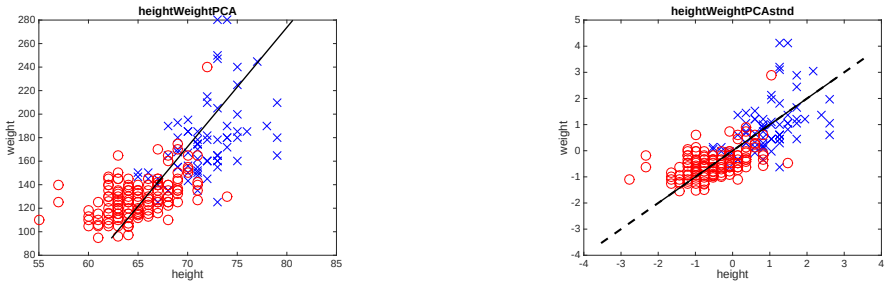
This is why it is often said that PCA finds the directions of maximal variance. (See Fig. 20.4 for an illustration.) However, the minimum error formulation easier to understand and is more general.

20.1.2.3 Induction step

Now let us find another direction $\mathbf{w}_2$ to further minimize the reconstruction error, subject to $\mathbf{w}_1^\top \mathbf{w}_2 = 0$ and $\mathbf{w}_2^\top \mathbf{w}_2 = 1$. The error is

$$\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1, \mathbf{w}_2, \tilde{\mathbf{z}}_2) = \frac{1}{N}\sum_{n=1}^{N} ||\mathbf{x}_n - z_{n1}\mathbf{w}_1 - z_{n2}\mathbf{w}_2||^2 \tag{20.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.5: Effect of standardization on PCA applied to the height/weight dataset. (Red=female, blue=male.) Left: PCA of raw data. Right: PCA of standardized data. Generated by code at figures.probml.ai/book1/20.5.

Optimizing wrt $\mathbf{w}_1$ and $\mathbf{z}_1$ gives the same solution as before. Exercise 20.3 asks you to show that $\frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} = 0$ yields $z_{n2} = \mathbf{w}_2^\top \mathbf{x}_n$. Substituting in yields

$$\mathcal{L}(\mathbf{w}_2) = \frac{1}{n}\sum_{n=1}^{N}[\mathbf{x}_n^\top \mathbf{x}_n - \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 - \mathbf{w}_2^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_2] = \text{const} - \mathbf{w}_2^\top \hat{\mathbf{\Sigma}} \mathbf{w}_2 \tag{20.18}$$

Dropping the constant term, plugging in the optimal $\mathbf{w}_1$ and adding the constraints yields

$$\tilde{\mathcal{L}}(\mathbf{w}_2) = \mathbf{w}_2^\top \hat{\mathbf{\Sigma}} \mathbf{w}_2 + \lambda_2(\mathbf{w}_2^\top \mathbf{w}_2 - 1) + \lambda_{12}(\mathbf{w}_2^\top \mathbf{w}_1 - 0) \tag{20.19}$$

Exercise 20.3 asks you to show that the solution is given by the eigenvector with the second largest eigenvalue:

$$\hat{\mathbf{\Sigma}} \mathbf{w}_2 = \lambda_2 \mathbf{w}_2 \tag{20.20}$$

The proof continues in this way to show that $\hat{\mathbf{W}} = \mathbf{U}_L$.

20.1.3 Computational issues

In this section, we discuss various practical issues related to using PCA.

20.1.3.1 Covariance matrix vs correlation matrix

We have been working with the eigendecomposition of the covariance matrix. However, it is better to use the correlation matrix instead. The reason is that otherwise PCA can be "misled" by directions in which the variance is high merely because of the measurement scale. Fig. 20.5 shows an example of this. On the left, we see that the vertical axis uses a larger range than the horizontal axis. This results in a first principal component that looks somewhat "unnatural". On the right, we show the results of PCA after standardizing the data (which is equivalent to using the correlation matrix instead of the covariance matrix); the results look much better.

20.1.3.2 Dealing with high-dimensional data

We have presented PCA as the problem of finding the eigenvectors of the $D \times D$ covariance matrix $\mathbf{X}^\top \mathbf{X}$. If $D > N$, it is faster to work with the $N \times N$ Gram matrix $\mathbf{X}\mathbf{X}^\top$. We now show how to do this.

First, let $\mathbf{U}$ be an orthogonal matrix containing the eigenvectors of $\mathbf{X}\mathbf{X}^\top$ with corresponding eigenvalues in $\mathbf{\Lambda}$. By definition we have $(\mathbf{X}\mathbf{X}^\top)\mathbf{U} = \mathbf{U}\mathbf{\Lambda}$. Pre-multiplying by $\mathbf{X}^\top$ gives

$$(\mathbf{X}^\top \mathbf{X})(\mathbf{X}^\top \mathbf{U}) = (\mathbf{X}^\top \mathbf{U})\mathbf{\Lambda} \tag{20.21}$$

from which we see that the eigenvectors of $\mathbf{X}^\top \mathbf{X}$ are $\mathbf{V} = \mathbf{X}^\top \mathbf{U}$, with eigenvalues given by $\mathbf{\Lambda}$ as before. However, these eigenvectors are not normalized, since $||\mathbf{v}_j||^2 = \mathbf{u}_j^\top \mathbf{X}\mathbf{X}^\top \mathbf{u}_j = \lambda_j \mathbf{u}_j^\top \mathbf{u}_j = \lambda_j$. The normalized eigenvectors are given by

$$\mathbf{V} = \mathbf{X}^\top \mathbf{U} \mathbf{\Lambda}^{-\frac{1}{2}} \tag{20.22}$$

This provides an alternative way to compute the PCA basis. It also allow us to use the kernel trick, as we discuss in Sec. 20.4.6.

20.1.3.3 Computing PCA using SVD

In this section, we show the equivalence between PCA as computed using eigenvector methods (Sec. 20.1) and the truncated SVD.[1]

Let $\mathbf{U}_\Sigma \mathbf{\Lambda}_\Sigma \mathbf{U}_\Sigma^\top$ be the top L eigendecomposition of the covariance matrix $\mathbf{\Sigma} \propto \mathbf{X}^\top \mathbf{X}$ (we assume $\mathbf{X}$ is centered). Recall from Sec. 20.1.2 that the optimal estimate of the projection weights $\mathbf{W}$ is given by the top L eigenvalues, so $\mathbf{W} = \mathbf{U}_\Sigma$.

Now let $\mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \approx \mathbf{X}$ be the L-truncated SVD approximation to the data matrix $\mathbf{X}$. From Eq. (7.169), we know that the right singular vectors of $\mathbf{X}$ are the eigenvectors of $\mathbf{X}^\top \mathbf{X}$, so $\mathbf{V}_X = \mathbf{U}_\Sigma = \mathbf{W}$. (In addition, the eigenvalues of the covariance matrix are related to the singular values of the data matrix via $\lambda_k = s_k^2/N$.)

Now suppose we are interested in the projected points (also called the principal components or PC scores), rather than the projection matrix. We have

$$\mathbf{Z} = \mathbf{X}\mathbf{W} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \mathbf{V}_X = \mathbf{U}_X \mathbf{S}_X \tag{20.23}$$

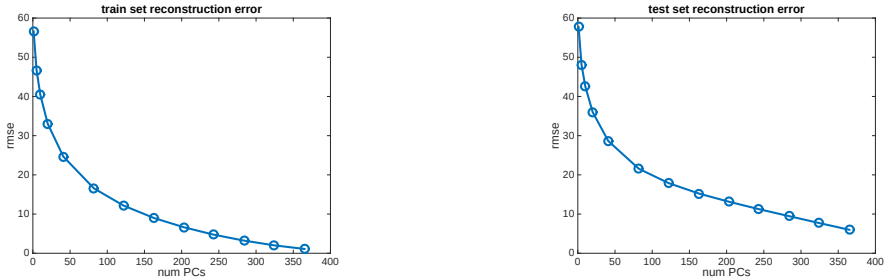
Finally, if we want to approximately reconstruct the data, we have

$$\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}^\top = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \tag{20.24}$$

This is precisely the same as a truncated SVD approximation (Sec. 7.5.5).

Thus we see that we can perform PCA either using an eigendecomposition of $\mathbf{\Sigma}$ or an SVD decomposition of $\mathbf{X}$. The latter is often preferable, for computational reasons. For very high dimensional problems, we can use a randomized SVD algorithm, see e.g., [HMT11; SKT14; DM16]. For example, the randomized solver used by sklearn takes $O(NL^2) + O(L^3)$ time for N examples and L principal components, whereas exact SVD takes $O(ND^2) + O(D^3)$ time.

1. A more detailed explanation can be found at https://bit.ly/2I566OK.



(a) *(b)*

Figure 20.6: Reconstruction error on MNIST vs number of latent dimensions used by PCA. (a) Training set. (b) Test set. Generated by code at figures.probml.ai/book1/20.6.

20.1.4 Choosing the number of latent dimensions

In this section, we discuss how to choose the number of latent dimensions L for PCA.

20.1.4.1 Reconstruction error

Let us define the reconstruction error on some dataset $\mathcal{D}$ incurred by the model when using L dimensions:

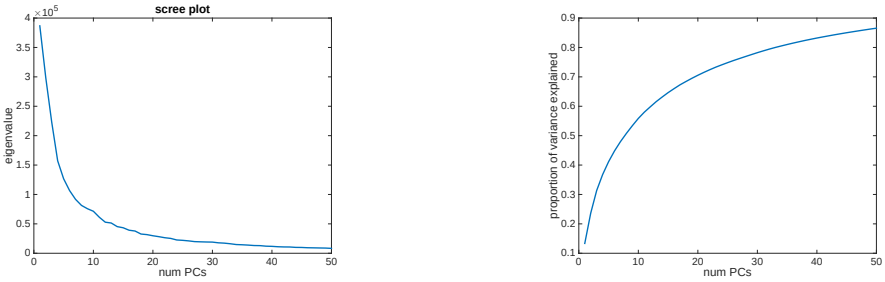
$$\mathcal{L}_L = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}} ||\mathbf{x}_n - \hat{\mathbf{x}}_n||^2 \tag{20.25}$$

where the reconstruction is given by by $\hat{\mathbf{x}}_n = \mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}$, where $\mathbf{z}_n = \mathbf{W}^\top(\mathbf{x}_n - \boldsymbol{\mu})$ and $\boldsymbol{\mu}$ is the empirical mean, and $\mathbf{W}$ is estimated as above. Fig. 20.6(a) plots $\mathcal{L}_L$ vs L on the MNIST training data. We see that it drops off quite quickly, indicating that we can capture most of the empirical correlation of the pixels with a small number of factors.

Of course, if we use $L = \text{rank}(\mathbf{X})$, we get zero reconstruction error on the training set. To avoid overfitting, it is natural to plot reconstruction error on the test set. This is shown in Fig. 20.6(b). Here we see that the error continues to go down even as the model becomes more complex! Thus we do not get the usual U-shaped curve that we typically expect to see in supervised learning. The problem is that PCA is not a proper generative model of the data: If you give it more latent dimensions, it will be able to approximate the test data more accurately. (A similar problem arises if we plot reconstruction error on the test set using K-means clustering, as discussed in Sec. 21.3.7.) We discuss some solutions to this below.

20.1.4.2 Scree plots

A common alternative to plotting reconstruction error vs L is to use something called a **scree plot**, which is a plot of the eigenvalues λ_j vs j in order of decreasing magnitude. One can show



(a) (b)

Figure 20.7: (a) Scree plot for training set, corresponding to Fig. 20.6(a). (b) Fraction of variance explained. Generated by code at figures.probml.ai/book1/20.7.

(Exercise 20.4) that

$$\mathcal{L}_L = \sum_{j=L+1}^{D} \lambda_j \tag{20.26}$$

Thus as the number of dimensions increases, the eigenvalues get smaller, and so does the reconstruction error, as shown in Fig. 20.7a).[2] A related quantity is the **fraction of variance explained**, defined as

$$F_L = \frac{\sum_{j=1}^{L} \lambda_j}{\sum_{j'=1}^{L^{\max}} \lambda_{j'}} \tag{20.27}$$

This captures the same information as the scree plot, but goes up with L (see Fig. 20.7b).

20.1.4.3 Profile likelihood

Although there is no U-shape in the reconstruction error plot, there is sometimes a "knee" or "elbow" in the curve, where the error suddenly changes from relatively large errors to relatively small. The idea is that for $L < L^*$, where L^* is the "true" latent dimensionality (or number of clusters), the rate of decrease in the error function will be high, whereas for $L > L^*$, the gains will be smaller, since the model is already sufficiently complex to capture the true distribution.

One way to automate the detection of this change in the gradient of the curve is to compute the **profile likelihood**, as proposed in [ZG06]. The idea is this. Let λ_L be some measure of the error incurred by a model of size L, such that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{L^{\max}}$. In PCA, these are the eigenvalues, but the method can also be applied to the reoconstruction error from K-means clustering (see Sec. 21.3.7). Now consider partitioning these values into two groups, depending on whether $k < L$ or $k > L$, where L is some threshold which we will determine. To measure the quality of L,

2. The reason for the term "scree plot" is that "the plot looks like the side of a mountain, and 'scree' refers to the debris fallen from a mountain and lying at its base". (Quotation from Kenneth Janda, https://bit.ly/2kqG1yW.)

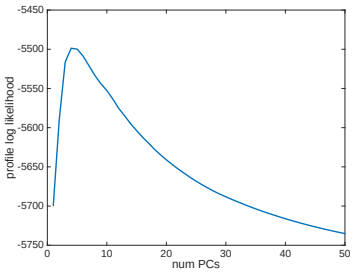


Figure 20.8: Profile likelihood corresponding to PCA model in Fig. 20.6(a). Generated by code at figures.probml.ai/book1/20.8.

we will use a simple change-point model, where $\lambda_k \sim \mathcal{N}(\mu_1, \sigma^2)$ if $k \leq L$, and $\lambda_k \sim \mathcal{N}(\mu_2, \sigma^2)$ if $k > L$. (It is important that σ^2 be the same in both models, to prevent overfitting in the case where one regime has less data than the other.) Within each of the two regimes, we assume the λ_k are iid, which is obviously incorrect, but is adequate for our present purposes. We can fit this model for each $L = 1 : L^{\max}$ by partitioning the data and computing the MLEs, using a pooled estimate of the variance:

$$\mu_1(L) = \frac{\sum_{k \leq L} \lambda_k}{L} \tag{20.28}$$

$$\mu_2(L) = \frac{\sum_{k > L} \lambda_k}{L^{\max} - L} \tag{20.29}$$

$$\sigma^2(L) = \frac{\sum_{k \leq L} (\lambda_k - \mu_1(L))^2 + \sum_{k > L} (\lambda_k - \mu_2(L))^2}{L^{\max}} \tag{20.30}$$

We can then evaluate the profile log likelihood

$$\ell(L) = \sum_{k=1}^{L} \log \mathcal{N}(\lambda_k | \mu_1(L), \sigma^2(L)) + \sum_{k=L+1}^{L^{\max}} \log \mathcal{N}(\lambda_k | \mu_2(L), \sigma^2(L)) \tag{20.31}$$

This is illustrated in Fig. 20.8. We see that the peak $L^* = \operatorname{argmax} \ell(L)$ is well determined.

20.2 Factor analysis *

PCA is a simple method for computing a linear low-dimensional representation of data. In this section, we present a generalization of PCA known as **factor analysis**. This is based on a probabilistic model, which means we can treat it as a building block for more complex models, such as the mixture of FA models in Sec. 20.2.6, or the nonlinear FA model in Sec. 20.3.5. We can recover PCA as a special limiting case of FA, as we discuss in Sec. 20.2.2.

20.2.1 Generative model

Factor analysis corresponds to the following linear-Gaussian latent variable generative model:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) \tag{20.32}$$

$$p(\mathbf{x}|\mathbf{z}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \tag{20.33}$$

where $\mathbf{W}$ is a $D \times L$ matrix, known as the **factor loading matrix**, and $\boldsymbol{\Psi}$ is a diagonal $D \times D$ covariance matrix.

FA can be thought of as a low-rank version of a Gaussian distribution. To see this, note that the induced marginal distribution $p(\mathbf{x}|\boldsymbol{\theta})$ is a Gaussian (see Eq. (3.39) for the derivation):

$$p(\mathbf{x}|\boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})\mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)d\mathbf{z} \tag{20.34}$$

$$= \mathcal{N}(\mathbf{x}|\mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}, \boldsymbol{\Psi} + \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\top) \tag{20.35}$$

Hence $\mathbb{E}[\mathbf{x}] = \mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}$ and $\text{Cov}[\mathbf{x}] = \mathbf{W}\text{Cov}[\mathbf{z}]\mathbf{W}^\top + \boldsymbol{\Psi} = \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\top + \boldsymbol{\Psi}$. From this, we see that we can set $\boldsymbol{\mu}_0 = \mathbf{0}$ without loss of generality, since we can always absorb $\mathbf{W}\boldsymbol{\mu}_0$ into $\boldsymbol{\mu}$. Similarly, we can set $\boldsymbol{\Sigma}_0 = \mathbf{I}$ without loss of generality, since we can always absorb a correlated prior by using a new weight matrix, $\tilde{\mathbf{W}} = \mathbf{W}\boldsymbol{\Sigma}_0^{-\frac{1}{2}}$. After these simplifications we have

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I}) \tag{20.36}$$

$$p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \tag{20.37}$$

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\top + \boldsymbol{\Psi}) \tag{20.38}$$

For example, suppose where $L = 1$, $D = 2$ and $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$. We illustrate the generative process in this case in Fig. 20.9. We can think of this as taking an isotropic Gaussian "spray can", representing the likelihood $p(\mathbf{x}|\mathbf{z})$, and "sliding it along" the 1d line defined by $\mathbf{w}z + \boldsymbol{\mu}$ as we vary the 1d latent prior z. This induces an elongated (and hence correlated) Gaussian in 2d. That is, the induced distribution has the form $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{w}\mathbf{w}^\top + \sigma^2\mathbf{I})$.

In general, FA approximates the covariance matrix of the visible vector using a low-rank decomposition:

$$\mathbf{C} = \text{Cov}[\mathbf{x}] = \mathbf{W}\mathbf{W}^\top + \boldsymbol{\Psi} \tag{20.39}$$

This only uses $O(LD)$ parameters, which allows a flexible compromise between a full covariance Gaussian, with $O(D^2)$ parameters, and a diagonal covariance, with $O(D)$ parameters.

From Eq. (20.39), we see that we should restrict $\boldsymbol{\Psi}$ to be diagonal, otherwise we could set $\mathbf{W} = \mathbf{0}$, thus ignoring the latent factors, while still being able to model any covariance. The marginal variance of each visible variable is given by $\mathbb{V}[x_d] = \sum_{k=1}^{L} w_{dk}^2 + \psi_d$, where the first term is the variance due to the common factors, and the second ψ_d term is called the **uniqueness**, and is the variance term that is specific to that dimension.

We can estimate the parameters of an FA model using EM (see Sec. 20.2.3). Once we have fit the model, we can compute probabilistic latent embeddings using $p(\mathbf{z}|\mathbf{x})$. Using Bayes rule for Gaussians we have

$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\mathbf{W}^\top\mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}), \mathbf{I} - \mathbf{W}^\top\mathbf{C}^{-1}\mathbf{W}) \tag{20.40}$$

where $\mathbf{C}$ is defined in Eq. (20.39).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

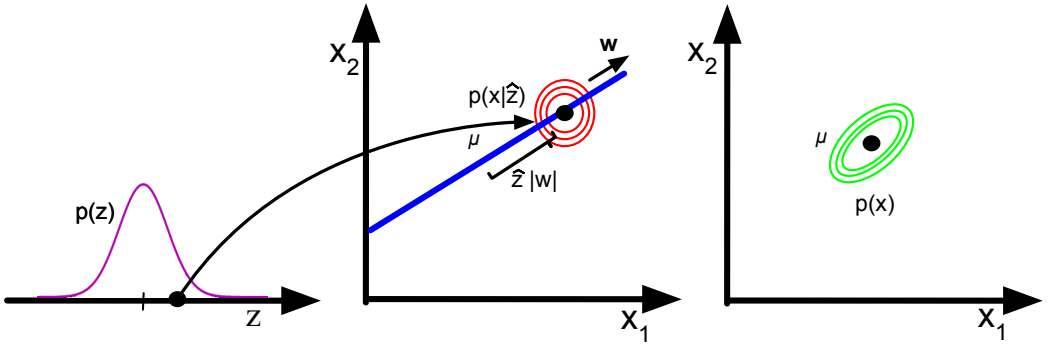


Figure 20.9: Illustration of the FA generative process, where we have $L = 1$ latent dimension generating $D = 2$ observed dimensions; we assume $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$. The latent factor has value $z \in \mathbb{R}$, sampled from $p(z)$; this gets mapped to a 2d offset $\boldsymbol{\delta} = z\mathbf{w}$, where $\mathbf{w} \in \mathbb{R}^2$, which gets added to $\boldsymbol{\mu}$ to define a Gaussian $p(\mathbf{x}|z) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu} + \boldsymbol{\delta}, \sigma^2\mathbf{I})$. By integrating over z, we "slide" this circular Gaussian "spray can" along the principal component axis $\mathbf{w}$, which induces elliptical Gaussian contours in $\mathbf{x}$ space centered on $\boldsymbol{\mu}$. Adapted from Figure 12.9 of [Bis06].

20.2.2 Probabilistic PCA

In this section, we consider a special case of the factor analysis model in which $\mathbf{W}$ has orthonormal columns, $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$ and $\boldsymbol{\mu} = \mathbf{0}$. This model is called **probabilistic principal components analysis** (**PPCA**) [TB99], or **sensible PCA** [Row97]. The marginal distribution on the visible variables has the form

$$p(\mathbf{x}|\boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I})\mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})d\mathbf{z} = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{C}) \tag{20.41}$$

where

$$\mathbf{C} = \mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I} \tag{20.42}$$

The log likelihood for PPCA is given by

$$\log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{ND}{2}\log(2\pi) - \frac{N}{2}\log|\mathbf{C}| - \frac{1}{2}\sum_{n=1}^{N}(\mathbf{x}_n - \boldsymbol{\mu})^\top\mathbf{C}^{-1}(\mathbf{x}_n - \boldsymbol{\mu}) \tag{20.43}$$

The MLE for $\boldsymbol{\mu}$ is $\overline{\mathbf{x}}$. Plugging in gives

$$\log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{N}{2}\left[D\log(2\pi) + \log|\mathbf{C}| + \mathrm{tr}(\mathbf{C}^{-1}\mathbf{S})\right] \tag{20.44}$$

where $\mathbf{S} = \frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}_n - \overline{\mathbf{x}})(\mathbf{x}_n - \overline{\mathbf{x}})^\top$ is the empirical covariance matrix.

In [TB99; Row97] they show that the maximum of this objective must satisfy

$$\mathbf{W} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})^{\frac{1}{2}}\mathbf{R} \tag{20.45}$$

where $\mathbf{U}_L$ is a $D \times L$ matrix whose columns are given by the L eigenvectors of $\mathbf{S}$ with largest eigenvalues, $\mathbf{L}_L$ is the $L \times L$ diagonal matrix of eigenvalues, and $\mathbf{R}$ is an arbitrary $L \times L$ orthogonal matrix, which (WLOG) we can take to be $\mathbf{R} = \mathbf{I}$. In the noise-free limit, where $\sigma^2 = 0$, we see that $\mathbf{W}_{\text{mle}} = \mathbf{U}_L \mathbf{L}_L^{\frac{1}{2}}$, which is proportional to the PCA solution.

The MLE for the observation variance is

$$\sigma^2 = \frac{1}{D-L} \sum_{i=L+1}^{D} \lambda_i \tag{20.46}$$

which is the average distortion associated with the discarded dimensions. If $L = D$, then the estimated noise is 0, since the model collapses to $\mathbf{z} = \mathbf{x}$.

To compute the likelihood $p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2)$, we need to evaluate $\mathbf{C}^{-1}$ and $\log|\mathbf{C}|$, where $\mathbf{C}$ is a $D \times D$ matrix. To do this efficiently, we can use the matrix inversion lemma to write

$$\mathbf{C}^{-1} = \sigma^{-2}\left[\mathbf{I} - \mathbf{W}\mathbf{M}^{-1}\mathbf{W}^\top\right] \tag{20.47}$$

where the $L \times L$ dimensional matrix $\mathbf{M}$ is given by

$$\mathbf{M} = \mathbf{W}^\top\mathbf{W} + \sigma^2\mathbf{I} \tag{20.48}$$

When we plug in the MLE for $\mathbf{W}$ from Eq. (20.45) (using $\mathbf{R} = \mathbf{I}$) we find

$$\mathbf{M} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\mathbf{U}_L^\top + \sigma^2\mathbf{I} \tag{20.49}$$

and hence

$$\mathbf{C}^{-1} = \sigma^{-2}\left[\mathbf{I} - \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\boldsymbol{\Lambda}_L^{-1}\mathbf{U}_L^\top\right] \tag{20.50}$$

$$\log|\mathbf{C}| = (D-L)\log\sigma^2 + \sum_{j=1}^{L}\log\lambda_j \tag{20.51}$$

Thus we can avoid all matrix inversions (since $\boldsymbol{\Lambda}_L^{-1} = \text{diag}(1/\lambda_j)$).

To use PPCA as an alternative to PCA, we need to compute the posterior mean $\mathbb{E}[\mathbf{z}|\mathbf{x}]$, which is the equivalent of the encoder model. Using Bayes rule for Gaussians we have

$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\mathbf{M}^{-1}\mathbf{W}^\top(\mathbf{x} - \boldsymbol{\mu}), \sigma^2\mathbf{M}^{-1}) \tag{20.52}$$

where $\mathbf{M}$ is defined in Eq. (20.48). In the $\sigma^2 = 0$ limit, the posterior mean using the MLE parameters becomes

$$\mathbb{E}[\mathbf{z}|\mathbf{x}] = (\mathbf{W}^\top\mathbf{W})^{-1}\mathbf{W}^\top(\mathbf{x} - \overline{\mathbf{x}}) \tag{20.53}$$

which is the orthogonal projection of the data into the latent space, as in standard PCA.

20.2.3 EM algorithm for FA/PPCA

In this section, we describe one method for computing the MLE for the FA model using the EM algorithm, based on [RT82; GH96].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.2.3.1 EM for FA

In the E step, we compute the posterior embeddings

$$p(\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_i|\mathbf{m}_i, \boldsymbol{\Sigma}_i) \tag{20.54}$$

$$\boldsymbol{\Sigma}_i \triangleq (\mathbf{I}_L + \mathbf{W}^\top \boldsymbol{\Psi}^{-1}\mathbf{W})^{-1} \tag{20.55}$$

$$\mathbf{m}_i \triangleq \boldsymbol{\Sigma}_i(\mathbf{W}^\top \boldsymbol{\Psi}^{-1}(\mathbf{x}_i - \boldsymbol{\mu})) \tag{20.56}$$

In the M step, it is easiest to estimate $\boldsymbol{\mu}$ and $\mathbf{W}$ at the same time, by defining $\tilde{\mathbf{W}} = (\mathbf{W}, \boldsymbol{\mu})$, $\tilde{\mathbf{z}} = (\mathbf{z}, 1)$, Also, define

$$\mathbf{b}_i \triangleq \mathbb{E}\left[\tilde{\mathbf{z}}|\mathbf{x}_i\right] = [\mathbf{m}_i; 1] \tag{20.57}$$

$$\mathbf{C}_i \triangleq \mathbb{E}\left[\tilde{\mathbf{z}}\tilde{\mathbf{z}}^T|\mathbf{x}_i,\right] = \begin{pmatrix} \mathbb{E}\left[\mathbf{z}\mathbf{z}^T|\mathbf{x}_i\right] & \mathbb{E}\left[\mathbf{z}|\mathbf{x}_i\right] \\ \mathbb{E}\left[\mathbf{z}|\mathbf{x}_i\right]^T & 1 \end{pmatrix} \tag{20.58}$$

Then the M step is as follows:

$$\hat{\tilde{\mathbf{W}}} = \left[\sum_i \mathbf{x}_i \mathbf{b}_i^\top\right]\left[\sum_i \mathbf{C}_i\right]^{-1} \tag{20.59}$$

$$\hat{\boldsymbol{\Psi}} = \frac{1}{N}\text{diag}\left\{\sum_i \left(\mathbf{x}_i - \hat{\tilde{\mathbf{W}}}\mathbf{b}_i\right)\mathbf{x}_i^T\right\} \tag{20.60}$$

Note that these updates are for "vanilla" EM. A much faster version of this algorithm, based on ECM, is described in [ZY08].

20.2.3.2 EM for (P)PCA

We can also use EM to fit the PPCA model, which provides a useful alternative to eigenvector methods. This relies on the probabilistic formulation of PCA. However the algorithm continues to work in the zero noise limit, $\sigma^2 = 0$, as shown by [Row97].

In particular, let $\tilde{\mathbf{Z}} = \mathbf{Z}^\top$ be a $L \times N$ matrix storing the posterior means (low-dimensional representations) along its columns. Similarly, let $\tilde{\mathbf{X}} = \mathbf{X}^\top$ store the original data along its columns. From Eq. (20.52), when $\sigma^2 = 0$, we have

$$\tilde{\mathbf{Z}} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\tilde{\mathbf{X}} \tag{20.61}$$

This constitutes the E step. Notice that this is just an orthogonal projection of the data.

From Equation 20.59, the M step is given by

$$\hat{\mathbf{W}} = \left[\sum_i \mathbf{x}_i \mathbb{E}\left[\mathbf{z}_i\right]^T\right]\left[\sum_i \mathbb{E}\left[\mathbf{z}_i\right]\mathbb{E}\left[\mathbf{z}_i\right]^T\right]^{-1} \tag{20.62}$$

where we exploited the fact that $\boldsymbol{\Sigma} = \text{Cov}\left[\mathbf{z}_i|\mathbf{x}_i, \boldsymbol{\theta}\right] = 0\mathbf{I}$ when $\sigma^2 = 0$.

It is worth comparing this expression to the MLE for multi-output linear regression (Equation 11.2), which has the form $\mathbf{W} = (\sum_i \mathbf{y}_i\mathbf{x}_i^T)(\sum_i \mathbf{x}_i\mathbf{x}_i^T)^{-1}$. Thus we see that the M step is like linear regression where we replace the observed inputs by the expected values of the latent variables.

In summary, here is the entire algorithm:

$$\tilde{\mathbf{Z}} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\tilde{\mathbf{X}} \text{ (E step)} \tag{20.63}$$

$$\mathbf{W} = \tilde{\mathbf{X}}\tilde{\mathbf{Z}}^T(\tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^T)^{-1} \text{ (M step)} \tag{20.64}$$

[TB99] showed that the only stable fixed point of the EM algorithm is the globally optimal solution. That is, the EM algorithm converges to a solution where $\mathbf{W}$ spans the same linear subspace as that defined by the first L eigenvectors. However, if we want $\mathbf{W}$ to be orthogonal, and to contain the eigenvectors in descending order of eigenvalue, we have to orthogonalize the resulting matrix (which can be done quite cheaply). Alternatively, we can modify EM to give the principal basis directly [AO03].

This algorithm has a simple physical analogy in the case $D = 2$ and $L = 1$ [Row97]. Consider some points in $\mathbb{R}^2$ attached by springs to a rigid rod, whose orientation is defined by a vector $\mathbf{w}$. Let z_i be the location where the i'th spring attaches to the rod. In the E step, we hold the rod fixed, and let the attachment points slide around so as to minimize the spring energy (which is proportional to the sum of squared residuals). In the M step, we hold the attachment points fixed and let the rod rotate so as to minimize the spring energy. See Figure 20.10 for an illustration.

20.2.3.3 Advantages

EM for PCA has the following advantages over eigenvector methods:

- EM can be faster. In particular, assuming $N, D \gg L$, the dominant cost of EM is the projection operation in the E step, so the overall time is $O(TLND)$, where T is the number of iterations. [Row97] showed experimentally that the number of iterations is usually very small (the mean was 3.6), regardless of N or D. (This results depends on the ratio of eigenvalues of the empirical covariance matrix.) This is much faster than the $O(\min(ND^2, DN^2))$ time required by straightforward eigenvector methods, although more sophisticated eigenvector methods, such as the Lanczos algorithm, have running times comparable to EM.
- EM can be implemented in an online fashion, i.e., we can update our estimate of $\mathbf{W}$ as the data streams in.
- EM can handle missing data in a simple way (see e.g., [IR10; DJ15]).
- EM can be extended to handle mixtures of PPCA/ FA models (see Sec. 20.2.6).
- EM can be modified to variational EM or to variational Bayes EM to fit more complex models (see e.g., Sec. 20.2.7).

20.2.4 Unidentifiability of the parameters

The parameters of a FA model are unidentifiable. To see this, consider a model with weights $\mathbf{W}$ and observation covariance $\mathbf{\Psi}$. We have

$$\text{Cov}[\mathbf{x}] = \mathbf{W}\mathbb{E}\left[\mathbf{z}\mathbf{z}^\top\right]\mathbf{W}^\top + \mathbb{E}\left[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top\right] = \mathbf{W}\mathbf{W}^\top + \mathbf{\Psi} \tag{20.65}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

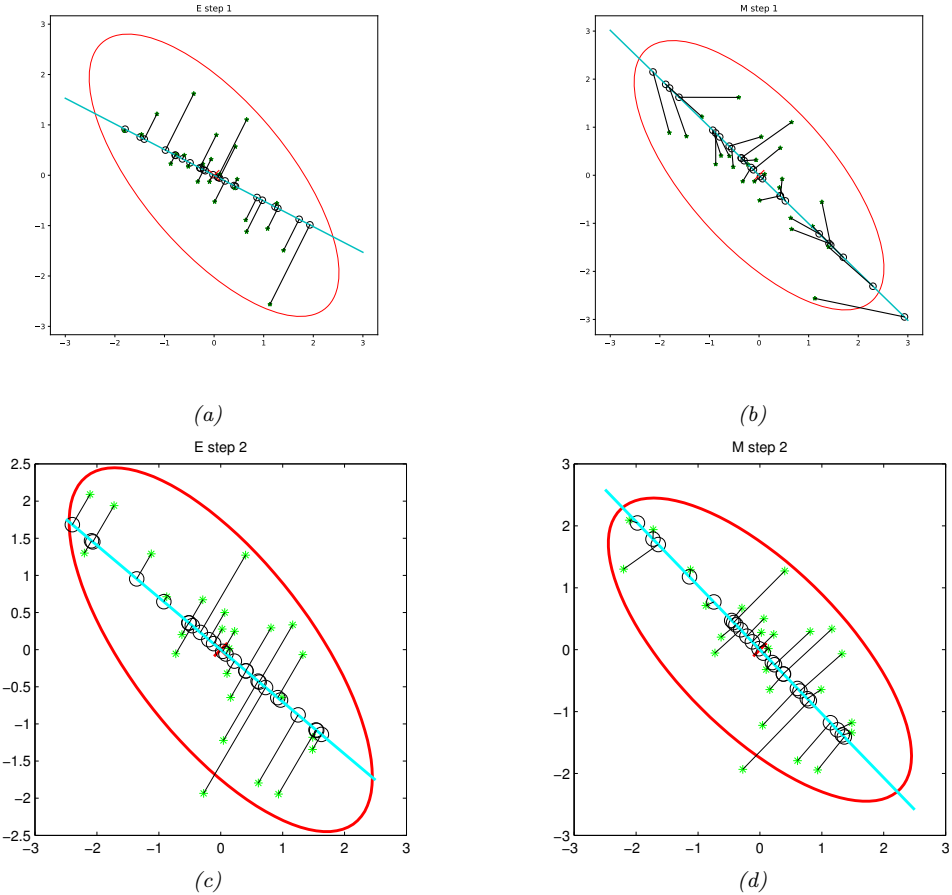


Figure 20.10: Illustration of EM for PCA when $D = 2$ and $L = 1$. Green stars are the original data points, black circles are their reconstructions. The weight vector $\mathbf{w}$ is represented by blue line. (a) We start with a random initial guess of $\mathbf{w}$. The E step is represented by the orthogonal projections. (b) We update the rod $\mathbf{w}$ in the M step, keeping the projections onto the rod (black circles) fixed. (c) Another E step. The black circles can 'slide' along the rod, but the rod stays fixed. (d) Another M step. Adapted from Figure 12.12 of [Bis06]. Generated by code at figures.probml.ai/book1/20.10.

Now consider a different moel with weights $\tilde{\mathbf{W}} = \mathbf{WR}$, where $\mathbf{R}$ is an arbitrary orthogonal rotation matrix, satisfying $\mathbf{RR}^\top = \mathbf{I}$. This has the same likelihood, since

$$\text{Cov}\left[\mathbf{x}\right] = \tilde{\mathbf{W}}\mathbb{E}\left[\mathbf{z}\mathbf{z}^\top\right]\tilde{\mathbf{W}}^\top + \mathbb{E}\left[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top\right] = \mathbf{WRR}^\top\mathbf{W}^\top + \boldsymbol{\Psi} = \mathbf{WW}^\top + \boldsymbol{\Psi} \tag{20.66}$$

Geometrically, multiplying $\mathbf{W}$ by an orthogonal matrix is like rotating $\mathbf{z}$ before generating $\mathbf{x}$; but since $\mathbf{z}$ is drawn from an isotropic Gaussian, this makes no difference to the likelihood. Consequently, we cannot uniquely identify $\mathbf{W}$, and therefore cannot uniquely identify the latent factors, either.

To break this symmetry, several solutions can be used, as we discuss below.

- **Forcing W to have orthonormal columns**.. Perhaps the simplest solution to the identifiability problem is to force $\mathbf{W}$ to have orthonormal columns. This is the approach adopted by PCA. The resulting posterior estimate will then be unique, up to permutation of the latent dimensions. (In PCA, this ordering ambiguity is resolved by sorting the dimensions in order of decreasing eigenvalues of $\mathbf{W}$.)
- **Forcing W to be lower triangular**. One way to resolve permutation unidentifiability, which is popular in the Bayesian community (e.g., [LW04c]), is to ensure that the first visible feature is only generated by the first latent factor, the second visible feature is only generated by the first two latent factors, and so on. For example, if $L = 3$ and $D = 4$, the correspond factor loading matrix is given by

$$\mathbf{W} = \begin{pmatrix} w_{11} & 0 & 0 \\ w_{21} & w_{22} & 0 \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{pmatrix} \tag{20.67}$$

  We also require that $w_{kk} > 0$ for $k = 1 : L$. The total number of parameters in this constrained matrix is $D + DL - L(L-1)/2$, which is equal to the number of uniquely identifiable parameters in FA.[3] The disadvantage of this method is that the first L visible variables, known as the **founder variables**, affect the interpretation of the latent factors, and so must be chosen carefully.
- **Sparsity promoting priors on the weights**. Instead of pre-specifying which entries in $\mathbf{W}$ are zero, we can encourage the entries to be zero, using ℓ_1 regularization [ZHT06], ARD [Bis99; AB08], or spike-and-slab priors [Rat+09]. This is called sparse factor analysis. This does not necessarily ensure a unique MAP estimate, but it does encourage interpretable solutions.
- **Choosing an informative rotation matrix**. There are a variety of heuristic methods that try to find rotation matrices $\mathbf{R}$ which can be used to modify $\mathbf{W}$ (and hence the latent factors) so as to try to increase the interpretability, typically by encouraging them to be (approximately) sparse. One popular method is known as **varimax** [Kai58].
- **Use of non-Gaussian priors for the latent factors**. If we replace the prior on the latent variables, $p(\mathbf{z})$, with a non-Gaussian distribution, we can sometimes uniquely identify $\mathbf{W}$, as well as the latent factors. See e.g., [KKH19] for details.

20.2.5 Nonlinear factor analysis

The FA model assumes the observed data can be modeled as arising from a linear mapping from a low-dimensional set of Gaussian factors. One way to relax this assumption is to let the mapping from $\mathbf{z}$ to $\mathbf{x}$ be a nonlinear model, such as a neural network. That is, the model becomes

$$p(\mathbf{x}) = \int d\mathbf{z}\, \mathcal{N}(\mathbf{x}|f(\mathbf{w};\boldsymbol{\theta}), \boldsymbol{\Psi})\mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I}) \tag{20.68}$$

3. We get D parameters for $\boldsymbol{\Psi}$ and DL for $\mathbf{W}$, but we need to remove $L(L-1)/2$ degrees of freedom coming from $\mathbf{R}$, since that is the dimensionality of the space of orthogonal matrices of size $L \times L$. To see this, note that there are $L-1$ free parameters in $\mathbf{R}$ in the first column (since the column vector must be normalized to unit length), there are $L-2$ free parameters in the second column (which must be orthogonal to the first), and so on.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

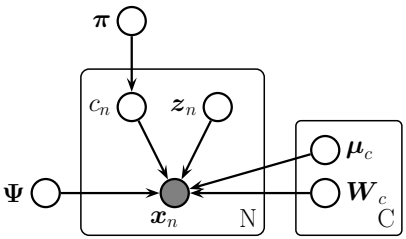


Figure 20.11: Mixture of factor analyzers as a PGM.

This is called **nonlinear factor analysis**. Unfortunately we can no longer compute the posterior or the MLE exactly, so we need to use approximate methods. In Sec. 20.3.5, we discuss variational autoencoders, which is the most common way to approximate a nonlinear FA model.

20.2.6 Mixtures of factor analysers

The factor analysis model (Sec. 20.2) assumes the observed data can be modeled as arising from a linear mapping from a low-dimensional set of Gaussian factors. One way to relax this assumption is to assume the model is only locally linear, so the overall model becomes a (weighted) combination of FA models; this is called a **mixture of factor analysers**. The overall model for the data is a mixture of linear manifolds, which can be used to approximate an overall curved manifold.

More precisely, let latent indicator $c_n \in \{1, \ldots, K\}$, specifying which subspace (cluster) we should use to generate the data. If $c_n = k$, we sample $\mathbf{z}_n$ from a Gaussian prior and pass it through the $\mathbf{W}_k$ matrix and add noise, where $\mathbf{W}_k$ maps from the L-dimensional subspace to the D-dimensional visible space.[4] More precisely, the model is as follows:

$$p(\mathbf{x}_n|\mathbf{z}_n, c_n = k, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_k + \mathbf{W}_k\mathbf{z}_n, \boldsymbol{\Psi}_k) \tag{20.69}$$

$$p(\mathbf{z}_n|\boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I}) \tag{20.70}$$

$$p(c_n|\boldsymbol{\theta}) = \mathrm{Cat}(c_n|\boldsymbol{\pi}) \tag{20.71}$$

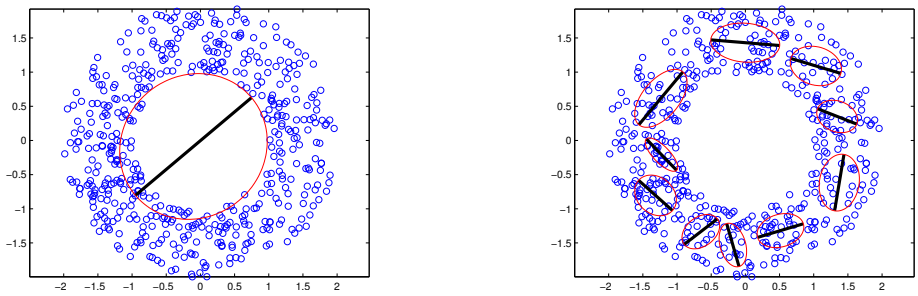
This is called a **mixture of factor analysers** (MFA) [GH96]. The corresponding distribution in the visible space is given by

$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_k p(c = k) \int d\mathbf{z}\; p(\mathbf{z}|c)p(\mathbf{x}|\mathbf{z}, c) = \sum_k \pi_k \int d\mathbf{z}; \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}_k, \mathbf{I})\mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I}) \tag{20.72}$$

In the special case that $\boldsymbol{\Psi}_k = \sigma^2\mathbf{I}$, we get a mixture of PPCA models (although it is difficult to ensure orthogonality of the $\mathbf{W}_k$ in this case). See Fig. 20.12 for an example of the method applied to some 2d data.

We can think of this as a low-rank version of a mixture of Gaussians. In particular, this model needs $O(KLD)$ parameters instead of the $O(KD^2)$ parameters needed for a mixture of full covariance Gaussians. This can reduce overfitting.

4. If we allow $\mathbf{z}_n$ to depend on c_n, we can let each subspace have a different dimensionality, as suggested in [KS15].



(a) *(b)*

Figure 20.12: Mixture of PPCA models fit to a 2d dataset, using $L = 1$ latent dimensions and $K = 1$ and $K = 10$ mixture components. Generated by code at figures.probml.ai/book1/20.12.

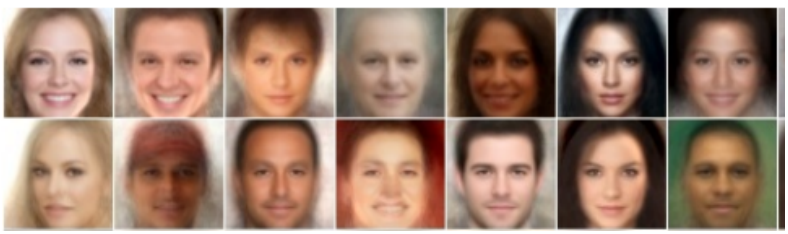


Figure 20.13: Random samples from the MixFA model fit to CelebA. From Figure 4 of [RW18]. Used with kind permission of Yair Weiss

20.2.6.1 Application: image generation

In this section, we show how to use the mixture of factor analysers model to generate images of faces by fitting to the CelebA dataset [Liu+15]. We use $K = 1000$ components, each of latent dimension $L = 10$; the observed data has dimension $D = 64 \times 64 \times 3 = 12,288$.

Fig. 20.13 shows some images generated from the fitted model. The results are suprisingly good for such a simple locally linear model. In fact, in [RW18], they showed that this MixFA model captures more of the modes of the data distribution than more sophisticated generative models, such as VAEs (Sec. 20.3.5) and GANs. Furthermore, the MixFA model has the advantage that we can compute the exact likelihood $p(\mathbf{x})$.

20.2.7 Exponential family factor analysis

So far we have assumed the observed data is real-valued, so $\mathbf{x}_n \in \mathbb{R}^D$. If we want to model other kinds of data (e.g., binary or categorical), we can simply replace the Gaussian output distribution with a suitable member of the exponential family, where the natural parameters are given by a linear function of $\mathbf{z}_n$. That is, we use

$$p(\mathbf{x}_n|\mathbf{z}_n) = \exp(\mathbf{t}(\mathbf{x})^\top \boldsymbol{\theta} + h(\mathbf{x}) - g(\boldsymbol{\theta})) \tag{20.73}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where the $N \times D$ matrix of natural parameters is assumed to be given by the low rank decomposition $\mathbf{\Theta} = \mathbf{ZW}$, where $\mathbf{Z}$ is $N \times L$ and $\mathbf{W}$ is $L \times D$. The resulting model is called **exponential family factor analysis**

Unlike the linear-Gaussian FA, we cannot compute the exact posterior $p(\mathbf{z}_n|\mathbf{x}_n, \mathbf{W})$ due to the lack of conjugacy between the expfam likelihood and the Gaussian prior. Furthermore, we cannot compute the exact marginal likelihood either, which prevents us from finding the optimal MLE.

[CDS02] proposed a coordinate ascent method for a deterministic variant of this model, known as **exponential family PCA**. This alternates between computing a point estimate of $\mathbf{z}_n$ and $\mathbf{W}$. This can be regarded as a degenerate version of variational EM, where the E step uses a delta function posterior for $\mathbf{z}_n$. [GS08] present an improved algoroithm that finds the global optimum, and [Ude+16] presents an extension called **generalized low rank models**, that covers many different kinds of loss function.

However, it is often preferable to use a probabilistic version of the model, rather than computing point estimates of the latent factors. In this case, we must represent the posterior use a non-degenerate distribution to avoid overfitting, since the number of latent variables is proportional to the number of datacases [WCS08]. Fortunately, we can use a non-degenerate posterior, such as a Gaussian, by optimizing the variational lower bound. We give some examples of this below.

20.2.7.1 Example: binary PCA

Consider a factored Bernoulli likelihood:

$$p(\mathbf{x}|\mathbf{z}) = \prod_d \text{Ber}(x_d|\sigma(\mathbf{w}_d^\top \mathbf{z})) \tag{20.74}$$

Suppose we observe $N = 150$ bit vectors of length $D = 6$. Each example is generated by choosing one of three binary prototype vectors, and then by flipping bits at random. See Fig. 20.14(a) for the data. We can fit this using the variational EM algorithm (see [Tip98] for details). We use $L = 2$ latent dimensions to allow us to visualize the latent space. In Fig. 20.14(b), we plot $\mathbb{E}\left[\mathbf{z}_n|\mathbf{x}_n, \hat{\mathbf{W}}\right]$. We see that the projected points group into three distinct clusters, as is to be expected. In Fig. 20.14(c), we plot the reconstructed version of the data, which is computed as follows:

$$p(\hat{x}_{nd} = 1|\mathbf{x}_n) = \int d\mathbf{z}_n \; p(\mathbf{z}_n|\mathbf{x}_n) p(\hat{x}_{nd}|\mathbf{z}_n) \tag{20.75}$$

If we threshold these probabilities at 0.5 (corresponding to a MAP estimate), we get the "denoised" version of the data in Fig. 20.14(d).

20.2.7.2 Example: categorical PCA

We can generalize the model in Sec. 20.2.7.1 to handle categorical data by using the following likelihood:

$$p(\mathbf{x}|\mathbf{z}) = \prod_d \text{Cat}(x_d|\mathcal{S}(\mathbf{W}_d \mathbf{z})) \tag{20.76}$$

We call this **categorical PCA** (**CatPCA**). A variational EM algorithm for fitting this is described in [Kha+10].

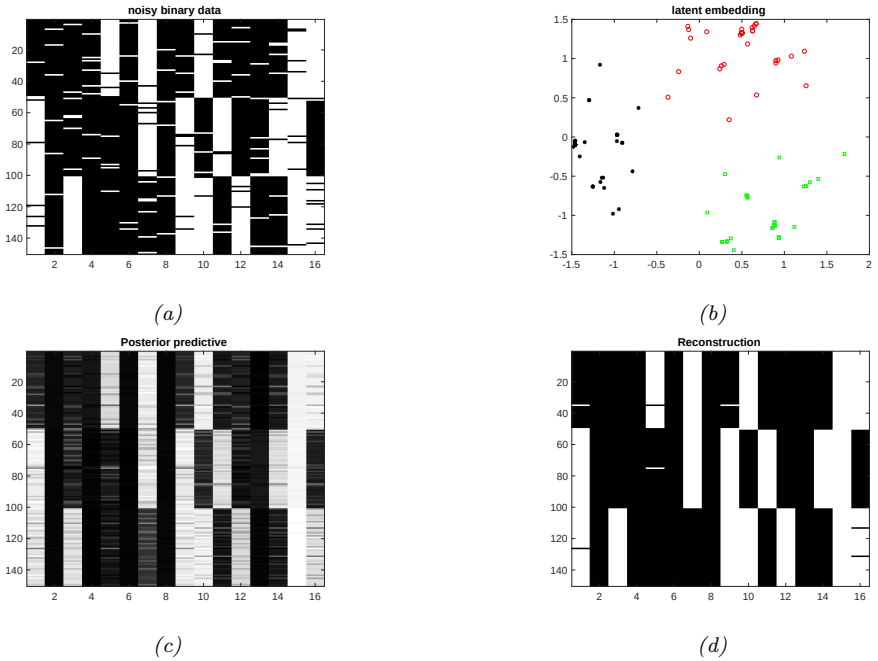


Figure 20.14: (a) 150 synthetic 16 dimensional bit vectors. (b) The 2d embedding learned by binary PCA, fit using variational EM. We have color coded points by the identity of the true "prototype" that generated them. (c) Predicted probability of being on. (d) Thresholded predictions. Generated by code at figures.probml.ai/book1/20.14.

20.2.8 Factor analysis models for paired data

In this section, we discuss linear-Gaussian factor analysis models when we have two kinds of observed variables, $\mathbf{x} \in \mathbb{R}^{D_x}$ and $\mathbf{y} \in \mathbb{R}^{D_y}$, which are paired. These often correspond to different sensors or modalities (e.g., images and sound). We follow the presentation of [Vir10].

20.2.8.1 Supervised PCA

In **supervised PCA** [Yu+06], we model the joint $p(\mathbf{x}, \mathbf{y})$ using a shared low-dimensional representation using the following linear Gaussian model:

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I}_L) \tag{20.77}$$

$$p(\mathbf{x}_n|\mathbf{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}_x\mathbf{z}_n, \sigma_x^2\mathbf{I}_{D_x}) \tag{20.78}$$

$$p(\mathbf{y}_n|\mathbf{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}_n|\mathbf{W}_y\mathbf{z}_n, \sigma_y^2\mathbf{I}_{D_y}) \tag{20.79}$$

This is illustrated as a graphical model in Fig. 20.15a. The intuition is that $\mathbf{z}_n$ is a shared latent subspace, that captures features that $\mathbf{x}_n$ and $\mathbf{y}_n$ have in common. The variance terms σ_x and σ_y control how much emphasis the model puts on the two different signals. If we put a prior on the parameters $\boldsymbol{\theta} = (\mathbf{W}_x, \mathbf{W}_y, \sigma_x, \sigma_y)$, we recover the **Bayesian factor regression** model of [Wes03].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

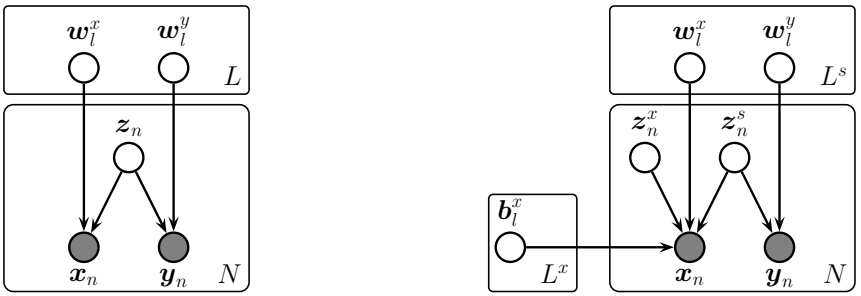


Figure 20.15: Gaussian latent factor models for paired data. (a) Supervised PCA. (b) Partial least squares.

We can marginalize out $\mathbf{z}_n$ to get $p(\mathbf{y}_n|\mathbf{x}_n)$. If $\mathbf{y}_n$ is a scalar, this becomes

$$p(y_n|\mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{N}(y_n|\mathbf{x}_n^\top \mathbf{v}, \mathbf{w}_y^\top \mathbf{C}\mathbf{w}_y + \sigma_y^2) \tag{20.80}$$

$$\mathbf{C} = (\mathbf{I} + \sigma_x^{-2}\mathbf{W}_x^\top \mathbf{W}_x)^{-1} \tag{20.81}$$

$$\mathbf{v} = \sigma_x^{-2}\mathbf{W}_x \mathbf{C}\mathbf{w}_y \tag{20.82}$$

To apply this to the classification setting, we can use supervised ePCA [Guo09], in which we replace the Gaussian $p(\mathbf{y}|\mathbf{z})$ with a logistic regression model.

This model is completely symmetric in $\mathbf{x}$ and $\mathbf{y}$. If our goal is to predict $\mathbf{y}$ from $\mathbf{x}$ via the latent bottleneck $\mathbf{z}$, then we might want to upweight the likelihood term for $\mathbf{y}$, as proposed in [Ris+08]. This gives

$$p(\mathbf{X}, \mathbf{Y}, \mathbf{Z}|\boldsymbol{\theta}) = p(\mathbf{Y}|\mathbf{Z}, \mathbf{W}_y)p(\mathbf{X}|\mathbf{Z}, \mathbf{W}_x)^\alpha p(\mathbf{Z}) \tag{20.83}$$

where $\alpha \leq 1$ controls the relative importance of modeling the two sources. The value of α can be chosen by cross-validation.

20.2.8.2 Partial least squares

Another way to improve the predictive performance in supervised tasks is to allow the inputs $\mathbf{x}$ to have their own "private" noise source that is independent on the target variable, since not all variation in $\mathbf{x}$ is relevant for predictive purposes. We can do thus by introducing an extra latent variable $\mathbf{z}_n^x$ just for the inputs, that is different from $\mathbf{z}_n^s$ that is the shared botteleneck between $\mathbf{x}_n$ and $\mathbf{y}_n$. In the Gaussian case, the overall model has the form

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n^s|\mathbf{0}, \mathbf{I})\mathcal{N}(\mathbf{z}_n^x|\mathbf{0}, \mathbf{I}) \tag{20.84}$$

$$p(\mathbf{x}_n|\mathbf{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}_x\mathbf{z}_n^s + \mathbf{B}_x\mathbf{z}_n^x, \sigma_x^2\mathbf{I}) \tag{20.85}$$

$$p(\mathbf{y}_n|\mathbf{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}_n|\mathbf{W}_y\mathbf{z}_n^s, \sigma_y^2\mathbf{I}) \tag{20.86}$$

See Fig. 20.15b. MLE for $\boldsymbol{\theta}$ in this model is equivalent to the technique of **partial least squares** (**PLS**) [Gus01; Nou+02; Sun+09].

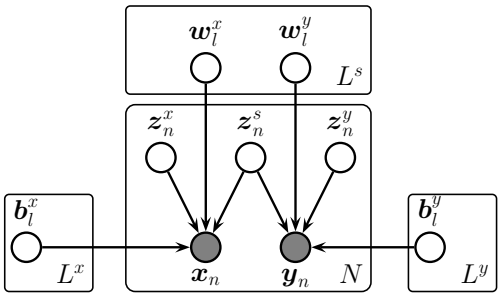


Figure 20.16: Canonical correlation analysis as a PGM.

20.2.8.3 Canonical correlation analysis

In some cases, we want to use a fully symmetric model, so we can capture the dependence between $\mathbf{x}$ and $\mathbf{y}$, while allowing for domain-specific or "private" noise sources We can do thus by introducing a latent variable $\mathbf{z}_n^x$ just for $\mathbf{x}_n$, a latent variable $\mathbf{z}_n^y$ just for $\mathbf{y}_n$, and a shared latent variable $\mathbf{z}_n^s$. In the Gaussian case, the overall model has the form

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n^s|\mathbf{0},\mathbf{I})\mathcal{N}(\mathbf{z}_n^x|\mathbf{0},\mathbf{I}) \tag{20.87}$$

$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}_x\mathbf{z}_n^s + \mathbf{B}_x\mathbf{z}_n^x, \sigma_x^2\mathbf{I}) \tag{20.88}$$

$$p(\mathbf{y}_n|\mathbf{z}_n,\boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}_n|\mathbf{W}_y\mathbf{z}_n^s + \mathbf{B}_y\mathbf{z}_n^y, \sigma_y^2\mathbf{I}) \tag{20.89}$$

where $\mathbf{W}_x$ and $\mathbf{W}_y$ are $L^s \times D$ dimensional, $\mathbf{V}_x$ is $L^x \times D$ dimensional, and $\mathbf{V}_y$ is $L^y \times D$ dimensional. See Fig. 20.16 for the PGM.

If we marginalize out all the latent variables, we get the following distribution on the visibles (where we assume $\sigma_x = \sigma_y = \sigma$):

$$p(\mathbf{x}_n,\mathbf{y}_n) = \int d\mathbf{z}_n p(\mathbf{z}_n) p(\mathbf{x}_n,\mathbf{y}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n,\mathbf{y}_n|\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}) \tag{20.90}$$

where $\boldsymbol{\mu} = (\boldsymbol{\mu}_x;\boldsymbol{\mu}_y)$, and $\mathbf{W} = [\mathbf{W}_x;\mathbf{W}_y]$. Thus the induced covariance is the following low rank matrix:

$$\mathbf{W}\mathbf{W}^\top = \begin{pmatrix} \mathbf{W}_x\mathbf{W}_x^\top & \mathbf{W}_x\mathbf{W}_y^\top \\ \mathbf{W}_y\mathbf{W}_x^\top & \mathbf{W}_y\mathbf{W}_y^\top \end{pmatrix} \tag{20.91}$$

[BJ05] showed that MLE for this model is equivalent to a classical statistical method known as **canonical correlation analysis** or **CCA** [Hot36]. However, the PGM perspective allows us to easily generalize to multiple kinds of observations (this is known as **generalized CCA** [Hor61]) or to nonlinear models (this is known as **deep CCA** [WLL16; SNM16]), or exponential family CCA [KVK10]. See [Uur+17] for further discussion of CCA and its extensions.

20.3 Autoencoders

We can think of PCA (Sec. 20.1) and factor analysis (Sec. 20.2) as learning a (linear) mapping from $\mathbf{x} \to \mathbf{z}$, called the **encoder**, f_e, and learning another (linear) mapping $\mathbf{z} \to \mathbf{x}$, called the **decoder**,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

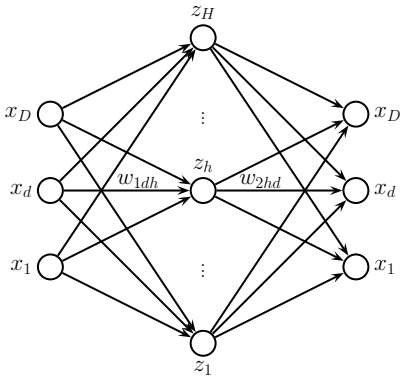


Figure 20.17: An autoencoder with one hidden layer.

f_d. The overall reconstruction function has the form $r(\mathbf{x}) = f_d(f_e(\mathbf{x}))$. The model is trained to minimize $\mathcal{L}(\boldsymbol{\theta}) = ||r(\mathbf{x}) - \mathbf{x}||_2^2$. More generally, we can use $\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathbf{x}|r(\mathbf{x}))$.

In this section, we consider the case where the encoder and encoder are nonlinear mappings implemented by neural networks. This is called an **autoencoder**. If we use an MLP with one hidden layer, we get the model shown Fig. 20.17. We can think of the hidden units in the middle as a low-dimensional **bottleneck** between the input and its reconstruction.

Of course, if the hidden layer is wide enough, there is nothing to stop this model from learning the identity function. To prevent this degenerate solution, we have to restrict the model in some way. The simplest approach is to use a narrow bottleneck layer, with $L \ll D$; this is called an **undercomplete representation**. The other approach is to use $L \gg D$, known as an **overcomplete representation**, but to impose some other kind of regularization, such as adding noise to the inputs, forcing the activations of the hidden units to be sparse, or imposing a penalty on the derivatives of the hidden units. We discuss these options in more detail below.

20.3.1 Bottleneck autoencoders

We start by considering the special case of a **linear autoencoder**, in which there is one hidden layer, the hidden units are computed using $\mathbf{z} = \mathbf{W}_1\mathbf{x}$, and the output is reconstructed using $\hat{\mathbf{x}} = \mathbf{W}_2\mathbf{z}$, where $\mathbf{W}_1$ is a $L \times D$ matrix, $\mathbf{W}_2$ is a $D \times L$ matrix, and $L < D$. Hence $\hat{\mathbf{x}} = \mathbf{W}_2\mathbf{W}_1\mathbf{x} = \mathbf{W}\mathbf{x}$ is the output of the model. If we train this model to minimize the squared reconstruction error, $\mathcal{L}(\mathbf{W}) = \sum_{n=1}^{N} ||\mathbf{x}_n - \mathbf{W}\mathbf{x}_n||_2^2$, one can show [BH89; KJ95] that $\hat{\mathbf{W}}$ is an orthogonal projection onto the first L eigenvectors of the empirical covariance matrix of the data. This is therefore equivalent to PCA.

If we introduce nonlinearities into the autoencoder, we get a model that is strictly more powerful than PCA, as proved in [JHG00]. Such methods can learn very useful low dimensional representations of data.

Consider fitting an autoencoder to the Fashion MNIST dataset. We consider both an MLP architecture (with 2 layers and a bottleneck of size 30), and a CNN based architecture (with 3 layers and a 3d bottleneck with 64 channels). We use a bernoulli likelihood model and binary cross entropy

(a) (b)

Figure 20.18: Results of applying an autoencoder to the Fashion MNIST data. Top row are first 5 images from validation set. Bottom row are reconstructions. (a) MLP model (trained for 20 epochs). The encoder is an MLP with architecture 784-100-30. The decoder is the mirror image of this. (b) CNN model (trained for 5 epochs). The encoder is a CNN model with architecture Conv2D(16, 3x3, same, selu), MaxPool2D(2x2), Conv2D(32, 3x3, same, selu), MaxPool2D(2x2), Conv2D(64, 3x3, same, selu), MaxPool2D(2x2). The decoder is the mirror image of this, using transposed convolution and without the max pooling layers. Adapted from Figure 17.4 of [Gér19]. Generated by code at figures.probml.ai/book1/20.18.

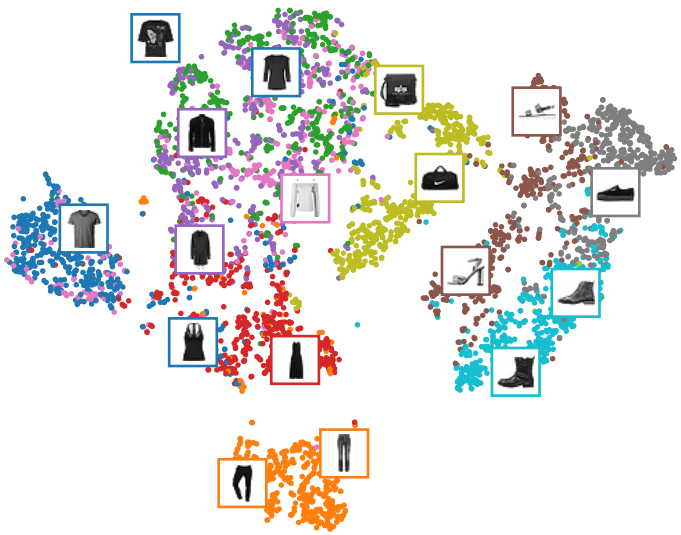
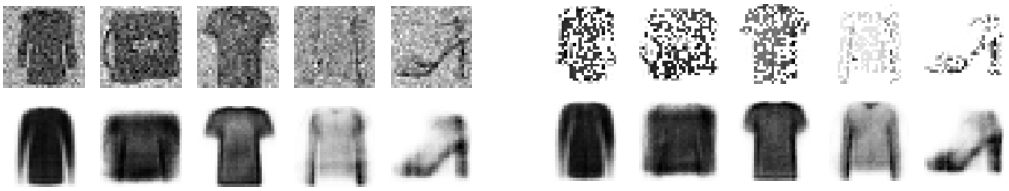


Figure 20.19: tSNE plot of the first 2 latent dimensions of the Fashion MNIST validation set computed using an MLP-based autoencoder. Adapted from Figure 17.5 of [Gér19]. Generated by code at figures.probml.ai/book1/20.19.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.20: Denoising autoencoder (MLP architecture) applied to some noisy Fashion MNIST images from the validation set. (a) Gaussian noise. (b) Bernoulli dropout noise. Top row: input. Bottom row: output Adapted from Figure 17.9 of [Gér19]. Generated by code at figures.probml.ai/book1/20.20.

as the loss. Fig. 20.18 shows some test images and their reconstructions.

Fig. 20.19 visualizes the first 2 (of 30) latent dimensions produced by the MLP-AE. More precisely, we plot the tSNE embeddings (see Sec. 20.4.10), color coded by class label. We also show some sample images. We see that the method has done a good job of separating the classes in a fully unsupervised way.

20.3.2 Denoising autoencoders

One useful way to control the capacity of an autoencoder is to add noise to its input, and then train the model to reconstruct a clean (uncorrupted) version of the original input. This is called a **denoising autoencoder** [Vin+10a].

We can implement this by adding Gaussian noise, or using Bernoulli dropout. Fig. 20.20 shows some reconstructions of corrupted images computed using a DAE. We see that the model is able to "hallucinate" details that are missing in the input, since it has seen similar images before, and can store this information in the parameters of the model.

Suppose we train a DAE using Gaussian corruption and squared error reconstruction, i.e., we use $p_c(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}}|\mathbf{x}, \sigma^2\mathbf{I})$ and $\ell(\mathbf{x}, r(\tilde{\mathbf{x}})) = ||\mathbf{e}||_2^2$, where $\mathbf{e}(\mathbf{x}) = r(\tilde{\mathbf{x}}) - \mathbf{x}$ is the residual error for example $\mathbf{x}$. Then one can show [AB14] the remarkable result that, as $\sigma \to 0$ (and with a sufficiently powerful model and enough data), the residuals approximate the **score function**, which is the log probability of the data, i.e., $\mathbf{e}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$. That is, the DAE learns a **vector field**, corresponding to the gradient of the log data density. Thus points that are close to the data manifold will be projected onto it via the sampling process. See Fig. 20.21 for an illustration.

20.3.3 Contractive autoencoders

A different way to regularize autoencoders is by adding the penalty term

$$\Omega(\mathbf{z}, \mathbf{x}) = \lambda ||\frac{\partial f_e(\mathbf{x})}{\partial \mathbf{x}}||_F^2 = \lambda \sum_k ||\nabla_{\mathbf{x}} h_k(\mathbf{x})||_2^2 \tag{20.92}$$

to the reconstruction loss, where h_k is the value of the k'th hidden embedding unit. That is, we penalize the Frobenius norm of the encoder's Jacobian. This is called a **contractive autoencoder**

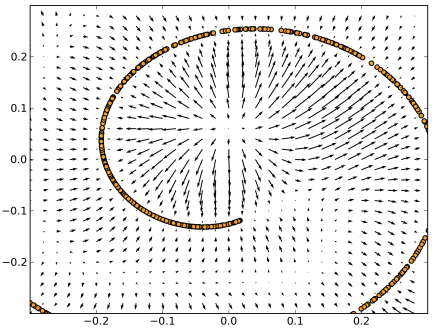


Figure 20.21: The residual error from a DAE, $\mathbf{e}(\mathbf{x}) = r(\tilde{\mathbf{x}}) - \mathbf{x}$, can learn a vector field corresponding to the score function. Arrows point towards higher probability regions. The length of the arrow is proportional to $||\mathbf{e}(\mathbf{x})||$, so points near the 1d data manifold (represented by the curved line) have smaller arrows. From Figure 5 of [AB14]. Used with kind permission of Guillaume Alain.

[Rif+11]. (A linear operator with Jacobian $\mathbf{J}$ is called a **contraction** if $||\mathbf{Jx}|| \leq 1$ for all unit-norm inputs $\mathbf{x}$.)

To understand why this is useful, consider Fig. 20.21. We can approximate the curved low-dimensional manifold by a series of locally linear manifolds. These linear approximations can be computed using the Jacobian of the encoder at each point. By encouraging these to be contractive, we ensure the model "pushes" inputs that are off the manifold to move back towards it.

Another way to think about CAEs is as follows. To minimize the penalty term, the model would like to ensure the encoder is a constant function. However, if it was completely constant, it would ignore its input, and hence incur high reconstruction cost. Thus the two terms together encourage the model to learn a representation where only a few units change in response to the most significant variations in the input.

One possible degenerate solution is that the encoder simply learns to multiply the input by a small constant ϵ (which scales down the Jacobian), followed by a decoder that divides by ϵ (which reconstructs perfectly). To avoid this, we can tie the weights of the encoder and decoder, by setting the weight matrix for layer ℓ of f_d to be the transpose of the weight matrix for layer ℓ of f_e, but using untied bias terms. Unfortunately CAEs are slow to train, because of the expense of computing the Jacobian.

20.3.4 Sparse autoencoders

Yet another way to regularizer autoencoders is to add a sparsity penalty to the latent activations of the form $\Omega(\mathbf{z}) = \lambda||\mathbf{z}||_1$. (This is called **activity regularization**.)

An alternative way to implement sparsity, that often gives better results, is to use logistic units, and then to compute the expected fraction of time each unit k is on within a minibatch (call this q_k), and ensure that this is close to a desired target value p, as proposed in [GBB11]. In particular, we use the regularizer $\Omega(\mathbf{z}_{1:L,1:N}) = \lambda \sum_k \mathbb{KL}\left(\mathbf{p}||\mathbf{q}_k\right)$ for latent dimensions $1:L$ and examples $1:N$, where

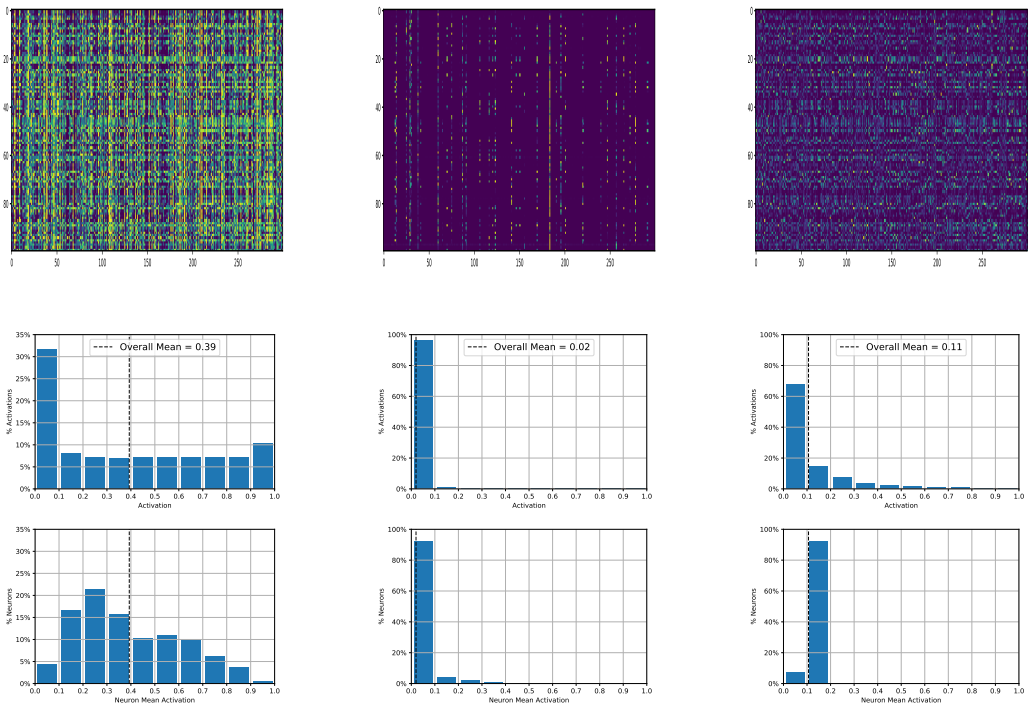


Figure 20.22: Neuron activity (in the bottleneck layer) for an autoencoder applied to Fashion MNIST. We show results for three models, with different kinds of sparsity penalty: no penalty (left column), ℓ_1 penalty (middle column), KL penalty (right column). Top row: Heatmap of 300 neuron activations (columns) across 100 examples (rows). Middle row: Histogram of activation levels derived from this heatmap. Bottom row: Histogram of the mean activation per neuron, averaged over all examples in the validation set. Adapted from Figure 17.11 of [Gér19]. Generated by code at figures.probml.ai/book1/20.22.

$\mathbf{p} = (p, 1-p)$ is the desired target distribution, and $\mathbf{q}_k = (q_k, 1-q_k)$ is the empirical distribution for unit k, computed using $q_k = \frac{1}{N}\sum_{n=1}^{N} \mathbb{I}(z_{n,k} = 1)$.

Fig. 20.22 shows the results when fitting an AE-MLP (with 300 hidden units) to Fashion MNIST. If we set $\lambda = 0$ (i.e., if we don't impose a sparsity penalty), we see that the average activation value is about 0.4, with most neurons being partially activated most of the time. With the ℓ_1 penalty, we see that most units are off all the time, which means they are not being used at all. With the KL penalty, we see that about 70% of neurons are off on average, but unlike the ℓ_1 case, we don't see units being permanently turned off (the average activation level is 0.1). This latter kind of sparse firing pattern is similar to that observed in biological brains (see e.g., [Bey+19]).

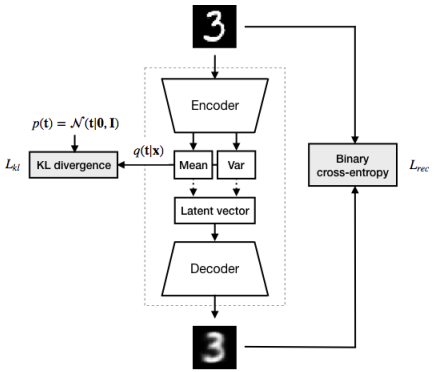


Figure 20.23: Schematic illustration of a VAE. From a figure from http://krasserm.github.io/2018/07/27/dfc-vae/. Used with kind permission of Martin Krasser.

20.3.5 Variational autoencoders

In this section, we discuss the **variational autoencoder** or **VAE** [KW14; RMW14; KW19a], which can be thought of as a probabilistic version of a deterministic autoencoder (Sec. 20.3) The principle advantage is that a VAE is a generative model that can create new samples, whereas an autoencoder just computes embeddings of input vectors.

In more detail, the VAE combines two key ideas. First we create a non-linear extension of the factor analysis generative model, i.e., we replacing $p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}, \sigma^2\mathbf{I})$ with

$$p(\mathbf{x}|\mathbf{z}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|f_d(\mathbf{z}; \boldsymbol{\theta}), \sigma^2\mathbf{I}) \tag{20.93}$$

where f_d is the decoder. (We can also use a Bernoulli likeliood if we wish.) Second, we create another model, $q(\mathbf{z}|\mathbf{x})$, called the **recognition network** or **inference network**, that is trained simultaneously with the generative model to do approximate posterior inference. If we assume the posterior is Gaussian, with diagonal covariance, we get

$$q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) = \mathcal{N}(\mathbf{z}|f_{e,\mu}(\mathbf{x}; \boldsymbol{\phi}), \text{diag}(f_{e,\sigma}(\mathbf{x}; \boldsymbol{\phi}))) \tag{20.94}$$

where f_e is the encoder. See Fig. 20.23 for a sketch.

The idea of training an inference network to "invert" a generative network, rather than running an optimization algorithm to infer the latent code, is called **amortized inference**. This idea was first proposed in the **Helmholtz machine** [Day+95]. However, that paper did not present a single unified objective function for inference and generation, but instead used the wake sleep method for training, which alternates between optimizing the generative model and inference model. By contrast, the VAE optimizes a variational lower bound on the log-likelihood, which is more principled, since it is a single unified objective.

20.3.5.1 Training VAEs

We cannot compute the exact marginal likelihood $p(\mathbf{x}|\boldsymbol{\theta})$ needed for MLE training, because posterior inference in a nonlinear FA model is intractable. However, we can use the inference network to

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

compute an approximate posterior, $q(\mathbf{z}|\mathbf{x})$. We can then use this to compute the evidence lower bound or ELBO. This is discussed in detail in Sec. 4.6.8.3; the difference here is that the unknowns that we want to infer are the latent variables $\mathbf{z}$; the model parameters $\boldsymbol{\theta}$ will be estimated using point estimation methods, as in variational EM (see Sec. 8.7.2). Thus our goal is to maximize the following objective:

$$\mathcal{E}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathcal{D}) = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\mathcal{E}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x})\right] \tag{20.95}$$

where the ELBO for a single datapoint is given by

$$\mathcal{E}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) - \log q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})\right] \tag{20.96}$$

$$= \mathbb{E}_{q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right] + \mathbb{KL}\left(q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}) \| p_{\boldsymbol{\theta}}(\mathbf{z})\right) \tag{20.97}$$

Thus we see that we apply the KL penalty to each individual encoding vector, rather than the approach in Sec. 20.3.4 where we applied the KL penalty to the aggregate posterior in each minibatch.

The ELBO is a lower bound of the log marginal likelihood (aka evidence), as can be seen from Jensen's inequality:

$$\mathcal{E}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x}) = \int q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}) \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})} d\mathbf{z} \tag{20.98}$$

$$\leq \log \int q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}) \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})} d\mathbf{z} = \log p_{\boldsymbol{\theta}}(\mathbf{x}) \tag{20.99}$$

Thus for fixed inference network parameters $\boldsymbol{\omega}$, increasing the ELBO should increase the log likelihood of the data, as we discuss in Sec. 8.7.2.

We can rewrite the ELBO as follows:

$$\mathcal{E}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x}, \boldsymbol{\omega})}\left[\log p(\mathbf{x}|\mathbf{z}, \boldsymbol{\theta})\right] - \mathbb{KL}\left(q(\mathbf{z}|\mathbf{x}, \boldsymbol{\omega}) \| p(\mathbf{z})\right) \tag{20.100}$$

For simplicity, let us suppose that the inference network estimates the parameters of a Gaussian posterior. Since $q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})$ is Gaussian, we can write

$$\mathbf{z} = f_{e,\mu}(\mathbf{x}; \boldsymbol{\phi}) + f_{e,\sigma}(\mathbf{x}; \boldsymbol{\phi}) \odot \boldsymbol{\epsilon} \tag{20.101}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This is known as the **reparameterization trick** Hence

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x}) = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z} = \mu_{\boldsymbol{\omega}}(\mathbf{x}) + \sigma_{\boldsymbol{\omega}}(\mathbf{x}) \odot \boldsymbol{\epsilon})\right] - \mathbb{KL}\left(q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})\right) \tag{20.102}$$

Now the expectation is independent of the parameters of the model, so we can safely push gradients inside and use backpropagation for training in the usual way, by minimizing $-\mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\omega}|\mathbf{x})\right]$ wrt $\boldsymbol{\theta}$ and $\boldsymbol{\omega}$.

To compute the loss and its gradient, we need to compute the ELBO for each example. The first term in the ELBO can be computed by sampling $\boldsymbol{\epsilon}$, and then "decoding" this sample to generate $\mathbf{x}$, and then evaluating $\log p(\mathbf{x}|\mathbf{z})$.

The second term in the ELBO is the KL of two Gaussians. This has a closed form. In particular, inserting $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$ and $q(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}))$ into Eq. (6.33), we get

$$\mathbb{KL}\left(q \| p\right) = -\frac{1}{2} \sum_{k=1}^{K} \left[\log \sigma_k^2 - \sigma_k^2 - \mu_k^2 + 1\right] \tag{20.103}$$

(The inference network often outputs the log variance, $\gamma_k = \log \sigma_k^2$, for numerical stability.)

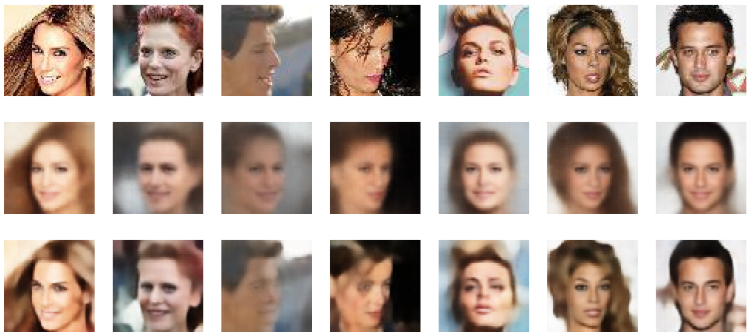


Figure 20.24: Comparison of reconstruction abilities of an autoencoder and VAE. Top row: Original images. Middle row: Reconstructions from a VAE. Bottom row: Reconstructions from an AE. We see that the VAE reconstructions (middle) are blurrier. Both models have the same shallow convolutional architecture (3 hidden layers, 200 latents), and are trained on identical data (20k images of size 64×64 extracted from CelebA) for the same number of epochs (20). Generated by code at figures.probml.ai/book1/20.24.

20.3.5.2 Comparison of VAEs and autoencoders

VAEs are very similar to autoencoders. In particular, the generative model, $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$, acts like the decoder, and the inference network, $q_{\boldsymbol{\omega}}(\mathbf{z}|\mathbf{x})$, acts like the encoder.

The primary advantage of the VAE is that it can be used to generate new data from random noise. In particular, we sample $\mathbf{z}$ from the Gaussian prior $\mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$, and then pass this through the decoder to get $\mathbb{E}[\mathbf{x}|\mathbf{z}] = f_d(\mathbf{z}; \boldsymbol{\theta})$. The VAE's decoder is trained to convert random points in the embedding space (generated by perturbing the input encodings) to sensible outputs. By contrast, the decoder for the deterministic autoencoder only ever gets as inputs the exact encodings of the training set, so it does not know what to do with random inputs that are outside what it was trained on.

We now give a visual illustration of this difference. In Fig. 20.24, we show results after training a convolutional AE and VAE with 4 hidden layers on a small version of the CelebA dataset [Liu+15]. The architecture is similar to the U-net model in Fig. 14.29, but without the shortcut skip connections. We use 20,000 images of size 64x64 and train for 20 epochs. (We can get better results if we train for longer and/or on more data, but we wanted to ensure the demo runs in just a couple of minutes.) We see that both models can encode and decode an image fairly reliably, although the the VAE reconstructions (middle row) are slightly blurrier (soft focus). However, Fig. 20.25 shows that novel samples from the VAE are reasoanable, whereas the AE samples are not.

20.3.5.3 Latent space interpolation

One powerful feature of latent variable models is the ability to generate samples that interpolate the "semantic properties" of two different inputs. Specifically, let $\mathbf{x}_1$ and $\mathbf{x}_2$ be two images, and let $\mathbf{z}_1 = \mathbb{E}_{q(\mathbf{z}|\mathbf{x}_1)}[\mathbf{z}]$ and $\mathbf{z}_1 = \mathbb{E}_{q(\mathbf{z}|\mathbf{x}_1)}[\mathbf{z}]$ be their encodings. We can now generate new images that interpolate between these two anchors by computing $\mathbf{z} = \lambda \mathbf{z}_1 + (1 - \lambda)\mathbf{z}_2$, where $0 \leq \lambda \leq 1$, and then decoding by computing $\mathbb{E}[\mathbf{x}|\mathbf{z}]$. This is called **latent space interpolation**. (The justification for taking a linear interpolation is that the learned manifold has approximately zero curvature, as

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

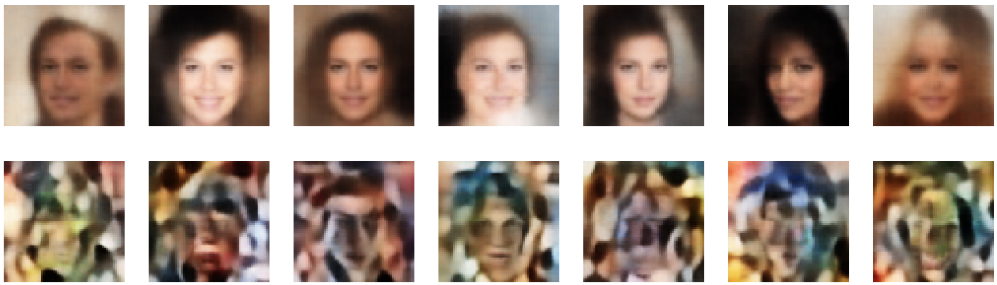


Figure 20.25: Unconditional samples from a VAE (top row) or AE (bottom row) trained on CelebA. Both models have the same structure and both are trained for 20 epochs. Generated by code at figures.probml.ai/book1/20.25.

Figure 20.26: Interpolation between two real images (first and last columns) in the latent space of a VAE. Adapted from Figure 3.22 of [Fos19]. Generated by code at figures.probml.ai/book1/20.26.

shown in [SKTF18].) See Fig. 20.26 for an example, where we interpolate between a woman without sunglasses and a man with sunglasses.

20.3.5.4 Latent space vector arithmetic

An intriguing property of some latent variable models is that they support the ability to perform vector addition and subtraction in latent space in order to increase or decrease the amount of a desired "semantic factor of variation" (c.f., word2vec model in Fig. 20.44). For example, consider the attribute of wearing sunglasses. Let $\mathbf{X}_i^+$ be a set of images which have attribute i, and $\mathbf{X}_i^-$ be a set of images which do not have this attribute. Let $\mathbf{Z}_i^+$ and $\mathbf{Z}_i^-$ be the corresponding embeddings, and $\overline{\mathbf{z}}^+$ and $\overline{\mathbf{z}}^-$ be the average of these embeddings. We define the offset vector as $\boldsymbol{\Delta} = \overline{\mathbf{z}}^+ - \overline{\mathbf{z}}^-$. If we add some positive multiple of $\boldsymbol{\Delta}$ to a new point $\mathbf{z}$, we increase the amount of this attribute; if we subtract some multiple of $\boldsymbol{\Delta}$, we decrease the amount of this attribute. (This idea was first proposed in [RMC16].)

20.4 Manifold learning *

In this section, we discuss the problem of recovering the underlying low-dimensional structure in a high-dimensional dataset. This structure is often assumed to be a curved manifold (explained in Sec. 20.4), so this problem is called **manifold learning**. or **nonlinear dimensionality reduction**. The key difference from methods such as autoencoders (Sec. 20.3) is that we will focus on non-parametric methods, in which we compute an embedding for each point in the training set, as opposed

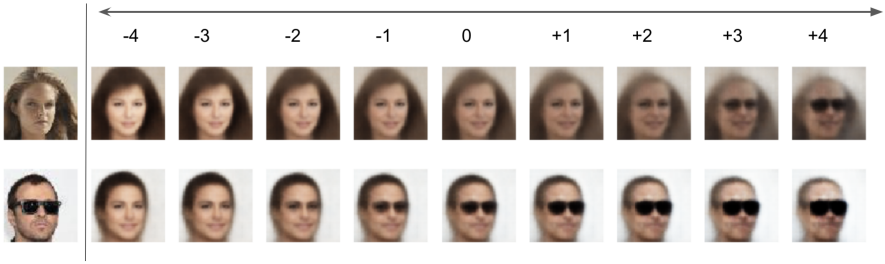


Figure 20.27: Adding or removing the "sunglasses" vector to an image using a VAE. The first column is an input image, with embedding $\mathbf{z}$*. Subsequent columns show the decoding of* $\mathbf{z} + s\boldsymbol{\Delta}$*, where* $s \in \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$ *and* $\boldsymbol{\Delta} = \overline{\mathbf{z}}^{+} - \overline{\mathbf{z}}^{-}$ *is the difference in the average embeddings of images of people with or without sunglasses. Adapted from Figure 3.21 of [Fos19]. Generated by code at figures.probml.ai/book1/20.27.*

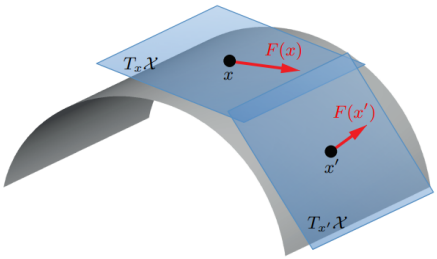


Figure 20.28: Illustration of the tangent space and tangent vectors at two different points on a 2d curved manifold. From Figure 1 of [Bro+17a]. Used with kind permission of Michael Bronstein

to learning a generic model that can embed any input vector. That is, the methods we discuss do not (easily) support **out-of-sample generalization**. However, they can be easier to fit, and are quite flexible. Such methods can be a useful for unsupervised learning (knowledge discovery), data visualization, and as a preprocessing step for supervised learning.

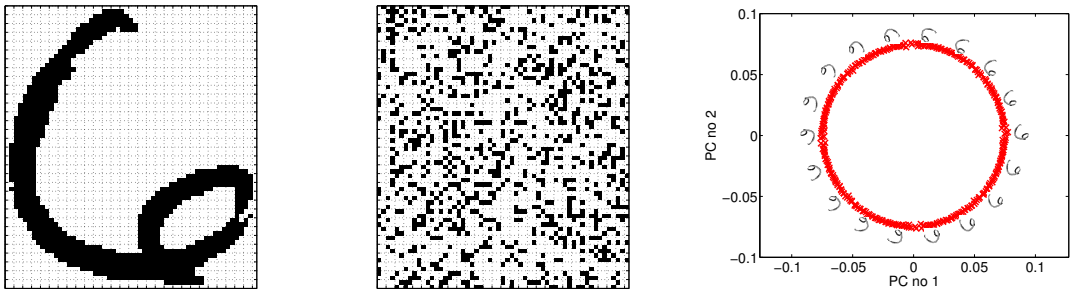
20.4.1 What are manifolds?

Roughly speaking, a **manifold** is a topological space which is locally Euclidean. One of the simplest examples is the surface of the earth, which is a curved 2d surface embedded in a 3d space. At each local point on the surface, the earth seems flat.

More formally, a d-dimensional manifold $\mathcal{X}$ is a space in which each point $x \in \mathcal{X}$ has a neighborhood which is topologically equivalent to a d-dimensional Euclidean space, called the **tangent space**, denoted $\mathcal{T}_x = T_x\mathcal{X}$. This is illustrated in Fig. 20.28.

A **Riemannian manifold** is a differentiable manifold that associates an inner product operator at each point x in tangent space; this is assumed to depend smoothly on the position x. The inner product induces a notion of distance, angles, and volume. The collection of these inner products is

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 20.29: Illustration of the image manifold. (a) An image of the digit 6 from the USPS dataset, of size $64 \times 57 = 3,648$. *(b) A random sample from the space* $\{0,1\}^{3648}$ *reshaped as an image. (c) A dataset created by rotating the original image by one degree 360 times. We project this data onto its first two principal components, to reveal the underlying 2d circular manifold. From Figure 1 of [Law12]. Used with kind permission of Neil Lawrence*

called a **Riemannian metric**. It can be shown that any sufficiently smooth Riemannian manifold can be embedded into a Euclidean space of potentially higher dimension; the Riemannian inner product at a point then becomes Euclidean inner product in that tangent space.

20.4.2 The manifold hypothesis

Most "naturally occuring" high dimensional dataset lie a low dimensional manifold. This is called the **manifold hypothesis** [FMN16]. For example, consider the case of an image. Fig. 20.29a shows a single image of size 64×57. This is a vector in a 3,648-dimensional space, where each dimension corresponds to a pixel intensity. Suppose we try to generate an image by drawing a random point in this space; it is unlikely to look like the image of a digit, as shown in Fig. 20.29b. However, the pixels are not independent of each other, since they are generated by some lower dimensional structure, namely the shape of the digit 6.

As we vary the shape, we will generate different images. We can often characterize the space of shape variations using a low-dimensional manifold. This is illustrated in Fig. 20.29c, where we apply PCA (Sec. 20.1) to project a dataset of 360 images, each one a slightly rotated version of the digit 6, into a 2d space. We see that most of the variation in the data is captured by an underlying curved 2d manifold. We say that the **intrinsic dimensionality** d of the data is 2, even though the **ambient dimensionality** D is 3,648.

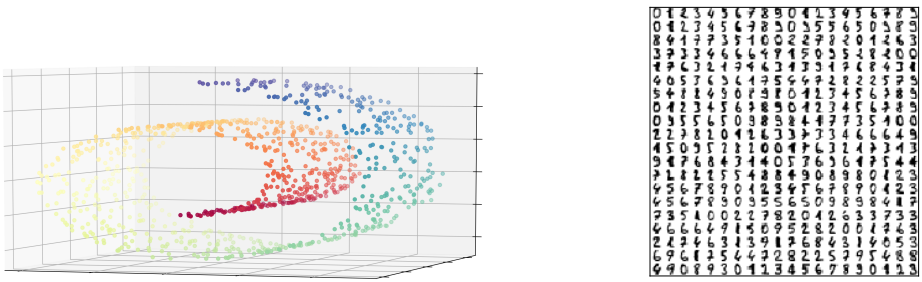
20.4.3 Approaches to manifold learning

In the rest of this section, we discuss ways to learn manifolds from data. There are many different algorithms that have been proposed, which make different assumptions about the nature of the manifold, and which have different computational properties. We discuss a few of these methods in the following sections. For more details, see e.g., [Bur10].

The methods can be categorized as shown in Table 20.1. The term "nonparametric" refers to

Method	Parametric	Convex	Section
PCA / classical MDS	N	Y (Dense)	Sec. 20.1
Kernel PCA	N	Y (Dense)	Sec. 20.4.6
Isomap	N	Y (Dense)	Sec. 20.4.5
LLE	N	Y (Sparse)	Sec. 20.4.8
Laplacian Eigenmaps	N	Y (Sparse)	Sec. 20.4.9
tSNE	N	N	Sec. 20.4.10
Autoencoder	Y	N	Sec. 20.3

Table 20.1: A list of some approaches to dimensionality reduction. If a method is convex, we specify in parentheses whether it requires solving a sparse or dense eigenvalue problem.



(a) *(b)*

Figure 20.30: Illustration of some data generated from low-dimensional manifolds. (a) The 2d Swiss-roll manifold embedded into 3d. Generated by code at figures.probml.ai/book1/20.30. (b) Sample of some UCI digits, which have size $8 \times 8 = 64$. Generated by code at figures.probml.ai/book1/20.30.

methods that learn a low dimensional embedding $\mathbf{z}_i$ for each datapoint $\mathbf{x}_i$, but do not learn a mapping function which can be applied to an out-of-sample datapoint. (However, [Ben+04b] discusses how to extend many of these methods beyond the training set by learning a kernel.)

In the sections below, we compare some of these methods using 2 different datasets: a set of 1000 3d-points sampled from the 2d "**Swiss roll**" manifold, and a set of 1797 64-dimensional points sampled from the UCI digits dataset. See Fig. 20.30 for an illustration of the data. We will learn a 2d manifold, so we can visualize the data.

20.4.4 Multi-dimensional scaling (MDS)

The simplest approach to manifold learning is **multidimensional scaling** (**MDS**). This tries to find a set of low dimensional vectors $\{\mathbf{z}_i \in \mathbb{R}^L : i = 1 : N\}$ such that the pairwise distances between these vectors is as similar as possible to a set of pairwise dissimilarities $\mathbf{D} = \{d_{ij}\}$ provided by the user. There are several variants of MDS, one of which turns out to be equivalent to PCA, as we discuss below.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.4.4.1 Classical MDS

Suppose we start an $N \times D$ data matrix $\mathbf{X}$ with rows $\mathbf{x}_i$. Let us define the centered Gram (similarity) matrix as follows:

$$\tilde{K}_{ij} = \langle \mathbf{x}_i - \overline{\mathbf{x}}, \mathbf{x}_j - \overline{\mathbf{x}} \rangle \tag{20.104}$$

In matrix notation, we have $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top$, where $\tilde{\mathbf{X}} = \mathbf{C}_N\mathbf{X}$ and $\mathbf{C}_N = \mathbf{I}_N - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\top$ is the centering matrix.

Now define the **strain** of a set of embeddings as follows:

$$\mathcal{L}_{\text{strain}}(\mathbf{Z}) = \sum_{i,j}(\tilde{K}_{ij} - \langle \tilde{\mathbf{z}}_i, \tilde{\mathbf{z}}_j \rangle)^2 = ||\tilde{\mathbf{K}} - \tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\top||_F^2 \tag{20.105}$$

where $\tilde{\mathbf{z}}_i = \mathbf{z}_i - \overline{\mathbf{z}}$ is the centered embedding vector. Intuitively this measures how well similarities in the high-dimensional data space, $\tilde{K}_{ij}$, are matched by similarities in the low-dimensional embedding space, $\langle \tilde{\mathbf{z}}_i, \tilde{\mathbf{z}}_j \rangle$. Minimizing this loss is called **classical MDS**.

We know from Sec. 7.5 that the best rank L approximation to a matrix is its truncated SVD representation, $\tilde{\mathbf{K}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. Since $\tilde{\mathbf{K}}$ is positive semi definite, we have that $\mathbf{V} = \mathbf{U}$. Hence the optimal embedding satisfies

$$\tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\top = \mathbf{U}\mathbf{S}\mathbf{U}^\top = (\mathbf{U}\mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}}\mathbf{U}^\top) \tag{20.106}$$

Thus we can set the embedding vectors to be the rows of $\tilde{\mathbf{Z}} = \mathbf{U}\mathbf{S}^{\frac{1}{2}}$.

Now we describe how to apply classical MDS to a dataset where we just have Euclidean distances, rather than raw features. First we compute a matrix of squared Euclidean distances, $\mathbf{D}^{(2)} = \mathbf{D} \odot \mathbf{D}$, which has the following entries:

$$D_{ij}^{(2)} = ||\mathbf{x}_i - \mathbf{x}_j||^2 = ||\mathbf{x}_i - \overline{\mathbf{x}}||^2 + ||\mathbf{x}_j - \overline{\mathbf{x}}||^2 - 2\langle \mathbf{x}_i - \overline{\mathbf{x}}, \mathbf{x}_j - \overline{\mathbf{x}} \rangle \tag{20.107}$$

$$= ||\mathbf{x}_i - \overline{\mathbf{x}}||^2 + ||\mathbf{x}_j - \overline{\mathbf{x}}||^2 - 2\tilde{K}_{ij} \tag{20.108}$$

We see that $\mathbf{D}^{(2)}$ only differs from $\tilde{\mathbf{K}}$ by some row and column constants (and a factor of -2). Hence we can compute $\tilde{\mathbf{K}}$ by double centering $\mathbf{D}^{(2)}$ using Eq. (7.88) to get $\tilde{\mathbf{K}} = -\frac{1}{2}\mathbf{C}_N\mathbf{D}^{(2)}\mathbf{C}_N$. In other words,

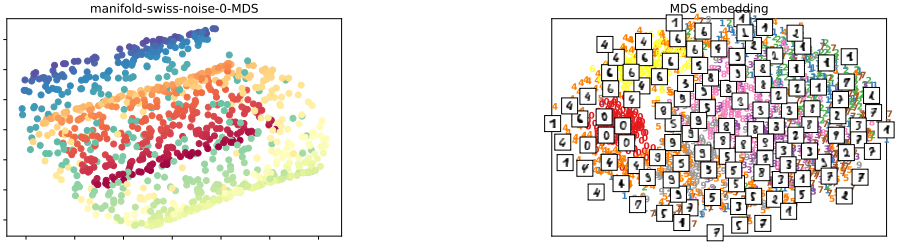
$$\tilde{K}_{ij} = -\frac{1}{2}\left(d_{ij}^2 - \frac{1}{N}\sum_{l=1}^N d_{il}^2 - \frac{1}{N}\sum_{l=1}^N d_{jl}^2 + \frac{1}{N^2}\sum_{l=1}^N\sum_{m=1}^N d_{lm}^2\right) \tag{20.109}$$

We can then compute the embeddings as before.

It turns out that classical MDS is equivalent to PCA (Sec. 20.1). To see this, let $\tilde{\mathbf{K}} = \mathbf{U}_L\mathbf{S}_L\mathbf{U}_L^\top$ be the rank L truncated SVD of the centered kernel matrix. The MDS embedding is given by $\mathbf{Z}_{\text{MDS}} = \mathbf{U}_L\mathbf{S}_L^{\frac{1}{2}}$. Now consider the rank L SVD of the centered data matrix, $\tilde{\mathbf{X}} = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\top$. The PCA embedding is $\mathbf{Z}_{\text{PCA}} = \mathbf{U}_X\mathbf{S}_X$. Now

$$\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\top\mathbf{V}_X\mathbf{S}_X\mathbf{U}_X^\top = \mathbf{U}_X\mathbf{S}_X^2\mathbf{U}_X^\top = \mathbf{U}_L\mathbf{S}_L\mathbf{U}_L^\top \tag{20.110}$$

Hence $\mathbf{U}_X = \mathbf{U}_L$ and $\mathbf{S}_X = \mathbf{S}_L^2$, and so $\mathbf{Z}_{\text{PCA}} = \mathbf{Z}_{\text{MDS}}$.



(a) (b)

Figure 20.31: Metric MDS applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.31 (b) UCI digits. Generated by code at figures.probml.ai/book1/20.31

20.4.4.2 Metric MDS

Classical MDS assumes Euclidean distances. We can generalize it to allow for any dissimilarity measure by defining the **stress function**

$$\mathcal{L}_{\text{stress}}(\mathbf{Z}) = \sqrt{\frac{\sum_{i<j}(d_{i,j} - \hat{d}_{ij})^2}{\sum_{ij} d_{ij}^2}} \tag{20.111}$$

where $\hat{d}_{ij} = ||\mathbf{z}_i - \mathbf{z}_j||$. This is called **metric MDS**. Note that this is a different objective than the one used by classical MDS, so even if d_{ij} are Euclidean distances, the results will be different.

We can use gradient descent to solve the optimization problem. However, it is better to use an bound optimization algorithm (Sec. 8.7) called **SMACOF** [Lee77], which stands for "Scaling by MAjorizing a COmplication Function". (This is the method implemented in scikit-learn.) See Fig. 20.31 for the results of applying this to our running example.

20.4.4.3 Non-metric MDS

Instead of trying to match the distance between points, we can instead just try to match the ranking of how similar points are. To do this, let $f(d)$ be a monotonic transformation from distances to ranks. Now define the loss

$$\mathcal{L}_{\text{NM}}(\mathbf{Z}) = \sqrt{\frac{\sum_{i<j}(f(d_{i,j}) - \hat{d}_{ij})^2}{\sum_{ij} \hat{d}_{ij}^2}} \tag{20.112}$$

where $\hat{d}_{ij} = ||\mathbf{z}_i - \mathbf{z}_j||$. Minimizing this is known as **non-metric MDS**.

This objective can be optimized iteratively. First the function f is optimized, for a given $\mathbf{Z}$, using isotonic regression; this finds the optimal monotonic transformation of the input distances to match the current embedding distances. Then the embeddings $\mathbf{Z}$ are optimized, for a given f, using gradient descent, and the process repeats.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

(a) (b)

Figure 20.32: (a) If we measure distances along the manifold, we find $d(1,6) > d(1,4)$, whereas if we measure in ambient space, we find $d(1,6) < d(1,4)$. The plot at the bottom shows the underlying 1d manifold. (b) The K-nearest neighbors graph for some datapoints; the red path is the shortest distance between A and B on this graph. From [Hin13]. Used with kind permission of Geoff Hinton.

20.4.4.4 Sammon mapping

Metric MDS tries to minimize the sum of squared distances, so it puts the most emphasis on large distances. However, for many embedding methods, small distances matter more, since they capture local structure. One way to capture this is to divide each term of the loss by d_{ij}, so small distances get upweighted:

$$\mathcal{L}_{\text{sammon}}(\mathbf{Z}) = \left(\frac{1}{\sum_{i<j} d_{ij}}\right) \sum_{i \neq j} \frac{(\hat{d}_{ij} - d_{ij})^2}{d_{ij}} \tag{20.113}$$

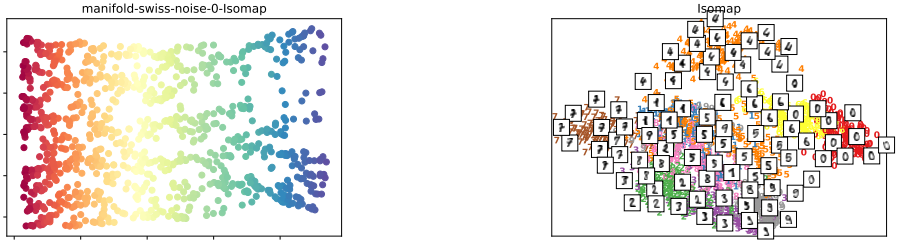
Minimizing this results in a **Sammon mapping**. (The coefficient in front of the sum is just to simplify the gradient of the loss.) Unfortunately this is a non-convex objective, and it arguably puts too much emphasis on getting very small distances exactly right. We will discuss better methods for capturing local structure later on.

20.4.5 Isomap

If the high-dimensional data lies on or near a curved manifold, such as the Swiss roll example, then MDS might consider two points to be close even if their distance along the manifold is large. This is illustrated in Fig. 20.32a.

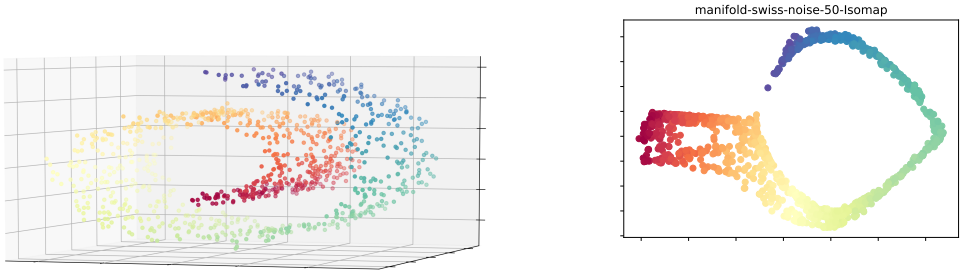
One way to capture this is to create the K-nearest neighbor graph between datapoints[5], and then approximate the manifold distance between a pair of points by the shortest distance along this graph; this can be computed efficiently using Dijkstra's shortest path algorithm. See Fig. 20.32b for an illustration. Once we have computed this new distance metric, we can apply classical MDS (i.e., PCA). This is a way to capture local structure while avoiding local optima. The overall method is called **isomap** [TSL00].

5. In scikit-learn, you can use the function `sklearn.neighbors.kneighbors_graph`.



(a) (b)

Figure 20.33: Isomap applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.33. (b) UCI digits. Generated by code at figures.probml.ai/book1/20.33.



(a) (b)

Figure 20.34: (a) Noisy version of Swiss roll data. We perturb each point by adding $\mathcal{N}(0, 0.5^2)$ noise. (b) Results of Isomap applied to this data. Generated by code at figures.probml.ai/book1/20.34

See Fig. 20.33 for the results of this method on our running example. We see that they are quite reasonable. However, if the data is noisy, there can be "false" edges in the nearest neighbor graph, which can result in "short circuits" which significantly distort the embedding, as shown in Fig. 20.34. This problem is known as "**topological instability**" [BS02]. Choosing a very small neighborhood does not solve this problem, since this can fragment the manifold into a large number of disconnected regions. Various other solutions have been proposed, e.g., [CC07].

20.4.6 Kernel PCA

PCA (and classical MDS) finds the best linear projection of the data, so as to preserve pairwise similarities between all the points. In this section, we consider nonlinear projections. The key idea is to solve PCA by finding the eigenvectors of the inner product (Gram) matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$, as in Sec. 20.1.3.2, and then to use the kernel trick (Sec. 17.5.4), which lets us replace inner products such as $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel function, $K_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_k)$. This is known as **kernel PCA** [SSM98].

Recall from Mercer's theorem that the use of a kernel implies some underlying feature space, so we are implicitly replacing $\mathbf{x}_i$ with $\boldsymbol{\phi}(\mathbf{x}_i) = \boldsymbol{\phi}_i$. Let $\boldsymbol{\Phi}$ be the corresponding (notional) design matrix,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

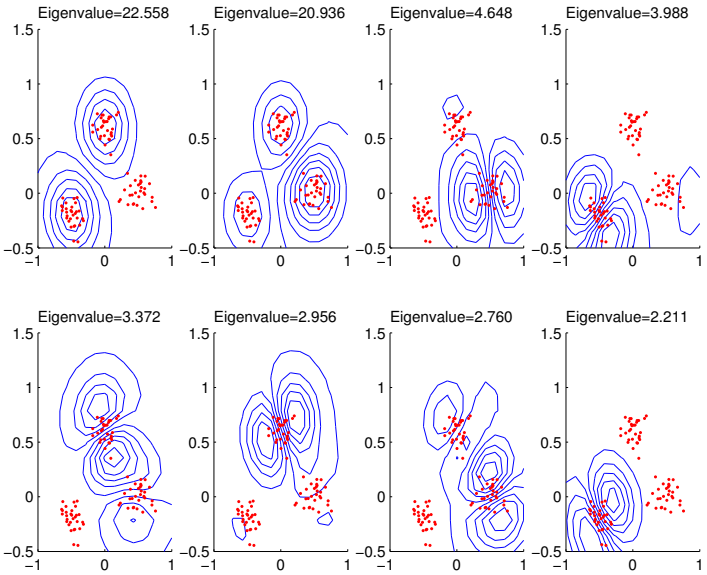


Figure 20.35: Visualization of the first 8 kernel principal component basis functions derived from some 2d data. We use an RBF kernel with $\sigma^2 = 0.1$. Generated by code at figures.probml.ai/book1/20.35.

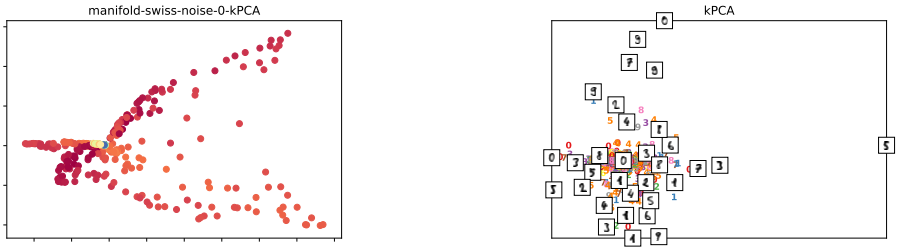
and $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$ be the Gram matrix. Finally, let $\mathbf{S}_\phi = \frac{1}{N}\sum_i \boldsymbol{\phi}_i \boldsymbol{\phi}_i^\top$ be the covariance matrix in feature space. (We are assuming for now the features are centered.) From Eq. (20.22), the normalized eigenvectors of $\mathbf{S}$ are given by $\mathbf{V}_{\text{kPCA}} = \boldsymbol{\Phi}^\top \mathbf{U}\boldsymbol{\Lambda}^{-\frac{1}{2}}$, where $\mathbf{U}$ and $\boldsymbol{\Lambda}$ contain the eigenvectors and eigenvalues of $\mathbf{K}$. Of course, we can't actually compute $\mathbf{V}_{\text{kPCA}}$, since $\boldsymbol{\phi}_i$ is potentially infinite dimensional. However, we can compute the projection of a test vector $\mathbf{x}_*$ onto the feature space as follows:

$$\boldsymbol{\phi}_*^\top \mathbf{V}_{\text{kPCA}} = \boldsymbol{\phi}_*^\top \boldsymbol{\Phi}^\top \mathbf{U}\boldsymbol{\Lambda}^{-\frac{1}{2}} = \mathbf{k}_*^\top \mathbf{U}\boldsymbol{\Lambda}^{-\frac{1}{2}} \tag{20.114}$$

where $\mathbf{k}_* = [\mathcal{K}(\mathbf{x}_*, \mathbf{x}_1), \ldots, \mathcal{K}(\mathbf{x}_*, \mathbf{x}_N)]$.

There is one final detail to worry about. The covariance matrix is only given by $\mathbf{S} = \boldsymbol{\Phi}^\top \boldsymbol{\Phi}$ if the features is zero-mean. Thus we can only use the Gram matrix $\mathbf{K} = \boldsymbol{\Phi}\boldsymbol{\Phi}^\top$ if $\mathbb{E}[\boldsymbol{\phi}_i] = \mathbf{0}$. Unfortunately, we cannot simply subtract off the mean in feature space, since it may be infinite dimensional. However, there is a trick we can use. Define the centered feature vector as $\tilde{\boldsymbol{\phi}}_i = \boldsymbol{\phi}(\mathbf{x}_i) - \frac{1}{N}\sum_{j=1}^N \boldsymbol{\phi}(\mathbf{x}_j)$. The Gram matrix of the centered feature vectors is given by $\tilde{K}_{ij} = \tilde{\boldsymbol{\phi}}_i^\top \tilde{\boldsymbol{\phi}}_j$. Using the double centering trick from Eq. (7.88), we can write this in matrix form as $\tilde{\mathbf{K}} = \mathbf{C}_N \mathbf{K} \mathbf{C}_N$, where $\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N}\mathbf{1}_N \mathbf{1}_N^\top$ is the centering matrix.

If we apply kPCA with a linear kernel, we recover regular PCA (classical MDS). This is limited to using $L \leq D$ embedding dimensions. If we use a non-degenerate kernel, we can use up to N components, since the size of $\boldsymbol{\Phi}$ is $N \times D^*$, where D^* is the (potentially infinite) dimensionality of embedded feature vectors. Fig. 20.35 gives an example of the method applied to some $D = 2$ dimensional data using an RBF kernel. We project points in the unit grid onto the first 8 components



(a) (b)

Figure 20.36: Kernel PCA applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.36. (b) UCI digits. Generated by code at figures.probml.ai/book1/20.36.

and visualize the corresponding surfaces using a contour plot. We see that the first two component separate the three clusters, and the following components split the clusters.

See Fig. 20.36 for some the results on kPCA (with an RBF kernel) on our running example. In this case, the results are arguably not very useful. In fact, it can be shown that kPCA with an RBF kernel expands the feature space instead of reducing it [WSS04], as we saw in Fig. 20.35, which makes it not very useful as a method for dimensionality reduction. We discuss a solution to this in Sec. 20.4.7.

20.4.7 Maximum variance unfolding (MVU)

kPCA with certain kernels, such as RBF, might not result in a low dimensional embedding, as discussed in Sec. 20.4.6. This observation led to the development of the **semidefinite embedding** algorithm [WSS04], also called **maximum variance unfolding**, which tries to learn an embedding $\{\mathbf{z}_i\}$ such that

$$\max \sum_{ij} ||\mathbf{z}_i - \mathbf{z}_j||_2^2 \;\; \text{s.t.} \;\; ||\mathbf{z}_i - \mathbf{z}_j||_2^2 = ||\mathbf{x}_i - \mathbf{x}_j||_2^2 \text{ for all } (i,j) \in G \tag{20.115}$$

where G is the nearest neighbor graph (as in Isomap). This approach explicitly tries to 'unfold' the data manifold while respecting the nearest neighbor constraints.

This can be reformulated as a **semidefinite programming** (SDP) problem by defining the kernel matrix $\mathbf{K} = \mathbf{Z}\mathbf{Z}^\top$ and then optimizing

$$\max \text{tr}(\mathbf{K}) \;\; \text{s.t.} \;\; ||\mathbf{z}_i - \mathbf{z}_j||_2^2 = ||\mathbf{x}_i - \mathbf{x}_j||_2^2, \; \sum_{ij} K_{ij} = 0, \; \mathbf{K} \succ 0 \tag{20.116}$$

The resulting kernel is then passed to kPCA, and the resulting eigenvectors give the low dimensional embedding.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.4.8 Local linear embedding (LLE)

The techniques we have discussed so far all rely on an eigendecomposition of a full matrix of pairwise similarities, either in the ambient space (PCA), in feature space (kPCA), or along the KNN graph (Isomap). In this section, we discuss **local linear embedding** (LLE) [RS00], a technique that solves a sparse eigenproblem, thus focusing more on local structure in the data.

LLE assumes the data manifold around each point $\mathbf{x}_i$ is locally linear. The best linear approximation can be found by predicting $\mathbf{x}_i$ as a linear combination of its K nearest neigbhbors using reconstruction weights $\mathbf{w}_i$. This can be found by solving

$$\hat{\mathbf{W}} = \min_{\mathbf{W}} \sum_{i=1}^{N} (\mathbf{x}_i - \sum_{j=1}^{N} w_{ij}\mathbf{x}_j)^2 \tag{20.117}$$

$$\text{subject to} \begin{cases} w_{ij} = 0 & \text{if } \mathbf{x}_j \notin \text{nbr}(\mathbf{x}_i, K) \\ \sum_{j=1}^{N} w_{ij} = 1 & \text{for } i = 1 : N \end{cases} \tag{20.118}$$

Note that we need the sum-to-one constraint on the weights to prevent the trivial solution $\mathbf{W} = \mathbf{0}$. The resulting vector of weights $\mathbf{w}_{i,:}$ constitute the **barycentric coordinates** of $\mathbf{x}_i$.

Any linear mapping of this hyperplane to a lower dimensional space perserves the reconstruction weights, and thus the local geometry. Thus we can solve for the low-dimensional embeddings for each point by solving

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\text{argmin}} \sum_{i} ||\mathbf{z}_i - \sum_{j=1}^{N} \hat{w}_{ij}\mathbf{z}_j||_2^2 \tag{20.119}$$

where $\hat{w}_{ij} = 0$ if j is not one of the K nearest neighbors of i. We can rewrite this loss as

$$\mathcal{L}(\mathbf{Z}) = ||\mathbf{Z} - \mathbf{W}\mathbf{Z}||^2 = \mathbf{Z}^\top (\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})\mathbf{Z} \tag{20.120}$$

Thus the solution is given by the eigenvectors of $(\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})$ corresponding to the smallest nonzero eigenvalues, as shown in Sec. 7.4.8.

See Fig. 20.37 for some the results on LLE on our running example. In this case, the results do not seem as good as those produced by Isompa. However, the method tends to be somewhat less sensitive to short-circuiting (noise).

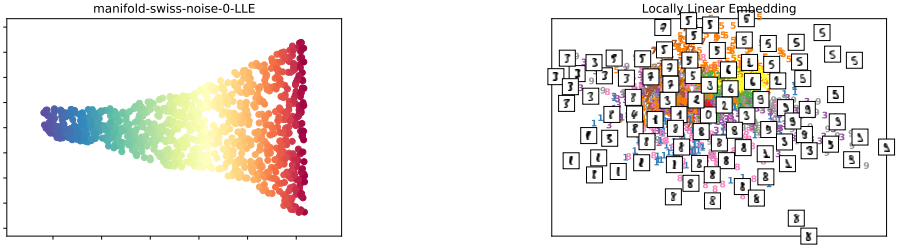
20.4.9 Laplacian eigenmaps

In this section, we describe **Laplacian eigenmaps** or **spectral embedding** [BN01]. The idea is to compute a low-dimensional representation of the data in which the weighted distances between a datapoint and its K nearest neighors are minimized. We put more weight on the first nearest neighbor than the second, etc. We give the details below.

20.4.9.1 Using eigenvectors of the graph Laplacian to compute embeddings

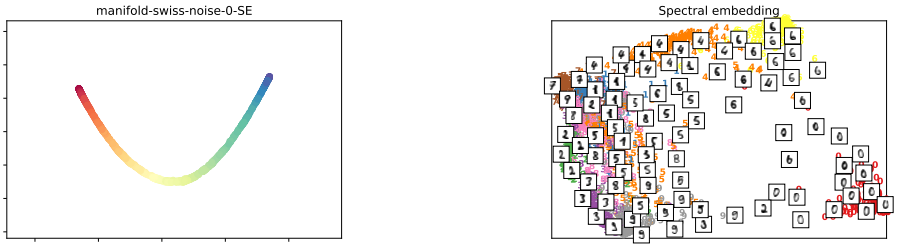
We want to find embeddings which minimize

$$\mathcal{L}(\mathbf{Z}) = \sum_{(i,j) \in E} W_{i,j} ||\mathbf{z}_i - \mathbf{z}_j||_2^2 \tag{20.121}$$



(a) (b)

Figure 20.37: LLE applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.37. (b) UCI digits. Generated by code at figures.probml.ai/book1/20.37.



(a) (b)

Figure 20.38: Laplacian eigenmaps applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.38. (b) UCI digits. Generated by code at figures.probml.ai/book1/20.38.

where $W_{ij} = \exp(-\frac{1}{2\sigma^2}||\mathbf{x}_i - \mathbf{x}_j||_2^2)$ if $i - j$ are neighbors in the KNN graph and 0 otherwise. We add the constraint $\mathbf{Z}^\top \mathbf{D}\mathbf{Z} = \mathbf{I}$ to avoid the degenerate solution where $\mathbf{Z} = \mathbf{0}$, where $\mathbf{D}$ is the diagonal weight matrix storing the degree of each node, $D_{ii} = \sum_j W_{i,j}$.

We can rewrite the above objective as follows:

$$\mathcal{L}(\mathbf{Z}) = \sum_{ij} W_{ij}(||\mathbf{z}_i||^2 + ||\mathbf{z}_j||^2 - 2\mathbf{z}_i\mathbf{z}_j^\top) \tag{20.122}$$

$$= \sum_i D_{ii}||\mathbf{z}_i||^2 + \sum_j D_{jj}||\mathbf{z}_j||^2 - 2\sum_{ij} W_{ij}\mathbf{z}_i\mathbf{z}_j^\top \tag{20.123}$$

$$= 2\mathbf{Z}^\top\mathbf{D}\mathbf{Z} - 2\mathbf{Z}^\top\mathbf{W}\mathbf{Z} = 2\mathbf{Z}^\top\mathbf{L}\mathbf{Z} \tag{20.124}$$

where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is the graph Laplacian (see Sec. 20.4.9.2). One can show that minimizing this is equivalent to solving the eigenvalue problem $\mathbf{L}\mathbf{z}_i = \lambda\mathbf{D}\mathbf{z}_i$ for the L smallest nonzero eigenvalues.

See Fig. 20.38 for the results of applying this method (with an RBF kernel) to our running example.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

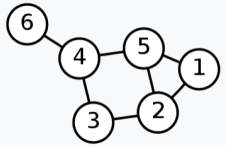
Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

Figure 20.39: Illustration of the Laplacian matrix derived from an undirected graph. From https://en.wikipedia.org/wiki/Laplacian_matrix. Used with kind permission of Wikipedia author AzaToth.

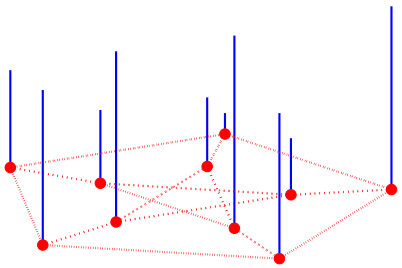


Figure 20.40: Illustration of a (positive) function defined on a graph. From Figure 1 of [Shu+13]. Used with kind permission of Pascal Frossard.

20.4.9.2 What is the graph Laplacian?

We saw above that we can compute the eigenvectors of the graph Laplacian in order to learn a good embedding of the high dimensional points. In this section, we give some intuition as to why this works.

Let $\mathbf{W}$ be a symmetric weight matrix for a graph, where $W_{ij} = W_{ji} \geq 0$. Let $\mathbf{D} = \text{diag}(d_i)$ be a diagonal matrix containing the weighted degree of each node, $d_i = \sum_j w_{ij}$. We define the **graph Laplacian** as follows:

$$\mathbf{L} \triangleq \mathbf{D} - \mathbf{W} \tag{20.125}$$

Thus the elements of $\mathbf{L}$ are given by

$$L_{ij} = \begin{cases} d_i & \text{if } i = j \\ -1 & \text{if } i \neq j \text{ and } w_{ij} \neq 0 \\ 0 & \text{otherwise} \end{cases} \tag{20.126}$$

See Fig. 20.39 for an example of how to compute this.

Suppose we associate a value $f_i \in \mathbb{R}$ with each node i in the graph (see Fig. 20.40 for example). Then we can use the graph Laplacian as a difference operator, to compute a discrete derivative of

the function at a point:

$$(\mathbf{Lf})(i) = \sum_{i \in \text{nbr}_i} W_{ij}[f(i) - f(j)] \tag{20.127}$$

where nbr_i is the set of neighbors of node i. We can also compute an overall measure of "smoothness" of the function f by computing its **Dirichlet energy** as follows:

$$\mathbf{f}^T\mathbf{Lf} = \mathbf{f}^T\mathbf{Df} - \mathbf{f}^T\mathbf{Wf} = \sum_i d_i f_i^2 - \sum_{i,j} f_i f_j w_{ij} \tag{20.128}$$

$$= \frac{1}{2}\left(\sum_i d_i f_i^2 - 2\sum_{i,j} f_i f_j w_{ij} + \sum_j d_j f_j^2\right) = \frac{1}{2}\sum_{i,j} w_{ij}(f_i - f_j)^2 \tag{20.129}$$

By studying the eigenvalues and eigenvectors of the Laplacian matrix, we can determine various useful properties of the function. (Applying linear algebra to study the adjacency matrix of a graph, or related matrices, is called **spectral graph theory** [Chu97].) For example, we see that $\mathbf{L}$ is symmetric and positive semi-definite, since we have $\mathbf{f}^T\mathbf{Lf} \geq 0$ for all $\mathbf{f} \in \mathbb{R}^N$, which follows from Eq. (20.129) due to the assumption that $w_{ij} \geq 0$. Consequently $\mathbf{L}$ has N non-negative, real-valued eigenvalues, $0 \leq \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_N$. The corresponding eigenvectors form an orthogonal basis for the function f defined on the graph, in order of decreasing smoothness.

In Sec. 20.4.9.1, we discuss Laplacian eigenmaps, which is a way to learn low dimensional embeddings for high dimensional data vectors. The approach is to let $z_{id} = f_i^d$ be the d'th embedding dimension for input i, and then to find a basis for these functions (i.e., embedding of the points) that varies smoothly over the graph, thus respecting distance of the points in ambient space.

There are many other applications of the graph Laplacian in ML. For example, in Sec. 21.5.1, we discuss normalized cuts, which is a way to learn a clustering of high dimensional data vectors based on pairwise similarity; and [WTN19] discusses how to use the eigenvectors of the state transition matrix to learn representations for RL.

20.4.10 t-SNE

In this section, we describe a very popular nonconvex technique for learning low dimensional embeddings called **t-SNE** [MH08]. This extends the earlier **stochastic neighbor embedding** method of [HR03], so we first describe SNE, before describing the t-SNE extension.

20.4.10.1 Stochastic neighborhood embedding (SNE)

The basic idea in SNE is to convert high-dimensional Euclidean distances into conditional probabilities that represent similarities. More precisely, we define $p_{j|i}$ to be the probability that point j would pick point i as its neighbor if neighbors were picked in proportion to their probability under a Gaussian centered at $\mathbf{x}_i$:

$$p_{j|i} = \frac{\exp(-\frac{1}{2\sigma_i^2}||\mathbf{x}_i - \mathbf{x}_j||^2)}{\sum_{k \neq i} \exp(-\frac{1}{2\sigma_i^2}||\mathbf{x}_i - \mathbf{x}_k||^2)} \tag{20.130}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Here σ_i^2 is the variance for data point i, which can be used to "magnify" the scale of points in dense regions of input space, and diminish the scale in sparser regions. (We discuss how to estimate the length scales σ_i^2 shortly).

Let $\mathbf{z}_i$ be the low dimensional embedding representing $\mathbf{x}_i$. We define similarities in the low dimensional space in an analogous way:

$$q_{j|i} = \frac{\exp(-||\mathbf{z}_i - \mathbf{z}_j||^2)}{\sum_{k \neq i} \exp(-||\mathbf{z}_i - \mathbf{z}_k||^2)} \tag{20.131}$$

In this case, the variance is fixed to a constant; changing it would just rescale the learned map, and not change its topology.

If the embedding is a good one, then $q_{j|i}$ should match $p_{j|i}$. Therefore, SNE defines the objective to be

$$\mathcal{L} = \sum_i \mathbb{KL}\left(P_i \| Q_i\right) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \tag{20.132}$$

where P_i is the conditional distribution over all other data points given $\mathbf{x}_i$, Q_i is the conditional distribution over all other latent points given $\mathbf{z}_i$, and $\mathbb{KL}\left(P_i \| Q_i\right)$ is the KL divergence (Sec. 6.2) between the distributions.

Note that this is an asymmetric objective. In particular, there is a large cost if a small $q_{j|i}$ is used to model a large $p_{j|i}$. This the objective will prefer to pull distant points together rather than push nearby points apart. We can get a better idea of the geometry by looking at the gradient for each embedding vector, which is given by

$$\nabla_{\mathbf{z}_i} \mathcal{L}(\mathbf{Z}) = 2 \sum_j (\mathbf{z}_j - \mathbf{z}_i)(p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j}) \tag{20.133}$$

Thus points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's.

Although this is an intuitively sensible objective, it is not convex. Nevertheless it can be minimized using SGD. In practice, it helps to add Gaussian noise to the embedding points, and to gradually anneal the amount of noise. [Hin13] recommends to "spend a long time at the noise level at which the global structure starts to form from the hot plasma of map points" before reducing it.[6]

20.4.10.2 Symmetric SNE

There is a slightly simpler version of SNE that minimizes a single KL between the joint distribution P in high dimensional space and Q in low dimensional space:

$$\mathcal{L} = \mathbb{KL}\left(P \| Q\right) = \sum_{i<j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{20.134}$$

This is called **symmetric SNE**.

6. See [Ros98; WF20] for a discussion of annealing and phase transitions in unsupervised learning. See also [CP10] for a discussion of the **elastic embedding** algorithm, which uses a homotopy method to more efficiently optimize a model that is related to both SNE and Laplacian eigenmaps.

The obvious way to define p_{ij} is to use

$$p_{ij} = \frac{\exp(-\frac{1}{2\sigma^2}||\mathbf{x}_i - \mathbf{x}_j||^2)}{\sum_{k<l} \exp(-\frac{1}{2\sigma^2}||\mathbf{x}_k - \mathbf{x}_l||^2)} \tag{20.135}$$

We can define q_{ij} similarily. The corresponding gradient becomes

$$\nabla_{\mathbf{z}_i} \mathcal{L}(\mathbf{Z}) = 2 \sum_j (\mathbf{z}_j - \mathbf{z}_i)(p_{ij} - q_{ij}) \tag{20.136}$$

As before, points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's.

Although symmetric SNE is slightly easier to implement, it loses the nice property of regular SNE that the data is its own optimal embedding if the embedding dimension L is set equal to the ambient dimension D. Nevertheless, the methods seems to give similar results in practice on real datasets where $L \ll D$.

20.4.10.3 t-distributed SNE

A fundamental problem with SNE and many other embedding techniques is that they tend to squeeze points that are relatively far away in the high dimensional space close together in the low dimensional (usually 2d) embedding space; this is called the **crowding problem**, and arises due to the use of squared errors (or Gaussian probabilities).

One solution to this is to use a probability distribution in latent space that has heavier tails, which eliminates the unwanted attractive forces between points that are relatively far in the high dimensional space. An obvious choice is the Student-t distribution (Sec. 2.7.1). In t-SNE, they set the degree of freedom parameter to $\nu = 1$, so the distribution becomes equivalent to a Cauchy:

$$q_{ij} = \frac{(1 + ||\mathbf{z}_i - \mathbf{z}_j||^2)^{-1}}{\sum_{k<l}(1 + ||\mathbf{z}_k - \mathbf{z}_l||^2)^{-1}} \tag{20.137}$$

We can use the same global KL objective as in Eq. (20.134). For t-SNE, the gradient turns out to be

$$\nabla \mathbf{z}_i \mathcal{L} = 4 \sum_j (p_{ij} - q_{ij})(\mathbf{z}_i - \mathbf{z}_j)(1 + ||\mathbf{z}_i - \mathbf{z}_j||^2)^{-1} \tag{20.138}$$

The gradient for symmetric (Gaussian) SNE is the same, but lacks the $(1 + ||\mathbf{z}_i - \mathbf{z}_j||^2)^{-1}$ term. This term is useful because $(1 + ||\mathbf{z}_i - \mathbf{z}_j||^2)^{-1}$ acts like an inverse square law. This means that points in embedding space act like stars and galaxies, forming many well-separated clusters (galaxies) each of which has many starts tightly packed inside. This can be useful for separating different classes of data in an unsupervised way (see Fig. 20.41 for an example).

20.4.10.4 Choosing the length scale

An important parameter in t-SNE is the local bandwidth σ_i^2. This is usually chosen so that P_i has a perplexity chosen by the user.[7] This can be interpreted as a smooth measure of the effective number

7. The perplexity is defined to be $2^{\mathbb{H}(P_i)}$, where $\mathbb{H}(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$ is the entropy; see Sec. 6.1.5 for details. A big radius around each point (large value of σ_i) will result in a high entropy, and thus high perplexity.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

(a) (b)

Figure 20.41: tSNE applied to (a) Swiss roll. Generated by code at figures.probml.ai/book1/20.41. (b) UCI digits. Generated by code at figures.probml.ai/book1/20.41.

Figure 20.42: Illustration of the effect of changing the perplexity parameter when t-SNE is applied to some 2d data. From [WVJ16]. See http://distill.pub/2016/misread-tsne for an animated version of these figures. Used with kind permission of Martin Wattenberg.

of neighbors.

Unfortunately, the results of t-SNE can be quite sensitive to the perplexity parameter, so it is wise to run the algorithm with many different values. This is illustrated in Fig. 20.42. The input data is 2d, so there is no distortion generating by mapping to a 2d latent space. If the perplexity is too small, the method tends to find structure within each cluster which is not truly present. At perplexity 30 (the default for scikit-learn), the clusters seem equi-distant in embedding space, even though some are closer than others in the data space. Many other caveats in interpreting t-SNE plots can be found in [WVJ16].

20.4.10.5 Computational issues

The naive implementation of t-SNE takes $O(N^2)$ time, as can be seen from the gradient term in Eq. (20.138). A faster version can be created by leveraging an analogy to N-body simulation in physics. In particular, the gradient requires computing the force of N points on each of N points. However, points that are far away can be grouped into clusters (computationally speaking), and their effective force can be approximated by a few representative points per cluster. We can then approximate the forces using the **Barnes-Hut algorithm** [BH86], which takes $O(N \log N)$ time, as proposed in [Maa14]. Unfortunately, this only works well for low dimensional embeddings, such as

$L = 2$.

20.4.10.6 UMAP

Various extensions of tSNE have been proposed, that try to improve its speed, the quality of the embedding space, or the ability to embed into more than 2 dimensions.

One popular recent extension is called **UMAP** (which stands for "Uniform Manifold Approximation and Projection"), was proposed in [MHM18]. At a high level, this is similar to tSNE, but it tends to preserve global structure better, and it is much faster. This makes it easier to try multiple values of the hyperparameters. For an interactive tutorial on UMAP, and a comparison to tSNE, see [CP19].

20.5 Word embeddings

Words are categorical random variables, so their corresponding one-hot vector representations are sparse. The problem with this binary representation is that semantically similar words may have very different vector representations. For example, the pair of related words "man" and "woman" will be Hamming distance 1 apart, as will the pair of unrelated words "man" and "banana".

The standard way to solve this problem is to use **word embeddings**, in which we map each sparse one-hot vector, $\mathbf{s}_{n,t} \in \{0,1\}^M$, representing the t'th word in document n, to a lower-dimensional dense vector, $\mathbf{z}_{n,t} \in \mathbb{R}^D$, such that semantically similar words are placed close by. This can significantly help with data sparsity. There are many ways to learn such embeddings, as we discuss below.

Before discussing methods, we have to define what we mean by "semantically similar" words. We will assume that two words are semantically similar if they occur in similar contexts. This is known as the **distributional hypothesis** [Har54], which is often summarized by the phase (originally from [Fir57]) "a word is characterized by the company it keeps". Thus the methods we discuss will all learn a mapping from a word's context to an embedding vector for that word.

20.5.1 Latent semantic analysis / indexing

In this section, we discuss a simple way to learn word embeddings based on singular value decomposition (Sec. 7.5) of a term-frequency count matrix.

20.5.1.1 Latent semantic indexing (LSI)

Let C_{ij} be the number of times "term" i occurs in "context" j. The definition of what we mean by "term" is application-specific. In English, we often take it to be the set of unique tokens that are separated by punctuation or whitespace; for simplicity, we will call these "words". However, we may preprocess the text data to remove very frequent or infrequent words, or perform other kinds of preprocessing. as we discuss in Sec. 1.5.4.1.

The definition of what we mean by "context" is also application-specific. In this section, we count how many times word i occurs in each document $j \in \{1, \ldots, N\}$ from a set or **corpus** of documents; the resulting matrx $\mathbf{C}$ is called a **term-document frequency matrix**, as in Fig. 1.16. (Sometimes we apply the TF-IDF transformation to the counts, as discussed in Sec. 1.5.4.2.)

Let $\mathbf{C} \in \mathbb{R}^{M \times N}$ be the count matrix, and let $\hat{\mathbf{C}}$ be the rank K approximation that minimizes the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

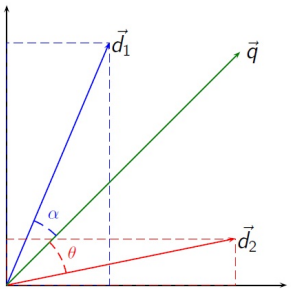


Figure 20.43: Illustration of the cosine similarity between a query vector $\mathbf{q}$ *and two document vectors* $\mathbf{d}_1$ *and* $\mathbf{d}_2$*. Since angle* α *is less than angle* θ*, we see that the query is more similar to document 1. From* https://en.wikipedia.org/wiki/Vector_space_model*. Used with kind permission of Wikipedia author Riclas.*

following loss:

$$\mathcal{L}(\hat{\mathbf{C}}) = ||\mathbf{C} - \hat{\mathbf{C}}||_F = \sum_{ij}(C_{ij} - \hat{C}_{ij})^2 \tag{20.139}$$

One can show that the minimizer of this is given by the rank K truncated SVD approximation, $\hat{\mathbf{C}} = \mathbf{USV}$. This means we can represent each c_{ij} as a bilinear product:

$$c_{ij} \approx \sum_{k=1}^{K} u_{ik} s_k v_{jk} \tag{20.140}$$

We define $\mathbf{u}_i$ to be the embedding for word i, and $\mathbf{s} \odot \mathbf{v}_j$ to be the embedding for context j.

We can use these embeddings for **document retrieval**. The idea is to compute an embedding for the query words using $\mathbf{u}_i$, and to compare this to the embedding of all the documents or contexts $\mathbf{v}_j$. This is known as **latent semantic indexing** or **LSI** [Dee+90].

In more detail, suppose the query is a bag of words $w_1, \ldots, w_B$; we represent this by the vector $\mathbf{q} = \frac{1}{B}\sum_{b=1}^{B} \mathbf{u}_{w_b}$, where $\mathbf{u}_{w_b}$ is the embedding for word w_b. Let document j be represented by $\mathbf{v}_j$. We then rank documents by the **cosine similarity** between the query vector and document, defined by

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}^\top \mathbf{d}}{||\mathbf{q}||\,||\mathbf{d}||} \tag{20.141}$$

where $||\mathbf{q}|| = \sqrt{\sum_i q_i^2}$ is the ℓ_2-norm of $\mathbf{q}$. This measures the angles between the two vectors, as shown in Fig. 20.43. Note that if the vectors are unit norm, cosine similarity is the same as inner product; it is also equal to the squared Euclidean distance, up to a change of sign and an irrelevant additive constant:

$$||\mathbf{q} - \mathbf{d}||^2 = (\mathbf{q} - \mathbf{d})^\top (\mathbf{q} - \mathbf{d}) = \mathbf{q}^\top \mathbf{q} + \mathbf{d}^\top \mathbf{d} - 2\mathbf{q}^\top \mathbf{d} = 2(1 - \text{sim}(\mathbf{q}, \mathbf{d})) \tag{20.142}$$

20.5.1.2 Latent semantic analysis (LSA)

Now suppose we define context more generally to be some local neighborhood of words $j \in \{1, \ldots, M^h\}$, where h is the window size. Thus C_{ij} is how many times word i occurs in a neighborhood of type j. We can compute the SVD of this matrix as before, to get $c_{ij} \approx \sum_{k=1}^{K} u_{ik} s_k v_{jk}$. We define $\mathbf{u}_i$ to be the embedding for word i, and $\mathbf{s} \odot \mathbf{v}_j$ to be the embedding for context j. This is known as **latent semantic analysis** or **LSA** [Dee+90].

For example, suppose we compute $\mathbf{C}$ on the British National Corpus.[8] For each word, let us retrieve the K nearest neighbors in embedding space ranked by cosine similarity (i.e., normalized inner product). If the query word is "dog", and we use $h = 2$ or $h = 30$, the nearest neighbors are as follows:

```
h=2: cat, horse, fox, pet, rabbit, pig, animal, mongrel, sheep, pigeon
h=30: kennel, puppy, pet, bitch, terrier, rottwiler, canine, cat, to bark
```

The 2-word context window is more sensitive to syntax, while the 30-word window is more sensitive to semantics. The "optimal" value of context size h depends on the application.

20.5.1.3 PMI

In practice LSA (and other similar methods) give much better results if we replace the raw counts C_{ij} with **pointwise mutual information** (**PMI**) [CH90], defined as

$$\mathbb{PMI}(i,j) = \log \frac{p(i,j)}{p(i)p(j)} \tag{20.143}$$

If word i is strongly associated with context j, we will have $\mathbb{PMI}(i,j) > 0$. If the PMI is negative, it means i and j co-occur less often that if they were independent; however, such negative correlations can be unreliable, so it is common to use the **positive PMI**: $\mathbb{PPMI}(i,j) = \max(\mathbb{PMI}(i,j), 0)$. In [BL07], they show that SVD applied to the PPMI matrix results in word embeddings that perform well on a many tasks related to word meaning. See Sec. 20.5.5 for a theoretical model that explains this empirical performance.

20.5.2 Word2vec

In this section, we discuss the popular **word2vec** model from [Mik+13a; Mik+13b], which are "shallow" neural nets for predicting a word given its context. In Sec. 20.5.5, we will discuss the connections with SVD of the PMI matrix.

There are two versions of the word2vec model. The first is called CBOW, which stands for "continuous bag of words". The second is called skipgram. We discuss both of these below.

8. This example is taken from [Eis19, p312].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

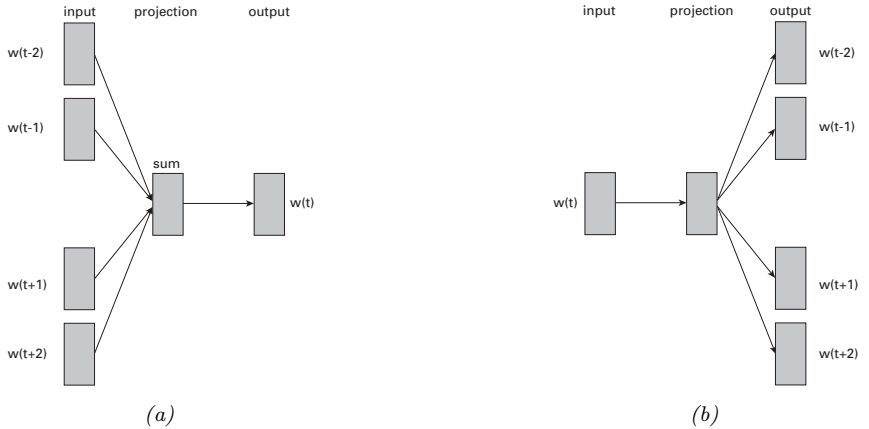


Figure 20.44: Illustration of word2vec model with window size of 2. (a) CBOW version. (b) Skip-gram version.

20.5.2.1 Word2vec CBOW model

In the continuous bag of words (**CBOW**) model (see Fig. 20.44(a)), the log likelihood of a sequence of words is computed using the following model:

$$\log p(\mathbf{w}) = \sum_{t=1}^{T} \log p(w_t|\mathbf{w}_{t-m:t+m}) = \sum_{t=1}^{T} \log \frac{\exp(\mathbf{v}_{w_t}^\top \overline{\mathbf{v}}_t)}{\sum_{w'} \exp(\mathbf{v}_{w'}^\top \overline{\mathbf{v}}_t)} \tag{20.144}$$

$$= \sum_{t=1}^{T} \mathbf{v}_{w_t}^\top \overline{\mathbf{v}}_t - \log \sum_{i \in \mathcal{V}} \exp(\mathbf{v}_i^\top \overline{\mathbf{v}}_t) \tag{20.145}$$

where $\mathbf{v}_{w_t}$ is the vector for the word at location w_t, $\mathcal{V}$ is the set of all words, m is the context size, and

$$\overline{\mathbf{v}}_t = \frac{1}{2m} \sum_{h=1}^{m} (\mathbf{v}_{w_{t+h}} + \mathbf{v}_{w_{t-h}}) \tag{20.146}$$

is the average of the word vectors in the window around word w_t. Thus we try to predict each word given its context. The model is called CBOW because it uses a bag of words assumption for the context, and represents each word by a continuous embedding.

20.5.2.2 Word2vec Skip-gram model

In CBOW, each word is predicted from its context. A variant of this is to predict the context (surrounding words) given each word. This yields the following objective:

$$-\log p(\mathbf{w}) = -\sum_{t=1}^{T}\left[\sum_{j=1}^{m}\log p(w_{t-j}|w_t) + \log p(w_{t+j}|w_t)\right] \tag{20.147}$$

$$= -\sum_{t=1}^{T}\sum_{-m\le j\le m, j\neq 0}\log p(w_{t+j}|w_t) \tag{20.148}$$

where m is the context window length. We define the log probability of some other context word w_o given the central word w_c to be

$$\log p(w_o|w_c) = \mathbf{u}_o^\top \mathbf{v}_c - \log\left(\sum_{i\in\mathcal{V}}\exp(\mathbf{u}_i^\top \mathbf{v}_c)\right) \tag{20.149}$$

where $\mathcal{V}$ is the vocabulary. Here $\mathbf{u}_i$ is the embedding of a word if used as context, and $\mathbf{v}_i$ is the embedding of a word if used as a central (target) word to be predicted. This model is known as the **skipgram model**. See Fig. 20.44(b) for an illustration.

20.5.2.3 Negative sampling

Computing the conditional probability of each word using Eq. (20.149) is expensive, due to the need to normalize over all possible words in the vocabulary. This makes it slow to compute the log likelihood and its gradient, for both the CBOW and skip-gram models.

In [Mik+13b], they propose a fast approximation, called **skip-gram with negative sampling** (**SGNS**). The basic idea is to create a set of $K+1$ context words for each central word w_t, and to label the one that actually occurs as positive, and the rest as negative. The negative words are called noise words, and can be sampled from a reweighted unigram distribution, $p(w) \propto \text{freq}(w)^{3/4}$, which has the effect of redistributing probability mass from common to rare words. The conditional probability is now approximated by

$$p(w_{t+j}|w_t) = p(D=1|w_t, w_{t+j})\prod_{k=1}^{K} p(D=0|w_t, w_k) \tag{20.150}$$

where $w_k \sim p(w)$ are noise words, and $D=1$ is the event that the word pair actually occurs in the data, and $D=0$ is the event that the word pair does not occur. The binary probabilities are given by

$$p(D=1|w_t, w_{t+j}) = \sigma(\mathbf{u}_{w_{t+j}}^\top \mathbf{v}_{w_t}) \tag{20.151}$$

$$p(D=0|w_t, w_k) = 1 - \sigma(\mathbf{u}_{w_k}^\top \mathbf{v}_{w_t}) \tag{20.152}$$

To train this model, we just need to compute the contexts for each central word, and a set of negative noise words. We associate a label of 1 with the context words, and a label of 0 with the noise words. We can then compute the log probability of the data, and optimize the embedding vectors $\mathbf{u}_i$ and $\mathbf{v}_i$ for each word using SGD. See code.probml.ai/book1/skipgram_torch for some sample code.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.5.3 GloVE

A popular alternative to Skipgram is the **GloVe** model of [PSM14a]. (GloVe stands for "global vectors for word representation".) This method uses a simpler objective, which is much faster to optimize.

To explain the method, recall that in the skipgram model, the predicted conditional probability of word j occuring in the context window of central word i as

$$q_{ij} = \frac{\exp(\mathbf{u}_j^\top \mathbf{v}_i)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_k^\top \mathbf{v}_i)} \tag{20.153}$$

Let x_{ij} be the number of times word j occurs in any context window of i. (Note that if word i occurs in the window of j, then j will occur in the window of i, so we have $x_{ij} = x_{ji}$.) Then we we can rewrite Eq. (20.148) as follows:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} x_{ij} \log q_{ij} \tag{20.154}$$

If we define $p_{ij} = x_{ij}/x_i$ to be the empirical probability of word j occuring in the context window of central word i, we can rewrite the skipgram loss as a cross entropy loss:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} x_i \sum_{j \in \mathcal{V}} p_{ij} \log q_{ij} \tag{20.155}$$

The problem with this objective is that computing q_{ij} is expensive, due to the need to normalize over all words. In GloVe, we work with unnormalized probabilities, $p'_{ij} = x_{ij}$ and $q'_{ij} = \exp(\mathbf{u}_j^\top \mathbf{v}_i + b_i + c_j)$, where b_i and c_j are bias terms to capture marginal probailities. In addition, we minimize the squared loss, $(\log p'_{ij} - \log q'_{ij})^2$, which is more robust to errors in estimating small probablities than log loss. Finally, we upweight rare words for which $x_{ij} < c$, where $c = 100$, by weighting the squared errors by $h(x_{ij})$, where $h(x) = (x/c)^{0.75}$ if $x < c$, and $h(x) = 1$ otherwise. This gives the final GloVe objective:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} h(x_{ij})(\mathbf{u}_j^\top \mathbf{v}_i + b_i + c_j - \log x_{ij})^2 \tag{20.156}$$

We can precompute x_{ij} offline, and then optimize the above objective using SGD. After training, we define the embedding of word i to be the average of $\mathbf{v}_i$ and $\mathbf{u}_i$.

Empirically GloVe gives similar results to skigram, but it is faster to train. See Sec. 20.5.5 for a theoretical model that explains why these methods work.

20.5.4 Word analogies

One of the most remarkable properties of word embeddings produced by word2vec, GloVe, and other similar methods is that the learned vector space seems to capture relational semantics in terms of simple vector addition. For example, consider the **word analogy problem** "man is to woman as king is to queen", often written as man:woman::king:queen. Suppose we are given the words a=man, b=woman, c=king; how do we find d=queen? Let $\boldsymbol{\delta} = \mathbf{v}_b - \mathbf{v}_a$ be the vector representing

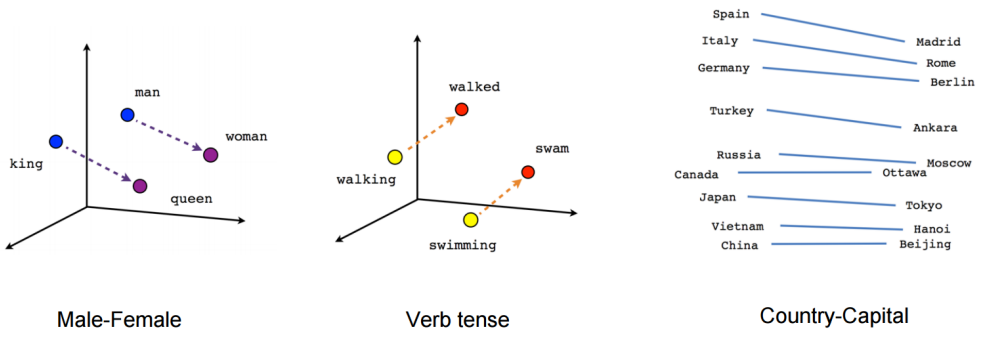


Figure 20.45: Visualization of arithmetic operations in word2vec embedding space. From https://www.tensorflow.org/tutorials/representation/word2vec.

the concept of "converting the gender from male to female". Intuitively we can find word d by computing $\mathbf{v}_d = \mathbf{c} + \boldsymbol{\delta}$, and then finding the closest word in the vocabulary to $\mathbf{v}_d$. See Fig. 20.45 for an illustration of this process, and code.probml.ai/book1/word_analogies_torch for some code.

In [PSM14a], they conjecture that $a : b :: c : d$ holds iff for every word w in the vocabulary, we have

$$\frac{p(w|a)}{p(w|b)} \approx \frac{p(w|c)}{p(w|d)} \tag{20.157}$$

In [Aro+16], they show that this follows from the RAND-WALK modeling assumptions in Sec. 20.5.5. See also [AH19; EDH19] for other explanations of why word analogies work, based on different modeling assumptions.

20.5.5 RAND-WALK model of word embeddings

Word embeddings significantly improve the performance of various kinds of NLP models compared to using one-hot encodings for words. It is natural to wonder why the above word embeddings work so well. In this section, we give a simple generative model for text documents that explains this phenomenon, based on [Aro+16].

Consider a sequence of words $w_1, \ldots, w_T$. We assume each word is generated by a latent context or discourse vector $\mathbf{z}_t \in \mathbb{R}^D$ using the following **log bilinear language model**, similar to [MH07]:

$$p(w_t = w|\mathbf{z}_t) = \frac{\exp(\mathbf{z}_t^\top \mathbf{v}_w)}{\sum_{w'} \exp(\mathbf{z}_t^\top \mathbf{v}_{w'})} = \frac{\exp(\mathbf{z}_t^\top \mathbf{v}_w)}{Z(\mathbf{z}_t)} \tag{20.158}$$

where $\mathbf{v}_w \in \mathbb{R}^D$ is the embedding for word w, and $Z(\mathbf{z}_t)$ is the partition function. We assume $D < M$, the number of words in the vocabulary.

Let us further assume the prior for the word embeddings $\mathbf{v}_w$ is an isotropic Gaussian, and that the latent topic $\mathbf{z}_t$ undergoes a slow Gaussian random walk. (This is therefore called the **RAND-WALK** model.) Under this model, one can show that $Z(\mathbf{z}_t)$ is approximately equal to a fixed constant, Z, independent of the context. This is known as the **self-normalization property** of log-linear

models [AK15]. Furthermore, one can show that the pointwise mutual information of predictions from the model is given by

$$\mathbb{PMI}(w, w') = \frac{p(w, w')}{p(w)p(w')} \approx \frac{\mathbf{v}_w^\top \mathbf{v}_{w'}}{D} \tag{20.159}$$

We can therefore fit the RAND-WALK model by matching the model's predicted values for PMI with the empirical values, i.e., we minimize

$$\mathcal{L} = \sum_{w,w'} X_{w,w'} (\mathbb{PMI}(w, w') - \mathbf{v}_w^\top \mathbf{v}_{w'})^2 \tag{20.160}$$

where $X_{w,w'}$ is the number of times w and w' occur next to each other. This objective can be seen as a frequency-weighted version of the SVD loss in Eq. (20.139). (See [LG14] for more connections between word embeddings and SVD.)

Furthermore, some additional approximations can be used to show that the NLL for the RAND-WALK model is equivalent to the CBOW and SGNS word2vec objectives. We can also derive the objective for GloVE from this approach.

20.5.6 Contextual word embeddings

Consider the sentences "I was eating an apple" and "I bought a new phone from Apple". The meaning of the word "apple" is different in both cases, but a fixed word embedding, of the type discussed in Sec. 20.5, would not be able to capture this. In Sec. 15.7, we discuss **contextual word embeddings**, where the embedding of a word is a function of all the words in its context (usually a sentence). This can give much improved results, and is currently the standard approach to representing natural language data, as a pre-processing step before doing transfer learning (see Sec. 19.2).

20.6 Exercises

Exercise 20.1 [EM for FA]

Derive the EM updates for the factor analysis model. For simplicity, you can optionally assume $\boldsymbol{\mu} = \mathbf{0}$ is fixed.

Exercise 20.2 [EM for mixFA]

Derive the EM updates for a mixture of factor analysers.

Exercise 20.3 [Deriving the second principal component]

a. Let

$$J(\mathbf{v}_2, \mathbf{z}_2) = \frac{1}{n} \sum_{i=1}^{n} (\mathbf{x}_i - z_{i1}\mathbf{v}_1 - z_{i2}\mathbf{v}_2)^T (\mathbf{x}_i - z_{i1}\mathbf{v}_1 - z_{i2}\mathbf{v}_2) \tag{20.161}$$

Show that $\frac{\partial J}{\partial \mathbf{z}_2} = 0$ yields $z_{i2} = \mathbf{v}_2^T \mathbf{x}_i$.

b. Show that the value of $\mathbf{v}_2$ that minimizes

$$\tilde{J}(\mathbf{v}_2) = -\mathbf{v}_2^T \mathbf{C} \mathbf{v}_2 + \lambda_2(\mathbf{v}_2^T \mathbf{v}_2 - 1) + \lambda_{12}(\mathbf{v}_2^T \mathbf{v}_1 - 0) \tag{20.162}$$

is given by the eigenvector of $\mathbf{C}$ with the second largest eigenvalue. Hint: recall that $\mathbf{C}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ and $\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$.

Exercise 20.4 [Deriving the residual error for PCA]

a. Prove that

$$||\mathbf{x}_i - \sum_{j=1}^{K} z_{ij}\mathbf{v}_j||^2 = \mathbf{x}_i^T\mathbf{x}_i - \sum_{j=1}^{K} \mathbf{v}_j^T\mathbf{x}_i\mathbf{x}_i^T\mathbf{v}_j \tag{20.163}$$

Hint: first consider the case $K = 2$. Use the fact that $\mathbf{v}_j^T\mathbf{v}_j = 1$ and $\mathbf{v}_j^T\mathbf{v}_k = 0$ for $k \neq j$. Also, recall $z_{ij} = \mathbf{x}_i^T\mathbf{v}_j$.

b. Now show that

$$J_K \triangleq \frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{x}_i^T\mathbf{x}_i - \sum_{j=1}^{K}\mathbf{v}_j^T\mathbf{x}_i\mathbf{x}_i^T\mathbf{v}_j\right) = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i^T\mathbf{x}_i - \sum_{j=1}^{K}\lambda_j \tag{20.164}$$

Hint: recall $\mathbf{v}_j^T\mathbf{C}\mathbf{v}_j = \lambda_j\mathbf{v}_j^T\mathbf{v}_j = \lambda_j$.

c. If $K = d$ there is no truncation, so $J_d = 0$. Use this to show that the error from only using $K < d$ terms is given by

$$J_K = \sum_{j=K+1}^{d}\lambda_j \tag{20.165}$$

Hint: partition the sum $\sum_{j=1}^{d}\lambda_j$ into $\sum_{j=1}^{K}\lambda_j$ and $\sum_{j=K+1}^{d}\lambda_j$.

Exercise 20.5 [PCA via successive deflation]

Let $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k$ be the first k eigenvectors with largest eigenvalues of $\mathbf{C} = \frac{1}{n}\mathbf{X}^T\mathbf{X}$, i.e., the principal basis vectors. These satisfy

$$\mathbf{v}_j^T\mathbf{v}_k = \begin{cases} 0 & \text{if } j \neq k \\ 1 & \text{if } j = k \end{cases} \tag{20.166}$$

We will construct a method for finding the $\mathbf{v}_j$ sequentially.

As we showed in class, $\mathbf{v}_1$ is the first principal eigenvector of $\mathbf{C}$, and satisfies $\mathbf{C}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$. Now define $\tilde{\mathbf{x}}_i$ as the orthogonal projection of $\mathbf{x}_i$ onto the space orthogonal to $\mathbf{v}_1$:

$$\tilde{\mathbf{x}}_i = \mathbf{P}_{\perp\mathbf{v}_1}\,\mathbf{x}_i = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)\mathbf{x}_i \tag{20.167}$$

Define $\tilde{\mathbf{X}} = [\tilde{\mathbf{x}}_1; ...; \tilde{\mathbf{x}}_n]$ as the **deflated matrix** of rank $d-1$, which is obtained by removing from the d dimensional data the component that lies in the direction of the first principal direction:

$$\tilde{\mathbf{X}} = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)^T\mathbf{X} = (\mathbf{I} - \mathbf{v}_1\mathbf{v}_1^T)\mathbf{X} \tag{20.168}$$

a. Using the facts that $\mathbf{X}^T\mathbf{X}\mathbf{v}_1 = n\lambda_1\mathbf{v}_1$ (and hence $\mathbf{v}_1^T\mathbf{X}^T\mathbf{X} = n\lambda_1\mathbf{v}_1^T$) and $\mathbf{v}_1^T\mathbf{v}_1 = 1$, show that the covariance of the deflated matrix is given by

$$\tilde{\mathbf{C}} \triangleq \frac{1}{n}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} = \frac{1}{n}\mathbf{X}^T\mathbf{X} - \lambda_1\mathbf{v}_1\mathbf{v}_1^T \tag{20.169}$$

b. Let $\mathbf{u}$ be the principal eigenvector of $\tilde{\mathbf{C}}$. Explain why $\mathbf{u} = \mathbf{v}_2$. (You may assume $\mathbf{u}$ is unit norm.)

c. Suppose we have a simple method for finding the leading eigenvector and eigenvalue of a pd matrix, denoted by $[\lambda, \mathbf{u}] = f(\mathbf{C})$. Write some pseudo code for finding the first K principal basis vectors of $\mathbf{X}$ that only uses the special f function and simple vector arithmetic, i.e., your code should not use SVD or the `eig` function. Hint: this should be a simple iterative routine that takes 2–3 lines to write. The input is $\mathbf{C}$, K and the function f, the output should be $\mathbf{v}_j$ and λ_j for $j = 1 : K$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 20.6 [PPCA variance terms]

Recall that in the PPCA model, $\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$. We will show that this model correctly captures the variance of the data along the principal axes, and approximates the variance in all the remaining directions with a single average value σ^2.

Consider the variance of the predictive distribution $p(\mathbf{x})$ along some direction specified by the unit vector $\mathbf{v}$, where $\mathbf{v}^T\mathbf{v} = 1$, which is given by $\mathbf{v}^T C\mathbf{v}$.

a. First suppose $\mathbf{v}$ is orthogonal to the principal subspace. and hence $\mathbf{v}^T\mathbf{U} = \mathbf{0}$. Show that $\mathbf{v}^T\mathbf{C}\mathbf{v} = \sigma^2$.
b. Now suppose $\mathbf{v}$ is parallel to the principal subspace. and hence $\mathbf{v} = \mathbf{u}_i$ for some eigenvector $\mathbf{u}_i$. Show that $\mathbf{v}^T\mathbf{C}\mathbf{v} = (\lambda_i - \sigma^2) + \sigma^2 = \lambda_i$.

Exercise 20.7 [Posterior inference in PPCA]

Derive $p(\mathbf{z}_n|\mathbf{x}_n)$ for the PPCA model.

Exercise 20.8 [Imputation in a FA model]

Derive an expression for $p(\mathbf{x}_h|\mathbf{x}_v, \boldsymbol{\theta})$ for a FA model, where $\mathbf{x} = (\mathbf{x}_h, \mathbf{x}_v)$ is a partition of the data vector.

Exercise 20.9 [Efficiently evaluating the PPCA density]

Derive an expression for $p(\mathbf{x}|\hat{\mathbf{W}}, \hat{\sigma}^2)$ for the PPCA model based on plugging in the MLEs and using the matrix inversion lemma.

21 Clustering

21.1 Introduction

Clustering is a very common form of unsupervised learning. There are two main kinds of methods. In the first approach, the input is a set of data samples $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$, where $\mathbf{x}_n \in \mathcal{X}$, where typically $\mathcal{X} = \mathbb{R}^D$. In the second approach, the input is an $N \times N$ pairwise disimilarity metric $D_{ij} \geq 0$. In both cases, the goal is to assign similar data points to the same cluster.

As is often the case with unsupervised learning, it is hard to evaluate the quality of a clustering algorithm. If we have labeled data for some of the data, we can use the similarity (or equality) between the labels of two data points as a metric for determining if the two inputs "should" be assigned to the same cluster or not. If we don't have labels, but the method is based on a generative model of the data, we can use log likelihood as a metric. We will see examples of both approaches below.

21.1.1 Evaluating the output of clustering methods

> The validation of clustering structures is the most difficult and frustrating part of cluster analysis. Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage. — Jain and Dubes [JD88]

Clustering is an unsupervised learning technique, so it is hard to evaluate the quality of the output of any given method [Kle02; LWG12]. If we use probabilistic models, we can always evaluate the likelihood of the data, but this has two drawbacks: first, it does not directly assess any clustering that is discovered by the model; and second, it does not apply to non-probabilistic methods. So now we discuss some performance measures not based on likelihood.

Intuitively, the goal of clustering is to assign points that are similar to the same cluster, and to ensure that points that are dissimilar are in different clusters. There are several ways of measuring these quantities e.g., see [JD88; KR90]. However, these internal criteria may be of limited use. An alternative is to rely on some external form of data with which to validate the method. For example, if we have labels for each object, then we can assume that objects with the same label are similar. We can then use the metrics we discuss below to quantify the quality of the clusters. (If we do not have labels, but we have a reference clustering, we can derive labels from that clustering.)

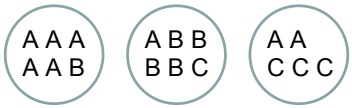


Figure 21.1: Three clusters with labeled objects inside.

21.1.1.1 Purity

Let N_{ij} be the number of objects in cluster i that belong to class j, and let $N_i = \sum_{j=1}^{C} N_{ij}$ be the total number of objects in cluster i. Define $p_{ij} = N_{ij}/N_i$; this is the empirical distribution over class labels for cluster i. We define the **purity** of a cluster as $p_i \triangleq \max_j p_{ij}$, and the overall purity of a clustering as

$$\text{purity} \triangleq \sum_i \frac{N_i}{N} p_i \tag{21.1}$$

For example, in Figure 21.1, we have that the purity is

$$\frac{6}{17}\frac{5}{6} + \frac{6}{17}\frac{4}{6} + \frac{5}{17}\frac{3}{5} = \frac{5+4+3}{17} = 0.71 \tag{21.2}$$

The purity ranges between 0 (bad) and 1 (good). However, we can trivially achieve a purity of 1 by putting each object into its own cluster, so this measure does not penalize for the number of clusters.

21.1.1.2 Rand index

Let $U = \{u_1, \ldots, u_R\}$ and $V = \{v_1, \ldots, V_C\}$ be two different partitions of the N data points. For example, U might be the estimated clustering and V is reference clustering derived from the class labels. Now define a 2×2 contingency table, containing the following numbers: TP is the number of pairs that are in the same cluster in both U and V (true positives); TN is the number of pairs that are in the different clusters in both U and V (true negatives); FN is the number of pairs that are in the different clusters in U but the same cluster in V (false negatives); and FP is the number of pairs that are in the same cluster in U but different clusters in V (false positives). A common summary statistic is the **Rand index**:

$$R \triangleq \frac{TP + TN}{TP + FP + FN + TN} \tag{21.3}$$

This can be interpreted as the fraction of clustering decisions that are correct. Clearly $0 \leq R \leq 1$.

For example, consider Figure 21.1, The three clusters contain 6, 6 and 5 points, so the number of "positives" (i.e., pairs of objects put in the same cluster, regardless of label) is

$$TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 40 \tag{21.4}$$

Of these, the number of true positives is given by

$$TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 20 \tag{21.5}$$

where the last two terms come from cluster 3: there are $\binom{3}{2}$ pairs labeled C and $\binom{2}{2}$ pairs labeled A. So $FP = 40 - 20 = 20$. Similarly, one can show $FN = 24$ and $TN = 72$. So the Rand index is $(20 + 72)/(20 + 20 + 24 + 72) = 0.68$.

The Rand index only achieves its lower bound of 0 if $TP = TN = 0$, which is a rare event. One can define an **adjusted Rand index** [HA85] as follows:

$$AR \triangleq \frac{\text{index} - \text{expected index}}{\text{max index} - \text{expected index}} \tag{21.6}$$

Here the model of randomness is based on using the generalized hyper-geometric distribution, i.e., the two partitions are picked at random subject to having the original number of classes and objects in each, and then the expected value of $TP + TN$ is computed. This model can be used to compute the statistical significance of the Rand index.

The Rand index weights false positives and false negatives equally. Various other summary statistics for binary decision problems, such as the F-score (Section 5.1.4), can also be used.

21.1.1.3 Mutual information

Another way to measure cluster quality is to compute the mutual information between two candidate partitions U and V, as proposed in [VD99]. To do this, let $p_{UV}(i,j) = \frac{|u_i \cap v_j|}{N}$ be the probability that a randomly chosen object belongs to cluster u_i in U and v_j in V. Also, let $p_U(i) = |u_i|/N$ be the be the probability that a randomly chosen object belongs to cluster u_i in U; define $p_V(j) = |v_j|/N$ similarly. Then we have

$$\mathbb{I}(U,V) = \sum_{i=1}^{R}\sum_{j=1}^{C} p_{UV}(i,j) \log \frac{p_{UV}(i,j)}{p_U(i)p_V(j)} \tag{21.7}$$

This lies between 0 and $\min\{\mathbb{H}(U), \mathbb{H}(V)\}$. Unfortunately, the maximum value can be achieved by using lots of small clusters, which have low entropy. To compensate for this, we can use the **normalized mutual information**,

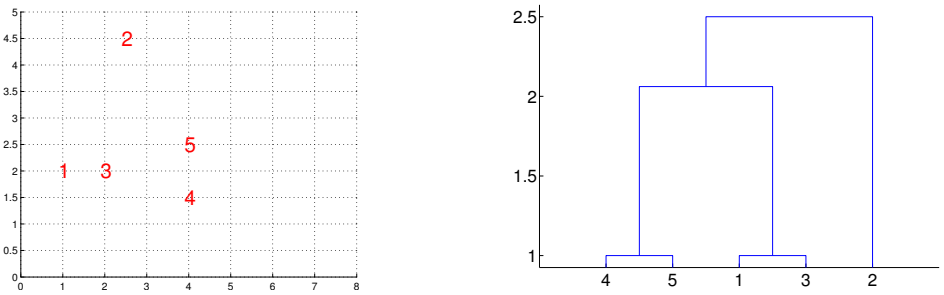
$$NMI(U,V) \triangleq \frac{\mathbb{I}(U,V)}{(\mathbb{H}(U) + \mathbb{H}(V))/2} \tag{21.8}$$

This lies between 0 and 1. A version of this that is adjusted for chance (under a particular random data model) is described in [VEB09]. Another variant, called **variation of information**, is described in [Mei05].

21.2 Hierarchical agglomerative clustering

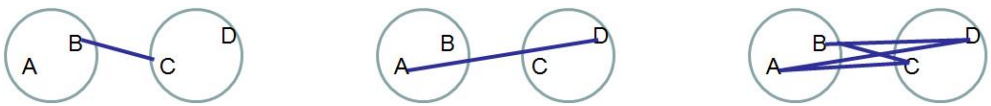
A common form of clustering is known as **hierarchical agglomerative clustering** or **HAC**. The input to the algorithm is an $N \times N$ dissimilarity matrix $D_{ij} \geq 0$, and the output is a tree structure in which groups i and j with small disimilarity are grouped together in a hierarchical fashion.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 21.2: (a) An example of single link clustering using city block distance. Pairs (1,3) and (4,5) are both distance 1 apart, so get merged first. (b) The resulting dendrogram. Adapted from Figure 7.5 of [Alp04]. Generated by code at figures.probml.ai/book1/21.2.



(a) (b) (c)

Figure 21.3: Illustration of (a) Single linkage. (b) Complete linkage. (c) Average linkage.

For example, consider the set of 5 inputs points in Fig. 21.2(a), $\mathbf{x}_n \in \mathbb{R}^2$. We will use **city block distance** between the points to define the dissimilarity, i.e.,

$$d_{ij} = \sum_{k=1}^{2} |x_{ik} - x_{jk}| \tag{21.9}$$

We start with a tree with N leaves, each corresponding to a cluster with a single data point. Next we compute the pair of points that are closest, and merge them. We see that (1,3) and (4,5) are both distance 1 apart, so they get merged first. We then measure the dissimilarity between the sets $\{1, 3\}$, $\{4, 5\}$ and $\{2\}$ using some measure (details below), and group them, and repeat. The result is a binary tree known as a **dendogram**, as shown in Fig. 21.2(b). By cutting this tree at different heights, we can induce a different number of (nested) clusters. We give more details below.

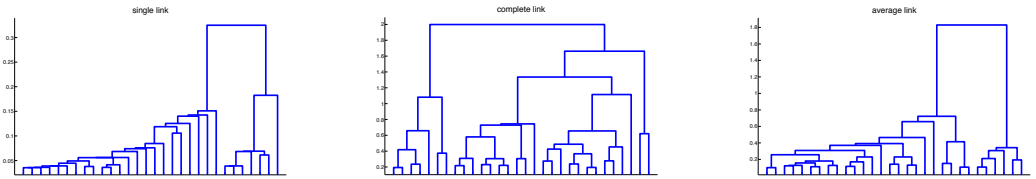
21.2.1 The algorithm

Agglomerative clustering starts with N groups, each initially containing one object, and then at each step it merges the two most similar groups until there is a single group, containing all the data. See Algorithm 13 for the pseudocode. Since picking the two most similar clusters to merge takes $O(N^2)$ time, and there are $O(N)$ steps in the algorithm, the total running time is $O(N^3)$. However, by using a priority queue, this can be reduced to $O(N^2 \log N)$ (see e.g., [MRS08, ch. 17] for details).

There are actually three variants of agglomerative clustering, depending on how we define the dissimilarity between groups of objects. We give the details below.

Algorithm 13: Agglomerative clustering

Initialize clusters as singletons: **for** $i \leftarrow 1$ **to** n **do** $C_i \leftarrow \{i\}$;
;
Initialize set of clusters available for merging: $S \leftarrow \{1, \ldots, n\}$; **repeat**
    Pick 2 most similar clusters to merge: $(j, k) \leftarrow \arg\min_{j,k \in S} d_{j,k}$;
    Create new cluster $C_\ell \leftarrow C_j \cup C_k$;
    Mark j and k as unavailable: $S \leftarrow S \setminus \{j, k\}$;
    if $C_\ell \neq \{1, \ldots, n\}$ **then**
        Mark ℓ as available, $S \leftarrow S \cup \{\ell\}$;
    foreach $i \in S$ **do**
        Update dissimilarity matrix $d(i, \ell)$;
until *no more clusters are available for merging*;



(a) *(b)* *(c)*

Figure 21.4: Hierarchical clustering of yeast gene expression data. (a) Single linkage. (b) Complete linkage. (c) Average linkage. Generated by code at figures.probml.ai/book1/21.4.

21.2.1.1 Single link

In **single link clustering**, also called **nearest neighbor clustering**, the distance between two groups G and H is defined as the distance between the two closest members of each group:

$$d_{SL}(G, H) = \min_{i \in G, i' \in H} d_{i,i'} \tag{21.10}$$

See Figure 21.3(a).

The tree built using single link clustering is a minimum spanning tree of the data, which is a tree that connects all the objects in a way that minimizes the sum of the edge weights (distances). To see this, note that when we merge two clusters, we connect together the two closest members of the clusters; this adds an edge between the corresponding nodes, and this is guaranteed to be the "lightest weight" edge joining these two clusters. And once two clusters have been merged, they will never be considered again, so we cannot create cycles. As a consequence of this, we can actually implement single link clustering in $O(N^2)$ time, whereas the other variants take $O(N^3)$ time.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

21.2.1.2 Complete link

In **complete link clustering**, also called **furthest neighbor clustering**, the distance between two groups is defined as the distance between the two most distant pairs:

$$d_{CL}(G, H) = \max_{i \in G, i' \in H} d_{i,i'} \tag{21.11}$$

See Figure 21.3(b).

Single linkage only requires that a single pair of objects be close for the two groups to be considered close together, regardless of the similarity of the other members of the group. Thus clusters can be formed that violate the **compactness** property, which says that all the observations within a group should be similar to each other. In particular if we define the **diameter** of a group as the largest dissimilarity of its members, $d_G = \max_{i \in G, i' \in G} d_{i,i'}$, then we can see that single linkage can produce clusters with large diameters. Complete linkage represents the opposite extreme: two groups are considered close only if all of the observations in their union are relatively similar. This will tend to produce clusterings with small diameter, i.e., compact clusters. (Compare Fig. 21.4(a) with Fig. 21.4(b).)

21.2.1.3 Average link

In practice, the preferred method is **average link clustering**, which measures the average distance between all pairs:

$$d_{avg}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{i,i'} \tag{21.12}$$

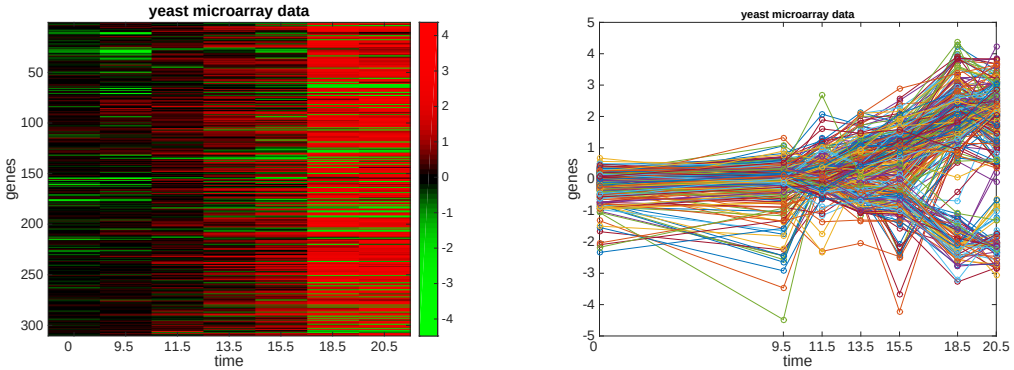
where n_G and n_H are the number of elements in groups G and H. See Figure 21.3(c).

Average link clustering represents a compromise between single and complete link clustering. It tends to produce relatively compact clusters that are relatively far apart. (See Fig. 21.4(c).) However, since it involves averaging of the $d_{i,i'}$'s, any change to the measurement scale can change the result. In contrast, single linkage and complete linkage are invariant to monotonic transformations of $d_{i,i'}$, since they leave the relative ordering the same.

21.2.2 Example

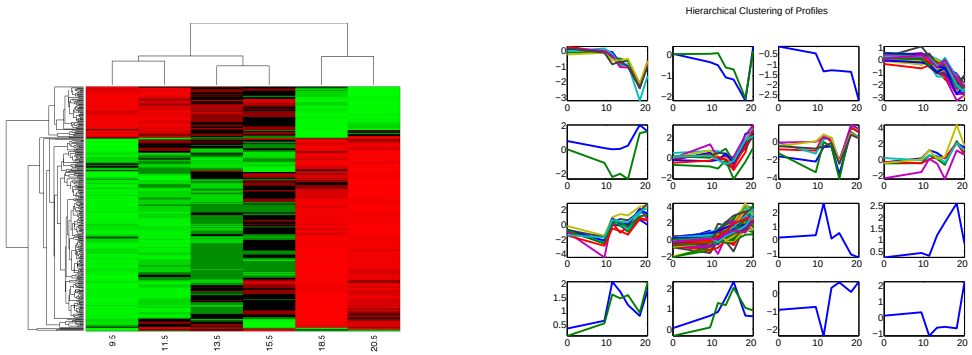
Syppose we have a set of time series measurements of the expression levels for $N = 300$ genes at $T = 7$ points. Thus each data sample is a vector $\mathbf{x}_n \in \mathbb{R}^7$. See Fig. 21.5 for a visualization of the data. We see that there are several kinds of genes, such as those whose expression level goes up monotonically over time (in response to a given stimulus), those whose expression level goes down monotonically, and those with more complex response patterns.

Suppose we use Euclidean distance to compute a pairwise dissimilarity matrix, $\mathbf{D} \in \mathbb{R}^{300 \times 300}$, and apply HAC using average linkage. We get the dendogram in Fig. 21.6(a). If we cut the tree at a certain height, we get the 16 clusters shown in Figure 21.6(b). The time series assigned to each cluster do indeed "look like" each other.



(a) (b)

Figure 21.5: (a) Some yeast gene expression data plotted as a heat map. (b) Same data plotted as a time series. Generated by code at figures.probml.ai/book1/21.5.



(a) (b)

Figure 21.6: Hierarchical clustering applied to the yeast gene expression data. (a) The rows are permuted according to a hierarchical clustering scheme (average link agglomerative clustering), in order to bring similar rows close together. (b) 16 clusters induced by cutting the average linkage tree at a certain height. Generated by code at figures.probml.ai/book1/21.6.

21.3 K means clustering

There are several problems with hierarchical agglomerative clustering (Sec. 21.2). First, it takes $O(N^3)$ time (for the average link method), making it hard to apply to big datasets. Second, it assumes that a dissimilarity matrix has already been computed, whereas the notion of "similarity" is often unclear and needs to be learned. Third, it is just an algorithm, not a model, and so it is hard to evaluate how good it is. That is, there is no clear objective that it is optimizing.

In this section, we discuss the **K-means algorithm** [Mac67; Llo82], which addresses these issues.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

In particular, it runs in $O(N)$ time, it computes similarity in terms of Euclidean distance to learned cluster centers $\boldsymbol{\mu}_k \in \mathbb{R}^D$, and it optimizes a well-defined cost function, as we will see.

21.3.1 The algorithm

We assume there are K cluster centers $\boldsymbol{\mu}_k \in \mathbb{R}^D$, so we can cluster the data by assigning each data point $\mathbf{x}_n \in \mathbb{R}^D$ to it closest center:

$$z_n^* = \arg\min_k ||\mathbf{x}_n - \boldsymbol{\mu}_k||_2^2 \tag{21.13}$$

Of course, we don't know the cluster centers, but we can estimate them by computing the average value of all points assigned to them:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n:z_n=k} \mathbf{x}_n \tag{21.14}$$

We can then iterate these steps to convergence.

More formally, we can view this as finding a local minimum of the following cost function.

$$J(\mathbf{M}, \mathbf{Z}) = \sum_{n=1}^{N} ||\mathbf{x}_n - \boldsymbol{\mu}_{z_n}||^2 = ||\mathbf{X} - \mathbf{Z}\mathbf{M}^\top||_F^2 \tag{21.15}$$

where $\mathbf{X} \in \mathbb{R}^{N \times D}$, $\mathbf{Z} \in [0,1]^{N \times K}$, and $\mathbf{M} \in \mathbb{R}^{D \times K}$ contains the cluster centers $\boldsymbol{\mu}_k$ in its columns. K-means optimizes this using alternating minimization. (This is closely related to the EM algorithm for GMMs, as we discuss in Sec. 21.4.1.1.)

21.3.2 Examples

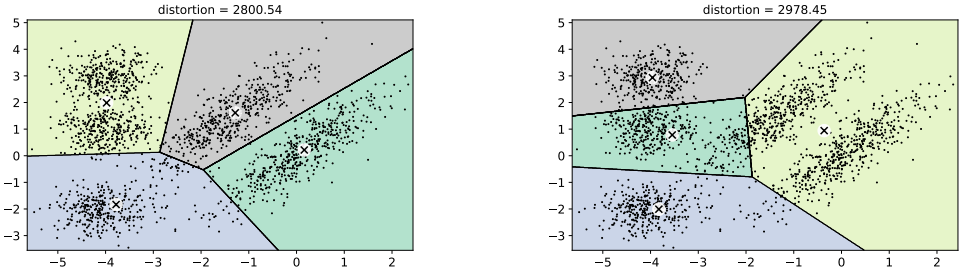
In this section, we give some examples of K-means clustering.

21.3.2.1 Clustering points in the 2d plane

Fig. 21.7 gives an illustration of K-means clustering applied to some points in the 2d plane. We see that the method induces a **Voronoi tessellation** of the points. The resulting clustering is sensitive to the initialization. Indeed, we see that the lower quality clustering on the right has higher distortion. By default, sklearn uses 10 random restarts (combined with the K-means++ initialization described in Sec. 21.3.4) and returns the clustering with lowest distortion. (In sklearn, the distortion is called the "inertia".)

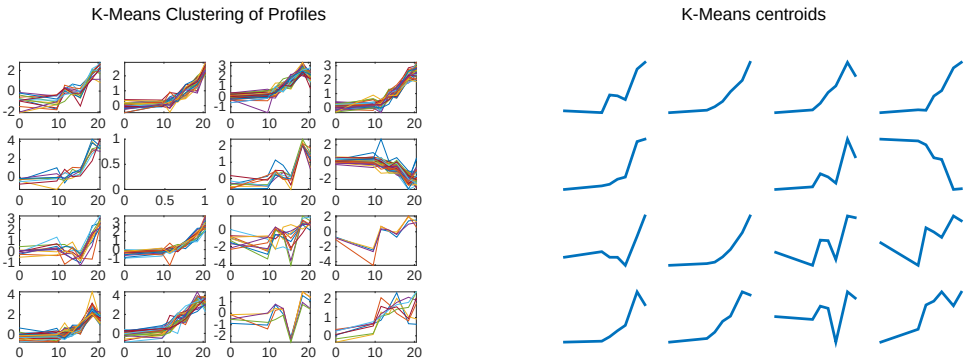
21.3.2.2 Clustering gene expression time series data from yeast cells

In Fig. 21.8, we show the result of applying K-means clustering with $K = 16$ to the 300×7 yeast time series matrix shown in Fig. 21.5. We see that time series that "look similar" to each other are assigned to the same cluster. We also see that the centroid of each cluster is a reasonabe summary all the data points assigned to that cluster. Finally we notice that group 6 was not used, since no points were assigned to it. However, this is just an accident of the initialization process, and we are not guaranteed to get the same clustering, or number of clusters, if we repeat the algorithm. (We discuss good ways to initialize the method in Sec. 21.3.4, and ways to choose K in Sec. 21.3.7.)



(a) (b)

Figure 21.7: Illustration of K-means clustering in 2d. We show the result of using two different random seeds. Adapted from Figure 9.5 of [Gér19]. Generated by code at figures.probml.ai/book1/21.7.



(a) (b)

Figure 21.8: Clustering the yeast data from Fig. 21.5 using K-means clustering with $K = 16$. (a) Visualizing all the time series assigned to each cluster. (d) Visualizing the 16 cluster centers as prototypical time series. Generated by code at figures.probml.ai/book1/21.8.

21.3.3 Vector quantization

Suppose we want to perform lossy compression of some real-valued vectors, $\mathbf{x}_n \in \mathbb{R}^D$. A very simple approach to this is to use **vector quantization** or **VQ**. The basic idea is to replace each real-valued vector $\mathbf{x}_n \in \mathbb{R}^D$ with a discrete symbol $z_n \in \{1, \ldots, K\}$, which is an index into a **codebook** of K prototypes, $\boldsymbol{\mu}_k \in \mathbb{R}^D$. Each data vector is encoded by using the index of the most similar prototype, where similarity is measured in terms of Euclidean distance:

$$\text{encode}(\mathbf{x}_n) = \arg\min_k ||\mathbf{x}_n - \boldsymbol{\mu}_k||^2 \tag{21.16}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

(a) *(b)*

Figure 21.9: An image compressed using vector quantization with a codebook of size K. (a) $K = 2$. (b) $K = 4$. Generated by code at figures.probml.ai/book1/21.9.

We can define a cost function that measures the quality of a codebook by computing the **reconstruction error** or **distortion** it induces:

$$J \triangleq \frac{1}{N}\sum_{n=1}^{N} ||\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n))||^2 = \frac{1}{N}\sum_{n=1}^{N} ||\mathbf{x}_n - \boldsymbol{\mu}_{z_n}||^2 \tag{21.17}$$

where $\text{decode}(k) = \boldsymbol{\mu}_k$. This is exactly the cost function that is minimized by the K-means algorithm.

Of course, we can achieve zero distortion if we assign one prototype to every data vector, by using $K = N$ and assigning $\boldsymbol{\mu}_n = \mathbf{x}_n$. However, this does not compress the data at all. In particular, it takes $O(NDB)$ bits, where N is the number of real-valued data vectors, each of length D, and B is the number of bits needed to represent a real-valued scalar (the quantization accuracy to represent each $\mathbf{x}_n$).

We can do better by detecting similar vectors in the data, creating prototypes or centroids for them, and then representing the data as deviations from these prototypes. This reduce the space requirement to $O(N \log_2 K + KDB)$ bits. The $O(N \log_2 K)$ term arises because each of the N data vectors needs to specify which of the K codewords it is using; and the $O(KDB)$ term arises because we have to store each codebook entry, each of which is a D-dimensional vector. When N is large, the first term dominates the second, so we can approximate the **rate** of the encoding scheme (number of bits needed per object) as $O(\log_2 K)$, which is typically much less than $O(DB)$.

One application of VQ is to image compression. Consider the 200×320 pixel image in Fig. 21.9; we will treat this as a set of $N = 64,000$ scalars. If we use one byte to represent each pixel (a gray-scale intensity of 0 to 255), then $B = 8$, so we need $NB = 512,000$ bits to represent the image in uncompressed form. For the compressed image, we need $O(N \log_2 K)$ bits. For $K = 4$, this is about 128kb, a factor of 4 compression, yet it results in negligible perceptual loss (see Fig. 21.9(b)).

Greater compression could be achieved if we modelled spatial correlation between the pixels, e.g., if we encoded 5x5 blocks (as used by JPEG). This is because the residual errors (differences from the model's predictions) would be smaller, and would take fewer bits to encode. his shows the deep connection between data compression and density estimation. See the sequel to this book, [Mur22], for more information.

21.3.4 The K-means++ algorithm

K-means is optimizing a non-convex objective, and hence needs to be initialized carefully. A simple approach is to pick K data points at random, and to use these as the initial values for $\boldsymbol{\mu}_k$. We can improve on this by using **multiple restarts**, i.e., we run the algorithm multiple times from different random starting points, and then pick the best solution. However, this can be slow.

A better approach is to pick the centers sequentially so as to try to "cover" the data. That is, we pick the initial point uniformly at random, and then each subsequent point is picked from the remaining points, with probability proportional to its squared distance to the point's closest cluster center. That is, at iteration t, we pick the next cluster center to be $\mathbf{x}_n$ with probability

$$p(\boldsymbol{\mu}_t = \mathbf{x}_n) = \frac{D_{t-1}(\mathbf{x}_n)^2}{\sum_{n'=1}^{N} D_{t-1}(\mathbf{x}_{n'})^2} \tag{21.18}$$

where

$$D_t(\mathbf{x}) = \min_{k=1}^{t-1} ||\mathbf{x} - \boldsymbol{\mu}_k||_2^2 \tag{21.19}$$

is the distance of $\mathbf{x}$ to the closest existing centroid. Thus points that are far away from a centroid are more likely to be picked, thus reducing the distortion. This is known as **farthest point clustering** [Gon85], or **K-means++** [AV07; Bah+12; Bac+16; BLK17; LS19a]. Surprisingly, this simple trick can be shown to guarantee that the recontruction error is never more than $O(\log K)$ worse than optimal [AV07].

21.3.5 The K-medoids algorithm

There is a variant of K-means called **K-medoids** algorithm, in which we estimate each cluster center $\boldsymbol{\mu}_k$ by choosing the data example $\mathbf{x}_n \in \mathcal{X}$ whose average dissimilarity to all other points in that cluster is minimal; such a point is known as a **medoid**. By contrast, in K-means, we take averages over points $\mathbf{x}_n \in \mathbb{R}^D$ assigned to the cluster to compute the center. K-medoids can be more robust to outliers (although that issue can also be tackled by using mixtures of Student distributions, instead of mixtures of Gaussians). More importantly, K-medoids can be applied to data that does not live in $\mathbb{R}^D$, where averaging may not be well defined. In K-medoids, the input to the algorithm is $N \times N$ pairwise distance matrix, $D(n, n')$, not an $N \times D$ feature matrix.

The classic algorithm for solving the K-medoids is the **partitioning around medoids** or **PAM** method [KR87]. In this approach, at each iteration, we loop over all K medoids. For each medoid m, we consider each non-medoid point o, swap m and o, and recompute the cost (sum of all the distances of points to their medoid). If the cost has decreased, we keep this swap. The running time of this algorithm is $O(K(N-K)^2 I)$, where I is the number of iterations.

There is also a simpler and faster method, known as the **Voronoi iteration** method due to [PJ09]. In this approach, at each iteration, we have two steps, similar to K-means. First, for each cluster k, look at all the points currently assigned to that cluster, $S_k = \{n : z_n = k\}$, and then set m_k to be the index of the medoid of that set. (To find the medoid requires examining all $|S_k|$ candidate points, and choosing the one that has the smallest sum of distances to all the other points in S_k.) Second, for each point n, assign it to its closest medoid, $z_n = \text{argmin}_k D(n, k)$. The pseudo-code is given in Algorithm 14. The running time of this algorithm is $O(NKI)$, which is comparable to K-means, and much faster than PAM.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 14: K-medoids algorithm

Initialize $m_{1:K}$ as a random subset of size K from $\{1, \ldots, N\}$;
repeat
  $z_n = \text{argmin}_k \, d(n, m_k)$ for $n = 1 : N$;
  $m_k \leftarrow \text{argmin}_{n:z_n=k} \sum_{n':z_{n'}=k} d(n, n')$ for $k = 1 : K$;
until *converged*;

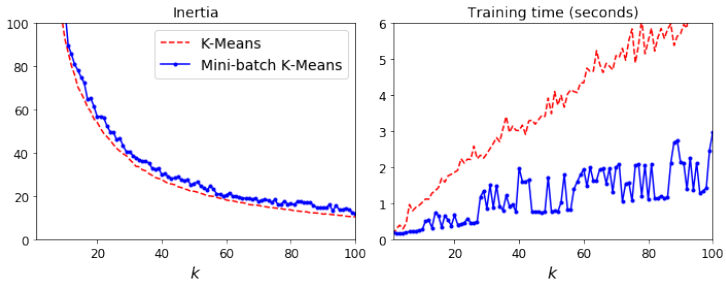


Figure 21.10: Illustration of batch vs mini-batch K-means clustering on the 2d data from Fig. 21.7. Left: distortion vs K. Right: Training time vs K. Adapted from Figure 9.6 of [Gér19]. Generated by code at figures.probml.ai/book1/21.10.

21.3.6 Speedup tricks

K-means clustering takes $O(NKI)$ time, where I is the number of iterations, but we can reduce the constant factors using various tricks. For example, [Elk03] shows how to use the triangle inequality to keep track of lower and upper bounds for the distances between inputs and the centroids; this can be used to eliminate some redundant computations. Another approach is to use a minibatch approximation, as proposed in [Scu10]. This can be significantly faster, although can result in slightly worse loss (see Fig. 21.10).

21.3.7 Choosing the number of clusters K

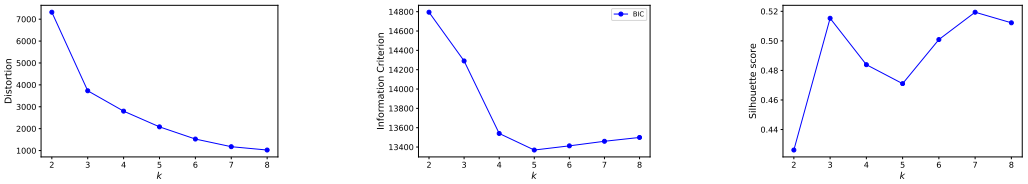
In this section, we discuss how to choose the number of clusters K in the K-means algorithm and other related methods.

21.3.7.1 Minimizing the distortion

Based on our experience with supervised learning, a natural choice for picking K is to pick the value that minimizes the reconstruction error on a validation set, defined as follows:

$$\text{err}(\mathcal{D}_{\text{valid}}, K) = \frac{1}{|\mathcal{D}_{\text{valid}}|} \sum_{n \in \mathcal{D}_{\text{valid}}} ||\mathbf{x}_n - \hat{\mathbf{x}}_n||_2^2 \tag{21.20}$$

where $\hat{\mathbf{x}}_n = \text{decode}(\text{encode}(\mathbf{x}_n))$ is the reconstruction of $\mathbf{x}_n$.



(a) (b) (c)

Figure 21.11: Performance of K-means and GMM vs K on the 2d dataset from Fig. 21.7. (a) Distortion on validation set vs K. Generated by code at figures.probml.ai/book1/21.11. (b) BIC vs K. Generated by code at figures.probml.ai/book1/21.11. (c) Silhouette score vs K. Generated by code at figures.probml.ai/book1/21.11.

Unfortunately, this technique will not work. Indeed, as we see in Fig. 21.11a, the distortion monotonically decreases with K. To see why, note that the K-means model is a degenerate density model which consists of K "spikes" at the $\boldsymbol{\mu}_k$ centers. As we increase K, we "cover" more of the input space. Hence any given input point is more likely to find a close prototype to accurately represent it as K increases, thus decreasing reconstruction error. Thus unlike with supervised learning, *we cannot use reconstruction error on a validation set as a way to select the best unsupervised model.* (This comment also applies to picking the dimensionality for PCA, see Sec. 20.1.4.)

21.3.7.2 Maximizing the marginal likelihood

A method that does work is to use a proper probabilistic model, such as a GMM, as we describe in Sec. 21.4.1. We can then to use the log marginal likelihood (LML) of the data to perform model selection.

We can approximate the LML using the BIC score as we discussed in Sec. 5.4.5.1. This has the following form:

$$\text{BIC}(K) = \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_k) - D_K \log(N) \tag{21.21}$$

where D_K is the number of parameters in a model with K clusters, and $\hat{\boldsymbol{\theta}}_K$ is the MLE. We see from Fig. 21.11b that this exhibits the typical U-shaped curve, where the penalty decreases and then increases.

The reason this works is that each cluster is associated with a Gaussian distribution that fills a volume of the input space, rather than being a degenerate spike,. Once we have enough clusters to cover the true modes of the distribution, the Bayesian Occam's razor (Sec. 5.4.3) kicks in, and starts penalizing the model for being unncessarily complex.

See Sec. 21.4.1.3 for more discussion of Bayesian model selection for mixture models.

21.3.7.3 Silhouette coefficient

In this section, we describe a common heuristic method for picking the number of clusters in a K-means clustering model. This is designed to work for spherical (not elongated) clusters. First we define the **silhouette coefficient** of an instance i to be $sc(i) = (b_i - a_i)/\max(a_i, b_i)$, where a_i is the mean distance to the other instances in cluster $k_i = \text{argmin}_k ||\boldsymbol{\mu}_k - \mathbf{x}_i||$, and b_i is the mean distance

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

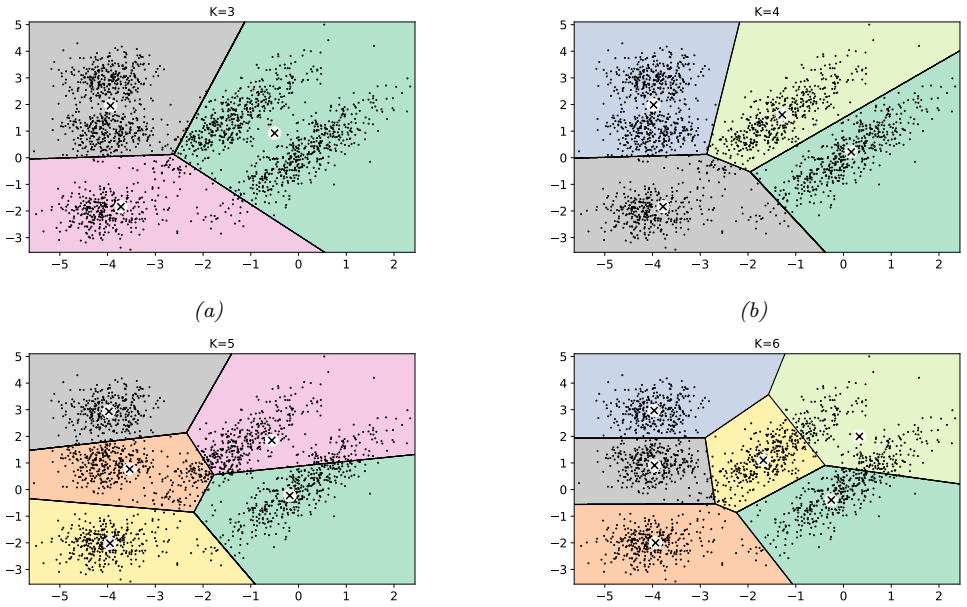


Figure 21.12: Voronoi diagrams for K-means for different K on the 2d dataset from Fig. 21.7. Generated by code at figures.probml.ai/book1/21.12.

to the other instances in the next closest cluster, $k_i' = \text{argmin}_{k \neq k_i} ||\boldsymbol{\mu}_k - \mathbf{x}_i||$. Thus a_i is a measure of compactness of i's cluster, and b_i is a measure of distance between the clusters. The silhouette coefficient varies from -1 to +1. A value of +1 means the instance is close to all the members of its cluster, and far from other clusters; a value of 0 means it is close to a cluster boundary; and a value of -1 means it may be in the wrong cluster. We define the **silhouette score** of a clustering K to be the mean silhouette coefficient over all instances.

In Fig. 21.11a, we plot the distortion vs K for the data in Fig. 21.7. As we explained above, it goes down monotonically with K. There is a slight "**kink**" or "**elbow**" in the curve at $K = 3$, but this is hard to detect. In Fig. 21.11c, we plot the silhouette score vs K. Now we see a more prominent peak at $K = 3$, although it seems $K = 7$ is almost as good. See Fig. 21.12 for a comparison of some of these clusterings.

It can be informative to look at the individual silhouette coefficients, and not just the mean score. We can plot these in a **silhouette diagram**, as shown in Fig. 21.13, where each colored region corresponds to a different cluster. The dotted vertical line is the average coefficient. Clusters with many points to the left of this line are likely to be of low quality. We can also use the silhouette diagram to look at the size of each cluster, even if the data is not 2d.

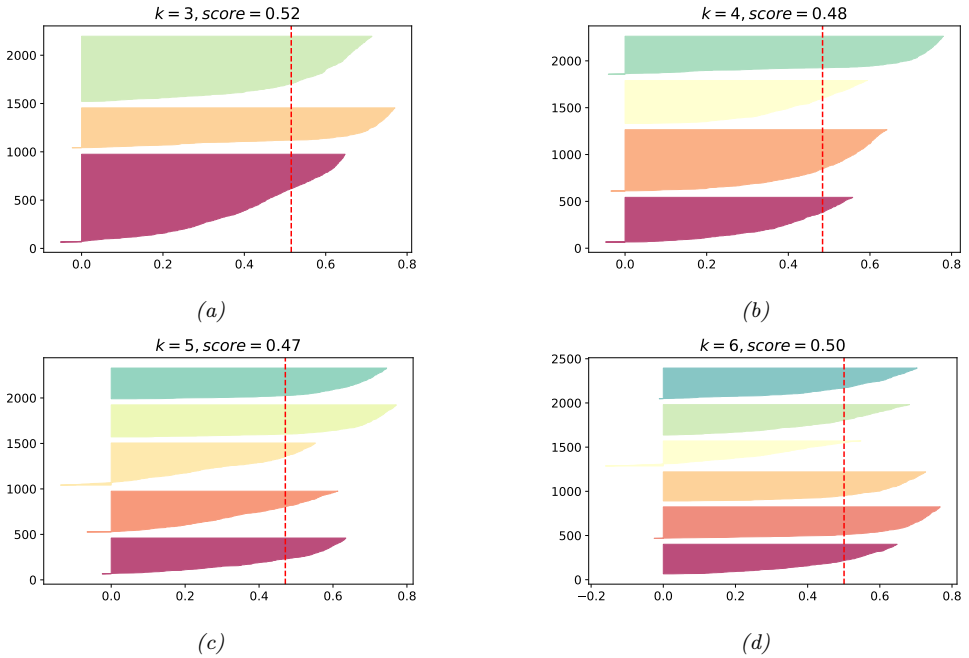


Figure 21.13: Silhouette diagrams for K-means for different K on the 2d dataset from Fig. 21.7. Generated by code at figures.probml.ai/book1/21.13.

21.3.7.4 Incrementally growing the number of mixture components

An alternative searching for the best value of K is to incrementally "grow" GMMs. We can start with a small value of K, and after each round of training, we consider splitting the cluster with the highest mixing weight into two, with the new centroids being random perturbations of the original centroid, and the new scores being half of the old scores. If a new cluster has too small a score, or too narrow a variance, it is removed. We continue in this way until the desired number of clusters is reached. See [FJ02] for details.

21.3.7.5 Sparse estimation methods

Another approach is to pick a large value of K, and then to use some kind of sparsity-promoting prior or inference method to "kill off" unneeded mixture components, such as variational Bayes. See the sequel to this book, [Mur22], for details.

21.4 Clustering using mixture models

We have seen how the K-means algorithm can be used to cluster data vectors in $\mathbb{R}^D$. However, this method assumes that all clusters have the same spherical shape, which is a very restrictive assumption. In addition, K-means assumes that all clusters can be described by Gaussians in the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

input space, so it cannot be applied to discrete data. By using mixture models (Sec. 3.4), we can overcome both of these problems, as we illustrate below.

21.4.1 Mixtures of Gaussians

Recall from Sec. 3.4.1.1 that a Gaussian mixture model (GMM) is a model of the form

$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{21.22}$$

If we know the model parameters $\boldsymbol{\theta} = (\boldsymbol{\pi}, \{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\})$, we can use Bayes rule to compute the responsibility (posterior membership probability) of cluster k for data point $\mathbf{x}_n$:

$$r_{nk} \triangleq p(z_n = k|\mathbf{x}_n, \boldsymbol{\theta}) = \frac{p(z_n = k|\boldsymbol{\theta})p(\mathbf{x}_n|z_n = k, \boldsymbol{\theta})}{\sum_{k'=1}^{K} p(z_n = k'|\boldsymbol{\theta})p(\mathbf{x}_n|z_n = k', \boldsymbol{\theta})} \tag{21.23}$$

Given the responsibilities, we can compute the most probable cluster assignment as follows:

$$\hat{z}_n = \arg\max_k r_{nk} = \arg\max_k \left[\log p(\mathbf{x}_n|z_n = k, \boldsymbol{\theta}) + \log p(z_n = k|\boldsymbol{\theta})\right] \tag{21.24}$$

This is known as **hard clustering**.

21.4.1.1 K-means is a special case of EM

We can estimate the parameters of a GMM using the EM algorithm (Sec. 8.7.3). It turns out that the K-means algorithm is a special case of this algorithm, in which we make two approximations: we fix $\boldsymbol{\Sigma}_k = \mathbf{I}$ and $\pi_k = 1/K$ for all the clusters (so we just have to estimate the means $\boldsymbol{\mu}_k$), and we approximate the E step, by replacing the soft responsibilities with hard cluster assignments, i.e., we compute $z_n^* = \text{argmax}_k\, r_{nk}$, and set $r_{nk} \approx \mathbb{I}(k = z_n^*)$ instead of using the soft responsibilities, $r_{nk} = p(z_n = k|\mathbf{x}_n, \boldsymbol{\theta})$. With this approximation, the weighted MLE problem in Eq. (8.175) of the M step reduces to Eq. (21.14), so we recover K-means.

However, the assumption that all the clusters have the same spherical shape is very restrictive. For example, Fig. 21.14 shows the marginal density and clustering induced using different shaped covariance matrices for some 2d data. We see that modeling this particular dataset needs the ability to capture off-diagonal covariance for some clusters (top row).

21.4.1.2 Unidentifiability and label switching

Note that we are free to permute the labels in a mixture model without changing the likelihood. This is called the **label switching problem**, and is an example of **non-identifiability** of the parameters. This can cause problems if we wish to perform posterior inference over the parameters (as opposed to just computing the MLE or a MAP estimate). For example, suppose we fit a GMM with $K = 2$ components to the data in Fig. 21.15 using HMC. The posterior over the means, $p(\mu_1, \mu_2|\mathcal{D})$, is shown in Fig. 21.16a. We see that the marginal posterior for each component, $p(\mu_k|\mathcal{D})$, is bimodal. This reflects the fact that there are two equally good explanations of the data: either $\mu_1 \approx 47$ and $\mu_2 \approx 57$, or vice versa.

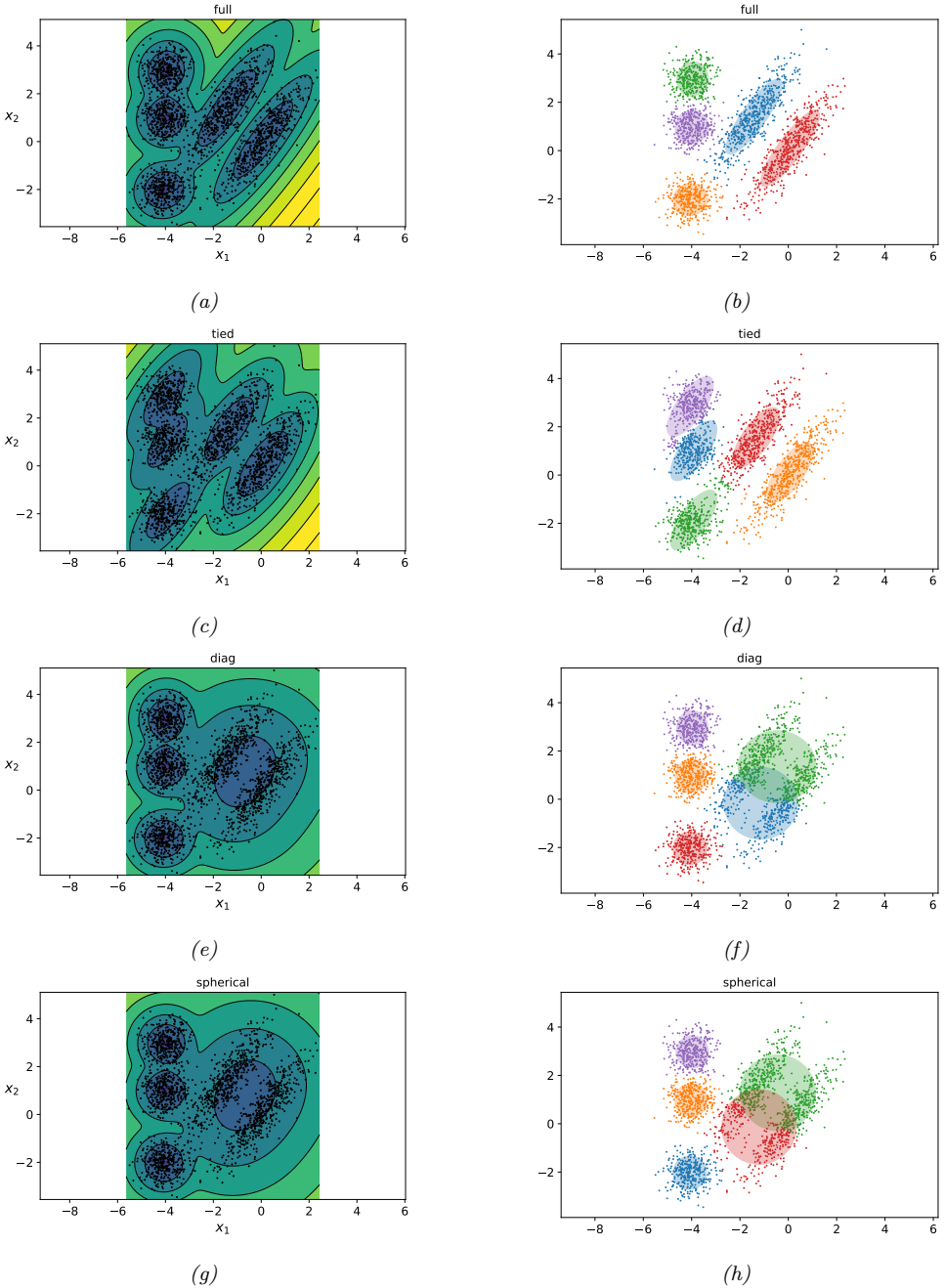


Figure 21.14: Some data in 2d fit using a GMM with $K = 5$ components. Left column: marginal distribution $p(\mathbf{x})$. Right column: visualization of each mixture distribution, and the hard assignment of points to their most likely cluster. (a-b) Full covariance. (c-d) Tied full covariance. (e-f) Diagonal covairance, (g-h) Spherical covariance. Color coding is arbitrary. Generated by code at figures.probml.ai/book1/21.14.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

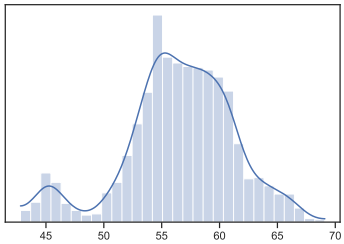
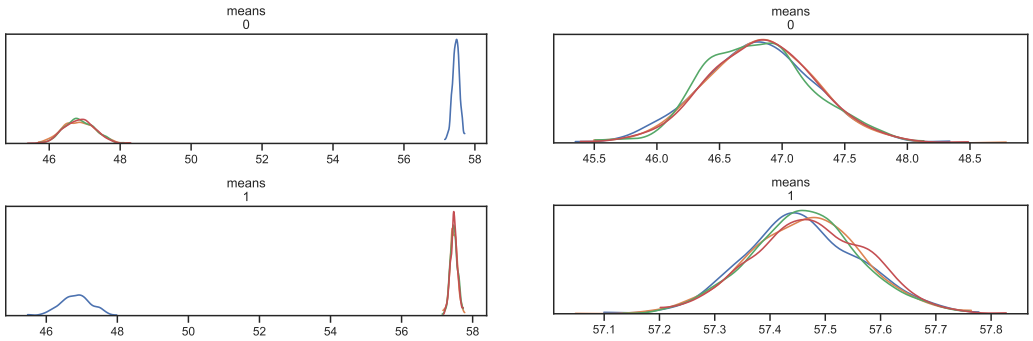


Figure 21.15: Some 1d data, with a kernel density estimate superimposed. Adapted from Figure 6.2 of [Mar18]. Generated by code at figures.probml.ai/book1/21.15.



(a) *(b)*

Figure 21.16: Illustration of the label switching problem when performing posterior inference for the parameters of a GMM. We show a KDE estimate of the posterior marginals derived from 1000 samples from 4 HMC chains. (a) Unconstrained model. Posterior is symmetric. (b) Constrained model, where we add a penalty to ensure $\mu_0 < \mu_1$. Adapted from Figure 6.6-6.7 of [Mar18]. Generated by code at figures.probml.ai/book1/21.16.

To break symmetry, we can add an **ordering constraint** on the centers, so that $\mu_1 < \mu_2$. We can do this by adding a penalty or potential function to the objective if the penalty is violated. More precisely, the penalized log joint becomes

$$\ell'(\boldsymbol{\theta}) = \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) + \phi(\boldsymbol{\mu}) \tag{21.25}$$

where

$$\phi(\boldsymbol{\mu}) = \begin{cases} -\infty & \text{if } \mu_1 < \mu_0 \\ 0 & \text{otherwise} \end{cases} \tag{21.26}$$

This has the desired effect, as shown in Fig. 21.16b.

A more general approach is to apply a transformation to the parameters, to ensure identifiability. That is, we sample the parameters $\boldsymbol{\theta}$ from a proposal, and then apply an invertible transformation $\boldsymbol{\theta}' = f(\boldsymbol{\theta})$ to them before computing the log joint, $\log p(\mathcal{D}, \boldsymbol{\theta}')$. To account for the change of

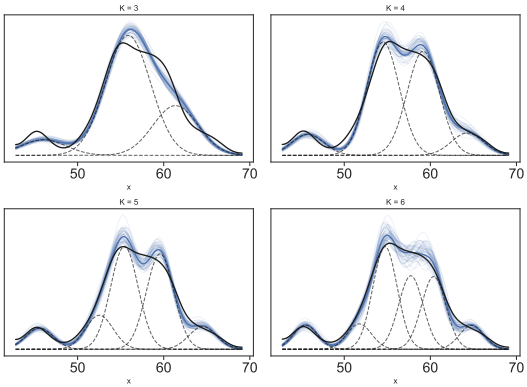


Figure 21.17: Fitting GMMs with different numbers of clusters K to the data in Fig. 21.15. Black solid line is KDE fit. Solid blue line is posterior mean; feint blue lines are posterior samples. Dotted lines show the individual Gaussian mixture components, evaluated by plugging in their posterior mean parameters. Adapted from Figure 6.8 of [Mar18]. Generated by code at figures.probml.ai/book1/21.17.

variables (Sec. 2.8.3), we add the log of the determinant of the Jacobian. In the case of a 1d ordering transformation, which just sorts its inputs, the determinant of the Jacobian is 1, so the log-det-Jacobian term vanishes.

Unfortunately, this approach does not scale to more than 1 dimensional problems, because there is no obvious way to enforce an ordering constraint on the centers $\boldsymbol{\mu}_k$.

21.4.1.3 Bayesian model selection

Once we have a reliable way to ensure identifiability, we can use Bayesian model selection techniques from Sec. 5.4.2 to select the number of clusters K. In Fig. 21.17, we illustrate the results of fitting a GMM with $K = 3 - 6$ components to the data in Fig. 21.15. We use the ordering transform on the means, and perform inference using HMC. We compare the resulting GMM model fits to the fit of a kernel density estimate (Sec. 16.3), which often over-smooths the data. We see fairly strong evidence for two bumps, corresponding to different subpopulations.

We can compare these models more quantitatively by computing their WAIC scores (Sec. 5.4.5.3), which is an approximation to the log marginal likelihood. The results are shown in Fig. 21.18. (This kind of visualization was proposed in [McE20, p228].) We see that the model with $K = 6$ scores significantly higher than for the other models, although $K = 5$ is a close second. This is consistent with the plot in Fig. 21.17.

21.4.2 Mixtures of Bernoullis

As we discussed in Sec. 3.4.2, we can use a mixtures of Bernoullis to cluster binary data. The model has the form

$$p(\mathbf{y}|z = k, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Ber}(y_d|\mu_{dk}) = \prod_{d=1}^{D} \mu_{dk}^{y_d}(1 - \mu_{dk})^{1-y_d} \tag{21.27}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

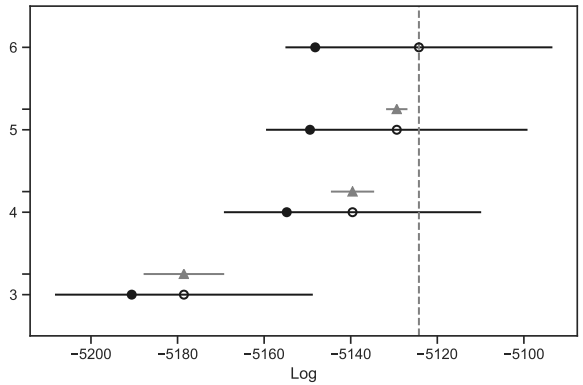


Figure 21.18: WAIC scores for the different GMMs. The empty circle is the posterior mean WAIC score for each model, and the black lines represent the standard error of the mean. The solid circle is the in-sample deviance of each model, i.e., the unpenalized log-likelihood. The dashed vertical line corresponds to the maximum WAIC value. The gray triangle is the difference in WAIC score for that model compared to the best model. Adapted from Figure 6.10 of [Mar18]. Generated by code at figures.probml.ai/book1/21.18.

Here μ_{dk} is the probability that bit d turns on in cluster k. We can fit this model with EM, SGD, MCMC, etc. See Fig. 3.13 for an example, where we cluster some binarized MNIST digits.

21.5 Spectral clustering *

In this section, we discuss an approach to clustering based on eigenvalue analysis of a pairwise similarity matrix. It uses the eigenvectors to derive feature vectors for each datapoint, which are then clustered using a feature-based clustering method, such as K-means (Sec. 21.3). This is known as **spectral clustering** [SM00; Lux07].

21.5.1 Normalized cuts

We start by creating a weighted undirected graph $\mathbf{W}$, where each data vector is a node, and the strength of the $i-j$ edge is a measure of similarity. Typically we only connected a node to its most similar neighbors, to ensure the graph is sparse, which speeds computation.

Our goal is to find K clusters of similar points. That is, we we want to find a **graph partition** into $S_1, \ldots, S_K$ disjoint sets of nodes so as to minimize some kind of cost.

Our first attempt at a cost function is to compute the weight of connections between nodes in each cluster to nodes outside each cluster:

$$\text{cut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} W(S_k, \overline{S}_k) \tag{21.28}$$

where $W(A, B) \triangleq \sum_{i \in A, j \in B} w_{ij}$ and $\overline{S}_k = V \setminus S_k$ is the complement of S_k, where $V = \{1, \ldots, K\}$.

Unfortunately the optimal solution to this often just partitions off a single node from the rest, since that minimizes the weight of the cut. To prevent this, we can divide by the size of each set, to

get the following objective, known as the **normalized cut**:

$$\text{Ncut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} \frac{\text{cut}(S_k, \overline{S}_k)}{\text{vol}(S_k)} \tag{21.29}$$

where $\text{vol}(A) \triangleq \sum_{i \in A} d_i$ is the total weight of set A and $d_i = \sum_{j=1}^{N} w_{ij}$ is the weighted degree of node i. This splits the graph into K clusters such that nodes within each cluster are similar to each other, but are different to nodes in other clusters.

We can formulate the Ncut problem in terms of searching for binary vectors $\mathbf{c}_i \in \{0,1\}^N$ that minimizes the above objective, where $c_{ik} = 1$ iff point i belongs to cluster k. Unfortunately this is NP-hard [WW93]. Below we discuss a continuous relaxation of the problem based on eigenvector methods that is easier to solve.

21.5.2 Eigenvectors of the graph Laplacian encode the clustering

In Sec. 20.4.9.2, we discussed the graph Laplacian, which is defined as $\mathbf{L} \triangleq \mathbf{D} - \mathbf{W}$, where $\mathbf{W}$ is a symmetric weight matrix for the graph, and $\mathbf{D} = \text{diag}(d_i)$ is a diagonal matrix containing the weighted degree of each node, $d_i = \sum_j w_{ij}$. To get some intuition as to why $\mathbf{L}$ might be useful for graph-based clustering, we note the following result.

Theorem 21.5.1. *The set of eigenvectors of $\mathbf{L}$ with eigenvalue 0 is spanned by the indicator vectors $\mathbf{1}_{S_1}, \ldots, \mathbf{1}_{S_K}$, where S_k are the K connected components of the graph.*

Proof. Let us start with the case $K = 1$. If $\mathbf{f}$ is an eigenvector with eigenvalue 0, then $0 = \sum_{ij} w_{ij}(f_i - f_j)^2$. If two nodes are connected, so $w_{ij} > 0$, we must have that $f_i = f_j$. Hence $\mathbf{f}$ is constant for all vertices which are connected by a path in the graph. Now suppose $K > 1$. In this case, $\mathbf{L}$ will be block diagonal. A similar argument to the above shows that we will have K indicator functions, which "select out" the connected components. □

This suggests the following clustering algorithm. Compute the eigenvectors and values of $\mathbf{L}$, and let $\mathbf{U}$ be an $N \times K$ matrix with the K eigenvectors with smallest eigenvalue in its columns. (Fast methods for computing such "bottom" eigenvectors are discussed in [YHJ09]). Let $\mathbf{u}_i \in \mathbb{R}^K$ be the i'th *row* of $\mathbf{U}$. Since these $\mathbf{u}_i$ will be piecewise constant, we can apply K-means clustering (Sec. 21.3) to them to recover the connected components. (Note that the vectors $\mathbf{u}_i$ are the same as those computed by Laplacian eigenmaps discussed in Sec. 20.4.9.)

Real data may not exhibit such clean block structure, but one can show, using results from perturbation theory, that that the eigenvectors of a "perturbed" Laplacian will be close to these ideal indicator functions [NJW01].

In practice, it is important to normalize the graph Laplacian, to account for the fact that some nodes are more highly connected than others. One way to do this (proposed in [NJW01]) is to create a symmetric matrix

$$\mathbf{L}_{sym} \triangleq \mathbf{D}^{-\frac{1}{2}} \mathbf{L} \mathbf{D}^{-\frac{1}{2}} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}} \mathbf{W} \mathbf{D}^{-\frac{1}{2}} \tag{21.30}$$

This time the eigenspace of 0 is spanned by $\mathbf{D}^{\frac{1}{2}} \mathbf{1}_{S_k}$. This suggests the following algorithm: find the smallest K eigenvectors of $\mathbf{L}_{sym}$, stack them into the matrix $\mathbf{U}$, normalize each *row* to unit norm by creating $t_{ij} = u_{ij} / \sqrt{(\sum_k u_{ik}^2)}$ to make the matrix $\mathbf{T}$, cluster the rows of $\mathbf{T}$ using K-means, then infer the partitioning of the original points.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

(a) (b)

Figure 21.19: Clustering data consisting of 2 spirals. (a) K-means. (b) Spectral clustering. Generated by code at figures.probml.ai/book1/21.19.

21.5.3 Example

Figure 21.19 illustrates the method in action. In Figure 21.19(a), we see that K-means does a poor job of clustering, since it implicitly assumes each cluster corresponds to a spherical Gaussian. Next we try spectral clustering. We compute a dense similarity matrix $\mathbf{W}$ using a Gaussian kernel, $W_{ij} = \exp(-\frac{1}{2\sigma^2}||\mathbf{x}_i - \mathbf{x}_j||_2^2)$. We then compute the first two eigenvectors of the normalized Laplacian L_{sym}. From this we infer the clustering using K-means, with $K = 2$; the results are shown in Figure 21.19(b).

21.5.4 Connection with other methods

Spectral clustering is closely related to several other methods for unsupervised learning, some of which we discuss below.

21.5.4.1 Connection with kPCA

Spectral clustering is closely related to kernel PCA (Sec. 20.4.6). In particular, kPCA uses the largest eigenvectors of $\mathbf{W}$; these are equivalent to the smallest eigenvectors of $\mathbf{I} - \mathbf{W}$. This is similar to the above method, which computes the smallest eigenvectors of $\mathbf{L} = \mathbf{D} - \mathbf{W}$. See [Ben+04a] for details. In practice, spectral clustering tends to give better results than kPCA.

21.5.4.2 Connection with random walk analysis

In practice we get better results by computing the eigenvectors of the normalized graph Laplacian. One way to normalize the graph Laplacian, which is used in [SM00; Mei01], is to define

$$\mathbf{L}_{rw} \triangleq \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{W} \tag{21.31}$$

One can show that for $\mathbf{L}_{rw}$, the eigenspace of 0 is again spanned by the indicator vectors $\mathbf{1}_{S_k}$ [Lux07], so we can perform clustering directly on the K smallest eigenvectors $\mathbf{U}$.

There is an interesting connection between this approach and random walks on a graph. First note that $\mathbf{P} = \mathbf{D}^{-1}\mathbf{W} = \mathbf{I} - \mathbf{L}_{rw}$ is a stochastic matrix, where $p_{ij} = w_{ij}/d_i$ can be interpreted as the

probability of going from i to j. If the graph is connected and non-bipartite, it possesses a unique stationary distribution $\boldsymbol{\pi} = (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_N)$, where $\pi_i = d_i/\text{vol}(V)$, and $\text{vol}(V) = \sum_i d_i$ is the sum of all the node degrees. Furthermore, one can show that for a partition of size 2,

$$\text{Ncut}(S, \overline{S}) = p(\overline{S}|S) + p(S|\overline{S}) \tag{21.32}$$

This means that we are looking for a cut such that a random walk spends more time transitioning to similar points, and rarely makes transitions from S to $\overline{S}$ or vice versa. This analysis can be extended to $K > 2$; for details, see [Mei01].

21.6 Biclustering *

In some cases, we have a data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and we want to cluster the rows *and* the columns; this is known as **biclustering** or **coclustering**. This is widely used in bioinformatics, where the rows often represent genes and the columns represent conditions. It can also be used for collaborative filtering, where the rows represent users and the columns represent movies.

A variety of ad hoc methods for biclustering have been proposed; see [MO04] for a review. In Sec. 21.6.1, we present a simple probabilistic generative model in which we assign a latent cluster id to each row, and a differnet latent cluster id to each column. In Sec. 21.6.2, we extend this to the case where each row can belong to multiple clusters, depending on which groups of features (columns) we choose to use to define the different groups of objects (rows).

21.6.1 Basic biclustering

Here we present a simple probabilistic generative model for biclustering based on [Kem+06] (see also [SMM03] for a related approach). The idea is to associate each row and each column with a latent indicator, $u_i \in \{1, \ldots, N_u\}$, $v_j \in \{1, \ldots, N_v\}$, where N_u is the number of row clusters, and N_v is the number of column clusters. We then use the following generative model:

$$p(\mathbf{U}) = \prod_{i=1}^{N_r} \text{Unif}(u_i|\{1, \ldots, N_u\}) \tag{21.33}$$

$$p(\mathbf{V}) = \prod_{j=1}^{N_c} \text{Cat}(v_j|\{1, \ldots, N_v\}) \tag{21.34}$$

$$p(\mathbf{X}|\mathbf{U}, \mathbf{V}, \boldsymbol{\theta}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} p(X_{ij}|\boldsymbol{\theta}_{u_i, v_j}) \tag{21.35}$$

where $\boldsymbol{\theta}_{a,b}$ are the parameters for row cluster a and column cluster b.

Fig. 21.20 shows a simple example. The data has the form $X_{ij} = 1$ iff animal i has feature j, where $i = 1:50$ and $j = 1:85$. The animals represent whales, bears, horses, etc. The features represent properties of the habitat (jungle, tree, coastal), or anatomical properties (has teeth, quadrapedal), or behavioral properties (swims, eats meat), etc. The method discovered 12 animal clusters and 33 feature clusters. ([Kem+06] use a Bayesian nonparametric method to infer the number of clusters.) For example, the O2 cluster is { antelope, horse, giraffe, zebra, deer }, which is characterized by

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

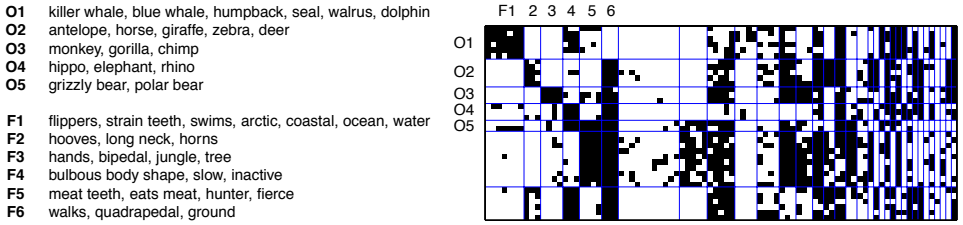


Figure 21.20: Illustration of biclustering. We show 5 of the 12 organism clusters, and 6 of the 33 feature clusters. The original data matrix is shown, partitioned according to the discovered clusters. From Figure 3 of [Kem+06]. Used with kind permission of Charles Kemp.

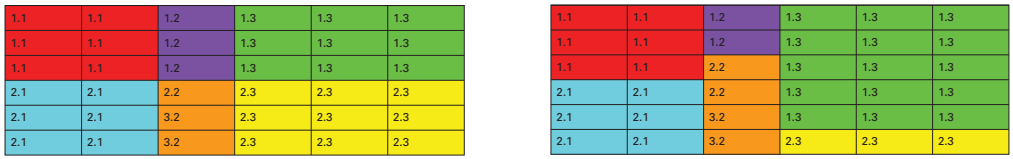


Figure 21.21: (a) Example of biclustering. Each row is assigned to a unique cluster, and each column is assigned to a unique cluster. (b) Example of multi-clustering using a nested partition model. The rows can belong to different clusters depending on which subset of column features we are looking at.

feature clusters F2 = { hooves, long kneck, horns} and F6 = { walks, quadrapedal, ground }, whereas the O4 cluster is { hippo, elephant, rhino }, which is characterized by feature clusters F4 = { bulbous body shape, slow, inactive } and F6.

21.6.2 Nested partition models (Crosscat)

The problem with basic biclustering (Sec. 21.6.1) is that each object (row) can only belong to one cluster. Intuitively, an object can have multiple roles, and can be assigned to different clusters depending on which subset of features you use. For example, in the animal dataset, we may want to group the animals on the basis of anatomical features (e.g., mammals are warm blooded, reptiles are not), or on the basis of behavioral features (e.g., predators vs prey).

We now present a model that can capture this phenomenon. We illustrate the method with an example. Suppose we have a 6×6 matrix, with $N_u = 2$ row clusters and $N_v = 3$ column clusters. Furthermore, suppose the latent column assignments are as follows: $\mathbf{v} = [1, 1, 2, 3, 3, 3]$. This means we put columns 1 and 2 into group 1, column 3 into group 2, and columns 4 to 6 into group 3. For the columns that get clustered into group 1, we cluster the rows as follows: $\mathbf{u}_{:,1} = [1, 1, 1, 2, 2, 2]$; For the columns that get clustered into group 2, we cluster the rows as follows: $\mathbf{u}_{:,2} = [1, 1, 2, 2, 2, 2]$; and for the columns that get clustered into group 3, we cluster the rows as follows: $\mathbf{u}_{:,3} = [1, 1, 1, 1, 1, 2]$. The resulting partition is shown in Fig. 21.21(b). We see that the clustering of the rows depends on which group of columns we choose to focus on.

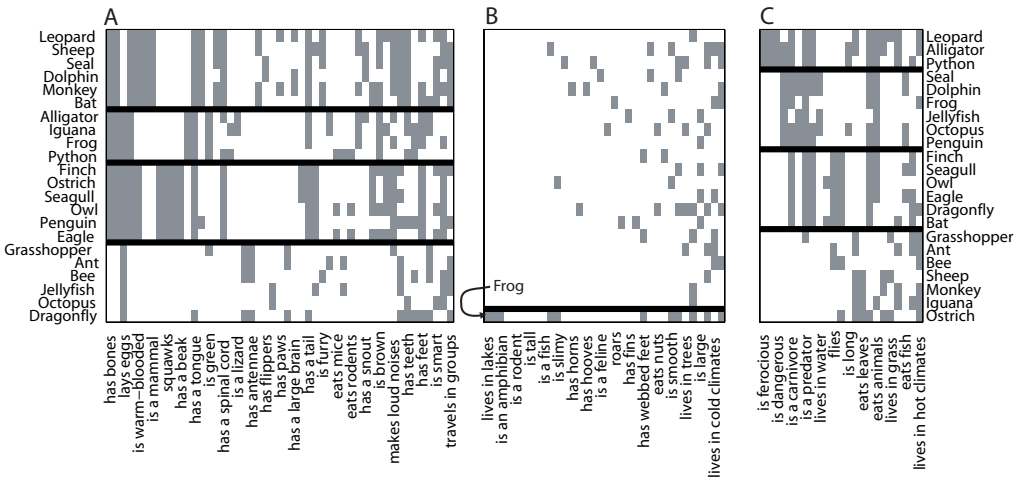


Figure 21.22: MAP estimate produced by the crosscat system when applied to a binary data matrix of animals (rows) by features (columns). See text for details. From Figure 7 of [Sha+06]. Used with kind permission of Vikash Mansingkha.

Formally, we can define the model as follows:

$$p(\mathbf{U}) = \prod_{i=1}^{N_r} \prod_{l=1}^{N_v} \text{Unif}(u_{il} | \{1, \ldots, N_u\}) \tag{21.36}$$

$$p(\mathbf{V}) = \prod_{j=1}^{N_c} \text{Unif}(v_j | \{1, \ldots, N_v\}) \tag{21.37}$$

$$p(\mathbf{Z}|\mathbf{U}, \mathbf{V}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} \mathbb{I}\left(Z_{ij} = (u_{i,v_j}, v_j)\right) \tag{21.38}$$

$$p(\mathbf{X}|\mathbf{Z}, \boldsymbol{\theta}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} p(X_{ij} | \boldsymbol{\theta}_{z_{ij}}) \tag{21.39}$$

where $\boldsymbol{\theta}_{k,l}$ are the parameters for cocluster $k \in \{1, \ldots, N_u\}$ and $l \in \{1, \ldots, N_v\}$.

This model was independently proposed in [Sha+06; Man+16] who call it **crosscat** (for cross-categorization), in [Gua+10; CFD10], who call it **multi-clust**, and in [RG11], who call it **nested partitioning**. In all of these papers, the authors propose to use Dirichlet processes, to avoid the problem of estimating the number of clusters. Here we assume the number of clusters is known, and show the parameters explicitly, for notational simplicity.

Fig. 21.22 illustrates the model applied to some binary data containing 22 animals and 106 features. The figures shows the (approximate) MAP partition. The first partition of the columns contains taxonomic features, such as "has bones", "is warm-blooded", "lays eggs", etc. This divides the animals into birds, reptiles/ amphibians, mammals, and invertebrates. The second partition of the columns

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

contains features that are treated as noise, with no apparent structure (except for the single row labeled "frog"). The third partition of the columns contains ecological features like "dangerous", "carnivorous", "lives in water", etc. This divides the animals into prey, land predators, sea predators and air predators. Thus each animal (row) can belong to a different cluster depending on what set of features are considered.

22 Recommender systems

Recommender systems are systems which recommend **items** (such as movies, books, ads) to **users** based on various information, such as their past viewing/ purchasing behavior (e.g., which movies they rated high or low, which ads they clicked on), as well as optional "side information" such as demographics about the user, or information about the content of the item (e.g., its title, genre or price). Such systems are widely used by various internet companies, such as Facebook, Amazon, Netflix, Google, etc. In this chapter, we give a brief introduction to the topic. More details can be found in e.g., [DKK12; Pat12; Yan+14; AC16; Agg16; Zha+19b]..

22.1 Explicit feedback

In this section, we consider the simplest setting in which the user gives **explicit feedback** to the system in terms of a **rating**, such as +1 or -1 (for like/dislike) or a score from 1 to 5. Let $Y_{ui} \in \mathbb{R}$ be the rating that user u gives to item i. We can represent this as an $M \times N$ matrix, where M is the number of users, and N is the number of items. Typically this matrix will be very large but very sparse, since most users will not provide any feedback on most items. See Fig. 22.1(a) for an example. We can also view this sparse matrix as a bipartite graph, where the weight of the $u - i$ edge is Y_{ui}. This reflects the fact that we are dealing with **relational data**, i.e., the values of u and i have no intrinsic meaning (they are just arbitrary indices), it is the fact that u and i are connected that matters.

If Y_{ui} is missing, it could be because user u has not interacted with item i, or it could be that they knew they wouldn't like it and so they chose not to engage with it. In the former case, some of the data is **missing at random**; in the latter case, the missingness is informative about the true value of Y_{ui}. (See e.g., [Mar+11] for further discussion of this point.) We will assume the data is missing at random, for simplicity.

22.1.1 Datasets

A famous example of an explicit ratings matrix was made available by the movie streaming company Netflix. In 2006, they released a large dataset of 100,480,507 movie ratings (on a scale of 1 to 5) from 480,189 users of 17,770 movies. Despite the large size of the training set, the ratings matrix is still 99% sparse (unknown). Along with the data, they offered a prize of $100,000, known as the **Netflix Prize**, to any team that could predict the true ratings of a set of test (user, item) pairs more accurately than their incumbent system. The prize was claimed on September 21, 2009 by a team known as "Pragmatic Chaos". They used an ensemble of different methods, as described in

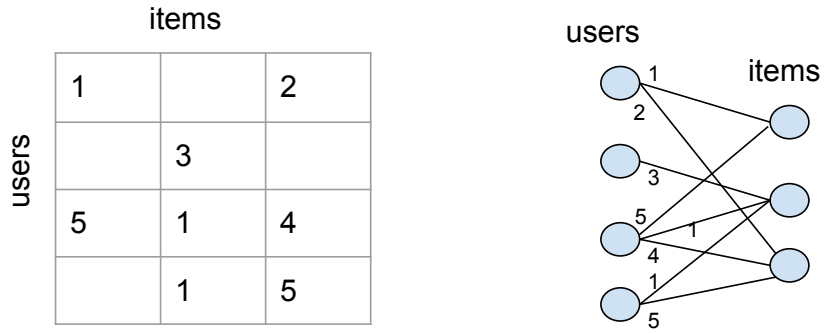


Figure 22.1: Example of a relational dataset represented as a sparse matrix (left) or a sparse bipartite graph (right). Values corresponding to empty cells (missing edges) are unknown. Rows 3 and 4 are similar to each other, indicating that users 3 and 4 might have similar preferences, so we can use the data from user 3 to predict user 4's preferences. However, user 1 seems quite different in their preferences, and seems to give low ratings to all items. For user 2, we have very little observed data, so it is hard to make reliable predictions.

[Kor09; BK07; FHK12]. However, a key component in their ensemble was the method described in Sec. 22.1.3.

Unfortunately the Netflix data is no longer available due to privacy concerns. Fortunately the **MovieLens** group at the University of Minnesota have released an anonymized public dataset of movie ratings, on a scale of 1-5, that can be used for research [HK15]. There are also various other public explicit ratings datasets, such as the **Jester** jokes dataset from [Gol+01] and the **BookCrossing** dataset from [Zie+05].

22.1.2 Collaborative filtering

The original approach to the recommendation problem is called **collaborative filtering** [Gol+92]. The idea is that users collaborate on recommending items by sharing their ratings with other users; then if u wants to know if they interact with i, they can see what ratings other users u' have given to i, and take a weighted average:

$$\hat{Y}_{ui} = \sum_{u': Y_{u',i} \neq ?} \text{sim}(u, u')\, Y_{u',i} \tag{22.1}$$

where we assume $Y_{u',i} =?$ if the entry is unknown. The traditional approach measured the similarity of two users by comparing the sets $S_u = \{Y_{u,i} \neq ? : i \in \mathcal{I}\}$ and $S_{u'} = \{Y_{u',i} \neq ? : i \in \mathcal{I}\}$, where $\mathcal{I}$ is the set of items. However, this can suffer from data sparsity. In Sec. 22.1.3 we discuss an approach based on learning dense embedding vectors for each item and each user, so we can compute similarity in a low dimensional feature space.

22.1.3 Matrix factorization

We can view the recommender problem as one of **matrix completion**, in which we wish to predict all the missing entries of $\mathbf{Y}$. We can formulate this as the following optimization problem:

$$\mathcal{L}(\mathbf{Z}) = \sum_{ij:Y_{ij} \neq ?} (Z_{ij} - Y_{ij})^2 = ||\mathbf{Z} - \mathbf{Y}||_F^2 \tag{22.2}$$

However, this is an under-specified problem, since there are an infinite number of ways of filling in the missing entries of $\mathbf{Z}$.

We need to add some constraints. Suppose we assume that $\mathbf{Y}$ is low rank. Then we can write it in the form $\mathbf{Z} = \mathbf{U}\mathbf{V}^\top \approx \mathbf{Y}$, where $\mathbf{U}$ is an $M \times K$ matrix, $\mathbf{V}$ is a $N \times K$ matrix, K is the rank of the matrix, M is the number of users, and N is the number of items. This corresponds to a prediction of the form by writing

$$\hat{y}_{ui} = \mathbf{u}_u^\top \mathbf{v}_i \tag{22.3}$$

This is called **matrix factorization**.

If we observe all the Y_{ij} entries, we can find the optimal $\mathbf{Z}$ using SVD (Sec. 7.5). However, when $\mathbf{Y}$ has missing entries, the corresponding objective is no longer convex, and does not have a unique optimum [SJ03]. We can fit this uisng **alternating least squares** (**ALS**), where we estimate $\mathbf{U}$ given $\mathbf{V}$ and then estimate $\mathbf{V}$ given $\mathbf{U}$ (for details, see e.g., [KBV09]). Alternatively we can just use SGD.

In practice, it is important to also allow for user-specific and item-specific baselines, by writing

$$\hat{y}_{ui} = \mu + b_u + c_i + \mathbf{u}_u^\top \mathbf{v}_i \tag{22.4}$$

This can capture the fact that some users might always tend to give low ratings and others may give high ratings; in addition, some items (e.g., very popular movies) might have unusually high ratings.

In addition, we can add some ℓ_2 regularization to the parameters to get the objective

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{ij:Y_{ij} \neq ?} (y_{ij} - \hat{y}_{ij})^2 + \lambda(b_u^2 + c_i^2 + ||\mathbf{u}_u||^2 + ||\mathbf{v}_i||^2) \tag{22.5}$$

We can optimize this using SGD by sampling a random (u, i) entry from the set of observed values, and performing the following updates:

$$b_u = b_u + \eta(e_{ui} - \lambda b_u) \tag{22.6}$$
$$c_i = c_i + \eta(e_{ui} - \lambda b_i) \tag{22.7}$$
$$\mathbf{u}_u = \mathbf{u}_u + \eta(e_{ui}\mathbf{u}_u - \lambda \mathbf{u}_u) \tag{22.8}$$
$$\mathbf{v}_i = \mathbf{v}_i + \eta(e_{ui}\mathbf{v}_i - \lambda \mathbf{v}_i) \tag{22.9}$$

where $e_{ui} = y_{ui} - \hat{y}_{ui}$ is the error term, and $\eta \geq 0$ is the learning rate. This approach was first proposed by Simon Funk, who was one of the first to do well in the early days of the Netflix competition.[1]

1. https://sifter.org/~simon/journal/20061211.html.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

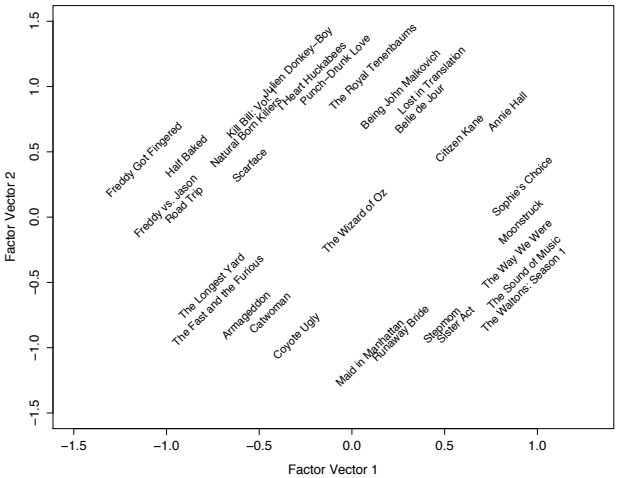


Figure 22.2: Visualization of the first two latent movie factors estimated from the Netflix challenge data. Each movie j is plotted at the location specified by $\mathbf{v}_j$. See text for details. From Figure 3 of [KBV09]. Used with kind permission of Yehuda Koren.

22.1.3.1 Probabilistic matrix factorization (PMF)

We can convert matrix factorization into a probabilistic model by defining

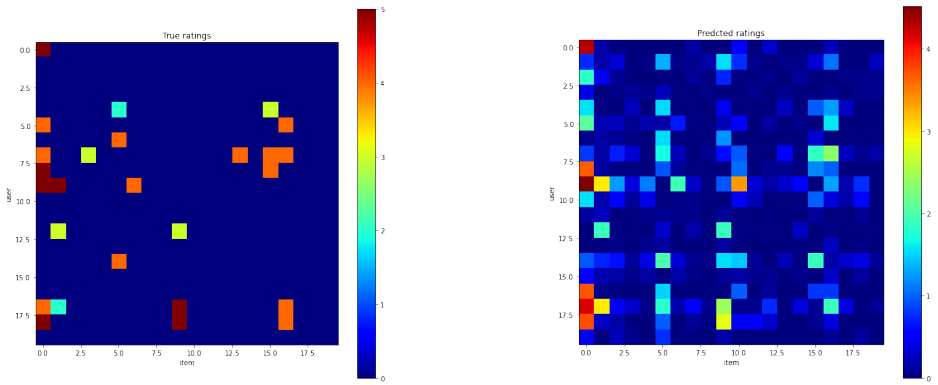
$$p(y_{ui} = y) = \mathcal{N}(y|\mu + b_u + c_i + \mathbf{u}_u^\top \mathbf{v}_i, \sigma^2) \tag{22.10}$$

This is known as **probabilistic matrix factorization** (**PMF**) [SM08]. The NLL of this model is equivalent to the matrix factorization objective in Eq. (22.2). However, the probabilistic perspective allows us to generalize the model more easily. For example, we can capture the fact that the ratings are integers (often mostly 0s), and not reals, using a Poisson or negative Binomial likelihood (see e.g., [GOF18]). This is similar to exponential family PCA (Sec. 20.2.7), except that we view rows and columns symmetrically.

22.1.3.2 Example: Netflix

Suppose we apply PMF to the Netflix dataset using $K = 2$ latent factors. Fig. 22.2 visualizes the learned embedding vectors $\mathbf{u}_i$ for a few movies. On the left of the plot we have low-brow humor and horror movies (*Half Baked*, *Freddy vs Jason*), and on the right we have more serious dramas (*Sophie's Choice*, *Moonstruck*). On the top we have critically acclaimed independent movies (*Punch-Drunk Love*, *I Heart Huckabees*), and on the bottom we have mainstream Hollywood blockbusters (*Armageddon*, *Runway Bride*). The *Wizard of Oz* is right in the middle of these axes, since it is in some senses an "average movie".

Users are embedded into the same spaces as movies. We can then predict the rating for any user-video pair using proximity in the latent embedding space.



(a) (b)

Figure 22.3: (a) A fragment of the observed ratings matrix from the MovieLens-1M dataset. (b) Predictions using SVD with 50 latent components. Generated by code at figures.probml.ai/book1/22.3.

22.1.3.3 Example: MovieLens

Now suppose we apply PMF to the MovieLens-1M dataset with 6040 users, 3706 movies, and 1,000,209 ratings. We will use $K = 50$ factors. For simplicity, we fit this using SVD applied to the dense ratings matrix, where we replace missing values with 0. (This is just a simple approximation to keep the demo code simple.) In Fig. 22.3 we show a snippet of the true and predicted ratings matrix. (We truncate the predictions to lie in the range [1,5].) We see that the model is not particularly accurate, but does capture some structure in the data.

Furthermore, it seems to behave in a qualitatively sensible way. For example, in Fig. 22.4 we show the top 10 movies rated by a given user as well as the top 10 predictions for movies they had not seen. The model seems to have "picked up" on the underlying preferences of the user. For example, we see that many of the predicted mpvies are action or film-noir, and both of these genres feature in the user's own top-10 list, even though explicit genre information is not used during model training.

22.1.4 Autoencoders

Matrix factorization is a (bi)linear model. We can make a nonlinear version using autoencoders. Let $\mathbf{y}_{:,i} \in \mathbb{R}^M$ be the i'th column of the ratings matrix, where unknown ratings are set to 0. We can predict this ratings vector using an autoencoder of the form

$$f(\mathbf{y}_{:,i}; \boldsymbol{\theta}) = \mathbf{W}^\top \varphi(\mathbf{V}\mathbf{y}_{:,i} + \boldsymbol{\mu}) + \mathbf{b} \tag{22.11}$$

where $\mathbf{V} \in \mathbb{R}^{KM}$ maps the ratings to an embedding space, $\mathbf{W} \in \mathbb{R}^{KM}$ maps the embedding space to a distribution over ratings, $\boldsymbol{\mu} \in \mathbb{R}^K$ are the biases of the hidden units, and $\mathbf{b} \in \mathbb{R}^M$ are the biases of the output units. This is called the (item-based) version of the **AutoRec** model [Sed+15]. This has $2MK + M + K$ parameters. There is also a user-based version, that can be derived in a similar manner, which has $2NK + N + K$ parameters. (On MovieLens and Netflix, the authors find that the item-based method works better.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

	MovieID	Title	Genres
36	858	Godfather, The (1972)	Action|Crime|Drama
35	1387	Jaws (1975)	Action|Horror
65	2028	Saving Private Ryan (1998)	Action|Drama|War
63	1221	Godfather: Part II, The (1974)	Action|Crime|Drama
11	913	Maltese Falcon, The (1941)	Film-Noir|Mystery
20	3417	Crimson Pirate, The (1952)	Adventure|Comedy|Sci-Fi
34	2186	Strangers on a Train (1951)	Film-Noir|Thriller
55	2791	Airplane! (1980)	Comedy
31	1188	Strictly Ballroom (1992)	Comedy|Romance
28	1304	Butch Cassidy and the Sundance Kid (1969)	Action|Comedy|Western

(a)

	MovieID	Title	Genres
516	527	Schindler's List (1993)	Drama|War
1848	1953	French Connection, The (1971)	Action|Crime|Drama|Thriller
596	608	Fargo (1996)	Crime|Drama|Thriller
1235	1284	Big Sleep, The (1946)	Film-Noir|Mystery
2085	2194	Untouchables, The (1987)	Action|Crime|Drama
1188	1230	Annie Hall (1977)	Comedy|Romance
1198	1242	Glory (1989)	Action|Drama|War
897	922	Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)	Film-Noir
1849	1954	Rocky (1976)	Action|Drama
581	593	Silence of the Lambs, The (1991)	Drama|Thriller

(b)

Figure 22.4: (a) Top 10 movies (from a list of 69) that user "837" has already highly rated. (b) Top 10 predictions (from a list of 3637) from the algorithm. Generated by code at figures.probml.ai/book1/22.4.

We can fit this by only updating parameters that are associated with the observed entries of $\mathbf{y}_{:,i}$. Furthermore, we can add an ℓ_2 regularizer to the weight matrices to get the objective

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \sum_{u:y_{ui}\neq ?} (y_{u,i} - f(\mathbf{y}_{:,i};\boldsymbol{\theta})_u)^2 + \frac{\lambda}{2}(||\mathbf{W}||_F^2 + ||\mathbf{V}||_F^2) \tag{22.12}$$

Despite the simplicity of this method, the authors find that this it does better than more complex methods such as restricted Boltzmann machines (RBMs, [SMH07]) and local low-rank matrix approximation (LLORMA, [Lee+13]).

22.2 Implicit feedback

So far, we have assumed tha the user gives explicit ratings for each item that they interact with. This is a very restrictive assumption. More generally, we would like to learn from the **implicit feedback**

that users give just by interacting with a system. For example, we can treat the list of movies that user u watches as positives, and regard all the other movies as negatives. Thus we get a sparse, positive-only ratings matrix.

Alternatively, we can view the fact that they watched movie i but did not watch movie j as an implicit signal that they prefer i to j. The resulting data can be represented as a set of tuples of the form $y_n = (u, i, j)$, where (u, i) is a positive pair, and (u, j) is a negative (or unlabeled) pair.

22.2.1 Bayesian personalized ranking

To fit a model to data of the form (u, i, j), we need to use a **ranking loss**, so that the model ranks i ahead of j for user u. A simple way to do this is to use a Bernoulli model of the form

$$p(y_n = (u, i, j)|\boldsymbol{\theta}) = \sigma(f(u, i; \boldsymbol{\theta}) - f(u, j; \boldsymbol{\theta})) \tag{22.13}$$

If we combine this with a Gaussian prior for $\boldsymbol{\theta}$, we get the following MAP estimation problem:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{(u,i,j)\in\mathcal{D}} \log \sigma(f(u, i; \boldsymbol{\theta}) - f(u, j; \boldsymbol{\theta})) - \lambda||\boldsymbol{\theta}||^2 \tag{22.14}$$

where $\mathcal{D} = \{(u, i, j) : i \in \mathcal{I}_u^+, j \in \mathcal{I} \setminus \mathcal{I}_u^+\}$, where $\mathcal{I}_u^+$ are the set of all items that user u selected, and $\mathcal{I} \setminus \mathcal{I}_u^+$ are all the other items (which they may dislike, or simply may not have seen). This is known as **Bayesian personalized ranking** or BPR [Ren+09].

Let us consider this example from [Zha+20, Sec 16.5]. There are 4 items in total, $\mathcal{I} = \{i_1, i_2, i_3, i_4\}$, and user u chose to interact with $\mathcal{I}_u^+ = \{i_2, i_3\}$. In this case, the implicit item-item preference matrix for user u has the form

$$\mathbf{Y}_u = \begin{pmatrix} . & + & + & ? \\ - & . & ? & - \\ - & ? & . & - \\ ? & + & + & . \end{pmatrix} \tag{22.15}$$

where $Y_{u,i,i'} = +$ means user u prefers i' to i, $Y_{u,i,i'} = -$ means user u prefers i to i', and $Y_{u,i,i'} =?$ means we cannot tell what the user's preference is. For example, focusing on the second column, we see that this user rates i_2 higher than i_1 and i_4, since they selected i_2 but not i_1 or i_4; however, we cannot tell if they prefer i_2 over i_3 or vice versa.

When the set of posssible items is large, the number of negatives in $\mathcal{I} \setminus \mathcal{I}_u^+$ can be very large. Fortunately we can approximate the loss by subsampling negatives.

Note that an alternative to the log-loss above is to use a hinge loss, similar to the approach used in SVMs (Sec. 17.5). This has the form

$$\mathcal{L}(y_n = (u, i, j), f) = \max(m - (f(u, i) - f(u, j)), 0) = \max(m - f(u, i) + f(u, j), 0) \tag{22.16}$$

where $m \geq 0$ is the safety margin. This tries to ensure the negative items j never score more than m higher than the positive items i.

22.2.2 Factorization machines

The AutoRec approach of Sec. 22.1.4 is nonlinear, but treats users and items asymmetrically. In this section, we discuss a more symmetric discriminative modeling approach. We start with a linear

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

version. The basic idea is to predict the output (such as a rating) for any given user-item pair, $\mathbf{x} = [\text{one-hot}(u), \text{one-hot}(i)]$, using

$$f(\mathbf{x}) = \mu + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D} \sum_{j=i+1}^{D} (\mathbf{v}_i^\top \mathbf{v}_j) x_i x_j \tag{22.17}$$

where $\mathbf{x} \in \mathbb{R}^D$ where $D = (M + N)$ is the number of inputs, $\mathbf{V} \in \mathbb{R}^{D \times K}$ is a weight matrix, $\mathbf{w} \in \mathbb{R}^D$ is a weight vector, and $\mu \in \mathbb{R}$ is a global offset. This is known as a **factorization machine** (FM) [Ren12].

The term $(\mathbf{v}_i^\top \mathbf{v}_j) x_i x_j$ measures the interaction between feature i and j in the input. This generalizes the matrix factorization model of Eq. (22.4), since it can handle other kinds of information in the input $\mathbf{x}$, beyond just user and item, as we discuss in Sec. 22.3.

Computing Eq. (22.17) takes $O(KD^2)$ time, since it considers all possible pairwise interactions between every user and every item. Fortunately we can rewrite this so that we can compute it in $O(KD)$ time as follows:

$$\sum_{i=1}^{D} \sum_{j=i+1}^{D} (\mathbf{v}_i^\top \mathbf{v}_j) x_i x_j = \frac{1}{2} \sum_{i=1}^{D} \sum_{j=1}^{D} (\mathbf{v}_i^\top \mathbf{v}_j) x_i x_j - \frac{1}{2} \sum_{i=1}^{D} (\mathbf{v}_i^\top \mathbf{v}_i) x_i x_i \tag{22.18}$$

$$= -\frac{1}{2} \left(\sum_{i=1}^{D} \sum_{j=1}^{D} \sum_{k=1}^{K} v_{ik} v_{jk} x_i x_j - \sum_{i=1}^{D} \sum_{k=1}^{K} v_{ik} v_{ik} x_i x_i \right) \tag{22.19}$$

$$= -\frac{1}{2} \sum_{k=1}^{K} \left((\sum_{i=1}^{D} v_{ik} x_i)^2 - \sum_{i=1}^{D} v_{ik}^2 x_i^2 \right) \tag{22.20}$$

For sparse vectors, the overall complexity is linear in the number of non-zero components. So if we use one-hot encodings of the user and item id, the complexity is just $O(K)$, analogous to the original matrix factorization objective of Eq. (22.4).

We can fit this model to minimize any loss we want. For example, if we have explicit feedback, we may choose MSE loss, and if we have implicit feedback, we may choosing ranking loss.

In [Guo+17], they propose a model called **deep factorization machines**, which combines the above method with an MLP applied to a concatenation of the embedding vectors, instead of the inner product. More precisely, it is a model of the form

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sigma(\text{FM}(\mathbf{x}) + \text{MLP}(\mathbf{x})) \tag{22.21}$$

This is closely related to the **wide and deep** model proposed in [Che+16]. The idea is that the bilinear FM model captures explicit interactions between specific users and items (a form of memorization), whereas the MLP captures implicit interactions between user features and item features, which allows the model to generalize.

22.2.3 Neural matrix factorization

In this section, we describe the **neural matrix factorization** model of [He+17]. This is another way to combine bilinear models with deep neural networks. The bilinear part is used to define the

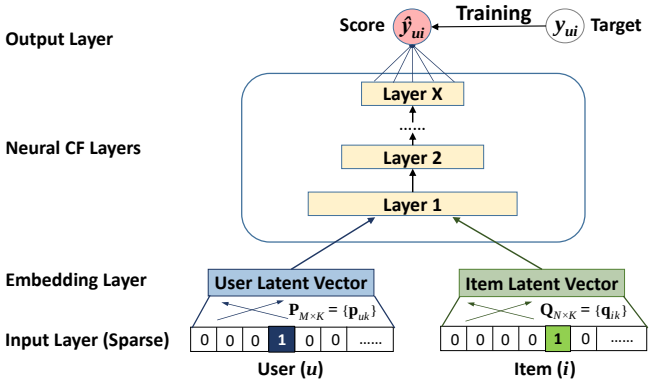


Figure 22.5: Illustration of the neural matrix factorization model. From Figure 2 of [He+17]. Used with kind permission of Xiangnan He.

following generalized matrix factorization (GMF) pathway, which computes the following feature vector for user u and item i:

$$\mathbf{z}_{ui}^1 = \mathbf{P}_{u,:} \odot \mathbf{Q}_{i,:} \tag{22.22}$$

where $\mathbf{P} \in \mathbb{R}^{MK}$ is a user embedding matrix, and $\mathbf{Q} \in \mathbb{R}^{NK}$ is an item embedding matrix. The DNN part is just an MLP applied to a concatenation of the embedding vectors (using different embedding matrices):

$$\mathbf{z}_{ui}^2 = \text{MLP}([\tilde{\mathbf{U}}_{u,:}, \tilde{\mathbf{V}}_{i,:}]) \tag{22.23}$$

Finally, the model combines these to get

$$f(u, i; \boldsymbol{\theta}) = \sigma(\mathbf{w}^\top [\mathbf{z}_{ui}^1, \mathbf{z}_{ui}^2]) \tag{22.24}$$

See Fig. 22.5 for an illustration.

In [He+17], the model is trained on implicit feedback, where $y_{ui} = 1$ if the interaction of user u with item i is observed, and $y_{ui} = 0$ otherwise. However, it could be trained to minimize BPR loss.

22.3 Leveraging side information

So far, we have assumed that the only information available to the predictor are the integer id of the user and the integer id of the item. This is an extremely impoverished representation, and will fail to work if we encounter a new user or new item (the so-called **cold start** problem). To overcome this, we need to leverage "**side information**", beyond just the id of the user/item.

There are many forms of side information we can use. For items, we often have rich meta-data, such text (e.g., title), images (e.g., cover), high-dimensional categorical variables (e.g., location), or just scalars (eg., price). For users, the side information available depends on the specific form of the interactive system. For search engines, it is the list of queries the user has issued, and (if they are

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

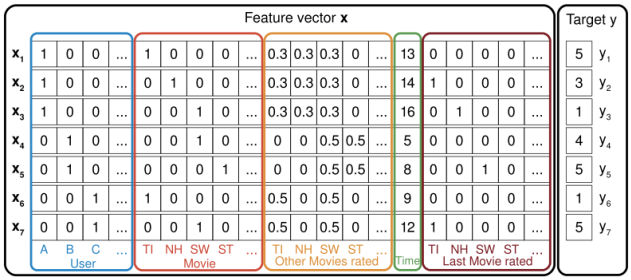


Figure 22.6: Illustration of a design matrix for a movie recommender system, where we show the id of the user and movie, as well as other side information. From Figure 1 of [Ren12]. Used with kind permission of Stefen Rendle.

logged in), information derived from websites they have visited (which is tracked via cookies). For online shopping sites, it is the list of searches plus past viewing and purchasing behavior. For social networking sites, there is information about the friendship graph of each user.

It is very easy to capture this side information in the factorization machines framework, by expanding our definition of $\mathbf{x}$ beyond the two one-hot vectors, as illustrated in Fig. 22.6. The same input encoding can of course be fed into other kinds of models, such as deepFM or neuralMF.

In addition to features about the user and item, there may be other contextual features, such as the time of the interaction (e.g., the day or evening). The order (sequence) of the most recently viewed items is often also a useful signal. The "Convolutional Sequence Embedding Recommendation" or **Caser** model proposed in [TW18] captures this by embedding the last M items, and then treating the $M \times K$ input as an image, by using a convolutional layer as part of the model.

Many other kinds of neural models can be designed for the recommender task. See e.g., [Zha+19b] for a review.

22.4 Exploration-exploitation tradeoff

An interesting "twist" to recommender systems that does not arise in other kinds of prediction problems is the fact that the data that the system is trained on is a consequence of recommendations made by earlier versions of the system. Thus there is a feedback loop [Bot+13]. For example, consider the YouTube video recommendation system [CAS16]. There are millions of videos on the site, so the system must come up with a shortlist, or "**slate**", of videos to show the user, to help them find what they want (see e.g., [Ie+19]). If the user watches one of these videos, the system can consider this positive feedback that it made a good recommendation, and it can update the model parameters accordingly. However, maybe there was some other video that the user would have liked even more? It is impossible to answer this counterfactual unless the system takes a chance and shows some items for which the user response is uncertain. This is an example of the **exploration-exploitation tradeoff** that we introduced in Sec. 5.3.3.

In addition to needing to explore, the system may have to wait for a long time until it can detect if a change it made its recommendation policies was beneficial. It is common to use **reinforcement learning** to learn policies which optimize long-term reward. See the sequel to this book, [Mur22],

for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

23 Graph embeddings *

This chapter is coauthored with Bryan Perozzi.

23.1 Introduction

We now turn our focus to data which has semantic relationships between training samples $\{\mathbf{x}_n\}_{n=1}^N$. The relationships (known as edges) connect training samples (nodes) with an application specific meaning (commonly similarity). Graphs provide the mathematical foundations for reasoning about these kind of relationships

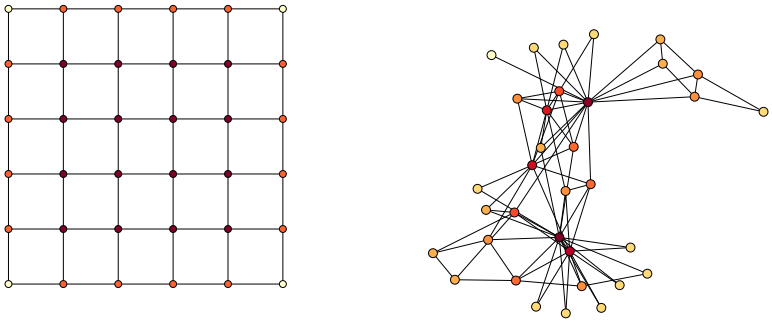
Graphs are universal data structures that can represent complex relational data (composed of nodes and edges), and appear in multiple domains such as social networks, computational chemistry [Gil+17], biology [Sta+06], recommendation systems [KSJ09], semi-supervised learning [GB18], and others.

Let $\mathbf{A} \in \{0,1\}^{N \times N}$ be the adjacency matrix, where N is the number of nodes, and let $\mathbf{W} \in \mathbb{R}^{N \times N}$ be a weighted version. In the methods we discuss below, some set $\mathbf{W} = \mathbf{A}$ while others set $\mathbf{W}$ to a transformation of $\mathbf{A}$, such as row-wise normalization. Finally, let $\mathbf{X} \in \mathbb{R}^{N \times D}$ be a matrix of node features.

When designing and training a neural network model over graph data, we desire the designed method be applicable to nodes which participate in different graph settings (e.g. have differing connections and community structure). Contrast this with a neural network model designed for images, where each pixel (node) has the same neighborhood structure. By contrast, an arbitrary graph has no specified alignment of nodes, and further, each node might have a different neighborhood structure. See Fig. 23.1 for a comparison. Consequently, operations like Euclidean spatial convolution cannot be directly applied on irregular graphs: Euclidean convolutions strongly rely on geometric priors (such as shift invariance), which don't generalize to non-Euclidean domains.

These challenges led to the development of **Geometric Deep Learning** (GDL) research [Bro+17b], which aims at applying deep learning techniques to non-Euclidean data. In particular, given the widespread prevalence of graphs in real-world applications, there has been a surge of interest in applying machine learning methods to graph-structured data. Among these, **Graph Representation Learning** (GRL) [Cha+20] methods aim at learning low-dimensional continuous vector representations for graph-structured data, also called embeddings.

We divide GRL here into two classes of problems: **unsupervised** and **supervised** (or semi-supervised) GRL. The first class aims at learning low-dimensional Euclidean representations optimizing an objective, e.g. one that preserve the structure of an input graph. The second class also learns



(a) Grid (Euclidean). *(b) Arbitrary graph (Non-Euclidean).*

Figure 23.1: An illustration of Euclidean vs. non-Euclidean graphs. Used with permission from [Cha+20].

low-dimensional Euclidean representations but for a specific downstream prediction task such as node or graph classification. Further, the graph structure can be fixed throughout training and testing, which is known as the **transductive** learning setting (e.g. predicting user properties in a large social network), or alternatively the model is expected to answer questions about graphs not seen during training, known as the **inductive** learning setting (e.g. classifying molecular structures). Finally, while most supervised and unsupervised methods learn representations in Euclidean vector spaces, there recently has been interest for **non-Euclidean representation learning**, which aims at learning non-Euclidean embedding spaces such as hyperbolic or spherical spaces. The main motivations for this body of work is to use a continuous embedding space that resembles the underlying discrete structure of the input data it tries to embed (e.g. the hyperbolic space is a continuous version of trees [Sar11]).

23.2 Graph Embedding as an Encoder/Decoder Problem

While there are many approaches to GRL, many methods follow a similar pattern. First, the network input (node features $\mathbf{X} \in \mathbb{R}^{N \times D}$ and graph edges in $\mathbf{A}$ or $\mathbf{W} \in \mathbb{R}^{N \times N}$) is encoded from the discrete domain of the graph into a continuous representation (embedding), $\mathbf{Z} \in \mathbb{R}^{N \times L}$. Next, the learned representation $\mathbf{Z}$ is used to optimize a particular objective (such as reconstructing the links of the graph). In this section we will use the graph encoder-decoder model (GraphEDM) proposed by Chami et al. [Cha+20] to analyze popular families of GRL methods.

The GraphEDM framework (Figure 23.2, [Cha+20]) provides a general framework that encapsulates a wide variety of supervised and unsupervised graph embedding methods: including ones utilizing the graph as a regularizer (e.g. [ZG02]), positional embeddings(e.g. [PARS14]), and graph neural networks such as ones based on message passing [Gil+17; Sca+09] or graph convolutions [Bru+14; KW16a]).

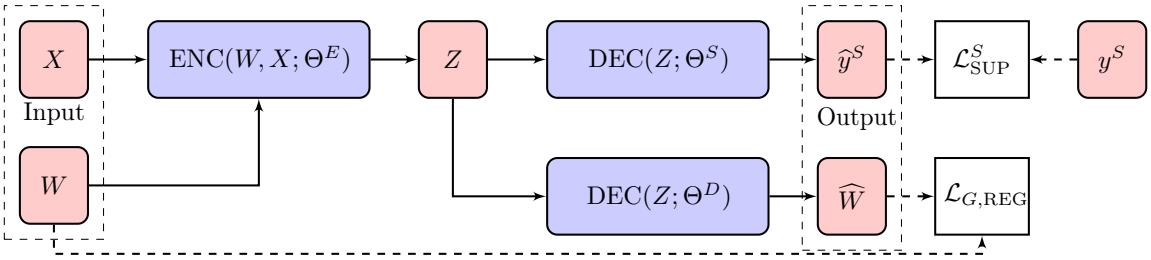


Figure 23.2: Illustration of the GraphEDM *framework from Chami et al. [Cha+20]. Based on the supervision available, methods will use some or all of the branches. In particular, unsupervised methods do not leverage label decoding for training and only optimize the similarity decoder (lower branch). On the other hand, semi-supervised and supervised methods leverage the additional supervision to learn models' parameters (upper branch). Used with permission.*

The GraphEDM framework takes as input a weighted graph $\mathbf{W} \in \mathbb{R}^{N \times N}$, and optional node features $\mathbf{X} \in \mathbb{R}^{N \times D}$. In (semi-)supervised settings, we assume that we are given training target labels for nodes (denoted N), edges (denoted E), and/or for the entire graph (denoted G). We denote the supervision signal as $S \in \{N, E, G\}$, as presented below.

The GraphEDM model itself can be decomposed into the following components:

- **Graph encoder network** $\text{ENC}_{\Theta^E} : \mathbb{R}^{N \times N} \times \mathbb{R}^{N \times D} \to \mathbb{R}^{N \times L}$, parameterized by Θ^E, which combines the graph structure with optional node features to produce a node embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times L}$ as follows:

$$\mathbf{Z} = \text{ENC}(\mathbf{W}, \mathbf{X}; \Theta^E). \tag{23.1}$$

  As we shall see next, this node embedding matrix might capture different graph properties depending on the supervision used for training.

- **Graph decoder network** $\text{DEC}_{\Theta^D} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times N}$, parameterized by Θ^D, which uses the node embeddings Z to compute similarity scores for all node pairs in matrix $\widehat{\mathbf{W}} \in \mathbb{R}^{N \times N}$ as follows:

$$\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D). \tag{23.2}$$

- **Classification network** $\text{DEC}_{\Theta^S} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times |\mathcal{Y}|}$, where $\mathcal{Y}$ is the label space. This network is used in (semi-)supervised settings and parameterized by Θ^S. The output is a distribution over the labels $\hat{y}^S$, using node embeddings, as follows:

$$\hat{y}^S = \text{DEC}(\mathbf{Z}; \Theta^S). \tag{23.3}$$

Specific choices of the aforementioned (encoder and decoder) networks allows GraphEDM to realize specific graph embedding methods, as we explain in the next subsections.

The output of a model, as described by GraphEDM framework, is a reconstructed graph similarity matrix $\widehat{W}$ (often used to train *unsupervised* embedding algorithms), and/or labels $\widehat{y}^S$ for *supervised*

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

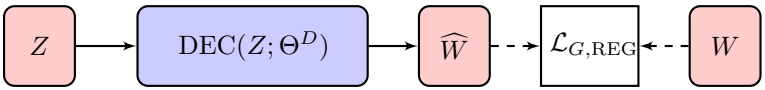


Figure 23.3: Shallow embedding methods. The encoder is a simple embedding look-up and the graph structure is only used in the loss function. Reprinted with permission from [Cha+20].

applications. The label output space $\mathcal{Y}$ is application dependent. For instance, in node-level classification, $\widehat{y}^N \in \mathcal{Y}^N$, with $\mathcal{Y}$ representing the node label space. Alternately, for edge-level labeling, $\widehat{y}^E \in \mathcal{Y}^{N\times N}$, with $\mathcal{Y}$ representing the edge label space. Finally, we note that other kinds of labeling are possible, such as graph-level labeling (where we would say $\widehat{y}^G \in \mathcal{Y}$, with $\mathcal{Y}$ representing the graph label space).

Finally, a loss must be specified. This can be used to optimize the parameters $\Theta = \{\Theta^E, \Theta^D, \Theta^S\}$. GraphEDM models can be optimized using a combination of three different terms. First, a supervised loss term, $\mathcal{L}^S_{\text{SUP}}$, compares the predicted labels $\hat{y}^S$ to the ground truth labels y^S. Next, a graph reconstruction loss term, $\mathcal{L}_{G,\text{RECON}}$, may leverage the graph structure to impose regularization constraints on the model parameters. Finally, a weight regularization loss term, $\mathcal{L}_{\text{REG}}$, allows representing priors on trainable model parameters for reducing overfitting. Models realizable by GraphEDM framework are trained by minimizing the total loss $\mathcal{L}$ defined as:

$$\mathcal{L} = \alpha(y^S, \hat{y}^S; \Theta) + \beta\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) + \gamma\mathcal{L}_{\text{REG}}(\Theta), \tag{23.4}$$

where α, β and γ are hyper-parameters, that can be tuned or set to zero. Note that graph embedding methods can be trained in a *supervised* ($\alpha \neq 0$) or *unsupervised* ($\alpha = 0$) fashion. Supervised graph embedding approaches leverage an additional source of information to learn embeddings such as node or graph labels. On the other hand, unsupervised network embedding approaches rely on the graph structure only to learn node embeddings.

23.3 Shallow graph embeddings

Shallow embedding methods are transductive graph embedding methods, where the encoder function maps categorical node IDs onto a Euclidean space through an embedding matrix. Each node $v_i \in V$ has a corresponding low-dimensional learnable embedding vector $\mathbf{Z}_i \in \mathbb{R}^L$ and the shallow encoder function is

$$\mathbf{Z} = \text{ENC}(\Theta^E) \triangleq \Theta^E \quad \text{where} \quad \Theta^E \in \mathbb{R}^{N\times L}. \tag{23.5}$$

Crucially, the embedding dictionary $\mathbf{Z}$ is directly learned as model parameters. In the unsupervised case, embeddings $\mathbf{Z}$ are optimized to recover some information about the input graph (e.g., the adjacency matrix $\mathbf{W}$, or a some transformation of it). This is somewhat similar to dimensionality reduction methods, such as PCA (Sec. 20.1), but for graph data structures. In the supervised case, the embeddings are optimized to predict some labels, for nodes, edges and/or the whole graph.

23.3.1 Unsupervised embeddings

In the unsupervised case, we will consider two main types of shallow graph embedding methods: distance-based and outer product-based. Distance-based methods optimize the embedding dictionary

$\mathbf{Z} = \Theta^E \in \mathbb{R}^{N \times L}$ such that nodes i and j which are close in the graph (as measured by some graph distance function) are embedded in $\mathbf{Z}$ such that $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ is small, where $d_2(.,.)$ is a pairwise distance function between embedding vectors. The distance function $d_2(\cdot,\cdot)$ can be customized, which can lead to Euclidean (Section 23.3.2) or non-Euclidean (Section 23.3.3) embeddings. The decoder outputs a node-to-node matrix $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$, with $\widehat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j)$.

Alternatively, some methods rely on pairwise dot-products to compute node similarities. The decoder network can be written as: $\widehat{W} = \text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\top$.

In both cases, unsupervised embeddings for distance- and product-based methods are learned by minimizing the graph regularization loss:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = d_1(s(\mathbf{W}), \widehat{\mathbf{W}}), \tag{23.6}$$

where $s(\mathbf{W})$ is an optional transformation of the adjacency matrix $\mathbf{W}$, and d_1 is pairwise distance function between matrices, which does not need to be of the same form as d_2. As we shall see, there are many plausible choices for s, d_1, d_2. For instance, we can s to be the adjacency matrix itself, $s(\mathbf{W}) = \mathbf{W}$ or a power of it e.g. $s(\mathbf{W}) = \mathbf{W}^2$. If the input is an weighted binary matrix $\mathbf{W} = \mathbf{A}$, we can set $s(\mathbf{W}) = 1 - \mathbf{W}$, so that connected nodes with $A_{ij} = 1$ get a weight (distance) of 0.

23.3.2 Distance-based: Euclidean methods

Distance-based method sminimize Euclidean distances between similar (connected) nodes. We give some examples below.

Multi-dimensional scaling (MDS, Sec. 20.4.4) is equivalent to setting $s(\mathbf{W})$ to some distance matrix measuring the dissimilarity between nodes (e.g. proportional to pairwise shortest distance) and then defining

$$d_1(s(W), \widehat{W}) = \sum_{i,j} (s(W)_{ij} - \widehat{W}_{ij})^2 = ||s(\mathbf{W}) - \widehat{\mathbf{W}}||_F^2 \tag{23.7}$$

where $\widehat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j) = ||\mathbf{Z}_i - \mathbf{Z}_j||$ (although other distance metrics are plausible).

Laplacian eigenmaps (Sec. 20.4.9) learn embeddingss by solving the generalized eigenvector problem

$$\min_{\mathbf{Z} \in \mathbb{R}^{|V| \times d}} \mathbf{Z}^\top \mathbf{L}\, \mathbf{Z} s.t.\ \mathbf{Z}^\top \mathbf{D}\mathbf{Z} = \mathbf{I} \ \text{ and } \ \mathbf{Z}^\top \mathbf{D}\mathbf{1} = 0 \tag{23.8}$$

where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is the graph Laplacian (Sec. 20.4.9.2), and $\mathbf{D}$ is a diagonal matrix containing the sum across columns for each row. The first constraint removes an arbitrary scaling factor in the embedding and the second one removes trivial solutions corresponding to the constant eigenvector (with eigenvalue zero for connected graphs). Further, note that $\mathbf{Z}^\top \mathbf{L}\, \mathbf{Z} = \frac{1}{2} \sum_{i,j} W_{ij} ||Z_i - Z_j||_2^2$ and therefore the minimization objective can be equivalently written as a graph reconstruction term, as follows:

$$d_1(s(\mathbf{W}), \widehat{\mathbf{W}}) = \sum_{i,j} \mathbf{W}_{ij} \times \widehat{\mathbf{W}}_{ij} \tag{23.9}$$

$$\widehat{\mathbf{W}}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j) = ||\mathbf{Z}_i - \mathbf{Z}_j||_2^2 \tag{23.10}$$

where $s(\mathbf{W}) = \mathbf{W}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

23.3.3 Distance-based: non-Euclidean methods

So far, we have discussed methods which assume that embeddings lie in an Euclidean Space. However, recent work has considered hyperbolic geometry for graph embedding. In particular, hyperbolic embeddings are ideal for embedding trees and offer an exciting alternative to Euclidean geometry for graphs that exhibit hierarchical structures. We give some examples below.

Nickel and Kiela [NK17] learn embeddings of hierarchical graphs using the **Poincaré model** of hyperbolic space. This is simple to represent in our notation as we only need to change $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ to the Poincaré distance function:

$$d_2(\mathbf{Z}_i, \mathbf{Z}_j) = d_{\text{Poincaré}}(\mathbf{Z}_i, \mathbf{Z}_j) = \text{arcosh}\left(1 + 2\frac{||\mathbf{Z}_i - \mathbf{Z}_j||_2^2}{(1 - ||\mathbf{Z}_i||_2^2)(1 - ||\mathbf{Z}_j||_2^2)}\right). \tag{23.11}$$

The optimization then learns embeddings which minimize distances between connected nodes while maximizing distances between disconnected nodes:

$$d_1(\mathbf{W}, \widehat{\mathbf{W}}) = \sum_{i,j} \mathbf{W}_{ij} \log \frac{e^{-\widehat{\mathbf{W}}_{ij}}}{\sum_{k|\mathbf{W}_{ik}=0} e^{-\widehat{\mathbf{W}}_{ik}}} \tag{23.12}$$

where the denominator is approximated using negative sampling. Note that since the hyperbolic space has a manifold structure, care needs to be taken to ensure that the embeddings remain on the manifold (using Riemannian optimization techniques [Bon13]).

Other variants of these methods have been proposed. Nickel and Kiela [NK18] explore the **Lorentz model** of hyperbolic space , and show that it provides better numerical stability than the Poincaré model. Another line of work extends non-Euclidean embeddings to mixed-curvature product spaces [Gu+18a], which provide more flexibility for other types of graphs (e.g. ring of trees). Finally, work by Chamberlain, Clough, and Deisenroth [CCD17] extends Poincaré embeddings using skip-gram losses with hyperbolic inner products.

23.3.4 Outer product-based: Matrix factorization methods

Matrix factorization approaches learn embeddings that lead to a low rank representation of some similarity matrix $s(\mathbf{W})$, with $s : \mathbb{R}^{N\times N} \to \mathbb{R}^{N\times N}$. The following are frequent choices: $s(\mathbf{W}) = \mathbf{W}$, $s(\mathbf{W}) = L$ (Graph Laplacian), or other proximity measure such as the Katz centrality index, Common Neighbors or Adamic/Adar index. The decoder function in matrix factorization methods is just a dot product:

$$\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\top \tag{23.13}$$

Matrix factorization methods learn $\mathbf{Z}$ by minimizing a regularization loss $\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = ||s(\mathbf{W}) - \widehat{\mathbf{W}}||_F^2$.

The **graph factorization** method of [Ahm+13] learns a low-rank factorization of a graph by minimizing the graph regularization loss $\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{(v_i,v_j)\in E}(\mathbf{W}_{ij} - \widehat{\mathbf{W}}_{ij})^2$.

Note that if $\mathbf{A}$ is the binary adjacency matrix, ($\mathbf{A}_{ij} = 1$ iff $(v_i, v_j) \in E$ and $\mathbf{A}_{ij} = 0$ otherwise), the graph regularization loss can be expressed in terms of the Frobenius norm:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = ||\mathbf{A} \odot (\mathbf{W} - \widehat{\mathbf{W}})||_F^2, \tag{23.14}$$

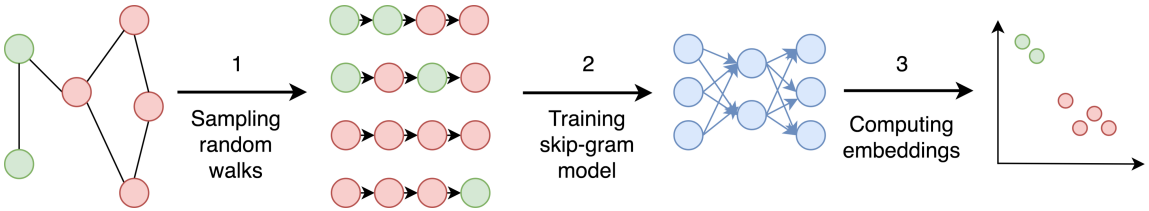


Figure 23.4: An overview of the pipeline for random-walk graph embedding methods. Reprinted with permission from [God18].

where $\odot$ is the element-wise matrix multiplication operator. Therefore, GF also learns a low-rank factorization of the adjacency matrix W measured in Frobenuis norm. We note that this is a sparse operation (summing only over edges which exist in the graph), and so the method has computational complexity $O(M)$.

The methods described so far are all symmetric, that is, they assume that $\mathbf{W}_{ij} = \mathbf{W}_{ji}$. This is a limiting assumption when working with directed graphs as some relationships are not reciprocal. The **GraRep** method of [CLX15] overcomes this limitation by learning two embeddings per node, a source embedding $\mathbf{Z}^s$ and a target embedding $\mathbf{Z}^t$, which capture asymmetric proximity in directed networks. In addition to asymmetry, GraRep learns embeddings that preserve k-hop neighborhoods via powers of the adjacency matrix and minimizes a graph reconstruction loss with:

$$\widehat{\mathbf{W}}^{(k)} = \mathbf{Z}^{(k),s}\mathbf{Z}^{(k),t\top} \tag{23.15}$$

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}^{(k)}; \Theta) = ||\mathbf{D}^{-k}\mathbf{W}^k - \widehat{\mathbf{W}}^{(k)}||_F^2, \tag{23.16}$$

for each $1 \leq k \leq K$. GraRep concatenates all representations to get source embeddings $\mathbf{Z}^s = [\mathbf{Z}^{(1),s}|\ldots|\mathbf{Z}^{(K),s}]$ and target embeddings $\mathbf{Z}^t = [\mathbf{Z}^{(1),t}|\ldots|\mathbf{Z}^{(K),t}]$. Unfortunately, GraRep is not very scalable, since it uses a matrix power, power, $\mathbf{D}^{-1}\mathbf{W}$, making it increasingly more dense. This limitation can be circumvented by using implicit matrix factorization [Per+17] as discussed below.

23.3.5 Outer product-based: Skip-gram methods

Skip-gram graph embedding models were inspired by research in natural language processing to model the distributional behavior of words [Mik+13c; PSM14b]. Skip-gram word embeddings are optimized to predict words in their context (the surrounding words) for each target word in a sentence. Given a sequence of words $(w_1, \ldots, w_T)$, skip-gram will minimize the objective:

$$\mathcal{L} = -\sum_{-K \leq i \leq K, i \neq 0} \log \mathbb{P}(w_{k-i}|w_k),$$

for each target words w_k. These conditional probabilities can be efficiently estimated using neural networks. See Sec. 20.5.2.2 for details.

This idea has been leveraged for graph embeddings in the **DeepWalk** framework of [PARS14]. They justified this by showing empirically how the frequency statistics induced by random walks in real graphs follow a distribution similar that of words used in natural language. In terms of

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

GraphEDM, skip-gram graph embedding methods use an outer product (Equation 23.13) as their decoder function and a graph reconstruction term computed over random walks on the graph.

In more detail, DeepWalk trains node embeddings to maximize the probability of predicting *context nodes* for each *center node*. The context nodes are nodes appearing adjacent to the center node, in simulated random walks on $\mathbf{A}$. To train embeddings, DeepWalk generates sequences of nodes using truncated unbiased random walks on the graph—which can be compared to sentences in natural language models—and then maximize their log-likelihood. Each random walk starts with a node $v_{i_1} \in V$ and repeatedly samples the next node uniformly at random: $v_{i_{j+1}} \in \{v \in V \mid (v_{i_j}, v) \in E\}$. The walk length is a hyperparameter. All generated random-walks can then be encoded by a sequence model. This two-step paradigm introduced by [PARS14] has been followed by many subsequent works, such as **node2vec** [GL16].

We note that it is common for underlying implementations to use two distinct representations for each node, one for when a node is center of a truncated random walk, and one when it is in the context. The implications of this modeling choice is studied further in [AEHPAR17].

To present DeepWalk in the GraphEDM framework, we can set:

$$s(\mathbf{W}) = \mathbb{E}_q \left[\left(\mathbf{D}^{-1}\mathbf{W} \right)^q \right] \text{ with } q \sim P(Q) = \text{Categorical}([1, 2, \ldots, T_{\max}]) \tag{23.17}$$

wwhere $P(Q = q) = \frac{T_{\max} - 1 + q}{T_{\max}}$ (see [AEH+18] for the derivation).

Training DeepWalk is equivalent to minimizing:

$$\mathcal{L}_{G,\text{RECON}}(W, \widehat{W}; \Theta) = \log Z(\mathbf{Z}) - \sum_{v_i \in V, v_j \in V} s(\mathbf{W})_{ij} \widehat{\mathbf{W}}_{ij}, \tag{23.18}$$

where $\widehat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^\top$, and the partition function is given by $Z(\mathbf{Z}) = \prod_i \sum_j \exp(\widehat{\mathbf{W}}_{ij})$ can be approximated in $O(N)$ time via hierarchical softmax (see Sec. 20.5.2). (It is also common to model $\widehat{\mathbf{W}} = \mathbf{Z}_{\text{out}}\mathbf{Z}_{\text{in}}{}^\top$ for directed graphs using embedding dictionaries $\mathbf{Z}_{\text{out}}, \mathbf{Z}_{\text{in}} \in \mathbb{R}^{N \times L}$.)

As noted by [LG14], Skip-gram methods can be viewed as implicit matrix factorization, and the methods discussed here are related to those of Matrix Factorization (see Sec. 23.3.4). This relationship is discussed in depth by [Qiu+18], who propose a general matrix factorization framework, **NetMF**, which uses the same underlying graph proximity information as DeepWalk, LINE [Tan+15], and node2vec [GL16]. Casting the node embedding problem as matrix factorization can inherit benefits of efficient sparse matrix operations [Qiu+19b].

23.3.6 Supervised embeddings

In many applications, we have labeled data in addition to node features and graph structure. While it is possible to tackle a supervised task by first learning unsupervised representations and then using them as features in a secondary model, this is not the ideal workflow. Unsupervised node embeddings might not preserve important properties of graphs (e.g., node neighborhoods or attributes), that are most useful for a downstream supervised task.

In light of this limitation, a number of methods combining these two steps, namely learning embeddings and predicting node or graph labels, have been proposed. Here, we focus on simple shallow methods. We discuss deep, nonlinear embeddings later on.

23.3.6.1 Label propagation

Label propagation (LP) [ZG02] is a very popular algorithm for graph-based semi-supervised node classification. The encoder is a shallow model represented by a lookup table $\mathbf{Z}$. LP uses the label space to represent the node embeddings directly (i.e. the decoder in LP is simply the identity function):

$$\hat{y}^N = \text{DEC}(\mathbf{Z}; \Theta^C) = \mathbf{Z}.$$

In particular, LP uses the graph structure to smooth the label distribution over the graph by adding a regularization term to the loss function, using the underlying assumption is that neighbor nodes should have similar labels (i.e. there exist some label consistency between connected nodes). Laplacian eigenmaps are utilized in the regularization to enforce this smoothness:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{i,j} \mathbf{W}_{ij} ||y_i^N - \hat{y}_j^N||_2^2 \tag{23.19}$$

LP minimizes this energy function over the space of functions that take fixed values on labelled nodes (i.e. $\hat{y}_i^N = y_i^N \; \forall i | v_i \in V_L$) using an iterative algorithm that updates an unlabelled node's label distribution via the weighted average of its neighbors' labels.

Label spreading (LS) [Zho+04] is a variant of label propagation which minimizes the following energy function:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{i,j} \mathbf{W}_{ij} \left|\left| \frac{\hat{y}_i^N}{\sqrt{D_i}} - \frac{\hat{y}_j^N}{\sqrt{D_j}} \right|\right|_2^2, \tag{23.20}$$

where $D_i = \sum_j W_{ij}$ is the degree of node v_i.

In both methods, the supervised loss is simply the sum of distances between predicted labels and ground truth labels (one-hot vectors):

$$\mathcal{L}_{\text{SUP}}^N(y^N, \hat{y}^N; \Theta) = \sum_{i | v_i \in V_L} ||y_i^N - \hat{y}_i^N||_2^2. \tag{23.21}$$

Note that while the regularization term is computed over all nodes in the graph, the supervised loss is computed over labelled nodes only. These methods are expected to work well with *consistent* graphs, that is graphs where node proximity in the graph is positively correlated with label similarity.

23.4 Graph Neural Networks

An extensive area of research focuses on defining convolutions over graph data. In the notation of Chami et al. [Cha+20], these (semi-)supervised neighborhood aggregation methods can be represented by an encoder of the form $\mathbf{Z} = \text{ENC}(\mathbf{X}, \mathbf{W}; \Theta^E)$, and decoders of the form $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$ and/or $\hat{y}^S = \text{DEC}(\mathbf{Z}; \Theta^S)$. There are many models in this family; we review some of them below.

23.4.1 Message passing GNNs

The original **graph neural network** (**GNN**) model of [GMS05; Sca+09] was the first formulation of deep learning methods for graph-structured data. It views the supervised graph embedding problem

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

as an information diffusion mechanism, where nodes send information to their neighbors until some stable equilibrium state is reached. More concretely, given randomly initialized node embeddings $\mathbf{Z}^0$, it applies the following recursion:

$$\mathbf{Z}^{t+1} = \text{ENC}(\mathbf{X}, \mathbf{W}, \mathbf{Z}^t; \Theta^E), \tag{23.22}$$

where parameters Θ^E are reused at every iteration. After convergence ($t = T$), the node embeddings $\mathbf{Z}^T$ are used to predict the final output such as node or graph labels:

$$\hat{y}^S = \text{DEC}(\mathbf{X}, \mathbf{Z}^T; \Theta^S). \tag{23.23}$$

This process is repeated several times and the GNN parameters Θ^E and Θ^D are learned with backpropagation via the Almeda-Pineda algorithm [Alm87; Pin88]. By Banach's fixed point theorem, this process is guaranteed to converge to a unique solution when the recursion provides a contraction mapping. In light of this, Scarselli et al. [Sca+09] explore maps that can be expressed using message passing networks:

$$\mathbf{Z}_i^{t+1} = \sum_{j|(v_i,v_j)\in E} f(\mathbf{X}_i, \mathbf{X}_j, \mathbf{Z}_j^t; \Theta^E), \tag{23.24}$$

where $f(\cdot)$ is a multi-layer perception (MLP) constrained to be a contraction mapping. The decoder function, however, has no constraints and can be any MLP.

Li et al. [Li+15] propose **Gated Graph Sequence Neural Networks** (GGSNNs), which remove the contraction mapping requirement from GNNs. In GGSNNs, the recursive algorithm in Equation 23.22 is relaxed by applying mapping functions for a fixed number of steps, where each mapping function is a gated recurrent unit [Cho+14a] with parameters shared for every iteration. The GGSNN model outputs predictions at every step, and so is particularly useful for tasks which have sequential structure (such as temporal graphs).

Gilmer et al. [Gil+17] provide a framework for graph neural networks called **message passing neural networks** (MPNNs), which encapsulates many recent models. In contrast with the GNN model which runs for an indefinite number of iterations, MPNNs provide an abstraction for modern approaches, which consist of multi-layer neural networks with a *fixed* number of layers. At every layer ℓ, message functions $f^\ell(.)$ receive messages from neighbors (based on neighbor's hidden state), which are then passed to aggregation functions $h^\ell(.)$:

$$\mathbf{m}_i^{\ell+1} = \sum_{j|(v_i,v_j)\in E} f^\ell(\mathbf{H}_i^\ell, \mathbf{H}_j^\ell) \tag{23.25}$$

$$\mathbf{H}_i^{\ell+1} = h^\ell(\mathbf{H}_i^\ell, \mathbf{m}_i^{\ell+1}), \tag{23.26}$$

where $\mathbf{H}^0 = \mathbf{X}$. After ℓ layers of message passing, nodes' hidden representations encode information within ℓ-hop neighborhoods.

Battaglia et al. [Bat+18] propose **GraphNet**, which further extends the MPNN framework to learn representations for edges, nodes and the entire graph using message passing functions. The explicit addition of edge and graph representations adds additional expressivity to the MPNN model, and allows the application of graph models to additional domains.

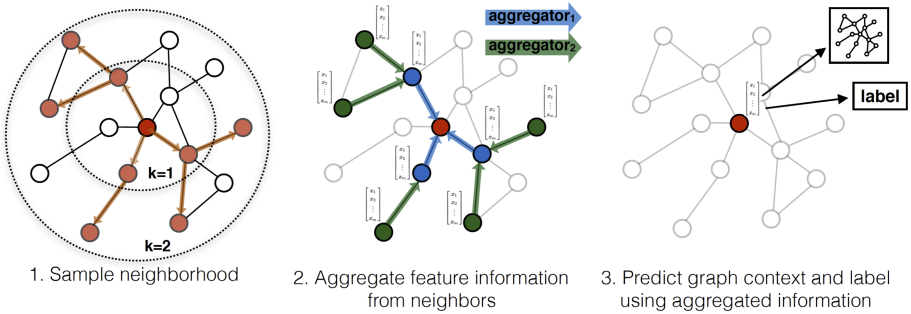


Figure 23.5: Illustration of the GraphSAGE model. Reprinted with permission from [HYL17].

23.4.2 Spectral Graph Convolutions

Spectral methods define graph convolutions using the spectral domain of the graph Laplacian matrix. These methods broadly fall into two categories: *spectrum-based methods*, which explicitly compute an eigendecomposition of the Laplacian (e.g., **spectral CNNs** [Bru+14]) and *spectrum-free* methods, which are motivated by spectral graph theory but do not actually perform a spectral decomposition (e.g., **Graph convolutional networks** or **GCN** [KW16a]).

A major disadvantage of spectrum-based methods is that they rely on the spectrum of the graph Laplacian and are therefore domain-dependent (i.e. cannot generalize to new graphs). Moreover, computing the Laplacian's spectral decomposition is computationally expensive. Spectrum-free methods overcome these limitations by utilizing approximations of these spectral filters. However, spectrum-free methods require using the whole graph $\mathbf{W}$, and so do not scale well.

For mroe details on spectral approaches, see e.g., [Bro+17b; Cha+20].

23.4.3 Spatial Graph Convolutions

Spectrum-based methods have an inherent domain dependency which limits the application of a model trained on one graph to a new dataset. Additionally, *spectrum-free* methods (e.g. GCNs) require using the entire graph $\mathbf{A}$, which can quickly become unfeasible as the size of the graph grows.

To overcome these limitations, another branch of graph convolutions (*spatial* methods) borrow ideas from standard CNNs – applying convolutions in the spatial domain as defined by the graph topology. For instance, in computer vision, convolutional filters are spatially localized by using fixed rectangular patches around each pixel. Combined with the natural ordering of pixels in images (top, left, bottom, right), it is possible to reuse filters' weights at every location. This process significantly reduces the total number of parameters needed for a model. While such spatial convolutions cannot directly be applied in graph domains, spatial graph convolutions take inspiration from them. The core idea is to use *neighborhood sampling* and *attention mechanisms* to create fixed-size graph patches, overcoming the irregularity of graphs.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

23.4.3.1 Sampling-based spatial methods

To overcome the domain dependency and storage limitations of GCNs, Hamilton, Ying, and Leskovec [HYL17] propose **GraphSAGE**, a framework to learn inductive node embeddings. Instead of averaging signals from all one-hop neighbors (via multiplications with the Laplacian matrix), SAGE samples fixed neighborhoods (of size q) for each node. This removes the strong dependency on fixed graph structure and allows generalization to new graphs. At every SAGE layer, nodes aggregate information from nodes sampled from their neighborhood (see Fig. 23.5). In the GraphEDM notation, the propagation rule can be written as:

$$\mathbf{H}_{:,i}^{\ell+1} = \sigma(\Theta_1^\ell \mathbf{H}_{:,i}^\ell + \Theta_2^\ell \text{AGG}(\{\mathbf{H}_{:,j}^\ell \mid v_j \in \text{Sample}(\text{nbr}(v_i), q)\})), \tag{23.27}$$

where $\text{AGG}(\cdot)$ is an aggregation function. This aggregation function can be any permutation invariant operator such as averaging (SAGE-mean) or max-pooling (SAGE-pool). As SAGE works with fixed size neighborhoods (and not the entire adjacency matrix), it also reduces the computational complexity of training GCNs.

23.4.3.2 Attention-based spatial methods

Attention mechanisms (Sec. 15.4) have been successfully used in language models where they, for example, allow models to identify relevant parts of long sequence inputs. Inspired by their success in language, similar ideas have been proposed for graph convolution networks. Such graph-based attention models learn to focus their attention on important neighbors during the message passing step via parametric patches which are learned on top of node features. This provides more flexibility in inductive settings, compared to methods that rely on fixed weights such as GCNs.

The **Graph attention network** (GAT) model of [Vel+18] is an attention-based version of GCNs. At every GAT layer, it attends over the neighborhood of each node and learns to selectively pick nodes which lead to the best performance for some downstream task. The intuition behind this is similar to SAGE [HYL17] and makes GAT suitable for inductive and transductive problems. However unlike SAGE, which limits the convolution step to fixed size-neighborhoods, GAT allows each node to attend over the entirety of its neighbors – assigning each of them different weights. The attention parameters are trained through backpropagation, and the attention scores are then row-normalized with a softmax activation.

23.4.3.3 Geometric spatial methods

Monti et al. [Mon+17] propose **MoNet**, a general framework that works particularly well when the node features lie in a geometric space, such as 3D point clouds or meshes. MoNet learns attention patches using parametric functions in a pre-defined spatial domain (e.g. spatial coordinates), and then applies convolution filters in the resulting graph domain.

MoNet generalizes spatial approaches which introduce constructions for convolutions on manifolds, such as the Geodesic CNN (GCNN) [Mas+15] and the Anisotropic CNN (ACNN) [Bos+16]. Both GCNN and ACNN use fixed patches that are defined on a specific coordinate system and therefore cannot generalize to graph-structured data. However, the MoNet framework is more general; any pseudo-coordinates (i.e. node features) can be used to induce the patches. More formally, if $\mathbf{U}^s$ are pseudo-coordinates and $\mathbf{H}^\ell$ are features from another domain, the MoNet layer can be expressed in

our notation as:

$$\mathbf{H}^{\ell+1} = \sigma\left(\sum_{k=1}^{K}(\mathbf{W} \odot g_k(\mathbf{U}^s))\mathbf{H}^{\ell}\Theta_k^{\ell}\right), \tag{23.28}$$

where $g_k(U^s)$ are the learned parametric patches, which are $N \times N$ matrices. In practice, MoNet uses Gaussian kernels to learn patches, such that:

$$g_k(\mathbf{U}^s) = \exp\left(-\frac{1}{2}(\mathbf{U}^s - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{U}^s - \boldsymbol{\mu}_k)\right), \tag{23.29}$$

where $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ are learned parameters, and $\boldsymbol{\Sigma}_k$ is restricted to be diagonal.

23.4.4 Non-Euclidean Graph Convolutions

As we discussed in Sec. 23.3.3, hyperbolic geometry enables learning of shallow embeddings of hierarchical graphs which have smaller distortion than Euclidean embeddings. However, one major downside of shallow embeddings is that they do not generalize well (if at all) across graphs. On the other hand, Graph Neural Networks, which leverage node features, have achieved good results on many inductive graph embedding tasks.

It is natural then, that there has been recent interest in extending Graph Neural Networks to learn non-Euclidean embeddings. One major challenge in doing so again revolves around the nature of convolution itself. How should we perform convolutions in a non-Euclidean space, where standard operations such as inner products and matrix multiplications are not defined?

Hyperbolic Graph Convolution Networks (HGCN) [Cha+19a] and Hyperbolic Graph Neural Networks (HGNN) [LNK19] apply graph convolutions in hyperbolic space by leveraging the Euclidean tangent space, which provides a first-order approximation of the hyperbolic manifold at a point. For every graph convolution step, node embeddings are mapped to the Euclidean tangent space at the origin, where convolutions are applied, and then mapped back to the hyperbolic space. These approaches yield significant improvements on graphs that exhibit hierarchical structure (Figure 23.6).

23.5 Deep graph embeddings

In this section, we use graph neural networks to devise graph embeddings in the unsupervised and semi-supervised cases.

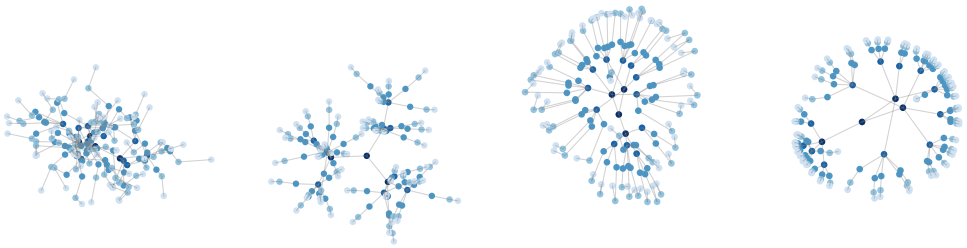
23.5.1 Unsupervised embeddings

In this section, we discuss unsupervised losses for GNNs, as illustrated in Fig. 23.7.

23.5.1.1 Structural deep network embedding

The **structural deep network embedding** (SDNE) method of [WCZ16] uses auto-encoders which preserve first and second-order node proximity. The SDNE encoder takes a row of the adjacency matrix as input (setting $s(\mathbf{W}) = \mathbf{W}$) and produces node embeddings $\mathbf{Z} = \text{ENC}(\mathbf{W}; \theta^E)$. (Note that this ignores any node features.) The SDNE decoder returns $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$, a reconstruction

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) GCN layers. (b) HGCN layers.

Figure 23.6: Euclidean (left) and hyperbolic (right) embeddings of a tree graph. Hyperbolic embeddings learn natural hierarchies in the embedding space (depth indicated by color). Reprinted with permission from [Cha+19a].

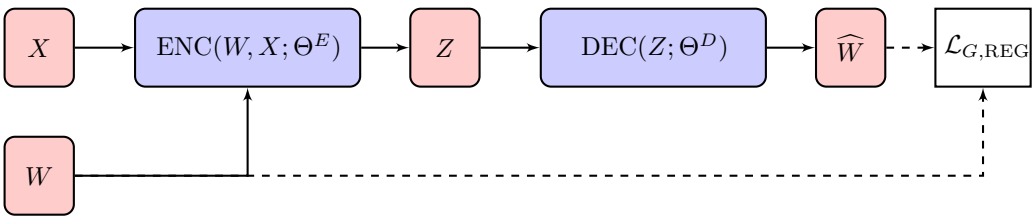


Figure 23.7: Unsupervised graph neural networks. Graph structure and input features are mapped to low-dimensional embeddings using a graph neural network encoder. Embeddings are then decoded to compute a graph regularization loss (unsupervised). Reprinted with permission from [Cha+20].

trained to recover the original graph adjacency matrix. SDNE preserves second order node proximity by minimizing the following loss:

$$||(s(\mathbf{W}) - \widehat{\mathbf{W}}) \cdot \mathbb{I}\left(s(\mathbf{W}) > 0\right)||_F^2 + \alpha_{\text{SDNE}} \sum_{ij} s(\mathbf{W})_{ij} ||\mathbf{Z}_i - \mathbf{Z}_j||_2^2 \tag{23.30}$$

The first term is similar to the matrix factorization regularization objective, except that $\widehat{\mathbf{W}}$ is not computed using outer products. The second term is the used by distance-based shallow embedding methods.

23.5.1.2 (Variational) graph auto-encoders

Kipf and Welling [KW16b] use graph convolutions (Sec. 23.4.2) to learn node embeddings $\mathbf{Z} = \text{GCN}(\mathbf{W}, \mathbf{X}; \Theta^E)$. The decoder is an outer product: $\text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\top$. The graph reconstruction term is the sigmoid cross entropy between the true adjacency and the predicted edge similarity scores:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = -\left(\sum_{i,j} (1 - \mathbf{W}_{ij}) \log(1 - \sigma(\widehat{\mathbf{W}}_{ij})) + \mathbf{W}_{ij} \log \sigma(\widehat{\mathbf{W}}_{ij}) \right). \tag{23.31}$$

Computing the regularization term over all possible nodes pairs is computationally challenging in practice, so the Graph Auto Encoders (GAE) model uses negative sampling to overcome this challenge.

Whereas GAE is a deterministic model, the authors also introduce variational graph auto-encoders (VGAE), which relies on variational auto-encoders (as in Sec. 20.3.5) to encode and decode the graph structure. In VGAE, the embedding $\mathbf{Z}$ is modelled as a latent variable with a standard multivariate normal prior $p(\mathbf{Z}) = \mathcal{N}(\mathbf{Z}|\mathbf{0}, \mathbf{I})$ and a graph convolution is used as the amortized inference network, $q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})$. The model is trained by minimizing the corresponding negative evidence lower bound:

$$\text{NELBO}(\mathbf{W}, \mathbf{X}; \Theta) = -\mathbb{E}_{q_\Phi(\mathbf{Z}|\mathbf{W},\mathbf{X})}[\log p(\mathbf{W}|\mathbf{Z})] + \text{KL}(q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})||p(\mathbf{Z})) \tag{23.32}$$

$$= \mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) + \text{KL}(q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})||p(\mathbf{Z})). \tag{23.33}$$

23.5.1.3 Iterative generative modelling of graphs (Graphite)

The **graphite** model of [GZE19] extends GAE and VGAE by introducing a more complex decoder. This decoder iterates between pairwise decoding functions and graph convolutions, as follows:

$$\widehat{\mathbf{W}}^{(k)} = \frac{\mathbf{Z}^{(k)}\mathbf{Z}^{(k)\top}}{||\mathbf{Z}^{(k)}||_2^2} + \frac{\mathbf{1}\mathbf{1}^\top}{N}$$

$$\mathbf{Z}^{(k+1)} = \text{GCN}(\widehat{\mathbf{W}}^{(k)}, \mathbf{Z}^{(k)})$$

where $\mathbf{Z}^{(0)}$ is initialized using the output of the encoder network. This process allows Graphite to learn more expressive decoders. Finally, similar to GAE, Graphite can be deterministic or variational.

23.5.1.4 Methods based on contrastive losses

The **deep graph infomax** method of [Vel+19] is a GAN-like method for creating graph-level embeddings. Given one or more *real* (positive) graphs, each with its adjacency matrix $\mathbf{W} \in \mathbb{R}^{N\times N}$ and node features $\mathbf{X} \in \mathbb{R}^{N\times D}$, this method creates *fake* (negative) adjacency matrices $\mathbf{W}^- \in \mathbb{R}^{N^-\times N^-}$ and their features $X^- \in \mathbb{R}^{N^-\times D}$. It trains (i) an encoder that processes both real and fake samples, respectively giving $Z = \text{ENC}(\mathbf{X}, \mathbf{W}; \Theta^E) \in \mathbb{R}^{N\times L}$ and $\mathbf{Z}^- = \text{ENC}(\mathbf{X}^-, \mathbf{W}^-; \Theta^E) \in \mathbb{R}^{N^-\times L}$, (ii) a (readout) graph pooling function $\mathcal{R} : \mathbb{R}^{N\times L} \to \mathbb{R}^L$, and (iii) a descriminator function $\mathcal{D} : \mathbb{R}^L \times \mathbb{R}^L \to [0, 1]$ which is trained to output $\mathcal{D}(\mathbf{Z}_i, \mathcal{R}(\mathbf{Z})) \approx 1$ and $\mathcal{D}(\mathbf{Z}_j^-, \mathcal{R}(\mathbf{Z}^-)) \approx 0$, respectively, for nodes corresponding to given graph $i \in V$ and fake graph $j \in V^-$. Specifically, DGI optimizes:

$$\min_\Theta - \mathop{\mathbb{E}}_{\mathbf{X},\mathbf{W}} \sum_{i=1}^{N} \log \mathcal{D}(\mathbf{Z}_i, \mathcal{R}(\mathbf{Z})) - \mathop{\mathbb{E}}_{\mathbf{X}^-,\mathbf{W}^-} \sum_{j=1}^{N^-} \log\left(1 - \mathcal{D}(\mathbf{Z}_j^-, \mathcal{R}(\mathbf{Z}^-))\right), \tag{23.34}$$

where Θ contains Θ^E and the parameters of $\mathcal{R}, \mathcal{D}$. In the first expectation, DGI samples from the real (positive) graphs. If only one graph is given, it could sample some subgraphs from it (e.g. connected components). The second expectation samples fake (negative) graphs. In DGI, fake samples use the real adjacency $W^- := W$ but fake features X^- are a row-wise random permutation of real X. The ENC used in DGI is a graph convolutional network, though any GNN can be used. The

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

readout $\mathcal{R}$ summarizes an entire (variable-size) graph to a single (fixed-dimension) vector. Veličković et al. [Vel+19] use $\mathcal{R}$ as a row-wise mean, though other graph pooling might be used e.g. ones aware of the adjacency.

The optimization of Eq. (23.34)) is shown by [Vel+19] to maximize a lower-bound on the mutual information between the outputs of the encoder and the graph pooling function, i.e., between individual node representations and the graph representation.

In [Pen+20] they present a variant called **Graphical Mutual Information** Rather than maximizing MI of node information and an entire graph, GMI maximizes the MI between the representation of a node and its neighbors.

23.5.2 Semi-supervised embeddings

In this section, we discuss semi-supervised losses for GNNs. We consider the simple special case in which we use a nonlinear encoder of the node features, but ignore the graph structure, i.e., we use $\mathbf{Z} = \text{ENC}(\mathbf{X}; \Theta^E)$.

23.5.2.1 SemiEmb

[WRC08] propose an approach called **semi-supervised embeddings** (SemiEmb) They use an MLP for the encoder of $\mathbf{X}$. For the decoder, we can use a distance-based graph decoder: $\widehat{\mathbf{W}}_{ij} = \text{DEC}(\mathbf{Z}; \Theta^D)_{ij} = ||\mathbf{Z}_i - \mathbf{Z}_j||^2$, where $||\cdot||$ can be the L2 or L1 norm.

SemiEmb regularizes intermediate or auxiliary layers in the network using the same regularizer as the label propagation loss in Eq. (23.19). SemiEmb uses a feed forward network to predict labels from intermediate embeddings, which are then compared to ground truth labels using the Hinge loss.

23.5.2.2 Planetoid

Unsupervised skip-gram methods like DeepWalk and node2vec learn embeddings in a multi-step pipeline, where random walks are first generated from the graph and then used to learn embeddings. These embeddings are likely not optimal for downstream classification tasks. The **Planetoid** method of [YCS16] extends such random walk methods to leverage node label information during the embedding algorithm.

Planetoid first maps nodes to embeddings $\mathbf{Z} = [\mathbf{Z}^c || \mathbf{Z}^F] = \text{ENC}(\mathbf{X}; \Theta^E)$ using a neural network (again ignoring graph structure). The node embeddings $\mathbf{Z}^c$ capture structural information while the node embeddings $\mathbf{Z}^F$ capture feature information. There are two variants, a transductive version that directly learns $\mathbf{Z}^c$ (as an embedding lookup), and an inductive model where $\mathbf{Z}^c$ is computed with parametric mappings that act on input features $\mathbf{X}$. The Planetoid objective contains both a supervised loss and a graph regularization loss. The graph regularization loss measures the ability to predict context using nodes embeddings:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = -\mathbb{E}_{(i,j,\gamma)} \log \sigma\left(\gamma \widehat{\mathbf{W}}_{ij}\right), \tag{23.35}$$

with $\widehat{\mathbf{W}}_{ij} = \mathbf{Z}_i^\top \mathbf{Z}_j$ and $\gamma \in \{-1, 1\}$ with $\gamma = 1$ if $(v_i, v_j) \in E$ is a positive pair and $\gamma = -1$ if (v_i, v_j) is a negative pair. The distribution under the expectation is directly defined through a sampling process

The supervised loss in Planetoid is the negative log-likelihood of predicting the correct labels:

$$\mathcal{L}^N_{\text{SUP}}(y^N, \widehat{y}^N; \Theta) = -\frac{1}{|V_L|} \sum_{i|v_i \in V_L} \sum_{1 \le k \le C} y^N_{ik} \log \widehat{y}^N_{ik}, \tag{23.36}$$

where i is a node's index while k indicates label classes, and $\widehat{y}^N_i$ are computed using a neural network followed by a softmax activation, mapping $\mathbf{Z}_i$ to predicted labels.

23.6 Applications

There are many applications of graph embeddings, both unsupervised and supervised. We give some examples in the sections below.

23.6.1 Unsupervised applications

In this section, we discuss common unsupervised applications.

23.6.1.1 Graph reconstruction

A popular unsupervised graph application is graph reconstruction. In this setting, the goal is to learn mapping functions (which can be parametric or not) that map nodes onto a manifold which can reconstruct the graph. This is regarded as *unsupervised* in the sense that there is no supervision beyond the graph structure. Models can be trained by minimizing a reconstruction error, which is the error in recovering the original graph from learned embeddings. Several algorithms were designed specifically for this task, and we refer to Sec. 23.3.1 and Sec. 23.5.1 for some examples of reconstruction objectives. At a high level, graph reconstruction is similar to dimensionality reduction in the sense that the main goal is summarize some input data into a low-dimensional embedding. Instead of compressing high dimensional vectors into low-dimensional ones as standard dimensionality reduction methods (e.g. PCA) do, the goal of graph reconstruction models is to compress data defined on graphs into low-dimensional vectors.

23.6.1.2 Link prediction

The goal in **link prediction** is to predict missing or unobserved links (e.g., links that may appear in the future for dynamic and temporal networks). Link prediction can also help identifying spurious link and remove them. It is a major application of graph learning models in industry, and common example of applications include predicting friendships in **social networks** predicting user-product interactions in **recommendation systems**, predicting suspicious links in a **fraud detection system** (see Fig. 23.8), or predicting missing relationships between entities in a **knowledge graph** (see e.g., [Nic+15]).

A common approach for training link prediction models is to mask some edges in the graph (positive and negative edges), train a model with the remaining edges and then test it on the masked set of edges. Note that link prediction is different from graph reconstruction. In link prediction, we aim at predicting links that are not observed in the original graph while in graph reconstruction, we only want to compute embeddings that preserve the graph structure through reconstruction error minimization.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

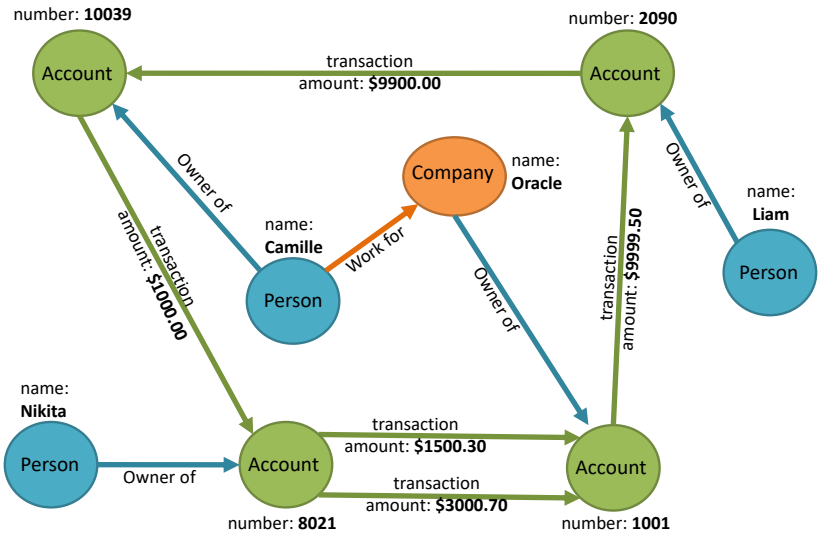


Figure 23.8: A graph representation of some financial transactions. Adapted from http://pgql-lang.org/spec/1.2/.

Finally, while link prediction has similarities with supervised tasks in the sense that we have labels for edges (positive, negative, unobserved), we group it under the unsupervised class of applications since edge labels are usually not used during training, but only used to measure the predictive quality of embeddings.

23.6.1.3 Clustering

Clustering is particularly useful for discovering communities and has many real-world applications. For instance, clusters exist in biological networks (e.g. as groups of proteins with similar properties), or in social networks (e.g. as groups of people with similar interests).

The unsupervised methods introduced in this chapter can be used to solve clustering problems by applying the clustering algorithm (e.g. k-means) to embeddings that are output by an encoder. Further, clustering can be joined with the learning algorithm while learning a shallow [Roz+19] or Graph Convolution [Chi+19a; CEL19] embedding model.

23.6.1.4 Visualization

There are many off-the-shelf tools for mapping graph nodes onto two-dimensional manifolds for the purpose of visualization. Visualizations allow network scientists to qualitatively understand graph properties, understand relationships between nodes or visualize node clusters. Among the popular tools are methods based on *Force-Directed Layouts*, with various web-app Javascript implementations.

Unsupervised graph embedding methods are also used for visualization purposes: by first training an encoder-decoder model (corresponding to a shallow embedding or graph convolution network),

and then mapping every node representation onto a two-dimensional space using t-SNE (Sec. 20.4.10) or PCA (Sec. 20.1). Such a process (embedding → dimensionality reduction) is commonly used to qualitatively evaluate the performance of graph learning algorithms. If nodes have attributes, one can use these attributes to color the nodes on 2D visualization plots. Good embedding algorithms embed nodes that have similar attributes nearby in the embedding space, as demonstrated in visualizations of various methods [PARS14; KW16a; AEH+18]. Finally, beyond mapping every node to a 2D coordinate, methods which map every graph to a representation [ARZP19] can similarly be projected into two dimensions to visualize and qualitatively analyze graph-level properties.

23.6.2 Supervised applications

In this section, we discuss common supervised applications.

23.6.2.1 Node classification

Node classification is an important supervised graph application, where the goal is to learn node representations that can accurately predict node labels. (This is sometimes called **statistical relational learning** [GT07].) For instance, node labels could be scientific topics in citation networks, or gender and other attributes in social networks.

Since labelling large graphs can be time-consuming and expensive, semi-supervised node classification is a particularly common application. In semi-supervised settings, only a fraction of nodes are labelled and the goal is to leverage links between nodes to predict attributes of unlabelled nodes. This setting is transductive since there is only one partially labelled fixed graph. It is also possible to do inductive node classification, which corresponds to the task of classifying nodes in multiple graphs.

Note that node features can significantly boost the performance on node classification tasks if these are descriptive for the target label. Indeed, recent methods such as GCN (Sec. 23.4.2) GraphSAGE (Sec. 23.4.3.1) have achieved state-of-the-art performance on multiple node classification benchmarks due to their ability to combine structural information and semantics coming from features. On the other hand, other methods such as random walks on graphs fail to leverage feature information and therefore achieve lower performance on these tasks.

23.6.2.2 Graph classification

Graph classification is a supervised application where the goal is to predict graph labels. Graph classification problems are inductive and a common example is classifying chemical compounds (e.g. predicting toxicity or odor from a molecule, as shown in Fig. 23.9).

Graph classification requires some notion of pooling, in order to aggregate node-level information into graph-level information. As discussed earlier, generalizing this notion of pooling to arbitrary graphs is non trivial because of the lack of regularity in the graph structure making graph pooling an active research area. In addition to the supervised methods discussed above, a number of unsupervised methods for learning graph-level representations have been proposed [Tsi+18; ARZP19; TMP20].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

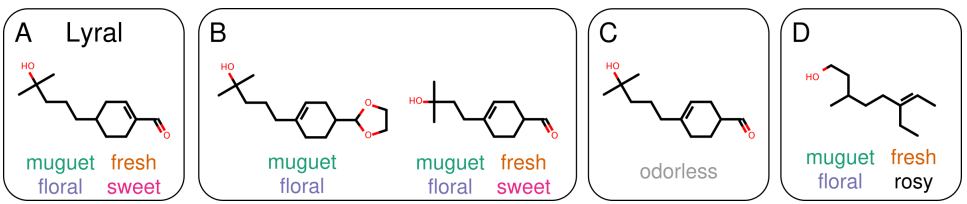


Figure 23.9: Structurally similar molecules do not necessarily have similar odor descriptors. (A) Lyral, the reference molecule. (B) Molecules with similar structure can share similar odor descriptors. (C) However, a small structural change can render the molecule odorless. (D) Further, large structural changes can leave the odor of the molecule largely unchanged. From Figure 1 of [SL+19], originally from [OPK12]. Used with kind permission of Benjamin Sanchez-Lengeling.

Appendices

Part VI

Appendix

A Notation

A.1 Introduction

It is very difficult to come up with a single, consistent notation to cover the wide variety of data, models and algorithms that we discuss in this book. Furthermore, conventions differ between different fields (such as machine learning, statistics and optimization), and between different books and papers within the same field. Nevertheless, we have tried to be as consistent as possible. Below we summarize most of the notation used in this book, although individual sections may introduce new notation. Note also that the same symbol may have different meanings depending on the context, although we try to avoid this where possible.

A.2 Common mathematical symbols

We list some common symbols below.

Symbol	Meaning
∞	Infinity
$\rightarrow$	Tends towards, e.g., $n \rightarrow \infty$
$\propto$	Proportional to, so $y = ax$ can be written as $y \propto x$
$\triangleq$	Defined as
$O(\cdot)$	Big-O: roughly means order of magnitude
$\mathbb{Z}_+$	The positive integers
$\mathbb{R}$	The real numbers
$\mathbb{R}_+$	The positive reals
$\mathcal{S}_K$	The K-dimensional probability simplex
$\mathcal{S}_{++}^D$	Cone of positive definite $D \times D$ matrices
$\approx$	Approximately equal to
$\{1, \ldots, N\}$	The finite set $\{1, 2, \ldots, N\}$
$1 : N$	The finite set $\{1, 2, \ldots, N\}$
$[\ell, u]$	The continuous interval $\{\ell \leq x \leq u\}$.

A.3 Functions

Generic functions will be denoted by f (and sometimes g or h). We will encounter many named functions, such as $\tanh(x)$ or $\sigma(x)$. A scalar function applied to a vector is assumed to be applied elementwise, e.g., $\mathbf{x}^2 = [x_1^2, \ldots, x_D^2]$. Functionals (functions of a function) are written using "blackboard" font, e.g., $\mathbb{H}(p)$ for the entropy of a distribution p. We list some common functions below.

A.3.1 Common functions of one argument

Symbol	Meaning
$\lfloor x \rfloor$	Floor of x, i.e., round down to nearest integer
$\lceil x \rceil$	Ceiling of x, i.e., round up to nearest integer
$\neg a$	logical NOT
$\mathbb{I}(x)$	Indicator function, $\mathbb{I}(x) = 1$ if x is true, else $\mathbb{I}(x) = 0$
$\delta(x)$	Dirac delta function, $\delta(x) = \infty$ if $x = 0$, else $\delta(x) = 0$
$\lvert x \rvert$	Absolute value
$\lvert \mathcal{S} \rvert$	Size (cardinality) of a set
$n!$	Factorial function
$\log(x)$	Natural logarithm of x
$\exp(x)$	Exponential function e^x
$\Gamma(x)$	Gamma function, $\Gamma(x) = \int_0^\infty u^{x-1} e^{-u} du$
$\Psi(x)$	Digamma function, $\Psi(x) = \frac{d}{dx} \log \Gamma(x)$
$\sigma(x)$	Sigmoid (logistic) function, $\frac{1}{1+e^{-x}}$

A.3.2 Common functions of two arguments

Symbol	Meaning
$a \wedge b$	logical AND
$a \vee b$	logical OR
$B(a, b)$	Beta function, $B(a, b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$
$\binom{n}{k}$	n choose k, equal to $n!/(k!(n-k)!)$
δ_{ij}	Kronecker delta, equals $\mathbb{I}(i = j)$
$\mathbf{u} \odot \mathbf{v}$	Elementwise product of two vectors
$\mathbf{u} \circledast \mathbf{v}$	Convolution of two vectors

A.3.3 Common functions of > 2 arguments

Symbol	Meaning
$B(\mathbf{x})$	Multivariate beta function, $\frac{\prod_k \Gamma(x_k)}{\Gamma(\sum_k x_k)}$
$\Gamma(\mathbf{x})$	Multi. gamma function, $\pi^{D(D-1)/4} \prod_{d=1}^{D} \Gamma\left(x + (1-d)/2\right)$
$\mathcal{S}(\mathbf{x})$	Softmax function, $[\frac{e^{a_c}}{\sum_{c'=1}^{C} e^{a_{c'}}}]_{c=1}^{C}$

A.4 Linear algebra

In this section, we summarize the notation we use for linear algebra (see Chapter 7 for details).

A.4.1 General notation

Vectors are bold lower case letters such as $\mathbf{x}$, $\mathbf{w}$. Matrices are bold upper case letters, such as $\mathbf{X}$, $\mathbf{W}$. Scalars are non-bold lower case. When creating a vector from a list of N scalars, we write $\mathbf{x} = [x_1, \ldots, x_N]$; this may be a column vector or a row vector, depending on the context. (Vectors are assumed to be column vectors, unless noted otherwise.) When creating an $M \times N$ matrix from a list of vectors, we write $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_N]$ if we stack along the columns, or $\mathbf{X} = [\mathbf{x}_1; \ldots; \mathbf{x}_M]$ if we stack along the rows.

A.4.2 Vectors

Here is some standard notation for vectors. (We assume $\mathbf{u}$ and $\mathbf{v}$ are both N-dimensional vectors.)

Symbol	Meaning
$\mathbf{u}^\top \mathbf{v}$	Inner (scalar) product, $\sum_{i=1}^N u_i v_i$
$\mathbf{u}\mathbf{v}^\top$	Outer product ($N \times N$ matrix)
$\mathbf{u} \odot \mathbf{v}$	Elementwise product, $[u_1 v_1, \ldots, u_N v_N]$
$\mathbf{v}^\top$	Transpose of $\mathbf{v}$
$\dim(\mathbf{v})$	Dimensionality of $\mathbf{v}$ (namely N)
$\mathrm{diag}(\mathbf{v})$	Diagonal $N \times N$ matrix made from vector $\mathbf{v}$
$\mathbf{1}$ or $\mathbf{1}_N$	Vector of ones (of length N)
$\mathbf{0}$ or $\mathbf{0}_N$	Vector of zeros (of length N)
$\|\|\mathbf{v}\|\| = \|\|\mathbf{v}\|\|_2$	Euclidean or ℓ_2 norm $\sqrt{\sum_{i=1}^N v_i^2}$
$\|\|\mathbf{v}\|\|_1$	ℓ_1 norm $\sum_{i=1}^N \|v_i\|$

A.4.3 Matrices

Here is some standard notation for matrices. (We assume $\mathbf{S}$ is a square $N \times N$ matrix, $\mathbf{X}$ and $\mathbf{Y}$ are of size $M \times N$, and $\mathbf{Z}$ is of size $M' \times N'$.)

Symbol	Meaning
$\mathbf{X}_{:,j}$	j'th column of matrix
$\mathbf{X}_{i,:}$	i'th row of matrix (treated as a column vector)
X_{ij}	Element (i, j) of matrix
$\mathbf{S} \succ 0$	True iff $\mathbf{S}$ is a positive definite matrix
$\mathrm{tr}(\mathbf{S})$	Trace of a square matrix
$\det(\mathbf{S})$	Determinant of a square matrix
$\|\mathbf{S}\|$	Determinant of a square matrix
$\mathbf{S}^{-1}$	Inverse of a square matrix
$\mathbf{X}^\dagger$	Pseudo-inverse of a matrix
$\mathbf{X}^\top$	Transpose of a matrix

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\text{diag}(\mathbf{S})$	Diagonal vector extracted from square matrix
$\mathbf{I}$ or $\mathbf{I}_N$	Identity matrix of size $N \times N$
$\mathbf{X} \odot \mathbf{Y}$	Elementwise product
$\mathbf{X} \otimes \mathbf{Z}$	Kronecker product (see Sec. 7.2.5)

A.4.4 Matrix calculus

In this section, we summarize the notation we use for matrix calculus (see Sec. 7.8 for details).

Let $\boldsymbol{\theta} \in \mathbb{R}^N$ be a vector and $f : \mathbb{R}^N \to \mathbb{R}$ be a scalar valued function. The derivative of f wrt its argument is denoted by the following:

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \triangleq \nabla f(\boldsymbol{\theta}) \triangleq \nabla f \triangleq \begin{pmatrix} \frac{\partial f}{\partial \theta_1} & \cdots & \frac{\partial f}{\partial \theta_N} \end{pmatrix} \tag{A.1}$$

The gradient is a vector that must be evaluated at a point in space. To emphasize this, we will sometimes write

$$\mathbf{g}_t \triangleq \mathbf{g}(\boldsymbol{\theta}_t) \triangleq \nabla f(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}_t} \tag{A.2}$$

We can also compute the (symmetric) $N \times N$ matrix of second partial derivatives, known as the **Hessian**:

$$\nabla^2 f \triangleq \begin{pmatrix} \frac{\partial^2 f}{\partial \theta_1^2} & \cdots & \frac{\partial^2 f}{\partial \theta_1 \partial \theta_N} \\ & \vdots & \\ \frac{\partial^2 f}{\partial \theta_N \theta_1} & \cdots & \frac{\partial^2 f}{\partial \theta_N^2} \end{pmatrix} \tag{A.3}$$

The Hessian is a matrix that must be evaluated at a point in space. To emphasize this, we will sometimes write

$$\mathbf{H}_t \triangleq \mathbf{H}(\boldsymbol{\theta}_t) \triangleq \nabla^2 f(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}_t} \tag{A.4}$$

A.5 Optimization

In this section, we summarize the notation we use for optimization (see Chapter 8 for details).

We will often write an objective or cost function that we wish to minimize as $\mathcal{L}(\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the variables to be optimized (often thought of as parameters of a statistical model). We denote the parameter value that achieves the minimum as $\boldsymbol{\theta}_* = \text{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta})$, where Θ is the set we are optimizing over. (Note that there may be more than one such optimal value, so we should really write $\boldsymbol{\theta}_* \in \text{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta})$.)

When performing iterative optimization, we use t to index the iteration number. We use η as a step size (learning rate) parameter. Thus we can write the gradient descent algorithm (explained in Sec. 8.4) as follows: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t$.

We often use a hat symbol to denote an estimate or prediction (e.g., $\hat{\boldsymbol{\theta}}$, $\hat{y}$), a star subscript or superscript to denote a true (but usually unknown) value (e.g., $\boldsymbol{\theta}_*$ or $\boldsymbol{\theta}^*$), an overline to denote a mean value (e.g., $\overline{\boldsymbol{\theta}}$).

A.6 Probability

In this section, we summarize the notation we use for probability theory (see Chapter 2 for details).

We denote a probability density function (pdf) or probability mass function (pmf) by p, a cumulative distribution function (cdf) by P, and the probability of a binary event by Pr. We write $p(X)$ for the distribution for random variable X, and $p(Y)$ for the distribution for random variable Y — these refer to different distributions, even though we use the same p symbol in both cases. (In cases where confusion may arise, we write $p_X(\cdot)$ and $p_Y(\cdot)$.) Approximations to a distribution p will often be represented by q, or sometimes $\hat{p}$.

In some cases, we distinguish between a random variable (rv) and the values it can take on. In this case, we denote the (scalar) rv in upper case (e.g., X), and its value in lower case (e.g., x). However, we often ignore this distinction between variables and values. For example, we sometimes write $p(x)$ to denote either the scalar value (the distribution evaluated at a point) or the distribution itself, depending on whether X is observed or not.

We write $X \sim p$ to denote that X is distributed according to distribution p. We write $X \perp Y \mid Z$ to denote that X is conditionally independent of Y given Z. If $X \sim p$, we denote the expected value of $f(X)$ using

$$\mathbb{E}\left[f(X)\right] = \mathbb{E}_{p(X)}\left[f(X)\right] = \mathbb{E}_X\left[f(X)\right] = \int_x f(x)p(x)dx \tag{A.5}$$

If f is the identity function, we write $\overline{X} \triangleq \mathbb{E}\left[X\right]$. Similarly, the variance is denoted by

$$\mathbb{V}\left[f(X)\right] = \mathbb{V}_{p(X)}\left[f(X)\right] = \mathbb{V}_X\left[f(X)\right] = \int_x (f(x) - \mathbb{E}\left[f(X)\right])^2 p(x)dx \tag{A.6}$$

If $\mathbf{x}$ is a random vector, the covariance matrix is denoted

$$\mathrm{Cov}\left[\mathbf{x}\right] = \mathbb{E}\left[(\mathbf{x} - \overline{\mathbf{x}})(\mathbf{x} - \overline{\mathbf{x}})^\top\right] \tag{A.7}$$

If $X \sim p$, the mode of a distribution is denoted by

$$\hat{x} = \mathrm{mode}\left[p\right] = \operatorname*{argmax}_x p(x) \tag{A.8}$$

We denote parametric distributions using $p(\mathbf{x}|\boldsymbol{\theta})$, where $\mathbf{x}$ are the random variables, $\boldsymbol{\theta}$ are the parameters and p is a pdf or pmf. For example, $\mathcal{N}(x|\mu, \sigma^2)$ is a Gaussian (normal) distribution with mean μ and standard deviation σ.

A.7 Information theory

In this section, we summarize the notation we use for information theory (see Chapter 6 for details).

If $X \sim p$, we denote the (differential) entropy of the distribution by $\mathbb{H}\left(X\right)$ or $\mathbb{H}\left(p\right)$. If $Y \sim q$, we denote the KL divergence from distribution p to q by $\mathbb{KL}\left(p \| q\right)$. If $(X, Y) \sim p$, we denote the mutual information between X and Y by $\mathbb{I}\left(X; Y\right)$.

A.8 Statistics and machine learning

We briefly summarize the notation we use for statistical learning.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

A.8.1 Supervised learning

For supervised learning, we denote the observed features (also called inputs or **covariates**) by $\mathbf{x} \in \mathcal{X}$. Often $\mathcal{X} = \mathbb{R}^D$, meaning the features are real-valued. (Note that this includes the case of discrete-valued inputs, which can be represented as one-hot vectors.) Sometimes we compute manually-specified features of the input; we denote these by $\boldsymbol{\phi}(\mathbf{x})$. We also have outputs (also called **targets** or **response variables**) $\mathbf{y} \in \mathcal{Y}$ that we wish to predict. Our task is to learn a conditional probability distribution $p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the parameters of the model. If $\mathcal{Y} = \{1, \ldots, C\}$, we call this **classification**. If $\mathcal{Y} = \mathbb{R}^C$, we call this **regression** (often $C = 1$, so we are just predicting a scalar response).

The parameters $\boldsymbol{\theta}$ are estimated from **training data**, denoted by $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n) : n \in \{1, \ldots, N\}\}$ (so N is the number of training cases). If $\mathcal{X} = \mathbb{R}^D$, we can store the training inputs in an $N \times D$ **design matrix** denoted by $\mathbf{X}$. If $\mathcal{Y} = \mathbb{R}^C$, we can store the training outputs in an $N \times C$ matrix $\mathbf{Y}$. If $\mathcal{Y} = \{1, \ldots, C\}$, we can represent each class label as a C-dimensional bit vector, with one element turned on (this is known as a **one-hot encoding**), so we can store the training outputs in an $N \times C$ binary matrix $\mathbf{Y}$.

A.8.2 Unsupervised learning and generative models

Unsupervised learning is usually formalized as the task of unconditional density estimation, namely modeling $p(\mathbf{x}|\boldsymbol{\theta})$. In some cases, we want to perform conditional density estimation; we denote the values we are conditioning on by $\mathbf{c}$, so the model becomes $p(\mathbf{x}|\mathbf{c}, \boldsymbol{\theta})$. This is similar to supervised learning, except that $\mathbf{x}$ is usually high dimensional (e.g., an image) and $\mathbf{c}$ is usually low dimensional (e.g., a class label or a text description).

In some models, we have **latent variables**, also called **hidden variables**, which are never observed in the training data. We call such models **latent variable models** (LVM). We denote the latent variables for data case n by $\mathbf{z}_n \in \mathcal{Z}$. Sometimes latent variables are known as **hidden variables**, and are denoted by $\mathbf{h}_n$. By contrast, the **visible variables** will be denoted by $\mathbf{v}_n$. Typically the latent variables are continuous or discrete, i.e., $\mathcal{Z} = \mathbb{R}^L$ or $\mathcal{Z} = \{1, \ldots, K\}$.

Most LVMs have the form $p(\mathbf{x}_n, \mathbf{z}_n|\boldsymbol{\theta})$; such models can be used for unsupervised learning. However, LVMs can also be used for supervised learning. In particular, we can either create a generative (unconditional) model of the form $p(\mathbf{x}_n, \mathbf{y}_n, \mathbf{z}_n|\boldsymbol{\theta})$, or a discriminative (conditional) model of the form $p(\mathbf{y}_n, \mathbf{z}_n|\mathbf{x}_n, \boldsymbol{\theta})$.

A.8.3 Bayesian inference

When working with Bayesian inference, we write the prior over the parameters as $p(\boldsymbol{\theta}|\boldsymbol{\phi})$, where $\boldsymbol{\phi}$ are the hyperparameters. For conjugate models, the posterior has the same form as the prior (by definition). We can therefore just update the hyperparameters from their prior value, $\breve{\boldsymbol{\phi}}$, to their posterior value, $\widehat{\boldsymbol{\phi}}$.

In variational inference (Sec. 4.6.8.3), we use $\boldsymbol{\xi}$ to represent the parameters of the variational posterior, i.e., $p(\boldsymbol{\theta}|\mathcal{D}) \approx q(\boldsymbol{\theta}|\boldsymbol{\xi})$. We optimize the ELBO wrt $\boldsymbol{\xi}$ to make this a good approximation.

When performing Monte Carlo sampling, we use a s subscript or superscript to denote a sample (e.g., $\boldsymbol{\theta}_s$ or $\boldsymbol{\theta}^s$).

A.9 Abbreviations

Here are some of the abbreviations used in the book.

Abbreviation	Meaning
cdf	Cumulative distribution function
CNN	Convolutional neural network
DAG	Directed acyclic graph
DML	Deep metric learning
DNN	Deep neural network
dof	Degrees of freedom
EB	Empirical Bayes
EM	Expectation maximization algorithm
GLM	Generalized linear model
GMM	Gaussian mixture model
HMC	Hamiltonian Monte Carlo
HMM	Hidden Markov model
iid	Independent and identically distributed
iff	If and only if
KDE	Kernel density estimation
KL	Kullback Leibler divergence
KNN	K nearest neighbor
LHS	Left hand side (of an equation)
LSTM	Long short term memory (a kind of RNN)
LVM	Latent variable model
MAP	Maximum A Posterior estimate
MCMC	Markov chain Monte Carlo
MLE	Maximum likelihood estimate
MLP	Multilayer perceptron
MSE	Mean squared error
NLL	Negative log likelihood
OLS	Ordinary least squares
psd	Positive definite (matrix)
pdf	Probability density function
pmf	Probability mass function
PNLL	Penalized NLL
PGM	Probabilistic graphical model
RNN	Recurrent neural network
RHS	Right hand side (of an equation)
RSS	Residual sum of squares
rv	Random variable
RVM	Relevance vector machine
SGD	Stochastic gradient descent
SSE	Sum of squared errors
SVI	Stochastic variational inference

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

SVM	Support vector machine
VB	Variational Bayes
wrt	With respect to

Index

A^* search, **536**
à trous algorithm, **501**
AMSGrad, 307
AdaDelta, **306**
AdaGrad, **305**
Adam, **307**
Padam, 307
RMSProp, **306**
RPROP, **306**
Yogi, 307
1x1 convolution, **491**

A/B test, **171**
abstractive summarization, **561**
action, **176**
action potential, **450**
actions, **159**
activation function, **442**
activation maximization, **509**
active, **311**
active learning, 413, **668**
active set, **409**
activity regularization, **701**
Adaboost.M1, **639**
AdaBoostClassifier, 639
Adam, 464
Adamic/Adar, 776
adaptive basis functions, **635**
adaptive instance normalization, **514**
adaptive learning rate, **305**, **307**
adaptive policies, **176**
add-one smoothing, **113**, **124**, **345**
additive attention, **541**
additive model, **635**
adjoint, **458**
adjusted Rand index, **735**
admissible, **195**
adversarial attack, **515**
adversarial bandit, **177**
adversarial image, **515**
adversarial risk, **518**
adversarial training, **518**
affine function, **10**
agent, **16**, **159**, 176
aggregated gradient, **304**
AI, **26**
AI ethics, **27**
AI safety, **27**
Akaike information criterion, **191**
aleatoric uncertainty, **7**, **32**
AlexNet, **497**
alignment, **541**
all pairs, **613**
all-reduce, **472**
ALS, **761**
alternating least squares, **761**
alternative hypothesis, **184**, **201**
ambient dimensionality, **708**
amortized inference, **703**
analysis by synthesis, **45**
anchor, **573**
anchor boxes, **505**
ANN, **449**
approximate posterior inference, **143**
approximation error, **197**
ARD, **414**, **590**, **618**
ARD kernel, **590**
area under the curve, **164**
Armijo backtracking method, **289**
Armijo-Goldstein, **289**
artificial intelligence, **26**
artificial neural networks, **449**
Asia network, **94**
associative, **244**
asymptotic normality, **147**
asynchronous training, **472**
atomic bomb, 68
attention, **541**
attention kernel, **673**
attention mechanisms, **538**
attention score, **539**
attention weights, **538**
AUC, **164**
AugMix, **649**
auto-covariance matrix, **75**
Auto-ML, **500**
AutoAugment, **649**
autocorrelation matrix, **75**
autodiff, **451**
autoencoder, **698**
automatic differentiation, **451**
automatic relevancy determination, **414**, **590**, **618**
AutoRec, **763**
autoregressive model, **93**
average link clustering, **738**
average pooling, **491**
average precision, **166**
axis aligned, **78**
axis parallel splits, **627**
axon, 449

backbone, **446**
backpropagation, **442**
backpropagation algorithm, **451**
backpropagation through time, **530**
backslash operator, **267**
backsubstitution, **267**, 383
bag of word embeddings, **24**
bag of words, **23**, 445
bagging, **633**
BALD, **670**
balloon kernel density estimator, **581**
band-diagonal matrix, **241**
bandit algorithms, 195
bandit problem, **176**
bandwidth, **477**, **578**, **588**
Barnes-Hut algorithm, **722**
barycentric coordinates, **716**
base measure, **420**
basis, **235**
basis function expansion, **439**
basis vectors, **245**
batch learning, **110**
batch normalization, **492**, **494**
batch renormalization, **494**
BatchBALD, **670**
Bayes error, **120**, 565
Bayes estimator, **160**, **194**
Bayes factor, **184**, 201
Bayes model averaging, **121**
Bayes risk, **194**
Bayes' rule, **41**, 41
Bayes' rule for Gaussians, **82**

Bayes's rule, 41
Bayesian, **31**
Bayesian active learning by disagreement, **670**
Bayesian decision theory, **159**
Bayesian deep learning, **475**
Bayesian factor regression, **695**
Bayesian inference, **41**, **42**
Bayesian information criterion, **190**
Bayesian machine learning, **139**
Bayesian network, **91**
Bayesian neural network, **475**
Bayesian Occam's razor, **187**
Bayesian optimization, **329**, **668**
Bayesian personalized ranking, **765**
Bayesian statistics, **121**, 146
BBO, **328**
Beam search, **535**
belief state, **42**, **179**
belief-state MDP, **180**
Berkson's paradox, **93**
Bernoulli bandit, **178**
Bernoulli distribution, **45**, **46**
BERT, **557**
Bessel function, 590
best-arm identification, **177**
beta distribution, **60**, 113, 123
beta function, **60**
beta-binomial, **127**
BFGS, **294**
bi-tempered logistic regression, **371**
bias, **10**, **151**, **379**
bias-variance tradeoff, **153**
biases, **439**
BIC, **190**, 191
BIC score, 745
biclustering, **755**
bidirectional RNN, **527**
big data, **4**
bigram model, **93**, 214
bijector, **62**
bilevel optimization, **198**
binary classification, **2**, **42**
binary connect, **317**
binary cross entropy, **352**
binary entropy function, **210**
binary logistic regression, **349**
binomial coefficient, **46**
binomial distribution, **46**, 46
binomial regression, **429**
BinomialBoost, **642**
BIO, **559**
BiT, **551**
bits, **209**
bivariate Gaussian, **78**
black box attack, **516**
black swan paradox, **113**
blackbox, **328**
blackbox optimization, **328**
blind inverse problem, **88**
block diagonal, **241**
block structured matrices, 251
Blue Brain Project, **451**
BMA, **121**
BN, **492**
BNN, **475**
Boltzmann distribution, **51**
Boltzmann policy, **183**
BookCrossing, **760**
Boolean logic, **32**
Boosting, **635**
bootstrap, **148**
bottleneck, **698**
bound optimization, **317**, 363
bounding boxes, **505**
bowl shape, **354**
box constraints, **314**
boxcar kernel, **578**, 579
branching factor, **214**
Brier score, **171**, 664
Brownian motion, 591
byte-pair encoding, **25**

C-way N-shot classification, **671**
C4, **562**
C4.5, **629**
calculus, **269**
calculus of variations, 428
canonical correlation analysis, **697**
canonical form, **420**, **423**
canonical link function, **431**
canonical parameters, **420**, **423**
CART, **627**, 629
Cartesian, 63
Caser, **768**
CatBoost, **643**
categorical, **49**
categorical PCA, **694**
CatPCA, **694**
Cauchy, **59**
causal, 76
causal CNN, **537**
causal convolution, **537**
CBOW, 725, **726**, 726
CCA, **697**
cdf, **35**, **53**
CelebA, 705
center, 412
centering matrix, **105**, **248**, 714
central interval, **138**
central limit theorem, 56, **68**
centroids, **477**
certify, **519**
chain rule for entropy, **213**
chain rule for mutual information, **221**
chain rule of calculus, **273**
chain rule of probability, **37**
change of variables, **63**
channels, **483**, **489**
characteristic equation, **254**
characteristic length scale, **590**
characteristic matrix, **224**
Chernoff-Hoeffding inequality, **181**
Chi-squared distribution, **61**
choice theory, **432**
Cholesky decomposition, 391
Cholesky factorization, **265**
CIFAR, **18**
CIFAR-10, **19**
city block distance, **736**
class conditional density, **333**
class confusion matrix, **164**
class imbalance, **165**, **368**, 613
class-balanced sampling, **368**
classes, **2**
classical MDS, **710**
classical statistics, **146**
classification, **2**, **800**
Classification and regression trees, **627**
click through rate, **175**, **177**
clinical trials, **177**
closed world assumption, **568**
cloze, **557**
cloze task, **653**
cluster assumption, **659**
Clustering, **733**
clustering, **86**
clusters, **14**
CNN, **3**, **440**, **483**
co-adaptation, 474
Co-training, **661**
coclustering, **755**
codebook, **741**
coefficient of determination, **389**
CoLA, **562**
cold start, **767**
collaborative filtering, 755, **760**
column rank, **239**
column space, **235**
column vector, **231**
column-major order, **233**
committee method, **632**
commutative, **244**
compactness, **738**
complementary log-log, **432**
complementary slackness, **311**
complete class theorem, **195**
complete link clustering, **738**
complexity penalty, **113**
composite objective, **286**
compositional, **448**
compositional pattern-producing network, 517
compound hypothesis, **202**
computation graph, **457**
computation graphs, **275**
computer graphics, **507**
concave, **282**, 427
concentration inequality, **181**
concentration of measure, **521**
condition number, 114, **239**, 290
conditional computation, **479**
conditional distribution, **36**, **80**
conditional entropy, **212**
conditional instance normalization, **514**
conditional mixture model, **478**
conditional mutual information, **221**
conditional probability, **33**
conditional probability distribution, **7**, **47**, **92**
conditional probability table, **92**
conditional variance formula, **40**
conditionally independent, **33**, **37**, **90**

confidence interval, 138, **149**, 150
confirmation bias, **658**
conjugate function, **283**
conjugate gradient, **290**, **383**
conjugate prior, **83**, **122**, 122, 123, **419**
consensus sequence, **211**
conservation of probability mass, **187**
Consistency regularization, **662**
consistent estimator, **195**, 411
constrained optimization, **281**
constrained optimization problem, **307**
constrained optimization problems, 395
constraints, **281**
content constrained, **517**
contextual bandit, **176**
contextual word embeddings, 24, **556**, **730**
continual learning, **569**
continuation method, **329**, **409**
continuous optimization, **279**
continuous random variable, **34**
contraction, **701**
contractive autoencoder, **700**
contradicts, **543**
contrastive loss, **573**
contrastive tasks, **654**
control, **171**
control variate, 304
controlled Markov chain, **177**
conversions, **175**
convex, **354**
convex combination, 125
convex function, **282**
convex optimization, **281**
convex relaxation, **398**
convex set, **282**
convolution, **66**, **484**
convolution theorem, **66**
convolution with holes, **501**
convolutional Markov model, **537**
convolutional neural network, **3**, 20
convolutional neural networks, **12**, **440**, **483**
cooling schedule, **329**
coordinate descent, **408**
coordinate vectors, **235**
coreference resolution, **546**
coreset, 577
corpus, **723**
correlation coefficient, **74**, **78**
correlation does not imply causation, **75**
correlation matrix, **74**, 106
cosine kernel, **592**
cosine similarity, **724**
cost function, **279**
covariance, **73**
covariance matrix, **73**, **77**
covariates, **2**, **379**, **800**
COVID-19, **42**
CPD, **92**
CPPN, 517
CPT, **92**
credible interval, **135**, **138**, 138, 150
critical point, **309**
cross correlation, **485**
cross entropy, **211**, 216, **218**
cross validation, **118**, **199**
cross-covariance, **73**
cross-entropy, **170**
cross-entropy method, **329**
cross-over rate, **164**
cross-validated risk, **118**, **199**
crosscat, **757**
crowding problem, **721**
CTR, **177**
cumulants, **425**
cumulative distribution function, **35**, **53**
cumulative regret, **180**
cumulative reward, **176**
curse of dimensionality, **566**
curve fitting, 13
curved exponential family, **420**
CV, **118**, **199**
cyclic permutation property, **238**
cyclical learning rate, **303**

DAG, **91**, **457**
data augmentation, **218**, 434, **649**
data compression, **16**, 742
data fragmentation, **629**
Data mining, **26**
data parallelism, **471**
data processing inequality, **225**
Data science, **26**
data uncertainty, **7**, **32**
dead ReLU, **467**
debiasing, **401**
decision boundary, **5**, **49**, **141**, **350**
decision making under uncertainty, **1**
decision rule, **4**
decision surface, **5**
decision tree, **5**
decision trees, **627**
decode, **675**
decoder, **697**, 705
deconvolution, **502**
deduction, **203**
deep CCA, **697**
deep factorization machines, **766**
deep graph infomax, **785**
deep metric learning, **570**, **572**
deep mixture of experts, **481**
deep neural networks, **12**, **439**
DeepDream, **511**
DeepWalk, **777**
default prior, **136**
defender's fallacy, **70**
deflated matrix, **731**
deflation, **259**
degree of normality, **58**
degrees of freedom, **12**, **58**, **190**, **392**
delta rule, **301**
demonstrations, 17
dendogram, **736**
dendrites, **450**
denoising autoencoder, **700**
dense prediction, **506**
dense sequence labeling, **527**
DenseNets, **500**
density estimation, **16**
density kernel, 539, **578**
dependent variable, **379**
depth prediction, **506**
depthwise separable convolution, **503**
derivative, **269**
derivative free optimization, **328**
derivative function, **275**
descent direction, **287**
design matrix, **3**, 247, **800**
determinant, **239**
development set, **117**
deviance, **190**, **630**
DFO, **328**
diagonal covariance matrix, **79**
diagonal matrix, **241**
diagonalizable, **255**
diagonally dominant, **242**
diameter, **738**
differentiable programming, **457**
differential entropy, **214**
differentiating under the integral sign, **66**
differentiation, 270
diffuse prior, **136**
dilated convolution, **501**, 506
dilation factor, **501**
dimensionality reduction, **4**, **675**
Dirac delta function, **57**, 140
directed acyclic graph, **91**
directed ayclic graph, **457**
directional derivative, **271**
Dirichlet distribution, **130**, 344
Dirichlet energy, **719**
discount factor, **176**
discrete AdaBoost, **639**
discrete optimization, **279**
discrete random variable, **33**
discretize, **215**, **222**
discriminant function, **334**
discriminative classifier, **333**, **346**
dispersion parameter, **427**
distance metric, **215**
distortion, **675**, **742**
distribution shift, **519**
distributional hypothesis, **723**
distributive, **244**
divergence measure, **215**
DNA sequence motifs, **210**
DNN, **12**, **439**
document retrieval, **724**
document summarization, **21**
domain adaptation, 651, **656**
domain adversarial learning, **656**
dominates, **195**, **202**
dot product, **244**
double centering trick, **249**
double sided exponential, **59**
dropout, **473**
dual feasibility, **310**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

dual form, **608**
dual problem, **607**
dual variables, **614**
dummy encoding, **22**, **50**
Dutch book theorem, 205
dynamic graph, **459**
dynamic programming, **535**
Dyspnea, **94**

E step, **319**, **320**
early stopping, **119**, **472**
earning while learning dilemma, **172**
EB, **137**
echo state network, **531**
ECM, 688
economy sized QR, **265**
economy sized SVD, **260**
edge devices, **317**, **503**
EER, **164**
EfficientNetv2, **500**
eigenfaces, **677**
eigenvalue, **254**
eigenvalue decomposition, **254**
eigenvalue spectrum, 115
eigenvector, **254**
Einstein summation, **250**
einsum, **250**
elastic embedding, **720**
elastic net, 402, **407**
ELBO, **145**, **320**, 704
elbow, **746**
electronic health records, **543**
ell-2 loss, 8
ELM, **604**
ELMo, **557**
ELU, **467**
EM, 81, **317**, **318**
EM algorithm, 394
email spam classification, **21**
embarassingly parallel, **471**
embedding, **675**
EMNIST, **18**
empirical Bayes, **137**, 186
empirical distribution, **68**, **101**, 196, 217
empirical Fisher, **298**
empirical risk, **6**, **196**
empirical risk minimization, **6**, **107**, **197**, 299
encode, **675**
encoder, **697**, 705
encoder-decoder, **506**
encoder-decoder architecture, **529**
endogeneous variables, **2**
energy based model, **655**
energy function, 144
ensemble, 303, **474**
ensemble learning, **632**
entails, **543**
entity linking, **569**
entity resolution, 560, **569**
entropy, **170**, **209**, 320, 630
entropy minimization, **658**
environment, **176**
Epanechnikov kernel, 579
epigraph, **282**
epistemic uncertainty, **7**, **32**
epistemology, 32
epoch, **300**
epochs, **444**
epsilon insensitive loss function, **616**
epsilon-greedy, 183
equal error rate, **164**
equality constraints, **281**, **307**
equitability, **224**
equivalent sample size, **123**
equivariance, **491**
ERM, **107**, **197**
error function, **53**
estimation error, **197**
estimator, **146**
evaluation functional, **623**
EVD, **254**
event, **32**, 32, **33**, 33
events, **31**
evidence, **127**, **186**
evidence lower bound, **145**, **320**, 704
evolutionary algorithms, **329**
EWMA, **111**, 306
exact line search, **289**
exchangeable, **95**
exclusive KL, **218**
exclusive or, 478
exemplar-based models, **565**
exemplars, **477**
exogeneous variables, **2**
expanded parameterization, **90**
expectation maximization, **317**, **318**
expected complete data log likelihood, **321**
expected sufficient statistics, **321**
expected value, **38**, **55**
experiment design, **669**
explaining away, **93**
explanatory variables, **379**
explicit feedback, **759**
exploding gradient probem, **464**
exploration-exploitation tradeoff, **172**, **178**, **768**
exploratory data analysis, **4**
exponential cooling schedule, **329**
exponential dispersion family, **427**
Exponential distribution, **61**
exponential family, 136, **419**, 419, 428
exponential family factor analysis, **694**
exponential family PCA, **694**
Exponential linear unit, 465
exponential loss, **637**
exponentially weighted moving average, **111**
exponentiated cross entropy, **213**
exponentiated quadratic, **588**
extractive summarization, **561**
extreme learning machine, **604**

F score, 167
face detection, **505**
face recognition, **505**
face verification, **569**
FaceNet, **575**
factor analysis, **14**, **684**
factor loading matrix, **685**
factorization machine, **766**
FAISS, **568**
false alarm rate, **164**
false negative rate, **42**
false positive rate, **42**, **164**
fan-in, **461**, **469**
fan-ou, **469**
fan-out, **277**, **461**
Fano's inequality, **227**
farthest point clustering, **743**
Fashion-MNIST dataset, **18**
FashionMNIST, **19**
fast adapation, **671**
fast gradient sign, **516**
fast Hadamard transform, **604**
fastfood, **604**
feasibility, **310**
feasibility problem, **281**
feasible set, **281**
feature crosses, **22**
feature detection, **486**
feature engineering, **10**
feature extraction, **11**
feature extractor, **380**
feature importance, **644**, 645
feature map, **486**
feature preprocessing, **10**
feature selection, **227**, **315**, **397**
features, **1**
featurization, **3**
feedforward neural network, **440**
few-shot classification, **569**
few-shot learning, **671**
FFNN, **440**
FGS, **516**
fill in, **25**
fill-in-the-blank, **557**, **653**
filter, **485**
filters, **483**
FIM, **147**
fine-grained classification, **20**, **672**
fine-grained visual classification, **651**
fine-tune, **556**
fine-tuning phase, **651**
finite difference, **270**
finite difference matrix, **586**
finite horizon, **176**
finite sum problem, **299**
first order, **286**, **356**
first order Markov condition, **92**
first-order, **292**
Fisher information matrix, **147**, 296, 357
Fisher scoring, **357**
Fisher's linear discriminant analysis, **339**
FISTA, **409**
flat local minimum, **280**
flat minima, **475**
flat prior, **136**
flatten, **444**
FLDA, **339**

folds, **118**, **199**
fooling images, **516**
forget gate, **533**
formal language, **463**
forward mode differentiation, **453**
forward stagewise additive modeling, **636**
forwards KL, **218**
forwards model, **45**
founder variables, **691**
fraction of variance explained, **683**
fraud detection system, **787**
frequentist, **31**
frequentist decision theory, **192**
frequentist statistics, **146**
Frobenius norm, **237**
frozen parameters, **652**
full covariance matrix, **79**
full rank, **239**
function space, **597**
functional, **275**
functional analysis, **621**
furthest neighbor clustering, **738**
fused batchnorm, **493**

gallery, **505**, **568**
gamma distribution, **60**
GANs, 656
gap, **180**
Gated Graph Sequence Neural Networks, **780**
gated recurrent units, **532**
gating function, **479**
Gaussian, **8**
Gaussian bandit, **179**
Gaussian discriminant analysis, **333**
Gaussian distribution, **53**
Gaussian kernel, **477**, 554, **578**, **588**
Gaussian mixture model, **85**
Gaussian process, **477**
Gaussian processes, 329, **594**
Gaussian scale mixture, **89**, 395
GCN, **781**
GDA, **333**
generalization error, **197**, 199
generalization gap, **12**, **197**
generalize, **6**, **112**
generalized CCA, **697**
generalized eigenvalue, **340**
generalized Lagrangian, **310**, 607
generalized linear model, **428**
generalized linear models, **419**
generalized low rank models, **694**
Generative adversarial networks, **666**
generative classifier, **333**, **346**
generative image model, **507**
Geometric Deep Learning, **771**
geometric series, **111**, 291
Gini index, **629**
Gittins index, **180**
GLM, **428**
glmnet, **408**
GLMs, **419**
global average pooling, **446**, **491**
global optimization, **279**
global optimum, **279**, 354
globally convergent, **280**
Glorot initialization, **469**
GloVe, **728**
GMM, **85**
GMRES, **383**
GNN, **440**, **779**
goodness of fit, **387**
GoogLeNet, **497**
GPT, **561**
GPT-2, **561**
GPT-3, **561**
GPUs, **448**, **471**
GPyTorch, **603**
gradient, **270**, **287**
gradient boosted regression trees, **642**
gradient boosting, **640**
gradient clipping, **465**
gradient sign reversal, **656**
gradient tree boosting, **642**
Gram matrix, **242**, **249**, **513**, **588**
Gram Schmidt, 243
Graph attention network, **782**
Graph convolutional networks, **781**
graph factorization, **776**
graph Laplacian, **718**, 753
graph neural network, **779**
graph neural networks, **440**
graph partition, **752**
Graph Representation Learning, **771**
graphical models, **37**
Graphical Mutual Information, **786**
graphics processing units, **471**
graphite, **785**
GraphNet, **780**
GraphSAGE, **782**
GraRep, **777**
greedy decoding, **535**
greedy forward selection, **409**
greedy policy, **178**
grid approximation, **144**
grid search, 117, **328**
group lasso, **405**
group normalization, **494**
group sparsity, **405**
grouping effect, **407**
GRU, **532**
GSM, **89**

HAC, **735**
half Cauchy, **59**
half spaces, **350**
Hamiltonian Monte Carlo, **146**
hard attention, **545**
hard clustering, **87**, **748**
hard negatives, **574**
hard thresholding, **401**, 403
hardware accelerators, **450**
harmonic mean, 167
hat matrix, **383**
HDI, **139**
He initialization, **469**
heads, **446**
heat map, **486**
Heaviside, 356
Heaviside step function, 456
heaviside step function, **48**, **440**
heavy ball, **291**
heavy tails, **58**, **393**
Helmholtz machine, **703**
Hessian, 354, **798**
Hessian matrix, **272**
heteroskedastic regression, **56**, **384**
heuristic function, **535**
heuristics, **451**
hidden, **41**
hidden common cause, **75**
hidden units, **441**
hidden variables, **95**, **319**, **800**
hierarchical, **448**
hierarchical agglomerative clustering, **735**
hierarchical Bayesian model, **137**
hierarchical mixture of experts, **482**
hierarchical softmax, **367**
hierarchy, **366**
highest density interval, **139**
highest posterior density, **138**
Hilbert space, **622**
hinge loss, **108**, 330, **611**, 765
Hinton diagram, **82**
hit rate, **164**
HMC, **146**
Hoeffding's inequality, **200**
hogwild training, **472**
holdout set, **198**
homogeneous, **93**
homoscedastic regression, **55**
homotopy, **409**
HPD, **138**
Huber loss, **169**, **396**, 641
Huffman encoding, 367
human pose estimation, **507**
Hutchinson trace estimator, **238**
hyper-parameters, **123**, 328
hypercolumn, **489**
hypernyms, **366**
hyperparameter, **198**
hyperparameters, **137**
hyperplane, **350**
hypothesis, **543**
hypothesis space, **197**
hypothesis testing, **184**

I-projection, **218**
ID3, **629**
identifiability, **365**
identifiable, **195**, 365
identity matrix, **241**
iGPT, **561**
iid, **67**, **100**, **122**
ill-conditioned, 106, **239**
ill-posed, **45**
ILP, **313**
image captioning, **525**
image classification, **3**
image compression, 742

image deblurring, **87**
image denoising, **87**
image inpainting, **87**
image patches, **483**
image super-resolution, **87**
image tagging, **360**, **504**
image-to-image, **506**
ImageNet, **20**, 448, 496
ImageNet-21k, 552
IMDB, 119
IMDB movie review dataset, **21**, **445**
implicit feedback, **764**
implicit regularization, **475**
impostors, **570**
imputation tasks, **653**
inception block, **497**
Inceptionism, **511**
inclusive KL, **218**
incremental learning, **569**
indefinite, **242**
independent, **37**
independent and identically distributed, **67**, **122**
independent variables, **379**
indicator function, **6**, 34
induced norm, **237**
inducing points, **603**
induction, **113**, **203**
inductive bias, **13**, 551
inductive learning, **662**
inequality constraint, 310
inequality constraints, **281**, **307**
infeasible, 311
inference, **99**, **121**
inference network, **703**
infinite horizon, **176**
infinitely wide, **602**
infinitely widely neural networks, 471
InfoNCE, **574**
information, 31
information content, **209**
information criteria, **190**
information diagram, **220**
information diagrams, 221
information extraction, **559**
information form, **423**
information gain, **215**, **669**
information gathering action, **7**
information projection, **218**
information retrieval, 165
information state, **179**
information theory, **170**, **209**
inner product, **244**, **622**
input gate, **533**
input nodes, **460**
Instagram, **504**
instance normalization, **494**
instance segmentation, **505**
instance-balanced sampling, **368**
instance-based learning, **565**
Integer linear programming, **313**
integrated risk, **194**
integrating out, **121**
interaction effects, **22**
intercept, **10**
interior point method, 312
internal covariate shift, **493**
interpolate, **10**, **586**
interpolated precision, **166**
interpolator, **594**
interpretable, **16**
intrinisic motivation, **183**
intrinsic dimensionality, **708**
intrinsic Gaussian random field, 586
inverse, **251**
inverse cdf, **36**
inverse document frequency, **23**
inverse Gamma distribution, **61**
inverse probability, **45**
inverse problems, **45**
inverse reinforcement learning, **27**
inverse Wishart, 114
iris, 2
iris dataset, **3**
IRLS, **357**
isomap, **712**
isotropic covariance matrix, **79**
ISTA, **409**
items, **759**
iterate averaging, **303**
iterative soft thresholding algorithm, **409**
iteratively reweighted least squares, **357**

Jacobian, 63, 362
Jacobian formulation, 271
Jacobian matrix, **271**
Jacobian vector product, **272**
Jensen's inequality, **216**, 320
Jeopardy, 162
Jester, **760**
JFT, 552
jittered, **478**
joint distribution, **36**
joint probability, **32**
JPEG, 742
just in time, **459**
JVP, **272**

K nearest neighbor, **565**
k-d tree, **568**
K-means algorithm, **739**
K-means clustering, **87**
K-means++, **743**
K-medoids, **743**
Kalman filter, **83**
Karl Popper, 113
Karush-Kuhn-Tucker, **311**
Katz centrality index, 776
KDE, **577**, **579**
kernel, **485**, 578
kernel basis function expansion, **416**
kernel density estimation, **577**
kernel density estimator, **579**
kernel function, **477**, 585, **588**
kernel PCA, **713**, 754
kernel regression, 539, **582**, 596
kernel ridge regression, 596, **615**
kernel smoothing, **582**
kernel trick, **610**
keys, **538**
keywords, 261
KFAC, **298**
kink, **746**
KKT, **311**
KL divergence, **101**, **169**, **215**, 320
KNN, **565**
Knowledge distillation, **668**
knowledge graph, **787**
Kronecker product, **249**
Krylov subspace methods, **603**
KSG estimator, **222**
Kullback Leibler divergence, **101**, **169**
Kullback-Leibler divergence, **215**, 320

L-BFGS, **295**
L0-norm, **397**, 398
L1 loss, 168
L1 regularization, 398
L1VM, **618**
L2, 622
L2 loss, 168
L2 regularization, 116, 359, 389
L2VM, **618**
label, **2**
label noise, **369**
Label propagation, **661**, **779**
label smearing, **366**
label smoothing, **668**
Label spreading, **779**
label switching problem, **325**, **748**
Lagrange multiplier, **309**
Lagrange multipliers, 103, 427
Lagrange notation, **270**
Lagrangian, 259, **281**, **309**, 399, 427
Lanczos algorithm, 689
language model, **92**
language modeling, **21**, **525**
language models, 213, **556**
Laplace, 397
Laplace approximation, **144**, 372
Laplace distribution, **59**
Laplace smoothing, **345**
Laplace vector machine, **618**
Laplace's rule of succession, **126**
Laplacian eigenmaps, **716**, 753, **775**
LAR, **409**
large margin classifier, **605**
large margin nearest neighbor, **570**
LARS, **409**
lasso, **398**
latent coincidence analysis, **571**
latent factors, **14**, **677**
latent semantic analysis, **725**
latent semantic indexing, **724**
latent space interpolation, **705**
latent variable, **85**
latent variable models, **800**
latent variables, **800**

latent vector, **677**
law of iterated expectations, **39**
law of total expectation, **39**
law of total variance, **40**
layer normalization, **494**
layer-sequential unit-variance, **469**
LCA, **571**
LDA, **333**, **335**
Leaky ReLU, 465
leaky ReLU, **467**
learning curve, **120**
learning rate, **287**
learning rate schedule, **288**, **301**, 301
learning rate warmup, **302**
learning to learn, **670**
learning with a critic, **17**
learning with a teacher, **17**
least angle regression, **409**
least favorable prior, **195**
least mean squares, **301**, 386
least squares boosting, **409**, **636**
least squares objective, **269**
least squares solution, **9**
leave-one-out cross-validation, **118**, **199**
LeCun initialization, **469**
Leibniz notation, **270**
LeNet, **492**, **495**
level sets, 78, **79**
life-long learning, **569**
LightGBM, **643**
likelihood, **41**
likelihood function, **121**
likelihood principle, **205**
likelihood ratio, **184**, **204**
likelihood ratio test, **201**
limited memory BFGS, **295**, 363
line search, **289**
Linear algebra, **231**
linear autoencoder, **698**
linear combination, **245**
linear discriminant analysis, **333**, **335**
linear Gaussian system, **82**
linear kernel, **601**
linear map, **235**
linear operator, **383**
linear programming, 395
linear rate, **290**
linear regression, **55**, **379**, 419, 439
linear subspace, **245**
linear threshold function, **440**
linear transformation, **235**
linearity of expectation, **38**
linearization point, **275**
linearly dependent, 234
linearly independent, 234
linearly separable, **350**
Linformer, **554**
link function, **428**, 431
link prediction, **787**
Lipschitz constant, **285**
liquid state machine, **531**
LMNN, **570**
LMS, **301**
local linear embedding, **716**
local maximum, **280**
local minimum, **279**
local optimum, **279**, 354
locality sensitive hashing, **568**
locally linear regression, **583**
locally-weighted scatterplot smoothing, **583**
LOESS, **583**
log bilinear language model, **729**
log likelihood, **100**
log loss, **170**, **612**
log odds, **48**
log partition function, **420**
log-sum-exp trick, **52**
logistic, **48**
logistic distribution, **433**
logistic function, **48**, **140**
Logistic regression, **349**
logistic regression, **48**, **140**, 419
logit, **47**, **349**
logit function, **48**
logitBoost, **640**
logits, **7**, **51**, **360**
long short term memory, **533**
long tail, 143, **368**
Lorentz, **59**
Lorentz model, **776**
loss function, **6**, 8, **159**, **279**
lossy compression, 741
lower triangular matrix, **241**
LOWESS, **583**
LSA, **725**
lse, **52**
LSH, **568**
LSI, **724**
LSTM, **533**

M step, **319**
M'th order Markov model, 93
M-projection, **218**
M1, **665**
M2, **665**
MAB, **176**
machine learning, **1**
machine translation, **21**, **529**
Mahalanobis distance, **79**, **565**
Mahalanobis whitening, **258**
main effects, **22**
majorize-minimize, **317**
MALA, 508
MAML, **670**, **671**
manifold, 706, **707**
manifold assumption, **661**
manifold hypothesis, **708**
manifold learning, **706**
mAP, **166**
MAP estimate, **161**
MAP estimation, **113**, 198
MAR, **25**
margin, **108**, **605**, **637**
margin errors, **610**
marginal distribution, **36**, **80**
marginal likelihood, **41**, **121**, **127**, 129, 137, **186**
marginalizing out, **121**, 140
marginalizing over, **121**
marginally independent, **37**
Markov chain, **93**
Markov chain Monte Carlo, **145**
Markov decision process, **177**
Markov kernel, **93**
Markov model, **93**
MART, **642**
masked attention, **539**
masked language model, **558**
matched filter, **484**
matching network, **673**
Matern kernel, **590**
matrix, **231**
matrix completion, **761**
matrix determinant lemma, **254**
matrix factorization, **761**
matrix inversion lemma, **253**, 614
matrix square root, 237, **246**, **265**
matrix vector multiplication, **603**
max pooling, **491**
maxent classifier, **366**
maximal information coefficient, 223
maximum a posterior estimation, **113**
maximum a posteriori, **161**
maximum entropy, 56, **209**
maximum entropy classifer, **366**
maximum entropy model, **428**
maximum entropy sampling, **670**
maximum expected utility principle, **160**
maximum likelihood estimate, **8**
maximum likelihood estimation, **99**
maximum risk, **194**
maximum variance unfolding, **715**
MCAR, **25**
McCulloch-Pitts model, **450**
McKernel, **604**
MCMC, **145**
MDL, **192**
MDN, **481**
MDP, **177**
MDS, **709**
mean, **38**, **55**
mean average precision, **166**
mean function, **428**
mean squared error, **8**, **107**
mean value imputation, **25**
measure, **622**
median, **36**, **53**
median absolute deviation, **580**
medoid, **743**
memory cell, **533**
memory-based learning, **565**
Mercer kernel, **588**
Mercer's theorem, **589**, **623**
message passing neural networks, **780**
meta learning, **670**
meta-learning, 673
method of moments, **108**
metric MDS, **711**
Metropolis-adjusted Langevin algorithm, **508**
MICe, **224**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

min-max scaling, **360**
minibatch, **299**
minimal, **420**
minimal representation, **420**
minimal sufficient statistic, **226**
minimally informative prior, **136**
minimax estimator, **194**
minimum description length, **192**
minimum mean squared error, **168**
minimum spanning tree, 737
minorize-maximize, **317**
MIP, **313**
misclassification rate, **5**, **108**
missing at random, **25**, **759**
missing completely at random, **25**
missing data, **25**, **319**, **325**
missing data mechanism, **25**, 666
missing value imputation, **81**
mixed ILP, **313**
mixing weights, 129
mixmatch, **659**
mixture density network, **481**
mixture model, **84**
mixture of Bernoullis, **89**
mixture of beta distributions, **128**
mixture of experts, **479**
mixture of factor analysers, **692**
mixture of Gaussians, **85**
ML, **1**
MLE, **8**, **99**
MLP, **440**
MM, **317**
MMSE, **168**
MNIST, **18**, **19**, **89**, 495
MobileNet, **503**
MoCo, **656**
mode, **39**, **161**
mode-covering, **218**
mode-seeking, **219**
model compression, **473**
model fitting, **6**, **99**
model parallelism, **471**
model selection, **185**
model selection consistent, **403**
model uncertainty, **7**, **32**
model-agnostic meta-learning, **671**
modus tollens, **203**
MoE, **479**
MoG, **85**
moment matching, **427**
moment parameters, **422**
moment projection, **218**
momentum, **291**
momentum contrastive learning, **656**
MoNet, **782**
Monte Carlo approximation, **68**, **145**, **375**
Monte Carlo dropout, **474**
Monte Carlo tree search, **536**
Monty Hall problem, **43**
Moore-Aronszajn theorem, **623**
Moore-Penrose pseudo-inverse, **261**
most powerful test, **202**
motes, 10
motif, 211
MovieLens, **760**
moving average, **111**
MSE, **8**, **107**
multi-armed bandit, **176**
multi-class classification, **360**
multi-clust, **757**
Multi-dimensional scaling, **775**
multi-headed attention, **546**
multi-label classification, **360**
multi-label classifier, 366
multi-level model, **137**
multi-object tracking, **569**
multiclass logistic regression, **349**
multidimensional scaling, **709**
multilayer perceptron, **440**
multimodal, **39**
multinomial coefficient, **50**
multinomial distribution, **50**
Multinomial logistic regression, **360**
multinomial logistic regression, **52**, **349**
multinomial logit, **51**
multinomial probit, **435**
multiple imputation, **81**
multiple linear regression, **9**, **379**
multiple restarts, **743**
multiplicative noise, **89**
multivariate Bernoulli naive Bayes, **343**
multivariate Gaussian, **76**
multivariate linear regression, **380**
multivariate normal, **76**
multivariate probit, **435**
mutual information, 75, **219**
mutually independent, 69
MVM, **603**
MVN, **76**
myopic, **670**

N-pairs loss, **574**
Nadaraya-Watson, **582**
naive Bayes assumption, 337, **342**
naive Bayes classifier, **342**
named entity recognition, **559**
NAS, **500**
nats, **209**
natural exponential family, **420**
natural gradient, **296**
natural gradient descent, **296**
natural language inference, **543**
natural language processing, **21**, 366
natural parameters, **420**, **423**
NBC, **342**
NCA, **570**
NCM, **338**
nearest centroid classifier, **338**
nearest class mean classifier, **338**, **368**
nearest class mean metric learning, **338**
nearest neighbor clustering, **737**
NEF, **420**
negative definite, **242**
negative log likelihood, **8**, **100**
negative semidefinite, **242**
neighborhood components analysis, **570**
neocognitron, **492**
nested optimization, **198**
nested partitioning, **757**
Nesterov accelerated gradient, **291**
Netflix Prize, **759**
NetMF, **778**
neural architecture search, **329**, **500**
neural language model, **93**
neural machine translation, **529**
neural matrix factorization, **766**
neural network Gaussian process, **471**
neural style transfer, **512**
NeurIPS, 18
neutral, **543**
Newton's method, **293**, **356**
next sentence prediction, **558**
Neyman-Pearson lemma, **202**
NGD, **296**
NHST, **202**
NHWC, **492**
NIPS, 18
NLL, **100**
NLP, **21**
NMAR, **25**
no free lunch theorem, **13**
node2vec, **778**
noise floor, **120**
non-factorial prior, **415**
non-identifiability, **748**
non-metric MDS, **711**
non-parametric bootstrap, **148**
non-parametric methods, 706
non-parametric model, **477**
non-saturating activation functions, **443**, **466**
noninformative, **136**
nonlinear dimensionality reduction, **706**
nonlinear factor analysis, **692**
nonparametric methods, **585**
nonparametric models, **565**
nonsmooth optimization, **285**
norm, **236**, 239
normal, **8**
normal distribution, **53**
normal equations, **269**, **381**
Normal-Inverse-Wishart distribution, **324**
normalization layer, **492**
normalized, **242**
normalized cut, **753**
normalized mutual information, **223**, **735**
normalizer-free networks, **495**
Normalizing flows, **667**
not missing at random, **25**
noun phrase chunking, **559**
novelty detection, **569**
NT-Xent, **574**
nu-SVM classifier, 610
nuclear norm, **237**
nucleotide, 211
nuisance variables, **81**

null hypothesis, **184**, **201**
null hypothesis significance testing, **202**
nullspace, **235**
numerator layout, 271

object detection, **505**
objective, **136**
objective function, **100**, **279**
observation distribution, **41**
Occam factor, **190**
Occam's razor, **186**
offset, **9**, **379**
Old Faithful, 322
Olivetti face dataset, 677
OLS, **107**, **269**, **381**
one-armed bandit, **176**
one-hot, **170**
one-hot encoding, **22**, 361, **800**
one-hot vector, **49**, **231**
one-shot learning, **338**, **671**
one-standard error rule, **118**
one-to-many functions, **478**
one-versus-one, **613**
one-versus-the-rest, **613**
one-vs-all, **613**
online advertising system, **177**
online learning, **110**, **569**
OOD, **569**
OOV, **23**, **24**
open class, **25**
open set recognition, **568**
open world, **504**
open world assumption, **569**
OpenPose, **507**
opportunity cost, **171**
opt-einsum, **251**
optimal policy, **160**
optimism in the face of uncertainty, **181**
optimism of the training error, **198**
optimization problem, **279**
order, **233**
order statistics, **110**
ordered Markov property, **91**
ordering constraint, **750**
ordinal regression, **435**
ordinary least squares, **107**, **269**, **381**
Ornstein-Uhlenbeck process, **591**
orthodox statistics, **146**
orthogonal, **242**, **243**, 255
orthogonal projection, **382**
orthogonal random features, **604**
orthonormal, **242**, **255**
orthonormal basis, **622**
out of vocabulary, **24**
out-of-bag instances, **633**
out-of-distribution, **569**
out-of-sample generalization, **707**
out-of-vocabulary, **23**
outer product, **244**
outliers, **57**, **168**, **369**, **393**
output gate, **533**
over-complete representation, **420**
over-parameterized, **52**
overcomplete representation, **698**
overdetermined, **266**
overdetermined system, **381**
overfitting, **12**, **112**, 125

p-value, 184, **203**
PAC learnable, **199**
PageRank, **258**
pair plot, **4**
pairwise independent, 69
PAM, **743**
panoptic segmentation, **505**
parameter space, **279**
parameter tying, **93**
parameters, **5**, **9**
parametric bootstrap, **148**
parametric models, **565**
parametric ReLU, **467**
part of speech tagging, 559
part-of-speech, **557**
partial dependency plot, **645**
partial derivative, **270**
partial least squares, **696**
partial pivoting, **264**
partial regression coefficient, **385**
partially observable Markov decision process, **178**
partially observed, **159**
partially observed data, **325**
partition function, **52**, **420**, **655**
partitioned inverse formulae, **252**
partitioning around medoids, **743**
Parzen window density estimator, **579**
pathologies, 205
pattern recognition, **2**
PCA, **14**, **675**, 676
PCA whitening, **257**
pdf, **35**, **54**
peephole connections, **534**
penalty term, **314**
per-step regret, **180**
percent point function, **36**
perceptron, **355**, **440**
perceptron learning algorithm, **356**
perfect information, **174**
Performer, **554**
periodic kernel, **592**
perplexity, **213**, 525
person re-identification, **569**
PersonLab, **507**
perturbation theory, 753
PGD, **516**
PGM, **90**
phi-exponential family, 424
Pitman-Koopman-Darmois theorem, **426**
placebo, **177**
Planetoid, **786**
plates, **95**
Platt scaling, **611**
PLS, **696**
plug-in approximation, **125**, **140**
plugin approximation, 375
PMF, **762**
pmf, **34**
PMI, **725**
Poincaré model, **776**
point estimate, **99**
pointwise convolution, **491**
pointwise mutual information, **725**
Poisson regression, **430**
polar, 64
policy, **16**, **176**
Polyak-Ruppert averaging, **303**
polynomial expansion, 380
polynomial regression, **10**, 116, **380**
polytope, **311**
POMDP, **178**
pool-based active learning, **668**
population risk, **12**, **117**, **196**
POS, **557**
position weight matrix, **210**
positional embedding, **547**
positive definite, **242**
positive definite kernel, **588**
positive PMI, **725**
positive semidefinite, **242**
posterior, 121
posterior distribution, **42**, 81, **121**
posterior expected loss, **159**
posterior inference, **42**
posterior mean, 168
posterior median, **168**
posterior mode, **113**
posterior predictive distribution, **121**, **126**, 140, **375**, 412
power, **202**
power method, **258**
PPCA, **686**
ppf, **36**
pre-activation, **349**
pre-activations, **442**
pre-train, **556**
pre-trained word embedding, **24**
pre-training phase, **651**
preactivation resnet, **499**
precision, **53**, 133, **166**, 166
precision at K, **166**
precision matrix, **104**, **423**
precision-recall curve, **165**
preconditioned SGD, **305**
preconditioner, **305**
preconditioning matrix, **305**
predictive analytics, **26**
predictors, **2**
preferences, **159**
premise, **543**
PreResnet, **499**
pretext tasks, **654**
prevalence, **42**, **167**
primal problem, **607**
primal variables, **614**
primitive nodes, **460**
principal components analysis, **14**, **675**
principal components regression, **393**
prior, **113**, **121**
prior distribution, **41**, 81
probabilistic forecasting, **169**
probabilistic graphical model, **90**

probabilistic inference, **42**
probabilistic matrix factorization, **762**
probabilistic model-building genetic algorithms, **329**
probabilistic PCA, 675
probabilistic perspective, **1**
probabilistic prediction, **169**
probabilistic principal components analysis, **686**
probability density function, **35**, **54**
probability distribution, **169**
probability distributions, **1**
probability mass function, **34**
probability matching, **182**
probability simplex, **130**
probability theory, 41
probably approximately correct, **199**
probit approximation, **376**
probit function, **54**, 376, **432**
probit link function, **432**
probit regression, **432**
product rule, **37**
product rule of probability, **41**
profile likelihood, **683**
profile log likelihood, 684
projected gradient descent, **315**, 408, **516**
projection, **236**
projection matrix, **383**
prompt, **561**
proper scoring rule, **170**
proposal distribution, 329
prosecutor's fallacy, **70**
proxies, **575**
proximal gradient descent, 409
proximal gradient method, **314**
proximal operator, **314**
ProxQuant, **317**
proxy tasks, **654**
prune, **630**
psd, **242**
pseudo counts, **123**, **344**
pseudo inputs, **603**
pseudo inverse, 269, 381
pseudo norm, **237**
pseudo-labeling, **657**
pseudo-likelihood, **557**
pure, **630**
pure exploration, **180**
purity, **734**
Pythagoras's theorem, **268**

QALY, **159**
QP, **312**
quadratic approximation, **144**
quadratic discriminant analysis, **334**
quadratic form, **241**, **256**
quadratic kernel, **589**
quadratic loss, **8**, **168**
quadratic program, **312**, 399, 616
quality-adjusted life years, **159**
quantile, **36**, **53**
quantile function, **36**
quantization, 214
quantize, **215**, **222**
quantized, **317**
quartiles, **36**, **54**
Quasi-Newton, **294**
quasi-Newton, 363
query, **538**
query synthesis, **668**
question answering, **21**, **560**

radial basis function, **578**
radial basis function kernel, **477**, 554
Rand index, **734**
RAND-WALK, **729**
random finite sets, **569**
random forests, **634**
random Fourier features, **604**
random number generator, **68**
random search, **328**
random shuffling, 300
random variable, **33**
random variables, **1**
random walk kernel, **594**
random walk proposal, 329
range, **235**
rank, **233**, **239**
rank deficient, **239**
rank one update, **253**
rank-nullity theorem, **262**
ranking loss, **572**, **765**
RANSAC, **397**
rate, **501**, **742**
rate of convergence, **290**
rating, **759**
Rayleigh quotient, **258**
RBF, **578**
RBF kernel, **477**, **588**
RBF network, **477**
real AdaBoost, **639**
recall, **164**, **166**
receiver operating characteristic, **164**
receptive field, **488**, 501
recognition network, **703**
recommendation systems, **787**
Recommender systems, **759**
reconstruction error, **675**, **677**, **742**
Rectified linear unit, 465
rectified linear unit, **443**, **466**
recurrent neural network, **523**
recurrent neural networks, **12**, **440**
recursive update, **110**
recursively, **385**
reduce-on-plateau, **302**
reduced QR, **265**
Reformer, **553**
regression, **8**, **800**
regression coefficient, **385**
regression coefficients, **10**, **379**
regret, **174**, **180**
regularization, **112**
regularization parameter, **113**
regularization path, **393**, **401**, 409
regularized discriminant analysis, **337**
regularized empirical risk, **197**
reinforcement learning, **16**, **178**, **768**
reject option, **162**
relational data, **759**
relative entropy, **215**
relevance vector machine, 416, **618**
ReLU, **443**, **466**
reparameterization trick, **704**
representation learning, **654**
representer theorem, **623**
Reproducing Kernel Hilbert Space, **623**
reproducing property, **623**
reservoir computing, **532**
reset gate, **533**
reshape, **233**
residual block, **467**, **497**
residual error, **107**
residual network, **467**
residual plot, **388**
residual sum of squares, **106**, **381**
residuals, **8**, 388
ResNet, **467**, **497**
ResNet-18, **499**
response, **2**
response variables, **800**
responsibility, **86**, **321**, **480**
reverse KL, **218**
reverse mode differentiation, **453**
reward, **17**, **171**, **176**
reward function, **279**
reward hacking, **26**
RFF, **604**
ridge regression, **116**, 154, 189, **389**, 473
Riemannian manifold, **707**
Riemannian metric, **708**
risk, **159**, **192**
risk averse, **161**, 162
risk neutral, **161**
risk sensitive, **161**
RKHS, **623**
RKHS kernel, **623**
RL, **16**
RMSE, **107**, **389**
RNN, **440**, **523**
Robbins-Monro conditions, **301**
robust, **8**, **57**, **168**
robust linear regression, 312
robust logistic regression, **369**
robust optimization, **518**
robustness, **393**
ROC, **164**
root mean squared error, **107**, **388**
rotation matrix, **243**
row rank, **239**
row-major order, **233**
RSS, **106**
rule of iterated expectation, **97**
rule of total probability, **36**
running sum, **111**
rv, **33**
RVM, **618**

saddle point, **281**, 284
SAGA, **304**, 355
same convolution, **487**
SAMME, **639**
Sammon mapping, **712**
sample efficiency, **16**
sample mean, 151
sample size, **2**, **102**, 122
sample space, **33**
sample variance, **135**
sampling distribution, **146**
SARS-CoV-2, **42**
saturates, **443**
scalar field, **270**
scalar product, **244**
scalars, **234**
scale of evidence, 184
scaled dot-product attention, **541**
scatter matrix, **105**, **248**
Schatten p-norm, **237**
scheduled sampling, **530**
Schur complement, **252**
score function, **147**, **279**, **700**
scree plot, **682**
second order, **356**
Second-order, **292**
self attention, **546**
self-normalization property, **729**
self-supervised, **653**
self-supervised learning, **15**
self-training, **657**, **668**
SELU, **467**
semantic role labeling, **366**
semantic segmentation, **505**
semantic textual similarity, **545**
semi-hard negatives, **574**
semi-supervised embeddings, **786**
Semi-supervised learning, **657**
semi-supervised learning, 347, 651
semidefinite embedding, **715**
semidefinite programming, **570**, **715**
sensible PCA, **686**
sensitivity, **42**, **164**
sensor fusion, **84**
sentiment analysis, **21**, **445**
seq-to-seq model, **21**
seq2seq, **527**
seq2vec, **526**
sequence logo, **211**
sequential decision problem, **176**
sequential minimal optimization, **608**
SGD, **299**
SGNS, **727**
shaded nodes, **95**
shallow parsing, **559**
Shannon's source coding theorem, **211**
shared, **335**
sharp minima, **475**
sharpness aware minimization, **475**
Sherman-Morrison formula, **253**
Sherman-Morrison-Woodbury formula, **253**
shooting, **408**
short and fat, **4**
shrinkage, **134**, **393**
shrinkage estimation, **114**
shrinkage factor, **401**, **641**
Siamese network, **573**, 654
side information, **767**
sifting property, **57**, 140
sigmoid, **48**, **140**, 336
significance, **202**
silhouette coefficient, **745**, 746
silhouette diagram, **746**
silhouette score, **746**
SimCLR, **654**
similarity, **565**
simple hypothesis, **202**
simple linear regression, **9**, **379**
simple regret, **180**
simplex algorithm, **312**
Simpson's paradox, **76**
Simpsons paradox, 385
Simulated annealing, **328**
single link clustering, **737**
single shot detector, **505**
singular, **251**
singular statistical model, **191**
singular value decomposition, **260**
singular values, 240, **260**
singular vectors, **260**
skip connections, 499
skip-gram with negative sampling, **727**
skipgram, 725
skipgram model, **727**
slack variables, **609**, **616**
slate, **768**
slope, **9**
slot machines, 176
SMACOF, **711**
SMO, **608**
smooth optimization, **285**
SNLI, **544**
Sobel edge detector, **509**
social networks, **787**
soft clustering, **87**
soft margin constraints, **609**
soft thresholding, **401**, 403
soft thresholding operator, **316**
soft triple, **575**
softmax, **51**, 422
softmax function, **7**
Softplus, 465
softplus, **56**
solver, **279**
source dataset, **651**
source domain, **656**
span, **234**
sparse, 131, **397**
sparse Bayesian learning, **414**
sparse factor analysis, 691
sparse GP, **603**
sparse kernel machine, **477**, 568
sparse linear regression, 313
sparse transformers, **561**
sparse vector machines, **618**
sparsity inducing regularizer, **316**
specificity, **42**
spectral clustering, **752**
spectral CNNs, **781**
spectral embedding, **716**
spectral graph theory, **719**
spectral radius, **464**
spherical covariance matrix, **79**
spherical embedding constraint, **577**
split variable trick, **408**
spurious correlation, **75**
spurious correlations, 76
square, **232**
square-integrable functions, **622**
square-root sampling, **368**
square-root schedule, **302**
squared error, **168**
squared exponential kernel, **588**
stacking, **633**
standard basis, **235**
standard deviation, **39**, **55**
standard error, **125**
standard error of the mean, **118**, **135**
standard form, **311**
standard normal, **53**
standardization operation, 248
standardize, **359**, **383**
standardized, 322
standardizing, 256
Stanford Natural Language Inference, **544**
state of nature, **159**, **176**
state space, **33**
state transition matrix, **93**
state transition model, **178**
static graph, **459**
stationary, **93**
stationary kernels, **589**
stationary point, **280**
statistical learning theory, **199**
statistical machine translation, **529**
statistical relational learning, **789**
statistically significant, **202**
statistics, **26**
steepest descent, **288**
Stein's paradox, **195**
step decay, **302**
step size, **287**
stochastic averaged gradient accelerated, **304**
stochastic bandit, **177**
stochastic gradient boosting, **642**
stochastic gradient descent, **299**, 355
stochastic gradient descent with warm restarts, **303**
stochastic local search, **328**
stochastic matrix, **93**
stochastic neighbor embedding, **719**
stochastic optimization, **299**
stochastic variance reduced gradient, **304**
stochastic volatility model, **447**
Stochastic Weight Averaging, **303**
stochastic weight averaging, **663**
stop word removal, **23**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

storks, 76
straight-through estimator, **317**
strain, **710**
stream-based active learning, **668**
stress function, **711**
strict, 195
strict local minimum, **280**
strictly concave, **282**
strictly convex, **282**
strided convolution, **488**
string kernel, **594**
strong learner, **636**
strongly convex, **284**
structural deep network embedding, **783**
structural risk minimization, **198**
STS Benchmark, **545**
STSB, **562**
Student distribution, **57**
Student t distribution, 57
subderivative, 456
subdifferentiable, **286**
subdifferential, **286**
subgradient, **286**
submodular, **670**
subword units, **25**
sufficient statistic, **226**
sufficient statistics, **102**, **104**, **122**, 419, **420**
sum of squares matrix, **248**
sum rule, **36**
supervised learning, **1**
supervised PCA, **695**
support vector machine, **605**
support vector machine regression, **617**
support vectors, **605**, **608**, **617**
surface normal prediction, **506**
surrogate function, **318**, **329**
surrogate loss function, **108**
surrogate splits, **630**
suspicious coincidence, 185
SVD, 115, **260**, 391
SVM, 313, **605**
SVM regression, **617**
SVRG, **304**
Swish, 465
swish, **467**
Swiss roll, **709**
symmetric, **232**
symmetric SNE, **720**
synaptic connection, 449
synchronous training, **472**
syntactic sugar, **95**
systems of linear equations, 266

t-SNE, **719**
T5, **562**
tabular data, **3**
tall and skinny, **4**
tangent space, **707**
target, **2**, **379**
target dataset, **651**
target domain, **656**
target neighbors, **570**
targeted attack, **515**
targets, **800**
tasks, **670**
taxonomy, **366**
Taylor series, 144
Taylor series expansion, 228
teacher forcing, **529**
temperature, **51**
tempered cross entropy, **370**
tempered softmax, **370**
template matching, **483**, **486**
tensor, **232**, **489**
tensor processing units, **450**
term frequency matrix, **23**
term-document frequency matrix, **723**
test and roll, **171**
test risk, **12**
test set, **12**
test statistic, **201**
text to speech, **537**
textual entailment, **543**
TF-IDF, **23**
The Pile, **561**
thin SVD, **260**
Thompson sampling, **183**
threat models, **515**
TICe, **224**
tied, **335**
Tikhonov damping, **296**
Tikhonov regularization, **296**
time series forecasting, 525
time-invariant, **93**
TinyImages, 18
TL;DR, **561**
token, **23**
topological instability, **713**
topological order, **91**
total derivative, **271**
total differential, **271**
total regret, **180**
total variation, **509**
TPUs, **450**
trace, **238**
trace norm, **237**
trace trick, **238**
tracing, **459**
training, **6**, **99**
training data, **800**
training set, **2**
transductive learning, **662**
transfer learning, **556**, 577, **651**
transformer, **546**
transition function, **93**
transition kernel, **93**
translation invariance, **483**
transpose, **232**
transposed convolution, **502**, 506
treatment, **171**
treewidth, 251
Tri-cube kernel, 579
Tri-Training, **661**
triangle inequality, **215**
tridiagonal, **241**, 587
trigram model, **93**
triplet loss, **573**
true negative rate, **42**
true positive rate, **42**, **164**
truncate, 531
truncated Gaussian, **434**
truncated SVD, **263**
trust-region optimization, **295**
tube, **616**
Turing machine, **524**
TV, **509**
type I error rate, **164**, **202**
type II maximum likelihood, **137**
typical patterns, **16**

U-net, **506**
U-shaped curve, **12**
UCB, **181**
UMAP, **723**
unadjusted Langevin algorithm, **508**
unbiased, **151**
uncertainty, **31**
unconditionally independent, **37**
unconstrained optimization, **281**
undercomplete representation, **698**
underdetermined, **266**
underfitting, **12**, **117**, 120
unidentifiable, 365
uninformative, **124**, **136**
uninformative prior, 135
union bound, **200**
uniqueness, **685**
unit vector, **231**
unit vectors, **49**
unitary, **243**
universal function approximator, **447**
UNK, **25**
unrolled, **95**
unstable, **632**
unstructured data, **440**
unsupervised learning, **13**
unsupervised pre-training, **653**
untargeted attack, **516**
update gate, **533**
upper confidence bound, **181**
upper triangular matrix, **241**
users, **759**
utility function, **160**

VAE, **703**
valid convolution, **487**
validation risk, **117**, **199**
validation set, **12**, **117**, **198**
value of information, **669**
values, **538**
vanishing gradient problem, **443**, **464**
variable metric, **294**
variable selection, **403**
variance, **38**, **55**
variation of information, **735**
variational autoencoder, **15**, 664, **703**
variational autoencoders, 692
variational EM, **320**, 321
variational inference, **145**, 419
variational RNN, **525**
varimax, **691**

VC dimension, **200**
vec2seq, **523**
vector, **231**
vector addition, 728
vector field, **270**, **700**
vector Jacobian product, **272**
vector quantization, **741**
vector space, **234**
vector space model, **23**
VI, **145**
vicinal risk minimization, **650**
virtual adversarial training, **663**
visible variables, **800**
vision as inverse graphics, **45**
ViT, **551**
Viterbi decoding, **535**
VJP, **272**
Voronoi iteration, **743**
Voronoi tessellation, **566**, **740**
VQ, **741**

wake sleep, 703
Wald interval, **151**
warm start, **393**
warm starting, **409**
Watanabe–Akaike information criterion, **191**
Watson, **162**
wavenet, **537**
weak learner, **636**
website testing, **174**
WebText, **561**
weight decay, **116**, **359**, **389**, **473**
weight space, **597**
weighted least squares, **384**, 395
weighted least squares problem, 357
weighted linear regression, **384**
weights, **10**, **379**, **439**
well-conditioned, **239**
whitebox attack, **516**
whiten, **257**
wide and deep, **766**
wide data, **4**
wide format, **22**
wide resnet, **499**
widely applicable information criterion, **191**
Widrow-Hoff rule, **301**
winner takes all, **51**
WMT dataset, **21**
Wolfe conditions, **294**
word analogy problem, **728**
word embeddings, **24**, **723**, 723
word sense disambiguation, **557**
word stemming, **23**
word2vec, 706, **725**
wordpieces, **25**
working response, **357**
World Health Organization, 224
WSD, **557**

Xavier initialization, **469**
XGBoost, **643**
XOR problem, **440**

YOLO, **505**

ZCA, **258**
zero count, **113**
zero-avoiding, **218**
zero-forcing, **219**
zero-one loss, **6**, **161**
zero-padding, **487**
zero-shot learning, **671**
zero-shot task transfer, **561**
zig-zag, **290**

Bibliography

[AB08] C. Archambeau and F. Bach. "Sparse probabilistic projections". In: *NIPS*. 2008.

[AB14] G. Alain and Y. Bengio. "What Regularized Auto-Encoders Learn from the Data-Generating Distribution". In: *JMLR* (2014).

[ABM10] J.-Y. Audibert, S. Bubeck, and R. Munos. "Best Arm Identification in Multi-Armed Bandits". In: *COLT*. 2010, pp. 41–53.

[AC16] D. K. Agarwal and B.-C. Chen. *Statistical Methods for Recommender Systems*. en. 1st edition. Cambridge University Press, Feb. 2016.

[AC93] J. Albert and S. Chib. "Bayesian analysis of binary and polychotomous response data". In: *JASA* 88.422 (1993), pp. 669–679.

[AEH+18] S. Abu-El-Haija, B. Perozzi, R. Al-Rfou, and A. A. Alemi. "Watch your step: Learning node embeddings via graph attention". In: *Advances in Neural Information Processing Systems*. 2018, pp. 9180–9190.

[AEHPAR17] S. Abu-El-Haija, B. Perozzi, and R. Al-Rfou. "Learning Edge Representations via Low-Rank Asymmetric Projections". In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. CIKM '17. 2017, 1787–1796.

[AFF19] C. Aicher, N. J. Foti, and E. B. Fox. "Adaptively Truncating Backpropagation Through Time to Control Gradient Bias". In: (May 2019). arXiv: 1905.07473 [cs.LG].

[AG13] S. Agrawal and N. Goyal. "Further Optimal Regret Bounds for Thompson Sampling". In: *AISTATS*. 2013.

[Aga+14] D. Agarwal, B. Long, J. Traupman, D. Xin, and L. Zhang. "LASER: a scalable response prediction platform for online advertising". In: *WSDM*. 2014.

[Agg16] C. C. Aggarwal. *Recommender Systems: The Textbook*. en. 1st ed. 2016 edition. Springer, Mar. 2016.

[Agg20] C. C. Aggarwal. *Linear Algebra and Optimization for Machine Learning: A Textbook*. en. 1st ed. 2020 edition. Springer, May 2020.

[AGM19] V. Amrhein, S. Greenland, and B. McShane. "Scientists rise up against statistical significance". In: *Nature* 567.7748 (Mar. 2019), p. 305.

[Agr70] A. Agrawala. "Learning with a probabilistic teacher". In: *IEEE Transactions on Information Theory* 16.4 (1970), pp. 373–379.

[AH19] C. Allen and T. Hospedales. "Analogies Explained: Towards Understanding Word Embeddings". In: *ICML*. 2019.

[AHK12] A. Anandkumar, D. Hsu, and S. M. Kakade. "A Method of Moments for Mixture Models and Hidden Markov Models". In: *COLT*. Vol. 23. Proceedings of Machine Learning Research. Edinburgh, Scotland: PMLR, 2012, pp. 33.1–33.34.

[Ahm+13] A. Ahmed, N. Shervashidze, S. Narayanamurthy, V. Josifovski, and A. J. Smola. "Distributed large-scale natural graph factorization". In: *Proceedings of the 22nd international conference on World Wide Web*. ACM. 2013, pp. 37–48.

[AIW19] B. Athiwaratkun, M. F. P. Izmailov, and A. G. Wilson. "There Are Many Consistent Explanations of Unlabeled Data: Why You Should Average". In: *ICLR*. 2019.

[AK15] J. Andreas and D. Klein. "When and why are log-linear models self-normalizing?" In: *Proc. ACL*. Denver, Colorado: Association for Computational Linguistics, 2015, pp. 244–249.

[Aka74] H. Akaike. "A new look at the statistical model identification". In: *IEEE Trans. on Automatic Control* 19.6 (1974).

[AKA91] D. W. Aha, D. Kibler, and M. K. Albert. "Instance-based learning algorithms". In: *Mach. Learn.* 6.1 (Jan. 1991), pp. 37–66.

[AL13] N. Ailon and E. Liberty. "An Almost Optimal Unrestricted Fast Johnson-Lindenstrauss Transform". In: *ACM Trans. Algorithms* 9.3 (2013), 21:1–21:12.

[Alb+17] M. Alber, P.-J. Kindermans, K. Schütt, K.-R. Müller, and F. Sha. "An Empirical Study on The Properties of Random Bases for Kernel Methods". In: *NIPS*. Curran Associates, Inc., 2017, pp. 2763–2774.

[Alb+18] D. Albanese, S. Riccadonna, C. Donati, and P. Franceschi. "A practical tool for maximal information coefficient analysis". en. In: *Gigascience* 7.4 (Apr. 2018), pp. 1–8.

[ALL18] S. Arora, Z. Li, and K. Lyu. "Theoretical Analysis of Auto Rate-Tuning by Batch Normalization". In: (Dec. 2018). arXiv: 1812.03981 [cs.LG].

[Alm87] L. B. Almeida. "A learning rule for asynchronous perceptrons with feedback in a combinatorial environment." In: *Proceedings, 1st First International Conference on Neural Networks*. Vol. 2. IEEE. 1987, pp. 609–618.

[Alo+09] D. Aloise, A. Deshpande, P. Hansen, and P. Popat. "NP-hardness of Euclidean sum-of-squares clustering". In: *Machine Learning* 75 (2009), pp. 245–249.

[Alp04] E. Alpaydin. *Introduction to machine learning*. MIT Press, 2004.

[AM74] D. Andrews and C. Mallows. "Scale mixtures of Normal distributions". In: *JRSSB* 36 (1974), pp. 99–102.

[Ama98] S Amari. "Natural Gradient Works Efficiently in Learning". In: *Neural Comput.* 10.2 (Feb. 1998), pp. 251–276.

[AMH19] A. Aubret, L. Matignon, and S. Hassas. "A survey on intrinsic motivation in reinforcement learning". In: (Aug. 2019). arXiv: 1908.06976 [cs.LG].

[Ami+19] E. Amid, M. K. Warmuth, R. Anil, and T. Koren. "Robust Bi-Tempered Logistic Loss Based on Bregman Divergences". In: *NIPS*. 2019.

[Amo+16] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. "Concrete Problems in AI Safety". In: (June 2016). arXiv: 1606.06565 [cs.AI].

[Amo17] Amoeba. *What is the difference between ZCA whitening and PCA whitening*. Stackexchange. 2017.

[And01] C. A. Anderson. "Heat and Violence". In: *Current Directions in Psychological Science* 10.1 (2001), pp. 33–38.

[And+18] R. Anderson, J. Huchette, C. Tjandraatmadja, and J. P. Vielma. "Strong convex relaxations and mixed-integer programming formulations for trained neural networks". In: (Nov. 2018). arXiv: 1811.01988 [math.OC].

[Ani+18] R. Anirudh, J. J. Thiagarajan, B. Kailkhura, and T. Bremer. "An Unsupervised Approach to Solving Inverse Problems using Generative Adversarial Networks". In: (May 2018). arXiv: 1805.07281 [cs.CV].

[AO03] J.-H. Ahn and J.-H. Oh. "A Constrained EM Algorithm for Principal Component Analysis". In: *Neural Computation* 15 (2003), pp. 57–65.

[AP19] M. Abadi and G. D. Plotkin. "A simple differentiable programming language". In: *Proceedings of the ACM on Programming Languages* 4.POPL (2019), pp. 1–28.

[Arc+19] F. Arcadu, F. Benmansour, A. Maunz, J. Willis, Z. Haskova, and M. Prunotto. "Deep learning algorithm predicts diabetic retinopathy progression in individual patients". en. In: *NPJ Digit Med* 2 (Sept. 2019), p. 92.

[Ard+20] R. Ardila et al. "Common Voice: A Massively-Multilingual Speech Corpus". In: *Proceedings of The 12th Language Resources and Evaluation Conference*. 2020, pp. 4218–4222.

[Arn+19] S. M. R. Arnold, P.-A. Manzagol, R. Babanezhad, I. Mitliagkas, and N. Le Roux. "Reducing the variance in online optimization by transporting past gradients". In: *NIPS*. 2019.

[Aro+16] S. Arora, Y. Li, Y. Liang, T. Ma, and A. Risteski. "A Latent Variable Model Approach to PMI-based Word Embeddings". In: *TACL* 4 (Dec. 2016), pp. 385–399.

[Aro+19] L. Aroyo, A. Dumitrache, O. Inel, Z. Szlávik, B. Timmermans, and C. Welty. "Crowdsourcing Inclusivity: Dealing with Diversity of Opinions, Perspectives and Ambiguity in Annotated Data". In: *WWW*. WWW '19. San Francisco, USA: Association for Computing Machinery, May 2019, pp. 1294–1295.

[ARZP19] R. Al-Rfou, D. Zelle, and B. Perozzi. "DDGK: Learning Graph Representations for Deep Divergence Graph Kernels". In: *Proceedings of the 2019 World Wide Web Conference on World Wide Web* (2019).

[AS17] A. Achille and S. Soatto. "On the Emergence of Invariance and Disentangling in Deep Representations". In: (June 2017). arXiv: 1706.01350 [cs.LG].

[AS19] A. Achille and S. Soatto. "Where is the Information in a Deep Neural Network?" In: (May 2019). arXiv: 1905.12213 [cs.LG].

[Ash18] J. Asher. "A Rise in Murder? Let's Talk About the Weather". In: *The New York Times* (Sept. 2018).

[ASR15] A. Ali, S. M. Shamsuddin, and A. L. Ralescu. "Classification with class imbalance problem: A Review". In: *Int. J. Advance Soft Compu. Appl* 7.3 (2015).

[Aue12] J. E. Auerbach. "Automated evolution of interesting images". In: *Artificial Life 13*. 2012.

[AV07] D. Arthur and S. Vassilvitskii. "k-means++: the advantages of careful seeding". In: *Proc. 18th ACM-SIAM symp. on Discrete algorithms*. 2007, 1027–1035.

[AWS19] E. Amid, M. K. Warmuth, and S. Srinivasan. "Two-temperature logistic regression based on the Tsallis divergence". In: *AISTATS*. 2019.

[Axl15] S. Axler. *Linear algebra done right*. 2015.

[Aze+20] E. M. Azevedo, A. Deng, J. L. Montiel Olea, J. Rao, and E. G. Weyl. "A/B Testing with Fat Tails". In: *J. Polit. Econ.* (July 2020), pp. 000–000.

[BA10] R. Bailey and J. Addison. *A Smoothed-Distribution Form of Nadaraya-Watson Estimation*. Tech. rep. 10-30. Univ. Birmingham, 2010.

[BA97a] A. Bowman and A. Azzalini. *Applied Smoothing Techniques for Data Analysis*. Oxford, 1997.

[BA97b] L. A. Breslow and D. W. Aha. "Simplifying decision trees: A survey". In: *Knowl. Eng. Rev.* 12.1 (Jan. 1997), pp. 1–40.

[Bab19] S. Babu. *A 2019 guide to Human Pose Estimation with Deep Learning*. 2019.

[Bac+16] O. Bachem, M. Lucic, H. Hassani, and A. Krause. "Fast and Provably Good Seedings for k-Means". In: *NIPS*. 2016, pp. 55–63.

[Bah+12] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii. "Scalable k-Means++". In: *VLDB*. 2012.

[Bah+20] Y. Bahri, J. Kadmon, J. Pennington, S. Schoenholz, J. Sohl-Dickstein, and S. Ganguli. "Statistical Mechanics of Deep Learning". In: *Annu. Rev. Condens. Matter Phys.* (Mar. 2020).

[Bal17] S. Baluja. "Learning deep models of optimization landscapes". In: *IEEE Symposium Series on Computational Intelligence (SSCI)* (2017).

[BAP14] P. Bachman, O. Alsharif, and D. Precup. "Learning with pseudo-ensembles". In: *Advances in neural information processing systems*. 2014, pp. 3365–3373.

[Bar09] M. Bar. "The proactive brain: memory for predictions". en. In: *Philos. Trans. R. Soc. Lond. B Biol. Sci.* 364.1521 (May 2009), pp. 1235–1243.

[Bar19] J. T. Barron. "A General and Adaptive Robust Loss Function". In: *CVPR*. 2019.

[Bat+18] P. W. Battaglia et al. "Relational inductive biases, deep learning, and graph networks". In: *arXiv preprint arXiv:1806.01261* (2018).

[BB08] O. Bousquet and L. Bottou. "The Tradeoffs of Large Scale Learning". In: *NIPS*. 2008, pp. 161–168.

[BB11] L. Bottou and O. Bousquet. "The Tradeoffs of Large Scale Learning". In: *Optimization for Machine Learning*. Ed. by S. Sra, S. Nowozin, and S. J. Wright. MIT Press, 2011, pp. 351–368.

[BB12] J. Bergstra and Y. Bengio. "Random Search for Hyper-Parameter Optimization". In: *JMLR* 13 (2012), pp. 281–305.

[BBV11] R. Benassi, J. Bect, and E. Vazquez. "Bayesian optimization using sequential Monte Carlo". In: (Nov. 2011). arXiv: 1111.4802 [math.OC].

[BC17] D. Beck and T. Cohn. "Learning Kernels over Strings using Gaussian Processes". In: *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*. Vol. 2. 2017, pp. 67–73.

[BCB12] S. Bubeck and N. Cesa-Bianchi. "Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems". In: *Foundations and Trends® in Machine Learning* 5.1 (2012), pp. 1–122.

[BCB15] D. Bahdanau, K. Cho, and Y. Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *ICLR*. 2015.

[BCD01] L. Brown, T. Cai, and A. DasGupta. "Interval Estimation for a Binomial Proportion". In: *Statistical Science* 16.2 (2001), pp. 101–133.

[BCN18] L. Bottou, F. E. Curtis, and J. Nocedal. "Optimization Methods for Large-Scale Machine Learning". In: *SIAM Rev.* 60.2 (2018), pp. 223–311.

[BCV13] Y. Bengio, A. Courville, and P. Vincent. "Representation learning: a review and new perspectives". en. In: *IEEE PAMI* 35.8 (Aug. 2013), pp. 1798–1828.

[BD17] S. Boyd and J. Duchi. *Lecture notes for EE364b: Convex Optimization II*. Stanford Univ., 2017.

[BD21] D. G. T. Barrett and B. Dherin. "Implicit Gradient Regularization". In: *ICLR*. 2021.

[BD87] G. Box and N. Draper. *Empirical Model-Building and Response Surfaces*. Wiley, 1987.

[BDEL03] S. Ben-David, N. Eiron, and P. M. Long. "On the difficulty of approximately maximizing agreements". In: *J. Comput. System Sci.* 66.3 (May 2003), pp. 496–514.

[Ben+04a] Y. Bengio, O. Delalleau, N. Roux, J. Paiement, P. Vincent, and M. Ouimet. "Learning eigenfunctions links spectral embedding and kernel PCA". In: *Neural Computation* 16 (2004), pp. 2197–2219.

[Ben+04b] Y. Bengio, J.-F. Paiement, P. Vincent, O. Delalleau, N. L. Roux, and M. Ouimet. "Out-of-Sample Extensions for LLE, Isomap, MDS, Eigenmaps, and Spectral Clustering". In: *NIPS*. MIT Press, 2004, pp. 177–184.

[Ben+15a] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks". In: *NIPS*. 2015.

[Ben+15b] Y. Bengio, D.-H. Lee, J. Bornschein, T. Mesnard, and Z. Lin. "Towards Biologically Plausible Deep Learning". In: (2015). arXiv: 1502.04156 [cs.LG].

[Ber15] D. Bertsekas. *Convex Optimization Algorithms*. Athena Scientific, 2015.

[Ber16] D. Bertsekas. *Nonlinear Programming*. Third. Athena Scientific, 2016.

[Ber19] D. Bertsekas. *Reinforcement learning and optimal control*. Athena Scientific, 2019.

[Ber+19a] D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, and C. Raffel. "Mixmatch: A holistic approach to semi-supervised learning". In: *Advances in Neural Information Processing Systems*. 2019, pp. 5049–5059.

[Ber+19b] D. Berthelot et al. "Remixmatch: Semi-supervised learning with distribution alignment and augmentation anchoring". In: *arXiv preprint arXiv:1911.09785* (2019).

[Ber85] J. Berger. "Bayesian Salesmanship". In: *Bayesian Inference and Decision Techniques with Applications: Essays in Honor of Bruno deFinetti*. Ed. by P. K. Goel and A. Zellner. North-Holland, 1985.

[Ber99] D. Bertsekas. *Nonlinear Programming*. Second. Athena Scientific, 1999.

[Bey+19] M. Beyeler, E. L. Rounds, K. D. Carlson, N. Dutt, and J. L. Krichmar. "Neural correlates of sparse coding and dimensionality reduction". en. In: *PLoS Comput. Biol.* 15.6 (June 2019), e1006908.

[BFO84] L. Breiman, J. Friedman, and R. Olshen. *Classification and regression trees*. Wadsworth, 1984.

[BG11] P. Buhlmann and S. van de Geer. *Statistics for High-Dimensional Data: Methodology, Theory and Applications*. Springer, 2011.

[BH07] P. Buhlmann and T. Hothorn. "Boosting Algorithms: Regularization, Prediction and Model Fitting". In: *Statistical Science* 22.4 (2007), pp. 477–505.

[BH69] A. Bryson and Y.-C. Ho. *Applied optimal control: optimization, estimation, and control*. Blaisdell Publishing Company, 1969.

[BH86] J. Barnes and P. Hut. "A hierarchical O(N log N) force-calculation algorithm". In: *Nature* 324.6096 (Dec. 1986), pp. 446–449.

[BH89] P. Baldi and K. Hornik. "Neural networks and principal components analysis: Learning from examples without local minima". In: *Neural Networks* 2 (1989), pp. 53–58.

[Bha+19] A. Bhadra, J. Datta, N. G. Polson, and B. T. Willard. "Lasso Meets Horseshoe: a survey". In: *Bayesian Anal.* 34.3 (2019), pp. 405–427.

[BHM92] J. S. Bridle, A. J. Heading, and D. J. MacKay. "Unsupervised Classifiers, Mutual Information and'Phantom Targets". In: *Advances in neural information processing systems*. 1992, pp. 1096–1101.

[BI19] P. Barham and M. Isard. "Machine Learning Systems are Stuck in a Rut". In: *Proceedings of the Workshop on Hot Topics in Operating Systems*. HotOS '19. Bertinoro, Italy: Association for Computing Machinery, May 2019, pp. 177–183.

[Big+11] B. Biggio, G. Fumera, I. Pillai, and F. Roli. "A survey and experimental evaluation of image spam filtering techniques". In: *Pattern recognition letters* 32.10 (2011), pp. 1436–1446.

[Bis06] C. Bishop. *Pattern recognition and machine learning*. Springer, 2006.

[Bis94] C. M. Bishop. *Mixture Density Networks*. Tech. rep. NCRG 4288. Neural Computing Research Group, Department of Computer Science, Aston University, 1994.

[Bis99] C. Bishop. "Bayesian PCA". In: *NIPS*. 1999.

[BJ05] F. Bach and M. Jordan. *A probabilistic interpretation of canonical correlation analysis*. Tech. rep. 688. U. C. Berkeley, 2005.

[BJM06] P. Bartlett, M. Jordan, and J. McAuliffe. "Convexity, Classification, and Risk Bounds". In: *JASA* 101.473 (2006), pp. 138–156.

[BK07] R. M. Bell and Y. Koren. "Lessons from the Netflix Prize Challenge". In: *SIGKDD Explor. Newsl.* 9.2 (Dec. 2007), pp. 75–79.

[BK20] E. M. Bender and A. Koller. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *ACL* (2020).

[BKC17] V. Badrinarayanan, A. Kendall, and R. Cipolla. "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation". In: *IEEE PAMI* 39.12 (2017).

[BKH16] J. L. Ba, J. R. Kiros, and G. E. Hinton. "Layer Normalization". In: (2016). arXiv: 1607.06450 [stat.ML].

[BKL10] S. Bird, E. Klein, and E. Loper. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. 2010.

[BL04] P. Bickel and E. Levina. "Some theory for Fisher's linear discriminant function, "Naive Bayes", and some alternatives when there are many more variables than observations". In: *Bernoulli* 10 (2004), pp. 989–1010.

[BL07] J. A. Bullinaria and J. P. Levy. "Extracting semantic representations from word co-occurrence statistics: a computational study". en. In: *Behav. Res. Methods* 39.3 (Aug. 2007), pp. 510–526.

[BL12] J. A. Bullinaria and J. P. Levy. "Extracting semantic representations from word co-occurrence statistics: stop-lists, stemming, and SVD". en. In: *Behav. Res. Methods* 44.3 (Sept. 2012), pp. 890–907.

[BL88] D. S. Broomhead and D Lowe. "Multivariable Functional Interpolation and Adaptive Networks". In: *Complex Systems* (1988).

[BLK17] O. Bachem, M. Lucic, and A. Krause. "Distributed and provably good seedings for k-means in constant rounds". In: *ICML*. 2017, pp. 292–300.

[BLM16] S. Boucheron, G. Lugosi, and P. Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2016.

[Blo20] M. Blondel. *Automatic differentiation*. 2020.

[BLV19] X. Bouthillier, C. Laurent, and P. Vincent. "Unreproducible Research is Reproducible". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. Long Beach, California, USA: PMLR, 2019, pp. 725–734.

[BM98] A. Blum and T. Mitchell. "Combining labeled and unlabeled data with co-training". In: *Proceedings of the eleventh annual conference on Computational learning theory*. 1998, pp. 92–100.

[BMS11] S. Bubeck, R. Munos, and G. Stoltz. "Pure Exploration in Finitely-armed and Continuous-armed Bandits". In: *Theoretical Computer Science* 412.19 (2011), pp. 1832–1852.

[BN01] M. Belkin and P. Niyogi. "Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering". In: *NIPS*. 2001, pp. 585–591.

[BNJ03] D. Blei, A. Ng, and M. Jordan. "Latent Dirichlet allocation". In: *JMLR* 3 (2003), pp. 993–1022.

[Bo+08] L. Bo, C. Sminchisescu, A. Kanaujia, and D. Metaxas. "Fast Algorithms for Large Scale Conditional 3D Prediction". In: *CVPR*. 2008.

[Boe+05] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein. "A Tutorial on the Cross-Entropy Method". en. In: *Ann. Oper. Res.* 134.1 (Feb. 2005), pp. 19–67.

[Boh92] D. Bohning. "Multinomial logistic regression algorithm". In: *Annals of the Inst. of Statistical Math.* 44 (1992), pp. 197–200.

[Bon13] S. Bonnabel. "Stochastic gradient descent on Riemannian manifolds". In: *IEEE Transactions on Automatic Control* 58.9 (2013), pp. 2217–2229.

[Bos+16] D. Boscaini, J. Masci, E. Rodolà, and M. Bronstein. "Learning shape correspondence with anisotropic con-

volutional neural networks". In: *Advances in Neural Information Processing Systems*. 2016, pp. 3189–3197.

[Bot+13] L. Bottou et al. "Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising". In: *JMLR* 14 (2013), pp. 3207–3260.

[Bow+15] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning. "A large annotated corpus for learning natural language inference". In: *EMNLP*. Lisbon, Portugal: Association for Computational Linguistics, Sept. 2015, pp. 632–642.

[BPC20] I. Beltagy, M. E. Peters, and A. Cohan. "Longformer: The Long-Document Transformer". In: *CoRR* abs/2004.05150 (2020). arXiv: 2004.05150.

[BR18] B. Biggio and F. Roli. "Wild patterns: Ten years after the rise of adversarial machine learning". In: *Pattern Recognition* 84 (2018), pp. 317–331.

[Bra+18] J. Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.1.55. 2018.

[Bre01] L. Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32.

[Bre96] L. Breiman. "Bagging predictors". In: *Machine Learning* 24 (1996), pp. 123–140.

[Bri50] G. W. Brier. "Verification of forecasts expressed in terms of probability". In: *Monthly Weather Review* 78.1 (Jan. 1950), pp. 1–3.

[Bri90] J. Bridle. "Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition". In: *Neurocomputing: Algorithms, Architectures and Applications*. Ed. by F. F. Soulie and J. Herault. Springer Verlag, 1990, pp. 227–236.

[Bro+17a] M. M. Bronstein, J Bruna, Y LeCun, A Szlam, and P Vandergheynst. "Geometric Deep Learning: Going beyond Euclidean data". In: *IEEE Signal Process. Mag.* 34.4 (July 2017), pp. 18–42.

[Bro+17b] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst. "Geometric deep learning: going beyond euclidean data". In: *IEEE Signal Processing Magazine* 34.4 (2017), pp. 18–42.

[Bro19] J. Brownlee. *Deep Learning for Computer Vision - Machine Learning Mastery*. Accessed: 2020-6-30. Machine Learning Mastery, Feb. 2019.

[Bro+20] T. B. Brown et al. "Language Models are Few-Shot Learners". In: (May 2020). arXiv: 2005.14165 [cs.CL].

[Bro+21] A. Brock, S. De, S. L. Smith, and K. Simonyan. "High-Performance Large-Scale Image Recognition Without Normalization". In: (Feb. 2021). arXiv: 2102 . 06171 [cs.CV].

[BRR18] T. D. Bui, S. Ravi, and V. Ramavajjala. "Neural Graph Machines: Learning Neural Networks Using Graphs". In: *WSDM*. 2018.

[Bru+14] J. Bruna, W. Zaremba, A. Szlam, and Y. Lecun. "Spectral networks and locally connected networks on graphs International Conference on Learning Representations (ICLR2014)". In: *CBLS, April* (2014).

[Bru+19] G. Brunner, Y. Liu, D. Pascual, O. Richter, and R. Wattenhofer. "On the Validity of Self-Attention as Explanation in Transformer Models". In: (Aug. 2019). arXiv: 1908.04211 [cs.CL].

[BS02] M. Balasubramanian and E. L. Schwartz. "The isomap algorithm and topological stability". en. In: *Science* 295.5552 (Jan. 2002), p. 7.

[BS16] P. Baldi and P. Sadowski. "A Theory of Local Learning, the Learning Channel, and the Optimality of Backpropagation". In: *Neural Netw.* 83 (2016), pp. 51–74.

[BS17] D. M. Blei and P. Smyth. "Science and data science". en. In: *Proc. Natl. Acad. Sci. U. S. A.* (Aug. 2017).

[BS94] J. Bernardo and A. Smith. *Bayesian Theory*. John Wiley, 1994.

[BS97] A. J. Bell and T. J. Sejnowski. "The "independent components" of natural scenes are edge filters". en. In: *Vision Res.* 37.23 (Dec. 1997), pp. 3327–3338.

[BT00] C. Bishop and M. Tipping. "Variational relevance vector machines". In: *UAI*. 2000.

[BT08] D. Bertsekas and J. Tsitsiklis. *Introduction to Probability*. 2nd Edition. Athena Scientific, 2008.

[BT09] A Beck and M Teboulle. "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems". In: *SIAM J. Imaging Sci.* 2.1 (Jan. 2009), pp. 183–202.

[BTEGN09] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski. *Robust optimization*. Vol. 28. Princeton University Press, 2009.

[Bul11] A. D. Bull. "Convergence rates of efficient global optimization algorithms". In: *JMLR* 12 (2011), 2879–2904.

[Bur10] C. J. C. Burges. "Dimension Reduction: A Guided Tour". en. In: *Foundations and Trends in Machine Learning* (July 2010).

[BV04] S. Boyd and L. Vandenberghe. *Convex optimization*. Cambridge, 2004.

[BW08] P. L. Bartlett and M. H. Wegkamp. "Classification with a Reject Option using a Hinge Loss". In: *JMLR* 9.Aug (2008), pp. 1823–1840.

[BW88] J. Berger and R. Wolpert. *The Likelihood Principle*. 2nd edition. The Institute of Mathematical Statistics, 1988.

[BWL19] Y. Bai, Y.-X. Wang, and E. Liberty. "ProxQuant: Quantized Neural Networks via Proximal Operators". In: *ICLR*. 2019.

[BY03] P. Buhlmann and B. Yu. "Boosting with the L2 loss: Regression and classification". In: *JASA* 98.462 (2003), pp. 324–339.

[Byr+16] R Byrd, S Hansen, J Nocedal, and Y Singer. "A Stochastic Quasi-Newton Method for Large-Scale Optimization". In: *SIAM J. Optim.* 26.2 (Jan. 2016), pp. 1008–1031.

[BZ20] A. Barbu and S.-C. Zhu. *Monte Carlo Methods*. en. Springer, 2020.

[Cao+18] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh. "OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields". In: (Dec. 2018). arXiv: 1812.08008 [cs.CV].

[Car+19] N. Carlini et al. "On evaluating adversarial robustness". In: *arXiv preprint arXiv:1902.06705* (2019).

[CAS16] P. Covington, J. Adams, and E. Sargin. "Deep Neural Networks for YouTube Recommendations". In: *Proceedings of the 10th ACM Conference on Recommender Systems*. RecSys '16. Boston, Massachusetts, USA: Association for Computing Machinery, Sept. 2016, pp. 191–198.

[CB02] G. Casella and R. Berger. *Statistical inference*. 2nd edition. Duxbury, 2002.

[CBD15] M. Courbariaux, Y. Bengio, and J.-P. David. "BinaryConnect: Training Deep Neural Networks with binary weights during propagations". In: *NIPS*. 2015.

[CC07] H. Choi and S. Choi. "Robust kernel Isomap". In: *Pattern Recognit.* 40.3 (Mar. 2007), pp. 853–862.

[CCD17] B. P. Chamberlain, J. Clough, and M. P. Deisenroth. "Neural embeddings of graphs in hyperbolic space". In: *arXiv preprint arXiv:1705.10359* (2017).

[CD14] K. Chaudhuri and S. Dasgupta. "Rates of Convergence for Nearest Neighbor Classification". In: *NIPS*. 2014.

[CD88] W. Cleveland and S. Devlin. "Locally-Weighted Regression: An Approach to Regression Analysis by Local Fitting". In: *JASA* 83.403 (1988), pp. 596–610.

[CDL19] S. Chen, E. Dobriban, and J. H. Lee. "Invariance reduces Variance: Understanding Data Augmentation in Deep Learning and Beyond". In: (July 2019). arXiv: 1907.10905 [stat.ML].

[CDS02] M. Collins, S. Dasgupta, and R. E. Schapire. "A Generalization of Principal Components Analysis to the Exponential Family". In: *NIPS-14*. 2002.

[CEL19] Z. Chen, J. B. Estrach, and L. Li. "Supervised community detection with line graph neural networks". In: *7th International Conference on Learning Representations, ICLR 2019*. 2019.

[Cer+17] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia. "SemEval-2017 Task 1: Semantic Textual Similarity Multilingual and Crosslingual Focused Evaluation". In: *Proc. 11th Intl. Workshop on Semantic Evaluation (SemEval-2017)*. Vancouver, Canada: Association for Computational Linguistics, Aug. 2017, pp. 1–14.

[CFD10] Y. Cui, X. Z. Fern, and J. G. Dy. "Learning Multiple Nonredundant Clusterings". In: *ACM Transactions on Knowledge Discovery from Data* 4.3 (2010).

[CG16] T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *KDD*. ACM, 2016, pp. 785–794.

[CG18] J. Chen and Q. Gu. "Closing the Generalization Gap of Adaptive Gradient Methods in Training Deep Neural Networks". In: (June 2018). arXiv: 1806.06763 [cs.LG].

[CGG17] S. E. Chazan, J. Goldberger, and S. Gannot. "Speech Enhancement using a Deep Mixture of Experts". In: (2017). arXiv: 1703.09302 [cs.SD].

[CH67] T. Cover and P. Hart. " Nearest neighbor pattern classification". In: *IEEE Trans. Inform. Theory* 13.1 (1967), pp. 21–27.

[CH90] K. W. Church and P. Hanks. "Word Association Norms, Mutual Information, and Lexicography". In: *Computational Linguistics* (1990).

[Cha+01] O. Chapelle, J. Weston, L. Bottou, and V. Vapnik. "Vicinal Risk Minimization". In: *NIPS*. MIT Press, 2001, pp. 416–422.

[Cha+19a] I. Chami, Z. Ying, C. Ré, and J. Leskovec. "Hyperbolic graph convolutional neural networks". In: *Advances in Neural Information Processing Systems*. 2019, pp. 4869–4880.

[Cha+19b] J. J. Chandler, I. Martinez, M. M. Finucane, J. G. Terziev, and A. M. Resch. "Speaking on Data's Behalf: What Researchers Say and How Audiences Choose". en. In: *Eval. Rev.* (Mar. 2019), p. 193841X19834968.

[Cha+20] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy. "Machine Learning on Graphs: A Model and Comprehensive Taxonomy". In: *arXiv preprint arXiv:2005.03675* (2020).

[Che+16] H.-T. Cheng et al. "Wide & Deep Learning for Recommender Systems". In: (June 2016). arXiv: 1606.07792 [cs.LG].

[Che+20a] M. Chen et al. "Generative Pretraining From Pixels". In: *ICML*. Ed. by H. D. Iii and A. Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 1691–1703.

[Che+20b] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. "A Simple Framework for Contrastive Learning of Visual Representations". In: *ICML*. 2020.

[Che+20c] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. "A simple framework for contrastive learning of visual representations". In: *ICML*. 2020.

[Che+20d] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. Hinton. "Big Self-Supervised Models are Strong Semi-Supervised Learners". In: *NIPS*. 2020.

[Chi+19a] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh. "Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks". In: *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 2019.

[Chi+19b] R. Child, S. Gray, A. Radford, and I. Sutskever. "Generating Long Sequences with Sparse Transformers". In: *CoRR* abs/1904.10509 (2019). arXiv: 1904.10509.

[CHL05] S. Chopra, R. Hadsell, and Y. LeCun. "Learning a Similarity Metric Discriminatively, with Application to Face Verification". en. In: *CVPR*. 2005.

[Cho+14a] K. Cho, B. Van Merriënboer, D. Bahdanau, and Y. Bengio. "On the properties of neural machine translation: Encoder-decoder approaches". In: *arXiv preprint arXiv:1409.1259* (2014).

[Cho+14b] K. Cho et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". In: *EMNLP*. 2014.

[Cho+15] Y. Chow, A. Tamar, S. Mannor, and M. Pavone. "Risk-Sensitive and Robust Decision-Making: a CVaR Optimization Approach". In: *NIPS*. 2015, pp. 1522–1530.

[Cho17] F. Chollet. *Deep learning with Python*. Manning, 2017.

[Cho+19] K. Choromanski, M. Rowland, W. Chen, and A. Weller. "Unifying Orthogonal Monte Carlo Methods". In: *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*. Ed. by K. Chaudhuri and R. Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 1203–1212.

[Cho+20a] K. Choromanski et al. "Masked Language Modeling for Proteins via Linearly Scalable Long-Context Transformers". In: (June 2020). arXiv: 2006.03555 [cs.LG].

[Cho+20b] K. Choromanski et al. "Rethinking Attention with Performers". In: *CoRR* abs/2009.14794 (2020). arXiv: 2009.14794.

[Cho21] F. Chollet. *Deep learning with Python (second edition)*. Manning, 2021.

[Chu+15] J. Chung, K. Kastner, L. Dinh, K. Goel, A. Courville, and Y. Bengio. "A Recurrent Latent Variable Model for Sequential Data". In: *NIPS*. 2015.

[Chu97] F. Chung. *Spectral Graph Theory*. AMS, 1997.

[Cir+10] D. C. Ciresan, U. Meier, L. M. Gambardella, and J. Schmidhuber. "Deep Big Simple Neural Nets For Handwritten Digit Recognition". In: *Neural Computation* 22.12 (2010), pp. 3207–3220.

[Cir+11] D. C. Ciresan, U. Meier, J. Masci, L. M. Gambardella, and J. Schmidhuber. "Flexible, High Performance Convolutional Neural Networks for Image Classification". In: *IJCAI*. 2011.

[CL11] O. Chapelle and L. Li. "An empirical evaluation of Thompson sampling". In: *NIPS*. 2011.

[CL96] B. P. Carlin and T. A. Louis. *Bayes and Empirical Bayes Methods for Data Analysis*. Chapman and Hall, 1996.

[CLX15] S. Cao, W. Lu, and Q. Xu. "Grarep: Learning graph representations with global structural information". In: *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. ACM. 2015, pp. 891–900.

[CNB17] C. Chelba, M. Norouzi, and S. Bengio. "N-gram Language Modeling using Recurrent Neural Network Estimation". In: (Mar. 2017). arXiv: 1703.10724 [cs.CL].

[Coh+17] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik. "EMNIST: an extension of MNIST to handwritten letters". In: (Feb. 2017). arXiv: 1702.05373 [cs.CV].

[Coh94] J. Cohen. "The earth is round ($p < .05$)". In: *American Psychologist* 49.12 (1994), pp. 997–1003.

[Con+17] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes. "Supervised learning of universal sentence representations from natural language inference data". In: *arXiv preprint arXiv:1705.02364* (2017).

[Cor+16] C. Cortes, X. Gonzalvo, V. Kuznetsov, M. Mohri, and S. Yang. "AdaNet: Adaptive Structural Learning of Artificial Neural Networks". In: (July 2016). arXiv: 1607.01097 [cs.LG].

[Cor+87] A Corana, M Marchesi, C Martini, and S Ridella. "Minimizing Multimodal Functions of Continuous Variables with the "Simulated Annealing" Algorithm". In: *ACM Trans. Math. Softw.* 13.3 (Sept. 1987), pp. 262–280.

[CP10] M. A. Carreira-Perpinan. "The Elastic Embedding Algorithm for Dimensionality Reduction". In: *ICML*. 2010.

[CP19] A. Coenen and A. Pearce. *Understanding UMAP*. 2019.

[CPS06] K. Chellapilla, S. Puri, and P. Simard. "High Performance Convolutional Neural Networks for Document Processing". In: *10th Intl. Workshop on Frontiers in Handwriting Recognition*. 2006.

[CRK19] J. M. Cohen, E. Rosenfeld, and J. Z. Kolter. "Certified adversarial robustness via randomized smoothing". In: *arXiv preprint arXiv:1902.02918* (2019).

[CRW17] K. M. Choromanski, M. Rowland, and A. Weller. "The Unreasonable Effectiveness of Structured Random Orthogonal Embeddings". In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*. Ed. by I. Guyon et al. 2017, pp. 219–228.

[CS07] D. Calvetti and E. Somersalo. *Introduction to Bayesian Scientific Computing*. Springer, 2007.

[CS20] F. E. Curtis and K Scheinberg. "Adaptive Stochastic Optimization: A Framework for Analyzing Stochastic Optimization Algorithms". In: *IEEE Signal Process. Mag.* 37.5 (Sept. 2020), pp. 32–42.

[Csu17] G. Csurka. "Domain Adaptation for Visual Applications: A Comprehensive Survey". In: *Domain Adaptation in Computer Vision Applications*. Ed. by G. Csurka. 2017.

[CT06] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. 2nd edition. John Wiley, 2006.

[CT91] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. John Wiley, 1991.

[Cub+19] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le. "AutoAugment: Learning Augmentation Policies from Data". In: *CVPR*. 2019.

[CUH16] D.-A. Clevert, T. Unterthiner, and S. Hochreiter. "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)". In: *ICLR*. 2016.

[Cui+19] X. Cui, K. Zheng, L. Gao, B. Zhang, D. Yang, and J. Ren. "Multiscale Spatial-Spectral Convolutional Network with Image-Based Framework for Hyperspectral Imagery Classification". en. In: *Remote Sensing* 11.19 (Sept. 2019), p. 2220.

[Cur+17] J. D. Curtó et al. "McKernel: A Library for Approximate Kernel Expansions in Log-linear Time". In: (Feb. 2017). arXiv: 1702.08159v14 [cs.LG].

[Cyb89] G. Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals, and Systems* 2 (1989), 303–331.

[D'A+20] A. D'Amour et al. "Underspecification Presents Challenges for Credibility in Modern Machine Learning". In: (Nov. 2020). arXiv: 2011.03395 [cs.LG].

[Dah+11] G. E. Dahl, D. Yu, L. Deng, and A. Acero. "Large vocabulary continuous speech recognition with context-dependent DBN-HMMS". In: *ICASSP*. IEEE, 2011, pp. 4688–4691.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[Dai+19] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context". In: *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*. Ed. by A. Korhonen, D. R. Traum, and L. Màrquez. Association for Computational Linguistics, 2019, pp. 2978–2988.

[Dal+04] N. Dalvi, P. Domingos, S. Sanghai, and D. Verma. "Adversarial classification". In: *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*. 2004, pp. 99–108.

[Dao+19] T. Dao, A. Gu, A. J. Ratner, V. Smith, C. De Sa, and C. Re. "A Kernel Theory of Modern Data Augmentation". In: *ICML*. 2019.

[Dau04] H. Daume. *From Zero to Reproducing Kernel Hilbert Spaces in Twelve Pages or Less*. 2004.

[Day+95] P. Dayan, G. Hinton, R. Neal, and R. Zemel. "The Helmholtz machine". In: *Neural Networks* 9.8 (1995).

[DB18] A. Defazio and L. Bottou. "On the Ineffectiveness of Variance Reduced Optimization for Deep Learning". In: (Dec. 2018). arXiv: 1812.04529 [cs.LG].

[DBLJ14] A. Defazio, F. Bach, and S. Lacoste-Julien. "SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives". In: *NIPS*. Curran Associates, Inc., 2014, pp. 1646–1654.

[DDDM04] I Daubechies, M Defrise, and C De Mol. "An iterative thresholding algorithm for linear inverse problems with a sparsity constraint". In: *Commun. Pure Appl. Math.* Advances in E 57.11 (Nov. 2004), pp. 1413–1457.

[DE04] J. Dow and J. Endersby. "Multinomial probit and multinomial logit: a comparison of choice models for voting research". In: *Electoral Studies* 23.1 (2004), pp. 107–122.

[Dee+90] S. Deerwester, S. Dumais, G. Furnas, T. Landauer, and R. Harshman. "Indexing by Latent Semantic Analysis". In: *J. of the American Society for Information Science* 41.6 (1990), pp. 391–407.

[DeG70] M. DeGroot. *Optimal Statistical Decisions*. McGraw-Hill, 1970.

[Den+12] J. Deng, J Krause, A. C. Berg, and L. Fei-Fei. "Hedging your bets: Optimizing accuracy-specificity trade-offs in large scale visual recognition". In: *CVPR*. June 2012, pp. 3450–3457.

[Den+14] J. Deng et al. "Large-Scale Object Classification using Label Relation Graphs". In: *ECCV*. 2014.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *NAACL*. 2019.

[Dex] *Dex: Research language for array processing in the Haskell/ML family*. https://github.com/google-research/dex-lang. 2019.

[DG06] J. Davis and M. Goadrich. "The Relationship Between Precision-Recall and ROC Curves". In: *ICML*. 2006, pp. 233–240.

[DHM07] P. Diaconis, S. Holmes, and R. Montgomery. "Dynamical Bias in the Coin Toss". In: *SIAM Review* 49.2 (2007), pp. 211–235.

[DHS01] R. O. Duda, P. E. Hart, and D. G. Stork. *Pattern Classification*. 2nd edition. Wiley Interscience, 2001.

[DHS11] J. Duchi, E. Hazan, and Y. Singer. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization". In: *JMLR* 12 (2011), pp. 2121–2159.

[Din+15] N. Ding, J. Deng, K. Murphy, and H. Neven. "Probabilistic Label Relation Graphs with Ising Models". In: *ICCV*. 2015.

[DJ15] S. Dray and J. Josse. "Principal component analysis with missing values: a comparative survey of methods". In: *Plant Ecol.* 216.5 (May 2015), pp. 657–667.

[DKK12] G Dror, N Koenigstein, and Y Koren. "Web-Scale Media Recommendation Systems". In: *Proc. IEEE* 100.9 (2012), pp. 2722–2736.

[DKS95] J. Dougherty, R. Kohavi, and M. Sahami. "Supervised and Unsupervised Discretization of Continuous Features". In: *ICML*. 1995.

[DLR77] A. P. Dempster, N. M. Laird, and D. B. Rubin. "Maximum likelihood from incomplete data via the EM algorithm". In: *J. of the Royal Statistical Society, Series B* 34 (1977), pp. 1–38.

[DM01] D. van Dyk and X.-L. Meng. "The Art of Data Augmentation". In: *J. Computational and Graphical Statistics* 10.1 (2001), pp. 1–50.

[DM16] P. Drineas and M. W. Mahoney. "RandNLA: Randomized Numerical Linear Algebra". In: *CACM* (June 2016).

[Do+19] T.-T. Do, T. Tran, I. Reid, V. Kumar, T. Hoang, and G. Carneiro. "A Theoretically Sound Upper Bound on the Triplet Loss for Improving the Efficiency of Deep Distance Metric Learning". In: *CVPR*. 2019, pp. 10404–10413.

[Don95] D. L. Donoho. "De-noising by soft-thresholding". In: *IEEE Trans. Inf. Theory* 41.3 (May 1995), pp. 613–627.

[Dos+21] A. Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *ICLR*. 2021.

[Doy+07] K. Doya, S. Ishii, A. Pouget, and R. P. N. Rao, eds. *Bayesian Brain: Probabilistic Approaches to Neural Coding*. MIT Press, 2007.

[DP97] P. Domingos and M. Pazzani. "On the Optimality of the Simple Bayesian Classifier under Zero-One Loss". In: *Machine Learning* 29 (1997), pp. 103–130.

[Dri+04] P. Drineas, A. Frieze, R. Kannan, S. Vempala, and V. Vinay. "Clustering Large Graphs via the Singular Value Decomposition". In: *Machine Learning* 56 (2004), pp. 9–33.

[Dru08] J. Drugowitsch. *Bayesian linear regression*. Tech. rep. U. Rochester, 2008.

[DS12] M. Der and L. K. Saul. "Latent Coincidence Analysis: A Hidden Variable Model for Distance Metric Learning". In: *NIPS*. Curran Associates, Inc., 2012, pp. 3230–3238.

[DSK16] V. Dumoulin, J. Shlens, and M. Kudlur. "A Learned Representation For Artistic Style". In: (2016). arXiv: 1610.07629 [cs.CV].

[Dum+18] A. Dumitrache et al. "Empirical Methodology for Crowdsourcing Ground Truth". In: *Semantic Web Journal* (Sept. 2018).

[Duv14] D. Duvenaud. "Automatic Model Construction with Gaussian Processes". PhD thesis. Computational and Biological Learning Laboratory, University of Cambridge, 2014.

[DV16] V. Dumoulin and F. Visin. "A guide to convolution arithmetic for deep learning". In: (2016). arXiv: 1603.07285 [stat.ML].

[DY79] P. Diaconis and D. Ylvisaker. "Conjugate priors for exponential families". In: vol. 7. 1979, pp. 269–281.

[EDH19] K. Ethayarajh, D. Duvenaud, and G. Hirst. "Towards Understanding Linear Word Analogies". In: *Proc. ACL*. Association for Computational Linguistics, July 2019, pp. 3253–3262.

[EF15] D. Eigen and R. Fergus. "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture". In: *ICCV*. 2015.

[Efr+04] B. Efron, I. Johnstone, T. Hastie, and R. Tibshirani. "Least angle regression". In: *Annals of Statistics* 32.2 (2004), pp. 407–499.

[Efr86] B. Efron. "Why Isn't Everyone a Bayesian?" In: *The American Statistician* 40.1 (1986).

[Ein16] A Einstein. "Die Grundlage der allgemeinen Relativitätstheorie". In: *Ann. Phys.* 354.7 (1916), pp. 769–822.

[Eis19] J. Eisenstein. *Introduction to Natural Language Processing*. 2019.

[Elk03] C. Elkan. "Using the triangle inequality to accelerate k-means". In: *ICML*. 2003.

[EMH19] T. Elsken, J. H. Metzen, and F. Hutter. "Neural Architecture Search: A Survey". In: *JMLR* 20 (2019), pp. 1–21.

[Erh+10] D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, and S. Bengio. "Why Does Unsupervised Pretraining Help Deep Learning?" In: *JMLR* 11 (2010), pp. 625–660.

[Eyk+18] K. Eykholt et al. "Robust Physical-World Attacks on Deep Learning Models". In: *CVPR*. 2018.

[FAL17] C. Finn, P. Abbeel, and S. Levine. "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks". In: *ICML*. 2017.

[FB19] E. M. Feit and R. Berman. "Test & Roll: Profit-Maximizing A/B Tests". In: *Marketing Science* 38.6 (Nov. 2019), pp. 1038–1058.

[FB81] M. A. Fischler and R. Bolles. "Random sample concensus: A paradigm for model fitting with applications to image analysis and automated cartography". In: *Comm. ACM* 24.6 (1981), pp. 381–395.

[Fer+10] D. Ferrucci et al. "Building Watson: An Overview of the DeepQA Project". In: *AI Magazine* (2010), pp. 59–79.

[FH20] E. Fong and C. Holmes. "On the marginal likelihood and cross-validation". In: *Biometrika* 107.2 (May 2020).

[FHK12] A. Feuerverger, Y. He, and S. Khatri. "Statistical Significance of the Netflix Challenge". In: *Stat. Sci.* 27.2 (May 2012), pp. 202–231.

[FHT00] J. Friedman, T. Hastie, and R. Tibshirani. "Additive logistic regression: a statistical view of boosting". In: *Annals of statistics* 28.2 (2000), pp. 337–374.

[FHT10] J. Friedman, T. Hastie, and R. Tibshirani. "Regularization Paths for Generalized Linear Models via Coordinate Descent". In: *J. of Statistical Software* 33.1 (2010).

[Fir57] J. Firth. "A synopsis of linguistic theory 1930-1955". In: *Studies in Linguistic Analysis*. Ed. by F. Palmer. 1957.

[FJ02] M. A. T. Figueiredo and A. K. Jain. "Unsupervised Learning of Finite Mixture Models". In: *IEEE PAMI* 24.3 (2002), pp. 381–396.

[FJL18] R. Frostig, M. J. Johnson, and C. Leary. "Compiling machine learning programs via high-level tracing". In: *Machine Learning and Systems (MLSys)* (2018).

[FM03] J. H. Friedman and J. J. Meulman. "Multiple additive regression trees with application in epidemiology". en. In: *Stat. Med.* 22.9 (May 2003), pp. 1365–1381.

[FMN16] C. Fefferman, S. Mitter, and H. Narayanan. "Testing the manifold hypothesis". In: *J. Amer. Math. Soc.* 29.4 (Feb. 2016), pp. 983–1049.

[FNW07] M. Figueiredo, R. Nowak, and S. Wright. "Gradient projection for sparse reconstruction: application to compressed sensing and other inverse problems". In: *IEEE. J. on Selected Topics in Signal Processing* (2007).

[For+19] N. Ford, J. Gilmer, N. Carlini, and D. Cubuk. "Adversarial Examples Are a Natural Consequence of Test Error in Noise". In: (Jan. 2019). arXiv: 1901.10513 [cs.LG].

[For+21] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur. "Sharpness-aware Minimization for Efficiently Improving Generalization". In: *ICLR*. 2021.

[Fos19] D. Foster. *Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play*. 1 edition. O'Reilly Media, July 2019.

[FR07] C. Fraley and A. Raftery. "Bayesian Regularization for Normal Mixture Estimation and Model-Based Clustering". In: *J. of Classification* 24 (2007), pp. 155–181.

[Fra+17] L. Franceschi, M. Donini, P. Frasconi, and M. Pontil. "Forward and Reverse Gradient-Based Hyperparameter Optimization". In: *ICML*. 2017.

[Fre98] B. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, 1998.

[Fri01] J. Friedman. "Greedy Function Approximation: a Gradient Boosting Machine". In: *Annals of Statistics* 29 (2001), pp. 1189–1232.

[Fri97a] J. Friedman. "On bias, variance, 0-1 loss and the curse of dimensionality". In: *J. Data Mining and Knowledge Discovery* 1 (1997), pp. 55–77.

[Fri97b] J. H. Friedman. "Data mining and statistics: What's the connection". In: *Proceedings of the 29th Symposium on the Interface Between Computer Science and Statistics*. 1997.

[Fri99] J. Friedman. *Stochastic Gradient Boosting*. Tech. rep. 1999.

[Fro+21] R. Frostig, M. Johnson, D. Maclaurin, A. Paszke, and A. Radul. "Decomposing reverse-mode automatic differentiation". In: *LAFI workshop at POPL 2021*. 2021.

[FS96] Y. Freund and R. R. Schapire. "Experiments with a new boosting algorithm". In: *ICML*. 1996.

[FSF10] S. Fruhwirth-Schnatter and R. Fruhwirth. "Data Augmentation and MCMC for Binary and Multinomial Logit Models". In: *Statistical Modelling and Regression Structures*. Ed. by T. Kneib and G. Tutz. Springer, 2010, pp. 111–132.

[FT05] M. Fashing and C. Tomasi. "Mean shift is a bound optimization". en. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 27.3 (Mar. 2005), pp. 471–474.

[Fu98] W. Fu. "Penalized regressions: the bridge versus the lasso". In: *J. Computational and graphical statistics* 7 (1998), 397– 416.

[Fuk75] K. Fukushima. "Cognitron: a self-organizing multilayered neural network". In: *Biological Cybernetics* 20.6 (1975), pp. 121–136.

[Fuk80] K Fukushima. "Neocognitron: a self organizing neural network model for a mechanism of pattern recognition unaffected by shift in position". en. In: *Biol. Cybern.* 36.4 (1980), pp. 193–202.

[Fuk90] K. Fukunaga. *Introduction to Statistical Pattern Recognition*. 2nd edition. Academic Press, 1990.

[FXL18] C. Finn, K. Xu, and S. Levine. "Probabilistic Model-Agnostic Meta-Learning". In: *NIPS*. 2018.

[Gag94] P. Gage. "A New Algorithm for Data Compression". In: *Dr Dobbs Journal* (1994).

[Gan+16] Y Ganin, E Ustinova, H Ajakan, P Germain, and others. "Domain-adversarial training of neural networks". In: *JMLR* (2016).

[Gao+20] L. Gao et al. "The Pile: An 800GB Dataset of Diverse Text for Language Modeling". In: (Dec. 2020). arXiv: 2101.00027 [cs.CL].

[Gär03] T. Gärtner. "A Survey of Kernels for Structured Data". In: *SIGKDD Explor. Newsl.* 5.1 (July 2003), pp. 49–58.

[Gar+18] J. Gardner, G. Pleiss, K. Q. Weinberger, D. Bindel, and A. G. Wilson. "GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration". In: *NIPS*. Ed. by S Bengio, H Wallach, H Larochelle, K Grauman, N Cesa-Bianchi, and R Garnett. Curran Associates, Inc., 2018, pp. 7576–7586.

[GASG18] D. G. A. Smith and J. Gray. "opt-einsum - A Python package for optimizing contraction order for einsum-like expressions". In: *JOSS* 3.26 (June 2018), p. 753.

[GB05] Y. Grandvalet and Y. Bengio. "Semi-supervised learning by entropy minimization". In: *Advances in neural information processing systems*. 2005, pp. 529–536.

[GB10] X. Glorot and Y. Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *AISTATS*. 2010, pp. 249–256.

[GB18] V. Garcia and J. Bruna. "Few-shot Learning with Graph Neural Networks". In: *International Conference on Learning Representations (ICLR)*. 2018.

[GBB11] X. Glorot, A. Bordes, and Y. Bengio. "Deep Sparse Rectifier Neural Networks". In: *AISTATS*. 2011.

[GBC16] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[GBD92] S. Geman, E. Bienenstock, and R. Doursat. "Neural networks and the bias-variance dilemma". In: *Neural Computing* 4 (1992), pp. 1–58.

[GC20] A. Gelman and B. Carpenter. "Bayesian analysis of tests with unknown specificity and sensitivity". In: *medRxiv* medrxiv;2020.05.22.20108944v2 (May 2020).

[GEB16] L. A. Gatys, A. S. Ecker, and M. Bethge. "Image style transfer using convolutional neural networks". In: *CVPR*. 2016, pp. 2414–2423.

[GEH19] T. Gale, E. Elsen, and S. Hooker. "The State of Sparsity in Deep Neural Networks". In: (Feb. 2019). arXiv: 1902.09574 [cs.LG].

[Gel+04] A. Gelman, J. Carlin, H. Stern, and D. Rubin. *Bayesian data analysis*. 2nd edition. Chapman and Hall, 2004.

[Gel16] A. Gelman. "The problems with p-values are not just with p-values". In: *American Statistician* (2016).

[Gér17] A. Géron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques for Building Intelligent Systems*. en. O'Reilly Media, Incorporated, Apr. 2017.

[Gér19] A. Géron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques for Building Intelligent Systems (2nd edition)*. en. O'Reilly Media, Incorporated, 2019.

[GEY19] Y. Geifman and R. El-Yaniv. "SelectiveNet: A Deep Neural Network with an Integrated Reject Option". In: *ICML*. 2019.

[GG16] Y. Gal and Z. Ghahramani. "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning". In: *ICML*. 2016.

[GH96] Z. Ghahramani and G. Hinton. *The EM Algorithm for Mixtures of Factor Analyzers*. Tech. rep. Dept. of Comp. Sci., Uni. Toronto, 1996.

[GHK17] Y. Gal, J. Hron, and A. Kendall. "Concrete Dropout". In: (May 2017). arXiv: 1705.07832 [stat.ML].

[GHV14] A. Gelman, J. Hwang, and A. Vehtari. "Understanding predictive information criteria for Bayesian models". In: *Statistics and Computing* 24.6 (Nov. 2014), pp. 997–1016.

[Gil+17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. "Neural message passing for quantum chemistry". In: *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org. 2017, pp. 1263–1272.

[Gil+18a] J. Gilmer, R. P. Adams, I. Goodfellow, D. Andersen, and G. E. Dahl. "Motivating the rules of the game

for adversarial example research". In: *arXiv preprint arXiv:1807.06732* (2018).

[Gil+18b] J. Gilmer et al. "Adversarial spheres". In: *arXiv preprint arXiv:1801.02774* (2018).

[GIM99] A. Gionis, P. Indyk, and R. Motwani. "Similarity Search in High Dimensions via Hashing". In: *Proc. 25th Intl. Conf. on Very Large Data Bases*. VLDB '99. 1999, pp. 518–529.

[Git89] J. Gittins. *Multi-armed Bandit Allocation Indices*. Wiley, 1989.

[GK19] L. Graesser and W. L. Keng. *Foundations of Deep Reinforcement Learning: Theory and Practice in Python*. en. 1 edition. Addison-Wesley Professional, Dec. 2019.

[GL15] B. Gu and C. Ling. "A New Generalized Error Path Algorithm for Model Selection". In: *ICML*. 2015.

[GL16] A. Grover and J. Leskovec. "node2vec: Scalable feature learning for networks". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2016, pp. 855–864.

[GMS05] M. Gori, G. Monfardini, and F. Scarselli. "A new model for learning in graph domains". In: *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.* Vol. 2. IEEE. 2005, pp. 729–734.

[GNK18] R. A. Güler, N. Neverova, and I. Kokkinos. "Densepose: Dense human pose estimation in the wild". In: *CVPR*. 2018, pp. 7297–7306.

[God18] P. Godec. https://towardsdatascience.com/graph-embeddings-the-summary-cc6075aba007. 2018.

[GOF18] O. Gouvert, T. Oberlin, and C. Févotte. "Negative Binomial Matrix Factorization for Recommender Systems". In: (Jan. 2018). arXiv: 1801.01708 [cs.LG].

[Gol+01] K. Goldberg, T. Roeder, D. Gupta, and C. Perkins. "Eigentaste: A Constant Time Collaborative Filtering Algorithm". In: *Information Retrieval* 4.2 (2001), pp. 133–151.

[Gol+05] J. Goldberger, S. Roweis, G. Hinton, and R. Salakhutdinov. "Neighbourhood Components Analysis". In: *NIPS*. 2005.

[Gol+92] D. Goldberg, D. Nichols, B. M. Oki, and D. Terry. "Using collaborative filtering to weave an information tapestry". In: *Commun. ACM* 35.12 (Dec. 1992), pp. 61–70.

[Gom+17] A. N. Gomez, M. Ren, R. Urtasun, and R. B. Grosse. "The Reversible Residual Network: Backpropagation Without Storing Activations". In: *NIPS*. 2017.

[Gon85] T. Gonzales. "Clustering to minimize the maximum intercluster distance". In: *Theor. Comp. Sci.* 38 (1985), pp. 293–306.

[Goo01] N. Goodman. "Classes for fast maximum entropy training". In: *ICASSP*. 2001.

[Goo+14] I. J. Goodfellow et al. "Generative Adversarial Networks". In: *NIPS*. 2014.

[Gor06] P. F. Gorder. "Neural Networks Show New Promise for Machine Vision". In: *Computing in science & engineering* 8.6 (2006), pp. 4–8.

[Got+19] A. Gotmare, N. S. Keskar, C. Xiong, and R. Socher. "A Closer Look at Deep Learning Heuristics: Learning rate restarts, Warmup and Distillation". In: *ICLR*. 2019.

[GOV18] W Gao, S Oh, and P Viswanath. "Demystifying Fixed k -Nearest Neighbor Information Estimators". In: *IEEE Trans. Inf. Theory* 64.8 (Aug. 2018), pp. 5629–5661.

[GR06] M. Girolami and S. Rogers. "Variational Bayesian multinomial probit regression with Gaussian process priors". In: *Neural Comptuation* 18.8 (2006), pp. 1790–1817.

[GR07] T. Gneiting and A. E. Raftery. "Strictly Proper Scoring Rules, Prediction, and Estimation". In: *JASA* 102.477 (2007), pp. 359–378.

[GR18] A. Graves and M.-A. Ranzato. "Tutorial on unsupervised deep learning: part 2". In: *NIPS*. 2018.

[Gra04] Y. Grandvalet. "Bagging Equalizes Influence". In: *Mach. Learn.* 55 (2004), pp. 251–270.

[Gra+10] T. Graepel, J. Quinonero-Candela, T. Borchert, and R. Herbrich. "Web-Scale Bayesian Click-Through Rate Prediction for Sponsored Search Advertising in Microsoft's Bing Search Engine". In: *ICML*. 2010.

[Gra11] A. Graves. "Practical variational inference for neural networks". In: *Advances in neural information processing systems*. 2011, pp. 2348–2356.

[Gra13] A. Graves. "Generating Sequences With Recurrent Neural Networks". In: (Aug. 2013). arXiv: 1308.0850 [cs.NE].

[Gra+17] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou. "Efficient softmax approximation for GPUs". In: *ICML*. 2017.

[Gra+18] E. Grant, C. Finn, S. Levine, T. Darrell, and T. Griffiths. "Recasting Gradient-Based Meta-Learning as Hierarchical Bayes". In: *ICLR*. 2018.

[Gra+20] W. Grathwohl, K.-C. Wang, J.-H. Jacobsen, D. Duvenaud, M. Norouzi, and K. Swersky. "Your classifier is secretly an energy based model and you should treat it like one". In: *ICLR*. 2020.

[Gre+17] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber. "LSTM: A Search Space Odyssey". In: *IEEE Transactions on Neural Networks and Learning Systems* 28.10 (2017).

[Gri20] T. L. Griffiths. "Understanding Human Intelligence through Human Limitations". en. In: *Trends Cogn. Sci.* 24.11 (Nov. 2020), pp. 873–883.

[GS08] Y Guo and D Schuurmans. "Efficient global optimization for exponential family PCA and low-rank matrix factorization". In: *2008 46th Annual Allerton Conference on Communication, Control, and Computing*. Sept. 2008, pp. 1100–1107.

[GS97] C. M. Grinstead and J. L. Snell. *Introduction to probability (2nd edition)*. American Mathematical Society, 1997.

[GSK18] S. Gidaris, P. Singh, and N. Komodakis. "Unsupervised Representation Learning by Predicting Image Rotations". In: *ICLR*. 2018.

[GSS15] I. J. Goodfellow, J. Shlens, and C. Szegedy. "Explaining and Harnessing Adversarial Examples". In: *ICLR*. 2015.

[GT07] L. Getoor and B. Taskar, eds. *Introduction to Relational Statistical Learning*. MIT Press, 2007.

[GTA00] G. Gigerenzer, P. M. Todd, and ABC Research Group. *Simple Heuristics That Make Us Smart*. en. Illustrated edition. Oxford University Press, Sept. 2000.

[Gu+18a] A. Gu, F. Sala, B. Gunel, and C. Ré. "Learning Mixed-Curvature Representations in Product Spaces". In: *International Conference on Learning Representations* (2018).

[Gu+18b] J. Gu et al. "Recent Advances in Convolutional Neural Networks". In: *Pattern Recognit.* 77 (2018), pp. 354–377.

[Gua+10] Y. Guan, J. Dy, D. Niu, and Z. Ghahramani. "Variational Inference for Nonparametric Multiple Clustering". In: *1st Intl. Workshop on Discovering, Summarizing and Using Multiple Clustering (MultiClust)*. 2010.

[Gua+17] S. Guadarrama, R. Dahl, D. Bieber, M. Norouzi, J. Shlens, and K. Murphy. "PixColor: Pixel Recursive Colorization". In: *BMVC*. 2017.

[Guo09] Y. Guo. "Supervised exponential family principal component analysis via convex optimization". In: *NIPS*. 2009.

[Guo+17] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He. "DeepFM: a factorization-machine based neural network for CTR prediction". In: *IJCAI*. IJCAI'17. Melbourne, Australia: AAAI Press, Aug. 2017, pp. 1725–1731.

[Gus01] M. Gustafsson. "A probabilistic derivation of the partial least-squares algorithm". In: *Journal of Chemical Information and Modeling* 41 (2001), pp. 288–294.

[GVZ16] A. Gupta, A. Vedaldi, and A. Zisserman. "Synthetic Data for Text Localisation in Natural Images". In: *CVPR*. 2016.

[GZE19] A. Grover, A. Zweig, and S. Ermon. "Graphite: Iterative Generative Modeling of Graphs". In: *International Conference on Machine Learning*. 2019, pp. 2434–2444.

[HA17] W. Hare and C. Audet. *Derivative Free and Black-Box Optimization*. Springer, 2017.

[HA85] L. Hubert and P. Arabie. "Comparing Partitions". In: *J. of Classification* 2 (1985), pp. 193–218.

[HAB19] M. Hein, M. Andriushchenko, and J. Bitterwolf. "Why ReLU networks yield high-confidence predictions far away from the training data and how to mitigate the problem". In: *CVPR*. 2019.

[Hac75] I. Hacking. *The Emergence of Probability: A Philosophical Study of Early Ideas about Probability, Induction and Statistical Inference*. Cambridge University Press, 1975.

[Háj08] A. Hájek. "Dutch Book Arguments". In: *The Oxford Handbook of Rational and Social Choice*. Ed. by P. Anand, P. Pattanaik, and C. Puppe. Oxford University Press, 2008.

[Haj88] B. Hajek. "Cooling Schedules for Optimal Annealing". In: *Math. Oper. Res.* 13.2 (1988), pp. 311–329.

[Har54] Z. Harris. "Distributional structure". In: *Word* 10.23 (1954), pp. 146–162.

[Has+04] T. Hastie, S. Rosset, R. Tibshirani, and J. Zhu. "The entire regularization path for the support vector machine". In: *JMLR* 5 (2004), pp. 1391–1415.

[Has+09] T. Hastie, S. Rosset, J. Zhu, and H. Zou. "Multi-class AdaBoost". In: *Statistics and tis Interface* 2.3 (2009), pp. 349–360.

[Has+17] D. Hassabis, D. Kumaran, C. Summerfield, and M. Botvinick. "Neuroscience-Inspired Artificial Intelligence". en. In: *Neuron* 95.2 (2017), pp. 245–258.

[Has87] J. Hastad. *Computational limits of small-depth circuits*. MIT Press, 1987.

[HB17] X. Huang and S. Belongie. "Arbitrary style transfer in real-time with adaptive instance normalization". In: *ICCV*. 2017.

[HCD12] D. Hoiem, Y. Chodpathumwan, and Q. Dai. "Diagnosing Error in Object Detectors". In: *ECCV*. 2012.

[HCL09] C.-W. Hsu, C.-C. Chang, and C.-J. Lin. *A Practical Guide to Support Vector Classification*. Tech. rep. Dept. Comp. Sci., National Taiwan University, 2009.

[He+15] K. He, X. Zhang, S. Ren, and J. Sun. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *ICCV*. 2015.

[He+16a] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition". In: *CVPR*. 2016.

[He+16b] K. He, X. Zhang, S. Ren, and J. Sun. "Identity Mappings in Deep Residual Networks". In: *ECCV*. 2016.

[He+17] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. "Neural Collaborative Filtering". In: *WWW*. 2017.

[HE18] D. Ha and D. Eck. "A Neural Representation of Sketch Drawings". In: *ICLR*. 2018.

[He+20] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. "Momentum contrast for unsupervised visual representation learning". In: *CVPR*. 2020, pp. 9729–9738.

[Hen+15] J. Hensman, A. Matthews, M. Filippone, and Z. Ghahramani. "MCMC for Variationally Sparse Gaussian Processes". In: *NIPS*. 2015, pp. 1648–1656.

[Hen+20] D. Hendrycks*, N. Mu*, E. D. Cubuk, B. Zoph, J. Gilmer, and B. Lakshminarayanan. "AugMix: A Simple Data Processing Method to Improve Robustness and Uncertainty". In: *ICLR*. 2020.

[HG20] J. Howard and S. Gugger. *Deep Learning for Coders with Fastai and PyTorch: AI Applications Without a PhD*. en. 1st ed. O'Reilly Media, Aug. 2020.

[HGD19] K. He, R. Girshick, and P. Dollár. "Rethinking ImageNet Pre-training". In: *CVPR*. 2019.

[Hin+12] G. E. Hinton et al. "Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups". In: *IEEE Signal Process. Mag.* 29.6 (2012), pp. 82–97.

[Hin13] G. Hinton. *CSC 2535 Lecture 11: Non-linear dimensionality reduction*. 2013.

[Hin14] G. Hinton. *Lecture 6e on neural networks*. 2014.

[HK15] F. M. Harper and J. A. Konstan. "The MovieLens Datasets: History and Context". In: *ACM Trans. Interact. Intell. Syst.* 5.4 (Dec. 2015), pp. 1–19.

[HL04] D. R. Hunter and K. Lange. "A Tutorial on MM Algorithms". In: *The American Statistician* 58 (2004), pp. 30–37.

[HMT11] N. Halko, P.-G. Martinsson, and J. A. Tropp. "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions". In: *SIAM Rev., Survey and Review section* 53.2 (2011), pp. 217–288.

[HN19] C. M. Holmes and I. Nemenman. "Estimation of mutual information for real-valued data with error bars and controlled bias". en. In: *Phys Rev E* 100.2-1 (Aug. 2019), p. 022404.

[Hoc+01] S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. "Gradient flow in recurrent nets: the difficulty of learning long-term dependencies". In: *A Field Guide to Dynamical Recurrent Neural Networks*. Ed. by S. C. Kremer and J. F. Kolen. 2001.

[Hoe+14] R. Hoekstra, R. D. Morey, J. N. Rouder, and E.-J. Wagenmakers. "Robust misinterpretation of confidence intervals". en. In: *Psychon. Bull. Rev.* 21.5 (Oct. 2014), pp. 1157–1164.

[Hoe+21] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste. "Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks". In: (Jan. 2021). arXiv: 2102.00554 [cs.LG].

[Hof09] P. D. Hoff. *A First Course in Bayesian Statistical Methods*. Springer, 2009.

[Hon+10] A. Honkela, T. Raiko, M. Kuusela, M. Tornio, and J. Karhunen. "Approximate Riemannian Conjugate Gradient Learning for Fixed-Form Variational Bayes". In: *JMLR* 11.Nov (2010), pp. 3235–3268.

[Hor61] P Horst. "Generalized canonical correlations and their applications to experimental data". en. In: *J. Clin. Psychol.* 17 (Oct. 1961), pp. 331–347.

[Hor91] K. Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2 (1991), pp. 251–257.

[Hos+18] M. Z. Hossain, F. Sohel, M. F. Shiratuddin, and H. Laga. "A Comprehensive Survey of Deep Learning for Image Captioning". In: *ACM Computing Surveys* (2018).

[HOT06] G. Hinton, S. Osindero, and Y. Teh. "A fast learning algorithm for deep belief nets". In: *Neural Computation* 18 (2006), pp. 1527–1554.

[Hot36] H. Hotelling. "Relations Between Two Sets of Variates". In: *Biometrika* 28.3/4 (1936), pp. 321–377.

[Hou+12] N. Houlsby, F. Huszar, Z. Ghahramani, and J. M. Hernández-lobato. "Collaborative Gaussian Processes for Preference Learning". In: *NIPS*. 2012, pp. 2096–2104.

[How+17] A. G. Howard et al. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications". In: *CVPR*. 2017.

[HR03] G. E. Hinton and S. T. Roweis. "Stochastic Neighbor Embedding". In: *NIPS*. 2003, pp. 857–864.

[HR76] L. Hyafil and R. Rivest. "Constructing Optimal Binary Decision Trees is NP-complete". In: *Information Processing Letters* 5.1 (1976), pp. 15–17.

[HS19] J. Haochen and S. Sra. "Random Shuffling Beats SGD after Finite Epochs". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. Long Beach, California, USA: PMLR, 2019, pp. 2624–2633.

[HS97a] S Hochreiter and J Schmidhuber. "Flat minima". en. In: *Neural Comput.* 9.1 (1997), pp. 1–42.

[HS97b] S. Hochreiter and J. Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 (1997), 1735–1780.

[HSW89] K. Hornik, M. Stinchcombe, and H. White. "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5 (1989), pp. 359–366.

[HTF01] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, 2001.

[HTF09] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. 2nd edition. Springer, 2009.

[HTW15] T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, 2015.

[Hua14] G.-B. Huang. "An Insight into Extreme Learning Machines: Random Neurons, Random Features and Kernels". In: *Cognit. Comput.* 6.3 (Sept. 2014), pp. 376–390.

[Hua+17a] G. Huang, Z. Liu, K. Q. Weinberger, and L. van der Maaten. "Densely Connected Convolutional Networks". In: *CVPR*. 2017.

[Hua+17b] J. Huang et al. "Speed/accuracy trade-offs for modern convolutional object detectors". In: *CVPR*. 2017.

[Hua+18] C.-Z. A. Huang et al. "Music Transformer". In: (Sept. 2018). arXiv: 1809.04281 [cs.LG].

[Hub+08] M. F. Huber, T Bailey, H Durrant-Whyte, and U. D. Hanebeck. "On entropy approximation for Gaussian mixture random vectors". In: *2008 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems*. Aug. 2008, pp. 181–188.

[Hub64] P. Huber. "Robust Estimation of a Location Parameter". In: *Annals of Statistics* 53 (1964), 73–101.

[Hut90] M. F. Hutchinson. "A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines". In: *Communications in Statistics - Simulation and Computation* 19.2 (Jan. 1990), pp. 433–450.

[HVD14] G. Hinton, O. Vinyals, and J. Dean. "Distilling the Knowledge in a Neural Network". In: *NIPS DL workshop*. 2014.

[HW62] D. Hubel and T. Wiesel. "Receptive fields, binocular interaction, and functional architecture in the cat's visual cortex". In: *J. Physiology* 160 (1962), pp. 106–154.

[HXW17] H. He, B. Xin, and D. Wipf. "From Bayesian Sparsity to Gated Recurrent Nets". In: *NIPS*. 2017.

[HY01] M. Hansen and B. Yu. "Model selection and the principle of minimum description length". In: *JASA* (2001).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[HYL17] W. Hamilton, Z. Ying, and J. Leskovec. "Inductive representation learning on large graphs". In: *Advances in Neural Information Processing Systems*. 2017, pp. 1024–1034.

[Idr+17] H. Idrees et al. "The THUMOS challenge on action recognition for videos "in the wild"". In: *Comput. Vis. Image Underst.* 155 (Feb. 2017), pp. 1–23.

[Ie+19] E. Ie et al. "SlateQ: A tractable decomposition for reinforcement learning with recommendation sets". In: *IJCAI*. Macao, China: International Joint Conferences on Artificial Intelligence Organization, Aug. 2019.

[Ily+19] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry. "Adversarial Examples Are Not Bugs, They Are Features". In: (May 2019). arXiv: 1905.02175 [stat.ML].

[Iof17] S. Ioffe. "Batch Renormalization: Towards Reducing Minibatch Dependence in Batch-Normalized Models". In: (2017). arXiv: 1702.03275 [cs.LG].

[IR10] A. Ilin and T. Raiko. "Practical Approaches to Principal Component Analysis in the Presence of Missing Values". In: *JMLR* 11 (2010), pp. 1957–2000.

[IS15] S. Ioffe and C. Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *ICML*. 2015, pp. 448–456.

[Isc+19] A. Iscen, G. Tolias, Y. Avrithis, and O. Chum. "Label Propagation for Deep Semi-supervised Learning". In: *CVPR*. 2019.

[Izm+18] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson. "Averaging Weights Leads to Wider Optima and Better Generalization". In: *UAI*. 2018.

[Izm+20] P. Izmailov, P. Kirichenko, M. Finzi, and A. G. Wilson. "Semi-supervised learning with normalizing flows". In: *ICML*. 2020, pp. 4615–4630.

[Jac+91] R. Jacobs, M. Jordan, S. Nowlan, and G. Hinton. "Adaptive mixtures of local experts". In: *Neural Computation* (1991).

[JAFF16] J. Johnson, A. Alahi, and L. Fei-Fei. "Perceptual Losses for Real-Time Style Transfer and Super-Resolution". In: *ECCV*. 2016.

[Jan18] E. Jang. *Normalizing Flows Tutorial*. 2018.

[Jay03] E. T. Jaynes. *Probability theory: the logic of science*. Cambridge university press, 2003.

[Jay76] E. T. Jaynes. "Confidence intervals vs Bayesian intervals". In: *Foundations of Probability Theory, Statistical Inference, and Statistical Theories of Science, vol II*. Ed. by W. L. Harper and C. A. Hooker. Reidel Publishing Co., 1976.

[JD88] A. Jain and R. Dubes. *Algorithms for Clustering Data*. Prentice Hall, 1988.

[JDJ17] J. Johnson, M. Douze, and H. Jégou. "Billion-scale similarity search with GPUs". In: (Feb. 2017). arXiv: 1702.08734 [cs.CV].

[Jef61] H. Jeffreys. *Theory of Probability*. Oxford, 1961.

[Jef73] H. Jeffreys. *Scientific Inference*. Third edition. Cambridge, 1973.

[JH04] H. Jaeger and H. Haas. "Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication". In: *Science* 304.5667 (2004).

[JHG00] N. Japkowicz, S. Hanson, and M. Gluck. "Nonlinear autoassociation is not equivalent to PCA". In: *Neural Computation* 12 (2000), pp. 531–545.

[Jia+19] R. Jia, A. Raghunathan, K. Göksel, and P. Liang. "Certified Robustness to Adversarial Word Substitutions". In: *EMNLP*. 2019.

[Jia+20] Y. Jiang, B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio. "Fantastic Generalization Measures and Where to Find Them". In: *ICLR*. 2020.

[Jin+17] Y. Jing, Y. Yang, Z. Feng, J. Ye, Y. Yu, and M. Song. "Neural Style Transfer: A Review". In: *arXiv [cs.CV]* (May 2017).

[JJ94] M. I. Jordan and R. A. Jacobs. "Hierarchical mixtures of experts and the EM algorithm". In: *Neural Computation* 6 (1994), pp. 181–214.

[JK13] A. Jern and C. Kemp. "A probabilistic account of exemplar and category generation". en. In: *Cogn. Psychol.* 66.1 (Feb. 2013), pp. 85–125.

[JM08] D. Jurafsky and J. H. Martin. *Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 2nd edition. Prentice-Hall, 2008.

[JM20] D. Jurafsky and J. H. Martin. *Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (Third Edition)*. Draft of 3rd edition. 2020.

[Jor19] M. I. Jordan. "Artificial Intelligence—The Revolution Hasn't Happened Yet". In: *Harvard Data Science Review* 1.1 (July 2019).

[JT19] L. Jing and Y. Tian. "Self-supervised Visual Feature Learning with Deep Neural Networks: A Survey". In: (Feb. 2019). arXiv: 1902.06162 [cs.CV].

[Jun+19] W. Jung, D. Jung, B. Kim, S. Lee, W. Rhee, and J. Anh. "Restructuring Batch Normalization to Accelerate CNN Training". In: *SysML*. 2019.

[JW19] S. Jain and B. C. Wallace. "Attention is not Explanation". In: *NAACL*. 2019.

[JZ13] R. Johnson and T. Zhang. "Accelerating Stochastic Gradient Descent using Predictive Variance Reduction". In: *NIPS*. Curran Associates, Inc., 2013, pp. 315–323.

[JZS15] R. Jozefowicz, W. Zaremba, and I. Sutskever. "An Empirical Exploration of Recurrent Network Architectures". In: *ICML*. 2015, pp. 2342–2350.

[KAG19] A. Kirsch, J. van Amersfoort, and Y. Gal. "BatchBALD: Efficient and Diverse Batch Acquisition for Deep Bayesian Active Learning". In: *NIPS*. 2019.

[Kai58] H. Kaiser. "The varimax criterion for analytic rotation in factor analysis". In: *Psychometrika* 23.3 (1958).

[Kan+12] E. Kandel, J. Schwartz, T. Jessell, S. Siegelbaum, and A. Hudspeth, eds. *Principles of Neural Science*. Fifth Edition. 2012.

[Kan+18] M. Kanagawa, P. Hennig, D. Sejdinovic, and B. K. Sriperumbudur. "Gaussian Processes and Kernel Methods: A Review on Connections and Equivalences". In: (July 2018). arXiv: 1807.02582 [stat.ML].

[Kan+20] B. Kang et al. "Decoupling Representation and Classifier for Long-Tailed Recognition". In: *ICLR*. 2020.

[Kat+17] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. "Reluplex: An efficient SMT solver for verifying deep neural networks". In: *International Conference on Computer Aided Verification*. Springer. 2017, pp. 97–117.

[Kat+20] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention". In: *ICML*. 2020.

[KB15] D. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization". In: *ICLR*. 2015.

[KB19] M. Kaya and H. S. Bilge. "Deep Metric Learning: A Survey". en. In: *Symmetry* 11.9 (Aug. 2019), p. 1066.

[KBH19] F. Kunstner, L. Balles, and P. Hennig. "Limitations of the Empirical Fisher Approximation". In: (May 2019). arXiv: 1905.12558 [cs.LG].

[KBV09] Y. Koren, R. Bell, and C. Volinsky. "Matrix factorization techniques for recommender systems". In: *IEEE Computer* 42.8 (2009), pp. 30–37.

[KD09] A. D. Kiureghian and O. Ditlevsen. "Aleatory or epistemic? Does it matter?" In: *Structural Safety* 31.2 (Mar. 2009), pp. 105–112.

[Kel+20] J. Kelly, J. Bettencourt, M. J. Johnson, and D. Duvenaud. "Learning Differential Equations that are Easy to Solve". In: *Neural Information Processing Systems*. 2020.

[Kem+06] C. Kemp, J. Tenenbaum, T. Y. T. Griffiths and, and N. Ueda. "Learning systems of concepts with an infinite relational model". In: *AAAI*. 2006.

[KF09] D. Krishnan and R. Fergus. "Fast Image Deconvolution using Hyper-Laplacian Priors". In: *NIPS*. 2009, pp. 1033–1041.

[KG05] A. Krause and C. Guestrin. "Near-optimal value of information in graphical models". In: *UAI*. 2005.

[KG17] A. Kendall and Y. Gal. "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?" In: *NIPS*. Curran Associates, Inc., 2017, pp. 5574–5584.

[KGS20] J. von Kügelgen, L. Gresele, and B. Schölkopf. "Simpson's paradox in Covid-19 case fatality rates: a mediation analysis of age-related causal effects". In: (May 2020). arXiv: 2005.07180 [stat.AP].

[KH09] A Krizhevsky and G Hinton. *Learning multiple layers of features from tiny images*. Tech. rep. U. Toronto, 2009.

[KH19] D. Krotov and J. J. Hopfield. "Unsupervised learning by competing hidden units". en. In: *PNAS* 116.16 (Apr. 2019), pp. 7723–7731.

[Kha+10] M. E. Khan, B. Marlin, G. Bouchard, and K. P. Murphy. "Variational bounds for mixed-data factor analysis". In: *NIPS*. 2010.

[KHB07] A. Kapoor, E. Horvitz, and S. Basu. "Selective Supervision: Guiding Supervised Learning with Decision-Theoretic Active Learning". In: *IJCAI*. 2007.

[Kim14] Y. Kim. "Convolutional Neural Networks for Sentence Classification". In: *EMNLP*. 2014.

[Kim19] D. H. Kim. *Survey of Deep Metric Learning*. 2019.

[Kin+14] D. P. Kingma, D. J. Rezende, S. Mohamed, and M. Welling. "Semi-Supervised Learning with Deep Generative Models". In: *NIPS*. 2014.

[Kir+19] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár. "Panoptic Segmentation". In: *CVPR*. 2019.

[KJ16] L Kang and V Joseph. "Kernel Approximation: From Regression to Interpolation". In: *SIAM/ASA J. Uncertainty Quantification* 4.1 (Jan. 2016), pp. 112–129.

[KJ95] J. Karhunen and J. Joutsensalo. "Generalizations of principal component analysis, optimization problems, and neural networks". In: *Neural Networks* 8.4 (1995), pp. 549–562.

[KJM19] N. M. Kriege, F. D. Johansson, and C. Morris. "A Survey on Graph Kernels". In: (Mar. 2019). arXiv: 1903.11835 [cs.LG].

[KJV83] S. Kirkpatrick, C. G. Jr., and M. Vecchi. "Optimization by simulated annealing". In: *Science* 220 (1983), pp. 671–680.

[KK06] S. Kotsiantis and D. Kanellopoulos. "Discretization Techniques: A recent survey". In: *GESTS Intl. Trans. on Computer Science and Engineering* 31.1 (2006), pp. 47–58.

[KKH19] I. Khemakhem, D. P. Kingma, and A. Hyvärinen. "Variational Autoencoders and Nonlinear ICA: A Unifying Framework". In: *arXiv preprint arXiv:1907.04809* (2019).

[KKL20] N. Kitaev, L. Kaiser, and A. Levskaya. "Reformer: The Efficient Transformer". In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[KKS20] F. Kunstner, R. Kumar, and M. Schmidt. "Homeomorphic-Invariance of EM: Non-Asymptotic Convergence in KL Divergence for Exponential Families via Mirror Descent". In: (Nov. 2020). arXiv: 2011.01170 [cs.LG].

[KL09] H. Kawakatsu and A. Largey. "EM algorithms for ordered probit models with endogenous regressors". In: *The Econometrics Journal* 12.1 (2009), pp. 164–186.

[KL19] W. M. Kouw and M. Loog. "A review of domain adaptation without target labels". en. In: *IEEE PAMI* (Oct. 2019).

[Kla+17] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter. "Self-Normalizing Neural Networks". In: *NIPS*. 2017.

[Kle02] J. Kleinberg. "An Impossibility Theorem for Clustering". In: *NIPS*. 2002.

[Kle+11] A. Kleiner, A. Talwalkar, P. Sarkar, and M. I. Jordan. *A scalable bootstrap for massive data*. Tech. rep. UC Berkeley, 2011.

[Kle13] P. N. Klein. *Coding the Matrix: Linear Algebra through Applications to Computer Science*. en. 1 edition. Newtonian Press, Sept. 2013.

[KLQ95] C. Ko, J. Lee, and M. Queyranne. "An exact algorithm for maximum entropy sampling". In: *Operations Research* 43 (1995), 684–691.

[KMY04] D. Kersten, P. Mamassian, and A. Yuille. "Object perception as Bayesian inference". en. In: *Annu. Rev. Psychol.* 55 (2004), pp. 271–304.

[Koc15] M. Kochenderfer. *Decision Making Under Uncertainty: Theory and Application*. en. 1 edition. The MIT Press, 2015.

[Kok17] I. Kokkinos. "UberNet: Training a Universal Convolutional Neural Network for Low-, Mid-, and High-Level Vision Using Diverse Datasets and Limited Memory". In: *CVPR*. Vol. 2. 2017, p. 8.

[Kol+19] A. Kolesnikov et al. "Large Scale Learning of General Visual Representations for Transfer". In: (Dec. 2019). arXiv: 1912.11370 [cs.CV].

[Kol+20] A. Kolesnikov et al. "Large Scale Learning of General Visual Representations for Transfer". In: *ECCV*. 2020.

[Kon20] M. Konnikova. *The Biggest Bluff: How I Learned to Pay Attention, Master Myself, and Win*. en. Penguin Press, June 2020.

[Koo03] G. Koop. *Bayesian econometrics*. Wiley, 2003.

[Kor09] Y. Koren. *The BellKor Solution to the Netflix Grand Prize*. Tech. rep. Yahoo! Research, 2009.

[KR19] M. Kearns and A. Roth. *The Ethical Algorithm: The Science of Socially Aware Algorithm Design*. en. Oxford University Press, Nov. 2019.

[KR87] L. Kaufman and P. Rousseeuw. "Clustering by means of Medoids". In: *Statistical Data Analysis Based on the L1-norm and Related Methods*. Ed. by Y. Dodge. North-Holland, 1987, 405–416.

[KR90] L. Kaufman and P. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, 1990.

[Kri+05] B. Krishnapuram, L. Carin, M. Figueiredo, and A. Hartemink. "Learning sparse Bayesian classifiers: multi-class formulation, fast algorithms, and generalization bounds". In: *IEEE Transaction on Pattern Analysis and Machine Intelligence* (2005).

[Kro+19] D. P. Kroese, Z. Botev, T. Taimre, and R. Vaisman. *Data Science and Machine Learning: Mathematical and Statistical Methods*. 1 edition. Chapman and Hall/CRC, Nov. 2019.

[Kru13] J. K. Kruschke. "Bayesian estimation supersedes the t test". In: *J. Experimental Psychology: General* 142.2 (2013), pp. 573–603.

[KS15] H. Kaya and A. A. Salah. "Adaptive Mixtures of Factor Analyzers". In: (July 2015). arXiv: 1507.02801 [stat.ML].

[KSG04] A. Kraskov, H. Stögbauer, and P. Grassberger. "Estimating mutual information". en. In: *Phys. Rev. E Stat. Nonlin. Soft Matter Phys.* 69.6 Pt 2 (June 2004), p. 066138.

[KSH12] A. Krizhevsky, I. Sutskever, and G. Hinton. "Imagenet classification with deep convolutional neural networks". In: *NIPS*. 2012.

[KSJ09] I. Konstas, V. Stathopoulos, and J. M. Jose. "On social networks and collaborative recommendation". In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 195–202.

[KST82] D. Kahneman, P. Slovic, and A. Tversky, eds. *Judgment under uncertainty: Heuristics and biases*. Cambridge, 1982.

[KTB11] D. P. Kroese, T. Taimre, and Z. I. Botev. *Handbook of Monte Carlo Methods*. en. 1 edition. Wiley, Mar. 2011.

[Kua+09] P. Kuan, G. Pan, J. A. Thomson, R. Stewart, and S. Keles. *A hierarchical semi-Markov model for detecting enrichment with application to ChIP-Seq experiments*. Tech. rep. U. Wisconsin, 2009.

[Kul13] B. Kulis. "Metric Learning: A Survey". In: *Foundations and Trends in Machine Learning* 5.4 (2013), pp. 287–364.

[Kum+18] K. Kumagai, I. Kobayashi, D. Mochihashi, H. Asoh, T. Nakamura, and T. Nagai. "Natural Language Generation Using Monte Carlo Tree Search". In: *Journal of Advanced Computational Intelligence and Intelligent Informatics* 22.5 (2018), pp. 777–785.

[KV94] M. J. Kearns and U. V. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[KVK10] A. Klami, S. Virtanen, and S. Kaski. "Bayesian exponential family projections for coupled data sources". In: *UAI*. 2010.

[KW14] D. P. Kingma and M. Welling. "Auto-encoding variational Bayes". In: *ICLR*. 2014.

[KW16a] T. N. Kipf and M. Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016).

[KW16b] T. N. Kipf and M. Welling. "Variational graph auto-encoders". In: *arXiv preprint arXiv:1611.07308* (2016).

[KW19a] D. P. Kingma and M. Welling. "An Introduction to Variational Autoencoders". In: *Foundations and Trends in Machine Learning* 12.4 (2019), pp. 307–392.

[KW19b] M. J. Kochenderfer and T. A. Wheeler. *Algorithms for Optimization*. en. The MIT Press, Mar. 2019.

[KW70] G. S. Kimeldorf and G. Wahba. "A Correspondence Between Bayesian Estimation on Stochastic Processes and Smoothing by Splines". en. In: *Ann. Math. Stat.* 41.2 (Apr. 1970), pp. 495–502.

[KWW22] M. J. Kochenderfer, T. A. Wheeler, and K. Wray. *Algorithms for Decision Making*. The MIT Press, 2022.

[Kyu+10] M. Kyung, J. Gill, M. Ghosh, and G. Casella. "Penalized Regression, Standard Errors and Bayesian Lassos". In: *Bayesian Analysis* 5.2 (2010), pp. 369–412.

[LA16] S. Laine and T. Aila. "Temporal ensembling for semi-supervised learning". In: *arXiv preprint arXiv:1610.02242* (2016).

[LA87] P. J. M. Laarhoven and E. H. L. Aarts, eds. *Simulated Annealing: Theory and Applications*. Norwell, MA, USA: Kluwer Academic Publishers, 1987.

[Lak+17] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman. "Building Machines That Learn and Think

Like People". en. In: *Behav. Brain Sci.* (2017), pp. 1–101.

[Law12] N. D. Lawrence. "A Unifying Probabilistic Perspective for Spectral Dimensionality Reduction: Insights and New Models". In: *JMLR* 13.May (2012), pp. 1609–1638.

[LeC18] Y. LeCun. *Self-supervised learning: could machines learn like humans?* 2018.

[LeC+98] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[Lee13] D.-H. Lee. "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks". In: *ICML Workshop on Challenges in Representation Learning*. 2013.

[Lee+13] J. Lee, S. Kim, G. Lebanon, and Y. Singer. "Local Low-Rank Matrix Approximation". In: *ICML*. Vol. 28. Proceedings of Machine Learning Research. Atlanta, Georgia, USA: PMLR, 2013, pp. 82–90.

[Lee+19] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh. "Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks". In: *ICML*. 2019.

[Lee77] J. de Leeuw. "Applications of Convex Analysis to Multidimensional Scaling". In: *Recent Developments in Statistics*. Ed. by J. R. Barra, F Brodeau, G Romier, and B Van Cutsem. 1977.

[LG14] O. Levy and Y. Goldberg. "Neural Word Embedding as Implicit Matrix Factorization". In: *NIPS*. 2014.

[LH17] I. Loshchilov and F. Hutter. "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *ICLR*. 2017.

[Li+10] L. Li, W. Chu, J. Langford, and R. E. Schapire. "A contextual-bandit approach to personalized news article recommendation". In: *WWW*. 2010.

[Li+15] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. "Gated graph sequence neural networks". In: *arXiv preprint arXiv:1511.05493* (2015).

[Li+19] J. Li, S. Qu, X. Li, J. Szurley, J. Z. Kolter, and F. Metze. "Adversarial Music: Real world Audio Adversary against Wake-word Detection System". In: *NIPS*. Curran Associates, Inc., 2019, pp. 11908–11918.

[Lia20] S. M. Liao, ed. *Ethics of Artificial Intelligence*. en. 1st ed. Oxford University Press, Sept. 2020.

[Lim+19] S. Lim, I. Kim, T. Kim, C. Kim, and S. Kim. "Fast AutoAugment". In: (May 2019). arXiv: 1905.00397 [cs.LG].

[Lin06] D. Lindley. *Understanding Uncertainty*. Wiley, 2006.

[Lin56] D. Lindley. "On a measure of the information provided by an experiment". In: *The Annals of Math. Stat.* (1956), 986–1005.

[Liu01] J. Liu. *Monte Carlo Strategies in Scientific Computation*. Springer, 2001.

[Liu+15] Z. Liu, P. Luo, X. Wang, and X. Tang. "Deep Learning Face Attributes in the Wild". In: *ICCV*. 2015.

[Liu+16] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, and S. Reed. "SSD: Single Shot MultiBox Detector". In: *ECCV*. 2016.

[Liu+18a] H. Liu, Y.-S. Ong, X. Shen, and J. Cai. "When Gaussian Process Meets Big Data: A Review of Scalable GPs". In: (July 2018). arXiv: 1807.01065 [stat.ML].

[Liu+18b] L. Liu, X. Liu, C.-J. Hsieh, and D. Tao. "Stochastic Second-order Methods for Non-convex Optimization with Inexact Hessian and Gradient". In: (Sept. 2018). arXiv: 1809.09853 [math.OC].

[Liu+20] F. Liu, X. Huang, Y. Chen, and J. A. K. Suykens. "Random Features for Kernel Approximation: A Survey on Algorithms, Theory, and Beyond". In: (Apr. 2020). arXiv: 2004.11154 [stat.ML].

[LJ09] H. Lukosevicius and H. Jaeger. "Reservoir computing approaches to recurrent neural network training". In: *Computer Science Review* 3.3 (2009), 127–149.

[LKB20] Q. Liu, M. J. Kusner, and P. Blunsom. "A Survey on Contextual Embeddings". In: (Mar. 2020). arXiv: 2003.07278 [cs.CL].

[Llo82] S Lloyd. "Least squares quantization in PCM". In: *IEEE Trans. Inf. Theory* 28.2 (Mar. 1982), pp. 129–137.

[LLT89] K. Lange, R. Little, and J. Taylor. "Robust Statistical Modeling Using the T Disribution". In: *JASA* 84.408 (1989), pp. 881–896.

[LM04] E. Learned-Miller. *Hyperspacings and the estimation of information theoretic quantities*. Tech. rep. 04-104. U. Mass. Amherst Comp. Sci. Dept, 2004.

[LM86] R. Larsen and M. Marx. *An introduction to mathematical statistics and its applications*. Prentice Hall, 1986.

[LNK19] Q. Liu, M. Nickel, and D. Kiela. "Hyperbolic graph neural networks". In: *Advances in Neural Information Processing Systems*. 2019, pp. 8228–8239.

[Loa00] C. F. V. Loan. "The ubiquitous Kronecker product". In: *J. Comput. Appl. Math.* 123.1 (Nov. 2000), pp. 85–100.

[Lod+02] H. Lodhi, C. Saunders, J. Shawe-Taylor, N. Cristianini, and C. Watkins. "Text classification using string kernels". en. In: *J. Mach. Learn. Res.* (Mar. 2002).

[LPM15] M.-T. Luong, H. Pham, and C. D. Manning. "Effective Approaches to Attention-based Neural Machine Translation". In: *EMNLP*. 2015.

[LR85] T. L. Lai and H. Robbins. "Asymptotically efficient adaptive allocation rules". en. In: *Adv. Appl. Math.* (Mar. 1985).

[LR87] R. J. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. New York: Wiley and Son, 1987.

[LRU14] J. Leskovec, A. Rajaraman, and J. Ullman. *Mining of massive datasets*. Cambridge, 2014.

[LS10] P. Long and R. Servedio. "Random classification noise beats all convex potential boosters". In: *JMLR* 78.3 (2010), pp. 287–304.

[LS19a] S. Lattanzi and C. Sohler. "A Better k-means++ Algorithm via Local Search". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. Long Beach, California, USA: PMLR, 2019, pp. 3662–3671.

[LS19b] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge, 2019.

[LS19c] T. Lattimore and C. Szepesvari. *Bayesian/minimax duality for adversarial bandits (blog post)*. 2019.

[LS19d] Z. C. Lipton and J. Steinhardt. "Troubling Trends in Machine Learning Scholarship: Some ML papers suffer from flaws that could mislead the public and stymie future research". In: *The Queue* 17.1 (Feb. 2019), pp. 45–77.

[LS88] S. L. Lauritzen and D. J. Spiegelhalter. "Local computations with probabilities on graphical structures and their applications to expert systems". In: *JRSSB* B.50 (1988), pp. 127–224.

[LSS13] Q. Le, T. Sarlos, and A. Smola. "Fastfood - Computing Hilbert Space Expansions in loglinear time". In: *ICML*. Vol. 28. Proceedings of Machine Learning Research. Atlanta, Georgia, USA: PMLR, 2013, pp. 244–252.

[LSY19] H. Liu, K. Simonyan, and Y. Yang. "DARTS: Differentiable Architecture Search". In: *ICLR*. 2019.

[Lu+19] L. Lu, Y. Shin, Y. Su, and G. E. Karniadakis. "Dying ReLU and Initialization: Theory and Numerical Examples". In: (Mar. 2019). arXiv: 1903.06733 [stat.ML].

[Luo16] M.-T. Luong. "Neural machine translation". PhD thesis. Stanford Dept. Comp. Sci., 2016.

[Luo+19] P. Luo, X. Wang, W. Shao, and Z. Peng. "Towards Understanding Regularization in Batch Normalization". In: *ICLR*. 2019.

[LUW17] C. Louizos, K. Ullrich, and M. Welling. "Bayesian Compression for Deep Learning". In: *NIPS*. 2017.

[Lux07] U. von Luxburg. "A tutorial on spectral clustering". In: *Statistics and Computing* 17.4 (2007), pp. 395–416.

[LW04a] O. Ledoit and M. Wolf. "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices". In: *J. of Multivariate Analysis* 88.2 (2004), pp. 365–411.

[LW04b] O. Ledoit and M. Wolf. "Honey, I Shrunk the Sample Covariance Matrix". In: *J. of Portfolio Management* 31.1 (2004).

[LW04c] H. Lopes and M. West. "Bayesian model assessment in factor analysis". In: *Statisica Sinica* 14 (2004), pp. 41–67.

[LW16] C. Li and M. Wand. "Precomputed Real-Time Texture Synthesis with Markovian Generative Adversarial Networks". In: *ECCV*. 2016.

[LWG12] U. von Luxburg, R. Williamson, and I. Guyon. "Clustering: science or art?" In: *Workshop on Unsupervised and Transfer Learning*. 2012.

[Lyu+20] X.-K. Lyu, Y. Xu, X.-F. Zhao, X.-N. Zuo, and C.-P. Hu. "Beyond psychology: prevalence of p value and confidence interval misinterpretation across different fields". In: *Journal of Pacific Rim Psychology* 14 (2020).

[MA10] I. Murray and R. P. Adams. "Slice sampling covariance hyperparameters of latent Gaussian models". In: *NIPS*. 2010, pp. 1732–1740.

[MA+17] Y. Movshovitz-Attias, A. Toshev, T. K. Leung, S. Ioffe, and S. Singh. "No Fuss Distance Metric Learning using Proxies". In: *ICCV*. 2017.

[Maa+11] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts. "Learning Word Vectors for Sentiment Analysis". In: *Proc. ACL*. 2011, pp. 142–150.

[Maa14] L. van der Maaten. "Accelerating t-SNE using Tree-Based Algorithms". In: *JMLR* (2014).

[Mac03] D. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[Mac09] L. W. Mackey. "Deflation Methods for Sparse PCA". In: *NIPS*. 2009.

[Mac+19] D. Maclaurin, A. Radul, M. J. Johnson, and D. Vytiniotis. "Dex: array programming with typed indices". In: *NeurIPS workshop: Program Transformations for Machine Learning* (2019).

[Mac67] J MacQueen. "Some methods for classification and analysis of multivariate observations". en. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*. The Regents of the University of California, 1967.

[Mac95] D. MacKay. "Probable networks and plausible predictions — a review of practical Bayesian methods for supervised neural networks". In: *Network: Computation in Neural Systems* 6.3 (1995), pp. 469–505.

[Mad+18] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. "Towards Deep Learning Models Resistant to Adversarial Attacks". In: *ICLR*. 2018.

[Mad+20] A. Madani et al. "ProGen: Language Modeling for Protein Generation". en. Mar. 2020.

[Mah07] R. P. S. Mahler. *Statistical Multisource-Multitarget Information Fusion*. Norwood, MA, USA: Artech House, Inc., 2007.

[Mah13] R Mahler. "Statistics 102 for Multisource-Multitarget Detection and Tracking". In: *IEEE J. Sel. Top. Signal Process.* 7.3 (June 2013), pp. 376–389.

[Mah+18] D. Mahajan et al. "Exploring the Limits of Weakly Supervised Pretraining". In: (May 2018). arXiv: 1805.00932 [cs.CV].

[Mai15] J Mairal. "Incremental Majorization-Minimization Optimization with Application to Large-Scale Machine Learning". In: *SIAM J. Optim.* 25.2 (Jan. 2015), pp. 829–855.

[Mal99] S. Mallat. *A Wavelet Tour of Signal Processing*. Academic Press, 1999.

[Man+16] V. Mansinghka, P. Shafto, E. Jonas, C. Petschulat, M. Gasner, and J. Tenenbaum. "Crosscat: A Fully Bayesian, Nonparametric Method For Analyzing Heterogeneous, High-dimensional Data." In: *JMLR* 17 (2016).

[Mar06] H. Markram. "The blue brain project". en. In: *Nat. Rev. Neurosci.* 7.2 (Feb. 2006), pp. 153–160.

[Mar08] B. Marlin. "Missing Data Problems in Machine Learning". PhD thesis. U. Toronto, 2008.

[Mar+11] B. M. Marlin, R. S. Zemel, S. T. Roweis, and M. Slaney. "Recommender Systems, Missing Data and Statistical Model Estimation". In: *IJCAI*. 2011.

[Mar16] J. Martens. "Second-order optimization for neural networks". PhD thesis. Toronto, 2016.

[Mar18] O. Martin. *Bayesian analysis with Python*. Packt, 2018.

[Mar20] G. Marcus. "The Next Decade in AI: Four Steps Towards Robust Artificial Intelligence". In: (Feb. 2020). arXiv: 2002.06177 [cs.AI].

[Mar72] G. Marsaglia. "Choosing a Point from the Surface of a Sphere". en. In: *Ann. Math. Stat.* 43.2 (Apr. 1972), pp. 645–646.

[Mar82] D. Marr. *Vision. A Computational Investigation into the Human Representation and Processing of Visual Information*. W.H. Freeman and Company, 1982.

[Mas+00] L. Mason, J. Baxter, P. L. Bartlett, and M. R. Frean. "Boosting Algorithms as Gradient Descent". In: *NIPS*. 2000, pp. 512–518.

[Mas+15] J. Masci, D. Boscaini, M. Bronstein, and P. Vandergheynst. "Geodesic convolutional neural networks on riemannian manifolds". In: *Proceedings of the IEEE international conference on computer vision workshops*. 2015, pp. 37–45.

[Mat00] R. Matthews. "Storks Deliver Babies (p = 0.008)". In: *Teach. Stat.* 22.2 (June 2000), pp. 36–38.

[Mat98] R. Matthews. *Bayesian Critique of Statistics in Health: The Great Health Hoax*. 1998.

[MAV17] D. Molchanov, A. Ashukha, and D. Vetrov. "Variational Dropout Sparsifies Deep Neural Networks". In: *ICML*. 2017.

[MB05] F. Morin and Y. Bengio. "Hierarchical Probabilistic Neural Network Language Model". In: *AISTATS*. 2005.

[MB06] N. Meinshausen and P. Buhlmann. "High dimensional graphs and variable selection with the lasso". In: *The Annals of Statistics* 34 (2006), pp. 1436–1462.

[MBL20] K. Musgrave, S. Belongie, and S.-N. Lim. "A Metric Learning Reality Check". In: *ECCV*. Mar. 2020.

[MC19] P. Moreno Comellas. "Vision as inverse graphics for detailed scene understanding". en. PhD thesis. July 2019.

[McE20] R. McElreath. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan (2nd edition)*. en. Chapman and Hall/CRC, 2020.

[McL75] G. J. McLachlan. "Iterative reclassification procedure for constructing an asymptotically optimal rule of allocation in discriminant analysis". In: *Journal of the American Statistical Association* 70.350 (1975), pp. 365–369.

[McM+13] H. B. McMahan et al. "Ad click prediction: a view from the trenches". In: *KDD*. 2013, pp. 1222–1230.

[MD97] X. L. Meng and D. van Dyk. "The EM algorithm — an old folk song sung to a fast new tune (with Discussion)". In: *J. Royal Stat. Soc. B* 59 (1997), pp. 511–567.

[MDA15] D. Maclaurin, D. Duvenaud, and R. P. Adams. "Gradient-based Hyperparameter Optimization through Reversible Learning". In: *ICML*. 2015.

[MDM19] S. Mahloujifar, D. I. Diochnos, and M. Mahmoody. "The curse of concentration in robust learning: Evasion and poisoning attacks from concentration of measure". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33. 2019, pp. 4536–4543.

[ME14] S. Masoudnia and R. Ebrahimpour. "Mixture of experts: a literature survey". In: *Artificial Intelligence Review* 42.2 (Aug. 2014), pp. 275–293.

[Mei01] M. Meila. "A random walks view of spectral segmentation". In: *AISTATS*. 2001.

[Mei05] M. Meila. "Comparing clusterings: an axiomatic view". In: *ICML*. 2005.

[Men+12] T. Mensink, J. Verbeek, F. Perronnin, and G. Csurka. "Metric Learning for Large Scale Image Classification: Generalizing to New Classes at Near-Zero Cost". In: *ECCV*. Springer Berlin Heidelberg, 2012, pp. 488–501.

[MG05] I. Murray and Z. Ghahramani. *A note on the evidence and Bayesian Occam's razor*. Tech. rep. Gatsby, 2005.

[MG15] J. Martens and R. Grosse. "Optimizing Neural Networks with Kronecker-factored Approximate Curvature". In: *ICML*. 2015.

[MH07] A. Mnih and G. Hinton. "Three new graphical models for statistical language modelling". en. In: *ICML*. 2007.

[MH08] L. v. d. Maaten and G. Hinton. "Visualizing Data using t-SNE". In: *JMLR* 9.Nov (2008), pp. 2579–2605.

[MHM18] L. McInnes, J. Healy, and J. Melville. "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction". In: (Feb. 2018). arXiv: 1802.03426 [stat.ML].

[MHN13] A. L. Maas, A. Y. Hannun, and A. Y. Ng. "Rectifier Nonlinearities Improve Neural Network Acoustic Models". In: *ICML*. Vol. 28. 2013.

[Mik+13a] T. Mikolov, K. Chen, G. Corrado, and J. Dean. "Efficient Estimation of Word Representations in Vector Space". In: *ICLR*. 2013.

[Mik+13b] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. "Distributed Representations of Words and Phrases and their Compositionality". In: *NIPS*. 2013.

[Mik+13c] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. "Distributed representations of words and phrases and their compositionality". In: *NIPS*. 2013, pp. 3111–3119.

[Min00] T. Minka. *Bayesian model averaging is not model combination*. Tech. rep. MIT Media Lab, 2000.

[Min99] T. Minka. *Learning how to learn is learning with point sets*. Tech. rep. MIT Media Lab, 1999.

[Mit97] T. Mitchell. *Machine Learning*. McGraw Hill, 1997.

[Miy+18] T. Miyato, S.-I. Maeda, M. Koyama, and S. Ishii. "Virtual Adversarial Training: A Regularization Method for Supervised and Semi-Supervised Learning". In: *IEEE PAMI* (2018).

[MK97] G. J. McLachlan and T. Krishnan. *The EM Algorithm and Extensions*. Wiley, 1997.

[MKS21] K. Murphy, A. Kumar, and S. Serghiou. "Risk score learning for COVID-19 contact tracing apps". In: (Apr. 2021). arXiv: 2104.08415 [cs.LG].

[MM16] D. Mishkin and J. Matas. "All you need is a good init". In: *ICLR*. 2016.

[MN89] P. McCullagh and J. Nelder. *Generalized linear models*. 2nd edition. Chapman and Hall, 1989.

[MNM02] W. Maass, T. Natschlaeger, and H. Markram. "Real-time computing without stable states: A new framework for neural computation based on perturbations". In: *Neural Computation* 14.11 (2002), 2531—2560.

[MO04] S. C. Madeira and A. L. Oliveira. "Biclustering Algorithms for Biological Data Analysis: A Survey". In: *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 1.1 (2004), pp. 24–45.

[Mol04] C. Moler. *Numerical Computing with MATLAB*. SIAM, 2004.

[Mon+14] G. F. Montufar, R. Pascanu, K. Cho, and Y. Bengio. "On the Number of Linear Regions of Deep Neural Networks". In: *NIPS*. 2014.

[Mon+17] F. Monti, D. Boscaini, J. Masci, E. Rodola, J. Svoboda, and M. M. Bronstein. "Geometric deep learning on graphs and manifolds using mixture model cnns". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 5115–5124.

[Mor+16] R. D. Morey, R. Hoekstra, J. N. Rouder, M. D. Lee, and E.-J. Wagenmakers. "The fallacy of placing confidence in confidence intervals". en. In: *Psychon. Bull. Rev.* 23.1 (Feb. 2016), pp. 103–123.

[MOT15] A. Mordvintsev, C. Olah, and M. Tyka. *Inceptionism: Going Deeper into Neural Networks*. https://ai.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html. Accessed: NA-NA-NA. 2015.

[MP43] W. McCulloch and W. Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 115–137.

[MP69] M. Minsky and S. Papert. *Perceptrons*. MIT Press, 1969.

[MRS08] C. Manning, P. Raghavan, and H. Schuetze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[MS11] D. Mayo and A. Spanos, "Error Statistics". In: *Handbook of Philosophy of Science*. Ed. by P. S. Bandyopadhyay and M. R. Forster. 2011.

[Muk+19] B. Mukhoty, G. Gopakumar, P. Jain, and P. Kar. "Globally-convergent Iteratively Reweighted Least Squares for Robust Regression Problems". In: *AISTATS*. 2019, pp. 313–322.

[Mur22] K. P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2022.

[MV15] A Mahendran and A Vedaldi. "Understanding deep image representations by inverting them". In: *CVPR*. June 2015, pp. 5188–5196.

[MV16] A. Mahendran and A. Vedaldi. "Visualizing Deep Convolutional Neural Networks Using Natural Pre-images". In: *Intl. J. Computer Vision* (2016), pp. 1–23.

[MWK16] A. H. Marblestone, G. Wayne, and K. P. Kording. "Toward an Integration of Deep Learning and Neuroscience". en. In: *Front. Comput. Neurosci.* 10 (2016), p. 94.

[MWP98] B Moghaddam, W Wahid, and A Pentland. "Beyond eigenfaces: probabilistic matching for face recognition". In: *Proceedings Third IEEE International Conference on Automatic Face and Gesture Recognition*. Apr. 1998, pp. 30–35.

[Nab01] I. Nabney. *NETLAB: algorithms for pattern recognition*. Springer, 2001.

[Nad+19] S. Naderi, K. He, R. Aghajani, S. Sclaroff, and P. Felzenszwalb. "Generalized Majorization-Minimization". In: *ICML*. 2019.

[Nau04] J. Naudts. "Estimators, escort probabilities and ϕ-exponential families in statistical physics". In: *J. of Inequalities in Pure and Applied Mathematics* 5.4 (2004).

[Nea96] R. Neal. *Bayesian learning for neural networks*. Springer, 1996.

[Nes04] Y. Nesterov. *Introductory Lectures on Convex Optimization. A basic course*. Kluwer, 2004.

[Neu04] A. Neumaier. "Complete search in continuous global optimization and constraint satisfaction". In: *Acta Numer.* 13 (May 2004), pp. 271–369.

[Neu17] G. Neubig. "Neural Machine Translation and Sequence-to-sequence Models: A Tutorial". In: (Mar. 2017). arXiv: 1703.01619 [cs.CL].

[NH98] R. M. Neal and G. E. Hinton. "A View of the EM Algorithm that Justifies Incremental, Sparse, and other Variants". In: *Learning in Graphical Models*. Ed. by M. I. Jordan. Dordrecht: Springer Netherlands, 1998, pp. 355–368.

[NHLS19] E. Nalisnick, J. M. Hernández-Lobato, and P. Smyth. "Dropout as a Structured Shrinkage Prior". In: *ICML*. 2019.

[Nic+15] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich. "A Review of Relational Machine Learning for Knowledge Graphs". In: *Proc. IEEE* (2015).

[Niu+11] F. Niu, B. Recht, C. Re, and S. J. Wright. "HOGWILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent". In: *NIPS*. 2011.

[NJ02] A. Y. Ng and M. I. Jordan. "On Discriminative vs. Generative Classifiers: A comparison of logistic regression and Naive Bayes". In: *NIPS-14*. 2002.

[NJW01] A. Ng, M. Jordan, and Y. Weiss. "On Spectral Clustering: Analysis and an algorithm". In: *NIPS*. 2001.

[NK17] M. Nickel and D. Kiela. "Poincaré embeddings for learning hierarchical representations". In: *Advances in neural information processing systems*. 2017, pp. 6338–6347.

[NK18] M. Nickel and D. Kiela. "Learning Continuous Hierarchies in the Lorentz Model of Hyperbolic Geometry". In: *International Conference on Machine Learning*. 2018, pp. 3779–3788.

[NK19] T. Niven and H.-Y. Kao. "Probing Neural Network Comprehension of Natural Language Arguments". In: *Proc. ACL*. 2019.

[NMC05] A. Niculescu-Mizil and R. Caruana. "Predicting Good Probabilities with Supervised Learning". In: *ICML*. 2005.

[Nou+02] M. N. Nounou, B. R. Bakshi, P. K. Goel, and X. Shen. "Process modeling by Bayesian latent variable regression". In: *Am. Inst. Chemical Engineers Journal* 48.8 (Aug. 2002), pp. 1775–1793.

[Nov62] A. Novikoff. "On convergence proofs on perceptrons". In: *Symp. on the Mathematical Theory of Automata* 12 (1962), pp. 615–622.

[NR18] G. Neu and L. Rosasco. "Iterate Averaging as Regularization for Stochastic Gradient Descent". In: *COLT*. 2018.

[NTL20] J. Nixon, D. Tran, and B. Lakshminarayanan. "Why aren't bootstrapped neural networks better?" In: *NIPS Workshop on "I can't believe it's not better"*. 2020.

[NW06] J. Nocedal and S. Wright. *Numerical Optimization*. Springer, 2006.

[NYC15] A. Nguyen, J. Yosinski, and J. Clune. "Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images". In: *CVPR*. 2015.

[Ode16] A. Odena. "Semi-supervised learning with generative adversarial networks". In: *arXiv preprint arXiv:1606.01583* (2016).

[OLV18] A. van den Oord, Y. Li, and O. Vinyals. "Representation Learning with Contrastive Predictive Coding". In: (July 2018). arXiv: 1807.03748 [cs.LG].

[OMS17] C. Olah, A. Mordvintsev, and L. Schubert. "Feature Visualization". In: *Distill* (2017).

[oor+16] A. Van den oord et al. "WaveNet: A Generative Model for Raw Audio". In: (Dec. 2016). arXiv: 1609.03499 [cs.SD].

[Oor+18] A. van den Oord et al. "Parallel WaveNet: Fast High-Fidelity Speech Synthesis". In: *ICML*. Ed. by J. Dy and A. Krause. Vol. 80. Proceedings of Machine Learning Research. Stockholmsmässan, Stockholm Sweden: PMLR, 2018, pp. 3918–3926.

[OPK12] G. Ohloff, W. Pickenhagen, and P. Kraft. *Scent and Chemistry*. en. Wiley, Jan. 2012.

[OPT00a] M. R. Osborne, B. Presnell, and B. A. Turlach. "A new approach to variable selection in least squares problems". In: *IMA Journal of Numerical Analysis* 20.3 (2000), pp. 389–403.

[OPT00b] M. R. Osborne, B. Presnell, and B. A. Turlach. "On the lasso and its dual". In: *J. Computational and graphical statistics* 9 (2000), pp. 319–337.

[Ort+19] P. A. Ortega et al. "Meta-learning of Sequential Strategies". In: (May 2019). arXiv: 1905.03030 [cs.LG].

[Osb16] I. Osband. "Risk versus Uncertainty in Deep Learning: Bayes, Bootstrap and the Dangers of Dropout". In: *NIPS workshop on Bayesian deep learning*. 2016.

[OTJ07] G. Obozinski, B. Taskar, and M. I. Jordan. *Joint covariate selection for grouped classification*. Tech. rep. UC Berkeley, 2007.

[Pai05] A. Pais. *Subtle Is the Lord: The Science and the Life of Albert Einstein*. en. Oxford University Press, Nov. 2005.

[Pan+15] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur. "Librispeech: an asr corpus based on public domain audio books". In: *ICASSP*. IEEE. 2015, pp. 5206–5210.

[Pap+17] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z Berkay Celik, and A. Swami. "Practical Black-Box Attacks against Deep Learning Systems using Adversarial Examples". In: *ACM Asia Conference on Computer and Communications Security*. 2017.

[Pap+18] G. Papandreou, T. Zhu, L.-C. Chen, S. Gidaris, J. Tompson, and K. Murphy. "PersonLab: Person Pose Estimation and Instance Segmentation with a Bottom-Up, Part-Based, Geometric Embedding Model". In: *ECCV*. 2018, pp. 269–286.

[Par+16] A. Parikh, O. Täckström, D. Das, and J. Uszkoreit. "A Decomposable Attention Model for Natural Language Inference". In: *EMNLP*. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2249–2255.

[Par+18] N. Parmar et al. "Image Transformer". In: *ICLR*. 2018.

[PARS14] B. Perozzi, R. Al-Rfou, and S. Skiena. "Deepwalk: Online learning of social representations". In: *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2014, pp. 701–710.

[Pas14] R. Pascanu. "On Recurrent and Deep Neural Networks". PhD thesis. U. Montreal, 2014.

[Pat12] A. Paterek. *Predicting movie ratings and recommender systems*. 2012.

[Pat+16] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. "Context Encoders: Feature Learning by Inpainting". In: *CVPR*. 2016.

[Pau+20] A. Paullada, I. D. Raji, E. M. Bender, E. Denton, and A. Hanna. "Data and its (dis)contents: A survey of dataset development and use in machine learning research". In: *NeurIPS 2020 Workshop: ML Retrospectives, Surveys & Meta-analyses (ML-RSA)*. Dec. 2020.

[PB+14] N. Parikh, S. Boyd, et al. "Proximal algorithms". In: *Foundations and Trends in Optimization* 1.3 (2014), pp. 127–239.

[Pea18] J. Pearl. *Theoretical Impediments to Machine Learning With Seven Sparks from the Causal Revolution*. Tech. rep. UCLA, 2018.

[Pel08] M. Pelikan. "Probabilistic model-building genetic algorithms". In: *GECCO*. GECCO '08. Atlanta, GA, USA: Association for Computing Machinery, July 2008, pp. 2389–2416.

[Pen+20] Z. Peng et al. "Graph Representation Learning via Graphical Mutual Information Maximization". In: *Proceedings of The Web Conference*. 2020.

[Per+17] B. Perozzi, V. Kulkarni, H. Chen, and S. Skiena. "Don't Walk, Skip! Online Learning of Multi-Scale Network Embeddings". In: *Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2017*. ASONAM '17. Sydney, Australia: Association for Computing Machinery, 2017, 258–265.

[Pet13] J. Peters. *When Ice Cream Sales Rise, So Do Homicides. Coincidence, or Will Your Next Cone Murder You?* https://slate.com/news-and-politics/2013/07/warm-weather-homicide-rates-when-ice-cream-sales-rise-homicides-rise-coincidence.html. Accessed: 2020-5-20. July 2013.

[Pet+18] M. E. Peters et al. "Deep contextualized word representations". In: *NAACL*. 2018.

[PH18] T. Parr and J. Howard. "The Matrix Calculus You Need For Deep Learning". In: (Feb. 2018). arXiv: 1802.01528 [cs.LG].

[Pin88] F. J. Pineda. "Generalization of back propagation to recurrent and higher order neural networks". In: *Neural information processing systems*. 1988, pp. 602–611.

[PJ09] H.-S. Park and C.-H. Jun. "A simple and fast algorithm for K-medoids clustering". In: *Expert Systems with Applciations* 36.2, Part 2 (2009), pp. 3336–3341.

[PJ92] B Polyak and A Juditsky. "Acceleration of Stochastic Approximation by Averaging". In: *SIAM J. Control Optim.* 30.4 (July 1992), pp. 838–855.

[Pla00] J. Platt. "Probabilities for SV machines". In: *Advances in Large Margin Classifiers*. Ed. by A. Smola, P. Bartlett, B. Schoelkopf, and D. Schuurmans. MIT Press, 2000.

[Pla98] J. Platt. "Using analytic QP and sparseness to speed training of support vector machines". In: *NIPS*. 1998.

[PM17] D. L. Poole and A. K. Mackworth. *Artificial intelligenceL foundations computational agents 2nd edition*. Cambridge University Press, Nov. 2017.

[PM18] J. Pearl and D. Mackenzie. *The book of why: the new science of cause and effect*. 2018.

[PMB19] J. Pérez, J. Marinkovic, and P. Barcelo. "On the Turing Completeness of Modern Neural Network Architectures". In: *ICLR*. 2019.

[Pog+17] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. "Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review". en. In: *Int. J. Autom. Comput.* (2017), pp. 1–17.

[PPS18] T. Pierrot, N. Perrin, and O. Sigaud. "First-order and second-order variants of the gradient descent in a unified framework". In: (Oct. 2018). arXiv: 1810.08102 [cs.LG].

[PSM14a] J. Pennington, R. Socher, and C. Manning. "GloVe: Global vectors for word representation". In: *EMNLP*. 2014, pp. 1532–1543.

[PSM14b] J. Pennington, R. Socher, and C. Manning. "Glove: Global vectors for word representation". In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014, pp. 1532–1543.

[PSW15] N. G. Polson, J. G. Scott, and B. T. Willard. "Proximal Algorithms in Statistics and Machine Learning". en. In: *Stat. Sci.* 30.4 (Nov. 2015), pp. 559–581.

[QC+06] J. Quiñonero-Candela, C. E. Rasmussen, F. Sinz, O. Bousquet, and B. Schölkopf. "Evaluating Predictive Uncertainty Challenge". In: *Machine Learning Challenges. Evaluating Predictive Uncertainty, Visual Object Classification, and Recognising Tectual Entailment*. Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2006, pp. 1–27.

[Qia+19] Q. Qian, L. Shang, B. Sun, J. Hu, H. Li, and R. Jin. "SoftTriple Loss: Deep Metric Learning Without Triplet Sampling". In: *ICCV*. 2019.

[Qiu+18] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, and J. Tang. "Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec". In: *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. 2018, pp. 459–467.

[Qiu+19a] J. Qiu, H. Ma, O. Levy, S. W. Yih, S. Wang, and J. Tang. "Blockwise Self-Attention for Long Document Understanding". In: *CoRR* abs/1911.02972 (2019). arXiv: 1911.02972.

[Qiu+19b] J. Qiu et al. "NetSMF: Large-Scale Network Embedding as Sparse Matrix Factorization". In: *The World Wide Web Conference*. WWW '19. San Francisco, CA, USA: Association for Computing Machinery, 2019, 1509–1520.

[Qui86] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1 (1986), pp. 81–106.

[Qui93] J. R. Quinlan. *C4.5 Programs for Machine Learning*. Morgan Kauffman, 1993.

[Rad+18] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. *Improving Language Understanding by Generative Pre-Training*. Tech. rep. OpenAI, 2018.

[Rad+19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[Raf+19] C. Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: (Oct. 2019). arXiv: 1910.10683 [cs.LG].

[Rag+17] M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, and J. Sohl-Dickstein. "On the Expressive Power of Deep Neural Networks". In: *ICML*. 2017.

[Rag+19] M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio. "Transfusion: Understanding transfer learning for medical imaging". In: *NIPS*. 2019, pp. 3347–3357.

[Raj+16] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. "SQuAD: 100,000+ Questions for Machine Comprehension of Text". In: *EMNLP*. 2016.

[Raj+18] A. Rajkomar et al. "Scalable and accurate deep learning with electronic health records". en. In: *NPJ Digit Med* 1 (May 2018), p. 18.

[Rao99] R. P. Rao. "An optimal estimation approach to visual perception and learning". en. In: *Vision Res.* 39.11 (June 1999), pp. 1963–1989.

[Rat+09] M. Rattray, O. Stegle, K. Sharp, and J. Winn. "Inference algorithms and learning theory for Bayesian sparse factor analysis". In: *Proc. Intl. Workshop on Statistical-Mechanical Informatics*. 2009.

[RB93] M. Riedmiller and H. Braun. "A direct adaptive method for faster backpropagation learning: The RPROP algorithm". In: *ICNN*. IEEE. 1993, pp. 586–591.

[RC04] C. Robert and G. Casella. *Monte Carlo Statisical Methods*. 2nd edition. Springer, 2004.

[Red+16] J Redmon, S Divvala, R Girshick, and A Farhadi. "You Only Look Once: Unified, Real-Time Object Detection". In: *CVPR*. 2016, pp. 779–788.

[Ren+09] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. "BPR: Bayesian Personalized Ranking from Implicit Feedback". In: *UAI*. 2009.

[Ren12] S. Rendle. "Factorization Machines with libFM". In: *ACM Trans. Intell. Syst. Technol.* 3.3 (May 2012), pp. 1–22.

[Ren19] Z. Ren. *List of papers on self-supervised learning*. 2019.

[Res+11] D. Reshef et al. "Detecting Novel Associations in Large Data Sets". In: *Science* 334 (2011), pp. 1518–1524.

[Res+16] Y. A. Reshef, D. N. Reshef, H. K. Finucane, P. C. Sabeti, and M. Mitzenmacher. "Measuring Dependence Powerfully and Equitably". In: *J. Mach. Learn. Res.* 17.211 (2016), pp. 1–63.

[RF17] J. Redmon and A. Farhadi. "YOLO9000: Better, Faster, Stronger". In: *CVPR*. 2017.

[RFB15] O. Ronneberger, P. Fischer, and T. Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *MICCAI (Intl. Conf. on Medical Image Computing and Computer Assisted Interventions)*. 2015.

[RG11] A. Rodriguez and K. Ghosh. *Modeling relational data through nested partition models*. Tech. rep. UC Santa Cruz, 2011.

[RH05] H. Rue and L. Held. *Gaussian Markov Random Fields: Theory and Applications*. Vol. 104. Monographs on Statistics and Applied Probability. London: Chapman & Hall, 2005.

[RHS05] C. Rosenberg, M. Hebert, and H. Schneiderman. "Semi-Supervised Self-Training of Object Detection Models". In: *Proceedings of the Seventh IEEE Workshops on Application of Computer Vision (WACV/MOTION'05)-Volume 1-Volume 01*. 2005, pp. 29–36.

[RHW86] D. Rumelhart, G. Hinton, and R. Williams. "Learning internal representations by error propagation". In: *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*. Ed. by D. Rumelhart, J. McClelland, and the PDD Research Group. MIT Press, 1986.

[Ric95] J. Rice. *Mathematical statistics and data analysis*. 2nd edition. Duxbury, 1995.

[Rif+11] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio. "Contractive Auto-Encoders: Explicit Invariance During Feature Extraction". In: *ICML*. 2011.

[Ris+08] I. Rish, G. Grabarnik, G. Cecchi, F. Pereira, and G. Gordon. "Closed-form supervised dimensionality reduction with generalized linear models". In: *ICML*. 2008.

[RK04] R. Rubinstein and D. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*. Springer-Verlag, 2004.

[RKK18] S. J. Reddi, S. Kale, and S. Kumar. "On the Convergence of Adam and Beyond". In: *ICLR*. 2018.

[RM01] N. Roy and A. McCallum. "Toward optimal active learning through Monte Carlo estimation of error reduction". In: *ICML*. 2001.

[RMC09] H. Rue, S. Martino, and N. Chopin. "Approximate Bayesian Inference for Latent Gaussian Models Using Integrated Nested Laplace Approximations". In: *JRSSB* 71 (2009), pp. 319–392.

[RMC16] A. Radford, L. Metz, and S. Chintala. "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks". In: *ICLR*. 2016.

[RMW14] D. J. Rezende, S. Mohamed, and D. Wierstra. "Stochastic Backpropagation and Approximate Inference in Deep Generative Models". In: *ICML*. Ed. by E. P. Xing and T. Jebara. Vol. 32. Proceedings of Machine Learning Research. Bejing, China: PMLR, 2014, pp. 1278–1286.

[RN10] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 3rd edition. Prentice Hall, 2010.

[Ros58] F. Rosenblatt. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6 (1958), pp. 386–408.

[Ros98] K. Rose. "Deterministic Annealing for Clustering, Compression, Classification, Regression, and Related Optimization Problems". In: *Proc. IEEE* 80 (1998), pp. 2210–2239.

[Rot+20] K. Roth, T. Milbich, S. Sinha, P. Gupta, B. Ommer, and J. P. Cohen. "Revisiting Training Strategies and Generalization Performance in Deep Metric Learning". In: *ICML*. 2020.

[Row97] S. Roweis. "EM algorithms for PCA and SPCA". In: *NIPS*. 1997.

[Roy+20] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. "Efficient Content-Based Sparse Attention with Routing Transformers". In: *CoRR* abs/2003.05997 (2020). arXiv: 2003.05997.

[Roz+19] B. Rozemberczki, R. Davies, R. Sarkar, and C. Sutton. "GEMSEC: Graph Embedding with Self Clustering". In: *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. ASONAM '19. Vancouver, British Columbia, Canada: Association for Computing Machinery, 2019, 65–72.

[RP99] M. Riesenhuber and T. Poggio. "Hierarchical Models of Object Recognition in Cortex". In: *Nature Neuroscience* 2 (1999), pp. 1019–1025.

[RR08] A. Rahimi and B. Recht. "Random Features for Large-Scale Kernel Machines". In: *NIPS*. Curran Associates, Inc., 2008, pp. 1177–1184.

[RR09] A. Rahimi and B. Recht. "Weighted Sums of Random Kitchen Sinks: Replacing minimization with randomization in learning". In: *NIPS*. Curran Associates, Inc., 2009, pp. 1313–1320.

[RS00] S. T. Roweis and L. K. Saul. "Nonlinear dimensionality reduction by locally linear embedding". en. In: *Science* 290.5500 (Dec. 2000), pp. 2323–2326.

[RT82] D. B. Rubin and D. T. Thayer. "EM algorithms for ML factor analysis". In: *Psychometrika* 47.1 (Mar. 1982), pp. 69–76.

[Rub84] D. B. Rubin. "Bayesianly Justifiable and Relevant Frequency Calculations for the Applied Statistician". In: *Ann. Stat.* 12.4 (1984), pp. 1151–1172.

[Rub97] R. Y. Rubinstein. "Optimization of computer simulation models with rare events". In: *Eur. J. Oper. Res.* 99.1 (May 1997), pp. 89–112.

[Rup88] D Ruppert. *Efficient Estimations from a Slowly Convergent Robbins-Monro Process*. Tech. rep. 1988.

[Rus+15] O. Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *Intl. J. Computer Vision* (2015), pp. 1–42.

[Rus15] S. Russell. "Unifying Logic and Probability". In: *Commun. ACM* 58.7 (June 2015), pp. 88–97.

[Rus18] A. M. Rush. "The Annotated Transformer". In: *Proceedings of ACL Workshop on Open Source Software for NLP*. 2018.

[Rus+18] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen. "A Tutorial on Thompson Sampling". In: *Foundations and Trends in Machine Learning* 11.1 (2018), pp. 1–96.

[Rus19] S. Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. en. Kindle. Viking, Oct. 2019.

[RW06] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[RW15] D. Rosenbaum and Y. Weiss. "The Return of the Gating Network: Combining Generative Models and Discriminative Training in Natural Image Priors". In: *NIPS*. 2015, pp. 2665–2673.

[RW18] E. Richardson and Y. Weiss. "On GANs and GMMs". In: *NIPS*. 2018.

[RZL17] P. Ramachandran, B. Zoph, and Q. V. Le. "Searching for Activation Functions". In: (Oct. 2017). arXiv: 1710.05941 [cs.NE].

[SA93] P Sinha and E Adelson. "Recovering reflectance and illumination in a world of painted polyhedra". In: *ICCV*. May 1993, pp. 156–163.

[Sal+16] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. "Improved Techniques for Training GANs". In: (2016). arXiv: 1606.03498 [cs.LG].

[Sal+20] H. Salman, A. Ilyas, L. Engstrom, A. Kapoor, and A. Madry. "Do Adversarially Robust ImageNet Models Transfer Better?" In: *arXiv preprint arXiv:2007.08489* (2020).

[SAM04] D. J. Spiegelhalter, K. R. Abrams, and J. P. Myles. *Bayesian Approaches to Clinical Trials and Health-Care Evaluation*. Wiley, 2004.

[San+18a] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. "Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation". In: (Jan. 2018). arXiv: 1801.04381 [cs.CV].

[San+18b] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry. "How Does Batch Normalization Help Optimization? (No, It Is Not About Internal Covariate Shift)". In: *NIPS*. 2018.

[San96] R. Santos. "Equivalence of regularization and truncated iteration for general ill-posed problems". In: *Linear Algebra and its Applications* 236.15 (1996), pp. 25–33.

[Sar11] R. Sarkar. "Low distortion delaunay embedding of trees in hyperbolic plane". In: *International Symposium on Graph Drawing*. Springer. 2011, pp. 355–366.

[SAV20] E. Stevens, L. Antiga, and T. Viehmann. *Deep Learning with PyTorch*. Manning, 2020.

[SB18] R. Sutton and A. Barto. *Reinforcement learning: an introduction (2nd edn)*. MIT Press, 2018.

[SBB01] T. Sellke, M. J. Bayarri, and J. Berger. "Calibration of p Values for Testing Precise Null Hypotheses". In: *The American Statistician* 55.1 (2001), pp. 62–71.

[SBP17] Y Sun, P Babu, and D. P. Palomar. "Majorization-Minimization Algorithms in Signal Processing, Communications, and Machine Learning". In: *IEEE Trans. Signal Process.* 65.3 (Feb. 2017), pp. 794–816.

[Sca+09] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. "The graph neural network model". In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80.

[Sca+17] S. Scardapane, D. Comminiello, A. Hussain, and A. Uncini. "Group Sparse Regularization for Deep Neural Networks". In: *Neurocomputing* 241 (2017).

[Sch+00] B Scholkopf, A. J. Smola, R. C. Williamson, and P. L. Bartlett. "New support vector algorithms". en. In: *Neural Comput.* 12.5 (May 2000), pp. 1207–1245.

[Sch78] G. Schwarz. "Estimating the dimension of a model". In: *Annals of Statistics* 6.2 (1978), pp. 461–464.

[Sch90] R. E. Schapire. "The strength of weak learnability". In: *Mach. Learn.* 5.2 (June 1990), pp. 197–227.

[Sco09] S. Scott. "Data augmentation, frequentist estimation, and the Bayesian analysis of multinomial logit models". In: *Statistical Papers* (2009).

[Sco10] S. Scott. "A modern Bayesian look at the multi-armed bandit". In: *Applied Stochastic Models in Business and Industry* 26 (2010), pp. 639–658.

[Sco79] D. Scott. "On optimal and data-based histograms". In: *Biometrika* 66.3 (1979), pp. 605–610.

[Scu10] D Sculley. "Web-scale k-means clustering". In: *WWW*. WWW '10. Raleigh, North Carolina, USA: Association for Computing Machinery, Apr. 2010, pp. 1177–1178.

[Scu65] H. Scudder. "Probability of error of some adaptive pattern-recognition machines". In: *IEEE Transactions on Information Theory* 11.3 (1965), pp. 363–371.

[SD12] J. Sohl-Dickstein. "The Natural Gradient by Analogy to Signal Whitening, and Recipes and Tricks for its Use". In: (May 2012). arXiv: 1205.1828 [cs.LG].

[Sed+15] S. Sedhain, A. K. Menon, S. Sanner, and L. Xie. "AutoRec: Autoencoders Meet Collaborative Filtering". In: *WWW*. WWW '15 Companion. Florence, Italy: Association for Computing Machinery, May 2015, pp. 111–112.

[Sej18] T. J. Sejnowski. *The Deep Learning Revolution*. en. Kindle. The MIT Press, Sept. 2018.

[Set12] B. Settles. "Active learning". In: *Synthesis Lectures on Artificial Intelligence and Machine Learning* 6 (2012), 1–114.

[SF12] R. Schapire and Y. Freund. *Boosting: Foundations and Algorithms*. MIT Press, 2012.

[SG12] D. Sejdinovic and A. Gretton. *What is an RKHS?* 2012.

[SGJ11] D. Sontag, A. Globerson, and T. Jaakkola. "Introduction to Dual Decomposition for Inference". In: *Optimization for Machine Learning*. Ed. by S. Sra, S. Nowozin, and S. J. Wright. MIT Press, 2011.

[Sha+06] P. Shafto, C. Kemp, V. Mansinghka, M. Gordon, and J. B. Tenenbaum. "Learning cross-cutting systems of categories". In: *Cognitive Science Conference*. 2006.

[Sha+16a] B Shahriari, K Swersky, Z. Wang, R. P. Adams, and N de Freitas. "Taking the Human Out of the Loop: A Review of Bayesian Optimization". In: *Proc. IEEE* 104.1 (Jan. 2016), pp. 148–175.

[Sha+16b] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter. "Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016, pp. 1528–1540.

[Sha+17] N. Shazeer et al. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *ICLR*. 2017.

[Sha+19] A. Shaikhha, A. Fitzgibbon, D. Vytiniotis, and S. Peyton Jones. "Efficient differentiable programming in a functional array-processing language". In: *Proceedings of the ACM on Programming Languages* 3.ICFP (2019), pp. 1–30.

[Sha88] T. Shallice. *From Neuropsychology to Mental Structure*. 1988.

[SHB16] R. Sennrich, B. Haddow, and A. Birch. "Neural Machine Translation of Rare Words with Subword Units". In: *Proc. ACL*. 2016.

[She+18] Z. Shen, M. Zhang, S. Yi, J. Yan, and H. Zhao. "Factorized Attention: Self-Attention with Linear Complexities". In: *CoRR* abs/1812.01243 (2018). arXiv: 1812.01243.

[She94] J. R. Shewchuk. *An introduction to the conjugate gradient method without the agonizing pain*. Tech. rep. CMU, 1994.

[SHF15] R. Steorts, R. Hall, and S. Fienberg. "A Bayesian Approach to Graphical Record Linkage and Deduplication". In: *JASA* (2015).

[SHS] D. Stutz, M. Hein, and B. Schiele. "Confidence-calibrated adversarial training: Generalizing to unseen attacks". In: ().

[SHS01] B. Schölkopf, R. Herbrich, and A. J. Smola. "A Generalized Representer Theorem". In: *COLT*. COLT '01/EuroCOLT '01. London, UK, UK: Springer-Verlag, 2001, pp. 416–426.

[Shu+13] D. I. Shuman, S. K. Narang, P Frossard, A Ortega, and P Vandergheynst. "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains". In: *IEEE Signal Process. Mag.* 30.3 (2013), pp. 83–98.

[Sil18] D. Silver. *Lecture 9L Exploration and Exploitation*. 2018.

[Sin+20] S. Sinha, H. Zhang, A. Goyal, Y. Bengio, H. Larochelle, and A. Odena. "Small-GAN: Speeding up GAN Training using Core-Sets". In: *ICML*. Vol. 119. Proceedings of Machine Learning Research. Virtual: PMLR, 2020, pp. 9005–9015.

[SIV17] C. Szegedy, S. Ioffe, and V. Vanhoucke. "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning". In: *AAAI*. 2017.

[SJ03] N. Srebro and T. Jaakkola. "Weighted low-rank approximations". In: *ICML*. 2003.

[SJT16] M. Sajjadi, M. Javanmardi, and T. Tasdizen. "Regularization with stochastic transformations and perturbations for deep semi-supervised learning". In: *Advances in neural information processing systems*. 2016, pp. 1163–1171.

[SKP15] F. Schroff, D. Kalenichenko, and J. Philbin. "FaceNet: A Unified Embedding for Face Recognition and Clustering". In: *CVPR*. 2015.

[SKT14] A. Szlam, Y. Kluger, and M. Tygert. "An implementation of a randomized algorithm for principal component analysis". In: (2014). arXiv: 1412.3510 [stat.CO].

[SKTF18] H. Shao, A. Kumar, and P Thomas Fletcher. "The Riemannian Geometry of Deep Generative Models". In: *CVPR*. 2018, pp. 315–323.

[SL18] S. L. Smith and Q. V. Le. "A Bayesian Perspective on Generalization and Stochastic Gradient Descent". In: *ICLR*. 2018.

[SL+19] B. Sanchez-Lengeling, J. N. Wei, B. K. Lee, R. C. Gerkin, A. Aspuru-Guzik, and A. B. Wiltschko. "Machine Learning for Scent: Learning Generalizable Perceptual Representations of Small Molecules". In: (Oct. 2019). arXiv: 1910.10685 [stat.ML].

[SL90] D. J. Spiegelhalter and S. L. Lauritzen. "Sequential updating of conditional probabilities on directed graphical structures". In: *Networks* 20 (1990).

[Sli19] A. Slivkins. "Introduction to Multi-Armed Bandits". In: (Apr. 2019). arXiv: 1904.07272 [cs.LG].

[SLRB17] M. Schmidt, N. Le Roux, and F. Bach. "Minimizing finite sums with the stochastic average gradient". In: *Mathematical Programming* 162.1-2 (2017), pp. 83–112.

[SM00] J. Shi and J. Malik. "Normalized Cuts and Image Segmentation". In: *IEEE PAMI* (2000).

[SM08] R. Salakhutdinov and A. Mnih. "Probabilistic Matrix Factorization". In: *NIPS*. Vol. 20. 2008.

[SMG14] A. M. Saxe, J. L. McClelland, and S. Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks". In: *ICLR*. 2014.

[SMH07] R. Salakhutdinov, A. Mnih, and G. Hinton. "Restricted Boltzmann machines for collaborative filtering". In: *ICML*. ICML '07. Corvalis, Oregon, USA: Association for Computing Machinery, June 2007, pp. 791–798.

[Smi18] L. Smith. "A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay". In: (2018).

[Smi+21] S. L. Smith, B. Dherin, D. Barrett, and S. De. "On the Origin of Implicit Regularization in Stochastic Gradient Descent". In: *ICLR*. 2021.

[SMM03] Q. Sheng, Y. Moreau, and B. D. Moor. "Biclustering Microarray data by Gibbs sampling". In: *Bioinformatics* 19 (2003), pp. ii196–ii205.

[SNM16] M. Suzuki, K. Nakayama, and Y. Matsuo. "Joint Multimodal Learning with Deep Generative Models". In: (2016). arXiv: 1611.01891 [stat.ML].

[Soh16] K. Sohn. "Improved Deep Metric Learning with Multi-class N-pair Loss Objective". In: *NIPS*. Curran Associates, Inc., 2016, pp. 1857–1865.

[Soh+20] K. Sohn et al. "Fixmatch: Simplifying semi-supervised learning with consistency and confidence". In: *arXiv preprint arXiv:2001.07685* (2020).

[Sör15] K. Sörensen. "Metaheuristics—the metaphor exposed". In: *Intl. Trans. in Op. Res.* 22.1 (Jan. 2015), pp. 3–18.

[SP97] M Schuster and K. K. Paliwal. "Bidirectional recurrent neural networks". In: *IEEE. Trans on Signal Processing* 45.11 (Nov. 1997), pp. 2673–2681.

[Spe11] T. Speed. "A correlation for the 21st century". In: *Science* 334 (2011), pp. 152–1503.

[Spi71] M. Spivak. *Calculus On Manifolds: A Modern Approach To Classical Theorems Of Advanced Calculus*. Westview Press; 5th edition, 1971.

[SR15] T. Saito and M. Rehmsmeier. "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets". en. In: *PLoS One* 10.3 (Mar. 2015), e0118432.

[SRG03] R. Salakhutdinov, S. T. Roweis, and Z. Ghahramani. "Optimization with EM and Expectation-Conjugate-Gradient". In: *ICML*. 2003.

[Sri+14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. "Dropout: A Simple Way to Prevent Neural Networks from Over tting". In: *JMLR* (2014).

[SS01] B. Schlkopf and A. J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond (Adaptive Computation and Machine Learning)*. en. 1st edition. The MIT Press, Dec. 2001.

[SS02] B. Schoelkopf and A. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[SS05] J. Schaefer and K. Strimmer. "A shrinkage approach to large-scale covariance matrix estimation and implications for functional genomics". In: *Statist. Appl. Genet. Mol. Biol* 4.32 (2005).

[SS19] S. Serrano and N. A. Smith. "Is Attention Interpretable?" In: *Proc. ACL*. 2019.

[SS95] H. T. Siegelmann and E. D. Sontag. "On the Computational Power of Neural Nets". In: *J. Comput. System Sci.* 50.1 (Feb. 1995), pp. 132–150.

[SSE18] Y. Song, J. Song, and S. Ermon. "Accelerating Natural Gradient with Higher-Order Invariance". In: *ICML*. 2018.

[SSM98] B. Schoelkopf, A. Smola, and K.-R. Mueller. "Nonlinear component analysis as a kernel Eigenvalue problem". In: *Neural Computation* 10 (5 1998), pp. 1299 –1319.

[Sta+06] C. Stark, B.-J. Breitkreutz, T. Reguly, L. Boucher, A. Breitkreutz, and M. Tyers. "BioGRID: a general repository for interaction datasets". In: *Nucleic acids research* 34.suppl_1 (2006), pp. D535–D539.

[Sta07] K. O. Stanley. "Compositional pattern producing networks: A novel abstraction of development". In: *Genet. Program. Evolvable Mach.* 8.2 (Oct. 2007), pp. 131–162.

[Sta+17] N. Stallard et al. "Determination of the optimal sample size for a clinical trial accounting for the population size". en. In: *Biom. J.* 59.4 (July 2017), pp. 609–625.

[STC04] J. Shawe-Taylor and N. Cristianini. *Kernel Methods for Pattern Analysis*. Cambridge, 2004.

[Ste56] C. Stein. "Inadmissibility of the usual estimator for the mean of a multivariate distribution". In: *Proc. 3rd Berkeley Symposium on Mathematical Statistics and Probability* (1956), 197–206.

[Str09] G. Strang. *Introduction to linear algebra*. 4th edition. SIAM Press, 2009.

[Sug+19] A. S. Suggala, K. Bhatia, P. Ravikumar, and P. Jain. "Adaptive Hard Thresholding for Near-optimal Consistent Robust Regression". In: *Proceedings of the Annual Conference On Learning Theory (COLT)*. 2019, pp. 2892–2897.

[Sun+09] L. Sun, S. Ji, S. Yu, and J. Ye. "On the Equivalence Between Canonical Correlation Analysis and Orthonormalized Partial Least Squares". In: *IJCAI*. 2009.

[Sun+19a] C. Sun, A. Myers, C. Vondrick, K. Murphy, and C. Schmid. "VideoBERT: A Joint Model for Video and Language Representation Learning". In: *ICCV*. 2019.

[Sun+19b] S. Sun, Z. Cao, H. Zhu, and J. Zhao. "A Survey of Optimization Methods from a Machine Learning Perspective". In: (June 2019). arXiv: 1906.06821 [cs.LG].

[SVK19] J. Su, D. V. Vargas, and S. Kouichi. "One pixel attack for fooling deep neural networks". In: *IEEE Trans. Evol. Comput.* 23.5 (2019).

[SVL14] I. Sutskever, O. Vinyals, and Q. V. V. Le. "Sequence to Sequence Learning with Neural Networks". In: *NIPS*. 2014.

[SVZ14] K. Simonyan, A. Vedaldi, and A. Zisserman. "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps". In: *ICLR*. 2014.

[SW13] G. J. Sussman and J. Wisdom. *Functional Differential Geometry*. Functional Differential Geometry. MIT Press, 2013.

[SW87] M. Shewry and H. Wynn. "Maximum entropy sampling". In: *J. Applied Statistics* 14 (1987), 165–170.

[SWY75] G Salton, A Wong, and C. S. Yang. "A vector space model for automatic indexing". In: *Commun. ACM* 18.11 (Nov. 1975), pp. 613–620.

[Sze10] C. Szepesvari. *Algorithms for Reinforcement Learning*. Morgan Claypool, 2010.

[Sze+14] C. Szegedy et al. "Intriguing properties of neural networks". In: *ICLR*. 2014.

[Sze+15a] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. "Rethinking the Inception Architecture for Computer Vision". In: (2015). arXiv: 1512.00567 [cs.CV].

[Sze+15b] C. Szegedy et al. "Going Deeper with Convolutions". In: *CVPR*. 2015.

[Tal07] N. Taleb. *The Black Swan: The Impact of the Highly Improbable*. Random House, 2007.

[Tan+15] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei. "Line: Large-scale information network embedding". In: *Proceedings of the 24th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee. 2015, pp. 1067–1077.

[Tan+18] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu. "A Survey on Deep Transfer Learning". In: *ICANN*. 2018.

[TAS18] M. Teye, H. Azizpour, and K. Smith. "Bayesian Uncertainty Estimation for Batch Normalized Deep Networks". In: *ICML*. 2018.

[Tay+20a] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. "Efficient Transformers: A Survey". In: (Sept. 2020). arXiv: 2009.06732 [cs.LG].

[Tay+20b] Y. Tay et al. "Long Range Arena: A Benchmark for efficient Transformers". In: *CoRR* (2020).

[TB97] L. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, 1997.

[TB99] M. Tipping and C. Bishop. "Probabilistic principal component analysis". In: *JRSSB* 21.3 (1999), pp. 611–622.

[TDP19] I. Tenney, D. Das, and E. Pavlick. "BERT Rediscovers the Classical NLP Pipeline". In: *Proc. ACL*. 2019.

[TF03] M. Tipping and A. Faul. "Fast marginal likelihood maximisation for sparse Bayesian models". In: *AI/Stats*. 2003.

[Tho16] M. Thoma. "Creativity in Machine Learning". In: (Jan. 2016). arXiv: 1601.03642 [cs.CV].

[Tho17] R. Thomas. *Computational Linear Algebra for Coders*. 2017.

[Tho+19] V. Thomas, F. Pedregosa, B. van Merriënboer, P.-A. Mangazol, Y. Bengio, and N. Le Roux. "Information matrices and generalization". In: (June 2019). arXiv: 1906.07774 [cs.LG].

[Tho33] W. R. Thompson. "On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples". In: *Biometrika* 25.3/4 (1933), pp. 285–294.

[Tib96] R. Tibshirani. "Regression shrinkage and selection via the lasso". In: *J. Royal. Statist. Soc B* 58.1 (1996), pp. 267–288.

[Tip01] M. Tipping. "Sparse Bayesian learning and the relevance vector machine". In: *JMLR* 1 (2001), pp. 211–244.

[Tip98] M. Tipping. "Probabilistic visualization of high-dimensional binary data". In: *NIPS*. 1998.

[Tit16] M. Titsias. "One-vs-Each Approximation to Softmax for Scalable Estimation of Probabilities". In: *NIPS*. 2016, pp. 4161–4169.

[TK86] L. Tierney and J. Kadane. "Accurate approximations for posterior moments and marginal densities". In: *JASA* 81.393 (1986).

[TL21] M. Tan and Q. V. Le. "EfficientNetV2: Smaller Models and Faster Training". In: (Apr. 2021). arXiv: 2104.00298 [cs.CV].

[TMD12] A. Talhouk, K. Murphy, and A. Doucet. "Efficient Bayesian Inference for Multivariate Probit Models with Sparse Inverse Correlation Matrices". In: *J. Comp. Graph. Statist.* 21.3 (2012), pp. 739–757.

[TMP20] A. Tsitsulin, M. Munkhoeva, and B. Perozzi. "Just SLaQ When You Approximate: Accurate Spectral Distances for Web-Scale Graphs". In: *Proceedings of The Web Conference 2020*. WWW '20. 2020, 2697–2703.

[TOB16] L. Theis, A. van den Oord, and M. Bethge. "A note on the evaluation of generative models". In: *ICLR*. 2016.

[TP10] P. D. Turney and P. Pantel. "From Frequency to Meaning: Vector Space Models of Semantics". In: *JAIR* 37 (2010), pp. 141–188.

[TP97] S. Thrun and L. Pratt, eds. *Learning to learn*. Kluwer, 1997.

[TS92] D. G. Terrell and D. W. Scott. "Variable kernel density estimation". In: *Annals of Statistics* 20.3 (1992), 1236–1265.

[Tsa88] C. Tsallis. "Possible generalization of Boltzmann-Gibbs statistics". In: *J. of Statistical Physics* 52 (1988), pp. 479–487.

[Tsi+18] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller. "NetLSD: Hearing the Shape of a Graph". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. 2018, 2347–2356.

[TSL00] J. Tenenbaum, V. de Silva, and J. Langford. "A global geometric framework for nonlinear dimensionality reduction". In: *Science* 290.550 (2000), pp. 2319–2323.

[Tur13] M. Turk. "Over Twenty Years of Eigenfaces". In: *ACM Trans. Multimedia Comput. Commun. Appl.* 9.1s (Oct. 2013), 45:1–45:5.

[TV17] A. Tarvainen and H. Valpola. "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results". In: *Advances in neural information processing systems*. 2017, pp. 1195–1204.

[TVW05] B. Turlach, W. Venables, and S. Wright. "Simultaneous Variable Selection". In: *Technometrics* 47.3 (2005), pp. 349–363.

[TW18] J. Tang and K. Wang. "Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding". In: *WSDM*. WSDM '18. Marina Del Rey, CA, USA: Association for Computing Machinery, Feb. 2018, pp. 565–573.

[TXT19] V. Tjeng, K. Xiao, and R. Tedrake. "Evaluating Robustness of Neural Networks with Mixed Integer Programming". In: *ICLR*. 2019.

[Ude+16] M. Udell, C. Horn, R. Zadeh, and S. Boyd. "Generalized Low Rank Models". In: *Foundations and Trends in Machine Learning* 9.1 (2016), pp. 1–118.

[Uly+16] D. Ulyanov, V. Lebedev, Andrea, and V. Lempitsky. "Texture Networks: Feed-forward Synthesis of Textures and Stylized Images". In: *ICML*. 2016, pp. 1349–1357.

[Uur+17] V. Uurtio, J. M. Monteiro, J. Kandola, J. Shawe-Taylor, D. Fernandez-Reyes, and J. Rousu. "A Tutorial on Canonical Correlation Methods". In: *ACM Computing Surveys* (2017).

[UVL16] D. Ulyanov, A. Vedaldi, and V. Lempitsky. "Instance Normalization: The Missing Ingredient for Fast Stylization". In: (2016). arXiv: 1607.08022 [cs.CV].

[Val00] H. Valpola. "Bayesian Ensemble Learning for Nonlinear Factor Analysis". PhD thesis. Helsinki University of Technology, 2000.

[Van06] L. Vandenberghe. *Applied Numerical Computing: Lecture notes*. 2006.

[Van14] J. VanderPlas. *Frequentism and Bayesianism III: Confidence, Credibility, and why Frequentism and Science do not Mix*. Blog post. 2014.

[Van18] J. Vanschoren. "Meta-Learning: A Survey". In: (Oct. 2018). arXiv: 1810.03548 [cs.LG].

[Vap98] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[Vas+17] A. Vaswani et al. "Attention Is All You Need". In: *NIPS*. 2017.

[Vas+19] S. Vaswani, A. Mishkin, I. Laradji, M. Schmidt, G. Gidel, and S. Lacoste-Julien. "Painless Stochastic Gradient: Interpolation, Line-Search, and Convergence Rates". In: *NIPS*. Curran Associates, Inc., 2019, pp. 3727–3740.

[VBW15] S. S. Villar, J. Bowden, and J. Wason. "Multi-armed Bandit Models for the Optimal Design of Clinical Trials: Benefits and Challenges". en. In: *Stat. Sci.* 30.2 (2015), pp. 199–215.

[VD99] S. Vaithyanathan and B. Dom. "Model Selection in Unsupervised Learning With Applications To Document Clustering". In: *ICML*. 1999.

[VEB09] N. Vinh, J. Epps, and J. Bailey. "Information Theoretic Measures for Clusterings Comparison: Is a Correction for Chance Necessary?" In: *ICML*. 2009.

[Vel+18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. "Graph attention networks". In: *International Conference on Learning Representations*. 2018.

[Vel+19] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm. "Deep Graph Infomax". In: *International Conference on Learning Representations*. 2019.

[VGS97] V. Vapnik, S. Golowich, and A. Smola. "Support vector method for function approximation, regression estimation, and signal processing". In: *NIPS*. 1997.

[Vig15] T. Vigen. *Spurious Correlations*. en. Gift edition. Hachette Books, May 2015.

[Vin+10a] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion". In: *JMLR* 11 (2010), pp. 3371–3408.

[Vin+10b] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion". In: *Journal of machine learning research* 11.Dec (2010), pp. 3371–3408.

[Vin+16] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra. "Matching Networks for One Shot Learning". In: *NIPS*. 2016.

[Vir10] S. Virtanen. "Bayesian exponential family projections". MA thesis. Aalto University, 2010.

[Vis+10] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgward. "Graph Kernels". In: *JMLR* 11 (2010), pp. 1201–1242.

[Vo+15] B.-N. Vo et al. *Multitarget tracking*. John Wiley and Sons, 2015.

[Vor+17] E. Vorontsov, C. Trabelsi, S. Kadoury, and C. Pal. "On orthogonality and learning recurrent networks with long term dependencies". In: *ICML*. 2017.

[VT17] C. Vondrick and A. Torralba. "Generating the Future with Adversarial Transformers". In: *CVPR*. 2017.

[VV13] G. Valiant and P. Valiant. "Estimating the unseen: improved estimators for entropy and other properties". In: *NIPS*. 2013.

[Vyt+19] D. Vytiniotis, D. Belov, R. Wei, G. Plotkin, and M. Abadi. "The differentiable curry". In: *NeurIPS 2019 Workshop Program Transformations*. 2019.

[Wah+21] O. Wahltinez et al. "COVID-19 Open-Data: a global-scale, spatially granular meta-dataset for SARS-CoV-2". In: (2021). To appear.

[Wal+20] M. Walmsley et al. "Galaxy Zoo: probabilistic morphology through Bayesian CNNs and active learning". In: *Monthly Notices Royal Astronomial Society* 491.2 (Jan. 2020), pp. 1554–1574.

[Wal47] A. Wald. "An Essentially Complete Class of Admissible Decision Functions". en. In: *Ann. Math. Stat.* 18.4 (Dec. 1947), pp. 549–555.

[Wan+15] J. Wang, W. Liu, S. Kumar, and S.-F. Chang. "Learning to Hash for Indexing Big Data - A Survey". In: *Proc. IEEE* (2015).

[Wan+17] Y. Wang et al. "Tacotron: Towards End-to-End Speech Synthesis". In: *Interspeech*. 2017.

[Wan+20a] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. "Linformer: Self-Attention with Linear Complexity". In: *CoRR* abs/2006.04768 (2020). arXiv: 2006.04768.

[Wan+20b] Y. Wang, Q. Yao, J. Kwok, and L. M. Ni, "Generalizing from a Few Examples: A Survey on Few-Shot Learning". In: *ACM Computing Surveys* 1.1 (2020).

[Wan+20c] Y. Wang, Y.-C. Chen, X. Tao, and J. Jia. "VCNet: A Robust Approach to Blind Image Inpainting". In: *ECCV*. 2020.

[Wan+21] R. Wang, M. Cheng, X. Chen, X. Tang, and C.-J. Hsieh. "Rethinking Architecture Selection in Differentiable NAS". In: *ICLR*. 2021.

[Wat10] S. Watanabe. "Asymptotic Equivalence of Bayes Cross Validation and Widely Applicable Information Criterion in Singular Learning Theory". In: *JMLR* 11 (Dec. 2010), pp. 3571–3594.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[Wat13] S. Watanabe. "A Widely Applicable Bayesian Information Criterion". In: *JMLR* 14 (2013), pp. 867–897.

[WCS08] M. Welling, C. Chemudugunta, and N. Sutter. "Deterministic Latent Variable Models and their Pitfalls". In: *ICDM*. 2008.

[WCZ16] D. Wang, P. Cui, and W. Zhu. "Structural deep network embedding". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2016, pp. 1225–1234.

[Wei76] J. Weizenbaum. *Computer Power and Human Reason: From Judgment to Calculation*. en. 1st ed. W H Freeman & Co, Mar. 1976.

[Wen+16] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li. "Learning Structured Sparsity in Deep Neural Networks". In: (2016). arXiv: 1608.03665 [cs.NE].

[Wen18] L. Weng. "Attention? Attention!" In: *lilianweng.github.io/lil-log* (2018).

[Wen19] L. Weng. "Generalized Language Models". In: *lilianweng.github.io/lil-log* (2019).

[Wer74] P. Werbos. "Beyond regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard, 1974.

[Wer90] P. J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proc. IEEE* 78.10 (1990), pp. 1550–1560.

[Wes03] M. West. "Bayesian Factor Regression Models in the "Large p, Small n" Paradigm". In: *Bayesian Statistics* 7 (2003).

[Wes87] M. West. "On scale mixtures of normal distributions". In: *Biometrika* 74 (1987), pp. 646–648.

[WF14] Z. Wang and N. de Freitas. "Theoretical Analysis of Bayesian Optimisation with Unknown Gaussian Process Hyper-Parameters". In: (June 2014). arXiv: 1406.7758 [stat.ML].

[WF20] T. Wu and I. Fischer. "Phase Transitions for the Information Bottleneck in Representation Learning". In: *ICLR*. 2020.

[WH18] Y. Wu and K. He. "Group Normalization". In: *ECCV*. 2018.

[WH60] B. Widrow and M. E. Hoff. "Adaptive Switching Circuits". In: *1960 IRE WESCON Convention Record, Part 4*. IRE, 1960, pp. 96–104.

[WI20] A. G. Wilson and P. Izmailov. "Bayesian Deep Learning and a Probabilistic Perspective of Generalization". In: *NIPS*. Feb. 2020.

[Wil14] A. G. Wilson. "Covariance kernels for fast automatic pattern discovery and extrapolation with Gaussian processes". PhD thesis. University of Cambridge, 2014.

[Wil20] C. K. I. Williams. "The Effect of Class Imbalance on Precision-Recall Curves". In: *Neural Comput.* (July 2020).

[Wiy+19] R. R. Wiyatno, A. Xu, O. Dia, and A. de Berker. "Adversarial Examples in Modern Machine Learning: A Review". In: (Nov. 2019). arXiv: 1911.05268 [cs.LG].

[WK18] E. Wong and Z. Kolter. "Provable defenses against adversarial examples via the convex outer adversarial polytope". In: *International Conference on Machine Learning*. PMLR. 2018, pp. 5286–5295.

[WL08] T. T. Wu and K. Lange. "Coordinate descent algorithms for lasso penalized regression". In: *Ann. Appl. Stat* 2.1 (2008), pp. 224–244.

[WL16] R. L. Wasserstein and N. A. Lazar. "The ASA Statement on p-Values: Context, Process, and Purpose". In: *The American Statistician* 70.2 (Apr. 2016), pp. 129–133.

[WLD20] J. Whang, Q. Lei, and A. G. Dimakis. "Compressed Sensing with Invertible Generative Models and Dependent Noise". In: *NeurIPS 2020 Workshop on Deep Inverse Problems*. 2020.

[WLL16] W. Wang, H. Lee, and K. Livescu. "Deep Variational Canonical Correlation Analysis". In: *arXiv* (Nov. 2016).

[WM00] D. R. Wilson and T. R. Martinez. "Reduction Techniques for Instance-Based Learning Algorithms". In: *Mach. Learn.* 38.3 (Mar. 2000), pp. 257–286.

[WN07] D. Wipf and S. Nagarajan. "A new view of automatic relevancy determination". In: *NIPS*. 2007.

[WN10] D. Wipf and S. Nagarajan. "Iterative Reweighted ℓ_1 and ℓ_2 Methods for Finding Sparse Solutions". In: *J. of Selected Topics in Signal Processing (Special Issue on Compressive Sensing)* 4.2 (2010).

[WNF09] S. Wright, R. Nowak, and M. Figueiredo. "Sparse reconstruction by separable approximation". In: *IEEE Trans. on Signal Processing* 57.7 (2009), pp. 2479–2493.

[WNS19] C. White, W. Neiswanger, and Y. Savani. "BANANAS: Bayesian Optimization with Neural Architectures for Neural Architecture Search". In: (Oct. 2019). arXiv: 1910.11858 [cs.LG].

[Wol92] D. Wolpert. "Stacked Generalization". In: *Neural Networks* 5.2 (1992), pp. 241–259.

[Wol96] D. Wolpert. "The lack of a priori distinctions between learning algorithms". In: *Neural Computation* 8.7 (1996), pp. 1341–1390.

[WP19] S. Wiegreffe and Y. Pinter. "Attention is not not Explanation". In: *EMNLP*. 2019.

[WRC08] J. Weston, F. Ratle, and R. Collobert. "Deep learning via semi-supervised embedding". In: *Proceedings of the 25th international conference on Machine learning*. ACM. 2008, pp. 1168–1175.

[WRN10] D. Wipf, B. Rao, and S. Nagarajan. "Latent Variable Bayesian Models for Promoting Sparsity". In: *IEEE Transactions on Information Theory* (2010).

[WS09] K. Weinberger and L. Saul. "Distance Metric Learning for Large Margin Classification". In: *JMLR* 10 (2009), pp. 207–244.

[WSH16] L. Wu, C. Shen, and A. van den Hengel. "PersonNet: Person Re-identification with Deep Convolutional Neural Networks". In: (2016). arXiv: 1601.07255 [cs.CV].

[WSS04] K. Q. Weinberger, F. Sha, and L. K. Saul. "Learning a kernel matrix for nonlinear dimensionality reduction". In: *ICML*. 2004.

[WTN19] Y. Wu, G. Tucker, and O. Nachum. "The Laplacian in RL: Learning Representations with Efficient Approximations". In: *ICLR*. 2019.

[Wu+16] Y. Wu et al. "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation". In: (Sept. 2016). arXiv: 1609.08144 [cs.CL].

[Wu+19] Y. Wu, E. Winston, D. Kaushik, and Z. Lipton. "Domain Adaptation with Asymmetrically-Relaxed Distribution Alignment". In: *ICML*. 2019.

[WVJ16] M. Wattenberg, F. Viégas, and I. Johnson. "How to Use t-SNE Effectively". In: *Distill* 1.10 (Oct. 2016).

[WW12] Y. Wu and D. P. Wipf. "Dual-Space Analysis of the Sparse Linear Model". In: *NIPS*. 2012.

[WW93] D. Wagner and F. Wagner. "Between min cut and graph bisection". In: *Proc. 18th Intl. Symp. on Math. Found. of Comp. Sci.* 1993, pp. 744–750.

[XC19] Z. Xia and A. Chakrabarti. "Training Image Estimators without Image Ground-Truth". In: *NIPS*. 2019.

[Xie+19a] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le. "Unsupervised Data Augmentation". In: (Apr. 2019). arXiv: 1904.12848 [cs.LG].

[Xie+19b] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le. "Unsupervised data augmentation for consistency training". In: *arXiv preprint arXiv:1904.12848* (2019).

[Xie+20] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le. "Self-training with noisy student improves imagenet classification". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 10687–10698.

[XJ96] L. Xu and M. I. Jordan. "On Convergence Properties of the EM Algorithm for Gaussian Mixtures". In: *Neural Computation* 8 (1996), pp. 129–151.

[XRV17] H. Xiao, K. Rasul, and R. Vollgraf. "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms". In: (2017). arXiv: 1708.07747 [stat.ML].

[Xu+15] K. Xu et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention". In: *ICML*. 2015.

[Yal+19] I. Z. Yalniz, H. Jégou, K. Chen, M. Paluri, and D. Mahajan. "Billion-scale semi-supervised learning for image classification". In: *arXiv preprint arXiv:1905.00546* (2019).

[Yan+14] X. Yang, Y. Guo, Y. Liu, and H. Steck. "A Survey of Collaborative Filtering Based Social Recommender Systems". In: *Comput. Commun.* 41 (Mar. 2014), pp. 1–10.

[Yar95] D. Yarowsky. "Unsupervised word sense disambiguation rivaling supervised methods". In: *33rd annual meeting of the association for computational linguistics*. 1995, pp. 189–196.

[YB19] C. Yadav and L. Bottou. "Cold Case: The Lost MNIST Digits". In: *arXiv* (May 2019).

[YCS16] Z. Yang, W. W. Cohen, and R. Salakhutdinov. "Revisiting semi-supervised learning with graph embeddings". In: *Proceedings of the 33rd International Conference on International Conference on Machine Learning-Volume 48*. JMLR. org. 2016, pp. 40–48.

[Yeu91] R. W. Yeung. "A new outlook on Shannon's information measures". In: *IEEE Trans. Inf. Theory* 37.3 (May 1991), pp. 466–474.

[YHJ09] D. Yan, L. Huang, and M. I. Jordan. "Fast approximate spectral clustering". In: *15th ACM Conf. on Knowledge Discovery and Data Mining*. 2009.

[Yin+19a] D. Yin, R. G. Lopes, J. Shlens, E. D. Cubuk, and J. Gilmer. "A Fourier Perspective on Model Robustness in Computer Vision". In: *NIPS*. 2019.

[Yin+19b] P. Yin, J. Lyu, S. Zhang, S. Osher, Y. Qi, and J. Xin. "Understanding Straight-Through Estimator in Training Activation Quantized Neural Nets". In: *ICLR*. 2019.

[YK06] A. Yuille and D. Kersten. "Vision as Bayesian inference: analysis by synthesis?" en. In: *Trends Cogn. Sci.* 10.7 (July 2006), pp. 301–308.

[YK16] F. Yu and V. Koltun. "Multi-Scale Context Aggregation by Dilated Convolutions". In: *ICLR*. 2016.

[YL06] M. Yuan and Y. Lin. "Model Selection and Estimation in Regression with Grouped Variables". In: *J. Royal Statistical Society, Series B* 68.1 (2006), pp. 49–67.

[YL18] A. L. Yuille and C. Liu. "Deep Nets: What have they ever done for Vision?" In: (May 2018). arXiv: 1805.04025 [cs.CV].

[Yon19] E. Yong. "The Human Brain Project Hasn't Lived Up to Its Promise". In: *The Atlantic* (July 2019).

[Yos+15] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs, and H. Lipson. "Understanding Neural Networks Through Deep Visualization". In: *ICML Workshop on Deep Learning*. 2015.

[Yu+06] S. Yu, K. Yu, V. Tresp, K. H-P., and M. Wu. "Supervised probabilistic principal component analysis". In: *KDD*. 2006.

[Yu+16] F. X. X. Yu, A. T. Suresh, K. M. Choromanski, D. N. Holtmann-Rice, and S. Kumar. "Orthogonal Random Features". In: *NIPS*. Curran Associates, Inc., 2016, pp. 1975–1983.

[Yua+19] X. Yuan, P. He, Q. Zhu, and X. Li. "Adversarial Examples: Attacks and Defenses for Deep Learning". en. In: *IEEE Trans. Neural Networks and Learning Systems* 30.9 (Sept. 2019), pp. 2805–2824.

[YWG12] S. E. Yuksel, J. N. Wilson, and P. D. Gader. "Twenty Years of Mixture of Experts". In: *IEEE Trans. on neural networks and learning systems* (2012).

[Zah+18] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar. "Adaptive Methods for Nonconvex Optimization". In: *NIPS*. Curran Associates, Inc., 2018, pp. 9815–9825.

[Zah+20] M. Zaheer et al. "Big Bird: Transformers for Longer Sequences". In: *CoRR* abs/2007.14062 (2020). arXiv: 2007.14062.

[Zei12] M. D. Zeiler. "ADADELTA: An Adaptive Learning Rate Method". In: (Dec. 2012). arXiv: 1212.5701 [cs.LG].

[Zel76] A. Zellner. "Bayesian and non-Bayesian analysis of the regression model with multivariate student-t error terms". In: *JASA* 71.354 (1976), pp. 400–405.

[ZG02] X. Zhu and Z. Ghahramani. *Learning from labeled and unlabeled data with label propagation*. Tech. rep. CALD tech report CMU-CALD-02-107. CMU, 2002.

[ZG06] M. Zhu and A. Ghodsi. "Automatic dimensionality selection from the scree plot via the use of profile likelihood". In: *Computational Statistics & Data Analysis* 51 (2006), pp. 918–930.

[ZH05] H. Zou and T. Hastie. "Regularization and Variable Selection via the Elastic Net". In: *JRSSB* 67.2 (2005), pp. 301–320.

[Zha+17a] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. "Understanding deep learning requires rethinking generalization". In: *ICLR*. 2017.

[Zha+17b] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. "mixup: Beyond Empirical Risk Minimization". In: *ICLR*. 2017.

[Zha+18] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu. "Object Detection with Deep Learning: A Review". In: (July 2018). arXiv: 1807.05511 [cs.CV].

[Zha+19a] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu. "PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization". In: (Dec. 2019). arXiv: 1912.08777 [cs.CL].

[Zha+19b] S. Zhang, L. Yao, A. Sun, and Y. Tay. "Deep Learning Based Recommender System: A Survey and New Perspectives". In: *ACM Comput. Surv.* 52.1 (Feb. 2019), pp. 1–38.

[Zha+20] A. Zhang, Z. Lipton, M. Li, and A. Smola. *Dive into deep learning*. 2020.

[Zho+04] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, and B. Schölkopf. "Learning with local and global consistency". In: *Advances in neural information processing systems*. 2004, pp. 321–328.

[Zho+18] D. Zhou, Y. Tang, Z. Yang, Y. Cao, and Q. Gu. "On the Convergence of Adaptive Gradient Methods for Nonconvex Optimization". In: (Aug. 2018). arXiv: 1808.05671 [cs.LG].

[ZHT06] H. Zou, T. Hastie, and R. Tibshirani. "Sparse principal component analysis". In: *JCGS* 15.2 (2006), pp. 262–286.

[Zhu05] X. Zhu. "Semi-supervised learning with graphs". PhD thesis. Carnegie Mellon University, 2005.

[Zhu+19] F. Zhuang et al. "A Comprehensive Survey on Transfer Learning". In: (Nov. 2019). arXiv: 1911.02685 [cs.LG].

[Zie+05] C.-N. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen. "Improving recommendation lists through topic diversification". In: *WWW*. WWW '05. Chiba, Japan: Association for Computing Machinery, May 2005, pp. 22–32.

[ZK16] S. Zagoruyko and N. Komodakis. "Wide Residual Networks". In: *BMVC*. 2016.

[ZL05] Z.-H. Zhou and M. Li. "Tri-training: Exploiting unlabeled data using three classifiers". In: *IEEE Transactions on knowledge and Data Engineering* 17.11 (2005), pp. 1529–1541.

[ZL17] B. Zoph and Q. V. Le. "Neural Architecture Search with Reinforcement Learning". In: *ICLR*. 2017.

[ZLZ20] D. Zhang, Y. Li, and Z. Zhang. "Deep metric learning with spherical embedding". In: *NIPS*. 2020.

[ZMG19] G. Zhang, J. Martens, and R. B. Grosse. "Fast Convergence of Natural Gradient Descent for Over-Parameterized Neural Networks". In: *NIPS*. 2019, pp. 8082–8093.

[ZMY19] D. Zabihzadeh, R. Monsefi, and H. S. Yazdi. "Sparse Bayesian approach for metric learning in latent space". In: *Knowledge-Based Systems* 178 (Aug. 2019), pp. 11–24.

[ZRY05] P. Zhao, G. Rocha, and B. Yu. *Grouped and Hierarchical Model Selection through Composite Absolute Penalties*. Tech. rep. UC Berkeley, 2005.

[ZS14] H. Zen and A Senior. "Deep mixture density networks for acoustic modeling in statistical parametric speech synthesis". In: *ICASSP*. May 2014, pp. 3844–3848.

[ZW11] D Zoran and Y Weiss. "From learning models of natural image patches to whole image restoration". In: *ICCV*. 2011.

[ZY08] J.-H. Zhao and P. L. H. Yu. "Fast ML Estimation for the Mixture of Factor Analyzers via an ECM Algorithm". In: *IEEE. Trans. on Neural Networks* 19.11 (2008).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license